Markov Decision Processes and Reinforcement Learning

Martin L. Puterman and Timothy C. Y. Chan

August 7, 2025

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

Dedication

Marty: To my wife, Dodie Katzenstein, who made helpful suggestions and encouraged me to write this book, despite claiming she has no idea what it is about.

Tim: To my family – mom, dad, Laura, Euclid, Escher – who support me in everything I do, and can now stop asking when the book will be done.

Preface

When we set out to write this book, at the onset of the COVID-19 pandemic, our goal was to provide an accessible and logically structured introduction to Markov decision process (MDP) theory, applications and algorithms. As our writing and research progressed, we came to appreciate the vast and rapidly evolving landscape of reinforcement learning as a major sub-field within artificial intelligence (AI)[1]. Consequently, a significant portion of this book now focuses on reinforcement learning methods. The advantage of grounding our treatment in Markov decision processes is that they offer a rigorous and unifying modeling framework, one that applies even in *model-free* reinforcement learning settings in which system dynamics are not fully known.

This book is not meant to be encyclopedic. Rather, we have chosen to focus on the core concepts of these disciplines, with the objective of providing a solid basis for further study, application, and research. Throughout, simple and transparent models illustrate how the fundamental Markov decision process entities – states, actions, transition probabilities and rewards – apply in each context. A sound grasp of these fundamentals is necessary for use in more complex settings. In our experiences reviewing journal submissions, we have observed that model formulations are frequently imprecise, resulting in ambiguity in explaining and interpreting results.

Numerous computational examples in this book illustrate how algorithms work in practice. Commentary accompanying algorithms and in computational examples provides insights into algorithmic details and guidelines for application. Hands-on coding and tuning is a necessary step in learning to use these methods, an aspect that is often missed when learners rely solely on reusing pre-written code found online.

Markov decision processes arise in many fields, most notably operations research, robotics, computer science, engineering, economics, and management. While our approach primarily reflects an operations research perspective, the concepts and tools presented here are broadly applicable. We believe this book will provide a solid foundation for anyone seeking a rigorous introduction to these topics, especially within the growing reinforcement learning community.

A word of caution: the extremely powerful methods in this book are broadly applicable yet have the potential to be misused. We encourage you to apply them responsibly. In the spirit of Google's original guiding principle, "Don't be evil."

[1] Coincidentally, ChatGPT, which leveraged reinforcement learning in its development, has assisted us in editing text, generating code, and producing figures.

Book objectives

The book aims to provide readers with a solid foundation in Markov decision process principles and algorithms, reinforcement learning methods, and the relationship between the two. Readers will learn to apply Markov decision processes, identify model components, solve models using exact or approximate methods, and interpret solutions.

We strongly believe that working through examples and coding algorithms is the best way to learn this material. Accordingly, through numerous examples, the book includes extensive numerical calculations and discussions of their implications. Additional exercises are provided at the end of each chapter to further illustrate applications and reinforce key concepts.

Book structure

The book is organized as follows:

- **Introduction:** Chapter 1 describes the foundations and applications of Markov decision processes and reinforcement learning. In addition it provides a historical perspective on the evolution of Markov decision processes in operations research and control theory, and reinforcement learning in behavioural science and artificial intelligence, setting the stage for the integration of these perspectives in contemporary research and applications.

- **Part I - Fundamentals:** Chapters 2-3 develop and apply a rigorous modeling foundation. Chapter 2 introduces basic model components and optimality criteria, illustrates elementary calculations that form the building blocks of more complex algorithms, and includes a numerical example that appears throughout the book to illustrate new ideas and methods. A diverse set of applications in Chapter 3 provides the reader with real-world examples of Markov decision process formulations.

- **Part II - Classical Markov decision process models:** Chapter 4 covers finite horizon MDPs, while Chapters 5-7 address infinite horizon MDPs under several optimality criteria. Chapter 5 focuses on discounted models, which are most widely used and theoretically complete. Chapter 6 characterizes and analyses expected total reward models, also known as episodic models, which are foundational in reinforcement learning. Chapter 7 examines models under the long-run average reward criteria. Finite and infinite horizon partially observable Markov decision processes (POMDPs) are the subject of Chapter 8.

- **Part III - Reinforcement learning:** Chapters 9-11 describe methods for solving large-scale, model-based and model-free sequential decision processes. Chapter 9 combines value function approximation with value iteration, policy iteration and linear programming to find approximate solutions for Markov decision process. This material is often referred to as *approximate dynamic programming.*

Simulation-based methods that apply in both a model-free and model-based environment are the subject of Chapter 10. This chapter considers *tabular models* which are of a size that allows value functions, state-action value functions, and policies to be represented explicitly in look-up tables. Chapter 11 extends these results to large-scale models by combining the approximation methods of Chapters 9 and 10.

- **Appendices:** Book appendices summarize key mathematical notation and conventions used throughout the book, as well as background on Markov chains and linear programming. Some chapters have their own appendices, which provide supplementary technical details and relevant background. Topics include stochastic approximation, regression, and gradient descent.

- **Bibliographic remarks:** Each chapter includes a brief section that provides historical commentary and references.

- **Exercises:** Exercises at the end of each chapter offer opportunities to formulate models, apply methods, or delve into theoretical details. Many are open-ended. Alternatively, readers are encouraged to choose their own models to formulate and try out the methods in the book, as we have done with the recurring two-state model, queuing control model and coffee-delivering robot model (featured on the book cover). Starred exercises indicate increased difficulty.

- **Starred sections:** Sections with an asterisk (*) contain important, technically demanding or supplementary material. They may be skipped on first reading.

- **Gray boxes:** In addition to theoretical results, algorithms, and examples, important equations and comments are placed in gray boxes throughout the book.

Relationship to Puterman [1994]

This book is not a revision of [Puterman, 1994] which was written to be a comprehensive and state-of-the-art reference targeted at researchers and advanced graduate students with strong mathematics backgrounds. Our goal in writing this book is to provide a rigorous yet more accessible introduction to Markov decision processes, reinforcement learning and their extensions that will be accessible to advanced undergraduate students, early graduate students, and practitioners. Since the field has rapidly expanded since the early 1990s, especially in the area of reinforcement learning, this book develops these concepts using the common notation and language of Markov decision processes.

Some specific differences with Puterman [1994] include:

- A broad discussion of reinforcement learning methods, many of which were not widely known, especially in the operations research community, in the early 1990s.

- Formulation and analysis of partially observable Markov decision processes (POMDPs).
- Descriptions of state-action value functions permeate the book in anticipation of their fundamental role in reinforcement learning methods.
- Increased emphasis on randomized policies in light of their fundamental role in linear programming, policy gradient and actor-critic methods.
- Illustrating all algorithms with numerical calculations applied to common examples. All computations were done from scratch using R [R Core Team, 2024]. With the rapidly advancing proficiency of AI models to generate useful code and translate code between languages, readers are encouraged to engage hands-on in the coding process (perhaps with AI assistance) in order to truly understand the workings of each algorithm.
- Omission of material on countable state models, multi-chain average reward MDPs and sensitive optimality criteria including Blackwell optimality.
- Limited bibliographic references. With the rapid development of internet resources as well as the presence of some excellent books, historical perspectives are readily available elsewhere.
- Representing component-wise maxima by "c-max" to avoid misinterpretation of "max" when using vector notation.
- Using $r(s, a, j)$ as the basic definition of the reward function instead of $r(s, a)$.
- Emphasizing transient and stochastic shortest path models in the chapter on infinite horizon models with the expected total reward criterion. The advantage of this categorization is that it applies easily to what have become known as *episodic* models. The earlier book adopted a classic approach based on positive and negative models.

Neural networks

The book intentionally omits the extensive and rapidly evolving body of research and application of *deep reinforcement learning*, a subfield of reinforcement learning that leverages neural network approximations. While neural networks have played a central role in many breakthroughs in reinforcement learning, we have chosen not to cover neural network design or training. Readers interested in learning about and applying neural networks are encouraged to consult dedicated texts on deep learning that describe network architectures, backpropagation, and training strategies.

Our focus is on foundational principles and the core Markov decision process and reinforcement learning theory and algorithms. Understanding these foundations is essential regardless of whether function approximation is through tabular representations, linear functions of features, or deep neural networks. For this reason, we believe

a mastery of the basics should precede the study of more complex approximation techniques.

How to use this book

Our intention in writing this book is that readers find the material self-explanatory and accessible so that independent study is possible. The following description of course structures might provide guidance in working through it. The requisite background includes a solid foundation in probability, linear algebra, real analysis and statistical estimation.

Every course should at a minimum cover Chapters 1-4. Subsequent chapters should be chosen to reflect course and learning objectives.

- An introductory Markov decision process course with an operations research orientation should supplement the above chapters with selected material on optimality equations and algorithms from Chapter 5 on infinite horizon discounted models and Chapter 6 on episodic models. The model formulation in Chapter 8 of partially observable MDPs and the material on simulation of episodic and infinite horizon discounted models in Chapter 10 nicely complements the above material.

- A more advanced MDP course should delve more deeply into theoretical issues in Chapters 5 and 6, and also include material on average reward models in Chapter 7, supplemented by material in Chapter 9 of Puterman [1994].

- A reinforcement learning course with a computer science orientation should supplement the material in Chapters 1-4 with methods chosen from Chapters 9-11. Such a course should delve more deeply into the topics in Section 11.7, especially neural networks. Instructors should add material on neural network approximations of value functions, state-action value functions and policies, as well as focusing on examples arising in robotics, vehicle guidance and generative models.

Additional resources

Additional resources and errata can be found at `https://www.cambridge.org/9781009098410`.

Our cover image

Our life-long friend, Flora Gordon, designed our whimsical and aptly-themed cover with assistance from her colleague Sergio Lopez. The robot, who we refer to as "Flow", appears in the Gridworld model of Section 3.2, an application that recurs throughout the book to illustrate Markov decision process and reinforcement learning algorithms. This example is inspired by the Hungarian mathematician Alfréd Rényi who is attributed with the quote:

A mathematician is a machine for turning coffee into theorems.

Moreover, robotics has now become one of the principal application areas of reinforcement learning.

Acknowledgments

We would like to acknowledge many individuals who assisted us on this journey. Significant contributors include Abhijit Gosavi and Nicolas Gast, whose insights helped shape Chapters 10 and 11 (AG) and Chapter 7 (NG). Special thanks go to Antoine Sauré who provided extensive and insightful comments on Chapters 9-11.

We are also grateful to those who have provided suggestions and guidance including Alan Mackworth, Rich Sutton, Reid Swanson, Steven Shechter, Dmitri Bertsekas, John Tsitsiklis, Sergey Levine, Bruno Scherrer and Gergely Neu. Others have offered minor editorial improvements and suggestions, and checked calculations.

Elise Liu, while still in high school and now at The University of St. Andrews, produced many of the book's figures. Discussions with Pritam Dash provided us with insight into a computer science graduate student's perspective on this material. Moreover, he developed a Github repository for code and text. Neal Kaw and Michael Gimelfarb developed exercise solutions and several figures for the early chapters. Ken Wong also contributed to the figures, as well as the bibliography.

In addition, we thank Lauren Cowles and Arman Chowdhury for their timely encouragement, attention to detail, and expert advice on manuscript preparation, and Diana Gillooly who was our initial contact with Cambridge University Press.

Contents

1 Introduction **19**

1.1 What is a Markov decision process? . . . 21
1.1.1 A historical perspective . . . 22
1.1.2 Markov decision process applications . . . 23
1.2 What is reinforcement learning? . . . 25
1.2.1 Trial-and-error learning . . . 25
1.2.2 A historical perspective . . . 26
1.3 Book structure . . . 31

2 Markov Decision Process Fundamentals **35**

2.1 Basic model components . . . 36
2.1.1 Planning horizons and decision epochs . . . 36
2.1.2 States . . . 38
2.1.3 Actions . . . 38
2.1.4 Transition probabilities . . . 39
2.1.5 Rewards . . . 39
2.1.6 Markov decision processes and decision trees . . . 40
2.2 Derived objects . . . 41
2.2.1 Decision rules . . . 42
2.2.2 Policies . . . 44
2.2.3 Derived stochastic processes . . . 45
2.2.4 Reward processes . . . 47
2.2.5 Assigning a value to a reward process . . . 48
2.3 Optimality criteria: Transforming a Markov decision process into a Markov decision problem . . . 50
2.3.1 Expected total reward . . . 51
2.3.2 Expected discounted reward . . . 56
2.3.3 Long-run average reward for an infinite horizon model . . . 61
2.4 The one-period problem: A fundamental building block . . . 63
2.4.1 Optimal value functions . . . 64
2.4.2 Optimal policies . . . 66
2.4.3 State-action value functions . . . 68
2.4.4 Summary of results for a one-period problem . . . 69

2.5 A two-state model . . . 70
2.5.1 A one-period version . . . 71
2.5.2 Policies in a two-period version of the two-state model . . . 72

3 Examples and Applications 80
3.1 Inventory management . . . 80
3.1.1 Inventory management with backlogged demand . . . 82
3.1.2 The newsvendor problem . . . 85
3.2 Gridworld navigation . . . 88
3.3 Revenue management: Using price to manage demand . . . 92
3.4 Discrete-time queuing models . . . 94
3.4.1 Service rate control . . . 95
3.4.2 Admission control . . . 97
3.5 Behavioral decision making: When should a lion hunt? . . . 103
3.6 Clinical decision making: An application to liver transplantation . . . 105
3.7 Advance appointment scheduling . . . 108
3.8 Optimal stopping . . . 113
3.8.1 Model formulation . . . 114
3.8.2 Examples . . . 115
3.9 Sports strategy . . . 117
3.9.1 When to pull the goalie in ice hockey . . . 118
3.9.2 A tennis handicap system . . . 120
3.10 The art of modeling . . . 122

4 Finite Horizon Models 135
4.1 The value of a policy in a finite horizon model . . . 136
4.1.1 Backwards induction . . . 137
4.1.2 Evaluating a Markovian deterministic policy . . . 138
4.1.3 Evaluating a history-dependent randomized policy* . . . 143
4.2 Optimal policies and their values . . . 144
4.2.1 Definitions . . . 144
4.2.2 Computing optimal value functions and finding optimal policies . . . 146
4.2.3 A numerical example . . . 150
4.3 Applications . . . 150
4.3.1 Queuing service rate control . . . 151
4.3.2 Revenue management . . . 154
4.3.3 Online dating: Finding the best match . . . 157
4.4 Interpretability: Optimality of structured policies . . . 162
4.4.1 A general approach . . . 163
4.4.2 Example: Preventive maintenance . . . 164
4.4.3 Monotonicity of optimal policies in service rate control queuing systems* . . . 166
4.5 State-action value functions . . . 171

4.5.1 State-action value function recursion for a fixed policy 171
4.5.2 Revisiting the two-state model from the state-action value function perspective 173
4.5.3 Optimal state-action value functions 174
4.5.4 An example 177
4.5.5 Optimality equations in the queuing admission control model* . 178
4.6 Technical appendix 180
4.6.1 Finding optimal policies in Π^{MD} 180
4.6.2 Finding optimal values in Π^{HR} 181
4.6.3 Optimality of Markov deterministic policies in Π^{HR} 182

5 Infinite Horizon Models: Expected Discounted Reward 189
5.1 Preliminaries 191
5.1.1 Reward functions 191
5.1.2 Key assumptions 191
5.1.3 Markovian policies are sufficient 192
5.1.4 Matrix and vector representation for stationary policies 192
5.1.5 Partial orders 194
5.1.6 Norms 195
5.2 The expected discounted reward of a policy 197
5.2.1 Insights into how the expected discounted reward is evaluated . 197
5.2.2 Representing value functions as vectors 199
5.2.3 Evaluating stationary policies 201
5.3 Optimal policies and the Bellman equation 204
5.3.1 Optimal policies 204
5.3.2 The Bellman equation for a discounted model 205
5.3.3 The maximum return operator 207
5.3.4 Existence of an optimal value function 210
5.3.5 Existence of an optimal stationary policy 212
5.4 State-action value functions 215
5.4.1 State-action value function recursions for a fixed policy 216
5.4.2 Bellman equation for state-action value functions 216
5.4.3 An example 217
5.4.4 Existence of solutions to the Bellman equation for state-action value functions* 218
5.5 Spans, bounds and other technical concepts 219
5.5.1 Spans and span contractions 220
5.5.2 Properties of $L\mathbf{v} - \mathbf{v}$ 225
5.5.3 Bounds 227
5.5.4 Stopping criteria 230
5.5.5 Action elimination 231
5.6 Value iteration 233
5.6.1 A value iteration algorithm 233

5.6.2 Gauss-Seidel value iteration . . . 238
5.6.3 Value iteration with action elimination . . . 244
5.7 Policy iteration . . . 246
5.7.1 The policy iteration algorithm . . . 246
5.7.2 Convergence of policy iteration . . . 251
5.7.3 Policy iteration and Newton's method* . . . 253
5.8 Modified policy iteration . . . 256
5.8.1 Motivation . . . 257
5.8.2 A modified policy iteration algorithm . . . 258
5.8.3 Convergence of modified policy iteration* . . . 263
5.9 Linear programming . . . 265
5.9.1 The primal linear program . . . 266
5.9.2 The dual linear program . . . 271
5.9.3 Dual feasible solutions and stationary policies . . . 273
5.9.4 Solving linear programs . . . 282
5.10 Optimality of structured policies . . . 284
5.10.1 The fundamental result . . . 284
5.10.2 A preventive maintenance example . . . 285
5.11 Application: Queuing service rate control . . . 286
5.11.1 The model . . . 286
5.11.2 Value iteration . . . 287
5.11.3 Gauss-Seidel value iteration . . . 288
5.11.4 Policy iteration . . . 289
5.11.5 Modified policy iteration . . . 292
5.11.6 Linear Programming . . . 294
5.11.7 Concluding remarks on queuing control application . . . 296
5.12 Application: Clinical decision making . . . 297
5.12.1 The Bellman equation . . . 298
5.12.2 A numerical example . . . 298
5.12.3 Numerical solution . . . 299
5.12.4 Interpretation of results . . . 301
5.12.5 Concluding remarks on clinical decision making application . . . 301
5.13 Technical appendix . . . 302
5.13.1 Proof of Theorem 5.1 . . . 302
5.13.2 Bounds for Gauss-Seidel iteration . . . 306
5.13.3 Convergence of policy iteration in non-finite models* . . . 310

6 Infinite Horizon Models: Expected Total Reward 320
6.1 Introduction . . . 320
6.1.1 Motivating examples . . . 321
6.2 Model classification . . . 323
6.2.1 Preliminaries . . . 324
6.2.2 Transient models . . . 326

6.2.3 Stochastic shortest path models . . . 329
6.2.4 Positive models . . . 330
6.2.5 Negative models . . . 330
6.2.6 Comparison of model classes . . . 331
6.3 Value functions, optimal policies and the Bellman equation . . . 333
6.3.1 The Bellman equation in an expected total reward model . . . 334
6.3.2 The transient Bellman equation . . . 335
6.3.3 Solutions of the Bellman equation in expected total reward models 336
6.3.4 Existence of optimal stationary policies in expected total reward models* . . . 338
6.3.5 Identification of optimal policies . . . 340
6.4 State-action value functions . . . 342
6.5 Value iteration . . . 342
6.5.1 Examples . . . 343
6.5.2 Transient models . . . 346
6.5.3 Stochastic shortest path models . . . 356
6.5.4 Positive models . . . 356
6.5.5 Negative models . . . 360
6.6 Policy iteration . . . 362
6.6.1 Transient models . . . 363
6.6.2 Stochastic shortest path models . . . 365
6.7 Modified policy iteration . . . 365
6.7.1 Transient models . . . 365
6.7.2 Convergence in transient models . . . 367
6.7.3 An example . . . 367
6.7.4 Stochastic shortest path models . . . 368
6.8 Linear programming in transient models . . . 368
6.8.1 The primal linear program . . . 369
6.8.2 The dual linear program . . . 369
6.8.3 Examples . . . 371
6.9 Optimality of structured policies . . . 373
6.9.1 The fundamental result . . . 374
6.10 Applications . . . 374
6.10.1 Gridworld navigation . . . 374
6.10.2 Optimal stopping . . . 376

7 Infinite Horizon Models: Long-Run Average Reward 390
7.1 Preliminaries . . . 391
7.2 The long-run average reward or gain . . . 392
7.2.1 The gain of a stationary policy . . . 392
7.2.2 The gain of a non-stationary policy need not exist . . . 393
7.2.3 A more general definition of the gain . . . 395
7.2.4 Average optimality . . . 396

7.3 Chain structure . . . 397
7.4 Bias of a stationary policy . . . 403
7.4.1 Interpreting the bias . . . 404
7.4.2 Computing the gain and bias of a stationary policy . . . 410
7.5 The Bellman equation and its properties . . . 418
7.5.1 The Bellman equation in recurrent and unichain models . . . 418
7.5.2 Bounds on the optimal gain . . . 420
7.5.3 Existence of solutions of the Bellman equation . . . 423
7.5.4 The Bellman equation and optimal policies . . . 424
7.5.5 State-action value functions . . . 427
7.5.6 The Bellman equation in multi-chain models* . . . 430
7.6 Value iteration . . . 430
7.6.1 Examples . . . 431
7.6.2 Removing periodicity . . . 433
7.6.3 A value iteration algorithm . . . 435
7.6.4 Relative value iteration . . . 440
7.7 Policy iteration . . . 442
7.7.1 An algorithm . . . 442
7.7.2 An example . . . 444
7.7.3 Convergence of recurrent and unichain policy iteration . . . 445
7.8 Linear programming . . . 451
7.8.1 Linear programming formulation . . . 452
7.8.2 Recurrent models . . . 454
7.8.3 Unichain models . . . 459
7.8.4 Constrained Markov decision processes . . . 460
7.9 Optimality of structured policies . . . 461
7.10 Queuing service rate control . . . 463
7.10.1 The model . . . 463
7.10.2 Value iteration . . . 463
7.10.3 Policy iteration . . . 464
7.10.4 Linear programming . . . 466
7.10.5 Concluding remarks on computation . . . 468

8 Partially Observable Markov Decision Processes 474
8.1 Model overview . . . 475
8.1.1 POMDP dynamics . . . 476
8.1.2 A two-state POMDP model . . . 480
8.2 Information and belief states . . . 484
8.3 Transforming a POMDP into a Markov decision process . . . 488
8.3.1 The derived Markov decision process . . . 489
8.3.2 The two-state model revisited . . . 493
8.4 A finite horizon model . . . 496
8.4.1 Optimality criterion . . . 496

8.4.2 The Bellman equation and its solution . . . 498
8.4.3 Structure of optimal value functions in a POMDP . . . 501
8.4.4 An exact algorithm for finite horizon POMDPs . . . 506
8.4.5 A finite-grid approximation for finite horizon POMDPs . . . 513
8.5 Infinite horizon models . . . 524
8.5.1 Optimality criterion . . . 524
8.5.2 Computing optimal values and finding optimal policies . . . 525
8.6 Examples . . . 528
8.6.1 Preventive maintenance and inspection . . . 528
8.6.2 Models with unknown parameters: Bayesian decision problems . . . 533
8.6.3 Multi-armed bandits . . . 536
8.6.4 Breast cancer screening . . . 542
8.6.5 Controlling autonomous robots with noisy sensors . . . 545

9 Value Function Approximation 559
9.1 Introduction . . . 559
9.2 Approximating policy value functions . . . 561
9.2.1 Features . . . 563
9.2.2 Feature-based value function approximators . . . 566
9.2.3 State-action value functions . . . 571
9.3 Parameter estimation for policy value functions: Linear architectures . . . 571
9.3.1 Least squares policy evaluation . . . 575
9.3.2 Linear programming approximations for policy value functions . . . 581
9.4 Parameter estimation for a fixed policy: Nonlinear architectures . . . 584
9.5 Combining approximation and optimization . . . 585
9.5.1 Iterative methods for linear architectures . . . 586
9.5.2 Iterative methods for nonlinear architectures . . . 593
9.5.3 Optimization using linear programming: Linear architectures . . . 595
9.6 Application: strategic scheduling . . . 600
9.6.1 A numerical example . . . 604
9.6.2 Value iteration without approximation . . . 604
9.6.3 LSVI with linear value function approximation . . . 605
9.6.4 LSVI with quadratic value function approximation . . . 607
9.6.5 Comparison of policies and values . . . 608
9.6.6 Further insights . . . 609
9.7 Application: Golf strategy . . . 612
9.7.1 Value functions and strokes gained in golf . . . 612
9.8 Technical appendix: Regression and nonlinear optimization . . . 619
9.8.1 Linear regression . . . 619
9.8.2 Matrix formulation . . . 621
9.8.3 Nonlinear regression . . . 625
9.8.4 Nonlinear optimization . . . 625

10 Simulation in Tabular Models **634**
10.1 Preliminaries . . . 636
10.1.1 Data generation . . . 637
10.1.2 Trade-offs . . . 640
10.2 Methods overview: The learning newsvendor . . . 641
10.2.1 Analysis by simulation . . . 642
10.2.2 A numerical study . . . 646
10.2.3 Conclusions . . . 649
10.3 Policy evaluation: Episodic models . . . 651
10.3.1 Monte Carlo methods . . . 652
10.3.2 Temporal differencing . . . 656
10.3.3 TD(γ) . . . 667
10.4 Policy evaluation: Infinite horizon discounted models . . . 677
10.4.1 Monte Carlo methods . . . 677
10.4.2 Online TD(γ) in an infinite horizon discounted model . . . 680
10.5 Policy evaluation: Infinite horizon average reward models . . . 685
10.5.1 Monte Carlo methods . . . 686
10.5.2 Temporal differencing (TD(0)) . . . 688
10.6 Policy optimization . . . 692
10.6.1 Two perspectives . . . 692
10.6.2 Model selection . . . 692
10.6.3 Preliminaries . . . 693
10.6.4 Selecting actions: Exploration vs. exploitation . . . 695
10.7 Q-learning and SARSA . . . 697
10.7.1 Episodic models . . . 699
10.7.2 Infinite horizon discounted models . . . 706
10.7.3 Infinite horizon average reward models . . . 712
10.8 Policy iteration type algorithms* . . . 719
10.8.1 Hybrid policy improvement . . . 720
10.8.2 Hybrid policy iteration bounds . . . 722
10.8.3 Online policy improvement . . . 725
10.9 Queuing service rate control revisited . . . 728
10.9.1 Monte Carlo policy evaluation . . . 728
10.9.2 TD(γ) policy evaluation . . . 729
10.9.3 Q-learning . . . 732
10.9.4 Policy iteration type algorithms . . . 735
10.9.5 Computational summary: Discounted model . . . 738
10.9.6 Average reward queuing control . . . 739
10.10 Conclusion . . . 741
10.11 Technical appendix . . . 741
10.11.1 Choosing good estimators . . . 741
10.11.2 Stochastic approximation . . . 745
10.11.3 Stochastic approximation and optimization . . . 750

10.11.4 Offline TD(γ): Technical details . . . 757

11 Simulation with Function Approximation **768**
11.1 The challenge of large models . . . 769
11.1.1 Features . . . 769
11.1.2 Policy value function approximation . . . 770
11.1.3 State-action value function approximation . . . 771
11.1.4 Policy approximation . . . 774
11.2 Policy value function approximation . . . 775
11.2.1 Monte Carlo policy value function approximation . . . 775
11.2.2 Temporal differencing with policy value function approximation 781
11.2.3 TD(γ) with linear policy value function approximation . . . 789
11.3 Optimization based on value function approximation: Q-learning . . . 792
11.3.1 Motivation . . . 793
11.3.2 Discounted Q-learning with function approximation . . . 794
11.3.3 Q-learning in an episodic model . . . 798
11.4 Q-policy iteration . . . 802
11.4.1 Motivation . . . 803
11.4.2 An algorithm . . . 803
11.4.3 Newsvendor model . . . 806
11.4.4 Queuing service rate control model . . . 810
11.4.5 Concluding remarks on examples . . . 812
11.5 Policy space methods . . . 812
11.5.1 Motivation: A policy gradient algorithm for a one-state one-period model . . . 813
11.5.2 A policy gradient algorithm for a Markov decision process . . . 817
11.5.3 The gradient of the policy value function for an undiscounted infinite horizon Markov decision process . . . 819
11.5.4 A simple policy gradient algorithm . . . 820
11.5.5 An example . . . 823
11.5.6 Policy gradient in a discounted model . . . 828
11.5.7 Policy gradient: Concluding remarks . . . 830
11.6 Actor-critic algorithms: Combining policy and policy value function approximation . . . 830
11.6.1 An online actor-critic algorithm . . . 833
11.6.2 A batch actor-critic algorithm . . . 837
11.6.3 Actor-critic methods: Concluding remarks and enhancements . 841
11.7 Further topics . . . 843
11.7.1 Simultaneous learning and control . . . 843
11.7.2 Experience replay . . . 844
11.7.3 Deep learning . . . 845
11.7.4 Importance sampling . . . 845
11.7.5 Monte Carlo tree search . . . 847

11.7.6 Partially observable models . . . 850
11.8 Technical appendix: Derivation of policy gradient representation . . . 850

A Notation and Conventions **858**
A.1 General math notation . . . 858
A.2 Notation specific to MDPs . . . 860
A.3 Conventions . . . 861
A.4 Abbreviations . . . 862

B Markov Chains **863**
B.1 What is a finite Markov chain? . . . 863
B.2 Classifying states . . . 864
B.3 Classes and class structure . . . 865
B.3.1 Chain structure . . . 866
B.4 Limiting behavior . . . 867
B.5 An important lemma . . . 870
B.6 The deviation matrix . . . 871
B.7 Structure of $\mathbf{P}^*$ and $\mathbf{H}$ . . . 872
B.8 Some examples . . . 874
B.9 Eigenvalues and eigenvectors of a transition matrix* . . . 876
B.10 Absorbing chains . . . 878
B.11 Countable state chains* . . . 880

C Linear Programming **883**
C.1 Duality . . . 885

Chapter 1

Introduction

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

> *Destiny is not a matter of chance: it is a matter of choice; it is not a thing to be waited for, it is a thing to be achieved*[1].
>
> William Jennings Bryan[2], American orator and politician, 1860-1925.

The sentiment in William Jennings Bryan's quote captures the essence of Markov decision processes (MDPs) and reinforcement learning (RL). These frameworks provide methods to assist a decision maker or agent who seeks, through a series of choices, to guide a system to achieve desirable outcomes. Unlike forecasting, which focuses on predicting what will happen under fixed or external dynamics, Markov decision processes and reinforcement learning regard future outcomes as consequences of actions. The goal of these methods is not merely to anticipate the future but to learn policies that actively shape it. In this view, destiny is determined by decision-making, not chance–a principle that underlies the design and analysis of control strategies in dynamic environments.

Consequently, this book explores the mathematical and algorithmic foundations of sequential decision-making under uncertainty, with a focus on problems involving trade-offs, randomness, and temporal structure. By learning to formulate and solve dynamic decision models, readers will gain tools not only to anticipate future outcomes but also to influence and optimize them through informed action.

[1] Bryan [1899].

[2] Marty's wife, Dodie, and William Jennings Bryan were both born in Salem, Illinois. In a curious coincidence, Dodie later lived on Bryan Lane, the location of the Bryan family farm. Hopefully, this shared hometown and street name will not give rise to any conflicts of interest.

This book examines two primary approaches to sequential decision making in uncertain environments:

1. A *model-based* Markov decision process approach in which the agent or decision maker has full knowledge of the environment's structure—states, actions, transition probabilities, and reward functions — and uses algorithms such as value iteration, policy iteration, or linear programming to compute optimal policies.

2. A *model-free* reinforcement learning approach in which the agent or decision maker learns good policies through trial-and-error interactions with the environment or a simulation model, without requiring explicit knowledge of transition probabilities or reward functions. Algorithms include Q-learning, policy gradient, and actor-critic methods.

These categories represent conceptual extremes; the boundaries between them have become blurred over time. Modern approaches often blend features of both. In some cases, models may be available but too difficult to explicitly formulate or to solve using standard algorithms, making model-free methods more attractive. On the other hand, in model-free settings, the agent may learn approximate models of the environment and leverage them to to enhance learning and planning. Two applications distinguish these differences and their overlap.

Managing pre-boarding security

Consider the challenge of managing the pre-boarding screening area at a large airport. This is an ongoing challenge that is best analyzed offline using a model-based approach. The objective is to find policies that specify how and when to augment or decrement the workforce size and composition.

A model-based MDP approach specifies the system state to be the number of passengers of each type waiting to be cleared and the configuration of the workforce at each instant of time. Transition probabilities encode the probability of changes in system state between decision epochs given decisions regarding workforce levels. Viewing this as a cost minimization problem, costs are associated with workforce wages and penalties for passenger delay.

In theory, a model of this system can be developed and solved using Markov decision process algorithms. However, doing so will be challenging because of the large number of possible states and actions and the difficulty to explicitly represent the impacts of decisions in terms of transition probabilities. Such a model suffers from *the curse of dimensionality*[3]. Instead, it may be far easier to represent this system by a simulation and analyze it using reinforcement learning methods that were developed for model-free environments.

[3]This terminology was introduced by Bellman [1957] to describe the exponential growth of state spaces in multi-dimensional applications.

Robot navigation

Consider a robot with sensors dropped in a foreign landscape with no other instructions but to sense and reason autonomously in order to reach a certain target. This is an episodic problem that terminates if and when the target is reached. Developing a model would be challenging since the environment is unknown, although the actions the robot can choose are determined by its configuration and guidance system. There is no obvious intermittent reward; a reward corresponds to reaching the target.

In this new environment, the robot must learn online using trial-and-error. Model-free reinforcement learning algorithms can be used to guide the robot to its target. From this perspective, the methods are focused on learning, through experimentation, often from scratch.

Under this scenario, the robot does not have the benefit of running simulations to explore policy options, it must make decisions in real time. However, in training robots to perform tasks, it has become common practice to develop simulation models of the robot's capabilities and train the robot offline in simulations[4]. A major advantage of this approach is that using a simulator enables the robot to explore actions that cause costly damage in a real environment.

Origins

The model-based and model-free approaches have evolved largely independently within distinct academic communities. The model-based, Markov decision process framework has its roots in operations research and control theory, with applications primarily in economics, engineering, and management. The model-free, reinforcement learning approach emerged from behavioral science, neuroscience and computer science and has been applied extensively in robotics, gaming, and generative artificial intelligence.

Despite significant conceptual differences in the foundations of these disciplines, they share a common mathematical base. Ultimately, both aim to determine optimal policies, albeit in different ways.

1.1 What is a Markov decision process?

This book's framework for describing decision-making will be a Markov decision process (MDP) model. Markov decision processes excel at representing the sequential nature of decision problems where current choices probabilistically impact future outcomes and options.

Relevant information for making a decision is captured by a *state* variable. In each state, the decision-maker chooses an *action* and receives an immediate *reward* with known functional form. This action choice must weigh the trade-off between the immediate reward and future rewards, both of which may be subject to uncertainty.

[4] See, for example, NVIDIA Isaac [2025].

Finally, the system is sequential: after choosing an action, the system *transitions* probabilistically according to a known transition probability function to a new state from which the decision-making process repeats. The objective in analyzing an MDP is to find a *policy*, that is a sequence of state-dependent actions that maximizes expected accumulated rewards over the future.

The expression *dynamic programming* is often used synonymously with *Markov decision process*. This book distinguishes these terms. It views dynamic programming as an inductive algorithm for solving sequential decision problems based on decomposing them into a sequence of interrelated single-period problems. Backwards induction and value iteration correspond to applying dynamic programming to a Markov decision process.

1.1.1 A historical perspective

The expression Markov decision process originates with Bellman [1957]. His comprehensive book formulates a wide range of problems as Markov decision processes and introduces the basic concepts of states, actions, transition probabilities, rewards and optimality equations.

A description of some antecedents follows. One of the earliest published examples of a sequential decision problem was that of the eminent English mathematician Arthur Cayley. His example is a precursor of the "secretary problem"[5], which is referred to as the online dating problem in Chapter 3. He described it as follows [Cayley, 1875]:

> *A lottery is arranged as follows: There are n tickets representing rewards of $a, b, c, \ldots$ pounds respectively. A person draws once; looks at his ticket; and if he pleases draws again (out of the remaining $n-1$ tickets); looks at his ticket and if he pleases draws again (out of the remaining $n-2$ tickets) and so on, drawing in all not more than k times; and he receives the value of the last drawn ticket. Suppose he regulates his drawings in the manner most advantageous to him according to the theory of probabilities, what is the value of his expectations?*

The foundations of Markov decision process theory and application were laid in the mid-20th century. Two important (and independent) precursors were the work of Wald [1947] on statistical sequential analysis and the research of Massé [1946] on water resource management, which is described in Gessford and Karlin [1958]. However, the modern development of "dynamic programming" began with collaborations of Arrow, Bellman, Blackwell, Dvoretsky, Girschik, Isaacs, Kalman, Karlin, Kiefer, La Salle, Robbins, Shapley and Wolfowitz beginning in the late 1940s at RAND corporation. Excellent summaries of this pioneering work appear in Bellman [1957] and Arrow et al.

[5] A nice overview of the secretary problem and Cayley's original formulation appears in Ferguson [1989].

[1958]. The work of Shapley [1953] regarding value iteration in stochastic games is especially prescient.

The next important milestone was Howard's PhD dissertation research at MIT, which is colorfully described in Howard [1960]. In this work, Howard developed *policy iteration* for discounted and undiscounted Markov decision processes. A path-breaking insight was that the average reward model required distinct analyses when underlying Markov chains had single and multiple closed classes.

The 1960s saw the development of a rigorous mathematical theory of Markov decision processes. Inspired by Howard's book, the seminal paper by Blackwell [1962] developed a framework for rigorously analyzing infinite horizon, finite state and action, discounted and undiscounted Markov decision process. He formulated the model using matrix notation and established the optimality of stationary policies in a discounted model by showing that policy improvement converges. However, the most significant results in this paper concern the existence and computation of optimal policies as the discount factor approaches one. To do so, he uses Markov chain results from Kemeny and Snell [1960] concerning the fundamental matrix to relate the average and discounted cases through a partial Laurent expansion. This concept is now referred to as *Blackwell optimality*. This chain of arguments was subsequently refined in Veinott [1969a], where he introduces the concepts of n-discount optimality.

Subsequent papers [Blackwell, 1965, 1967, Strauch, 1966] advanced theoretical foundations in abstract settings. In recognition of his influential contributions, in 2024, NVIDIA Corporation introduced its Blackwell architecture for AI computing.

Other noteworthy contributions to the development of MDP theory include Derman [1970], Schweitzer and Federgruen [1977] and Puterman and Brumelle [1979]. The books Bertsekas [1987], Ross [1983], Sennott [1999] and Puterman [1994] represented milestones in the formalization and consolidation of this body of theoretical development.

Ironically, while Markov decision process research had matured by the late 1980s, laying a rigorous mathematical and algorithmic foundation for sequential decision-making, (computational) reinforcement learning emerged as an active and complementary field of research and application. In the next decade, the relationship between these two distinct disciplines began to become apparent with the publication of Barto et al. [1989] and Bertsekas and Tsitsiklis [1996].

1.1.2 Markov decision process applications

Originally, Markov decision processes found widespread application in operations research problems including inventory management, queuing control, and equipment maintenance and replacement. In addition to providing algorithms for computing optimal policies, Markov decision process methods provided an approach for determining the structure of optimal policies. This is exemplified by Scarf [1960], who established

the optimality of an (s, S)-policy[6] in a single-product inventory model with a fixed cost of placing an order (see Figure 1.1). In addition to establishing an elegant result, it enabled development of specialized computational algorithms that restricted search for optimal policies within the class of (s, S)-policies.

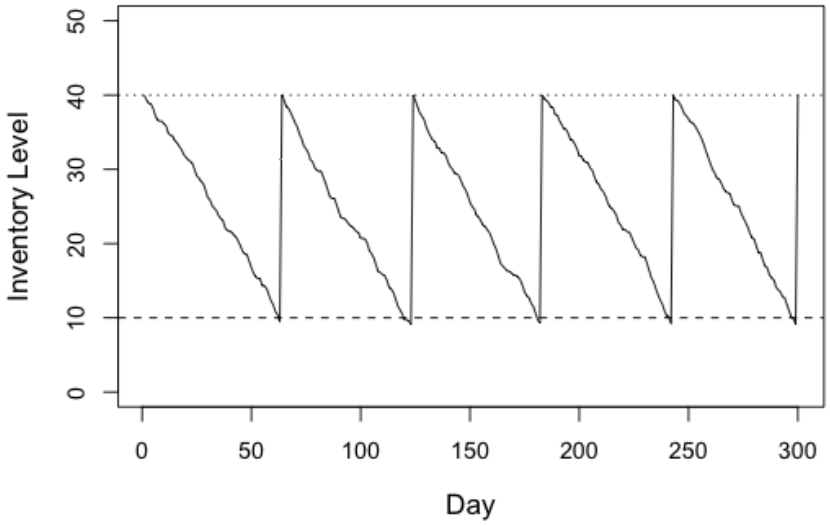


Figure 1.1: Graphical representation of the daily inventory level when controlled by an (s, S)-policy with $s = 10$ and $S = 40$. Jagged lines represent randomness in demand.

This avenue of research provided results in equipment replacement and queuing admission control models in which, under mild conditions, an optimal policy was shown to be of *control limit* form. That is, if the equipment condition deteriorated below a specified level, then it was optimal to replace it. Similarly, in a queuing admission control model, if the number of jobs in the system exceeded a specified level, then it was optimal to not admit further jobs.

Subsequently, the range of Markov decision process applications have broadened to include decision problems in:

> **Revenue Management:** Markov decision processes are used to dynamically adjust prices to maximize returns from time-limited or perishable resources. These techniques have been widely applied in the airline, hospitality, entertainment, and fashion retail industries.

[6] An (s, S) policy controls the inventory level as follows. Whenever the daily inventory level i falls below s, an (s, S)-policy places an order for $S - i$ units, raising the starting inventory level at the next day to S.

Healthcare: Applications include developing personalized treatment strategies—such as determining the optimal time to initiate therapy—and scheduling scarce resources like MRI/CT scanners and surgical staff, under uncertainty about future demand.

Finance: Markov decision process-based methods have been applied to dynamic portfolio optimization, risk management, and option pricing.

Communications Systems: Markov decision processes support the efficient allocation of network resources, including bandwidth distribution, job routing, and congestion control in dynamic environments.

Sports Analytics: Strategic decisions, such as when to pull a goalie in hockey or whether or not to punt on fourth down in American football, have been analyzed using Markov decision process models. They can also be used to optimize training regimens for individual athletes.

1.2 What is reinforcement learning?

Computational reinforcement learning (RL) refers to a collection of artificial intelligence methods that trains agents (or computer programs) to make effective decisions in dynamic environments by using feedback from actions to influence future decisions. *Behavioral reinforcement learning*, on the other hand, refers to a body of psychological research originating in the early 20th century that developed a theory of how organisms "learn through reinforcement". The expression "reinforcement learning" appears to have originated in the work of Barto and Sutton in 1980s. Over time, these two disciplines have grown to share a common vocabulary; behavioral concepts are used to describe computational models, while computational concepts have become increasingly adopted in behavioral science discourse. With the exception of this introductory chapter, this book will focus on computational reinforcement learning.

1.2.1 Trial-and-error learning

Both behavioral and computational reinforcement learning are grounded in trial-and-error experimentation. Consider, for example, the experience of a child learning to walk. Falls may lead to tears while progress may brings smiles. The child's innate reward mechanisms reinforce the movements that produce smiles and diminish those that result in tears. Over time, this reinforcement pattern shapes neural pathways that enable the child to eventually walk.

Reinforcement learning concepts have roots in behavioral science. Building on these roots, artificial intelligence research has sought to replicate trial-and-error learning on

a computer. A friend of ours[7] asked "*How can a computer be rewarded and learn?*" This insightful question strikes at the core of computational reinforcement learning.

In the language of this book, the environment is represented on the computer, explicitly or implicitly, by a Markov decision process. When code selects an action in a state the computer model generates both a reward and a new state. The computer "learns" by using this information to increase the likelihood of choosing actions with positive rewards and decrease the likelihood of obtaining negative rewards. Over repeated interactions, this process leads to the development of effective operating policies. Chapters 10 and 11 make this high-level description precise.

Behavioral reinforcement learning focuses on investigating the psychological and neurological mechanisms underlying learning. In contrast, computational reinforcement learning is concerned with designing reward systems and developing algorithms that enable agents to learn effective behavior through interaction with their environment.

1.2.2 A historical perspective

Reinforcement learning has it roots in behavioral science, neuroscience and artificial intelligence.

Behavioral antecedents

Early research in behavioral science underlies the conceptual framework of reinforcement learning.

> **Pavlov**, in the 1890s, developed a theory of "conditioned learning" or *classical conditioning* to explain his observations that a dog's salivation in response to a reward (food) could instead be triggered by an unrelated stimulus (a metronome or bell) through conditioning.
>
> **Thorndike**, in the early 1900s, developed the "law of effect" to explain the learning that cats exhibited when placed in puzzle boxes where they needed to press a lever to escape and obtain a reward (food). He observed that over time, the cats were able to escape and obtain their reward more quickly, suggesting that behaviors learned by trial-and-error influence future behavior.
>
> **Skinner**, beginning in the 1930s, developed a theory of "operant conditioning" to explain how behavior could be modified through different reinforcement regimes and schedules. He tested his theory in a controlled environment now referred to as a "Skinner box" where he was able to control the timing and frequency of rewards and punishment.

[7] Dr. Robin Friedlander, Psychologist, BC Children's Hospital

Overall, this historical work on classical and operant conditioning provided a foundation for understanding how organisms learn through reinforcement. It highlighted the role of rewards, punishments, and their frequency and timing on shaping and modifying behavior.

This line of research has increasingly focused on determining the underlying neurological mechanisms that reinforce behavior. While the exact neural level processes are complex, growing evidence suggests that dopamine release following a reward plays a critical role in reinforcing reward-producing actions by strengthening synaptic connections in the brain.

Seminal work by Schultz et al. [1997] showed that specific midbrain neurons encode *reward prediction errors* - the difference between actual and expected reward. Positive prediction errors are associated with increased release of dopamine and reinforcement of preceding actions. Conversely, negative prediction errors lead to decreased dopamine activity and suppression of the likelihood of repeating behavior. This concept aligns with the notion of *temporal differencing* in computational reinforcement learning, which is the subject of Chapters 10 and 11.

Credit assignment in behavioral reinforcement learning

It is not clear how organisms determine which actions to reinforce when a long sequence of actions is required to attain a reward. *Credit assignment* refers to the process of determining which actions or stimuli are responsible for an outcome.

Researchers in psychology and neuroscience study the mechanisms animals and humans use to solve the credit assignment problem. Their studies investigated mechanisms including action similarity, temporal discounting and dopamine signaling in the brain's reward system. Experiments, like those reported in Tang et al. [2024], explore how organisms, in this case mice, learn from controlled delayed and temporally distributed feedback. It found that when action pairs were separated by long time intervals, credit was assigned in a stepwise fashion: initially, behaviors closest in time to the reward were reinforced, followed by gradual refinement of earlier, more distant actions. As the authors note, *"Thus, a retrospective reinforcement mechanism promotes not only reinforcement, but also gradual refinement of the entire behavioral repertoire to assign credit to specific actions and action sequences that lead to dopamine release."*

This concept aligns with the notion of backward induction in Markov decision processes and computational reinforcement learning where rewards are propagated backwards from terminal nodes to earlier actions in a sequence.

Towards machine learning: Samuel's checkers

The transition from studies of learning in animals to machine learning was exemplified by the seminal work of Arthur Samuel in the 1950s. Samuel, a computer scientist at IBM, used the game of checkers [Samuel, 1959] to study how computers might learn through interacting with an environment. He chose checkers rather than chess because

"the simplicity of its rules permits greater emphasis on learning techniques".

Features of checkers that made it appealing to study were:

- Checkers is a *sequential decision process*. The current decision affects subsequent outcomes and decisions.
- Checkers is *Markovian*. The current board position summarizes the current state. No past information is necessary to determine the future.
- Checkers is *episodic*. Episodes end with a win, loss or stalemate.
- The *objective*, winning, is well-defined.
- Opponents' strategies introduce *randomness*. In most cases, the player does not have a model for the opponent's strategy.
- There is no algorithm that guarantees a win.
- The rules of checkers are well-defined and widely known.
- Checkers, including the ability to accept an opponent's moves and self-play, can be easily programmed on a computer.
- Computer-generated results can be tested in real-time against human players.

His research introduced many fundamental reinforcement learning concepts including:

Self-improvement: The goal of this research was to show that it was possible to develop a computer program that improved its outcomes over time without external interaction by repetitive play.

High dimensionality: The number of possible board configurations was extremely large.

Value function approximation: Weighted combinations of features[8] values were used to assign a "score" to each board configuration. Features with the largest weights were regarded to be most important.

Roll out: Actions (moves) were evaluated on the basis of multi-step roll out from the current board position using the value function approximation to assess the quality of the move. Using a value function approximation avoided the need to play the game to its conclusion to evaluate each alternative move.

Value function backup: The largest value from each node in the roll out was backed up and assigned to the current board position.

[8] Features included piece advantage, the number of kings and indicators of board control.

Parameter updating: Feature weights were updated on the basis of outcomes. The described updating method had some similarity with temporal differencing. Feature sets and weights were initialized on the basis of available games.

Replay: Games can start from any configuration and more than one game can be played at the same time against different opponents.

Self-play: To speed up learning and fulfill the objective of autonomy, games were played in which the player and opponent switched roles at each turn. The same value function was used for each player but the features were adjusted to accommodate the board configuration facing each player.

Empirical validation: Results were validated through play with human opponents.

A fundamental dichotomy: Distinction between general purpose methods and special purpose methods such as that developed to play checkers was discussed.

Although not noted therein, it should be obvious to the reader that many ingredients of a Markov decision process formulation and analysis, especially backwards induction, were present in Samuel's work.

Credit assignment in computational reinforcement learning

Checkers provides a classic example of the credit assignment problem. In checkers, a long series of actions (moves) leads eventually to winning, losing or ending in a draw. However, it is difficult to identify which specific moves most greatly impacted the outcome.

This challenge is fundamental to learning in computational reinforcement learning. For an agent to improve performance, it must accurately identify which actions are beneficial and which are not. If the agent fails to assign credit accurately it may reinforce ineffective actions or fail to reinforce effective ones, making the learning process counterproductive.

With the recognition of the formal relationship between computational reinforcement learning and Markov decision processes, it became apparent that *Markov decision process value functions solve the credit assignment problem.* In online methods, value functions (or policies) are updated incrementally during task execution. As a result, actions that are consequential for generating rewards increase value functions, effectively assigning more credit to beneficial actions and less credit to those with negligible impact.

For example, in a soccer game, online methods would incrementally update value estimates in real time after each pass, gradually improving understanding of how sequences of passes contribute to scoring a goal. In contrast offline or batch methods require waiting until the end of the game to adjust estimates. This online approach underlies real-time updating of odds in real-time sports gambling.

Foundations of reinforcement learning

Barto, Sutton and colleagues have been significant contributors to the development of computational reinforcement learning. Their text, Sutton and Barto [2018], provides a modern introduction to reinforcement learning from the computer science perspective. Motivated by this early work, Bertsekas and Tsitsiklis developed rigorous foundations for reinforcement learning methods from a control theory perspective. Much of their early work appears in Bertsekas and Tsitsiklis [1996].

A brief description of some fundamental references in the development of reinforcement learning follows. Barto et al. [1983] introduces many of the concepts and models underlying reinforcement learning including temporal difference (TD) learning, neural network approximations and an actor-critic architecture. It applied the actor-critic algorithm to solve the "cart-pole" problem and explored the relationship of this approach to behavioral theories.

Sutton [1988] provides an in-depth analysis of model-free temporal difference learning. The paper includes illustrative examples, a convergence proof, a discussion of the relationship to gradient descent and explores its relationship to Samuel's work. It distinguishes temporal difference learning as a form of incremental learning distinct from supervised learning, in which learning does not occur until a goal is achieved. Tsitsiklis and Roy [1997] develop and apply a rigorous foundation for analyzing temporal difference methods.

Sutton et al. [1999] introduced an approach to reinforcement learning that focuses on optimizing policies directly instead of state or state-action value functions. The approach is based on using a parametric model of a randomized decision rule. A key result, referred to as *The Policy Gradient Theorem*, provides a representation for the gradient of the value functions with respect to policy parameters that can be evaluated through simulation.

Watkins and Dayan [1992] proposed Q-learning as a method for model-free learning by shifting the object of analysis from a value function to a state-action value function. Updating the state-action value function through experience avoided the need for an underlying model. This paper described a Q-learning recursion and provided a convergence proof. Tsitsiklis [1994] explores the relationship between Q-learning and stochastic approximation and provides a proof of the convergence of Q-learning in greater generality.

Reinforcement learning applications

Reinforcement learning has demonstrated remarkable success in a variety of domains, including game playing, robotics, and natural language processing.

Perhaps motivated by Samuel's seminal study of checkers, many advances in reinforcement learning have come about through game playing including backgammon (Tesauro [1992]), Atari games (Mnih et al. [2013]), chess and Go (Silver et al. [2016]) and the Brio Labyrinth (Bi and D'Andrea [2023]). Bertsekas [2022] provides a control

theory perspective on the foundations of DeepMind's work on Go.

Reinforcement learning methods have been fundamental in driving advancements in robotics and autonomous vehicle guidance. By treating robotic systems as agents that learn from interactions with their environment, reinforcement learning has enabled the development of robots capable of performing complex tasks such as manipulation, navigation, and decision-making in dynamic settings. In the realm of autonomous vehicles, reinforcement learning has been instrumental in addressing challenges like path planning, obstacle avoidance, and adaptive control, leading to significant improvements in vehicle safety and efficiency.

Recently, reinforcement learning methods have played a fundamental role in the broad impact and remarkable progress of the generative AI applications ChatGPT, Gemini and Copilot. Reinforcement learning methods provide a framework for reward function design, policy optimization, trading off exploration with exploitation, and incorporating user feedback. Note we have occasionally called on such applications while writing this book, most notably to improve wording, generate and debug code, generate TikZ-based figures, and identify historical references.

1.3 Book structure

Figure 1.2 provides a graphical representation of the interrelationship between topics covered in this book. Note that Markov decision process and reinforcement learning methods lie in distinct boxes, ignoring the fact reinforcement learning methods are frequently used to solve large, intractable Markov decision process.

The book is divided into three parts:

- Part I establishes foundations by providing a rigorous formulation of the Markov decision process model and illustrating it through applications. It corresponds to the content itemized in the boxes **Foundations** and **Derived Quantities**, and provides an introduction to **Optimality** concepts.

- Part II covers classic Markov decision process theory and algorithms. This part of the book provides an in-depth treatment of the items in the **Optimality** and **MDP methods** boxes. For technical reasons, these methods are discussed separately for discounted, total, and average reward criteria. This part concludes with a description of partially observable Markov decision process (POMDP) models and results.

- Part III focuses on function approximation and simulation-based methods developed within the reinforcement learning literature. It corresponds to the topics in the **RL methods** box. Reinforcement learning provides a model-free perspective by developing methods that learn good policies without requiring prior knowledge of transition probabilities or reward functions.

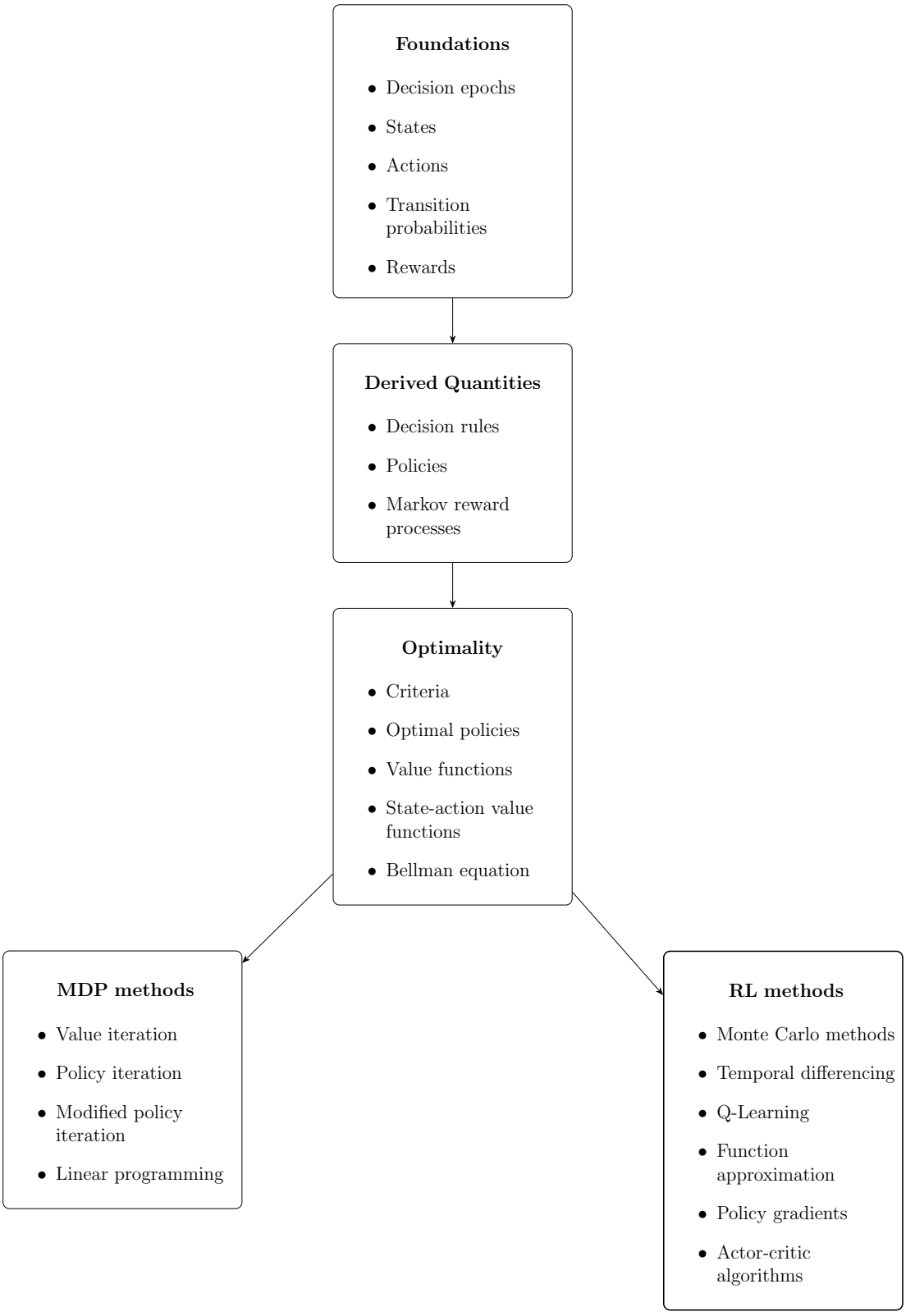


Figure 1.2: Relationship between fundamental topics in this book.

Part I - Foundations

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

Nature makes trees put down deep roots before having them bear fruit, and even this is done gradually[1].

Saint Vincent de Paul, French Catholic Priest, 1581-1660.

This part of the book contains two chapters:

- Chapter 2: Markov decision process fundamentals
- Chapter 3: Examples and applications

These chapters provide an important foundation for the rest of the book. As the above quote suggests, developing this foundation is critical to understanding and using the methods presented subsequently.

Chapter 2 introduces and discusses the core elements of a Markov decision process including basic components, derived quantities, and optimality criteria. Moreover, it introduces two simple Markov decision process models that provide a foundation for much of the material in this book:

1. The simple *one-period model* that highlights the fundamental trade-off between immediate and future rewards. It lays the groundwork for understanding a key feature of more complex, multi-period decision models.
2. The *two-state model* is a numerical example of a Markov decision process with two states. The two-state model recurs frequently throughout the book to illustrate key ideas and algorithms.

[1] de Paul (1995).

Familiarity with these models is crucial. They provide intuitive and concrete illustrations of the main Markov decision process and reinforcement learning concepts.

To make the model components in Chapter 2 concrete, Chapter 3 presents a broad range of illustrative applications drawn from diverse disciplines. Each explicitly identifies the key model components: states, actions, transition probabilities and rewards. As readers engage with these examples, they are encouraged to reflect on examples from their experience that can be formulated as Markov decision processes.

A strong grasp of model fundamentals is essential for successfully applying MDPs in new domains. For this reason, readers must take the time to carefully study the examples in Chapter 3 , paying particular attention to how the abstract model components are defined in each context.

Chapter 2

Markov Decision Process Fundamentals

Any artisan who wishes to do his job well must first sharpen his tools[1].

Confucius, Chinese philosopher, 551-479 BCE.

To effectively apply Markov decision processes and reinforcement learning, one must establish a solid grounding in the fundamental concepts and tools introduced in this chapter.

The discussion begins with a description of the core components of a Markov decision process: decision epochs, states, actions, transition probabilities, and rewards. From these elements, decision rules, policies, stochastic processes and reward processes are derived. Although not formally part of the model definition, these constructs arise naturally and inherit structure from the model's core components. Building on them, the chapter then describes optimality criteria, which are the basis for comparing the quality of decisions and underpin the concepts of optimal policies, policy and optimal value functions, state-action value functions and the Bellman equation.

The book primarily focuses on discrete time models with finite discrete state and action spaces. It briefly discusses generalizations where appropriate such as the case of partially observable Markov decision processes (Chapter 8) in which continuous state spaces arise naturally.

2.1 Basic model components

A Markov decision process model describes the fundamental elements of a recurring decision problem in which today's decision impacts future options. Decisions take place over a *planning horizon*. *Decision epochs* represent specific points of time when a decision can be made (Figure 2.1). Time is divided into *periods* or *stages*; a period begins at one decision epoch and ends at the next decision epoch.

[1] Confucius [2003].

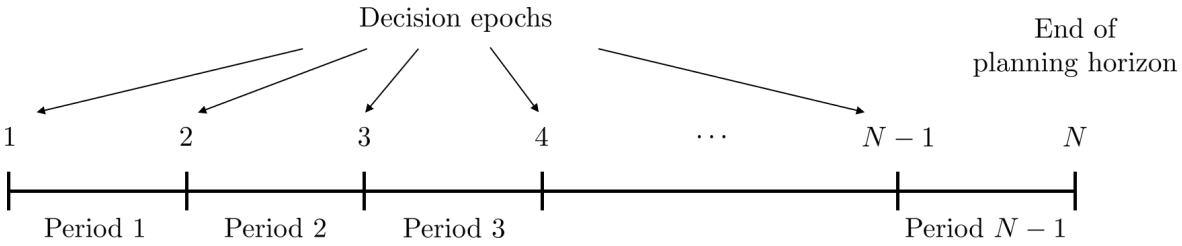


Figure 2.1: Decisions are made over a planning horizon divided into periods. The planning horizon may be finite (as shown) or infinite.

At a decision epoch, the decision maker[2] observes the state of the system, chooses an *action* and as a result of this choice, the system evolves to a new state according to a probability distribution that depends on the current state and the choice of action. After the system moves to the subsequent state, the decision maker receives a reward that depends on the starting state, action and possibly the subsequent state[3] These steps are repeated at each subsequent decision epoch. Figure 2.2 illustrates the timing of these events between two decision epochs. Such timelines are fundamental to developing a Markov decision process model. These concepts are formalized in the following subsections.

2.1.1 Planning horizons and decision epochs

The *planning horizon* is the time interval over which decisions are made. It may be *finite* or *infinite*. A *discrete time*[4] Markov decision process model is either:

- A *Finite Horizon Model*, in which the planning horizon is a bounded interval divided into $N-1$ periods, or
- An *Infinite Horizon Model*, in which the planning horizon is unbounded and divided into an infinite number of periods.

Each period begins at a *decision epoch*, a point in time at which the decision maker observes the system state and chooses an action. A period ends immediately before the subsequent decision epoch. For example, each day at midnight, a retailer's inventory management system observes the stock level of all products and decides how many additional units of each to order from suppliers. In this case, the period is a day, and the decision epoch corresponds to midnight of that day.

[2]This book primarily uses the expression *decision maker* to refer to a possibly animate entity who makes decisions. In the computer science literature, the decision maker is often referred to as an *agent* and in the engineering literature as a *controller* reflecting a reality in which an inanimate entity in some way chooses and executes decisions.

[3]The reinforcement learning literature uses the acronym SARSA to refer to the sequence: *state, action, reward, state, action.*

[4]This book will not describe continuous time models.

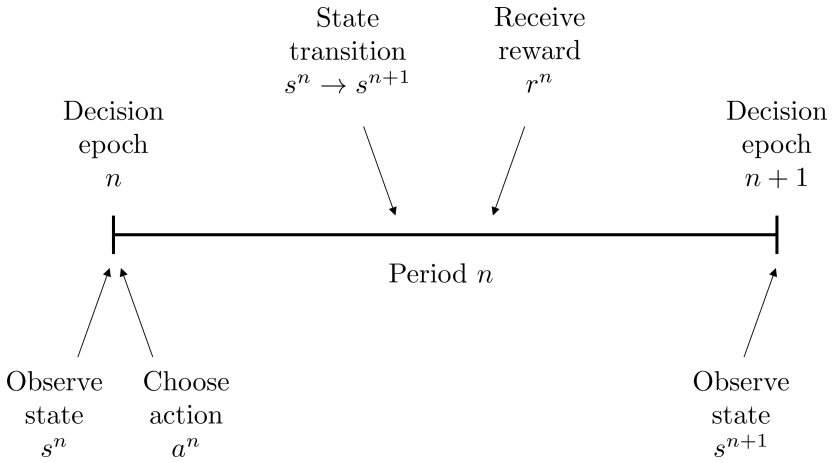


Figure 2.2: Timeline showing key events in period n. The decision maker observes state s^n at decision epoch n and chooses action a^n. The state transitions to state s^{n+1} according to the transition probabilities, the decision maker receives a reward r^n (that could depend on s^{n+1}), after which the decision maker is in the position to choose a new action at decision epoch $n+1$ after observing state s^{n+1}.

When the planning horizon is of finite length, it is divided it into $N-1$ periods with $N-1$ decision epochs. No decision is made at the end of the planning horizon, epoch N, which is referred to as the *terminal epoch*. It is included to evaluate the consequences of the decision at epoch $N-1$. Using $N-1$ also leads to cleaner formulas. Accordingly, the book uses the convention that a finite horizon problem is referred to as an $(N-1)$-period problem. In a limited number of finite horizon applications such as shortest path problems, decision epochs may index the order in which decisions are made, rather than pre-defined points in time. Chapter 4 focuses on finite horizon models, while Chapters 5 – 7 focus on infinite horizon models.

Throughout the book, "realizations", that is, states visited, actions implemented or rewards received following a decision or terminal epoch will be represented by **superscripts**. Moreover, transition probability functions and reward functions will be indexed by subscripts corresponding to the epoch to which they pertain.

In contrast, specific states and actions within the set of possible states and actions will be represented by **subscripts**[a].

[a]For example a^n will denote the action chosen at decision epoch n and a_m will denote the m-th action from a set of actions.

2.1.2 States

The state of the system contains all *relevant* information available to the decision maker at a decision epoch. This means that by knowing this information and choosing an action, the transition probabilities and rewards are fully specified. Let S denote the set of all states (the *state space*) and s denote a specific *state* in S. States are necessarily *exhaustive* and *mutually exclusive*. At each decision epoch, the system must occupy a unique state in S[5].

Elements of S may be scalar (e.g., the inventory level of a single product) or vector-valued (e.g., queue lengths of each priority class in a multi-class queuing system). They may be ordered (e.g., the number of people in a queue) or abstract (e.g., the configuration of a Tetris screen and the shape awaiting placement in the wall).

Although our primary focus in this book is on models with S discrete and finite, there are many possible generalizations, including choosing S to be countably infinite or a subset (either bounded or unbounded) of finite-dimensional Euclidean space. Also, specifically in network or decision analysis settings, S may vary with the decision epoch, in which case one may use an epoch-specific state space S_n at epoch n. An alternative is simply to define S as the union of S_n over all decision epochs n, though this approach may unnecessarily expand the state space at each decision epoch (because some states may not be reachable at certain epochs).

The book assumes that S is independent of the decision epoch.

2.1.3 Actions

Denote by A_s the set of all actions available to the decision maker in state $s \in S$. Refer to A_s as an *action set* and each $a \in A_s$ as an *action*. Actions are mutually exclusive; the decision maker cannot choose two actions at the same time, although, as seen later, the decision maker may specify a distribution over the set of actions.

Like the state space, each A_s may be a finite set, a countably infinite set, a subset of finite-dimensional Euclidean space, or an abstract set. The action set may be independent of s, as in an infinite-capacity queuing control model in which the action is to decide whether to admit or not admit a job. The book will focus on A_s being discrete and finite. Assuming the states and actions are ordered, $a_{s,j}$ represents the j-th action in the s-th state.

Note that in some states, A_s may contain a single element corresponding to "no action". Such states are *non-actionable*. These situations occur most often when the system reaches an absorbing state or set of states, Δ, from which it cannot exit. In that case, $A_\Delta = \{\text{Do nothing}\}$. This applies to episodic models (such as controlling a robot to move from an origin to a destination or in problems in which the decision maker can stop the system at any time).

[5]Unlike Schrödingers's cat, the system cannot occupy two different states at the same time.

2.1.4 Transition probabilities

Let $p_n(j|s,a)$ denote the probability that the system state becomes j at decision epoch $n+1$ when the decision maker chooses action a in state s at decision epoch n. Note that several random events may occur throughout the period between decision epochs; the transition probability does not explicitly represent each one, but instead represents the net change in state. The transition probability function[6], $p_n(j|s,a)$ has the following properties:

$$\begin{aligned} p_n(j|s,a) &\geq 0 \quad \text{for all } j \in S, a \in A_s, s \in S \\ \sum_{j \in S} p_n(j|s,a) &= 1 \quad \text{for all } a \in A_s, s \in S. \end{aligned}$$

Note that in a non-actionable state, δ, $p_n(\delta|\delta, \text{“Do nothing”}) = 1$.

When the transition probabilities do not vary over decision epochs, they are referred to as *stationary*. For infinite horizon models presented in this book, transition probabilities are assumed to be stationary, so that the subscript n is dropped.

2.1.5 Rewards

Consider two representations for a reward: $r_n(s,a,j)$ and $r_n(s,a)$. The quantity $r_n(s,a,j)$ denotes the reward received in period n when the decision maker chooses action a in state s and the system transitions to state j at decision epoch $n+1$. The latter quantity, $r_n(s,a)$, has a similar interpretation, but is independent of the subsequent state. The choice of reward function depends on the application – usually one of these two forms better describes a specific context. The examples in Chapter 3 will illustrate both.

Assume $r_n(s,a,j)$ and $r_n(s,a)$ are scalar and real-valued. Generalizations include vector-valued or abstract rewards. For example, as the result of an action in a fantasy game, the decision maker may receive a sword, a shield and a vial of magic potion. Instead of keeping track of this bundle of goods in the reward function, the decision maker might assign a numerical value to each item and be indifferent between collections of items with the same total value.

These quantities are *rewards* because the book formulates the problem of a decision maker seeking to *maximize* rewards. *Costs* can be represented as negative rewards to allow for a decision maker who seeks to minimize costs. Consequently, minimizing costs corresponds to maximizing rewards. To avoid awkward minus signs in expressions, on some occasions when minimizing costs, rewards will be replaced by $c_n(s,a)$ to represent the cost of being state s and choosing action a or $c_n(s,a,j)$ to include the possibility that the cost depends on the state at the next decision epoch j.

[6]When S represents an uncountable subset of finite-dimensional Euclidean space, the transition probability may be represented by a probability density function.

When using optimality criteria based on expected rewards[7], the quantity $r_n(s,a)$ may also represent the *expected* reward in period n where the expectation is taken over the possible states at decision epoch $n+1$:

$$r_n(s,a) = \sum_{j \in S} r_n(s,a,j) p_n(j|s,a). \tag{2.1}$$

For discrete time models, the model formulation does not account for how the reward is accumulated throughout a period between two decision epochs. For example, in an inventory control model with weekly decision epochs, the inventory levels may change during the week resulting in varying holding costs between decision epochs. A discrete time formulation accumulates all of these costs and summarizes them in the reward function for that one period.

In finite horizon models, a *terminal reward* or *scrap value* $r_N(s)$ is used to represent the consequence of ending the planning horizon in state s. In infinite horizon models, the terminal reward is omitted. Furthermore, the subscript n is deleted from the reward function in the infinite horizon setting, since the focus in that case is on *stationary* rewards.

> Rewards are an intrinsic part of the model that arise naturally from the application. However, in reinforcement learning models, the modeler may need to construct an *artificial* reward function that conforms with the objectives of the task.

2.1.6 Markov decision processes and decision trees

A decision tree provides a visual display of the basic model components. In Figure 2.3 square boxes represent states, arcs from boxes to circles represent possible actions in each state, arcs from circles to subsequent states denote transitions that occur according to a transition probability and result in a reward.

It should be evident from the decision tree representation that, in general, there is an exponential explosion in the number of possible trajectories through the tree as the length of the planning horizon is increased.

2.2 Derived objects

A Markov decision process is fully specified[8] by the five model components described in the previous section:

[7] In some applications, particularly in economic contexts, a decision maker seeks to maximize expected *utility* or some other risk sensitive criterion.

[8] Contrary to formulations in some disciplines, the discount rate is not specified as a model component. It is more appropriately associated with the optimality criterion.

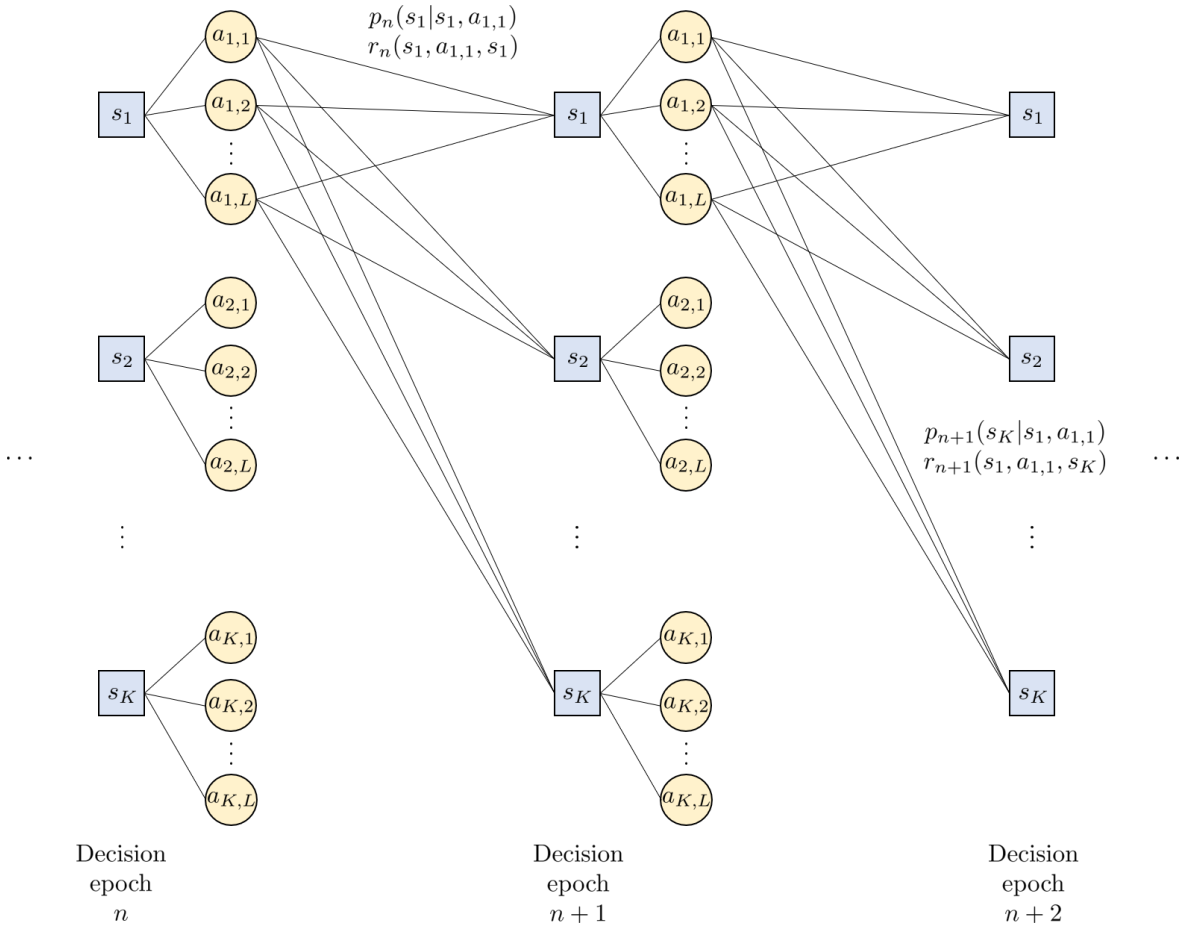


Figure 2.3: Graphical representation of a Markov decision process by a decision tree.

- the planning horizon N (which may be finite or infinite),
- the set of states S,
- the sets of actions A_s for each $s \in S$,
- the transition probabilities $p_n(j|s, a)$, and
- the rewards $r_n(s, a, j)$ or $r_n(s, a)$.

In this section, decision rules, policies, derived stochastic processes and reward processes are described. They are not part of the basic formulation, but are fundamental concepts derived from the basic model components.

2.2.1 Decision rules

A *decision rule* describes both the information and mechanism a decision maker uses to select an action in a given state at a single, specific decision epoch. Decision rules can be classified on the basis of two independent dimensions:

Information: Markovian or history-dependent

Mechanism: Deterministic or randomized

Information

The Markovian vs. history-dependent dichotomy describes the *information* used by the decision maker when choosing an action. A *Markovian* decision rule uses only the state at the current decision epoch to select actions, while a *history-dependent* decision rule considers some or all of the previous states and actions up to and including the current state when choosing an action. That is, at decision epoch n, a Markovian decision rule is a function of s^n and a history-dependent decision rule is a function of a *trajectory* $(s^1, a^1, s^2, \ldots, a^{n-1}, s^n)$[9]. Thus, a Markovian decision rule is a special case of a history-dependent decision rule in which the history is summarized in a single state. Write

$$H_n = \{h^n = (s^1, a^1, s^2, \ldots, a^{n-1}, s^n) \mid s^1 \in S, a^1 \in A_{s^1}, s^2 \in S, \ldots, a^{n-1} \in A_{s^{n-1}}, s^n \in S\} \tag{2.2}$$

as the set of all histories, h^n, leading up to decision epoch n. Note that $H_1 = S$. While the past sequence of rewards could also be explicitly included in the history, its inclusion is redundant in the Markov decision process model, since the history of states and actions alone allows one to reconstruct the past rewards through the reward functions[10]. Note that $h^1 = s^1$ and for $n = 2, 3, \ldots, N$,

$$h^n = (h^{n-1}, a^{n-1}, s^n). \tag{2.3}$$

This recursion is used explicitly in Section 4.1.3.

Mechanism

The deterministic vs. randomized dichotomy describes the *mechanism* used to select an action at a given decision epoch. A *deterministic* decision rule selects an action with certainty, while a *randomized* decision rule selects an action according to a specified probability distribution. A deterministic decision rule is a special case of a randomized decision rule corresponding to a degenerate probability distribution[11].

[9] Recall the convention that superscripts refer to the state and/or action chosen at decision epoch n. Also, history-dependent decision rules need not consider the entire history, but a subset of past actions and states.

[10] In model-free reinforcement learning, a functional form for $r_n(s, a, j)$ is not be known or specified. In this case the history may be augmented to include the reward.

[11] A degenerate probability distribution places all of the probability on one action.

Types of decision rules

The two dimensions described above lead to four classes of decision rules.

- A *Markovian deterministic* (MD) decision rule, d_n, is a function from states to actions. More formally, $d_n(s^n) = a^n$ denotes the decision rule that at decision epoch n chooses action $a^n \in A_{s^n}$ when the system is in state $s^n \in S$.

- A *history-dependent deterministic* (HD) decision rule, d_n, is a function from the set of histories to actions. More formally, $d_n(h^n) = a^n$ denotes the decision rule that chooses action $a^n \in A_{s^n}$ when the system history is $h^n \in H_n$ and s^n is the state at decision epoch n. The subtlety here is that although the decision rule's action choice may vary with history, it can only choose actions from the set A_{s^n} at epoch n, which depends only on the state at decision epoch n. This means that given two different histories h^n and $\hat{h}^n$ that both arrive at state s^n, $d_n(h^n)$ and $d_n(\hat{h}^n)$ may choose different actions, but each action will be chosen from the same action set A_{s^n}.

- A *Markovian randomized* (MR) decision rule, d_n, is a mapping from the set of states to the set of probability distributions over the action set. More formally, $w^n_{d_n}(a^n|s^n)$ denotes the probability that decision rule d_n chooses action a^n in state s^n.

- A *history-dependent randomized* (HR) decision rule, d_n, is a mapping from the set of histories to the set of probability distributions over the action set. More formally, $w^n_{d_n}(a^n|h^n)$ denotes the probability that decision rule d_n chooses action a^n given history h^n.

These classes of decision rules are denoted as D^{MD}, D^{HD}, D^{MR} and D^{HR}, respectively.

It is important to note that because of the ability to construct history-dependent decision rules, the Markov decision process formulation gives rise to a system that might not evolve in a Markovian fashion. The expression "Markov decision process" refers to the fact that rewards and transition probabilities depend on the past only through the state and action at the present decision epoch. It does not, however, mean that every stochastic process generated by this model is a Markov chain. See Section 2.2.3 below for more details on this point.

Subsequent chapters will establish the optimality of Markovian deterministic decision rules so it will be unnecessary to consider randomized decision rules when seeking optimal policies. However, randomized decision rules are fundamental in linear programming formulations of the infinite-horizon Markov decision process model (Chapters 5-7), exploration-based simulation algorithms (Chapter 10) and policy-based reinforcement learning algorithms (Chapter 11).

2.2.2 Policies

A *policy* π, sometimes referred to as a *contingency plan* or *strategy*, is a sequence of decision rules, one for each decision epoch. In finite horizon models, write $\pi = (d_1, d_2, \ldots, d_{N-1})$. In infinite horizon models, $\pi = (d_1, d_2, \ldots)$. In the classical Markov decision process setting, once an optimality criterion has been specified, the decision maker's goal is to find an optimal policy – one that maximizes or minimizes the designated criterion.

The four classes of decision rules defined in Section 2.2.1 result in four classes of policies, Π^{MD}, Π^{HD}, Π^{MR} and Π^{HR}. In addition, stationary deterministic (SD) and stationary randomized (SR) policies, denoted by the classes Π^{SD} and Π^{SR}, respectively, refer to sets of *stationary policies*, that is policies[12] that choose the same decision rule at every decision epoch. In infinite horizon models, stationary policies that use the decision rule d at every decision epoch will be denoted by d^∞, that is,

$$d^\infty := (d, d, \ldots). \tag{2.4}$$

Some comments about policies follow:

- The class Π^{HR} denotes the most general class of policies; all policies considered in the book are in this class. This level of generality is required to define optimality, but often not required to find optimal policies. For example, in finite horizon models it will be sufficient to restrict one's search for an optimal policy to the class of Markovian deterministic policies (see Section 2.4).

- Policies in Π^{HR} generally cannot be implemented in long finite-horizon or infinite horizon models because storage requirements increase exponentially with respect to the planning horizon length. Fortunately, in finite horizon models, there exist optimal policies in this class that are Markovian and deterministic, and in infinite horizon models, there exist optimal policies that are stationary and deterministic.

- Stationary policies are most relevant to infinite horizon models. Proofs and methods in Chapters 5 – 6 will use the result that there exist stationary deterministic policies that are optimal in the class of history-dependent randomized policies under several different optimality criteria.

- In finite horizon models, a Markovian deterministic policy may be characterized by a *lookup table*. A lookup table is an array in which rows correspond to states, columns correspond to decision epochs and an entry indicates which action the policy selects in that state at that decision epoch. Although a useful conceptual framework, a lookup table may be an impractical method of encoding a policy in models with a large number of states. Chapters 9 and 11 replace lookup tables

[12] In most cases, a stationary policy is necessarily Markovian.

by function approximation. Models in which lookup tables apply are referred to as *tabular models* in the reinforcement learning literature.

- Figure 2.4 summarizes the relationship between policy classes[13].

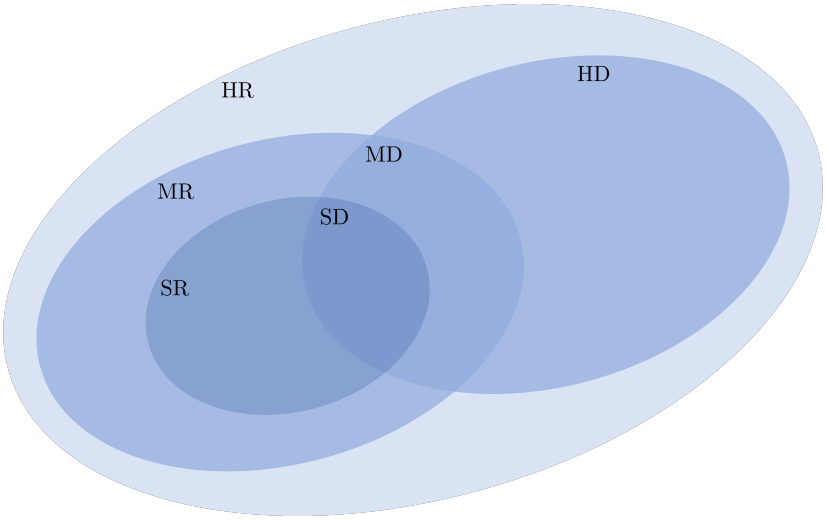


Figure 2.4: Relationships between policy classes. Labels near the edge of an oval refer to the entire oval, whereas labels inside the intersection of two ovals denote the entire intersection between them. For example, SD is the intersection of SR and HD. MD is the intersection of MR and HD, which includes SD.

2.2.3 Derived stochastic processes

Once a policy is chosen and a probability distribution over the starting state of the process is specified, the probabilistic evolution of a Markov decision process is completely determined. Let the random variable X_1 denote the initial state of the Markov decision process and let $\rho(s) := P[X_1 = s]$ denote the initial state probability distribution, which by assumption is independent of policy choice. When, as in most applications, the system starts in a specific state at decision epoch 1, say s^1, write $\rho(s) = 1$ for $s = s^1$ and $\rho(s) = 0$ for $s \neq s^1$.

Let π denote a policy for either a finite or infinite horizon problem and let the random variable Y_1 denote the action chosen by d_1 at decision epoch 1. There are two potential sources of variability that affect Y_1:

[13] Although not represented in the figure, there may exist policies that are stationary and history-dependent (i.e., non-Markovian). An example is a policy that makes decisions based on the current state as well as the starting state. However, these examples represent "edge cases" and are uncommon. Thus, in this book, only stationary policies are considered within the Markovian class.

1. Variability in X_1, which occurs when X_1 is random, and

2. Variability in action choice in X_1, which occurs if d_1 is a randomized decision rule.

Once Y_1 is chosen, the next state, X_2, is random with a distribution determined by the transition probabilities. This is true even if X_1 and Y_1 are not random, that is when the system starts in a specific state s^1 and d_1 is deterministic. Consequently, the second action Y_2 will be random as well, and so on. Thus, a policy and initial state distribution generate the stochastic process

$$(X_1, Y_1, X_2, Y_2, \ldots, X_{N-1}, Y_{N-1}, X_N) \tag{2.5}$$

in a finite horizon model and

$$(X_1, Y_1, X_2, Y_2, \ldots) \tag{2.6}$$

in an infinite horizon model. In these expressions, X_n represents the state of the stochastic process and Y_n denotes the action chosen by policy π at decision epoch n.

Some comments about policies with respect to derived stochastic processes follow:

1. When $\pi \in \Pi^{\text{MR}}$, the stochastic process is a discrete time Markov chain. See Exercise 3.

2. When π is history-dependent, the stochastic process need not be a Markov chain. That is, at each decision epoch, given a state and action, the transition probabilities may depend on all or some of the past. See Exercise 3.

3. A policy $\pi = (d_1, d_2, \ldots)$ induces a probability distribution p^π on realizations of the stochastic process $(X_1, Y_1, X_2, Y_2, \ldots)$. These realizations are referred to as *trajectories*. To determine this distribution requires enumerating realizations and multiplying the transition probabilities and action randomization distributions corresponding to each realization as follows:

   (a) For $\pi \in \Pi^{\text{HR}}$,

$$\begin{aligned} &p^\pi(s^1, a^1, s^2, a^2, s^3, \ldots) \\ &\quad = \rho(s^1) w^1_{d_1}(a^1|s^1) p_1(s^2|s^1, a^1) w^2_{d_2}(a^2|h^2) p_2(s^3|s^2, a^2) w^3_{d_3}(a^3|h^3) \ldots \end{aligned} \tag{2.7}$$

   where $h^1 = s^1$, $h^2 = (s^1, a^1, s^2)$, and so on. Recall that $w^n_{d_n}(a|h)$, defined in Section 2.2.1, denotes the probability that decision rule d_n chooses action a in state s at epoch n given history h. That is $w^n_{d_n} = P^\pi[Y_n = a]$ for $\pi = (d_1, d_2, \ldots)$.

   Note that the entire history first enters into this expression through the randomization distribution at decision epoch 2 and subsequently, but not

through the transition probabilities which only depend on the state and action. If a deterministic policy was used, the history would also only enter through the decision rule as in (2.7), but the decision rule would explicitly choose the action in each transition probability.

(b) For $\pi \in \Pi^{\text{MR}}$,

$$\begin{aligned} &p^\pi(s^1, a^1, s^2, a^2, s^3, \ldots) \\ &\quad = \rho(s^1) w^1_{d_1}(a^1|s^1) p_1(s^2|s^1, a^1) w^2_{d_2}(a^2|s^2) p_2(s^3|s^2, a^2) w^3_{d_3}(a^3|s^3) \ldots \end{aligned} \tag{2.8}$$

This expression is similar to (2.7), except that histories are replaced with states.

(c) For $\pi \in \Pi^{\text{MD}}$

$$p^\pi(s^1, a^1, s^2, a^2, s^3, \ldots) = \rho(s^1) p_1(s^2|s^1, a^1) p_2(s^3|s^2, a^2) \ldots \tag{2.9}$$

since $d_1(s^1) = a^1$, $d_2(s^2) = a^2$, and so on. In this case, transition probabilities need only be multiplied with each other (and the initial state distribution) to find the probability distribution of the stochastic process generated by π.

2.2.4 Reward processes

Section 2.2.3 showed that a policy π generates a stochastic process of states and actions represented by (2.5) in finite horizon models and (2.6) for infinite horizon models with distribution $p^\pi(\cdot)$ defined in (2.7) in the most general case. These processes generate corresponding stochastic processes of rewards

$$(r_1(X_1, Y_1, X_2), r_2(X_2, Y_2, X_3) \ldots, r_{N-1}(X_{N-1}, Y_{N-1}, X_N), r_N(X_N)) \tag{2.10}$$

in finite horizon models, and

$$(r_1(X_1, Y_1, X_2), r_2(X_2, Y_2, X_3), \ldots) \tag{2.11}$$

in infinite horizon models.

In greater generality, let these stochastic processes be represented by either

$$\mathcal{R}_N := (R_1, R_2, \ldots, R_N) \tag{2.12}$$

or

$$\mathcal{R}_\infty := (R_1, R_2, \ldots) \tag{2.13}$$

where the random variable R_n represents the reward received in period n.

Letting r^n denote a realization of R_n, the expressions

$$P[\mathcal{R}_N = (r^1, r^2, \ldots, r^n)] \quad \text{and} \quad P[\mathcal{R}_\infty = (r^1, r^2, \ldots)]$$

equal the probability of the given finite or infinite sequence of rewards. When reward processes arise in a Markov decision process, the policy-dependent probability $p^\pi(\cdot)$ replaces the generic probability $P[\cdot]$.

Simulation or observing repeated experiments in the real world will generate many different sequences of rewards. The probability distribution of $\mathcal{R}_N$ and $\mathcal{R}^\infty$ describes the likelihood of observing each sequence. In this book, there will be little occasion to explicitly compute such probability distributions, but they will provide the basis for interpreting several expressions. Most calculations will avoid this onerous task by evaluating distributions and expectations sequentially. The example in Section 4.1.2 shows how to derive the distribution of $\mathcal{R}_N$ in an Markov decision process by determining which states and actions map into the same reward sequence.

In the discussion of values in the next section, it is sufficient to work directly with the reward process described above. However, in the simulation methods in Chapters 10 and 11, realizations (trajectories) that include states, actions and rewards of the form[14]

$$(X_1, Y_1, R_1, X_2, Y_2, \ldots, R_N) \quad \text{and} \quad (X_1, Y_1, R_1, X_2, Y_2, \ldots)$$

will be considered. Thus a realization of the process will be written as either

$$(s^1, a^1, r^1, s^2, \ldots, r^N) \quad \text{or} \quad (s^1, a^1, r^1, s^2, \ldots).$$

Following convention, a stochastic process together with a reward function will be referred to as a *reward process*. When the stochastic process is a Markov chain, it is a *Markov reward process*. Markov reward processes most often arise as the sequence of rewards of a Markovian policy in a Markov decision process.

2.2.5 Assigning a value to a reward process

This section shows how reward processes lead to decision making criteria. As noted in Section 2.1.5, assume that reward functions $r_n(s, a, j)$ are real-valued so that each R_n, as defined implicitly through (2.12) or (2.13), is real-valued. Thus, the random reward sequence $\mathcal{R}_N = (R_1, R_2, \ldots, R_N)$ takes values in $\mathbb{R}^N$[15] and $\mathcal{R}_\infty = (R_1, R_2, \ldots)$ takes values in $\mathbb{R}^\infty$.

Expected utility

To assign a value to a sequence of rewards, a decision maker may use *expected utility*. Let $u(\cdot)$ denote a real-valued function on $\mathbb{R}^N$ or $\mathbb{R}^\infty$, and let $E[\cdot]$ denote expectation with respect to the probability distribution of $\mathcal{R}_N$ or $\mathcal{R}_\infty$. The expected utility of these reward sequences equals $E[u(R_1, R_2, \ldots, R_N)]$ and $E[u(R_1, R_2, \ldots)]$, respectively.

[14] Technically speaking when $r_n(\cdot, \cdot, \cdot)$ is a function of the current state, the current action and the next state, we need to observe X_{n+1} before we can evaluate the reward. So technically, at least, we should write the stochastic process $(X_1, Y_1, X_2, R_2, Y_2, \ldots, X_N, R_N)$.

[15] Denotes N-dimensional Euclidean space.

Often utility functions are additive, so

$$u(R_1, R_2, \ldots, R_N) = \sum_{n=1}^{N} u_n(R_n) \tag{2.14}$$

for N finite or infinite, where $u_n(\cdot)$ is a real-valued function for each n. When N is finite, the expected utility becomes

$$E[u(\mathcal{R}_N)] = E[u(R_1, R_2, \ldots, R_N)] = E\left[\sum_{n=1}^{N} u_n(R_n)\right]. \tag{2.15}$$

When N is infinite, define[16]

$$E[u(\mathcal{R}_\infty)] = E[u(R_1, R_2, \ldots)] := \lim_{N\to\infty} E\left[\sum_{n=1}^{N} u_n(R_n)\right]. \tag{2.16}$$

Interpreting utilities

Utilities $u_n(\cdot)$ encode a decision maker's attitude towards both risk (variability) and timing of rewards. The most common choice for $u_n(\cdot)$ is

$$u_n(r) = \lambda^{n-1} r$$

where $0 \leq \lambda \leq 1$ so that

$$E[u_n(R_n)] = \lambda^{n-1} E[u_n(R_n)].$$

Refer to the quantity λ as the *discount factor* or *discount rate*. When $\lambda = 1$ the decision maker is indifferent to the timing of rewards. When $\lambda < 1$ the decision maker prefers receiving rewards sooner than later. For example, if $\lambda = 0.95$, a decision maker would be indifferent between receiving one unit of utility next period and 0.95 units now.

When using $E[R_n]$ to make decisions, a decision maker is said to be *risk-neutral*. For example, a risk-neutral decision maker would be indifferent between the following two gambles since they have the same expected value:

- Gamble A: win \$100 with probability 1, or
- Gamble B: win \$0 with probability 0.5 and \$200 with probability 0.5.

A *risk-seeking* decision maker would prefer Gamble B and a *risk-averse* decision maker would prefer Gamble A. Convex increasing utility functions such as r^2, correspond

[16] In (2.16), it would be preferable to take the limit inside the expectation but that limit need not exist. An example below explores this point further

to risk-seeking decision making while concave increasing utility functions such as $\sqrt{r}$ correspond to risk-averse decision making[17].

As an example, consider coaching decisions in sport. If a team is behind near the end of the game, its coach may be risk-seeking in an attempt to catch up. Alternatively, if a team is ahead, its coach may choose to make more conservative decisions, based on a risk-averse utility function, in order to hold on to the lead.

The Markov decision process models considered in this book will focus on decision making by risk-neutral decision makers, so utility is replaced by rewards or discounted rewards directly. However, it is important to be aware that other utility functions apply, and optimal policies with respect to one utility function will not necessarily be optimal for a different utility function.

2.3 Optimality criteria: Transforming a Markov decision process into a Markov decision problem

Up to this point, the Markov decision process has been formulated without considering the decision maker's preferences for reward sequences generated by different policies. The following sections define and comment on the following three optimality criteria that are commonly used to evaluate and compare policies:

- expected total reward,
- expected discounted reward, and
- long-run average reward.

The expected total reward criterion applies to both finite and infinite horizon models, while the latter two are most often used in infinite horizon models. A policy is said to be *optimal* when it maximizes the appropriate criterion over the set of all history-dependent randomized policies. To be precise, the expression *Markov decision problem* will correspond to a Markov decision process *together* with an optimality criterion[18]. However, as with most of the published literature, this book will use the general phrase *Markov decision process* to refer to both cases, whether an optimality criterion is included or not.

2.3.1 Expected total reward

This section defines the expected total reward of a reward sequence and policy. Technical considerations lead us to distinguish finite horizon from infinite horizon models.

[17] Which would better reflect your attitude towards gambles? Why?

[18] We emphasize that a Markov decision process is defined independent of the optimality criterion.

Finite horizon

Define the *expected total reward* of a finite sequence of random rewards $(R_1, R_2, \ldots, R_N)$ by

$$E\left[\sum_{n=1}^{N} R_n\right] \tag{2.17}$$

where the expectation is over the distribution of reward sequences. Since many reward sequences might map into the same total reward, in theory, when R_n is discrete, this quantity can be computed by first enumerating all possible values for the sum and and then accumulating the probabilities of the trajectories with the same sum.

As noted in Section 2.2.3, a policy π generates a random sequence

$$(X_1, Y_1, X_2, Y_2, \ldots, X_{N-1}, Y_{N-1}, X_N)$$

of states and actions and consequently generates a random sequence of rewards by setting $R_n = r_n(X_n, Y_n, X_{n+1})$ or $R_n = r_n(X_n, Y_n)$ for $n = 1, 2, \ldots, N-1$ and $R_N = r_N(X_N)$.

Thus, the expected total reward of a policy $\pi \in \Pi^{\text{HR}}$ can be defined as follows.

Definition 2.1. For all $s \in S$, the *finite horizon expected total reward* of policy π is

$$v^\pi(s) := E^\pi\left[\sum_{n=1}^{N} R_n \,\middle|\, X_1 = s\right]. \tag{2.18}$$

In equation (2.18), the expectation is with respect to the probability distribution of random rewards that result from different realizations of the stochastic process generated by π with $X_1 = s$ (Section 2.2.4).

Refer to the expected total reward of policy π, $v^\pi(s)$, as its *value*. Implicit in this notation for finite horizon models is that $v^\pi(s)$ gives the value for a model with $N-1$ decision epochs and terminal epoch N[19]. The function $v^\pi(\cdot)$ will be called a *policy value function*. Note that $v^\pi(s)$ will most often be seen written as either

$$v^\pi(s) = E^\pi\left[\sum_{n=1}^{N-1} r_n(X_n, Y_n, X_{n+1}) + r_N(X_n) \,\middle|\, X_1 = s\right] \tag{2.19}$$

or

$$v^\pi(s) = E^\pi\left[\sum_{n=1}^{N-1} r_n(X_n, Y_n) + r_N(X_n) \,\middle|\, X_1 = s\right] \tag{2.20}$$

depending on the form of r_n. To simplify notation, $E_s^\pi[\cdot]$ will often be used instead of $E^\pi[\cdot|X_1 = s]$.

[19] Some authors use the notation $v_N^\pi(s)$ where N denotes the planning horizon.

Simulating the expected total reward

The following algorithm describes how to use simulation to estimate $v^\pi(s)$. It generates a single replicate for a single starting state s^1. Since the decision maker often needs $v^\pi(s)$ for all $s \in S$, this algorithm would be repeated for all states. *Monte Carlo estimates* (Chapter 10) are obtained averaging $v^\pi(s)$ over many replicates.

Algorithm 2.1. Simulating a policy value function for a deterministic policy based on a single replicate for a finite horizon model.

1. **Initialize:**
   (a) Specify $\pi = (d_1, d_2, \ldots, d_{N-1})$.
   (b) Specify $s^1 \in S$.
   (c) $n \leftarrow 1$ and $v \leftarrow 0$.
2. **Iterate:** While $n \leq N - 1$:
   (a) $a^n \leftarrow d(s^n)$.
   (b) Sample s^{n+1} from $p_n(\cdot|s^n, a^n)$.
   (c) $r^n \leftarrow r_n(s^n, a^n, s^{n+1})$.
   (d) $v \leftarrow v + r^n$.
   (e) $n \leftarrow n + 1$.
3. $v \leftarrow v + r_N(s^N)$
4. **Terminate:** Return v.

Some comments about simulating the expected total reward follow:

1. Although you might not intend to simulate this process, this algorithm describes how the process would evolve in an implementation.
2. The functions $p_n(\cdot|\cdot,\cdot)$ and $r_n(\cdot,\cdot,\cdot)$ are explicitly used in the above procedure. When this is done, the approach is said to be *model-based*. Alternatively if s^{n+1} in step 2(b) and r^n in step 2(c) were outputs of a simulation or real-time process, the approach is said to be *model-free*. This distinction will be most significant in Chapters 10 and 11.
3. If one wished to instead simulate a randomized policy, step 2(a) would be replaced by "Sample a^n from $w^n_{d_n}(\cdot|s^n)$".
4. Obvious modifications would be required to evaluate a history-dependent policy. This would be impractical if N is large and the decision rules depended on the whole past.

Chapter 4 will develop a straightforward approach to compute $v^\pi(s)$ numerically or analytically for any $s \in S$ and any π. However, in problems with large state spaces, simulation and approximation may be preferable.

Infinite horizon models

The expected total reward (value) of a policy π in an infinite horizon model can be defined as follows.

Definition 2.2. For all $s \in S$, the *infinite horizon expected total reward* of policy π is

$$v^\pi(s) := \lim_{N\to\infty} E_s^\pi \left[\sum_{n=1}^{N} R_n \right]. \tag{2.21}$$

Note that the expected total reward of a policy is defined as the limit of the expected value of its finite sums. This is because

$$\lim_{n\to\infty} \sum_{n=1}^{N} R_n$$

need not exist so that its expectation may be undefined. Equation (2.21) can be written in terms of the reward function directly as

$$v^\pi(s) = \lim_{N\to\infty} E_s^\pi \left[\sum_{n=1}^{N} r_n(X_n, Y_n, X_{n+1}) \right] \text{ or } v^\pi(s) = \lim_{N\to\infty} E_s^\pi \left[\sum_{n=1}^{N} r_n(X_n, Y_n) \right]. \tag{2.22}$$

In contrast to the finite horizon setting, it is now of concern whether the limits in (2.22) exist. Some possible limiting behaviors for this (or any) sequence include:

Convergence: when $E[R_n]$ decreases sufficiently quickly or becomes zero eventually,

Divergence: when $E[R_n]$ remains sufficiently positive or negative,

Oscillation: when $E[R_n]$ alternates between positive and negative values and does not die out.

When using the expected total reward criterion, the infinite horizon Markov decision process literature has focused on models in which the limit in (2.21) exists. At first glance, it may not be apparent how these expectations can be finite when all rewards are positive. The reason is that most examples of this kind are either *episodic models* or *optimal stopping problems*, namely, models that terminate in *reward-free* absorbing

states[20]. Such models include policies that ensure the expected time to reach one of the absorbing states is finite so that effectively they behave like finite horizon problems but with policy-specific variable horizons. Often the random horizon is referred to as the *effective horizon*. .

In such problems, the decision maker attempts to delay reaching an absorbing state as long as possible when rewards are mostly positive. When rewards are mostly negative, the decision maker attempts to reach an absorbing state as quickly as possible. These models include *transient* models and *stochastic shortest path* models. See Sections 3.5, 3.2, and 3.8 for concrete examples of such problems.

An illustrative example*

The following example justifies the above definition of the expected total reward in infinite horizon models.

Example 2.1. Consider the Markov reward process depicted in Figure 2.5. It contains three states $S = \{s_1, s_2, s_3\}$. Rewards and transition probabilities are represented in brackets below the arcs. The first number in the bracket represents the reward received if that transition takes place and the second number represents the probability of that transition. Thus, from state s_1 the system transitions to state s_2 or s_3 with probability 0.5 and generates a reward of 0. From state s_2 the Markov chain transitions to state s_3 with certainty and generates a reward of 1 and from state s_3 the system transitions to state s_2 with certainty and generates a reward of -1. No other transitions are possible.

Next consider the calculation of $v(s_1)$. There are two possible realizations of the Markov chain describing the state at epoch n, namely

$$(s_1, s_2, s_3, s_2, s_3, \ldots) \quad \text{or} \quad (s_1, s_3, s_2, s_3, s_2, \ldots).$$

Each occurs with probability 0.5. The corresponding Markov reward process generates sequences

$$\xi_1 := (0, -1, 1, -1, 1, \ldots) \quad \text{and} \quad \xi_2 := (0, 1, -1, 1, -1, \ldots).$$

each occurring with probability 0.5. Thus

$$P[\mathcal{R}_\infty = \xi_1 | X_1 = s_1] = P[\mathcal{R}_\infty = \xi_2 | X_1 = s_1] = 0.5.$$

The sequence of **total rewards** corresponding to each realization are

$$\sigma_1 := (0, -1, 0, -1, 0, \ldots) \quad \text{and} \quad \sigma_2 := (0, 1, 0, 1, 0, \ldots).$$

[20] Appendix B defines basic Markov chain concepts.

For finite N even, the total reward

$$P\left[\sum_{n=1}^{N} R_n = -1 \,\middle|\, X_1 = s_1\right] = P\left[\sum_{n=1}^{N} R_n = 1 \,\middle|\, X_1 = s_1\right] = 0.5,$$

while for N odd,

$$P\left[\sum_{n=1}^{N} R_n = 0 \,\middle|\, X_1 = s_1\right] = 1.$$

Next let $N \to \infty$. Since both possible total reward series oscillate, neither σ_1 or σ_2 has a limit. Hence the quantity $\lim_{N\to\infty} \sum_{n=1}^{N} R_n$ does not exist, so that the expression

$$E\left[\lim_{N\to\infty} \sum_{n=1}^{N} R_n \,\middle|\, X_1 = s_1\right]$$

is not well defined. However for any N,

$$E\left[\sum_{n=1}^{N} R_n \,\middle|\, X_1 = s_1\right] = 0,$$

so

$$\lim_{N\to\infty} E\left[\sum_{n=1}^{N} R_n \,\middle|\, X_1 = s_1\right] = 0$$

so that the limit in (2.21) exists.

This is the reason the expected total reward in an infinite horizon model is defined as the "limit of expected partial sums" as opposed to the "expectation of the limit of partial sums".

Note the limit can be passed inside the expectation by considering the *lim inf* and *lim sup* of the sequence of partial sums. These quantities always exist and are well defined. Thus, in this example

$$P\left[\limsup_{N\to\infty} \sum_{n=1}^{N} R_n = 0 \,\middle|\, X_1 = s_1\right] = P\left[\limsup_{N\to\infty} \sum_{n=1}^{N} R_n = 1 \,\middle|\, X_1 = s_1\right] = 0.5$$

so that

$$E\left[\limsup_{N\to\infty} \sum_{n=1}^{N} R_n \,\middle|\, X_1 = s_1\right] = 0.5.$$

Moreover

$$E\left[\liminf_{N\to\infty} \sum_{n=1}^{N} R_n \,\middle|\, X_1 = s_1\right] = -0.5.$$

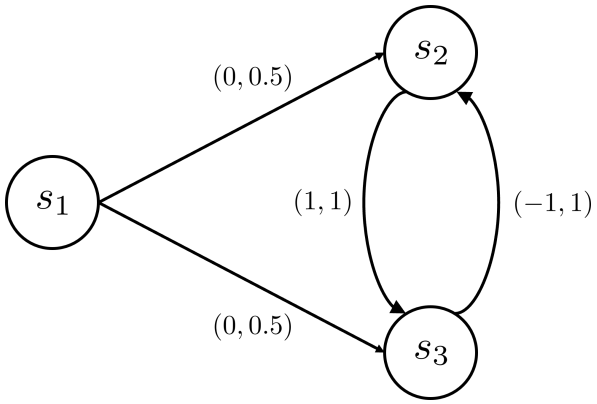


Figure 2.5: Markov reward process analyzed in Example 2.1. The labels (r, p) refer to the reward and transition probabilities, respectively.

Optimal policies

Definition 2.3. A policy π^* is *optimal* if

$$v^{\pi^*}(s) \geq v^{\pi}(s) \tag{2.23}$$

for all $s \in S$ and $\pi \in \Pi^{\text{HR}}$.

Chapter 4 analyzes finite horizon Markov decision process models and Chapter 6 considers infinite horizon models under this optimality criterion.

2.3.2 Expected discounted reward

In contrast to the expected total reward criterion, the expected discounted reward criterion accounts for the "time value of money" – that is, receiving a reward at some future epoch is worth less than receiving an identical reward now. This is the most widely used criterion in infinite horizon models.

Finite horizon models

Define the *expected discounted reward* of the reward sequence $\mathcal{R}_\infty = (R_1, R_2, \ldots)$ by

$$E\left[\sum_{n=1}^{N} \lambda^{n-1} R_n\right] \tag{2.24}$$

where the expectation is respect to the distribution of the sequence of rewards and the quantity $0 \leq \lambda < 1$ is the *discount factor*. In finite horizon models, discounting has no impact on theory or algorithms, but may affect a decision maker's preference for policies. For example, with a discount factor close to 0, a decision maker will prefer

actions that lead to larger immediate rewards, in contrast to a situation with a discount factor close to 1 when future rewards would be of greater significance.

Definition 2.4. For all $s \in S$, the *finite horizon expected discounted reward* of policy π is

$$v_\lambda^\pi(s) := E_s^\pi \left[\sum_{n=1}^{N} \lambda^{n-1} R_n \right]. \tag{2.25}$$

This definition is analogous to the expected total reward case. Above, the expectation is respect to the probability distribution of random rewards that result under different realizations of the stochastic process determined by π as described in Section 2.2.4.

Infinite horizon models

Discounting plays a fundamental role in the application and analysis of infinite horizon models and is the most studied optimality criterion.

Definition 2.5. For all $s \in S$, the *infinite horizon expected discounted reward* of policy π is

$$v_\lambda^\pi(s) := \lim_{N\to\infty} E_s^\pi \left[\sum_{n=1}^{N} \lambda^{n-1} R_n \right]. \tag{2.26}$$

Unlike in the expected total reward case, no special model structure is required for the infinite sum (2.26) to converge, only that $E[\lambda^{n-1} R_n]$ decays sufficiently quickly. To ensure this, it is assumed throughout the book that rewards are bounded. This means that there exists a finite M for which,

Bounded reward assumption:

$$|r(s,a,j)| \leq M \text{ for all } a \in A_s,\ s \in S \text{ and } j \in S. \tag{2.27}$$

Since the sum of a geometric series satisfies $\sum_{n=1}^{\infty} \lambda^{n-1} = 1/(1-\lambda)$, under the bounded reward assumption it follows that

$$\frac{-M}{1-\lambda} \leq v_\lambda^\pi(s) \leq \frac{M}{1-\lambda} \tag{2.28}$$

for all $\pi \in \Pi^{\text{HR}}$. Equation (2.28) shows the key role of the assumption that $\lambda < 1$. As noted previously when $\lambda = 1$ the series may not converge even when rewards are bounded.

Note that in contrast to the infinite horizon expected total reward, the quantity

$$\lim_{N\to\infty} \sum_{n=1}^{N} \lambda^{n-1} R_n$$

exists in a discounted model[21].

Hence it follows[22] that

$$\lim_{N\to\infty} E_s^\pi \left[\sum_{n=1}^{N} \lambda^{n-1} R_n \right] = E_s^\pi \left[\lim_{N\to\infty} \sum_{n=1}^{N} \lambda^{n-1} R_n \right] = E_s^\pi \left[\sum_{n=1}^{\infty} \lambda^{n-1} R_n \right]. \tag{2.29}$$

where the expression on the right is a shorthand for the middle expression.

[21] This is true because when rewards are bounded, the "tail" of the sum can be made arbitrarily small for every trajectory.

[22] The result follows from the bounded convergence theorem.

Example 2.1 (ctd.) Returning to Example 2.1, this example shows that when $\lambda < 1$, the limit of the sequence of partial sums for each of the discounted sequences of rewards exists and is finite.

For ξ_1, the sequence of its discounted partial sums σ_1' is

$$\sigma_1' = (0, -\lambda, -\lambda + \lambda^2, -\lambda + \lambda^2 - \lambda^3, \ldots)$$

and for ξ_2, the sequence of partial sums is

$$\sigma_1' = (0, \lambda, \lambda - \lambda^2, \lambda - \lambda^2 + \lambda^3, \ldots).$$

Thus the limit for ξ_1 is given by[a]

$$\sum_{n=1}^{\infty} \lambda^{n-1} R_n = -\frac{\lambda}{1+\lambda}.$$

Similarly, the limit of the sum of discounted rewards of sequence ξ_2 equals $\lambda/(1+\lambda)$. Thus when $\lambda < 1$

$$E\left[\lim_{N\to\infty} \sum_{n=1}^{N} \lambda^{n-1} R_n \,\middle|\, X_1 = s_1\right] = -0.5\frac{\lambda}{1+\lambda} + 0.5\frac{\lambda}{1+\lambda} = 0.$$

Since $\sigma_1' + \sigma_2' = (0, 0, 0, \ldots)$, it follows that

$$\lim_{N\to\infty} E\left[\sum_{n=1}^{N} \lambda^{n-1} R_n \,\middle|\, X_1 = s_1\right] = 0,$$

since the expectation equals 0 for any finite N. Hence (2.29) holds in this model.

[a] To see this, let $X = 1 - \lambda + \lambda^2 + \ldots$. Then $X = 1 - \lambda X$ so that $X = 1/(1+\lambda)$. Since the limit for $\xi_1 = X - 1$, the result follows.

An alternative and important interpretation of discounting

Discounting arises naturally in models where rewards are units of currency. Due to inflation, near-term rewards are preferable to future rewards. Discounting also arises in another, rather surprising way which extends its applicability.

Suppose the planning horizon length is not fixed but represented by a random variable, T, distributed according to a geometric distribution[23] with parameter λ, independent of $(R_1, R_2, \ldots)$ or $(X_1, Y_1, X_2, Y_2, \ldots)$.

[23] A random variable T follows a *geometric distribution* with parameter λ if $P[T = k] = (1-\lambda)\lambda^{k-1}$ for $k = 1, 2, \ldots$.

Let the expected total reward over a random horizon of length T be

$$E_T\left[E\left[\sum_{n=1}^{T} R_n\right]\right], \tag{2.30}$$

where E_T denotes the expectation with respect to T. Assuming R_n is bounded and $0 \leq \lambda < 1$,

$$\begin{aligned} E_T\left[E\left[\sum_{n=1}^{T} R_n\right]\right] &= E\left[\sum_{k=1}^{\infty}(1-\lambda)\lambda^{k-1}\sum_{n=1}^{k} R_n\right] \\ &= E\left[\sum_{n=1}^{\infty}\sum_{k=n}^{\infty}(1-\lambda)\lambda^{k-1}R_n\right] = E\left[\sum_{n=1}^{\infty}\lambda^{n-1}R_n\right] \end{aligned}$$

where the last inequality follows from the relationship $\sum_{k=n}^{\infty}\lambda^{k-1} = \lambda^{n-1}/(1-\lambda)$. The assumptions on R_n and λ allow interchange of the order of summation. Thus

$$E_T\left[E\left[\sum_{n=1}^{T} R_n\right]\right] = E\left[\sum_{n=1}^{\infty}\lambda^{n-1}R_n\right]. \tag{2.31}$$

When the rewards are generated by policy π starting from state s, this representation of the expected discounted reward follows:

Alternative representation for the expected discounted reward:

$$v_\lambda^\pi(s) = E_T\left[E_s^\pi\left[\sum_{n=1}^{T} R_n\right]\right]. \tag{2.32}$$

This result provides an alternative interpretation for discounting. Since the random variable T may be regarded as the time until the first "failure" in an independent, identically distributed series of Bernoulli trials with "success" probability λ, it means that when a decision maker uses expected total reward to evaluate policies in a system that terminates at the time of a random failure independent of the decision maker's policy it is equivalent to using the expected discounted reward. This is particularly appropriate in applications where the process can terminate suddenly for exogenous reasons. Examples include:

- **Animal behavior modeling**, when the animal might die of unanticipated causes (e.g., predation) independent of its decision making. See Section 3.5.
- **Clinical decision making**, in which a patient may die as a consequence of some event independent of the disease being treated. See Section 3.6.

Thus, discounting makes sense even in models that do not use economic values for rewards.

Note this provides an approach for estimating a discounted reward through simulation. The two steps are:

1. Generate T from a geometric distribution.
2. Apply Algorithm 2.1.

Optimal policies

Definition 2.6. A policy π^* is *discount optimal* if

$$v_\lambda^{\pi^*}(s) \geq v_\lambda^{\pi}(s) \tag{2.33}$$

for all $s \in S$ and $\pi \in \Pi^{\text{HR}}$.

Chapter 5 analyzes infinite horizon Markov decision process models under this optimality criterion.

2.3.3 Long-run average reward for an infinite horizon model

In contrast to discounting, which emphasizes short term behavior, the long-run average reward criterion focuses on steady state or limiting behavior of derived stochastic processes. For that reason, the long-run average reward is most appropriate for infinite horizon, non-terminating models with frequent decision epochs. Note that in finite horizon models, the average reward is equivalent to the expected total reward (why?).

As an example, consider a queuing system in which the decision maker inspects the system state very frequently, such as every second, and decides which service rate to use. Section 3.4 provides a rigorous formulation of such a problem. For such a problem:

- The **expected total reward** criterion would not be able to distinguish between policies because expected total costs or rewards would be unbounded.
- The **expected discounted reward** criterion would not be appropriate because the decision maker is interested in long-term system performance. Given the time scale of decision making, rewards at future decision epochs should **not** be less valuable than current rewards. However, if random failure of the system could occur discounting may be appropriate but it would require discount rates very close to 1.

The *average reward*[24] or more accurately the *long-run average reward* of the reward

[24] This quantity is interchangeably referred to as the *long-run average reward* or *gain*.

sequence $(R_1, R_2, \ldots)$ is given by

$$\lim_{N\to\infty} \frac{1}{N} E\left[\sum_{n=1}^{N} R_n\right].$$

When rewards are generated by a policy π, the average reward is defined as follows.

Definition 2.7. For all $s \in S$, the *average reward*, $g^\pi(s)$, of policy π is

$$g^\pi(s) := \lim_{N\to\infty} \frac{1}{N} E_s^\pi\left[\sum_{n=1}^{N} R_n\right]. \tag{2.34}$$

Note that the limit in (2.34) exists for all stationary policies when S is finite, as will be shown in Chapter 7. It need not exist when S is countable or policies are history-dependent. In such cases replace the limit above by the "lim inf" or the "lim sup" (see Chapter 7). The quantity g^π is sometimes referred to as the *gain* because the expected total reward in state s grows ("gains value") at rate $g^\pi(s)$ per epoch in the limit.

Similarly to the discounted model, when rewards are bounded,

$$\lim_{N\to\infty} \frac{1}{N} E_s^\pi\left[\sum_{n=1}^{N} R_n\right] = E_s^\pi\left[\lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} R_n\right]. \tag{2.35}$$

Example 2.1 (ctd.) Returning to Example 2.1, this example shows that when rewards are bounded, the limit of the sequence of the average partial sums for each of the sequences of rewards exists and is finite.

For ξ_1, the sequence of average partial sums σ_1'' is

$$\sigma_1'' = \left(0, -\frac{1}{2}, 0, -\frac{1}{4}, \ldots\right)$$

so that its limit exists and equals 0. Similarly, the limit of average of partial sums for ξ_2 equals 0. Thus

$$E\left[\lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} R_n \middle| X_1 = s_1\right] = 0.$$

since the expectation equals 0 for any finite N. Hence it is easy to see that (2.35) holds in this model.

Optimal policies

Definition 2.8. A policy π^* is *average reward optimal* or *gain optimal* if

$$g^{\pi^*}(s) \geq g^{\pi}(s) \tag{2.36}$$

for all $\pi \in \Pi^{\text{HR}}$ and $s \in S$.

Chapter 7 analyzes infinite horizon Markov decision process models under this optimality criterion.

2.4 The one-period problem: A fundamental building block

One-period models are the building blocks of Markov decision processes. Most of the algorithms presented later in the book are based on decomposing a multi-period model into a series of interlinked one-period models.

A one-period model begins with a decision epoch, evolves for one period of time and ends at the terminal non-decision epoch. Thus, it is the simplest representation of a finite horizon Markov decision process, namely the case of $N = 2$. In it, a policy is a single Markovian decision rule and the derived stochastic process is (X_1, Y_1, X_2). Note that in a one-period model, history-dependent policies and Markovian policies are equivalent because the only information available to the decision maker at the first (only) decision epoch is the state.

Figure 2.6 illustrates the timing of events and the nomenclature of the one-period problem.

Let S denote the state space and A_s denote the sets of actions defined for each $s \in S$. Assume that S and each A_s are discrete and finite. Let $r_1(s, a, j)$ denote the reward function in period 1, $p_1(j|s, a)$ denote the transition probability function in period 1, and $r_2(j)$ denote the terminal reward function. Analysis of this simple model provides intuition for more complex models.

Definition 2.9. Given a policy π in a one-period problem, the *policy value function* denoted $v^{\pi}(s)$, is the expected total reward when the process starts in state s at decision epoch 1. It is given by

$$v^{\pi}(s) := E^{\pi}[r_1(X_1, Y_1, X_2) + r_2(X_2)|X_1 = s], \tag{2.37}$$

where the expectation is with respect to the probability distribution of (X_1, Y_1, X_2) induced by policy π when the system starts in state $s \in S$.

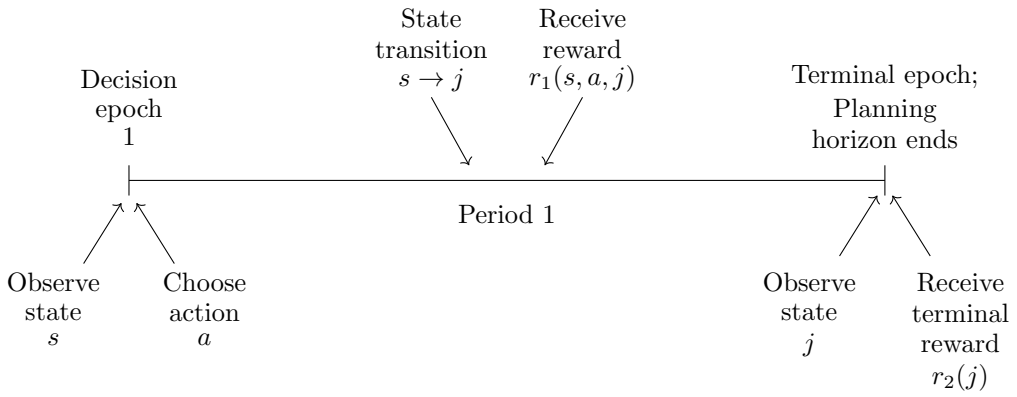


Figure 2.6: Illustration of timing of events and notation for the one-period problem. Action a may be chosen deterministically or probabilistically. Technically speaking, there is no need to observe state j since it is only used to evaluate the terminal reward $r_2(j)$.

Suppose $\pi = (d_1)$ is deterministic and $d_1(s) = a'$. Since s is fixed and $d_1(s) = a'$, the only random quantity in (2.37) is X_2. Under this assumption, (2.37) is equivalent to

$$v^\pi(s) = \sum_{j \in S} p_1(j|s, a')(r_1(s, a', j) + r_2(j)). \tag{2.38}$$

Therefore, computing $v^\pi(s)$ requires only $p_1(j|s, a') = P[X_2 = j|X_1 = s, Y_1 = a']$; the distribution of $r_1(X_1, Y_1, X_2) + r_2(X_2)$ does not need to be explicitly derived. Consequently, this avoids an enumeration of all of the latter's possible values as described in Section 2.2.4. Expressions of the form (2.38) appear frequently in the book, so it is important to understand why (2.38) is equivalent to (2.37).

When d_1 is randomized, Y_1 is also random with $P[Y_1 = a|X_1 = s] = w^1_{d_1}(a|s)$. In this case

$$v^\pi(s) = \sum_{a' \in A_s} w^1_{d_1}(a'|s) \sum_{j \in S} p_1(j|s, a')(r_1(s, a', j) + r_2(j)). \tag{2.39}$$

In both cases, $v^\pi(s)$ is referred to as the *value* of policy π.

2.4.1 Optimal value functions

The following definition is fundamental.

Definition 2.10. The *optimal value function* in a finite horizon Markov decision process, $v^*(s)$, satisfies

$$v^*(s) := \sup_{\pi \in \Pi^{\mathrm{HR}}} v^\pi(s) \tag{2.40}$$

for all $s \in S$.

Sometimes, the optimal value function is simply referred to as the *value function*. The adjective "optimal" distinguishes it from a policy value function.

In the one-period model, $\Pi^{\mathrm{HR}} = \Pi^{\mathrm{MR}}$. This identity holds because in a one-period model, the only decision is at the first decision epoch and the history at that decision epoch is the state s. Thus in the one-period model,

$$v^*(s) = \sup_{\pi \in \Pi^{\mathrm{MR}}} v^\pi(s), \tag{2.41}$$

where $v^\pi(s)$ is given by (2.39). As a result of (2.39) and (2.41)

$$v^*(s) = \sup_{w \in \mathscr{P}(A_s)} \left\{ \sum_{a \in A_s} w(a) \sum_{j \in S} p_1(j|s,a)(r_1(s,a,j) + r_2(j)) \right\}, \tag{2.42}$$

where $\mathscr{P}(A_s)$ denotes the set of probability distributions on A_s. The reason that (2.42) holds is that there is a one-to-one relationship between the set $\mathscr{P}(A_s)$ and the set of Markovian randomized decision rules. Since $\mathscr{P}(A_s)$ is an uncountable infinite set[25], *sup*[26] instead of *max* appears in (2.42).

The following easily proved lemma is fundamental here and in latter chapters.

Lemma 2.1. Let U be an arbitrary finite set, let $f(\cdot)$ be a real-valued function on U, $\mathscr{P}(U)$ denote the set of probability distributions on U and $w \in \mathscr{P}(U)$. Then

$$\max_{u \in U} f(u) \geq \sum_{u \in U} w(u) f(u) \tag{2.43}$$

and

$$\max_{u \in U} f(u) = \sup_{w \in \mathscr{P}(U)} \sum_{u \in U} w(u) f(u). \tag{2.44}$$

Proof. To prove (2.43)

$$\sum_{u \in U} w(u) f(u) \leq \sum_{u \in U} w(u) \left(\max_{u \in U} f(u) \right) = \max_{u \in U} f(u).$$

[25] For each state, the set of randomized decision rules in that state can be represented by a unit simplex corresponding to *all* probability distributions on A_s. The set Π^{MR} is equivalent to the Cartesian product of these simplices.

[26] When maximizing over a non-finite set, "sup" is used even when it is attained to emphasize that the set is not finite. This point is explored further in the book appendix.

To prove (2.44), choose $u^* \in \arg\max_{u \in U} f(u)$ and define $w^* \in \mathscr{P}(U)$ by $w^*(u) = 1$ if $u = u^*$ and $w^*(u) = 0$, if $u \neq u^*$. Then, by the definition of w^* and (2.43),

$$\sum_{u \in U} w^*(u) f(u) = \max_{u \in U} f(u) \geq \sum_{u \in U} w(u) f(u)$$

for all $w \in \mathscr{P}(U)$ so the "sup" is attained by w^* and the result follows. □

Applying Lemma 2.1 to (2.42) gives the first key result for one-period models.

Theorem 2.1. The optimal value function in a one-period model satisfies

$$v^*(s) = \max_{a \in A_s} \left\{ \sum_{j \in S} p_1(j|s,a)(r_1(s,a,j) + r_2(j)) \right\} \tag{2.45}$$

for all $s \in S$.

Proof. Choose $s \in S$, set $U = A_s$ and

$$f(s) = \sum_{j \in S} p_1(j|s,a)(r_1(s,a,j) + r_2(j)),$$

and apply (2.44). □

The main consequence of this result is that in a one-period problem the value can be obtained by restricting attention to Markovian deterministic policies, which are equivalent to Markovian deterministic decision rules. The following corollary states this result more formally.

Corollary 2.1. In a one-period model,

$$v^*(s) = \max_{\pi \in \Pi^{\mathrm{MD}}} v^\pi(s) \tag{2.46}$$

for all $s \in S$.

The result is expressed this way so that it generalizes to multi-period models.

2.4.2 Optimal policies

Attention now turns to identifying optimal policies. The concept of an "arg max" is critical and will be used throughout the book.

Definition 2.11. Let U denote an arbitrary set and $f(u)$ denote a real-valued function on U that attains its maximum on U. The *arg max* of $f(u)$ is defined by

$$\arg\max_{u\in U} f(u) := \left\{ u^* \in U \mid f(u^*) = \max_{u\in U} f(u) \right\}. \tag{2.47}$$

The "arg max" of a function returns the set of arguments that maximize the function. Thus, in general, the arg max is a set with multiple elements. Only when there is a unique maximizer of a function does the arg max return a single value. Of course the definition assumes that the maximum if attained. If not, the arg max is empty.

The following theorem shows how to identify optimal policies in the one-state model.

Theorem 2.2. In a finite state and action one-period Markov decision problem, let $d_1^*(\cdot)$ denote a deterministic decision rule that for each $s \in S$ satisfies

$$d_1^*(s) \in \arg\max_{a\in A_s} \left\{ \sum_{j\in S} p_1(j|s,a)(r_1(s,a,j) + r_2(j)) \right\}. \tag{2.48}$$

Then the policy $\pi^* = (d_1^*)$ is an optimal policy.

Proof. From (2.48) and (2.38),

$$v^{\pi^*}(s) = \sum_{j\in S} p_1(j|s,d_1^*(s))\big(r_1(s,d_1^*(s),j) + r_2(j)\big) \tag{2.49}$$

$$= \max_{a\in A_s} \left\{ \sum_{j\in S} p_1(j|s,a)(r_1(s,a,j) + r_2(j)) \right\} = v^*(s) \tag{2.50}$$

for all $s \in S$. Hence from (2.41), for all $s \in S$, $v^{\pi^*}(s) \geq v^{\pi}(s)$ for all $\pi \in \Pi^{\mathrm{MR}}$ so that it is an optimal policy. □

Some observations follow:

1. **Greedy actions:** Any action that achieves the maximum in the right hand side of (2.48) is referred to as a *greedy* action.

2. **Independent problems:** To find an optimal policy in this model, one solves an independent problem (2.48) for each state $s \in S$.

3. **Inter-temporal trade-offs:** Expressions like

$$\max_{a\in A_s} \left\{ \sum_{j\in S} p_1(j|s,a)(r_1(s,a,j) + r_2(j)) \right\}$$

appear frequently in Markov decision process theory and algorithms. This maximization highlights the trade-off between the immediate reward $r_1(s, a, j)$ and the future reward $r_2(j)$. This notion of balancing a current or *myopic* reward with future rewards is fundamental to Markov decision processes.

4. **Future values:** The future reward is encapsulated in the model component $r_2(X_2)$ in the one-period model. An important question in multi-period finite or infinite horizon models is: *Can the future reward be replaced with a single function that captures the cumulative reward associated with system evolution beyond the current decision epoch*? The answer is "yes". Subsequent chapters elaborate on this key concept.

5. **Rollout and approximations:** In modern applications with very large state spaces, the terminal reward might represent an approximation to the "value" of being in state s that has been determined "offline". Then to determine the next action in state s "online", the decision maker solves the one-period problem and uses the greedy action. Such an approach has been used to determine good strategies in checkers, chess, go and backgammon.

2.4.3 State-action value functions

A significant development in computer-based model-free reinforcement learning is to shift the focus from value functions to *state-action value functions*. While not essential here and in Chapters 4-8, they play a key role in Chapters 10 and 11.

In general, a state-action value function gives the expected total reward (or discounted reward) when choosing action a in state s. For problems with planning horizons exceeding $N = 2$, the state-action value function must be distinguished according to whether the decision maker follows a specific policy or the optimal policy after action specification. This distinction is irrelevant in one-period problems in which there are no decisions after the first decision epoch.

Definition 2.12. In a one-period problem, the *state-action value function*, $q(s, a)$, is defined for all $a \in A_s$ and $s \in S$ by

$$q(s, a) := \sum_{j \in S} p_1(j|s, a)(r_1(s, a, j) + r_2(j)). \tag{2.51}$$

State-action value functions often referred to as q-functions.

The state-action value function provides the decision maker with all the information necessary to find policy value functions, optimal policies and optimal value functions. Let $\pi = (d_1)$ denote an arbitrary deterministic policy. Then from (2.38), its value is given by

$$v^\pi(s) = q(s, d_1(s)). \tag{2.52}$$

Moreover, as a direct result of Theorem 2.1,

$$v^*(s) = \max_{a \in A_s} q(s,a) \tag{2.53}$$

and from Theorem 2.2

$$d_1^*(s) \in \arg\max_{a \in A_s} q(s,a). \tag{2.54}$$

Finding $d_1^*(s)$ using (2.54) is equivalent to greedy action choice with the q-function.

Figure 2.6 provides another way to distinguish state-action value functions from optimal value functions. The optimal value function corresponds to the largest total reward that can be obtained when the system starts in state s *before* choosing an action at the first decision epoch, while the (optimal) state-action value function refers to the optimal total reward *after* choosing action a in state s. Since there are no further decisions after action choice at decision epoch 1, the state-action value function does not involve a maximization.

As covered in Part III of this book, one might choose to estimate $q(s,a)$ by simulation. In such a situation, (2.52) can be used to find the value of a policy and (2.54) and (2.53) to find an optimal policy and its value, respectively.

2.4.4 Summary of results for a one-period problem

Putting this all together, the following has been demonstrated for a one-period model:

1. There exists a Markovian deterministic policy $\pi^* = (d_1^*)$ that is optimal in the class of all policies.

2. To "solve" the one-period problem under the expected total reward criterion, compute $v^*(s)$ for all $s \in S$ according to

$$v^*(s) = \max_{a \in A_s} \left\{ \sum_{j \in S} p_1(j|s,a)(r_1(s,a,j) + r_2(j)) \right\} \tag{2.55}$$

   and choose $d_1^*(s)$ so that

$$d_1^*(s) \in \arg\max_{a \in A_s} \left\{ \sum_{j \in S} p_1(j|s,a)(r_1(s,a,j) + r_2(j)) \right\}. \tag{2.56}$$

3. The expected total reward of the optimal policy $\pi^* = (d_1^*)$, satisfies

$$\begin{aligned} v^{\pi^*}(s) &= \sum_{j \in S} p_1(j|s, d_1^*(s))(r_1(s, d_1^*(s), j) + r_2(j)) \\ &= \max_{a \in A_s} \left\{ \sum_{j \in S} p_1(j|s,a)(r_1(s,a,j) + r_2(j)) \right\} = v^*(s). \end{aligned}$$

The goal in later parts of the book will be to generalize these ideas to multi-period models under different optimality criteria.

2.5 A two-state model

This section introduces a simple example, frequently referred to as *the two-state model*, that illustrates model components and notation, and previews calculations that will be seen later in the book. It formulates a finite horizon version; an infinite horizon version is analyzed in depth in later chapters. To simplify notation assume the transition probabilities and rewards are stationary, that is, they do not vary from decision epoch to decision epoch. Moreover, the reward is a function of the current state, the current chosen action, and the subsequent state.

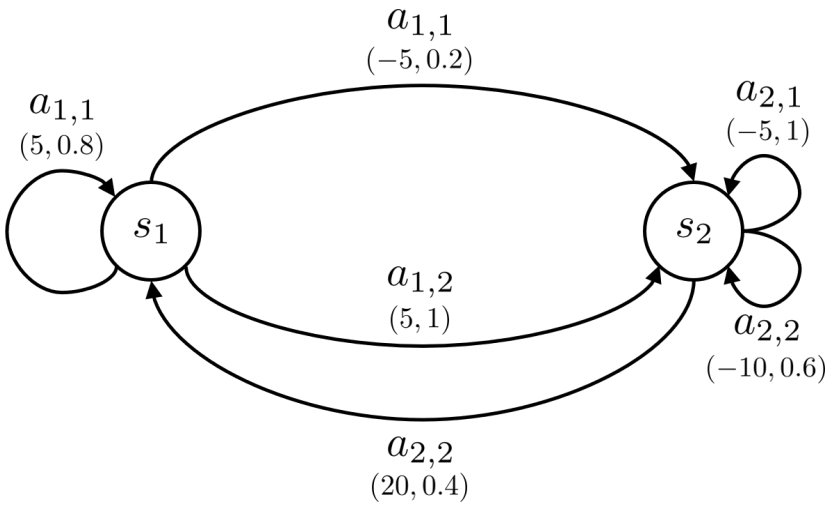


Figure 2.7: Graphical representation of two-state model. Circles denote states, arcs denote actions and transitions between states, and the expressions in parentheses denote rewards and transition probabilities, respectively. Zero probability transitions have been omitted.

Example 2.2. This example provides a concrete realization of the two-state model graphically represented in Figure 2.7. In this model, there are two states, and two actions to choose from in each state. Actions $a_{1,2}$ and $a_{2,1}$ result in deterministic outcomes – when the decision maker chooses these actions, transitions to a specified state occur with certainty. Consequently, rewards $r_n(s_1, a_{1,2}, s_1)$ and $r_n(s_2, a_{2,1}, s_1)$ are superfluous since they correspond to outcomes that cannot occur. Assume terminal rewards of 0. A precise formulation follows.

Decision epochs:

$$\{1, 2, \ldots, N\}, \quad N < \infty$$

States:

$$S = \{s_1, s_2\}$$

Actions:

$$A_{s_1} = \{a_{1,1}, a_{1,2}\}, \quad A_{s_2} = \{a_{2,1}, a_{2,2}\}$$

Rewards: For $n = 1, 2, \ldots, N$;

$$r_n(s_1, a_{1,1}, s_1) = 5, \quad r_n(s_1, a_{1,1}, s_2) = -5$$

$$r_n(s_1, a_{1,2}, s_1) = 0, \quad r_n(s_1, a_{1,2}, s_2) = 5$$

$$r_n(s_2, a_{2,1}, s_1) = 0, \quad r_n(s_2, a_{2,1}, s_2) = -5$$

$$r_n(s_2, a_{2,2}, s_1) = 20, \quad r_n(s_2, a_{2,2}, s_2) = -10$$

$$r_N(s_1) = 0, \quad r_N(s_2) = 0$$

Transition probabilities: For $n = 1, 2, \ldots, N$;

$$p_n(s_1|s_1, a_{1,1}) = 0.8, \quad p_n(s_2|s_1, a_{1,1}) = 0.2$$

$$p_n(s_1|s_1, a_{1,2}) = 0, \quad p_n(s_2|s_1, a_{1,2}) = 1$$

$$p_n(s_1|s_2, a_{2,1}) = 0, \quad p_n(s_2|s_2, a_{2,1}) = 1$$

$$p_n(s_1|s_2, a_{2,2}) = 0.4, \quad p_n(s_2|s_2, a_{2,2}) = 0.6$$

2.5.1 A one-period version

This section shows how to compute optimal value functions and policies directly, and with state-action value functions as intermediaries, in Example 2.2. Recall that in a one-period model, $N = 2$.

Optimal value functions

From (2.45), $v^*(s)$ satisfies

$$\begin{aligned} v^*(s_1) &= \max_{a \in \{a_{1,1}, a_{1,2}\}} \{p_1(s_1|s_1, a)(r_1(s_1, a, s_1) + r_2(s_1)) + p_1(s_2|s_1, a)(r_1(s_1, a, s_2) + r_2(s_2))\} \\ &= \max\{0.8 \times (5 + 0) + 0.2 \times (-5 + 0), 5 + 0\} = \max\{3, 5\} = 5. \end{aligned}$$

and

$$v^*(s_2) = \max_{a \in \{a_{1,1}, a_{1,2}\}} \{p_1(s_1|s_1, a)(r_1(s_1, a, s_1) + r_2(s_1)) + p_1(s_2|s_1, a)(r_1(s_1, a, s_2) + r_2(s_2))\}$$

$$= \max\{-5+0, 0.4\times(20+0)+0.6\times(-10+0)\} = \max\{-5,2\} = 2.$$

The first term in the "max" corresponds to action $a_{i,1}$, $i = 1, 2$ and the second term in the "max" corresponds to $a_{i,2}$, $i = 1, 2$. Thus, from (2.48), $\pi^* = (d_1^*)$ where $d_1^*(s_1) = a_{1,2}$ and $d_1^*(s_2) = a_{2,2}$ and from (2.38), $v^{\pi^*}(s_1) = 5$ and $v^{\pi^*}(s_2) = -2$.

Observe that this calculation was particularly simple because the terminal reward in each state equals 0. Exercise 5 asks you to explore the sensitivity of the optimal decision to the terminal reward.

State-action value functions

These calculations are now repeated using state-action value functions. From (2.51),

$$\begin{aligned}
q(s_1, a_{1,1}) &= p(s_1|s_1, a_{1,1})r(s_1, a_{1,1}, s_1) + p(s_2|s_1, a_{1,1})r(s_1, a_{1,1}, s_2) = 3\\
q(s_1, a_{1,2}) &= p(s_1|s_1, a_{1,2})r(s_1, a_{1,2}, s_1) + p(s_2|s_1, a_{1,2})r(s_1, a_{1,2}, s_2) = 5\\
q(s_2, a_{2,1}) &= p(s_1|s_2, a_{2,1})r(s_1, a_{2,1}, s_1) + p(s_2|s, a_{2,1})r(s_1, a_{2,1}, s_2) = -5\\
q(s_2, a_{2,2}) &= p(s_1|s_2, a_{2,2})r(s_2, a_{2,1}, s_1) + p(s_2|s_2, a_{2,2})r(s_2, a_{2,2}, s_2) = 2.
\end{aligned}$$

Hence from (2.54)

$$\begin{aligned}
d_1^*(s_1) &= \arg\max_{a\in\{a_{1,1},a_{1,2}\}} q(s_1, a) = a_{1,2}\\
d_1^*(s_2) &= \arg\max_{a\in\{a_{1,1},a_{1,2}\}} q(s_2, a) = a_{2,2}
\end{aligned}$$

and from (2.53)

$$\begin{aligned}
v^*(s_1) &= \max_{a\in\{a_{1,1},a_{1,2}\}} q(s_1, a) = 5\\
v^*(s_2) &= \max_{a\in\{a_{1,1},a_{1,2}\}} q(s_2, a) = 2
\end{aligned}$$

in agreement with direct calculation of optimal value functions.

As an aside, when coding algorithms in more complex problems, it was easier to organize calculations by first specifying $q(s, a)$.

2.5.2 Policies in a two-period version of the two-state model

In a one-period model, history-dependent and Markovian decision rules (and policies) are equivalent since the history at decision epoch 1 is the same as the state at decision epoch 1. Thus, to provide examples of the different types of policies, this section considers a two-period version. In a two-period example $N = 3$ representing two-decision epochs and a subsequent point in time when the terminal state is realized.

A Markovian deterministic policy in $\Pi^{\mathbf{MD}}$

Let $\pi_1 = (d_1, d_2)$, where

$$d_1(s) = \begin{cases} a_{1,1}, & s = s_1 \\ a_{2,1}, & s = s_2 \end{cases} \quad \text{and} \quad d_2(s) = \begin{cases} a_{1,2}, & s = s_1 \\ a_{2,1}, & s = s_2. \end{cases} \tag{2.57}$$

In state s_1, π_1 chooses action $a_{1,1}$ at the first decision epoch and $a_{1,2}$ at the second decision epoch. In state s_2, π chooses action $a_{2,1}$ at both decision epochs.

The expected total reward generated by policy π_1 for each starting state was computed by enumerating sample paths and assigning probabilities to each. These simple calculations are summarized in Table 2.1.

In state s_1 π_1 chooses $a_{1,1}$ and remains in state s_1 with probability 0.8 and jumps to state s_2 with probability 0.2. The corresponding rewards for these transitions are 5 and -5, respectively. At decision epoch 2, if the system is in state s_1, policy π_1 chooses $a_{1,2}$ resulting in a transition to state s_2 with certainty and a reward of 5. This sample path, which occurs with probability 0.8, has a total reward of $5 + 5 = 10$.

If the state at decision epoch 2 is s_2, this policy chooses action $a_{2,1}$, resulting in a self-transition and a reward of -5. This sample path occurs with probability 0.2 and has a total reward of $-5 - 5 = -10$. Since the terminal rewards are 0, the expected total reward of this Markovian deterministic policy is $0.8(10) + 0.2(-10) = 6$. Note that in these calculations the state at the end of period 2 was required to evaluate the reward in period 2.

Starting state	Sample path	Total reward	Probability	Expected total reward
s_1	$(s_1, a_{1,1}, s_1, a_{1,2}, s_2)$	10	0.8	6
	$(s_1, a_{1,1}, s_2, a_{2,1}, s_2)$	-10	0.2	
s_2	$(s_1, a_{2,1}, s_2, a_{2,1}, s_2)$	-10	1	-10

Table 2.1: Evaluation of expected total reward for Markovian deterministic policy π_1 by enumerating sample paths.

Clearly these calculations are tedious when N and the states and action sets were larger. Inductive methods in Chapter 4 provide a more efficient way to evaluate the expected rewards in a finite horizon model.

If instead, the initial state is chosen *randomly* with $\rho(s_1) = p$ and $\rho(s_2) = 1 - p$ for some $0 \leq p \leq 1$, the expected total reward corresponding to π_1 equals $16p - 10$.

A Markovian randomized policy in $\Pi^{\mathbf{MR}}$

Next is an example of a Markovian randomized policy $\pi_2 = (d_1, d_2)$. The randomized decision rule d_n, for $n = 1, 2$, choose action $a_{1,1}$ in state s_1, with probability q_1^n and action $a_{1,2}$ with probability $1 - q_1^n$ and in state s_2, choose action $a_{2,1}$ with probability

q_2^n and action $a_{2,2}$ with probability $1 - q_2^n$. Then, in the notation of Section 2.2.1, for $n = 1, 2$;

$$w_{d_n}^n(a|s) = \begin{cases} q_1^n, & a = a_{1,1},\ s = s_1 \\ 1 - q_1^n, & a = a_{1,2},\ s = s_1 \\ q_2^n, & a = a_{2,1},\ s = s_2 \\ 1 - q^2, & a = a_{2,2},\ s = s_2. \end{cases} \tag{2.58}$$

Computing the expected total reward of a Markovian randomized policy is slightly more complicated than in the Markovian deterministic case because it requires taking into account action choice probabilities specified by d_1 and d_2. Table 2.2 exhibits these calculations when the process starts in state s_1 and $q_1^1 = 0.1$, $q_1^2 = 0.5$ and $q_2^2 = 0.7$. Note that since the system starts in s_1 with certainty, q_2^1 is not used in the calculations below. In this table, a sample path represents a realization of the stochastic process $(X_1, Y_1, X_2, Y_2, X_3)$. As above, the realization of X_3 is required to compute the reward in period 2 which depends on the state at end of the horizon s^3.

Sample path	Total reward	Probability
$(s_1, a_{1,1}, s_1, a_{1,1}, s_1)$	$5 + 5 = 10$	$0.1 \times 0.8 \times 0.5 \times 0.8 = 0.0320$
$(s_1, a_{1,1}, s_1, a_{1,1}, s_2)$	$5 - 5 = 0$	$0.1 \times 0.8 \times 0.5 \times 0.2 = 0.0080$
$(s_1, a_{1,1}, s_1, a_{1,2}, s_2)$	$5 + 5 = 10$	$0.1 \times 0.8 \times 0.5 \times 1.0 = 0.0400$
$(s_1, a_{1,1}, s_2, a_{2,1}, s_2)$	$-5 - 5 = -10$	$0.1 \times 0.2 \times 0.7 \times 1.0 = 0.0140$
$(s_1, a_{1,1}, s_2, a_{2,2}, s_1)$	$-5 + 20 = 15$	$0.1 \times 0.2 \times 0.3 \times 0.4 = 0.0024$
$(s_1, a_{1,1}, s_2, a_{2,2}, s_2)$	$-5 - 10 = -15$	$0.1 \times 0.2 \times 0.3 \times 0.6 = 0.0036$
$(s_1, a_{1,2}, s_2, a_{2,1}, s_2)$	$5 - 5 = 0$	$0.9 \times 1.0 \times 0.7 \times 1.0 = 0.6300$
$(s_1, a_{1,2}, s_2, a_{2,2}, s_1)$	$5 + 20 = 25$	$0.9 \times 1.0 \times 0.3 \times 0.4 = 0.1080$
$(s_1, a_{1,2}, s_2, a_{2,2}, s_2)$	$5 - 10 = -5$	$0.9 \times 1.0 \times 0.3 \times 0.6 = 0.1620$

Table 2.2: Sample paths, rewards and probabilities corresponding to Markovian randomized policy π_2 starting in state s_1. Zero-probability sample paths are not shown.

The following expression mathematically describes the quantities used to obtain the probability in the first row of Table 2.2.

$$\begin{aligned} P^{\pi_2}\left[(X_1, Y_1, X_2, Y_2, X_3) = (s_1, a_{1,1}, s_1, a_{1,1}, s_1)\right] \\ = w_{d_1}^1(a_{1,1}|s_1)p(s_1|a_{1,1}, s_1)w_{d_2}^2(a_{1,1}|s_1)p(s_1|a_{1,1}, s_1) \end{aligned}$$

Combining sample paths which generate the same reward provides the following probability distribution of total rewards generated by π_2 and enables computation of its mean and standard deviation.

From Table 2.3 it follows that the expected total reward corresponding to starting π_2 in s_1 is 2.45 and its standard deviation is 9.35.

Reward	Probability
25	0.1080
15	0.0024
10	0.0720
0	0.6380
−5	0.1620
−10	0.0140
−15	0.0036

Table 2.3: Derived probability distribution of total reward over decision epochs 1 and 2 for Markovian randomized policy π_2 starting in state s_1.

A history-dependent deterministic policy in $\Pi^{\mathbf{HD}}$

Let $\pi_3 = (d_1, d_2)$, where

$$d_1(h) = \begin{cases} a_{1,1}, & h = s_1 \\ a_{2,1}, & h = s_2 \end{cases} \quad \text{and} \quad d_2(h) = \begin{cases} a_{1,2}, & h = (s_1, a_{1,1}, s_1) \\ a_{2,1}, & h = (s_1, a_{1,1}, s_2) \\ a_{2,2}, & h = (s_2, a_{2,1}, s_2) \end{cases} \tag{2.59}$$

For this policy, the chosen action (at decision epoch 2) depends on the entire history. At decision epoch 1, the history is simply the initial state. But at decision epoch epoch 2, the history includes the initial state, the first action chosen, and the subsequent state. If the state is s_1 at decision epoch 2, there is only one way for this to happen, given d_1 (initial state s_1 with action $a_{1,1}$). However, there are two histories that lead to s_2 at epoch 2. Thus, the action chosen in state s_2 at epoch 2 depends on whether the process started in s_1 or s_2: starting in s_1 leads to choosing action $a_{2,1}$ in the second decision epoch, otherwise $a_{2,2}$. Notice that if the policy selects action $a_{2,1}$ for the history $(s_2, a_{2,1}, s_2)$, then this policy coincides with the Markovian deterministic policy described previously.

It is left as an exercise to derive the probabilities of each realization and expected total reward.

A history-dependent randomized policy in $\Pi^{\mathbf{HR}}$

Define the policy $\pi_4 = (d_1, d_2)$ as follows. At the first decision epoch, the randomized decision rule is described by the following probability distribution:

$$w^1_{d_1}(a|h) = \begin{cases} q^1_1, & a = a_{1,1},\ h = s_1 \\ 1 - q^1_1, & a = a_{1,2},\ h = s_1 \\ q^1_2, & a = a_{2,1},\ h = s_2 \\ 1 - q^1_2, & a = a_{2,2},\ h = s_2. \end{cases} \tag{2.60}$$

In state s_1, action $a_{1,1}$ is chosen with probability q_1^1 and action $a_{1,2}$ is chosen with probability $1 - q_1^1$. In state s_2, action $a_{2,1}$ is chosen with probability q_2^1 and action a_2^1 is chosen with probability $1 - q_{1,2}$.

At the second decision epoch, there are eight histories to consider, since there are two states and two actions in each state. However, $a_{1,2}$ and $a_{2,1}$ result in deterministic transitions to state s_2, so two of the eight histories are impossible. The remaining six are listed below:

$$\begin{aligned} h_1 &:= (s_1, a_{1,1}, s_1) \\ h_2 &:= (s_1, a_{1,1}, s_2) \\ h_3 &:= (s_1, a_{1,2}, s_2) \\ h_4 &:= (s_2, a_{2,1}, s_2) \\ h_5 &:= (s_2, a_{2,2}, s_1) \\ h_6 &:= (s_2, a_{2,2}, s_2). \end{aligned}$$

Given these possible histories at the second decision epoch, a possible specification for a randomized decision rule is

$$w_{d_2}^2(a|h) = \begin{cases} q_1^2, & a = a_{1,1},\ h = h_i \\ 1 - q_i^2, & a = a_{1,2},\ h = h_i \end{cases} \tag{2.61}$$

for $i \in \{1, 5\}$ and

$$w_{d_2}^2(a|h) = \begin{cases} q_i^2, & a = a_{2,1},\ h = h_i \\ 1 - q_i^2, & a = a_{2,2},\ h = h_i \end{cases} \tag{2.62}$$

for $i \in \{2, 3, 4, 6\}$.

Again it is left as an exercise to derive the probabilities of each realization and expected total reward.

A stationary deterministic policy $\pi \in \Pi^{\mathbf{SD}}$

Suppose $\pi_5 = (d, d)$, where

$$d(s) = \begin{cases} a_{1,1}, & s = s_1 \\ a_{2,1}, & s = s_2 \end{cases} \tag{2.63}$$

This policy uses the same decision rule in both decision epochs. It is similar to the Markovian deterministic policy above, except that if the system is in state s_1 at epoch 2, the SD policy chooses action $a_{1,1}$, whereas the MD policy chooses $a_{1,2}$.

A stationary randomized policy $\pi \in \Pi^{\mathbf{SR}}$

The Markovian randomized policy above can be transformed into a stationary randomized policy $\pi = (d, d)$ by simply omitting its dependence on n, for example:

$$w_d(a|s) = \begin{cases} q_1, & a = a_{1,1},\ s = s_1 \\ 1 - q_1, & a = a_{1,2},\ s = s_1 \\ q_2, & a = a_{2,1},\ s = s_2 \\ 1 - q_2, & a = a_{2,2},\ s = s_2. \end{cases} \tag{2.64}$$

This policy has a stationary probability distribution of choosing action $a_{1,1}$ versus $a_{1,2}$ when the system is in state s_1, and similarly when the state is s_2.

Bibliographic remarks

See the historical summary in Section 1.1.1.

Exercises

1. Consider the following three-state model with $S = \{s_1, s_2, s_3\}$, actions $A_{s_i} = \{a_{i,1}, a_{i,2}\}$ for $i = 1, 2, 3$, rewards $r_n(s_i, a_{i,1}, s_j) = i - j, r_n(s_i, a_{i,2}, s_j) = j - i$ for $n = 1, 2, \ldots, N-1$ and $r_N(s_i) = -i^3$. Transition probabilities are given by $p_n(s_i|s_i, a_{i,1}) = 1 - 1/i$, $p_n(s_j|s_i, a_{i,1}) = 1/(2i)$ for $j \neq i$ and $p_n(s_i|s_i, a_{i,2}) = 1 - 1/(i+1)$, $p_n(s_j|s_i, a_{i,2}) = 1/(2(i+1))$ for $j \neq i$ for $n = 1, 2, \ldots, N$.

   (a) Provide a graphical representation of the model as in Figure 2.7.

   (b) Represent a one- and two-period version of the model as a decision tree when $\rho(s_i) = P[X_1 = s_i] = 1/3$ for $i = 1, 2, 3$. Compute the expected total reward by rolling back calculations on each path through the tree.

   (c) In a one-period version, find the distribution of X_2 for each possible initial state s_1, s_2, s_3 when the deterministic decision rule $d(s_i) = a_{i,1}$ is used at decision epoch 1.

   (d) Use (2.38) to find the expected total reward $v^{\pi}(s)$ for each $s \in S$ in a one-period version for the policy π that uses the above decision rule d at decision epoch 1.

   (e) In a one-period version, find the distribution of X_2 for each possible initial state s_1, s_2, s_3 when the randomized decision rule d' with distribution $P[d'(s_i) = a_{i,1}] = 1 - P[d'(s_i) = a_{i,2}] = e^{-0.5i}$ for $i = 1, 2, 3$ is used at decision epoch 1.

   (f) Use (2.39) to find the expected total reward $v^{\pi'}(s)$ for each $s \in S$ in a one-period version for the policy π' that uses the above decision rule d' at decision epoch 1.

   (g) Find the optimal policy in a one-period version by computing optimal value functions and state-action value functions.

2. Using 5,000 replications of Algorithm 2.1, simulate the total reward for the policies π and π' from Exercise 1 when $N = 2$.

   (a) Estimate the expected total reward of each policy and compare your results to those in Exercise 1.

   (b) Provide histograms of your estimates and comment on their shape and any differences you observe between the histograms of π and π'.

   (c) Compute the standard deviation and 95th percentile of the total returns for each policy. Interpret these quantities verbally and note why one might be interested in such quantities.

   (d) Suppose one measures the value of a reward stream (R_1, R_2) by multiplicative utility $e^{\gamma R_1} e^{\gamma R_2}$. Compare policies on the basis of their expected utility. Repeat parts (a) to (c) above using this utility function.

3. Show that when $\pi \in \Pi^{\text{MR}}$, the sequence of state and action pairs is a discrete time Markov Chain. Devise an example that shows that when $\pi \in \Pi^{\text{HR}}$, the sequence may not be Markov Chain.

4. Construct an example where an epoch-dependent state space S_n is appropriate (i.e., where using $S = \cup_n S_n$ would unnecessarily enlarge the state space at each decision epoch).

5. Write out (2.55) and (2.56) for the one-period version of the two-state problem in which $r_2(s_1) = c_1$ and $r_2(s_2) = c_2$. Plot the optimal policy as a function of the terminal rewards c_1 and c_2.

6. Evaluate the probability distributions of stochastic process generated by the deterministic history-dependent policy π_3 and the randomized history-dependent policy π_4 for the two-period model in Section 2.5.2. To organize calculations create analogs of Tables 2.2 and 2.3.

7. *Consider the following deterministic model. Let $S = \{s_1, s_2\}$, $A_{s_1} = \{a_{1,1}, a_{1,2}\}$, $A_{s_2} = \{a_{2,1}, a_{2,2}\}$, $r(s_1, a_{1,1}) = r(s_1, a_{1,2}) = 2$, $r(s_2, a_{2,1}) = r(s_2, a_{2,2}) = -2$, and $p(s_1|s_1, a_{1,1}) = p(s_2|s_1, a_{1,2}) = p(s_1|s_2, a_{2,1}) = p(s_2|s_2, a_{2,2}) = 1$.

   (a) Provide a graphical representation of the model as in Figure 2.7.

   (b) Show that for each stationary policy π and each state $s \in S$ that the following limit exists

$$\lim_{N\to\infty} \frac{1}{N} E^{\pi} \left[\sum_{n=1}^{N} r(X_n, Y_n) \;\middle|\; X_1 = s \right]. \tag{2.65}$$

   (c) Consider a history-dependent policy π that when the initial state is s_1 chooses action $a_{1,1}$ for one period, then chooses $a_{1,2}$ so that the system

proceeds to s_2 and then chooses action $a_{2,2}$ so the system remains in s_2 for three periods, at which point it chooses action $a_{2,1}$ so that the system returns to s_1 and then chooses action $a_{1,1}$ so that it stays in state s_1 for $3^2 = 9$ periods and then chooses actions so that it proceeds to s_2 and remains there for $3^3 = 27$ periods and so forth.

Show that for π the limit in (2.65) does not exist by showing that

$$\liminf_{N\to\infty} \frac{1}{N} E^{\pi}\left[\sum_{n=1}^{N} r(X_n, Y_n) \;\middle|\; X_1 = s\right] \neq \limsup_{N\to\infty} \frac{1}{N} E^{\pi}\left[\sum_{n=1}^{N} r(X_n, Y_n) \;\middle|\; X_1 = s\right].$$

Chapter 3

Examples and Applications

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

For the things we have to learn before we can do them, we learn by doing them[1].

Aristotle, Greek philosopher, 384-322 BCE.

Representing a sequential decision problem as a Markov decision process requires specifying five objects: decision epochs, states, actions, transition probabilities and rewards. This chapter shows how to do so by identifying these objects in applications spanning a broad range of disciplines. The objective of this chapter is to highlight both the wide applicability of the MDP framework and the diverse aspects of model formulation. Note that several of these examples will be revisited in subsequent chapters.

We encourage readers to identify decision epochs, states, actions, transition probabilities and rewards on their own before looking at our approach. The chapter concludes with guidance on how to formulate Markov decision process models in general. The problems at the end of the chapter provide additional opportunities to formulate Markov decision processes or revise the provided examples when assumptions differ.

3.1 Inventory management

Inventory models represent some of the earliest and most widely studied applications of Markov decision processes. They concern determining appropriate inventory levels for

[1] Aristotle [1925].

a product in the face of uncertain customer demand. The following passage from a New York Times article[2] nicely summarizes the core challenge of inventory management.

> *"When you have an inventory-based business, most people think only about the first order," Mr. Green said. With long lead times from the factory in China, he was almost immediately trying to figure out how big his second and third orders should be. Underestimating would hurt not just sales but the overall status of his Amazon listing; overestimating would drain him of cash upfront, and he would incur further charges from Amazon for storing excess inventory in its warehouses.*

As the quote indicates, having too little inventory results in losing sales and reputation, while having too much inventory leads to excess storage and capital charges. These costs are key components of inventory models; balancing this trade-off is the primary challenge.

The models considered here assume that an *inventory manager* periodically (hourly, daily, weekly or monthly) observes the inventory level of a product and, if deemed opportunistic, places a replenishment order with a *supplier*. This replenishment may arrive immediately, at the end of the current period (prior to the next review period) or several periods in the future. The delay between placing and receiving an order is referred to as a *lead time*.

Inventory management applications have many features that impact an Markov decision process formulation.

1. **Ordering costs** consist of a fixed component and a variable component. The fixed component K represents the administrative cost of placing an order[3] and the variable component $c(u)$ represents the cost of ordering u units. Thus when an order of u units is placed, the cost is $K + c(u)$ and when no order is placed, the cost is 0.

2. **Holding costs**, denoted by $h(u)$, represent the cost to the inventory manager of storing u units of product for one period. This cost only applies when $u > 0$. It is convenient to assume that $h(0) = 0$.

3. **Lost sales** occur if there is insufficient inventory to fulfill demand in the current period.

4. **Backlogged demand** means that unmet demand may be fulfilled at some period in the future when inventory is available. *A different formulation applies depending on whether unmet demand is lost or backlogged.*

[2] Herrman [2021].

[3] In some applications, $K = 0$, but usually $K > 0$. It is the presence of this cost that makes it advantageous for the inventory manager to not order every period.

5. **Penalty costs**, denoted by $p(u)$, represent the cost to the inventory manager of backlogging u units of demand for one period. This cost applies only when $u < 0$. It is convenient to assume $p(0) = 0$.

6. **Total customer demand** in each period is random. It may arrive in one batch at a specific point in time within a period or may arrive at random times within a period. The timing of demand will impact the formulation. The demand distribution may be known or unknown, and may be static or time-varying. Let the non-negative[4] random variable Z_n denote the total demand in period n. Its distribution will be represented by $P[Z_n = z]$.

7. **Revenue** corresponds to the amount spent by customers purchasing units of a product. It is represented by an increasing function $R(\cdot)$ and its relationship to demand depends on some of the above features. If unfulfilled demand is backlogged, it is simplest to assume that payments are made "up front", that is when an order is placed[5] so that if the demand is z, the revenue is $R(z)$. In the lost sales case, sales are capped by inventory on hand so that the revenue is $R(z)$ if the demand is less than inventory level s and $R(s)$ if the demand exceeds the inventory s. In this latter case, low inventory levels may result in lost sales and reduced revenue.

8. **Product shelf-life** may be finite or "infinite[6]". Products with finite shelf lives are said to be *perishable*. Perishable items may last for one-period (newspapers, fresh bread, defrosted vaccine or a seat at a sporting event) or multiple-periods (blood products, consumer electronics or fashion goods). Non-perishable goods include tools, books and basic clothing. Our discrete-state formulation implicitly assumes product is only available in whole units.

9. **Scrap value**, denoted by $H(s)$, equals the value of the ending inventory when there are s units on hand at the end of a finite planning horizon. If $s \geq 0$, $H(s)$ may include any potential holding cost before the inventory is liquidated. If $s < 0$, then $H(s)$ represents a penalty associated with not being able to fulfill the $|s|$ items backlogged (e.g., the loss associated with buying these items at a premium from a back-up supplier, or a loss in goodwill due to lost sales).

3.1.1 Inventory management with backlogged demand

Formulating sequential inventory management as a Markov decision process requires precisely specifying the timing of events and assumptions. Figure 3.1 depicts the

[4] Most inventory models assume non-negative demand.

[5] Alternatively, payments may be made when the order is fulfilled.

[6] No product lasts forever. Assuming infinite shelf life is a modeling convention to facilitate representing future inventory levels as functions of the present inventory, the quantity ordered and the demand only.

event sequence for the following model. At the start of period N, the decision maker observes the inventory level s and decides on how many units a of product to order from the supplier to replenish inventory. Demand arrives randomly throughout the period and is fulfilled at the end of the period, using inventory on hand together with units that arrive from the current period's order. If demand exceeds inventory, it is backlogged for fulfillment from future inventory replenishments. An Markov decision process formulation follows.

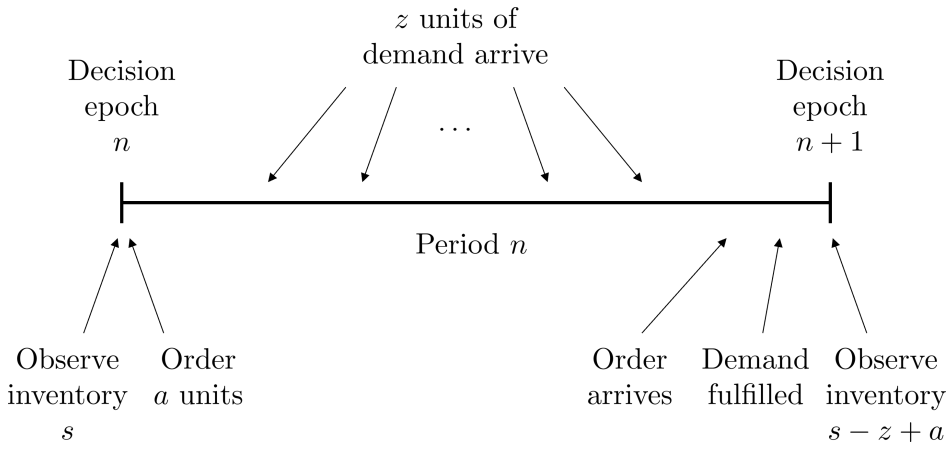


Figure 3.1: Timing of events in the periodic review inventory model.

Decision epochs: Decision epochs correspond to the times at which the decision maker reviews the inventory. This problem can be modeled either with a finite or infinite planning horizon. Hence

$$T = \{1, 2, \ldots, N\}, \quad N \leq \infty.$$

States: States represent the number of units on hand at a decision epoch. A negative value indicates backlogged demand.

$$S = \{\ldots, -2, -1, 0, 1, 2, \ldots\}.$$

Note that the quantities backlogged and in inventory may be truncated at large values (e.g., the capacity of a warehouse) to ensure a finite state space. Doing so makes the formulation slightly more complex because of the resulting boundary conditions.

Actions: Actions represent the quantity ordered from the supplier for delivery prior to the next decision epoch. For each $s \in S$,

$$A_s = \{0, 1, 2, \ldots\}.$$

Similar to the state space, the action space can be truncated so that it is finite.

Rewards: The reward in a period equals revenue minus cost. As noted above, revenue depends on whether unmet demand is backlogged or lost. Since this formulation assumes backlogging, when the total demand in a period is z, the revenue equals $R(z)$. Note that the inventory level at the next decision epoch, j is given by $j = s + a - z$ so that knowing only the inventory levels s and j and the order quantity a one can infer that the demand is given $z = s + a - j$.

There are three cost components: ordering costs, which are incurred only if $a > 0$, holding costs if s is positive and penalty costs if s is negative. A simplifying assumption is that holding and penalty costs are assessed at the beginning of the period based on the *starting* inventory.

The reward can be written[7]

$$r_n(s, a, j) = R(s + a - j) - KI_{\mathbb{Z}_{>0}}(a) - c(a) - h(s)I_{\mathbb{Z}_{>0}}(s) - p(-s)I_{\mathbb{Z}_{>0}}(s)$$

when $n < N$. The argument of $p(\cdot)$ is $-s$, which equals the backlogged demand when s is negative.

If N is finite, $r_N(s) = H(s)$, corresponding to the scrap value of the remaining inventory at the end of the planning horizon.

Transition probabilities: As noted above, when the state at decision epoch $n + 1$ is j, this means that demand in period n was $s + a - j$. Moreover, the assumption of non-negative demand ensures that the state at the next decision epoch cannot exceed $s + a$. Hence, the transition probabilities are

$$p_n(j|s, a) = \begin{cases} P[Z_n = s + a - j], & j \leq s + a,\ j \text{ integer} \\ 0, & j = s + a + 1, s + a + 2, \ldots \end{cases}$$

Application challenges

Applying this model presents several challenges. In particular, one must determine demand distributions, ordering costs, holding costs and penalty costs. Demand distributions may be estimated from historical data; parameterizing the model in terms of a known distribution reduces the challenge in estimating model parameters. Moreover it is likely that demand has seasonal components at the day, week and month levels.

Per unit ordering costs should be easily obtainable but fixed ordering costs may be more challenging to determine. The fixed component encompasses administrative, shipping and handling costs that may be hard to untangle from the per unit cost. Holding costs are real and involve cost of capital and space charges.

Penalty costs are the most challenging to determine since they involve intangibles such as loss of goodwill due to unfulfilled or delayed orders. Instead, the decision maker

[7] In the definition of the reward $\mathbb{Z}_{>0}$ denotes the positive integers so that $I_{\mathbb{Z}_{>0}}(x)$ equals 1 when x is positive and 0 otherwise.

may specify a service level such as 95% of orders be processed from stock on hand and use a constrained Markov decision process model formulation.

Note that major disruptions to supply chain or consumer behavior arising from, for example, a global pandemic, can lead to significant challenges in estimating appropriate parameters. Procurement costs might be much higher due to increased demand for raw materials and lower manufacturing capacity. Demand for certain products could be significantly increased or decreased compared to historical levels.

3.1.2 The newsvendor problem

This section describes a simple yet widely studied and applied inventory model referred to as the *newsvendor problem*[8]. This model is fundamental in the operations research literature and applies to perishable goods with a shelf-life of one period.

As shown in Figure 3.2, events unfold as follows. At the start of the period, the newsvendor purchases a units of product (newspapers) from a supplier at a cost of c per unit. After receiving the units, a random demand of z units arrives throughout the period. Items are sold at a price of g with $g > c$. Each unit sold results in a profit (selling price minus cost) $G := g - c$. At the end of the period each unsold item is disposed of at a scrap value of h where $c > h$ resulting in a loss (cost minus salvage value) $L := c - h$ and an ending of inventory zero.

The newsvendor seeks an order quantity a^* that maximizes the one-period expected reward. The choice of order quantity *trades off* ordering too many items and incurring a loss on unsold items with ordering too few items and foregoing potential additional profit. Another concern is that if a is set too low, the newsvendor will observe only *censored* demand complicating estimation of the demand distribution when it is unknown. Thus, initially the newsvendor may set a higher, trading off potential losses for gains in information about the demand distribution.

Formulation as a one-period Markov decision process follows.

Decision epochs: There is only one decision epoch so that $T = \{1\}$, consequently $N = 2$.

States: The formulation is a bit non-standard as the set of possible states differs at decision epoch 1 and at the terminal epoch. At the first decision epoch $S_1 = \{0\}$ corresponding to the initial inventory of 0. At the terminal epoch, $S_2 = A_0 = \{0, 1, \ldots, a_{\max}\}$ where $a_{\max}$ is defined below. This quantity is required to evaluate the final inventory and accounts for the implicit assumption that demand cannot be negative.

[8]This was first called a "newsboy" problem. It modeled the problem of a newsboy who purchased papers from a dealer, sold them at a street corner and returned the unsold papers to the dealer at the end of the day.

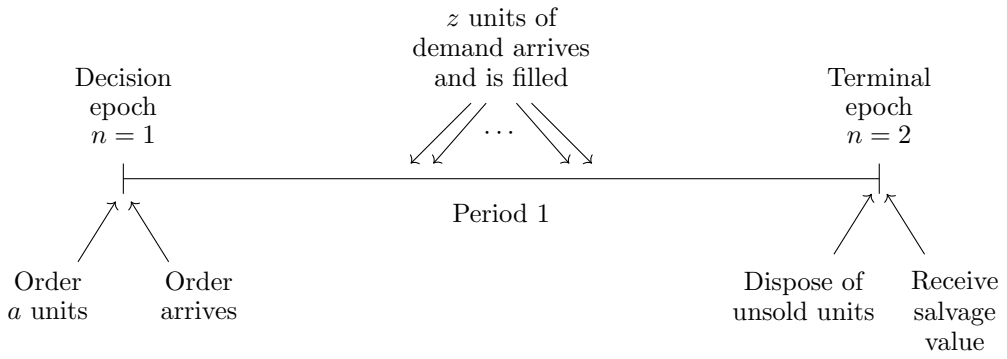


Figure 3.2: Timing of events in the newsvendor problem.

Actions: Actions represent the quantity ordered from the supplier for immediate delivery. This means that

$$A_0 = \{0, 1, 2, \ldots, a_{\max}\},$$

where the set of actions is bounded by some "reasonable" value $a_{\max}$.

Rewards: The reward in the first period includes the cost of purchasing the product plus the revenue from sales prior to the end of the period. Thus

$$r_1(0, a, j) = \begin{cases} -ca + g(a - j) & \text{for } j \leq a \\ 0 & \text{for } j > a. \end{cases}$$

Note it is impossible for ending inventory to exceed a.

The terminal reward results from disposing of the unsold product. That is

$$r_2(s) = hs.$$

Note that on account of the transition probabilities, $s \leq a$.

Transition probabilities: The transition probabilities at decision epoch 1 account are computed differently depending on whether some or all inventory is sold. The case $j = 0$ occurs when the demand equals or exceeds the order quantity a. Moreover the inventory at the end of the period cannot exceed the order quantity.

$$p_1(j|s, a) = \begin{cases} P[Z = a - j] & \text{for } j = 1, \ldots, a \\ \sum_{z=a}^{\infty} P[Z = z] & \text{for } j = 0 \\ 0 & \text{for } j > a. \end{cases}$$

Optimizing the order quantity

Suppose the newsvendor orders a units and the demand is z units. Then the total revenue combining $r_1(0, a, j)$ and $r_2(j)$ is given by

$$\begin{aligned} r(a,z) :=& \begin{cases} (g-c)z + (h-c)(a-z) & \text{if } z < a \\ (g-c)a & \text{if } z \geq a \end{cases} \\ =& \begin{cases} Gz - L(a-z) & \text{if } z < a \\ Ga & \text{if } z \geq a \end{cases} \\ =& G\min(z,a) - L(a-z)^+, \end{aligned} \tag{3.1}$$

where G and L are defined above. The representation (3.1) expresses the revenue in a single equation.

Consequently the expected revenue, denoted[9] by $q(a)$, is given by

$$\begin{aligned} q(a) :=& E[r(a,Z)] = \sum_{z=0}^{\infty} r(a,z) f(z) \\ =& \sum_{z=0}^{a-1} \big(Gz - L(a-Z)\big) P[Z=z] + Ga \sum_{z=a}^{\infty} P[Z=z]. \end{aligned} \tag{3.2}$$

The first term corresponds to the case when the demand z is less than the order quantity a. In this case, $a - z$ units of product remain unsold resulting in a loss of L units per item. The second term corresponds to the situation when demand is greater than or equal to the inventory. In this case, $z - a$ units of potential demand are lost and the reward is Ga.

Thus, the newsvendor problem can be summarized as that of finding

$$a^* \in \arg\max_{a \in A_0} q(a) = \arg\max_{a \in A_0} E[r(a,Z)],$$

where A_0 denotes the set of possible order quantities. The purpose of writing it this way is to prepare for its analysis using reinforcement learning methods in Chapters 10 and 11.

One particularly attractive feature of this model is that if the demand distribution is known, the optimal order quantity can be shown to satisfy

$$a^* = F^{-1}\left(\frac{G}{G+L}\right), \tag{3.3}$$

where $F(z) = \sum_{u=0}^{z} P[Z=u]$. Since the product is assumed to be available only in discrete units, a^* may need to be rounded up or down. Note that if demand is continuous this expression is exact.

[9] The quantity $q(a) := q(0,a)$ represents the state-action value function in the unique starting state 0.

The ratio $(G/G+L)$ is sometimes refereed to as the *critical fractile*. It represents the fraction of demand that will be met by choosing a^* in this way. For example if $G = 2L$, two-thirds of the demand will be satisfied on average[10].

Application challenges

The newsvendor model has been widely applied in the retail fashion-goods industry but also applies to other perishable goods. One novel application (unpublished) was its use by one us to the determine the number of instructors to have available for drop-in ski lessons each day. In applications, the greatest challenge is to model the demand distribution, which may vary by day-of-week or month-of-year and change over time.

3.2 Gridworld navigation

A mathematician is a machine which turns coffee into theorems[11].

Alfréd Rényi, Hungarian mathematician, 1921-1970.

As noted by Rényi, a working mathematician frequently requires coffee. To avoid time away from theorem proving, the mathematician may engage a robot to bring coffee when needed. In the example described here, the robot's task is to carry the mathematician's empty cup from the office to the coffee room, fill the cup with coffee and bring it back to the mathematician's office, all while avoiding barriers and pitfalls such as an open stairwell. When the mathematician is working on a particularly difficult theorem, a whole pot of coffee may be required, as shown by the robot on the book's cover and in Figure 3.3.

In general, a *Gridworld* represents a simplified environment commonly used to illustrate reinforcement learning methods. A Gridworld possesses a grid-like structure (like a chess board) where each cell in the grid represents a discrete state that an agent can occupy. The agent learns to move through the grid, typically aiming to achieve a goal while navigating around obstacles, collecting rewards, and avoiding penalties.

The model formulation in this section assumes that the robot *knows* the arrangement of the grid, which grid cell it occupies and the location of boundaries, target cells and penalties. This means the robot will never attempt to move outside the grid boundary. Alternative formulations assume:

1. the robot does not know its location,
2. the robot does not know the configuration of the grid, its boundaries, or the location of the stairwell,
3. the robot does not know the uncertainty involved when choosing which direction to move, or

[10] Assuming the newsvendor faces this situation on many occasions.

[11] Wieschenberg [1999].

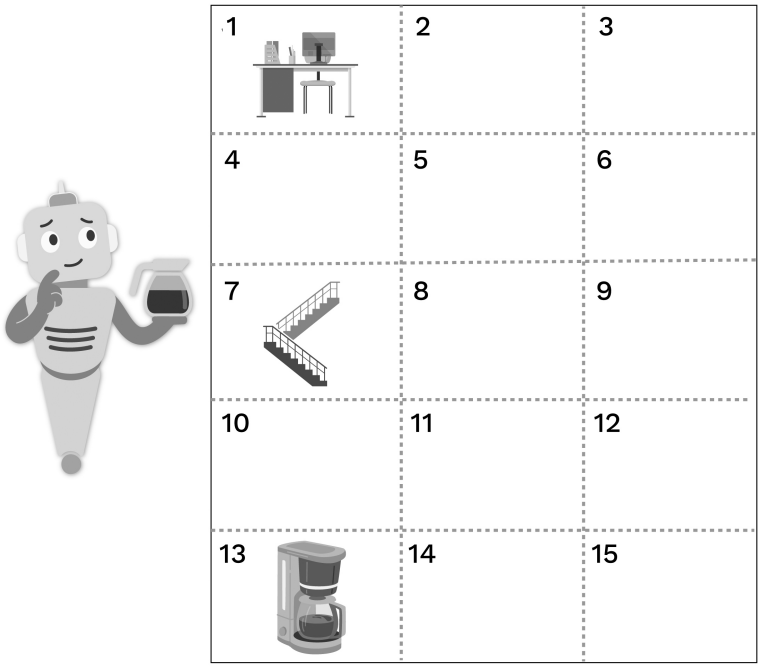


Figure 3.3: Schematic layout for Gridworld navigation example.

4. some combination of the above.

In these alternative situations, navigating through the grid becomes a *learning* problem. Such problems will be studied in depth in Chapters 10 and 11.

The problem description below assumes that the robot knows the grid configuration and which cell it occupies. In each cell except the stairwell, regardless of whether the coffee cup is empty or full, the robot can move in any of the four directions that does not take it outside the grid boundary. If the robot falls into the stairwell, or returns to the office with a full coffee cup, the episode terminates[12].

Assume movement on the grid is subject to uncertainty as follows. Let p_E and p_F denote the probability that the robot moves in its intended direction when the coffee cup is empty and full, respectively. If in some cell there are k possible locations where the robot can end up (including the current location), the probability the robot moves in each unintended direction or remains where it was is $(1 - p_E)/(k - 1)$ or $(1 - p_F)/(k - 1)$, depending on the status of the coffee cup. For example, suppose the robot has a full coffee cup and is in cell 6. Then if it intends to move down, it does so

[12]In some numerical examples in subsequent chapters, the episodes terminate only when the robot successfully delivers coffee.

with probability p_F and it moves left, moves up or remains in the same location with probability $(1 - p_F)/3$. The robot is more likely to be error prone with a full coffee cup because of the energy required not to spill the coffee. Thus, assume $p_E > p_F$.

The goal is to return with a full coffee cup as quickly as possible. If the robot successfully delivers coffee to the mathematician it receives a reward of B. If it falls down the stairs it incurs a penalty of X because the mathematician has to interrupt work to rescue the robot. Assume $X \gg B \gg 1$.

Decision epochs: Assume the process evolves in discrete time, where decision epochs correspond to the instant at which the robot decides in which direction to move. The first decision epoch after the robot enters the coffee room is used to fill up the cup, and the next one corresponds to the instant immediately after the cup has been filled and it decides where to move next. Thus

$$T = \{1, 2, \ldots\}.$$

The set of decision epochs is unbounded because the robot continues attempting to deliver the coffee to the mathematician until the random time when it is either successful or falls down the stairs.

States: States represent the location of the robot in the numbered grid, plus an additional variable to indicate whether or not the coffee cup is empty (E) or full (F). The status of the coffee cup is required because it informs the success probability of an intended action. It also influences the direction in which the robot should proceed: a robot with an empty coffee cup seeks the coffee room, while a robot with a full coffee cup seeks the office. Therefore,

$$S = \{1, 2, \ldots, 15\} \times \{E, F\}.$$

Actions: Actions represent the direction the robot attempts to travel. Assume the robot can only move up (north), down (south), right (east) and left (west), denoted by U, D, R and L, respectively. Under the assumption that the robot knows the layout of the grid and its location, the grid boundary constrains its intended movement so the set of permissable actions must take this into account. For example,

$$A_{(5,\cdot)} = \{U, D, R, L\},\ A_{(7,\cdot)} = \{U, D, R\},\ \text{and}\ A_{(15,\cdot)} = \{U, L\}.$$

Action sets for some particular states follow:

$$A_{(1,F)} = A_{(7,F)} = A_{(7,E)} = \{a_0\},\ A_{13,E} = \{a_1\}.$$

The states $(1, F), (7, F)$ and $(7, E)$ are *absorbing* states. Once entered, the robot remains there forever; action a_0 corresponds to remaining in that state. Once such a state is entered, decision making stops. When the robot enters the coffee room with an empty cup, it takes one period to fill it. Denote the action of filling the cup by a_1.

Rewards: Each intended movement action and the act of filling the coffee cup costs c. Often $c = 1$. Transitions into the office with a full coffee cup result in a reward of $B - c$. For example,

$$r((2, F), a, (1, F)) = B - c, \quad a \in A_{(2,F)} \tag{3.4}$$

Transitions into the stairwell receive a reward of $-X - c$ regardless of the status of the cup. For example,

$$r((4, k), a, (7, k)) = -X - c, \quad a \in A_{(4,k)} \text{ and } k \in \{E, F\}. \tag{3.5}$$

All other feasible state transitions between neighboring cells result in a reward of $-c$. For example,

$$r((5, k), a, (6, k)) = -c, \quad a \in A_{(5,k)} \text{ and } k \in \{E, F\}. \tag{3.6}$$

Finally, in this formulation, no rewards are received (or costs incurred) once the robot completes its task or falls down the stairs:

$$r((1, F), a_0, (1, F)) = 0 \text{ and } r((7, k), a_0, (7, k)) = 0, \ k \in \{E, F\}. \tag{3.7}$$

Note that in the absence of uncertainty, it is easy to see that the robot can complete its task in 13 steps, so the maximum possible reward is $B - 13c$.

Transition probabilities: Some typical probabilities follow:

$$p((k, F)|(5, F), U) = \begin{cases} p_F & k = 2 \\ (1 - p_F)/4 & k \in \{4, 5, 6, 8\} \\ 0 & k \notin \{2, 4, 5, 6, 8\} \end{cases}$$

$$p((k, E)|(6, E), D) = \begin{cases} p_E & k = 9 \\ (1 - p_E)/3 & k \in \{5, 6, 9\} \\ 0 & k \notin \{3, 5, 6, 9\} \end{cases}$$

Transitions are deterministic when the system is in one of the absorbing states or when the robot enters the coffee room with an empty cup (since the only action is to fill the cup):

$$\begin{aligned} p((13, F)|(13, E), a_1) &= p((1, F)|(1, F), a_0) \\ &= p((7, F)|(7, F), a_0) = p((7, E)|(7, E), a_0) = 1. \end{aligned}$$

A key feature of this model is that the robot must trade off between safe but slow and risky but fast policies. This type of trade-off is characteristic of the key tension in many applications modeled by Markov decision processes. Here, by trying to reach the coffee room and returning to the office by the shortest route, the robot risks a high probability of falling down the stairs. If robot motion with an empty cup does not involve any randomness, that is $p_E = 1$, the robot will travel from the office to the coffee room by the shortest path but most likely take a more cautious path when returning to the office with a full cup.

Application challenges

The above example is artificial but Markov decision processes have been widely applied to robotic control. Realistic challenges include:

1. converting sensor readings to state variables,
2. transforming actions to motor commands,
3. representing variability in intended movement,
4. providing the robot with a mapping of the area, and developing reward structures.

These challenges are amplified when the application involves robotic movement in three-dimensional space.

3.3 Revenue management: Using price to manage demand

This section describes a different approach to inventory management. It pertains primarily to perishable products that decline in value over multiple periods.

As a concrete example, consider the challenges faced by a friend, Frank Z., previous owner of a chain of women's fashion stores in Vancouver, Canada. One evening, on the way to a poker session, he told Marty that:

Setting prices is like a game of chance; if I mark down prices too early in the season, I lose revenue, but if I wait too late, I can't sell my inventory and also must pay to store it.

A more general formulation of Frank's problem for a single product follows[13]. A retailer acquires M units of a product (for example, a size 10 woman's dress in a particular pattern and design) and prices it at a_K. The retailer hopes that fashion-conscious and well-off customers will purchase all of the inventory at this price but if not, some inventory will remain. To sell the remaining inventory, the retailer may choose to *markdown* the price to make it accessible to more price-sensitive customers.

More formally, prices are set at the beginning of each month, are chosen from a finite set of prices, and remain constant throughout the month. Assume that when the price is a the demand in period n is random and Poisson distributed with rate $\lambda_{n,a}$. It is reasonable to assume that $\lambda_{n,a}$ decreases in both time (n) and price (a). The rationale is that fashion products become less attractive as time goes on so that expected demand at a fixed price decreases over time. Moreover, fundamental economic principles suggest that in any month demand should increase when price is reduced[14].

[13] Frank stocks hundreds of products each season so that this problem must be solved hundreds of times each season, a daunting task without a formal model to do so.

[14] In economic modelling, this is referred to as a downward sloping demand curve.

Assume a monthly holding cost of $h(s)$ when the end of month inventory equals s units with $h(0) = 0$. Any goods left over at the end of month N are sold to an outlet store at a low price H per unit, representing the scrap value.

Decision epochs: Prices are set at the beginning of each month, so

$$T = \{1, 2, \ldots, N\}.$$

States: States represent the number of items in stock at the start of each month in the planning horizon:

$$S = \{0, 1, \ldots, M\}.$$

Actions: Actions represent the possible set of prices to choose from in each period. Assuming there are K candidate prices,

$$A_s = \{a_1, a_2, \ldots, a_K\}$$

for $s \in \{1, \ldots, M\}$. Note the price when $s = 0$ is irrelevant; set $A_0 = \{0\}$ for concreteness. Assume that $H < a_1 \leq a_2 \leq \cdots \leq a_K$.

Rewards: If the inventory at the end of the month is j, that means $s - j \geq 0$ units were sold during that month. Thus, for $n < N$, $a_k \in A_s$, and $s \in S$,

$$r_n(s, a_k, j) = \begin{cases} a_k(s - j) - h(j), & j = 0, \ldots, s \\ 0, & j = s + 1, \ldots \end{cases}$$

and $r_N(s) = Hs$.

Transition probabilities: Since in this formulation no inventory is added during the planning horizon, only transitions to states with ending inventory $j \leq s$ have non-zero probability. If the demand equals or exceeds s, then the ending inventory is 0. This logic leads to the following transition probabilities for $s \in \{1, \ldots, M\}$ and $n = 1, \ldots, N - 1$:

$$p_n(j|s, a) = \begin{cases} e^{-\lambda_{n,a}} \lambda_{n,a}^{s-j} / (s - j)! & j = 1, \ldots, s \\ \sum_{i=s}^{\infty} e^{-\lambda_{n,a}} \lambda_{n,a}^{i} / i! & j = 0 \\ 0 & j = s + 1, \ldots \end{cases}$$

and[15]

$$p_n(j|0, 0) = \begin{cases} 1 & j = 0 \\ 0 & j \neq 0. \end{cases}$$

[15]Note the only action in state 0 is to set the price to 0.

Application challenges

Applying this model presents two challenges: determining the set of markdown prices and estimating the time-varying demand function parameters. In retail, the markdown prices can be set as a percentage of the original price such as "50% off" or "80% off". Estimating the demand function requires considerable amounts of data and may be product specific. Data from similar products may provide guidance when there is insufficient historical data for the product. The parameter $\lambda_{n,a}$ may itself be represented as a function of n and a and learned during the decision problem.

3.4 Discrete-time queuing models

A queuing system consists of arrivals, a queue and one or more servers. Jobs arrive, wait in a queue if all servers are busy, are served by a free server and then depart the system when service is completed. Queuing systems have been well-studied in the operations research and engineering literature, and are applicable to a wide variety of service systems, including retail (jobs represent customers), healthcare (jobs represent patients), communication systems (jobs represent packets) and computer systems (jobs represent computing tasks).

From a decision perspective, the most widely studied models are:

1. **Service rate control:** The decision maker varies the service rate to control the queue length and throughput. The service rate may be controlled directly or through the addition and removal of servers.

2. **Admission control:** The decision maker chooses whether or not to admit an arriving job.

3. **Routing control:** In a network of queues, the decision maker chooses how to route jobs based on the workload at each queue.

The section formulates Markov decision process models for optimizing service rate and admission control. Exercise 8 provides a routing control example.

Some general comments regarding the formulation of discrete-time queuing systems follow:

1. Queuing systems are usually modeled as continuous-time Markov processes or semi-Markov processes. This example considers a discrete-time formulation. To do so, assume observation of the system starts at time 0. Let h denote a "small" unit of time and let decision epoch n correspond to "time" nh. Then the set of decision epochs denoted by $\{1, 2, \ldots\}$ correspond to times $\{h, 2h, \ldots\}$. The time increment h is chosen to be sufficiently small so that it is very unlikely more than one event (an arrival or service completion) occurs during a period of length h.

2. Queuing systems are usually modeled with infinite planning horizons to reflect that they are on-going and decision epochs occur frequently. No terminal reward is specified.

3. Rewards and transition probabilities are assumed to be independent of the decision epoch.

4. The system state is the number of jobs in the queue *and* in service.

3.4.1 Service rate control

Consider a single-server infinite capacity queuing system. At each decision epoch, the decision maker chooses a service completion probability from the set $\{a_1, a_2, \ldots, a_K\}$. Let b denote the probability that a job arrives between two decision epochs, independent of the number of jobs in the system. Assume that $a_1 \leq a_2 \leq \cdots \leq a_K$ and $a_K + b \leq 1$. Figure 3.4 provides a schematic representation of this queuing system.

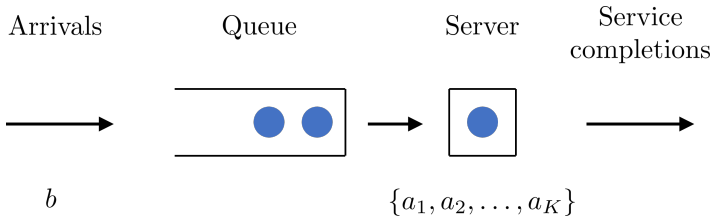


Figure 3.4: Schematic representation of a single server queuing system with adjustable service rate.

There are two costs to consider: a cost $m(a)$ per period for serving at rate a, and a delay cost of $f(s)$ per period when there are s jobs in the system at the start of the period. It makes sense to assume that both $m(a)$ and $f(s)$ are non-decreasing in their arguments.

A Markov decision process formulation follows. Note it assumes h is pre-specified and does not appear in the Markov decision process specification.

Decision epochs:

$$T = \{1, 2, \ldots\},$$

corresponding to time points $\{h, 2h, \ldots\}$.

States: States represent the number of jobs in the system (queue plus server):

$$S = \{0, 1, 2, \ldots\}.$$

Actions: Actions represent the probability that a job currently being served is completed in the current period prior to the next decision epoch. For $s \in S$,

$$A_s = \{a_1, a_2, ..., a_K\}^{16}.$$

Rewards: Costs should be represented as negative rewards, so

$$r(s, a) = -m(a) - f(s).$$

Note that this reward function is independent of the subsequent state.

Transition probabilities: For $s = 1, 2, \ldots$ and $k = 1, 2, \ldots, K$,

$$p(j|s, a_k) = \begin{cases} a_k & j = s - 1 \\ b & j = s + 1 \\ 1 - a_k - b & j = s. \end{cases} \tag{3.8}$$

For $s = 0$ and $k = 1, 2, \ldots, K$,

$$p(j|0, a_k) = \begin{cases} b & j = 1 \\ 1 - b & j = 0 \\ 0 & \text{otherwise.} \end{cases} \tag{3.9}$$

The rationale for these probabilities follows. When $s = 0$, a transition only occurs when there is an arrival. When $s > 0$, the assumption that h is small comes into play. This means that three things can happen, the state increases to $s + 1$ when an arrival occurs, the state decreases to $s - 1$ when a service is completed, and the state remains the same when neither occur.

A truncated model

An alternate formulation suitable for direct computation requires *truncating* the state space and modifying some transition probabilities. This is because direct numerical computation is impossible with a non-finite discrete state space[17]. In such a truncated model, all other model elements remain the same.

Truncated states: For some integer $W > 0$,

$$S = \{0, 1, \ldots, W\}.$$

[16] Since the server is idle when there are no jobs in the system, it is optimal to use the slowest service rate so that one could set $A_0 = \{a_1\}$. Ideally the server should be turned off, but this formulation does not allow that. Moreover because h is small, it might be impractical to turn off the server.

[17] However, such computation may not be needed if the structure of an optimal policy and value function can be succinctly encoded, e.g., a control limit policy or monotonic value function. See Section 4.4.

Truncated transition probabilities: When $s = W$, the system becomes blocked so arrivals are not possible. This means that for $k = 1, \ldots, K$,

$$p(s|W, a_k) = \begin{cases} a_k & \text{for } s = W - 1 \\ 1 - a_k & \text{for } s = W. \end{cases}$$

Choosing W presents some challenges. A well-controlled system will spend most of its time in low-occupancy states so an optimal policy might not be sensitive to the choice of W if it is chosen to be sufficiently large. A continuous approximation, where the mean and variance of the queue length can be computed in closed form, can provide some guidance. Alternatively, W may be varied and its impact noted.

3.4.2 Admission control

In admission control models, the decision maker assumes the role of a "gate keeper" by deciding whether or not to admit an arriving job into a queuing system. This provides an example of a model with a vector-valued state space and non-actionable states, which occur when no job arrives in the preceding period.

Again assume discretized time intervals of length h where h is sufficiently small so that at most one job can arrive between decision epochs and does so with probability b. If not admitted, the job is lost. Let $f(j)$ be the holding cost when there are j jobs in the system at the start of a period (after the admission decision, but before an arrival or service completion in the same period). Let w be the probability of a service completion between in a time interval of length h. Admitting a job into the system, generates a payment of R. Assume $b + w \leq 1$, which is reasonable when the time discretization step h is small. Figure 3.5 provides a schematic representation of the system and Figure 3.6 depicts the one-period dynamics.

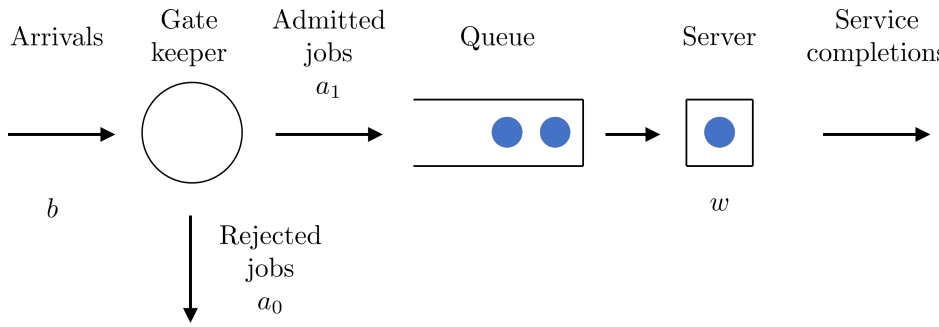


Figure 3.5: Schematic representation of a single server queuing system with admission control.

A Markov decision process formulation follows.

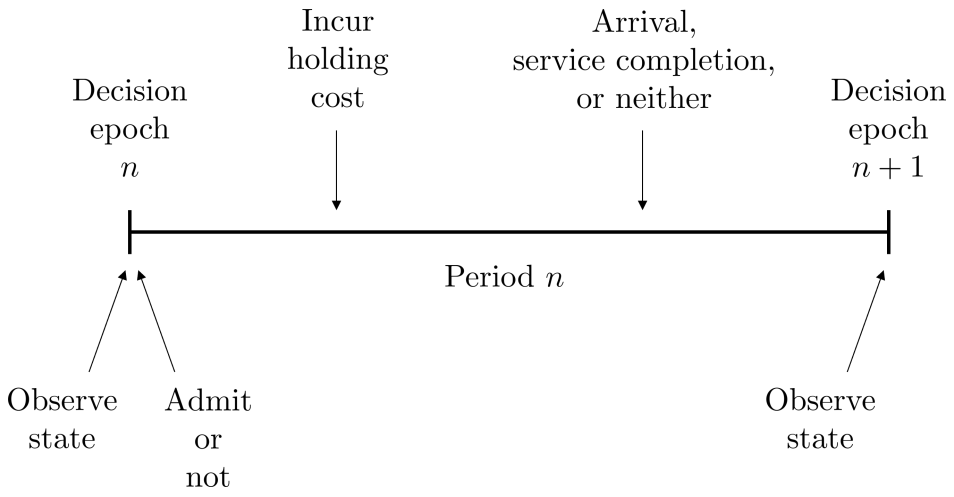


Figure 3.6: Timing of events in the queuing admission control model.

Decision epochs:

$$T = \{1, 2, \ldots\}$$

corresponding to time points $\{h, 2h, \ldots\}$.

States: The state has two components. The first component, denoted j, represents the number of jobs in the system and takes values in $J = \{0, 1, \ldots\}$, and the second component, denoted k, indicates whether there is a job waiting for admission ($k = 1$) or not ($k = 0$) so that $k \in \{0, 1\}$. Then the state space can be written as

$$S = J \times \{0, 1\},$$

with a typical state represented by the two-dimensional vector (j, k).

Actions: Let a_0 correspond to the action "do not admit" and a_1 to the action "admit"[18]. Since admission is possible only if an arrival occurred since the previous decision epoch, there is no choice in states[19] where $k = 0$. Thus, for $j = 0, 1, \ldots$,

$$A_{(j,k)} = \begin{cases} \{a_0, a_1\}, & k = 1 \\ \{a_0\}, & k = 0 \end{cases}$$

Recall that to formulate a Markov decision process model, actions need to be specified in all states even when the action set contains a single element.

[18] As a general convention, when the action space comprises two actions where one action is to "do nothing", that action will be referred to as a_0 and the other as a_1.

[19] Such states are said to be *non-actionable*.

Rewards: For $j = 0, 1, \ldots,$

$$r((j,k),a) = \begin{cases} R - f(j+1), & k = 1,\ a = a_1 \\ -f(j), & k = 0,\ a = a_0. \end{cases}$$

Note that in this model, the rewards are independent of the subsequent state (j', k').

Transition probabilities: If the action is to not admit ($a = a_0$), then whether there is a job currently waiting to be admitted or not ($k = 0$ or 1) is irrelevant since a waiting job will not be admitted. Thus, when $a = a_0$ for $j = 1, 2, \ldots$ and $k = 0$ or 1,

$$p((j',k')|(j,k),a_0) = \begin{cases} w & j' = j-1, k' = 0 \\ b & j' = j, k' = 1 \\ 1-b-w & j' = j, k' = 0 \\ 0 & \text{otherwise.} \end{cases} \tag{3.10}$$

Since service completions are not possible if the system is empty, the transition probabilities when $j = 0$ and $a = a_0$ are given by

$$p((j',k')|(0,k),a_0) = \begin{cases} b & j' = 0, k' = 1, \\ 1-b & j' = 0, k' = 0 \\ 0 & \text{otherwise.} \end{cases} \tag{3.11}$$

The "admit" action a_1 applies only when $k = 1$. So for $j = 0, 1, 2, \ldots$

$$p((j',k')|(j,k),a_1) = \begin{cases} w & j' = j,\ k' = 0 \\ b & j' = j+1,\ k' = 1 \\ 1-b-w & j' = j+1,\ k' = 0 \\ 0 & \text{otherwise.} \end{cases} \tag{3.12}$$

In contrast to (3.10), (3.12) is valid when $j = 0$ since even when the system is empty at the decision epoch, a decision to admit will add a job to the system, which can be completed during the same period with probability w.

Figure 3.7 summarizes the possible state transitions for each action. Despite the simplifying assumption that at most one event can happen in a period, keeping track of all transitions requires a careful accounting of events and actions and their interactions. This example is revisited in Chapter 4, which provides a simpler formulation of the model in terms of the *post-decision state* and *state-action value functions*. The post-decision state provides an alternative view of the decision timeline depicted in Figure 3.6 that allows the transition dynamics to be modeled more easily. As emphasized frequently in this chapter, drawing a correct timeline is an important part of formulating the model correctly.

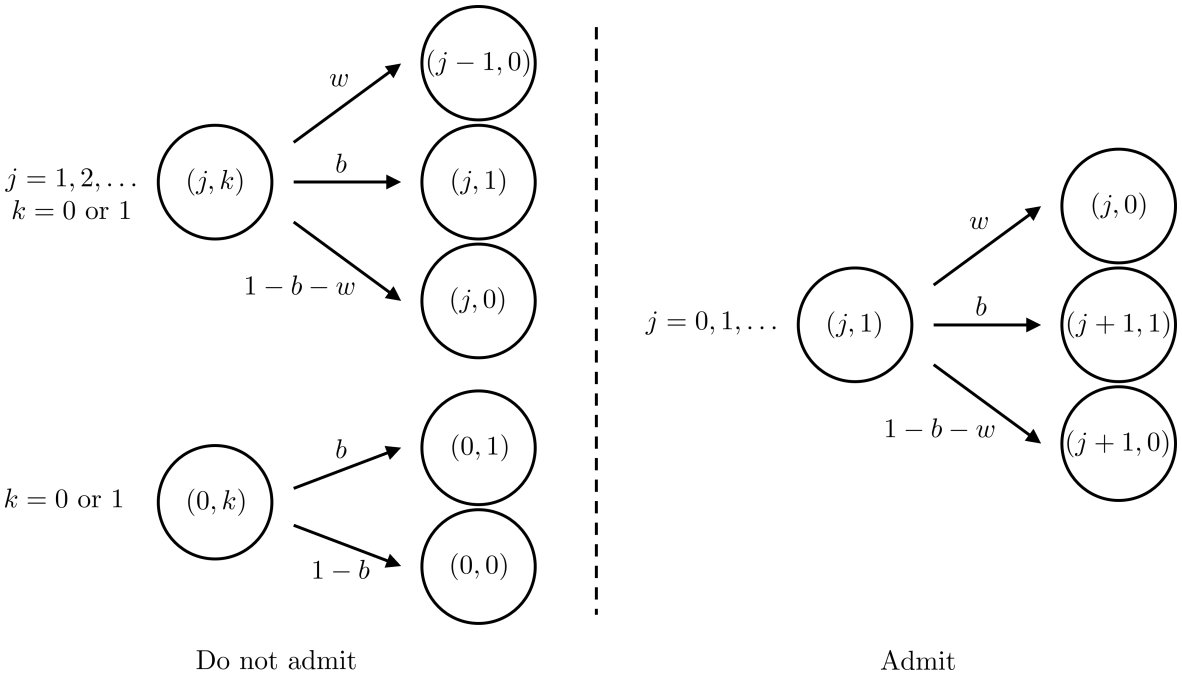


Figure 3.7: Possible state transitions for admission control problem. Note that not admitting a waiting job results in the same transitions as when no job arrives.

A post-decision state formulation

The following formulation is based on *post-decision states* in contrast to the above formulation based on *pre-decision* states.

Shifting the timing of decision epochs leads to a simpler formulation. Suppose instead that decision epochs occur just prior to resolving the uncertain event "Arrival, service completion or neither" in Figure 3.6 so the timing now follows Figure 3.8. In this formulation at a decision epoch, the previous decision has been implemented so that the state need only represent the number of jobs in the system at this decision epoch. Actions need to be modified to be contingent on the resolution of the uncertain event as described below. Note that the holding cost is assessed after the action has been implemented.

Decision epochs:

$$T = \{1, 2, \ldots\}$$

corresponding to time points $\{h, 2h, \ldots\}$, where $t = 1$ represents the time of the first decision.

States:

$$S = \{0, 1, \ldots\}$$

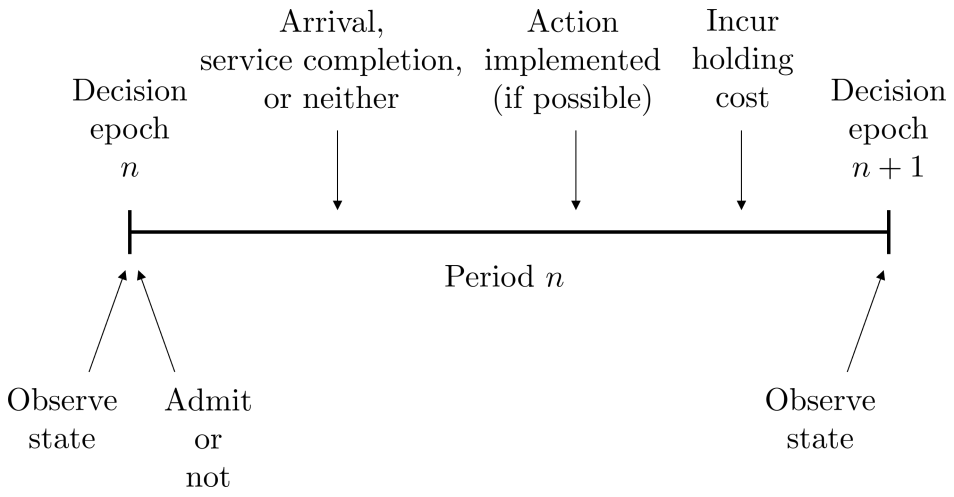


Figure 3.8: Timing of events in alternative formulation of the queuing admission control model. The expression "action implemented (if possible)" applies only if there is a prior arrival. Otherwise it cannot be implemented.

where $s \in S$ represents the number of jobs in the system at a decision epoch.

Actions: Let a_0 correspond to the action "do not admit" and a_1 to the action "admit". Since the decision maker does not know whether or not there will be an arrival, the result of this action must be contingent on the resolution of the event. Thus a_0 must be interpreted as "do not admit if there is an arrival" and a_1 as "admit if there is an arrival". By default if there is no arrival, the action a_0 applies since it will have the same effect as when there is an arrival. Thus for all $s \in S$,

$$A_s = \{a_0, a_1\}.$$

Rewards: In this formulation, rewards depend on the subsequent state and when the holding cost is assessed. Assuming the holding cost is incurred after the uncertain event is resolved and the decision implemented yields:

$$r(s, a_0, j) = \begin{cases} -f(j) & \text{for } j = s, s-1 \text{ and } s > 0 \\ -f(0) & \text{for } j = 0 \text{ and } s = 0 \\ 0 & \text{otherwise} \end{cases}$$

$$r(s, a_1, j) = \begin{cases} R - f(j+1) & \text{for } j = s+1 \text{ and } s \geq 0 \\ -f(j) & \text{for } j = s, s-1 \text{ and } s > 0 \\ -f(0) & \text{for } j = 0 \text{ and } s = 0 \\ 0 & \text{otherwise} \end{cases} \tag{3.13}$$

Observe that the payment R is received only when choosing a_1 and an arrival occurs, which corresponds to the case when $j = s+1$ in (3.13). In all other cases there are no admissions and only the holding cost is incurred.

Transition probabilities: Under a_0 a service completion is the only possible event that can occur when $s > 0$. It results in a transition to $s-1$. When the system is empty, such a transition cannot occur so that:

$$p(j|s,a_0) = \begin{cases} 1-w & \text{for } j = s \text{ and } s > 0 \\ w & \text{for } j = s-1 \text{ and } s > 0 \\ 1 & \text{for } j = 0 \text{ and } s = 0 \\ 0 & \text{otherwise} \end{cases}$$

Under a_1, an arrival results in a transition to $s+1$. When there is no arrival the system either remains in the same state or moves to state $s-1$ if $s > 0$ if there is a service completion.

$$p(j|s,a_1) = \begin{cases} b & \text{for } j = s+1 \text{ and } s \geq 0 \\ w & \text{for } j = s-1 \text{ and } s > 0 \\ 1-b-w & \text{for } j = s \text{ and } s > 0 \\ 1-b & \text{for } j = 0 \text{ and } s = 0 \\ 0 & \text{otherwise} \end{cases} \tag{3.14}$$

Note that this formulation may be more natural when using state-action value functions and simulation or when formulating it directly as a continuous time model. It is left to the reader to see which is preferred from the perspectives of ease of formulation and computation.

Application challenges

Applying these models requires estimates of arrival probabilities, service probabilities and costs. Queuing models are more commonly formulated in continuous time with inter-arrival and service times modeled using exponential random variables. Thus, if arrivals occur at rate λ, the probability of one arrival in an interval of length h is given by $\lambda h + o(h)$, the probability of no arrivals in an interval of length h is $1-\lambda h+o(h)$, and the probability of greater than one arrival in an interval of length h is $o(h)$ where $o(h)$ is an expression that converges to zero as h decreases to zero. Thus, it is convenient to set the probability of an arrival in a short time interval of length h to be λh.

As in other models, determining costs and rewards is somewhat arbitrary, and may depend heavily on the application. It is important to investigate the impact of specific choices through sensitivity analyses.

3.5 Behavioral decision making: When should a lion hunt?

Markov decision processes provide a natural framework for modeling behavior when an organism faces a decision that trades off survival with reserving energy. Examples include choosing a location for food acquisition, deciding when to hunt for food, choosing a group size when hunting and deciding when to abandon its offspring. The primary objective in such research is the determine whether results from an optimization model agree with observed animal behavior.

As an example, consider the challenge facing a lion (*panthera leo*) when deciding to hunt for food. Suppose the lion seeks to maximize its probability of survival over a season of N days. A mature lion has an energy storage capacity of C units. Each day it does not hunt the lion depletes its energy reserves by d units. Hunting requires h units of energy with $h > d$. If its energy reserves fall below c_0 units, it will not survive to the next day.

At the start of each day, the lion decides whether or not to hunt and if so, what prey to seek. Typically, lions hunt impalas, gazelles, wildebeests, giraffes and zebras. About half of the time they hunt in groups. This development assumes that the lion hunts alone[20]. Catch probability varies with species hunted. Assume that there are M species to choose from. Let w_m denote the probability that the lion catches an animal of species m, $m = 1, 2, \ldots, M$. A successful hunt for species m yields e_m units of energy. For simplicity, assume all relevant quantities have been discretized.

Decision epochs: Decisions are made at the start of each day during the season, so

$$T = \{1, 2, \ldots, N\}.$$

States: The state represents the lion's energy reserves at each decision epoch. Thus

$$S = \{0, 1, \ldots, C\}.$$

Actions: Actions in state s may be denoted by

$$A_s = \begin{cases} \{a_0, a_1, \ldots, a_M\} & s \in \{c_0, c_0 + 1, \ldots, C\} \\ \{a_0\} & s \in \{0, 1, \ldots, c_0 - 1\}, \end{cases}$$

where action a_0 corresponds to "do not hunt" and action a_m corresponds to "hunt for species m".

[20] Exercise 19 asks you to formulate the group size decision problem.

Rewards: Given the lion's survival objective, the lion receives a reward of 1 if alive at the end of the season and a reward 0 if not. Therefore

$$r_N(s) = \begin{cases} 1 & s \in \{c_0, c_0+1, \ldots, C\} \\ 0 & s \in \{0, 1, \ldots, c_0-1\}. \end{cases}$$

No rewards are accrued throughout the planning horizon, so $r_n(s, a, j) = 0$ for $n = 1, 2, \ldots, N-1$, $a \in A_s$, $s \in S$ and $j \in S$.

Transition probabilities: When the lion has energy reserves of s units at the start of the day, hunts for species m and is successful, its energy reserves at the start of the next day is $\min\{s - h + e_m, C\}$. The logic underlying this observation is that starting in state s, the lion uses h units of energy hunting and if successful obtains prey that provides e_m units of energy. However, the total energy acquired cannot exceed it capacity C. As the result of an unsuccessful hunt, its energy reserves fall to $\max\{s - h, 0\}$. Therefore

$$p_n(j|s, a) = \begin{cases} 1 & j = \max\{s-d, 0\}, s = c_0, \ldots, C, a = a_0 \\ w_m & j = \min\{s-h+e_m, C\}, s = c_0, \ldots, C, a = a_m \\ 1 - w_m & j = \max\{s-h, 0\}, s = c_0, \ldots, C, a = a_m \\ 1 & j = s, s = 0, 1, \ldots, c_0 - 1, a = a_0 \\ 0 & \text{otherwise.} \end{cases}$$

Note that the transition probabilities take into account the fact that the lion's energy level cannot exceed the maximum capacity or fall below 0, and if its energy level falls below c_0, it cannot hunt.

Application challenges

This application highlights the fact that it can be challenging to determine parameter values for a Markov decision process, and to do so, one must often appeal to a wide range of sources. Moreover, in this particular example, it is essential to understand the underlying animal behavior and model it correctly, ideally with expert input. An added benefit of developing a formal Markov decision process model is that it identifies relevant parameters that can motivate related research.

The ecology literature suggests values for many of the key model parameters although not always exactly in the form needed. One can use $C = 30$ and $d = 6$ kilograms, based on averages of male and female lions. If a lion-specific parameter is not available from the literature, borrowing values from other species may provide some guidance. For example, wild dogs expend about 23% more energy per hour when hunting, with the average duration of a hunt approximately equal to 40 minutes. Noting that lions undertake on average three chases per day and assuming that they expend

the same incremental amount of energy while hunting, on a day they decide to hunt, they will spend 3% more energy than on day they decide to rest so that $h = 1.03d$.

The literature suggests that gazelles yield a mean biomass of 12 kilograms with a catch probability of 0.15 on a single hunt. Observational data suggests that a lion may hunt up to three times a day if earlier hunts are unsuccessful; such dynamics can be incorporated into our model. Modeling the hunting of zebras presents additional challenges. Zebras yield an estimated 164 kilograms of edible biomass with a catch probability between 0.15 and 0.19 depending on the hunt location. Since they are large, their carcasses last for several days and are shared among several lions. Determining how much is available for the hunter requires further assumptions. Other sources provide estimates of the edible biomass for other types of prey such as impalas (29 kg), wildebeests (150 kg), and giraffes (468 kg), but not catch probabilities, which are harder to estimate. It is quite common for domain-specific literature to report data that allows us to estimate only some of the parameters of a Markov decision process, since such data is typically reported without a decision-making objective in mind. As a result, many assumptions must often be made, as described above. Especially when estimates are quite variable and many assumptions are needed, it is recommended to conduct sensitivity analyses of these parameters.

3.6 Clinical decision making: An application to liver transplantation

Markov decision processes have been widely applied to medical decision problems including organ transplantation, HIV treatment, cholesterol management and cancer diagnostics. As an illustration, the following describes a decision problem that arises when a patient requires a liver transplant.

This section describes the decision process for a patient with end-stage liver disease (ESLD)[21] who requires a liver transplant. Organs intermittently become available[22] and vary in quality. Depending upon the quality of the organ, the patient's medical team may either accept the organ and transplant it immediately, or reject it and wait for a higher quality organ.

To make this concrete, a relatively healthy patient may reject a lower-quality organ in the hopes of being offered a higher-quality organ in the future. On the other hand, a patient in poor health may accept the first liver available. A Markov decision process model can be used to formalize this decision problem and explore trade-offs.

To facilitate modeling, patient health status and organ quality are represented by discrete categories. Health states vary from 1 to H where 1 represents the healthiest state and H represents the least healthy state. The state Δ represents death by liver

[21] *End-stage liver disease* or *cirrhosis* refers to a condition in which a patient's liver is severely damaged and no longer able to function adequately. Without a transplant, it is usually fatal.

[22] From a recently deceased individual. These are referred to as *cadaveric* organs in the medical literature.

failure. Similarly, liver quality states are ordered from 1 (highest) to L (lowest). The state Φ represents an occasion when no liver is available.

Rewards measure life expectancy in days. On a day when there is no transplant, the patient accrues a reward of 1 (an extra day of life) independent of the health state. The (discounted) life expectancy post-transplant of a patient in health state h who accepts a liver of quality l is represented by $R(h, l)$. It is reasonable to assume (why?) that $R(h, l)$ is non-increasing in h and l.

To facilitate modeling, assume decisions are made daily, immediately after determining whether a liver is available and, if so, its quality. If a patient does not receive a transplant, their health state either remains stable or worsens. Furthermore, the likelihood of a patient being offered a liver depends on their current health state.

At each decision epoch, let

- $\gamma(h'|h)$ denote the probability that a patient in health state h deteriorates to state h' with $h' = h, h+1, \ldots, H$ at the next decision epoch,
- $\delta(h)$ denote the probability a patient in health state h dies from liver failure prior to the next decision epoch,
- $\beta(l|h)$ denote the probability that a patient in health state h is offered a liver of quality $l = 1, \ldots, L$ immediately prior to the next decision epoch, and
- $\phi(h)$ denote the probability a patient in health state h is *not* offered a liver prior to the next decision epoch.

Assume these distributions are stationary and independent. A Markov decision process formulation follows.

Decision epochs: Assume decisions are made daily after ascertaining whether a liver has been offered and if so, its quality. Further assume an infinite[23] horizon. Thus,

$$T = \{1, 2, \ldots\}.$$

States: States represent the patient's health (if alive) and liver quality (if available). Let the absorbing state Γ represent the post-transplant state. For convenience, define $S_H = \{1, \ldots, H\}$, $S_L = \{1, \ldots, L\}$ and $S_L^+ = S_L \cup \{\Phi\}$. Then,

$$S = (S_H \times S_L^+) \cup \{\Delta, \Gamma\}.$$

Note there is no need to distinguish Δ and Γ since the decision process ends in either case.

[23] A practical upper bound on the number of decision epochs might be 26,000 (assuming no transplants for people over 90 or younger than 20). Given this upper bound, an infinite horizon model may be appropriate and simpler, especially since the process will reach an absorbing state eventually, either post-transplant or death most likely before reaching 90 years old.

Some explanation may be helpful. States of the form $(h, l) \in S_H \times S_L$, correspond to being in health state h and having an organ of quality l available. Φ replaces L when no organ is available and $s = \Gamma$ when no more decisions are possible.

Actions: Let a_1 represent the action to accept an organ (assuming one is available) and a_0 represent the action to wait (do not accept an organ). Also let a_0 represent the "do nothing" action, which applies in the death state, in the post-transplant state, and in any state when no organ is offered. Hence

$$A_s = \begin{cases} \{a_0, a_1\} & s \in S_H \times S_L \\ \{a_0\} & s \in (S_H \times \{\Phi\}) \cup \{\Delta, \Gamma\}. \end{cases}$$

Note that a_1 is applicable only when the patient is alive and an organ is available, whereas a_0 applies regardless of whether an organ is available, the patient has passed away, or a transplant has already occurred.

Rewards: The reward function may be represented for $s = (h, l)$ as:

$$r(s, a, j) = \begin{cases} R(h, l) & (h, l) \in S_H \times S_L,\ a = a_1,\ j = \Gamma \\ 1 & (h, l) \in S_H \times S_L^+,\ a = a_0,\ j = (h', l') \in \{h, \ldots, H\} \times S_L^+ \\ 0 & (h, l) \in S_H \times S_L^+, a = a_0,\ j = (h', l') \in \{1, \ldots, h-1\} \times S_L^+ \\ 0 & (h, l) \in S_H \times S_L^+, a = a_0,\ j = \Delta \\ 0 & s \in \{\Delta, \Gamma\},\ a = a_0,\ j = s. \end{cases}$$

Note the second and third expressions correspond to the assumption that the patient's health state cannot improve[24]. Further the fourth expression indicates that the patient does not accrue an additional day of life if death occurs prior to the next decision epoch.

Transition probabilities: If the patient does not receive a transplant, then a transition from health state h to state h' with $h' \geq h$ may occur[25]. If the patient chooses to receive a transplant, then there is a deterministic transition to the post-transplant state, Γ. In the post-transplant or death state, the system remains in that state forever. Therefore

[24]Note that the reward under a transition to a $j = (h', l') \in \{1, \ldots, h-1\} \times S_L^+$ is arbitrary since this transition cannot occur.

[25]Recall that smaller values of h correspond to better health.

$$p(j|s,a) = \begin{cases} \gamma(h'|h)\beta(l'|h) & s=(h,l)\in S_H\times S_L^+,\ a=a_0,\ j=(h',l')\in\{h,\dots,H\}\times S_L \\ \gamma(h'|h)\phi(h) & s=(h,l)\in S_H\times S_L^+,\ a=a_0,\ j=(h',l')\in\{h,\dots,H\}\times \Phi \\ \delta(h) & s=(h,l)\in S_H\times S_L^+,\ a=a_0,\ j=\Delta \\ 1 & s\in S_H\times S_L, a=a_1, j=\Gamma \\ 1 & s\in\{\Delta,\Gamma\}, a=a_0, j=s \\ 0 & \text{otherwise.} \end{cases}$$

Application challenges

Application of Markov decision process models to clinical decision making requires detailed medical domain knowledge including the nature and progression of the disease, and the treatment options and processes. For instance, applying this model to liver transplantation requires in-depth knowledge of the liver transplant system, the process used to allocate organs and the progression of end-stage liver disease. Data required includes discretized patient health and liver quality states, an estimate of post-transplant life expectancy, probabilities of health state deterioration, and arrival distributions of organs for transplantation by quality. Such data may be obtained from transplant centers and organizations that manage the transplantation system, such as the United Network for Organ Sharing (UNOS) in the United States. Patient health status can be measured using the Model for End Stage Liver Disease (MELD) score, which is a function of various laboratory values. Higher scores indicate poorer health and a higher mortality rate[26]. When data is sparse, MELD scores can be aggregated or smoothed. A similar approach can be taken when defining liver quality states, which may depend on donor age, race and sex. UNOS data may be used to estimate $R(h,l)$ (days of survival post-transplant) with a proportional hazards model[27] and organ arrival rates. Transitions between health states can be modeled using a natural history model[28].

3.7 Advance appointment scheduling

In many applications, decision makers must allocate scarce resources prior to the arrival of future random demand. The formulation below is from the perspective of a scheduler in a hospital diagnostic imaging department who faces the challenge of scheduling appointments for current medical imaging requests to meet patient-specific clinical wait time targets, without knowing how many and when future requests will arrive. Ideally,

[26] Internet sources suggest that the 3-month survival rate of 27% in patients with the highest MELD score of 40 and 98% for patients with MELD score between 1 and 9.

[27] This is a statistical model that can be used to determine the impact of covariates on survival times when some patients in the data set are still alive.

[28] A commonly used epidemiological model that simulates disease progression accounting for different risk factors.

the scheduler would strive to minimize the fraction of patients scheduled beyond their target dates. The formulation[29] below introduces a cost to achieve this objective.

Suppose appointment requests arrive throughout the day and at the end of each day a radiologist assigns each request to one of K *urgency classes*. Urgency class k is associated with a target wait time of T_k days, $k = 1, 2, \ldots, K$, which is chosen based on clinical considerations. A patient in urgency class k should be scheduled prior to day T_k. The urgency classes are ordered, with 1 having the highest priority, so $T_1 < T_2 < \cdots < T_K$. If a patient in urgency class k is scheduled after T_k, a cost[30] C_k is incurred proportional to the number of days past T_k the appointment is scheduled. This cost can be thought of as a penalty related to worse clinical outcomes due to delayed diagnosis. Assuming $C_1 > C_2 > \cdots > C_K$ corresponds to setting higher delay costs for the most urgent cases. If a patient in class k is scheduled before the target T_k, no cost is incurred.

Let $p_k(w), k = 1, 2, \ldots, K$ denote the probability that w new class k appointments arrive each day. Let $\mathcal{W} = \{0, 1, \ldots, M\}$ denote the set of possible values of w. To ensure a finite state and action formulation, assume M is finite. Daily capacity is divided into B appointment slots. This means that at most B regular time appointments can be booked each day. Assume there are an unlimited number of overtime slots available and the system incurs a cost of h for each patient scheduled to overtime. Implicit is that $h > C_1$, which means that delaying a patient by a day beyond the target time is less costly than scheduling the patient to overtime. But for a sufficiently large delay, the cost of overtime will be less than the cost of delay.

Once all requests have an assigned urgency class, the scheduler assigns an appointment date to each request and informs the patient. Figure 3.9 provides a timeline for this process. The challenge is to schedule today's requests before realizing future requests in the face of limited capacity.

There are numerous issues arising in this and other similar scheduling problems that can impact modeling:

1. Does the system have access to surge capacity or overtime?
2. How does appointment length vary between patients?
3. Can overbooking be used to account for patient no-shows and late cancellations?
4. Can appointment dates be changed after scheduling?
5. Are the targets flexible or must they be met?
6. Does demand have any seasonal patterns or correlation across urgency classes?

[29] Such an application presents many modeling challenges and requires formulating the problem carefully. The formulation described below was not immediately obvious to the investigators (see Section 3.10) and required considerable trial and error to achieve.

[30] Note these costs are artificial, wait time targets in medical settings are often specified in terms of the proportion of demand that is scheduled prior to its target date.

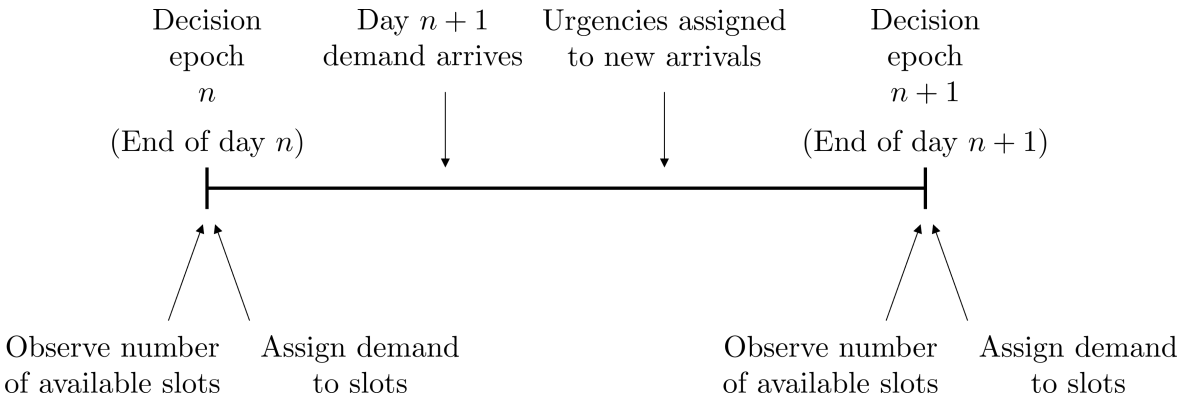


Figure 3.9: Timing of events in the advance appointment scheduling model.

7. How far in advance can appointments be scheduled?

The formulation below assumes:

- access to overtime,
- fixed appointment lengths,
- no cancellations,
- no rescheduling,
- flexible target dates (but with a penalty for exceeding the target date),
- stationary arrivals and uncorrelated demand between urgency classes,
- a fixed *booking horizon* of N days.

Note that the booking horizon refers to how far into the future current appointment requests can be scheduled, and not the length of the planning horizon. As an alternative to overtime, appointments not scheduled during a particular day may be held over for scheduling in the future at some cost. Exercise 16 considers this variation.

Decision epochs: Decision epochs correspond to the time in the day when the scheduling clerk assigns an appointment date to each appointment request waiting to be scheduled. This is naturally modeled as an infinite horizon problem, so

$$T = \{1, 2, , \ldots\}.$$

States: A typical state of the system is represented by $s = (b_1, \ldots, b_N, w_1, \ldots, w_K)$ where $b_n \in \mathcal{B} = \{0, 1, \ldots, B\}$ denotes the number of appointments that have already been booked on day n for $n = 1, \ldots, N$ and $w_k \in \mathcal{W}$ denotes the number of appointments of urgency class k waiting to be scheduled at a decision epoch. Note that if there are b_n appointments booked on day n, then there are $B - b_n$ remaining appointment slots on that day.

To simplify notation, introduce the vectors $\mathbf{b} := (b_1, \ldots, b_N)$ and $\mathbf{w} := (w_1, \ldots, w_K)$. Thus, a state is denoted by $(\mathbf{b}, \mathbf{w})$, and the state space is

$$S = \mathcal{B}^N \times \mathcal{W}^K.$$

Actions: Actions represent the number of waiting patients in urgency class k to schedule on each day within the booking window and possibly through overtime if B appointments if advantageous. Let x_{kn} denote the number of class k patients to book on day n and y_k denote the number of class k patients to book for overtime the next day. Note that it is not necessary to consider booking overtime slots at some point in the future since there is no limit on the number of patients booked through overtime, and booking overtime further in the future would simply increase costs.

Define the vectors $\mathbf{x} := (x_{11}, \ldots, x_{1N}, x_{21}, \ldots, x_{2N}, \ldots, x_{K1}, \ldots, x_{KN})$, $\mathbf{y} := (y_1, \ldots, y_K)$ and $\mathbf{0} := (0, 0, \ldots, 0)$ with lengths NK, K and $(N+1)K$, respectively. The action set $A_{(\mathbf{b},\mathbf{w})}$ is

$$A_{(\mathbf{b},\mathbf{w})} = \left\{ (\mathbf{x}, \mathbf{y}) \geq \mathbf{0} \;\middle|\; \sum_{n=1}^{N} x_{kn} + y_k = w_k \text{ for } k = 1, \ldots, K \text{ and } b_n + \sum_{k=1}^{K} x_{kn} \leq B \text{ for } n = 1, \ldots, N \right\}. \quad (3.15)$$

The first condition in (3.15) ensures that all class k requests must be scheduled either to a specific day or to overtime. The second condition ensures that at most B patients may be scheduled to regular time each day.

Rewards: Assume that the penalty associated with scheduling an urgency class k request n days from the current day costs $C_k(n - T_k)^+$. This function specifically models the cost as linear in the number of days a class k appointment is scheduled beyond its target, while incurring zero costs for scheduling prior to the target. Exercise 17 considers the variation where instead of the target representing a fixed day, a target window with both and earliest and latest time is assumed[31].

[31] This applies to chemotherapy schedules where there is a narrow window when a treatment can be given.

The reward for choosing actions $(\mathbf{x}, \mathbf{y})$ in state $(\mathbf{b}, \mathbf{w})$ is

$$r((\mathbf{b}, \mathbf{w}), (\mathbf{x}, \mathbf{y})) = -\sum_{k=1}^{K}\sum_{n=1}^{N} C_k(n - T_k)^+ x_{kn} - h\sum_{k=1}^{K} y_k.$$

The reward (negative of cost) captures the costs associated with exceeding the target times as well as overtime costs for the current set of appointment requests. Observe that the reward does not depend on the next state since the subsequent $\mathbf{b}$ vector is a deterministic function of the current action.

Transition probabilities: Transition probabilities depend on both action choice and the random arrival distribution. At the start of a period, the calendar rolls forward one day so previous bookings that were n days from the previous decision epoch are now $n-1$ days from the current decision epoch. Added to these bookings are the newly arriving demand that is booked over the N-day horizon starting at the current decision epoch. Finally, since there were no bookings $N+1$ days from the previous decision epoch, there are 0 appointments booked N days from the current decision epoch. The demand that has arrived since the last decision epoch is the only stochastic element in this model. Once this random demand has been assigned to urgency classes, the probability transitions are:

$$p((\mathbf{b}', \mathbf{w}') \mid (\mathbf{b}, \mathbf{w}), (\mathbf{x}, \mathbf{y})) = \begin{cases} \prod_{k=1}^{K} p_k(w'_k) & \mathbf{b}' = (b_2 + \sum_{k=1}^{K} x_{k2}, b_3 + \sum_{k=1}^{K} x_{k3}, \ldots, b_N + \sum_{k=1}^{K} x_{kN}, 0) \\ 0 & \text{otherwise.} \end{cases} \tag{3.16}$$

Some comments on this formulation follow:

1. The concept of a booking horizon may be regarded as an artifact of the modeling process and imposed to maintain a finite state space. Since overtime is unlimited, a fixed booking horizon applies. Without overtime, an unbounded booking horizon may be needed.

2. Because the booking horizon remains constant between decision epochs, but moves forward each day, the model may be regarded as using a *rolling horizon* model.

3. Note that the transitions decompose into a deterministic part corresponding to the number of booked appointments each day and a random part corresponding to the random demand for each urgency class.

4. When the maximum daily demand for each urgency class is M, the model has $(C+1)^N(M+1)^K$ states. This makes direct computation infeasible for practical sizes of these parameters and motivates the need for approximation (see Chapter 9).

5. After an action is implemented at decision epoch n, the $\mathbf{b}$ component of the state does not change until after decision epoch $n+1$. For reasons discussed in Section 4.5 and Chapter 11, it may be more convenient to formulate the model in terms of post-decision states.

6. In reality, many possible reward structures may be applicable for this problem. For example, as an alternative to incurring costs for appointments scheduled beyond their target date, a decision maker could strive to maximize the fraction of patients scheduled within their target dates.

Application challenges

Application challenges include specifying a booking horizon, specifying how unscheduled demand is satisfied, determining urgency classes and targets, specifying costs for delays, and estimating demand.

In real problem settings, a booking horizon may be determined by the decision maker based on their typical clinical processes. It has also been shown that when unlimited appointment diversion is possible, such as through overtime, an optimal policy is independent of the booking horizon, provided it exceeds the largest wait time target. Urgency classes should be defined based on clinical guidelines by medical professionals. The above model formulation was based on a real application. In that application the classes were "urgent" (7 day target), "semi-urgent" (14 day target) and "non-urgent" (28 day target). Emergency cases were scheduled to a different resource. Relative delay costs can potentially be quantified by calculating the impact on clinical outcomes of delayed treatment due to the delayed imaging.

Future demand may be forecasted using historical data. In practice, these distributions may be non-stationary as volumes generally increase over time. As an alternative to estimating the demand for each urgency class separately, if the historical data suggests that the relative proportion of cases from each urgency class is stable, it may be best to estimate total demand and then split it among each class based on the fixed proportion.

3.8 Optimal stopping

An elegant collection of applications are referred to as "optimal stopping problems". Examples include selling an asset, finding a parking spot, and online dating. Optimal stopping problems have attracted considerable research effort, which has primarily focused on showing that optimal policies have intuitively appealing structure.

In optimal stopping problems, the system evolves as a (possibly non-stationary) Markov chain on a set of states S' with transition probabilities $b_n(j|s)$ for $s \in S'$ and $j \in S'$ at epoch n. If the decision maker decides to "stop" in state s at decision epoch n, a reward of $g_n(s)$ is received. If the decision maker decides to "continue," the decision maker incurs a cost $f_n(s)$. In the finite horizon case, when the problem terminates

after N decision epochs, the decision maker receives a reward $h(s)$ if the Markov chain is in state s at epoch N.

3.8.1 Model formulation

Decision epochs: As noted above, this can be either a finite or infinite horizon model, so

$$T = \{1, \ldots, N\}, \quad N \leq \infty.$$

States: The state space is the union of S' and a state Δ that denotes the stopped state:

$$S = S' \cup \{\Delta\}.$$

Actions: Let the action a_0 denote the decision to continue and a_1 represent the stopping decision. Then the action set is

$$A_s = \begin{cases} \{a_0, a_1\} & s \in S' \\ \{a_0\} & s = \Delta. \end{cases}$$

The action to continue in the stopped state is included for completeness.

Rewards: The reward does not explicitly depend on the destination state j, so for $n < N$

$$r_n(s,a) = \begin{cases} -f_n(s) & s \in S', a = a_0 \\ g_n(s) & s \in S', a = a_1 \\ 0 & s = \Delta, a = a_0, \end{cases}$$

with $r_N(s) = h(s)$ for $s \in S'$ if the horizon is finite. In the infinite horizon case, it is more appropriate that $f_n(s) = f(s)$ and $g_n(s) = g(s)$ for all $s \in S'$ and $n = 1, 2, \ldots$.

Transition probabilities: For $n \leq N$

$$p_n(j|s,a) = \begin{cases} b_n(j|s) & s \in S', a = a_0, j \in S' \\ 1 & s \in S', a = a_1, j = \Delta \text{ or } s = j = \Delta, a = a_0 \\ 0 & \text{otherwise.} \end{cases}$$

Again, in the infinite horizon case, $b_n(j|s) = b(j|s)$ for $n = 1, 2, \ldots$.

3.8.2 Examples

Selling an asset

A homeowner who is moving to another city has N days to sell a house. Offers arrive throughout the day and by the end of the day, the homeowner has to decide whether to accept the best offer received that day, or wait until the next day for new offers. The set S' represents the set of possible values of the best daily offer for the house, assumed to be around its market value (i.e., bounded) and rounded to the nearest dollar (finite). By waiting until the next day, the homeowner incurs costs $f_n(s)$ related to continued home ownership such as maintenance, mortgage interest, property taxes, and advertising. By accepting the best offer on a given day, the homeowner receives a reward of s, minus the costs associated with selling the house, $L_n(s)$, which includes realtor fees and taxes. Hence $g_n(s) = s - L_n(s)$. At day N, the homeowner must accept the best offer that day, receiving a terminal reward of $h(s) = s - L_N(s)$. The best offer j at decision epoch $n+1$ is determined by a probability distribution $b_n(j|s)$ that may be conditional on the best offer s in epoch n. This distribution may be non-stationary. For example, early in the planning horizon, rejections could signal that the homeowner expects higher offers in the future than the current best offer. Later in the planning horizon, if bidders know the homeowner must sell, the offers might decrease in value.

Finding a parking spot

A driver seeks a parking spot as close as possible to a restaurant. Assume the driver can move in one direction only and see only one spot ahead. If the spot is not occupied, the driver may choose to park in it or proceed forward. The probability that any spot is unoccupied is assumed to be equal to p, independent of all other spots.

The most natural formulation is as a stationary infinite horizon model with an infinite state space. Elements of S' consist of vectors, (j,k), where j indicates the location of the parking spot and k indicates whether the spot is free (1) or occupied (0). Let $\mathbb{Z} = \{\ldots, 2, 1, 0, -1, -2, \ldots\}$ represent the distance of each parking spot to the restaurant where 0 denotes the location of the restaurant, positive numbers denote locations before the restaurant and negative numbers denote locations past the restaurant. Thus, $S' = \mathbb{Z} \times \{0, 1\}$.

States of the form $(j, 1)$ are the only ones in which the action a_1 corresponding to stopping is available. No rewards are received if the individual does not park, that is $f\big((j, a_0)\big) = 0$. When the individual parks, the reward is the distance from the parking location to the restaurant, so that $g((j, 1)) = -|j|$.

Transition probabilities of the underlying Markov chain are given by

$$b\big((j',k')|(j,k)\big) = \begin{cases} p & j' = j-1, k = 1 \\ 1-p & j' = j-1, k = 0. \end{cases} \tag{3.17}$$

This can be reduced to a finite state formulation by assuming:

- the driver will not start looking for a spot until reaching a distance M before the restaurant, and
- that if the driver has not parked before reaching the restaurant the driver will choose the next available spot.

As a result of the second assumption, the last decision is made in state $(1, 1)$ because the driver will definitely park in state $(0, 1)$ and continue in state $(0, 0)$. Hence the terminal reward is

$$h\big((0,k)\big) = \begin{cases} 0 & k = 1 \\ -\frac{1}{p} & k = 0. \end{cases}$$

To see this, note that from 0 onward, the distance to the next available parking spot follows a geometric distribution with parameter p so that

$$\big|h\big((0,0)\big)\big| = 1p + 2(1-p)p + 3(1-p)^2p + \ldots = \frac{1}{p}.$$

Online dating

This example provides a modern take on a classical problem that is often referred to as the *secretary problem*[32].

An individual is searching for people to date on a dating app. The dating app shares a brief profile for each potential match, including a photo and description, one at a time. For each potential match, the individual can either "swipe left" to pass or "swipe right" to indicate interest. If the individual swipes left, that profile will not be shown again. If the individual swipes right, a match is made and the two will go on a date. The app offers a free trial until the first match is made or until N profiles have been viewed, whichever comes first. The individual is interested in maximizing the probability of finding the best match during the free trial.

Assume that the N potential matches have an unobservable ranking from 1 to N, with 1 representing the best match. Through the process of examining the profiles, the individual will be able to rank the candidates seen so far relative to each other in a manner that is consistent with the true ranking. If the best profile is seen, the individual will only know that it is the best seen so far, but will not know whether any future profiles will be better. The order of profiles is completely random, so every permutation of the ranks 1 to N is equally likely.

The following formulation may not be immediately obvious, but is the most succinct way to model the problem. Let $S' = \{0, 1\}$, where the state indicates whether the current profile is the best seen so far (1) or not (0). No rewards or costs are accrued if the individual passes by swiping left ($f_n(s) = 0$). In this formulation, rewards correspond to the probability of choosing the best candidate and are only received

[32] A closely related problem was first formulated by Cayley [1875] and described in the historical summary in Chapter 1.

upon stopping. If the individual reaches the last profile, the match is automatically made. The terminal reward is thus $h(1) = 1$ and $h(0) = 0$, since the last profile is either the best profile seen so far or not.

If the individual swipes right on the n-th profile, $n \leq N$, and it is not the best profile seen so far, then the probability that the n-th profile corresponds to the best match is $g_n(0) = 0$. But if it is the best profile seen so far, then $g_n(1) = n/N$. To see why this is true, write out the probability that $g_n(1)$ represents explicitly:

$$\begin{aligned} g_n(1) &:= P[\text{profile } n \text{ is rank 1} \mid \text{profile } n \text{ has the highest rank of first } n \text{ profiles}] \\ &= \frac{P[\text{profile } n \text{ is rank 1 “and” profile } n \text{ has the highest rank of first } n \text{ profiles}]}{P[\text{profile } n \text{ has the highest rank of first } n \text{ profiles}]} \\ &= \frac{P[\text{profile } n \text{ is rank 1}]}{P[\text{profile } n \text{ has the highest rank of first } n \text{ profiles}]} \\ &= \frac{1/N}{1/n} = \frac{n}{N}. \end{aligned} \tag{3.18}$$

The second equality follows from the definition of a conditional probability. The third equality is due to the fact that if profile n is the top ranked profile, it must be the top ranked profile within the first n profiles as well. Finally, the fourth equality is due to the fact that the order of the profiles is completely random. So the top ranked profile is equally likely to be in any of the N positions. Similarly, any of the first n profiles is equally likely to be the one with the highest rank.

To determine the transition probabilities, again appeal to the fact that the order of the profiles is completely random. Thus, regardless of the current state, the probability that profile $n+1$ will be the best among the first $n+1$ is $1/(n+1)$. Thus, the transition probabilities are

$$b_n(j|s) = \begin{cases} \frac{1}{n+1} & j = 1, s \in \{0, 1\} \\ \frac{n}{n+1} & j = 0, s \in \{0, 1\}. \end{cases} \tag{3.19}$$

An elegant solution to this problem is provided in Chapter 4.

3.9 Sports strategy

Analytical methods have recently found widespread use in sport decision making, providing many opportunities for applying Markov decision processes. Some applications involve decisions made throughout a game while others are situational. Situational decisions such as whether to “go for it” on fourth down in North American football or whether to steal a base or sacrifice in baseball concern a decision in a particular state in a model of the whole game. They reduce to one-period problems (Section 2.4) when the value function for all games states is estimated from historical data. Other examples such as those described below concern recurrent decisions throughout a game or some portion of it.

3.9.1 When to pull the goalie in ice hockey

In ice hockey, a team has the option of replacing its goalie with an offensive player at any time during a game. This is called "pulling the goalie". Doing so can be beneficial since there is a greater probability of scoring when an extra offensive player is in the game. However, pulling the goalie also results in a greater likelihood that the opponent scores, since the goal is undefended. Such a goal is referred to as an "empty net goal". Such a strategy is often employed late in a game when the team is behind, in order to achieve a tie and send the game to overtime.

This decision problem is naturally modeled in continuous time but in keeping with the development of the book, a discrete time formulation follows. Assume that every h seconds a decision is made whether or not to pull the goalie, and that such a decision is not considered prior to M seconds remaining in the game. Usually, M is on the order of 180 seconds (3 minutes).

For concreteness, assume that Team A trails Team B by g goals. Let p_A (p_B) denote the probability Team A (Team B) scores one goal in an interval of length h when Team A pulls its goalie and let w_A (w_B) denote the probability Team A (Team B) scores a goal in an interval of length h when Team A does not pull its goalie. Naturally, $p_A > w_A$ and $p_B > w_B$. It may also be the case that no team scores during an interval of length h, so $p_A + p_B < 1$ and $w_A + w_B < 1$. As is customary when discretizing continuous time models, assume that h is sufficiently small so the likelihood of scoring more than one goal in that interval is negligible.

Decision epochs: Because decisions are made every h seconds up to M seconds before the end of the game,

$$T = \{1, 2, \ldots, N\},$$

where $N = M/h$. The first decision epoch corresponds to M seconds left in the game, the second corresponds to $M - h$ seconds left in the game, and so on.

States: The state represents the goal differential, defined as Team B's goals minus Team A's goals. Let G be the maximum goal differential at which the coach would consider pulling the goalie. Then

$$S = \{0, 1, \ldots, G + 1\}.$$

This formulation adds the absorbing state $G + 1$ to ensure a finite state space. If the goal differential reaches $G + 1$ the coach of Team A will not consider pulling the goalie anymore. If the score differential returns to G because Team A scored a goal without its goalie pulled, the decision problem starts anew. In practice, $G = 3$. A team would not pull its goalie if it is leading, so negative values are omitted from the state space. The state 0 corresponds to a tie score, which also is an absorbing state and the objective of pulling the goalie.

Actions: In all states the coach has the option to not pull the goalie (action a_0) or to pull the goalie (action a_1) when the goal differential is between 1 and G. Thus

$$A_s = \begin{cases} \{a_0, a_1\} & s = 1, \ldots, G \\ \{a_0\} & s = 0 \text{ or } G+1. \end{cases}$$

Rewards: Since the objective is to tie or win by the end of the planning horizon, rewards are only received at termination. So $r_n(s, a) = 0$ for $n < N$ and all s and a. The terminal reward is given by

$$r_N(s) = \begin{cases} 0 & s > 0 \\ 1 & s = 0. \end{cases}$$

Transition probabilities: Under the assumption that one event can occur in a time interval of length h, the transitions probabilities satisfy

$$p_n(j|s, a) = \begin{cases} w_A & j = s-1, s = 1, \ldots, G, a = a_0, \\ w_B & j = s+1, s = 1, \ldots, G, a = a_0, \\ 1 - w_A - w_B & j = s, s = 1, \ldots, G, a = a_0 \\ p_A & j = s-1, s = 1, \ldots, G, a = a_1, \\ p_B & j = s+1, s = 1, \ldots, G, a = a_1, \\ 1 - p_A - p_B & j = s, s = 1, \ldots, G, a = a_1, \\ 1 & j = s = 0, G+1, a = a_0, \\ 0 & \text{otherwise.} \end{cases}$$

Application challenges

The key parameters in this model are the relative scoring probabilities. They may vary by team and also depend on the whether the opposing team has been assessed a penalty so that pulling the goalie results in a "two-man advantage" and an increased scoring probability.

The following data comes from the (North American) National Hockey League for the 2013-2020 seasons. When both teams are at full strength (no player is in the penalty box), teams score goals at the rate of 2.25 goals per 60 minutes. When a team pulls its goalie, its scoring rate increases to 6.39 goals per 60 minutes. However, the opposing team's scoring rate increases to 19.16 goals per 60 minutes. Assuming $h = 5$ seconds, these goal scoring rates correspond to $w_A = w_B = 0.003125$, $p_A = 0.008875$ when pulling the goalie and $p_B = 0.02661$ for an empty net goal. On average, teams pull their goalie around 4 minutes prior to end of the game with a three goal deficit, 2.3 minutes with a two goal deficit, and 1.4 minutes with a one goal deficit. The success rate for pulling the goalie with a one goal deficit is about 14%.

Penalties play a large role in ice hockey. Pulling the goalie when Team B is penalized[33] significantly affects the scoring probabilities, increasing p_A and decreasing p_B relative to the previous values. The same data shows that pulling the goalie when the opposing team has one penalized player increases Team A's scoring rate to approximately 12 goals per 60 minutes ($p_A = 0.0167$), and decreases Team B's scoring rate to approximately 11 goals per 60 minutes ($p_B = 0.0153$).

3.9.2 A tennis handicap system

This section proposes a handicapping system for tennis. Consider a tennis match between two players of unequal skill level. In order to have a fair (and enjoyable) match, the stronger player (Player B) offers the weaker player (Player A) a handicap. The handicap takes the form of a budget of "credits" that Player A can use to win a point without playing it. To our knowledge, such a system has been yet to be widely applied but conceptually presents an interesting strategic challenge: when should Player A use these credits?

Before describing how such a handicapping system may be employed, a basic understanding of tennis scoring is needed. A tennis match consists of games and sets. A player wins a game by scoring four points first, provided that the player "wins by at least two points." That is, if both players have scored three points, the winning score needs to be at least 5-3. A player wins a set by being to first to win 6 games, again with a "win by two game" rule in effect. If the set score reaches 6-6, then a tiebreaker is played, with the winner winning the set by a score of 7-6. Finally, a match is typically the best two out of three sets or best three out of five sets.

A key feature of this scoring system is that it is "hierarchical": the match score decomposes into sets and games, each with its own scoring system. Thus the score with Player A serving may be 1-1 in sets, 5-3 in games and 3-1 (commonly referred to as 40-15) in the current game. Faced with this situation, Player A could use a credit to win the game, and hence the set and the match.

Unlike sports such as soccer, in which every goal contributes equally to the final score, not every point is equally valuable in tennis. In fact, a player could win more than 50% of the total points in the match, but still lose the match, due to the hierarchical nature of the scoring system.

To make this precise, if Player A uses a credit at the start of a point, then Player A wins the point, the handicap budget is decremented by one, and the players proceed to play the next point. If Player A decides not to use a credit, then the point is played as usual. Let p_1 (p_0) be the probability[34] that Player A wins the point on serve (return) if it the point is played out. Assume that the budget applies to the entire match and that Player A can use a credit regardless of which player is serving.

[33] The consequence of a penalty is that one or more fewer players are "on the ice".

[34] In most matches the probability of winning a point on serve is greater than when the opponent is serving.

Decision epochs: The start of each point is a decision epoch. Given the scoring system described above, the horizon is infinite but with a random stopping time corresponding to the end of the match. Thus

$$T = \{1, 2, \ldots\}.$$

States: The state comprises the current match score, q, the budget of credits remaining, b, and an indicator for the serving player, k. Suppose the set of possible match scores is $Q = \{q_1, q_2, \ldots\}$, the starting number of credits is B, and $k = 1$ (0) indicates that Player A (Player B) is serving. The two absorbing states, W and L correspond to Player A winning and losing the match, respectively. Thus, the state space is

$$S = (Q \times \{0, 1, \ldots, B\} \times \{0, 1\}) \cup \{W, L\}. \tag{3.20}$$

Each state q represents a set score, a game score and a within-game score. Note that B is at most 24 times the number of sets[35] to win the match. In a best two-out-of-three sets match, $B \leq 48$, and in a best three-out-of-five sets match $B \leq 72$.

Actions: Actions are to use a credit (a_1) or not (a_0) at each decision epoch when there are credits remaining. Once no credits remain, the only action is a_0.

$$A_s = \begin{cases} \{a_0, a_1\}, & s = (q, b, k) \text{ with } b > 0 \\ \{a_0\}, & \text{otherwise.} \end{cases} \tag{3.21}$$

Rewards: Since Player A's objective is to allocate handicap credits so as to maximize the probability of winning the match, the only non-zero reward is when there is a transition to state W, in which case the reward equals 1. Thus

$$r(s, a, s') = \begin{cases} 1, & \text{if } s \neq W, a \in A_s, s' = W \\ 0, & \text{otherwise.} \end{cases} \tag{3.22}$$

Similar to the online dating application, to maximize the probability of an event, the model formulation should be set up so that decision maker receives a reward of 1 only when that event occurs. This follows directly from the observation that the expected value of an indicator variable equals the probability of the indicated event.

Transition probabilities: If a credit is used, then there is a deterministic transition to the state in which Player A has one extra point. Using such a credit could also affect the game and set score, and even result in winning the match. Otherwise, the transitions follow the point-winning distribution of Player A against Player B. For convenience, let q^+ (q^-) denote the score if Player A wins (loses) the point when the

[35] Since it requires four points to win a game, using 24 credits would allow a player to win a set.

current score is q. Similarly, let k_q^+ (k_q^-) denote the server if Player A wins (loses) the point when the current score is q and the current server is indicated by k. The server changes only if by using the credit, Player A wins the current game. Thus, given that the current state is $s = (q, b, k)$,

$$p(j|s,a) = \begin{cases} 1, & \text{if } j = (q^+, b-1, k_q^+), a = a_1 \\ p_k, & \text{if } j = (q^+, b, k_q^+), a = a_0 \\ 1-p_k, & \text{if } j = (q^-, b, k_q^-), a = a_0 \\ 0, & \text{otherwise.} \end{cases} \tag{3.23}$$

Application challenges

The primary challenge in the application of this model revolves around the estimation of the point-win probabilities p_0 and p_1. Such probabilities depend on several factors including the strength of the opponent (perhaps considering the specific opponent and previous head-to-head successes), the court surface, the weather, and recent playing history. These probabilities are likely to be non-stationary as well due to factors like fatigue or injury.

3.10 The art of modeling

One learns to formulate Markov decision processes by studying how others have done so and trying it out for themselves. Formulations that appear particularly crisp are likely the result of numerous iterations of formulating and re-formulating the problem. By being exposed to and working through many different examples, one starts to build an intuition for how certain problem types are formulated. Below, a systematic approach to model formulation is presented.

How to formulate a Markov decision process model

- **Clearly define the problem.** Verbally describe what the decision maker wishes to achieve, what information is available on which to base decisions, how the system responds to these decisions, and what rewards or costs are incurred as a consequence of the decisions taken. A precise problem description facilitates identifying all model components. Often, however, one must return to, revise or redefine problem characteristics to ensure that the Markov decision process model properly represents the specified situation. The examples above present problem statements that are self-contained, with the information needed to fully formulate the problem.

- **Draw a timeline of events.** Carefully specify when the **state** information becomes available, when **actions** are chosen (i.e., the **decision epochs**), when

rewards are received and when **transitions** occur. Changes to the timing of events can impact the specification of certain model components. Several examples in this chapter illustrate the sequence of events in a typical period. Selected problems at the end of the chapter ask you to reformulate models under modified assumptions about the timing of events.

- **Identify decision epochs.** Specify the precise time at which actions are chosen, using the timeline as a guide.

- **Determine the planning horizon.** Applications may have a horizon that is finite with fixed length, finite with variable length or infinite. Variable length models arise when the policy or realization of a probability take the system to an absorbing state such as in the lion hunting model (Section 3.5), liver transplant model (Section 3.6), the Gridworld model (Section 3.2) and the optimal stopping models (Section 3.8). Most infinite horizon models may be transformed into finite horizon problems through an appropriate reformulation or simply through truncation. An example is provided in the parking problem (Section 3.8.2).

- **Identify states.** The main challenge when formulating a Markov decision process is determining an appropriate state space. Doing so requires taking into account all other model components. Hence, this is the most important step. A well-defined state space can make the rest of the formulation appear obvious; a poorly defined state space may result in complicated (non-Markovian) dynamics, extra computational burden, or insufficient information to write down a complete model. The advance appointment scheduling (Section 3.7) and online dating application (Section 3.8.2) illustrate two challenging examples of state space formulation.

  States should encapsulate all of the information available to the decision maker (and no more than is necessary) to specify actions, rewards and transition probabilities. Do not include actions as part of the state space except when one needs to know the current action to decide on future actions[36]. Often, a time component is required for decision making, but the decision epoch itself may be sufficient to avoid including an extra variable in the state space. Some models may require the addition of zero-reward absorbing states to account for early or random termination times.

- **Specify actions.** Be sure to note whether different sets of actions may be available in each state. Since the Markov decision process formulation requires specifying an action for each state, in *non-actionable* states, that is where there is no meaningful action choice such as an absorbing state, the set of actions should be specified as a single element representing the "do nothing" action.

[36] As an example consider the queuing service rate control model when there is a fixed cost for changing an action. In this case one needs to know the current action to determine if a switching cost applies.

- **Determine rewards.** Rewards may be stationary or vary with decision epoch. In finite horizon models, be sure to specify a terminal reward function. Also, note whether rewards depend on the subsequent state. It may be possible to model a problem both ways, with rewards that do or do not depend on the next state. However, usually one of these two is more natural. Some of the above examples use a reward function that does not depend on subsequent state. When using criteria based on expected reward, rewards that depend on subsequent states my be replaced by their expectations.

  In cases when the decision maker seeks to maximize the probability of an outcome, such as surviving or winning, specify a reward of zero in all states not corresponding to that outcome and a reward of one when that outcome occurs. The reason for this is that the expected value of an indicator of an event equals the probability of the event occurring.

  In some reinforcement learning formulations there is no explicit reward function available or its realization is delayed far into the future. In episodic models, the reward may correspond to winning a game or reaching a target state so that the reward is only learned when the episode is completed. In such cases it may prove useful to modify the reward function to indicate progress towards a goal.

  While a Markov decision process is generally concerned with maximizing rewards, its formulation, and the reward function specifically, should be independent of the specific choice of optimality criterion such as expected total reward, expected discounted reward, long-run average reward, or expected utility. Also, note that in many applications a decision maker may seek to minimize costs, which can be regarded as negative rewards. Since the Markov decision process formulation here seeks to maximize rewards, costs are best regarded as negative rewards.

- **Specify transition probabilities.** These are often quite complicated and contain many special cases. It is important to appeal to the timeline and the order of events when writing down the transitions. Challenges include taking into account "edge cases" at state space boundaries and noting that some components of the state may evolve deterministically, while others may evolve stochastically. In the presence of absorbing states, be sure to note that under the "do nothing" action the system remains in that state with probability one. For completeness, be sure to note zero probability transitions corresponding to impossible combinations of states and actions, which can be captured under the catch-all heading "otherwise".

  Recall that a Markov decision process with a fixed policy results in a Markov reward process that evolves over a Markov chain. Drawing a Markov chain with directed arcs indicating transitions and denoting probabilities on arcs is a simple but effective method to help ensure that all transitions are accounted for (e.g., probabilities leaving a state for a given action sum to 1) and that they make sense (e.g., transitions occur between states as described in the problem statement).

With complicated multi-dimensional state spaces, drawing such a picture is often a must. Figure 3.7 provides an example.

- **Estimate model parameters.** Most of the examples in this chapter are abstracted from real problem situations. To apply the models in concrete settings, one must estimate the model parameters such as transition probabilities and rewards.

  In some cases, such as inventory control (Section 3.1), revenue management (Section 3.3) and queuing control (Section 3.4) the transition probabilities may be derived from parametric distributions with the parameters estimated from historical data. When there are no parametric forms for transition probabilities, care must be taken in estimating probabilities because some may be non-zero but very small. In applications such as the lion hunting model (Section 3.5), clinical decision making (Section 3.6) and sports strategy (Section 3.9) one may appeal to the data and literature from those fields to obtain parameter estimates.

  Another challenge when applying models in practice is specifying rewards. In applications where rewards refer to concrete monetary values, specifying rewards can be relatively straightforward. In examples such as lion hunting (Section 3.5), optimal parking and online dating (Section 3.8.2), pulling the goalie (Section 3.9.1), and tennis handicapping (Section 3.9.2) the reward is implicit in the chosen model objective. In other cases, rewards may be derived in consultation with the decision maker.

Bibliographic remarks

Inventory models (Section 3.1) date back at least to Arrow et al. [1951] and Dvoretsky et al. [1952]. The book of Arrow et al. [1958] is an important early reference. Porteus [2002] provides an overview of the historical development in his book on inventory models. Much current research in inventory theory is subsumed under the heading "supply chain management". Section 8.9.2 of Puterman [1994] provides an inventory example of a constrained MDP with a service level constraint.

The newsvendor model seems to originate with Edgeworth [1888] where it is developed to determine optimal cash reserves to meet random withdrawals from a bank. Arrow et al. [1951] derived the critical fractile solution. It is now described in all operations management textbooks; Chen et al. [2016] provide a modern survey.

The model in Section 3.3 is in the spirit of Gallego and van Ryzin [1994] who provide one of the first examples of dynamic pricing. The book of Talluri and van Ryzin [2004] provides a comprehensive overview of revenue management. Dynamic pricing combined with overbooking has been applied extensively in the airline industry, where it is referred to as yield management [Smith et al., 1992].

The queuing control models in Section 3.4 have mostly been studied in continuous time. Early references include Yadin and Naor [1967], Heyman [1968], Naor [1969]

and Sobel [1969]. Our formulation of the discrete time service rate control model follows de Farias and Roy [2003] where they use it to illustrate approximate dynamic programming methods.

The lion hunting example in Section 3.5 is adopted from Clark [1987]. That paper provides many of the parameter values described in the section on application challenges, including the energy storage capacity, daily energy depletion, biomass yield and catch probabilities of gazelles and zebra. The estimate of energy expenditure while hunting was based on Hubel et al. [2016]. Edible biomass of other prey listed were taken from Smuts [1979]. Other applications in ecology include Mangel and Clark [1986], Kelly and Kennedy [1993] and Sirot and Bernstein [1996].

Numerous applications of using Markov decision processes in clinical decision making have appeared in the literature. The model described herein is based on Alagoz et al. [2007], who focus on liver transplantation. The discussion around application challenges is based on methods they employed to specify their model and estimate parameters. Other examples of clinical decision making using Markov decision processes include Shechter et al. [2008], who consider HIV therapy, and Kurt et al. [2011], who model statin treatments for diabetes patients.

The formulation of the advance scheduling model in Section 3.7 follows Patrick et al. [2008]. The result about an optimal policy being independent of the booking window if the window is longer than the largest wait time target and if the system has access to unlimited appointment diversion is given in that paper. Sauré et al. [2012] and Goçgun and Puterman [2014] analyze variants of this model. An example of using the impact of delayed treatment to quantify the cost of delayed imaging appointments appears in Sauré et al. [2012] in the context of radiation therapy.

A Gridworld model appears in Sutton and Barto [2018]. Such models have been widely used in the computer science community to illustrate Markov decision process and reinforcement learning concepts.

Optimal stopping problems originate with early work of Wald [1947], Wald and Wolfowitz [1948] and Arrow et al. [1949]. Karlin [1962] proposes and solves the asset selling problem. The optimal parking problem appears in Chow et al. [1971]. The online dating (i.e., secretary) problem was first proposed by Cayley [1875] in the context of evaluating a lottery.

Sports applications have appeared broadly. The path-breaking monograph of Howard [1960] introduces many of the key Markov decision process concepts, and contains an example of using Markov decision processes in baseball strategy. Carter and Machol [1971] and Chan et al. [2021] develop value functions in football. The pulling the goalie model in Section 3.9.1 originates with Morrison [1976]. A dynamic programming formulation was provided in Washburn [1991]. Hall [2020] provides the recent data quoted in the section on application challenges. The tennis handicapping model in Section 3.9.2 appears in Chan and Singal [2016]. The amazing book by Kemeny and Snell [1960] includes a Markov Chain model for a tennis game.

Exercises

1. Formulate a periodic inventory control problem in which orders arrive after demand has been fulfilled. Clearly show how the timing of events in Figure 3.1 changes.

2. Formulate a periodic inventory control problem in which sales are lost if demand exceeds supply in a period.

3. Formulate a periodic inventory control model in which there is limited storage capacity, limited backlogging and a bound on order size. Assume if the quantity backlogged exceeds its bound, an extra cost incurred.

4. Formulate the reward function of the inventory management example when the revenue of the inventory sold is included.

5. **A newsvendor model with one replenishment opportunity.** Formulate the following variant of the newsvendor model as a Markov decision process. Assume at the start of the period that the newsvendor purchases units of a product from the supplier at a cost of c_1 per item but can purchase additional units at a prespecified later time during the period at a cost c_2 with $c_2 > c_1$. Such a problem arises when sales exceed initial expectation early in the period so that it may be opportune for the newsvendor to purchase additional items halfway through the period.

6. Formulate a service rate control queuing model with a fixed cost C for changing the service rate. Note that as alluded to above in Section 3.10 it is not sufficient to just modify the reward function.

7. Formulate a combined admission and service rate queuing control problem as a Markov decision process.

8. **Call center routing.** Consider a call center with monolingual (English-only) and bilingual (English and French) call takers. Assume there are two call takers of each type. English-speaking and French-speaking customers call in to the center and indicate their preferred language. English-speaking customers can be served by either type of call taker, but French-speaking customers can only be served by a bilingual call taker. Every minute an English-speaking customer calls in with probability p_E and a French-speaking customer calls in with probability p_F, where $p_E + p_F < 1$. Assume the probability of more than one caller per epoch is negligible. Customers incur a cost of C for every minute spent waiting before service. The duration of a call is geometric with parameter q, regardless of the language. Formulate this routing control problem as a Markov decision process.

9. **Control of a tandem queuing network.** Consider a discrete-time queuing system composed of two single-server queues in tandem and two types of jobs.

Type 1 jobs arrive at queue 1 and after completing service by server 1 also require service by server 2. Type 2 jobs arrive directly to queue 2 and require service at server 2 only. In each period, no job arrives or either a type 1 job arrives, a type

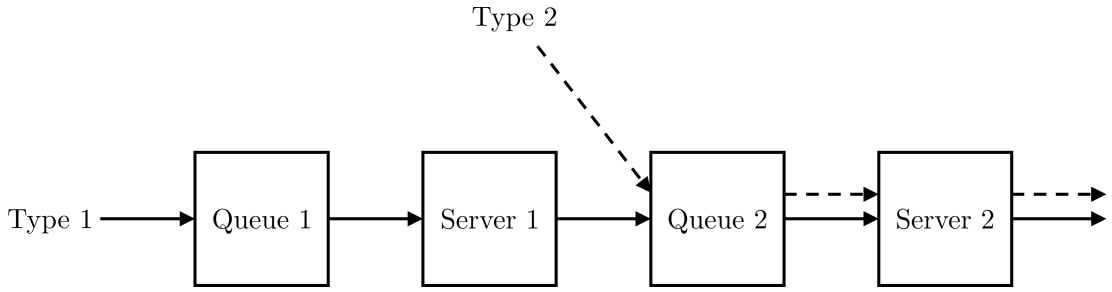


Figure 3.10: Tandem queuing network

2 job arrives or one of each type arrives. Assume the probability of an arrival of a type i job is p_i independent of the whether or not the other type job arrives.

Assume a finite buffer (waiting room) of size M_i in front of queue i. When the buffer is full jobs are blocked (lost) at penalty cost c_i. Completion of a type i job yields revenue R_i with $R_1 > R_2$. In addition, assume a holding cost of $h(s_1, s_2)$ when there are s_i type i jobs in the system (either in the queue or in service).

Formulate the following revenue maximization problems as Markov decision process.

(a) Job selection: In each period the controller of queue 2 can choose whether to serve a type 1 or type 2 job. Assume that the service is completed with probability q_2 independent of job type and whether or not a job at queue 1 has completed service with probability q_1. To simplify the formulation assume that if the service is not completed in the current period, the job reverts to the queue prior to the start of the subsequent period.

(b) Service rate control: In each period the system can choose the service probability at queue 1 from the set $\{q_{1,1}, q_{1,2}, \ldots, q_{1,N_1}\}$ with cost $f_1(q)$ that is non-decreasing in q.

(c) Describe and formulate other possible control problems that can apply to this configuration.

10. Formulate a version of the Gridworld navigation model in which there is a positive probability that the robot drops the coffee cup, which is larger if the cup is full. Assume the cup is breakable and if it is dropped, the robot needs to return to the office to retrieve another one.

    Clearly state any assumptions you are making in formulating this model.

11. Formulate a finite horizon version of the Gridworld problem in which if the robot does not return with coffee after N decision epochs, the mathematician gets his

or her own coffee and incurs a penalty of C units. How should C be related to X and R?

12. **Equipment maintenance.** Formulate the following maintenance problem. You own a piece of equipment that deteriorates over time. While it is operating, it contributes revenue of r dollars per month. When it has been operating for i months since its last maintenance, the probability it fails in the current month is $p(i)$ and the probability it does not fail is $1 - p(i)$. If it fails at any time during a month the cost of repairing it is c_A and it is available for use at the start of the subsequent month. Assume that if it fails during a month, no revenue is generated during that month. On the other hand, the maintenance manager can schedule preventive maintenance in a month at cost c_B. Assume preventive maintenance is always scheduled at the beginning of the month, starts in the first week of the month and takes one month.

    (a) Draw a timeline for the decision problem and clearly state any assumptions you are making.

    (b) Formulate the maintenance manager's problem as a Markov decision process. Be clear to state any assumptions you make.

    (c) In a real application, how do you think $p(i)$ will vary with i and what would be the relationship between c_A and c_B?

    (d) Propose a "real life" application of this model.

13. Stengos and Thomas [1980]. Consider the following generalization of the previous problem. You own two machines which sometimes require maintenance that takes three weeks. Maintenance on one machine costs c_1 per week while maintenance on two machines costs c_2 per week. The probability either piece of equipment breaks down if it has operating for i weeks is $p(i)$. Assume that if the equipment breaks down during a week, maintenance begins at the start of the next week. However, if you decide to perform preventive maintenance, you do so at the start of a week. Moreover, assume that the two pieces of equipment break down independently.

    (a) Formulate this problem as an infinite horizon Markov decision process.

    (b) How are c_1 and c_2 related? Why?

14. Reformulate Exercise 13 assuming that the time it takes to complete maintenance on a piece of equipment is random. In particular, assume that maintenance is completed in any period with probability q, independent of other periods.

15. Bertsimas and Shioda [2003]. A restaurant contains both two-seat and four-seat tables. Parties of two and four arrive randomly and request service. No reservations are taken. When a party of four arrives and a four-seat table is

empty, they should be seated but should the manager ever seat a party of two at a four seat table? If so, when? Also, when should requests for service be denied, if ever?

Formulate this problem as Markov decision process assuming the following, unrealistic as it may be. Decisions are made every 10 minutes and the restaurant operates 24 hours a day. There are two two-seat and two four-seat tables. Meals consist of two courses, durations of each are geometrically distributed independent of party size. Course 1's completion probability per epoch is 0.7, while course 2's is 0.8. In any period there is at most one arriving party. A party of two arrives with probability 0.2 and a party of four with probability 0.1. Assume that the waiting area holds at most 6 people. If it is full, arrivals are blocked and do not enter. Also any waiting party may leave in a 10 minute period with probability 0.05.

Revenue is as follows. A party of 2 contributes \$50 and a party of 4 contributes \$100. The cost of waiting (incurred by the restaurant) is \$6 per person per hour.

16. Formulate the advance scheduling problem when appointments not booked on first day available are added to the next days demand with cost C'.

17. Formulate the advance scheduling problem where instead of the target representing a fixed day, target windows are used for each urgency class. Let T_k^l and T_k^u be the lower and upper limits of the target window for urgency class k. If an appointment is scheduled within this window, no costs are incurred. If a class k appointment is scheduled before (after) T_k^l (T_k^u), then a cost of C_k^l (C_k^u) is incurred.

18. Modify the lion hunting problem to take into account that on any day, the lion may be captured by poachers with probability $1 - \lambda$. How is this related to discounting?

19. At the start of each day, a lion decides whether or not to hunt and if so, in what group size. The probability of catching prey varies with group size. This presents a trade-off: a larger group has a greater probability of a successful hunt, but then less food is available for each lion in the group. Assume a maximum group size of M and that all captured prey is split evenly among the group. Let λ_m, $m = 1, 2, \ldots, M$, denote the probability that a group of of size m is successful in its hunt. Assume the prey being hunted yields a total edible biomass of e units. Thus if the lion hunts in a group of size m and is successful, it receives e/m units of edible biomass.

20. Reformulate the original lion hunting behavior model so that the objective is to maximize the number of days of survival instead of the probability of survival. Clearly note what changes are necessary.

21. **The ride-sharing driver's dilemma.** At random times throughout the day, a ride-sharing driver receives offers of potential trips, including their expected revenue and time to complete the trip. The driver can either accept the trip or decline it and wait for the next offer. Formulate this problem as a discrete time Markov decision process clearly stating all assumptions being made.

22. Consider a variant of the online dating problem in which the decision maker's goal is to maximize the probability of choosing one of the two best candidates. How would you modify the formulation to take this into account.

23. Consider a variant of the online dating problem in which the decision maker's goal is to maximize the rank of the selected date. Modify the formulation accordingly.

24. Reformulate the tennis handicapping problem to account for second serves. That is, if a player misses a first serve, they have a second chance to get the ball in before losing the point. Suppose the probability that the first serve goes in is q_1 and the probability the second serve goes in is q_2. Conditioned on the serve going in, the probability of winning the point is $p_{1,1}$ and $p_{1,2}$ for the first and second serve, respectively. Since first serves tend to be more aggressive, $q_1 \leq q_2$ and $p_{1,1} \geq p_{1,2}$. Assume handicap credits can only be used when serving.

25. *"Scrabble, like life, is a trade-off between today and tomorrow-between spending and saving. It's what an economist would call a dynamic programming problem."* [Roeder, 2022] The game of Scrabble provides an opportunity for applying Markov decision processes. Develop a model for a decision of whether a player should replace some or all tiles during a turn. This is a rather complex model that will be challenging to formulate in its entirety. We do not believe it has been addressed in the Markov decision process literature. It may be amenable to reinforcement learning methods.

Part II - Classical Markov Decision Process Models

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

If I have seen further it is by standing on the shoulders of giants[1].

Sir Isaac Newton, mathematician, physicist, astronomer and much more, 1642-1727.

This part of the book contains five chapters:

- Chapter 4: Finite horizon models
- Chapter 5: Infinite horizon models: Expected discounted reward
- Chapter 6: Infinite horizon models: Expected total reward
- Chapter 7: Infinite horizon models: Long-run average reward
- Chapter 8: Partially observable Markov decision processes

Overview

The chapters in this part describe the classical models, theory, and algorithms of Markov decision processes. The core idea in multi-period Markov decision processes builds directly on the one-period model in Chapter 2: a decision maker aims to identify an action that balances the immediate reward from choosing that action and the future reward that can be gained from the new state to which the process transitions. However, instead of the process ending in the new state as in the one-period model, the process will continue for many more decision epochs. Thus, the decision maker

[1] Newton [1959].

needs to consider the possible actions that can be taken in all subsequent states, and the corresponding rewards that can be generated.

For example, consider a driver in traffic trying to get to their destination as quickly as possible. At an intersection, there is an option to take a side street that appears less busy. Choosing this action might lead to more immediate progress, but comes with uncertainty about whether subsequent streets will be busy or whether it will be possible to merge back on to the main street easily. Alternatively, the driver may choose to stay the course, going less slowly in the near term, but possibly making up time later by staying on the main street.

The best way to learn this material is through analyzing simple examples. Models with a few states and actions, such as the two-state example from Section 2.5, provide excellent vehicles to try out algorithms. By extending analyses to larger models, you will gain an appreciation of the challenges faced when a model has a large number of states and actions. Approaches for such models are the subject of the third part of the book.

Chapter details

Chapter 4 studies finite horizon models, those that end at a fixed future stage that is known at the start of the planning horizon. It introduces the concepts of the value of a policy, an optimal policy, the optimal value function, the optimality (Bellman) equation, and backwards induction. It illustrates these concepts through examples and also shows how to establish the structure of an optimal policy.

Chapters 5-7 focus on infinite horizon models, each with a different optimality criterion. The reason for this separation is that each requires different theoretical considerations.

- In **discounted reward models**, the inclusion of a non-negative discount factor less than one ensures, under boundedness of rewards, that values are well-defined and bounded. Moreover this enables analysis based on properties of contraction mappings.

- In **total reward models**, assumptions on rewards or transition probabilities are needed to ensure values are well defined. These models are most widely applied to *episodic* problems, which reach a reward-free absorbing state in a finite but variable time under some or all policies. A special class of these models, referred to as *transient*, inherit some of the contraction properties of discounted models.

- Analysis of **average reward models** depends explicitly on the structure of Markov chains corresponding to stationary policies. Most complete results are available when these Markov chains are aperiodic and in which all states are accessible from each other. Generalizations to models with transient states and multiple recurrent classes require more subtle analyses. Moreover the auxiliary concept of the *bias* of a policy is fundamental to theory and computation.

As closely as possible, each of these chapters evolves as follows. After defining optimality, they establish the existence and optimality of solutions of the appropriate Bellman equation, and that stationary deterministic policies are optimal in the class of all policies. They then show how to find optimal values and policies using value iteration, policy iteration and its variants, and linear programming.

Chapter 8 is rather distinct. It studies finite and infinite horizon partially observable Markov decision processes (POMDPs), where instead of knowing the state with certainty, the decision maker only observes a signal that is dependent on the true unobservable state of the process. Decisions need to account for this extra source of uncertainty, which leads to analyzing related models with a continuous state space corresponding to probability distributions on the unobservable state. This is the only chapter which considers continuous state spaces.

Chapter 4

Finite Horizon Models

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

"Begin at the beginning," the King said gravely, "and go on till you come to the end: then stop."[1]

From *Alice's Adventures in Wonderland*, by Lewis Carroll, 1832-1898.

This chapter focuses on models in which the planning horizon, N, is finite and fixed, and decision makers evaluate and compare policies on the basis of their expected total reward. Such models will be referred to as *finite horizon models*.

The assumption of a *fixed* horizon length means that the decision maker knows the number of decision epochs in the planning horizon in advance. For example, in an airline reservation model, the date of the departure of a specific flight is fixed. This contrasts to situations in which the planning horizon is finite in expected value but its actual length may vary from realization to realization. Examples include clinical decision making problems (Section 3.6), which terminate at disease onset or death, or Gridworld models (Section 3.2) in which the robot's actions determine the time to complete a task. Models with variable horizon lengths are best analyzed with methods from Chapters 5 or 6 by converting a variable horizon length model into an infinite horizon model though the introduction of a zero-reward absorbing state.

Recall that a policy π is a sequence of decision rules $(d_1, d_2, \ldots, d_{N-1})$ that determines action choice at each decision epoch n, $n = 1, \ldots, N-1$, either deterministically or randomly. Although policies can be quite general (recall the taxonomy in Chapter 2), Section 2.4 showed that in the special case of a one-period problem there always exists a Markovian deterministic policy that is optimal. It turns out that this result

[1] Carroll [1865].

holds in finite horizon models under mild assumptions, such as S finite and A_s finite for all $s \in S$. Thus, in this chapter, much of the exposition will focus on Markovian deterministic policies.

The results in this chapter, while important on their own, also provide a framework for analyzing and solving infinite horizon models in later chapters. In particular, this chapter introduces the following fundamental concepts:

1. Value of a policy,
2. Optimality equations,
3. Backwards induction (dynamic programming),
4. Existence of optimal policies,
5. Optimality of structured policies, and
6. Algorithms based on state-action value functions

The most significant results are that the existence of optimal policies within the class of Markovian deterministic policies and the development of a recursive algorithm for finding them.

4.1 The value of a policy in a finite horizon model

Definition 4.1. For each $s \in S$, the *value* of policy $\pi = (d_1, d_2, \ldots, d_{N-1}) \in \Pi^{\mathrm{HR}}$ is defined by

$$v^{\pi}(s) := E^{\pi}\left[\sum_{n=1}^{N-1} r_n(X_n, Y_n, X_{n+1}) + r_N(X_N) \,\middle|\, X_1 = s\right]. \tag{4.1}$$

In (4.1) X_n denotes the system state and Y_n the action chosen (either deterministically or randomly) by π at decision epoch n. The quantity $v^{\pi}(s)$ is frequently referred to as a *policy value function*.

A fundamental insight in Markov decision process theory and computation is that evaluation of $v^{\pi}(s)$ can be accomplished by decomposing it into a series of one-period problems that can be evaluated recursively. Although (4.1) is valid for any type of policy, the equations for computing the value differ between policy classes. Section 4.1.2 discusses this recursion for a Markovian deterministic policy and Section 4.1.3 focuses on history-dependent randomized policies.

4.1.1 Backwards induction

This section describes a fundamental recursion underlying computation and analysis in a finite horizon model.

Definition 4.2. For each $s \in S$, define the *future expected reward from stage n* as

$$v_n^\pi(s) := E^\pi \left[\sum_{i=n}^{N-1} r_i(X_i, Y_i, X_{i+1}) + r_N(X_N) \,\middle|\, X_n = s \right]. \tag{4.2}$$

Observe that:

1. This quantity represents the expected total reward from decision epoch n to the end of the planning horizon using policy π, starting in state $s \in S$ at decision epoch n.

2. Alternatively, this quantity can be interpreted as the expected total reward in an $N-n$ period problem that starts at state $s \in S$.

3. This expression is similar to (4.1) except that the summation runs from decision epoch n through decision epoch $N-1$, and the subscript on the reward and decision rules agree with the decision epoch number.

4. When rewards correspond to costs, $v_n^\pi(s)$ is often referred to as the *cost-to-go*.

5. The expressions $v^\pi(s)$ and $v_1^\pi(s)$ represent the same quantity.

Recursions

This section motivates the basic recursion in finite horizon models by showing how $v_1^\pi(s)$ and $v_2^\pi(s)$ are related. It is easy to see that

$$\begin{aligned} v_1^\pi(s) &= E^\pi \left[r_1(X_1, Y_1, X_2) + \sum_{n=2}^{N-1} r_n(X_n, Y_n, X_{n+1}) + r_N(X_N) \,\middle|\, X_1 = s \right] \\ &= E^\pi \left[r_1(X_1, Y_1, X_2) + v_2^\pi(X_2) \,\middle|\, X_1 = s \right]. \end{aligned} \tag{4.3}$$

The expression above shows that the value of policy $\pi = (d_1, \ldots, d_{N-1})$ is the expected total reward obtained by using decision rule d_1 at decision epoch 1 in state s to generate action Y_1, plus the expected total reward from decision epoch 2 onward (to the end of the planning horizon) starting in state X_2. and using decision rules $d_2, \ldots, d_{N-1}$. State X_2 is random because it is determined by transition probabilities and potentially random action selection that result from applying d_1 at decision epoch 1.

Since $v_2(\cdot)$ is unknown, forward recursions of this type cannot serve as the basis for computation. Rather, by starting at the end of the planning horizon and applying

similar recursions backwards one obtains the following process that can be used for computation.

Begin by settting $v_N^\pi(s) = r_N(s)$ for all $s \in S$. Then set

$$\begin{aligned} v_{N-1}^\pi(s) &= E^\pi \left[r_{N-1}(X_{N-1}, Y_{N-1}, X_N) + r_N(X_N) \,\middle|\, X_{N-1} = s \right] \\ &= E^\pi \left[r_{N-1}(X_{N-1}, Y_{N-1}, X_N) + v_N^\pi(X_N) \,\middle|\, X_{N-1} = s \right]. \end{aligned} \tag{4.4}$$

Since $v_N^\pi(s)$ is know, the expectation in (4.4) can be evaluated. Continuing in this way, write $v_n^\pi(s)$ as

$$\begin{aligned} v_n^\pi(s) &= E^\pi \left[r_n(X_n, Y_n, X_{n+1}) + \sum_{i=n+1}^{N-1} r_i(X_i, Y_i, X_{i+1}) + r_N(X_N) \,\middle|\, X_n = s \right] \\ &= E^\pi \left[r_n(X_n, Y_n, X_{n+1}) + v_{n+1}^\pi(X_{n+1}) \,\middle|\, X_n = s \right]. \end{aligned} \tag{4.5}$$

for $n = 1, \ldots, N-1$.

Once $v_{N-1}^\pi(s)$ for all $s \in S$ has been computed, use (4.5) to successively evaluate $v_{N-2}^\pi(s)$, $v_{N-3}^\pi(s)$ continuing all the way back to $v_1^\pi(s)$. Since $v_1^\pi(s) = v^\pi(s)$, this provides a way to find the value of policy π in every state. Such a computational approach is often referred to as *backwards induction* or sometimes as *dynamic programming*.

What backwards induction does is decompose the calculation of $v^\pi(s)$ into a series of calculations in one-period models like those described in Section 2.4. However the one-period models are linked through (4.5), where the quantity $v_{n+1}(s)$ is the terminal reward.

4.1.2 Evaluating a Markovian deterministic policy

This section provides an algorithm to compute the value of a Markovian deterministic policy and illustrates computations with an example. Since a Markovian deterministic policy generates a Markov reward process[2], the calculations also apply to any Markov reward process. As noted above, the fundamental idea is to decompose a $(N-1)$-period problem into a sequence of $N-1$ one-period problems.

Algorithm 4.1. Finite horizon policy evaluation for a Markovian deterministic policy

1. **Initialize:** Specify $\pi = (d_1, \ldots, d_{N-1}) \in \Pi^{\text{MD}}$. Set $n \leftarrow N-1$ and $u_N(s) \leftarrow r_N(s)$ for all $s \in S$.

[2]Recall that a finite state Markov reward process is a Markov chain together with a reward function defined on the states of the chain.

2. **Iterate:** While $n \geq 1$:

   (a) For all $s \in S$, evaluate $u_n(s)$ according to

   $$u_n(s) \leftarrow \sum_{j \in S} p_n(j|s, d_n(s))(r_n(s, d_n(s), j) + u_{n+1}(j)). \tag{4.6}$$

   (b) $n \leftarrow n - 1$.

3. **Terminate:** Return $u_1(s)$ for all $s \in S$.

Some comments about this algorithm follow:

1. Theorem 4.1 below shows that this algorithm calculates the expected total reward of policy π for the entire planning horizon and also from each decision epoch onwards. Specifically, it shows that $u_n(s) = v_n^\pi(s)$ for $n = 1, \ldots, N-1$ and $s \in S$. Most importantly, $u_1(s) = v_1^\pi(s) = v^\pi(s)$.

2. This algorithm employs the dummy variables $u_n(s)$ to distinguish calculations in the algorithm from the definition of the value function, facilitating a formalization of their connection below.

3. When $|S| = M$, at each iteration of the algorithm, step 2(a) requires on the order of M operations (multiplications and additions) to compute $u_n(s)$ for a given s. Since there are M states and the algorithm runs for $N-1$ iterations, the total number of operations required is $O(NM^2)$. In terms of storage, each iteration requires the storage of M values of $u_n(s)$, so storing all iterates results in total storage of $O(NM)$. However, if only the values from the most recent iteration, then storage requirements reduce to $O(M)$. While this book will not focus extensively much on algorithmic efficiency, it is important to recognize that efficient implementation can significantly impact on computation.

4. If $r_n(s, a, j)$ does not depend on the subsequent state, equation (4.6) simplifies to

   $$u_n(s) = r_n(s, d_n(s)) + \sum_{j \in S} p_n(j|s, d_n(s))u_{n+1}(j). \tag{4.7}$$

5. When π is a Markovian randomized policy, one needs to add an additional expectation to take account of the randomness in action choice under d_n (see Section 4.1.3).

A numerical example

The following calculations compare two approaches for evaluating a Markovian deterministic policy in a finite horizon model:

1. Backwards induction

2. Sample path enumeration.

It does so in the context of the following instance of the two-state model from Section 2.5. Let $N = 3$, set $r_3(s_1) = r_3(s_2) = 0$ and consider the Markovian deterministic policy $\pi = (d_1, d_2)$ where

$$d_1(s_1) = a_{1,2}, \quad d_1(s_2) = a_{2,2}$$

and

$$d_2(s_1) = a_{1,1}, \quad d_2(s_2) = a_{2,1}.$$

Backwards induction: Explicit calculations using backwards induction as described in Algorithm 4.1 proceed as follows.

1. Set $n = 3$. Since $r_3(s) = 0$ for $s = s_1, s_2$, set $u_3(s_1) = u_3(s_2) = 0$.

2. Set $n = 2$ and

$$\begin{aligned} u_2(s_1) &= p_2(s_1|s_1, a_{1,1})\big(r_2(s_1, a_{1,1}, s_1) + u_3(s_1)\big) + p_2(s_2|s_1, a_{1,1})\big(r_2(s_1, a_{1,1}, s_2) + u_3(s_2)\big) \\ &= 0.8(5 + 0) + 0.2(-5 + 0) = 3 \end{aligned}$$

and

$$\begin{aligned} u_2(s_2) &= p_2(s_1|s_2, a_{2,1})\big(r_2(s_2, a_{2,1}, s_1) + u_3(s_1)\big) + p_2(s_2|s_2, a_{2,1})\big(r_2(s_2, a_{2,1}, s_2) + u_3(s_2)\big) \\ &= 0 + 1(-5 + 0) = -5. \end{aligned}$$

3. Set $n = 1$ and

$$\begin{aligned} u_1(s_1) &= p_2(s_1|s_1, a_{1,2})\big(r_2(s_1, a_{1,2}, s_1) + u_2(s_1)\big) + p_2(s_2|s_1, a_{1,2})\big(r_2(s_1, a_{1,2}, s_2) + u_2(s_2)\big) \\ &= 0 + 1(5 + (-5)) = 0 \end{aligned}$$

and

$$\begin{aligned} u_1(s_2) &= p_2(s_1|s_2, a_{2,2})\big(r_2(s_2, a_{2,2}, s_1) + u_2(s_1)\big) + p_2(s_2|s_2, a_{2,2})\big(r_2(s_2, a_{2,2}, s_2) + u_2(s_2)\big) \\ &= 0.4(20 + 3) + 0.6(-10 + (-5)) = 0.2 \end{aligned}$$

4. Since $n = 1$, stop.

Since $v^\pi(s) = v_1^\pi(s)$, the calculations above show that $v^\pi(s_1) = 0$ and $v^\pi(s_2) = 0.2$.

Sample path enumeration As described in Section 2.2.4, this requires determining the probability and value of each sample path. Suppose the process starts in state s_1. At the first decision epoch, d_1 chooses action $a_{1,2}$, which results in a deterministic transition to state s_2. At the second decision epoch, at state s_2, d_2 chooses $a_{2,1}$, which results in a deterministic self-transition. Thus, the only possible realization of the process is $(s_1, a_{1,2}, s_2, a_{2,1}, s_2)$. The total reward along this sample path is 0, in agreement with the backwards induction calculations.

If the process starts in s_2, then several realizations are possible. They are summarized in Table 4.1.

Table 4.1: Realizations of states and actions, probabilities and total rewards under policy π starting in state s_2.

Realization	Probability	Total Reward
$(s_2, a_{2,2}, s_2, a_{2,1}, s_2)$	0.6	$-10-5=-15$
$(s_2, a_{2,2}, s_1, a_{1,1}, s_1)$	$0.4 \times 0.8 = 0.32$	$20+5=25$
$(s_2, a_{2,2}, s_1, a_{1,1}, s_2)$	$0.4 \times 0.2 = 0.08$	$20-5=15$

It follows that the expected total reward starting in s_2 equals $0.6(-15)+0.32(25)+0.08(15) = 0.2$, in agreement with the previous calculations.

For large N, the set of possible realizations becomes enormous, making sample path enumeration impractical for exact policy value function calculation. However, it forms the basis for the simulation methods in Chapter 10 so it is crucial to understand this concept.

Confirming that Algorithm 4.1 computes the value of a policy

This section proves that Algorithm 4.1 produces the value of a policy defined by equation (4.1). The proof uses the following basic result, which is sometimes referred to as the *Law of Iterated Expectations*[3].

Lemma 4.1. Let X and Y be random variables. Then

$$E[X] = E[E[X|Y]].$$

[3]For a concrete example of this lemma, let X and Y be random variables that respectively denote the lifetime earnings and years of post-secondary education of a randomly chosen individual. Computing the average earnings of all individuals $E[X]$ can be done by computing the average earnings conditioned on the number of years of post-secondary education $E[X|Y]$ and then averaging these values based on the probability distribution of post-secondary education years $P[Y]$. For example, if everybody has between 0 and 10 years of post-secondary education, then $E[X] = E[E[X|Y]] = \sum_{y=0}^{10} E[X|Y=y]P[Y=y]$.

Theorem 4.1. Let $\pi = (d_1, \ldots, d_{n-1}) \in \Pi^{\text{MD}}$. Suppose $u_n(s)$ is computed using Algorithm 4.1, $v^\pi(s)$ is defined as in (4.1) and $v_n^\pi(s)$ is as defined in equation (4.2). Then

1. $u_n(s) = v_n^\pi(s)$ for $n = 1, \ldots, N$ and all $s \in S$.
2. $v^\pi(s) = v_1^\pi(s)$ for all $s \in S$.

Proof. The first result is proved by (backwards) induction. Clearly, the result holds for $n = N$ since

$$u_N(s) = r_N(s) = E^\pi[r_N(X_n)|X_n = s] = v_N^\pi(s)$$

for all $s \in S$.

Now, assume for $t = n+1, \ldots, N$ and for all $s \in \mathcal{S}$,

$$u_t(s) = E^\pi\left[\sum_{i=t}^{N-1} r_i(X_i, d_i(X_i), X_{i+1}) + r_N(X_N)\middle| X_t = s\right] = v_t^\pi(s). \tag{4.8}$$

From Algorithm 4.1 and the induction hypothesis,

$$\begin{aligned}
u_n(s) &= \sum_{j\in S} p_n(j|s, d_n(s))\big(r_n(s, d_n(s), j) + u_{n+1}(j)\big) \\
&= \sum_{j\in S} p_n(j|s, d_n(s))\big(r_n(s, d_n(s), j) + v_{n+1}^\pi(j)\big) \\
&= E^\pi\left[r_n(X_n, d_n(X_n), X_{n+1}) + v_{n+1}^\pi(X_{n+1})|X_n = s\right] \\
&= E^\pi\left[r_n(X_n, d_n(X_n), X_{n+1}) + E^\pi\left[\sum_{i=n+1}^{N-1} r_i(X_i, d_i(X_i), X_{i+1}) + r_N(X_N)\middle| X_{n+1}\right]\middle| X_n = s\right] \\
&= E^\pi\left[r_n(X_n, d_n(X_n), X_{n+1})|X_n = s\right] + E^\pi\left[\sum_{i=n+1}^{N-1} r_i(X_i, d_i(X_i), X_{i+1}) + r_N(X_N)\middle| X_n = s\right] \\
&= E^\pi\left[\sum_{i=n}^{N-1} r_i(X_i, d_i(X_i), X_{i+1}) + r_N(X_N)\middle| X_n = s\right] \\
&= v_n^\pi(s),
\end{aligned}$$

where the fifth equality follows from Lemma 4.1 and the fact that the expectation of a finite sum equals the sum of expectations.

The second result simply restates the definition of $v_1^\pi(s)$. □

This result holds as well for Markovian randomized policies. The proof requires only a few modifications and is left as an exercise.

4.1.3 Evaluating a history-dependent randomized policy*

You probably would not want to evaluate a history-dependent randomized policy, but in case you do, this section will show you how. Moreover the development herein will be used in the technical appendix to establish the optimality of Markovian deterministic policies within the class of history-dependent randomized policies. Exercise 8 asks you to use it to evaluate a history-dependent randomized policy in a simple example.

Let $\pi = (d_1, d_2, \ldots, d_{N-1}) \in \Pi^{\text{HR}}$ be a history-dependent randomized policy. This means that at decision epoch n, d_n chooses action a according to some probability distribution $w^n_{d_n}(a^n|h^n)$. The following algorithm computes the value of a fixed history-dependent randomized policy. Note that since the number of histories grows exponentially with n, policy evaluation quickly becomes computationally intractable: it requires calculating $v^\pi_n(h^n)$ for each $h^n \in H_n$, where H_n is the set of all histories up to and including decision epoch n.

Algorithm 4.2. Finite horizon policy evaluation for a history-dependent randomized policy

1. **Initialize:** Specify $\pi = (d_1, \ldots, d_{N-1}) \in \Pi^{\text{HR}}$. Set $n \leftarrow N-1$ and $u_N(h^N) \leftarrow r_N(s^N)$ for all $h^N = (h^{N-1}, a^{N-1}, s^N) \in H_N$.
2. **Iterate:** While $n \geq 1$:
   (a) For all $h^n = (h^{n-1}, a^{n-1}, s^n) \in H_n$, evaluate $u_n(h^n)$ according to
   $$u_n(h^n) \leftarrow \sum_{j \in S} \sum_{a \in A_{s^n}} w^n_{d_n}(a|h^n) p_n(j|s^n, a)\big(r_n(s^n, a, j) + u_{n+1}(h^{n+1})\big), \tag{4.9}$$
   where $h^{n+1} = (h^n, a, j)$.
   (b) $n \leftarrow n-1$.
3. **Terminate:** Return $u_1(h^1)$ for all $h^1 \in H_1$.

Some comments about Algorithm 4.2 follow:

1. It is left as an exercise to show that $u_n(h^n) = v^\pi_n(h^n)$ for all $h^n \in H_n$ and $n = 1, \ldots, N$.
2. In the first iteration (when $n = N$), the algorithm assigns the same value to all histories
   $$h^N = (s^1, a^1, s^2, \ldots, s^{N-1}, a^{N-1}, s^N)$$
   that agree in state s^N. That is, if the state at decision epoch N is s, then all histories that end at s are given the same value.

3. In (4.9), recall that $w^n_{d_n}(a|h^n)$ represents the probability that action a is chosen in epoch n, given the history h^n and decision rule d_n. Since decision rules are history-dependent, the randomization distribution $w^n_{d_n}(a|h^n)$ may be different for different histories. However, rewards and transition probabilities depend only on s^n and not the entire history. Moreover, if action a is chosen in state h^n at decision epoch n and the next transition is to state j, the history at decision epoch $n+1$ is (h^n, a, j) (see (2.3)), which is the argument of $u_{n+1}(\cdot)$.

4.2 Optimal policies and their values

The goal in a finite horizon model is to find an optimal policy, that is, one that maximizes the expected total reward over all possible policies. This section, defines optimal policies and provides a recursion to compute them.

4.2.1 Definitions

Some key definitions follow.

Definition 4.3. A policy $\pi^* \in \Pi^{\mathrm{HR}}$ is an *optimal policy* if

$$v^{\pi^*}(s) \geq v^{\pi}(s) \tag{4.10}$$

for all $\pi \in \Pi^{\mathrm{HR}}$ and $s \in S$.

Definition 4.4. The *optimal value function* $v^*(s)$ is defined by

$$v^*(s) := \sup_{\pi \in \Pi^{\mathrm{HR}}} v^{\pi}(s). \tag{4.11}$$

for all $s \in S$. Similarly, the optimal value from decision epoch n to the end of the planning horizon is given by

$$v^*_n(s) := \sup_{\pi \in \Pi^{\mathrm{HR}}} v^{\pi}_n(s). \tag{4.12}$$

As a consequence of (4.10) and (4.11), when an optimal policy π^* exists, its value satisfies

$$v^{\pi^*}(s) = v^*(s) \tag{4.13}$$

for all $s \in S$. Exercise 5 provides a simple example (with a countable action set) in which an optimal policy need not exist. In cases where there is no optimal policy, focus shifts to finding a policy that is suboptimal by an amount no greater than ϵ.

Definition 4.5. Suppose $\epsilon > 0$. Then a policy $\pi^\epsilon \in \Pi^{\text{HR}}$ is an *ϵ-optimal policy* if

$$v^{\pi^\epsilon}(s) \geq v^*(s) - \epsilon \tag{4.14}$$

for all $s \in S$.

Of course $v^*(s) \geq v^{\pi^\epsilon}(s)$ for all $s \in S$. Exercise 6 asks you to establish that for any Markov decision process, there always exists an ϵ-optimal policy.

The fundamental result

The following theorem states that under mild assumptions, Markovian deterministic policies are optimal within the broader class of history-dependent randomized policies. This means that the search for an optimal policy can be limited to the class of Markovian deterministic policies.

Theorem 4.2. Suppose A_s is finite for each $s \in S$. Then there exists $\pi^* \in \Pi^{\text{MD}}$ that achieves the optimal value $v^*(s)$ for all $s \in S$. That is,

$$v^*(s) = v^{\pi^*}(s) = \max_{\pi \in \Pi^{\text{MD}}} v^\pi(s). \tag{4.15}$$

Moreover, the same policy π^* achieves the optimal value starting at any epoch $n = 1, \ldots, N$

$$v_n^*(s) = v_n^{\pi^*}(s) = \max_{\pi \in \Pi^{\text{MD}}} v_n^\pi(s) \tag{4.16}$$

This is a fundamental result within Markov decision processes theory that is often overlooked. A constructive proof is given below. Specifically, an algorithm is presented that generates value functions, which are subsequently proven to be optimal within the class of Markovian deterministic policies. Following this, it is shown that Markovian deterministic policies are optimal within the class of history-dependent randomized policies. Both of these proofs are provided in the chapter appendix.

Note that not only does Theorem 4.2 speak to an optimal policy and optimal value starting from the first epoch, it establishes that the same policy is optimal from any decision epoch n to the end of the planning horizon. Richard Bellman coined the expression *The Principle of Optimality*[4], to describe this property of optimal policies. He wrote (Bellman, 1957):

[4]This result is true when maximizing the expected total reward but may not hold under other optimality criteria. See pp. 97-98 in Puterman (1994).

> "An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision."

To understand why this must be true, consider driving from location A to Z along the shortest route, which involves visiting locations B, C, D, ..., Y on the way. Then it must be the case that the shortest route from B to Z visits C, D, ..., Y as well in the same order. The reason is that if there is a different route from B to Z that is shorter, then it could be used when driving from A to Z, contradicting the fact that the initial route from A to Z was shortest.

4.2.2 Computing optimal value functions and finding optimal policies

This section shows how to use backwards induction to compute $v^*(s)$ and use it to find an optimal policy, π^* when one exists. As a consequence of Theorem 4.2, the exposition below assumes policies are Markovian and deterministic.

In the spirit of the one-period model of Section 2.4, set $v^*_N(s) = r_N(s)$ and define $v^*_{N-1}(s)$ for all $s \in S$ by

$$v^*_{N-1}(s) := \max_{a \in A_s} \left\{ \sum_{j \in S} p_{N-1}(j|s,a)\big(r_{N-1}(s,a,j) + v^*_N(j)\big) \right\}. \tag{4.17}$$

The quantity $v^*_{N-1}(s)$ represents the maximum expected total reward starting in state s in a one-period model with immediate reward $r_{N-1}(s,a,\cdot)$ and terminal reward $v^*_N(\cdot) = r^*_N(\cdot)$.

Define $A^*_{N-1,s}$ to be the set of actions at decision epoch $N-1$ that achieve the maximum in equation (4.17):

$$A^*_{N-1,s} := \arg\max_{a \in A_s} \left\{ \sum_{j \in S} p_{N-1}(j|s,a)\big(r_{N-1}(s,a,j) + v^*_N(j)\big) \right\}. \tag{4.18}$$

Now construct a decision rule $d^*_{N-1}(s)$ by setting it equal to some $a^*_{N-1} \in A^*_{N-1,s}$ for each $s \in S$. Since the maximum need not be unique, selecting any element of $A^*_{N-1,s}$ suffices. By construction, $v^{d^*_{N-1}}(s) = v^*_{N-1}(s)$, so it is an optimal decision rule at decision epoch $N-1$ in the above one-period problem.

Next, define $v^*_{N-2}(s)$ for all $s \in S$ by

$$v^*_{N-2}(s) := \max_{a \in A_s} \left\{ \sum_{j \in S} p_{N-2}(j|s,a)\big(r_{N-2}(s,a,j) + v^*_{N-1}(j)\big) \right\}. \tag{4.19}$$

By the same argument as above, $v^*_{N-2}(s)$ is the maximum expected total reward achievable starting in state s in a one-period model with immediate reward $r_{N-2}(s,a,\cdot)$ and

terminal reward $v^*_{N-1}(\cdot)$. Considering the interpretation of $v^*_{N-1}(s)$, $v^*_{N-2}(s)$ represents as the maximum expected total reward in a two-period problem with immediate rewards $r_{N-2}(s,a,\cdot)$ and $r_{N-1}(s,a,\cdot)$, and terminal reward $r_N(\cdot)$[5]. Repeating this argument leads to a recursive algorithm for computing $v^*(s)$ for all $\in S$.

Continuing in this way, define

$$v^*_n(s) := \max_{a\in A_s}\left\{\sum_{j\in S} p_n(j|s,a)\big(r_n(s,a,j)+v^*_{n+1}(j)\big)\right\}. \tag{4.20}$$

Let $A^*_{n,s}$ be the set of optimal actions at epoch n, that is, those that achieve the maximum in equation (4.20):

$$A^*_{n,s} = \arg\max_{a\in A_s}\left\{\sum_{j\in S} p_n(j|s,a)\big(r_n(s,a,j)+v^*_{n+1}(j)\big)\right\}. \tag{4.21}$$

Letting $d^*_n(s)$ be some $a^*_n \in A^*_{n,s}$ for each s results in an optimal decision rule at epoch n. By choosing $d^*_n(s)$ to be an element of $A^*_{n,s}$ for $n = 1, 2, \ldots, N-1$ and $s \in S$, then $\pi^* = (d^*_1, d^*_2, \ldots, d^*_{N-1})$ is an optimal policy. In other words, $v^{\pi^*}(s) = v^*(s)$.

Backwards induction and the Bellman equation

The following algorithm computes an optimal value function and finds an optimal policy. This algorithm is often referred to as *Backwards Induction* or *Dynamic Programming*. Expression (4.22) is the fundamental relationship in finite horizon models. It is often called the *Bellman Equation*, after Richard Bellman, who is regarded as the founder of dynamic programming. Others refer to it simply as the *optimality equation*[6].

Algorithm 4.3. Finite horizon policy optimization

1. **Initialize:** Set $n \leftarrow N-1$ and $u_N(s) \leftarrow r_N(s)$ for all $s \in S$.
2. **Iterate:** While $n \geq 1$:
   (a) For all $s \in S$, evaluate $u_n(s)$ according to

$$u_n(s) \leftarrow \max_{a\in A_s}\left\{\sum_{j\in S} p_n(j|s,a)\big(r_n(s,a,j)+u_{n+1}(j)\big)\right\}, \tag{4.22}$$

[5] This is the Principle of Optimality at work.

[6] Note that in a finite horizon model, the singular expression Bellman equation corresponds to an expression which holds for each n and all $s \in S$.

set

$$A^*_{n,s} \leftarrow \arg\max_{a \in A_s} \left\{ \sum_{j \in S} p_n(j|s,a)\big(r_n(s,a,j) + u_{n+1}(j)\big) \right\} \tag{4.23}$$

and select $d_n(s) \in A^*_{n,s}$.

(b) $n \leftarrow n-1$.

3. **Terminate:** Return $u_1(s)$ for all $s \in S$ and $\pi = (d_1, \ldots, d_{N-1})$.

Some comments about Algorithm 4.3 follow:

1. Theorem 4.3 below establishes that this algorithm determines that for all n, $v^*_n(s)$, the optimal expected total reward over decision epochs $n, n+1, \ldots, N-1$ starting in state s at decision epoch n, and $v^*(s)$, the optimal value function, as defined by (4.11) for all $s \in S$.

2. The sets $A^*_{n,s}$ may contain one or more elements. Finding this requires no additional computation. It is obtained when evaluating the maximum in (4.22) and storing those actions that achieve the maximum.

3. Any policy that chooses an action in $A^*_{n,s}$ for each $s \in S$ and $n = 1, \ldots, N-1$ is optimal. If this set contain a single element for all s and n, there is a unique optimal policy. However, when there are multiple optimal policies, one might be preferred to another because it has a more interpretable structure[7] or has smaller variance[8].

4. As in the case of the finite horizon policy evaluation algorithm, the finite horizon policy optimization algorithm decomposes the multi-period optimization problem into a sequence of one-period optimization problems. This approach avoids enumerating all Markovian deterministic policies and their corresponding sample paths. If there are M states and K actions in each, the algorithm requires $(N-1)KM^2$ multiplications. When only a single optimal policy is required, one needs only store the policy elements obtained at each iteration and $v^*_n(s)$.

5. Theorem 4.2 is applicable for both finite and countable state spaces. Accordingly, Algorithm 4.3 is applicable to both cases. However, one usually cannot calculate $u_n(s)$ for every s when S is countably infinite. Practical options are to truncate the state space at a large state value (and modify the transition probabilities accordingly) or to establish some structure of the value function or optimal policy that makes it easy to identify an optimal action for all states above a certain

[7] Section 4.4 provides a systematic approach to identifying structured optimal policies.

[8] Recall, that $v^*_n(s)$ is an expected value. You may think of its variance as the variance of this value over multiple simulations or realizations.

threshold. The queuing service rate control application illustrates both options in Sections 4.3.1 and 4.4.3. Another option is to approximate $v_n^*(s)$ by a parametric function of s as in Chapter 9 but this does not guarantee finding an optimal policy.

The following theorem states that Algorithm 4.3 is guaranteed to find an optimal policy that is Markovian and deterministic, as well as the corresponding optimal values. The lengthy but insightful development needed to formally prove this result appears in the Appendix to this chapter. Combining Theorem 4.3 with Theorem 4.2 leads to the conclusion that that Algorithm 4.3 finds an optimal policy within the class of history-dependent randomized policies.

Theorem 4.3. Let A_s be finite for all $s \in S$. Suppose $u_n(s)$ is computed using Algorithm 4.3, $v^*(s)$ is defined as in (4.15), and $v_n^*(s)$ is defined as in (4.16).

1. For $n = 1, 2, \ldots, N$ and $s \in S$, $u_n(s) = v_n^*(s)$. In particular, $u_1(s) = v^*(s)$.
2. Suppose for $n = 1, 2, \ldots, N$ and $s \in S$ that

$$\sum_{j \in S} p_n(j|s, d_n(s)) \big(r_n(s, d_n(s), j) + u_{n+1}(j)\big) = \max_{a \in A_s} \left\{ \sum_{j \in S} p_n(j|s, a) \big(r_n(s, a, j) + u_{n+1}(j)\big) \right\}. \quad (4.24)$$

Then $\pi^* = (d_1, d_2, \ldots, d_{N-1})$ is an optimal policy.

Returning to the discussion surrounding the Principle of Optimality, given Theorem 4.2, it follows that Algorithm 4.3 provides a method to construct optimal policies and determine optimal values from any decision epoch n onward.

When do optimal policies exist?

The astute reader will recognize that the assumption of finite action sets ensures that the maximum is attained in equation (4.22). Another condition that ensures attainment of the maximum is that A_s is a compact set for all $s \in S$ and $r_n(s, a, j)$ and $p_n(j|s, a)$ are *continuous* functions of a for all $s \in S$ and $j \in S$. However, optimal policies need not exist when A_s is countable or an open set. More formally, the attainment of the "max" in equation (4.22) is sufficient for the existence of an optimal policy, regardless of whether the state space is finite, countable or continuous.

Corollary 4.1. Suppose the maximum is attained in (4.22) for $n = 1, 2, \ldots, N-1$ and $v_N^*(s) = r_N(s)$ for all $s \in S$ and $d_n^*(s) \in A_{n,s}^*$ for $n = 1, 2, \ldots, N-1$. Then the policy $\pi^* = (d_1^*, d_2^*, \ldots, d_{N-1}^*) \in \Pi^{\text{MD}}$ is optimal.

4.2.3 A numerical example

The calculations below use Algorithm 4.3 to find an optimal policy and the corresponding optimal value in the two-state model from Section 2.5.

Example 4.1.

Let $N = 3$.
1. Set $n = 3$. Since $r_3(s) = 0$ for $s = s_1$ and $s = s_2$, set $u_3(s_1) = u_3(s_2) = 0$.
2. Set $n = 2$ and

$$u_2(s_1) = \max\{0.8(5) + 0.2(-5) + 0,\ 5 + 0\} = 5$$

and

$$u_2(s_2) = \max\{-5 + 0,\ 0.4(20) + 0.6(-10) + 0\} = 2$$

Consequently $A_{2,s_1}^* = \{a_{1,2}\}$ and $A_{2,s_2}^* = \{a_{2,2}\}$.

3. Set $n = 1$ and

$$u_1(s_1) = \max\{5 + 2,\ 0.8(5 + 5) + 0.2(-5 + 2)\} = 7.4$$

and

$$u_1(s_2) = \max\{-5 + 2,\ 0.4(20 + 5) + 0.6(-10 + 2)\} = 5.2$$

Consequently $A_{1,s_1}^* = \{a_{1,2}\}$ and $A_{1,s_2}^* = \{a_{2,2}\}$.
4. Since $n = 1$, stop.

These calculations show that $v^*(s_1) = 7.4$ and $v^*(s_2) = 5.2$, and the unique optimal policy is $\pi^* = (d_1^*, d_2^*)$ where $d_1^*(s_1) = d_2^*(s_1) = a_{1,2}$ and $d_1^*(s_2) = d_2^*(s_2) = a_{2,2}$. This policy is unique because the optimal action sets contain a single element at each decision epoch and state.

4.3 Applications

This section finds optimal policies in three of the applications from Chapter 3. In the first two applications, Algorithm 4.3 is used to numerically find optimal policies. An optimal policy is derived analytically in the third application.

Note that in the first two applications optimal policies are computed over a range of parameter values. The reason for doing this is to examine how the structure of the optimal policy varies as the model parameters change. We recommend that you carry out such *sensitivity analyses* in all applications.

4.3.1 Queuing service rate control

Consider a finite horizon variant of the service rate control problem described in Section 3.4.1. The calculations below examine how the optimal policy changes under two cost regimes (linear delay cost $f(s) = s$ and linear service cost $m(a) = 5a$; linear delay cost $f(s) = s$ and cubic service cost $m(a) = 5a^3$) and two horizon lengths ($N = 10$ and $N = 50$). Assume a terminal cost $r_N(s) = 0$ for all s, arrival probability $b = 0.1$ and three service probabilities $a_1 = 0.2$, $a_2 = 0.4$ and $a_3 = 0.6$.

Truncation

Calculations below use truncation to provide numerically tractable instances. To determine an appropriate truncation level W, steady state results for an $M/M/1$ queuing system[9] are used as follows.

Setting[10] the service rate to a and arrival rate to b ($b < a$), the mean queue length $\mu = b/(a-b)$ and the variance in queue length $\sigma^2 = ba/(a-b)^2$. Thus setting the truncation level

$$W = \mu + 4\sigma = \frac{b}{a_1 - b} + 4\frac{\sqrt{ba_1}}{a_1 - b} \tag{4.25}$$

ensures that W is reached with low probability even under the lowest service probability. With the above parameter values, $\mu = 1$ and $\sigma = 1.4$ so that equation (4.25) implies that $W = 6$. Note that if $b = 0.199$ instead, then $\mu = 199$ and $W = 555$.

The Bellman equation

Since $m(a) + f(s)$ are costs it is more convenient to regard this as a cost minimization problem. To do so replaces the "max" in the Bellman equation by a "min" as follows:

$$u_n(s) = \min_{k \in \{1,2,3\}} \left\{ m(a_k) + f(s) + \sum_{j \in S} p(j|s, a_k) u_{n+1}(j) \right\} \tag{4.26}$$

for $n = 1, \ldots, N-1$. Writing this out explicitly for each state gives

$$u_n(0) = \min_{k \in \{1,2,3\}} \{ m(a_k) + f(0) + (1-b)u_{n+1}(0) + bu_{n+1}(1) \}$$

[9] An $M/M/1$ queuing system is a continuous time model for a single server queuing system with exponential inter-arrival times and service times.

[10] Using a and b as service and arrival rates, respectively, instead of probabilities is a reasonable approximation to the corresponding continuous time parameters for the chosen parameter values.

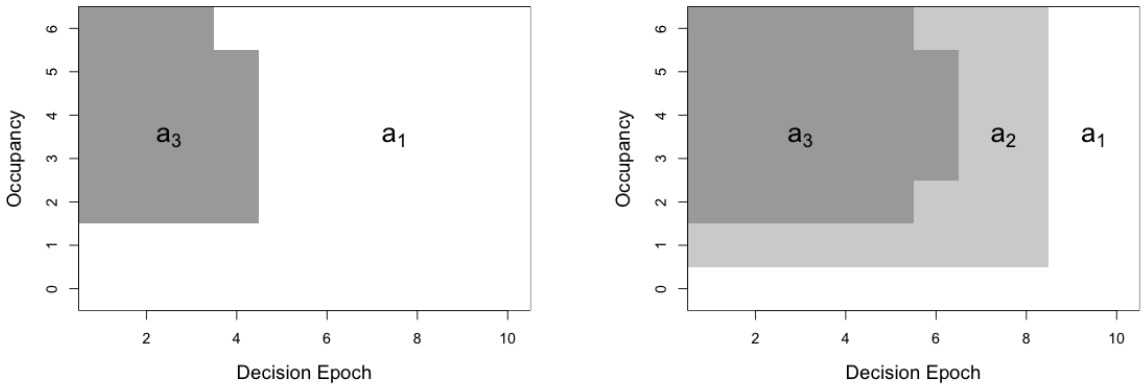
$$u_n(s) = \min_{k\in\{1,2,3\}} \{m(a_k) + f(s) + a_k u_{n+1}(s-1) + (1-a_k-b)u_{n+1}(s) + bu_{n+1}(s+1)\}, \quad s = 1,\dots,W-1$$
$$u_n(W) = \min_{k\in\{1,2,3\}} \{m(a_k) + f(W) + a_k u_{n+1}(W-1) + (1-a_k)u_{n+1}(W)\}.$$

By assumption, $u_N(s) = r_N(s) = 0$ for all $s \in S$.

Note that it follows from the Bellman equation that in state 0 that the optimal service rate is always a_1 because $m(a)$ is increasing in a. This result is observed in all calculations.

Linear delay cost, linear service cost, $N = 10$

Figure 4.1a shows the optimal policy in each decision epoch for this instance.



(a) Linear delay cost and linear service cost (b) Linear delay cost and cubic service cost

Figure 4.1: Optimal policy for queuing service rate control problem under two cost regimes and for $N = 10$. Occupancy refers to the state.

Observe the following structural properties of the optimal policy:

1. It is never optimal to use a_2.
2. The optimal policy at decision epochs 5 to 10 uses the least costly service rate a_1. This is because $r_{10}(s) = 0$, so there is no penalty for having the system occupied at termination.
3. In each state, the optimal service level decreases with respect to decision epoch. This again is a consequence of the end of planning horizon cost function.
4. The optimal action in state 0 is a_1 as noted above.

5. At all decision epochs except $n = 4$, the optimal policy is non-decreasing in the occupancy level. This reflects the reasonable assumption that the busier the system is, the faster the server should work. Theoretical justification for this observation is provided in Section 4.4.3 below.

6. The non-monotone decision rule at decision epoch 4 is an anomaly. It is a consequence of truncation of the state space at $s = 6$. The cost of using action a_1 (45.33) was close to that of using a_3 (45.35) so that if instead, the monotone policy was used at decision epoch 4, there would be a small increase in expected cost. More importantly when the truncation level $W > 6$, the optimal action in state 6 was a_3.

Linear delay cost, cubic service cost, $N = 10$

Figure 4.1b shows the optimal policy in each decision epoch for this instance. It shares the following properties with the linear service cost instance, namely:

1. It uses the least costly service rate in latter epochs.

2. It uses a_1 in state 0.

3. The service rate is non-increasing as a function of decision epoch.

4. The service rate is non-decreasing as a function of occupancy level (except at epoch 6).

5. The decision rule is a non-monotonic function of the occupancy at decision epoch 6.

Moreover the rationale for each are the same. However, the key difference with the optimal policy in the linear service cost case is that it uses a_2 in some states and decision epochs.

Linear delay cost, cubic service cost, $N = 50$

Figure 4.2 shows the optimal policy for this instance. The most important point to note is that the optimal policy remains the same up to decision epoch 46 at which point the terminal reward impacts actions choice, eventually using the slowest service rate.

Such behavior is described by what are know as *turnpike*[11] theorems.

[11] Turnpike theory is discussed in length in Section 6.8 in Puterman [1994]. The basic idea is starting from the end of the planning horizon, there exists a decision epoch after which the set of optimal policies remains the same.

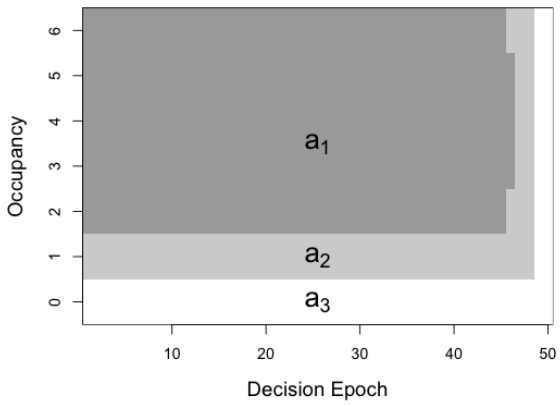


Figure 4.2: Optimal policy for queuing service rate control problem with linear delay cost, cubic service cost, and for $N = 50$. Occupancy refers to the state.

Conclusions

The figures above illustrate the optimal policies for each of three instances. The discussions that follow provide examples of the types of conclusions that can be drawn regarding the structure of an optimal policy. We encourage you try to draw similar conclusions when solving your own problems.

For these examples, truncation was applied to obtain numerical solutions. However this resulted in some unexpected "edge cases". To avoid such anomalies, when seeking solutions on a state space S, it is advisable to solve the problem on a larger state space $S' \supset S$ and then restrict the resulting policy to S.

It is also noteworthy that although there were three actions available, the optimal policy in the linear service cost case only used the two extreme actions a_1 and a_3. Such policies that abruptly switch between extreme actions are often referred to as *bang-bang*[12] *policies*. One can prove that when costs are linear only extreme actions are optimal.

4.3.2 Revenue management

This section computes optimal policies in an example that is naturally represented by a finite horizon model: the revenue management problem in Section 3.3.

Assume the selling season is $N = 6$ months and the retailer has $M = 15$ units of inventory to sell. The candidate prices are $A_s = \{20, 23, 25, 27, 30, 35\}$ for $s \in \{1, \ldots, M\}$ and there are two possible scrap values, $H = 0$ or $H = 5$, allowing investigation of the

[12]This refers to a policy that switches from the "smallest" action to the "largest" or vice versa, assuming the actions are ordered. When there are only two actions, this is equivalent to a control-limit or threshold policy.

sensitivity of the optimal policy to the scrap value. The holding cost is calculated based on the end of month inventory item and is set to 2 per item so that $h(j) = 2j$. Monthly demand is Poisson with rate $\lambda_{n,a} = (1.1 - 0.1n)(9 - 0.25a)$, which is linearly decreasing in both a and n. As a consequence, $0.25 \leq \lambda_{n,a} \leq 4$ for $n = 1$ and $0.15 \leq \lambda_{n,a} \leq 2.4$ for $n = 5$.

Bellman equation

The Bellman equations for this model are $u_N(s) = Hs$ for $s = 0, \ldots, M$,

$$u_n(s) = \max_{a \in A_s} \left\{ \sum_{j=0}^{s-1} \left(\frac{e^{-\lambda_{n,a}} \lambda_{n,a}^j}{j!} \big(aj - h(s-j) + u_{n+1}(s-j)\big) \right) + \left(\sum_{j=s}^{\infty} \frac{e^{-\lambda_{n,a}} \lambda_{n,a}^j}{j!} \right) \big(as + u_{n+1}(0)\big) \right\}$$

for $s = 1, \ldots, M$ and $u_n(0) = 0$ for $n = 1, \ldots, N-1$. Since $u_n(0) = 0$, the above expression simplifies to

$$u_n(s) = \max_{a \in A_s} \left\{ \sum_{j=0}^{s-1} \left(\frac{e^{-\lambda_{n,a}} \lambda_{n,a}^j}{j!} \big(aj - h(s-j) + u_{n+1}(s-j)\big) \right) + as \sum_{j=s}^{\infty} \frac{e^{-\lambda_{n,a}} \lambda_{n,a}^j}{j!} \right\}.$$

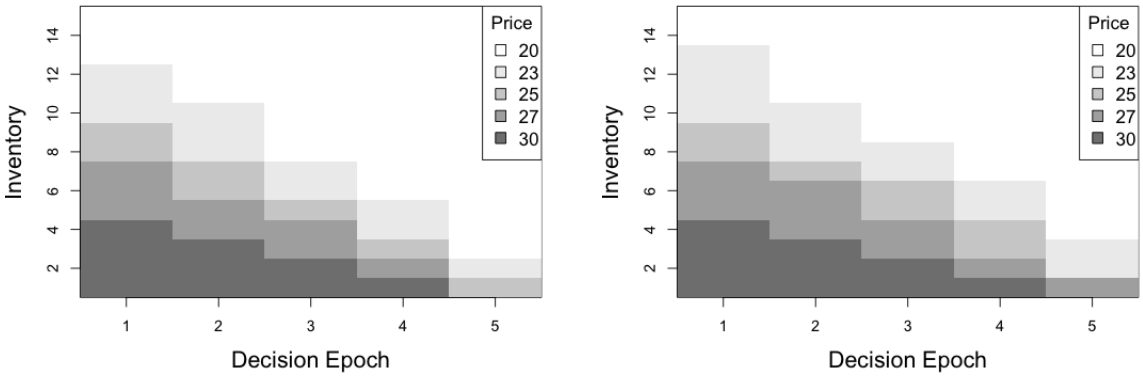
Note the first summation in the "max" corresponds to the possibility that demand j falls below the inventory level s, so the remaining inventory is $s - j$, while the second term corresponds to the possibility that demand equals or exceeds the inventory level. In the latter case, the number of items sold at price a is the number of items in inventory. Consequently the inventory at the start of the next decision epoch is zero.

Results

The following is the result of applying Algorithm 4.3 to this example. Figure 4.3 depicts optimal policies for the two scrap values. For example at decision epoch 2, with $H = 0$, the optimal prices are given by

$$d_2^*(s) = \begin{cases} 30 & s = 1, 2, 3 \\ 27 & s = 4, 5 \\ 25 & s = 6, 7 \\ 23 & s = 8, 9, 10 \\ 20 & s = 11, \ldots, 15. \end{cases}$$

Note that for both scrap values, it is never optimal to set the price equal to 35. This is because of the low mean demand at this price which varies between 0.25 and 0.15.



(a) Scrap value $H = 0$ (b) Scrap value $H = 5$

Figure 4.3: Optimal policy for revenue management example. Grey tones represent the different optimal prices.

Observe for each scrap value, the optimal policies have the same general shape. At each decision epoch, optimal prices are non-increasing in the inventory level. This means that in each decision epoch, it is optimal for the retailer to retain the current price or set a lower price when the inventory level is higher. Conversely, as the inventory level decreases it is optimal for the price to increase or remain the same. For a fixed inventory level, the price is non-increasing over time. This means that as the end of the season nears it is optimal to markdown the price or keep it the same.

With a higher scrap value $H = 5$, the optimal decision rule will set prices to be the same or higher at each inventory level and decision epoch compared to when with $H = 0$. For example, with $H = 5$, $d_5^*(3) = 23$ and $d_5^*(1) = 27$ while when $H = 0$, $d_5^*(3) = 20$ and $d_5^*(1) = 25$. The reason for this is that when the scrap value is higher, the retailer can take on some of the risk of lower demand at higher prices.

The results of these calculations agree with what is observed in practice: prices for fashion goods decline over time, but do so more slowly when fewer items are available.

Fixed pricing

One might ask, "What is the benefit of using dynamic pricing?" This can be investigated by using the policy evaluation equations with the policy set equal to a fixed price. At price $\bar{a}$ they are given by $u_N(s) = Hs$,

$$u_n(s) = \sum_{j=0}^{s-1} \left(\frac{e^{-\lambda_{n,\bar{a}}} \lambda_{n,\bar{a}}^j}{j!} \left(\bar{a}j - h(s-j) + u_{n+1}(s-j) \right) \right) + \bar{a}s \sum_{j=s}^{\infty} \frac{e^{-\lambda_{n,\bar{a}}} \lambda_{n,\bar{a}}^j}{j!}$$

for $n = 1, \ldots, N-1$ and $s = 1, \ldots, M$ and $u_n(0) = 0$ for $n = 1, \ldots, N$. Note that this expression is equivalent to choosing $A_s = \{\bar{a}\}$ for $s = 1, \ldots, M$ so that the same

code used to maximize the expected total reward can be used by simply modifying A_s accordingly.

Table 4.2 compares the expected total reward over 5 periods using a fixed price to that using the optimal policy. Observe that the optimal policy outperforms all specified fixed price policies for both salvage values. Moreover observe that the expected total reward is negative when the price is set to 35. This is a result of both the low demand and holding costs of 2 dollars per period.

Note the expected total revenue is slightly higher than that obtained using the fixed price of 20. This is especially consequential in retail inventory management where profit margins are low and a large number of products are sold.

Fixed price ($\bar{a}$)	$H = 0$	$H = 5$
18	215.58	218.35
20	225.16	230.77
23	216.05	229.43
25	190.52	211.60
30	68.97	113.98
35	−108.50	−38.50
Optimal policy	230.65	237.55

Table 4.2: Comparison of the expected total reward obtained using a range of fixed prices denoted by $\bar{a}$ with that of the optimal dynamic policy in the revenue management example with salvage values $H = 0$ and $H = 5$ and a starting inventory of 15 units.

4.3.3 Online dating: Finding the best match

This section uses the Bellman equations to derive the optimal policy *analytically*, instead of numerically, in the online dating[13] problem formulated in Section 3.8.2. As noted in the formulation, the objective in this model is to find a policy that maximizes the *probability* of obtaining the best match.

In this model, the state space is $S = \{0, 1, \Delta\}$, where 1 indicates the current profile is the best encountered so far, 0 indicates that it is not, and Δ denotes the stopped state. When $s = 0$ or $s = 1$, $A_s = \{a_0, a_1\}$ where a_0 represents not choosing the current match and a_1 denotes choosing the current match. Also $A_\Delta = \{a_0\}$, meaning that once the stopped state is entered there are no further decisions to make[14].

The Bellman equations follow. One can interpret the quantity $u_n(s)$ as the probability of obtaining the best match given that the decision process is in state s at decision epoch n. Clearly $u_N(1) = 1$ and $u_N(0) = u_N(\Delta) = 0$. For $n < N$,

$$u_n(0) = \max\left\{u_{n+1}(\Delta),\ \frac{1}{n+1}u_{n+1}(1) + \frac{n}{n+1}u_{n+1}(0)\right\}, \tag{4.27}$$

[13] As previously noted, this example is typically referred to as the *secretary problem*.

[14] The reward is received only at epoch when the decision to stop is implemented.

$$u_n(1) = \max\left\{\frac{n}{N} + u_{n+1}(\Delta),\ \frac{1}{n+1}u_{n+1}(1) + \frac{n}{n+1}u_{n+1}(0)\right\} \tag{4.28}$$

and $u_n(\Delta) = u_{n+1}(\Delta)$.

Since $u_N(\Delta) = 0$ and Δ is an absorbing state[15], $u_n(\Delta) = 0$ for all $n < N$. Noting that $u_n(s) \geq 0$ for $s = 0$ and 1, or arguing that it is never optimal to stop if the current match is not the best so far, (4.27) and (4.28) reduce to:

$$u_n(0) = \frac{1}{n+1}u_{n+1}(1) + \frac{n}{n+1}u_{n+1}(0) \tag{4.29}$$

and

$$u_n(1) = \max\left\{\frac{n}{N}, u_n(0)\right\}. \tag{4.30}$$

Of course one could have derived these equations directly.

Thus in state 0, one should pass on the profile and in state 1, an optimal decision rule (or action a_n^*) is given by

$$d_n^*(1) = a_n^* = \begin{cases} \text{stop} & \text{if } u_n(0) \leq n/N \\ \text{continue} & \text{if } u_n(0) > n/N. \end{cases} \tag{4.31}$$

The above expression assumes the convention that it is optimal to accept the current profile when there is a tie on the right hand side of (4.30). Equation (4.31) provides a specific structure for an optimal action as a function of the optimal value and the epoch. However, to implement it requires knowing $u_n(0)$.

An example

The following example with four potential matches ($N = 4$), illustrates the computation of an optimal policy using equations (4.29) and (4.30).

1. Initialize calculations by setting $n = 4$ and $u_4(0) = 0$ and $u_4(1) = 1$. This makes sense since if the last profile is the best so far, it is the best overall and if not, one cannot obtain the best match.

2. Set $n = 3$,

$$u_3(0) = \frac{1}{4}u_4(1) + \frac{3}{4}u_4(0) = \frac{1}{4} \tag{4.32}$$

and

$$u_3(1) = \max\left\{\frac{3}{4}, u_3(0)\right\} = \frac{3}{4}. \tag{4.33}$$

[15] See Appendix B for the definition of this concept.

Hence the set of optimal actions, $A^*_{3,1} = a_1$, and it is optimal to stop at decision epoch 3 if the profile is the best so far. Moreover the probability of getting the best match if in state 0 is 1/4 (which occurs only if the last profile is the best) and 3/4 by stopping in state 1 since the likelihood the last profile is the best is $1 - 3/4$.

3. Set $n = 2$ and

$$u_2(0) = \frac{1}{3}u_3(1) + \frac{2}{3}u_3(0) = \frac{1}{3} \cdot \frac{3}{4} + \frac{2}{3} \cdot \frac{1}{4} = \frac{5}{12} \tag{4.34}$$

and

$$u_2(1) = \max\left\{\frac{2}{4}, u_2(0)\right\} = \frac{2}{4} \tag{4.35}$$

Hence $A^*_{2,1} = a_1$ and it is again optimal to stop at decision epoch 2 if the profile is the best so far. This makes sense since if the state is 1 and one continues, the probability of getting the best match is 5/12 while if one stops, it is 1/2.

4. Set $n = 1$ and

$$u_1(0) = \frac{1}{2}u_2(1) + \frac{1}{2}u_2(0) = \frac{1}{2} \cdot \frac{2}{4} + \frac{1}{2} \cdot \frac{5}{12} = \frac{11}{24} \tag{4.36}$$

and

$$u_2(1) = \max\left\{\frac{1}{4}, u_2(0)\right\} = \frac{11}{24} \tag{4.37}$$

Hence $A^*_{1,1} = a_0$ and it is optimal to continue at decision epoch 1. Again this makes sense since by default the first match is the best so far and the probability it is the best match overall is 1/4 and the probability of achieving the best match by continuing is 11/24.

Thus the optimal policy for this example is

$$d^*_n(0) = a_0 \text{ for } n = 1, 2, 3 \quad \text{and} \quad d^*_n(1) = \begin{cases} a_0 \text{ for } n = 1 \\ a_1 \text{ for } n = 2, 3. \end{cases}$$

It can be interpreted as continue at the first decision epoch and then stop whenever a subsequent match is the best observed so far. The probability that this strategy finds the best candidate is $11/24 = 0.458$.

The optimal policy

Repeating the above calculations for larger values of N reveals that the optimal policy rejects any profile for an initial period of length M after which it selects the first profile that is better than all of those observed so far. Of course, if the online dater does not choose a profile before the last one, then the last one must be chosen.

The following theorem demonstrates that, indeed, the optimal policy has this structure.

Theorem 4.4. Suppose $N > 2$. Then there exists a decision epoch $M \in \{1, \dots N-1\}$ for which the optimal policy $\pi^* = (d_1^*, \dots, d_{N-1}^*)$ satisfies:

$$d_n^*(0) = a_0 \text{ for } 1 \le n \le N-1 \quad \text{and} \quad d_n^*(1) = \begin{cases} a_0 \text{ for } 1 \le n \le M \\ a_1 \text{ for } M+1 \le n \le N-1. \end{cases} \tag{4.38}$$

Proof. First, the proof shows that if it is optimal to continue when $s = 1$ at decision epoch n' it is optimal to continue when $s = 1$ at all earlier decision epochs $n < n'$.

Assuming it is optimal to continue at n', it follows from (4.30) that $u_{n'}(1) = u_{n'}(0)$ so that

$$u_{n'-1}(0) = \frac{1}{n'} u_{n'}(1) + \frac{n'-1}{n'} u_{n'}(0) = u_{n'}(0) > \frac{n'}{N} > \frac{n'-1}{N}. \tag{4.39}$$

So by (4.31), it is optimal to continue when $s = 1$ at decision epoch $n' - 1$ and $u_{n'-1}(1) = u_{n'-1}(0)$. Repeating this argument shows that the result holds for all $n < n'$ and

$$u_1(1) = u_1(0) = u_2(1) = u_2(0) = \cdots = u_{n'}(1) = u_{n'}(0). \tag{4.40}$$

This establishes that the optimal policy cannot switch from continue to stop and then back to continue as a function of the decision epoch index.

The proof is completed by showing that it is not optimal to always stop, or equivalently for some n, $u_n(0) > n/N$. Suppose $N > 2$. Now assume to the contrary that it is always optimal to stop. Then for all n, $u_n(1) = n/N$ and

$$u_n(0) = \frac{1}{n+1}\frac{n+1}{N} + \frac{n}{n+1} u_{n+1}(0) = \frac{1}{N} + \frac{n}{n+1} u_{n+1}(0). \tag{4.41}$$

Noting $u_N(0) = 0$ and applying (4.41) recursively shows that $u_{N-1}(0) = 1/N$,

$$u_{N-2}(0) = \frac{N-2}{N}\left(\frac{1}{N-2} + \frac{1}{N-1}\right) \tag{4.42}$$

and in general under this assumption

$$u_n(0) = \frac{n}{N}\left(\frac{1}{n} + \frac{1}{n+1} + \ldots + \frac{1}{N-1}\right). \tag{4.43}$$

Hence it is easy to see $u_1(0) > 1/N$, which from (4.31) implies that the optimal action at decision epoch 1 is to continue, contradicting the assumption that it is always optimal to stop. Thus $N > 1$. Since one must stop when $n = N$, $M < N$. □

Some comments regarding the conclusions and hypothesis of the theorem follow:

1. It states that the optimal policy is to choose action a_0 (continue) for $n \leq M$ and then choose action a_1 (stop) at the first epoch $n > M$ for which the current match is better than all previously observed matches.

2. Using this policy it is possible that one might not obtain the best match. This occurs if the best profile is among the first M.

3. The theorem excludes the case $N = 2$, since the probability of getting the best match is $1/2$ regardless of whether the online dater selects the first match or waits until the second since the two rankings $(1, 2)$ and $(2, 1)$ are equally likely.

An approximation to M

The second part of the proof above suggests a convenient way to find an optimal policy. It implies that it is optimal to stop at epoch n if $u_n(0) \geq u_n(1)$, or equivalently if

$$\frac{1}{n} + \frac{1}{n+1} + \cdots + \frac{1}{N-1} \leq 1.$$

This yields the following important characterization of M:

$$M = \max\left\{ n \geq 1 \;\middle|\; \frac{1}{n} + \frac{1}{n+1} + \ldots + \frac{1}{N-1} > 1 \right\}. \tag{4.44}$$

Note that is consistent with the numerical results above since (4.44) returns $M = 1$ when $N = 4$. This means that it is optimal to pass over the first profile and then choose any profile that is better than all of those previously seen. Note that if $N = 5$, $M = 2$.

Equation (4.44) leads to the following elegant rule of thumb for choosing M when N is large. In this case the summation in (4.44) can be approximated by an integral as follows:

$$\frac{1}{M} + \frac{1}{M+1} + \ldots + \frac{1}{N-1} \approx \int_M^N \frac{1}{x} dx = \ln \frac{N}{M}. \tag{4.45}$$

Thus, it is optimal to stop if

$$\ln \frac{N}{M} \approx 1 \quad \text{or equivalently,} \quad M \approx Ne^{-1} = 0.368N. \tag{4.46}$$

This result can be summarized succinctly as follows.

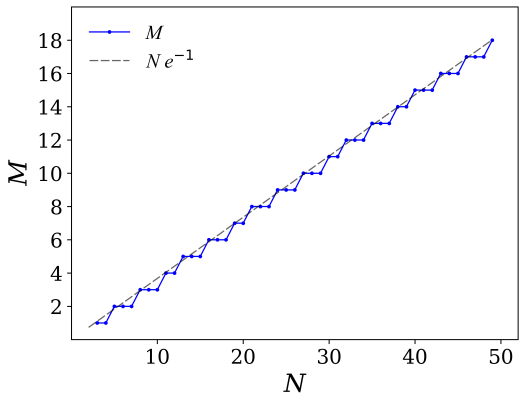
> Observe 36.8% of the potential profiles and then choose the next one that is the best observed so far.

Combining (4.43) with (4.40) and (4.45) shows that

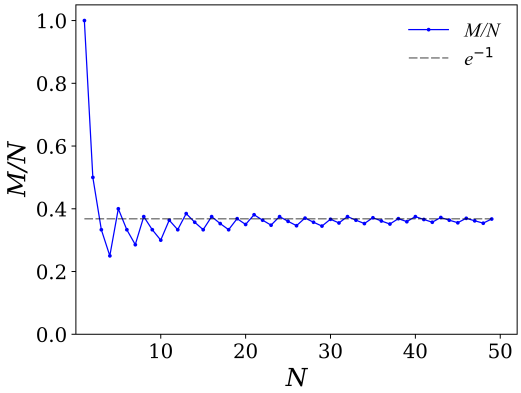
$$u_1(0) = u_1(1) = u_M(0) = \frac{Ne^{-1}}{N} = e^{-1} = 0.368.$$

This means that for large N, the optimal policy finds the best match with probability 0.368 or 36.8% of the time.

Figure 4.4a plots M versus N. Notice that the relationship closely tracks the line $M = Ne^{-1}$, as suggested by the theory. Figure 4.4b plots M/N versus N. This figure clearly shows how M/N converges to e^{-1}, the probability that the optimal policy finds the best match.



(a) Comparing M to Ne^{-1}. Since Ne^{-1} closely approximates M, stopping after viewing Ne^{-1} profiles is a good approximation to the optimal policy.



(b) The function M/N as a function of N. As N increases, M/N converges rapidly to e^{-1}, the probability that the optimal policy finds the best match.

Figure 4.4: Graphical representation of the optimal solution to the online dating problem and its approximation.

4.4 Interpretability: Optimality of structured policies

Interpretability refers to the ability of a user to clearly understand why a model prescribes a specific action in a specific state. This concept has become especially important as decision problems have become increasingly complex and often result in black box solutions that users may not be confident in implementing.

Along these lines, a significant stream of research dating back to the origins of operations research in the 1950s focused on interpretability by examining the structure of optimal policies in Markov decision process applications. One of the most well-known examples comes from inventory control. Consider an inventory model with fixed ordering costs and linear holding and shortage costs. Scarf [1960] showed that there exists an (s, S) policy (Figure 1.1) within the set of optimal policies. Such a policy is implemented as follows: if at any decision epoch the inventory level falls below s, an order is placed to raise the inventory level up to S. Such policies are also referred

to as *min-max* policies and are implemented in many commercial inventory systems. Policies of this form are easy to understand and implement. The primary challenge is to determine optimal values for the quantities s and S.

In addition to providing a theoretical basis for the structure of an optimal policy, there is also a computational advantage to identifying the structure of an optimal policy. Knowledge that such a policy exists reduces the search for an optimal policy to a smaller policy class. For example, if one can compute the cost of an (s, S) policy by other analytical methods, then optimizing over s and S will provide an optimal policy.

The online dating problem above provides another instance of a model in which the optimal policy has an interpretable structure. The analysis in Section 4.3.3 shows that it is optimal for the online dater to observe the first M profiles and then choose the first subsequent profile better than all of those previously observed. By doing so, the online dater is able to establish a benchmark on which to base decisions following decision epoch M. Moreover, knowing that the optimal policy has this form, one can use an approximation to determine M.

4.4.1 A general approach

Assume for simplicity that S is ordered[16], for example $S = \{0, 1, \ldots, M\}$. Let V^F denote the set[17] of real-valued functions of a specified form (such as convex) on S and let D^F be a subset of D^{MD} in which all decision rules have a specified structure. The sets D^F and V^F are *compatible* if for every $v(\cdot)$ in V^F and $s \in S$, there exists a

$$d(s) \in \arg\max_{a \in A_s} \left\{ \sum_{p \in S} p_n(j|s,a)\big(r_n(s,a,j) + v(j)\big) \right\} \tag{4.47}$$

in D^F for $n = 1, 2, \ldots, N-1$.

Analyses below establish that:

1. In a machine maintenance model, non-decreasing value functions are compatible with control-limit policies, and

2. in a queuing service rate control model, convex non-decreasing value functions are compatible with monotone non-decreasing Markovian deterministic decision rules.

The following theorem provides a framework for establishing the optimality of a structured policy. It relies heavily on backwards induction.

[16] These analyses generalize easily to partially ordered S such as $\mathbb{R}^n$.

[17] Subsequent chapters will express the Bellman equation as an operator on a set of value functions.

Theorem 4.5. Suppose that $r_N(\cdot)$ is in V^F and for each $v(\cdot)$ in V^F, $v'(\cdot)$ defined by

$$v'(s) = \max_{a \in A_s} \left\{ \sum_{p \in S} p_n(j|s,a)\big(r_n(s,a,j) + v(j)\big) \right\} \tag{4.48}$$

for all $s \in S$ is in V^F for $n = 1, 2, \ldots, N-1$.

Then

1. $v_n^*(s)$ is in V^F for $n = 1, 2, \ldots, N-1$.
2. If D^F is compatible with V^F, there exists a Markovian deterministic optimal policy
$$\pi^* = \big(d_1^*(s), \ldots, d_{N-1}^*(s)\big)$$
with $d_n^*(\cdot) \in D^F$ for $n = 1, \ldots, N-1$.

Proof. The proof is by backwards induction. Since $v_N^*(s) = r_N(s)$ for all $s \in S$, $v_N^*(.)$ is in V^F. Next assume that for $t = N-1, N-2, \ldots, n+1$, $v_t^*(\cdot)$ is in V^F. Then by (4.48) $v_n^*(\cdot)$ is in V^F. Hence $v_n^*(s)$ is in V^F for $n = 1, 2, \ldots, N-1$. Thus, if D^F is compatible with V^F, $d_n^*(\cdot)$ is in D^F for $n = 1, 2, \ldots, N-1$. □

By providing the necessary ingredients to establish the optimality of a structured policy, this result avoids the need to recreate the induction argument in every application. The challenge when applying it is to verify (4.48).

4.4.2 Example: Preventive maintenance

This section illustrates the use of Theorem 4.5 by applying it to determine the structure of an optimal policy in a simple dynamic machine maintenance model. In it, a machine deteriorates probabilistically over time and becomes more costly to operate as its state deteriorates. Alternatively, the machine can be restored to a pristine state at any time at a fixed cost.

Model formulation

Let $S = \{0, 1, \ldots, M\}$ represent the state of a machine; a higher state value represents greater deterioration. Assume that the state of the machine is known at the beginning of period n, at which time the decision maker may either restore it (action a_1) to state 0 or operate it as is (action a_0). Hence $A_s = \{a_0, a_1\}$ for $s \in S$.

Restoring the machine in period n costs K_n and operating the machine in state s for one-period, incurs an operating cost $f_n(s)$ for $n = 1, 2, \ldots, N-1$. Assume that $f_n(s)$ is a non-decreasing function of s for all $n = 1, \ldots, N$. This means that operating

the machine in a more deteriorated state is at least as costly as operating it in a less deteriorated state[18].

If the machine is not restored (action $a_0 0$) in period n and is in state $s < M$, it deteriorates to state $s+1$ with probability p_n and remains in state s with probability $1-p_n$. Furthermore assume the machine remains in state M, unless it is restored. Restoring the machine (action a_1) requires one period resulting in a machine in state 0 at the start of the next period.

Consequently, for $n = 1, 2, \ldots, N-1$

$$p_n(j|s,a) = \begin{cases} p_n & j = s+1, a = a_0, s \leq M-1 \\ 1-p_n & j = s, a = a_0, s \leq M-1 \\ 1 & j = 0, a = a_1, s \leq M \quad \text{or} \quad j = M, a = 0, s = M \\ 0 & \text{otherwise.} \end{cases}$$

Since the decision maker seeks to minimize the expected cost of operating the machine over N periods, this problem is best expressed as a minimization problem. The Bellman equations for the model are

$$u_n(s) = f_N(s)$$

for $s \in S$ and

$$u_n(s) = \begin{cases} \min\{K_n + u_{n+1}(0), f_n(s) + p_n v_{n+1}(s+1) + (1-p_n)u_{n+1}(s)\} & 0 \leq s \leq M-1 \\ \min\{K_n + u_{n+1}(0), f_n(M) + u_{n+1}(M)\} & s = M \end{cases} \tag{4.49}$$

and $n = 1, 2, \ldots, N-1$.

Control-limit policies are optimal

To apply Theorem 4.5 requires specifying V^F and showing that when $v(\cdot)$ is in V^F, so is $v'(\cdot)$ as defined by (4.48).

Let V^F denote the set of non-decreasing functions on S. By assumption $f_N(\cdot)$ is in V^F. Now suppose $v(s)$ is non-decreasing in s. Then

$$f_n(s+1) + p_n v(s+2) + (1-p_n)v(s+1) - \big(f_n(s) + p_n v(s+1) + (1-p_n)v(s)\big) \geq 0$$

for $s \leq M-2$ and

$$f_n(M) + v(M) - \big(f_n(s) + p_n v(M) + (1-p_n)v(M-1)\big) \geq 0,$$

so that the second expression in (4.49) is non-decreasing.

Since $K_n + v(0)$ is a constant, it is non-decreasing. Noting that the minimum of two non-decreasing functions is non-decreasing, establishes that $v'(s)$ is non-decreasing and

[18] This cost might be the result of producing lower quality output and reducing revenue.

in V^F. Hence the hypotheses of Theorem 4.5 are satisfied and $v_n^*(s)$ is non-decreasing in s

What remains is to determine the form of D^F. At decision epoch n, for $0 \leq s \leq M-1$,

$$d_n(s) = \begin{cases} 0 & \text{if } f_n(s) + p_n v(s+1) + (1-p_n)v(s) \leq K_n + v(0) \\ 1 & \text{if } f_n(s) + p_n v(s+1) + (1-p_n)v(s) \geq K_n + v(0) \end{cases} \tag{4.50}$$

and

$$d_n(M) = \begin{cases} 0 & \text{if } f_n(M) + v(M) \leq K_n + v(0) \\ 1 & \text{if } f_n(M) + v(M) \geq K_n + v(0). \end{cases} \tag{4.51}$$

For any non-decreasing $v(\cdot)$, $f_n(s) + p_n v(s+1) + (1-p_n)v(s)$ crosses $K_n + v(0)$ at most once so it follows from (4.50) and (4.51)[19] that the set of compatible decision rules has the form

$$d_n(s) = \begin{cases} 0 & \text{if } s \leq s_n \\ 1 & \text{if } s \geq s_n \end{cases} \tag{4.52}$$

for some constants s_n. Figure 4.5 shows how s_n is determined.

Such a policy is referred to as a *control-limit* or *threshold* policy and the quantity s_n denotes the control-limit or threshold at decision epoch n. Thus D^F represents the set of decision rules of form (4.52) establishing the following result.

Theorem 4.6. Suppose $f_n(s)$ is non-decreasing for $n = 1, 2, \ldots, n$. Then there exists an optimal control-limit policy in the machine maintenance model.

Figure 4.5 shows that it is possible that in (4.52)[20] that $s_n < 0$ meaning that it is optimal to restore the machine in every state in period n or $s_n > M$ meaning it is optimal to not restore the machine in period n. Moreover, if $s = s_n$, both actions are optimal.

It is left as an exercise to provide conditions on the model parameters that ensure $s_n \in \{0, 1, \ldots, M\}$ and also to determine an easy way to compute it. Furthermore one might be interested in determining how s_n behaves as a function of n.

4.4.3 Monotonicity of optimal policies in service rate control queuing systems*

Section 4.3.1 showed optimal policies exhibited the following interpretable structures:

[19] It is left to the reader to complete the argument for $s = M$.

[20] After extending the function beyond $\{0, \ldots, M\}$.

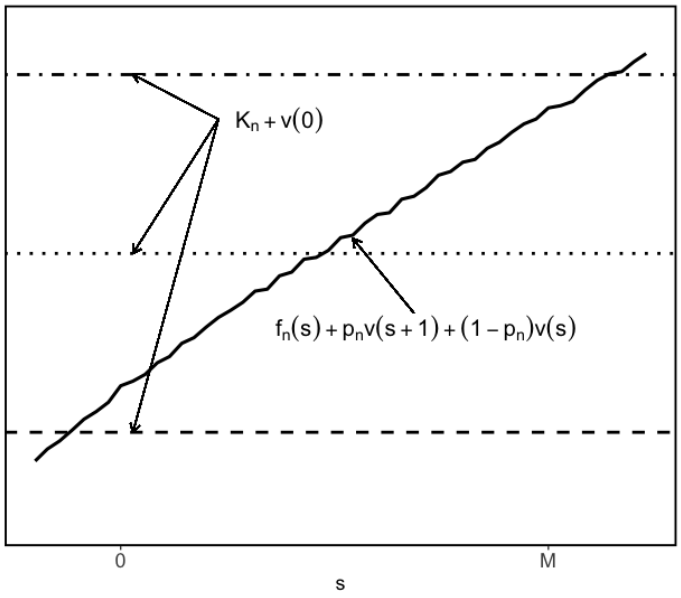


Figure 4.5: Graphical representation of calculation of s_n. It is the value at which the non-decreasing function (extended beyond $\{0, \ldots, M\}$) crosses one of the horizontal lines corresponding to possible values for $K_n + v(0)$. There are three cases; $s_n > M$, ($K_n + v(0)$ denoted by dash-dot line), $0 < s_n < M$ (dotted line) and $0 > s_n$ (dashed line). Note the computations must be adjusted for the comparison of $f(M) + v(M)$ to $K_n + v(0)$.

1. When delay costs and holding costs were linear the optimal policy chose either the lowest service probability a_1 or the highest service probability a_3. Furthermore, except at one decision epoch when truncation resulted in non-monotone behavior at the boundary, the optimal policy exhibited a "bang-bang" structure. That is, there was a particular state s' such that it chose the slowest service rate a_1 when $0 \leq s \leq s'$ and the fastest service rate a_3 when $s' < s \leq W$.

2. In another parameter regime with linear delay costs and cubic serving costs, the optimal policy selected all three actions, with larger values of the action being optimal in larger state values. Again at one decision epoch, state-space truncation introduced non-monotone behavior.

3. In all cases and in all states, the optimal policy was monotone in the decision epoch.

This section formalizes the first two observations theoretically. It shows that under reasonable conditions, an optimal policy for the service rate control problem is a monotone policy. That is, assuming ordered states and actions, the optimal action a_s^* is non-decreasing in s. To avoid the non-monotonicity due to state space truncation, this section assumes a *countably infinite* state space $S = \{0, 1, \ldots\}$.

Definition 4.6. A decision rule $d(s)$ is *monotone* if $d(s)$ is non-decreasing or non-increasing in s. A policy $\pi = (d_1, d_2, \ldots, d_N)$ is monotone if it consists of monotone decision rules $d_1, d_2, \ldots, d_N$.

The Bellman equation

For $n = 1, \ldots, N-1$ and $s = 1, 2, \ldots$ the Bellman equation for this model, written in terms of cost minimization, is given by

$$u_n(s) = \min_{a \in A_s} \{f(s) + m(a) + a u_{n+1}(s-1) + (1-a-b)u_{n+1}(s) + b u_{n+1}(s+1)\} \quad (4.53)$$

and for $s = 0$ by

$$u_n(0) = \min_{a \in A_0} \{f(0) + m(a) + b u_{n+1}(1) + (1-b)u_{n+1}(0)\}. \quad (4.54)$$

Moreover $u_N(s) = r_N(s)$ for all $s \in S$.

Given a solution $v_n^*(s)$ to this system of equations subject to the boundary condition, define $A_{n,s}^*$ by

$$A_{n,s}^* = \arg\min_{a \in A} \{f(s) + m(a) + a v_{n+1}^*(s-1) + (1-a-b)v_{n+1}^*(s) + b v_{n+1}^*(s+1)\}. \quad (4.55)$$

Since this set may have multiple elements, define $a_{n,s}^* = \min\{a \mid a \in A_{n,s}^*\}$ to be the smallest element in the set.

Note that when $s = 0$, (4.54) reduces to

$$u_n(0) = \min_{a \in A_0} \{m(a)\} + f(0) + b u_{n+1}(1) + (1-b)u_{n+1}(0).$$

Since $m(a)$ is non-decreasing, a_1 is the optimal action at state $s = 0$ for all n, or in other words, $A_{n,0}^* = \{a_1\}$ for $n = 1, \ldots, N-1$. This intuitive observation is important since it allows the subsequent development to focus on the value function structure and optimal policy for states 1 and higher, avoiding inconvenient boundary conditions at $s = 0$.

The main result

A definition of convexity for functions on discrete sets, which is needed to prove optimality of a monotone policy, follows. This definition is the discrete analogue of the condition that a differentiable function is convex if its first derivative is non-decreasing.

Definition 4.7. A function $g(x)$ defined on $\mathbb{Z}_+ = \{0, 1, \ldots\}$ is *convex* if for all $x = 1, 2, \ldots$

$$g(x+1) - g(x) \geq g(x) - g(x-1). \tag{4.56}$$

Alternatively, define $\Delta g(x) := g(x) - g(x-1)$. Then $g(x)$ is convex if

$$\Delta g(x+1) \geq \Delta g(x) \tag{4.57}$$

for all $x = 1, 2, \ldots$.

The following arguments establish optimality of a monotone policy by first establishing the convexity of solution of the Bellman equations under the assumption that the delay cost $f(s)$ and terminal reward $r_N(s)$ are convex functions of s.

Lemma 4.2. Suppose $f(s)$ and $r_N(s)$ are convex functions of s and let $v_n^*(s)$ be solutions of (4.53) and (4.54) subject to $u_n(s) = r_N(s)$ for all $s \in S$.

Then $v_n^*(s)$ is convex in s for $n = 1, \ldots, N$.

Proof. Let $s \in \{1, 2, \ldots\}$. The proof proceeds by (backwards) induction on n.

Clearly, the result holds for $n = N$ since $v_N^*(s) = r_N(s)$ by definition. Now assume that $v_{n+1}^*(s)$ is convex. Under this assumption, the following arguments establish that $\Delta v_n^*(s+1) \geq \Delta v_n^*(s)$ for $s = 1, 2, \ldots$.

Since $a_{n,s+1}^*$ need not attain the minimum at s, it follows from (4.53) that

$$v_n^*(s) \leq f(s) + m(a_{n,s+1}^*) + a_{n,s+1}^* v_{n+1}^*(s-1) + (1 - a_{n,s+1}^* - b) v_{n+1}^*(s) + b v_{n+1}^*(s+1). \tag{4.58}$$

By the definition of $a_{n,s+1}^*$,

$$v_n^*(s+1) = f(s+1) + m(a_{n,s+1}^*) + a_{n,s+1}^* v_{n+1}^*(s) + (1 - a_{n,s+1}^* - b) v_{n+1}^*(s+1) + b v_{n+1}^*(s+2)\}. \tag{4.59}$$

Subtracting (4.58) from (4.59) yields

$$\begin{aligned}
\Delta v_n^*(s+1) &\geq \Delta f(s+1) + a_{n,s+1}^* \Delta v_{n+1}^*(s) + (1 - a_{n,s+1}^* - b) \Delta v_{n+1}^*(s+1) + b \Delta v_{n+1}^*(s+2) \\
&\geq \Delta f(s+1) + a_{n,s+1}^* \Delta v_{n+1}^*(s) + (1 - a_{n,s+1}^* - b) \Delta v_{n+1}^*(s) + b \Delta v_{n+1}^*(s+2) \\
&= \Delta f(s+1) + (1-b) \Delta v_{n+1}^*(s) + b \Delta v_{n+1}^*(s+2),
\end{aligned}$$

where the second inequality follows from convexity of $v_{n+1}^*(s)$.

By a similar argument applied at s and $s-1$,

$$\begin{aligned}
\Delta v_n^*(s) &\leq \Delta f(s) + a_{n,s-1}^* \Delta v_{n+1}^*(s-1) + (1 - a_{n,s-1}^* - b) \Delta v_{n+1}^*(s) + b \Delta v_{n+1}^*(s+1) \\
&\leq \Delta f(s) + (1-b) \Delta v_{n+1}^*(s) + b \Delta v_{n+1}^*(s+1),
\end{aligned}$$

where the convexity of $v^*_{n+1}(s)$ implies inequality $\Delta v^*_{n+1}(s) \geq \Delta v^*_{n+1}(s-1)$ resulting in the inequality above.

Combining the above inequalities, gives

$$\Delta v^*_n(s+1) - \Delta v^*_n(s) \geq \Delta f(s+1) - \Delta f(s) + b(\Delta v^*_{n+1}(s+2) - \Delta v^*_{n+1}(s+1)),$$

which is non-negative, due to convexity of $f(s)$ and $v^*_{n+1}(s)$. Hence $v^*_n(s)$ is convex. Thus the induction hypothesis is satisfied from which the convexity of $v^*_n(s)$ for $n = 1, 2, \ldots, N$ follows.

□

Note that for a fixed action a, the value function satisfies

$$v_n(s) = f(s) + m(a) + av_{n+1}(s-1) + (1-a-b)v_{n+1}(s) + bv_{n+1}(s+1).$$

Since the sum of convex functions is convex, it may be tempting to conclude that $v_n(s)$ is convex since the above equation holds for each $a \in A_s$ and consequently would hold for an optimal choice of a as well. However, the subtlety here is that an optimal a is dependent on the state s, which is why it is denoted $a_{n,s}$. Thus, establishing the convexity of $v^*_n(s)$ would require analyzing how all terms that involve $a_{n,s}$ vary as functions of s.

Lemma 4.3. Suppose $v^*_n(s)$ is a convex function of s for $n = 1, \ldots, N-1$. Then $a^*_{n,s}$ is non-decreasing in s for $n = 1, \ldots, N-1$.

Proof. Rewrite (4.53) as

$$u_n(s) = f(s) + b\Delta u_{n+1}(s+1) + u_{n+1}(s) + \min_{a \in A}\{m(a) - a\Delta u_{n+1}(s)\}. \tag{4.60}$$

Thus, it suffices to show that the smallest minimizer of $m(a) - a\Delta v^*_{n+1}(s)$ is non-decreasing in s.

Fix $s \in \{1, 2, \ldots\}$. Let $a_{n,s}$ be the smallest minimizer of $m(a) - a\Delta v^*_{n+1}(s)$. Let $s' > s$. To show that $a_{n,s'} \geq a_{n,s}$ assume to the contrary that $a_{n,s'} < a_{n,s}$. Then

$$\begin{aligned} m(a_{n,s'}) - a_{n,s'}\Delta v^*_{n+1}(s) &= \big(m(a_{n,s'}) - a_{n,s'}\Delta v^*_{n+1}(s')\big) + a_{n,s'}(\Delta v^*_{n+1}(s') - \Delta v^*_{n+1}(s)) \\ &\leq \big(m(a_{n,s}) - a_{n,s}\Delta v^*_{n+1}(s')\big) + a_{n,s'}(\Delta v^*_{n+1}(s') - \Delta v^*_{n+1}(s)) \\ &< \big(m(a_{n,s}) - a_{n,s}\Delta v_{n+1}(s')\big) + a_{n,s}(\Delta v^*_{n+1}(s') - \Delta v^*_{n+1}(s)) \\ &= m(a_{n,s}) - a_{n,s}\Delta v^*_{n+1}(s), \end{aligned}$$

where the first inequality is due to optimality of $a_{n,s'}$ and the second inequality is due to the assumption that $a'_{n,s} < a_{n,s}$ and convexity of $v^*_{n+1}(s)$. However, this contradicts the assumption that $a_{n,s}$ is the smallest minimizer of $m(a) - a\Delta v^*_{n+1}(s)$, so the result follows. □

Combining these two lemmas and noting that $a^*_{n,0} = a_1$ gives the main result.

Theorem 4.7. Suppose $f(s)$ and $r_N(s)$ are convex functions of s. Then for $n = 1, \ldots, N-1$, $d^*_n(s)$ is monotone non-decreasing in s.

Note this section established Theorem 4.7 from first principles. Alternatively one could prove it using Theorem 4.5 by choosing V^F to be the set of convex functions on S and showing that V^F is compatible with D^F the set of monotone non-decreasing decision rules.

4.5 State-action value functions

Using the Bellman equation

$$u_n(s) = \max_{a \in A_s} \left\{ \sum_{j \in S} p_n(j|s,a)(r_n(s,a,j) + u_{n+1}(j)) \right\} \tag{4.61}$$

may not be the most effective approach for finding an optimal policy when S is large because computing expectations inside the maximum for each action may be computationally prohibitive. Instead, basing calculations on a recursion for state-action value functions might alleviate this problem by providing a basis for analyzing such models by simulation. They are fundamental in Chapters 10 and 11.

A Bellman-like recursion for state-action value functions requires the concept of a *shifted period*, one that starts immediately after selecting an action at decision epoch n and concludes immediately *after* selecting an action at decision epoch $n+1$. This contrasts with the traditional Markov decision process viewpoint where a period starts immediately before action choice at decision epoch n and ends *prior* to action choice at decision epoch $n+1$.

Figure 4.6 compares these two perspectives. The shifted period perspective represents transitions from *post-decision* states to *post-decision* states while the usual Markov decision process perspective represents transitions from *pre-decision* states to *pre-decision* states.

4.5.1 State-action value function recursion for a fixed policy

The *state-action value function*[21] $q^\pi_n(s,a)$ of the Markovian deterministic[22] policy $\pi = (d_1, \ldots, d_{N-1})$ represents the expected total reward to the end of the planning horizon

[21] This is also commonly referred to as a q-function, following the use of q to represent the function mathematically.

[22] A modification for Markovian randomized policies appears below.

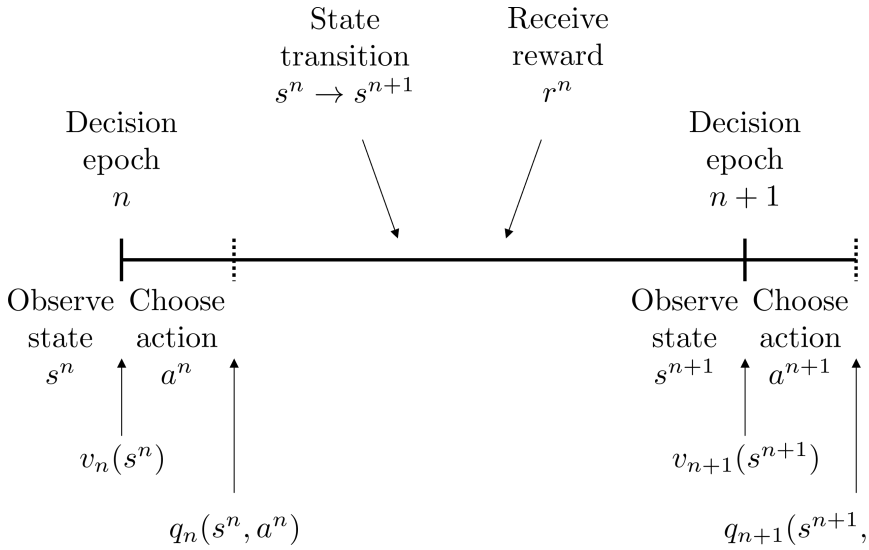


Figure 4.6: Timeline showing that $v_n(s^n)$ is computed after observing state s^n but before choosing action a^n, whereas $q_n(s^n, a^n)$ is computed after observing both s^n and a^n. In this diagram, a period starts at decision epoch n and terminates immediately before decision epoch $n+1$ after observing s^{n+1}.

after choosing action a in state s at decision epoch n and following $(d_{n+1}, d_{n+2}, \ldots, d_{N-1})$ thereafter.

It is easy to show that for $n = 1, 2, \ldots, N-1$, $q_n^\pi(s, a)$ satisfies the following equations.

State-action value function recursion to evaluate a fixed Markovian deterministic policy:

For all $s \in S$ and $a \in A_s$,

$$q_N(s, a) = r_N(s) \tag{4.62}$$

and

$$q_n(s, a) = \sum_{j \in S} p_n(j|s, a) \Big(r_n(s, a, j) + q_{n+1}(j, d_{n+1}(j)) \Big) \tag{4.63}$$

for $n = 1, \ldots, N-1$.

In contrast, the value function for π satisfies the following recursions, which appeared as (4.6).

Value function recursion to evaluate a fixed Markovian deterministic policy:

For $s \in S$,

$$u_N(s) = r_N(s) \tag{4.64}$$

and

$$u_n(s) = \sum_{j \in S} p_n(j|s, d_n(s))\big(r_n(s, d_n(s), j) + u_{n+1}(j)\big) \tag{4.65}$$

for $n = 1, 2, \ldots, N-1$.

Some important and subtle distinctions between these two recursions follow:

1. The state-action value function involves d_{n+1} while the value function depends on d_n.

2. The policy enters into (4.63) only through the appearance of $d_{n+1}(j)$ in the expression $q_{n+1}(j, d_{n+1}(j))$. It does not appear in either $p_n(j|s, a)$ or $r_n(s, a, j)$. On the other hand, in (4.65), the policy appears explicitly through $d_n(s)$ in both $p_n(j|s, d_n(s))$ and $r_n(s, d_n(s), j)$ but not explicitly in $u_{n+1}(j)$.

3. Expression (4.65) only depends on a single action $d_n(s)$, whereas (4.63) relates the value of two actions, the action a chosen at decision epoch n and the policy-specific action $d_{n+1}(j)$, where j is the subsequent state to s.

4. When evaluating a deterministic policy, one needs only $q_{n+1}(j, d_{n+1}(j))$ at the next iterate and not $q_{n+1}(j, a)$ for $a \neq d_{n+1}(j)$.

5. If instead π is a Markovian randomized policy, the quantity $q_{n+1}(j, d_{n+1}(j))$ is replaced by $\sum_{a' \in A_s} w^n_{d_n}(a'|j) q_{n+1}(j, a')$.

4.5.2 Revisiting the two-state model from the state-action value function perspective

This section analyzes state-action value functions for the two-state example in Section 4.1.2. Set $N = 3$, the terminal reward $r_3(s_1) = r_3(s_2) = 0$, and evaluate the Markovian deterministic policy $\pi = (d_1, d_2)$, in which

$$d_1(s_1) = a_{1,2}, \quad d_1(s_2) = a_{2,2}$$

and

$$d_2(s_1) = a_{1,1}, \quad d_2(s_2) = a_{2,1}.$$

When $n = 2$:

$$q^\pi_2(s_1, a_{1,1}) = p_2(s_1|s_1, a_{1,1})\big(r_2(s_1, a_{1,1}, s_1) + r_3(s_1)\big) + p_2(s_2|s_1, a_{1,1})\big(r_2(s_1, a_{1,1}, s_2) + r_3(s_2)\big)$$

$$= 0.8(5+0) + 0.2(-5+0)$$
$$= 3$$

and

$$q_2^\pi(s_2, a_{2,1}) = p_2(s_1|s_2, a_{2,1})\big(r_2(s_2, a_{2,1}, s_1) + r_3(s_1)\big) + p_2(s_2|s_2, a_{2,1})\big(r_2(s_2, a_{2,1}, s_2) + r_3(s_2)\big)$$
$$= 1(-5+0)$$
$$= -5$$

When $n = 1$:

$$q_1^\pi(s_1, a_{1,2}) = p_1(s_1|s_1, a_{1,2})\big(r_1(s_1, a_{1,2}, s_1) + q_2^\pi(s_1, d_2(s_1))\big) + p_1(s_2|s_1, a_{1,2})\big(r_1(s_1, a_{1,2}, s_2) + q_2^\pi(s_2, d_2(s_2))\big) \quad (4.66)$$

and

$$q_1^\pi(s_2, a_{2,2}) = p_1(s_1|s_2, a_{2,2})\big(r_1(s_2, a_{2,2}, s_1) + q_2^\pi(s_1, d_2(s_1))\big) + p_1(s_2|s_2, a_{2,2})\big(r_1(s_2, a_{2,2}, s_2) + q_2^\pi(s_2, d_2(s_2))\big). \quad (4.67)$$

Substituting values in (4.66) and (4.67) and noting that $d_2(s_1) = a_{1,1}$ and $d_2(s_2) = a_{2,1}$ yields

$$q_1^\pi(s_1, a_{1,2}) = 0(0+3) + 1(5+(-5)) = 0$$
$$q_1^\pi(s_2, a_{2,2}) = 0.4(20+3) + 0.6(-10+(-5)) = 0.2$$

Since $d_1(s_1) = a_{1,2}$ and $d_1(s_2) = a_{2,2}$ it follows that that $q_1^\pi(s_1, d_1(s_1)) = 0$ and $q_1^\pi(s_2, d_1(s_2)) = 0.2$.

Observe that:

1. These values agree with those in Section 4.1 and require the same computational effort. That is, $q_1^\pi(s_1, d_1(s_1)) = v_1^\pi(s_1) = 0$ and $q_1^\pi(s_2, d_1(s_2)) = v_1^\pi(s_2) = 0.2$.

2. There is no need to calculate q-function values for actions that π does not select at decision epochs 1 and 2 because they do not enter into subsequent evaluations and are not required to determine $v_1^\pi(s)$.

4.5.3 Optimal state-action value functions

State-action value functions are more important when seeking optimal policies. Let

$$q_n^*(s, a) := \sup_{\pi \in \Pi^{\text{HR}}} q_n^\pi(s, a) = \max_{\pi \in \Pi^{\text{MD}}} q_n^\pi(s, a)$$

for $s \in S$ and $a \in A_s$, where the last equality follows from an obvious modification of the proof of Theorem 4.2.

Since there is no action to be taken at decision epoch N,

$$q_N^*(s, a) = r_N(s) \quad (4.68)$$

for all $s \in S$ and $a \in A_s$. In decision epoch $N-1$,

$$q^*_{N-1}(s,a) = \sum_{j \in S} p_{N-1}(j|s,a)\big(r_{N-1}(s,a,j) + r_N(j)\big). \tag{4.69}$$

At decision epoch $N-2$, the recursion now must choose the decision at the next decision epoch, $N-1$ optimally so that

$$q^*_{N-2}(s,a) = \max_{a' \in A_s} \left\{ \sum_{j \in S} p_{N-1}(j|s,a)\big(r_{N-1}(s,a,j) + q^*_{N-1}(j,a')\big) \right\} \tag{4.70}$$

$$= \sum_{j \in S} p_{N-1}(j|s,a)\big(r_{N-1}(s,a,j) + \max_{a' \in A_s}\{q^*_{N-1}(j,a')\}\big). \tag{4.71}$$

The second equality follows by noting that a' only appears in $q^*_{N-1}(j,a')$ so the maximization can be brought inside the expectation. Continuing in this way it is easy to prove that $q^*_n(s,a)$ satisfies the following recursions.

Bellman equation for state-action value functions:

For $s \in S$ and $a \in A_s$,

$$q_N(s,a) = r_N(s) \tag{4.72}$$

and

$$q_n(s,a) = \sum_{j \in S} p_n(j|s,a)\big(r_n(s,a,j) + \max_{a' \in A_j}\{q_{n+1}(j,a')\}\big) \tag{4.73}$$

for $n = 1, \ldots, N-1$.

A few key observations about this development follow:

1. It is important to note that the Bellman equation for $q^*_n(s,a)$ differs from the Bellman equation for $v^*_n(s)$ in (4.22) in that the maximization is now *inside* the expectation. This change may not seem significant but it substantially reduces computation when $v^*_n(\cdot)$ is evaluated through simulation.

2. There is a very close connection between $v^*_n(s)$ and $q^*_n(s,a)$. In particular, for $n = 1, 2, \ldots, N$

$$v^*_n(s) = \max_{a \in A_s} q^*_n(s,a). \tag{4.74}$$

It then follows that

$$q^*_n(s,a) = \sum_{j \in S} p_n(j|s,a)(r_n(s,a,j) + v^*_{n+1}(j)). \tag{4.75}$$

Therefore $q^*_n(s,a)$ may be interpreted as *the expected total reward from decision epoch n onward when action a is chosen in state s and then the system evolves optimally.*

3. For $n = 1, 2, \ldots, N-1$ and $s \in S$, let $A^*_{n,s}$ denote the optimal actions in state s at decision epoch n. Previously, $A^*_{n,s}$ was defined in equation (4.21) using $v^*_{n+1}(\cdot)$:

$$A^*_{n,s} = \arg\max_{a \in A_s} \left\{ \sum_{j \in S} p_n(j|s,a)(r_n(s,a,j) + v^*_{n+1}(j)) \right\}.$$

Using q-functions, and noting equation (4.75), $A^*_{n,s}$ can be written much more succinctly as

$$A^*_{n,s} = \arg\max_{a \in A_s} q^*_n(s,a). \tag{4.76}$$

4. We found that when coding backwards induction based on value functions as in Algorithm 4.3 it was convenient to first evaluate

$$q_n(s,a) = r_n(s,a) + \sum_{j \in S} p_n(j|s,a) u_{n+1}(j)$$

for each $a \in A_s$ and then set

$$u_n(s) = \max_{a \in A_s} q_n(s,a).$$

Backwards induction using q-functions

Algorithm 4.4. Finite horizon policy optimization using state-action value functions

1. **Initialize:** Set $n \leftarrow N-1$, $q_N(s,a) \leftarrow r_N(s)$ for all $s \in S$ and $a \in A_s$, and $d_N(s)$ to be an arbitrary action in A_s.
2. **Iterate:** While $n \geq 1$:
   (a) For all $s \in S$ and $a \in A_s$, evaluate $q_n(s,a)$ according to

$$q_n(s,a) \leftarrow \sum_{j \in S} p_n(j|s,a)\big(r_n(s,a,j) + q_{n+1}(j, d_{n+1}(j))\big), \tag{4.77}$$

   where $d_{n+1}(j)$ can be defined as an arbitrary action $a \in A_j$. Set

$$A^*_{n,s} \leftarrow \arg\max_{a \in A_s} q_n(s,a) \tag{4.78}$$

   and choose $d_n(s)$ to be any action in $A^*_{n,s}$.
   (b) $n \leftarrow n-1$.
3. **Terminate:** Return $q_1(s, d_1(s))$ for all $s \in S$ and $\pi = (d_1, \ldots, d_{N-1})$.

It is left to the reader to establish that the computed q from Algorithm 4.4 is equal to q^*, in a similar way as that used to show that u from Algorithm 4.3 is equal to v^* (Theorem 4.3).

4.5.4 An example

This section finds an optimal policy in the two-state model using Algorithm 4.4. First set $q_3^*(s,a) = 0$ for all $a \in A_s$ and $s \in S$.

To find q_1^* and q_2^*, requires accounting for each possible action in each state at each decision epoch. For example:

$$\begin{aligned} q_1^*(s_1, a_{1,1}) &= p_1(s_1|s_1, a_{1,1})(r_1(s_1, a_{1,1}, s_1) + \max\{q_2^*(s_1, a_{1,1}), q_2^*(s_1, a_{1,2})\}) + \\ &\quad p_1(s_2|s_1, a_{1,1})(r_1(s_1, a_{1,1}, s_2) + \max\{q_2^*(s_2, a_{2,1}), q_2^*(s_2, a_{2,2})\}) \\ &= 0.8(5+5) + 0.2(-5+2) = 7.4 \end{aligned}$$

Table 4.3 gives all values for $q_n^*(s,a)$ for $n = 1, 2$.

State	Action	$q_2^*(s,a)$	$q_1^*(s,a)$
s_1	$a_{1,1}$	3	**7.4**
s_1	$a_{1,2}$	**5**	7
s_2	$a_{2,1}$	-5	-3
s_2	$a_{2,2}$	**2**	**5.2**

Table 4.3: Values of $q_n^*(s,a)$ for two-state model in Section 2.5. The maximum value in each state at each decision epoch is in **bold**.

It is easy to identify optimal policies and values from Table 4.3 by noting which actions attain the maximum at each decision epoch. These appear in Table 4.4.

Decision Epoch	State	Optimal Action	$v_n^*(s)$
1	s_1	$a_{1,1}$	7.4
1	s_2	$a_{2,2}$	5.2
2	s_1	$a_{2,2}$	5
2	s_2	$a_{2,2}$	2

Table 4.4: Optimal policies and values for two-state model in Section 2.5.

Inspecting Table 4.4 shows, for example, that

$$v_1^*(s_1) = \max\{q_1^*(s_1, a_{1,1}), q_1^*(s_1, a_{1,2})\} = 7.4$$

and from (4.76) the optimal action in state s_1 at decision epoch 1 is given by

$$A_{1,s_1}^* = \arg\max\{q_1^*(s_1, a_{1,1}), q_1^*(s_1, a_{1,2})\} = a_{1,1}.$$

Observe that the optimal policies and values agree with those computed in Section 4.2.3 using the corresponding algorithm for value functions.

4.5.5 Optimality equations in the queuing admission control model*

This section considers a finite horizon version of the queuing admission control model in Section 3.4.2. It first compares the traditional Bellman equation for value functions expressed in terms of pre-decision states, with the post-decision state model described in Section 3.4.2. Finally, it provides the Bellman equation using q-functions, which also corresponds to a post-decision state perspective.

Value function optimality equations: Pre-decision state formulation

The optimality equations expressed in terms of value functions for the pre-decision state formulation become, for $j = 0$,

$$u_n\big((0,1)\big) = \max\Big\{R - f(1) + bu_{n+1}\big((1,1)\big) + (1-b)u_{n+1}\big((1,0)\big), \\ - f(0) + bu_{n+1}\big((0,1)\big) + (1-b)u_{n+1}\big((0,0)\big)\Big\} \quad (4.79)$$

and

$$u_n\big((0,0)\big) = -f(0) + bu_{n+1}\big((0,1)\big) + (1-b)u_{n+1}\big((0,1)\big) \quad (4.80)$$

and for $j > 0$,

$$u_n\big((j,1)\big) \\ = \max\Big\{R - f(j+1) + bu_{n+1}\big((j+1,1)\big) + (1-b-w)u_{n+1}\big(j+1,0)\big) + wu_{n+1}\big((j,0)\big), \\ - f(j) + bu_{n+1}\big((j,1)\big) + (1-b-w)u_{n+1}\big((j,0)\big) + wu_{n+1}\big((j-1,0)\big)\Big\} \quad (4.81)$$

and

$$u_n\big(j,0)\big) = -f(j) + bu_{n+1}\big((j,1)\big) + (1-b-w)u_{n+1}\big((j,0)\big) + wu_{n+1}\big((j-1,0)\big) \quad (4.82)$$

for $n = 1, \ldots, N-1$. Observe that (4.80) and (4.82) do not contain the expression "max" since there is no decision when there are no jobs to admit.

Value function optimality equations: Post-decision state formulation

The optimality equations in the alternative post-decision state formulation become, for $j = 0$,

$$u_n(0) = \max\big\{R - f(1) + bu_{n+1}(1) + (1-b)u_{n+1}(0), -f(0) + u_{n+1}(0)\big\} \quad (4.83)$$

and for $j > 0$,

$$u_n(j) = \max\big\{R - f(j+1) + bu_{n+1}(j+1) + (1-b-w)u_{n+1}(j) + wu_{n+1}(j-1)$$
$$-f(j) + (1-w)u_{n+1}(j) + wu_{n+1}(j-1)\big\}. \quad (4.84)$$

Note that the main difference between (4.83) and (4.84) is the inclusion of the possibility of a service completion when $j > 0$. Note also that the first term in the "max" corresponds to admitting an arrival if it occurs and the second term to not doing so. Thus the second term does not involve b because whether or not an arrival occurs does not affect the "do not admit" decision.

Clearly the alternative formulation based on the post-decision states provides a more concise recursion.

State-action value function optimality equations

Now consider the optimality equations based on state-action value functions using the post-decision state formulation corresponding to Figure 3.8.

For $j = 0$,

$$q_n(0, a_0) = -f(0) + \max\{q_{n+1}(0, a_0), q_{n+1}(0, a_1)\} \quad (4.85)$$

and

$$q_n(0, a_1) = (1-b)\big(-f(0) + \max\{q_{n+1}(0, a_0), q_{n+1}(0, a_1)\}\big)$$
$$+ b\big(R - f(1) + \max\{q_{n+1}(1, a_0), q_{n+1}(1, a_1)\}\big). \quad (4.86)$$

Since whether or not an arrival occurs doesn't impact the "do not admit" action a_0, when $j > 0$

$$q_n(j, a_0) = (1-w)\big(-f(j) + \max\{q_{n+1}(j, a_0), q_{n+1}(j, a_1)\}\big)$$
$$+ w\big(-f(j-1) + \max\{q_{n+1}(j-1, a_0), q_{n+1}(j-1, a_1)\}\big) \quad (4.87)$$

and for action a_1 and $j > 0$

$$q_n(j, a_1) = (1-b-w)\big(-f(j) + \max\{q_{n+1}(j, a_0), q_{n+1}(j, a_1)\}\big)$$
$$+ w\big(-f(j-1) + \max\{q_{n+1}(j-1, a_0), q_{n+1}(j-1, a_1)\}\big])$$
$$+ b\big(R - f(j) + \max\{q_{n+1}(j+1, a_0)), q_{n+1}(j+1, a_1)\}\big). \quad (4.88)$$

Some comments on these equations follow:

1. The state-action value function optimality equations involve actions at two decision epochs. For example, in (4.88), a_1 on the left hand side of the equation corresponds to the choice of this action at decision epoch n while the quantity within the "max" corresponds to the action at the decision epoch $n+1$. The expression "max" ensures that the later decision is taken optimality.

2. In the value function optimality equations (4.83) and (4.84) the expectation over the next event is taken *before* evaluating the "max" while in the state-action value function optimality equations (4.85)-(4.88), the expectation over the next event is taken after evaluating the "max". This observation does not affect exact calculation using backwards induction but underlies the use of q-functions in simulations, especially in large models. In this example, simulation means using a random mechanism to generate the next event (arrival, service completion or neither).

Exercise 22 at the end of this chapter provides an opportunity to compare these three approaches to finding an optimal policy in an instance of the admission control model.

4.6 Technical appendix

This appendix shows that the finite horizon policy optimization algorithm (Algorithm 4.3) finds optimal values and optimal policies within the class of Markovian deterministic policies. It then extends this result to establish that these policies are optimal in the class of history-dependent policies.

4.6.1 Finding optimal policies in Π^{MD}

Theorem 4.8. Suppose that the maximum is attained in (4.22) for all $s \in S$, that $u_n(s)$ and $A^*_{n,s}$ for $n = 1, 2, \ldots, N-1$, $s \in S$ are computed by Algorithm 4.3 and $v^\pi_n(s)$ is as defined in equation (4.2). Then:

1. For any $\pi \in \Pi^{\mathrm{MD}}$, $u_n(s) \geq v^\pi_n(s)$ for $n = 1, 2, \ldots, N$ and $s \in S$.
2. Suppose $\pi^* = (d^*_1, d^*_2, \ldots, d^*_{N-1})$ where $d^*_n(s) \in A^*_{n,s}$ for $n = 1, 2, \ldots, N-1$ and $s \in S$. Then $v^{\pi^*}_n(s) = u_n(s)$ for all $s \in S$ and $n = 1, 2, \ldots, N$.

Proof. The proof of the first result is by induction. Let $\pi = (d_1, d_2, \ldots, d_{N-1}) \in \Pi^{\mathrm{MD}}$. The result holds trivially for $n = N$ since $u_N(s) = r_N(s) = v^\pi_N(s)$ for all $s \in S$.

Assume that $u_t(s) \geq v^\pi_t(s)$ for $t = n+1, \ldots, N$ and $s \in S$. From (4.22)

$$\begin{aligned} u_n(s) &= \max_{a \in A_s} \left\{ \sum_{j \in S} p_n(j|s,a)(r_n(s,a,j) + u_{n+1}(j)) \right\} \\ &\geq \sum_{j \in S} p_n(j|s, d_n(s))(r_n(s, d_n(s), j) + v^\pi_{n+1}(j)) \\ &= v^\pi_n(s), \end{aligned}$$

where the last equality holds from Theorem 4.1. Hence the result holds for $n = 1, \ldots, N$.

The proof of the second result is by induction as well. By the same argument as above, $u_N(s) = r_N(s) = v^{\pi^*}(s)$ for all $s \in S$. Now assume $v_t^{\pi^*}(s) = u_t(s)$ for all $s \in S$ and $t = n+1, \ldots, N$. By the definition of d_n^* together with (4.22) and (4.23) it follows for all $s \in S$ that

$$u_n(s) = \max_{a \in A_s} \left\{ \sum_{j \in S} p_n(j|s,a)(r_n(s,a,j) + u_{n+1}(j)) \right\} \tag{4.89}$$

$$= \sum_{j \in S} p_n(j|s, d_n^*(s))(r_n(s, a, d_n^*(s)) + u_{n+1}(j)) \tag{4.90}$$

$$= \sum_{j \in S} p_n(j|s, d_n^*(s))(r_n(s, a, d_n^*(s)) + v_{n+1}^{\pi^*}(j)) \tag{4.91}$$

$$= v_n^{\pi^*}(s), \tag{4.92}$$

where the third equality follows from the induction hypothesis and the last equality follows from Theorem 4.1. Hence the result follows.

□

Corollary 4.2. Let π^* be as defined in Theorem 4.8. Then π^* is optimal in the class of Markov deterministic policies.

The main takeaway from Theorem 4.8 and Corollary 4.2 is that Algorithm 4.3 is guaranteed to find an optimal policy within the class of Markov deterministic policies, along with the corresponding optimal value. However, not only does it find the optimal policy and value starting from the first epoch, it provides a method to construct optimal policies from any decision epoch n to the end of the planning horizon, along with the associated optimal value from epoch n onward.

4.6.2 Finding optimal values in $\Pi^{\mathbf{HR}}$

Before proving that there exist Markovian deterministic policies that are optimal in the class of history-dependent randomized policies, an algorithm is presented for finding optimal values only. Since results in the chapter Appendix establish that the search for optimal policies can be restricted to Markovian deterministic policies, this algorithm is not intended for use in practice.

Algorithm 4.5. Finite horizon policy optimization in $\Pi^{\mathbf{HR}}$

1. **Initialize:** Set $n \leftarrow N-1$ and $u'_N(h^N) \leftarrow r_N(s^N)$ for all $h^N = (h^{N-1}, a^{N-1}, s^N) \in H_N$.

2. **Iterate:** While $n \geq 1$:

   (a) For all $h^n = (h^{n-1}, a^{n-1}, s^n) \in H_n$, evaluate $u'_n(h^n)$ according to

   $$u'_n(h^n) \leftarrow \sup_{w \in \mathscr{P}(A_{s^n})} \left\{ \sum_{a \in A_{s^n}} w(a) \sum_{j \in S} p_n(j|s^n, a)\big(r_n(s^n, a, j) + u'_{n+1}((h^n, a, j))\big) \right\}. \tag{4.93}$$

   (b) $n \leftarrow n - 1$.

3. **Terminate:** Return $u'_1(s)$ for all $s \in S$ and $\pi = (d_1, \ldots, d_{N-1})$.

Equation (4.93) provides the most general form of the Bellman equation. A similar but slightly more tedious proof than that of Theorem 4.8 establishes that this algorithm finds optimal values. The following proposition states this formally; proving it is left as an exercise.

Proposition 4.1. Suppose for each $h^n \in H_n$, $n = 1, \ldots, N$, $u'_n(h^n)$ is determined by Algorithm 4.5. Then $u'_n(h^n) = v^*_n(h^n)$ and $u'_1(s) = v^*(s)$ for all $s \in S$.

4.6.3 Optimality of Markov deterministic policies in $\Pi^{\mathbf{HR}}$

What remains to be shown is that in (4.93), a deterministic decision rule attains the "sup" over randomized decision rules and that u_n depends on $h^n = (h^{n-1}, a^n, s^n)$ only through s^n. This is accomplished through two propositions. The first is an immediate consequence of the second part of Lemma 2.1 and its proof is omitted.

Proposition 4.2. Suppose for each $s \in S$, A_s is finite, and $u'_n(h^n), h^n \in H_N$ is computed by Algorithm 4.5. Then

$$\begin{aligned} u'_n(h^n) &= \sup_{w \in \mathscr{P}(A_{s^n})} \left\{ \sum_{a \in A_{s^n}} w(a) \sum_{j \in S} p_n(j|s^n, a)(r_n(s^n, a, j) + u'_{n+1}(h^{n+1})) \right\} \\ &= \max_{a \in A_{s^n}} \left\{ \sum_{j \in S} p_n(j|s^n, a)(r_n(s^n, a, j) + u'_{n+1}(h^{n+1})) \right\}. \end{aligned} \tag{4.94}$$

Proposition 4.3. Suppose for each $s \in S$, A_s is finite, $u_n(s)$ is computed by Algorithm 4.3, and $u'_n(h^n), h^n \in H_n$ is computed by Algorithm 4.5.

Then for $n = 1, \ldots, N-1$ and each $h^n = (h^{n-1}, a^n, s^n) \in H_n$, $u'_n(h^n)$ depends on h^n only through s^n. In other words, $u'_n(h^n) = u_n(s^n)$.

Proof. The proof is by induction. By construction, $u'_N(h^N) = r_N(s^N) = u_N(s^N)$. Assume now that $u'_t(h^t) = u_t(s^t)$ for $t = n+1, \ldots, N$. By the induction hypothesis and (4.94),

$$\begin{aligned} u'_n(h^n) &= \max_{a \in A_{s^n}} \left\{ \sum_{j \in S} p_n(j|s^n, a)(r_n(s^n, a, j) + u'_{n+1}(h^{n+1})) \right\} \\ &= \max_{a \in A_{s^n}} \left\{ \sum_{j \in S} p_n(j|s^n, a)(r_n(s^n, a, j) + u_{n+1}(j)) \right\} \end{aligned} \tag{4.95}$$

where $h^n = (h^{n-1}, a^n, s^n)$. Observe that all the quantities on the right hand side of (4.95) depend on h^n only through s^n. Hence, $u'_n(h^n) = u_n(s^n)$, the induction hypothesis holds and the result follows. □

The essence of Proposition 4.2 is that randomization is not necessary when there are a finite number of actions. Proposition 4.3 states that only the current state matters if the transition probabilities, reward functions, and especially the terminal reward do not depend on states prior to the current state. Together, these two results establish the following equalities:

$$\sup_{\pi \in \Pi^{\mathrm{HR}}} v_n^\pi(h^n) = \max_{\pi \in \Pi^{\mathrm{HD}}} v_n^\pi(h^n) = \max_{\pi \in \Pi^{\mathrm{MD}}} v_n^\pi(s^n). \tag{4.96}$$

Thus, combining Propositions 4.1, 4.2 and 4.3 provides a proof of Theorem 4.2. As a consequence of Theorem 4.2, a Markovian deterministic policy is optimal over the set of all history-dependent randomized policies. This justifies the restriction to Markovian deterministic policies at the beginning of this chapter. More formally

$$v^*(s) = \max_{\pi \in \Pi^{\mathrm{MD}}} v^\pi(s)$$

for all $s \in S$ and

$$v^*(s) \geq v^\pi(s)$$

for all $\pi \in \Pi^{\mathrm{HR}}$.

Bibliographic remarks

The use of the backwards induction to find optimal policies in a general Markov decision process model originates with Bellman [1957], where on page 87 he also states the

Principle of Optimality. Precursors in specific application areas include Massé's work on reservoir management (see Gessford and Karlin [1958]) and Wald's research on sequential analysis [Wald, 1947]. See Chapter 4 in Puterman [1994] for a more thorough discussion of this background.

Derman [1970] provides the first rigorous treatment in book form of the existence of optimal Markovian deterministic policies in the class of history-dependent policies. His approach differs considerably from that in this chapter. His book is well worth reading for those interested in a concise exposition of Markov decision process theory.

For additional reading on relevant fundamentals in probability (e.g., nested conditional expectations) and queuing (e.g., steady state equations of the $M/M/1$ queue), the reader is referred to Feller [1982] and Kleinrock [1975].

The identification of structured optimal policies dates back at least to Scarf [1960], who proves the optimality of (s, S) policies in a periodic review inventory model with fixed ordering costs. Other examples include monotone optimal policies in queuing models, control limit policies in replacement problems and structured policies in clinical applications. Interpretability has become increasingly important in machine learning, see, for example, Rudin [2019].

The proof of optimality of monotone policies in queuing service rate control follows Lippman [1975]. That result can also be proved using submodularity, see Section 4.4.2 of Puterman [1994]. The use of submodularity in Markov decision processes originates with the elegant paper of Serfozo [1976]. As noted in Chapter 3, the online dating problem originates with Cayley [1875]. The analysis herein is based on Bather [1982].

The concept of q-functions originates in the reinforcement learning literature, especially in the seminal paper by Watkins and Dayan [1992]. Our development follows Bertsekas [2012].

Exercises

1. Consider the variant of the two-state model analyzed in Section 4.1.2. Propose a randomized policy and evaluate it using backwards induction and sample path enumeration.

2. Find an optimal policy for the model in Example 2.2 for $N = 2$ by enumerating and evaluating all Markovian deterministic policies using Algorithm 4.1.

3. Find an optimal policy for the model in Example 2.2 for $N = 4$ and $N = 5$ using Algorithm 4.3. For the same values of N, determine the smallest positive value of $r_N(s)$ for each s that would result in a different optimal policy.

4. Prove that the values of $u_n(h^n)$ computed using Algorithm 4.2 are equal to $v_n^\pi(h^n)$ for all $h^n \in H_n$ and $n = 1, \ldots, N$.

5. Consider a model in which $S = \{s_0, s_1\}$, $A_{s_0} = \{a = 1, 2, \ldots\}$, $A_{s_1} = \{b\}$, $r(s_0, a) = 1 - 1/a$, $r(s_1, b) = 0$, $p(s_1|s_0, a) = 1$ and $p(s_1|s_1, b) = 1$.

(a) Represent the model graphically as in Figure 2.7.

(b) Show that an optimal policy does not exist.

(c) Show that for any $\epsilon > 0$, there is a policy π^ϵ for which

$$v^{\pi^\epsilon}(s_0) \geq v^*(s_0) - \epsilon.$$

6. Prove in general that for any $\epsilon > 0$ there always exists an ϵ-optimal policy.

7. Consider the model proposed in Exercise 1 in Chapter 2.

   (a) Evaluate the expected total reward of the policy that uses action $a_{i,1}$ in each state in a 5-period model.

   (b) Find an optimal policy and its value in a 5-period version of this model.

   (c) Express the optimality equations in terms of q-functions.

8. For Example 2.2 with $N = 2$, use Algorithm 4.2 to evaluate the history-dependent randomized policy in Section 2.5.2 with $r_2(s_1) = r_2(s_2) = 0$ and parameter values $q_{1,1} = 0.8, q_{1,2} = 0.4$, $q_{2,1} = 0.1, q_{2,5} = 0.5, q_{2,2} = 0.2, q_{2,3} = 0.3, q_{2,4} = 0.4$ and $q_{2,6} = 0.6$

9. Consider the queuing service rate control model in Section 4.3.1. In that example, the terminal cost was $r_N(s) = 0$ in all states so that the optimal policy was to use the least costly service rate in all states at the last decision epoch. To further investigate the impact of the terminal cost, repeat the analysis with linear, quadratic and cubic terminal costs. Discuss how the terminal cost impacts optimal policies.

10. Solve the queuing service rate control problem from Section 4.3.1 with the following changes to the parameters: a horizon length of $N = 10$, a delay cost of $f(s) = \sqrt{s}$ and a serving cost of $m(a) = 5a^3$. Comment on the structure of the optimal policy compared to the policies depicted in that section.

11. Solve the restaurant revenue management problem (Exercise 15 from Chapter 3) over a two-hour planning horizon. Clearly state and graphically represent the optimal policy.

12. Consider a 5-period version of the periodic inventory review problem of Section 3.1 with a finite warehouse capacity of 10 units, no backlogging, a fixed ordering cost of \$12, a per unit ordering cost of \$2, revenue of \$5 per item sold, and a holding cost of \$1 per unit per period. Assume a scrap value of \$1 per item. If demand cannot be fulfilled by stock on hand after receipt of an order, it is lost. Assume demand is geometrically distributed with $p = 0.3$.

    (a) Solve the problem assuming there is no delay between when the order is placed and it arrives. Does the optimal policy have any obvious structure?

(b) Solve the problem assuming than when an order is placed and it arrives at the end of the period after demand is met from stock on hand.

13. Consider the lion hunting problem from Section 3.5 with a horizon of $N = 30$. Assume the lion only hunts gazelles. On a day it hunts, the lion catches a young gazelle with probability 0.2 and edible biomass of 8kg, an adult gazelle with probability 0.1 and edible biomass of 15kg, or nothing with probability 0.7. Assume the following additional parameter values: $C = 30, d = 6, h = 7, c_0 = 6$. Formulate and solve this problem.

14. Consider the network shown in Figure 4.7.

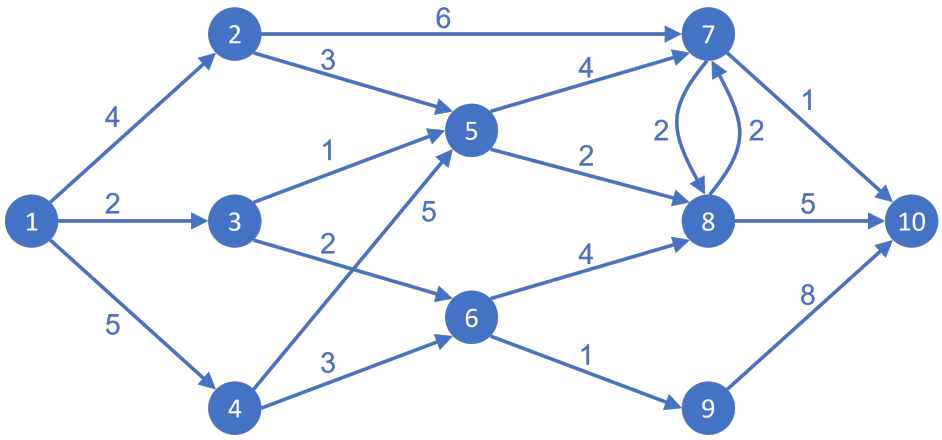


Figure 4.7: Network for Problem 14.

(a) Formulate the shortest path problem on this network as a finite-horizon Markov decision process where the states are the nodes and the actions are what arc to follow in each node. Identify all shortest paths from node 1 to node 10 using backwards induction. Note that the use of periods in this example is artificial.

(b) This example has multiple shortest paths. Which would you prefer and why?

(c) Find the longest path from node 1 to node 10 using backwards induction. In what application area might this be a useful problem to solve?

(d) Consider a modified network in which the arc from node 8 to node 7 has been deleted. Solve a stochastic version of this problem in which there is uncertainty about arc choice: if a node has two outgoing arcs, the chosen arc is followed with probability 0.8; if a node has three outgoing arcs, the chosen arc is followed with probability 0.6 and each of the remaining arcs is chosen with probability 0.2.

(e) Repeat the previous question but with the arc from node 8 to node 7 included.

15. Consider the "Pulling the goalie" model in Section 3.9.1.

    (a) Using the data at the end of that section, determine the optimal policy for when Team A should pull its goalie. Assume decisions are made every 20 seconds and that Team A first considers this decision when there are 5 minutes remaining.

    (b) Repeat this analysis for the situation when Team A's opponent, Team B, has a player in the penalty box.

16. Solve the single machine maintenance problem (Exercise 12 in Chapter 3), assuming it is a finite horizon problem with $N = 5$, using the following data: $r = 100$, $p(i) = (1 - e^{-i})/(1 + e^{-i})$, $c_A = 60$, $c_B = 20$.

17. Solve the house selling problem from Section 3.8.2 with $N = 7$, $f_n(s) = 0.001s$ for all n and $L_n(s) = 0.05s$ for all n. Assume the initial list price of the house is \$1M and that the best offer on the first day follows a discrete uniform distribution centered on the list price with a range of \$100K in \$10K increments. After the first day, assume the best offer follows a similar discrete uniform distribution, but centered at the best offer from the previous day.

18. Consider the house selling problem in a hot housing market. The problem parameters are the same as in Exercise 17, except that the discrete uniform distribution of the best offer on day $n > 1$ is centered at the best offer seen so far since the start of the horizon.

19. Consider the house selling problem in an inflationary market. The central bank is considering a one-time interest rate increase, which will depress the offer values for the house by 5% compared to no rate increase. On each day, there is a probability 0.1 that interest rates will increase, as long as there was no previous increase. Modify the formulation accordingly and solve it using the parameters from Exercise 17.

20. Formulate and solve a variant of the online dating model of Section 4.3.3 in which the objective is to maximize the probability of selecting one of the two best matches.

21. Consider the queuing admission control problem from Section 4.5.5. Derive the optimality equations for the value and state-action value functions for the case of $j = 0$.

22. Consider a finite horizon version of the queuing admission control problem from Section 4.5.5 with $N = 10$. Assume $b = 0.2$, $w = 0.2$, $f(j) = 5j$, and $R = 30$. At the final epoch, assume any remaining jobs in the system are penalized at a cost of 10 per job.

    (a) Solve this problem using both the value function optimality equations and the state-action value function optimality equations.

    (b) Compare and discuss the computational efforts in each approach.

23. Consider the machine maintenance model in Section 4.4.2.

    (a) Verify the result that a control-limit policy is optimal by numerically solving the problem over a range of parameter values of your choosing. It is easiest to consider instances in which the holding costs and transition probabilities are stationary.

    (b) *Provide conditions on $f_n(s)$ and p_n that ensure that $0 \leq s_n \leq M - 1$ for all n where s_n is defined in (4.52).

    (c) *Provide conditions on the model components that ensure that the control limit is monotone in the decision epoch n.

24. Express the state-action value function recursion that replaces (4.63) when evaluating a fixed Markovian *randomized* policy.

25. Prove Theorem 4.1 for Markovian randomized policies and history-dependent randomized policies.

Chapter 5

Infinite Horizon Models: Expected Discounted Reward

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

I early inquired the rate of interest on invested money, and worried my child's brain into an understanding of the virtues and excellencies of that remarkable invention of man, compound interest[1].

Jack London, American author, 1876-1916.

Chapter 5 concerns *infinite horizon Markov decision processes under the expected discounted reward criterion*. To avoid the above long-winded expression, they are referred to simply as *discounted models*.

The discounted model provides a gold standard for theory and algorithms in infinite horizon models. Unlike undiscounted models, most results do not depend on the underlying Markov chain structure. This is because the discount factor $0 \leq \lambda < 1$ dampens out the limiting behavior of the system since $\lambda^n \to 0$ and $n \to \infty$. Consequently, theory and methods draw from linear algebra and analysis, requiring little probability theory.

As noted in Section 2.3.2, discounting is usually interpreted as incorporating the time value of money when making decisions: a reward of one dollar next year is equivalent to $0 \leq \lambda < 1$ dollars today. You may already be familiar with the concept of an *interest rate*, which accounts for the time value of money in the forward direction. If you invest one dollar today and the (risk-less) annual interest rate equals i, then you

[1] London [1906].

would have $1+i$ dollars next year, $(1+i)^2$ dollars in two years, etc. This phenomenon is referred to as compounding. In relation to interest, the discount rate is $\lambda = 1/(1+i)$. That is, receiving one dollar a year from now is equivalent to investing λ dollars now. In other words the discount factor accounts for the time value of money in the backwards direction. See Figure 5.1. Note also that discounted models are equivalent to infinite horizon models with the expected total reward criterion that terminate at an independent geometric stopping time with parameter $1-\lambda$.

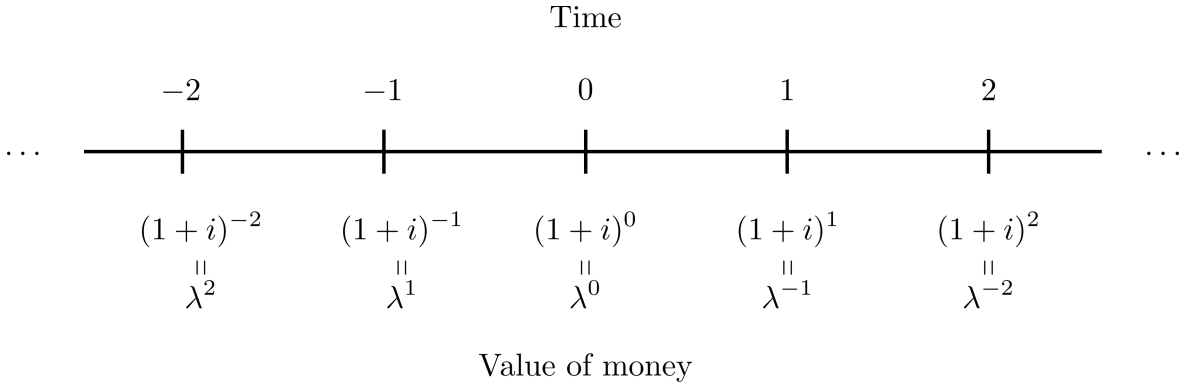


Figure 5.1: This diagram depicts the relationship between compound interest and discounting. From the compound interest perspective, \$1 invested at time 0 at interest rate i will increase to $(1+i)^2$ two-periods in the future ($t=2$). Conversely, receiving \$1 now is equivalent to investing λ^2 dollars at time $t=-2$ or in other words, an investor would be indifferent between receiving 1 two-periods in the future and possessing λ^2 now.

This chapter unfolds as follows. In several steps, culminating with establishing the existence and uniqueness of the solution to the Bellman equation, it establishes the optimality of stationary deterministic policies in the class of history-dependent randomized policies. The chapter then analyzes three classes of iterative algorithms:

- Value iteration
- Policy iteration
- Modified policy iteration

In each case, it describes the algorithm and its variants, provides stopping rules, proves convergence and presents illustrative examples. It then explores the equivalence between discounted Markov decision processes and linear programming. Finally it revisits several applications and shows how to extend the results on structured optimal policies in Section 4.4 to discounted models.

This and subsequent chapters use vector and matrix notation and rely on vector space concepts. We encourage the reader to carefully go through this chapter before

studying other optimality criteria, approximate dynamic programming, simulation-based methods, and reinforcement learning.

5.1 Preliminaries

This section introduces notation and basic recursions.

5.1.1 Reward functions

Recall that the reward function $r_n(s, a, j)$ equals the reward received between decision epoch n and $n + 1$ when action a is chosen in state s at decision epoch n, and a transition occurs to state j prior to decision epoch $n + 1$. In infinite horizon models under an expected reward based criterion, equations may appear more familiar and transparent if they are expressed in terms of $r_n(s, a)$ instead of $r_n(s, a, j)$. This may also be the case when the reward does not depend on j such as in the queuing control models in Section 3.4. When $r_n(s, a, j)$ is a model primitive, the expected reward from choosing action a in state s at decision epoch n can be written

$$r_n(s, a) = \sum_{j \in S} r_n(s, a, j) p_n(j|s, a). \tag{5.1}$$

This calculation in the two-state model in Section 2.5 as follows.

Example 5.1. Assume the rewards and probabilities are stationary. Then,

$$\begin{aligned} r(s_1, a_{1,1}) &= r(s_1, a_{1,1}, s_1)p(s_1|s_1, a_{1,1}) + r(s_1, a_{1,1}, s_2)p(s_2|s_1, a_{1,1}) \\ &= 5 \cdot 0.8 - 5 \cdot 0.2 = 3 \end{aligned}$$

The remaining rewards can be calculated similarly and shown to be given by $r(s_1, a_{1,2}) = 5$, $r(s_2, a_{2,1}) = -5$ and $r(s_2, a_{2,2}) = 2$.

This change in notation is primarily for convenience; it does not affect the principles of our development of infinite horizon Markov decision processes. One consequence of this change in the form of the reward function is that the recursions and Bellman equation will look slightly different from the ones presented in the previous chapter. The chapters on simulation methods (Chapters 10 and 11) retain $r_n(s, a, j)$ as a primitive because simulation replaces expectations by simulated sample paths.

5.1.2 Key assumptions

In addition to the standard assumptions of i finite states and finite actions, the following additional assumptions hold throughout this chapter:

Stationary rewards: The reward function does not vary from epoch to epoch. It will be written as $r(s, a)$, independent of n.

Bounded rewards: There is a finite W such that $|r(s, a)| \leq W$ for all $a \in A_s, s \in S$.

Stationary transition probabilities: The transition probabilities will be written $p(j|s, a)$, independent of n.

Discounting: Future rewards are discounted by a multiplicative factor λ per period, where $0 \leq \lambda < 1$.

5.1.3 Markovian policies are sufficient

Chapter 4 (Theorem 4.2) showed that Markovian deterministic policies are optimal among the class of history-dependent randomized policies for finite horizon Markov decision process. A stronger result holds in the infinite horizon setting: stationary deterministic policies, a subset of Markovian deterministic policies, are optimal within the class of history-dependent randomized policies.

To simplify exposition, this chapter considers Markovian randomized policies instead of history-dependent randomized policies. The following result formally justifies this simplification. A proof of it appears in Appendix A at the end of this chapter. It is the consequence of technical Lemma 5.9, which establishes that given any history-dependent policy, for each initial state, there exists a Markovian randomized policy that has the same action selection probabilities at each decision epoch.

Theorem 5.1. For each $s \in S$, given any policy $\pi = (d_1, d_2, \ldots) \in \Pi^{\mathrm{HR}}$, there exists a policy $\pi' = (d'_1, d'_2, \ldots) \in \Pi^{\mathrm{MR}}$ with the same expected discounted reward conditional on $X_1 = s$.

5.1.4 Matrix and vector representation for stationary policies

To streamline results and simplify many proofs and formulae, value functions and rewards will often be represented by (column) vectors and transition probabilities by matrices. Vectors and matrices are denoted by **bold** symbols. Key notation is defined below, with all definitions assuming S finite. All concepts can be extended to S countable, continuous or abstract, but such generality will not be needed here.

Matrix and vector notation

V: Vector or linear space[a] of real-valued $|S|$-dimensional vectors.

$\mathbf{v}$: $|S|$-dimensional real-valued vector usually representing a value function or an approximation.

$\mathbf{r}_d$: $|S|$-dimensional vector of rewards corresponding to $d \in D^{\text{MR}}$ (see equation (5.2)).

$\mathbf{e}$: Vector with all components equal to one.

$\mathbf{e}_s$: Vector with a 1 in the s-th component and zeroes elsewhere.

$\mathbf{0}$: Vector with all components equal to zero.

$\mathbf{P}_d$: $|S| \times |S|$-dimensional transition probability matrix corresponding to $d \in D^{\text{MR}}$ (see equation (5.3)).

$\mathbf{I}$: $|S| \times |S|$-dimensional identity matrix.

[a]For our purposes a vector or linear space is a set of elements closed under addition and scalar multiplication. In addition, several other technical properties hold.

Note that $\mathbf{v}$, $\mathbf{r}_d$, $\mathbf{e}$, $\mathbf{e}_s$ and $\mathbf{0}$ are elements of V and the matrices $\mathbf{P}_d$ and $\mathbf{I}$ map $V \to V$.

Some further notational conventions follow. Be sure you understand them before proceeding through the technical parts of this chapter. Assume for concreteness that $S = \{s_1, \ldots, s_M\}$ is M-dimensional. The vector $\mathbf{v}$ has components $v(s_1), \ldots, v(s_M)$. When written inline, $\big(v(s_1), \ldots, v(s_M)\big)$ will denote the column vector $\mathbf{v}$. The transpose of any vector $\mathbf{v}$ will be written $\mathbf{v}^\mathsf{T}$. So $(v(s_1), \ldots, v(s_M))^\mathsf{T}$ represents a row vector. Note that components of a vector are *not* expressed in bold. However, when multiplying a vector $\mathbf{v}$ by a matrix $\mathbf{P}_d$, $\mathbf{P}_d\mathbf{v}(s_k)$ denoted the s_k-th component of the resulting vector. Notice that the vector $\mathbf{v}$ remains bold here, since the quantity of interest is not the s_k-th component of $\mathbf{v}$, but of $\mathbf{P}_d\mathbf{v}$.

Operators

We find it convenient, transparent and elegant to use *operators* to express fundamental Markov decision process relationships. From the perspective of this book, an operator $T : V \to V$ is a (possibly nonlinear) function that assigns the value $T\mathbf{v} \in V$ to each vector $\mathbf{v} \in V$. When V is M-dimensional, $T\mathbf{v}$ has M components. Moreover, in contrast to how functions are written as $f(x)$, parentheses are not used when writing

$T\mathbf{v}$[2]. Define $T\mathbf{v}(s)$ as the s-th component of $T\mathbf{v}$ and note that $T^n\mathbf{v} = T\cdots T\mathbf{v}$.

Rewards and transition probabilities corresponding to Markovian policies

Next, consider $\mathbf{r}_d$ and $\mathbf{P}_d$ for deterministic and randomized Markovian decision rules. The s-th component of $\mathbf{r}_d$, denoted by $r_d(s)$, satisfies

$$r_d(s) = \begin{cases} r(s, d(s)) & d \in D^{\mathrm{MD}} \\ \sum_{a\in A_s} w_d(a|s) r(s,a) & d \in D^{\mathrm{MR}}. \end{cases} \tag{5.2}$$

Similarly, the (s,j)-th component of $\mathbf{P}_d$ denoted by $P_d(j|s)$, equals

$$P_d(j|s) = \begin{cases} p(j|s, d(s)) & d \in D^{\mathrm{MD}} \\ \sum_{a\in A_s} w_d(a|s) p(j|s,a) & d \in D^{\mathrm{MR}}. \end{cases} \tag{5.3}$$

Note that if the model had been formulated in terms of $r(s,a,j)$, $\mathbf{r}_d$ would be replaced by a matrix[3].

Given a policy $\pi = (d_1, d_2, \ldots) \in \Pi^{\mathrm{MR}}$, the n-step transition probability matrix $\mathbf{P}_\pi^n$ is

$$\mathbf{P}_\pi^n := \mathbf{P}_{d_1}\mathbf{P}_{d_2}\cdots\mathbf{P}_{d_n} \tag{5.4}$$

with its (s,j)-th component representing $p^\pi(X_n = j | X_1 = s)$.

Since V is a vector space, it is closed under addition and scalar multiplication, so that for $\mathbf{v} \in V$, $\mathbf{P}_d\mathbf{v} \in V$, $\lambda\mathbf{P}_d\mathbf{v} \in V$, and most importantly, $\mathbf{r}_d + \lambda\mathbf{P}_d\mathbf{v} \in V$.

5.1.5 Partial orders

To express the Bellman equation in vector notation, the "max" over a set of vectors must be defined carefully. Since the definition of a vector space does not include any notion of an ordering of vectors, the concept of a *partial order* is needed, in particular the *component-wise* partial order. This means that $\mathbf{v} \geq \mathbf{u}$ if $v(s) \geq u(s)$ for all $s \in S$.

Given two vectors, $\mathbf{v}$ and $\mathbf{u}$, there are four possibilities: $\mathbf{v} \geq \mathbf{u}$, $\mathbf{u} \geq \mathbf{v}$, $\mathbf{u} = \mathbf{v}$ or they are not comparable. An example of vectors that are not comparable are $\mathbf{v} = (1,2)$ and $\mathbf{u} = (2,1)$, since they are not equal, nor is one larger (component-wise) than the other. Because there exist incomparable vectors, this is a partial order, rather than a total order.

For the component-wise partial order, a corresponding component-wise maximum is needed.

[2]This approach to expressing operators generalizes the approach used to indicate a product of a matrix (a linear operator) and a vector. If $\mathbf{A}$ is a matrix and $\mathbf{v}$ a vector, it is standard to write $\mathbf{A}\mathbf{v}$ instead of $\mathbf{A}(\mathbf{v})$.

[3]If $d \in D^{\mathrm{MD}}$, $\mathbf{R}_d$ would represent the matrix with components $r(s, d(s), j)$ with an obvious generalization to $d \in D^{\mathrm{MR}}$. This representation will not be used in the sequel.

Definition 5.1. The *component-wise maximum* of a set of vectors $\mathbf{v}_1, \ldots, \mathbf{v}_K$ is a vector $\mathbf{v}^*$ that satisfies $\mathbf{v}^* \geq \mathbf{v}_k$ for $k = 1, \ldots, K$ and for all $s \in S$, $v^*(s) = v_k(s)$ for at least one $k = 1, \ldots, K$. Let c-max denote the component-wise maximum, written as

$$\mathbf{v}^* := \text{c-max}\{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_K\} \tag{5.5}$$

If vector $\mathbf{v}_{k^*}$ achieves the component-wise maximum, then

$$k^* \in \arg\underset{k=1,\ldots,K}{\text{c-max}}\{\mathbf{v}_k\} \tag{5.6}$$

Note that the component-wise maximum vector may not coincide with any of the vectors over which the component-wise maximum is taken, as the following example shows.

Example 5.2. Suppose $\mathbf{v}_1 = (2, 1)$, $\mathbf{v}_2 = (1, 3)$, and $\mathbf{v}_3 = (1, 5)$. Then $\mathbf{v}^* = \text{c-max}\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\} = (2, 5)$. Observe that $\mathbf{v}^* \notin \{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$.

Where the component-wise maximum in a Markov decision process model is used, the situation in Example 5.2 cannot occur. In particular, c-max is used when taking the maximum of values defined over the set of all deterministic decision rules, where each decision rule is a vector of actions in $|S|$-dimensional space. Thus, the set of vectors over which the c-max is taken will correspond to the Cartesian product of the possible actions in all states, which will coincide with one of the decision rules.

Example 5.3. Consider a two-dimensional example where the first component can assume values 1 and 2 and the second component can assume values 3 and 5. The set of vectors generated by the Cartesian products of these component values is

$$A = \{(1, 3), (1, 5), (2, 3), (2, 5)\}.$$

The c-max of this set is (2,5), which is an element of A.

It is worth emphasizing that the component-wise maximum, c-max*, can be found by maximizing over each component separately, thus avoiding the enumeration of all possible combinations of component values.* This observation is especially important in Markov decision process computation, as shown later.

5.1.6 Norms

To establish the convergence of a sequence of iterates of a computational algorithm, a notion of distance in a vector space is required.

Definition 5.2. For $\mathbf{v} \in V$, define the *norm* of $\mathbf{v}$ by

$$\|\mathbf{v}\| := \max_{s \in S} |v(s)|. \tag{5.7}$$

This norm is often called the *sup-norm*[4]. It assigns as a "length" to $\mathbf{v}$ the absolute value of its largest component. For example, if $\mathbf{v} = (-2, 1)$, $\|\mathbf{v}\| = 2$. This norm applies equally well to non-finite dimensional state spaces where it is appropriate to replace "max" in (5.7) by a supremum (hence the name sup-norm). That degree of generality is not required here. There are many other norms (such as the Euclidean norm) that could be assigned to vectors in V, but in most of this book only the sup-norm is used.

The sup-norm on V induces a norm on $|S| \times |S|$ matrices $\mathbf{Q} : V \to V$ through the following definition.

Definition 5.3. For $\mathbf{Q} \in V \times V$, define the *matrix norm* of $\mathbf{Q}$ by

$$\|\mathbf{Q}\| := \sup_{\|\mathbf{v}\|=1} \|\mathbf{Q}\mathbf{v}\| = \max_{s \in S} \sum_{j \in S} |q(s, j)|. \tag{5.8}$$

The following easily proved result provides a useful inequality relating matrix and vector norms.

Lemma 5.1. For $\mathbf{v} \in V$ and matrix $\mathbf{Q} : V \to V$,

$$\|\mathbf{Q}\mathbf{v}\| \leq \|\mathbf{Q}\|\|\mathbf{v}\|. \tag{5.9}$$

The following definition combines the concepts of partial order and norm with a vector space.

Definition 5.4. A vector space V together with a partial order $\geq$ and a norm $\|\cdot\|$ is referred to as a *partially ordered normed linear space*.

This book (except Chapter 8) focuses on the vector space V of real-valued $|S|$-dimensional vectors with component-wise partial order and sup-norm.

[4]The sup-norm is often written as $\|\cdot\|_\infty$, to distinguish it from other norms such as the Euclidean norm, which is typically written as $\|\cdot\|_2$. For convenience, the subscript on the sup-norm is omitted.

5.2 The expected discounted reward of a policy

Definition 5.5. The *expected discounted reward* of policy $\pi \in \Pi^{\text{HR}}$ starting in state s with discount factor λ is[a]

$$v_\lambda^\pi(s) := \lim_{N\to\infty} E^\pi\left[\sum_{n=1}^{N} \lambda^{n-1} r(X_n, Y_n) \,\middle|\, X_1 = s\right]. \tag{5.10}$$

[a]Recall Y_n is a random variable that refers to the action chosen at decision epoch n, where it may be the result of a deterministic decision rule d applied to state X_n, or a randomized decision rule distributed according to $w_d(\cdot|X_n)$.

Below are two equivalent representations for $v_\lambda^\pi(s)$. Be sure to note the subtle distinctions between them. As a result of assuming bounded rewards and discount factor $0 \le \lambda < 1$, the bounded convergence theorem[5] justifies interchanging the limit and the expectation above to obtain

$$v_\lambda^\pi(s) = \lim_{N\to\infty} E^\pi\left[\sum_{n=1}^{N} \lambda^{n-1} r(X_n, Y_n) \,\middle|\, X_1 = s\right] = E^\pi\left[\sum_{n=1}^{\infty} \lambda^{n-1} r(X_n, Y_n) \,\middle|\, X_1 = s\right]. \tag{5.11}$$

Since the expectation of a sum equals the sum of expectations it follows again from the bounded convergence theorem that

$$v_\lambda^\pi(s) = \lim_{N\to\infty} \sum_{n=1}^{N} \lambda^{n-1} E^\pi\left[r(X_n, Y_n) \,|\, X_1 = s\right] = \sum_{n=1}^{\infty} \lambda^{n-1} E^\pi\left[r(X_n, Y_n) \,|\, X_1 = s\right]. \tag{5.12}$$

The representation on the right hand side of (5.12) will provide the basis for most of the following development.

5.2.1 Insights into how the expected discounted reward is evaluated

This section shows why the second representation for $v_\lambda^\pi(s)$ is so important, by looking into the expectations on the right hand side of (5.12) in depth. For any n,

$$E^\pi[r(X_n, Y_n) \,|\, X_1 = s] = \sum_{j\in S} \sum_{a\in A_j} r(j,a) p^\pi(X_n = j, Y_n = a | X_1 = s), \tag{5.13}$$

[5]The bounded convergence theorem states that if $|f_n(s)| \le W$ for all $s \in S$, $n \ge 1$ and some finite value W, then

$$\lim_{n\to\infty} \int_S f_n(s) ds = \int_S \lim_{n\to\infty} f_n(s) ds.$$

When S is finite sums replace integrals.

where p^π is a probability distribution incorporating both the transition probabilities and the action randomization distribution (see Section 2.2.3). Thus, combining (5.12) and (5.13),

$$v_\lambda^\pi(s) = \sum_{n=1}^{\infty} \sum_{j \in S} \sum_{a \in A_j} \lambda^{n-1} r(j,a) p^\pi(X_n = j, Y_n = a | X_1 = s). \tag{5.14}$$

As a consequence of Theorem 5.1, focus can be restricted to $\pi = (d_1, d_2, \ldots) \in \Pi^{\mathrm{MR}}$ without loss of generality.

To gain insight into equation (5.14), the first three terms in the summation over n are written explicitly:

$$n = 1: \quad \sum_{a \in A_j} r(s,a) w_{d_1}(a|s) \tag{5.15}$$

$$n = 2: \quad \lambda \sum_{j \in S} \sum_{a' \in A_j} r(j,a') w_{d_2}(a'|j) \sum_{a \in A_s} p(j|s,a) w_{d_1}(a|s) \tag{5.16}$$

$$n = 3: \quad \lambda^2 \sum_{k \in S} \sum_{a'' \in A_j} r(k,a'') w_{d_3}(a''|k) \sum_{j \in S} \sum_{a' \in A_j} p(k|j,a') w_{d_2}(a'|j) \sum_{a \in A_s} p(j|s,a) w_{d_1}(a|s). \tag{5.17}$$

To derive the first term, note that states $j \neq s$ will have probability zero and $p^\pi(X_1 = s, Y_1 = a | X_1 = s) = p^\pi(Y_1 = a | X_1 = s) = w_{d_1}(a|s)$ by definition. Thus, the first term in the sum is simply the expected reward, where the expectation is with respect to the conditional probability of actions in decision epoch 1 given state s.

Using the law of total probability and the Markov property, the second term follows from

$$\begin{aligned} p^\pi(X_2 = j, Y_2 = a' | X_1 = s) &= p^\pi(Y_2 = a' | X_2 = j, X_1 = s) p^\pi(X_2 = j | X_1 = s) \\ &= w_{d_2}(a'|j) p^\pi(X_2 = j | X_1 = s) \\ &= w_{d_2}(a'|j) \sum_{a \in A_s} p^\pi(X_2 = j | X_1 = s, Y_1 = a) p^\pi(Y_1 = a | X_1 = s) \\ &= w_{d_2}(a'|j) \sum_{a \in A_s} p(j|s,a) w_{d_1}(a|s). \end{aligned}$$

Thus, (5.16) represents the expected discounted reward in decision epoch 2, where the expectation is with respect to the joint probability distribution of state and action at decision epoch 2 conditional on starting in state s at decision epoch 1. This probability distribution comprises both the distribution of actions that results from using the Markovian randomized decision rule d_2 in state j at decision epoch 2 and the distribution governing the transition from state s in epoch 1 to state j in epoch 2. Furthermore, the latter distribution is itself a product of the transition probabilities out of state s in epoch 1 for the possible actions in epoch 1 and the action randomization probabilities associated with decision rule d_1.

The third term (5.17) can be derived similarly and is left as an exercise. Its interpretation is similar to the previous term, but extended for one more decision epoch.

Simulating the discounted reward of a Markovian randomized policy

To provide another perspective on equation (5.10) (or equivalently (5.14)), consider the use of Monte Carlo simulation to estimate the value of a Markovian randomized policy $\pi = (d_1, d_2, \ldots)$. More details on how to implement it and other approaches appear in Chapter 10.

A challenge when simulating discounted rewards is to account for the infinite number of terms in (5.14). This can be achieved through truncation or using geometric stopping times. Putting that issue aside until Chapter 10, assume the goal is to obtain an estimate of the expected discounted reward after truncating it to K terms. The algorithm is expressed in terms of $r(s, a, j)$ and assumes knowledge of $p(j|s, a)$.

Algorithm 5.1. Simulating a single replicate of an estimate of $v_\lambda^\pi(s)$ for fixed $s \in S$.

1. **Initialize:** Specify $\pi = (d_1, d_2, \ldots) \in \Pi^{\text{MR}}$, $s \in S$ and $K \in \mathbb{Z}_+$. Set $k \leftarrow 1$, $v \leftarrow 0$.
2. **Iterate:** While $k \leq K$:
   (a) Sample a from $w_{d_k}(\cdot|s)$.
   (b) Sample s' from $p(\cdot|s, a)$.
   (c) $v \leftarrow v + \lambda^{k-1} r(s, a, s')$.
   (d) $s \leftarrow s'$.
   (e) $k \leftarrow k + 1$.
3. **Terminate:** Return v.

Note that if the reward was not a function of the next state steps 2(b) and 2(c) could be interchanged, with the update replaced by $v \leftarrow v + \lambda^{k-1} r(s, a)$. By generating many sample paths in this way, an estimate of the *distribution* of $v_\lambda^\pi(s)$ can be obtained.

5.2.2 Representing value functions as vectors

This section expresses quantities in the previous section using in vector/matrix notation. Doing simplifies explanations and derivations below.

The first three terms in the sum over n in equation (5.14) are

$$n = 1: \quad r_{d_1}(s) \tag{5.18}$$

$$n = 2: \quad \lambda \sum_{j \in S} r_{d_2}(j) P_{d_1}(j|s) \tag{5.19}$$

$$n = 3: \quad \lambda^2 \sum_{j \in S} r_{d_3}(j) P_{d_2}(j|k) P_{d_1}(k|s). \tag{5.20}$$

These expressions are the s-th components of $\mathbf{r}_{d_1}$, $\lambda \mathbf{P}_{d_1} \mathbf{r}_{d_2}$, and $\lambda^2 \mathbf{P}_{d_1} \mathbf{P}_{d_2} \mathbf{r}_{d_3}$, respectively. The last term can be written equivalently[6] as $\lambda^2 \mathbf{P}_\pi^2 \mathbf{r}_{d_3}$ using the two-step transition probability matrix $\mathbf{P}_\pi^2 = \mathbf{P}_{d_1} \mathbf{P}_{d_2}$.

Letting $\mathbf{P}_\pi^0 = \mathbf{I}$, the above pattern suggests that $\mathbf{v}_\lambda^\pi$ has the following closed form representation:

$$\mathbf{v}_\lambda^\pi = \sum_{n=1}^{\infty} \lambda^{n-1} \mathbf{P}_\pi^{n-1} \mathbf{r}_{d_n} \tag{5.21}$$

This infinite sum converges because $||\mathbf{P}_\pi^n|| = 1$, $0 \leq \lambda < 1$ and

$$\|\mathbf{v}_\lambda^\pi\| \leq \max_{a \in A_s, s \in S} |r(s,a)| (1-\lambda)^{-1}.$$

Expanding this sum and grouping terms appropriately, yields

$$\mathbf{v}_\lambda^\pi = \mathbf{r}_{d_1} + \lambda \mathbf{P}_{d_1} \mathbf{r}_{d_2} + \lambda^2 \mathbf{P}_{d_1} \mathbf{P}_{d_2} \mathbf{r}_{d_3} + \cdots \tag{5.22}$$

$$= \mathbf{r}_{d_1} + \lambda \mathbf{P}_{d_1} \left(\mathbf{r}_{d_2} + \lambda \mathbf{P}_{d_2} \mathbf{r}_{d_3} + \cdots \right). \tag{5.23}$$

Thus, if $\pi' = (d_2, d_3, \ldots)$ is a new policy that uses the same decision rules as π from decision epoch 2 onwards,

$$\mathbf{v}_\lambda^\pi = \mathbf{r}_{d_1} + \lambda \mathbf{P}_{d_1} \mathbf{v}_\lambda^{\pi'}. \tag{5.24}$$

This equation can be interpreted as follows: the expected discounted reward associated with following policy π equals the immediate reward associated with using decision rule d_1 in the first epoch, plus the expected discounted reward over the infinite horizon, following policy π' from the second decision epoch onwards. Equivalently, this equation may be viewed as the expected value in a one-period problem, in which the terminal reward is the expected discounted reward associated with π'.

In component form, equation (5.24) can be written in general as

$$v_\lambda^\pi(s) = r_{d_1}(s) + \lambda \sum_{j \in S} P_{d_1}(j|s) v_\lambda^{\pi'}(j), \tag{5.25}$$

[6] Note the expression $\mathbf{P}_\pi^n$ denotes the n-step transition probability matrix corresponding to policy π. For stationary policies $\pi = d^\infty$, $\mathbf{P}_\pi^n$ equals the matrix power $\mathbf{P}_d^n$.

for all $s \in S$. If $\pi \in \Pi^{\text{MD}}$, then equation (5.24) becomes

$$v_\lambda^\pi(s) = r(s, d_1(s)) + \lambda \sum_{j \in S} p(j|s, d_1(s)) v_\lambda^{\pi'}(j) \tag{5.26}$$

for all $s \in S$. The subtle difference between (5.25) and (5.26) is that the former accommodates randomized decision rules (see equations (5.2) and (5.3)). Note that equation (5.26) is the discounted, infinite horizon generalization of equation (4.7) from the finite horizon setting with stationary rewards and transition probabilities.

Writing the above two expressions as expectations will be especially useful in Chapter 10. Regardless of whether π is deterministic or randomized

$$v_\lambda^\pi(s) = E^\pi[r(X_1, Y_1) + \lambda v_\lambda^{\pi'}(X_2)|X_1 = s]. \tag{5.27}$$

When the reward is a function of the next state as well, this expression becomes

$$v_\lambda^\pi(s) = E^\pi[r(X_1, Y_1, X_2) + \lambda v_\lambda^{\pi'}(X_2)|X_1 = s]. \tag{5.28}$$

5.2.3 Evaluating stationary policies

Stationary policies play an especially important role in discounted models. Restricting attention to such policies greatly simplifies the above expressions and more.

Let $d^\infty := (d, d, \ldots)$ denote the stationary policy that uses decision rule $d \in D^{\text{MR}}$ in every epoch. In this case, (5.21) becomes

$$\mathbf{v}_\lambda^{d^\infty} = \sum_{n=0}^{\infty} \lambda^n \mathbf{P}_d^n \mathbf{r}_d \tag{5.29}$$

and (5.24) becomes

$$\mathbf{v}_\lambda^{d^\infty} = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}_\lambda^{d^\infty}. \tag{5.30}$$

In other words, $\mathbf{v}_\lambda^{d^\infty}$ satisfies the system of linear equations. That it is the unique solution will now be established.

$$\mathbf{v} = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}. \tag{5.31}$$

A key lemma and its application

The following lemma, which is proved in the book Appendix (see Lemma B.1), is fundamental. It provides a matrix generalization of the representation for a convergent geometric series $\sum_{n=0}^{\infty} a^n = (1-a)^{-1}$ when $|a| < 1$.

Lemma 5.2. Let $\mathbf{Q}$ denote an $|S| \times |S|$ real-valued matrix for which $\mathbf{Q}^N \to \mathbf{0}$ as $N \to \infty$. Then the inverse of $\mathbf{I} - \mathbf{Q}$ exists and satisfies

$$(\mathbf{I} - \mathbf{Q})^{-1} = \sum_{n=0}^{\infty} \mathbf{Q}^n. \tag{5.32}$$

Applying it to (5.29) gives:

Theorem 5.2. Suppose $0 \leq \lambda < 1$ and $d \in D^{\mathrm{MR}}$. Then

$$\mathbf{v}_\lambda^{d^\infty} = (\mathbf{I} - \lambda \mathbf{P}_d)^{-1} \mathbf{r}_d \tag{5.33}$$

and is the unique solution of equation (5.31).

Proof. Let $d \in D^{\mathrm{MR}}$. Rewriting (5.31) yields

$$(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d. \tag{5.34}$$

As a result of Lemma 5.2, $\mathbf{I} - \lambda \mathbf{P}_d$ is invertible, since each row sum of $\lambda \mathbf{P}_d$ is strictly less than 1[7].

To establish uniqueness suppose $\mathbf{u}$ and $\mathbf{v}$ satisfy (5.34). Then $(\mathbf{I} - \lambda \mathbf{P}_d)(\mathbf{u} - \mathbf{v}) = \mathbf{0}$. That $\mathbf{v} = \mathbf{u}$ follows from the invertibility of $(\mathbf{I} - \lambda \mathbf{P}_d)$. □

The following example illustrates the computation of $\mathbf{v}_\lambda^{d^\infty}$ in two ways: direct solution of the system of linear equations (5.31) and using representation (5.33).

Example 5.4. Consider a discounted, infinite horizon version of the two-state model with a randomized stationary policy d^∞ that, in every epoch, chooses action $a_{1,1}$ with certainty in state s_1 and chooses state $a_{2,1}$ and $a_{2,2}$ with equal probability in state s_2.

Solving (5.31) directly: In component form, (5.31) becomes

$$v(s_1) = 5 \cdot 0.8 - 5 \cdot 0.2 + \lambda(0.8v(s_1) + 0.2v(s_2)) \tag{5.35}$$

$$v(s_2) = 0.5(-5 \cdot 1) + 0.5(20 \cdot 0.4 - 10 \cdot 0.6) + \lambda((0.5 \cdot 0 + 0.5 \cdot 0.4)v(s_1) + (0.5 \cdot 1 + 0.5 \cdot 0.6)v(s_2)), \tag{5.36}$$

[7] Alternatively, it suffices to show that the columns of $\mathbf{I} - \lambda \mathbf{P}_d$ are linearly independent. Suppose to the contrary that the columns are linearly dependent. That is, there exists $\mathbf{v} \neq \mathbf{0}$ such that $(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{0}$, which is equivalent to $\mathbf{v} = \lambda \mathbf{P}_d \mathbf{v}$. In other words, 1 is an eigenvalue of the matrix $\lambda \mathbf{P}_d$. From Theorem B.3 in the Appendix, since the largest eigenvalue of a transition probability matrix is 1, the largest eigenvalue of $\lambda \mathbf{P}_d$ is $\lambda < 1$, which is a contradiction.

which simplifies to

$$v(s_1) = 3 + \lambda(0.8v(s_1) + 0.2v(s_2)) \tag{5.37}$$
$$v(s_2) = -1.5 + \lambda(0.2v(s_1) + 0.8v(s_2)). \tag{5.38}$$

Solving for $v(s_1)$ and $v(s_2)$,

$$v_\lambda^{d^\infty}(s_1) = \frac{3(1-2.7\lambda)}{(1-\lambda)(1-0.6\lambda)}, \qquad v_\lambda^{d^\infty}(s_2) = \frac{3(0.6\lambda-0.5)}{(1-\lambda)(1-0.6\lambda)}. \tag{5.39}$$

Using (5.33): From (5.33):

$$\mathbf{I} - \lambda \mathbf{P}_d = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \lambda \begin{pmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{pmatrix} = \begin{pmatrix} 1-0.8\lambda & -0.2\lambda \\ -0.2\lambda & 1-0.8\lambda \end{pmatrix} \tag{5.40}$$

Then

$$(\mathbf{I} - \lambda \mathbf{P}_d)^{-1}\mathbf{r}_d = \frac{1}{(1-\lambda)(1-0.6\lambda)} \begin{pmatrix} 1-0.8\lambda & 0.2\lambda \\ 0.2\lambda & 1-0.8\lambda \end{pmatrix} \begin{pmatrix} 3 \\ -1.5 \end{pmatrix}, \tag{5.41}$$

which can easily be verified to be the same as (5.39).

Next, the expected discounted reward for the four deterministic stationary policies are computed using (5.33). Setting $\lambda = 0.9$, $d_1 = (a_{1,1}, a_{2,1})$, $d_2 = (a_{1,1}, a_{2,2})$, $d_3 = (a_{1,2}, a_{2,1})$ and $d_4 = (a_{1,2}, a_{2,2})$ yields

$$\mathbf{v}_\lambda^{d_1^\infty} = \begin{bmatrix} -21.429 \\ -50 \end{bmatrix}, \mathbf{v}_\lambda^{d_2^\infty} = \begin{bmatrix} 27.188 \\ 25.625 \end{bmatrix}, \mathbf{v}_\lambda^{d_3^\infty} = \begin{bmatrix} -40 \\ -50 \end{bmatrix} \text{ and } \mathbf{v}_\lambda^{d_4^\infty} = \begin{bmatrix} 30.147 \\ 27.941 \end{bmatrix}.$$

Observe that $\mathbf{v}_\lambda^{d_4^\infty} \geq \mathbf{v}_\lambda^{d_2^\infty} \geq \mathbf{v}_\lambda^{d_1^\infty} \geq \mathbf{v}_\lambda^{d_3^\infty}$ so that the stationary policy d_4^∞ is optimal within the class of stationary policies. This implies d_4^∞ is optimal, once it is established that stationary policies are optimal within the class of all policies, which is done below.

Properties of $(\mathbf{I} - \lambda \mathbf{P}_d)^{-1}$

From Lemma 5.2 it follows immediately that:

Theorem 5.3. Suppose $0 \leq \lambda < 1$ and $d \in D^{\mathrm{MR}}$, then

$$(\mathbf{I} - \lambda \mathbf{P}_d)^{-1} = \sum_{n=0}^{\infty} \lambda^n \mathbf{P}_d^n. \tag{5.42}$$

Several properties of $(\mathbf{I} - \lambda \mathbf{P}_d)^{-1}$ appear in the following lemma.

Lemma 5.3. Suppose $0 \leq \lambda < 1$ and $\mathbf{u}, \mathbf{v} \in V$. Then, for any $d \in D^{\mathrm{MR}}$:

1. If $\mathbf{u} \geq \mathbf{0}$, then $(\mathbf{I} - \lambda \mathbf{P}_d)^{-1}\mathbf{u} \geq \mathbf{u} \geq \mathbf{0}$.
2. If $\mathbf{u} \geq \mathbf{v}$, then $(\mathbf{I} - \lambda \mathbf{P}_d)^{-1}\mathbf{u} \geq (\mathbf{I} - \lambda \mathbf{P}_d)^{-1}\mathbf{v}$.
3. $(\mathbf{I} - \lambda \mathbf{P}_d)^{-1}\mathbf{e} = (1 - \lambda)^{-1}\mathbf{e}$.

Proof. From Theorem 5.3,

$$(\mathbf{I} - \lambda \mathbf{P}_d)^{-1}\mathbf{u} = \mathbf{u} + \lambda \mathbf{P}_d \mathbf{u} + \lambda^2 \mathbf{P}_d^2 \mathbf{u} + \ldots \geq \mathbf{u} \geq \mathbf{0}$$

since $\lambda \geq 0$ and all entries of $\mathbf{P}_d$ are non-negative. The second result follows by applying the first result to $\mathbf{u} - \mathbf{v}$. The third result follows from (5.42), the fact that $\mathbf{P}_d \mathbf{e} = \mathbf{e}$ and the formula for the sum of a geometric series. □

5.3 Optimal policies and the Bellman equation

This section defines optimal policies for discounted models, introduces the Bellman (optimality) equation and shows that the existence of a solution to the Bellman equation implies the existence of an optimal policy that is stationary.

5.3.1 Optimal policies

Definitions of optimal policies and optimal value functions for infinite horizon discounted models follow.

Definition 5.6. An *optimal policy* $\pi^* \in \Pi^{\mathrm{HR}}$ satisfies

$$v_\lambda^{\pi^*}(s) \geq v_\lambda^\pi(s) \tag{5.43}$$

for all $\pi \in \Pi^{\mathrm{HR}}$ and $s \in S$.

Definition 5.7. The *optimal value function* $v_\lambda^*(s)$ is defined by

$$v_\lambda^*(s) := \sup_{\pi \in \Pi^{\mathrm{HR}}} v_\lambda^\pi(s) \tag{5.44}$$

for all $s \in S$.

Definition 5.8. For any $\epsilon > 0$, an *ϵ-optimal policy* $\pi^\epsilon \in \Pi^{\text{HR}}$ satisfies

$$v_\lambda^{\pi^\epsilon}(s) \geq v_\lambda^*(s) - \epsilon \tag{5.45}$$

for all $s \in S$.

These definitions are similar to those in finite horizon models under the expected total reward criterion. Recall that in finite horizon models with finite action sets, Markovian deterministic policies are optimal within the class of history-dependent randomized policies. In infinite horizon discounted models with stationary rewards and transition probabilities, a stronger result holds, namely that *stationary deterministic policies are optimal within the class of history-dependent randomized policies*. Theorem 5.4 provides the main result of this section. The remainder of the section develops the necessary machinery to establish it. The assumption of finite action sets is added for emphasis.

Theorem 5.4. Suppose A_s is finite for each $s \in S$. Then there exists $\pi^* \in \Pi^{\text{SD}}$ for which

$$v_\lambda^{\pi^*}(s) = v_\lambda^*(s)$$

for all $s \in S$.

The main consequence of this theorem is that the search for optimal policies can be restricted to the class of stationary deterministic policies[8].

5.3.2 The Bellman equation for a discounted model

Chapter 4 showed that in a finite horizon model, the following sequence of recursive equations together with a terminal condition (re-expressed in the notation of this chapter) could be used to determine optimal value functions and policies:

$$v_n(s) = \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v_{n+1}(j)) \right\}. \tag{5.46}$$

Informally, passing to the limit on both sides of (5.46) suggests that the Bellman equation for a discounted model should be of the form:

[8]Note that stationary optimal policy and optimal stationary policy are two different concepts. The former refers to the fact that the optimal policy is stationary. The latter refers to the best stationary policy among all stationary policies, but it may not be optimal among all policies, including those that are non-stationary. The focus here is on the former.

$$v(s) = \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\}. \tag{5.47}$$

The following development will show that this heuristic derivation is justified by showing that the Bellman equation has a unique solution and that the solution equals the optimal value function.

Vector form of the Bellman equation and c-max notation

Equation (5.47) can be expressed in vector notation as:

$$\mathbf{v} = \underset{d \in D^{\text{MD}}}{\text{c-max}} \left\{ \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v} \right\}. \tag{5.48}$$

It is important to observe that the vector Bellman equation uses c-max instead of max[9]. The use of the expression c-max on the right-hand side of (5.48) emphasizes that the maximum is taken component-wise. This means that given a $\mathbf{v} \in V$, for each $s \in S$ one evaluates

$$r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \tag{5.49}$$

for each $a \in A_s$, and then sets the right hand side of (5.48) to the vector of values that achieve this maximum. Example 5.2 expands on this point and emphasizes that the maximum can be taken component by component, thus avoiding enumerating $|A_{s_1}| \times |A_{s_2}| \times \ldots \times |A_{s_M}|$ Markovian deterministic decision rules.

The expression

$$d^* \in \arg \underset{d \in D^{\text{MD}}}{\text{c-max}} \left\{ \mathbf{r}_d + \lambda \mathbf{P} \mathbf{v} \right\} \tag{5.50}$$

[9]The c-max notation is not standard, but in our teaching experience, using max instead was a source of confusion among students. When the equation was written with max instead of c-max, it was frequently interpreted incorrectly to represent the process of:

1. Enumerating all decision rules.
2. Evaluating $\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}$ for each decision rule d.
3. Setting $\max_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}$ to the maximum value obtained in step 2.

Such an approach runs contrary to the dynamic programming principle of reducing a complex problem into a series of simpler problems that can be analyzed state by state.

indicates that the decision rule d^* is obtained by choosing

$$d^*(s) \in \arg\max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\}$$

for each $s \in S$.

5.3.3 The maximum return operator

Definition 5.9. The *maximum return operator* or *Bellman operator* $L : V \to V$ is defined as

$$L\mathbf{v} := \text{c-}\max_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}. \tag{5.51}$$

This is a nonlinear operator[10] that is applied to a vector $\mathbf{v} \in V$ and returns the vector of values associated with the deterministic decision rule that obtains the component-wise maximum of $\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}$. Note that for $\mathbf{v} \in V$, $L\mathbf{v}$ is a vector in V and its s-th component is represented by $L\mathbf{v}(s)$. Using L simplifies notation, makes proofs more transparent and suggests analyses for other optimality criteria.

The following lemma is a direct application of Lemma 2.1 expressed in vector notation. It asserts that the component-wise maximum over Markovian deterministic decision rules equals the component-wise maximum over Markovian randomized decision rules. Note it doesn't require that λ be less than 1.

Lemma 5.4. For any $\mathbf{v} \in V$ and $0 \leq \lambda \leq 1$

$$\text{c-}\max_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\} = \text{c-}\max_{d \in D^{\text{MR}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}. \tag{5.52}$$

Consequently (5.48) can be expressed[11] as

Bellman equation: operator notation

$$\mathbf{v} = L\mathbf{v}. \tag{5.53}$$

A solution of equation (5.53) is called a *fixed point* of L. Theoretical results regarding existence of a fixed point for a contraction mapping, which is defined below, are used to prove that the Bellman equation has a unique solution.

[10] In a finite state and action model, L is *piecewise linear* and concave.

[11] This representation indicates why this book typically refers to the Bellman equation in the singular and not "Bellman equations". However, in some instances, when the corresponding system of equations is written in component notation, they may be referred to as the Bellman equations.

Define the operator $L_d : V \to V$ for $d \in D^{\text{MR}}$ as

$$L_d\mathbf{v} := \mathbf{r}_d + \lambda \mathbf{P}_d\mathbf{v}. \tag{5.54}$$

That is, L_d gives the value of using decision rule d for one period and then receiving a terminal reward $\mathbf{v}$. Note that the fixed point equation $L_d\mathbf{v} = \mathbf{v}$ previously appeared as (5.31) where it was shown that $\mathbf{v}_\lambda^{d^\infty}$ is the unique fixed point of L_d when $0 \leq \lambda < 1$.

The expression $L^2\mathbf{v}$ corresponds to applying L to $\mathbf{v}$ twice[12]. Since $L\mathbf{v} \in V$, it is in the domain of L, so that the operator L can be applied to it yielding $L^2\mathbf{v} = L(L\mathbf{v})$. In general $L^n\mathbf{v} = L(L^{n-1}\mathbf{v})$.

Contraction mappings

Contraction mappings are an important analytic concept that is fundamental to establishing many key results in this chapter and Chapter 6.

Definition 5.10. An operator $T : V \to V$ is a *contraction mapping* on V with *modulus* κ, if for all $\mathbf{u}$ and $\mathbf{v}$ in V there exists a scalar $0 \leq k < 1$ for which

$$\|T\mathbf{v} - T\mathbf{u}\| \leq \kappa\|\mathbf{v} - \mathbf{u}\|. \tag{5.55}$$

It is easy to show that for any $n \geq 1$,

$$\|T^n\mathbf{v} - T^n\mathbf{u}\| \leq \kappa^n\|\mathbf{v} - \mathbf{u}\|.$$

A normed linear space is said to be *complete* if whenever $\|\mathbf{v}^n - \mathbf{v}^m\| \to 0$ as $m \to \infty$ and $n \to \infty$, there exists a $\mathbf{v}^* \in V$ for which $\|\mathbf{v}^n - \mathbf{v}^*\| \to 0$[13]. Note that the set of real numbers is complete, while the set of rational numbers is not. In the context of this book, the set of all $|S|$-dimensional real vectors, V, is complete.

The following result, often referred to as the *Banach fixed point theorem* or the *contraction mapping fixed point theorem* will be the basis for establishing a unique solution of the Bellman equation expressed as $L\mathbf{v} = \mathbf{v}$.

[12] It does not mean squaring $L\mathbf{v}$ which would be written as $(L\mathbf{v})^2$.

[13] Equivalently, a normed linear space is complete if every Cauchy sequence converges to an element of that space. A complete normed linear space is called a *Banach* space.

Theorem 5.5. Banach Fixed Point Theorem
Let V be a complete normed linear space and $T : V \to V$ be a contraction mapping. Then:

1. there exists a unique $\mathbf{v}^* \in V$ that satisfies $T\mathbf{v}^* = \mathbf{v}^*$, and
2. for any $\mathbf{v}^0 \in V$, the sequence $\{\mathbf{v}^n, n = 0, 1, 2, \ldots\}$, defined by

$$\mathbf{v}^{n+1} = T\mathbf{v}^n = T^{n+1}\mathbf{v}^0 \tag{5.56}$$

converges to $\mathbf{v}^*$. That is, $\|\mathbf{v}^n - \mathbf{v}^*\| \to 0$ as $n \to \infty$.

Proofs of this result are widely available and surprisingly straightforward. This result is important because when T is a contraction mapping, then the first part of Theorem 5.5 establishes the existence of a unique fixed point for T in V. The second part establishes that repeated application of the recursion $\mathbf{v} \leftarrow T\mathbf{v}$, converges to a fixed point of T.

Proposition 5.1. Suppose $0 \leq \lambda < 1$. Then the operator L defined in (5.51) is a contraction mapping on V with modulus λ.

Proof. Let[14] $\mathbf{u}$ and $\mathbf{v}$ be elements of V. Fix $s \in S$ and assume without loss of generality that $L\mathbf{v}(s) \geq L\mathbf{u}(s)$. Then by the finiteness of A_s, there exists an $a_s^* \in A_s$ for which

$$r(s, a_s^*) + \lambda \sum_{j \in S} p(j|s, a_s^*)v(j) = \max_{a \in A_s} \left\{ r(s, a) + \lambda \sum_{j \in S} p(j|s, a)v(j) \right\} = L\mathbf{v}(s). \tag{5.57}$$

Since a_s^* need not be a maximizer of $L\mathbf{u}(s)$,

$$r(s, a_s^*) + \lambda \sum_{j \in S} p(j|s, a_s^*)u(j) \leq \max_{a \in A_s} \left\{ r(s, a) + \lambda \sum_{j \in S} p(j|s, a)u(j) \right\} = L\mathbf{u}(s). \tag{5.58}$$

Subtracting (5.58) from (5.57) and noting the assumption that $L\mathbf{v}(s) \geq L\mathbf{u}(s)$, it follows that

$$\begin{aligned} 0 &\leq L\mathbf{v}(s) - L\mathbf{u}(s) \\ &\leq r(s, a_s^*) + \lambda \sum_{j \in S} p(j|s, a_s^*)v(j) - \left(r(s, a_s^*) + \lambda \sum_{j \in S} p(j|s, a_s^*)u(j) \right) \\ &= \lambda \sum_{j \in S} p(j|s, a_s^*)\Big(v(j) - u(j)\Big) \leq \lambda \sum_{j \in S} p(j|s, a_s^*)\|\mathbf{v} - \mathbf{u}\| \end{aligned}$$

[14] The proof uses component notation because two vectors need not be comparable. Note also for scalars, that $x \leq y$ does not imply $|x| \leq |y|$. It requires $0 \leq x \leq y$.

$$= \lambda \|\mathbf{v} - \mathbf{u}\|.$$

Since s was arbitrary and a similar result holds for $L\mathbf{u}(s) - L\mathbf{v}(s)$ assuming $L\mathbf{u}(s) \geq L\mathbf{v}(s)$, it follows that

$$\|L\mathbf{v} - L\mathbf{u}\| \leq \lambda \|\mathbf{v} - \mathbf{u}\|$$

completing the proof. □

Combining the above results and noting that the set of real-valued n-dimensional vectors V is complete leads to the following important result.

Theorem 5.6. Suppose $0 \leq \lambda < 1$. Then $L\mathbf{v} = \mathbf{v}$ has a unique solution in V.

Note that by replacing D^{MD} with a single decision rule[15] $d \in D^{\text{MD}}$, this result also establishes that $\mathbf{v}_\lambda^{d^\infty}$ is the unique fixed point of $L_d\mathbf{v} = \mathbf{v}$ in V. As a result of Lemma 5.4 it generalizes to $d \in D^{\text{MR}}$ to give the following result that previously appeared as Theorem 5.2 and was proved without appealing to the Banach fixed point theorem.

Corollary 5.1. For each $d \in D^{\text{MR}}$, $\mathbf{v}_\lambda^{d^\infty}$ is the unique fixed point of $L_d\mathbf{v} = \mathbf{v}$ in V.

5.3.4 Existence of an optimal value function

That the solution of the Bellman equation is the optimal value function follows from the following important theorem, which also can be used to derive upper and lower bounds on the optimal value function.

Theorem 5.7. Let $0 \leq \lambda < 1$. If there exists a $\mathbf{v} \in V$ for which:

1. $\mathbf{v} \geq L\mathbf{v}$, then $\mathbf{v} \geq \mathbf{v}_\lambda^*$,
2. $\mathbf{v} \leq L\mathbf{v}$, then $\mathbf{v} \leq \mathbf{v}_\lambda^*$ an
3. $\mathbf{v} = L\mathbf{v}$, then $\mathbf{v} = \mathbf{v}_\lambda^*$. Moreover, it is the unique solution of this equation.

Proof. To prove the first result, choose an arbitrary policy $\pi = (d_1, d_2, \ldots) \in \Pi^{\text{MR}}$ and an $\epsilon > 0$.

Combining $L\mathbf{v} \geq \mathbf{v}$ with Lemma 5.4, it follows that

$$\mathbf{v} \geq L\mathbf{v} = \underset{d \in D^{\text{MD}}}{\text{c-max}}\{\mathbf{r}_d + \lambda \mathbf{P}_d\mathbf{v}\} = \underset{d \in D^{\text{MR}}}{\text{c-max}}\{\mathbf{r}_d + \lambda \mathbf{P}_d\mathbf{v}\} \geq \mathbf{r}_d + \lambda \mathbf{P}_d\mathbf{v}$$

[15] This "trick" will be used on occasion in the sequel to obtain results for individual policies by first establishing results in terms of the maximum return operator.

for all $d \in D^{\mathrm{MR}}$. Hence

$$\mathbf{v} \geq \mathbf{r}_{d_1} + \lambda \mathbf{P}_{d_1}\mathbf{v} \geq \mathbf{r}_{d_1} + \lambda \mathbf{P}_{d_1}(\mathbf{r}_{d_2} + \lambda \mathbf{P}_{d_2}\mathbf{v}) = \mathbf{r}_{d_1} + \lambda \mathbf{P}_{d_1}\mathbf{r}_{d_2} + \lambda^2 \mathbf{P}_{d_1}\mathbf{P}_{d_2}\mathbf{v}$$

Repeating this argument, it follows that for $n \geq 1$,

$$\begin{aligned}\mathbf{v} &\geq \mathbf{r}_{d_1} + \lambda \mathbf{P}_{d_1}\mathbf{r}_{d_2} + \cdots + \lambda^{n-1}\mathbf{P}_{\pi}^{n-1}\mathbf{r}_{d_n} + \lambda^n \mathbf{P}_{\pi}^n \mathbf{v} \\ &= \mathbf{v}_{\lambda}^{\pi} - \sum_{k=n}^{\infty} \lambda^k \mathbf{P}_{\pi}^k \mathbf{r}_{d_{k+1}} + \lambda^n \mathbf{P}_{\pi}^n \mathbf{v}\end{aligned}$$

where the last equality follows from the definition of $\mathbf{v}_{\lambda}^{\pi}$ in (5.21). Choosing n sufficiently large, ensures that each of the last two terms can be bounded above in norm by $\epsilon/2$. This establishes that

$$\mathbf{v} \geq \mathbf{v}_{\lambda}^{\pi} - \epsilon \mathbf{e}$$

for any $\epsilon > 0$ and $\pi \in \Pi^{\mathrm{MR}}$.

From this inequality and Theorem 5.1 it follows that

$$\mathbf{v} \geq \sup_{\pi \in \Pi^{\mathrm{MR}}} \mathbf{v}_{\lambda}^{\pi} = \sup_{\pi \in \Pi^{\mathrm{HR}}} \mathbf{v}_{\lambda}^{\pi} = \mathbf{v}_{\lambda}^{*}$$

which proves part 1.

To prove the second result, since

$$\mathbf{v} \leq L\mathbf{v} = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\},$$

there exists a specific $d \in D^{\mathrm{MD}}$ for which $\mathbf{v} \leq \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}$. Hence

$$\mathbf{v} \leq \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v} \leq \mathbf{r}_d + \lambda \mathbf{P}_d(\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}) = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{r}_d + \lambda^2 \mathbf{P}_d^2 \mathbf{v}$$

so by repeating this argument, it follows that

$$\mathbf{v} \leq \sum_{n=1}^{\infty} \lambda^{n-1} \mathbf{P}_d^{n-1} \mathbf{r}_d = \mathbf{v}_{\lambda}^{d^\infty} \leq \sup_{\pi \in \Pi^{\mathrm{HR}}} \mathbf{v}_{\lambda}^{\pi},$$

as desired.

Combining parts 1 and 2 establishes that if $\mathbf{v} = L\mathbf{v}$, $\mathbf{v} = \mathbf{v}_{\lambda}^{*}$. □

Combining Theorem 5.7 with Theorem 5.6 yields the following fundamental result.

Theorem 5.8. The optimal value function $\mathbf{v}_{\lambda}^{*}$ is the unique solution to the optimality equation

$$\mathbf{v} = L\mathbf{v} = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}. \tag{5.59}$$

A version of Theorem 5.7 in component notation follows. It will be especially useful for formulating a discounted Markov decision process as a linear program in Section 5.9.

Corollary 5.2. Let $\mathbf{v} \in V$.

1. If
$$v(s) \geq \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\}$$
for all $s \in S$, then $v(s) \geq v^*_\lambda(s)$ for all $s \in S$, and

2. If
$$v(s) \leq \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\}$$
for all $s \in S$, then $v(s) \leq v^*_\lambda(s)$ for all $s \in S$.

3. The optimal value function $v^*_\lambda(s)$ is the unique solution of
$$v(s) = \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\}.$$

5.3.5 Existence of an optimal stationary policy

This section shows how to find an optimal policy from a solution of the Bellman equation $\mathbf{v}^*_\lambda$. The following is the fundamental result.

Theorem 5.9. Suppose there exists a $d^* \in D^{\mathrm{MD}}$ satisfying
$$d^* \in \arg \operatorname*{c\text{-}max}_{d \in D^{\mathrm{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}^*_\lambda\}. \tag{5.60}$$
Then $(d^*)^\infty$ is an optimal deterministic stationary policy.

Proof. Consider any decision rule $d^* \in D^{\mathrm{MD}}$ satisfying (5.60). Then
$$\mathbf{v}^*_\lambda = L\mathbf{v}^*_\lambda = \operatorname*{c\text{-}max}_{d \in D^{\mathrm{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}^*_\lambda\} = \mathbf{r}_{d^*} + \lambda \mathbf{P}_{d^*} \mathbf{v}^*_\lambda = L_{d^*} \mathbf{v}^*_\lambda$$
But since $\mathbf{v}_\lambda^{(d^*)^\infty}$ is the unique solution of $L_{d^*}\mathbf{v} = \mathbf{v}$, it follows from Corollary 5.1 that $\mathbf{v}_\lambda^{(d^*)^\infty} = \mathbf{v}^*_\lambda$. Hence $(d^*)^\infty$ is optimal. □

Note that this result shows that a stationary policy derived from *any* d^* satisfying (5.60) is optimal. Moreover, if more than one policy is optimal, any policy that ran-

domizes between actions chosen by different deterministic policies is optimal. This is particularly relevant because some policies with the same expected discounted reward might have smaller variance.

What is required to ensure that such a d^* exists? Clearly when A_s is finite for each $s \in S$ this holds, but it holds in greater generality under continuity and compactness assumptions.

A restatement of this result in component notation follows.

Corollary 5.3. Suppose for all $s \in S$ there exists an a_s^* satisfying

$$a_s^* \in \arg\max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} \lambda p(j|s,a) v_\lambda^*(j) \right\} \tag{5.61}$$

and let $d^*(s) = a_s^*$ for all $s \in S$. Then $(d^*)^\infty$ is an optimal policy.

The following terminology will be useful.

Definition 5.11. A stationary policy $(d^*)^\infty$ where d^* satisfies (5.60) is said to be a *greedy policy*. A decision rule satisfying (5.60) is said to be a *greedy decision rule* and an action satisfying (5.61) is said to be a *greedy action*. Decision rules and actions selected in this way are said to be chosen *greedily*.

Two examples

Example 5.5. This example revisits the two-state model. After simplification, the optimality equations in component form are given by

$$v(s_1) = \max\{3 + \lambda(0.8v(s_1) + 0.2v(s_2)),\ 5 + \lambda v(s_2)\} \tag{5.62}$$
$$v(s_2) = \max\{-5 + \lambda v(s_2),\ 2 + \lambda(0.4v(s_1) + 0.6v(s_2))\}. \tag{5.63}$$

Direct computation in Example 5.4 based on enumerating all stationary policies showed that when $\lambda = 0.9$, the stationary policy d_4^∞, where $d_4 = (a_{1,2}, a_{2,2})$, is optimal and the optimal value function is $\mathbf{v}_\lambda^{d_4^\infty} = (30.147, 27.941)$. It can be easily verified that this value function satisfies the Bellman equation. In fact, d_4^∞ is the optimal stationary policy for all λ, which is left as an exercise to show.

The following example illustrates an algebraic solution of the Bellman equation. Because of its simple structure, solving it does not require enumeration or knowledge of the optimal value function *a priori*. In contrast to the previous example, it also shows that an optimal policy may depend on the specific value of λ.

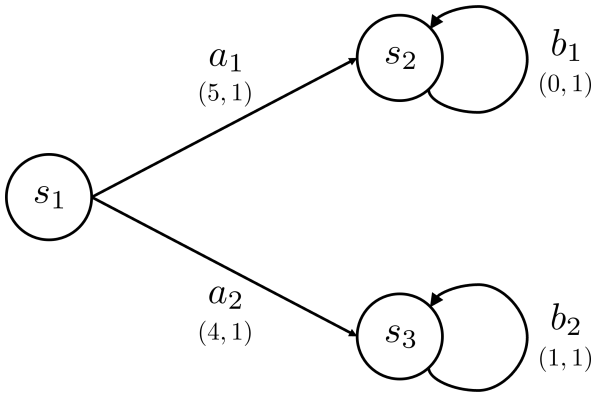


Figure 5.2: The three-state Markov decision process from Example 5.6 to illustrate the effect of varying λ on the optimal policy.

Example 5.6. Consider a three-state Markov decision process with $S = \{s_1, s_2, s_3\}$, $A_{s_1} = \{a_1, a_2\}$, $A_{s_2} = \{b_1\}$ and $A_{s_3} = \{b_2\}$, shown in Figure 5.2. Under action a_1, the process generates a reward of 5, transitions to state s_2 with probability 1 and then remains at state s_2 forever. Subsequent self-transitions in state s_2, (under action b_1), generate a reward of 0. Under action a_2, the process generates a reward of 4, transitions to state s_3 with probability 1, and then remains at state s_3 forever. Subsequent self-transitions in state s_3, (under action b_2), generate a reward of 1. The horizon is infinite and the discount factor is λ.

The Bellman equation for this model is:

$$\begin{aligned} v(s_1) &= \max\{5 + \lambda v(s_2), 4 + \lambda v(s_3)\} \\ v(s_2) &= 0 + \lambda v(s_2) \\ v(s_3) &= 1 + \lambda v(s_3) \end{aligned}$$

Because of the simple structure, these equations can be solved directly to obtain

$$v^*_\lambda(s) = \begin{cases} \max\{5, 4 + \lambda/(1-\lambda)\} & s = s_1 \\ 0 & s = s_2, \\ 1/(1-\lambda) & s = s_3, \end{cases}$$

The solution depends on λ as follows:

$$v^*_\lambda(s_1) = \begin{cases} 5 & 0 \le \lambda \le 0.5 \\ 4 + \lambda/(1-\lambda) & 0.5 \le \lambda < 1 \end{cases}$$

Thus, from (5.60) the stationary policy that uses action a_1 in state s_1 is optimal when $0 \le \lambda \le 0.5$ and the stationary policy that uses action a_2 in state s_1 is

optimal when $0.5 \leq \lambda < 1$. Note that both actions are optimal at $\lambda = 0.5$, which means that a randomized policy is also optimal at $\lambda = 0.5$.

Intuitively, small λ values mean that future rewards are heavily discounted relative to immediate rewards, so the decision maker would prefer to take the larger immediate reward of 5, and forego all future rewards. On the other hand, a large λ means that it is better to take the smaller initial reward of 4, but be assured of a future, infinite stream of rewards of 1, even though they are discounted.

Demonstrating that there exists an optimal stationary policy in Theorem 5.9 required first finding $\mathbf{v}_\lambda^*$. While an enumeration approach in Example 5.5 may be feasible for small "toy" problems, it is not scalable to larger problems. Thus, subsequent sections present algorithms to compute $\mathbf{v}_\lambda^*$ and corresponding optimal stationary policies.

5.4 State-action value functions

Section 4.5 introduced the concept of a state-action value function. This concept will not be central to this chapter, it is include for ease of reference. They will be fundamental in Chapters 10 and 11.

For a discounted model, define the *state-action value function corresponding to policy* π for $\pi \in \Pi^{\text{HR}}$, denoted by $q_\lambda^\pi(s,a)$, to be the infinite horizon expected discounted reward when choosing action $a \in A_s$ in state $s \in S$ at the current decision epoch and using policy π subsequently. Consequently,

$$q_\lambda^\pi(s,a) = r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v_\lambda^\pi(j) \tag{5.64}$$

Similarly, define *the optimal state-action value function*, denoted by $q_\lambda^*(s,a)$, to be the infinite horizon expected discounted reward when action $a \in A_s$ is chosen in state $s \in S$ at the current decision epoch and the optimal policy is used subsequently. It satisfies

$$q_\lambda^*(s,a) = r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v_\lambda^*(j) \tag{5.65}$$

for all $a \in A_s$ and $s \in S$. When an optimal policy π^* exists, (5.65) is equivalent to (5.64) with $\pi = \pi^*$.

When either $v_\lambda^\pi(s)$ or $v_\lambda^*(s)$ are available the above two representations can be used to evaluate the state-action value functions. However, from the perspective of their intended use, recursions expressed directly in terms of either $q_\lambda^\pi(s,a)$ or $q_\lambda^*(s,a)$ are preferable.

5.4.1 State-action value function recursions for a fixed policy

When $\pi = d^\infty$ for some $d \in D^{\mathrm{MD}}$,

$$v_\lambda^{d^\infty}(s) = q_\lambda^{d^\infty}(s, d(s)).$$

Hence from (5.64)

$$q_\lambda^{d^\infty}(s,a) = r(s,a) + \lambda \sum_{j \in S} p(j|s,a) q_\lambda^{d^\infty}(j, d(j)). \tag{5.66}$$

When the reward depends on the subsequent state, (5.66), can be expressed as

$$q_\lambda^{d^\infty}(s,a) = \sum_{j \in S} p(j|s,a) \Big(r(s,a,j) + \lambda q_\lambda^{d^\infty}(j, d(j)) \Big). \tag{5.67}$$

When d^∞ is randomized, the state action value function satisfies

$$q_\lambda^{d^\infty}(s,a) = \sum_{j \in S} p(j|s,a) \Big(r(s,a,j) + \lambda \sum_{a' \in A_j} w_d(a'|s) q_\lambda^{d^\infty}(j,a') \Big). \tag{5.68}$$

Observe that in contrast to the standard value function recursion

$$v_\lambda^{d^\infty} = r(s, d(s)) + \lambda \sum_{j \in S} p(j|s, d(s)) v_\lambda^{d^\infty}(j)$$

the decision rule $d(s)$ in (5.66) only appears in the q-function, and not the reward function or transition probabilities.

5.4.2 Bellman equation for state-action value functions

It follows from the Bellman equation (5.47) that

$$v_\lambda^*(s) = \max_{a \in A_s} q_\lambda^*(s,a).$$

Consequently, the optimal state-action value function may be represented as follows.

Bellman equation for state-action value functions
For $s \in S$ and $a \in A_s$,

$$q_\lambda^*(s,a) = r(s,a) + \lambda \sum_{j \in S} p(j|s,a) \max_{a' \in A_s} q_\lambda^*(j,a') \tag{5.69}$$

or, when the reward depends on the subsequent state,

$$q_\lambda^*(s,a) = \sum_{j \in S} p(j|s,a) \left(r(s,a,j) + \lambda \max_{a' \in A_s} q_\lambda^*(j,a') \right). \tag{5.70}$$

Choosing $d^*(s) \in \arg\max_{a \in A_s} q^*(s,a)$ produces an optimal stationary policy.

Written as an expected value, (5.69) is equivalent to

$$q^*(s,a) = E^{d^*}\left[r(X,a) + \lambda \max_{a' \in A_{X'}} q^*(X',a') \,\middle|\, X = s, Y = a \right], \tag{5.71}$$

where $(d^*)^\infty$ is an optimal policy for the discounted model and X' is distributed according to $p(\cdot|s,a)$. When $r(s,a,j)$ is the model primitive, (5.70) becomes:

$$q^*(s,a) = E^{d^*}\left[r(X,Y,X') + \lambda \max_{a' \in A_{X'}} q^*(X',a') \,\middle|\, X = s, Y = a \right]. \tag{5.72}$$

Observe that these two expressions differ from those representing the Bellman equation based on the value function. Here, the maximization is *inside* the expectation. While this difference is unimportant in this chapter, it will have a significant impact in Chapters 10 and 11, where expectations are estimated by random draws from a transition probability distribution or samples from the environment. As before, (5.66) may be viewed as special case of (5.69) when $A_s = \{d(s)\}$.

5.4.3 An example

This example derives the Bellman equation in component form for the two-state model using representation (5.70). For $s = s_1$,

$$\begin{aligned}
q(s_1,a_{1,1}) &= p(s_1|s_1,a_{1,1})\left(r(s_1,a_{1,1},s_1) + \lambda \max_{a \in A_{s_1}} q(s_1,a) \right) \\
&\quad + p(s_2|s_1,a_{1,1})\left(r(s_1,a_{1,1},s_2) + \lambda \max_{a \in A_{s_2}} q(s_2,a) \right) \\
&= 0.8\left(5 + \lambda \max_{a \in A_{s_1}} q(s_1,a) \right) + 0.2\left(-5 + \lambda \max_{a \in A_{s_2}} q(s_2,a) \right) \qquad (5.73) \\
&= 3 + 0.8\lambda \max_{a \in A_{s_1}} q(s_1,a) + 0.2\lambda \max_{a \in A_{s_2}} q(s_2,a), \qquad (5.74)
\end{aligned}$$

and since under $a_{1,2}$ the only possible transition is to s_2,

$$\begin{aligned}
q(s_1,a_{1,2}) &= p(s_2|s_1,a_{1,2})\left(r(s_1,a_{1,2},s_2) + \lambda \max_{a \in A_{s_2}} q(s_2,a) \right) \\
&= 5 + \lambda \max_{a \in A_{s_2}} q(s_2,a).
\end{aligned}$$

When $s = s_2$,

$$\begin{aligned}
q(s_2,a_{2,1}) &= p(s_2|s_2,a_{2,1})\left(r(s_2,a_{2,1},s_2) + \lambda \max_{a \in A_{s_2}} q(s_2,a) \right) \\
&= -5 + \lambda \max_{a \in A_{s_2}} q(s_2,a)
\end{aligned}$$

and

$$\begin{aligned} q(s_2, a_{2,2}) &= p(s_1|s_2, a_{2,2}) \left(r(s_2, a_{2,2}, s_1) + \lambda \max_{a \in A_{s_1}} q(s_1, a) \right) \\ &\quad + p(s_2|s_2, a_{2,2}) \left(r(s_2, a_{2,2}, s_2) + \lambda \max_{a \in A_{s_2}} q(s_2, a) \right) \\ &= 0.4 \left(20 + \lambda \max_{a \in A_{s_1}} q(s_1, a) \right) + 0.6 \left(-10 + \lambda \max_{a \in A_{s_2}} q(s_2, a) \right) \\ &= 2 + 0.4\lambda \max_{a \in A_{s_1}} q(s_1, a) + 0.6\lambda \max_{a \in A_{s_2}} q(s_2, a). \end{aligned}$$

Note that (5.73) is of the form (5.70) and (5.74) is of the form (5.69). The former would be the basis of a simulation model that generated $5 + \lambda \max_{a \in A_{s_1}} q(s_1, a)$ with probability 0.8 and $-5 + \lambda \max_{a \in A_{s_2}} q(s_2, a)$ with probability 0.2 corresponding to probabilities of transitions under action $a_{1,1}$ in s_1.

From Theorem 5.10 below, $q^*_\lambda(s, a)$ is the unique solution of these equations. It is left as an exercise to solve these equations and show that (5.65) holds.

5.4.4 Existence of solutions to the Bellman equation for state-action value functions*

Establishing the existence of solutions to either (5.66) or (5.69) follows the same contraction mapping approach as when the Bellman equation was expressed in terms of value functions.

Let V_q denote the set of real valued functions on $A_{s_1} \times A_{s_2} \times \ldots \times A_{s_M}$, or equivalently, indexed by the state s and action a. For an element $\mathbf{q} \in V_q$, define its norm by

$$\|\mathbf{q}\| = \max_{s \in S, a \in A_s} |q(s, a)|.$$

Note $\max_{s \in S, a \in A_s} |q(s, a)| = \max_{s \in S} \max_{a \in A_s} |q(s, a)|$.

Analogous to the operator $L : V \to V$, define the (s, a)-th component of $F : V_q \to V_q$ by

$$F\mathbf{q}(s, a) := r(s, a) + \lambda \sum_{j \in S} p(j|s, a) \max_{a' \in A_s} q(j, a'). \tag{5.75}$$

The following result establishes that the operator F is a contraction mapping on V_q so that as a result of the Banach fixed point theorem, there exists a unique solution to $F\mathbf{q} = \mathbf{q}$ in V_q.

Proposition 5.2. Suppose $0 \leq \lambda < 1$. Then the operator F defined in (5.75) is a contraction mapping on V_q with modulus λ.

Proof. Let $\mathbf{q}_1$ and $\mathbf{q}_2$ be elements of V_q. Then[16]

$$\begin{aligned}
\|F\mathbf{q}_1 - F\mathbf{q}_2\| &= \max_{s\in S, a\in A_s} \left| r(s,a) + \lambda \sum_{j\in S} p(j|s,a) \max_{a'\in A_j} q_1(j,a') - r(s,a) - \lambda \sum_{j\in S} p(j|s,a) \max_{a'\in A_j} q_2(j,a') \right| \\
&= \lambda \max_{s\in S, a\in A_s} \left| \sum_{j\in S} p(j|s,a) \max_{a'\in A_j} q_1(j,a') - \sum_{j\in S} p(j|s,a) \max_{a'\in A_j} q_2(j,a') \right| \\
&\leq \lambda \max_{s\in S, a\in A_s} \sum_{j\in S} p(j|s,a) \left| \max_{a'\in A_j} q_1(j,a') - \max_{a'\in A_j} q_2(j,a') \right| \\
&\leq \lambda \max_{s\in S, a\in A_s} \sum_{j\in S} p(j|s,a) \max_{a'\in A_j} |q_1(j,a') - q_2(j,a')| \\
&\leq \lambda \max_{s\in S, a\in A_s} \sum_{j\in S} p(j|s,a) \max_{j\in S} \max_{a'\in A_j} |q_1(j,a') - q_2(j,a')| \\
&= \lambda \|\mathbf{q}_1 - \mathbf{q}_2\|.
\end{aligned}$$

□

As a consequence of the above Proposition, applying the Banach fixed point theorem (Theorem 5.5) yields the following important result.

Theorem 5.10. Suppose $0 \leq \lambda < 1$.

1. Then the equation $F\mathbf{q} = \mathbf{q}$ has a unique solution $\mathbf{q}^* \in V_q$.
2. For any $\mathbf{q}^0 \in V_q$, the sequence $(\mathbf{q}^n)$, $n = 0, 1, \ldots$ defined by $\mathbf{q}^{n+1} = F\mathbf{q}^n = F^{n+1}\mathbf{q}^0$ converges to $\mathbf{q}^*$.

5.5 Spans, bounds and other technical concepts

This long and rather technical section serves as a bridge to the analysis of computational algorithms. The key results appear as Theorems 5.12, 5.13, and 5.14.

A formal statement of an iterative algorithm for solving an infinite horizon Markov decision process requires:

1. a stopping criterion,
2. an estimate of the optimal value function at termination, and
3. an estimate of the optimal policy at termination.

[16] The proof uses the inequality $\max_{a\in A} |f(a) - g(a)| \geq |\max_{a\in A} f(a) - \max_{a\in A} g(a)|$ for real-valued functions $f(a)$ and $g(a)$ on a set A. A proof of this result is left as an exercise.

Terminating such an algorithm requires determining when the difference in value between successive iterates is sufficiently small. This requires a concept of distance between two vectors. Section 5.1.6 introduced the norm $||\cdot||$; two vectors $\mathbf{u}$ and $\mathbf{v}$ are close if $||\mathbf{u}-\mathbf{v}||$ is small. However, using an alternative construct, referred to as the *span* of a vector, may detect convergence sooner.

Moreover, the span:

1. provides bounds on the distance of the value of a stationary policy derived from the "current" greedy decision rule to the optimal value function,
2. is the basis for identifying sub-optimal actions, and
3. provides enhanced estimates of the optimal value function.

5.5.1 Spans and span contractions

Definition 5.12. The *span semi-norm* or simply the *span*, $\text{sp}(\mathbf{v})$, is defined as

$$\text{sp}(\mathbf{v}) := \max_{s\in S} v(s) - \min_{s\in S} v(s). \tag{5.76}$$

It is easy to show that the span is a semi-norm[17]. Moreover, for $\mathbf{v} \in V$:

1. $\text{sp}(\mathbf{v}+k\mathbf{e}) = \text{sp}(\mathbf{v})$ for any scalar k,
2. $\text{sp}(\mathbf{v}) = \text{sp}(-\mathbf{v})$, and
3. $\text{sp}(\mathbf{v}) \leq 2\|\mathbf{v}\|$.

The first property means that $\text{sp}(\mathbf{v}) = 0$, implies that $\mathbf{v}$ is a constant vector. On the other hand, $\|\mathbf{v}\| = 0$, implies that $\mathbf{v} = 0$. Hence the span and the sup-norm measure different properties of a vector. The norm measures how close it is to $\mathbf{0}$ while the span measures how close it is to a constant vector. Note that the third condition is satisfied with equality when, for example, $\mathbf{v} = (1, -1)$.

[17] A *semi-norm* $\sigma(\mathbf{v})$ on V is a real-valued function that satisfies the following properties:

1. $\sigma(\mathbf{v}) \geq 0$ for all $\mathbf{v} \in V$,
2. $\sigma(\mathbf{v}+\mathbf{u}) \leq \sigma(\mathbf{v}) + \sigma(\mathbf{u})$ for $\mathbf{v}$ and $\mathbf{u}$ in V,
3. $\sigma(k\mathbf{v}) = |k|\sigma(\mathbf{v})$ for scalar k and $\mathbf{v} \in V$.

This is a semi-norm (and not a norm) because $\sigma(\mathbf{v}) = 0$ need not imply $\mathbf{v} = \mathbf{0}$.

Why the span is useful?

This section will show that under mild conditions, transition probability matrices have contraction properties with respect to the span. They are based on the following more general observation, which is stated without proof[18].

Proposition 5.3. Let $\mathbf{Q}$ denote an $K \times M$ matrix where each row of $\mathbf{Q}$ is probability distribution on $\{1, \ldots, M\}$. Then for any M-dimensional vector $\mathbf{v}$,

$$\text{sp}(\mathbf{Q}\mathbf{v}) \leq \gamma \text{sp}(\mathbf{v}) \tag{5.77}$$

where

$$\gamma = 1 - \min_{1 \leq k \leq K,\, 1 \leq k' \leq K} \sum_{m=1}^{M} \min\{Q(m|k), Q(m|k')\}. \tag{5.78}$$

Moreover, there exists a $\mathbf{v}$ for which (5.77) holds with equality.

When $\gamma < 1$ this result implies that the elements of $\mathbf{Q}\mathbf{v}$ will be closer together than those of $\mathbf{v}$. In other words $\mathbf{Q}\mathbf{v}$ is closer than $\mathbf{v}$ to a constant vector.

The main application of Proposition 5.3 in this book is when $\mathbf{Q} = \mathbf{P}_d$ so that $K = M = |S|$ and $\mathbf{Q}$ is a transition probability matrix corresponding to a Markovian decision rule d.

Proposition 5.4. For any $d \in D^{\text{MR}}$, transition probability matrix $\mathbf{P}_d : S \to S$, and $\mathbf{v} \in V$,

$$\text{sp}(\mathbf{P}_d \mathbf{v}) \leq \gamma_d \text{sp}(\mathbf{v}), \tag{5.79}$$

where

$$\gamma_d = \quad 1 - \min_{(s,u) \in S \times S} \sum_{j \in S} \min\left\{P_d(j|s), P_d(j|u)\right\} \tag{5.80}$$

Moreover, there exists a $\mathbf{v} \in V$ for which (5.79) holds with equality.

The quantity γ_d is referred to as the *delta coefficient* of $\mathbf{P}_d$. It provides an upper bound on the second largest eigenvalue (*sub-radius*) of $\mathbf{P}_d$. Note that (5.80) is not intended for computation[19] but rather to provide insight into the meaning of γ_d. In

[18] A slightly less general version of this result is stated and proved in Hübner [1977] and restated as Proposition 6.6.1 in Puterman [1994].

[19] Alternatively

$$\gamma_d = \frac{1}{2} \max_{(s,u) \in S \times S} \sum_{j \in S} |P_d(j|s) - P_d(j|u)| = \max_{(s,u) \in S \times S} \sum_{j \in S} (P_d(j|s) - P_d(j|u))^+.$$

particular, when $\mathbf{P}_d = \mathbf{I}$ or when $\mathbf{P}_d$ has exactly one 1 in each row and column, direct computation using (5.80) establishes that $\gamma_d = 1$. On the other hand if the rows of $\mathbf{P}_d$ are equal, it is easy to see then $\gamma_d = 0$.

Note that the following simpler expression γ'_d provides an easier to compute upper bound on γ_d:

$$\gamma_d \leq \gamma'_d := 1 - \sum_{j \in S} \min_{s \in S} P_d(j|s). \tag{5.81}$$

From Lemma 5.1, it follows for any Markovian decision rule that

$$\|\mathbf{P}_d \mathbf{v}\| \leq ||\mathbf{P}_d||\, ||\mathbf{v}|| = \|\mathbf{v}\|,$$

so in contrast to the span, $\mathbf{P}_d$ is **not** a contraction with respect to the norm.

Example 5.7. This example computes γ_d for the Markov deterministic decision rule $d(s_1) = a_{1,1}$, $d(s_2) = a_{2,1}$ in the two-state model. Since

$$\mathbf{P}_d = \begin{bmatrix} 0.8 & 0.2 \\ 0 & 1 \end{bmatrix},$$

for any $\mathbf{v} = (v_1, v_2)$,

$$\mathbf{P}_d \mathbf{v} = \begin{bmatrix} 0.8v_1 + 0.2v_2 \\ v_2 \end{bmatrix}.$$

Hence

$$\mathrm{sp}(\mathbf{P}_d \mathbf{v}) = |0.8v_1 - 0.8v_2| = 0.8|v_1 - v_2| = 0.8\, \mathrm{sp}(\mathbf{v}).$$

Thus

$$\mathrm{sp}(\mathbf{P}_d^n \mathbf{v}) = 0.8^n \mathrm{sp}(\mathbf{v})$$

so that $\mathrm{sp}(\mathbf{P}_d^n \mathbf{v})$ converges to $\mathbf{0}$ for any $\mathbf{v} \in V$.

Direct computation shows that $\gamma_d = \gamma'_d = 1 - (0 + 0.2) = 0.8$ in agreement with the above calculations. Note also that the eigenvalues of $\mathbf{P}_d$ equal 1 and 0.8 and that γ_d equals the second largest eigenvalue of $\mathbf{P}_d$[a].

It is left to the reader to compute the values of γ_d and γ'_d, and the corresponding eigenvalues for the three other Markovian deterministic decision rules.

[a]For any square matrix, the sum of the diagonal elements (the *trace*) is equal to the sum of its eigenvalues. Since 1 is an eigenvalue of any finite transition probability matrix, for a 2×2 transition probability matrix one can find the second eigenvalue by subtracting 1 from its trace.

The following theorem identifies broad classes of transition probability matrices for which $\gamma_d < 1$. It is one of the few results for discounted models that depend on the structure of $\mathbf{P}_d$. It is a consequence of the Perron-Frobenius Theorem, which appears as Theorem B.3 in the book Appendix.

Proposition 5.5. Let $d \in D^{\text{MD}}$. Suppose the Markov chain corresponding to $\mathbf{P}_d$ is either[a]

1. regular or
2. has a single irreducible set and a non-empty set of transient states.

Then $\gamma_d < 1$.

[a]See Appendix B for definitions of the concepts below.

Note that the matrices

$$\mathbf{P}_d = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \text{or} \quad \mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

do not satisfy the hypotheses of Theorem 5.5. The one on the left has two closed classes and the one on the right is periodic so it is not regular. On the other hand the matrix in Example 5.7 satisfies the second condition in this theorem.

Span contraction mappings

Establishing that L is a span contraction with modulus at most λ requires the following application of Proposition 5.3 with a different choice for $\mathbf{Q}$ defined in the following proposition.

Proposition 5.6. Let $\mathscr{P}$ represent a $\sum_{s\in S} |A_s| \times |S|$ matrix[a] where each row consists of the vector of probabilities $p(\cdot|s,a)$ for a different (s,a)-pair where $s \in S$ and $a \in A_s$. Then for any $\mathbf{v} \in V$

$$\text{sp}(\mathscr{P}\mathbf{v}) \leq \gamma^* \text{sp}(\mathbf{v}), \tag{5.82}$$

where

$$\gamma^* = 1 - \min_{s\in S, a\in A_s, s'\in S, a'\in A_{s'}} \sum_{j\in S} \min\{p(j|s,a), p(j|s',a')\} \leq 1. \tag{5.83}$$

Moreover, there exists a $\mathbf{v} \in V$ for which (5.82) holds with equality.

[a]The order of the rows is material for what follows.

The following theorem is the main result of this section. Its proof is quite subtle but uses some of the same arguments that were the basis for establishing that L was a sup-norm contraction.

Theorem 5.11. For any $\mathbf{v}$ and $\mathbf{u}$ in V, and $0 \leq \lambda < 1$, there exists $\gamma^* \leq 1$ such that

$$\mathrm{sp}(L\mathbf{v} - L\mathbf{u}) \leq \lambda\gamma^* \mathrm{sp}(\mathbf{v} - \mathbf{u}). \tag{5.84}$$

Proof. Choose $\mathbf{v}$ and $\mathbf{u}$ in V and let Markovian deterministic decision rule $d_{\mathbf{v}}$ and $d_{\mathbf{u}}$ be greedy with respect to $\mathbf{v}$ and $\mathbf{u}$ respectively. Then

$$L\mathbf{v} - L\mathbf{u} \leq L_{d_{\mathbf{v}}}\mathbf{v} - L_{d_{\mathbf{v}}}\mathbf{u} = \lambda \mathbf{P}_{d_{\mathbf{v}}}(\mathbf{v} - \mathbf{u})$$

and

$$L\mathbf{v} - L\mathbf{u} \geq L_{d_{\mathbf{u}}}\mathbf{v} - L_{d_{\mathbf{u}}}\mathbf{u} = \lambda \mathbf{P}_{d_{\mathbf{u}}}(\mathbf{v} - \mathbf{u}).$$

Since $P_{d_{\mathbf{v}}}$ and $P_{d_{\mathbf{u}}}$ consist[20] of rows of $\mathscr{P}$, it follows that

$$\max_{s\in S}\big(L\mathbf{v}(s) - L\mathbf{u}(s)\big) \leq \lambda \max_{s\in S} \mathbf{P}_{d_{\mathbf{v}}}(\mathbf{v} - \mathbf{u}) \leq \lambda \max_{s\in S} \mathscr{P}(\mathbf{v} - \mathbf{u})$$

and

$$\min_{s\in S}\big(L\mathbf{v}(s) - L\mathbf{u}(s)\big) \geq \lambda \min_{s\in S} \mathbf{P}_{d_{\mathbf{u}}}(\mathbf{v} - \mathbf{u}) \geq \lambda \min_{s\in S} \mathscr{P}(\mathbf{v} - \mathbf{u}).$$

Combining these two inequalities and applying Proposition 5.6 gives the desired result that

$$\mathrm{sp}(L\mathbf{v} - L\mathbf{u}) \leq \lambda\, \mathrm{sp}(\mathscr{P}(\mathbf{v} - \mathbf{u})) \leq \lambda\gamma^*\, \mathrm{sp}(\mathbf{v} - \mathbf{u}).$$

□

Note that in most settings, γ^* cannot be easily determined *a priori*. Moreover, there is no need to do so. When either of the conditions in Theorem 5.5 holds and the sets of states and action are finite, $\gamma^* < 1$. Hence, in this case *the contraction modulus with respect to the span semi-norm is strictly less than that of the sup-norm* that is, $\lambda\gamma^* < \lambda$. This suggests that algorithms of the form $\mathbf{v} \leftarrow L\mathbf{v}$ will terminate faster when using the span as a stopping criterion.

But when $\gamma^* = 1$, as long as $\lambda < 1$, L will remain a span contraction but with modulus possibly equal to λ. Note the subtle point that algorithms of the form $\mathbf{v} \leftarrow L\mathbf{v}$ might indeed converge faster with respect to the span because the series of greedy decision rules chosen might all have $\gamma_d < 1$ even when $\gamma^* = 1$.

The following example illustrates these calculations.

[20] Since $\mathbf{P}_{d_{\mathbf{v}}}$ need not equal $\mathbf{P}_{d_{\mathbf{u}}}$ the use of $\mathscr{P}$ is required to ensure that the maximum and minimum are with respect to the same quantity.

Example 5.8. In the two-state model

$$\mathscr{P} = \begin{bmatrix} 0.8 & 0.2 \\ 0 & 1 \\ 0 & 1 \\ 0.6 & 0.4 \end{bmatrix} \tag{5.85}$$

where the first two rows correspond to $(s_1, a_{1,1})$ and $(s_1, a_{1,2})$ and the last two rows to $(s_2, a_{2,1})$ and $(s_2, a_{2,2})$. Direct calculation shows that $\gamma^* = 0.8$. Hence the contraction modulus with respect to the span is 0.8λ, while that with respect to the norm is λ.

Restricting the set of possible decision rules to a single decision rule d leads to the following corollary, which can easily be proved directly.

Corollary 5.4. For any $d \in D^{\text{MD}}$, $\mathbf{v}$ and $\mathbf{u}$ in V, and $0 \leq \lambda < 1$,

$$\text{sp}(L_d\mathbf{v} - L_d\mathbf{u}) \leq \lambda\gamma_d \text{sp}(\mathbf{v} - \mathbf{u}), \tag{5.86}$$

where γ_d is defined in (5.80).

5.5.2 Properties of $L\mathbf{v} - \mathbf{v}$

The quantity $L\mathbf{v} - \mathbf{v}$ plays an important role in deriving bounds, eliminating actions and proving convergence of several iterative algorithms. Since it arises so frequently, the operator B is defined to represent it.

Definition 5.13. The *optimal one-step improvement operator* is the operator $B : V \to V$ defined by

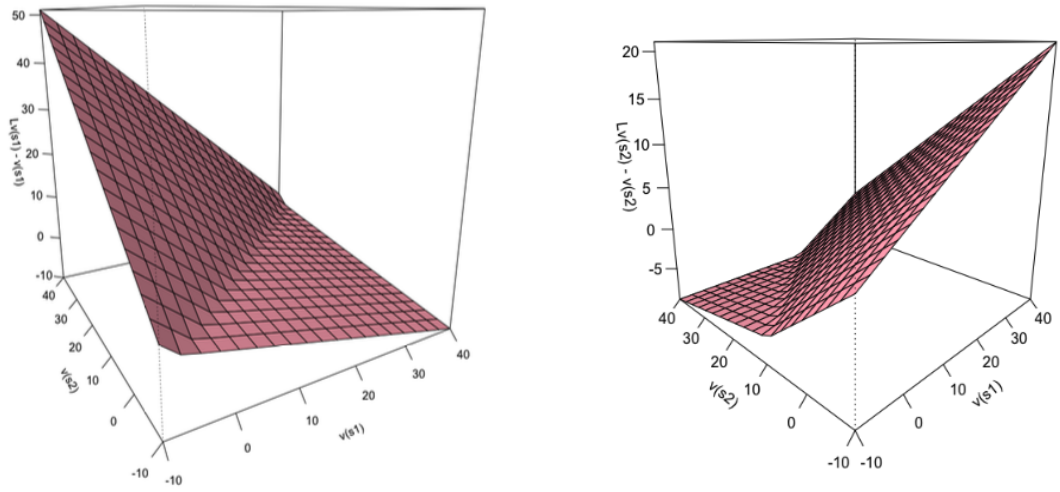
$$B\mathbf{v} := L\mathbf{v} - \mathbf{v}. \tag{5.87}$$

Note that the Bellman equation $L\mathbf{v} = \mathbf{v}$ can now be expressed as

$$B\mathbf{v} = \mathbf{0} \tag{5.88}$$

so that solving the Bellman equation may be regarded as finding a zero of B. This observation will be important when analyzing policy iteration below. For completeness, define the operator B_d on V by $B_d := L_d\mathbf{v} - \mathbf{v}$. Using this notation, $B\mathbf{v}$ can be rewritten as $B\mathbf{v} = \text{c-max}_{d \in D^{\text{MD}}} B_d\mathbf{v}$.

Figure 5.3 shows *each* component of $B\mathbf{v}$ as a function of the components of $\mathbf{v}$ for the two-state model. Because there are only two states, the components of $\mathbf{v}$ can be



(a) $B\mathbf{v}(s_1)$ as a function of $v(s_1)$ and $v(s_2)$. (b) $B\mathbf{v}(s_2)$ as a function of $v(s_1)$ and $v(s_2)$.

Figure 5.3: Each components of $B\mathbf{v}$ as a function of the components of $\mathbf{v}$ for the two-state example.

shown on the horizontal axes and the value of **each** component of $B\mathbf{v}$ on the vertical axis in a separate figure.

Observe that in each case the surfaces represent the maximum of two hyperplanes, each corresponding to a different action. Hence each component is piecewise linear and convex. Moreover $B\mathbf{v}(s_1)$ is non-decreasing as a function of $v(s_1)$ for each fixed value of $v(s_2)$ and $B\mathbf{v}(s_2)$ is non-increasing as a function of $v(s_2)$ for each fixed value of $v(s_1)$.

To get another perspective on the form of $B\mathbf{v}$, Figure 5.4 provides a one-dimensional plot of $B\mathbf{v}(s_1)$ as a function of $v(s_1)$ when $v(s_2)$ equals its optimal value 27.941. Observe that it is the maximum of two lines, each corresponding to one of the deterministic decision rules and that it is convex non-increasing in $v(s_1)$ in agreement with Figure 5.3a. The value of each deterministic decision rule can be found by solving $B_d\mathbf{v}(s_1) = 0$ for $d = d_1$ and d_2, and the optimal value by solving $B\mathbf{v}(s_1) = 0$. This figure also shows that the decision rule d_1 (dotted line) corresponds to the optimal policy.

Convexity of $B\mathbf{v}$

The following lemma, which generalizes the gradient inequality for convex functions[21], establishes convexity-like properties of B and L. Expression (5.89) is often called the

[21] If $f(x)$ is a convex and differentiable function mapping $\mathbb{R}^1$ into $\mathbb{R}^1$, then for any x and y in $\mathbb{R}^1$,

$$f(y) \geq f(x) + f'(x)(y - x)$$

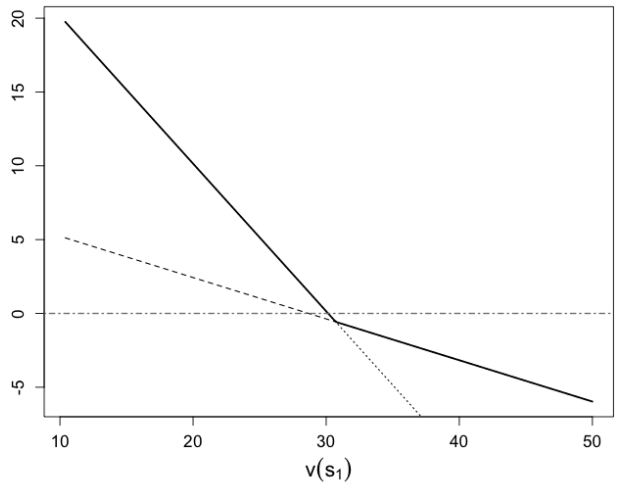


Figure 5.4: Plot showing $B\mathbf{v}(s_1)$ (solid line), $B_{d_1}\mathbf{v}(s_1)$ (dotted line corresponding to $d_1 = (a_{1,2}, a_{2,2})$) and $B_{d_2}\mathbf{v}(s_1)$ (dashed line corresponding to $d_2 = (a_{1,1}, a_{2,2})$) as a function of $v(s_1)$ for the two-state model when $v(s_2)$ is set to its optimal value 27.941. The optimal value $v(s_1) = 30.147$ is obtained when $Bv(s_1) = 0$.

support inequality.

Lemma 5.5. For any $\mathbf{u}$ and $\mathbf{v}$ in V, and $d_{\mathbf{v}}$ that satisfies $\mathbf{r}_{d_{\mathbf{v}}} + \lambda \mathbf{P}_{d_{\mathbf{v}}}\mathbf{v} = L\mathbf{v}$,

$$B\mathbf{u} \geq B\mathbf{v} + (\lambda \mathbf{P}_{d_{\mathbf{v}}} - \mathbf{I})(\mathbf{u} - \mathbf{v}), \tag{5.89}$$

or equivalently,

$$L\mathbf{u} - \mathbf{u} \geq L\mathbf{v} - \mathbf{v} + (\lambda \mathbf{P}_{d_{\mathbf{v}}} - \mathbf{I})(\mathbf{u} - \mathbf{v}). \tag{5.90}$$

Proof. By the definition of $d_{\mathbf{v}}$,

$$L\mathbf{u} \geq \mathbf{r}_{d_{\mathbf{v}}} + \lambda \mathbf{P}_{d_{\mathbf{v}}}\mathbf{u} = \mathbf{r}_{d_{\mathbf{v}}} + \lambda \mathbf{P}_{d_{\mathbf{v}}}\mathbf{v} + \lambda \mathbf{P}_{d_{\mathbf{v}}}(\mathbf{u} - \mathbf{v}) = L\mathbf{v} + \lambda \mathbf{P}_{d_{\mathbf{v}}}(\mathbf{u} - \mathbf{v})$$

subtracting $\mathbf{u} - \mathbf{v}$ from both sides and rearranging terms gives the result. □

5.5.3 Bounds

Bounds will be used to determine how accurately $\mathbf{v}^n$ (the n-th iterate of an algorithm) approximates $\mathbf{v}^*_\lambda$ when an algorithm terminates, although they apply in greater generality. They will also provide the basis for action elimination procedures in Section 5.5.5.

Some additional notation and a definition simplify exposition. For $\mathbf{v} \in V$, define

the scalars[22]:

$$\underline{\mathbf{v}} := \min_{s \in S} v(s) \quad \text{and} \quad \overline{\mathbf{v}} := \max_{s \in S} v(s). \tag{5.91}$$

Definition 5.14. For any $\mathbf{v} \in V$, $d_{\mathbf{v}} \in D^{\text{MR}}$ is $\mathbf{v}$-*greedy* if

$$\mathbf{r}_{d_{\mathbf{v}}} + \lambda \mathbf{P}_{d_{\mathbf{v}}} \mathbf{v} = L\mathbf{v}. \tag{5.92}$$

Note that $d_{\mathbf{v}} \in D^{\text{MD}}$ is $\mathbf{v}$-greedy if

$$d_{\mathbf{v}} \in \arg\operatorname*{c\text{-}max}_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}.$$

However, the definition also holds for randomized decision rules that obtain the componentwise maximums.

The following theorem provides two sets of upper and lower bounds on the optimal value function and the value function of a $\mathbf{v}$-greedy policy. It is the main result in this section.

Theorem 5.12. For any $\mathbf{v} \in V$, and $\mathbf{v}$-greedy $d_{\mathbf{v}} \in D^{\text{MR}}$

$$\begin{aligned} \mathbf{v} + (1-\lambda)^{-1}(\overline{L\mathbf{v} - \mathbf{v}})\mathbf{e} \geq L\mathbf{v} + \lambda(1-\lambda)^{-1}(\overline{L\mathbf{v} - \mathbf{v}})\mathbf{e} \geq \mathbf{v}_\lambda^* \geq \mathbf{v}_\lambda^{d_{\mathbf{v}}^\infty} \\ \geq L\mathbf{v} + \lambda(1-\lambda)^{-1}(\underline{L\mathbf{v} - \mathbf{v}})\mathbf{e} \geq \mathbf{v} + (1-\lambda)^{-1}(\underline{L\mathbf{v} - \mathbf{v}})\mathbf{e} \end{aligned} \tag{5.93}$$

Proof. A derivation of the lower bounds in (5.93) follows. The upper bounds can be obtained by reversing $\mathbf{v}$ and $\mathbf{v}_\lambda^*$ in the following.

Applying Lemma 5.5 with $\mathbf{u} = \mathbf{v}_\lambda^*$ and noting that $L\mathbf{v}_\lambda^* = \mathbf{v}_\lambda^*$ gives

$$\mathbf{0} \geq L\mathbf{v} - \mathbf{v} + (\lambda \mathbf{P}_{d_{\mathbf{v}}} - \mathbf{I})(\mathbf{v}_\lambda^* - \mathbf{v}).$$

From part 1 of Lemma 5.3, for $\mathbf{w} \in V$, $(\mathbf{I} - \lambda \mathbf{P}_{d_{\mathbf{v}}})^{-1}\mathbf{w} \leq \mathbf{0}$ when $\mathbf{w} \leq \mathbf{0}$ so that left-multiplying both sides of the above expression by $(\mathbf{I} - \lambda \mathbf{P}_{d_{\mathbf{v}}})^{-1}$ and rearranging terms yields

$$\begin{aligned} \mathbf{v}_\lambda^* &\geq \mathbf{v} + (\mathbf{I} - \lambda \mathbf{P}_{d_v})^{-1}(L\mathbf{v} - \mathbf{v}) && (5.94) \\ &= \mathbf{v} + (\mathbf{I} + \lambda \mathbf{P}_{d_{\mathbf{v}}} + \lambda^2 \mathbf{P}_{d_{\mathbf{v}}}^2 + \ldots)(L\mathbf{v} - \mathbf{v}) && (5.95) \\ &= \mathbf{v} + (L\mathbf{v} - \mathbf{v}) + \lambda \mathbf{P}_{d_{\mathbf{v}}}(\mathbf{I} + \lambda \mathbf{P}_{d_{\mathbf{v}}} + \lambda^2 \mathbf{P}_{d_{\mathbf{v}}}^2 + \ldots)(L\mathbf{v} - \mathbf{v}) && (5.96) \\ &\geq \mathbf{v} + (L\mathbf{v} - \mathbf{v}) + \lambda \mathbf{P}_{d_{\mathbf{v}}}(\mathbf{I} + \lambda \mathbf{P}_{d_{\mathbf{v}}} + \lambda^2 \mathbf{P}_{d_{\mathbf{v}}}^2 + \ldots)(\underline{L\mathbf{v} - \mathbf{v}})\mathbf{e} && (5.97) \\ &= \mathbf{v} + (L\mathbf{v} - \mathbf{v}) + \lambda \mathbf{P}_{d_{\mathbf{v}}}(1-\lambda)^{-1}(\underline{L\mathbf{v} - \mathbf{v}})\mathbf{e} && (5.98) \end{aligned}$$

[22] The notation here is a bit awkward. It is best understood by viewing $\underline{\mathbf{v}}$ and $\overline{\mathbf{v}}$ as real-valued functions applied to the vector $\mathbf{v}$.

$$= \mathbf{v} + (L\mathbf{v} - \mathbf{v}) + \lambda(1-\lambda)^{-1}(\underline{L\mathbf{v} - \mathbf{v}})\mathbf{e} \tag{5.99}$$
$$\geq \mathbf{v} + (1-\lambda)^{-1}(\underline{L\mathbf{v} - \mathbf{v}})\mathbf{e}. \tag{5.100}$$

Equality (5.95) is an immediate consequence of Theorem 5.3 and (5.96) follows by rewriting (5.95). Inequality (5.97) follows by noting $L\mathbf{v} - \mathbf{v}$ is lower bounded by $(\underline{L\mathbf{v} - \mathbf{v}})\mathbf{e}$. Equation (5.98) follows from the third part of Lemma 5.3. Equation (5.99) follows from the fact the rows of $\mathbf{P}_d^n$ sum to 1, i.e., $\mathbf{P}_d^n\mathbf{e} = \mathbf{e}$, for all $n \geq 0$. Noting that the terms involving $\mathbf{v}$ cancel in (5.99) yields the inner lower bound in (5.93). Equation (5.100) follows by replacing $L\mathbf{v} - \mathbf{v}$ in (5.99) by $(\underline{L\mathbf{v} - \mathbf{v}})\mathbf{e}$.

Rewriting the right hand side of (5.94) and using the assumption that $d_\mathbf{v}$ is $\mathbf{v}$-greedy gives

$$\mathbf{v}+(\mathbf{I}-\lambda\mathbf{P}_{d_\mathbf{v}})^{-1}(L\mathbf{v}-\mathbf{v}) = \mathbf{v}+(\mathbf{I}-\lambda\mathbf{P}_{d_\mathbf{v}})^{-1}(\mathbf{r}_{d_\mathbf{v}}+(\lambda\mathbf{P}_{d_\mathbf{v}}-\mathbf{I})\mathbf{v}) = (\mathbf{I}-\lambda\mathbf{P}_{d_\mathbf{v}})^{-1}\mathbf{r}_{d_\mathbf{v}} = \mathbf{v}_\lambda^{d_\mathbf{v}^\infty},$$

implying

$$\mathbf{v}_\lambda^{d_v^\infty} \geq L\mathbf{v} + \lambda(1-\lambda)^{-1}(\underline{L\mathbf{v} - \mathbf{v}})\mathbf{e}.$$

□

Some comments about the bounds in the above theorem follow:

1. Several generalizations of these bounds are possible by splitting (5.96) at higher order terms, but those above are most relevant to what follows.

2. The lower bound based on $\mathbf{v}_\lambda^{d_\mathbf{v}^\infty}$ is also an obvious consequence of the definition of $\mathbf{v}_\lambda^*$. It is not intended for direct evaluation but instead enables you to understand how close $d_\mathbf{v}$ is to optimal.

3. Computing the tighter inner bound requires no additional computational effort since one must evaluate $L\mathbf{v}$ to obtain the looser outer bound.

The following calculations illustrate computation of these bounds in the two-state example.

Example 5.9. From Example 5.5,

$$L\mathbf{v} = \begin{bmatrix} \max\{3 + \lambda(0.8v(s_1) + 0.2v(s_2)), 5 + \lambda v(s_2)\} \\ \max\{-5 + \lambda v(s_2), 2 + \lambda(0.4v(s_1) + 0.6v(s_2))\} \end{bmatrix}.$$

Setting $\lambda = 0.9$ and choosing $\mathbf{v} = (5, -5)$ gives

$$L\mathbf{v} = \begin{bmatrix} \max\{5.7, 0.5\} \\ \max\{-9.5, 1.1\} \end{bmatrix} = \begin{bmatrix} 5.7 \\ 1.1 \end{bmatrix}.$$

Hence $L\mathbf{v} - \mathbf{v} = (0.7, 6.1)$ and $\underline{L\mathbf{v} - \mathbf{v}} = 0.7$ and $\overline{L\mathbf{v} - \mathbf{v}} = 6.1$. Moreover $d_\mathbf{v} = (a_{1,1}, a_{2,2})$ is $\mathbf{v}$-greedy.

The bounds in (5.93) become

$$\begin{bmatrix}66\\56\end{bmatrix} \geq \begin{bmatrix}60.6\\56\end{bmatrix} \geq \mathbf{v}_\lambda^* \geq \begin{bmatrix}27.188\\25.625\end{bmatrix} \geq \begin{bmatrix}12\\7.4\end{bmatrix} \geq \begin{bmatrix}12\\2\end{bmatrix}$$

Since $\mathbf{v}_\lambda^* = (30.147, 27.941)$ neither bound based on $L\mathbf{v}-\mathbf{v}$ is close to the optimal value but the second bound is tighter. Note also that the average of the upper and lower bounds provides an enhanced estimate of $\mathbf{v}_\lambda^*$. It is left as an exercise to evaluate these bounds for other choices of $\mathbf{v}$.

5.5.4 Stopping criteria

Bounds provide a guarantee on the quality of an approximation when terminating an algorithm. The following result will apply to value iteration and any other algorithm that evaluates $L\mathbf{v}$.

Theorem 5.13. For any $\mathbf{v} \in V$ and $\epsilon > 0$, if

$$\mathrm{sp}(L\mathbf{v} - \mathbf{v}) < \frac{1-\lambda}{\lambda}\epsilon \tag{5.101}$$

then

$$\|L\mathbf{v} + \lambda(1-\lambda)^{-1}(\underline{L\mathbf{v}-\mathbf{v}})\mathbf{e} - \mathbf{v}_\lambda^*\| < \epsilon \tag{5.102}$$

and for any $\mathbf{v}$-greedy $d_\mathbf{v} \in D^{\mathrm{MR}}$

$$\|\mathbf{v}_\lambda^{d_\mathbf{v}^\infty} - \mathbf{v}_\lambda^*\| < \epsilon. \tag{5.103}$$

Proof. Note that for any vectors satisfying $\mathbf{x} \geq \mathbf{y} \geq \mathbf{z} \geq \mathbf{w}$, it follows that $\mathbf{x}-\mathbf{w} \geq \mathbf{y}-\mathbf{z} \geq \mathbf{0}$ and $\mathbf{x}-\mathbf{w} \geq \mathbf{y}-\mathbf{w} \geq \mathbf{0}$. Consequently $\|\mathbf{x}-\mathbf{w}\| \geq \|\mathbf{y}-\mathbf{z}\|$ and $\|\mathbf{x}-\mathbf{w}\| \geq \|\mathbf{y}-\mathbf{w}\|$.

Equations (5.102) and (5.103) follow from (5.93) by setting $\mathbf{x} = L\mathbf{v} + \lambda(1-\lambda)^{-1}(\overline{L\mathbf{v}-\mathbf{v}})\mathbf{e}$, $\mathbf{y} = \mathbf{v}_\lambda^*$, $\mathbf{z} = \mathbf{v}_\lambda^{d_\mathbf{v}^\infty}$ and $\mathbf{w} = L\mathbf{v} + \lambda(1-\lambda)^{-1}(\underline{L\mathbf{v}-\mathbf{v}})\mathbf{e}$, applying the above inequalities involving $\mathbf{x}, \mathbf{y}, \mathbf{z}$ and $\mathbf{w}$ and noting that $\mathrm{sp}(L\mathbf{v}-\mathbf{v}) = \overline{L\mathbf{v}-\mathbf{v}} - (\underline{L\mathbf{v}-\mathbf{v}})$ and $\|\mathbf{e}\| = 1$. □

Since most iterative algorithms will not generate a direct estimate of $\mathbf{v}_\lambda^{d_\mathbf{v}^\infty}$, the easily computable estimate referred to as the *lower bound extrapolation*,

$$L\mathbf{v} + \lambda(1-\lambda)^{-1}(\underline{L\mathbf{v}-\mathbf{v}})\mathbf{e},$$

well approximates $\mathbf{v}_\lambda^*$ when (5.101) holds for some small $\epsilon > 0$. However even without using this estimate, it still follows that $d_\mathbf{v}^\infty$ is ϵ-optimal although its value may not be determined without further calculations.

5.5.5 Action elimination

This section begins with the following obvious result, which provides the basis for eliminating non-optimal actions.

Proposition 5.7. Suppose in state $s \in S$,

$$a' \notin \arg\max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v_\lambda^*(j) \right\}$$

or equivalently

$$r(s,a') + \lambda \sum_{j \in S} p(j|s,a') v_\lambda^*(j) < \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v_\lambda^*(j) \right\}. \quad (5.104)$$

Then any stationary policy that selects a' in state s with positive probability cannot be optimal.

Since $\mathbf{v}_\lambda^*$ is unknown, (5.104) cannot be used to identify non-optimal actions. However judiciously replacing it by upper and lower bounds leads to the following practical result.

Theorem 5.14. Suppose $\mathbf{v}^L \leq \mathbf{v}_\lambda^* \leq \mathbf{v}^U$. If

$$r(s,a') + \lambda \sum_{j \in S} p(j|s,a') v^U(j) < v^L(s), \quad (5.105)$$

any stationary policy that selects a' in state s with positive probability cannot be optimal.

Proof. Suppose (5.105) holds. Then

$$r(s,a') + \lambda \sum_{j \in S} p(j|s,a') v_\lambda^*(j) \leq r(s,a') + \lambda \sum_{j \in S} p(j|s,a') v^U(j)$$

$$< v^L(s) \leq v_\lambda^*(s) = \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v_\lambda^*(j) \right\}.$$

Hence the result follows from Proposition 5.7 □

The previous section provided explicit bounds that can be used to eliminate non-optimal actions. Substituting them into (5.105) gives the main result of this section.

Theorem 5.15. Suppose

$$\frac{\lambda}{(1-\lambda)}\text{sp}(L\mathbf{v}-\mathbf{v}) < L\mathbf{v}(s) - \left(r(s,a') + \lambda\sum_{j\in S} p(j|s,a')v(j)\right) \tag{5.106}$$

for some $\mathbf{v} \in V$. Then any stationary policy that selects a' in state s with positive probability cannot be optimal.

Proof. Substitute the outer upper bound and inner lower bound from (5.93) into (5.105) gives

$$r(s,a') + \lambda\sum_{j\in S} p(j|s,a')\Big(v(j)+(1-\lambda)^{-1}(\overline{L\mathbf{v}-\mathbf{v}})\Big) < L\mathbf{v}(s) + \lambda(1-\lambda)^{-1}(\underline{L\mathbf{v}-\mathbf{v}}).$$

Rearranging terms and applying Proposition 5.7 gives the result. □

Some comments about this result follow:

1. Note that the choice of bounds in (5.106) is guided by avoiding additional computation to evaluate
$$r(s,a') + \lambda\sum_{j\in S} p(j|s,a')L\mathbf{v}(j),$$
which would have been necessary if instead the tighter upper bound in (5.93) had been used.

2. By organizing calculations efficiently, the right hand side of (5.106) can be evaluated with minimal extra effort.

3. The next section will show how to integrate this result into value iteration.

4. The literature also proposes action elimination methods that identify actions for temporary elimination, that is, they will not be part of a $\mathbf{v}^n$-greedy decision rule, where $\mathbf{v}^n$ is the n-th iterate of an algorithm but could be in a $\mathbf{v}^m$-greedy decision rule for some $m \geq n$.

This section concludes with an example.

Example 5.10. Continuing with the calculations in Example 5.9 let

$$q(s,a') := L\mathbf{v}(s) - \left(r(s,a') + \lambda\sum_{j\in S} p(j|s,a')v(j)\right)$$

denote the quantity on the right hand side of (5.106). Start with $\mathbf{v}^0 = (5,-5)$

and compute $\mathbf{v}^1 = L\mathbf{v}^0$ and $\mathbf{v}^2 = L\mathbf{v}^1$ and in each case evaluate the expressions on either side of (5.106). Results are given in the following table.

n	$\mathbf{v}^n$	$\frac{\lambda}{1-\lambda}\text{sp}(L\mathbf{v}^n - \mathbf{v}^n)$	$q(s_1, a_{1,1})$	$q(s_1, a_{1,2})$	$q(s_2, a_{2,1})$	$q(s_2, a_{2,2})$
0	$(5, -5)$	48.6	0	5.2	10.6	0
1	$(5.7, 1.1)$	17.50	0	1.31	8.66	0
2	$(7.30, 4.65)$	**5.51**	0.09	0	**7.96**	0

Observe that when $\mathbf{v} = \mathbf{v}^2$, the inequality in (5.106) holds in state s_2 with $a' = a_{2,1}$ as denoted by bold text. Hence this action cannot be optimal in state 2 and at subsequent iterations does not need to be evaluated.

Continuing the above calculations shows that when $\mathbf{v} = \mathbf{v}^5 = (13.24, 11.06)$, action $a_{1,1}$ is eliminated in s_1. Hence six iterations of $\mathbf{v}^{n+1} = L\mathbf{v}^n$, establish that the stationary policy d^∞ with $d = (a_{1,2}, a_{2,2})$ is optimal. This is very surprising because $\mathbf{v}^5$ is quite far from the optimal value $\mathbf{v}^*_\lambda = (30.147, 27.941)$. Note further that that value obtained by the lower bound extrapolation in (5.102) equals $(30.08, 27.87)$, which is a close approximation to $\mathbf{v}^*_\lambda$.

This example illustrates how to incorporate action elimination in an iterative procedure. It shows that the optimal policy has been identified when all but one action in each state has been eliminated. While this may be the case in certain situations, in our experience, action elimination methods are not always so rewarding.

5.6 Value iteration

Value iteration is the most widely used and easiest to implement method for solving discounted models. It generalizes the standard successive approximations approach for solving fixed point equations of the form $f(x) = x$. It is frequently referred to as *backwards induction*, *successive approximation* or even *dynamic programming*.

5.6.1 A value iteration algorithm

Figure 5.5 provides a schematic representation of a value iteration algorithm. Observe its simplicity: it repeats $\mathbf{v}' \leftarrow L\mathbf{v}$ until a stopping criterion (described below) is satisfied.

The following algorithmic statement formalizes the above flow chart. It combines all steps in the loop and specifies how to choose a policy and estimate its value at termination. The stopping criterion will be discussed subsequently.

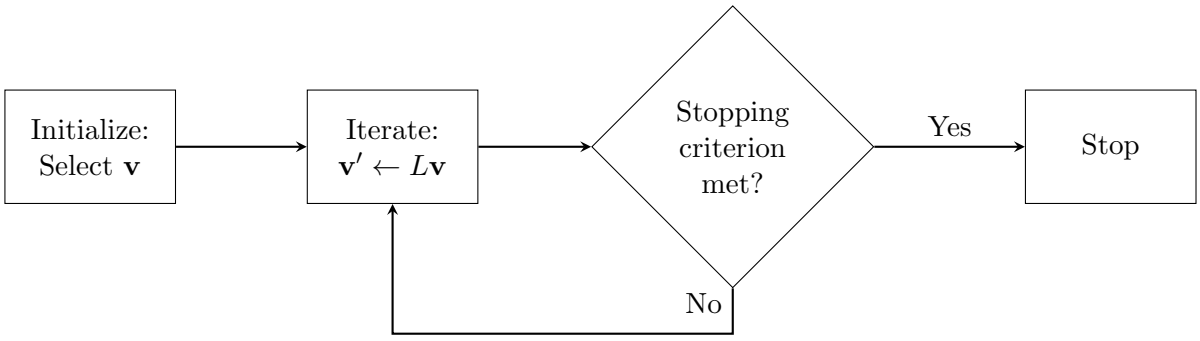


Figure 5.5: Flow diagram for value iteration

Algorithm 5.2. Value iteration: vector form

1. **Initialize:** Specify $\mathbf{v} \in V$ and a stopping criterion that involves an $\epsilon > 0$.
2. **Iterate:** Do until stopping criterion met:
$$\mathbf{v}' \leftarrow L\mathbf{v}.$$
3. **Choose an ϵ-optimal policy:** Select
$$d_\epsilon \in \arg\operatorname*{c\text{-}max}_{d \in D^{\mathrm{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}.$$
4. **Approximate the optimal value function:** Set
$$\mathbf{v}_\lambda^\epsilon \leftarrow \mathbf{v}' + \frac{\lambda}{1-\lambda}(\underline{\mathbf{v}' - \mathbf{v}})\mathbf{e}.$$

In practice, value iteration is applied component-wise as follows. Iterates in the algorithm below are represented by superscripts on the value function to remain consistent with this book's use of subscripts to denote decision epochs or specific states.

Algorithm 5.3. Value iteration: component form

1. **Initialize:**
   (a) Specify $v'(s)$ for all $s \in S$.
   (b) Specify $\epsilon > 0$ and $\sigma > (1-\lambda)\epsilon/\lambda$.

2. **Iterate:** While
$$\sigma \geq \frac{1-\lambda}{\lambda}\epsilon : \tag{5.107}$$
   (a) $v(s) \leftarrow v'(s)$ for all $s \in S$.
   (b) For all $s \in S$, compute
$$v'(s) \leftarrow \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\}. \tag{5.108}$$
   (c) $\sigma \leftarrow \mathrm{sp}(\mathbf{v}' - \mathbf{v})$.
3. **Terminate:** For all $s \in S$, return
$$d_\epsilon(s) \in \arg\max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\}$$
   and
$$v_\lambda^\epsilon(s) = v'(s) + \frac{\lambda}{1-\lambda}(\underline{\mathbf{v}' - \mathbf{v}}).$$

Before providing a proof of convergence, some comments are in order:

1. The span (as opposed to the norm) is used as the basis for terminating the algorithm. Section 5.5.1 suggested that the span-based stopping criterion will be met in fewer iterations than the norm-based stopping criterion. This will be explored in numerical examples below.
2. The condition $\sigma \geq (1-\lambda)\epsilon/\lambda$ will be shown to result in an ϵ-optimal policy upon termination. This should be sufficient in most practical examples.
3. The expression for $v^\epsilon(s)$, referred to as the lower bound extrapolation in Section 5.5.4, can be determined with little extra computational effort. It is important to emphasize that $\mathbf{v}^\epsilon$ should be used instead of $\mathbf{v}$ to approximate $\mathbf{v}_\lambda^*$ because Theorem 5.13 ensures that each component of the lower bound extrapolation will be within ϵ of the optimal value function.
4. The iterative step (5.108) represents the essence of the algorithm. In operator notation, it was represented by $\mathbf{v}' = L\mathbf{v}$.
5. Judicious choice of a starting value can enhance convergence. Choosing the lower bound $v(s) = (1-\lambda)^{-1} \min_{a \in A_s} r(s,a)$ ensures monotone convergence.
6. In most practical examples, the transition probabilities will be mostly zero so that for each $s \in S$, the sum on the right hand side of (5.108) will only contain a

few terms. For example, in the queuing service rate control model (Section 3.4.1), there will be at most three non-zero probabilities in each state, enabling the sum to be easily coded and quickly computed.

7. When the stopping criterion is satisfied, specifying $d_\epsilon(s)$ requires no addition computation. It can be determined while computing (5.108). Theorem 5.13 ensures that it is ϵ-optimal. Note that its value is not determined by the algorithm and $d_\epsilon(s)$ need not be unique.

8. Action elimination can be used to enhance the algorithm. This will be discussed below.

9. In addition to computing ϵ-optimal policies, value iteration can be used to demonstrate the structure of optimal policies.

10. Some authors refer to the above algorithm as *synchronous* value iteration because at each iteration, the estimate is updated in all states. Gauss-Seidel value iteration (described below) provides an *asynchronous* alternative.

The following theorem establishes convergence of value iteration to an ϵ-optimal policy. Its proof follows by combining several of the above theorems.

Theorem 5.16. Suppose $0 \leq \lambda < 1$ and $\epsilon > 0$. Then for any $\mathbf{v}^0 \in V$ the sequence $\mathbf{v}^n, n = 1, 2, \ldots$ generated by value iteration has the following properties:

1. $\|\mathbf{v}^n - \mathbf{v}^*_\lambda\| \to 0$ as $n \to \infty$.
2. $\mathrm{sp}(\mathbf{v}^n - \mathbf{v}^*_\lambda) \to 0$.
3. For each $\epsilon > 0$, there exists a finite N which for $n \geq N$, (5.107) holds.
4. When (5.107) is satisfied,
   (a) $\|\mathbf{v}^\epsilon - \mathbf{v}^*_\lambda\| < \epsilon$, and
   (b) $\|\mathbf{v}_\lambda^{d_\epsilon^\infty} - \mathbf{v}^*_\lambda\| < \epsilon$.

Proof. Proposition 5.1 establishes that L is a contraction mapping. Hence by Theorem 5.5, $\mathbf{v}^n$ converges to a fixed point of L in norm. From Theorem 5.8, the limit is unique and equals $\mathbf{v}^*_\lambda$. Since $\mathrm{sp}(\mathbf{v}) \leq 2\|\mathbf{v}\|$, part 2 follows. Part 3 follows from Theorem 5.11 and the observation that

$$\mathrm{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n) = \mathrm{sp}(L^{n+1}\mathbf{v}^0 - L^n\mathbf{v}^0) \leq (\lambda\gamma^*)^n \mathrm{sp}(L\mathbf{v}^0 - \mathbf{v}^0).$$

Part 4 is a consequence of Theorem 5.13. □

Example 5.11. This example finds an ϵ-optimal policy in the two-state model using value iteration with $\lambda = 0.9$, $\epsilon = 0.000001$ and starting value $\mathbf{v} = \mathbf{0}$.

Using $\|\mathbf{v}' - \mathbf{v}\|$, the algorithm terminates after 162 iterations and using $\text{sp}(\mathbf{v}' - \mathbf{v})$, the algorithm terminates after 16 iterations. In both cases the algorithm terminates with the *optimal* stationary policy that uses decision rule $d_\epsilon = (a_{1,2}, a_{2,2,})$.

Figure 5.6 compares the speed of convergence of $\mathbf{v}' - \mathbf{v}$ with respect to these two measures. Observe that the span-based criterion converges more quickly than the norm-based criterion for the same sequence of iterates. This in a consequence of the observation in Example 5.8, that $\gamma^* = 0.8$.

At termination using the span stopping criterion, $\mathbf{v} = (25.39, 23.19)$ and $\mathbf{v}^\epsilon = (30.15, 27.94)$. Using the norm stopping criterion, $\mathbf{v} = \mathbf{v}^\epsilon = (30.15, 27.94)$. Thus the span converges 10 times faster than the norm. Moreover the lower bound extrapolation produces identical estimates of the optimal value function at termination under either a norm-based or span-based stopping criterion.

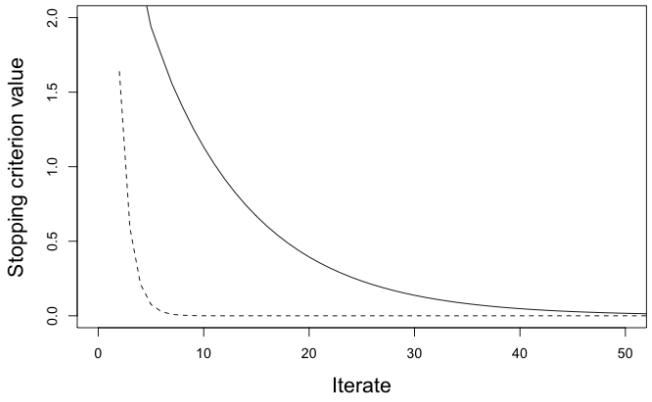


Figure 5.6: Comparison of convergence rate of $\text{sp}(\mathbf{v}' - \mathbf{v})$ (dashed line) to $||\mathbf{v}' - \mathbf{v}||$ (solid line) in two-state example.

Convergence rates and error bounds

Algorithms are frequently compared on the basis of their convergence rates. A discussion of some relevant concepts follows.

Let $\mathbf{u}^n; n = 1, 2, \ldots$ denote a sequence of vectors converging (in norm) to $\mathbf{u}^*$. Then $\mathbf{u}^n$ *converges with order* $\alpha > 0$ if there exists a constant $K > 0$ for which

$$||\mathbf{u}^{n+1} - \mathbf{u}^*|| \leq K||\mathbf{u}^n - \mathbf{u}^*||^\alpha.$$

Convergence of order 1 is referred to as *linear convergence* and convergence with order

2 is referred to as *quadratic convergence*[23]. Clearly for fixed K, the higher the order, the faster the convergence[24].

The sequence $\mathbf{u}^n, n = 1, 2, \ldots$ *converges geometrically at rate* β if there exists a constant $k > 0$ and $0 < \beta < 1$ for which

$$||\mathbf{u}^n - \mathbf{u}^*|| \leq K\beta^n.$$

This is often expressed as $\mathbf{u}^n = \mathbf{u}^* + O(\beta^n)$.

The following result summarizes convergence rate properties of value iteration.

Theorem 5.17. Let $\mathbf{v}^n : n = 1, 2, \ldots$ denote a sequence of iterates of value iteration where $\mathbf{v}^0$ is arbitrary. Then:

1. $\mathbf{v}^n$ converges linearly to $\mathbf{v}^*_\lambda$.
2. $\mathbf{v}^n$ converges to $\mathbf{v}^*_\lambda$ at rate λ^n.
3. For $n = 1, 2, \ldots$

$$||\mathbf{v}^{n+1} - \mathbf{v}^*_\lambda|| = \frac{\lambda^n}{(1-\lambda)}||\mathbf{v}^1 - \mathbf{v}^0|| \tag{5.109}$$

Proof. Parts 1 and 2 follow directly from the definitions. To prove part 3, note that

$$\begin{aligned}
\|\mathbf{v}^{n+1} - \mathbf{v}^*_\lambda\| &= \|L\mathbf{v}^n - L\mathbf{v}^*_\lambda\| \\
&\leq \|L\mathbf{v}^n - L\mathbf{v}^{n+1}\| + \|L\mathbf{v}^{n+1} - L\mathbf{v}^*_\lambda\| \\
&\leq \|L^n\mathbf{v}^1 - L^n\mathbf{v}^0\| + \lambda\|\mathbf{v}^{n+1} - \mathbf{v}^*_\lambda\| \\
&\leq \lambda^n\|\mathbf{v}^1 - \mathbf{v}^0\| + \lambda\|\mathbf{v}^{n+1} - \mathbf{v}^*_\lambda\|.
\end{aligned}$$

The result follows by rearranging terms in the last inequality. □

Expression (5.109) is referred to as an *error bound* and is consistent with the first two statements in the theorem.

5.6.2 Gauss-Seidel value iteration

Gauss-Seidel is a variant of value iteration that updates values as soon as they have been computed instead of waiting until finishing a complete pass through step 2 of Algorithm 5.3. Its motivation is that using updated values as soon as possible will speed up convergence. Proving that this is the case requires machinery that is outside the scope of this book but numerical results below support this conjecture.

[23] For example when solving $f(x) = x$ or equivalently $f(x) - x = 0$, under mild conditions, successive approximations converges linearly and Newton's method converges quadratically.

[24] Note that when $\alpha > 1$, this statement only makes when the error is less than 1.

A description of a Gauss-Seidel value iteration algorithm follows. Assume that value iteration evaluates states using (5.108) in order $s_1, s_2, \ldots, s_M$. Noting that $v'(s_j)$ for $j < k$ have been calculated prior to evaluating $v'(s_k)$, Gauss-Seidel value iteration accounts for this by replacing (5.108) in state s_k with

$$v'(s_k) = \max_{a \in A_{s_k}} \left\{ r(s_k, a) + \lambda \left(\sum_{j<k} p(s_j|s_k, a) v'(s_j) + \sum_{j \geq k} p(s_j|s_k, a) v(s_j) \right) \right\}. \quad (5.110)$$

Thus, it uses the most current estimate of the value function in all states when doing its update. It is most straightforward to state in component notation.

For $k = 1, \ldots, M$, inductively define the s_k-th component of the operator $G : V \to V$ by

$$G\mathbf{v}(s_k) := \max_{a \in A_{s_k}} \left\{ r(s_k, a) + \lambda \left(\sum_{j<k} p(s_j|s_k, a) G\mathbf{v}(s_j) + \sum_{j \geq k} p(s_j|s_k, a) v(s_j) \right) \right\}. \quad (5.111)$$

This means that $G\mathbf{v}(s_1) = L\mathbf{v}(s_1)$,

$$G\mathbf{v}(s_2) := \max_{a \in A_{s_2}} \left\{ r(s_2, a) + \lambda \left(p(s_1|s_2, a) G\mathbf{v}(s_1) + \sum_{j=2}^{M} p(s_j|s_2, a) v(s_j) \right) \right\},$$

$$G\mathbf{v}(s_3) := \max_{a \in A_{s_3}} \left\{ r(s_3, a) + \lambda \left(\sum_{j=1}^{2} p(s_j|s_3, a) G\mathbf{v}(s_j) + \sum_{j=3}^{M} p(s_j|s_3, a) v(s_j) \right) \right\}$$

and so forth.

An iterative algorithm that uses Gauss-Seidel updates follows. It assumes that states are labeled $s_1, \ldots, s_M$ and are evaluated in that order. It is clearest when stated in component form.

Algorithm 5.4. Gauss-Seidel value iteration: component form

1. **Initialize:**
   (a) Specify $v'(s)$ for all $s \in S$.
   (b) Specify $\epsilon > 0$ and $\sigma > (1 - \lambda)\epsilon/2\lambda$
2. **Iterate:** While
$$\sigma \geq \frac{(1 - \lambda)}{2\lambda} \epsilon : \quad (5.112)$$
   (a) $v(s) \leftarrow v'(s)$ for all $s \in S$.

(b) For $k = 1, \ldots, M$, compute

$$v'(s_k) \leftarrow \max_{a \in A_s} \left\{ r(s,a) + \lambda \left(\sum_{j<k} p(s_j|s_k,a) v'(s_j) + \sum_{j \geq k} p(s_j|s_k,a) v(s_j) \right) \right\}. \tag{5.113}$$

(c) $\sigma \leftarrow \|\mathbf{v}' - \mathbf{v}\|$.

3. **Terminate:** For $k = 1, \ldots, M$, return

$$d_\epsilon(s_k) \in \arg\max_{a \in A_s} \left\{ r(s,a) + \lambda \left(\sum_{j<k} p(s_j|s_k,a) v'(s_j) + \sum_{j \geq k} p(s_j|s_k,a) v(s_j) \right) \right\}. \tag{5.114}$$

Some comments about the algorithm follow.

1. This update is said to be *asynchronous* because state values are updated prior to completing a pass through an iteration. Online reinforcement learning and simulation-based algorithms (Chapters 10 and 11) use asynchronous updates since values are updated as soon as states are observed or simulated.

2. This algorithm uses a norm-based stopping criterion since the bounds used to derive the span-based criterion for value iteration are not available. If one wishes to instead use a span-based stopping criterion, it can be applied intermittently after using a value iteration update.

3. The decision rule specification in step 3 is consistent with the key iterative step based on (5.113). Doing so simplifies the proof that $\|\mathbf{v}^{(d_\epsilon)^\infty} - \mathbf{v}^*_\lambda\| < \epsilon$ at termination. Alternatively one can choose d_ϵ to be $\mathbf{v}'$-greedy (corresponding to an extra value iteration step)

4. The ordering of the states is immaterial and can vary from iteration to iteration. The analysis in this section remains valid as long as there is a complete pass through all states at each iteration.

5. When the transition matrices corresponding to all decision rules are upper-triangular, Gauss-Seidel value iteration and value iteration are identical.

Convergence of Gauss-Seidel iteration

A proof of the convergence of Gauss-Seidel iteration to $\mathbf{v}^*_\lambda$ is based on showing that the operator representing a Gauss-Seidel update is a contraction mapping and then applying the Banach fixed point theorem. The following result is obtained by modifying the proof of Proposition 5.1 accordingly.

Proposition 5.8. Suppose $G : V \to V$ is defined by (5.111) and $0 \leq \lambda < 1$. Then G is a contraction mapping with modulus (at most) λ with respect to the sup-norm.

Proof. The proof uses induction over states $s \in S$. Choose $\mathbf{v}$ and $\mathbf{u}$ in V. For $s = s_1$, since $G\mathbf{v}(s_1) = L\mathbf{v}(s_1)$, it follows that

$$|G\mathbf{v}(s_1) - G\mathbf{u}(s_1)| \leq \lambda \|\mathbf{v} - \mathbf{u}\| \tag{5.115}$$

from the same argument used in the proof of Proposition 5.1.

Next assume (5.115) holds when s_j replaces s_1 for $j = 1, \ldots, k-1$. Define $a' \in A_{s_k}$ by

$$\begin{aligned}
&r(s_k, a') + \lambda \left(\sum_{j<k} p(s_j|s_k, a') G\mathbf{v}(s_j) + \sum_{j \geq k} p(s_j|s_k, a') v(s_j) \right) \\
&= \max_{a \in A_s} \left\{ r(s_k, a) + \lambda \left(\sum_{j<k} p(s_j|s_k, a) G\mathbf{v}(s_j) + \sum_{j \geq k} p(s_j|s_k, a) v(s_j) \right) \right\} = G\mathbf{v}(s_k).
\end{aligned}$$

Assuming first that $G\mathbf{v}(s_k) \geq G\mathbf{u}(s_k)$,

$$\begin{aligned}
0 &\leq G\mathbf{v}(s_k) - G\mathbf{u}(s_k) \\
&\leq r(s_k, a') + \lambda \left(\sum_{j<k} p(s_j|s_k, a') G\mathbf{v}(s_j) + \sum_{j \geq k} p(s_j|s_k, a') v(s_j) \right) \\
&\quad - \left\{ r(s_k, a') + \lambda \left(\sum_{j<k} p(s_j|s_k, a') G\mathbf{u}(s_j) + \sum_{j \geq k} p(s_j|s_k, a') u(s_j) \right) \right\} \\
&= \lambda \left(\sum_{j<k} p(j|s_k, a') \Big(G\mathbf{v}(s_j) - G\mathbf{u}(s_j) \Big) + \sum_{j \geq k} p(j|s_k, a') \Big(v(s_j) - u(s_j) \Big) \right) \\
&\leq \lambda \left(\sum_{j<k} p(s_j|s_k, a') \big| G\mathbf{v}(s_j) - G\mathbf{u}(s_j) \big| + \sum_{s_j \geq k} p(j|s_k, a') \|\mathbf{v} - \mathbf{u}\| \right) \\
&\leq \lambda \|\mathbf{v} - \mathbf{u}\|,
\end{aligned}$$

where the last inequality follows from the induction hypothesis.

Repeating the above argument for $G\mathbf{u}(s_k) \geq G\mathbf{v}(s_k)$ establishes that (5.115) holds for s_k, completing the induction step and the proof. □

The following theorem establishes convergence properties of Gauss-Seidel iteration. The proof of parts 2 and 3 differ from that for value iteration since bounds like those in (5.93) are not available for $G\mathbf{v}$.

Theorem 5.18. Suppose $0 \leq \lambda < 1$ and $\epsilon > 0$. Let the sequence $\mathbf{v}^n, n = 1, 2, \ldots$ be generated by Gauss-Siedel value iteration. Then:

1. $\|\mathbf{v}^n - \mathbf{v}^*_\lambda\| \to 0$ as $n \to \infty$,
2. $\text{sp}(\mathbf{v}^n - \mathbf{v}^*_\lambda) \to 0$.
3. For each $\epsilon > 0$, there exists a finite N which for $n \geq N$, (5.112) holds.
4. When (5.112) is satisfied,
   (a) d^∞_ϵ is ϵ-optimal,
   (b) for $n \geq N$, $\|\mathbf{v}^n - \mathbf{v}^*_\lambda\| < \epsilon/2\lambda$ and $\text{sp}(\mathbf{v}^n - \mathbf{v}^*_\lambda) < \epsilon/\lambda$.

Proof. Since G is a contraction, $G^n\mathbf{v}_0$ converges to a fixed point $\mathbf{v}^*$ by Theorem 5.5. Since $G\mathbf{v}^* = \mathbf{v}^*$, from (5.111),

$$v^*(s_k) = G\mathbf{v}^*(s_k) = \max_{a \in A_s} \left\{ r(s_k, a) + \lambda \left(\sum_{j<k} p(s_j|s_k, a) G\mathbf{v}^*(s_j) + \sum_{j \geq k} p(s_j|s_k, a) v^*(s_j) \right) \right\}$$

$$= \max_{a \in A_s} \left\{ r(s_k, a) + \lambda \left(\sum_{j<k} p(s_j|s_k, a) v^*(s_j) + \sum_{j \geq k} p(s_j|s_k, a) v^*(s_j) \right) \right\} = L\mathbf{v}^*(s_k)$$

Hence, $\mathbf{v}^*$ is also a fixed point of L. Since L has the unique fixed point $\mathbf{v}^*_\lambda$ the first result follows. Part 2 follows by noting again that $\text{sp}(\mathbf{v}) \leq 2\|\mathbf{v}\|$. Part 3 follows directly from part 1.

To prove 4(a), note that in Step 4 of Algorithm 5.4 that $G\mathbf{v}^{n+1} = G_{d_\epsilon}\mathbf{v}^{n+1}$ where G_d denotes the operator that corresponds to implementing a single step of Gauss-Seidel value iteration with a fixed decision rule d. Hence

$$\begin{aligned} \|\mathbf{v}^{(d_\epsilon)^\infty}_\lambda - \mathbf{v}^*_\lambda\| &\leq \|\mathbf{v}^{(d_\epsilon)^\infty}_\lambda - G_{d_\epsilon}\mathbf{v}^n\| + \|G\mathbf{v}^n - \mathbf{v}^*_\lambda\| \\ &\leq \lambda\|\mathbf{v}^{(d_\epsilon)^\infty} - \mathbf{v}^n\| + \lambda\|\mathbf{v}^n - \mathbf{v}^*_\lambda\|, \end{aligned} \tag{5.116}$$

where the last inequality follows by noting that G and G_{d_ϵ} are contraction mappings with fixed points $\mathbf{v}^*_\lambda$ and $\mathbf{v}^{(d_\epsilon)^\infty}_\lambda$, respectively. It is easy to show[25] that each of the quantities on the right hand side of (5.116) is bounded by $(1-\lambda)^{-1}\|\mathbf{v}^{n+1} - \mathbf{v}^n\|$, from which the result follows.

To prove 4(b), since $\mathbf{v}^{n+1} = G\mathbf{v}^n$ and $\mathbf{v}^*_\lambda$ is the fixed point of G,

$$\begin{aligned} \|\mathbf{v}^n - \mathbf{v}^*_\lambda\| &\leq \|\mathbf{v}^{n+1} - \mathbf{v}^n\| + \|\mathbf{v}^{n+1} - \mathbf{v}^*_\lambda\| \\ &\leq \|\mathbf{v}^{n+1} - \mathbf{v}^n\| + \|G\mathbf{v}^n - G\mathbf{v}^*_\lambda\| \leq \|\mathbf{v}^{n+1} - \mathbf{v}^n\| + \lambda\|\mathbf{v}^n - \mathbf{v}^*_\lambda\|. \end{aligned}$$

Hence $(1-\lambda)\|\mathbf{v}^n - \mathbf{v}^*_\lambda\| \leq \|\mathbf{v}^{n+1} - \mathbf{v}^n\|$, which gives the result. □

[25] See Puterman (1994) p. 162 or apply a telescoping sum argument.

Example 5.12. This example applies Gauss-Seidel value iteration to the two-state model starting from $\mathbf{v} = \mathbf{0}$ and setting $\lambda = 0.9$ and $\epsilon = 0.000001$.

Using stopping criterion (5.112), the algorithm terminates in 120 iterations with $\mathbf{v}' = (30.147, 27.941)$. Note that $\mathbf{v}' = \mathbf{v}^{(d_\epsilon)^\infty} = \mathbf{v}^*_\lambda$ (to at least 3 decimal places) so that the ϵ-optimal policy is optimal.

Figure 5.7 compares the speed of convergence of $\|\mathbf{v}' - \mathbf{v}'\|$ for value iteration and Gauss-Seidel value iteration. Note that for successive iterates $\mathbf{v}'', \mathbf{v}'$ and $\mathbf{v}$, the ratio $\|\mathbf{v}'' - \mathbf{v}'\| / \|\mathbf{v}' - \mathbf{v}\| \approx 0.86$ for Gauss-Seidel, while equaling 0.9 for value iteration. This suggests a smaller contraction modulus for Gauss-Seidel in this example.

The figure also shows $\text{sp}(\mathbf{v}' - \mathbf{v})$ for Gauss-Seidel indicating faster convergence with respect to this measure, even in the absence of theoretical justification.

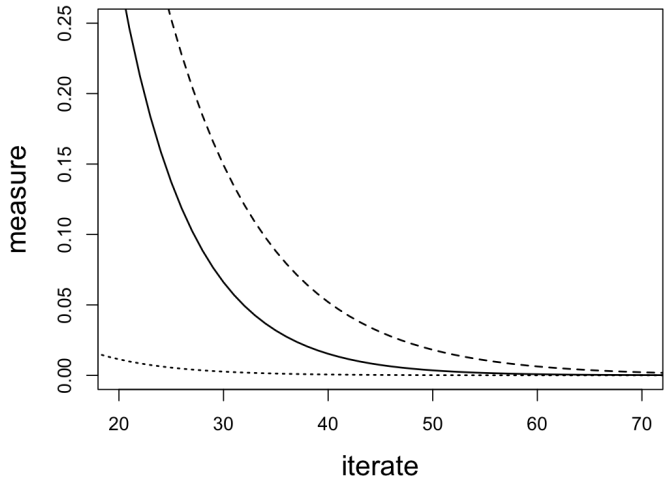


Figure 5.7: Comparison of speed of convergence of Gauss-Seidel and value iteration for 2-state example. The solid line shows $\|\mathbf{v}' - \mathbf{v}\|$ for Gauss-Seidel, the dashed line shows $\|\mathbf{v}' - \mathbf{v}\|$ for value iteration and the dotted line shows $\text{sp}(\mathbf{v}' - \mathbf{v})$ for Gauss-Seidel.

A hybrid approach plus some discussion

Appendix 5.13.2 provides bounds and develops some interesting matrix representations for Gauss-Seidel iteration. As shown in Section 5.5.4, the span stopping criterion for value iteration is based on subtracting the lower bound from the upper bound as in Theorem 5.13. Since the lower bound for Gauss-Seidel cannot be evaluated easily, a hybrid algorithm that intersperses a value iteration step in Gauss-Seidel iteration

enables computation of bounds and allows use of the span-based criterion. Hence, we recommend in practice to either:

1. use such a hybrid approach when solving large problems with Gauss-Seidel, or
2. simply use value iteration with the span stopping criterion.

The advantage of the later approach is the availability of bounds, the applicability of the span-based stopping criterion and termination with a good estimate of the value function. The advantage of the former approach is that in large problems, updates of the value function can be used more quickly, perhaps leading to faster convergence.

Asynchronous value iteration

In an asynchronous value iteration algorithm states are updated one at a time. In particular a single update is given by

$$v^{n+1}(s) = \begin{cases} \max_{a \in A_s} \{r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v^n(j)\} & s = s' \\ v^n(s) & s \neq s'. \end{cases} \tag{5.117}$$

for some chosen state s'. If all states are cycled through without repeating any states, this is exactly a Gauss-Seidel update. However, if states are chosen randomly, some might repeat before others are evaluated even once. As long as all states are evaluated infinitely often, the algorithm converges to $\mathbf{v}^*_\lambda$.

Asynchronous updates underlie simulation-based algorithms discussed in Chapter 10 and also updates generated by parallel computation. The reader is referred to the literature for more details and a proof of convergence.

5.6.3 Value iteration with action elimination

The purpose of augmenting an algorithm with an action elimination procedure is to reduce the number of actions evaluated when performing a maximization. This brief section shows how to use the result in Proposition 5.7 to do so in the context of value iteration. It requires minimal extra computation but some additional storage.

Algorithm 5.5. Value iteration with action elimination: component form

1. **Initialize:**
   (a) Specify $v'(s)$ for all $s \in S$.
   (b) Set $H_s \leftarrow A_s$ for all $s \in S$.
   (c) Specify $\epsilon > 0$ and $\sigma > (1-\lambda)\epsilon/\lambda$.

2. **Iterate:** While $\sigma \geq (1-\lambda)\epsilon/\lambda$ and H_s contains two or more elements for some $s \in S$:

   (a) $v(s) \leftarrow v'(s)$ for all $s \in S$.

   (b) **Update values:** For all $s \in S$ and $a' \in H_s$, compute

   $$q(s,a') \leftarrow r(s,a') + \lambda \sum_{j \in S} p(j|s,a')v(j) \tag{5.118}$$

   and

   $$v'(s) \leftarrow L\mathbf{v}(s) = \max_{a \in H_s} q(s,a).$$

   (c) **Identify and eliminate non-optimal actions:** For all $s \in S$,

   $$H_s \leftarrow \left\{ a' \in H_s \;\middle|\; \frac{\lambda}{(1-\lambda)} \mathrm{sp}(\mathbf{v}' - \mathbf{v}) \geq v'(s) - q(s,a') \right\}. \tag{5.119}$$

   (d) $\sigma \leftarrow \mathrm{sp}(\mathbf{v}' - \mathbf{v})$.

3. **Terminate:** For all $s \in S$, return

   $$d_\epsilon(s) \in \arg\max_{a \in H_s} q(s,a) \tag{5.120}$$

   and

   $$v_\lambda^\epsilon(s) = v'(s) + \frac{\lambda}{1-\lambda}(\underline{\mathbf{v}' - \mathbf{v}}). \tag{5.121}$$

Some comments on this algorithm follow:

1. Applying this algorithm results in *either*:

   (a) an ϵ-optimal policy when terminated with the span-based stopping criterion, or

   (b) the unique optimal policy when H_s contains a single element for all $s \in S$.

   In both cases $\mathbf{v}^\epsilon$ is within ϵ of the optimal value function.

2. The algorithm identifies an optimal policy, rather than an ϵ-optimal policy, only when it terminates when H_s contains a single element for all $s \in S$.

3. Note that it is necessary to evaluate $L\mathbf{v}(s)$ for all $s \in S$ before eliminating actions. This requires steps 2(b) and 2(c).

4. The quantity $q(s,a)$ does not need to be stored from iteration to iteration. Its evaluation requires no extra computation at each iteration since it is an interme-

diate calculation in determining $L\mathbf{v}$. When using value iteration without action elimination, it need not be stored for any state.

5. Note that $|H_s|$ is non-increasing for all $s \in S$ between successive iterations.

6. This algorithm does not generalize easily to to Gauss-Seidel value iteration because of the unavailability of easily computable lower bounds.

Example 5.10 shows that starting with $\mathbf{v}^0 = (5, -5)$, action $a_{2,1}$ is eliminated in s_1 at the third iteration and action $a_{1,1}$ is eliminated in state s_1 after the sixth iteration. Thus it requires only **six** iterations to identify the optimal policy using action elimination in the two-state example. Such dramatic results cannot be expected in all applications.

5.7 Policy iteration

Policy iteration (also known as policy improvement or approximation in policy space) offers an attractive alternative to value iteration for finding optimal policies in discounted Markov decision processes. Actor-critic methods (see Chapter 11) may be regarded as policy iteration using function approximation and simulation.

While value iteration has its foundations in fixed point theory, policy iteration exploits the special structure of Markov decision processes, but may be also viewed as an application of Newton's method for finding the zero of a function. While value iteration may only find an ϵ-optimal policy, policy iteration will find an optimal policy in a finite state and action model.

5.7.1 The policy iteration algorithm

A schematic representation of policy iteration appears in Figure 5.8. It shows that policy iteration cycles between an evaluation step that solves a linear system to find the value of a policy and an improvement step that chooses a greedy decision rule with respect to the value of the previously evaluated policy. When greedy decision rules repeat, which occurs with certainty in a finite state and action model, the algorithm terminates with an optimal stationary deterministic policy.

A statement of policy iteration in vector notation follows. We recommend that you implement the component form of the algorithm stated below, because it makes the improvement step more explicit.

Algorithm 5.6. Policy iteration: vector form

1. **Initialize:** Specify $d \in D^{\text{MD}}$ and set $\Gamma \leftarrow S$.

2. **Iterate:** While $\Gamma \neq \emptyset$:

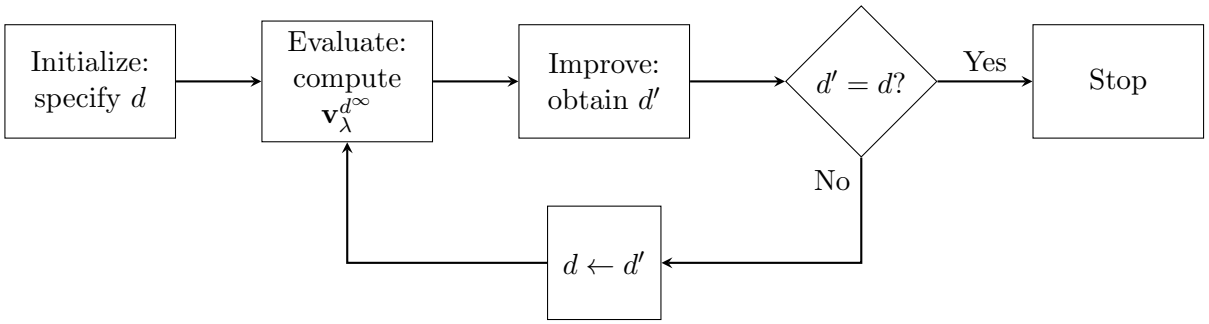


Figure 5.8: Flow diagram for policy iteration

(a) **Evaluate:** Find $\mathbf{v}'$ by solving

$$(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d. \tag{5.122}$$

(b) **Improve:** Choose

$$d' \in \underset{\delta \in D^{\mathrm{MD}}}{\arg\,\text{c-max}} \left\{ \mathbf{r}_\delta + \lambda \mathbf{P}_\delta \mathbf{v}' \right\}, \tag{5.123}$$

setting $d' = d$ if possible.

(c) $\Gamma \leftarrow \{s \in S \,|\, d'(s) \neq d(s)\}$.

(d) **Update:** $d \leftarrow d'$.

3. **Terminate:** Return $d^* = d$ and $\mathbf{v}_\lambda^* = \mathbf{v}'$.

Before stating the algorithm in a form that is more suitable for computation, note that:

1. In contrast to value iteration, policy iteration begins with a Markovian deterministic decision rule instead of a value. On the other hand, the algorithm can start in step 2(b) with any $\mathbf{v} \in V$.

2. The evaluation step involves solving a system of linear equations, which requires at most $O(|S|^3)$ operations. Modified policy iteration, described below, replaces this step with an iterative method for approximating the value function of d^∞.

3. The improvement step requires the same effort as one pass through value iteration. We emphasize that it is implemented component-wise thus avoiding enumerating all Markovian deterministic decision rules.

4. The instruction "set $d' = d$ if possible" prevents cycling when the maximizer is non-unique in some states. It is also ensures that algorithm terminates in a

finite number of iterations when there are finitely many deterministic stationary policies. This rule can be replaced by a stopping criterion expressed in terms of value functions: stop when successive value functions are sufficiently close. The advantage of such a rule is that it can be used to terminate the algorithm when the set of states and/or actions is not finite.

5. An action elimination procedure can be incorporated into policy iteration.

6. The next section shows that policy iteration can be implemented asynchronously. That is, as soon as a strict improvement is identified in a state or subset of states, the improved policy can be updated.

A statement of policy iteration in component notation follows. This description is more suitable for computation and emphasizes that the improvement step should be implemented state by state.

Algorithm 5.7. Policy iteration: component form

1. **Initialize:** Specify $a_s \in A_s$ for all $s \in S$ and set $\Gamma \leftarrow S$.

2. **Iterate:** While $\Gamma \neq \emptyset$:

   (a) **Evaluate:** Obtain $v'(s)$ for all $s \in S$ by solving the system of linear equations

   $$v(s) - \lambda \sum_{j \in S} p(j|s, a_s) v(j) = r(s, a_s). \tag{5.124}$$

   (b) **Improve:** For all $s \in S$, choose

   $$a'_s \in \arg\max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v'(j) \right\}, \tag{5.125}$$

   setting $a'_s = a_s$ if possible.

   (c) $\Gamma \leftarrow \{s \in S \,|\, a'_s \neq a_s\}$.

   (d) **Update:** For all $s \in S$, $a_s \leftarrow a'_s$.

3. **Terminate:** Return $d^*(s) = a_s$ and $v^*_\lambda(s) = v'(s)$ for all $s \in S$.

An alternative (asynchronous) form of the algorithm jumps immediately to the evaluation step as soon as there is a strict improvement in some state in step 2(b).

That is for some s and $\delta > 0$

$$\max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\} = r(s,a_s) + \lambda \sum_{j \in S} p(j|s,a_s) v(j) + \delta. \tag{5.126}$$

The proof of the theorem below also establishes that this variant of policy iteration terminates in a finite number of iterations.

Examples

The following example applies Algorithm 5.7 to the two-state model.

Example 5.13. Let $\lambda = 0.9$. The algorithm proceeds as follows:

1. Choose $a_{s_1} = a_{1,2}$ and $a_{s_2} = a_{2,1}$.
2. Evaluate[a] the stationary policy that use these actions to obtain $v(s_1) = -40$ and $v(s_2) = -50$.
3. In the improvement step

$$\begin{aligned} a'_{s_1} &= \arg\max_{i=1,2} \left\{ r(s_1, a_{1,i}) + \lambda \sum_{j=1,2} p(s_j|s_1, a_{1,i}) v(s_j) \right\} \\ &= \arg\max\{3 + 0.9\big(0.8v(s_1) + 0.2v(s_2)\big), 5 + 0.9v(s_2)\} \\ &= \arg\max\{-34.8, -40\} = a_{1,1} \end{aligned}$$

   and

$$\begin{aligned} a'_{s_2} &= \arg\max_{i=1,2} \left\{ r(s_2, a_{2,i}) + \lambda \sum_{j=1,2} p(s_j|s_2, a_{2,i}) v(s_j) \right\} \\ &= \arg\max\{-5 + 0.9v(s_2), 2 + 0.9\big(0.4v'(s_1) + 0.6v(s_2)\big)\} \\ &= \arg\max\{-50, -39.4\} = a_{2,2} \end{aligned}$$

4. Since $a'_s \neq a_s$ for all $s \in S$, replace a_s by a'_s for all $s \in S$ and return to the evaluation step.
5. Appealing again to Example 5.39, shows that $v(s_1) = 27.188$ and $v(s_2) = 25.625$.
6. At the next improvement step,

$$\begin{aligned} a'_{s_1} &= \arg\max\{27.188, 28.063\} = a_{1,2} \\ a'_{s_2} &= \arg\max\{18.063, 25.625\} = a_{2,2}. \end{aligned}$$

7. Since $a'_s \neq a_s$ for all $s \in S$, replace a_s by a'_s for all $s \in S$ and return to the evaluation step.

8. From Example 5.39, $v(s_1) = 30.147$ and $v(s_2) = 27.941$.

9. Applying the improvement step gives

$$a'_{s_1} = \arg\max\{29.735, 30.147\} = a_{1,2}$$
$$a'_{s_2} = \arg\max\{20.146, 27.941\} = a_{2,2}.$$

10. Since $a'_s = a_s$ for all $s \in S$, stop.

The above calculations showed that it required two improvement steps to find the optimal policy and an additional improvement step to confirm that it was optimal. Using action elimination may have avoided this additional step. Observe that the algorithm terminates with:

1. the optimal policy,
2. its value function, and
3. the solution of the Bellman equation.

Observe also that the sequence of value functions is non-decreasing.

[a]Note that Example 5.4 evaluates all four stationary deterministic policies.

As illustrated in the example above, it usually requires only a few iterations to find an optimal policy. However, the following example[26] shows that policy iteration may require $|S| + 1$ iterations to find an optimal policy.

Example 5.14. Let $S = \{1, 2, \ldots, M, \Delta\}$, $A_s = \{a_0, a_1\}$, $s \leq M$ and $A_\Delta = \{a_0\}$ where a_0 represents "stay" and a_1 represents "go right". The reward function is $r(s, a_0) = 0,\ s \in S$, $r(s, a_1) = -1,\ s < M$, and $r(M, a_1) = M^2$. The transition probabilities are $p(s|s, a_0) = 1$ for $s \in S$ and $p(s + 1|s, a_1) = 1$ for $s < M$. See Figure 5.9.

Implement policy iteration as follows. Initialize by choosing $a_s = a_0$ for all $s \in S$. Evaluation shows that $v(s) = 0$ for all $s \in S$. In the improvement step, $a'_s = a_0$ for $s \neq M$ and $a'_M = a_1$. Since $a'_s \neq a_s$ for all $s \in S$, replace a_s by a'_s for

[26]John Tsitsiklis provided this example in a personal communication. A different example reaching the same conclusion appears in Bertsekas [2012].

all $s \in S$ and return to the start of step 2 in the algorithm.

Evaluating this policy shows that $v(s) = 0$ for $s < M$ and $v(M) = M^2$. At the next improvement step $a'_s = a_1$ for $s \in \{M-1, M\}$ and $a'_s = a_0$ otherwise. Evaluating this policy shows that

$$v(s) = \begin{cases} 0 & s \notin \{M-1, M\} \\ -1 + \lambda M^2 & s = M-1 \\ M^2 & s = M. \end{cases}$$

Continuing in this way shows that it requires $M+1$ iterations to identify an optimal policy. The reason for this slow convergence is that it takes M iterations for the reward of M^2 to "make its way" back to state 1. Note that this example is representative of games like chess where the reward corresponding to winning, losing or ending in a draw may require many transitions to be realized.

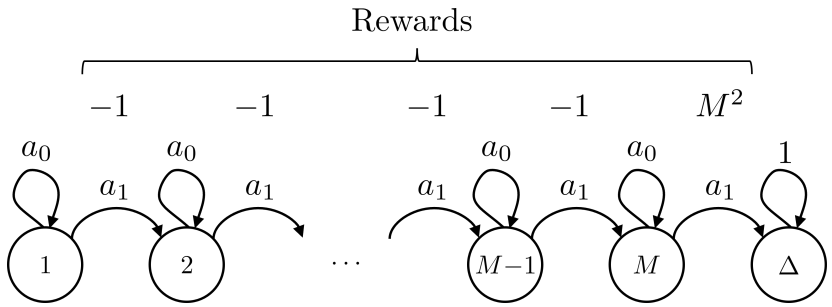


Figure 5.9: Schematic representation of model in Example 5.14. Actions a_0 and a_1 indicate transitions with probability 1 given the chosen action. Rewards are associated with transitions to the "right".

5.7.2 Convergence of policy iteration

This section formally establishes the result that policy iteration finds an optimal policy in a finite number of iterations, assuming a finite state and action model. It begins by showing that the sequence of values generated by policy iteration is non-decreasing and moreover a *strict* improvement leads to a strict increase in value. Vector notation simplifies exposition.

Proposition 5.9. Suppose $\mathbf{v}$ is the solution of $(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d$ and for some $s' \in S$ and $\delta > 0$

$$\mathbf{r}_{d'} + \lambda \mathbf{P}_{d'}\mathbf{v} = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v} + \delta \mathbf{e}_{s'}, \tag{5.127}$$

where $\mathbf{e}_{s'}$ denotes a vector with a one in the s'-th position and zeroes in all other components. Then

$$\mathbf{v}_\lambda^{(d')^\infty} \geq \mathbf{v} + \delta \mathbf{e}_{s'}.$$

Proof. Rewriting the hypothesis of the proposition as

$$\mathbf{r}_{d'} + \lambda \mathbf{P}_{d'}\mathbf{v} = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v} + \delta \mathbf{e}_{s'} = \mathbf{v} + \delta \mathbf{e}_{s'}, \tag{5.128}$$

it follows that

$$\mathbf{r}_{d'} = (\mathbf{I} - \lambda \mathbf{P}_{d'})\mathbf{v} + \delta \mathbf{e}_{s'}.$$

Multiplying both sides of the above equality by $(\mathbf{I} - \lambda \mathbf{P}_{d'})^{-1}$ and noting that

$$(\mathbf{I} - \lambda \mathbf{P}_{d'})^{-1}\mathbf{e}_{s'} = \sum_{n=0}^{\infty} (\lambda \mathbf{P}_{d'})^n \mathbf{e}_{s'} \geq \mathbf{I}\mathbf{e}_{s'} = \mathbf{e}_{s'}$$

it follows from Theorem 5.2 that

$$\mathbf{v}_\lambda^{(d')^\infty} = (\mathbf{I} - \lambda \mathbf{P}_{d'})^{-1}\mathbf{r}_{d'} \geq \mathbf{v} + \delta \mathbf{e}_{s'}.$$

□

The above proposition guarantees that if inequality (5.127) is strict in state s', then the value of the new policy d' is strictly greater than $\mathbf{v}$ in that state. Clearly, this generalizes to improvements in multiple states.

The following important result follows immediatley from Proposition 5.9.

Theorem 5.19. In a finite state and action model, the policy iteration algorithm converges monotonically and in a finite number of iterations to an optimal policy and value function.

Proof. As a consequence of Proposition 5.9 at each iteration, either $d' = d$ or there is a strict improvement in at least one state. Since there are only finitely many stationary deterministic policies, the algorithm must terminate at some iteration with $d' = d$. In this case

$$\mathbf{v} = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v} = \mathbf{r}_{d'} + \lambda \mathbf{P}_{d'}\mathbf{v}' = \underset{\delta \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_\delta + \lambda \mathbf{P}_\delta \mathbf{v}\} = L\mathbf{v}. \tag{5.129}$$

This means that $\mathbf{v}$ is the unique solution of the Bellman equation $\mathbf{v} = L\mathbf{v}$ and d^∞ is an optimal policy since its value satisfies the Bellman equation. □

Some comments follow:

1. It follows from the above proof that an asynchronous version of policy iteration, which carries out an evaluation step as soon as a strict improvement in one state is identified, converges.

2. Theorem 5.19 provides a constructive proof of the existence of an optimal policy and a solution of the optimality equation without appealing to the Banach fixed point theorem.

3. The proof above strongly depends on the finiteness of the set of stationary deterministic policies. The next section provides a general approach for analyzing policy iteration based on relating it to Newton's method.

5.7.3 Policy iteration and Newton's method*

It is now widely recognized that policy iteration may be viewed as Newton's method for finding the zero of a differentiable function. This equivalence enables proving convergence of policy iteration when the sets of states and/or actions are non-finite, and also provides some insight into its fast convergence.

In one dimension, Newton's method (Newton-Raphson iteration) seeks to find a solution x^* of $f(x) = 0$ through iterations of the form

$$x' = x - \left(\frac{df(x)}{dx}\right)^{-1} f(x).$$

When $f(x)$ is a differentiable, convex function and its derivative is Lipschitz continuous[27], Newton's method converges quadratically[28].

Newton's method can be generalized to solve $F(\mathbf{x}) = \mathbf{0}$ where $F : \mathbb{R}^n \to \mathbb{R}^n$. In this case the Newton recursion becomes

$$\mathbf{x}' = \mathbf{x} - (J(\mathbf{x}))^{-1} F(\mathbf{x}) \tag{5.130}$$

where $J(\mathbf{x})$ denotes the *Jacobian matrix*[29] of $F(\cdot)$ evaluated at $\mathbf{x}$.

To illustrate how Newton's method arises in a Markov decision process, rewrite the problem of finding a fixed point of L as that of finding a *zero* of B where

[27] A function $f(x)$ is said to be *Lipschitz continuous* if $|f(x) - f(y)| \leq K|x - y|$ for some $K > 0$ and all x and y.

[28] As noted earlier, a real-valued sequence $x_0, x_1, x_2, \ldots$ is said to *converge quadratically* to x^* if $|x_{n+1} - x^*| \leq K|x_n - x^*|^2$ for some constant $K > 0$. This means that eventually the number of decimal places of accuracy doubles when x_{n+1} replaces x_n as an approximation of x^*.

[29] The *Jacobian matrix* of a function $F(\cdot) : \mathbb{R}^n \to \mathbb{R}^n$ is an $n \times n$ matrix with its (i, j) component equal to the partial derivative of the ith component of $F(\cdot)$ with respect to the jth component of $\mathbf{x}$. The argument of $J(\cdot)$ denotes at which value of $\mathbf{x}$ it is evaluated.

$$B\mathbf{v} = L\mathbf{v} - \mathbf{v} = \underset{d \in D^{\text{MD}}}{\text{c-max}}\{\mathbf{r}_d + (\lambda \mathbf{P}_d - \mathbf{I})\mathbf{v}\} \tag{5.131}$$

for $\mathbf{v} \in V$.

From Lemma 5.5, for $\mathbf{u} \in V$ and $\mathbf{v} \in V$,

$$B\mathbf{u} \geq B\mathbf{v} + (\lambda \mathbf{P}_{d_\mathbf{v}} - \mathbf{I})(\mathbf{u} - \mathbf{v}) \tag{5.132}$$

where $d_\mathbf{v}$ is a $\mathbf{v}$-greedy decision rule; that is $d_\mathbf{v} \in \arg\,\text{c-max}_{d \in D^{\text{MD}}}\{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}$. This inequality generalizes the following property of differentiable, convex functions mapping $\mathbb{R}^n$ into $\mathbb{R}^n$, namely

$$F(\mathbf{y}) \geq F(\mathbf{x}) + J(\mathbf{x})(\mathbf{y} - \mathbf{x}) \tag{5.133}$$

for $\mathbf{x}$ and $\mathbf{y}$ in $\mathbb{R}^n$.

The following proposition provides a **closed form** representation for the iterates of the policy iteration algorithm.

Proposition 5.10. (**Newton method representation for policy iteration**) Let $\mathbf{v}$ and $\mathbf{v}'$ be successive iterates of the policy iteration algorithm. Then

$$\mathbf{v}' = \mathbf{v} - (\lambda \mathbf{P}_{d_\mathbf{v}} - \mathbf{I})^{-1} B\mathbf{v}, \tag{5.134}$$

where $d_\mathbf{v}$ is any $\mathbf{v}$-greedy decision rule.

Proof. Let $d_\mathbf{v}$ be selected in (5.123) in the improvement step when entering with $\mathbf{v}$ and let $\mathbf{v}'$ be obtained at the next evaluation step by solving (5.122). Then

$$\begin{aligned}
\mathbf{v}' &= (\mathbf{I} - \lambda \mathbf{P}_{d_\mathbf{v}})^{-1} \mathbf{r}_{d_\mathbf{v}} + \mathbf{v} - \mathbf{v} \\
&= (\mathbf{I} - \lambda \mathbf{P}_{d_\mathbf{v}})^{-1} (\mathbf{r}_{d_\mathbf{v}} + (\lambda \mathbf{P}_{d_\mathbf{v}} - \mathbf{I})\mathbf{v}) + \mathbf{v} \\
&= \mathbf{v} + (\mathbf{I} - \lambda \mathbf{P}_{d_\mathbf{v}})^{-1} B\mathbf{v} \\
&= \mathbf{v} - (\lambda \mathbf{P}_{d_\mathbf{v}} - \mathbf{I})^{-1} B\mathbf{v}.
\end{aligned}$$

□

Referring to (5.130), it suggests that the matrix $\lambda \mathbf{P}_{d_\mathbf{v}} - \mathbf{I}$ may be viewed as the Jacobian of B evaluated at $\mathbf{v}$. This is not quite precise because B is piecewise linear, so it does not have a unique "derivative" at its break points (see Figure 5.10). However using $\lambda \mathbf{P}_{d_\mathbf{v}} - \mathbf{I}$ corresponding to any $\mathbf{v}$-greedy decision rule can assume this role.

The following lemma provides the basis for an easy proof that policy iteration converges in general. It shows that iterates of policy iteration are non-decreasing, bounded below by the iterates of value iteration and bounded above by the optimal value function. Hence if value iteration converges monotonically to $\mathbf{v}^*_\lambda$, which is the case if $L\mathbf{v}^0 \geq \mathbf{v}^0$, the iterates of policy iteration are "squeezed" between the iterates of value iteration and the optimal value function. This last argument is formalized

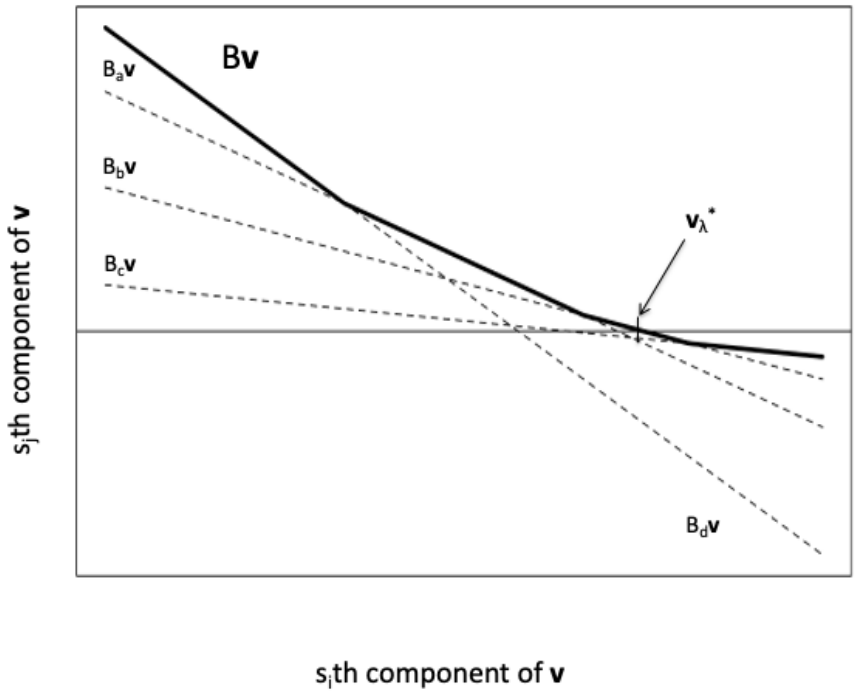


Figure 5.10: Bold line represents $B\mathbf{v}(s_j)$ as a function of $\mathbf{v}(s_i)$ for two states s_i and s_j. Observe that it is *piecewise linear and convex decreasing* with respect to $\mathbf{v}(s_i)$. Each dashed line corresponds to a component of $B_{d'}\mathbf{v}(s_j) = \mathbf{r}_{d'}(s_j) + (\lambda \mathbf{P}_{d'} - \mathbf{I})\mathbf{v}(s_j)$ as the s_i-th component of $\mathbf{v}$ varies for each of four decision rules $\{a, b, c, d\}$. The horizontal line represents $B\mathbf{v}(s_j) = 0$. The value indicated by $\mathbf{v}^*_\lambda$ corresponds to the s_ith component of the optimal value function when $B\mathbf{v}(s_j) = 0$.

in Appendix 5.13.3, which establishes the convergence of policy iteration in non-finite settings.

Lemma 5.6. For $\mathbf{v} \in V$ define $Z\mathbf{v} := \mathbf{v} - (\lambda \mathbf{P}_{d_\mathbf{v}} - \mathbf{I})^{-1} B\mathbf{v}$. Suppose for some $\mathbf{v} \in V$ that $B\mathbf{v} \geq \mathbf{0}$, then

1. $\mathbf{v}^*_\lambda \geq Z\mathbf{v}$,
2. $Z\mathbf{v} \geq L\mathbf{v}$,
3. $Z\mathbf{v} \geq \mathbf{v}$, and
4. $B(Z\mathbf{v}) \geq \mathbf{0}$.

Proof. The first result follows by reversing the steps in the proof of Proposition 5.10 to show that $Z\mathbf{v}$ is the value of $d^{\infty}_{\mathbf{v}}$.

The second result follows from rewriting (5.134) as

$$Z\mathbf{v} = \mathbf{v} + (\mathbf{I} - \lambda \mathbf{P}_{d_{\mathbf{v}}})^{-1} B\mathbf{v} \geq \mathbf{v} + B\mathbf{v} = L\mathbf{v}$$

where the inequality follows from the result in Lemma 5.3 that for $\mathbf{u} \geq \mathbf{0}$, $(\mathbf{I} - \lambda \mathbf{P}_{d_{\mathbf{v}}})^{-1}\mathbf{u} \geq \mathbf{u}$ and the assumption that $B\mathbf{v} \geq \mathbf{0}$.

The third result follows the same result in Lemma 5.3 to conclude that $(\mathbf{I} - \lambda \mathbf{P}_{d_{\mathbf{v}}})^{-1} B\mathbf{v} \geq \mathbf{0}$.

To prove the fourth result, apply equation (5.132) and note that

$$Z\mathbf{v} - \mathbf{v} = (\mathbf{I} - \lambda \mathbf{P}_{d_{\mathbf{v}}})^{-1} B\mathbf{v}$$

to obtain

$$B(Z\mathbf{v}) \geq B\mathbf{v} + (\lambda \mathbf{P}_{d_{\mathbf{v}}} - \mathbf{I})(Z\mathbf{v} - \mathbf{v}) = B\mathbf{v} - B\mathbf{v} = \mathbf{0}.$$

□

A proof of convergence of policy iteration in non-finite settings appears in Appendix C of this chapter.

5.8 Modified policy iteration

Modified policy iteration (MPI) algorithms provide a bridge between value iteration in which a maximization over the action set is performed in each state at each iteration and policy iteration in which a maximization takes place only after evaluating a policy by solving $\mathbf{r}_d + (\lambda \mathbf{P}_d - \mathbf{I})\mathbf{v} = \mathbf{0}$. Modified policy iteration can be viewed as either:

1. a value iteration algorithm in which the maximization is only done intermittently, or

2. a policy iteration algorithm where a policy is only evaluated approximately.

MPI is sometimes referred to as *value-oriented successive approximation* or *optimistic policy iteration*. A more descriptive alternative might be *truncated policy iteration* but this book will stick with modified policy iteration since it is widely used.

The greatest benefit of modified policy iteration with respect to value iteration occurs when there are many actions in each state, that is, when computing

$$\max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\}$$

is costly such as in the case of advance appointment scheduling and multi-product inventory control.

On the other hand, its main advantage with respect to policy iteration is that it avoids solving $(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d$, when the number of states is large.

5.8.1 Motivation

This exposition takes a policy iteration perspective. From Proposition 5.10, successive iterates of policy iteration may be represented by

$$\begin{aligned}\mathbf{v}' &= \mathbf{v} + (\mathbf{I} - \lambda \mathbf{P}_{d_\mathbf{v}})^{-1} B\mathbf{v} \\ &= \mathbf{v} + \sum_{n=0}^{\infty} (\lambda \mathbf{P}_{d_\mathbf{v}})^n B\mathbf{v}.\end{aligned}$$

where the second representation follows from Lemma 5.2. MPI truncates the series expansion $\sum_{n=0}^{\infty}(\lambda \mathbf{P}_{d_\mathbf{v}})^n$ at a value of m that may vary from iteration to iteration. That is, a step of MPI can be expressed as

Modified policy iteration: truncation representation

$$\mathbf{v}' = \mathbf{v} + \sum_{n=0}^{m} (\lambda \mathbf{P}_{d_\mathbf{v}})^n B\mathbf{v}. \tag{5.135}$$

Note that when $m = 0$, (5.135) becomes $\mathbf{v}' = \mathbf{v} + L\mathbf{v} - \mathbf{v} = L\mathbf{v}$, which is the value iteration recursion. And as noted above, when $m = \infty$, this is the recursive form for policy iteration. The index m may be referred to as the *truncation order* of the algorithm.

Equation (5.135) is not efficient for calculation. Instead MPI is implemented by specifying $\mathbf{v}$, performing one value iteration step $\mathbf{v}' = L\mathbf{v}$ to identify a $\mathbf{v}$-greedy decision rule, $d_\mathbf{v}$ according to

$$d_\mathbf{v} \in \underset{d \in D^{\mathrm{MD}}}{\arg\,\text{c-max}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\},$$

setting $\mathbf{u} = \mathbf{v}'$ and then applying value iteration m times using the *fixed* decision rule $d_\mathbf{v}$ as follows

$$\mathbf{u} \leftarrow L_{d_\mathbf{v}} \mathbf{u} = \mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{u}} \mathbf{u}.$$

Thus, since $L\mathbf{v} = L_{d_\mathbf{v}}\mathbf{v}$, an MPI update can be expressed as

Modified policy iteration: implementation representation

$$\mathbf{v}' = L_{d_\mathbf{v}}^m L\mathbf{v} = L_{d_\mathbf{v}}^{m+1}\mathbf{v}. \tag{5.136}$$

Note that since $d_\mathbf{v}$ is not known when implementing the above, $L\mathbf{v}$ must be evaluated to determine $d_\mathbf{v}$ and also update $\mathbf{v}$.

The next result shows that these two expressions for MPI are equivalent.

Proposition 5.11. For each $\mathbf{v} \in V$,

$$\mathbf{v} + \sum_{n=0}^{m} (\lambda \mathbf{P}_{d_\mathbf{v}})^n B\mathbf{v} = L_{d_\mathbf{v}}^m L\mathbf{v}. \tag{5.137}$$

Proof. To prove this result, expand and re-group terms as follows:

$$\begin{aligned}
\mathbf{v} + \sum_{n=0}^{m} (\lambda \mathbf{P}_{d_\mathbf{v}})^n B\mathbf{v} &= \mathbf{v} + \left(\mathbf{I} + \lambda \mathbf{P}_{d_\mathbf{v}} + (\lambda \mathbf{P}_{d_\mathbf{v}})^2 + \ldots + (\lambda \mathbf{P}_{d_\mathbf{v}})^m\right)\left(\mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{v}}\mathbf{v} - \mathbf{v}\right) \\
&= \mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{v}}\mathbf{v} + \lambda \mathbf{P}_{d_\mathbf{v}}(\mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{v}}\mathbf{v} - \mathbf{v}) + \ldots + (\lambda \mathbf{P}_{d_\mathbf{v}})^m(\mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{v}}\mathbf{v} - \mathbf{v}) \\
&= \mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{v}}\mathbf{r}_{d_\mathbf{v}} + \ldots + (\lambda \mathbf{P}_{d_\mathbf{v}})^m \mathbf{r}_{d_\mathbf{v}} + (\lambda \mathbf{P}_{d_\mathbf{v}})^{m+1}\mathbf{v} \\
&= \mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{v}}(\mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{v}}(\mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{v}}(\ldots(\mathbf{r}_{d_\mathbf{v}} + \lambda \mathbf{P}_{d_\mathbf{v}}\mathbf{v})))) \\
&= L_{d_\mathbf{v}}^{m+1}\mathbf{v} = L_{d_\mathbf{v}}^m L\mathbf{v}.
\end{aligned}$$

□

5.8.2 A modified policy iteration algorithm

This section describes an MPI algorithm using vector notation and then an implementable form using component notation. Unlike policy iteration, MPI is not a finite algorithm, therefore it requires a stopping criterion. The algorithm below uses a span-based stopping criterion that terminates the algorithm with an ϵ-optimal policy and an approximate value function that is within ϵ of the optimal value function. Figure 5.11 provides a schematic representation where m represents the order of MPI.

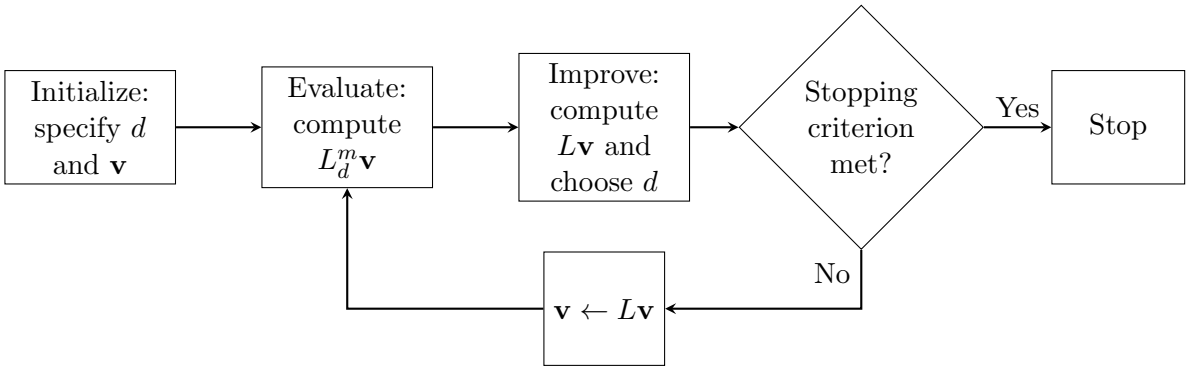


Figure 5.11: Flow diagram for modified policy iteration

The algorithm can start with either a value or a decision rule. This choice impacts whether improvement precedes evaluation or vice versa and when the stopping criterion is evaluated. The algorithm below initializes both the decision rule and value, begins with an evaluation step and checks the stopping criterion after improvement. Starting with a value only requires some modifications.

Algorithm 5.8. Modified policy iteration: vector form

1. **Initialize:**
   (a) Specify a decision rule $d' \in D^{\text{MD}}$.
   (b) Specify $\epsilon > 0$ and $\sigma > (1-\lambda)\epsilon/\lambda$.
   (c) Specify $\mathbf{v} \in V$.
   (d) Specify a sequence of non-negative integers $m_n, n = 1, 2, \ldots$.
   (e) $n \leftarrow 1$.
2. **Iterate:** While $\sigma \geq (1-\lambda)\epsilon/\lambda$:
   (a) **Update:** $d \leftarrow d'$.
   (b) **Evaluate:**
   $$\mathbf{u} \leftarrow L_d^{m_n}\mathbf{v}. \tag{5.138}$$
   (c) **Improve:** $\mathbf{v} \leftarrow L\mathbf{u}$ and choose
   $$d' \in \underset{\delta \in D^{\text{MD}}}{\arg\,\text{c-max}} \left\{\mathbf{r}_\delta + \lambda \mathbf{P}_\delta \mathbf{u}\right\} \tag{5.139}$$
   (d) $\sigma \leftarrow \text{sp}(\mathbf{v} - \mathbf{u})$.
   (e) $n \leftarrow n + 1$.
3. **Terminate:** Return $d_\epsilon = d'$ and
   $$\mathbf{v}_\lambda^\epsilon = \mathbf{v} + \frac{\lambda}{1-\lambda}(\underline{\mathbf{v} - \mathbf{u}})\mathbf{e}.$$

Some comments on the above algorithm follow:

1. The algorithm reduces to value iteration when $m_n = 0$ for $n = 1, 2, \ldots$ and policy iteration when $m_n = \infty$ for $n = 1, 2, \ldots$.
2. Specifying both a starting value and policy in the initialization avoids calculating $L\mathbf{v}$ prior to evaluation. Alternatively, if the algorithm starts without a decision rule, iteration begins with an improvement step.
3. A convenient choice for $\mathbf{v}$ is $v(s) = (1-\lambda)^{-1} \min_{a \in A_s} r(s,a)$, which is also a lower bound on $\mathbf{v}_\lambda^*$. Choosing such an initial value guarantees monotone convergence (see below).
4. The value at iteration n corresponds to the non-stationary deterministic Markovian policy that uses d_1 for m_1 periods, d_2 for $m_2 + 1$ periods, ..., d_n for $m_n + 1$

periods and then receives a terminal reward $\mathbf{v}$ where d_n denotes the decision rule evaluated at the n-th pass through the algorithm. Note if the algorithm was initiated by setting $\mathbf{u} = \mathbf{v}$ and beginning in the improve step, d_1 would be used for $m_1 + 1$ periods.

5. This general version of the algorithm keeps track of the iteration number because the pre-specified sequence m_n of orders may vary with n.

6. The impact of m_n on the speed of convergence will be explored empirically in Section 5.11.5.

7. Instead of specifying the order in advance, one might instead specify a stopping rule for the evaluation step. In other words, stop when $\text{sp}(L_d^{k+1}\mathbf{v} - L_d^k\mathbf{v})$ achieves some pre-specified tolerance. The tolerance may vary with n; presumably less accuracy is needed initially and more accuracy is needed later.

8. Note that at termination $\mathbf{v} = L\mathbf{u}$ to ensure that Theorem 5.13 applies. This specification is consistent with value iteration.

9. Similarly to value iteration, action elimination may be incorporated in the algorithm prior to applying the stopping criteria.

10. The evaluation step can be implemented using Gauss-Seidel iteration.

11. As a result of Theorem 5.13, the algorithm terminates with a decision rule corresponding to the ϵ-optimal stationary policy d_ϵ^∞ and a value function approximation that is within ϵ of the optimal value function in all components.

An implementable version

A statement of MPI in component notation follows. It is initialized by specifying an action for each state and a value to initiate the evaluation step. For transparency, the statement denotes the iterates of the evaluation step by $u(s)$. The reason for including the iteration index n is to specify the appropriate truncation order m_n at each iteration. The algorithm terminates with an ϵ-optimal policy d_ϵ^∞ and an approximation in which all components are within ϵ of the optimal value function.

Algorithm 5.9. Modified policy iteration: component form

1. **Initialize:**
   (a) For all $s \in S$, specify $a'_s \in A_s$.
   (b) Specify $\epsilon > 0$ and $\sigma > (1 - \lambda)\epsilon/\lambda$.
   (c) Specify $v(s)$ for all $s \in S$.

(d) Specify a sequence of non-negative integers $m_n, n = 1, 2, \ldots$.

(e) $n \leftarrow 1$.

2. **Iterate:** While $\sigma \geq (1-\lambda)\epsilon/\lambda$:

(a) **Update:** $a_s \leftarrow a'_s$ for all $s \in S$.

(b) **Evaluate:**

i. $m \leftarrow 1$ and $u(s) \leftarrow v(s)$ for all $s \in S$.

ii. While $m \leq m_n$:

A. For all $s \in S$,

$$u(s) \leftarrow r(s, a_s) + \lambda \sum_{j \in S} p(j|s, a_s) u(j).$$

B. $m \leftarrow m + 1$.

(c) **Improve:**

i. For all $s \in S$,

$$v(s) \leftarrow \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) u(j) \right\}$$

and choose

$$a'_s \in \arg\max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) u(j) \right\}.$$

ii. $\sigma \leftarrow \mathrm{sp}(\mathbf{v} - \mathbf{u})$.

iii. $n \leftarrow n + 1$.

(d) **Terminate:** For all $s \in S$, return $d_\epsilon(s) = a'_s$ and

$$v^\epsilon_\lambda(s) = v(s) + \frac{\lambda}{1-\lambda}(\underline{\mathbf{v} - \mathbf{u}}).$$

The following calculations illustrate this algorithm applied the two-state model.

Example 5.15. This examples finds an ϵ-optimal policy in the two-state model using modified policy iteration with $v(s) = 0$, $\lambda = 0.9$ and $\epsilon = 0.000001$. For simplicity, let $m_n = 3$ for all $n = 1, 2, \ldots$.

1. Choose $a'_{s_1} = a_{1,2}$ and $a'_{s_2} = a_{2,1}$ and
2. Three iterations of value iteration with the above fixed policy, yields $u(s_1) = -3.55$ and $u(s_2) = -13.55$.
3. To implement the improve step evaluate

$$\begin{aligned} v(s_1) &= \max_{i=1,2} \left\{ r(s_1, a_{1,i}) + \lambda \sum_{j=1,2} p(s_j|s_1, a_{1,i}) u(s_j) \right\} \\ &= \max\{3 + 0.9(0.8u(s_1) + 0.2u(s_2)), 5 + 0.9u(s_2)\} \\ &= \max\{-1.995, -7.195\} = -1.955, \end{aligned}$$

$$\begin{aligned} v(s_2) &= \max_{i=1,2} \left\{ r(s_2, a_{2,i}) + \lambda \sum_{j=1,2} p(s_j|s_2, a_{2,i}) u(s_j) \right\} \\ &= \max\{-5 + 0.9v(s_2), 2 + 0.9(0.4u(s_1) + 0.6u(s_2))\} \\ &= \max\{-17.195, -6.595\} = -6.595, \end{aligned}$$

and choose

$$\begin{aligned} a'_{s_1} &\in \arg\max_{i=1,2} \left\{ r(s_1, a_{1,i}) + \lambda \sum_{j=1,2} p(s_j|s_1, a_{1,i}) u(s_j) \right\} \\ &= \arg\max\{-1.995, -7.195\} = a_{1,1}, \end{aligned}$$

and

$$\begin{aligned} a'_{s_2} &\in \arg\max_{i=1,2} \left\{ r(s_2, a_{2,i} + \lambda \sum_{j=1,2} p(s_j|s_2, a_{2,i}) v(s_j) \right\} \\ &\in \arg\max\{-17.195, -6.595\} = a_{2,2}. \end{aligned}$$

4. Since $\text{sp}(\mathbf{v} - \mathbf{u}) = 5.4 \geq \lambda(1-\lambda)^{-1}\epsilon = 0.000009$, $\mathbf{v} \leftarrow \mathbf{v}'$ and return to the evaluation step.
5. Table 5.1 describes the results of four additional iterations of modified policy iteration.
6. Since $\text{sp}(\mathbf{v} - \mathbf{u}) = 0.0000032 < \lambda(1-\lambda)^{-1}\epsilon = 0.000009$, the algorithm terminates with $d_\epsilon(s_1) = a_{1,2}$ and $d_\epsilon(s_2) = a_{2,2}$.
7. The value function approximation is $v^\epsilon_\lambda(s_1) = 30.147$ and $v^\epsilon_\lambda(s_2) = 27.941$, which equal $\mathbf{v}^*_\lambda$.

Observe that it requires 5 maximizations and 15 iterations with a fixed policy to determine an ϵ-optimal policy. Note further that the ϵ-optimal policy is optimal and was first identified at iteration 2.

Iteration	$u(s_1)$	$u(s_2)$	$v(s_1)$	$v(s_2)$	a_{s_1}	a_{s_2}	$\text{sp}(\mathbf{v}' - \mathbf{v})$
2	5.2225	3.5184	8.1665	5.7800	$a_{1,2}$	$a_{2,2}$	0.6822
3	14.0232	11.8257	15.6432	13.4342	$a_{1,2}$	$a_{2,2}$	0.0115
4	19.5720	17.3663	20.6296	18.4237	$a_{1,2}$	$a_{2,2}$	0.00019
5	23.2089	21.0029	23.9027	21.6967	$a_{1,2}$	$a_{2,2}$	0.0000032

Table 5.1: Iterates of modified policy iteration in Example 5.15.

5.8.3 Convergence of modified policy iteration*

Figure 5.12 illustrates how the iterates of modified policy iteration evolve and motivates the convergence proof below. The following discussion suppresses the component index for clarity, that is, $\mathbf{u}^k$ substitutes for $\mathbf{u}^k(s_i)$ and $B\mathbf{u}^k$ for $B\mathbf{u}^k(s_j)$. Observe that:

1. Iteration starts at $\mathbf{u}^0$.

2. $\mathbf{u}^1 = \mathbf{u}^0 + B\mathbf{u}^0 = L\mathbf{u}^0$ and $\mathbf{u}^2 = \mathbf{u}^1 + B\mathbf{u}^1 = L\mathbf{u}^1$ may be regarded as the result of applying two iterates of value iteration or equivalently modified policy iteration of order 0. Since $L\mathbf{u} = B\mathbf{u} + \mathbf{u}$ the iterates of value iteration may be represented by adding the increment obtained when the "45°" line in the figure crosses the axis at 0 to the original value.

3. Iterates $\mathbf{u}^1$, $\mathbf{u}^2$ and $\mathbf{u}^3$ may be regarded as the results of applying one step of modified policy iteration or order 2. That is, the first iteration identifies the $\mathbf{u}^0$-greedy decision rule d and sets $\mathbf{u}^1 = L\mathbf{u}_0 = B\mathbf{u}^0 + \mathbf{u}^0$. The next two steps compute $\mathbf{u}^2 = L_d\mathbf{u}^1$ and $\mathbf{u}^2 = L_d\mathbf{u}^2$. We emphasize that the update is based on L_d and not L. Although they are equivalent at $\mathbf{u}^1$, using L_d instead of L avoids an extra maximization. Then the next full step of modified policy iteration of order 2 begins at $\mathbf{u}^3$ by identifying a $\mathbf{u}^3$-greedy decision rule and computing $\mathbf{u}^4 = L\mathbf{u}_3$ and then uses this decision rule to evaluate $\mathbf{u}^5$. This approach avoids two maximizations at each full MPI iteration.

The following shows that modified policy iteration converges using the same approach that was suggested for establishing the convergence of policy iteration in a non-finite setting in Appendix 5.13.3. To do so, define the operator $W_m : V \to V$ to represent the right hand side of (5.135), that is, for any integer $m \geq 0$,

$$W_m\mathbf{v} := \mathbf{v} + \sum_{n=0}^{m} (\lambda \mathbf{P}_{d_\mathbf{v}})^n B\mathbf{v}.$$

Hence the iterates of MPI can be represented by $\mathbf{v}^{n+1} = W_{m_n}\mathbf{v}^n$.

The proof of Lemma 5.6 can be easily modified to establish the analogous result for W_m. Only statement 4 requires a revised argument.

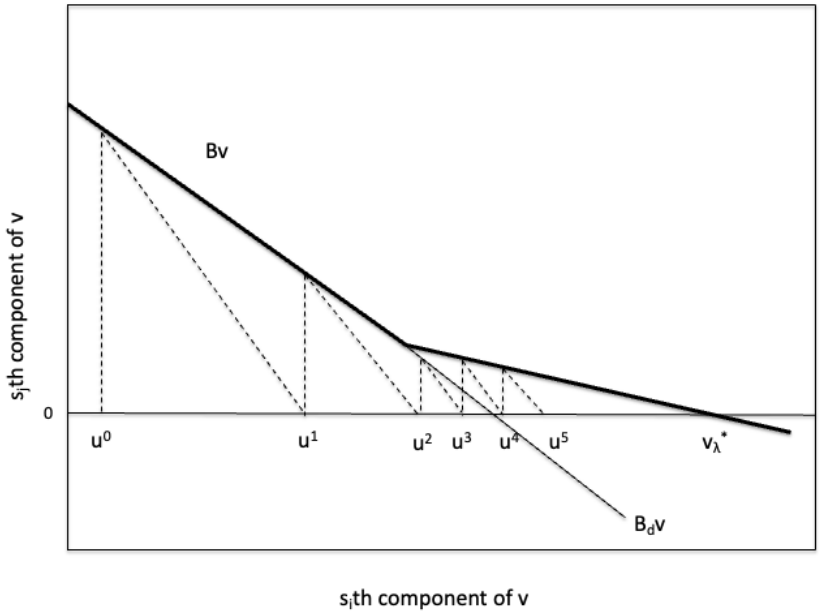


Figure 5.12: Symbolic representation of modified policy iteration updates. The bold line represents $B\mathbf{v}(s_j)$ as a function of $\mathbf{v}(s_i)$ for two states s_i and s_j and the lighter line represents $B_d\mathbf{v}(s_j)$ as a function of $\mathbf{v}(s_i)$ for the $\mathbf{u}^0$-greedy decision rule d. Note that $\mathbf{u}^2 = L_d\mathbf{u}^1 = L\mathbf{u}^2$ and $\mathbf{u}^3 = L_d\mathbf{u}^2 \neq L\mathbf{u}^2$.

Lemma 5.7. Suppose for some $\mathbf{v} \in V$ that $B\mathbf{v} \geq \mathbf{0}$ then for any integer $m \geq 0$,

1. $\mathbf{v}_\lambda^* \geq W_m\mathbf{v}$,
2. $W_m\mathbf{v} \geq L\mathbf{v}$,
3. $W_m\mathbf{v} \geq \mathbf{v}$, and
4. $B(W_m\mathbf{v}) \geq \mathbf{0}$.

The following theorem establishes the convergence of MPI using monotonicity arguments under the condition that $B\mathbf{v}^0 \geq \mathbf{0}$ or equivalently $L\mathbf{v}^0 \geq \mathbf{v}^0$. A more general result is referenced below.

Theorem 5.20. Let $m_n, n = 0, 1, \ldots$ denote a sequence of non-negative integers and let $\mathbf{v}^n, n = 0, 1, \ldots$ denote a sequence of iterates of modified policy iteration. Then if $B\mathbf{v}^0 \geq \mathbf{0}$, $\mathbf{v}^n$ converges monotonically and in norm to $\mathbf{v}^*_\lambda$.

Proof. Suppose $\mathbf{u}^0 = \mathbf{v}^0$. For each $n \geq 0$ define $\mathbf{u}^{n+1} = L\mathbf{u}^n$ and $\mathbf{v}^{n+1} = W_{m_n}\mathbf{v}^n$. Note that the sequence $\mathbf{u}^n$ corresponds to iterates of value iteration and the sequence $\mathbf{v}^n$ corresponds to iterates of MPI.

Now apply induction to establish that for all n, $B\mathbf{v}^n \geq \mathbf{0}$ and

$$\mathbf{u}^n \leq \mathbf{v}^n \leq \mathbf{v}^*_\lambda. \tag{5.140}$$

Since $B\mathbf{v} \geq \mathbf{0}$ is equivalent to $L\mathbf{v} \geq \mathbf{v}$, $L^2\mathbf{v}^0 \geq L\mathbf{v}^0 \geq \mathbf{v}^0$ so that for any n, $L^n\mathbf{v}^0 \geq \mathbf{v}^0$. That $\mathbf{v}^0 \leq \mathbf{v}^*_\lambda$ follows by letting $n \to \infty$ and noting that $L^n\mathbf{v}^0 \to \mathbf{v}^*_\lambda$. Consequently $\mathbf{v}^0 \leq \mathbf{v}^*_\lambda$.

For the induction step, assume now that (5.140) and $B\mathbf{v}^n \geq \mathbf{0}$ hold for all $m \leq n$. By the second result in Lemma 5.7 and since $L\mathbf{u}^n \leq L\mathbf{v}^n$, $\mathbf{v}^{n+1} = W_{m_n}\mathbf{v}^n \geq L\mathbf{v}^n \geq L\mathbf{u}^n = \mathbf{u}^{n+1}$. That $\mathbf{v}^{n+1} \leq \mathbf{v}^*_\lambda$ follows from the first result of Lemma 5.7 and that $B\mathbf{v}^{n+1} \geq \mathbf{0}$ follows from the fourth result of Lemma 5.7. That $\mathbf{v}^{n+1} \geq \mathbf{v}^n$ is an application of the third result of Lemma 5.7. Hence the induction hypothesis is satisfied.

Thus from (5.140) the convergence of $\mathbf{u}^n$ to $\mathbf{v}^*_\lambda$ implies the convergence of $\mathbf{v}^n$ to the same value, completing the proof. □

The proof above establishes that the iterates of value iteration (modified policy iteration of order 0) are dominated by those of modified policy iteration of order m, if $B\mathbf{v}^0 \geq \mathbf{0}$. Consequently one might conjecture that the iterates of MPI of order $m + k$ ($k > 0$) dominate those of MPI of order m. It turns out that this conjecture is false[30].

The following more general convergence result was established by Rothblum in an obscure reference in 1979 and republished as Canbolat and Rothblum [2013].

Theorem 5.21. For any $\mathbf{v}^0$, modified policy iteration converges to $\mathbf{v}^*_\lambda$.

5.9 Linear programming

Linear programming provides an elegant approach for formulating, analyzing and solving infinite horizon discounted MDPs. Moreover, it provides an effective computational approach for solving approximate dynamic programs (Chapter 9). This section describes the relationship between linear programs (LPs) and Markov decision processes and shows that key MDP concepts have direct analogs in linear programming. The main result is that an optimal solution to a (dual) linear program corresponds to an

[30] See example 6.5.1 in Puterman [1994].

optimal policy to the Markov decision process. This long section requires background in linear algebra and linear programming, some of which appears in Appendix C.

5.9.1 The primal linear program

If $v(s)$ satisfies the Bellman equation

$$v(s) = \max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \right\}$$

for all $s \in S$, then it satisfies the system of inequalities

$$v(s) \geq \; r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v(j) \tag{5.141}$$

for all $a \in A_s$ and $s \in S$.

By Corollary 5.2, any $v(s)$ satisfying (5.141) for all $a \in A_s$ and $s \in S$ is an upper bound on $v^*_\lambda(s)$. Hence a solution of the Bellman equation may be thought of as the *smallest* (in a component-wise sense) $v(s)$ that satisfies (5.141) for all $a \in A_s$ and $s \in S$. To formulate the problem of finding such a $v(s)$ using linear programming requires a single objective function. To obtain one, selecting $\alpha(s) > 0$ for all $s \in S$ leads to the following linear program.

Primal LP formulation of a discounted MDP

$$\text{minimize} \quad \sum_{s \in S} \alpha(s) v(s) \tag{5.142a}$$

$$\text{subject to} \quad v(s) - \lambda \sum_{j \in S} p(j|s,a) v(j) \geq r(s,a), \quad a \in A_s, s \in S \tag{5.142b}$$

Some comments about this formulation follow:

1. The above model is often referred to as the *primal* linear program. Associated with every primal linear is a *dual* linear program, which is described below. Note that the relationship between policies and values becomes more apparent through analysis of the dual linear program.

2. In this formulation, $v(s)$ is *unrestricted* in sign for all $s \in S$. That is, it is not restricted to be non-negative as is often the case in LP formulations.

3. This LP has $|S|$ variables and $\sum_{s \in S} |A_s|$ constraints. Thus, the number of constraints might far exceed the number of variables.

4. Choosing $\alpha(s)$ so that $\sum_{s\in S}\alpha(s) = 1$ allows interpretation of this quantity as a probability distribution on S.

5. Note that there is exactly one positive coefficient in each row[31] corresponding to the coefficient of $v(s)$ with value $1 - \lambda p(s|s, a)$.

6. Since $v^*_\lambda(s)$ for all $s \in S$ satisfies the Bellman equation, it is a feasible[32] solution for the primal LP.

The following example illustrates this primal LP formulation.

Example 5.16. This example formulates the primal LP for the two-state example. The Bellman equations are given in equations (5.62) and (5.63). The primal linear program is given by

$$\begin{aligned} [1]\text{minimize} \quad & \alpha(s_1)v(s_1) + \alpha(s_2)v(s_2) \\ \text{subject to} \quad & (1 - 0.8\lambda)v(s_1) - 0.2\lambda v(s_2) \geq 3 \\ & 1v(s_1) - \lambda v(s_2) \geq 5 \\ & 0v(s_1) + (1 - \lambda)v(s_2) \geq -5 \\ & (1 - 0.4\lambda)v(s_1) - 0.6\lambda v(s_2) \geq 2. \end{aligned} \tag{5.143}$$

The four constraints correspond respectively to choosing actions $a_{1,1}$ or $a_{1,2}$ in s_1 and $a_{2,1}$ or $a_{2,2}$ in s_2. For $\lambda = 0.9$, the four constraints can be written as:

$$\begin{aligned} v(s_2) &\leq \frac{14}{9}v(s_1) - \frac{50}{3} \\ v(s_2) &\leq \frac{10}{9}v(s_1) - \frac{50}{9} \\ v(s_2) &\geq -50 \\ v(s_2) &\geq \frac{18}{23}v(s_1) + \frac{100}{23}. \end{aligned}$$

Written as above, it is easy to visualize the feasible region in two dimensions. The first and fourth constraints intersect at $(v(s_1), v(s_2)) = (27.1875, 25.625)$, while the second and fourth constraints intersect at $(v(s_1), v(s_2)) = (30.147, 27.941)$. Since the objective is to maximize the value function in each state, the optimal solution is the latter, which is the same value function found by value iteration in Example 5.11. The figure below illustrates the region described by the above constraints together with a graphical solution for $\lambda = 0.9$.

Note that the optimal policy can be obtained by looking at which constraints are tight (hold with equality) at the optimal solution. The second constraint cor-

[31] A linear system with this property is referred to as *Leontief substitution system*.

[32] A feasible solution is one that satisfies all the LP constraints.

responds to choosing $a_{1,2}$ in s_1 and the fourth constraint corresponds to choosing $a_{2,2}$ in s_2, again matching the policy identified from previous methods.

Finally, note that the optimal solution corresponds to the "lower left" vertex of the feasible region. So any values of $\alpha(s_1) > 0$ and $\alpha(s_2) > 0$ will result in the same optimal solution, as will be show formally in Corollary 5.5.

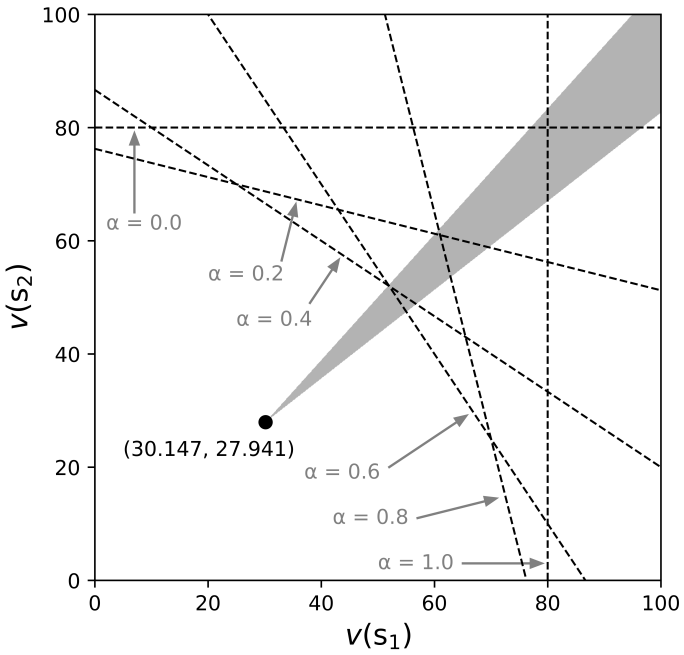


Figure 5.13: Figure for Example 5.16. Shaded region represents the feasible region, defined by the constraints. This figure assumes that $\alpha(s_1) = \alpha$ and $\alpha(s_2) = 1 - \alpha$ for some $\alpha > 0$. Different α values represent different objective functions. The identified vertex is the optimal solution for all α.

LP formulation: vector notation*

This section discusses the matrix-vector formulation of the primal linear program. Unfortunately, the Bellman equation expressed in terms of "c-max" as

$$\mathbf{v} = \underset{d \in D^{\text{MD}}}{\text{c-max}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}, \tag{5.144}$$

does not have a direct analog in terms of vector inequalities. If instead, one considered the Bellman equation expressed as

$$\mathbf{v} = \max_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}, \tag{5.145}$$

where the maximum was taken over all Markov deterministic decision rules, then this would have a direct analog as

$$\mathbf{v} \geq \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v} \tag{5.146}$$

for all $d \in D^{\text{MD}}$. Choosing this as a starting point and letting $\boldsymbol{\alpha} > \mathbf{0}$ denote a vector of length $|S|$ leads to the following linear program.

Primal LP formulation of a discounted MDP: vector notation

$$\begin{aligned} \text{minimize} \quad & \boldsymbol{\alpha}^{\mathsf{T}} \mathbf{v} && \text{(5.147a)} \\ \text{subject to} \quad & (\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} \geq \mathbf{r}_d, \quad d \in D^{\text{MD}}. && \text{(5.147b)} \end{aligned}$$

While elegant, this formulation is impractical for computation because it requires enumeration of all decision rules, which runs contrary to Markov decision process principles. Moreover it repeats constraints when two or more decision rules choose the same action in a particular state. For example, consider the decision rules $d_1 = \{a_{1,1}, a_{2,1}\}$ and $d_2 = \{a_{1,1}, a_{2,2}\}$ in the two-state model. Since both choose the same action in state s_1, the inequalities $\mathbf{v} \geq \mathbf{r}_{d_1} + \lambda \mathbf{P}_{d_1} \mathbf{v}$ and $\mathbf{v} \geq \mathbf{r}_{d_2} + \lambda \mathbf{P}_{d_2} \mathbf{v}$ would both contain the same constraint involving $a_{1,1}$ in s_1.

Existence of a solution of the primal linear program*

The following result shows that the primal LP has an optimal solution, and that solving the LP finds the optimal value function $\mathbf{v}^*_\lambda$ and an optimal policy for the MDP. Its proof is subtle and uses the structural result that the primal LP formulation of an MDP has exactly one positive coefficient in each row.

Theorem 5.22. Let $\boldsymbol{\alpha} > \mathbf{0}$ and assume $0 \leq \lambda < 1$.

1. The linear program (5.142) has an optimal solution $v^*(s)$ for all $s \in S$.
2. For each $s \in S$ there exists an $a^*_s \in A_s$ for which
$$v^*(s) - \lambda \sum_{j \in S} p(j|s, a^*_s) v^*(j) = r(s, a^*_s). \tag{5.148}$$
3. For each $s \in S$, let $d^*(s) = a^*_s$ as defined in (5.148). Then $(d^*)^\infty$ is an optimal policy.
4. The solution $v^*(s) = v^*_\lambda(s)$ for all $s \in S$ is the unique optimal solution of the LP.

Proof. Define $R = \max_{a \in A_s, s \in S} r(s,a)$ and let $v'(s) = (1-\lambda)^{-1}R$ for all $s \in S$. Then direct substitution into (5.142b) gives

$$v'(s) - \lambda \sum_{j \in S} p(j|s,a)v'(j) = R \geq r(s,a)$$

for all $a \in A_s, s \in S$. Hence, the linear program is feasible ($\mathbf{v}'$ is a feasible solution). Now, consider any feasible solution $\mathbf{v}$. For every $d \in D^{\text{MD}}$, it satisfies $(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} \geq \mathbf{r}_d$ and therefore $\mathbf{v} \geq (\mathbf{I} - \lambda \mathbf{P}_d)^{-1}\mathbf{r}_d$, by the second result of Lemma 5.3. Since $\boldsymbol{\alpha} > \mathbf{0}$, $\boldsymbol{\alpha}^{\mathsf{T}}\mathbf{v} \geq \boldsymbol{\alpha}^{\mathsf{T}}(\mathbf{I} - \lambda \mathbf{P}_d)^{-1}\mathbf{r}_d$ for all $d \in D^{\text{MD}}$. In other words, the objective function value of every feasible $\mathbf{v}$ is bounded below by $\max_{d \in D^{\text{MD}}} \boldsymbol{\alpha}^{\mathsf{T}}(\mathbf{I} - \lambda \mathbf{P}_d)^{-1}\mathbf{r}_d$. Hence by Theorem C.1 in Appendix C, the linear program has an optimal solution, denoted by $v^*(s)$ for all $s \in S$.

To prove the second result, assume to the contrary that there exists an $s' \in S$ such that for all $a \in A_{s'}$

$$v^*(s') - \lambda \sum_{j \in S} p(j|s',a)v^*(j) > r(s',a). \tag{5.149}$$

Hence there exists an $\epsilon > 0$ for which

$$v^*(s') - \lambda \sum_{j \in S} p(j|s',a)v^*(j) - \epsilon \geq r(s',a).$$

Since $\lambda p(s'|s',a)\epsilon > 0$, it follows that

$$\begin{aligned}(v^*(s') - \epsilon) - \lambda \left(\sum_{j \in S \setminus \{s'\}} p(j|s',a)v^*(j) + p(s'|s',a)(v^*(s') - \epsilon) \right) \\ \geq v^*(s') - \lambda \sum_{j \in S} p(j|s',a)v^*(j) - \epsilon\end{aligned}$$

Hence, the value function defined by

$$v'(s) := \begin{cases} v^*(s) & s \neq s' \\ v^*(s) - \epsilon & s = s' \end{cases}$$

is a feasible solution for the linear program. Since $\alpha(s) > 0$ for all $s \in S$, this implies the value function $v'(s)$ has a lower objective function value than that of $v^*(s)$ (by $\alpha(s')\epsilon$), which contradicts the optimality of $v^*(s)$. Hence, for each $s \in S$ there exists an $a_s^* \in A_s$ for which

$$v^*(s) - \lambda \sum_{j \in S} p(j|s,a_s^*)v^*(j) = r(s,a_s^*).$$

To prove the third result, the second result and Theorem 5.2 imply that $v^*(s) = v_\lambda^{(d^*)^\infty}(s)$ where $d^*(s) = a_s^*$ for all $s \in S$. By Theorem 5.7, any feasible solution of the

LP must be an upper bound on $v^*_\lambda(s)$ for all $s \in S$. In particular, $v_\lambda^{(d^*)^\infty}(s) \geq v^*_\lambda(s)$ for all $s \in S$, which implies that the stationary policy on $d^*(s)$ is an optimal policy. Note also that $v^*_\lambda(s) \geq v_\lambda^{(d^*)^\infty}(s)$ for all $s \in S$ by definition of $v^*_\lambda(s)$. Hence, $v^*(s) = v_\lambda^{(d^*)^\infty}(s) = v^*_\lambda(s)$. Since $\mathbf{v}^*_\lambda$ is the unique solution to the Bellman equation, it is also the unique solution to the primal LP, proving the fourth result. □

The above theorem establishes the existence of a solution to the Bellman equation without using the Banach fixed point theorem (Theorem 5.5) and as well the existence of a stationary optimal policy. Moreover it establishes that for each $s \in S$ there exists at least one action for which the inequality

$$v^*(s) - \lambda \sum_{j \in S} p(j|s,a) v^*(j) \geq r(s,a) \tag{5.150}$$

holds with equality. Hence solving the LP and noting which constraints hold with equality identifies an optimal policy. When two or more actions satisfy (5.150) in some state $s \in S$, any stationary policy that randomizes between them in state s is optimal.

It may appear that the optimal solution and policy depend on the particular choice of $\boldsymbol{\alpha}$. However, the proof of Theorem 5.22 remains valid for any $\boldsymbol{\alpha} > \mathbf{0}$, yielding the following important result.

Corollary 5.5. Fix $\boldsymbol{\alpha} > \mathbf{0}$. Let $\mathbf{v}^*$ be an optimal solution to the LP and $(d^*)^\infty$ be a corresponding optimal policy. Then $\mathbf{v}^*$ and $(d^*)^\infty$ are optimal for any other $\boldsymbol{\alpha} > \mathbf{0}$.

This result suggests that the solution of the primal LP lies at the "lower left most" vertex of the feasible region defined by the constraints (5.142b). Furthermore, the feasible region must be contained in the region[33]

$$\{\mathbf{v} \in V \mid \mathbf{v} = \mathbf{v}^* + \mathbf{y}, \mathbf{y} \geq \mathbf{0}\}.$$

5.9.2 The dual linear program

Analyzing a Markov decision process through its dual linear program provides a direct and elegant way of identifying optimal policies and insight into the relationship between between feasible solutions and stationary policies. The key result of this section is Theorem 5.27, which shows that there is a one-to-one correspondence between optimal solutions to the dual LP and optimal policies of the MDP.

Appealing to LP duality theory (see Appendix C), the dual linear program for the discounted MDP may be expressed as:

[33] This region is a *convex cone*, a fundamental concept in linear programming.

Dual LP formulation of discounted MDP

$$\text{maximize} \quad \sum_{s\in S}\sum_{a\in A_s} r(s,a)x(s,a) \tag{5.151a}$$

$$\text{subject to} \quad \sum_{a\in A_s} x(s,a) - \sum_{j\in S}\sum_{a\in A_j} \lambda p(s|j,a)x(j,a) = \alpha(s), \quad s\in S \tag{5.151b}$$

$$x(s,a) \geq 0, \quad a\in A_s, s\in S \tag{5.151c}$$

Some comments regarding this formulation follow.

1. The dual formulation reverses the role of objective function and right-hand side constraint coefficients. Observe that $\alpha(s)$ appears on the right-hand side of (5.151b) as constraints and $r(s,a)$ appears as coefficients in the objective function (5.151a).

2. Note that in equation (5.151b), the index j represents the starting state and the index s represents the state reached after one transition. The reflects the observation below that a matrix representation of the dual may be expressed in terms of the transpose of the transition probability matrix.

3. In contrast to the primal, the objective function of the dual is maximized, instead of minimized.

4. The dual linear program has $|S|$ rows and $\sum_{s\in S}|A_s|$ variables. Hence, usually the number of columns far exceeds the number of rows.

5. In contrast to the primal variables, the dual variables are constrained to be non-negative.

The following example provides the dual formulation of the linear program for the two-state model.

Example 5.17. The dual linear program of (5.143) can be written as :

$$\begin{aligned}
\text{minimize} \quad & 3x(s_1,a_{1,1}) + 5x(s_1,a_{1,2}) - 5x(s_2,a_{2,1}) + 2x(s_2,a_{2,2}) \\
\text{subject to} \quad & x(s_1,a_{1,1}) + x(s_1,a_{1,2}) - 0.8\lambda x(s_1,a_{1,1}) - 0.4\lambda x(s_2,a_{2,2}) = \alpha(s_1) \\
& x(s_2,a_{2,1}) + x(s_2,a_{2,2}) - 0.2\lambda x(s_1,a_{1,1}) \\
& \quad - \lambda x(s_1,a_{1,2}) - \lambda x(s_2,a_{2,1}) - 0.6\lambda x(s_2,a_{2,2}) = \alpha(s_2) \\
& x(s_1,a_{1,1}), x(s_1,a_{1,2}), x(s_2,a_{2,1}), x(s_2,a_{2,2}) \geq 0.
\end{aligned}$$

Letting $\lambda = 0.9$ and $\alpha(s_1) = \alpha = 0.5 = 1 - \alpha(s_2)$, one can use any LP solver to find the optimal solution $x^*(s_1,a_{1,1}) = 0, x^*(s_1,a_{1,2}) = 3.015, x^*(s_2,a_{2,1}) = 0$

and $x^*(s_2, a_{2,2}) = 6.985$. The corresponding dual objective function value is $\sum_s \sum_{a \in A_s} x^*(s,a) r(s,a) = 29.044$.

Recall from Example 5.16 that $v^*(s_1) = 30.147$ and $v^*(s_2) = 27.941$. Thus,

$$\sum_{s \in S} \alpha(s) v^*(s) = 29.044 = \sum_{s \in S} \sum_{a \in A_s} x^*(s,a) r(s,a).$$

This result, where the primal and dual optimal solutions have the same objective function value, is known as the Strong Duality Theorem (see Theorem 5.26 and Theorem C.6).

Where convenient, the dual variables $x(s,a), a \in A_s, s \in S$ will be represented by the vector $\mathbf{x}$. One may think of $\mathbf{x}$ as a "long" vector, where the first $|A_{s_1}|$ components correspond to $x(s_1,a), a \in A_{s_1}$, the second $|A_{s_2}|$ components correspond to $x(s_2,a), a \in A_{s_2}$, and so on.

5.9.3 Dual feasible solutions and stationary policies

This section establishes an elegant result relating feasible solutions to the dual linear program to randomized stationary policies in a Markov decision process. To do so, for each $d \in D^{\mathrm{MR}}, a \in A_s$ and $s \in S$, define $x_d(s,a)$ by:

$$x_d(s,a) := \sum_{j \in S} \alpha(j) \sum_{n=1}^{\infty} \lambda^{n-1} P^{d^\infty}[X_n = s, Y_n = a \,|\, X_1 = j]. \tag{5.152}$$

Let $\mathbf{x}_d$ denote the corresponding vector representation for $x_d(s,a)$.

Assuming that $\sum_{j \in S} \alpha(j) = 1$, the quantity $x_d(s,a)$ can be interpreted as the discounted joint probability under decision rule d^∞, that the process is in state s and chooses action a over the infinite horizon when the process starts in state $j \in S$ with probability $\alpha(j)$.

The following lemma is fundamental when constructing a randomized decision rule from a feasible solution to the dual linear program using (5.153) below.

Lemma 5.8. Suppose $\alpha(s) > 0$ for all $s \in S$. Then if $x(s,a)$ is a feasible solution to the dual linear program

$$\sum_{a \in A_s} x(s,a) > 0.$$

Proof. From (5.151b) and the condition that $x(s,a) \geq 0$ for all $a \in A_s, s \in S$ it follows

immediately that for all $s \in S$,

$$\sum_{a \in A_s} x(s,a) \geq \alpha(s) > 0.$$

□

Given a feasible solution $x(s,a), a \in A_s, s \in S$ to the dual linear program, define the Markovian randomized decision rule $d_{\mathbf{x}}$ by:

$$w_{d_{\mathbf{x}}}(a|s) := \frac{x(s,a)}{\sum_{a' \in A_s} x(s,a')}. \tag{5.153}$$

As a consequence of Lemma 5.8, the denominator is positive so that $w_{d_{\mathbf{x}}}(a|s)$ is well defined. Recall that $w_{d_{\mathbf{x}}}(a|s)$ denotes the probability that the randomized decision rule $d_{\mathbf{x}}$ chooses action a in state s. Consequently, if for each $s \in S$, $x(s,a) > 0$ for exactly one $a \in A_s$, $d_{\mathbf{x}}$ is deterministic.

Feasible solutions and randomized stationary policies

The following theorem establishes the relationship between solutions to the dual and randomized stationary policies. More specifically, when $\mathbf{x}_{d_{\mathbf{x}}}$ is constructed by the following process:

1. obtain a feasible solution to the dual $\mathbf{x}$,
2. construct $d_{\mathbf{x}}(s)$ using (5.153),
3. compute $x_{d_{\mathbf{x}}}(s,a), a \in A_s, s \in S$ by (5.152),

it follows that $\mathbf{x}_{d_{\mathbf{x}}} = \mathbf{x}$. Part 2 of the following theorem shows that this relationship holds. Its proof is straightforward but intricate.

Theorem 5.23. Suppose $0 \leq \lambda < 1$ and $\alpha(s) > 0$ for all $s \in S$. Then

1. For any $d \in D^{\mathrm{MR}}$, if $x_d(s,a)$ is defined by (5.152) for all $a \in A_s$ and $s \in S$, then it is a feasible solution to the dual LP.
2. Suppose $x(s,a)$ is a feasible solution to the dual LP and define $d_{\mathbf{x}}$ by (5.153). Then $x_{d_{\mathbf{x}}}(s,a) = x(s,a)$ for all $a \in A_s$ and $s \in S$.

Proof. By definition $x_d(s,a) \geq 0$ for all $a \in A_s$, $s \in S$. Write the expression on the left hand side of the equality constraint (5.151b) as

$$\begin{aligned}
\sum_{j\in S}\sum_{a\in A_j} \lambda p(s|j,a)x_d(j,a) &= \sum_{j\in S}\sum_{a\in A_j} \lambda p(s|j,a) \sum_{k\in S}\alpha(k)\sum_{n=1}^{\infty}\lambda^{n-1}P^{d^\infty}[X_n = j, Y_n = a\,|X_1 = k] \\
&= \sum_{k\in S}\alpha(k)\sum_{n=1}^{\infty}\lambda^{n}P^{d^\infty}[X_{n+1} = s\,|X_1 = k] \\
&= \sum_{k\in S}\alpha(k)\left(\sum_{n=1}^{\infty}\lambda^{n-1}P^{d^\infty}[X_n = s\,|X_1 = k] - P^{d^\infty}[X_1 = s\,|X_1 = k]\right) \\
&= \sum_{a\in A_s}\sum_{k\in S}\alpha(k)\sum_{n=1}^{\infty}\lambda^{n-1}P^{d^\infty}[X_n = s, Y_n = a\,|X_1 = k] - \alpha(s) \\
&= \sum_{a\in A_s} x_d(s,a) - \alpha(s).
\end{aligned}$$

Rearranging terms shows that $x_d(s,a)$ is a feasible solution to the dual LP.

To prove the second result, assume $x(s,a)$ is a feasible solution to the dual LP. Let

$$u(s) = \sum_{a\in A_s} x(s,a), \tag{5.154}$$

which by Lemma 5.8 is strictly positive for each $s \in S$. From (5.151b) it follows that

$$\begin{aligned}
\alpha(s) &= u(s) - \sum_{j\in S}\sum_{a\in A_j}\lambda p(s|j,a)x(j,a) \\
&= u(s) - \sum_{j\in S}\sum_{a\in A_j}\lambda p(s|j,a)x(j,a)\frac{u(j)}{\sum_{a\in A_j}x(j,a)} \\
&= u(s) - \sum_{j\in S}\sum_{a\in A_j}\lambda p(s|j,a)w_{d_{\mathbf{x}}}(a|j)u(j) \\
&= u(s) - \sum_{j\in S}\lambda P_{d_{\mathbf{x}}}(s|j)u(j),
\end{aligned}$$

where $P_{d_{\mathbf{x}}}(s|j)$ is defined in (5.3). Writing the last expression in matrix notation yields

$$\boldsymbol{\alpha}^{\mathsf{T}} = \mathbf{u}^{\mathsf{T}}(\mathbf{I} - \lambda\mathbf{P}_{d_{\mathbf{x}}}).$$

From Lemma 5.2

$$\mathbf{u}^{\mathsf{T}} = \boldsymbol{\alpha}^{\mathsf{T}}(\mathbf{I} - \lambda\mathbf{P}_{d_{\mathbf{x}}})^{-1} = \boldsymbol{\alpha}^{\mathsf{T}}\left(\sum_{n=1}^{\infty}\lambda^{n-1}\mathbf{P}_{d_{\mathbf{x}}}^{n-1}\right). \tag{5.155}$$

Expressing (5.155) in component notation gives

$$u(s) = \sum_{j\in S}\alpha(j)\sum_{n=1}^{\infty}\lambda^{n-1}P_{d_{\mathbf{x}}}^{n-1}(s|j)$$

$$= \sum_{j \in S} \alpha(j) \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{a \in A_s} P^{d^\infty_{\mathbf{x}}}[X_n = s, Y_n = a \,|X_1 = j] \tag{5.156}$$

$$= \sum_{a \in A_s} x_{d_{\mathbf{x}}}(s, a) \tag{5.157}$$

for all $s \in S$, where (5.156) is a consequence of the law of total probability applied as follows:

$$P^{n-1}_{d_{\mathbf{x}}}(s|j) = P^{d^\infty_{\mathbf{x}}}[X_n = s \,|X_1 = j] = \sum_{a \in A_s} P^{d^\infty_{\mathbf{x}}}[X_n = s, Y_n = a \,|X_1 = j].$$

This establishes that

$$\sum_{a \in A_s} x(s, a) = \sum_{a \in A_s} x_{d_{\mathbf{x}}}(s, a). \tag{5.158}$$

To complete the proof, from the definition of $x_{d_{\mathbf{x}}}(s, a)$, for all $s \in S$ and $a \in A_s$,

$$x_{d_{\mathbf{x}}}(s, a) = \sum_{j \in S} \alpha(j) \sum_{n=1}^{\infty} \lambda^{n-1} P^{d^\infty_{\mathbf{x}}}[X_n = s, Y_n = a \,|X_1 = j]$$

$$= \sum_{j \in S} \alpha(j) \sum_{n=1}^{\infty} \lambda^{n-1} P^{d^\infty_{\mathbf{x}}}[X_n = s|X_1 = j] w_{d_{\mathbf{x}}}(a|s)$$

$$= \sum_{a \in A_s} x_{d_{\mathbf{x}}}(s, a) w_{d_{\mathbf{x}}}(a|s) \tag{5.159}$$

$$= \sum_{a \in A_s} x_{d_{\mathbf{x}}}(s, a) \frac{x(s, a)}{\sum_{a' \in A_s} x(s, a')} \tag{5.160}$$

$$= x(s, a). \tag{5.161}$$

where the third equality (5.159) is a consequence of (5.157) and the last equality is due to (5.158). □

Basic feasible solutions and deterministic stationary policies

Theorem 5.23 established that any stationary randomized policy generates a feasible solution to the dual problem, and that all feasible solutions to the dual problem can be generated by stationary randomized policies.

The following result establishes a correspondence between *basic* feasible solutions[34] to the dual problem and stationary *deterministic* policies.

[34] A basic feasible solution is an extreme point of the region defined by the constraints of the linear program. This means that it cannot be expressed as a convex combination of any other points in the region.

Theorem 5.24. Suppose $0 \leq \lambda < 1$ and $\alpha(s) > 0$ for all $s \in S$. Then

1. For any $d \in D^{\text{MD}}$, if $x_d(s,a)$ is defined by (5.152) for all $a \in A_s$, and $s \in S$ then it is a basic feasible solution to the dual LP.

2. Suppose $x(s,a)$ is a basic feasible solution to the dual LP. Then $d_{\mathbf{x}} \in D^{\text{MD}}$ and $d_{\mathbf{x}}(s) = a_s$ if $x(s,a_s) > 0$.

Proof. If $d \in D^{\text{MD}}$, it follows from Theorem 5.23 that $x_d(s,a)$ is feasible to the dual linear program.

Suppose to the contrary that $x_d(s,a)$ is not a basic feasible solution. Then there exists basic feasible solutions $y(s,a)$ and $z(s,a)$, with $y(s,a) \neq z(s,a)$ for some $a \in A_s$ and $s \in S$ and $\beta \in (0,1)$ for which

$$x_d(s,a) = \beta y(s,a) + (1-\beta) z(s,a)$$

for all $a \in A_s$ and $s \in S$. From Lemma 5.8,

$$\sum_{a \in A_s} y(s,a) > 0 \text{ and } \sum_{a \in A_s} z(s,a) > 0$$

for each s. And since they are distinct solutions, there must be some $\hat{s}$ such that $y(\hat{s},a') > 0$ and $z(\hat{s},a'') > 0$ where a' and a'' are distinct actions in $A_{\hat{s}}$. Since $x(s,a)$ is a convex combination of $y(s,a)$ and $z(s,a)$, $x(\hat{s},a)$ must be positive for at least two distinct actions, implying that $d_{\mathbf{x}}$ is a randomized policy, which is a contradiction.

To prove the second result, first note that for each s, since $\sum_{a \in A_s} x(s,a) > 0$, $x(s,a) > 0$ for at least one $a \in A_s$. If $x(s,a)$ is a basic feasible solution, then it has at most $|S|$ positive components, since the dual problem has $|S|$ equality constraints. Since $\alpha(s) > 0$ for all $s \in S$, $x(s,a) > 0$ for exactly one $a \in A_s$. Thus for each $s \in S$, $d_{\mathbf{x}}(s)$ defined by (5.153) chooses one action in A_s with probability 1, so that it is deterministic. □

Figure 5.14 provides further insight into the consequences of Theorems 5.23 and 5.24. The shaded polyhedron[35] represents the set of feasible solutions to the dual LP. Vertices of the polyhedron are basic feasible solutions, which correspond to stationary deterministic policies. All other points in the polyhedron are feasible solutions, which correspond to stationary randomized policies. Any point in the polyhedron can be written as a convex combination of the vertices, which is equivalent to the fact that randomized policies may be viewed as weighted combinations of deterministic policies.

To make things concrete, consider the two vertices $\mathbf{x}^1$ and $\mathbf{x}^2$ shown in Figure 5.14. They correspond to Markovian deterministic decision rules d_1 and d_2 that choose actions $d_1(s)$ and $d_2(s)$, respectively, in state s. Now, consider a feasible solution

[35] A *polyhedron* is a region formed by the intersection of a finite number of linear inequalities. See Appendix C for definitions of additional linear programming concepts.

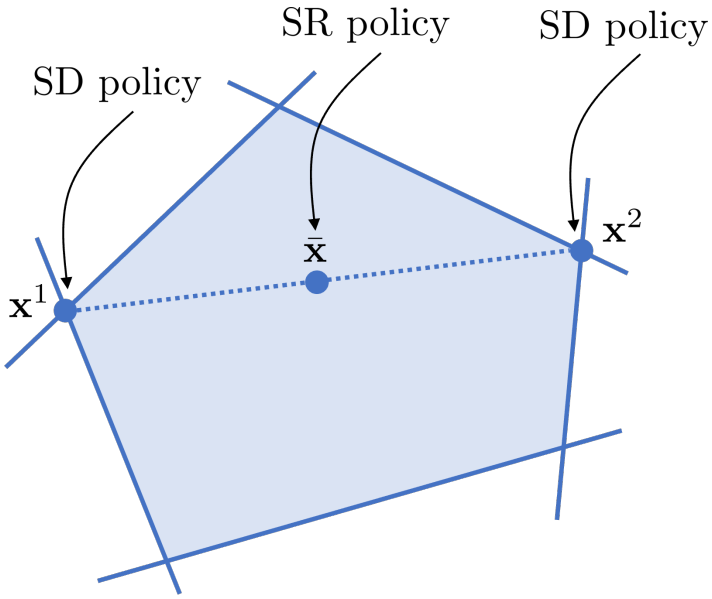


Figure 5.14: Relationship between feasible solutions of the dual LP and stationary policies of the MDP. Basic feasible solutions $\mathbf{x}^1$ and $\mathbf{x}^2$ correspond to stationary deterministic policies. Their convex combination, $\bar{\mathbf{x}}$, corresponds to a stationary randomized policy.

$\bar{\mathbf{x}} = \mu\mathbf{x}^1 + (1-\mu)\mathbf{x}^2$, where $\mu \in (0,1)$, with corresponding Markovian randomized decision rule $d_{\bar{\mathbf{x}}}$. From equation (5.153),

$$w_{d_{\bar{\mathbf{x}}}}(a|s) = \frac{\bar{x}(s,a)}{\sum_{a' \in A_s} \bar{x}(s,a')} \tag{5.162}$$

$$= \frac{\mu x^1(s,a) + (1-\mu)x^2(s,a)}{\sum_{a' \in A_s} (\mu x^1(s,a') + (1-\mu)x^2(s,a'))} \tag{5.163}$$

By definition, for $i = 1, 2$

$$x^i(s,a) = \begin{cases} \sum_{j \in S} \alpha(j) \sum_{n=1}^{\infty} \lambda^{n-1} P^{d_i^\infty}[X_n = s \,|X_1 = j] & \text{if } a = d_i(s) \\ 0 & \text{if } a \neq d_i(s). \end{cases} \tag{5.164}$$

Hence

$$w_{d_{\bar{\mathbf{x}}}}(a|s) = \begin{cases} \frac{\mu x^1(s,d_1(s))}{\mu x^1(s,d_1(s)) + (1-\mu)x^2(s,d_2(s))} & \text{if } a = d_1(s) \\ \frac{(1-\mu)x^2(s,d_2(s))}{\mu x^1(s,d_1(s)) + (1-\mu)x^2(s,d_2(s))} & \text{if } a = d_2(s) \\ 0 & \text{otherwise.} \end{cases} \tag{5.165}$$

Given this 1-1 relationship between feasible solutions of the dual LP and policies of the MDP, and the fact that randomized policies are weighted averages of deterministic policies, it follows that the dual feasible region must be bounded. The proof is left as

an exercise. This result allows us to provide a first principles proof of the existence of solutions to the dual LP.

Theorem 5.25. The feasible region of the dual LP is bounded.

As a preview of the results in the next section, the following result establishes a connection between primal solutions (value functions) and dual solutions (discounted joint probability distributions).

Proposition 5.12. Let $\mathbf{x}$ be a dual feasible solution and $d_{\mathbf{x}} \in D^{\text{MR}}$ be the randomized policy defined by (5.153). Then,

$$\sum_{s\in S}\sum_{a\in A_s} x_{d_{\mathbf{x}}}(s,a)r(s,a) = \sum_{j\in S}\alpha(j)v_{\lambda}^{d_{\mathbf{x}}^{\infty}}(j). \tag{5.166}$$

Proof. From equations (5.14) and (5.152),

$$v_{\lambda}^{d_{\mathbf{x}}^{\infty}}(j) = \sum_{s\in S}\sum_{a\in A_s}\sum_{n=1}^{\infty}\lambda^{n-1}P^{d_{\mathbf{x}}^{\infty}}[X_n = s, Y_n = a\,|X_1 = j]r(s,a), \tag{5.167}$$

and

$$x_{d_{\mathbf{x}}}(s,a) = \sum_{j\in S}\alpha(j)\sum_{n=1}^{\infty}\lambda^{n-1}P^{d_{\mathbf{x}}^{\infty}}[X_n = s, Y_n = a\,|X_1 = j], \tag{5.168}$$

respectively. Thus,

$$\sum_{j\in S}\alpha(j)v_{\lambda}^{d_{\mathbf{x}}^{\infty}}(j) = \sum_{s\in S}\sum_{a\in A_s} x_{d_{\mathbf{x}}}(s,a)r(s,a). \tag{5.169}$$

□

The objective functions for both the primal and dual LPs can be interpreted as the expected discounted reward of policy d^{∞}, in which the expectation is taken over the initial state distribution, in addition to over actions chosen by randomized decision rules and state transitions determined by transition probability function. Note the above result does not ensure that $\mathbf{v}_{\lambda}^{d_{\mathbf{x}}^{\infty}}$ is primal feasible. However, when $\mathbf{x}$ is optimal for the dual, then $\mathbf{v}_{\lambda}^{d_{\mathbf{x}}^{\infty}}$ will be optimal (and feasible, by definition) for the primal.

Optimal solutions and optimal policies

This section examines the relationship between optimal solutions to the dual linear program and it optimal policies.

Theorem 5.26. Let $\boldsymbol{\alpha} > \mathbf{0}$.

1. The dual linear program has an optimal solution.
2. Let $\mathbf{v}^*$ be an optimal solution to the primal linear program and $\mathbf{x}^*$ be an optimal solution to the dual linear program. Then

$$\sum_{s \in S} \alpha(s) v^*(s) = \sum_{s \in S} \sum_{a \in A_s} x^*(s,a) r(s,a). \tag{5.170}$$

Proof. The first result follows from Theorem 5.25 and Theorem C.1. The second result follows from Proposition 5.12. □

Theorem 5.26 is a restatement of the Theorem of Strong Duality of linear programming (Theorem C.6 in Appendix C) for discounted Markov decision processes. It is an immediate consequence of Theorem 5.22 that establishes that the primal LP has an optimal solution.

The next result formalizes the connection between optimal solutions to the dual LP and optimal policies of the MDP. This is the main result of this section.

Theorem 5.27.
1. Suppose $\mathbf{x}^*$ is an optimal solution to the dual LP. Then $d^\infty_{\mathbf{x}^*}$ is an optimal policy.
2. Suppose $(d^*)^\infty$ is an optimal policy. Then $\mathbf{x}_{d^*}$ is an optimal solution to the dual LP.

Proof. Let $\mathbf{v}^*$ and $\mathbf{x}^*$ be optimal solutions to the primal and dual LP, respectively. Let $d_{\mathbf{x}^*}$ be as defined in equation (5.153). Combining equation (5.169) and (5.170) gives

$$\sum_{s \in S} \alpha(s) v_\lambda^{d^\infty_{\mathbf{x}^*}}(s) = \sum_{s \in S} \sum_{a \in A_s} x^*(s,a) r(s,a) = \sum_{s \in S} \alpha(s) v^*(s).$$

For all s, since $\alpha(s) > 0$,

$$v_\lambda^{d^\infty_{\mathbf{x}^*}}(s) = v^*(s). \tag{5.171}$$

Since $\mathbf{v}^*$ is an optimal solution to the primal, it satisfies $\mathbf{v}^* \geq \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}^*$ for all $d \in D^{\mathrm{MD}}$, or equivalently, $\mathbf{v}^* \geq L\mathbf{v}^*$. By Theorem 5.7, $\mathbf{v}^* \geq \mathbf{v}^*_\lambda$. Hence,

$$v_\lambda^{d^\infty_{\mathbf{x}^*}}(s) \geq v^*_\lambda(s) \tag{5.172}$$

for all s. By the definition of $v^*_\lambda(s)$, (5.172) holds with equality so that $d^\infty_{\mathbf{x}^*}$ is an optimal policy.

To prove the second result, from (5.169) it follows that

$$\sum_{s \in S} \sum_{a \in A_s} x_{d^*}(s,a) r(s,a) = \sum_{s \in S} \alpha(s) v_\lambda^{(d^*)^\infty}(s)$$

$$\begin{aligned}
&\geq \sum_{s\in S} \alpha(s) v_\lambda^{d^\infty}(s) \\
&= \sum_{s\in S} \sum_{a\in A_s} x_d(s,a) r(s,a) \\
&= \sum_{s\in S} \sum_{a\in A_s} x(s,a) r(s,a),
\end{aligned}$$

for all $d \in D^{\mathrm{MR}}$. Since any feasible solution $\mathbf{x}$ to the dual represents a decision rule $d \in D^{\mathrm{MR}}$ for which $\mathbf{x}_d = \mathbf{x}$, no feasible solution has objective function value strictly greater than $\mathbf{x}_{d^*}$. Hence, $\mathbf{x}_{d^*}$ is an optimal solution to the dual LP. □

The following theorem is an immediate consequence of Theorem 5.27 applied to basic feasible solutions. It is also a direct consequence of Theorem 5.24.

Theorem 5.28. 1. Suppose $\mathbf{x}^*$ is an optimal basic feasible solution to the dual LP. Then $d^\infty_{\mathbf{x}^*}$ is a deterministic optimal policy.

2. Suppose $(d^*)^\infty$ is a deterministic optimal policy. Then $\mathbf{x}_{d^*}$ is an optimal basic feasible solution to the dual LP.

This theorem is of special note because:

1. Any linear program that has an optimal solution and whose feasible region has at least one basic feasible solution has an optimal basic feasible solution[36].
2. Earlier results establish the existence of an optimal deterministic stationary policy using properties of the Bellman equation.
3. When one exists, LP solvers will return an optimal basic feasible solution.

Some further results*

The dual LP can be used to provide the following direct proof of the existence of a stationary deterministic optimal policy in a discounted Markov decision process.

Since the feasible region includes non-negativity constraints, which means it does not contain a line[37], it follows that it has at least one vertex (see Theorem C.4). A fundamental theorem in linear programming states that if an LP has an optimal solution and the feasible region contains at least one vertex, then at least one of the vertices will

[36] Note that linear programs can have optimal solutions but no optimal basic feasible solution. For example, every point on the line $(x_1, 0)$ in $\mathbb{R}^2$ is optimal if the objective is to minimize x_2 subject to $x_2 \geq 0$, but none are basic feasible solutions. In the case of a Markov decision process, the dual LP polyhedron will always have extreme points since it is bounded.

[37] A set X *contains a line* if there exists a vector $\mathbf{x} \in X$ and a nonzero vector $\mathbf{d}$ such that $\mathbf{x} + \theta\mathbf{d} \in X$ for all $\theta \in \mathbb{R}$.

be an optimal solution. Since each vertex corresponds to a stationary deterministic policy, this argument establishes that there will always exist a deterministic stationary optimal policy.

Finally, recall that it was previously established that an optimal policy is optimal for all $\boldsymbol{\alpha} > \mathbf{0}$, using the primal LP. A proof using the dual follows.

Theorem 5.29. Let $\mathbf{x}^*$ be an optimal dual solution. The corresponding policy $d^\infty_{\mathbf{x}^*}$ is optimal for all $\boldsymbol{\alpha} > \mathbf{0}$.

Proof. First, consider the case where $\mathbf{x}^*$ is an optimal basic feasible solution. Let $d_{\mathbf{x}^*}$ be the policy corresponding to $\mathbf{x}^*$. Then, since $x^*(s,a) = 0$ for all $a \neq d_{\mathbf{x}^*}(s)$, $\mathbf{x}^*$ satisfies (5.151b):

$$(\mathbf{I} - \lambda \mathbf{P}_{d_{\mathbf{x}^*}})^\mathsf{T} \mathbf{x}^* = \boldsymbol{\alpha}. \tag{5.173}$$

Since $\mathbf{I} - \lambda \mathbf{P}_{d_{\mathbf{x}^*}}$ is invertible and since the transpose of the inverse is the inverse of the transpose,

$$\mathbf{x}^* = (\mathbf{I} - \lambda \mathbf{P}_{d_{\mathbf{x}^*}})^{-1} \boldsymbol{\alpha} \geq \mathbf{0}, \tag{5.174}$$

where the inequality follows from Lemma 5.3. This result shows that $\mathbf{x}^*$ is feasible for all $\boldsymbol{\alpha} > \mathbf{0}$. Since changing $\boldsymbol{\alpha}$ does not affect the objective function, $\mathbf{x}^*$ is optimal for the dual LP and $d^\infty_{\mathbf{x}^*}$ is an optimal policy for the MDP for all $\boldsymbol{\alpha} > \mathbf{0}$.

In the case where $\mathbf{x}^*$ is an optimal solution that is not basic, then $\mathbf{x}^*$ can be written as the convex combination of optimal basic feasible solutions. By the above argument, for a given basic feasible solution, the same action is chosen in a given state for all $\boldsymbol{\alpha} > \mathbf{0}$. So the randomized policy corresponding to $\mathbf{x}^*$ will also be optimal for all $\boldsymbol{\alpha} > \mathbf{0}$. □

5.9.4 Solving linear programs

This section describes algorithms and solvers that can be used to find solutions to a linear programs.

Simplex algorithm

The simplex algorithm consists of two phases. Phase one of the simplex algorithm obtains a basic feasible solution to an LP. Phase two iteratively evaluates adjacent basic feasible solutions with improved objective function values until no improvement is possible. Since there are only finitely many basic feasible solutions, it must stop in a finite number of iterations with a local optimum. Since linear programs are convex optimization problems, finding a local optimum is equivalent to finding a global optimum.

Because basic feasible solutions of the dual LP correspond to stationary deterministic policies of the MDP, moving between adjacent basic feasible solutions means that improved actions are identified one state at a time. In other words:

> The simplex algorithm is equivalent to policy iteration in which the value function is updated after an improvement has been identified in a **single** state.

Conversely,

> Policy iteration in which the value function is updated after the improvement step cycles through all states is equivalent to a simplex algorithm with "block pivoting".

Block pivoting in simplex algorithms means that instead of obtaining the value of an improved *adjacent* vertex, the algorithm may jump to and evaluate a further away basic feasible solution, which represents a new deterministic stationary policy with improved actions in multiple states.

Figure 5.14 provides insight into this distinction. Suppose that the simplex algorithm starts at $\mathbf{x}^1$ and the optimal basic feasible solution is at $\mathbf{x}^2$. Then successive iterates would move through adjacent vertices and corresponding stationary policies to reach $\mathbf{x}^2$, while on the other hand, policy iteration might jump immediately to $\mathbf{x}^2$.

Interior point algorithms

Because the simplex algorithm repeatedly seeks and evaluates vertices, it is referred to as an exterior point algorithm. That is, it moves along the outside of the feasible region.

In contrast, interior point algorithms move towards an optimal basic feasible solution through the interior of the feasible region. That is, they approach an optimal deterministic policy through a sequence of randomized policies[38]. No Markov decision process algorithm is a direct analogue. Interior point methods have proven to be quite effective at solving many types of large linear programming problems, and so they may also be effective at solving Markov decision processes.

To visualize this distinction, consider Figure 5.14. Suppose the optimal basic feasible solution is at $\mathbf{x}^2$ and either algorithm starts iteration at $\mathbf{x}^1$. The simplex algorithm would iterate around the outside of the polyhedron while interior point methods would cut across the middle through points of the form $\bar{\mathbf{x}}$.

[38] There may be a relationship between interior point methods and policy gradient algorithms (Chapter 11), which improve over the class of stationary randomized policies.

Solvers

At the time of writing this book, the two most widely used commercial LP solvers are CPLEX (IBM [2024]) and Gurobi (Gurobi Optimization [2024]). They reliably solve large, industrial scale linear programs by exploiting modern computing architectures and encoding variants of the simplex and interior point algorithms.

Several open-source solvers are also available including GLPK (Makhorin [2024]) and COIN-OR CLP (Forrest and the COIN-OR contributors [2024]). These and other open-source solvers interface with all popular languages and applications.

Note that software that solves a primal (dual) linear program will also return the optimal values of its dual (primal) variables. This feature will be extremely useful when using the dual linear program to solve approximate dynamic programs.

5.10 Optimality of structured policies

This section extends results in Section 4.4 to infinite horizon models. The main result is stated in considerable generality.

5.10.1 The fundamental result

Define V^F to be a subset of V that contains elements of a specific form (such as convex) and define D^F to be a subset D^{MD} in which all decision rules are structured. They are *compatible* if $\mathbf{v} \in V^F$ implies that there exists a $d' \in \arg\,\text{c-max}_{d \in D^{\text{MD}}} L\mathbf{v}$ that is in D^F.

For example, in the queuing service rate control model (Section 3.4.1), if V^F denotes the set of convex non-decreasing functions then D^F is the set of monotone non-decreasing decision rules in D^{MD}.

The following theorem uses vector and operator notation. It generalizes Theorem 4.5 for finite horizon models to infinite horizon discounted models.

Theorem 5.30. Suppose that

1. V^F is non-empty,
2. $\mathbf{v} \in V^F$ implies $L\mathbf{v} \in V^F$, and
3. if $\lim_{n\to\infty} \|\mathbf{v}^n - \mathbf{v}^*\| = 0$ and $\mathbf{v}^n \in V^F$ for all $n = 1, 2, \ldots$ implies that $\mathbf{v}^* \in V^F$.

Then if D^F is compatible with V^F, there exists an optimal stationary policy $(d^*)^\infty$ with $d^* \in D^F$.

Proof. The first part of the proof is by induction. Since V^F is non-empty, let $\mathbf{v}^1 \in V^F$ be arbitrary and suppose that $\mathbf{v}^n \in V^F$ for $n = 1, \ldots, t$. Hypothesis 2 implies that

$\mathbf{v}^{t+1} = L\mathbf{v}^t \in V^F$. Hence $\mathbf{v} \in V^F$ for all n.

As a consequence of Theorem 5.16, $\mathbf{v}^n$ converges to $\mathbf{v}^*_\lambda$, the unique solution of the Bellman equation $L\mathbf{v} = \mathbf{v}$. Hence by Hypothesis 3, $\mathbf{v}^*_\lambda \in V^F$. Consequently there exists

$$d^* \in \arg\operatorname*{c\text{-}max}_{d \in D^{\mathrm{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}^*_\lambda\}$$

that is also in D^F. □

Some comments about this theorem follow:

1. The proof relies on the already demonstrated convergence of value iteration to the solution of the Bellman equation.

2. This result differs from that in the finite horizon case by requiring that the $\lim_{n\to\infty} \mathbf{v}^n$ retains the structure of all previous iterates. To establish this requires results from analysis on the limiting behavior of functions[39].

3. Note that the result does *not* say that *every* $d \in \arg\operatorname{c\text{-}max}_{d\in D^{\mathrm{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}^*_\lambda\}$ is in D^F, only that there exists at least one that is in D^F.

5.10.2 A preventive maintenance example

This section illustrates Theorem 5.30 by applying it to determine the structure of an optimal policy in a stationary infinite horizon version of the preventive maintenance model of Section 4.4.2. The Markov decision process formulation is repeated to make this section self-contained.

As before, $S = \{0, 1, \ldots, M\}$ represents the state of the machine with higher state values representing greater deterioration and $A_s = \{0, 1\}$ denotes the set of available actions in each state. Assuming that the state of the machine is known at each decision epoch, the decision maker may either restore the machine (action 1) to state 0 at cost K or operate the machine as is (action 0) incurring a per-period operating cost of $f(s)$ where $f(s)$ is a non-decreasing function of s. Consequently operating the machine in a more deteriorated state costs more than operating it in a less deteriorated state. Assume further that restoring the machine takes one period so that a restored machine in state 0 is available at the start of the next period.

Define

$$p(j|s,a) = \begin{cases} p & j = s+1, a = 0, s \le M-1 \\ 1-p & j = s, a = 0, s \le M-1 \\ 1 & j = 0, a = 1, s \le M \\ 0 & \text{otherwise.} \end{cases}$$

[39] For example, if $S = [0, 1]$ and V^F denotes the set of continuous functions on S, then to establish the third result requires the result that the *uniform* limit of continuous functions is continuous.

Thus in each period the machine deteriorates one state with probability p and remains in the same state with probability $1-p$ except in state M. Since the decision maker seeks to minimize the expected discounted cost of operating the machine over the infinite horizon the problem is best expressed as a minimization problem.

The Bellman equation becomes

$$v(s) = \min\left\{K + \lambda v(0), f(s) + \lambda\big(pv(s+1) + (1-p)v(s)\big)\right\} \tag{5.175}$$

for $s \in \{0, \ldots, M\}$.

Existence of an optimal control limit policy

To apply Theorem 5.30, let V^F represent the set of non-decreasing real-valued functions on S. It was shown in Section 4.4.2 to be compatible with D^F, the set of control limit decision rules of the form:

$$d(s) = \begin{cases} 0 & \text{if } s \leq s^* \\ 1 & \text{if } s > s^*. \end{cases} \tag{5.176}$$

Clearly V^F is non-empty. Section 4.4.2 establishes Hypothesis 2 of the Theorem 5.30. Since the (uniform) limit of non-decreasing functions is non-decreasing, Hypothesis 3 holds. Consequently, there exists an optimal non-decreasing (control limit) policy. Moreover this result is valid if s is replaced with any non-decreasing function.

It is left as an exercise to verify this conclusion numerically, to develop an algorithm for finding an optimal control limit policy by searching within the class of control limit policies, and to provide conditions under which $s^* \in \{0, \ldots, M-1\}$.

5.11 Application: Queuing service rate control

This section compares numerical algorithms and investigates the structure of the optimal policy in the queuing service rate control model formulated in Section 3.4.1.

5.11.1 The model

Consider an infinite horizon discounted version of the discrete-time queuing service rate control model described in Section 3.4.1 and analyzed in the finite horizon case in Section 4.3.1. Assume three service probabilities $a_1 = 0.2$, $a_2 = 0.4$ and $a_3 = 0.6$ and an arrival probability of $b = 0.2$. Further assume that the delay cost is $f(s) = s^2$ and the cost per period of serving at rate a_k is $m(a_k) = 5k^3$.

To use a finite state algorithm requires truncation of the state space. Assume throughout this section that N denotes the truncation level. When the state space is truncated, the transition probabilities in state N for $k = 1, 2, 3$ are

$$p(j|N, a_k) = \begin{cases} a_k & j = N-1 \\ 1 - a_k & j = N. \end{cases}$$

This means that in state N, arrivals are blocked and the only possible event is a service completion.

Since the goal is to minimize costs, a formulation using "min" instead of "max" in the Bellman equation provides more transparent expressions. The Bellman equation for the truncated model follows:

$$v(s) = \min_{k=1,2,3} \begin{cases} m(a_k) + \lambda((1-b)v(0) + bv(1)) & s = 0 \\ f(s) + m(a_k) + \lambda(a_k v(s-1) + (1-b-a_k)v(s) + bv(s+1)) & 0 < s < N \\ f(N) + m(a_k) + \lambda(a_k v(N-1) + (1-a_k)v(N)) & s = N \end{cases}$$

When $m(a_k)$ is increasing in k, which is reasonable for this application, there is no need to minimize in state 0 since it will always be optimal to use the least expensive service rate (or turn off the server if that were possible). Hence the Bellman equation in state 0 simplifies to

$$v(0) = m(a_1) + \lambda((1-b)v(0) + bv(1)). \tag{5.177}$$

The beauty of this model is that as a result of the simple transition structure, there is no need to write out the entire transition matrix when implementing an iterative algorithm.

5.11.2 Value iteration

This section explores the use of value iteration to solve the queuing service rate control model. It terminates with an ϵ-optimal policy by using a span-based stopping criterion and estimates the value function using the lower bound extrapolation.

Iterates of value iteration (Algorithm 5.3) satisfy the following easy-to-code recursions:

$$\begin{aligned} v'(0) &= m(a_1) + \lambda((1-b)v(0) + bv(1)), \\ v'(s) &= \min_{k=1,2,3} \{f(s) + m(a_k) + \lambda(a_k v(s-1) + (1-b-a_k)v(s) + bv(s+1))\},\ 0 < s < N, \\ v'(N) &= \min_{k=1,2,3} \{f(N) + m(a_k) + \lambda(a_k v'(N-1) + (1-a_k)v(N))\}. \end{aligned}$$

The problem is solved for $N = 50$, 200 and 1,000, and $\lambda = 0.5$, 0.9 and 0.99. Each instance starts with $v'(s) = 0$ for all $s \in S$ as a starting value and $\epsilon = 0.0001$ for the stopping criterion. Value iteration terminates with an ϵ-optimal policy when

$$\text{sp}(\mathbf{v}' - \mathbf{v}) < \lambda(1-\lambda)^{-1}\epsilon.$$

At termination the value function is approximated by

$$\mathbf{v}_\lambda^\epsilon = \mathbf{v}' + \lambda(1-\lambda)^{-1}(\underline{\mathbf{v}' - \mathbf{v}})\mathbf{e},$$

which is within ϵ of the optimal value function. While value iteration is only guaranteed to produce an ϵ-optimal policy, analysis using policy iteration below shows that the policy obtained by value iteration is optimal.

Table 5.2 shows the number of iterations for value iteration to achieve the stopping criterion as a function of N and λ. Observe that the number of iterations is increasing in both N and λ. In all cases, the time to solve it on a MacBook Pro M3 was negligible.

N \ λ	0.5	0.9	0.99
50	27	157	384
200	31	202	754
1,000	36	240	1,983

Table 5.2: Iterations of value iteration required to achieve span-based stopping criterion for the discounted queuing service rate control as a function of N and λ.

Figure 5.15 shows the ϵ-optimal stationary policy as a function of λ for two choices of N. Observe that in both instances the ϵ-optimal policy is **monotone** in the service rate. That is, when the queue length is longer, the server should work faster in spite of the higher cost. Moreover, the states at which ϵ-optimal actions change for each value of λ remain the same for all choices of N.

In addition observe that in each state, the service probability increases with respect to λ. For example when $s = 15$, the left figure[40] shows that the ϵ-optimal action is $a_1 = 0.2$ for $\lambda = 0.5$, $a_2 = 0.4$ for $\lambda = 0.9$ and $a_3 = 0.6$ when $\lambda = 0.99$. The reason for this is that when the future becomes less important to the decision maker (that is when λ is smaller), it is not worth the extra cost of using a faster service rate to reduce the queue length. In the extreme, when $\lambda = 0.01$, the ϵ-optimal action in all states (verified up to $N = 5{,}000$) is a_1.

Figure 5.16 shows that the value function is non-decreasing in the queue length. Providing a good parametric approximation to this function will provide the basis for calculations in Chapter 9 using approximate dynamic programming.

5.11.3 Gauss-Seidel value iteration

This section finds ϵ-optimal policies using Gauss-Seidel value iteration (Algorithm 5.4). Its implementation updates states in sequential order as follows:

$$v(0) = m(a_1) + \lambda\big((1-b)v(0) + bv(1)\big),$$

$$v'(s) = \min_{k=1,2,3}\{f(s) + m(a_k) + \lambda\big(a_k v'(s-1) + (1-b-a_k)v(s) + bv(s+1)\big)\},\ 0 < s < N$$

[40] This observation is based on explicitly comparing the ϵ-optimal policies.

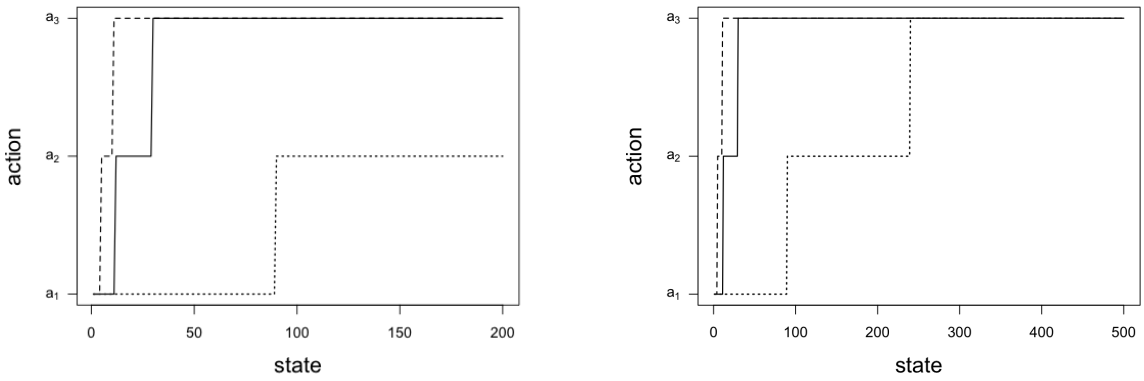


Figure 5.15: The ϵ-optimal stationary policy for the discounted queuing service rate control model as a function of λ for $\lambda = 0.5$ (dotted), $\lambda = 0.9$ (solid) and $\lambda = 0.99$ (dashed) truncated at $N = 200$ (left) and $N = 500$ (right).

$$v'(N) = \min_{k=1,2,3} \{f(N) + m(a_k) + \lambda\big(a_k v'(N-1) + (1-a_k)v(N)\big)\}.$$

Observe that these updates differ from value iteration in that they use $v'(s-1)$ instead of $v(s-1)$ for $s > 1$. As noted earlier, the $s = 1$ case for Gauss-Seidel and value iteration are identical.

Section 5.6.2 indicated that using a span-based stopping criterion for Gauss-Seidel iteration lacks theoretical justification[41]. Instead the Gauss-Seidel implementation uses the norm-based stopping criterion

$$\|\mathbf{v}' - \mathbf{v}\| < (1-\lambda)^{-1}\epsilon,$$

which by Theorem 5.18 guarantees that $\mathbf{v}'$ is within ϵ of the optimal value function and that a stationary policy derived using (5.114) or alternatively from a $\mathbf{v}$-greedy decision rule is an ϵ-optimal policy.

Table 5.3 shows that with the exception of the 50-state model with $\lambda = 0.99$, Gauss-Seidel requires fewer iterations than value iteration to identify an ϵ-optimal policy in spite of using the norm-based stopping criterion. At termination $\|\mathbf{v}'' - \mathbf{v}'\|/\|\mathbf{v}' - \mathbf{v}\|$, where $\mathbf{v}''$, $\mathbf{v}'$ and $\mathbf{v}$ denote three successive iterates, was slightly less than λ (0.987 vs. 0.99, 0.887 vs. 0.9 and 0.443 vs. 0.5), consistent with faster convergence.

5.11.4 Policy iteration

This section describes the use of policy iteration (Algorithm 5.6). The algorithm terminates when the $\mathbf{v}$-greedy decision rule found using (5.123) repeats or equivalently Γ

[41] An alternative is to intermittently perform a value iteration update, followed by evaluating the span of the change in values.

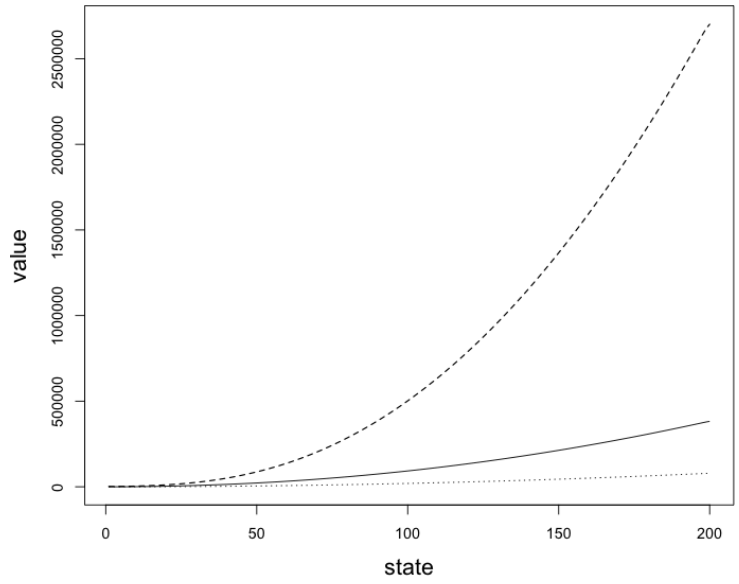


Figure 5.16: Lower bound extrapolation approximations to optimal value function for the discounted queuing service rate control model as a function of s for $\lambda = 0.5$ (dotted), $\lambda = 0.9$ (solid) and $\lambda = 0.99$ (dashed) truncated at $N = 200$.

N \ λ	0.5	0.9	0.99
50	23	90	634
200	24	90	634
1,000	24	102	727

Table 5.3: Iterations of Gauss-Seidel iteration required to achieve norm-based stopping criterion for the discounted queuing service rate control model for different values of N and λ.

is empty. In this implementation, policies are evaluated using the linear system solver in R Core Team [2024]. To do so requires generating the transition probability matrix corresponding to a decision rule, which in this model is tri-diagonal[42].

To explore the robustness of policy iteration, the algorithm is initialized with the sub-optimal decision rule $d^0 = \{a_1, a_2, a_3, a_1, a_2, a_3, \ldots\}$. The number of iterations required and the optimal policy are reported in Table 5.4.

Table 5.4 shows that in all instances except the first, three iterations (policy evaluations and minimizations) were required to find an optimal policy. Since the optimal

[42] A matrix is *tri-diagonal* if all non-zero entries lie on the sub-diagonal, diagonal and super-diagonal. The $row(\cdot)$ and $column(\cdot)$ functions in R Core Team [2024] facilitate generating this matrix.

N	λ	Iterations	First a_2	First a_3
50	0.50	2	n/a	n/a
50	0.90	3	11	29
50	0.99	3	4	10
200	0.50	3	89	n/a
200	0.90	3	11	29
200	0.99	3	4	10
1,000	0.50	3	89	239
1,000	0.90	3	11	29
1,000	0.99	3	3*	10

Table 5.4: Number of iterations and optimal policy obtained when solving the discounted queuing control model with policy iteration. Since the optimal policy is monotone in service rate, the column "First a_2" indicates the lowest state (shortest queue length) at which action a_2 (service rate 0.4) is optimal. Interpret the column "First a_3" analogously. The starred cell is discussed in the text and "n/a" indicates that this action was not used in the optimal policy.

policy for this model was monotone increasing in the service rate as a function of the queue length, the optimal policies can be described by the change points between actions as noted in the table caption. In all cases except one (denoted by *), the optimal policy for the smaller problem instance agreed with that of the larger problem instance.

The instance denoted by * is an anomaly arising from the dual effects of truncation and discreteness of the action space. When solving the problem with $N = 999$ and $N = 1001$, the optimal policy used a_1 for $s \leq 3$ and a_2 for $s \geq 4$ in agreement with the 50 and 200 state versions with $\lambda = 0.99$. When $N = 1{,}000$, it was observed that in the improvement step selecting the "argmax" using

$$a'_s \in \arg\max_{a \in A_s} \left\{ r(s,a) + \lambda \sum_{j \in S} p(j|s,a) v_\lambda^{d^\infty}(j) \right\}$$

resulted in the quantities in brackets in state 4 being 2080.04 when $a = a_1$ and 2080.87 when $a = a_2$. Hence a_1 was selected when $N = 1{,}000$.

To further explore the performance of policy iteration and shed more light on the service rate control model, a larger problem instance was solved. In it, $A_s = \{a_1, \ldots, a_6\}$, corresponding to service rates $\{0.2, 0.3, \ldots, 0.7\}$. The model parameters were $m(a_k) = 2k^3$, $f(s) = s^2$, $b = 0.2$ and $N = 5{,}000$. The discount rate was set at $\lambda = 0.4$[43].

Policy iteration again found an optimal policy in 3 iterations, starting from the decision rule $d^0 = (a_1, a_6, a_1, a_6, \ldots)$. Solving the $5{,}000 \times 5{,}000$ linear system of equations

[43] A small discount rate was chosen in order to obtain a policy in which the change in optimal actions was not restricted to low states. With $\lambda = 0.9$, it was optimal to use action a_6 when $s \geq 106$.

required about 11 seconds on a MacBook Pro M3 computer using the solver in R Core Team [2024]. Figure 5.17 shows that the optimal policy is monotone and the value function is non-decreasing in the queue length.

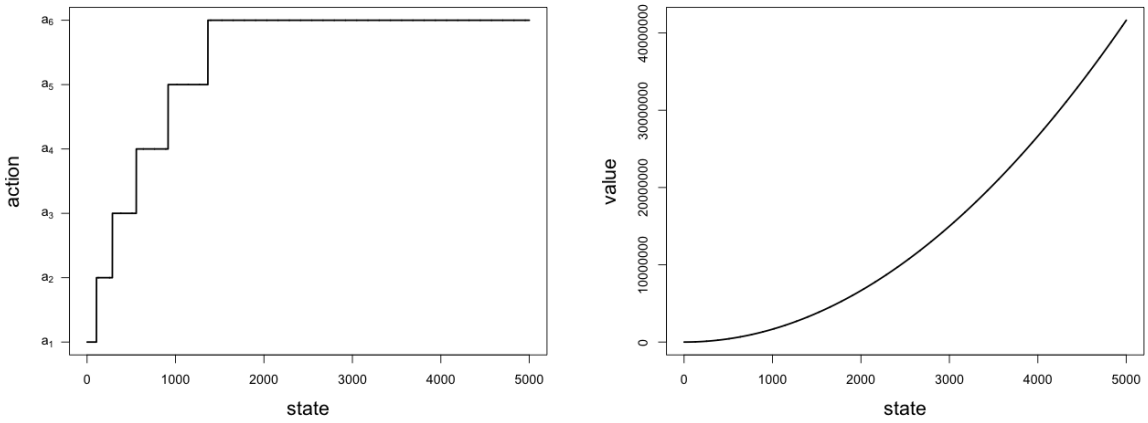


Figure 5.17: Optimal policy (left) and value function (right) obtained when using policy iteration to solve the larger instance of the service rate control problem.

5.11.5 Modified policy iteration

This section solves several versions this model using modified policy iteration with the main objectives being:

1. comparing the computational effort of MPI to that of value iteration,
2. comparing computation time of MPI to policy iteration in large problems, and
3. exploring the effect of the truncation sequence m_n on computational effort.

Theorem 5.13 is used to obtain an ϵ-optimal policy at termination.

Implementation follows Algorithm 5.8. It starts with a decision rule d' and value $\mathbf{v} = \mathbf{0}$ and applies the evaluation recursion (5.138) represented by

$$\mathbf{v} \leftarrow \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v} \tag{5.178}$$

m_n times. The improvement step, (5.139), implemented component-wise, generates both an updated decision rule d and $L_d\mathbf{v}$.

To compare different truncation sequences, note that one improvement step requires the same number of calculations as

$$\frac{\sum_{s \in S} |A_s|}{|S|}$$

evaluation steps (5.138). More formally, define the quantity *Effort* to represent the number of calculations required to perform one evaluation step. MPI trades off the number of "cheap" evaluation steps with "more expensive" improvement steps. In this example, $|A_s| = 3$ for all $s \in S$. Hence an improvement step requires three times more calculations than an evaluation step. This means that if MPI terminated after M iterations, then the total *Effort* is $\sum_{n=1}^{M} m_n + 3M$.

Table 5.5 summarizes *Effort* as a function of the truncation sequence for the instance when the state space is truncated at $N = 200$, $\lambda = 0.9$ and MPI is initialized with the sub-optimal decision rule $d' = (a_1, a_2, a_3, a_1, a_2, a_3, \ldots)$. It shows that all implementations of modified policy iteration required fewer equivalent calculations than value iteration ($m_n = 0$). Note than the number of iterates for value iteration differed from that in Table 5.2 because this instance used a smaller value of ϵ. Observe also that the best choice of a fixed truncation value was between 10 and 20. Moreover the increasing sequences n and $\text{int}(n^{0.5})$[44] required more computational effort than the decreasing truncation sequence $\max(30 - n, 0)$, which required the least.

Truncation sequence m_n	Number of iterations, M	*Effort* $= \sum_{n=1}^{M} m_n + 3M$
0 (value iteration)	221	663
1	111	444
5	37	296
10	21	273
15	14	252
20	11	253
n	21	273
$\text{int}(n^{0.5})$	43	311
$\max(30 - n, 0)$	8	234

Table 5.5: Comparison of *Effort* of modified policy iteration in the queuing service rate control model using various truncation sequences for the instance with $N = 200$ and $\lambda = 0.9$. Entries above the line used fixed m_n and those below the line used m_n that varied in n.

Guided by the above results, a 1,000 state version with $\lambda = 0.9$ was solved. The sequence $m_n = \max(30 - n, 0)$ required the lowest *Effort*, 255, and $M = 10$ iterations. In contrast, *Effort* for $m_n = 20$ was 299 and that of value iteration was 783.

Solution methods were compared for variants of the six-action model (solved in the policy iteration section above) with 5,000, 10,000 and 15,000 states. Algorithms were initialized with the decision rule $d' = (a_1, a_2, a_3, a_1, a_2, a_3, \ldots)$ and $\epsilon = 0.00001$. Value iteration required 293 iterations corresponding to an *Effort* of 1758. Moreover, it required 21.69 minutes, to obtain an ϵ-optimal solution. In contrast, modified policy

[44] $\text{int}(x)$ denotes the integer part of x.

iteration with $m_n = \max(30-n, 0)$ required 12 iterations or, equivalently, *Effort* of 366 to find an ϵ-optimal solution. In contrast to value iteration, execution time for modified policy iteration was 1.38 minutes and that for policy iteration was 2.40 minutes. Thus, both are preferred to value iteration in this example.

Table 5.6 provides execution time for policy iteration and modified policy iteration in larger problems. Observe that modified policy iteration required less time than policy iteration in all cases with the greatest reduction when $N = 15{,}000$. In all cases policy iteration required three iterations to find an optimal policy. The majority of the execution time was spent in the evaluation step solving

$$(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d.$$

Note that constructing the matrix $\mathbf{P}_d$ required considerable computational time. Because of the sparsity of the $\mathbf{P}_d$ matrix, computational time for modified policy iteration could be reduced if (5.178) was implemented component-wise to take into account the matrix structure.

	Policy iteration		Modified policy iteration
N	Total time	Evaluation time	Total time
5,000	2.40	2.00	1.38
10,000	18.92	17.34	16.17
15,000	71.76	67.03	28.23

Table 5.6: Comparison of computation times (in minutes) on a MacBook M3 Pro for policy iteration and modified policy iteration with truncation series varying with respect to problem size. Total time solving systems of linear equations in the evaluation step of policy iteration appears in the column "Evaluation time".

5.11.6 Linear Programming

This section provides the primal and dual formulation of the queuing service rate control model and solves a problem instance using the dual.

Let $\alpha(s) > 0, s \in \{0, 1, \ldots, N\}$ satisfy $\sum_{s=0}^{N} \alpha(s) = 1$ and set $c(s, a_k) = f(s) + m(a_k) = s^2 + 5k^3$.

Primal linear program

Since this is a cost minimization problem, it is natural to formulate the primal linear program as a maximization problem[45] as follows:

[45]The primal LP formulation corresponding to a cost minimization Markov decision process is

$$\text{maximize} \quad \sum_{s \in S} \alpha(s) v(s)$$

$$
\begin{aligned}
\text{maximize} \quad & \alpha(0)v(0) + \cdots + \alpha(N)v(N) \\
\text{subject to} \quad & v(0) - \lambda((1-b)v(0) + bv(1)) \le c(0, a_k),\ k = 1, 2, 3 \\
& v(s) - \lambda(a_k v(s-1) + (1 - a_k - b)v(s) + bv(s+1)) \le c(s, a_k),\ s = 1, \ldots, N-1, k = 1, 2, 3 \\
& v(N) - \lambda(a_k v(N-1) + (1 - a_k)v(N)) \le c(N, a_k),\ k = 1, 2, 3.
\end{aligned}
$$

Observe that there are $3(N+1)$ inequalities, $N+1$ unknowns and no non-negativity constraints. Moreover, the left hand side of the first constraint does not depend on a_k, so it only needs to hold for the least costly action, a_1 (see equation (5.177)). Moreover with a little algebra, the constraints can be simplified.

Dual linear program

As noted earlier, it is preferable to solve the problem through its dual (5.151). The dual formulation is given by:

$$
\begin{aligned}
\text{minimize} \quad & \sum_{s=0}^{N} \sum_{k=1}^{3} c(s, a_k) x(s, a_k) \\
\text{subject to} \quad & \sum_{k=1}^{3} x(0, a_k) - \lambda \sum_{k=1}^{3} \big((1-b)x(0, a_k) + a_k x(1, a_k)\big) = \alpha(0), \\
& \sum_{k=1}^{3} x(s, a_k) - \lambda \sum_{k=1}^{3} \big(bx(s-1, a_k) + (1 - a_k - b)x(s, a_k) + a_k x(s+1, a_k)\big) = \alpha(s), \\
& \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \text{for } s = 1, \ldots, N-1, \\
& \sum_{k=1}^{3} x(N, a_k) - \lambda \sum_{k=1}^{3} \big(bx(N-1, a_k) + (1 - a_k)x(N, a_k)\big) = \alpha(N), \\
& x(s, a_k) \ge 0, \quad k = 1, \ldots, 3,\ s = 0, \ldots, N,
\end{aligned}
$$

Observe that the dual is a minimization problem and has $3(N+1)$ variables, $N+1$ equalities and $3(N+1)$ non-negativity constraints.

Solution

Recall that every feasible solution of the dual corresponds to a randomized policy through (5.153). Moreover from Theorem 5.24, it follows that decision rules corresponding to basic feasible solutions form deterministic policies in which $d^*(s) = a$

$$\text{subject to} \quad v(s) - \lambda \sum_{j \in S} p(j|s, a)v(j) \le r(s, a), \quad a \in A_s, s \in S.$$

Alternatively, the LP problem could be formulated as as a minimization problem, but this would require setting rewards equal to negative costs.

when $x(s,a) > 0$. Since there is an optimal basic feasible solution to the dual, the optimal policy can be easily obtained from it.

Consider a problem instance with $N = 50$, $\lambda = 0.9$ and $\alpha(s) = 1/(N+1)$. Since there are $N+1$ equality constraints, an optimal basic feasible solution to the dual has $N+1$ positive variables and $2(N+1)$ variables equal to zero. Solving the dual using either the function "simplex" in the package "boot" or the function "lpSolve" in the package "linprog" in R , yielded a dual solution of the form:

$$x(s,a_k) > 0 \quad \text{when} \quad \begin{cases} k = 1 \text{ and } s = 0,\ldots,10 \\ k = 2 \text{ and } s = 11,\ldots,28 \\ k = 3 \text{ and } s = 29,\ldots,50. \end{cases}$$

By the above argument, the policy $(d^*)^\infty$ which uses

$$d^*(s) = \begin{cases} a_1, \text{ for } s = 0,\ldots,10 \\ a_2, \text{ for } s = 11,\ldots,28 \\ a_3, \text{ for } s = 29,\ldots,50. \end{cases}$$

is optimal and agrees with that found using policy iteration.

Other observations include:

1. the primal solution, which yielded the optimal value function $\mathbf{v}^*_\lambda$, was identical to that found using the iterative methods above,
2. the optimal primal and dual objective function values were equal (Theorem 5.26),
3. the optimal policy and primal value functions did not vary with $\alpha(s)$ (Theorem 5.29), and
4. larger instances of the problem could be solved easily.

5.11.7 Concluding remarks on queuing control application

This section solved variants of the queuing service rate control model using all of the algorithms in this chapter. A summary of results follow:

1. In all cases the algorithms terminated with monotone non-decreasing policies and value functions. Although modified policy iteration and value iteration found ϵ-optimal policies, they were actually optimal.
2. Policy iteration (and linear programming) are the only algorithms that guarantee optimal (as opposed to ϵ-optimal) policies. However policy iteration requires constructing $\mathbf{P}_d$ and solving a linear system, which is $O(|S|^3)$, so that it becomes time consuming as the problem size increases.

3. Gauss-Seidel iteration required fewer iterations than value iteration but improvements were not remarkable.

4. Modified policy iteration with any reasonable truncation sequence significantly outperformed value iteration. It appeared that a decreasing truncation sequence worked best.

5. Modified policy iteration (with truncation sequence $m_n = \max(30 - n, 0)$) was the fastest method for finding optimal policies in large instances ($N \geq 5{,}000$).

6. Linear programming required some effort to formulate and specialized software to solve. The dual linear program identifies optimal policies directly and the primal yields optimal value functions. An advantage is that it allows constraints to be included.

For large problems, modified policy iteration with a decreasing truncation sequence appeared to be the best approach. Moreover it can be enhanced by using Gauss-Seidel updates when evaluating a fixed policy, coding the updates component-wise so as to exploit the sparsity of the transition matrices, and perhaps augmenting it with action elimination. Policy iteration can be used for small problems but in such settings, any solution method should suffice.

5.12 Application: Clinical decision making

This section analyzes a numerical instance of an infinite horizon discounted variant of the liver transplantation model that was formulated in Section 3.6. Refer to that section for model components and details.

The main purposes of this example are:

1. to analyze a model with a complex transition structure,

2. to illustrate an insightful graphical description of an optimal policy in a multi-dimensional state space,

3. to provide an optimal policy with a clinically meaningful structure, and

4. to delve deeper into an innovative application.

In this application, discounting may be justified in one of two ways: (1) from the perspective that one minus the discount rate represents the probability that a patient dies on any day from a cause other than organ failure or (2) from the perspective that a day alive today is worth more than a potential day alive in the future.

5.12.1 The Bellman equation

Given the model components defined previously, the Bellman equation can be written as

$$v(h,l) = \max\Bigg\{R(h,l), \sum_{\substack{h'\in S_H\\ l'\in S_L}}\Bigg(\big(1+\lambda v(h',l')\big)\gamma(h'|h)\beta(l'|h) + \big(1+\lambda v(h',\Phi)\big)\gamma(h'|h)\phi(h) + \lambda v(\Delta)\delta(h)\Bigg)\Bigg\} \tag{5.179}$$

for $h \in S_H, l \in S_L$,

$$v(h,\Phi) = \sum_{\substack{h'\in S_H\\ l'\in S_L}}\Bigg(\big(1+\lambda v(h',l')\big)\gamma(h'|h)\beta(l'|h)+\big(1+\lambda v(h',\Phi)\big)\gamma(h'|h)\phi(h)+\lambda v(\Delta)\delta(h)\Bigg) \tag{5.180}$$

and $v(\Delta) = 0$[46].

In the maximization, the first term corresponds to accepting the offered organ (a_0) while the second term corresponds to the decision to not accept an organ and wait one period (a_1). The second term is composed of the sum of three expressions: the first corresponds to a transition to state (h', l'), the second to no organ being available and the third corresponding to death due to liver failure. Note further that in each of the first two expressions, the quantity 1 represents an additional day of life.

Observe also that the second term in the maximization is identical to $v(h, \Phi)$ because rejecting an organ results in the same transition probabilities as when there is no organ to transplant[47].

5.12.2 A numerical example

Without access to primary data[48] the implementation here chose parameters that produce an optimal policy with a similar structure to that in the article.

This example specified $H = 20$ and $L = 10$. The daily discount rate $\lambda = 0.99978$ corresponded to an annual discount rate of 0.99. To be compatible with discounting, $R(h, l)$ represents the average *discounted* survival time post transplant for a patient in health state h and organ quality l. It was set to

$$R(h,l) = \alpha\big(365(14 - 0.25(h-1) - 0.6(l-1))\big).$$

[46] Because Δ is a zero-reward absorbing state, this application could be modeled without discounting as a transient model as described in Chapter 6.

[47] This observation simplifies coding the algorithm. Note a similar structure was observed in the online dating model of Section 4.3.3.

[48] Recall this example is based on Alagoz et al. 2007.

With $\alpha = 0.15$, mean survival varies between 7 and 25.6 months[49]

Transition probabilities without transplant were chosen to be

$$\delta(20) = 0.001 \text{ and } \gamma(20|20) = 0.999$$

and for $h = 1, \ldots, 19$

$$\delta(h) = 0.0003h \quad \text{and} \quad \gamma(h'|h) = \begin{cases} 0.997 - 0.0004h & h' = h \\ 0.003 + 0.0001h & h' = h + 1. \end{cases}$$

These probabilities assume that each day, a patient may die, remain in the same health state, or transition to the next higher (worse) health state[50]. Note that properties of transient Markov chains (see Appendix B) can be used to compute the expected number of days to death without transplant by solving $(\mathbf{I} - \mathbf{Q})\mathbf{u} = \mathbf{1}$ where $\mathbf{Q}$ denotes the matrix with components $q(h'|h)$. Using this approach showed that expected survival times without transplant are between 3.3 months in health state 20 and 35.2 months in health state 1[51].

The probability of not being offered an organ for transplant, $\phi(h)$, depends on h, the health state at the start a day. To model the policy that patients in poorer health states are most likely to be offered an organ, the probability of a patient in health state not being offered an organ was chosen to be

$$\phi(h) = 1 - 0.01h \quad \text{for} \quad h = 1, \ldots, 20$$

and

$$\beta(l|h) = 0.001h \quad \text{for} \quad h = 1, \ldots, 20,\ l = 1, \ldots, 10.$$

This probability distribution assumes that the probability of being offered an organ on any day, varies between 0.01 and 0.2 and that organ quality varies uniformly over the quality classes.

5.12.3 Numerical solution

Numerical solutions are obtained by solving the model using policy iteration with iterative policy evaluation[52] and begin the iteration with the decision rule d' that uses

[49] In contrast, the literature suggests survival of 14 years when a low-health state patient receives a high quality organ. The multiplier of 0.15 on R provides interesting policies. With $\alpha = 1$, it is always optimal to transplant.

[50] When the probability of transitioning to worse health states or death increases as the patient's health state degrades, a property known as *increasing failure rate*

[51] Literature suggests that 3-month survival rates vary between 27% and 98% depending on patient health state (MELD score).

[52] This is equivalent to modified policy iteration with a very high order (20,000). Iterative evaluation was used for coding since it is tedious to generate and manipulate matrices with vector-valued states in R.

action a_1 (reject an offered organ) in every state (h, l) for $h \in S_H$ and $L \in S_L$. Note that under this policy, $v(h, l)$ does not depend on l so it can be written as $u(h)$. To find $u(h)$ solve

$$(\mathbf{I} - \lambda \mathbf{Q})\mathbf{u} = \mathbf{1} \tag{5.181}$$

where $\mathbf{I}$ denotes an $|S_H| \times |S_H|$ identity matrix, $\mathbf{Q}$ denotes an $|S_H| \times |S_H|$ matrix with elements $\gamma(h'|h)$[53] and $\mathbf{u}$ denotes an $|S_H|$-component vector with entries $u(h)$.

In Figure 5.18, $u(h)$ is depicted with a solid line. Observe that it declines from over 1056 days for a patient in health state 1 (the best health state) to under 99 days for a patient in health state 20 (the worst health state). This figure also compares survival without transplant to survival post-transplant for the best ($l = 1$) and worst ($l = 10$) organs. It suggests that if the best organ is available, it should be accepted in health states 3-20 and if the worst organ is available it should be accepted in health states 8-20[54].

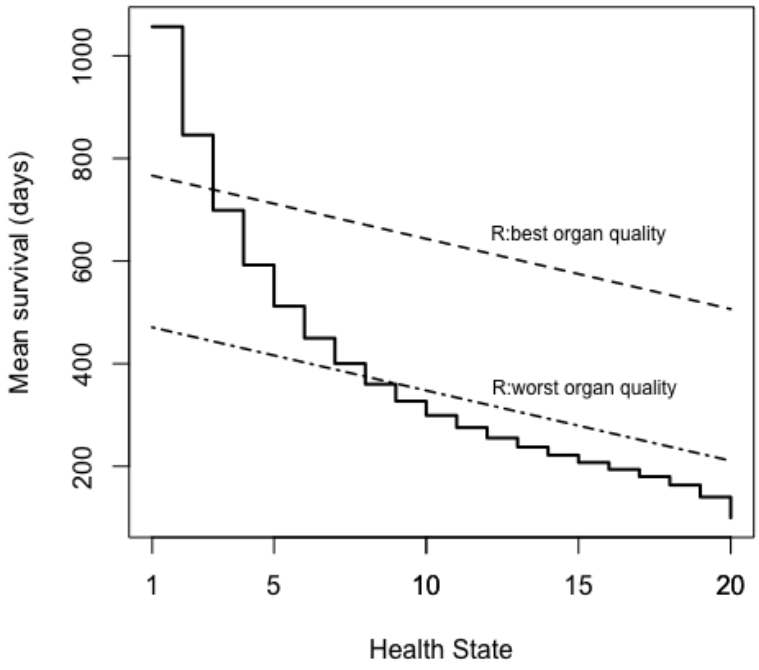


Figure 5.18: Comparison of mean (discounted) survival days $u(h)$ without transplant (solid line) and specified survival post transplant reward $R(h, l)$ for two organ qualities labelled "R:best organ quality" and "R:worst organ quality".

[53] This matrix corresponds to transient states of the health state change process.

[54] Note that choice of $R(h, l)$ was not based on data but specified to generate the pattern in Figure 5.18.

To implement the first improvement step of policy iteration set $v(h,l) = u(h)$ for $L = 1, \ldots, L$ and use the right hand side of (5.179) to identify an improvement. The algorithm converges after two additional iterations.

5.12.4 Interpretation of results

Figure 5.19a displays the optimal policy graphically. Observe that regardless of organ type, it is optimal to wait in health states 1 and 2 and to accept a transplant for any type of organ in health states 8-20. In health states 3-7, the decision to accept an organ depends on the organ quality. In health state 3, it is optimal to accept an organ of quality 1 and 2 while in health state 7 it is optimal to accept an organ of quality of 1 to 9. Observe also that the transplant region shrinks as health quality improves.

From the other perspective, it is optimal to accept the lowest quality organ in health states 8 to 20 and optimal to accept the best organ in health states 3 to 20. Observe also that the transplant region shrinks as the organ quality worsens.

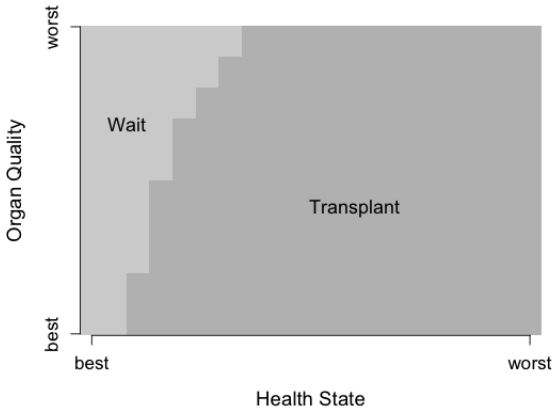
What this means clinically is that a patient in a poor health state should be willing to accept any available organ, while a patient in healthier state can be more selective. At the extremes, a patient in health states 1 or 2 should reject any organ and only consider transplantation when the disease worsens. In other words, a patient should become less selective as disease progresses. From a technical perspective, the optimal policy is a control limit policy in each dimension and jointly[55].

Figure 5.19b shows discounted survival times for the no transplantation option (as in Figure 5.18 and under the optimal policy when the best ($l = 1$) and worst ($l = 10$) organ types are available. Observe that survival times are greater than the always wait action in all health states under either transplant option. This may seem surprising in health state 1 since a patient in that state will not accept any organ. However after eventual transition to a poorer health state, the patient will follow the optimal policy and eventually accept a transplant. Thus improved survival follows because when the patient reaches a poorer health state, the improved survival under transplantation $R(h,l)$ will replace the survival under no transplantation $u(h)$.

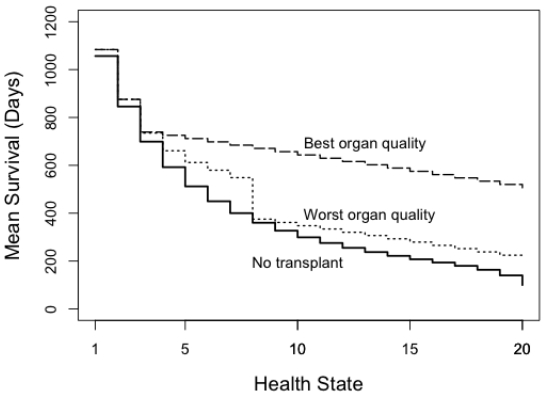
5.12.5 Concluding remarks on clinical decision making application

This section has formulated and used Markov decision processes to analyze a stylized version of a realistic clinical decision problem. It provides a intuitive graphical representation of an optimal policy and interprets it from a clinical perspective. Moreover by comparing and interpreting survival (value functions) under several options, it shows the benefits of using a dynamic decision making as opposed to a myopic approach.

[55] Using methods discussed in Section 5.10, Alagoz et al. [2007] establish this result analytically under specific assumptions on the model components.



(a) Optimal policy as a function of health state and organ quality.



(b) Survival under the optimal policy as a function of health state for two organ qualities and without transplantation.

Figure 5.19: Graphical representation of the optimal policy and value function in the liver transplantation application.

5.13 Technical appendix

5.13.1 Proof of Theorem 5.1

This section proves Theorem 5.1 for the case where $v_\lambda^\pi(s)$ is written as:

$$v_\lambda^\pi(s) = \sum_{n=1}^{\infty} \sum_{j \in S} \sum_{a \in A_j} \lambda^{n-1} r(j,a,k) P^\pi[X_n = j, Y_n = a, X_{n+1} = k | X_1 = s]. \qquad (5.182)$$

Theorem 5.1 follows immediately from the following important yet subtle lemma[56]. Moreover this lemma will provide the basis for establishing an analogous result to Theorem 5.1 for models with the expected total reward and average reward criteria.

Given a history dependent randomized policy, the proof constructs for each starting state $s \in S$, a Markovian randomized policy with equivalent conditional action selection probabilities.

Lemma 5.9. Let $\pi = (d_1, d_2, \ldots) \in \Pi^{\text{HR}}$. For each $s \in S$ and $n = 1, 2, \ldots$, there exists a $\pi' = (d'_1, d'_2, \ldots) \in \Pi^{\text{MR}}$ for which

$$P^\pi[X_{n+1} = k, X_n = j, Y_n = a \,|X_1 = s] \qquad (5.183)$$
$$= P^{\pi'}[X_{n+1} = k, X_n = j, Y_n = a \,|X_1 = s]$$

[56] This result was proved by Strauch [1966] in greater generality.

for $k \in S$, $j \in S$, and $a \in A_j$.

Proof. Fix $s \in S$ and construct $\pi' = (d'_1, d'_2, \ldots) \in \Pi^{\text{MR}}$ by setting

$$w_{d'_n}(a|j,s) := P^\pi[Y_n = a \,|X_n = j, X_1 = s] = \frac{P^\pi[Y_n = a, X_n = j \,|X_1 = s]}{P^\pi[X_n = j \,|X_1 = s]} \tag{5.184}$$

for $j \in S$, $a \in A_j$ and $n = 1, 2, \ldots$[57].

Note the dependence on s in the definition of $w_{d'_n}(a|j,s)$. This is because the action selection probability distribution may differ across initial states. Note further the two quantities in the final term in (5.184) are marginal distributions over all intermediate states and actions[58].

The left hand side of (5.183) can be expressed as a product of conditional probabilities as follows:

$$\begin{aligned} &P^\pi[X_{n+1} = k, X_n = j, Y_n = a | X_1 = s] = \\ &\quad P^\pi[X_{n+1} = k | X_n = j, Y_n = a, X_1 = s]\, P^\pi[Y_n = a | X_n = j, X_1 = s]\, P^\pi[X_n = j | X_1 = s]. \end{aligned} \tag{5.185}$$

The following arguments establish that for $n = 1, 2, \ldots$, each expression in terms of $\pi \in \Pi^{\text{HR}}$ in (5.185) equals the equivalent expression with respect to $\pi' \in \Pi^{\text{MR}}$.

By the Markovian assumption on transition probabilities, X_{n+1} is independent of X_1, conditional on $X_n = j$ and $Y_n = a$. Therefore

$$P^\pi[X_{n+1} = k | X_n = j, Y_n = a, X_1 = s] = p(k|j,a) = P^{\pi'}[X_{n+1} = k | X_n = j, Y_n = a, X_1 = s].$$

By the construction of d'_n, it follows from (5.184) that

$$P^\pi[Y_n = a | X_n = j, X_1 = s] = w_{d'_n}(a|j,s) = P^{\pi'}[Y_n = a | X_n = j, X_1 = s] \tag{5.186}$$

for all n. Thus, what remains is to show is that

$$P^\pi[X_n = j | X_1 = s] = P^{\pi'}[X_n = j | X_1 = s]. \tag{5.187}$$

This follows by induction as follows. Clearly (5.187) holds for $n = 1$. Assume it is true for $t = 2, \ldots, n-1$. Then

$$\begin{aligned} P^\pi[X_n = j | X_1 = s] &= \sum_{l \in S} \sum_{a \in A_l} p(j|l,a) P^\pi[X_{n-1} = l, Y_{n-1} = a | X_1 = s] \\ &= \sum_{l \in S} \sum_{a \in A_l} p(j|l,a) P^\pi[Y_{n-1} = a | X_{n-1} = l, X_1 = s] P^\pi[X_{n-1} = l | X_1 = s] \\ &= \sum_{l \in S} \sum_{a \in A_l} p(j|l,a) P^{\pi'}[Y_{n-1} = a | X_{n-1} = l, X_1 = s] P^{\pi'}[X_{n-1} = l | X_1 = s] \end{aligned}$$

[57] The Markovian randomized decision rule d'_n induces a probability distribution over the actions that matches the probability distribution induced by the given history-dependent randomized decision rule d_n.

[58] See Example 5.19 below.

$$= \sum_{l \in S} \sum_{a \in A_l} p(j|l,a) P^{\pi'}[X_{n-1} = l, Y_{n-1} = a | X_1 = s]$$
$$= P^{\pi'}[X_n = j | X_1 = s],$$

where the third equality is a consequence of the induction hypothesis and (5.186). Hence (5.187) is valid for all n completing the proof. □

Substituting the result of the above lemma into (5.182) proves the following result that appears as Theorem 5.1 in the body of this chapter.

Theorem 5.31. For each $\pi \in \Pi^{\text{HR}}$, there exists a $\pi' \in \Pi^{\text{MR}}$ for which

$$v_\lambda^\pi(s) = v_\lambda^{\pi'}(s)$$

Comments and examples

Lemma 5.9 and its proof are quite subtle. Comments follow.

1. Given a history-dependent policy, this lemma guarantees that for each initial state s there exists a Markovian (randomized) policy $\pi'(s)$, that achieves the same state action probabilities. However, as Example 5.18 shows, it does not guarantee that same policy will achieve this result for all states.

2. The construction of d'_n in (5.184) requires some further explanation. The quantities $P^\pi[Y_n = a, X_n = j, |X_1 = s]$ and $P^\pi[X_n = j | X_1 = s]$ may be computed by *marginalizing* the distribution summing over all omitted terms, for example when deriving

$$P^\pi[X_2 = j \,|X_1 = s] = \sum_{a \in A_a} p(j|s,a) P^\pi[Y_1 = a \,|X_1 = s].$$

3. This result does not guarantee that distribution of sample paths of π and π' will be the same, only that the marginal conditional probabilities necessary to evaluate the expected reward in period n given in (5.183) agree.

4. The result generalizes to a model in which the initial state is chosen according to a probability distribution $\rho(s) := P[X_1 = s]$ in which the conditional action selection probabilities in (5.184) depend on $\rho(\cdot)$.

5. This result could also be applied to a finite horizon model to establish that for every history-dependent policy there exists a Markovian randomized policy with the same state and action probabilities.

Example 5.18. To show that the corresponding Markovian policy depends on the initial system state, consider a deterministic model with $S = \{u, w\}$ and $A_u = A_w = \{a, b\}$. Action a maintains the system in its current state and action b causes the system to move to the other state.

Define the history-dependent policy π that uses a twice and then b once and repeats this sequence indefinitely when the system starts in u and uses b once then uses a twice and repeats this sequence indefinitely when the system starts in w. Since the system is deterministic the action at decision epochs when a state is not visited can be arbitrary.

Hence if the system starts in state u, π generates the sequence of states and actions (history) $(u, a, u, a, u, b, w, a, w, a, w, b, u, \ldots)$ and if it starts in w, it generates the sequence of states and actions $(w, b, u, a, u, a, u, b, w, a, w, a, w, b, \ldots)$. Now construct the decision rules corresponding to a (non-stationary) Markovian policy. Starting in u, $d_1(u) = a$, $d_2(u) = a$ and $d_3(u) = b$. Starting in w, $d'_1(w) = b$, $d'_2(u) = a$ and $d'_3(u) = a$. Since $d_3(u) \neq d'_3(w)$, the Markovian decision rule needed to match the state-action distribution of π varies with the starting state.

The following numerical example illustrates the computation implicit in (5.184).

Example 5.19. Let $S = \{u, w\}$ and $A_u = A_w = \{a, b\}$, $p(u|u, b) = 0.1$ and $p(u|u, a) = 0.4$.

Suppose $\pi \in \Pi^{\mathrm{HR}}$ is such that

$$P^\pi[Y_1 = a|X_1 = u] = 0.7$$

and

$$P^\pi[Y_2 = a|X_2 = u, Y_1 = a, X_1 = u] = 0.2$$
$$P^\pi[Y_2 = a|X_2 = u, Y_1 = b, X_1 = u] = 0.4.$$

Note that at the second decision epoch, the probability distribution of action selection under π depends on the entire history.

Then for example

$$P^\pi[Y_2 = a, X_2 = u, Y_1 = a|X_1 = u] = 0.2 \times 0.4 \times 0.7 = 0.056$$
$$P^\pi[Y_2 = a, X_2 = u, Y_1 = b|X_1 = u] = 0.4 \times 0.1 \times 0.3 = 0.012$$

so that

$$P^\pi[Y_2 = a, X_2 = u|X_1 = u] = 0.056 + 0.012 = 0.068$$
$$P^\pi[X_2 = u|X_1 = u] = 0.4 \times 0.7 + 0.1 \times 0.3 = 0.31$$

Hence d_2' chooses actions according to

$$w_{d_2'}(a|u,u) = \frac{0.068}{0.31} = 0.219 \text{ and } w_{d_2'}(b|u,u) = \frac{0.224}{0.31} = 0.781$$

where $w_{d_2'}(\cdot|u,u)$ is defined for $n=2$ by (5.184). Thus at decision epoch 2, π' chooses actions according to d_2' and is in Π^{MR}.

5.13.2 Bounds for Gauss-Seidel iteration

This section generalizes arguments in Section 5.5.3 to derive bounds for Gauss-Seidel iteration. To do so requires developing some interesting matrix theory.

Choose $\mathbf{v} \in V$ and let $d' \in D^{\text{MD}}$ be such that for $k = 1, \ldots, M$

$$r(s, d'(s_k)) + \lambda \left(\sum_{j<k} p\big(s_j|s_k, d'(s_k)\big) G\mathbf{v}(s_j) + \sum_{j \geq k} p\big(s_j|s_k, d'(s_k)\big) v(s_j) \right)$$
$$= \max_{a \in A_s} \left\{ r(s,a) + \lambda \left(\sum_{j<k} p(s_j|s_k,a) G\mathbf{v}(s_j) + \sum_{j \geq k} p(s_j|s_k,a) v^n(s_j) \right) \right\} = G\mathbf{v}(s_k).$$

Decompose $\mathbf{P}_{d'}$ in to its lower and upper triangular parts, $\mathbf{P}_{d'} = \mathbf{P}_{d'}^L + \mathbf{P}_{d'}^U$, where

$$\mathbf{P}_{d'}^L = \begin{bmatrix} 0 & 0 & 0 & \ldots & 0 & 0 \\ p_{21} & 0 & 0 & \ldots & 0 & 0 \\ p_{31} & p_{32} & 0 & \ldots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ p_{M-1,1} & p_{M-1,2} & p_{M-1,3} & \ldots & \ddots & 0 \\ p_{M1} & p_{M2} & p_{M3} & \ldots & p_{M,M-1} & 0 \end{bmatrix}$$

and

$$\mathbf{P}_{d'}^U = \begin{bmatrix} p_{11} & p_{12} & p_{13} & \ldots & p_{1,M-1} & p_{1M} \\ 0 & p_{22} & p_{23} & \ldots & p_{2,M-1} & p_{2M} \\ 0 & 0 & p_{33} & \ldots & p_{3,M-1} & p_{3M} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \ldots & \ddots & p_{M-1,M} \\ 0 & 0 & 0 & \ldots & 0 & p_{MM} \end{bmatrix}.$$

Then in vector form

$$G\mathbf{v} = \underset{d \in D^{\text{MD}}}{\text{c-max}} \left\{ \mathbf{r}_d + \lambda \mathbf{P}_d^L G\mathbf{v} + \lambda \mathbf{P}_d^U \mathbf{v} \right\} = \mathbf{r}_{d'} + \lambda \mathbf{P}_{d'}^L G\mathbf{v} + \lambda \mathbf{P}_{d'}^U \mathbf{v}.$$

From the above equation, it is easy to see that if $\mathbf{P}_d$ is upper-triangular, that is $\mathbf{P}_d^L$ is a matrix of zeroes, Gauss-Seidel and value iteration are identical. Rearranging terms

in the above expression yields

$$(\mathbf{I} - \lambda \mathbf{P}_{d'}^L) G\mathbf{v} = \mathbf{r}_{d'} + \lambda \mathbf{P}_{d'}^U \mathbf{v}.$$

Since $(\mathbf{I} - \lambda \mathbf{P}_{d'}^L)^{-1}$ exists, it provides the following elegant representation for G

$$\begin{aligned} G\mathbf{v} &= (\mathbf{I} - \lambda \mathbf{P}_{d'}^L)^{-1} \mathbf{r}_{d'} + (\mathbf{I} - \lambda \mathbf{P}_{d'}^L)^{-1} \lambda \mathbf{P}_{d'}^U \mathbf{v} \\ &= \underset{d \in D^{\mathrm{MD}}}{\text{c-max}} \left\{ (\mathbf{I} - \lambda \mathbf{P}_d^L)^{-1} \mathbf{r}_d + (\mathbf{I} - \lambda \mathbf{P}_d^L)^{-1} \lambda \mathbf{P}_d^U \mathbf{v} \right\}. \end{aligned} \tag{5.188}$$

The following additional notation simplifies exposition and suggests some generalizations. For $d \in D^{\mathrm{MD}}$, define two matrices as follows:

$$\mathbf{Q}_d := \mathbf{I} - \lambda \mathbf{P}_d^L \quad \text{and} \quad \mathbf{R}_d := \lambda \mathbf{P}_d^U.$$

Consequently[59]

$$\mathbf{I} - \lambda \mathbf{P}_d = \mathbf{Q}_d - \mathbf{R}_d. \tag{5.189}$$

Hence G can be expressed in this notation as:

$$G\mathbf{v} = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}} \{ \mathbf{Q}_d^{-1} \mathbf{r}_d + \mathbf{Q}_d^{-1} \mathbf{R}_d \mathbf{v} \}. \tag{5.190}$$

The following result generalizes Lemma 5.5 to the operator G. The proof is identical to that lemma in which $L\mathbf{v}$ is replaced by the representation (5.190) for $G\mathbf{v}$.

Lemma 5.10. Let $\mathbf{u} \in V$ and $\mathbf{u} \in V$. If $d_{\mathbf{v}}$ attains the maximum of $G\mathbf{v}$ in (5.190), then

$$G\mathbf{v} - \mathbf{v} \geq G\mathbf{u} - \mathbf{u} + (\mathbf{Q}_{d_{\mathbf{v}}}^{-1} \mathbf{R}_{d_{\mathbf{v}}} - \mathbf{I})(\mathbf{u} - \mathbf{v}). \tag{5.191}$$

An additional result follows.

Lemma 5.11. For any $d \in D^{\mathrm{MR}}$, $\|\mathbf{Q}_d^{-1} \mathbf{R}_d\| = \lambda$ and

$$(\mathbf{I} - \mathbf{Q}_d^{-1} \mathbf{R}_d)^{-1} = \sum_{n=0}^{\infty} (\mathbf{Q}_d^{-1} \mathbf{R}_d)^n. \tag{5.192}$$

Proof. From Lemma 5.2,

$$\mathbf{Q}_d^{-1} = (\mathbf{I} - \lambda \mathbf{P}_d^L)^{-1} = \sum_{n=0}^{\infty} (\lambda \mathbf{P}_d^L)^n \geq \mathbf{I}.$$

[59] This decomposition of $\mathbf{I} - \lambda \mathbf{P}_d$ is referred to as a *splitting*. Other iterative schemes for solving $(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d$ can be expressed in this way.

Since $(\mathbf{I} - \lambda\mathbf{P}_d)\mathbf{e} \geq \mathbf{0}$, the above inequality implies

$$\mathbf{Q}_d^{-1}(\mathbf{I} - \lambda\mathbf{P}_d)\mathbf{e} \geq (\mathbf{I} - \lambda\mathbf{P}_d)\mathbf{e},$$

and it follows from (5.189) that

$$(\mathbf{I} - \mathbf{Q}_d^{-1}\mathbf{R}_d)\mathbf{e} = \mathbf{Q}_d^{-1}(\mathbf{Q}_d - \mathbf{R}_d)\mathbf{e} \geq (\mathbf{I} - \lambda\mathbf{P}_d)\mathbf{e}.$$

Hence $\lambda\mathbf{P}_d\mathbf{e} \geq \mathbf{Q}_d^{-1}\mathbf{R}_d\mathbf{e} \geq \mathbf{R}_d\mathbf{e} \geq \mathbf{0}$, from which it follows that $\lambda = \|\lambda\mathbf{P}_d\| \geq \|\mathbf{Q}_d^{-1}\mathbf{R}_d\|$. But since the first row of Gauss-Seidel is equivalent to value iteration, its row sum, $\mathbf{Q}_d^{-1}\mathbf{R}_d\mathbf{e}(s_1) = \lambda$ from which the first result follows.

Equation (5.192) follows the first result and Lemma 5.2. □

This lemma is used in the proof of the following theorem but also guarantees the row sums of $\mathbf{Q}_d^{-1}\mathbf{R}_d$ are less than or equal to λ. If $\mathbf{P}_d$ is not upper triangular, some, but not necessarily all, of the row sums will be strictly less than λ.

For a matrix $\mathbf{A} \geq \mathbf{0}$, let $\mu(\mathbf{A})$ denote its minimum row sum. Of course $\|\mathbf{A}\|$ equals its maximum row sum. Note that the proof of the following theorem differs slightly from that of Theorem 5.12 because the row sums of $\mathbf{Q}_d^{-1}\mathbf{R}_d$ need not all equal λ.

Theorem 5.32. Suppose $0 \leq \lambda < 1$ and let $\alpha = \min_{d \in D^{\text{MD}}} \mu(\mathbf{Q}_d^{-1}\mathbf{R}_d)$. Then for any $\mathbf{v} \in V$,

$$\begin{aligned}\mathbf{v} + (1-\lambda)^{-1}(\overline{G\mathbf{v} - \mathbf{v}})\mathbf{e} \geq G\mathbf{v} + \lambda(1-\lambda)^{-1}(\overline{G\mathbf{v} - \mathbf{v}})\mathbf{e} \geq \mathbf{v}_\lambda^* \geq \mathbf{v}_\lambda^{d_\mathbf{v}^\infty} \\ \geq G\mathbf{v} + \alpha(1-\alpha)^{-1}(\underline{G\mathbf{v} - \mathbf{v}})\mathbf{e} \geq \mathbf{v} + (1-\alpha)^{-1}(\underline{G\mathbf{v} - \mathbf{v}})\mathbf{e}\end{aligned} \tag{5.193}$$

Proof. Following the argument used to obtain the upper bound in the proof of Theorem 5.12 and (5.192), it follows that

$$\mathbf{v} + \sum_{n=0}^{\infty}(\mathbf{Q}_{d_\mathbf{v}}^{-1}\mathbf{R}_{d_\mathbf{v}})^n(\overline{G\mathbf{v} - \mathbf{v}})\mathbf{e} \geq \mathbf{v} + (\mathbf{I} - \mathbf{Q}_{d_\mathbf{v}}^{-1}\mathbf{R}_{d_\mathbf{v}})^{-1}(G\mathbf{v} - \mathbf{v}) \geq \mathbf{v}_\lambda^*.$$

From Lemma 5.11, $\|\mathbf{Q}_{d_\mathbf{v}}^{-1}\mathbf{R}_{d_\mathbf{v}}\| = \lambda$ so that sum above is bounded by $(1-\lambda)^{-1}$ giving the outer upper bound. The inner upper bound follows from the same argument used to prove Theorem 5.12.

To obtain the lower bound, apply Lemma 5.10 and (5.192) to obtain

$$\mathbf{v}_\lambda^* \geq \mathbf{v} + \sum_{n=0}^{\infty}(\mathbf{Q}_{d_\mathbf{v}}^{-1}\mathbf{R}_{d_\mathbf{v}})^n(\underline{G\mathbf{v} - \mathbf{v}})\mathbf{e}.$$

Follow the steps in the proof of Theorem 5.12 using the lower bounds on $\sum_{n=0}^{\infty}(\mathbf{Q}_{d_\mathbf{v}}^{-1}\mathbf{R}_{d_\mathbf{v}})^n\mathbf{e}$ based on α to complete the proof. □

Note that the upper bounds can be computed throughout the iterative process. However the lower bound cannot be evaluated without knowledge of α or a lower bound on it. When $\|G\mathbf{v} - \mathbf{v}\|$ is sufficiently small to terminate the iterations, the upper bound should provide a reasonable approximation to the optimal value function.

The following example illustrates these bounds in a case where α can be evaluated.

Example 5.20. Consider to the two-state example with $\lambda = 0.9$. Since there are only four Markovian decision rules, one can evaluate α. The following table summarizes calculations.

d	$\mathbf{Q}_d$	$\mathbf{R}_d$	$\mathbf{Q}_d^{-1}\mathbf{R}_d$	$\|\mathbf{Q}_d^{-1}\mathbf{R}_d\|$	$\mu(\mathbf{Q}_d^{-1}\mathbf{R}_d)$
$(a_{1,1}, a_{2,1})$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0.72 & 0.8 \\ 0 & 0.9 \end{bmatrix}$	$\begin{bmatrix} 0.72 & 0.18 \\ 0 & 0.9 \end{bmatrix}$	0.9	0.9
$(a_{1,1}, a_{2,2})$	$\begin{bmatrix} 1 & 0 \\ -0.36 & 1 \end{bmatrix}$	$\begin{bmatrix} 0.72 & .18 \\ 0 & 0.54 \end{bmatrix}$	$\begin{bmatrix} 0.72 & 0.8 \\ 0.26 & 0.60 \end{bmatrix}$	0.9	0.86
$(a_{1,2}, a_{2,1})$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & 0.9 \\ 0 & 0.9 \end{bmatrix}$	$\begin{bmatrix} 0 & 0.9 \\ 0 & 0.9 \end{bmatrix}$	0.9	0.9
$(a_{1,2}, a_{2,2})$	$\begin{bmatrix} 1 & 0 \\ -0.36 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & 0.9 \\ 0 & 0.54 \end{bmatrix}$	$\begin{bmatrix} 0 & 0.9 \\ 0 & 0.54 \end{bmatrix}$	0.9	**0.54**

Observe that $\|\mathbf{Q}_d^{-1}\mathbf{R}_d\| = 0.9$ as guaranteed by Lemma 5.11. Also note that $\alpha = 0.54$. Since α corresponds to the optimal policy, one would expect the lower bound to apply.

The following table of iterates of Gauss-Seidel with $\mathbf{v}^0 = \mathbf{0}$ shows how these bounds behave empirically. Each entry of the table represents a bound on each component of the vector of iterates.

Iteration	Upper bound	Lower bound
10	$(32.958, 31.022)$	$(24.597, 22.661)$
20	$(30.799, 28.655)$	$(28.860, 26.717)$
30	$(30.299, 28.107)$	$(29.849, 27.657)$
40	$(30.182, 27.980)$	$(30.078, 27.875)$

Since $\mathbf{v}^*_\lambda = (30.147, 27.941)$, the bounds are quite adequate after 40 iterations with the upper bound being a closer approximation of $\mathbf{v}^*_\lambda$ than the lower bound. Note also that when α replaced λ in the upper bound and λ by α in the lower bound, neither were valid.

5.13.3 Convergence of policy iteration in non-finite models*

This section represents a departure from most of the book in that it applies to models with non-finite state and/or action sets. Figure 5.20 symbolically represents the logic underlying the Newton's method representation for policy iteration, suggests why it converges in a few iterations in a finite model and also provides the basis for proofs of convergence of modified policy iteration.

Iteratively applying Lemma 5.6 and using the established convergence of value iteration yields the following important convergence result for policy iteration. See the proof of Theorem 5.20 for a detailed proof using the same logic.

Theorem 5.33. Suppose $B\mathbf{v}^0 \geq \mathbf{0}$ and for all $\mathbf{v} \in V$ there exists a

$$d_{\mathbf{v}} \in \arg\max_{d \in D^{\mathrm{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}. \tag{5.194}$$

Then the sequence of iterates $\mathbf{v}^n, n = 0, 1, \ldots$ of policy iteration converges monotonically and in norm to $\mathbf{v}^*_\lambda$.

The condition $B\mathbf{v}^0 \geq \mathbf{0}$, or equivalently $L\mathbf{v}^0 \geq \mathbf{v}^0$, holds when policy iteration starts with a decision rule d^0 in step 2, which is evaluated in step 3 to obtain $\mathbf{v}^0$, because

$$L\mathbf{v}^0 \geq \mathbf{r}_{d^0} + \lambda \mathbf{P}_{d^0} \mathbf{v}^0 = B\mathbf{v}^0.$$

Alternatively policy iteration can start in step 3 with any $\mathbf{v}^0$ satisfying $B\mathbf{v}^0 \geq \mathbf{0}$. Recall that if $B\mathbf{v}^0 \geq \mathbf{0}$, $\mathbf{v}^0 \leq \mathbf{v}^*_\lambda$.

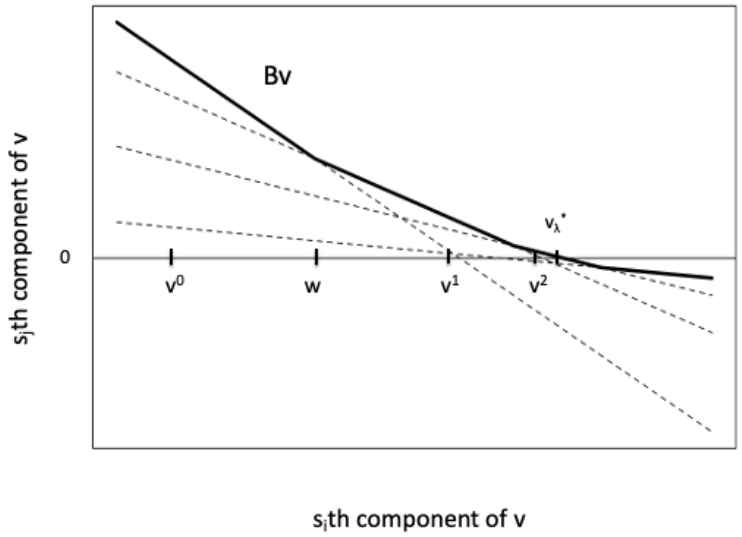


Figure 5.20: The bold line represents $B\mathbf{v}(s_j)$ as a function of $v(s_i)$ for two states s_i and s_j and the horizontal line represents $v(s_j) = 0$. The dashed lines represent $B_d\mathbf{v}(s_j)$ as a function of $v(s_i)$ for each of four decision rules. When policy iteration starts at $\mathbf{v}^0$, the improvement step identifies a $\mathbf{v}^0$-greedy decision rule d^0 with support $B_{d^0}\mathbf{v}$. In the evaluation step, it solves $B_{d^0}\mathbf{v} = \mathbf{0}$ to obtain the value $\mathbf{v}^1$. Repeating this process shows that policy iteration generates the iterates $\mathbf{v}^2$ and $\mathbf{v}^3 = \mathbf{v}^*_\lambda$. Observe that if one were able to identify $\mathbf{w}$ with less effort than solving $B_{d^0}\mathbf{v} = \mathbf{0}$, the $\mathbf{w}$-greedy decision rule would be identical to that at $\mathbf{v}^1$. Modified policy iteration seeks to do this.

The crucial hypothesis is that the maximum is attained in (5.194). Conditions under which this occurs include:

1. A_s finite for all $s \in S$, and

2. A_s compact for all $s \in S$, and $r(s,a)$ and $p(j|s,a)$ continuous in a for each $s \in S$.

Note that S no longer needs to be finite. It can also be compact or countable, and convergence will be guaranteed.

In non-finite circumstances, policy iteration may never satisfy the stopping criterion $d' = d$. In such cases, terminate the algorithm using a stopping criterion based on $\|\mathbf{v}^{n+1} - \mathbf{v}^n\|$ or $\text{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n)$. The value of the decision rule at termination provides the tightest lower bound while the tightest upper bound from Theorem 5.12 applies.

Note that this approach can also be used when the problem is finite if one seeks only an ϵ-optimal policy. Note further that action elimination can also be incorporated in policy iteration.

Quadratic convergence of policy iteration

As observed in examples, policy iteration usually converges in a few iterations in finite state and action models. The following theorem provides conditions under which policy iteration converges quadratically in non-finite models.

Theorem 5.34. Suppose for all $\mathbf{u}$ and $\mathbf{v}$ in V, there exists a $K > 0$ for which

$$\|\mathbf{P}_{d_\mathbf{u}} - \mathbf{P}_{d_\mathbf{v}}\| \leq K\|\mathbf{u} - \mathbf{v}\| \tag{5.195}$$

where $d_\mathbf{v}$ and $d_\mathbf{u}$ are greedy with respect to $\mathbf{u}$ and $\mathbf{v}$. Then

$$\|\mathbf{v}^{n+1} - \mathbf{v}^*_\lambda\| \leq \frac{K\lambda}{1-\lambda}\|\mathbf{v}^n - \mathbf{v}^*_\lambda\|^2. \tag{5.196}$$

Refer to Section 6.4.4 in Puterman [1994] for a proof of this theorem, an example illustrating quadratic convergence and conditions on the model components that ensure (5.195) holds.

Note that condition (5.195) does not hold for all $\mathbf{u}$ and $\mathbf{v}$ in finite state and action models. To understand why consider the value $\mathbf{w}$ in Figure 5.20 at which there is a "kink" in $B\mathbf{v}$. This means that at $\mathbf{w}$ there are two $\mathbf{w}$-greedy decision rules, say d_1 and d_2, where d_1 is greedy at $\mathbf{u} = \mathbf{w} - \epsilon\mathbf{1}$ and d_2 is greedy at $\mathbf{v} = \mathbf{w} + \epsilon\mathbf{1}$ for small $\epsilon > 0$. Thus $\|\mathbf{P}_{d_1} - \mathbf{P}_{d_2}\| > 0$ will remain constant even when ϵ becomes extremely small so the bound cannot hold.

Sufficient conditions for (5.195) to hold[60] include that for each $s \in S$:

1. A_s compact and convex,
2. $p(j|s,a)$ is affine in a for each $a \in A_s$, and
3. $r(s,a)$ is strictly convex and twice continuously differentiable in $a \in A_s$.

Bibliographic remarks

Howard [1960], in his doctoral thesis, was the first to describe and analyze the discounted model in considerable generality. However, discounting appeared earlier in inventory models such those described in Arrow et al. [1958]. Blackwell [1962] and

[60] Results that describe the form of $\mathbf{v}$-greedy decision rules are referred to as *selection theorems*.

Blackwell [1965] were seminal papers on the theory of discounted Markov decision process models, with the former being easy to read and most relevant to this book.

The use of value iteration and contraction mappings for discounted models originates with the analysis of stochastic sequential games by Shapley [1953]. However, value iteration has antecedents in earlier unpublished work described in Bellman [1957].

Policy iteration originates with Howard [1960] and Bellman [1957]. The relationship between policy iteration and Newton's method was formalized in Puterman and Brumelle [1979]. The example that policy iteration requires $|S|$ steps is based on an example described by Tsitsiklis in a personal communication. The discussion of strict improvement for policy iterations is motivated by results on pp. 40-41 in Derman [1970].

The discussion of the span follows Section 6.6.1 of Puterman [1994] which is based on the excellent paper Hübner [1977]; its use in Markov decision processes originates with Bat. Theorem 4.1.3 in Kemeny and Snell [1960] develops a precursor to Proposition 5.4.

Bounds and their use to identify sub-optimal actions originate with MacQueen [1966] and MacQueen [1967]. Porteus [2002] provides a good reference to this material.

Hastings [1968] and Kushner and Kleimnan [1971] independently proposed using Gauss-Seidel iteration to improve the performance of value iteration algorithms. The proof that the Gauss-Seidel operator G is a contraction uses concepts in Bertsekas [2012]. The bounds for GS in Section 5.13.2 are new and use several results from Puterman [1994] regarding representing Gauss-Seidel updates as a regular splitting. The concepts of hybrid algorithms and details on asynchronous value iteration may be found in Bertsekas [2012].

Modified policy iteration was independently proposed by Puterman and Shin [1978] and van Nunen [1976]. Its discussion here follows Puterman [1994] and the references therein. Canbolat and Rothblum [2013] establishes convergence of modified policy iteration and bounds for any starting value. Note that this paper originally appeared as an unpublished manuscript in 1979 and was published posthumously in 2013. See also Bertsekas [2012], who refers to this algorithm as *optimistic* policy iteration. At this point in time, we think *approximate* policy iteration would have been a better descriptor.

The linear programming formulation of discounted models originates with D'Epenoux [1960]. Kallenberg [1983] provides a comprehensive study of this model under a range of optimality criteria. Hillier and Lieberman [2021] provides a good introduction to linear programming and Bertsimas and Tsitsiklis [1997] provides a more theoretical treatment.

Our formulation and analysis of the clinical decision making application is based on Alagoz et al. [2007].

Exercises

1. Consider Exercise 1 from Chapter 2. Assume $\lambda = 0.9$.

(a) Write out the Bellman equation for the value function and the state-action value function.

(b) Find an ϵ-optimal policy using value iteration and Gauss-Seidel iteration. Plot the norm and span for each as a function of the iteration number.

(c) Evaluate the bounds for value iteration and Gauss-Seidel iteration after 10, 20 and 30 iterates.

(d) Apply an asynchronous value iteration algorithm in which the state to update is chosen from a uniform distribution on the set of states.

(e) Apply value iteration with action elimination.

(f) Solve the problem with policy iteration and a variant that re-evaluates the value function as soon as a strictly improving action is identified. Compare the convergence and progress of these two variants of policy iteration.

(g) Formulate and solve the problem using linear programming.

2. Find an ϵ-optimal policy for the model in Exercise 1 from Chapter 2 using the following variants of modified policy iteration.

(a) Initiate it with $\mathbf{v} = \mathbf{0}$.

(b) Initiate it with a decision rule of your choice.

(c) Choose m_n fixed, increasing and decreasing.

(d) Integrate the algorithm with Gauss-Seidel updating where appropriate.

(e) Summarize your conclusions on the best approach for implementing modified policy iteration.

3. Consider an infinite horizon version of the periodic inventory review problem of Section 3.1 with a finite warehouse capacity of 10 units, no backlogging, fixed ordering cost of \$12, per unit ordering cost of \$2, revenue of \$5 per item sold, and holding cost of \$1 per unit per period. Assume a scrap value of \$1 per item. If demand cannot be fulfilled by stock on hand after receipt of an order, it is lost. Assume demand is geometrically distributed with $p = 0.3$.

(a) Solve the problem assuming there is no delay between when the order is placed and it arrives. Does the optimal policy have any obvious structure?

(b) Solve the problem assuming that when an order is placed it arrives at the end of the period after demand is met from stock on hand. How do your results compare to those when the stock arrives instantaneously?

4. Consider the replacement model in Section 5.10.2 with $M = 100$, $p = 0.5$, $\lambda = 0.9$ and $K = 50$.

(a) Use policy iteration to find an optimal policy and comment on its structure.

(b) Use linear programming to find an optimal policy.

(c) Develop a policy iteration type algorithm that exploits the result that there exists an optimal control limit policy for this model.

5. Solve a discounted infinite horizon version of the restaurant management problem (Exercise 15 in Chapter 3) using $\lambda = 0.9$.

6. Consider a discounted infinite horizon version of the queuing admission control model of Section 3.4.2. Assume the arrival probability is $b = 0.3$, the service probability is $w = 0.2$, the holding cost is $h(j) = 3j$, the payment is $R = 60$ and the discount rate is 0.95. Moreover, assume that the queue capacity is 100 and that the controller must reject a job if the queue is full.

   (a) Write the Bellman equation for this model.

   (b) Find an optimal policy using policy iteration.

   (c) Find an ϵ-optimal policy using value iteration and modified policy iteration.

   (d) Prove that there exists an optimal control limit policy.

7. **Knowing when to quit.** Consider a game in which transitions occur between states according to a random walk on the integers $\{0, \ldots, M\}$ with transition probabilities $p(j+1|j) = 1 - p(j-1|j) = q$ for $1 \leq j \leq M-1$. Assume further that the game stops when either state 0 or state M is reached. In state 0, the player receives 0 and in state M, the player receives M.

   At each decision epoch the decision maker may quit the game and receive a reward of j if the game is in state $j < M$ or continue playing. Recall that stopping corresponds to a transition to a reward-free absorbing state Δ. Assume a discount rate of $\lambda < 1$.

   (a) Provide the Bellman equation for this model.

   (b) Show that the optimal strategy is to stop immediately if $q \leq 0.5$.

   (c) Find an optimal strategy for all combinations of $M \in \{20, 100\}, q \in \{0.6, 0.75, 0.9\}$ and $\lambda \in \{0.60, 0.90, 0.99\}$ using policy iteration and linear programming.

   (d) Verbally describe the optimal policy and how it varies with the model parameters.

   (e) *Derive the optimal policy analytically.

   (f) How would results change if on stopping in state s, the player receives s^2 or $\sqrt{s}$?

8. Solve the primal and dual linear programs for the two-state model for $\lambda = 0.9$ and $\alpha \in \{0.2, 0.4, 0.6, 0.8\}$. Verify that Strong Duality holds, and that the optimal policies are the same as the ones derived through value iteration and policy iteration.

9. **Discounted optimal stopping.**[61] The purpose of the exercise is to investigate policy evaluation in a simple optimal stopping model. Consider an optimal stopping problem on states $S = \{s_1, s_2, s_3, s_4\}$. Assume in states s_1 and s_2 stopping is possible and in states s_3 and s_4 stopping is not possible. Stopping yields a reward $R = 25$ and continuing yields a reward of 1 in states s_1, s_2, s_3 and 0 in s_4. Let Δ denote the stopped state. Assume the unstopped Markov chain transitions are determined by the transition probability matrix

$$\mathbf{P} = \begin{bmatrix} 0.98 & 0.01 & 0.01 & 0 \\ 0.01 & 0.98 & 0.01 & 0 \\ 0.005 & 0.005 & 0.985 & 0.005 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

   (a) Evaluate the policy d^∞ that stops in state s_1 and continues otherwise. Assume $\lambda = 0.999$.

      i. Evaluate this policy by solving by solving $(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d$.

      ii. Evaluate this policy using the recursion $\mathbf{v}' = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}$. How many iterations are required to accurately compute $\mathbf{v}$?

      iii. Evaluate this policy using the component form recursion

$$v'(s) = r(s, d(s)) + \lambda \sum_{j \in S} p(j|s, d(s))v(j)$$

      where the summation is computed recursively.

   (b) Find an optimal policy for the optimal stopping problem.

   (c) Investigate the sensitivity of the optimal policy to the value of R.

10. Consider a version of the liver transplantation model of Section 5.12 with $H = 2$ and $L = 2$[62].

   (a) Depict the model symbolically in the form of Figure 2.7 in two ways, one in which the state corresponding to death from transplant failure is distinguished from the state which results from accepting an organ for transplantation, and one in which these two stopped states are combined.

   (b) Using the later specification, provide the transition probability matrix where the two-dimensional state is indexed by (h, l). Also specify the reward function $r((h, l), a)$.

   (c) Using parameters of your choice and the discount rate $\lambda = 0.9997$, evaluate the policy that accepts an organ only in state $(2, 2)$ in three ways:

[61] This example is a good jumping off point for analyzing the transplant model in the next few exercises.

[62] We used this example to debug our code for solving the model in Section 5.12.

i. By solving $(\mathbf{I} - \lambda\mathbf{P})\mathbf{v} = \mathbf{r}$.

ii. Using an iterative scheme in vector form $\mathbf{v}' = \mathbf{r} + \lambda\mathbf{P}\mathbf{v}$. How many iterations are required to accurately compute the exact value found by solving the linear equation?

iii. Using a similar recursion in component form in which the summations are computed recursively.

11. This example revisits the liver transplantation application formulated and analyzed in Section 5.12 using policy iteration and value iteration. Consider reducing the annual discount rate to 0.8. Determine the daily discount rate, generate a policy map, and compare it to Figure 5.19a. Are the changes in the policy map intuitive, given the change in the discount factor?

12. Consider a variant of the liver transplantation problem with only one organ type.

    (a) Solve the model numerically using the specification in Section 5.12 when only an organ of quality 1 is available.

    (b) Repeat your analysis when only an organ of quality 10 is available and compare your results.

    (c) Provide conditions under which the optimal policy is a control limit policy in the health state. Prove that the optimal policy has this form.

    (d) Solve a variant of the problem where there is a single health state (state 6) and multiple organ types. Be sure to redefine transition probabilities.

13. **A really neat problem**[63] Suppose d and e are two decision rules in D^{MD} and define a new decision rule $f \in D^{\text{MD}}$ by

$$f(s) = \begin{cases} d(s) & \text{whenever } v_\lambda^{d^\infty}(s) \geq v_\lambda^{e^\infty}(s) \\ e(s) & \text{whenever } v_\lambda^{d^\infty}(s) < v_\lambda^{e^\infty}(s) \end{cases}$$

    Show that $v_\lambda^{f^\infty}(s) \geq \max\{v_\lambda^{d^\infty}(s), v_\lambda^{e^\infty}(s)\}$ for all $s \in S$.

14. Show that the third term of equation (5.14) in the sum over n is equal to (5.17).

15. Show that $\text{sp}(\mathbf{v})$ is a semi-norm on V.

16. Prove that $\text{sp}(\mathbf{v})$ is a semi-norm and in addition satisfies the three properties following its definition.

17. Evaluate γ and γ' for each of the three other Markovian deterministic policies using the approach in Example 5.7 and equations (5.80) and (5.81). Also, compute the eigenvalues of each of the transition matrices and compare them to γ.

[63] This was on author MLP's final exam when he took a Markov decision process course in graduate school many, many years ago.

18. Evaluate the bounds in Example 5.9 for $\mathbf{v} = (0, 0)$, $\mathbf{v} = (-10, -10)$, and $\mathbf{v} = (20, 20)$. Comment on the tightest of the generated bounds from these value functions.

19. Implement value iteration in Example 5.11 using $\mathbf{v}^0 = (0, 0)$, $\mathbf{v}^0 = (-10, -10)$, and $\mathbf{v}^0 = (20, 20)$ with the span-based stopping criterion. Comment on convergence as a function of the initial value function.

20. Suppose $\pi^* = (d_1, d_2, d_3, \ldots) \in \Pi^{\text{HR}}$ is an optimal policy. Show that $(d_1)^\infty$ is a stationary optimal policy.

21. Prove a variant of Proposition 5.9 when there are strict improvements in two states.

22. Provide a formal proof of Theorem 5.33 using induction and Lemma 5.6.

23. Prove Lemma 5.7.

24. Write out and prove Lemma 5.4 in component notation.

25. Of the four value functions in Example 5.5, verify that $\mathbf{v}_\lambda^{d_4^\infty}$ is the only one that satisfies the Bellman equation and does so for arbitrary λ.

26. Letting $\mathbf{y}_d$ be the vector of dual variables corresponding to the constraints in the vector version of the primal LP, formulate the dual LP in vector notation. Show that by creating a suitable "long" vector, the dual LP can be written as a standard form LP.

27. Prove Theorem 5.25.

28. Consider the preventive maintenance example from Section 5.10.2.

    (a) Verify the result that a control limit policy is optimal by numerically solving the problem over a range of parameter values of your choosing.

    (b) Develop an algorithm for finding an optimal control limit policy by searching only within the class of control limit policies.

    (c) *Provide conditions on $f(s)$ and p that ensure that $0 \leq s^* \leq M - 1$ where s^* is defined in (5.176).

29. Apply Gauss-Seidel iteration directly in Example 5.20 and verify that it gives the same result.

30. Suppose in Example 5.19 that

$$P^\pi[Y_2 = a | X_2 = u, Y_1 = a, X_1 = w] = 0.5$$
$$P^\pi[Y_2 = a | X_2 = u, Y_1 = b, X_1 = w] = 0.3.$$

(a) Compute $w_{d_2'}(a|u,w)$ and $w_{d_2'}(b|u,w)$ where these quantities are defined in (5.184).

(b) What quantities are needed to specify d_2' ?

(c) Suppose the initial state distribution is $\rho(u) = 0.75$ and $\rho(w) = 0.25$. Compute the probability $w_{d_2'}(a|u)$ under this initial state distribution.

Chapter 6

Infinite Horizon Models: Expected Total Reward

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

Starting a long way off the true point, and proceeding by loops and zigzags, we now and then arrive just where we ought to be[1].

From the novel *Middlemarch*, by George Eliot, British novelist, 1819-1880.

6.1 Introduction

In the spirit of the above quote from *Middlemarch*, this chapter focuses on models in which there is a destination that the decision-maker seeks to reach as quickly as possible, or on the other hand avoid as long as possible. These models evaluate policies using their infinite horizon expected total reward defined below. Throughout this chapter such models will be referred to as *expected total reward models*.

[1] Eliot [1874].

Definition 6.1. For each $s \in S$ and $\pi \in \Pi^{\mathrm{HR}}$, define the *infinite horizon expected total reward*[a] by

$$v^{\pi}(s) := \lim_{N\to\infty} E^{\pi}\left[\sum_{n=1}^{N} r(X_n, Y_n) \,\middle|\, X_1 = s\right]. \tag{6.1}$$

[a]This is also referred to as the policy value function.

Upon examining this expression, it is natural to ask "How can such expectations be finite?". Finiteness is achieved by restricting models to those containing zero-reward absorbing states that can be reached in a finite number of transitions by some or all policies. Additional conditions, either explicitly or implicitly stated, ensure that non-terminating policies cannot be optimal.

Using (6.1) to evaluate policies raises a number of technical issues, namely, this quantity may be unbounded or not even exist for some policies π. While many of the challenges arise in countable state[2] models, finite state and action models still require subtle analyses.

Initially, as a result of their intrinsic complexity, expected total reward models were formulated and analyzed by mathematicians. But such models also provided a unified framework to study colourful and significant applications including:

1. optimal stopping,
2. Gridworld models,
3. sequential statistical hypothesis testing,
4. finding the shortest path in a network with random transitions, and
5. gambling strategies.

The recent trend towards using Markov decision process and reinforcement learning methods in *task-oriented episodic* applications such as game playing, gaming, autonomous vehicle guidance, and robotic control, has rekindled interest in such models, underscoring the need for this chapter.

6.1.1 Motivating examples

To appreciate some of the technical complexities faced when using the expected total reward criterion, consider three examples.

First, consider the two-state model in Section 2.5. In this model there are two stationary policies (those choosing action $a_{2,2}$ in s_2) with expected total reward $+\infty$ and two stationary policies (those choosing action $a_{2,1}$ in s_2) with expected total reward

[2]See Chapter 7 in Puterman [1994].

reward $-\infty$. Hence, the expected total reward criterion would not be effective for choosing among these policies regardless of whether the goal is to maximize rewards or minimize costs.

The next example illustrates further pitfalls that arise when using the expected total reward criterion, and how seemingly insignificant changes to the reward function can have significant changes on the optimal value.

Example 6.1. Let $S = \{s_1, s_2\}$, $A_{s_i} = \{a_{i,1}, a_{i,2}\}$ for $i = 1, 2$, $r(s_1, a_{1,1}) = -1$, $r(s_1, a_{1,2}) = 1$, $r(s_2, a_{2,1}) = 0$, $r(s_2, a_{2,2}) = -1$ and $p(s_1|s_1, a_{1,1}) = p(s_2|s_1, a_{1,2}) = p(s_2|s_2, a_{2,1}) = p(s_1|s_2, a_{2,2}) = 1$. See Figure 6.1.

In this model there are four stationary policies: $d_i^\infty, i = 1, 2, 3, 4$ where $d_1 = (a_{1,1}, a_{2,1}), d_2 = (a_{1,1}, a_{2,2}), d_3 = (a_{1,2}, a_{2,1})$ and $d_4 = (a_{1,2}, a_{2,2})$. For the first three

$$\mathbf{v}^{d_1^\infty} = \begin{bmatrix} -\infty \\ 0 \end{bmatrix}, \quad \mathbf{v}^{d_2^\infty} = \begin{bmatrix} -\infty \\ -\infty \end{bmatrix}, \quad \mathbf{v}^{d_3^\infty} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

For d_4^∞, when $X_1 = s_1$,[a]

$$\liminf_{N\to\infty} \sum_{n=1}^{N} r(X_n, d_4(X_n)) = 0 \quad \text{and} \quad \limsup_{N\to\infty} \sum_{n=1}^{N} r(X_n, d_4(X_n)) = 1,$$

and when $X_1 = s_2$,

$$\liminf_{N\to\infty} \sum_{n=1}^{N} r(X_n, d_4(X_n)) = -1 \quad \text{and} \quad \limsup_{N\to\infty} \sum_{n=1}^{N} r(X_n, d_4(X_n)) = 0.$$

If the goal is to maximize expected total reward the decision-maker would prefer d_3^∞. However this example shows that there are stationary policies for which the expected total reward is $-\infty$ and stationary policies for which the limit used to define the expected total reward of a policy does not exist.

Suppose now -1 is subtracted from all rewards so that $r(s, a) \leq 0$ for all states and actions. In the resulting model, all policies have expected total reward $-\infty$. Thus something is clearly special about the existence of policies with absorbing states with zero reward.

[a] Note that since all transitions are deterministic, expectations are not needed for a deterministic policy.

Finally, consider the Gridworld navigation model of Section 3.2. This is an example of what is often referred to as an *episodic* model, one in which the decision process terminates at the end of an episode and the episode length varies among policies. For another concrete example, in the game of golf, an elite player may usually require 4 shots on a par 4 hole but on some occasions may require many more.

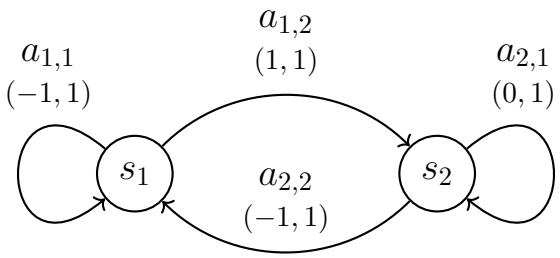


Figure 6.1: Symbolic representation of the model in Example 6.1. The labels (r, p) refer, respectively, to the reward and transition probabilities associated with using the specified action in the given state.

Example 6.2. Consider the Gridworld model of Section 3.2 in which the objective is to deliver coffee from the coffee room (cell 13) to the professor's office (cell 1). In this model, the decision process terminates when the robot successfully delivers coffee, or falls down the stairs (cell 7). The goal in this application is to choose a series of actions for the robot so as to maximize the expected total reward consisting of a positive reward for delivering the coffee, a negative reward (cost) if the robot falls down the stairs and a negative reward (cost) per transition.

In the presence of randomness in the robot's motion in all cells, under any policy, the decision process terminates at a random time so that a finite horizon model with a *fixed* horizon would not apply. Discounting makes little sense in this application since the time scale is so short and averaging is inappropriate because the process terminates when the task is completed so the long-run average reward of any policy would be zero.

On the other hand, when transitions are deterministic there exist polices that never terminate, so that their expected total reward equals $-\infty$. However, even in this case, there are also polices with finite total reward.

The examples above show that the expected total reward of (stationary) policies may be either finite, infinite, or non-existent.

6.2 Model classification

Model classification is based on either:

1. the chain structure of Markov chains generated by stationary policies, or
2. the sign of the rewards.

Historically, expected total reward model research focused on cases with either positive or negative rewards. It did so to ensure that the expected total reward of stationary

policies was well defined. Most of this research concerned establishing the existence of optimal policies in countable state models. A newer and distinct research stream focuses on finite state models with zero-reward absorbing states. Such models are more relevant for modern applications, possess some nice theoretical properties and will be the primary focus of this chapter.

6.2.1 Preliminaries

Throughout this chapter assume stationary rewards and transition probabilities, and without loss of generality, the rewards do not depend on the subsequent state[3].

For each $s \in S$ and $a \in A_s$, define

$$r_+(s,a) := \max\{r(s,a), 0\} \quad \text{and} \quad r_-(s,a) := \min\{r(s,a), 0\}. \tag{6.2}$$

and for each $\pi \in \Pi^{\text{HR}}$ define

$$v_+^\pi(s) := E^\pi \left[\sum_{n=1}^{\infty} r_+(X_n, Y_n) \middle| X_1 = s \right] \tag{6.3}$$

and

$$v_-^\pi(s) := E^\pi \left[\sum_{n=1}^{\infty} r_-(X_n, Y_n) \middle| X_1 = s \right]. \tag{6.4}$$

The limits implicit in defining $v_+^\pi(s)$ and $v_-^\pi(s)$ are always assumed to exist but may be infinite.

Whenever *at least one* of $v_+^\pi(s)$ and $v_-^\pi(s)$ is finite, then $v^\pi(s)$ can be written

$$v^\pi(s) = v_+^\pi(s) + v_-^\pi(s). \tag{6.5}$$

The following example shows that $v^\pi(s)$ may exist, yet need not equal $v_+^\pi(s) + v_-^\pi(s)$ when both quantities are infinite.

Example 6.3. Consider the policy π represented symbolically by the transitions in Figure 6.2. In it all transitions are deterministic with the exception of that from s_3 which goes to the right and left each with probability 1/2. Rewards are 0 except on the arcs from s_4 to s_5 and s_2 to s_1 where they are 2 and -1, respectively. Then its easy to see that $v_+^\pi(s_3) = +\infty$, $v_-^\pi(s_3) = -\infty$ but $v^\pi(s_3) = +\infty$ so that (6.5) does not hold.

Note that if there was another policy π' which had similar transitions as above but with the reward on the arc from s_4 to s_5 equal to 3, then $v^{\pi'}(s_3) = +\infty$ but clearly π' is preferable to π. Unfortunately, the expected total reward criterion

[3] If they do, replace $r(s,a,j)$ by $r(s,a) = \sum_{j \in S} r(s,a,j)p(j|s,a)$. Simulation models are best analyzed in terms of $r(s,a,j)$ since their purpose is to avoid computing this expectation.

does not distinguish these two policies. Either the discounted reward criterion (Chapter 5) or average reward criterion (Chapter 7) would be appropriate in such situations.

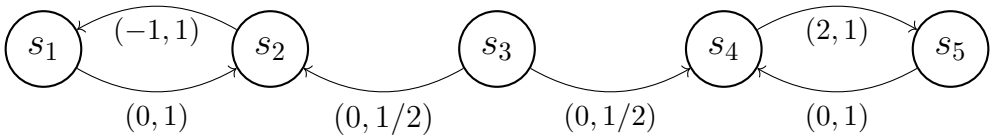


Figure 6.2: Symbolic representation of policy π analyzed in Example 6.3. The labels (r, p) refer, respectively, to the reward and transition probabilities associated with using the specified action in the given state.

To exclude such pathologies and simplify analyses the following assumption will be imposed throughout the remainder of this chapter and frequently be restated for emphasis:.

Assumption 6.1. For each $\pi \in \Pi^{\text{HR}}$ and $s \in S$, either:

1. $v_+^\pi(s)$ is finite,
2. $v_-^\pi(s)$ is finite, or
3. both $v_+^\pi(s)$ or $v_-^\pi(s)$ are finite.

The most significant consequence of this assumption is that when it holds, the limit in (6.1) exists and moreover is finite under condition 3. Furthermore, it excludes models such as those in Example 6.3.

The next four subsections describe the following expected total reward model classes:

- Transient
- Stochastic shortest path
- Positive
- Negative.

6.2.2 Transient models

First, distinguish the class of *transient*[4] models. Appendix B provides an overview of relevant Markov chain concepts and results.

Definition 6.2. A *transient model* is characterized by the existence of an absorbing state $\Delta \in S$ with $A_\Delta = \{\delta\}$ such that :

1. $r(\Delta, \delta) = 0$ and $p(\Delta|\Delta, \delta) = 1$;
2. for every stationary policy d^∞,

$$\lim_{n\to\infty} P^{d^\infty}[X_n = \Delta | X_1 = s] = 1 \tag{6.6}$$

for all $s \in S$.

The only assumptions on rewards in a transient model are that $r(\Delta, \delta) = 0$. Consequently, a transient model can have both positive and negative rewards. The condition that there is a single state Δ satisfying conditions 1 and 2 can be relaxed as follows:

1. The quantity Δ may represent a *set* of absorbing states with the property that for each $s' \in \Delta$, $A_{s'}$ contains a single action a' for which $r(s', a') = 0$ and $p(s'|s', a') = 1$.
2. The set of absorbing states may vary by stationary policy. This can be reduced to the above definition by adding a zero-reward transition from each of these absorbing states to a common absorbing state Δ.

The most important consequence of the transient model definition is that under every stationary policy the corresponding Markov chain is *unichain* with a *single* absorbing state Δ and a set of transient states $S \setminus \Delta$. This means that the transition probabilities and rewards corresponding to decision rule d can be written as

$$\mathbf{P}_d = \begin{bmatrix} \mathbf{Q}_d & \mathbf{R}_d \\ \mathbf{0} & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{r}_d = \begin{bmatrix} (\mathbf{r}_d)_T \\ 0 \end{bmatrix} \tag{6.7}$$

where $\mathbf{Q}_d$ is an $|S| - 1 \times |S| - 1$ matrix that represents the probability of transitions between transient states in $S \setminus \Delta$ and $\mathbf{R}_d$ represents the probability of transitions from $S \setminus \Delta$ to Δ. The last row of the matrix corresponds to transition from Δ. Since it

[4]The expression *transient model* originates with Veinott [1969a]. However, he defines this concept in terms of transition probability matrices of stationary policies and does not explicitly distinguish the set Δ of absorbing states. Derman [1970] referred to such models as *optimal first passage models*. Others have referred to them as *contracting models*.

is absorbing, states in $S \setminus \Delta$ cannot be reached from it. Instead the system remains in Δ in perpetuity, corresponding to the entry 1 in the lower right-hand corner of the matrix.

The reward vector can be partitioned into two parts, with $(\mathbf{r}_d)_T$ representing the reward in the transient states $S \setminus \Delta$ and a reward of 0 in state Δ. For convenience, let $(\mathbf{P}_d)_T := \mathbf{Q}_d$, the sub-matrix of $\mathbf{P}_d$ corresponding to the transient states.

Key properties of transient models

The following proposition accumulates key properties of $\mathbf{Q}_d$ in a transient model. Recall that the spectral radius of a finite square matrix $\mathbf{Q}$ denoted by $\sigma(\mathbf{Q})$ is its largest eigenvalue in absolute value.

Proposition 6.1. In a transient model for every $d \in D^{\text{MD}}$:

1. The *spectral radius* of $\mathbf{Q}_d$ satisfies
$$\sigma(\mathbf{Q}_d) < 1. \tag{6.8}$$
2. $(\mathbf{I} - \mathbf{Q}_d)^{-1}$ exists and satisfies
$$(\mathbf{I} - \mathbf{Q}_d)^{-1} = \sum_{n=0}^{\infty} \mathbf{Q}_d^n. \tag{6.9}$$
3. For $\mathbf{u} \geq \mathbf{0}$, $(\mathbf{I} - \mathbf{Q}_d)^{-1}\mathbf{u} \geq \mathbf{u}$.

Proof. The first result follows from Veinott [1969a] or p. 223 in Berman and Plemmons [1979].

The second part follows directly from the first part by Lemma B.2. Alternatively, in a transient model the system reaches state Δ with certainty under each stationary policy, so that[5] $\mathbf{Q}_d^n \to \mathbf{0}$. Consequently it follows from Lemma B.1 that $(\mathbf{I} - \mathbf{Q}_d)^{-1}$ exists.

To prove the third part, since $\mathbf{Q}_d \geq \mathbf{0}$, it follows from (6.9)
$$(\mathbf{I} - \mathbf{Q}_d)^{-1}\mathbf{u} = \mathbf{u} + \mathbf{Q}_d\mathbf{u} + \mathbf{Q}_d^2\mathbf{u} + \ldots \geq \mathbf{u}.$$
□

Consequently in a transient model, the value of a stationary policy can be expressed in closed form as:
$$\mathbf{v}^{d^\infty} = \sum_{n=1}^{\infty} \mathbf{P}_d^{n-1}\mathbf{r}_d = \sum_{n=1}^{\infty} \mathbf{Q}_d^{n-1}(\mathbf{r}_d)_T = (\mathbf{I} - \mathbf{Q}_d)^{-1}(\mathbf{r}_d)_T. \tag{6.10}$$

[5] Otherwise $S \setminus \Delta$ would be a closed class and not transient.

One might hypothesize that the condition that $\sigma(\mathbf{Q}_d) < 1$ for all $d \in D^{\text{MD}}$ could serve as a definition of a transient model, since this condition underpins many key results for transient models. The downside of doing this is that it admits models in which transition matrix row sums could exceed one, which challenges the probabilistic interpretation of a transition matrix.

Define the *absorption time* N_Δ as follows.

Definition 6.3. The random variable N_Δ defined by

$$N_\Delta := \inf\{n \geq 1 \,|\, X_n = \Delta\} \tag{6.11}$$

is referred to as an *absorption time* or *effective horizon*[a].

[a] Absorption times are often referred to as first passage times or stopping times in the probability literature. It is often referred to as the *effective horizon* in the reinforcement learning literature.

As a consequence of (6.6) or directly from (6.9) the following holds.

Proposition 6.2. In a transient model

$$E^{d^\infty}[N_\Delta \,|\, X_1 = s] < \infty \tag{6.12}$$

for all $s \in S \setminus \Delta$.

Applying this result gives the following:

Theorem 6.1. In a transient model, the expected total reward of every stationary policy exists and is finite.

Proof. Since

$$v^{d^\infty}(s) = E^{d^\infty}\left[\sum_{n=1}^{N_\Delta} r(X_n, d(X_n)) \middle| X_1 = s\right].$$

it follows from Proposition 6.2 that

$$-\infty < E^{d^\infty}[N_\Delta | X_1 = s] r_{\min} \leq v^{d^\infty}(s) \leq E^{d^\infty}[N_\Delta | X_1 = s] r_{\max} < \infty,$$

where

$$r_{\min} = \min_{a \in A_s, s \in S} r(s,a) \quad \text{and} \quad r_{\max} = \max_{a \in A_s, s \in S} r(s,a).$$

The finiteness of $r_{\min}$ and $r_{\max}$ are a consequence of the assumption of a finite state and action model. □

Note that Veinott [1969a] showed this result holds for all Markovian policies by establishing that when all stationary policies are transient, then all policies are transient.

As noted in Section 2.24, a discounted model may be regarded as a transient model by interpreting discounting as random termination at a geometrically distributed stopping time independent of the policy. Conversely, transient models inherit many of the properties of discounted reward models.

6.2.3 Stochastic shortest path models

Stochastic shortest path[6] (SSP) models generalize transient models by allowing the possibility that some stationary policies do not reach a zero-reward absorbing state with certainty. However they exclude the possibility that such a policy can be optimal by requiring that the value of any such policy be equal to $-\infty$ in some state.

Definition 6.4. A *stochastic shortest path model* is characterized by an absorbing state Δ with a single action $A_\Delta = \{\delta\}$ for which:

1. $r(\Delta, \delta) = 0$ and $p(\Delta|\Delta, \delta) = 1$,
2. there exists at least one stationary policy d^∞ for which
$$\lim_{N\to\infty} P^{d^\infty}[X_N = \Delta | X_1 = s] = 1$$
for all $s \in S$, and
3. if
$$\lim_{N\to\infty} P^{(d')^\infty}[X_N = \Delta | X_1 = s] < 1 \tag{6.13}$$
for some stationary policy d' and $s \in S$, then $v^{(d')^\infty}(s) = -\infty$.

A stationary policy is *proper*[7] if it satisfies

$$\lim_{N\to\infty} P^{d^\infty}[X_N = \Delta | X_1 = s] = 1.$$

If (6.13) holds, the policy is said to be *improper*. Thus, an SSP generalizes a transient model by allowing improper policies, but adds the condition that the expected total reward of an improper policy must exist and have value $-\infty$ for at least one state[8].

[6] The nomenclature stochastic shortest path model and the associated terminology originates with Bertsekas and Tsitsiklis [1991] in the context of cost minimization models. Since the models are formulated in terms of rewards, they may be thought of as stochastic longest path models. Nevertheless, they will continue to be referred to as stochastic shortest path models to be consistent with the literature.

[7] Following the terminology of Bertsekas and Tsitsiklis [1991].

[8] This excludes models with policies that can loop between two states and have rewards 1 and -1 respectively, as in Example 6.1 so that the limit defining $v^{(d')^\infty}$ does not exist.

Consequently, an improper policy cannot be optimal. Note that improper policies will contain closed classes of states in which there are negative rewards. On the other hand, an SSP model in which every stationary policy is proper is a transient model.

6.2.4 Positive models

A discussion of positive and negative models is included to relate results herein to the classical expected total reward literature. Moreover, derivations of results for these models provide additional insight into the structure of Markov decision process models.

Positive and negative models are based on the decomposition of $v^\pi(s)$ in (6.5). Positive models were first formulated as those in which $r(s,a) \geq 0$ for all $a \in A_s, s \in S$. The following definition is more general and isolates the critical components of the model.

Definition 6.5. A *positive model* satisfies:

1. $v_+^\pi(s) < \infty$ for all $s \in S$ and $\pi \in \Pi^{\text{HR}}$, and
2. for each $s \in S$, $r(s,a) \geq 0$ for some $a \in A_s$.

The first condition ensures that there are no policies with infinite expected total reward. Thus, in a positive model it is possible to distinguish among policies with non-negative expected total reward. The second condition ensures that there is at least one (stationary) policy with non-negative expected total reward. Note that in this model, some policies may have $v^\pi(s) = -\infty$ but it follows from (6.5) that the first condition in the definition of a positive model excludes the possibility that the limit defining the expected total reward does not exist.

6.2.5 Negative models

Negative models were first formulated as ones in which $r(s,a) \leq 0$ for all $a \in A_s, s \in S$. The following definition is more general.

Definition 6.6. A *negative model* satisfies:

1. $v_+^\pi(s) = 0$ for all $s \in S$ and $\pi \in \Pi^{\text{HR}}$, and
2. there exists a $\pi \in \Pi^{\text{HR}}$ for which $v_-^\pi(s) > -\infty$ for all $s \in S$.

The first condition ensures that $v^\pi(s) \leq 0$ for all policies and the second condition ensures there is at least one policy with a finite non-positive value. Observe also that in a negative model, the limit defining $v^\pi(s)$ exists for all $\pi \in \Pi^{\text{HR}}$. Note that a subclass

of models with non-negative rewards in some states can be transformed to a negative model. This point is expanded on below in the context of the Gridworld navigation model in Example 6.4.

6.2.6 Comparison of model classes

At first glance it might appear that by changing signs of rewards, positive and negative models are equivalent. This is not the case because with an objective of maximizing the expected total reward in positive models, the decision maker seeks a policy with its expected total reward as far above zero as possible while in a negative model the decision maker seeks a policy with its expected total reward as close to zero as possible. By regarding negative rewards as costs, maximizing the expected total reward in a negative model is equivalent to minimizing the expected total cost.

Note that the classifications positive and negative model are *not* mutually exclusive as shown by the model in Figure 6.3, which satisfies both model definitions.

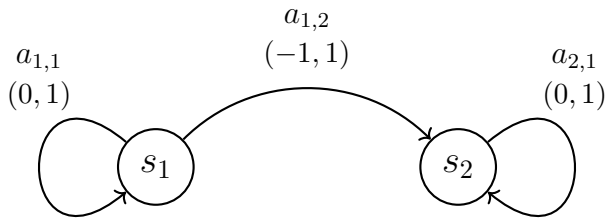


Figure 6.3: Symbolic representation of a model that is both negative and positive. The labels (r, p) refer, respectively, to the reward and transition probabilities associated with using the specified action in the given state.

Transient and SSP models allow more general cost structures than positive and negative models, however there are two special cases to consider:

1. When all rewards are less than or equal to zero, transient and SSP models are negative models.

2. An SSP model in which at least one stationary policy has a non-negative reward is a positive model.

Examples

The following two examples provide concrete illustrations of the above model classes.

Example 6.4. Recall that in the Gridworld model, there is a reward of $B > 0$ for successfully delivering the coffee cup, a cost of $X > 0$ (reward of $-X$) for falling down the stairs and a cost of c (reward of $-c$) for each transition. Since the (expected) positive reward for delivering the coffee can only be obtained from cells 2 and 4 (see Figure 3.3 in Section 3.2), the condition that for each $s \in S$ there exists an $a \in A_s$ for which $r(s,a) \geq 0$ does not hold. Moreover the condition that $v_+^\pi(s) = 0$ in the negative model definition does not hold. Hence this model is neither positive nor negative.

However, this model can be transformed into a negative model by subtracting B from all rewards (or just from the reward associated with delivering the coffee and adding it to the penalty for falling down the stairs). On the other hand, two special cases are noteworthy: 1) when $B = 0$, this is negative model; 2) when $X = 0$ and there is no transition cost, the model becomes a positive model. If $B = 1$ and all other costs are 0, maximizing the expected total reward is equivalent to maximizing the probability that the robot succeeds at its task of delivering the coffee. So, positive models include those that maximize the probability of reaching a particular set of states.

With regards to transient and SSP models, there are two zero-reward absorbing states, so the model does not satisfy the first condition in the definition of a transient model. However, by adding a zero-reward absorbing state Δ and deterministic zero-reward transitions from cells 1 and 7 to Δ, the first transient model condition is satisfied. When transitions are random, all stationary policies are proper so that the model is transient. When transitions are deterministic, infinite loops are possible so that there are improper stationary policies. However, since each transition has a reward of $-c$, when $c \neq 0$ such improper policies have expected total reward equal to $-\infty$ so that it is an SSP model. Thus, in this example, the SSP classification allows deterministic actions while the transient model classification does not.

Example 6.5. Classifying the model in Example 6.1 is more problematic. Because $v_+^{d_4^\infty}(s) = +\infty$ and $v_-^{d_4^\infty}(s) = -\infty$ the model is neither positive nor negative. Subtracting 1 from all rewards satisfies condition 1 for a negative model, but after this transformation, $v_-^\pi(s) = -\infty$ for each stationary policy so condition 2 does not hold. Moreover it is not an SSP model because the improper policy d_3^∞ does not have an expected total reward equal to $-\infty$.

6.3 Value functions, optimal policies and the Bellman equation

Suppose that $v^\pi(s)$ exists for all $\pi \in \Pi^{\text{HR}}$ and $s \in S$. As noted above, a sufficient condition for this to hold is given by Assumption 6.1, namely that either $v^\pi_+(s)$, $v^\pi_-(s)$ or both are finite. This assumption ensures that $v^\pi(s)$ is well-defined, although it may be infinite. It is important to emphasize that:

Assumption 6.1 is satisfied in transient, SSP, positive, and negative models.

More specifically:

1. In transient models $v^\pi(s)$ is finite for all policies.
2. In SSP, positive and negative models, $v^\pi(s)$ exists but may equal $-\infty$ for some policies. Other assumptions ensure that such a policy cannot be optimal.

Given a model for which this assumption is satisfied, define its optimal value function $v^*(s)$ by

$$v^*(s) := \sup_{\pi \in \Pi^{\text{HR}}} v^\pi(s). \tag{6.14}$$

for all $s \in S$. Equivalently, $v^*(s)$ is the "smallest" function that satisfies $v(s) \geq v^\pi(s)$ for all $\pi \in \Pi^{\text{HR}}$ and $s \in S$.

What this condition means is that given $\epsilon > 0$, for each $s \in S$ there exists a $\pi^* \in \Pi^{\text{HR}}$ (which can vary with s) for which

$$v^{\pi^*}(s) \geq v^*(s) - \epsilon.$$

Consequently it follows that:

Theorem 6.2. The optimal value function $v^*(s)$ is finite in transient, SSP, positive and negative models.

The following proposition establishes that for each state, the supremum over history-dependent randomized policies equals the supremum over Markovian randomized policies. This was proved in Chapter 5 using the key Lemma 5.9, which showed that for any history-dependent randomized policy, for each state there exists a Markovian randomized policy with the same transition probabilities. The same result applies here.

Proposition 6.3. For each $s \in S$,

$$v^*(s) = \sup_{\pi \in \Pi^{\mathrm{MR}}} v^\pi(s). \tag{6.15}$$

Definition 6.7. A policy π^* is *optimal* if

$$v^{\pi^*}(s) \geq v^\pi(s) \tag{6.16}$$

for all $\pi \in \Pi^{\mathrm{HR}}$ and $s \in S$.

This definition along with the previous proposition allows the decision maker to restrict the search for optimal policies to the class of Markovian randomized policies.

A technical aside*

When the limit defining $v^\pi(s)$ does not exist for some $\pi \in \Pi^{\mathrm{HR}}$ the above definitions can be modified as follows. Define $v^*(s)$ to be the smallest function $v(s)$ that satisfies

$$v(s) \geq \limsup_{N\to\infty} E^\pi \left[\sum_{n=1}^{N} r(X_n, Y_n) \middle| X_1 = s \right] \tag{6.17}$$

for all $\pi \in \Pi^{\mathrm{HR}}$. Using this definition, it follows that $v^*(s)$ is well defined in Example 6.1.

Moreover, policy $\pi^* \in \Pi^{\mathrm{HR}}$ is *optimal* if

$$\liminf_{N\to\infty} E^{\pi^*} \left[\sum_{n=1}^{N} r(X_n, Y_n) \middle| X_1 = s \right] \geq \limsup_{N\to\infty} E^\pi \left[\sum_{n=1}^{N} r(X_n, Y_n) \middle| X_1 = s \right] \tag{6.18}$$

for all $\pi \in \Pi^{\mathrm{HR}}$. This definition establishes the optimality of d_3^∞ in Example 6.1.

6.3.1 The Bellman equation in an expected total reward model

This section investigates properties of the Bellman equation for expected total reward models. In component notation, the Bellman equation can be expressed as:

$$v(s) = \max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a) v(j) \right\} \quad \text{for all } s \in S. \tag{6.19}$$

In vector notation it is given by:

$$\mathbf{v} = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_d + \mathbf{P}_d \mathbf{v}\}. \tag{6.20}$$

Observe that the Bellman equation in expected total reward models is identical to that in a discounted model with $\lambda = 1$.

In the special case that D^{MD} consists of a single decision rule d, (6.19) reduces to the (deterministic stationary) policy evaluation equation

$$v(s) = r(s, d(s)) + \sum_{j \in S} p(j|s, d(s))v(j). \tag{6.21}$$

In vector notation (6.21) reduces to the policy evaluation equation

$$\mathbf{v} = \mathbf{r}_d + \mathbf{P}_d \mathbf{v}. \tag{6.22}$$

In greater generality[9] this expression becomes:

$$v(s) = E^d \left[r(X, Y, X') + v(X') \middle| X = s \right]. \tag{6.23}$$

Recalling that V denotes the set of real valued functions on S, define the *Bellman operator*, $L : V \to V$ and the *policy evaluation* operator, $L_d : V \to V$ by

$$L\mathbf{v} := \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_d + \mathbf{P}_d \mathbf{v}\} \quad \text{and} \quad L_d \mathbf{v} := \mathbf{r}_d + \mathbf{P}_d \mathbf{v}. \tag{6.24}$$

Consequently, the Bellman equation and policy evaluation equations can be written in vector notation as:

$$\mathbf{v} = L\mathbf{v} \quad \text{and} \quad \mathbf{v} = L_d \mathbf{v}. \tag{6.25}$$

Noting that Lemma 5.4 holds when $\lambda = 1$ enables restricting attention to Markovian deterministic decision rules in the definition of operator L.

6.3.2 The transient Bellman equation

In transient models, the Bellman equation is only of interest on the set of transient states $S \setminus \Delta$, since the definition of a transient model implies $v^{d^\infty}(\Delta) = 0$ for any $d \in D^{\mathrm{MD}}$. Let $\mathbf{v}_T$ denote the vector of components of $\mathbf{v}$ restricted to transient states

[9] Allowing rewards to depend on the subsequent state and Markovian randomized decision rules. Such a representation will be useful in Chapters 10 and 11.

and let V_T denote the set of bounded functions on $S \setminus \Delta$. Then, using the matrix partitioning in (6.7), the Bellman equation can be expressed as

$$\begin{bmatrix} \mathbf{v}_T \\ v(\Delta) \end{bmatrix} = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}} \left\{ \begin{bmatrix} (\mathbf{r}_d)_T \\ 0 \end{bmatrix} + \begin{bmatrix} \mathbf{Q}_d & \mathbf{R}_d \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{v}_T \\ v(\Delta) \end{bmatrix} \right\}. \tag{6.26}$$

Although (6.26) does not explicitly determine $v(\Delta)$, it is obvious in such models that $v(\Delta) = 0$. Substituting $v(\Delta) = 0$ into the above equation gives:

$$\mathbf{v}_T = \underset{d \in D^{MD}}{\text{c-max}} \{(\mathbf{r}_d)_T + \mathbf{Q}_d \mathbf{v}_T\}. \tag{6.27}$$

Refer to (6.27) as the *transient Bellman equation* and define the *transient Bellman operator* $L_T : V_T \to V_T$ by

$$L_T \mathbf{v} := \underset{d \in D^{\mathrm{MD}}}{\text{c-max}} \{(\mathbf{r}_d)_T + \mathbf{Q}_d \mathbf{v}_T\}. \tag{6.28}$$

Then the transient Bellman equation can be represented in operator form by

$$\mathbf{v}_T = L_T \mathbf{v}_T. \tag{6.29}$$

By replacing D^{MD} with a single decision rule d, the policy evaluation equation in a transient model, $(L_d)_T$, reduces to

$$\mathbf{v}_T = (\mathbf{r}_d)_T + \mathbf{Q}_d \mathbf{v}_T = (L_d)_T \mathbf{v}_T. \tag{6.30}$$

It will be established below that as a consequence of the observation that at least one row sum of $\mathbf{Q}_d$ is strictly less than 1 in a transient model, the transient Bellman operator is a contraction mapping so that it can be analyzed using a similar approach as for discounted models.

6.3.3 Solutions of the Bellman equation in expected total reward models

This section establishes properties of the Bellman operator that apply for any model class. It will show that:

1. the operator L is monotone,
2. the optimal value function satisfies the Bellman equation, and
3. the solution of the Bellman equation is not unique.

In contrast to the discounted model for which the Bellman operator was a contraction mapping because $\lambda < 1$, without further assumptions, the operator L is not a contraction mapping in expected total reward models. Therefore contraction mapping properties cannot be used to establish existence of solutions to the Bellman equation in all expected total reward models.

Proposition 6.4. Let $\mathbf{u}$ and $\mathbf{v}$ be elements of V. Then

1. For any scalar c, $L(\mathbf{v} + c\mathbf{e}) = L\mathbf{v} + c\mathbf{e}$.
2. If $\mathbf{u} \geq \mathbf{v}$, $L\mathbf{u} \geq L\mathbf{v}$.

Proof. To prove the first part,

$$L(\mathbf{v} + c\mathbf{e}) = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_d + \mathbf{P}_d(\mathbf{v} + c\mathbf{e})\} = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_d + \mathbf{P}_d\mathbf{v} + c\mathbf{e}\} = L\mathbf{v} + c\mathbf{e}.$$

The second part follows by noting that there exists a $d \in D^{\mathrm{MD}}$ for which

$$L\mathbf{v} = \mathbf{r}_d + \mathbf{P}_d\mathbf{v} \leq \mathbf{r}_d + \mathbf{P}_d\mathbf{u} \leq L\mathbf{u}.$$

□

The following result establishes that $v^*(s)$ is a solution of the Bellman equation. Its subtle proof also applies when $\lambda < 1$, however in that case a more general and obvious approach was used.

Theorem 6.3. Suppose $v^*(s)$ is finite for all $s \in S$, then $\mathbf{v}^*$ is a solution of the Bellman equation.

Proof. First show that $\mathbf{v}^* \geq L\mathbf{v}^*$. From the definition of supremum, given $\epsilon > 0$ for each $s \in S$, there exists a $\pi_s^* \in \Pi^{\mathrm{HR}}$ for which

$$v^{\pi_s^*}(s) \geq v^*(s) - \epsilon. \tag{6.31}$$

Since for all $a \in A_s$

$$v^*(s) \geq r(s,a) + \sum_{j \in S} p(j|s,a) v^{\pi_j^*}(j)$$

it follows from the monotonicity of L (part 2 of Proposition 6.4) that

$$v^*(s) \geq \max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a) v^{\pi_j^*}(j) \right\} \geq \max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a) v^*(j) \right\} - \epsilon.$$

Since the result holds for every $\epsilon > 0$, and s was arbitrary, $\mathbf{v}^* \geq L\mathbf{v}^*$.

Now show that $\mathbf{v}^* \leq L\mathbf{v}^*$. Write π_s^* in (6.31), as $\pi_s^* = (a_s, \pi_s')$. That is, in state s the policy chooses action a_s at the first decision epoch and then uses the history-dependent randomized policy π_s' thereafter. It follows that

$$v^*(s) - \epsilon \leq v^{\pi_s^*}(s) = r(s, a_s) + \sum_{j \in S} p(j|s, a_s) v^{\pi_j'}(j) \leq \max_{a \in A_S} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a) v^*(j) \right\}.$$

Since ϵ was arbitrary, it follows that $\mathbf{v}^* \leq L\mathbf{v}^*$.

Thus, $\mathbf{v}^* = L\mathbf{v}^*$ and $\mathbf{v}^*$ is a solution of the Bellman equation. □

While $\mathbf{v}^*$ is a solution of the Bellman equation, as a consequence of part 1 of Proposition 6.4, the solution is not unique. Note that if $\mathbf{v}^*$ is infinite, it could also be determined by solving the Bellman equation[10].

The following is an immediate corollary of Theorem 6.3.

Corollary 6.1. For every $d \in D^{\mathrm{MR}}$, $\mathbf{v}^{d^\infty} = L_d \mathbf{v}^{d^\infty}$.

6.3.4 Existence of optimal stationary policies in expected total reward models*

This section shows that under mild assumptions, in a finite state and action Markov decision process, there exists a stationary policy that maximizes the expected total reward. Its proof is rather technical and uses the result (Theorem 5.9) that in a discounted model, for each non-negative $\lambda < 1$ there exists an optimal stationary policy.

Analysis relies on the following technical result, the proof of which is a direct application of Abel's Theorem[11], which is valid even if $v^\pi(s)$ is infinite as would be the case in a one-state one-action model with $r(s) = 1$ and $p(s|s) = 1$.

Lemma 6.1. Suppose for some $\pi \in \Pi^{\mathrm{HR}}$ and $s \in S$

$$\lim_{N \to \infty} E_s^\pi \left[\sum_{n=0}^{N} r(X_n, Y_n) \,\middle|\, X_1 = s \right] = v^\pi(s) \tag{6.32}$$

exists. Then

$$\lim_{\lambda \uparrow 1} v_\lambda^\pi(s) = v^\pi(s). \tag{6.33}$$

[10] Consider a one-state Markov chain resulting in the Bellman equation $v(1) = 1 + v(1)$. This equation has no finite solution, but $v(1) = \infty$ can be regarded to be its solution.

[11] Abel's Theorem (Theorem 8.2, Rudin [1964]) states that if $\sum_{n=0}^\infty x_n$ converges, and $f(\lambda) = \sum_{n=0}^\infty \lambda^n x_n$ converges for $|\lambda| < 1$, then $\lim_{\lambda \uparrow 1} f(\lambda) = \sum_{n=0}^\infty x_n$.

The following example shows that the assumption the limit in (6.32) exists cannot be easily relaxed.

Example 6.6. Consider the stationary policy d_4^∞ in Example 6.1. Letting $\lambda < 1$, gives

$$v_\lambda^{d_4^\infty}(s_1) = 1 - \lambda + \lambda^2 - \lambda^3 + ... = \frac{1}{1+\lambda}.$$

Hence $\lim_{\lambda\uparrow 1} v_\lambda^{d_4^\infty}(s_1) = 1/2$, but $v_\lambda^{d_4^\infty}(s_1)$ does not exist as shown previously in Example 6.1. Moreover

$$\liminf_{N\to\infty} \sum_{n=1}^{N} r(X_n, d_4(X_n)) = 0 < \lim_{\lambda\uparrow 1} v_\lambda^{d_4^\infty}(s_1) < \limsup_{N\to\infty} \sum_{n=1}^{N} r(X_n, d_4(X_n)) = 1,$$

so that the limit of $v_\lambda^{d_4^\infty}(s_1)$ provides no insight about the lim inf or lim sup of the partial sums of (expected) rewards.

Theorem 6.4. Suppose Assumption 6.1 holds. Then there exists an optimal stationary deterministic policy.

Proof. Let $\lambda_n, n = 1, 2, \ldots$ denote a sequence that converges monotonically to 1 from below. From Theorem 5.9 there exists an optimal deterministic stationary policy for all non-negative $\lambda < 1$ in a discounted model. Thus, for each λ_n, there exists an optimal deterministic stationary policy. Since there are only finitely many deterministic stationary policies, there must exist a deterministic stationary policy $(d^*)^\infty$ and a subsequence $\lambda_{n_k}, k = 1, 2, \ldots$ for which $(d^*)^\infty$ is optimal.

Therefore for all $\pi \in \Pi^{\text{HR}}$, $s \in S$, and $k = 1, 2, \ldots$,

$$v_{\lambda_{n_k}}^{\pi}(s) \le v_{\lambda_{n_k}}^{(d^*)^\infty}(s).$$

Hence from Assumption 6.1 and Lemma 6.1, both limits below exist so that

$$v^{\pi}(s) = \lim_{k\to\infty} v_{\lambda_{n_k}}^{\pi}(s) \le \lim_{k\to\infty} v_{\lambda_{n_k}}^{(d^*)^\infty}(s) = v^{(d^*)^\infty}(s). \tag{6.34}$$

Thus $(d^*)^\infty$ is an optimal policy. □

Since Assumption 6.1 holds for all four model classes defined above, the following important result follows.

Corollary 6.2. There exists an optimal deterministic stationary policy in transient, SSP, positive, and negative models.

6.3.5 Identification of optimal policies

This section describes how to identify an optimal policy. One would suspect that using the same approach as in a discounted model, namely that a stationary policy derived from any $\mathbf{v}^*$-greedy policy is optimal. The following simple example shows that this is not the case.

Example 6.7. Let $S = \{s_1, s_2\}, A_{s_1} = \{a_{1,1}, a_{1,2}\}, A_{s_2} = \{a_{2,1}\}, r(s_1, a_{1,1}) = 0, r(s_1, a_{1,2}) = 1, r(s_2, a_{2,1}) = 0$ and $p(s_1|s_1, a_{1,1}) = p(s_2|s_1, a_{1,2}) = p(s_2|s_2, a_{2,1}) = 1$.

This is an example of a positive model. If it is modified by adding a zero-reward absorbing state Δ and changing transition probabilities for actions $a_{1,1}$ and $a_{2,1}$ to $p(\Delta|s_1, a_{1,1}) = p(\Delta|s_2, a_{2,1}) = 1$, it is a transient model.

The Bellman equation for this model is given by:

$$v(s_1) = \max\{v(s_1), 1 + v(s_2)\} \quad \text{and} \quad v(s_2) = v(s_2).$$

Since $v^*(s_1) = 1$ and $v^*(s_2) = 0$, it follows that both $d_1 = (a_{1,1}, a_{2,1})$ and $d_2 = (a_{1,2}, a_{2,1})$ are in $\arg\,\text{c-max}_{d \in D^{\text{MD}}}\{\mathbf{r}_d + \mathbf{P}_d\mathbf{v}^*\}$ but only d_2^∞ is optimal.

The following general result addresses this complication by providing a condition under which stationary policies derived from $\mathbf{v}^*$-greedy policies are optimal.

Theorem 6.5. Suppose Assumption 6.1 holds and d^* satisfies

$$d^* \in \arg\,\text{c-max}_{d \in D^{\text{MD}}}\{\mathbf{r}_d + \mathbf{P}_d\mathbf{v}^*\}. \tag{6.35}$$

Then $\mathbf{v}^{(d^*)^\infty} = \mathbf{v}^*$ if

$$\limsup_{N\to\infty} \mathbf{P}_{d^*}^N \mathbf{v}^* \leq 0. \tag{6.36}$$

Proof. Iteratively applying Theorem 6.3 and (6.35), yields

$$\mathbf{v}^* = \mathbf{r}_{d^*} + \mathbf{P}_{d^*}\mathbf{v}^* = \mathbf{r}_{d^*} + \mathbf{P}_{d^*}\mathbf{r}_{d^*} + \mathbf{P}_{d^*}^2\mathbf{v}^* = \ldots = \sum_{n=1}^{N} \mathbf{P}_{d^*}^{n-1}\mathbf{r}_{d^*} + \mathbf{P}_{d^*}^N\mathbf{v}^*.$$

Thus

$$\mathbf{v}^* \leq \limsup_{N\to\infty} \left(\sum_{n=1}^{N} \mathbf{P}_{d^*}^{n-1}\mathbf{r}_{d^*} + \mathbf{P}_{d^*}^N\mathbf{v}^* \right) \leq \limsup_{N\to\infty} \sum_{n=1}^{N} \mathbf{P}_{d^*}^{n-1}\mathbf{r}_{d^*} + \limsup_{N\to\infty} \mathbf{P}_{d^*}^N\mathbf{v}^* \tag{6.37}$$

As a consequence of Assumption 6.1, $\lim_{N\to\infty}\sum_{n=1}^{N}\mathbf{P}_{d^*}^{n-1}\mathbf{r}_{d^*}$ exists[12] so (6.36) gives

$$\mathbf{v}^* \leq \sum_{n=1}^{\infty}\mathbf{P}_{d^*}^{n-1}\mathbf{r}_{d^*} = \mathbf{v}^{(d^*)^\infty} \leq \mathbf{v}^*,$$

From which the result follows. □

Observe that the form of the rewards and/or the structure of transition probabilities corresponding to d^* determines whether (6.36) holds. The following corollary summarizes these results.

Corollary 6.3. Suppose d^* satisfies (6.35). Then $(d^*)^\infty$ is optimal in:

1. a transient model,
2. an SSP model in which d^* is proper,
3. a positive model in which (6.36) holds,
4. a negative model[a].

[a]Equation (6.36) is automatically satisfied in a negative model since $\mathbf{v}^* \leq \mathbf{0}$.

Returning to Example 6.7, since the model can be classified as a positive model, it requires the addition of condition (6.36) to establish the optimality of d_2^∞ since $\mathbf{P}_{d_1}^N\mathbf{v}^*(s_1) = 1$ for all N. On the other hand, under the modification used to obtain a transient model described in the example, only d_2^∞ attains the maximum in the Bellman equation.

The following example explores the implications of Theorem 6.5 in the context of Example 6.1.

Example 6.8. Recall that the model in Example 6.1 fell into none of the model classes described above. Its Bellman equation may be written as:

$$v(s_1) = \max\{-1 + v(s_1), 1 + v(s_2)\} \quad \text{and} \quad v(s_2) = \max\{v(s_2), -1 + v(s_1)\}.$$

Clearly $v^*(s_1) = 1$ and $v^*(s_2) = 0$ (taking into consideration that the limit defining $v^{d_4^\infty}(s)$ does not exist). Observe that in state s_2 both actions achieve the maximum on the right hand side of the Bellman equation, but only d_3, which satisfies (6.36) is optimal. Thus, this condition can be used to identify optimal policies even in the case when the model does not fall into any of the above

[12]Recall that if the limit of a sequence exists, its lim inf and lim sup are equal to each other and to the limit.

classes or even when Assumption 6.1 doesn't hold.

6.4 State-action value functions

State-action value functions will not play a significant role in this chapter but will be fundamental for analysing episodic models using simulation in Chapters 10 and 11.

For an expected total reward model, define the state-action value function corresponding to π by

$$q^{\pi}(s,a) := r(s,a) + \sum_{j \in S} p(j|s,a) v^{\pi}(j) \tag{6.38}$$

for any $\pi \in \Pi^{\mathrm{HR}}$ and define the optimal state-action value function by

$$q^{*}(s,a) := r(s,a) + \sum_{j|in S} p(j|s,a) v^{*}(j). \tag{6.39}$$

For a deterministic stationary policy d^{∞}, $q^{d^{\infty}}(s,a)$ is a solution of

$$q(s,a) = r(s,d(s)) + \sum_{j \in S} p(j|s,d(s)) q(j,d(j)) \tag{6.40}$$

and for a randomized stationary policy d^{∞} it is a solution of

$$q(s,a) = \sum_{a \in A_s} w_d(a|s) \left(r(s,a) + \sum_{j \in S} p(j|s,a) q(j,a) \right). \tag{6.41}$$

The optimal state-action value function satisfies the Bellman equation:

$$q(s,a) = r(s,a) + \sum_{j \in S} p(j|s,a) \left(\max_{a' \in A_s} q(j,a') \right). \tag{6.42}$$

6.5 Value iteration

This chapter now turns to computation in expected total reward models. It begins with an analysis of value iteration, which can be expressed in vector notation as follows.

Algorithm 6.1. Value iteration for an expected total reward model

1. **Initialize:** Specify $\mathbf{v}'$, $\epsilon > 0$ and $\sigma > \epsilon$.
2. **Iterate:** While $\sigma \geq \epsilon$:
   (a) $\mathbf{v} \leftarrow \mathbf{v}'$.

(b)

$$\mathbf{v}' \leftarrow \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_d + \mathbf{P}_d \mathbf{v}\} = L\mathbf{v}. \tag{6.43}$$

(c) $\sigma \leftarrow \|\mathbf{v}' - \mathbf{v}\|$.

3. **Terminate:** Return $\mathbf{v}^\epsilon = \mathbf{v}'$ and

$$d_\epsilon \in \underset{d \in D^{\mathrm{MD}}}{\arg\,\text{c-max}}\{\mathbf{r}_d + \mathbf{P}_d \mathbf{v}\}. \tag{6.44}$$

The above algorithm includes a norm-based stopping criterion. This is because we observed that when solving many examples numerically, the norm-based stopping criterion gave the same results as a span-based criterion (see Section 5.5.1)[13]. Note also that when applying value iteration to transient models, recursions use the transient Bellman operator L_T.

This section presents the analysis of the convergence of value iteration in transient and SSP models, and positive and negative models separately. Convergence in transient and SSP models relies on contraction properties of the transient Bellman operator while convergence in positive and negative models uses monotonicity. Note that in the latter case error bounds are not available.

6.5.1 Examples

Before analyzing convergence properties of value iteration, consider the following examples. The first two illustrate some pathological behavior in small deterministic examples. The third one illustrates more typical numerical results and the recursions underlying value iteration.

Example 6.9. (Example 6.7 revisited.) The value iteration recursion for this positive model may be written in component form as

$$v'(s_1) = \max\{v(s_1), 1 + v(s_2)\} \quad \text{and} \quad v'(s_2) = v(s_2).$$

Starting with $v(s_1) = v(s_2) = 0$, Algorithm 6.1 terminates in two iterations with the optimal value function $v'(s_1) = 1$ and $v'(s_2) = 0$ and $d_\epsilon = (a_{1,2}, a_{2,1})$. However, suppose the algorithm started from $v(s_1) = v(s_2) = 1$ then it would terminate with $v'(s_1) = 2$ and $v'(s_2) = 1$ and $d_\epsilon = (a_{1,2}, a_{2,1})$.

Thus, in this example the choice of the initial value affects the solution of the Bellman equation found by the algorithm but not the policy. This is a

[13] We observed that the iterates remain unchanged on states that lead to absorbing states. For example, if in state $s \in S$ there is a single action leading to zero-reward absorbing state Δ with reward c, then value iteration will eventually set $v(s) = c$ so that the quantity involved in defining the span, $\min_{s \in S}\{v'(s) - v(s)\} = 0$ resulting in $\mathrm{sp}(\mathbf{v}' - \mathbf{v}) = \|\mathbf{v}' - \mathbf{v}\|$.

consequence of the non-uniqueness of the solution of the Bellman equation in expected total reward models.

Note that the optimal value function of the discounted version of the problem, $v^*(s_1) = 1, v^*(s_2) = 0$, is in agreement with the first solution above.

Example 6.10. (Example 6.1 revisited.) Recall that the model in Example 6.1 did not fit into any of the specified model classes. The value iteration recursion (6.43) becomes

$$v'(s_1) = \max\{-1 + v(s_1), 1 + v(s_2)\} \quad \text{and} \quad v'(s_2) = \max\{v(s_2), -1 + v(s_1)\}.$$

Starting value iteration at $v(s_1) = v(s_2) = 0$, generates the value $v'(s_1) = 1$ and $v'(s_2) = 0$ at the next and all subsequent iterations. Moreover, (6.44), selects $d_\epsilon(s_1) = a_{1,2}$ and $d_\epsilon(s_2) \in \{a_{2,1}, a_{2,2}\}$.

Starting value iteration at $v(s_1) = v(s_2) = 1$ generates the value $v'(s_1) = 2$ and $v'(s_2) = 1$ at the next and all subsequent iterations, as well as the set of decision rules.

Note that in a discounted version of the problem, value iteration identifies the same optimal value function as when started at $v(s_1) = v(s_2) = 0$ but only the decision rule $d_\epsilon(s_1) = a_{1,2}$ and $d_\epsilon(s_2) = a_{2,1}$.

The next example applies value iteration to the Gridworld model.

Example 6.11. (Example 6.2 revisited.) This example will focus primarily on computational issues; the structure of policies will be explored in Section 6.10.1. Recall that B denotes the reward received when the robot successfully delivers the coffee, X denotes the penalty received if the robot falls down the stairs and c represents the cost of a single transition.

Consider three instances:

Instance 1: $B = 50$, $X = 200$ and $c = 1$,

Instance 2: $B = 0$, $X = 150$ and $c = 1$, and

Instance 3: $B = 1, X = 0$ and $c = 0$.

As noted previously, Instance 1 can be transformed into a negative model, Instance 2 is a negative model and Instance 3 is a positive model. On the other hand, when the probability the robot moves in the intended direction, p is strictly less than 1, the model is transient. When this probability equals 1, it is an SSP model.

The following relationships describe the value iteration recursion for selected states. Let $q = 1 - p$ and assume the only actions in each state are "left", "up" and "right". Note also that the recursions are easier to express when using $r(s, a, j)$ instead of $r(s, a)$ because reward and penalties are received after the transition occurs. It is left to the reader as an exercise to provide recursions for the remaining states.

$$
\begin{aligned}
v'(1) &= v(1) \\
v'(2) &= -c + \max\left\{ p\big(B + v(1)\big) + \frac{1}{2}q\big(v(3) + v(5)\big), pv(3) + \frac{1}{2}q\big(B + v(1) + v(5)\big) \right\} \\
v'(3) &= -c + pv(2) + qv(6), \\
v'(8) &= -c + \max\Big\{ p\big(-X + v(7)\big) + \frac{1}{3}q(v(5) + v(9) + v(11)), \\
&\qquad pv(5) + \frac{1}{3}q\big(-X + v(7) + v(9) + v(11)\big), \\
&\qquad pv(9) + \frac{1}{3}q\big(-X + v(5) + v(7) + v(11)\big) \Big\}.
\end{aligned}
$$

Observe that these recursions account for the consequences of choosing one of the available actions in a state. For example, in cell 2 there are two possible actions "left" and " right". Choosing either results in a transition in the intended direction with probability p, and transitions in the other direction and "down" with probability $q/2$. Other expressions are derived similarly.

Value iteration starts from $\mathbf{v} = \mathbf{0}$ and uses a stopping criterion $\|\mathbf{v}' - \mathbf{v}\| < 0.0001$. In all instances value iteration converges; the number of iterations to achieve the stopping criterion varies with p and instance as shown in Table 6.1. It shows that:

1. For each p the number iterations varied by instance with Instance 1 and 2 the most similar. Moreover, convergence was faster in Instance 3 for all values of p.

2. In each instance the number of iterations to achieve convergence was decreasing with p.

Inspection of calculations revealed that in Instance 1 the iterates of $v(13)$ were non-monotone[a] when $p \geq 0.6$, in Instance 2 the iterates of $v(13)$ were monotone non-increasing, and in Instance 3 the iterates of $v(13)$ were monotone non-decreasing for all values of p.

[a] With respect to iteration number.

p	Instance 1	Instance 2	Instance 3
0	480	481	305
0.25	199	201	101
0.50	114	124	81
0.75	51	44	38
0.95	21	19	17

Table 6.1: Number of iterations to achieve stopping criterion $\|\mathbf{v}' - \mathbf{v}\| < 0.0001$ in Gridworld model categorized by p and instance.

6.5.2 Transient models

Value iteration calculations in the Gridworld model in Example 6.11 showed that as p – the probability the robot moves in the intended direction – increased, the number of iterates required to achieve a specified level of precision decreased (Table 6.1). This makes sense since the larger the value of p, the more quickly the system under an optimal policy will reach an absorbing state, suggesting an underlying contraction behavior. The following analysis of the convergence of value iteration in transient models provides insight into why this is the case.

Recall that in a transient model all deterministic stationary policies have the same reward-free absorbing state Δ. Let $\mathbf{v}_T$ and $\mathbf{v}'_T$ denote the restriction of $\mathbf{v}$ and $\mathbf{v}'$ to the set of transient states, $S \setminus \Delta$. Then the value iteration recursion in a transient model may be written as:

$$\mathbf{v}'_T \leftarrow L_T \mathbf{v}_T = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}} \{(\mathbf{r}_d)_T + \mathbf{Q}_d \mathbf{v}_T\}, \tag{6.45}$$

so that its convergence depends on properties of $\mathbf{Q}_d$.

Solutions of the Bellman equation in a transient model

Theorem 6.3 showed that $\mathbf{v}^*$ was a solution of the Bellman equation in an expected total reward model. The following stronger result follows for transient models.

Theorem 6.6. In a transient model, the transient Bellman equation

$$L_T \mathbf{v} = \mathbf{v}$$

has a unique solution $\mathbf{v}^*_T \in V_T$.

Proof. The result is trivially true if $\mathbf{Q}_d = \mathbf{0}$ for all $d \in D^{\mathrm{MD}}$.

Assume now that $\mathbf{Q}_d$ has at least one positive row sum for each $d \in D^{\mathrm{MD}}$. As a consequence of Part 1 of Proposition 6.4 applied to a transient model, $\mathbf{v}^* + c\mathbf{e}$ is a solution of the transient Bellman equation for any scalar c. Hence it follows for its restriction to transient states, $\mathbf{v}_T^*$, that

$$\mathbf{v}_T^* + c\mathbf{e} = L_T(\mathbf{v}_T^* + c\mathbf{e}) \leq L_T\mathbf{v}_T^* + \underset{d\in D^{\mathrm{MD}}}{\text{c-max}}\, c\mathbf{Q}_d\mathbf{e} = \mathbf{v}_T^* + c\mathbf{Q}_{d'}\mathbf{e}$$

for some $d' \in D^{\mathrm{MD}}$. Hence $c \leq c\mathbf{Q}_{d'}\mathbf{e}(s)$ for all $s \in S \setminus \Delta$. That $c = 0$ follows from the above assumption regarding the positivity of at least one row sum of $\mathbf{Q}_{d'}$. □

Theorem 6.6 establishes the uniqueness of the solution of the Bellman equation on transient states $S \setminus \Delta$. Since the solution on all of S is not unique (Proposition 6.4), adding the condition that $v(\Delta) = 0$ specifies a unique solution as follows.

Corollary 6.4. In a transient model, the Bellman equation

$$L\mathbf{v} = \mathbf{v}$$

subject to $v(\Delta) = 0$ has a unique solution $\mathbf{v}^* \in V$.

Restricting attention to a single decision rule leads to the following result concerning policy evaluation.

Corollary 6.5. In a transient model:

1. The transient policy evaluation equation

$$\mathbf{v} = (\mathbf{r}_d)_T + \mathbf{Q}_d\mathbf{v} = (L_d)_T\mathbf{v} \tag{6.46}$$

has the unique solution $\mathbf{v}_T^{d^\infty} \in V_T$.

2. The policy evaluation equation

$$\mathbf{v} = \mathbf{r}_d + \mathbf{P}_d\mathbf{v} = (L_d)\mathbf{v} \tag{6.47}$$

subject to $v(\Delta) = 0$ has the unique solution $\mathbf{v}^{d^\infty} = (\mathbf{v}_T^{d^\infty}, 0) \in V$.

Convergence of value iteration in transient models

Proving convergence of value iteration in a transient model is based on establishing one of the following results:

1. L_T^N is a contraction mapping with respect to the sup-norm for some $N > 0$,
2. L_T is a contraction mapping with respect to the Euclidean norm,

3. there is an equivalent Markov decision process in which L_T is a contraction mapping with respect to the sup-norm, or

4. L_T is a contraction mapping with respect to a specific weighted sup-norm,

where the concepts of equivalent Markov decision process and weighted sup-norm are defined below.

Under each of these results, it follows from the Banach fixed-point Theorem (Theorem 5.5) that:

Theorem 6.7. In a transient model, value iteration converges geometrically to the unique fixed point of L_T.

The following sections establish this result in different ways. Recall that a sequence $\mathbf{v}^n$, $n = 0, 1, \ldots$ converges geometrically to $\mathbf{v}^*$ if there exists constants $K > 0$ and $0 < \alpha < 1$ for which:

$$\|\mathbf{v}^n - \mathbf{v}^*\| \leq K\alpha^n. \tag{6.48}$$

This is sometimes written as $\|\mathbf{v}^n - \mathbf{v}^*\| = O(\alpha^n)$.

N-stage contractions

By the defining property of a transient model, for each $d \in D^{\text{MD}}$ there exists a state in $S \setminus \Delta$ in which the probability of reaching Δ in one transition is positive. Hence for all states in $S \setminus \Delta$, there is positive probability of reaching Δ in *at most* $N = |S \setminus \Delta|$ transitions. Consequently, there exists a positive $\alpha < 1$ for which $\|\mathbf{Q}_d^N\| \leq \alpha < 1$ for all $d \in D^{\text{MD}}$ so that L_T satisfies

$$\|L_T^N \mathbf{v}_T - L_T^N \mathbf{v}'_T\| \leq \alpha \|\mathbf{v}_T - \mathbf{v}'_T\|$$

for any $\mathbf{v}_T$ and $\mathbf{v}'_T$ in V_T. Thus L_T^N is a contraction mapping so value iteration converges.

Euclidean norm convergence

This approach[14] is based on some convenient properties of the Euclidean norm[15]. Recall that the Euclidean norm of vector $\mathbf{v} \in V_T$ is defined by

$$\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{|S\setminus\Delta|} v(s_i)^2}.$$

Given a non-negative matrix $\mathbf{A} : V_T \to V_T$, its subordinate matrix norm with respect to the Euclidean norm is

$$\|\mathbf{A}\|_2 = \sigma(\mathbf{A}),$$

[14] In most settings Markov decision processes are analyzed with respect to the sup-norm.

[15] See Faddeeva [1959].

where $\sigma(A)$ denotes the spectral radius (largest eigenvalue of $\mathbf{A}$).

Modifying the proof of Theorem 5.1 in the discounted case[16], it follows that for $\mathbf{u}$ and $\mathbf{v}$ in V_T, L_T is a contraction mapping. In other words,

$$\|L_T\mathbf{v} - L_T\mathbf{u}\|_2 \leq \alpha\|\mathbf{v} - \mathbf{u}\|_2, \tag{6.49}$$

where

$$\alpha = \max_{d \in D^{\text{MD}}} \sigma(\mathbf{Q}_d) < 1. \tag{6.50}$$

That this quantity is less than 1 follows from the assumptions of transience and the finiteness of the sets of states and actions.

Thus it follows from Theorem 6.7 that value iteration converges geometrically in the Euclidean norm with

$$\|\mathbf{v}^n - \mathbf{v}^*\|_2 \leq K\alpha^n.$$

Since $\|\mathbf{v}\| \leq \|\mathbf{v}\|_2$, it follows that this inequality holds as well for the sup-norm.

Positively similar Markov decision processes*

This section describes Veinott's elegant analysis of the transient model[17]. The rationale for this analysis is that the condition $\|\mathbf{Q}_d^N\| < 1$ for some $N > 1$ does not imply that $\|\mathbf{Q}_d\| < 1$. This means that L_T^N being a contraction does not imply that the "Bellman operator" L_T is a contraction. Hence, the analysis of contraction mappings in Chapter 5 *does not* extend directly to this model.

Example 6.12. To illustrate the above dilemma, consider a model with $S = \{s_1, s_2, \Delta\}$ in which

$$\mathbf{P}_d = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/4 & 0 & 3/4 \\ 0 & 0 & 1 \end{bmatrix}$$

so that

$$\mathbf{Q}_d = \begin{bmatrix} 1/2 & 1/2 \\ 1/4 & 0 \end{bmatrix} \quad \text{and} \quad \mathbf{Q}_d^2 = \begin{bmatrix} 3/8 & 1/4 \\ 1/8 & 1/8 \end{bmatrix}.$$

Then $\|\mathbf{Q}_d^2\| = 5/8$ but $\|\mathbf{Q}_d\| = 1$, so $(L_d)_T^2$ is a contraction while $(L_d)_T$ is not. Note further that $\sigma(\mathbf{Q}_d) = 0.6830$.

What Veinott [1969a] showed is that there exists a related Markov decision process model that shares many key properties with the original Markov decision process. This related model is constructed in a way that ensures that the Bellman operator is a contraction mapping, which is sufficient to establish that value iteration converges

[16] A proof is outlined in Problem 3 at the end of this chapter.

[17] See Veinott [1969a].

geometrically. Since this is a property shared by both models, it follows that value iteration converges geometrically in the original model. Details of this analysis follow.

Definition 6.8. Two Markov decision processes with the same state and action spaces, and rewards and transition probabilities given by $(\mathbf{r}_d, \mathbf{P}_d)$ and $(\tilde{\mathbf{r}}_d, \tilde{\mathbf{P}}_d)$ are said to be *positively similar* if there exists a positive diagonal matrix $\mathbf{B}$ for which:

$$\tilde{\mathbf{r}}_d = \mathbf{B}\mathbf{r}_d \quad \text{and} \quad \tilde{\mathbf{P}}_d = \mathbf{B}\mathbf{P}_d\mathbf{B}^{-1}. \tag{6.51}$$

Equivalently, in component form, Markov decision processes with the same state and action spaces, and rewards and transition probabilities given by $\big(r(s,a), p(j|s,a)\big)$ and $\big(\tilde{r}(s,a), \tilde{p}(j|s,a)\big)$ are positively similar if there exists a vector $b(s) > 0$ on $s \in S$ for which:

$$\tilde{r}(s,a) = b(s)r(s,a) \quad \text{and} \quad \tilde{p}(j|s,a) = \frac{1}{b(s)}p(j|s,a)b(j). \tag{6.52}$$

Let $\mathbf{B}_T$ be the restriction of the $\mathbf{B}$ matrix to the transient states of $\mathbf{P}_d$. Observe that if

$$\mathbf{v} = (\mathbf{r}_d)_T + \mathbf{Q}_d\mathbf{v},$$

then

$$\mathbf{B}_T\mathbf{v} = \mathbf{B}_T(\mathbf{r}_d)_T + \mathbf{B}_T\mathbf{Q}_d\mathbf{v} = \mathbf{B}_T(\mathbf{r}_d)_T + \mathbf{B}_T\mathbf{Q}_d(\mathbf{B}_T)^{-1}\mathbf{B}\mathbf{v} = (\tilde{\mathbf{r}}_d)_T + \tilde{\mathbf{Q}}_d(\mathbf{B}_T\mathbf{v}),$$

where $\tilde{\mathbf{Q}}_d := \mathbf{B}_T\mathbf{Q}_d(\mathbf{B}_T)^{-1}$ is the restriction of $\tilde{\mathbf{P}}_d$ to the transient states. Hence, $\mathbf{B}_T\mathbf{v}$ is a solution of

$$\mathbf{v} = (\tilde{\mathbf{r}}_d)_T + \tilde{\mathbf{Q}}_d\mathbf{v}.$$

What this shows that if $\mathbf{v}$ is a solution of the policy evaluation equation in the model with $(\mathbf{r}_d, \mathbf{P}_d)$, then $\mathbf{B}\mathbf{v}$ is a solution in the positively similar model with $(\tilde{\mathbf{r}}_d, \tilde{\mathbf{P}}_d)$. Because the Bellman operator L_T applies the maximum component-wise, it follows that if $\mathbf{v}^*$ is the optimal value in $(\mathbf{r}_d, \mathbf{P}_d)$, then $\mathbf{B}\mathbf{v}^*$ is the optimal value in the positively similar model $(\tilde{\mathbf{r}}_d, \tilde{\mathbf{P}}_d)$. Moreover for any $\mathbf{v} \in V_T$, the same decision rule obtains the arg max in both models.

It is left as an exercise to show that positively similar Markov decision processes:

1. have the same sets of transient policies,
2. have the same optimal policy,
3. have the same spectral radii, and
4. have value iteration iterates that converge to the unique fixed point of the corresponding Bellman operator when the models are transient.

Given a Markov decision process model, the following argument shows how to construct a positively similar Markov decision process with $\|\tilde{\mathbf{Q}}_d\| < 1$, ensuring that value iteration converges geometrically.

Consider an expected total reward Markov decision process with the objective to maximize the number of transitions to reach Δ from each $s \neq \Delta$. This maximum can be found by solving the transient Bellman equation[18]:

$$\mathbf{v}_T = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{e} + \mathbf{Q}_d \mathbf{v}_T\}. \tag{6.53}$$

Note that for any $d \in D^{\mathrm{MD}}$, $\mathbf{v}_T^{d^\infty} \geq \mathbf{e}$ so that $\mathbf{v}_T \geq \mathbf{e}$. When $\mathbf{Q}_d \neq \mathbf{0}$, this inequality is strict[19].

The following more general result will be fundamental to what follows. The key assumption is that $\sigma(\mathbf{Q}_d) < 1$, so it also applies to matrices with row sums exceeding 1.

Proposition 6.5. Suppose for all $d \in D^{\mathrm{MD}}$

1. $\mathbf{Q}_d \geq \mathbf{0}$, and
2. $\sigma(\mathbf{Q}_d) < 1$.

Then if $\mathbf{b} \in V_T$ is the solution of

$$\mathbf{v} = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{e} + \mathbf{Q}_d \mathbf{v}\} \tag{6.54}$$

and $\mathbf{B}$ is an $|S \setminus \Delta| \times |S \setminus \Delta|$ diagonal matrix with entries $1/b(s)$,

$$\underset{d \in D^{\mathrm{MD}}}{\text{c-max}} \|\mathbf{B}\mathbf{Q}_d\mathbf{B}^{-1}\| < 1. \tag{6.55}$$

Proof. Let $\mathbf{b}$ denote the unique solution of (6.54), which exists by Theorem 6.6. Then

$$\mathbf{b} = \mathbf{e} + \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\, \mathbf{Q}_d \mathbf{b}. \tag{6.56}$$

Let $d^* \in \arg\text{c-max}_{d \in D^{\mathrm{MD}}} \mathbf{Q}_d\mathbf{b}$. Then from Proposition 6.1, $\mathbf{b} = (\mathbf{I} - \mathbf{Q}_{d^*})^{-1}\mathbf{e} \geq \mathbf{e}$ so that $b(s) > 0$ for all $s \in S \setminus \Delta$. Thus, $\mathbf{B}$ is well defined and $\mathbf{B}\mathbf{e}$ is positive.

Noting that $\mathbf{b} = \mathbf{B}^{-1}\mathbf{e}$ and multiplying both sides of (6.56) on the left by $\mathbf{B}$ gives:

$$\mathbf{e} = \mathbf{B}\mathbf{b} = \mathbf{B}\mathbf{e} + \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\, \mathbf{B}\mathbf{Q}_d\mathbf{b} = \mathbf{B}\mathbf{e} + \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\, \mathbf{B}\mathbf{Q}_d\mathbf{B}^{-1}\mathbf{e}.$$

[18] Recall that $\mathbf{e}$ denotes a vector with all components equal to one.

[19] To illustrate the need for this extra condition consider an example with $|S \setminus \Delta| = 1$ and $\mathbf{Q}_d = \mathbf{0}$. In this case, then $\mathbf{v}^{d^\infty} = 1$.

Since $\mathbf{Be}$ is positive,

$$\mathbf{e} > \underset{d\in D^{\mathrm{MD}}}{\text{c-max}} \|\mathbf{B}\mathbf{Q}_d\mathbf{B}^{-1}\|\mathbf{e},$$

which is the desired result.

□

The following result shows that a positively similar model can be constructed that has its norm arbitrarily close to its spectral radius. Its proof below is rather subtle and appeared in Veinott [1969b]. Note this result is trivially true and often overlooked in discounted models because $\sigma(\lambda\mathbf{P}) = \|\lambda\mathbf{P}\| = |\lambda|$ for any probability matrix $\mathbf{P}$.

Theorem 6.8. Given a transient Markov decision process with transition probability matrices $\mathbf{Q}_d$, $d \in D^{\mathrm{MD}}$ on the transient states of S. Then for any $\epsilon > 0$ there exists a positively similar Markov decision process with corresponding transition probability matrix $\tilde{\mathbf{Q}}_d, d \in D^{\mathrm{MD}}$ for which

$$\max_{d\in D^{\mathrm{MD}}} \sigma(\mathbf{Q}_d) \le \max_{d\in D^{\mathrm{MD}}} \|\tilde{\mathbf{Q}}_d\| \le \max_{d\in D^{\mathrm{MD}}} \sigma(\mathbf{Q}_d) + \epsilon. \tag{6.57}$$

Proof. Since $\sigma(\mathbf{Q}) \le \|\mathbf{Q}\|$ for any square matrix $\mathbf{Q}$ the first inequality in (6.57) follows by noting that the spectral radii of matrices in positively similar Markov decision processes are equal.

To obtain the second inequality choose ϵ' such that $\epsilon' < \epsilon$ and $0 < \sigma(\mathbf{Q}_d) + \epsilon' < 1$ for all $d \in D^{\mathrm{MD}}$. Therefore there exists an $\alpha > 1$ for which

$$1 = \alpha \max_{d\in D^{\mathrm{MD}}} \sigma(\mathbf{Q}_d) + \epsilon' = \max_{d\in D^{\mathrm{MD}}} \sigma(\alpha\mathbf{Q}_d) + \epsilon'. \tag{6.58}$$

Since $\sigma(\alpha\mathbf{Q}_d) < 1$, it follows from Proposition 6.5 that there exists a matrix $\mathbf{B}$ for which

$$\max_{d\in D^{\mathrm{MD}}} \|\alpha\mathbf{B}\mathbf{Q}_d\mathbf{B}^{-1}\| < 1.$$

Hence from (6.58)

$$\alpha \max_{d\in D^{\mathrm{MD}}} \|\tilde{\mathbf{Q}}_d\| = \alpha \max_{d\in D^{\mathrm{MD}}} \|\mathbf{B}\mathbf{Q}_d\mathbf{B}^{-1}\| < 1 = \alpha \max_{d\in D^{\mathrm{MD}}} \sigma(\mathbf{Q}_d) + \epsilon'.$$

Dividing through by α, noting that $\alpha > 1$ and $\epsilon > \epsilon'$, gives the result.

□

Example 6.13. Returning to Example 6.12, choosing $\epsilon = 0.01$ and following the argument in the proof of Theorem 6.8 gives $\alpha = \epsilon/\sigma(\mathbf{Q}_d) = 1.45$. Solving (6.54) with $\mathbf{Q}_d$ replaced by $\alpha\mathbf{Q}_d$ gives $\|\tilde{\mathbf{Q}}_d\| = 0.6833$, which is within 0.0003 of $\sigma(\mathbf{Q}_d)$.

Since as a result of Proposition 6.1, $\sigma(\mathbf{Q}_d) < 1$ for all $d \in D^{\mathrm{MD}}$, it follows from Theorem 6.8 and the finiteness of D^{MD} that

$$\max_{d \in D^{\mathrm{MD}}} \|\tilde{\mathbf{Q}}_d\| := \alpha < 1.$$

Hence the same argument as in the discounted case[20] establishes that

$$\tilde{L}_T \mathbf{v} := \underset{d \in D^{\mathrm{MD}}}{\text{c-max}} \{(\tilde{\mathbf{r}}_d)_T + \tilde{\mathbf{Q}}_d \mathbf{v}\} \tag{6.59}$$

is a contraction mapping with modulus α on V_T in the positively similar Markov decision process. Applying the Banach fixed-point theorem (Theorem 5.5) results in:

Proposition 6.6. $\tilde{L}_T$ has a unique fixed point $\tilde{\mathbf{v}}^* \in V_T$ and for any $\mathbf{v} \in V_T$, $\tilde{L}_T^n \mathbf{v}$ converges geometrically to $\tilde{\mathbf{v}}^*$.

Given the relationship between the iterates of value iteration in positively similar Markov decision processes gives:

Theorem 6.9. L_T has a unique fixed point $\mathbf{v}^* \in V_T$ and for any $\mathbf{v} \in V_T$, $L_T^n \mathbf{v}$ converges geometrically to $\mathbf{v}^*$.

In the case of policy evaluation, this result reduces to:

Corollary 6.6. For any $d \in D^{\mathrm{MD}}$, $(L_d)_T$ has a unique fixed point $\mathbf{v}^{d^\infty} \in V_T$ and for any $\mathbf{v} \in V_T$, $(L_d)_T^n \mathbf{v}$ converges geometrically to $\mathbf{v}^{d^\infty}$.

A weighted norm approach*

A third approach to analyzing convergence of value iteration in transient models is based on using weighted norms. by the following.

Definition 6.9. For $\mathbf{v} \in V$, define the *weighted sup-norm* of $\mathbf{v}$ with respect to $\mathbf{b}$ by[a]

$$\|\mathbf{v}\|_{\mathbf{b}} := \max_{s \in S} \left| \frac{v(s)}{b(s)} \right|, \tag{6.60}$$

where $b(s) > 0$ for all $s \in S$.

[a] As in Chapter 5, define the sup-norm using max, since the max is achieved on a finite state space.

[20] See the proof of Proposition 5.1.

Let $V_{\mathbf{b}}$ denote the set of real valued functions on $|S|$ that are bounded in the weighted sup-norm with respect to $\mathbf{b}$. In other words, for each $\mathbf{v} \in V_{\mathbf{b}}$ there exists a $M > 0$ such that $\|\mathbf{v}\|_{\mathbf{b}} \leq M$. Consequently for all $s \in S$,

$$|v(s)| \leq Mb(s).$$

If S is partially ordered, this corresponds to a bounded growth rate. The corresponding *subordinate* matrix norm is defined by:

Definition 6.10. For $\mathbf{A} \in V \times V$, define the *weighted matrix norm* of $\mathbf{A}$ with respect to $\mathbf{b}$ by

$$\|\mathbf{A}\|_{\mathbf{b}} := \max_{s \in S} \frac{\sum_{j \in S} b(j)|a(s,j)|}{b(s)} \tag{6.61}$$

where $b(s) > 0$ for all $s \in S$.

When $\|\mathbf{A}\|_{\mathbf{b}} \leq M$,

$$\sum_{j \in S} b(j)|a(s,j)| \leq Mb(s)$$

for all $s \in S$.

Hence, boundedness in the weighted sup-norm imposes growth conditions on values and structure on matrices. Such norms play an important role in the analysis of countable state models and approximate dynamic programs.

Surprisingly this approach is very closely related to the analysis using positively similar Markov decision process. When $b(s)$ is determined by the methods in that section, it is easy to see that

$$\|\mathbf{Q}_d\|_{\mathbf{b}} = \|\tilde{\mathbf{Q}}_d\|, \tag{6.62}$$

where the norm on the left is the weighted sup-norm on the original Markov decision process while the norm on the right is the *unweighted* sup-norm applied to the constructed positively similar Markov decision process[21]. Therefore it follows that:

Proposition 6.7. L_T is a contraction mapping with respect to the weighted sup-norm $\|\cdot\|_{\mathbf{b}}$, where $\mathbf{b}$ is the unique solution to (6.54).

As a result of Proposition 6.7 value iteration converges geometrically in the weighted sup-norm in the original model. In other words,

$$\|L_T^n \mathbf{v}^0 - \mathbf{v}^*\|_{\mathbf{b}} \leq \alpha^n \|\mathbf{v}^0 - \mathbf{v}^*\|_{\mathbf{b}} \tag{6.63}$$

for some $0 < \alpha < 1$.

[21] Recall the definition of the matrix norm from equation (5.8): $\|\mathbf{A}\| := \max_{s \in S} \sum_{j \in S} |a(s,j)|$.

Error bounds

An error bound in a transient model is derived below. Consider the relationship

$$\|\mathbf{v} - \mathbf{v}^*\| \leq \|\mathbf{v} - \mathbf{v}'\| + \|\mathbf{v}' - \mathbf{v}^*\|.$$

Viewing L_T as an N-stage contraction and setting $\mathbf{v}' = L_T^N \mathbf{v}$ where $N = |S \setminus \Delta|$, it follows that

$$\|\mathbf{v}' - \mathbf{v}^*\| = \|L_T^N \mathbf{v} - L_T^N \mathbf{v}^*\| \leq \alpha \|\mathbf{v} - \mathbf{v}^*\|$$

with[22]

$$\alpha = \max_{d \in D^{\mathrm{MD}}} \|\mathbf{Q}_d^N\| < 1.$$

That this quantity is less than 1 follows from the finiteness of the model. Hence, this gives the error bound

$$\|\mathbf{v}^* - \mathbf{v}\| \leq \left(\frac{1}{1-\alpha}\right) \|L_T^N \mathbf{v} - \mathbf{v}\|. \tag{6.64}$$

In Example 6.12, in which there is a single decision rule and $N = 2$, $\alpha = 5/8$.

A second approach is to derive an error bound in terms of the *Euclidean* norm $\|\cdot\|_2$. Following the above argument shows that

$$\|\mathbf{v}^* - \mathbf{v}\|_2 \leq \left(\frac{1}{1-\alpha}\right) \|L_T \mathbf{v} - \mathbf{v}\|_2, \tag{6.65}$$

with

$$\alpha = \max_{d \in D^{\mathrm{MD}}} \sigma(\mathbf{Q}_d) < 1.$$

Since $\sigma(\mathbf{Q}_d) \leq \|\mathbf{Q}_d\|$ for any norm, the error bound in the Euclidean norm is *tighter* than that using the sup-norm and is consistent with our computations. Moreover since

$$\|\mathbf{v}\| \leq \|\mathbf{v}\|_2 \leq \sqrt{M}\|\mathbf{v}\|,$$

where $M = |S \setminus \Delta|$, it follows from (6.65) that

$$\|\mathbf{v}^* - \mathbf{v}\| \leq \left(\frac{\sqrt{M}}{1-\alpha}\right) \|L_T \mathbf{v} - \mathbf{v}\|. \tag{6.66}$$

In practice this error bound might not be useful since M may be large.

Other approaches for deriving error bounds include constructing a positively similar Markov decision process or using the weighted norm approach.

While these bounds exist in theory, computing them in practice is problematic due to the computation of α, whicch requires a priori knowledge of the optimal policy, i.e., the decision rule that achieves the max in the definition of α.

[22] In component notation,

$$\alpha = \max_{s \in S \setminus \Delta, a \in A_s} \sum_{j \in S \setminus \Delta} q^N(j|s,a)$$

where $q^N(j|s,a)$ denotes components of $p^N(j|s,a)$ restricted to $s \in S \setminus \Delta$ and $j \in S \setminus \Delta$.

6.5.3 Stochastic shortest path models

Recall that in an SSP model, an improper policy cannot be optimal.

Theorem 6.10. In an SSP model:

1. The transient Bellman equation
$$L_T\mathbf{v} = \mathbf{v}$$
has a unique solution $\mathbf{v}_T^* \in V_T$.
2. The Bellman equation
$$L\mathbf{v} = \mathbf{v}$$
subject to $v(\Delta) = 0$ has a unique solution $\mathbf{v}^* \in V$.
3. For any proper stationary policy d^∞ with $d \in D^{\text{MR}}$, the transient policy evaluation equation
$$\mathbf{v} = (\mathbf{r}_d)_T + \mathbf{Q}_d\mathbf{v}$$
has a unique solution $\mathbf{v}_T^{d^\infty} \in V_T$.
4. For any proper stationary policy d^∞ with $d \in D^{\text{MR}}$, the policy evaluation equation
$$\mathbf{v} = \mathbf{r}_d + \mathbf{P}_d\mathbf{v}$$
has a unique solution $\mathbf{v}^{d^\infty} \in V$.

Analyses in the previous section showed that value iteration converges in a transient model or equivalently in an SSP model in which every policy is proper[23].

6.5.4 Positive models

The following sections provide analyses of the convergence of value iteration in positive and negative models. The following result holds for value iteration in any expected total reward model.

Lemma 6.2. Suppose the sequence $\mathbf{v}^n, n = 0, 1, 2, \ldots$ is generated by value

[23] Bertsekas and Tsitsiklis [1991] establish this result in SSP models with improper policies, however convergence may not be at a geometric rate.

iteration with $\mathbf{v}^0 = \mathbf{0}$. Then for each $N \geq 1$,

$$\mathbf{v}^n = \mathbf{v}_N^\pi = \sum_{n=1}^{N} \mathbf{P}_{d_1} \cdots \mathbf{P}_{d_n} \mathbf{r}_{d_n}, \tag{6.67}$$

where $\pi = (d_1, d_2, ...) \in \Pi^{\text{MD}}$. Moreover $(d_1, \ldots, d_N)$ is an optimal policy in an $(N+1)$-period model with terminal reward 0.

The Bellman equation in positive models

The following technical result summarizes useful properties of the Bellman operator L in positive models. It is convenient to denote the set of non-negative values, $V^+ := \{\mathbf{v} \in V \,|\, \mathbf{v} \geq \mathbf{0}\}$.

Proposition 6.8. In a positive model:

1. There exists a $d \in D^{\text{MD}}$ for which $\mathbf{v}^{d^\infty} \geq \mathbf{0}$,
2. $L\mathbf{0} \geq \mathbf{0}$,
3. $\mathbf{v}^* \geq \mathbf{0}$,
4. $L : V^+ \to V^+$, and
5. for any $d \in D^{\text{MD}}$, $r_d(s) \leq 0$ for all s in each recurrent set of $\mathbf{P}_d$.

Proof. Parts 1-3 are immediate consequences of the second condition defining a positive model. Part 4 is an immediate consequence of Proposition 6.4 and Part 5is an immediate consequence of the first condition defining a positive model. □

The model in Example 6.1 with action $a_{2,2}$ deleted is a positive model and exhibits all the results in the above proposition. Note that under action $a_{1,1}$, s_1 is recurrent but its expected total reward is $-\infty$. Note that this is an SSP model since the policy using action $a_{1,1}$ every period is improper.

The following result identifies $\mathbf{v}^*$ among the solutions of the Bellman equation in a positive model.

Theorem 6.11. In a positive model:

1. If $\mathbf{v} \in V^+$ satisfies $\mathbf{v} \geq L\mathbf{v}$, then $\mathbf{v} \geq \mathbf{v}^*$.
2. The optimal value function $\mathbf{v}^*$ is the *minimal* solution in V^+ of
$$L\mathbf{v} = \mathbf{v}.$$
3. For any $d \in D^{\mathrm{MR}}$ the policy value function $\mathbf{v}^{d^\infty} \in V^+$ is the minimal solution of
$$L_d\mathbf{v} = \mathbf{v}.$$

Proof. To prove Part 1 choose $\pi = (d_1, d_2, \ldots) \in \Pi^{\mathrm{MR}}$. By Lemma 5.4 and the assumption that $\mathbf{v} \geq L\mathbf{v}$,

$$\begin{aligned}\mathbf{v} &\geq \mathbf{r}_{d_1} + \mathbf{P}_{d_1}\mathbf{v} \geq \mathbf{r}_{d_1} + \mathbf{P}_{d_1}\mathbf{r}_{d_2} + \mathbf{P}_{d_1}\mathbf{P}_{d_2}\mathbf{v} \geq \cdots \\ &\geq \sum_{n=1}^{N} \mathbf{P}_{d_1}\cdots\mathbf{P}_{d_n}\mathbf{r}_{d_n} + \mathbf{P}_{d_1}\cdots\mathbf{P}_{d_N}\mathbf{v} \geq \sum_{n=1}^{N} \mathbf{P}_{d_1}\cdots\mathbf{P}_{d_n}\mathbf{r}_{d_n},\end{aligned}$$

where the last inequality follows by noting that $\mathbf{v} \geq \mathbf{0}$. Hence, passing to the limit on the right hand side of the above expression establishes that $\mathbf{v} \geq \mathbf{v}^*$.

Part 2 follows immediately from Part 1 applied to $L\mathbf{v} = \mathbf{v}$. Part 3 follows by restricting Part 2 to a single decision rule and Lemma 5.4. □

Example 6.14. The deterministic positive model represented by Figure 6.4 illustrates how to use this result. Decision rules are specified by action choice in state s_1. The Bellman equation for this model can be written as:

$$\begin{aligned}v(s_1) &= \max\{v(s_1), 1 + v(s_2)\} \\ v(s_2) &= v(s_2).\end{aligned}$$

Any solution of the Bellman equation can be expressed as $v(s_1) = b$ and $v(s_2) = c$ where $b \geq c + 1$. Thus the minimal solution is $v^*(s_1) = 1$ and $v(s_2) = 0$, and the corresponding greedy decision rule is $d(s_1) = a_{1,2}$, $d(s_2) = a_{2,1}$.

Observe that s_2 is recurrent (absorbing) under the optimal policy so that the requirement that it be the minimal value in V^+ means that $v^*(s_2)$ must be 0.

If instead if $r(s_1, a_{1,2}) = -1$, surprisingly this model would still be a positive model even though one of the rewards is negative. The minimal solution of the Bellman equation for this modification would be $v(s_1) = v(s_2) = 0$.

Thus in both of these models, the minimal solution sets $v(s) = 0$ when s is in a recurrent state under the optimal policy.

Note that when $r(s_1, a_{1,2}) = -1$ it would also be a negative model (see Figure 6.3). Moreover either model could be transformed to a transient model,

by adding a zero-reward absorbing state Δ and replacing the zero-reward self transitions by ones from states s_1 and s_2 to Δ.

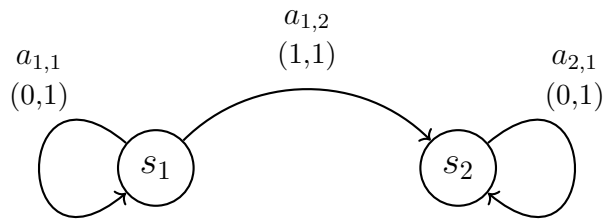


Figure 6.4: Symbolic representation of positive model in Example 6.14. Recall that the labels (r, p) refer to the reward and transition probabilities, respectively, associated with using the specified action in the given state.

Convergence of value iteration in positive models

Analysis of the Gridworld model in Example 6.11 showed that value iteration converged monotonically in Instance 3, which was a positive model. Such monotonicity holds for any positive model and provides the basis for a proof of convergence of value iteration.

Theorem 6.12. Suppose in a positive model the sequence $\mathbf{v}^n, n = 0, 1, 2, \ldots$ is generated by value iteration with $\mathbf{v}^0 = \mathbf{0}$. Then $\mathbf{v}^n$ converges monotonically to $\mathbf{v}^*$ and $\|\mathbf{v}^n - \mathbf{v}^*\| \to 0$.

Proof. Since $\mathbf{v}^1 = L\mathbf{0} \geq \mathbf{0} = \mathbf{v}^0$, it follows from part 2 of Proposition 6.4 that $\mathbf{v}^n$ is monotonically non-decreasing. Since $\mathbf{v}^n$ is the value of a policy, condition 1 in the definition of positive models implies that it is bounded above. Hence, by a standard result in analysis,[24] $v^n(s)$ converges for each $s \in S$. Call the limit $v'(s)$. Since S is finite, $\|\mathbf{v}^n - \mathbf{v}'\| \to 0$.

Since $\mathbf{v}^n$ is monotonically non-decreasing $L\mathbf{v}' \geq \mathbf{v}^n$ for all $n \geq 0$ so that $L\mathbf{v}' \geq \mathbf{v}'$. On the other hand, for any $d \in D^{\mathrm{MD}}$

$$L_d\mathbf{v}^n \leq L\mathbf{v}^n = \mathbf{v}^{n+1} \leq \mathbf{v}'.$$

Since $L_d\mathbf{v}$ is linear in $\mathbf{v}$, $L_d\mathbf{v}^n$ converges to $L_d\mathbf{v}'$ so that $L_d\mathbf{v}' \leq \mathbf{v}'$. Hence

$$L\mathbf{v}' = \operatorname*{c\text{-}max}_{d \in D^{\mathrm{MD}}} L_d\mathbf{v}' \leq \mathbf{v}'.$$

[24] Theorem 3.14 in Rudin [1964].

Therefore, $L\mathbf{v}' = \mathbf{v}'$.

Since for every $n \geq 0, \mathbf{v}^n$ is the value of a policy, it follows from Theorem 6.3 that $\mathbf{v}' = \mathbf{v}^*$. □

6.5.5 Negative models

Instance 2 in the Gridworld model in Example 6.11 is an example of a negative model. For this instance, value iteration generated a convergent monotonically non-increasing sequence of iterates. This section establishes that result in greater generality.

The Bellman equation in negative models

The following technical result summarizes useful properties of the Bellman operator L in negative models. As above, it is convenient to denote the set of non-positive values, $V^- := \{\mathbf{v} \in V \mid \mathbf{v} \leq \mathbf{0}\}$.

Proposition 6.9. In a negative model:

1. For all $d \in D^{\mathrm{MD}}$, $\mathbf{0} \geq \mathbf{v}^{d^\infty}$,
2. $L\mathbf{0} \leq \mathbf{0}$,
3. $\mathbf{v}^* \leq \mathbf{0}$,
4. $L : V^- \to V^-$, and
5. if $\mathbf{v}^{d^\infty}(s) > -\infty$ for some $s \in S$ and $d \in D^{\mathrm{MD}}$, then $r_d(s') = 0$ for all s' in the recurrent set of $\mathbf{P}_d$ containing s.

Proof. Condition 1 in the definition of a negative model implies that $r(s,a) \leq 0$ for $a \in A_s, s \in S$. From this observation, results 1-4 follow immediately. Part 5 follows by noting that if $r_d(s') < 0$ for some s' in a recurrent set of $\mathbf{P}_d$, then $v_d(s) = -\infty$ for all s in that recurrent set of $\mathbf{P}_d$, which would be a contradiction. □

The following result is the equivalent of Theorem 6.11 for negative models. The first part is also used to prove that value iteration converges below.

Theorem 6.13. In a negative model:

1. If $\mathbf{v} \in V^-$ satisfies $\mathbf{v} \leq L\mathbf{v}$, then $\mathbf{v} \leq \mathbf{v}^*$.
2. The optimal value function $\mathbf{v}^*$ is the *maximal* solution in V^- of
$$L\mathbf{v} = \mathbf{v}.$$
3. For any $d \in D^{\mathrm{MR}}$ the policy value function $\mathbf{v}^{d^\infty} \in V^-$ is the maximal solution of
$$L_d\mathbf{v} = \mathbf{v}.$$

Proof. A proof of Part 1 follows. Since $L\mathbf{v} \geq \mathbf{v}$, there exists a $d_1 \in D^{\mathrm{MD}}$ for which:

$$\mathbf{r}_{d_1} + \mathbf{P}_{d_1}\mathbf{v} \geq \mathbf{v}.$$

But since $\mathbf{v} \in V^-$, $\mathbf{P}_{d_1}\mathbf{v} \leq \mathbf{0}$ so that $\mathbf{r}_{d_1} \geq \mathbf{v}$. Applying L to both sides of this inequality implies that there exists a $d_2 \in D^{\mathrm{MD}}$ for which

$$\mathbf{r}_{d_2} + \mathbf{P}_{d_2}\mathbf{r}_{d_1} = L\mathbf{r}_{d_1} \geq L\mathbf{v} \geq \mathbf{v}.$$

By repeated application of this argument it follows that there exists a policy $\pi = (d_1, d_2, \ldots) \in \Pi^{\mathrm{MD}}$ for which $\mathbf{v}^\pi \geq \mathbf{v}$. Hence $\mathbf{v}^* \geq \mathbf{v}$. Part 2 follows immediately from Part 1 applied to $L\mathbf{v} = \mathbf{v}$. Part 3 follows by restricting Part 2 to a single decision rule and Lemma 5.4. □

Example 6.14 showed that when $r(s_1, a_{1,2}) = -1$, the model was also negative (see Figure 6.3). For this model the Bellman equation is:

$$\begin{aligned} v(s_1) &= \max\{v(s_1), -1 + v(s_2)\} \\ v(s_2) &= v(s_2). \end{aligned}$$

Therefore, any solution has the form $v(s_1) = b$, $v(s_2) = c$ where $b \geq c - 1$. Hence the maximal solution is $v^*(s_1) = v^*(s_2) = 0$, which is zero on the recurrent states of the optimal policy.

Convergence of value iteration in negative models

The following result establishes convergence of value iteration. Its proof is a bit more complicated than that for positive models.

Theorem 6.14. Suppose in a negative model the sequence $\mathbf{v}^n, n = 0, 1, 2, \ldots$ is generated by value iteration with $\mathbf{v}^0 = \mathbf{0}$. Then $\mathbf{v}^n$ converges monotonically to $\mathbf{v}^*$ and $\|\mathbf{v}^n - \mathbf{v}^*\| \to 0$.

Proof. Since $\mathbf{v}^1 = L\mathbf{0} \leq \mathbf{0} = \mathbf{v}^0$, it follows from part 2 of Proposition 6.4 that $\mathbf{v}^n$ is monotonically non-increasing. Since $\mathbf{v}^* \leq \mathbf{0}$ and satisfies the optimality equation, it follows from the monotonicity of L that for any $n \geq 1$

$$\mathbf{v}^* = L^n\mathbf{v}^* \leq L^n\mathbf{0} = \mathbf{v}^{n+1}.$$

Hence by the same argument as in the positive case, $\mathbf{v}^n$ converges to a limit $\mathbf{v}'$ which satisfies $L\mathbf{v}' \leq \mathbf{v}'$ and $\mathbf{v}' \geq \mathbf{v}^*$. Since S is finite, $\|\mathbf{v}^n - \mathbf{v}'\| \to 0$.

The following arguments establish the reverse inequality. Let $\epsilon > 0$.

$$\begin{aligned}
\mathbf{v}' = \lim_{n\to\infty} \mathbf{v}^n = \lim_{n\to\infty} L\mathbf{v}^n &= \lim_{n\to\infty} \text{c-max}_{d\in D^{\text{MD}}}\{\mathbf{r}_d + \mathbf{P}_d\mathbf{v}^n\} \\
&= \lim_{n\to\infty} \text{c-max}_{d\in D^{\text{MD}}}\{\mathbf{r}_d + \mathbf{P}_d\mathbf{v}' + \mathbf{P}_d(\mathbf{v}^n - \mathbf{v}')\} \\
&\leq \lim_{n\to\infty} \text{c-max}_{d\in D^{\text{MD}}}\{\mathbf{r}_d + \mathbf{P}_d\mathbf{v}' + \|\mathbf{v}^n - \mathbf{v}'\|\mathbf{e}\} \\
&\leq L\mathbf{v}' + \epsilon\mathbf{e},
\end{aligned}$$

where the last inequality follows by noting that for any $\epsilon > 0$ and n sufficiently large $\|\mathbf{v}^n - \mathbf{v}'\| \leq \epsilon$. Since ϵ was arbitrary, $\mathbf{v}' \leq L\mathbf{v}'$ so that it follows from Part 1 of Theorem 6.13 that $\mathbf{v}' \leq \mathbf{v}^*$.

That $\mathbf{v}' = \mathbf{v}^*$ follows by combining both parts of the proof. □

6.6 Policy iteration

This section concerns the convergence of policy iteration in transient and SSP models. Analysis of positive and negative models is more subtle[25] and not described here. In these models, policy iteration solves the transient Bellman equation by iterating between evaluation and improvement steps.

A statement of the algorithm follows.

Algorithm 6.2. Policy iteration for a transient model: vector form

1. **Initialize:** Specify $d \in D^{\text{MD}}$ and set $\Gamma \leftarrow S \setminus \Delta$.
2. **Iterate:** While $\Gamma \geq \emptyset$:
   (a) **Evaluate:** Find $\mathbf{v}'_T$ by solving
   $$\mathbf{v}_T = (\mathbf{r}_d)_T + \mathbf{Q}_d\mathbf{v}_T. \tag{6.68}$$
   (b) **Improve:** Choose
   $$d' \in \arg\text{c-max}_{\delta\in D^{\text{MD}}}\{(\mathbf{r}_\delta)_T + \mathbf{Q}_\delta\mathbf{v}'_T\}, \tag{6.69}$$

[25] See Sections 7.2.5 and 7.3.4 in Puterman [1994].

setting $d' = d$ if possible.

(c) $\Gamma \leftarrow \{s \in S \setminus \Delta \,|\, d'(s) \neq d(s)\}$.

(d) **Update:** $d \leftarrow d'$.

3. **Terminate:** Return $d^* = d$ and $\mathbf{v}^* = \mathbf{v}'_T$.

Note that the improvement step is executed component-wise as follows. For each $s \in S \setminus \Delta$ it chooses

$$a'_s \in \arg\max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a) v(j) \right\}, \tag{6.70}$$

setting $a'_s = d(s)$ if possible.

The conditions "setting $d' = d$ if possible" or "setting $a'_s = d(s)$ if possible" are referred to as *anti-cycling rules*. They apply to the case when there is more than one decision rule or action that attains the max in the improvement step. The inclusion of this stipulation ensures that the stopping criterion is eventually satisfied.

Moreover, the evaluation step can be implemented recursively by iterating

$$\begin{aligned} \mathbf{v}'_T &\leftarrow (\mathbf{r}_d)_T + \mathbf{Q}_d \mathbf{v}_T \\ \mathbf{v}_T &\leftarrow \mathbf{v}'_T \end{aligned}$$

to achieve a high degree of precision. By the argument in the previous section, this iterative scheme converges to $\mathbf{v}^{d^\infty}$ in a transient model. Truncating this iterative scheme corresponds to modified policy iteration below.

6.6.1 Transient models

This section analyzes convergence of policy iteration in transient models. The development parallels that in the discounted case (Section 5.7) by showing that the iterates of policy iteration have a "Newton" representation. To do so, define the operator $B_T : V_T \to V_T$ by $B_T \mathbf{v}_T := L_T \mathbf{v}_T - \mathbf{v}_T$.

Proposition 6.10. Let $\mathbf{v}_T$ and $\mathbf{v}'_T$ be successive iterates of transient policy iteration (restricted to the set of transient states). Then

$$\mathbf{v}'_T = \mathbf{v}_T + (\mathbf{I} - \mathbf{Q}_{d_\mathbf{v}})^{-1} B_T \mathbf{v}_T \tag{6.71}$$

where $d_\mathbf{v} \in \arg\,\text{c-max}_{\delta \in D^{\text{MD}}} \{(\mathbf{r}_\delta)_T + \mathbf{Q}_\delta \mathbf{v}_T\}$.

Proof. The proof is identical to that in the discounted case (Proposition 5.10) after taking into account that iterates are restricted to transient states. It follows that:

$$\begin{aligned}\mathbf{v}'_T &= (\mathbf{I} - \mathbf{Q}_{d_\mathbf{v}})^{-1}(\mathbf{r}_{d_\mathbf{v}})_T \\ &= (\mathbf{I} - \mathbf{Q}_{d_\mathbf{v}})^{-1}(\mathbf{r}_{d_\mathbf{v}})_T - \mathbf{v}_T + \mathbf{v}_T \\ &= (\mathbf{I} - \mathbf{Q}_{d_\mathbf{v}})^{-1}\left((\mathbf{r}_{d_\mathbf{v}})_T + (\mathbf{Q}_{d_\mathbf{v}} - \mathbf{I})\mathbf{v}_T\right) + \mathbf{v}_T \\ &= \mathbf{v}_T + (\mathbf{I} - \mathbf{Q}_{d_\mathbf{v}})^{-1} B_T \mathbf{v}_T.\end{aligned}$$

□

This result leads to an easy proof of the convergence of policy iteration in transient models.

Theorem 6.15. In a transient model, policy iteration converges monotonically and in a finite number of iterations to the optimal value function $\mathbf{v}_T^*$ and an optimal stationary policy.

Proof. By construction

$$\mathbf{v}_T = (\mathbf{r}_d)_T + \mathbf{Q}_d \mathbf{v}_T \leq \underset{\delta \in D^{\mathrm{MD}}}{\text{c-max}}\{(\mathbf{r}_\delta)_T + \mathbf{Q}_\delta \mathbf{v}_T\} = L_T \mathbf{v}_T$$

with strict inequality whenever $d' \neq d$ where d' satisfies (6.69). Therefore, either $B_T \mathbf{v}_T = L_T \mathbf{v}_T - \mathbf{v}_T = \mathbf{0}$ or $B_T \mathbf{v}_T(s) > 0$ for some $s \in S \setminus \Delta$. In the former case $\mathbf{v}_T$ solves the transient Bellman equation and in the latter case it follows immediately from (6.71) and the observation that $(\mathbf{I} - \mathbf{Q}_d)^{-1} \geq \mathbf{I}$ that $v'_T(s) > v_T(s)$ for some $s \in S - \Delta$. Since there are only finitely many policies, the stopping criterion must be satisfied at some finite N at which point $\mathbf{v}_T$ satisfies the Bellman equation. From Corollary (6.3), d^∞ is an optimal stationary policy. □

Example 6.15. This example solves Instance 1 of the Gridworld model in Example 6.2 using policy iteration. Choose the initial decision rule as "go up" when possible, otherwise "go right" in cell 2 and "go left" in cell 3[a]. To avoid explicitly writing our $\mathbf{Q}_d$ and thus simplify coding, the evaluation step was carried out iteratively.

The number of iterations (improvement steps) required for convergence varied with p as shown in Table 6.2.

p	0.01	0.1	0.5	0.9	0.99
iterations	5	4	3	3	6

Table 6.2: Number of iterations for policy iteration to converge in Gridworld model as a function of p.

Note that when $p = 0$ or $p = 1$ the stationary policy corresponding to the initial decision rule was improper because it cycled between cells 2 and 3 forming a closed class consisting of cells 2 and 3. Since the cost associated with these states is negative infinity the model in this case is an SSP model. Hence, as noted below, in such a case, starting the algorithm with a proper policy ensure convergences.

[a] To simplify coding one could allow all actions in all cells but set $q(s, a)$ to a large negative value when action a is impossible in state s. If this adjustment is made, policy iteration could start with the decision rule "go up" in all states.

6.6.2 Stochastic shortest path models

The above result extends directly to SSP models in which policy iteration starts with a proper policy. The argument used to prove Theorem 6.15 ensures that an improper policy cannot be identified at a subsequent iterate of policy iteration.

Corollary 6.7. In a stochastic shortest path model, suppose transient policy iteration begins with a *proper* policy. Then it converges monotonically to $\mathbf{v}_T^*$ in a finite number of iterations to the optimal value function and an optimal stationary policy.

Numerical examples such as that in Example 6.18 below, show that when starting with an improper policy, (6.68) will not have a solution.

6.7 Modified policy iteration

Since transient models behave like discounted models, a modified policy iteration (MPI) algorithm can be analyzed by previously developed methods. Because $v^*(\Delta) = 0$ and all policies stop when reaching Δ, MPI can be restricted to transient states. A vector version of the algorithm follows.

6.7.1 Transient models

Algorithm 6.3. Modified policy iteration for a transient model: vector form

1. **Initialize:**
   (a) Specify $d' \in D^{\text{MD}}$.
   (b) Specify $\epsilon > 0$ and $\sigma > \epsilon$.

(c) Specify $\mathbf{v}_T \in V_T$.

(d) Specify a sequence of non-negative integers m_n, $n = 1, 2, \ldots$.

(e) $n \leftarrow 1$.

2. **Iterate:** While $\sigma \geq \epsilon$:

   (a) **Update:** $d \leftarrow d'$.

   (b) **Evaluate:**

      i. $m \leftarrow 1$ and $\mathbf{u}_T \leftarrow \mathbf{v}_T$.

      ii. While $m \leq m_n$:

         A. $\mathbf{u}_T \leftarrow (\mathbf{r}_d)_T + \mathbf{Q}_d \mathbf{u}_T$.

         B. $m \leftarrow m + 1$.

   (c) **Improve:**

      i. Choose
      $$d' \in \arg\operatorname*{c\text{-}max}_{\delta \in D^{\mathrm{MD}}} \{(\mathbf{r}_\delta)_T + \mathbf{Q}_\delta \mathbf{u}_T\}. \tag{6.72}$$

      ii. $\mathbf{v}_T \leftarrow L_T \mathbf{u}_T$.

      iii. $\sigma \leftarrow \|\mathbf{v}_T - \mathbf{u}_T\|$.

      iv. $n \leftarrow n + 1$.

3. **Terminate:** Return $d_\epsilon = d'$ and $\mathbf{v}_T^\epsilon = \mathbf{v}_T$.

Some comments about the algorithm follow:

1. The algorithm could start directly from the improvement step, avoiding the need to specify an initial policy.

2. The index n is necessary to determine m_n.

3. Setting $m_n = 0$ (so the "Evaluate" step above is skipped) for all n is identical to value iteration and $m_n = \infty$ corresponds to policy iteration.

4. The stopping criterion is based on the value function, while that of policy iteration is based on the decision rule.

Let $\mathbf{v}_T^n$ and d_n for $n = 1, 2, \ldots$ denote the sequence of values and decision rules, respectively, generated by modified policy iteration[26]. Then with the exception of the first iterate, which can be written as

$$\mathbf{v}_T^2 = L_{d^1}^{m_1} \mathbf{v}_T^1,$$

[26] This notation assumes the initial value is $\mathbf{v}_T^0$ and the initial decision rule is d_0.

iterates can be written as:

$$\mathbf{v}_T^{n+1} = L_{d'}^{m_n+1}\mathbf{v}_T^n. \tag{6.73}$$

Therefore

$$\mathbf{v}_T^{n+1} = L_{d_n}^{m_n+1}\dots L_{d_1}^{m_1}\mathbf{v}_T^1. \tag{6.74}$$

This quantity can be interpreted as the expected total reward obtained using the policy implied by the above sequence of operators with terminal reward $\mathbf{v}_T^1$. If instead the algorithm began with the improvement step, $\mathbf{v}_T^2 = L^{m_1+1}\mathbf{v}_T^1$ consistent with the remaining terms. Consequently, the assumption of a transient model, causes the impact of the terminal reward to decrease as n increases.

6.7.2 Convergence in transient models

An outline of a proof of the convergence of MPI under the assumption that the algorithm starts from a value that satisfies $L\mathbf{v}_T \geq \mathbf{v}_T$ follows. All the details appear in the analysis of the discounted case in Section 5.8.3.

Theorem 6.16. Let $m_n, n = 1, 2, \dots$ denote a sequence of non-negative integers and let $\mathbf{v}_T^n, n = 1, 2, \dots$ denote a sequence of iterates of modified policy iteration. Then if $L\mathbf{v}_T^1 \geq \mathbf{v}_T^1$, $\mathbf{v}_T^n$ converges monotonically to $\mathbf{v}_T^*$ and $\|\mathbf{v}_T^n - \mathbf{v}_T^*\| \to 0$.

Proof. Let $\mathbf{u}_T^n$ denote the iterates of value iteration starting with $\mathbf{u}_T^1 = \mathbf{v}_T^1$ for $n = 1, 2, \dots$ Then by induction it follows that

$$\mathbf{u}_T^n \leq \mathbf{v}_T^n \leq \mathbf{v}_T^*.$$

Since $\mathbf{u}_T^n$ converges monotonically to the optimal value function $\mathbf{v}_T^*$ when $L\mathbf{u}_T^1 \geq \mathbf{u}_T^1$, the above inequalities imply the result. □

This result also holds if MPI is started in the improvement step with $\mathbf{v}_T^1$ satisfying the above condition.

6.7.3 An example

The following example applies modified policy iteration to an instance of the Gridworld model, investigating the choice of m_n.

Example 6.16. This example considers Instance 1 of the Gridworld model in Example 6.11 with $p = 0.1$ and $\epsilon = 0.0001$. The probability p is chosen to be small so that value iteration converges slowly and the impact of the choice of m_n can be examined. The algorithm is initialized with the same decision rule used to start policy iteration in the previous section. Values of m_n are chosen to be

constant, increasing and decreasing with n.

Table 6.3 gives computational results. The quantity *Evaluation equivalents* equals the sum of the number of evaluations plus twice the number of improvements because each maximization required evaluating 28 state-action pairs while each evaluation step required updating 13 values. Hence a maximization step is "roughly" twice as costly as an evaluation step.

m_n	Improvement steps (maximizations)	Evaluation equivalents	Iteration at which optimal policy first appears
0	251	502	31
2	88	352	7
10	21	252	2
20	13	286	2
50	7	364	1
n^2	10	405	3
$\max\{40-n,0\}$	9	666	4

Table 6.3: Computational results for various specifications for m_n.

The results show that a moderate fixed value of m_n or an increasing sequence has the best performance. Moreover all choices of m_n with the exception of $m_n = \max\{40-n,0\}$ require less computational effort than value iteration.

6.7.4 Stochastic shortest path models

In an SSP model, modified policy iteration converges when

1. initiated with a proper policy, or
2. it begins in the improvement step, or
3. when $m_0 < \infty$.

In the latter case, $\mathbf{v}'_T$ will be finite so at the subsequent improvement step, a proper policy will be identified.

6.8 Linear programming in transient models

This section discusses the formulation and properties of linear programs in transient expected total reward models.

6.8.1 The primal linear program

If $v(s)$ satisfies the transient Bellman equation

$$v(s) = \max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S \setminus \Delta} p(j|s,a) v(j) \right\}$$

for all $s \in S \setminus \Delta$, then it satisfies

$$v(s) \geq \; r(s,a) + \sum_{j \in S \setminus \Delta} p(j|s,a) v(j) \tag{6.75}$$

for all $a \in A_s$ and $s \in S \setminus \Delta$[27]. By a generalization of Corollary 5.2 in the case of discounted models, any $v(s)$ satisfying (6.75) for all $a \in A_s$ and $s \in S \setminus \Delta$ is an upper bound on $v^*(s)$. Hence a solution of the transient Bellman equation may be thought of as the *smallest* (in a component-wise sense) $v(s)$ that satisfies (6.75) for all $a \in A_s$ and $s \in S \setminus \Delta$. To find such a value function, choose $\alpha(s) > 0$ for all $s \in S \setminus \Delta$ and solve the following linear program.

Primal LP formulation of a transient model

$$\text{minimize} \quad \sum_{s \in S \setminus \Delta} \alpha(s) v(s) \tag{6.76a}$$

$$\text{subject to} \quad v(s) - \sum_{j \in S \setminus \Delta} p(j|s,a) v(j) \geq r(s,a), \quad a \in A_s, \; s \in S \setminus \Delta \tag{6.76b}$$

Because the dual variables have interesting properties the dual linear program is analyzed next.

6.8.2 The dual linear program

The corresponding dual linear program may be written as:

[27] Recall that in a transient model $v(\Delta) = 0$ so that term in the summation in the Bellman equation can be removed.

Dual LP formulation of a transient model

$$\text{maximize} \quad \sum_{s\in S\setminus\Delta}\sum_{a\in A_s} r(s,a)x(s,a) \tag{6.77a}$$

$$\text{subject to} \quad \sum_{a\in A_s} x(s,a) - \sum_{j\in S\setminus\Delta}\sum_{a\in A_j} p(s|j,a)x(j,a) = \alpha(s), \quad s\in S\setminus\Delta \tag{6.77b}$$

$$x(s,a)\geq 0, \quad a\in A_s, s\in S\setminus\Delta \tag{6.77c}$$

Using the same approach as in the discounted model (see Theorem 5.23), Theorem 6.17 shows that there is a one-to-one relationship between:

1. feasible solutions to the dual problem and randomized stationary policies, and
2. *basic* feasible solutions to the dual problem and deterministic stationary policies.

Theorem 6.17. Suppose $\alpha(s) > 0$ for all $s \in S \setminus \Delta$.

1. For each $d \in D^{\text{MR}}$

$$x_d(s,a) := \sum_{j\in S\setminus\Delta} \alpha(j) \sum_{n=1}^{\infty} p^{d^\infty}(X_n = s, Y_n = a \,|\, X_1 = j) \tag{6.78}$$

   is a feasible solution to the dual linear program. That is, it satisfies the constraints (6.77b) and (6.77c).

2. Let $x(s,a)$ denote a feasible solution to the dual linear program. Let $d_{\mathbf{x}}$ denote the randomized decision rule that selects action $a \in A_s$ in state $s \in S \setminus \Delta$ with probability

$$w_{d_{\mathbf{x}}}(a|s) := \frac{x(s,a)}{\sum_{a\in A_s} x(s,a)}. \tag{6.79}$$

   Then $x_{d_{\mathbf{x}}}(s,a)$ defined by (6.78) is a feasible solution to the dual linear program and $x_{d_{\mathbf{x}}}(s,a) = x(s,a)$ for all $a \in A_s$, $s \in S \setminus \Delta$.

Noting that there are $|S-1|$ dual constraints[28] and assuming $\alpha(s) > 0$, for each $s \in S \setminus \Delta$, a basic feasible solution has $x(s,a') > 0$ for exactly one $a' \in A_s$.

[28] Not including the non-negativity constraints.

Corollary 6.8. Suppose $\alpha(s) > 0$ for all $s \in S \setminus \Delta$. Then

1. For each $d \in D^{\text{MD}}$ and $s \in S \setminus \Delta$

$$x_d(s,a) := \begin{cases} \sum_{j \in S\setminus\Delta} \alpha(j) \sum_{n=1}^{\infty} P^{d^\infty}[X_n = s, Y_n = a \,|\, X_1 = j] & \text{for} \quad a = d(s) \\ 0 & \text{for} \quad a \neq d(s) \end{cases} \tag{6.80}$$

   is a basic feasible solution of the dual linear program.

2. Let $\mathbf{x}$ be a basic feasible solution to the dual linear program. Then $d_{\mathbf{x}} \in D^{\text{MD}}$.

From these results it follows that:

Corollary 6.9. There exists an optimal basic feasible solution $x^*(s,a)$ to the dual linear program and an optimal stationary deterministic policy $d^\infty_{\mathbf{x}^*}$ where $d_{\mathbf{x}^*}(s) = a^*_s$ if $x^*(s, a^*_s) > 0$.

Observe that when $\sum_{s\in S\setminus\Delta} \alpha(s) = 1$, the expression on the right hand side of (6.78) represents the expected number of times (under policy d^∞) that the system with initial state chosen with probability $\alpha(s)$ visits state s and chooses action a[29]. Hence,

$$\sum_{s\in S\setminus\Delta} \sum_{a\in A_s} r(s,a) x_d(s,a) = \sum_{s\in S\setminus\Delta} \alpha(s) v^{d^\infty}(s). \tag{6.81}$$

This means that when $\sum_{s\in S\setminus\Delta} \alpha(s) = 1$, the objective function value when $x(s,a) = x_d(s,a)$ is the expected total reward corresponding to policy d^∞ averaged with respect to initial distribution $\alpha(s)$. An alternative derivation of equation (6.81) follows from strong duality of linear programming.

6.8.3 Examples

This section provides examples of the application of the above LP to a transient and SSP model.

[29] To see this note that

$$\sum_{n=1}^{\infty} P^{d^\infty}[X_n = s, Y_n = a \,|\, X_1 = j] = E\left[\sum_{n=1}^{\infty} I_{\{X_n = s, Y_n = a\}} \,\middle|\, X_1 = j\right].$$

Example 6.17. A simple transient example.
Suppose $S = \{s', \Delta\}$ and $A_{s'} = \{a_1, a_2\}$ with $r(s', a_1) = 5$ and $p(s'|s', a_1) = 0.2$ and $r(s', a_2) = 3$ and $p(s'|s', a_2) = 0.5$.

Setting $\alpha(s') = 1$ yields the primal linear program

$$\begin{aligned} \text{minimize} \quad & v(s') \\ \text{subject to} \quad & v(s') - 0.2v(s') \geq 5, \\ & v(s') - 0.5v(s') \geq 3. \end{aligned}$$

Note that the constraints can be rewritten as $v(s') \geq 6.25$ and $v(s') \geq 6$, respectively. Therefore the solution is $v^*(s') = 6.25$. Since equality holds in the first constraint, the optimal decision rule is $d^*(s') = a_1$.

The dual formulation is

$$\begin{aligned} \text{maximize} \quad & 5x(s', a_1) + 3x(s', a_2) \\ \text{subject to} \quad & x(s', a_1) + x(s', a_2) - 0.2x(s', a_1) - 0.5x(s', a_2) = 1 \\ & x(s', a_1) \geq 0 \text{ and } x(s', a_2) \geq 0. \end{aligned}$$

The equality constraint can be written as $0.8x(s', a_1) + 0.5x(s', a_2) = 1$. There are two basic feasible solutions: $\mathbf{x}^1 = (x^1(s', a_1), x^1(s', a_2)) = (1.25, 0)$ and $\mathbf{x}^2 = (x^2(s', a_1), x^2(s', a_2)) = (0, 2)$. Clearly, $\mathbf{x}^1$ is optimal with an objective function value of 6.25, in agreement with the solution of the primal linear program. Hence the deterministic stationary policy derived from $d^*(s') = a_1$ is optimal and under this policy, on average, the system spends 1.25 decision epochs in state s' prior to absorption in Δ.

Example 6.18. An SSP model

Consider the SSP model depicted in Figure 6.5 with $S = \{s_1, s_2, \Delta\}$, $A_{s_1} = \{a_{1,1}, a_{1,2}\}$ and $A_{s_2} = \{a_{2,1}, a_{2,2}\}$, with rewards $r(s_1, a_{1,1}) = -3$, $r(s_1, a_{1,2}) = 1$, $r(s_2, a_{2,1}) = 1$ and $r(s_2, a_{2,2}) = -2$, and non-zero transition probabilities $p(s_1|s_1, a_{1,1}) = p(s_2|s_1, a_{1,1}) = 0.5$, $p(s_2|s_1, a_{1,2}) = 1$, $p(\Delta|s_2, a_{2,1}) = 1$ and $p(s_1|s_2, a_{2,2}) = 1$.

Observe that the model is *not* transient since the stationary policy based on the decision rule $d'(s_1) = a_{1,2}, d'(s_2) = a_{2,2}$ is improper as is the stationary policy based on $d''(s_1) = a_{1,1}$ and $d''(s_2) = a_{2,2,}$. However it is a positive model.

Taking $\alpha(s_1) = \alpha(s_2) = 0.5$, the primal linear program becomes

$$\begin{aligned} \text{minimize} \quad & 0.5v(s_1) + 0.5v(s_2) \\ \text{subject to} \quad & v(s_1) - 0.5v(s_1) - 0.5v(s_2) \geq -3, \\ & v(s_1) - v(s_2) \geq 1, \end{aligned}$$

$$v(s_2) \geq 1,$$
$$-v(s_1) + v(s_2) \geq -2.$$

The corresponding dual linear program is

$$\begin{aligned}
\text{maximize} \quad & -3x(s_1, a_{1,1}) + x(s_1, a_{1,2}) + x(s_2, a_{2,1}) - 2x(s_2, a_{2,2}) \\
\text{subject to} \quad & x(s_1, a_{1,1}) + x(s_1, a_{1,2}) - 0.5x(s_1, a_{1,1}) - x(s_2, a_{2,2}) = 0.5 \\
& x(s_2, a_{2,1}) + x(s_2, a_{2,2}) - 0.5x(s_1, a_{1,1}) - x(s_1, a_{1,2}) = 0.5 \\
& x(s, a) \geq 0, \quad a \in A_s, s \in S \setminus \Delta.
\end{aligned}$$

Solving the dual yields $x(s_1, a_{1,2}) = 0.5$, $x(s_2, a_{2,1}) = 1$ with the remaining variables equal to 0. Hence the optimal policy d^* uses decision rule $d^*(s_1) = a_{1,2}$ and $d^*(s_2) = a_{2,1}$.

Note that the same policy is optimal for all values of $r(s_2, a_{2,1})$. On the other hand, when either $r(s_2, a_{2,2}) > -1$, $r(s_1, a_{1,2}) > 2$ or $r(s_1, a_{1,1}) > 1$, the primal linear program is infeasible[a]. In each of these cases the condition that the value of an improper policy equals $-\infty$ were violated so it is not an SSP model.

[a] You can easily confirm this by plotting the feasible region of the primal.

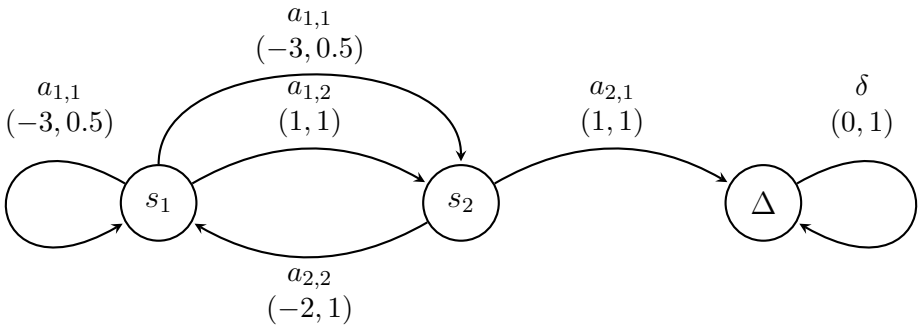


Figure 6.5: Symbolic representation of model in Example 6.18. The labels (r, p) refer, respectively, to the reward and transition probabilities associated with using the specified action in the given state.

6.9 Optimality of structured policies

This section extends results in Section 5.10 to expected total reward models in which value iteration is convergent.

6.9.1 The fundamental result

Define V^F to be a subset of V with a specific form (such as convex or monotone) and define D^F to be a subset D^{MD} in which all decision rules have a particular structure. They are *compatible* if $\mathbf{v} \in V^F$ implies that there exists a

$$d' \in \arg\operatorname*{c\text{-}max}_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \mathbf{P}_d \mathbf{v}\}$$

that is in D^F. For example in an optimal stopping problem, V^F might denote the set of non-decreasing functions and D^F might denote decision rules that stop after a threshold is reached.

Theorem 6.18. Suppose

1. V^F is non-empty,
2. D^F is non-empty and compatible with V^F,
3. $\mathbf{v} \in V^F$ implies $L\mathbf{v} \in V^F$, and
4. if $\lim_{n\to\infty} \|\mathbf{v}^n - \mathbf{v}^*\| = 0$ and $\mathbf{v}^n \in V^F$ for all $n = 0, 1, \ldots$ then $\mathbf{v}^* \in V^F$.

Then there exists an optimal stationary policy $(d^*)^\infty$ with $d^* \in D^F$.

The proof is identical to that of Theorem 5.30 and is omitted. Note that for hypothesis 4 to apply requires at a minimum the convergence of value iteration, which holds in all the models considered previously in this section.

6.10 Applications

6.10.1 Gridworld navigation

Example 6.11 focused on convergence of value iteration. In this example attention shifts to the properties and structure of solutions and how they are impacted by model parameters. Solution is straightforward and it is easy to solve the model by value iteration, policy iteration, modified policy iteration or linear programming. Investigating computational aspects of these approaches is left to the reader.

Consider Instance 1 of Example 6.11 with a reward of 50 units when successfully delivering the coffee, a cost of 200 units if the robot falls down the stairs and a cost per step of 1 unit. Recall that the only available actions are "up", "left" or "right", with not every action available in each cell.

When $p = 1$ this is a deterministic shortest path problem. Its optimal policy starting in cell 13 is

$$13 \to (10 \text{ or } 14) \to 11 \to 8 \to 5 \to 2 \to 1.$$

Both paths yield $v^*(13) = 50 - 6 = 44$. Moreover, solving this problem also provides the shortest path from each cell not on the shortest path to the office.

When $p = 0.95$, a decision rule that specifies action choice in each sate is required. However starting in cell 13, the "optimal path"[30] becomes

$$13 \to 14 \to 15 \to 12 \to 9 \to 6 \to 3 \to 2 \to 1.$$

For such a policy, the expected value starting in cell 13 is $v^*(13) = 40.96$. Observe that this policy is conservative in that it aims to take a longer path so as to avoid the costly cell 7. Moreover if the robot ever occupies cell 11, the optimal policy moves it to cell 12 for the same reason.

Note that when $p = 0.65$, the optimal policy remains the same as when $p = 0.95$ but $v^*(13) = -4.63$ implying that even under the optimal policy there is a non-negligible probability of falling down the stairs.

For small values of p such as $p = 0.2$, the robot's movement becomes unreliable and the optimal policy becomes quite strange. In such cases the robot must aim in the "wrong" direction so as to actually head in the right direction. The optimal policy when $p = 0.2$ appears in Table 6.4.

State	Intended cell
2	3
3	2
4	5
5	6
6	3
8	7
9	8
10	11
11	10
12	11
13	10
14	13
15	14

Table 6.4: State and optimal direction when $p = 0.2$.

Observe that in cell 8, the optimal policy aims the robot in the counter-intuitive direction of cell 7. This is because the probability it moves in some other direction is 0.8. Note that $v^*(13) = -150.26$ corresponding to the high probability of falling down the stairs when the robot is unreliable. The reader is encouraged to investigate whether the optimal value function is monotone as a function of p, particularly as p approaches 0, when it is possible to avoid falling down the stairs.

[30] That is, the path the robot follows if the implemented action results in a movement in the intended direction.

Interesting variants to consider include models with cell-dependent movement probabilities or a tilted grid where it is more likely to move in one direction than the other.

6.10.2 Optimal stopping

This section analyzes a stationary version of the optimal stopping problem (Section 3.8) in which the system moves probabilistically between states in S' until the decision maker decides to stop. Thus $S = S' \cup \{\Delta\}$ where Δ denotes the stopped state. For each state in S' the decision maker can either continue (action C) or stop (action Q). In state Δ, the only option is to continue (action C) and remain in that state and receive a reward of 0. Therefore the action sets may be written as

$$A_s = \begin{cases} \{C, Q\} & s \in S' \\ \{C\} & s = \Delta. \end{cases}$$

For simplicity assume continuing costs c and stopping in state $s' \in S$ generates revenue $g(s)$. Hence the reward function[31] is given by:

$$r(s,a) = \begin{cases} -c & s \in S', a = C \\ g(s) & s \in S', a = Q \\ 0 & s = \Delta, a = a_C. \end{cases}$$

Note that the reward $g(s)$ is only received upon stopping, at which time the system moves to the zero reward absorbing state Δ.

Assume that transitions are governed by an underlying $|S'| \times |S'|$ transition probability matrix $\mathbf{B}$ with components $b(j|s)$. Thus the transition probabilities for the corresponding Markov decision process can be expressed as

$$p(j|s,a) = \begin{cases} b(j|s) & s \in S', a = C, j \in S' \\ 1 & s \in S', a = Q, j = \Delta \text{ or } s = j = \Delta, a = C \\ 0 & \text{otherwise.} \end{cases}$$

Note that in this model the stationary policy that always continues, that is when $d'(s) := a_0$ for all $s \in S'$, has $v^{(d')^\infty}(s) = -\infty$ for all $s \in S'$.

Model classification

The optimal stopping model can be classified as follows:

1. If $g(s) \geq 0$ for all $s \in S'$, it is a positive model.
2. If $g(s) \leq 0$ for all $s \in S'$, it is a negative model.

[31] It would be more precise to formulate this model in terms of $r(s,a,j)$ especially $r(s,a_1,\Delta) = g(s)$, but since under a_1 the transition is deterministic, representing this by $r(s,a_1)$ suffices.

3. It is not a transient model since under $(d')^\infty$ the state Δ cannot be reached.

4. The model is an SSP model. When the underlying Markov chain forms a single closed class, the policy $(d')^\infty$ is improper and satisfies condition 3 in the definition of an SSP model. This holds also for other Markov chain structures but application of condition 3 is more subtle[32].

The Bellman equation

The Bellman equation in component notation can be expressed as:

$$v(s) = \max\left\{-c + \sum_{j\in S'} b(j|s)v(j), g(s) + v(\Delta)\right\}, \quad s \in S', \tag{6.85}$$

$$v(\Delta) = v(\Delta). \tag{6.86}$$

Since $v(\Delta) = 0$, the Bellman equation can be rewritten for $s \in S'$ as:

$$v(s) = \max\left\{-c + \sum_{j\in S'} b(j|s)v(j),\ g(s)\right\}. \tag{6.87}$$

It can be solved using any iterative algorithm or using the (primal) linear program:

$$\text{minimize} \quad \sum_{s\in S'} v(s) \tag{6.88}$$

$$\text{subject to} \quad v(s) - \sum_{j\in S'} b(j|s)v(j) \geq c \tag{6.89}$$

$$v(s) \geq g(s) \tag{6.90}$$

Let $v^*(s)$ for $s \in S'$ denote the optimal value function and d^* denote a decision rule corresponding to the optimal stationary policy. Then it follows from the primal linear program that

$$d^*(s) = \begin{cases} Q & \text{if} \quad v^*(s) = g(s) \\ C & \text{if} \quad v^*(s) > g(s). \end{cases}$$

Since $(d^*)^\infty$ must stop in at least one state in S' (why?), $\mathbf{P}_{d^*}^N$ must converge to a matrix that has entries equal to zero in the first $|S'|$ columns and ones in the column corresponding to Δ. Since $v^*(\Delta) = 0$, (6.36) holds. Therefore it follows from Theorem 6.5 that the policy d^* is optimal.

[32] To see this, consider the case of of Markov chain with two closed classes and some transient states.

Optimal stopping on a random walk

The following example applies the above results to the problem of optimal stopping in a finite random walk on the integers with reflecting barriers at 1 and N.

Example 6.19. Let $S = \{1, \ldots, N\}$, c be the continuation cost, $g(s) = \alpha s^2$ and p denote the probability of a transition from s to $s+1$, except in state N where p is the probability of remaining in state N. Let $q = 1 - p$, denote the probability of a transition from state s to $s-1$, except in state 1 where it denotes the probability of staying there. Thus the (transient) Bellman equation (6.87) is given by:

$$\begin{aligned} v(1) &= \max\{-c + qv(1) + pv(2),\ g(1)\}, \\ v(s) &= \max\{-c + qv(s-1) + pv(s+1),\ g(s)\}, \quad \text{for} \quad s = 2, \ldots, N-1 \\ v(N) &= \max\{-c + qv(N-1) + pv(N),\ g(N)\}. \end{aligned}$$

The decision maker trades off the cost of continuing versus the cost of reaching the high reward states. When $\alpha > 0$ these are states with large values of s and when $\alpha < 0$ they correspond to small values of s.

Since this is an SSP model, it can be solved numerically using value iteration[a] and a sup-norm stopping criterion with $\epsilon = 0.000001$. For simplicity, initialize computation with $v(s) = 0$ for all $s \in S$.

Consider the following instances:

Instance 1: $N = 25$, $\alpha = 0.2$, $p = 0.65$ and $c = 2$,

Instance 2: $N = 25$, $\alpha = 0.2$, $p = 0.5$ and $c = 2$,

Instance 3: $N = 500$, $\alpha = 0.05$, $p = 0.65$ and $c = 10$,

Instance 4: $N = 500$, $\alpha = -0.05$, $p = 0.65$ and $c = 10$,

Instance 5: $N = 500$, $\alpha = -0.05$, $p = 0.35$ and $c = 10$.

Note that Instances 1-3 represent reward maximization problems and Instances 4-5 represent cost minimization problems. The parameters were chosen so as to obtain interesting policies.

Instance	Iterations	Continuation region
1	249	$\{8, \ldots, 24\}$
2	2	none
3	1777	$\{299, \ldots, 499\}$
4	1287	$\{500\}$
5	1677	$\{334, \ldots, 500\}$

Table 6.5: Computational results for Example 6.19.

Table 6.5 provides the number of iterations to converge and the *continuation region* in which the optimal policy chooses the action a_0 for each instance. It shows that:

1. Value iteration converged in all instances.
2. In all instances $v^*(s)$ was a monotone function of s.
3. With the exception of Instance 2 in which $v^*(s) = g(s)$ for $S = 1, \ldots, 25$, and Instance 4 where the continuation region is a singleton, the continuation region is a set of consecutive states.
4. In Instance 2, value iteration converges in 2 iterations with iterates $v^n(s) = g(s)$ for all $n \geq 1$.
5. In Instances 1 and 3, the optimal policy stops for s less than some threshold (7 and 298, respectively) and when $s = N$.
6. In Instance 4 when $g(s) < 0$ for all $s \in S$, and there is a probability of 0.65 of moving to a higher cost state, it is better to stop unless the system is in the highest cost state.
7. In Instance 5, in which the system moves to a lower cost state with probability 0.65, it is optimal to continue in high cost states.

We hypothesize that one can prove analytically that when $g(s)$ is monotone and p is constant, then $v^*(s)$ is monotone. Hence under further conditions on $g(s)$, one can prove that the continuation region is either empty, or a set of consecutive states. This is left as an exercise.

[a] This is the easiest to code and converges quickly for small N.

Selling an asset*

Consider a stationary version of the asset selling problem in Section 3.8.2. Assume you are trying to sell an asset such as a home. Offers arrive daily and you can either accept the best offer from that day (action Q) or wait another day in anticipation of better offers (action S). If you do not accept the current best offer, it is withdrawn at the end of the day.

Let the random variable X denote the offer amount which is assumed to be discrete and independent between days. Let $b(n) := P[X = n]$, $0 \leq n \leq N$, for some N and assume a cost c for holding the asset for an additional day.

Consider first, a related problem, referred to as the *modified asset selling problem*, in which all past offers remain available in perpetuity. In it, $S = S' \cup \{\Delta\}$, $S' = \{0, \ldots, N\}$ and Δ denotes the stopped state. The set of actions $A_s = \{C, Q\}$ for $s \in S'$ and

$A_\Delta = \{C\}$. Transition probabilities satisfy $p(\Delta|s, Q) = 1$ for all $s \in S'$, $p(\Delta|\Delta, C) = 1$ and

$$p(j|s, a_0) = \begin{cases} 0 & 0 \leq j < s \\ \sum_{k \leq s} p(k) & j = s \\ p(j) & s < j \leq N. \end{cases}$$

Rewards satisfy

$$r(s, a) = \begin{cases} -c & s \in S', a = C \\ s & s \in S', a = Q \\ 0 & s = \Delta, a = C. \end{cases}$$

Again, this can be classified as a positive or SSP model. The (transient) Bellman equation can be written[33] as

$$v(s) = \max \left\{ s, -c + v(s) \sum_{j=0}^{s} p(j) + \sum_{j=s+1}^{N} p(j)v(j) \right\}. \tag{6.91}$$

Instead of solving this equation numerically, the following theorem provides a relationship that can be used to easily compute the optimal policy. Its proof uses policy iteration. Let $\mu > 0$ denote the expected offer size, that is,

$$\mu = \sum_{j=1}^{N} jp(j).$$

Moreover, define

$$F(s) := \sum_{j=s+1}^{N} (j - s)p(j)$$

for $s = 0, \ldots, N$. Then, it is easy to see that $F(0) = \mu$, $F(N) = 0$ and $F(s)$ is non-increasing in s.

[33] Note typo in the Bellman equation on p. 307 in Puterman [1994]. This typo could have been avoided by carefully writing out the transition probabilities.

Theorem 6.19. Let

$$s^* = \min\{s \in \{0, 1, \ldots, N\} \mid F(s) < c\}. \tag{6.92}$$

Then the stationary policy $(d^*)^\infty$ where[a]

$$d^*(s) = \begin{cases} C & s < s^* \\ Q & s \geq s^* \end{cases} \tag{6.93}$$

is an optimal policy for the modified asset selling problem. Moreover, if $\mu < c$, $s^* = 0$ and $d^*(s) = Q$. That is, it is optimal to accept the first offer.

[a] If $s^* = 0$, the following decision rule never chooses C.

Proof. Begin policy iteration with $d(s) = Q$ for all $s \in S'$. Then $v(s) = s$ for all $s \in S'$. The improvement step chooses a $d'(s)$ as

$$\begin{aligned} d'(s) &\in \arg\max \left\{ s, -c + s\sum_{j=0}^{s} p(j) + \sum_{j=s+1}^{N} jp(j) \right\} \\ &= \arg\max \left\{ s, -c + s + \sum_{j=s+1}^{N} (j-s)p(j) \right\} = \arg\max \{0, -c + F(s)\}, \end{aligned}$$

where the first equality follows by adding and subtracting $s\sum_{j=s+1}^{N} p(j)$ in the second expression. Therefore

$$d'(s) = \begin{cases} a_1 & \text{if} \quad F(s) - c < 0 \\ \{a_0, a_1\} & \text{if} \quad F(s) - c = 0 \\ a_0 & \text{if} \quad F(s) - c > 0. \end{cases}$$

Hence $d'(s)$ has the form (6.93).

Let $d''(s)$ denote the decision rule chosen at the next iteration of policy iteration. When $\mu < c$, $s^* = 0$ and $d''(s) = a_1$ for all $s \in S'$. Hence policy iteration terminates because $d''(s) = d'(s)$ for all $s \in S$.

Now assume $\mu \geq c$. Consequently $s^* > 0$, so it follows without further computation that $v'(s) := v^{(d')^\infty}(s) = s$ for $s \geq s^*$ and $v'(s) > s$ for $s < s^*$ (Why?).

Now assume $s \geq s^*$. Then

$$\begin{aligned} &-c + v'(s)\sum_{j=0}^{s} p(j) + \sum_{j=s+1}^{N} v'(j)p(j) = -c + s\sum_{j=0}^{s} p(j) + \sum_{j=s+1}^{N} jp(j) \\ &= -c + \sum_{j=s+1}^{N} (j-s)p(j) + s = -c + F(s) + s < s, \end{aligned}$$

where the last inequality follows from the definition of s^*. Therefore $d''(s) = Q = d'(s)$ for all $s \geq s^*$.

Now assume $s < s^*$. Then

$$
\begin{aligned}
&-c + v'(s)\sum_{j=0}^{s} p(j) + \sum_{j=s+1}^{N} v'(j)p(j) \\
&= -c + v'(s)\sum_{j=0}^{s} p(j) + \sum_{j=s+1}^{s^*} v'(j)p(j) + \sum_{j=s^*+1}^{N} v'(j)p(j) \\
&> -c + s\sum_{j=0}^{s} p(j) + \sum_{j=s+1}^{s^*} jp(j) + \sum_{j=s^*+1}^{N} v'(j)p(j) \\
&= -c + s\sum_{j=0}^{s} p(j) + \sum_{j=s+1}^{s^*} jp(j) + \sum_{j=s^*+1}^{N} jp(j) \\
&= -c + \sum_{j=s+1}^{N} (j-s)p(j) + s = -c + F(s) + s > s,
\end{aligned}
$$

where the last inequality follows from the definition of s^*. Therefore $d''(s) = C = d'(s)$. Hence policy iteration terminates and the stationary policy derived from d' is optimal. □

Since the above policy is admissible in the (unmodified) asset selling problem and the value of the modified asset selling problem is necessarily greater than or equal to the value of the (unmodified) asset selling problem, it follows that:

Corollary 6.10. The stationary policy derived from the decision rule $d^*(s)$ defined by equation (6.93) is optimal in the (unmodified) asset selling problem.

Note this result extends to continuous distributions and those on $0, 1, 2 \ldots$ but it requires extension of underlying theory to continuous and countable state spaces, respectively.

To see how this works in practice consider an example in which the offer size is generated from a truncated (at $N = 20$) Poisson with parameter 10. Since $F(8) = 2.44$ and $F(9) = 1.78$ when $c = 2$, $s^* = 9$. Moreover, it follows from Figure 6.6 that if $c > 9.98$ (the mean of the truncated Poisson), it would be optimal to stop after receiving the first offer and if $c = 0$ it would be optimal to continue until receiving the maximum offer.

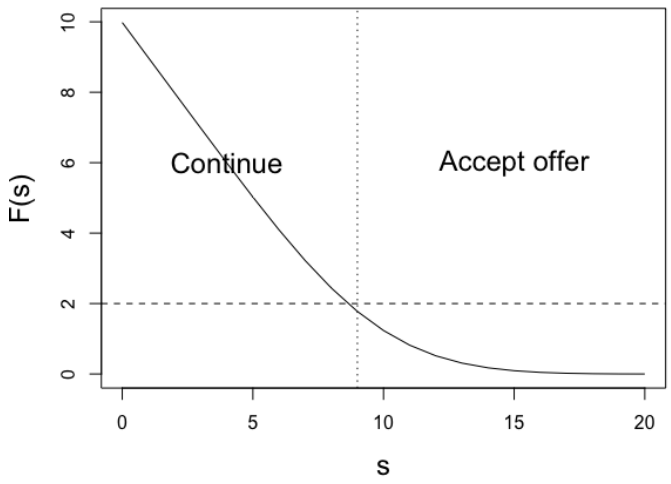


Figure 6.6: Plot of $F(s)$ vs. s in asset selling problem when offers follow a truncated Poisson distribution with parameter 10. The dashed horizontal line corresponds to $c = 2$ and the dotted vertical line to $s^* = 9$.

Bibliographic remarks

This chapter analyzes finite state and action expected total reward models focusing primarily on transient and SSP models, which arise naturally in modern applications. On the other hand, Chapter 7 in Puterman [1994] considers countable state models and restricts attention to positive and negative models.

Foundational papers include Strauch [1966] (negative models), Blackwell [1967] (positive models), and Veinott [1969a] (transient models).

Transient models were first referred to as *optimal first passage problems* and studied by Eaton and Zadeh [1962] and Derman [1970]. Bertsekas and Tsitsiklis [1991] extended them to include stochastic shortest path models and provide a rich bibliographic background.

The definition of a transient model in Veinott [1969a] differs from ours in that he requires at least one row sum of $\mathbf{P}_d$ be strictly less than 1. Thus he does not explicitly require the existence of a single reward-free absorbing state. Our use of the Euclidean norm to analyze transient models does not seem to appear in the literature. Wessels [1977] introduced the use of the weighted supremum norm for analyzing value iteration in expected total reward models.

The discussion of optimal stopping is based on Çinlar [1975] and analysis of the asset selling problem follows Karlin [1962]. Puterman [1994] analyzes an optimal parking problem where it is formulated as a countable state model and Ross [1983] analyzes an application in gambling. In the exercises, the tennis serving problem is adapted from

Norman [1985], the tennis handicapping problem is adapted from Chan and Singal [2016], and the darts handicapping problem is adapted from Chan et al. [2024]

Exercises

1. Consider the following deterministic expected total reward model with $S = \{s_1, s_2, s_3\}$; $A_{s_1} = \{a_{1,1}\}$, $A_{s_2} = \{a_{2,1,}, a_{2,2}\}$ and $A_{s_3} = \{a_{3,1}\}$; $r(s_1, a_{1,1}) = 1$, $r(s_2, a_{2,1}) = 1$, $r(s_2, a_{2,2}) = -3$, $r(s_3, a_{3,1}) = 0$ and $p(s_2|s_1, a_{1,1}) = 1$, $p(s_1|s_2, a_{2,1}) = 1$, $p(s_3|s_2, a_{2,2}) = 1$, $p(s_3|s_3, a_{3,1}) = 1$.

   (a) Represent the model graphically.

   (b) Classify the model.

   (c) Find the value of both deterministic stationary policies. What is $\mathbf{v}^*$ in each case?

   (d) Write out and solve the Bellman equation.

2. **Value iteration in a transient model.** Consider a stationary policy d^∞ in a transient model on $S = \{s_1, s_2, s_3, \Delta\}$ in which

$$\mathbf{P}_d = \begin{bmatrix} 0.3 & 0.4 & 0.3 & 0 \\ 0.8 & 0.2 & 0 & 0 \\ 0.2 & 0.2 & 0.2 & 0.4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{r}_d = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}.$$

   Let $\mathbf{Q}_d$ denote the 3×3 sub-matrix of $\mathbf{P}_d$ restricted to the transient states $\{s_1, s_2, s_3\}$.

   (a) Verify that the model is transient.

   (b) Compute the max norm and the Euclidean norm (spectral radius) of $\mathbf{Q}_d$.

   (c) Show that for some N, all row sums of $\mathbf{Q}_d^N$ are strictly less than 1 and compute the norm of $\mathbf{Q}_d^N$.

   (d) Find the value of this policy using value iteration and interpret the result (see Section 6.5.2).

   (e) Compare the rate of convergence with respect to the span, the max norm and the Euclidean norm and describe the results.

3. Prove that in a transient model:

   (a) (Easy) For any deterministic stationary policy d^∞, L_d is a contraction mapping in the Euclidean norm.

   (b) (Harder) L_T is a contraction mapping in the Euclidean norm. Hint: Complete the following steps.

i. Use the definition of the Euclidean norm to write out $\|L_T\mathbf{v} - L_T\mathbf{u}\|_2^2$.

ii. Partition S into subsets S_1 in which $L_T\mathbf{v}(s) \geq L_T\mathbf{u}(s)$ and S_2 where $L_T\mathbf{v}(s) < L_T\mathbf{u}(s)$.

iii. Define a decision rule d' that attains $L_T\mathbf{v}$ on S_1 and $L_T\mathbf{u}$ on S_2.

iv. Show that for each $s \in S$

$$(L_T\mathbf{v}(s) - L_T\mathbf{u}(s))^2 \leq (L_{d'}\mathbf{v}(s) - L_{d'}\mathbf{u}(s))^2$$

v. Apply Part 1.

4. Show that if value iteration converges to $\mathbf{v}^*$ with $\mathbf{v}^0 = \mathbf{0}$, it converges to $\mathbf{v}^* + c\mathbf{e}$ if $\mathbf{v}^0 = c\mathbf{e}$.

5. Prove that if two Markov decision processes are positively similar, then they have the same sets of transient policies, they have the same optimal policy and value iteration converges at the same rate in each.

6. Give an example of a transition probability matrix in a transient model with norm greater than one and spectral radius less than one.

   (a) Find a positively similar model with transition probability matrix with norm less than one.

   (b) Show that value iteration converges in both the untransformed model and the transformed model for an arbitrary reward function.

   (c) What is the rate of convergence?

7. Consider a version of the Gridworld problem in Section 6.10.1 in which an additional column has been added to the right of grid displayed in Figure 3.3. Show how the optimal policy changes as a function of the probability, p, of successfully exercising the intended movement in the following two cases:

   (a) There is a penalty for falling down the stairs combined with a reward for successfully delivering the coffee;

   (b) The robot seeks to maximize the probability of successfully delivering the coffee.

   Solve the problems using value iteration and linear programming.

8. Formulate and solve the following variant of the Gridworld problem described in Section 3.2. Assume that the robot starts at the professor's office, needs to navigate to the coffee room, fill the coffee cup and then return to the professor's office. Assume in each cell that the robot can decide to move up, down, right or left. If a wall (or boundary) is in the way, an attempted move in that direction is unsuccessful and the robot remains where it is. Assume probabilistic transitions

in each state; if the robot attempts to move in a specific direction, then it executes that move with probability p and moves in another direction with probability $(1-p)/3$.

Note that to formulate and solve this model requires augmenting the state with the content of the coffee cup. Assume a reward of 50 for delivering a filled coffee cup, a penalty of 200 for falling down the stairs and a cost of one unit for each attempted move. Find a policy that maximizes the expected total reward with $p = 0.5, 0.9, 1$ and discuss how it varies as a function of p. Clearly state any assumptions made.

9. **The rational burglar.** Consider a Markov chain on $S = \{0, 1, \ldots, N\}$ with probability p of moving from s to $s+1$ and a probability of $1-p$ of moving from s to 0. Suppose in addition that states 0 and N are absorbing.

   (a) Formulate this as an optimal stopping problem in which the burglar receives a reward of $g(s) = s$ if he/she decides to stop in state s.

   (b) Why do you think this is called the burglar problem?

   (c) Classify the model.

   (d) Find an optimal policy numerically for $p = 0.01, 0.25, 0.5, 0.75, 0.99$ using policy iteration and linear programming.

   (e) Describe the structure of the optimal policy and how it varies with p.

   (f) *Formally prove the optimality of the structured policy.

10. **Sequential gambling model.** As long as you have some money to bet, you can repeatedly play a game of chance that has a win probability of p. If your fortune reaches 0 you can no longer play.

    Prior to each round of the game you may place a bet of b. If you win you receive $2b$ (the amount bet and an equal pay out) and if you lose, you receive 0. You must decide how much to bet in each round of the game given that your fortune can assume any (integer) value between 0 and N dollars.

    (a) Formulate this as an expected total reward model that seeks to develop a betting strategy that maximizes the probability of reaching N.

    (b) How is this related to the rational burglar problem and how is it different?

    (c) Classify this model.

    (d) Solve the model numerically for $N = 100$ and $p = 0.01, 0.25, 0.5, 0.75, 0.99$ and describe how the optimal strategy varies as a function of p.

    (e) *Hypothesize and prove the form of the optimal policy as a function of p.

11. Determine the optimal policy in the asset selling problem when the demand distribution is binomially and normally distributed.

12. **Lion hunting behavior.** Consider a simplified infinite horizon version of the lion hunting behavior model in Section 3.5 in which the lion seeks a hunting policy to maximize survival time. Assume a lion's energy capacity is $C = 30$ kgs, that a lion requires $d = 4$ kgs of energy per day to survive, and hunting requires 1 additional kg of energy per day. A successful hunt, which occurs with probability 0.25, yields 12 kgs of energy.

    (a) Formulate this as an expected total reward model in which the objective is to maximize the expected number of days of survival and the decision is whether to hunt or not.

    (b) Classify this model.

    (c) Provide the Bellman equation and transient Bellman equation.

    (d) Find and interpret the optimal policy and the expected survival time if the lion starts the planning horizon at its highest energy level.

    (e) Suppose there are two other species to hunt: one yields 7 kgs of energy with a catch probability of 0.5 and another which yields 20 kg of energy with a catch probability of 0.15. Reformulate and solve this more general model where the objective is to decide whether or not to hunt and if so, what species to hunt.

13. **Serving in Tennis.** Consider the problem of choosing the type of serve to use at each point in a game of tennis. Tennis scoring is described in Section 3.9.2. The salient issue is that whenever the game reaches *deuce*, the game continues until a player wins two points in a row[34]. Moreover at each point, if the first serve is "out", that is, it lands out of bounds, the server has a second serve. If the second serve is out, the server loses the point.

    From the perspective of this chapter, a tennis game must be analyzed using an infinite horizon expected total reward model because the length of the game is random.

    Consider the simplified situation where the server can use either a "fast" or "slow" serve. A fast serve lands in bounds with probability p_f and the server wins the point with probability q_f. Similarly a slow serve lands in bounds with probability p_s and the server wins the point with probability q_s. To be realistic, $q_f > q_s$ and $p_f < p_s$.

    The objective is choose a service strategy that maximizes the probability of winning the game as a function of the score and whether it is a first serve or second serve.

    (a) Formulate this problem as an expected total reward model.

[34] Deuce corresponds to a score of 3-3 in points in a game, or "40-40" in tennis jargon.

(b) Classify it.

(c) Provide the Bellman equation.

(d) Find an optimal policy numerically when $p_f = 0.4$, $p_s = 0.8$, $q_f = 0.9$ and $q_s = 0.5$.

(e) Characterize the optimal strategy as a function of the model parameters.

(f) Reformulate and solve the problem when the parameters depend on the game score. Use your knowledge of tennis (or empirical data) to determine realistic parameter choices.

14. **Handicapping in Tennis.** Consider the tennis handicapping example from Chapter 3.

(a) Provide the Bellman equation.

(b) Prove that the optimal value function is equal to the probability of the weaker player winning the match at the current state.

(c) Find an optimal policy numerically for a one-set match when $p_1 = 0.5$, $p_0 = 0.46$ and $B = 2$.

(d) With $p_1 = 0.45$ and $p_0 = 0.45$, determine the smallest value of B such that the weaker player's probability of winning the match is at least 0.5. Repeat this calculation with $p_1 = 0.6$ and $p_0 = 0.3$, comment on whether B increases or decreases, and explain why.

(e) Reformulate this problem accounting for "hot streaks". Suppose if the serving player wins the point their probability of winning the next point increases by $\epsilon > 0$. If the serving player loses, the probability of winning the point reverts to the original value.

15. **501 darts.** In 501 darts, players start with a score of 501. Darts thrown at a dart board are scored based on the region they land in and subtracted from their current score. The scoring regions are $\{1, 2, \ldots, 20\}$ (singles), $\{2, 4, \ldots, 40\}$ (doubles), $\{3, 6, \ldots, 60\}$ (triples), and $\{25, 50\}$ (single and double bullseye). Players take turns, with each turn comprising three dart throws. The first player to reach 0 wins, with the added restriction that the dart that takes the score to zero must be a double. If a player goes below zero in a given turn or ends with a score that is impossible to complete with a double (i.e., 1), then the turn ends and the player's score reverts to the score at the start of the turn. The objective is to choose a strategy that minimizes the number of turns until a player reaches zero.

(a) Formulate this problem as an infinite horizon expected total reward model, starting with the simpler model where each turn comprises a single dart.

(b) Classify it.

(c) Provide the Bellman equation.

(d) Repeat the above steps where each turn comprises three dart throws.

(e) Find an optimal policy numerically assuming that a player lands their dart in the region they are aiming for with probability 0.8, with the remaining 0.2 distributed equally among all regions that share a border with the target region, regardless of the size of those regions. This will require looking up an image of a dart board to see which regions are adjacent to which.

16. Prove in an optimal stopping problem on a random walk that when $g(s)$ is monotone and p is constant, that $v^*(s)$ is monotone and the continuation region is either empty, or a set of consecutive states.

17. Complete the details of the proof of Theorem 6.16.

18. Show by example that in an SSP, the evaluation equations will not have a solution when you choose an improper policy.

19. Show that a finite horizon model is a (transient) infinite horizon expected total reward model.

20. **Tetris.** Formulate the game of Tetris as an expected total reward Markov decision process in which the objective is to maximize the length of the game. Classify this model.

Chapter 7

Infinite Horizon Models: Long-Run Average Reward

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

> *The field of average cost optimization is a strange one. Counterexamples exist to almost all natural conjectures, yet these conjectures are the basis of a proper intuition and are valid if reformulated right or if natural conditions are imposed*[1].
>
> Peter Whittle, Mathematician and statistician, 1927-2021.

This chapter focuses on infinite horizon Markov decision processes with the long-run average reward criterion, referred to as *average reward models*. The average reward criterion applies to non-terminating systems, such as controlled queues, in which decisions are made frequently and in perpetuity. In such models neither discounted or expected total reward models are appropriate but for different reasons: discounting trades off recent decisions with future decisions while the expected total reward does not discriminate between policies.

Average reward models pose numerous analytical challenges, as noted by Whittle in the quote above. This is because the existence and structure of the average reward depends on the *limiting* properties[2] of underlying Markov chains. In contrast to discounted models in which Markov chain properties do not impact results, they are fundamental here. Challenges faced include:

[1] Whittle (1983), p. 118.

[2] Limiting properties refer to the behavior of large powers of transition probability matrices.

1. For stationary policies with periodic transition probabilities, limits need not exist due to oscillation of the powers of transition probability matrices.

2. In finite state models, limits needed to define the average reward for history-dependent and Markovian policies need not exist.

3. In countable state models, the limit of transition probabilities may exist but it need not be a transition probability matrix.

4. The form of the Bellman equation depends on whether the average reward is constant or state-dependent. This is determined by the class structure of underlying Markov chains.

In light of these challenges, this chapter will focus primarily on finite state and actions models that are aperiodic and have constant (state-independent) average reward, a point that will be emphasized throughout. The reader is encouraged to review Appendix B for background on Markov chains relevant for Markov decision process.

7.1 Preliminaries

This chapter assumes:

Stationary rewards: The (expected) reward function does not vary from epoch to epoch and does not depend on the subsequent state[3]. It will be written as $r(s, a)$, independent of n.

Bounded rewards: There is a finite W such that $|r(s, a)| \leq W$ for all $a \in A_s, s \in S$.

Stationary transition probabilities: The transition probabilities will be written $p(j|s, a)$, independent of n.

This chapter examines how the form of the average reward is influenced by the structural properties of Markov chains induced by stationary policies. Key concepts of Markov chains are explored and illustrated to provide the necessary foundation. Several constructs unique to Markov decision processes are introduced, including *partial Laurent expansions*, *bias vectors*, and *relative value vectors*, which are essential for deriving and determining properties of the Bellman equation. These constructs underlie the three primary algorithms discussed: value iteration, policy iteration, and linear programming. The analysis and application of Gauss-Seidel iteration, modified policy iteration, and action elimination are left to the reader.

[3]As in previous chapters, when the reward depends on the subsequent state, that is it is given by $r(s, a, j)$, it is replaced by the expected reward $r(s, a) = \sum_{j \in S} r(s, a, j)p(j|s, a)$.

7.2 The long-run average reward or gain

At first glance, one might consider defining the *long-run average reward* or *gain* of a policy $\pi \in \Pi^{\text{HR}}$ by

$$g^{\pi}(s) := \lim_{N\to\infty} \frac{1}{N} E^{\pi}\left[\sum_{n=1}^{N} r(X_n, Y_n)\middle| X_1 = s\right]. \tag{7.1}$$

It will be shown below that the limit in this definition exists for stationary policies, however, it need not exist for non-stationary policies. This necessitates a more robust definition of the long-run average reward of a policy.

7.2.1 The gain of a stationary policy

For a stationary policy $\pi = d^{\infty}$ where $d \in D^{\text{MR}}$, (7.1) becomes

$$g^{d^{\infty}}(s) = \lim_{N\to\infty} \frac{1}{N} E^{d^{\infty}}\left[\sum_{n=1}^{N} r(X_n, Y_n)\middle| X_1 = s\right]. \tag{7.2}$$

Recalling the notation

$$r_d(s) = \sum_{a\in A_s} w_d(a|s) r(s,a) \quad \text{and} \quad p_d(j|s) = \sum_{a\in A_s} w_d(a|s) p(j|s,a), \tag{7.3}$$

equation (7.2) can be expressed in vector notation as

$$\mathbf{g}^{d^{\infty}} = \lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} \mathbf{P}_d^{n-1} \mathbf{r}_d, \tag{7.4}$$

where $\mathbf{r}_d$ denotes a vector with components $r_d(s)$, $\mathbf{P}_d$ denotes the matrix with components $P_d(j|s)$, and $\mathbf{P}_d^0 = \mathbf{I}$.

Applying the first result of Theorem B.1 in Appendix B provides the following representation for the gain.

Theorem 7.1. Let S be finite and let $\pi = d^{\infty}$ where $d \in D^{\text{MR}}$. Then the limit in (7.1) exists and satisfies

$$\mathbf{g}^{d^{\infty}} = \mathbf{P}_d^* \mathbf{r}_d \tag{7.5}$$

where

$$\mathbf{P}_d^* := \lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} \mathbf{P}_d^{n-1}. \tag{7.6}$$

Note that when $\mathbf{P}_d^n$ converges to $\mathbf{P}_d^*$, which occurs for example when the underlying Markov chain is regular[4], the gain may be also regarded as the *steady state reward*. A fundamental result in Markov chain theory also establishes that for regular chains, $\mathbf{P}_d^n$ converges to $\mathbf{P}_d^*$ at an *exponential* rate.

7.2.2 The gain of a non-stationary policy need not exist

The following example[5] shows that the limit in (7.1) need not exist for history-dependent and non-stationary Markovian policies. The analysis is quite subtle and can be skipped, but the conclusion is important and justifies the need for a more robust definition of the gain of a policy.

Let $S = \{s_1, s_2\}$, $A_{s_i} = \{a_{i,1}, a_{i,2}\}$ for $i = 1, 2$, $p(s_1|s_1, a_{1,1}) = 1$, $p(s_2|s_1, a_{1,2}) = 1$, $p(s_1|s_2, a_{2,1}) = 1$ and $p(s_2|s_2, a_{2,2}) = 1$ and $r(s_1, a_{1,1}) = r(s_1, a_{1,2}) = 1$ and $r(s_2, a_{2,1}) = r(s_2, a_{2,2}) = 0$. (See Figure 7.1).

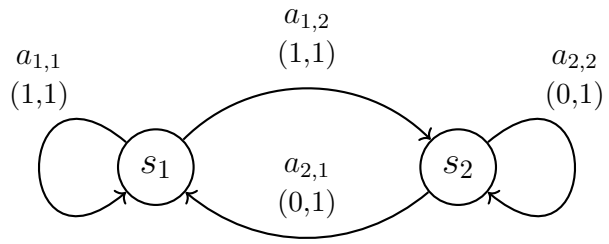


Figure 7.1: Symbolic representation of model in Section 7.2.2. The labels (r, p) refer to the reward and transition probabilities, respectively, associated with using the specified action in the given state.

A problematic non-stationary policy

Consider the deterministic history-dependent policy that when starting in state s_1 chooses the following sequence of actions:

$$a_{1,2}, a_{2,1}, a_{1,2}, a_{2,1}, a_{1,1}, a_{1,2}, a_{2,2}, a_{2,1}, a_{1,1}, a_{1,1}, a_{1,1}, a_{1,2}, a_{2,2}, a_{2,2}, a_{2,2}, a_{2,1}, \ldots$$

and generates the following sequence of rewards where parentheses correspond to blocks of length $k = 2, 2, 4, 8, 16, \ldots$ consisting of $k/2$ ones followed by $k/2$ zeroes.

$$(1, 0), (1, 0), (1, 1, 0, 0), (1, 1, 1, 1, 0, 0, 0, 0), \ldots.$$

[4]See Appendix B for definitions of relevant Markov chain concepts. Note this result holds for other aperiodic Markov chains as well.

[5]This elegant example and its analysis was adopted from Section A.4 in Sennott [1999]. Example 8.1.1 in Puterman [1994] provides a model with similar properties.

The corresponding sequence of averages w_n is given by

$$\left(1, \frac{1}{2}\right), \left(\frac{2}{3}, \frac{2}{4}\right), \left(\frac{3}{5}, \frac{4}{6}, \frac{4}{7}, \frac{4}{8}\right), \left(\frac{5}{9}, \frac{6}{10}, \frac{7}{11}, \frac{8}{12}, \frac{8}{13}, \frac{8}{14}, \frac{8}{15}, \frac{8}{16}\right), \ldots$$

Therefore

$$1/2 = \liminf_{n\to\infty} w_n < \limsup_{n\to\infty} w_n = 2/3$$

where the lim inf corresponds to the subsequence that chooses the last 0 in each block and the lim sup corresponds to the subsequence that chooses the last 1 in each block. Since a sequence converges when every subsequence has the same limit, this analysis shows that the limit in (7.1) **does not** exist for this history-dependent policy starting in state s_1.

When the system starts in state s_2, it chooses the actions

$$a_{2,2}, a_{2,1}, a_{1,2}, a_{2,1}, a_{1,1}, a_{1,2}, a_{2,2}, a_{2,1}, a_{1,1}, a_{1,1}, a_{1,1}, a_{1,2}, a_{2,2}, a_{2,2}, a_{2,2}, a_{2,1}, \ldots$$

that are identical to that in s_1 after the first decision epoch. For states not visited by this deterministic policy at a decision epoch, action choice is arbitrary and deterministic.

As a consequence of Lemma 5.9 for s_1 and s_2 there exist Markovian randomized policies that have the same state-action probabilities and generate the same sequence of rewards after the first decision epoch. Moreover, after defining $d_1(s_1) = a_{1,2}$ and $d_1(s_2) = a_{2,2}$ the Markovian policies are identical and deterministic with $d_2(s_2) = a_{2,1}$, $d_3(s_1) = a_{1,2}, \ldots$. Hence, in fact, there is a Markovian deterministic policy for which the limit in (7.1) does not exist.

Stationary policies

Now consider stationary policies. Simple calculations show:

action in s_1	action in s_2	gain in s_1	gain in s_2
$a_{1,1}$	$a_{2,2}$	1	0
$a_{1,1}$	$a_{2,1}$	1	1
$a_{1,2}$	$a_{2,2}$	0	0
$a_{1,2}$	$a_{2,1}$	1/2	1/2

Since the largest one-step reward is 1, the stationary policy, $(d^*)^\infty$ that uses $a_{1,1}$ in s_1 and $a_{2,1}$ in s_2 is optimal with respect to the average reward criterion.

Observe also that for the policy $d'(s_1) = a_{1,1}$ and $d'(s_2) = a_{2,2}$, the gain varies with the starting state. This is because this policy generates a Markov chain with two closed classes[6] .

[6]This issue will be discussed in the next section on model classification.

7.2.3 A more general definition of the gain

The above example illustrates the need for a more rigorous definition of the gain that as well applies to non-stationary policies. For any $\pi = (d_1, d_2, \ldots) \in \Pi^{\text{HR}}$, let

$$v_N^\pi(s) := E^\pi \left[\sum_{n=1}^{N} r(X_n, Y_n) \middle| X_1 = s \right]. \tag{7.7}$$

In vector notation,

$$\mathbf{v}_N^\pi = \sum_{n=1}^{N} \mathbf{P}_\pi^{n-1} \mathbf{r}_{d_n}, \tag{7.8}$$

where $\mathbf{P}_\pi^0 = \mathbf{I}$ and $\mathbf{P}_\pi^n = \mathbf{P}_{d_1} \mathbf{P}_{d_2} \cdots \mathbf{P}_{d_n}$.

Definition 7.1. For $\pi \in \Pi^{\text{HR}}$ define the *lim inf average reward* $g_-^\pi(s)$ and *lim sup average reward* $g_+^\pi(s)$ by:

$$g_-^\pi(s) := \liminf_{N\to\infty} \frac{1}{N} v_N^\pi(s) \quad \text{and} \quad g_+^\pi(s) := \limsup_{N\to\infty} \frac{1}{N} v_N^\pi(s) \tag{7.9}$$

for all $s \in S$.

Clearly $g_-^\pi(s) \le g_+^\pi(s)$. The example in the previous section shows that this inequality may be strict. When they are equal the limit in (7.1) exists, so that:

Definition 7.2. Whenever $g_-^\pi(s) = g_+^\pi(s)$ for all $s \in S$, the *average reward* of policy $\pi \in \Pi^{\text{HR}}$ is defined by:

$$g^\pi(s) := \lim_{N\to\infty} \frac{1}{N} v_N^\pi(s). \tag{7.10}$$

Applying Lemma 5.9, gives the following important result which simplifies subsequent analyses in this chapter in the same way as Theorem 5.1 did in Chapter 5. It says that given any history-dependent policy, there exists a Markov randomized policy with the same lim inf average reward and lim sup average reward.

Theorem 7.2. Let $\pi \in \Pi^{\text{HR}}$. Then for each $s \in S$, there exists a $\pi' \in \Pi^{\text{MR}}$ for which

$$g_-^{\pi'}(s) = g_-^{\pi}(s) \quad \text{and} \quad g_+^{\pi'}(s) = g_+^{\pi}(s). \tag{7.11}$$

Furthermore if $g_-^{\pi}(s) = g_+^{\pi}(s)$,

$$g^{\pi'}(s) = g^{\pi}(s). \tag{7.12}$$

7.2.4 Average optimality

It would be desirable to say that a policy π^* is optimal with respect to the average reward criterion whenever

$$g^{\pi^*}(s) \geq g^{\pi}(s) \tag{7.13}$$

for all $\pi \in \Pi^{\text{HR}}$ and $s \in S$. Unfortunately, as shown above, the limits implicit in defining the quantities in (7.13) need not exist, even when S is finite. Thus the expressions in (7.9) are needed to define optimality. (Recall the quote at the beginning of this chapter.)

Hence one must be content (at least until establishing the existence of optimal stationary policies) to adopt the following more rigorous notion of optimality.

Definition 7.3. A policy π^* is *average optimal* (*gain optimal*) if for all $s \in S$

$$g_-^{\pi^*}(s) \geq g_+^{\pi}(s) \tag{7.14}$$

for all $\pi \in \Pi^{\text{HR}}$.

What this definition means is that a policy is average optimal if its worst possible value (corresponding to its smallest limit point) is at least as large as the best possible value (corresponding to the largest limit point) of any other policy.

The example in Section 7.2.2 provides an illustration of this definition in action. That example showed that there exists a non-stationary Markovian policy π' for which

$$g_-^{\pi'}(s) = 1/2 \quad \text{and} \quad g_+^{\pi'}(s) = 2/3$$

for $s \in S$. Moreover from Theorem 7.1 for any stationary policy,

$$g^{d^\infty}(s) = g_+^{d^\infty}(s) = g_-^{d^\infty}(s).$$

Thus it follows for the stationary policy $(d^*)^\infty$ where $d^*(s_1) = a_{1,1}$ and $d^*(s_2) = a_{2,1}$ that:

$$g^{(d^*)^\infty}(s) = g_-^{(d^*)^\infty}(s) \geq g_+^{\pi'}(s)$$

for all $s \in S$. Noting that $g_+^{\pi}(s) \leq 1$ for all $\pi \in \Pi^{\text{HR}}$ and $s \in S$ and $g^{(d^*)^\infty}(s) = 1$ for all $s \in S$, confirms the optimality of d^* under this more general definition.

Optimal value functions

Defining optimal value functions is more subtle than in discounted and expected total reward models because as illustrated by the example in Section 7.2.2, the limit corresponding to the average reward of a policy need not exist. In light of this result, the *optimal value function* $g^*(s)$ is defined by[7]:

Definition 7.4. An *optimal value function* $g^*(s)$ satisfies

$$g^*(s) := \sup_{\pi \in \Pi^{\text{HR}}} g^\pi_-(s) = \sup_{\pi \in \Pi^{\text{HR}}} \liminf_{N \to \infty} \frac{1}{N} v^\pi_N(s) \tag{7.15}$$

This means that $g^*(s) \geq g^\pi_-(s)$ for all $\pi \in \Pi^{\text{HR}}$ so that if a policy attains this value, it has the best worst case behavior among all policies[8].

Later in this chapter, it will be shown that there exists a stationary deterministic policy that is optimal in certain average reward models so that

$$g^*(s) = \max_{d \in D^{\text{SD}}} g^{d^\infty}(s). \tag{7.16}$$

In this case, a policy π is average optimal whenever $g^\pi(s) = g^*(s)$ for all $s \in S$.

7.3 Chain structure

In contrast to discounted models, in average reward models transition properties of Markov chains generated by stationary policies impact the form of the average reward and the structure of the evaluation and Bellman equation. Material in Appendix B is fundamental here.

[7] Not this definition differs from that in Puterman [1994]

[8] In a minimization problem, define $g^*(s) := \inf_{\pi \in \Pi^{\text{HR}}} g^\pi_+(s)$.

Definition 7.5. A Markov decision process is said to be:

- *Regular* if the Markov chain corresponding to every Markovian deterministic decision rule is irreducible and aperiodic, in other words, consists of a single closed aperiodic class and no transient states.
- *Recurrent* or *ergodic* if the Markov chain corresponding to every Markovian deterministic decision rule consists of a single closed class and no transient states. Note that a regular model is a special case of a recurrent model in which every stationary deterministic policy generates an aperiodic Markov chain.
- *Unichain* if the Markov chain corresponding to every Markovian deterministic decision rule consists of a single closed class and the Markov chain corresponding to at least one decision rule contains a non-empty set of transient states[a].
- *Multi-chain* if the Markov chain corresponding to at least one Markovian deterministic decision rule contains two or more closed classes and possibly transient states[b].
- *Communicating* if for each pair of states s and j in S, there exists a Markovian deterministic decision rule d for which j is accessible from s, that is $p_d^n(j|s) > 0$ for some $n > 0$.
- *Weakly communicating* if the state space can be decomposed into two disjoint sets of states, one containing states that are transient under every deterministic decision rule and the other which forms a communicating Markov decision process[c].

[a] In Puterman (1994), a unichain model need not have any transient states in which case recurrent models and regular models are special cases. The definition here requires that at least one policy have transient states.

[b] This definition does not preclude the possibility of a model in which some decision rules generate Markov chains with a single closed class.

[c] Some authors refer to such a model as *weakly accessible*.

The first four categories depend on the chain structure of *every* Markovian deterministic decision rule. On the other hand, determining whether a model is communicating or weakly communicating requires checking whether every pair of states is accessible from each other under some policy. Note that regular and recurrent models are by definition communicating, a unichain model is necessarily weakly communicating and a multi-chain model may or may not be communicating or weakly communicating.

This classification scheme must be taken into consideration because it determines whether the value function is constant, as well as the appropriate form of the evaluation

equations and the Bellman equation. These relationships are summarized in Table 7.1.

Model class	Evaluation equations	Gain of a stationary policy	Gain of optimal policy
Regular or Recurrent	(7.43)	constant	constant
Unichain	(7.43)	constant	constant
Multi-chain	(7.35)	constant or non-constant	constant or non-constant
Communicating	(7.35)	constant or non-constant	constant
Weakly communicating	(7.35)	constant or non-constant	constant

Table 7.1: Relationship between chain structure and model features.

Communicating models

Note that in the definition of a communicating model, a different stationary deterministic decision rule may be required to reach states j and j' from s. The following result shows that this is not a limitation and moreover establishes the form of the optimal gain in a communicating model.

Proposition 7.1. In a communicating model:

1. There exists a randomized decision rule under which all pairs of states are accessible from each other.

2. The optimal average reward is constant.

Proof. The first part follows by noting that if a different decision rule is required to reach j and j' from state s, then randomizing between the action chosen by these two policies in state s generates a single randomized policy in which both j and j' are accessible from s.

To prove the second part, suppose there exists a stationary policy with closed[9] classes $C_1, C_2, \ldots, C_m$ with different average rewards on each. Assume further that C_1 has the greatest average reward g_1. From the first part of this proposition, there exists a randomized stationary policy under which there is a positive probability of reaching C_1 from each state in $C_2, \ldots, C_m$. Hence under this policy, all states in $C_2, \ldots, C_m$ are transient and have average reward g_1. □

Note that the presence of transient states requires only slight modification to the above proof to obtain the following corollary.

[9]Meaning that states in this class cannot be reached by states outside this class under the policy (see Appendix for formal definition).

Corollary 7.1. In a weakly communicating model, the optimal average reward is constant.

As noted in Table 7.1 communicating and weakly communicating models have constant gain but require the multi-chain evaluation equations, (7.35), because some policies may have two or more closed classes.

Chain structure examples

The following examples illustrate these properties.

Example 7.1. Consider the two-state model in Section 2.5. In it there are four deterministic decision rules. The Markov chain corresponding to decision rule $d(s_1) = a_{1,1}$ and $d(s_2) = a_{2,1}$ has a unichain transition matrix with state s_1 transient and state s_2 being a closed class (absorbing state). Also, all other deterministic decision rules correspond to regular Markov chains. Thus this model is unichain. Moreover it is easy to see that it is communicating as well.

A variant: Suppose a third action $a_{1,3}$ is added in s_1 with $p(s_1|s_1, a_{1,3}) = 1$, $p(s_2|s_1, a_{1,3}) = 0$ and arbitrary reward k (see Figure 7.2). Then as shown in Figure 7.3 the Markov chain corresponding to decision rule $d'(s_1) = a_{1,3}$ and $d'(s_2) = a_{2,1}$ has two closed classes. Hence with this additional action the model is multi-chain and communicating. This illustrates the interesting property that a model remains communicating when additional actions become available.

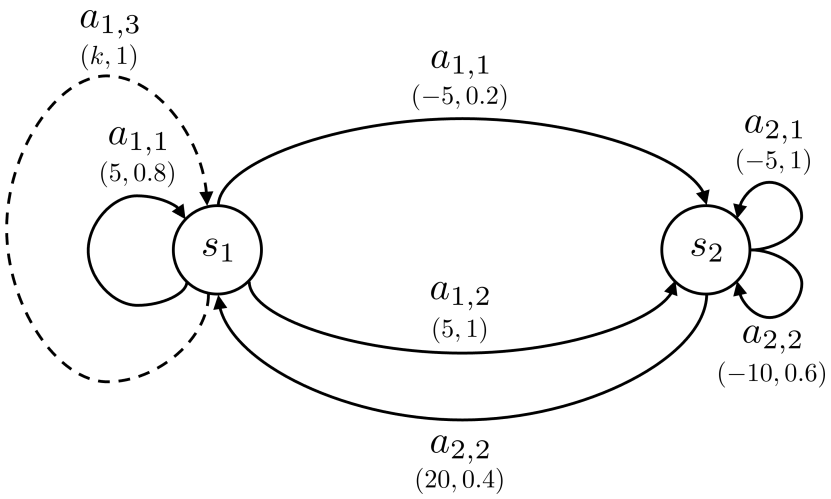
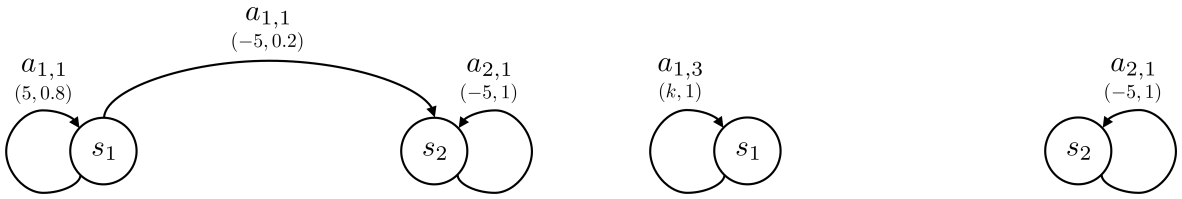


Figure 7.2: Model variant discussed in Example 7.1. The labels (r, p) refer to the reward and transition probabilities, respectively, associated with using the specified action in the given state.



(a) Chain structure under policy d^∞ (b) Chain structure under policy $(d')^\infty$

Figure 7.3: Symbolic representation of the two policies discussed in Example 7.1. Observe that under d^∞, s_1 is transient and s_2 is recurrent so that this policy generates a unichain model while under policy $(d')^\infty$, s_1 and s_2 each form closed classes so that the model is multi-chain. The labels (r, p) refer to the reward and transition probabilities, respectively, associated with using the specified action in the given state.

Example 7.2. This example considers a modified version of the Gridworld model in Section 3.2 in which the robot starts in the coffee room, cell 13, with a full cup of coffee and seeks to deliver it to the office, cell 1. In this model, $S = \{1, \ldots, 15\}$, where each state represents the location of the robot. Recall that actions specify an intended direction for the robot. Letting p denote the probability the robot moves in the intended direction and $(1-p)/(k-1)$ the probability it moves in each of the remaining $k-1$ directions including the possibility of remaining in the same cell.

In this model, cells 1 (deliver coffee) and 7 (fall down the stairs), each correspond to a closed class consisting of one absorbing state. When $p < 1$ the remaining set of states are transient under any decision rule since the robot reaches cells 1 or 7 with probability 1 under any stationary policy. When $p = 1$, decision rules can generate other closed classes. For example, by choosing the action R (right) in cell 2 and L (left) in cell 3 the robot will cycle between cells 2 and 3 forever and never reach cell 1 or cell 7. Moreover when $p = 1$ and it costs 1 unit per transition, then the long run average reward on this closed class is -1 while the average reward on any policy that reaches cells 1 or 7 is zero. Hence in either case the model is multi-chain but when $p < 1$, the only closed classes are $\{1\}$ and $\{7\}$.

A variant: Suppose, as shown in Figure 7.4, the model is augmented by adding an absorbing state Δ and an action a_Δ in states 1 and 7, which results in a transition to Δ with certainty so that $p(\Delta|1, a_\Delta) = p(\Delta|7, a_\Delta) = 1$. Moreover if the system reaches Δ it remains there forever and receives no reward. This can be represented by adding an action a' for which $p(\Delta|\Delta, a') = 1$. Moreover $r(1, a_\Delta, \Delta) = r(7, a_\Delta, \Delta) = r(\Delta, a', \Delta) = 0$. When $p < 1$, $\{\Delta\}$ is the only closed class, all other states are transient and the model is unichain.

When $p < 1$ in both the first formulation above and its variant, the average

reward of every policy is zero (since the robot reaches a zero-reward absorbing state), so the average reward criterion does not distinguish policies. The expected total reward[a], considered in the previous chapter, is a more appropriate optimality criterion for this model.

Another variant: Suppose instead of being absorbed in Δ, an action is added that loops to cell 13 after reaching cell 1 or 7. Then when $p < 1$, the model is regular. For the same reasons as above, when $p = 1$ the model is multi-chain and communicating.

The case of $p = 1$ can also be used to illustrate the construction in the proof of Proposition 7.1. Starting in cell 4, it is easy to see that a different policy is required to reach cells 1 and 6 but by randomizing action choice in cell 4 between the actions "Up" and "Right", the same randomized policy can be used to reach these two cells.

[a] In this model, it is equivalent to the *bias*, a concept defined below.

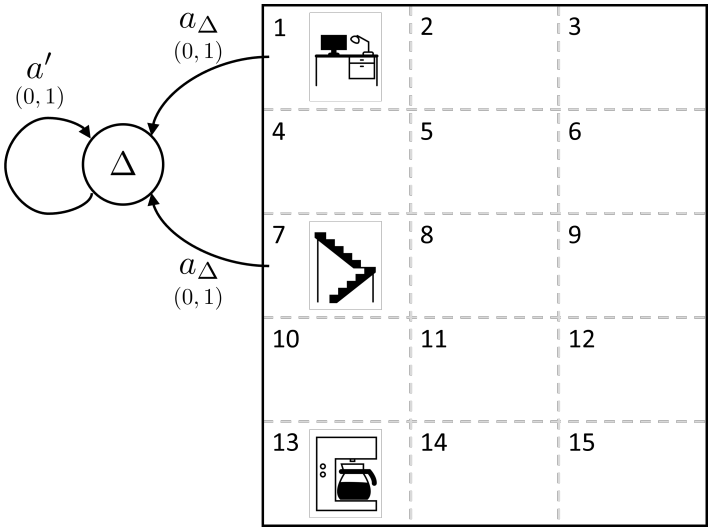


Figure 7.4: Variant of coffee delivering robot model with added absorbing state. The labels (r, p) refer to the reward and transition probabilities, respectively, associated with using the specified action in the given state.

In other common models, such as the queuing admission control model in Exercise 12, one may encounter real applications that are multi-chain and (weakly) communicating.

7.4 Bias of a stationary policy

In order to develop efficient algorithms to find average optimal policies, this section introduces an additional related quantity referred to as the *bias*. Definitions of the bias require distinguishing between two cases that will be described in separate subsections. The latter is an "edge case" that does not apply to most real applications and can be ignored when reading this for the first time.

1. **Aperiodic case:** When the sub-chains[10] corresponding to all closed classes are aperiodic.
2. **Periodic case:** When at least one closed class has a periodic sub-chain.

The aperiodic case

Definition 7.6. The *bias* of a stationary policy $d^\infty \in \Pi^{\text{SR}}$ in the aperiodic case is defined to be:

$$h^{d^\infty}(s) := E^{d^\infty}\left[\sum_{n=1}^{\infty}\left(r(X_n, Y_n) - g^{d^\infty}(X_n)\right)\middle| X_1 = s\right]. \tag{7.17}$$

In the above definition, the expectation of the infinite sum represents the limit of expectations of partial sums.

Equivalently, the bias of a stationary policy can be written in vector notation as

$$\mathbf{h}^{d^\infty} = \sum_{n=1}^{\infty}\left(\mathbf{P}_d^{n-1}\mathbf{r}_d - \mathbf{g}^{d^\infty}\right) = \sum_{n=1}^{\infty}\left(\mathbf{P}_d^{n-1} - \mathbf{P}_d^*\right)\mathbf{r}_d, \tag{7.18}$$

where the second equality follows from (7.5).

The following frequently used closed form representation for $\mathbf{h}^{d^\infty}$ holds in general, that is for **both** periodic and aperiodic models. It follows directly from Proposition B.1 in Appendix B:

[10] The expression *sub-chain* refers to a Markov chain on a closed class of the original Markov chain.

Theorem 7.3. Let $d \in D^{\text{MR}}$. Then

$$\mathbf{h}^{d^\infty} = \mathbf{H}_d \mathbf{r}_d, \tag{7.19}$$

where[a]

$$\mathbf{H}_d := (\mathbf{I} - (\mathbf{P}_d - \mathbf{P}_d^*))^{-1}(\mathbf{I} - \mathbf{P}_d^*).$$

[a]Note that $\mathbf{H}_d$ refers to the deviation matrix $\mathbf{H}$ introduced in Appendix B applied to the transition probability matrix $\mathbf{P}_d$.

Although Theorems 7.1 and 7.3 provide closed form representations for $\mathbf{g}^{d^\infty}$ and $\mathbf{h}^{d^\infty}$, they will not be used for computation except in simple examples. Instead, a system of equations will be derived that can be solved to find these quantities.

The periodic case*

Example B.4 describes a Markov chain where the following more general definitions are necessary:

Definition 7.7. The *bias* of a stationary policy $d^\infty \in \Pi^{\text{SR}}$ in the periodic case is defined to be:

$$h^{d^\infty}(s) := \lim_{N\to\infty} \frac{1}{N} \sum_{j=1}^{N} E^{d^\infty} \left[\sum_{n=1}^{j} \left(r(X_n, Y_n) - g^{d^\infty}(X_n) \right) \middle| X_1 = s \right]. \tag{7.20}$$

It is left to the reader to show that when the limit in (7.17) exists the two definitions are equivalent. However, when this limit does not exist, the second definition is necessary.

In the periodic case, when the limit in (7.18) does not exist, the equivalent matrix representation is

$$\mathbf{h}^{d^\infty} = \lim_{N\to\infty} \frac{1}{N} \sum_{j=1}^{N} \sum_{n=1}^{j} \left(\mathbf{P}_d^{n-1} \mathbf{r}_d - \mathbf{g}^{d^\infty} \right) = \lim_{N\to\infty} \frac{1}{N} \sum_{j=1}^{N} \sum_{n=1}^{j} \left(\mathbf{P}_d^{n-1} - \mathbf{P}_d^* \right) \mathbf{r}_d. \tag{7.21}$$

7.4.1 Interpreting the bias

The bias will be explained from three perspectives:

1. Directly from its definition.
2. In relation to the discounted reward as $\lambda \uparrow 1$.
3. In relation to expected total reward.

Using the definition of the bias

The nth term in the sum (7.17), $r(X_n, Y_n) - g^{d^\infty}(X_n)$, represents the difference in the reward received in period n and the gain. Hence the bias represents the expected total sum of these differences.

Turning to the matrix representation for the bias in (7.18), it follows that when $\mathbf{P}_d$ is regular, part 6 of Theorem B.1 implies that $\mathbf{P}_d^n - \mathbf{P}_d^*$ converges to $\mathbf{0}$ exponentially quickly, so that terms in the sum become very small quickly. Hence, the bias may be regarded as the total *initial* difference between the expected reward and the steady state reward. For this reason, the bias is sometimes referred to as the *transient reward*.

Example 7.3. Consider the stationary policy d^∞ where $d(s_1) = a_{1,1}$ and $d(s_2) = a_{2,2}$ in the model in Section 2.5. For this policy

$$\mathbf{P}_d = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} \quad \text{and} \quad \mathbf{r}_d = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.$$

Since $\mathbf{P}_d$ is regular, $\mathbf{P}^n \to \mathbf{P}_d^*$. Define the largest component difference between $\mathbf{P}^n$ and $\mathbf{P}_d^*$ by

$$\Delta_n := \max_{(s,j)\in S\times S} |p_d^n(j|s) - p_d^*(j|s)|.$$

Direct computation shows that $\Delta_2 = 0.1067$, $\Delta_3 = 0.0427$, $\Delta_4 = 0.0171$, ... indicative of the exponential convergence of $\mathbf{P}_d^n$ to $\mathbf{P}_d^*$. Because the eigenvalues of $\mathbf{P}_d$ are 1 and 0.4 the convergence is exponential at the rate of the second eigenvalue as noted in the Markov chain appendix.

Moreover,

$$\mathbf{P}_d^* = \begin{bmatrix} 2/3 & 1/3 \\ \\ 2/3 & 1/3 \end{bmatrix} \quad \text{and} \quad \mathbf{g}^{d^\infty} = \begin{bmatrix} 8/3 \\ \\ 8/3 \end{bmatrix}.$$

Observe that the rows of $\mathbf{P}_d^*$ are equal and so are the components of $\mathbf{g}^{d^\infty}$.

Evaluating the bias of d^∞ yields:

$$\mathbf{H}_d = \begin{bmatrix} 5/9 & -5/9 \\ \\ -10/9 & 10/9 \end{bmatrix} \quad \text{and} \quad \mathbf{h}^{d^\infty} = \mathbf{H}_d\mathbf{r}_d = \begin{bmatrix} 5/9 \\ \\ -10/9 \end{bmatrix}.$$

Let

$$\mathbf{S}_N := \sum_{n=1}^{N} \left(\mathbf{P}_d^{n-1}\mathbf{r}_d - \mathbf{g}^{d^\infty}\right).$$

denote the N-th partial sum in (7.18). Table 7.2 below provides the first few terms and the limit of $\mathbf{S}_N$. It shows that $\mathbf{S}_N \to \mathbf{h}^{d^\infty} = (5/9, -10/9)$. Observe that starting in s_1 the bias is positive since the expected reward $r_d(s_1)$ exceeds $g^{d^\infty}(s_1)$ and negative when starting in s_2 since $r_d(s_2)$ is less than $g^{d^\infty}(s_2)$.

	$N=1$	$N=2$	$N=3$	...	$N=\infty$
$S_N(s_1)$	0.33	0.47	0.52	...	0.5556
$S_N(s_2)$	−0.67	−0.93	1.04	...	−1.1111

Table 7.2: Terms in $\mathbf{S}_N$.

Relationship to the discounted reward

The most elegant analysis of finite state average reward models uses the relationship between the discounted reward and the average reward referred to as the *vanishing discount rate* approach. It is based on the following key result. The straightforward proof, provided in the aperiodic case[11], is informative as it includes several calculations that will be used below.

Theorem 7.4. Let S be finite, $d \in D^{\mathrm{MR}}$, and $0 \le \lambda < 1$. Then

$$\mathbf{v}_\lambda^{d^\infty} = \frac{\mathbf{g}^{d^\infty}}{1-\lambda} + \mathbf{h}^{d^\infty} + \mathbf{f}(\lambda), \tag{7.22}$$

where $\mathbf{h}^{d^\infty} = \mathbf{H}_d\mathbf{r}_d$ and $\mathbf{f}(\lambda)$ is a vector that converges to $\mathbf{0}$ as $\lambda \uparrow 1$.

Proof. From (5.29) the expected discounted reward of stationary policy d^∞ may be written as

$$\mathbf{v}_\lambda^{d^\infty} = \sum_{n=0}^{\infty} \lambda^n \mathbf{P}_d^n \mathbf{r}_d.$$

Adding and subtracting $\mathbf{P}_d^*$ inside the summation yields:

$$\sum_{n=0}^{\infty} \lambda^n \mathbf{P}_d^n \mathbf{r}_d = \sum_{n=0}^{\infty} \lambda^n (\mathbf{P}_d^* + \mathbf{P}_d^n - \mathbf{P}_d^*)\mathbf{r}_d$$

[11] A proof in the periodic case requires additional averaging as noted in the definitions above.

$$
\begin{aligned}
&= \sum_{n=0}^{\infty} \lambda^n \mathbf{P}_d^* \mathbf{r}_d + \left(\mathbf{I} - \mathbf{P}_d^* + \sum_{n=1}^{\infty} \lambda^n (\mathbf{P}_d^n - \mathbf{P}_d^*) \right) \mathbf{r}_d \\
&= \frac{\mathbf{P}_d^* \mathbf{r}_d}{1-\lambda} + \left(\sum_{n=0}^{\infty} \lambda^n (\mathbf{P}_d - \mathbf{P}_d^*)^n - \mathbf{P}_d^* \right) \mathbf{r}_d \qquad (7.23) \\
&= \frac{\mathbf{P}_d^* \mathbf{r}_d}{1-\lambda} + \left((\mathbf{I} - \lambda(\mathbf{P}_d - \mathbf{P}_d^*))^{-1} - \mathbf{P}_d^* \right) \mathbf{r}_d \qquad (7.24) \\
&= \frac{\mathbf{P}_d^* \mathbf{r}_d}{1-\lambda} + \mathbf{H}_d \mathbf{r}_d + \left((\mathbf{I} - \lambda(\mathbf{P}_d - \mathbf{P}_d^*))^{-1} - \mathbf{P}_d^* - \mathbf{H}_d \right) \mathbf{r}_d \qquad (7.25) \\
&= \frac{\mathbf{g}^{d^\infty}}{1-\lambda} + \mathbf{h}^{d^\infty} + \mathbf{f}(\lambda),
\end{aligned}
$$

where

$$
\mathbf{f}(\lambda) := \left((\mathbf{I} - \lambda(\mathbf{P}_d - \mathbf{P}_d^*))^{-1} - \mathbf{P}_d^* - \mathbf{H}_d \right) \mathbf{r}_d.
$$

An explanation of some key steps above follows. That $\mathbf{P}_d^n - \mathbf{P}_d^* = (\mathbf{P}_d - \mathbf{P}_d^*)^n$ for $n \geq 1$ in (7.23) follows from properties of $\mathbf{P}_d^*$ in part 2 of Theorem B.1. Its derivation is left as an exercise. The existence and representation for the inverse in (7.24) for $\lambda \leq 1$ follows from Proposition B.1 in Appendix B. Equation (7.25) follows by adding and subtracting $\mathbf{H}_d$ in (7.24) and the final equality follows from the representations for $\mathbf{g}^{d^\infty}$ and $\mathbf{h}^{d^\infty}$ in Theorems 7.1 and 7.3. Lastly, noting that $\mathbf{H}_d$ can be defined equivalently as $(\mathbf{I} - (\mathbf{P}_d - \mathbf{P}_d^*))^{-1} - \mathbf{P}_d^*$ (see Proposition B.1), $\mathbf{f}(\lambda)$ converges to $\mathbf{0}$, completing the proof. □

The expression (7.22) is referred to as the *partial Laurent expansion* of the expected discounted reward. When $\mathbf{f}(\lambda)$ is written as an infinite series, the representation is referred to as the *Laurent expansion* of the expected discounted reward. It is the basis for the concept of sensitive discount optimality[12], which will not be discussed in this book.

Note that the above result provides the approximation for the bias:

$$
\mathbf{h}^{d^\infty} \approx \mathbf{v}_\lambda^{d^\infty} - \frac{\mathbf{g}^{d^\infty}}{1-\lambda}. \qquad (7.26)
$$

This means that the bias of a stationary policy approximately equals the difference between its expected discounted reward and the expected discounted reward in a system that accrues the average reward every period.

The above proof also provides the matrix relationship

$$
\sum_{n=0}^{\infty} \lambda^n \mathbf{P}_d^n = \frac{\mathbf{P}_d^*}{1-\lambda} + \mathbf{H}_d + \mathbf{F}(\lambda), \qquad (7.27)
$$

where all of the components of the matrix $\mathbf{F}(\lambda)$ converge to 0 as $\lambda \uparrow 1$.

[12] See Chapter 10 in Puterman 1994

The following corollary uses Theorem 7.4 to relate the discounted reward to the average reward[13].

Corollary 7.2. Let $d \in D^{\mathrm{MR}}$. Then

$$\lim_{\lambda \uparrow 1} (1-\lambda)\mathbf{v}_\lambda^{d^\infty} = \mathbf{g}^{d^\infty} \tag{7.28}$$

The following table continues Example 7.3 by illustrating the calculations implicit in Theorem 7.4 and Corollary 7.2. Observe that approximations are accurate to two decimal places for $\lambda = 0.99$ and three decimal places for $\lambda = 0.999$.

λ	$(1-\lambda)\mathbf{v}_\lambda^{d^\infty}$	$\mathbf{v}_\lambda^{d^\infty} - \left((1-\lambda)^{-1}\mathbf{g}^{d^\infty} - \mathbf{h}^\infty\right)$
0.9	$(2.719, 2.563)$	$(-0.0347, 0.0694)$
0.95	$(2.694, 2.613)$	$(-0.0179, 0.0358)$
0.99	$(2.672, 2.656)$	$(-0.0037, 0.0074)$
0.999	$(2.667, 2.666)$	$(-0.0004, 0.0007)$

Table 7.3: Illustration of the approximation to $\mathbf{g}^{d^\infty} = (2.667, 2.667)$ based on Corollary 7.2 and the accuracy of the partial Laurent series approximation in Theorem 7.4 as a function of λ for the policy in Example 7.3.

Relationship to the expected total reward

The following theorem describes the relationship between the average reward and the expected total reward of a stationary policy in the aperiodic case. A representation like (7.5) in the periodic case requires averaging (Exercise 4).

[13]Results of this kind are often called Tauberian theorems because they relate two different ways of summing a divergent infinite series (see Derman [1970] or Sennott [1999]). For a sequence of real numbers $a_n, n = 0, 1, 2, \ldots$ it is analogous to

$$\lim_{\lambda \uparrow 1} (1-\lambda) \sum_{n=0}^{\infty} a_n \lambda^n = \lim_{N \to \infty} \sum_{n=1}^{N} a_n$$

Theorem 7.5. Let $d \in D^{\text{MR}}$. Then if the Markov chain corresponding to $\mathbf{P}_d$ has no closed periodic classes

$$\mathbf{v}_N^{d^\infty} = N\mathbf{g}^{d^\infty} + \mathbf{h}^{d^\infty} + \mathbf{o}(1), \tag{7.29}$$

where $\mathbf{o}(1)$ is a vector that converges to $\mathbf{0}$ as $N \to \infty$.

Proof. Adding and subtracting $\mathbf{P}_d^*$ from the matrix representation for the expected total reward over N decision epochs yields:

$$\begin{aligned}
\mathbf{v}_N^{d^\infty} &= \sum_{n=1}^{N} \mathbf{P}_d^{n-1}\mathbf{r}_d \qquad (7.30)\\
&= \sum_{n=1}^{N} (\mathbf{P}_d^* + \mathbf{P}_d^{n-1} - \mathbf{P}_d^*)\mathbf{r}_d \\
&= N\mathbf{P}_d^*\mathbf{r}_d + \sum_{n=1}^{N} (\mathbf{P}_d^{n-1} - \mathbf{P}_d^*)\mathbf{r}_d \\
&= N\mathbf{P}_d^*\mathbf{r}_d + \sum_{n=1}^{N} (\mathbf{P}_d^{n-1} - \mathbf{P}_d^*)\mathbf{r}_d + \sum_{n=N+1}^{\infty} (\mathbf{P}_d^{n-1} - \mathbf{P}_d^*)\mathbf{r}_d - \sum_{n=N+1}^{\infty} (\mathbf{P}_d^{n-1} - \mathbf{P}_d^*)\mathbf{r}_d \\
&= N\mathbf{P}_d^*\mathbf{r}_d + \mathbf{H}_d\mathbf{r}_d - \left(\sum_{n=N+1}^{\infty} (\mathbf{P}_d^{n-1} - \mathbf{P}_d^*) \right) \mathbf{r}_d, \qquad (7.31)
\end{aligned}$$

where the last expression in (7.31) is $\mathbf{o}(1)$ because $\sum_{n=1}^{N}(\mathbf{P}_d^{n-1} - \mathbf{P}_d^*)$ converges to $\mathbf{H}_d$ as a consequence of Proposition B.1. □

This result means that for each $s \in S$, $v_N^{d^\infty}(s)$ is *approximately linear* in N with slope $g^{d^\infty}(s)$ and intercept $h^{d^\infty}(s)$ (see Figure 7.5). Furthermore this theorem provides another interpretation for the bias as

$$\mathbf{h}^{d^\infty} \approx \mathbf{v}_N^{d^\infty} - N\mathbf{g}^{d^\infty}. \tag{7.32}$$

An immediate consequence of Theorem 7.5 follows.

Corollary 7.3. Let $d \in D^{\text{MR}}$. Then

$$\mathbf{g}^{d^\infty} = \lim_{N\to\infty} \frac{1}{N}\mathbf{v}_N^{d^\infty}. \tag{7.33}$$

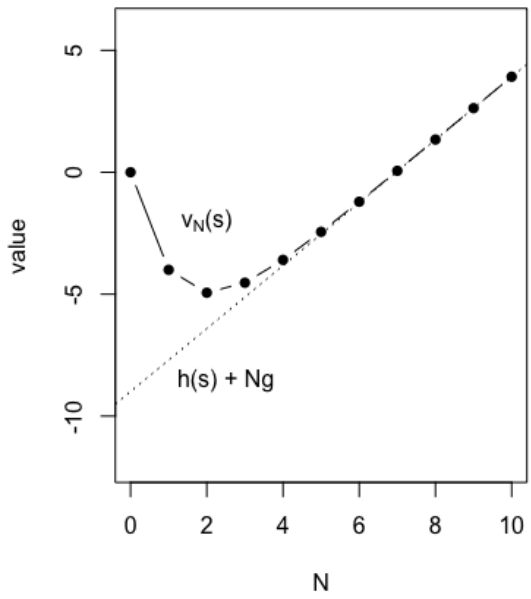


Figure 7.5: Sample plot of $v_N^{d^\infty}(s)$ vs. $h^{d^\infty}(s) + Ng^{d^\infty}$.

Observe that Theorem 7.5 also suggests the following approximation

$$\mathbf{g}^{d^\infty} \approx \mathbf{v}_N^{d^\infty} - \mathbf{v}_{N-1}^{d^\infty}, \tag{7.34}$$

which will be very useful when analyzing value iteration in average reward models.

The following discussion illustrates these results in the context of the decision rule in Example 7.3. Table 7.4 shows that $\mathbf{v}_N^{d^\infty} - \mathbf{v}_{N-1}^{d^\infty}$ better approximates $\mathbf{g}^{d^\infty}$ than $\mathbf{v}_N^{d^\infty}/N$. In fact the approximation by $\mathbf{v}_N^{d^\infty} - \mathbf{v}_{N-1}^{d^\infty}$ is accurate to three decimal places when $N = 11$. Note also that the approximation of the expected total reward based on (7.29) is very accurate even for small N.

7.4.2 Computing the gain and bias of a stationary policy

This section shows how to compute the gain and bias without evaluating $\mathbf{P}_d^*$ and $\mathbf{H}_d$. It derives a system of equations that when solved yields the gain and bias, and moreover can be generalized to obtain the Bellman equation for average reward models. The derivation is motivated in two ways: using the partial Laurent expansion of the expected discounted reward and using the approximation based on the finite horizon expected reward described in the previous section.

It will show that the gain and bias satisfy the following system of matrix equations[14]:

[14]Many of you may be familiar with this system without the first equation. This is because in most

N	$\frac{1}{N}\mathbf{v}_N^{d^\infty}$	$\mathbf{v}_N^{d^\infty} - \mathbf{v}_{N-1}^{d^\infty}$	$\mathbf{v}_N^{d^\infty} - \left(N\mathbf{g}^{d^\infty} - \mathbf{h}^\infty\right)$
5	$(2.777, 2.447)$	$(2.675, 2.644)$	$(-0.006, 0.011)$
10	$(2.722, 2.556)$	$(2.667, 2.666)$	$(-0.0001, 0.0001)$
20	$(2.694, 2.611)$	$(2.667, 2.667)$	*
50	$(2.678, 2.644)$	$(2.667, 2.667)$	*

Table 7.4: Illustration of the approximation to $\mathbf{g}^{d^\infty} = (2.667, 2.667)$ based on Corollary 7.3 and (7.34). The last column shows the accuracy of the approximation in Theorem 7.5 as a function of N for the policy in Example 7.3. The entries denoted by * are less than 10^{-8}.

Average reward evaluation equations

$$\mathbf{g} = \mathbf{P}_d\mathbf{g} \quad \text{and} \quad \mathbf{h} = \mathbf{r}_d - \mathbf{g} + \mathbf{P}_d\mathbf{h}. \tag{7.35}$$

These equations are sometimes written as

$$(\mathbf{I} - \mathbf{P}_d)\mathbf{g} = \mathbf{0} \quad \text{and} \quad \mathbf{g} + (\mathbf{I} - \mathbf{P}_d)\mathbf{h} = \mathbf{r}_d, \tag{7.36}$$

with all variables on the left hand side. For a deterministic decision rule, (7.35) can be expressed in component notation as:

$$g(s) = \sum_{s\in S} p(j|s, d(s))g(j) \quad \text{and} \quad h(s) = r(s, d(s)) - g(s) + \sum_{j\in S} p(j|s, d(s))h(j). \tag{7.37}$$

An obvious modification is required for a randomized decision rule.

models in the literature, $\mathbf{g}^{d^\infty}$ is a constant vector so this equation becomes redundant. This issue will be discussed below.

Heuristic derivation based on the partial Laurent series approximation to the discounted reward.

As a result of equation (5.31), the expected discounted reward of stationary policy d^∞, $\mathbf{v}_\lambda^{d^\infty}$ satisfies

$$\mathbf{v} = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}.$$

Substituting (7.22) into both sides of this equation yields

$$\begin{aligned}\frac{\mathbf{g}^{d^\infty}}{1-\lambda} + \mathbf{h}^{d^\infty} + \mathbf{f}(\lambda) &= \mathbf{r}_d + \lambda \mathbf{P}_d \left(\frac{\mathbf{g}^{d^\infty}}{1-\lambda} + \mathbf{h}^{d^\infty} + \mathbf{f}(\lambda) \right) \\ &= \mathbf{r}_d + \frac{\mathbf{P}_d \mathbf{g}^{d^\infty}}{1-\lambda} - \mathbf{P}_d \mathbf{g}^{d^\infty} + \lambda \mathbf{P}_d \mathbf{h}^{d^\infty} + \lambda \mathbf{P}_d \mathbf{f}(\lambda).\end{aligned}$$

For this last relationship to be valid in the limit as $\lambda \uparrow 1$ requires that $\mathbf{g}^{d^\infty} = \mathbf{P}_d \mathbf{g}^{d^\infty}$. When this is the case[15]

$$\begin{aligned}\mathbf{h}^{d^\infty} &= \mathbf{r}_d - \mathbf{P}_d \mathbf{g}^{d^\infty} + \lambda \mathbf{P}_d \mathbf{h}^{d^\infty} + \mathbf{f}(\lambda) \\ &= \mathbf{r}_d - \mathbf{g}^{d^\infty} + \lambda \mathbf{P}_d \mathbf{h}^{d^\infty} + \mathbf{f}(\lambda).\end{aligned}$$

Letting $\lambda \uparrow 1$ shows that $\mathbf{h}^{d^\infty} = \mathbf{r}_d - \mathbf{g}^{d^\infty} + \mathbf{P}_d \mathbf{h}^{d^\infty}$. Hence $\mathbf{g}^{d^\infty}$ and $\mathbf{h}^{d^\infty}$ must be solutions of (7.35).

Heuristic derivation based on the expected total reward approximation.

From equation (6.24), subject to $\mathbf{v}_0 = \mathbf{0}$, the expected total reward over n periods of stationary policy d^∞, $\mathbf{v}_n^{d^\infty}$, satisfies

$$\mathbf{v}_{n+1} = \mathbf{r}_d + \mathbf{P}_d \mathbf{v}_n$$

for $n = 0, 1, \ldots$. Substituting (7.29) into both sides of this equation yields

$$(n+1)\mathbf{g}^{d^\infty} + \mathbf{h}^{d^\infty} + \mathbf{o}(1) = \mathbf{r}_d + \mathbf{P}_d \left(n\mathbf{g}^{d^\infty} + \mathbf{h}^{d^\infty} + \mathbf{o}(1) \right)$$

so that

$$n\mathbf{g}^{d^\infty} + \mathbf{h}^{d^\infty} = \mathbf{r}_d - \mathbf{g}^{d^\infty} + n\mathbf{P}_d \mathbf{g}^{d^\infty} + \mathbf{P}_d \mathbf{h}^{d^\infty} + \mathbf{o}(1).$$

For this to hold for all $n = 0, 1, \ldots$ requires that the expressions multiplied by n be equal. Hence $\mathbf{g}^{d^\infty} = \mathbf{P}_d \mathbf{g}^{d^\infty}$. If this holds it follows that $\mathbf{h}^{d^\infty} = \mathbf{r}_d - \mathbf{g}^{d^\infty} + \mathbf{P}_d \mathbf{h}^{d^\infty}$ as $n \to \infty$. Consequently $\mathbf{g}^{d^\infty}$ and $\mathbf{h}^{d^\infty}$ must be solutions of (7.35).

[15] The precise form of $\mathbf{f}(\lambda)$ is not required, only that it converges to $\mathbf{0}$ as $\lambda \uparrow 1$. So analogously to "small o notation" $\mathbf{f}(\lambda)$ represents $\lambda \mathbf{P}_d \mathbf{f}(\lambda) - \mathbf{f}(\lambda)$.

An example

Before describing some important properties of the system of equations (7.35), an analysis of Example 7.3 illustrates the subtleties involved in solving this system.

For the decision rule d^∞ analyzed in Example 7.3, (7.35) becomes:

$$\begin{bmatrix} g(s_1) \\ g(s_2) \end{bmatrix} = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} \begin{bmatrix} g(s_1) \\ g(s_2) \end{bmatrix}$$

and

$$\begin{bmatrix} h(s_1) \\ h(s_2) \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix} - \begin{bmatrix} g(s_1) \\ g(s_2) \end{bmatrix} + \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} \begin{bmatrix} h(s_1) \\ h(s_2) \end{bmatrix}.$$

Expressing this system of equations as

$$(\mathbf{I} - \mathbf{P}_d)\mathbf{g} = \mathbf{0} \quad \text{and} \quad \mathbf{I}\mathbf{g} + (\mathbf{I} - \mathbf{P}_d)\mathbf{h} = \mathbf{r}_d \tag{7.38}$$

they can be combined and written in terms of a partitioned matrix as:

$$\begin{bmatrix} \mathbf{I} - \mathbf{P}_d & \mathbf{0} \\ \mathbf{I} & \mathbf{I} - \mathbf{P}_d \end{bmatrix} \begin{bmatrix} \mathbf{g} \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{r}_d \end{bmatrix}. \tag{7.39}$$

Substituting appropriate values, this matrix equation becomes

$$\begin{bmatrix} 0.2 & -0.2 & 0 & 0 \\ -0.4 & 0.4 & 0 & 0 \\ 1 & 0 & 0.2 & -0.2 \\ 0 & 1 & -0.4 & 0.4 \end{bmatrix} \begin{bmatrix} g(s_1) \\ g(s_2) \\ h(s_1) \\ h(s_2) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 3 \\ 2 \end{bmatrix}. \tag{7.40}$$

Applying Gauss-Jordan elimination[16] reduces this equation to

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} g(s_1) \\ g(s_2) \\ h(s_1) \\ h(s_2) \end{bmatrix} = \begin{bmatrix} 8/3 \\ 0 \\ 8/3 \\ 5/3 \end{bmatrix}. \tag{7.41}$$

Hence $g(s_1) = g(s_2) = 8/3$ and $h(s_1) - h(s_2) = 5/3$, so calculations in Example 7.3 show that $\mathbf{g} = \mathbf{g}^{d^\infty}$. Moreover for an arbitrary constant c,

$$\mathbf{h} = \begin{bmatrix} 5/3 \\ 0 \end{bmatrix} + c \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

where the vector $(5/3, 0)$ corresponds to the solution when $h(s_2) = c = 0$. By choosing $c = -10/9$ it follows that $\mathbf{h} = \mathbf{h}^{d^\infty}$.

[16] Some might refer to this as Gaussian elimination.

Making this formal

This above example illustrates the following key features of the system of equations (7.35):

1. When $\mathbf{g}$ satisfies these equations, $\mathbf{g} = \mathbf{g}^{d^\infty}$.
2. These equations do not uniquely determine $\mathbf{h}$. As can be seen directly from (7.40) or from the observation that the second row of the reduced matrix (7.41) contains all zero entries, the partitioned matrix in (7.39) is not full rank[17].
3. The singular matrix $\mathbf{I} - \mathbf{P}_d$ plays a key role in finding $\mathbf{g}^{d^\infty}$ and $\mathbf{h}^{d^\infty}$.

These observations are formalized and extended in the following important theorem.

Theorem 7.6. Let $d \in D^{\text{MR}}$.

1. Then $\mathbf{g}^{d^\infty} = \mathbf{P}_d^* \mathbf{r}_d$ and $\mathbf{h}^{d^\infty} = \mathbf{H}_d \mathbf{r}_d$ are solutions of (7.35).
2. Suppose $(\mathbf{g}, \mathbf{h})$ is a solution of (7.35). Then $\mathbf{g} = \mathbf{g}^{d^\infty}$ and $\mathbf{h} = \mathbf{h}^{d^\infty} + \mathbf{u}$ where $(\mathbf{I} - \mathbf{P}_d)\mathbf{u} = \mathbf{0}$.
3. Suppose $(\mathbf{g}, \mathbf{h})$ is a solution of (7.35) and $\mathbf{P}_d^* \mathbf{h} = \mathbf{0}$. Then $\mathbf{g} = \mathbf{g}^{d^\infty}$ and $\mathbf{h} = \mathbf{h}^{d^\infty}$.

Proof. Since $\mathbf{P}_d^* = \mathbf{P}_d \mathbf{P}_d^*$, that $\mathbf{g}^{d^\infty}$ satisfies the first equation in (7.35) follows from noting that

$$(\mathbf{I} - \mathbf{P}_d)\mathbf{g}^{d^\infty} = (\mathbf{I} - \mathbf{P}_d)\mathbf{P}_d^* \mathbf{r}_d = \mathbf{0}.$$

To establish the second equality in (7.35), note that

$$\begin{aligned} \mathbf{g}^{d^\infty} + (\mathbf{I} - \mathbf{P}_d)\mathbf{h}^{d^\infty} &= \mathbf{P}_d^* \mathbf{r}_d + (\mathbf{I} - \mathbf{P}_d)\mathbf{H}_d \mathbf{r}_d \\ &= \mathbf{P}_d^* \mathbf{r}_d + (\mathbf{I} - \mathbf{P}_d^*)\mathbf{r}_d \\ &= \mathbf{r}_d, \end{aligned}$$

where the second equality follows from the expression $(\mathbf{I} - \mathbf{P}_d)\mathbf{H}_d = \mathbf{I} - \mathbf{P}_d^*$ in (B.16) in Appendix B. Hence part 1 is proved.

To prove[18] the result in part 2 let $(\mathbf{g}_1, \mathbf{h}_1)$ and $(\mathbf{g}_2, \mathbf{h}_2)$ denote two solutions of (7.35) and $\Delta\mathbf{g} := \mathbf{g}_1 - \mathbf{g}_2$ and $\Delta\mathbf{h} := \mathbf{h}_1 - \mathbf{h}_2$. Then it is easy to see that

$$(\mathbf{I} - \mathbf{P}_d)\Delta\mathbf{g} = \mathbf{0} \quad \text{and} \quad \Delta\mathbf{g} = (\mathbf{P}_d - \mathbf{I})\Delta\mathbf{h}. \tag{7.42}$$

[17] In the example it has three independent rows and one redundant row and $\mathbf{I} - \mathbf{P}_d$ has one redundant row.

[18] An alternative proof that $\Delta\mathbf{g} = \mathbf{0}$ is obtaining by multiplying the second equation in (7.42) by $\mathbf{P}_d^*$, to obtain $\mathbf{P}_d^* \Delta\mathbf{g} = \mathbf{0}$ and then adding this to the first expression to obtain $(\mathbf{I} - (\mathbf{P}_d - \mathbf{P}_d^*))\Delta\mathbf{g} = \mathbf{0}$. Since this matrix is invertible, $\Delta\mathbf{g} = \mathbf{0}$.

Multiplying both sides of the second equation on the left by $\mathbf{P}_d$ and noting the first equation gives

$$\Delta\mathbf{g} = \mathbf{P}_d\Delta\mathbf{g} = (\mathbf{P}_d^2 - \mathbf{P}_d)\Delta\mathbf{h}.$$

Repeating this argument (or applying induction) establishes that for all $n \geq 0$,

$$\Delta\mathbf{g} = (\mathbf{P}_d^n - \mathbf{P}_d^{n-1})\Delta\mathbf{h}.$$

Adding these expressions for $\Delta\mathbf{g}$ together yields

$$n\Delta\mathbf{g} = (\mathbf{P}_d^n - \mathbf{I})\Delta\mathbf{h}.$$

Dividing both sides by n and take the limit as $n \to \infty$ establishes that

$$\Delta\mathbf{g} = \lim_{n\to\infty} \frac{1}{n}(\mathbf{P}_d^n - \mathbf{I})\Delta\mathbf{h} = \mathbf{0}.$$

Hence $\mathbf{g}_1 = \mathbf{g}_2 = \mathbf{g}^{d^\infty}$, where the last equality follows from part 1. Substituting $\Delta\mathbf{g} = \mathbf{0}$ into the second equation in (7.42) shows that $(\mathbf{I} - \mathbf{P}_d)\Delta\mathbf{h} = \mathbf{0}$. Part 1 established that $\mathbf{h}^{d^\infty}$ satisfies

$$\mathbf{r}_d = \mathbf{g}^{d^\infty} + (\mathbf{I} - \mathbf{P}_d)\mathbf{h}.$$

So setting $\mathbf{h}_1 = \mathbf{h}^{d^\infty}$ implies that $\mathbf{h}_2 = \mathbf{h}^{d^\infty} + \Delta\mathbf{h}$, which completes the proof of part 2.

To establish part 3 add $\mathbf{P}_d^*\mathbf{h} = \mathbf{0}$ to the right hand side of $\mathbf{r}_d = \mathbf{g}^{d^\infty} + (\mathbf{I} - \mathbf{P}_d)\mathbf{h}$ and note that $\mathbf{g}^{d^\infty} = \mathbf{P}_d^*\mathbf{r}_d$ and rearrange terms to obtain

$$(\mathbf{I} - \mathbf{P}_d^*)\mathbf{r}_d = (\mathbf{I} - \mathbf{P}_d + \mathbf{P}_d^*)\mathbf{h}.$$

Multiplying both sides on the left by $(\mathbf{I} - \mathbf{P}_d + \mathbf{P}_d^*)^{-1}$ yields

$$\mathbf{h} = (\mathbf{I} - \mathbf{P}_d + \mathbf{P}_d^*)^{-1}(\mathbf{I} - \mathbf{P}_d^*)\mathbf{r}_d = \mathbf{H}_d\mathbf{r}_d = \mathbf{h}^{d^\infty}$$

completing the proof. □

Some implications of this result follow:

1. It will be shown below that the system of equations (7.35) provides the basis for the average reward Bellman equation. Consequently, understanding its properties is fundamental.

2. Theorem 7.6 establishes that (7.35) uniquely determines $\mathbf{g}^{d^\infty}$ and determines $\mathbf{h}^{d^\infty}$ up to a vector in the null space[19] of $\mathbf{I} - \mathbf{P}_d$. Part 3 provides a way of identifying $\mathbf{h}^{d^\infty}$ but it requires determining $\mathbf{P}_d^*$, which can be burdensome.

[19]The vector $\mathbf{x} \neq \mathbf{0}$ is in the *null space* of the matrix $\mathbf{A}$ if $\mathbf{A}\mathbf{x} = \mathbf{0}$.

3. The equation $(\mathbf{I}-\mathbf{P}_d)\mathbf{g}=\mathbf{0}$ shows only that $\mathbf{g}$ is an element of the null space of $\mathbf{I}-\mathbf{P}_d$. Since the null space of $\mathbf{I}-\mathbf{P}_d$ is equivalent to the space of right eigenvectors of $\mathbf{P}_d$ corresponding to the eigenvalue 1, this means that this equation determines $\mathbf{g}$ up to m arbitrary constants where m represents the number of closed classes of $\mathbf{P}_d$. Special attention will be paid to the case $m=1$ where this system of equations simplifies further as the corollary below shows.

4. As an alternative to adding the condition $\mathbf{P}^*\mathbf{h}=\mathbf{0}$ to uniquely determine $\mathbf{h}^{d^\infty}$ one can instead add the *third* equation

$$-\mathbf{h}+(\mathbf{P}_d-\mathbf{I})\mathbf{w}=\mathbf{0}$$

to the system (7.35). Solving this system of three matrix equations uniquely determines $\mathbf{g}^{d^\infty}$ and $\mathbf{h}^{d^\infty}$ and determines $\mathbf{w}$ up to an element of the null space of $\mathbf{I}-\mathbf{P}_d$. When $\mathbf{w}$ is uniquely determined by the condition $\mathbf{P}_d^*\mathbf{w}=\mathbf{0}$ or by adding a *fourth* equation, it corresponds to the third term in the Laurent series[20] expansion of $\mathbf{v}_\lambda^{d^\infty}$.

Simplifications for regular and unichain models

The following corollary provides simplified evaluation equations that apply when a Markovian decision rule has a single closed class, that is, it is recurrent or unichain. It follows from the previous theorem by noting for such a model that $\mathbf{P}_d^*$ has equal rows so that all components of g^{d^∞} are equal, and so that the equation $(\mathbf{I}-\mathbf{P}_d)\mathbf{g}=\mathbf{0}$ is superfluous. Note this is the form of the evaluation you are most likely to come across in the literature.

Corollary 7.4. Suppose $\mathbf{P}_d$ is the transition probability matrix of a Markovian decision rule d in a recurrent or unichain model.

1. If $(\mathbf{I}-\mathbf{P}_d)\mathbf{g}=\mathbf{0}$, then $\mathbf{g}=g\mathbf{e}$, for some scalar g[a].

2. Suppose $(g,\mathbf{h})$ satisfy
$$g\mathbf{e}+(\mathbf{I}-\mathbf{P}_d)\mathbf{h}=\mathbf{r}_d. \tag{7.43}$$
Then $g\mathbf{e}=\mathbf{g}^{d^\infty}$ and $\mathbf{h}=\mathbf{h}^{d^\infty}+k\mathbf{e}$, where k is a scalar.

3. Suppose $(g,\mathbf{h})$ satisfy
$$g\mathbf{e}+(\mathbf{I}-\mathbf{P}_d)\mathbf{h}=\mathbf{r}_d$$
and $\mathbf{P}_d^*\mathbf{h}=\mathbf{0}$. Then $g\mathbf{e}=\mathbf{g}^{d^\infty}$ and $\mathbf{h}=\mathbf{h}^{d^\infty}$.

[a] Recall that $\mathbf{e}$ denotes a vector with all components equal to one.

[20] See Theorem 8.2.8 and Chapter 9 in Puterman [1994] for details on this concept.

Hence for recurrent and unichain models, $\mathbf{g}^{d^\infty}$ and $\mathbf{h}^{d^\infty}$, up to a constant, can be obtained by solving (7.43). For $d \in D^{\mathrm{MD}}$ and all $s \in S$, (7.43) can be expressed in component notation as:

$$g + h(s) - \sum_{j \in S} p(j|s, d(s))h(j) = r(s, d(s)) \tag{7.44}$$

Of course the standard modification applies when $d \in D^{\mathrm{MR}}$.

Example 7.4. This examples continues Example 7.3. For the stationary policy analyzed there, $\mathbf{P}_d$ is recurrent (in fact it is regular) so that results in Corollary 7.4 can be used to find its gain and bias. Equation (7.44) becomes

$$\begin{aligned} g + 0.2h(s_1) - 0.2h(s_2) &= 3 \\ g - 0.4h(s_1) + 0.4h(s_2) &= 2. \end{aligned}$$

Note that this under-determined system has three variables and two unknowns.

Multiplying the first equation by 2 and adding it to the second equation shows that $g = 8/3$. Substituting this value back into the first (or second) equation shows that $h(s_1) - h(s_2) = 5/3$. Thus $\mathbf{h}$ is unique up to an additive constant and agrees with previous analyses.

Since

$$\mathbf{P}_d^* = \begin{bmatrix} 2/3 & 1/3 \\ \\ 2/3 & 1/3 \end{bmatrix},$$

the condition $\mathbf{P}_d^*\mathbf{h} = \mathbf{0}$ implies that $2h(s_1)/3 + h(s_2)/3 = 0$ or equivalently $h(s_2) = -2h(s_1)$. Hence $h^{d^\infty}(s_1) = 5/9$ and $h^{d^\infty}(s_2) = -10/9$.

Alternatively other particular solutions can be obtained by setting $h(s_2) = 0$ to yield $h(s_1) = 5/3$ or setting $h(s_1) = 0$ to yield $h(s_2) = -5/3$.These solutions will be referred to as relative values and defined formally below.

Relative values

It is sometimes sufficient in a recurrent or unichain model to distinguish specific representations for $\mathbf{h}$, referred to as *relative values*.

Definition 7.8. The *relative value* of d^∞ for a specified $s^* \in S$, is the $\mathbf{h}$ part of the solution of (7.43) subject to the condition $h(s^*) = 0$. It is denoted by $\mathbf{h}_{\mathrm{rel}}^{d^\infty}$.

The calculation in the previous section distinguished relative values with respect to

$s^* = s_1$ and $s^* = s_2$. More generally if $(g, \mathbf{h})$ satisfy (7.43), then

$$h_{\text{rel}}^{d^\infty}(s) = h(s) - h(s^*). \tag{7.45}$$

In some applications, for instance in single-server queuing control models or single-product inventory models, it is convenient to set $s^* = 0$ when computing the relative values. Most importantly it will be seen below that relative values suffice for optimization, so from this perspective it is not necessary to undertake the extra effort to compute $\mathbf{h}^{d^\infty}$ (unless you want to).

7.5 The Bellman equation and its properties

As in the case of other optimality criteria, the Bellman equation plays a fundamental role in average reward Markov decision processes. The key difference is that multi-chain models require a pair of nested Bellman equations, while recurrent and unichain models require only a single (vector) equation. Because of this, this section will focus on recurrent and unichain models. A brief discussion of the multi-chain Bellman equation will be included for completeness[21].

The main results in this section are that:

1. The Bellman equation has a solution.
2. The first component of the solution corresponds to the optimal gain[22].
3. The Bellman equation provides the basis for algorithms for finding average optimal policies.

7.5.1 The Bellman equation in recurrent and unichain models

The Bellman equation for a recurrent or unichain model is given in component form by:

$$h(s) = \max_{a \in A_s} \left\{ r(s,a) - g + \sum_{j \in S} p(j|s,a) h(j) \right\}, \quad \forall s \in S \tag{7.46}$$

or equivalently in vector form as:

[21] See Chapter 8 in Puterman [1994] for a rigorous analysis of the multi-chain Bellman equation.

[22] It does not determine the optimal bias. Instead it determines the gain of an average optimal policy and its bias up to a constant. A further optimality equation is required to determine the optimal bias. Moreover as Example 7.7 shows, there may exist average optimal policies with gain and bias that do not satisfy the Bellman equation.

$$\mathbf{h} = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}} \{\mathbf{r}_d - g\mathbf{e} + \mathbf{P}_d \mathbf{h}\}. \tag{7.47}$$

This equation (in either form), is sometimes referred to as the *average optimality equation.*

Example 7.5. The Bellman equation (7.46) for the unichain model in Section 2.5 becomes

$$h(s_1) = \max\{3 - g + 0.8h(s_1) + 0.2h(s_2), 5 - g + h(s_2)\} \tag{7.48}$$
$$h(s_2) = \max\{-5 - g + h(s_2), 2 - g + 0.4h(s_1) + 0.6h(s_2)\}. \tag{7.49}$$

Observe that there are two equations with three variables so the solution is not unique.

In most instances, solving the Bellman equation requires value iteration, policy iteration or linear programming. This example solves it by using the result (which is established below) that there exists a deterministic stationary policy that is optimal. Hence by enumerating the stationary policies and computing g^{d^∞} for each, it follows that $(d^*)^\infty$ with $d^*(s_1) = a_{1,2}$ and $d^*(s_2) = a_{2,2}$ is optimal and has gain $g^{(d^*)^\infty} = 20/7 = 2.857$.

Solving the evaluation equations (7.44) subject to the condition $\mathbf{P}_{d^*}\mathbf{h} = \mathbf{0}$ gives the solution

$$g^{(d^*)^\infty} = \frac{20}{7}, \quad h^{(d^*)^\infty}(s_1) = \frac{75}{49} \quad \text{and} \quad h^{d^\infty}(s_2) = -\frac{30}{49}.$$

If instead the evaluation equations were solved subject to $h(s_2) = 0$, the solution is $g^{(d^*)^\infty} = 20/7$ together with the relative values $h_{\mathrm{rel}}^{(d^*)^\infty}(s_1) = 15/7$ and $h_{\mathrm{rel}}^{(d^*)^\infty}(s_2) = 0$. Finally, if $h(s_1)$ and $h(s_2)$ together with $g^{(d^*)^\infty}$ solves the evaluation equations, then $g^{(d^*)^\infty}$ and $h(s_1) + k$ and $h(s_2) + k$ for a constant k also satisfies this system of equations.

To verify that these values satisfy the Bellman equation it is easiest to do so for the relative values. Consider equation (7.48)

$$h(s_1) = \max\{3 - g + 0.8h(s_1) + 0.2h(s_2), 5 - g + h(s_2)\}.$$

Since $h_{\mathrm{rel}}^{(d^*)^\infty}(s_1) = 5 - g^{(d^*)^\infty} + h_{\mathrm{rel}}^{(d^*)^\infty}(s_2)$, it is only necessary to check the first expression in the "max". Since $3 - g^{(d^*)^\infty} + 0.8h_{\mathrm{rel}}^{(d^*)^\infty}(s_1) + 0.2h_{\mathrm{rel}}^{(d^*)^\infty}(s_2) = 13/7 < 15/7 = h_{\mathrm{rel}}^{(d^*)^\infty}(s_1)$, equation (7.48) holds.

Now consider the equation (7.49)

$$h(s_2) = \max\{-5 - g + h(s_2), 2 - g + 0.4h(s_1) + 0.6h(s_2)\}.$$

Since $h_{\text{rel}}^{(d^*)^\infty}(s_2) = 2 - g^{(d^*)^\infty} + 0.4h_{\text{rel}}^{(d^*)^\infty}(s_1) + 0.6h_{\text{rel}}^{(d^*)^\infty}(s_2)$, it is necessary only to check the first equation in the "max". Since $-5 - g^{(d^*)^\infty} + h_{\text{rel}}^{(d^*)^\infty}(s_2) = -49/7 < 0 = h_{\text{rel}}^{(d^*)^\infty}(s_2)$, equation (7.49) holds. Hence

$$g^{(d^*)^\infty} = \frac{20}{7}, \quad h^{(d^*)^\infty}(s_1) = \frac{15}{7} \quad \text{and} \quad h^{(d^*)^\infty}(s_2) = 0$$

is a solution to the Bellman equation.

7.5.2 Bounds on the optimal gain

The following result establishes that the first component of the solution to the Bellman equation equals the optimal gain. It does so by first establishing upper bounds on the lim sup average reward and lower bounds on the lim inf average reward of each policy. Its instructive proof provides insight into some fundamental Markov decision process concepts especially how the Bellman equation, which is expressed in terms of Markovian deterministic decision rules, accounts for history-dependent randomized policies.

Theorem 7.7. Let g be a scalar and $\mathbf{h}$ a bounded $|S|$-dimensional vector.

1. If
$$\mathbf{h} \geq \underset{d \in D^{\text{MD}}}{\text{c-max}}\{\mathbf{r}_d - g\mathbf{e} + \mathbf{P}_d\mathbf{h}\}, \tag{7.50}$$
then $g\mathbf{e} \geq \mathbf{g}_+^\pi$ for all $\pi \in \Pi^{\text{HR}}$.

2. If
$$\mathbf{h} \leq \underset{d \in D^{\text{MD}}}{\text{c-max}}\{\mathbf{r}_d - g\mathbf{e} + \mathbf{P}_d\mathbf{h}\}, \tag{7.51}$$
then $g\mathbf{e} \leq \mathbf{g}_-^\pi$ for all $\pi \in \Pi^{\text{HR}}$.

3. If
$$\mathbf{h} = \underset{d \in D^{\text{MD}}}{\text{c-max}}\{\mathbf{r}_d - g\mathbf{e} + \mathbf{P}_d\mathbf{h}\}, \tag{7.52}$$
then $g\mathbf{e} = \mathbf{g}^*$.

Proof. Let $\pi = (d_1, d_2, \ldots)$ denote an arbitrary policy in Π^{MR}. As a consequence of Lemma 2.43 and equation (7.50),

$$g\mathbf{e} \geq \underset{d \in D^{\text{MD}}}{\text{c-max}}\{\mathbf{r}_d + (\mathbf{P}_d - \mathbf{I})\mathbf{h}\} = \underset{d \in D^{\text{MR}}}{\text{c-max}}\{\mathbf{r}_d + (\mathbf{P}_d - \mathbf{I})\mathbf{h}\}.$$

This means that

$$g\mathbf{e} \geq \mathbf{r}_d + (\mathbf{P}_d - \mathbf{I})\mathbf{h} \tag{7.53}$$

for any $d \in D^{\text{MR}}$. Setting $d = d_2$ gives

$$g\mathbf{e} \geq \mathbf{r}_{d_2} + (\mathbf{P}_{d_2} - \mathbf{I})\mathbf{h}.$$

Multiplying this expression by $\mathbf{P}_{d_1}$ and noting that $\mathbf{P}_{d_1}\mathbf{e} = \mathbf{e}$, it follows that

$$g\mathbf{e} \geq \mathbf{P}_{d_1}\mathbf{r}_{d_2} + \mathbf{P}_{d_1}(\mathbf{P}_{d_2} - \mathbf{I})\mathbf{h}.$$

Adding this to (7.53) applied to d_1 implies that

$$\begin{aligned} 2g\mathbf{e} &\geq \mathbf{r}_{d_1} + \mathbf{P}_{d_1}\mathbf{r}_{d_2} + (\mathbf{P}_{d_1}\mathbf{P}_{d_2} - \mathbf{P}_{d_1} + \mathbf{P}_{d_1} - \mathbf{I})\mathbf{h} \\ &= \sum_{n=1}^{2} \mathbf{P}_\pi^{n-1}\mathbf{r}_{d_n} + (\mathbf{P}_\pi^2 - \mathbf{I})\mathbf{h} \end{aligned}$$

Hence by induction it can be shown that for $N \geq 1$

$$Ng\mathbf{e} \geq \sum_{n=1}^{N} \mathbf{P}_\pi^{n-1}\mathbf{r}_{d_n} + (\mathbf{P}_\pi^N - \mathbf{I})\mathbf{h}$$

where $\mathbf{P}_\pi^0 = \mathbf{I}$ and $\mathbf{P}_\pi^n = \mathbf{P}_{d_1}\mathbf{P}_{d_2}\cdots\mathbf{P}_{d_n}$. From this it follows that

$$g\mathbf{e} \geq \limsup_{N\to\infty} \frac{1}{N}\left[\sum_{n=1}^{N} \mathbf{P}_\pi^{n-1}\mathbf{r}_{d_n} + (\mathbf{P}_\pi^N - \mathbf{I})\mathbf{h}\right] = \limsup_{N\to\infty} \frac{1}{N}\mathbf{v}_N^\pi = \mathbf{g}_+^\pi,$$

where the first equality follows from the boundedness of $\mathbf{h}$. Thus, $g\mathbf{e} \geq \mathbf{g}_+^\pi$ for all $\pi \in \Pi^{\text{MR}}$. As a consequence of Theorem 7.12, this is also valid for all $\pi \in \Pi^{\text{HR}}$ establishing part 1.

The proof of part 2 is more direct. From (7.51) there exists a $d^* \in D^{\text{MD}}$ for which

$$g\mathbf{e} \leq \mathbf{r}_{d^*} + (\mathbf{P}_{d^*} - \mathbf{I})\mathbf{h}.$$

Repeating the argument in the proof of part 1 with the single decision rule d^* and accounting for the reversed direction of the inequality implies that

$$g\mathbf{e} \leq \lim_{N\to\infty} \frac{1}{N}\mathbf{v}_N^{(d^*)^\infty} = \liminf_{N\to\infty} \frac{1}{N}\mathbf{v}_N^{(d^*)^\infty} \leq \sup_{\pi\in\Pi^{\text{HR}}} \left\{ \liminf_{N\to\infty} \frac{1}{N}\mathbf{v}_N^{d^\infty} \right\} = \sup_{\pi\in\Pi^{\text{HR}}} \mathbf{g}_-^\pi = \mathbf{g}^*,$$

where the limit in the first inequality exits and equals its lim inf because $(d^*)^\infty$ is stationary. The last equality follows from Definition 7.4.

The third result follows by combining the first two to obtain

$$g\mathbf{e} \leq \mathbf{g}^* = \sup_{\pi\in\Pi^{\text{HR}}} \mathbf{g}_-^\pi \leq \sup_{\pi\in\Pi^{\text{HR}}} \mathbf{g}_+^\pi \leq g\mathbf{e}.$$

□

Some comments on this result follow:

1. Part 3 expresses the most important result, namely that the existence of a solution to the Bellman equation (7.47) identifies the optimal average reward.

2. Parts 1 and 2 hold regardless of chain structure.

3. Note that no assumptions are made on the chain structure. However part 3 may be vacuous when the optimal gain is not constant because there need not exist a scalar g and vector $\mathbf{h}$ that satisfy this specification of the Bellman equation. The following example illustrates this situation.

Example 7.6. Consider the deterministic model shown in Figure 7.6 with $S = \{s_1, s_2\}$, $A_{s_1} = \{a_{1,1}, a_{1,2}\}$ and $A_{s_2} = \{a_{2,1}\}$ with $r(s_1, a_{1,1}) = 3, r(s_1, a_{1,1}) = 1$ and $r(s_2, a_{2,1}) = 2$. Clearly the model is multi-chain and not communicating. Because $g^\pi(s_2) = 2$ and $g^\pi(s_1) \leq 3$ for any policy π, the policy d^∞ with $d(s_1) = a_{1,1}$ and $d(s_2) = a_{2,1}$ is optimal with values $g^*(s_1) = 3$ and $g^*(s_2) = 2$.

Note $g = 3$ and $h(s_1) = h(s_2) = 0$ satisfy (7.50) (expressed in component notation):

$$\begin{aligned} h(s_1) &\geq \max\{3 - g + h(s_1), 1 - g + h(s_2)\} \\ h(s_2) &\geq 2 - g + h(s_2) \end{aligned}$$

Hence $g \geq g_+^\pi(s)$ for $s \in S$ and $\pi \in \Pi^{\text{HR}}$ in agreement with part 1 of Theorem 7.7.

Moreover $g' = 2$ and $h'(s_1) = h'(s_2) = 0$ satisfy (7.51) so that $g' \leq g_-^\pi(s)$ for $s \in S$ and $\pi \in \Pi^{\text{HR}}$ in agreement with part 2. This establishes that

$$2\mathbf{e} \leq \mathbf{g}^* \leq 3\mathbf{e}.$$

Writing out (7.52):

$$\begin{aligned} h(s_1) &= \max\{3 - g + h(s_1), 1 - g + h(s_2)\} \\ h(s_2) &= 2 - g + h(s_2). \end{aligned}$$

The second equation suggests $g = 2$, but then the first equation becomes $h(s_1) = \max\{1 + h(s_1), -1 + h(s_2)\}$, which has no solution. Thus part 3 of the theorem does not apply for this multi-chain model.

If on the other hand $r(s_2, a_{2,1}) = 3$, then part 3 would be valid even though the model is multi-chain.

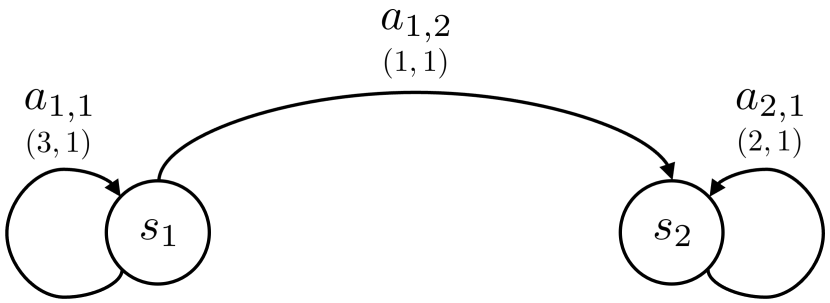


Figure 7.6: Model in Example 7.6. The labels (r, p) refer to the reward and transition probabilities, respectively, associated with using the specified action in the given state.

7.5.3 Existence of solutions of the Bellman equation

This section establishes the existence of a solution to the unichain optimality equation. The proof formalizes the use of the partial Laurent series of the expected discounted reward to link the gain and bias of a policy to its discounted reward. The fundamental idea is that in a model with finitely many deterministic stationary policies, there exists a sequence of discount factors converging to one for which the *same* policy is discount optimal[23].

Theorem 7.8. In a finite state, finite action recurrent or unichain model there exist:

1. A unique scalar g and a vector $\mathbf{h}$ that satisfy the Bellman equation (7.47)
2. A stationary deterministic optimal policy.

Proof. Choose a sequence of discount factors $\lambda'_n, n = 1, 2, \ldots$ for which $\lambda'_n \uparrow 1$. For each n there exists a stationary deterministic discount optimal policy by Theorem 5.4. Since the set of stationary policies is finite, there exists a subsequence of λ'_n, denoted by $\lambda_n, n = 1, 2, \ldots$ for which a specific policy δ^∞ is optimal. Hence $\mathbf{v}_{\lambda_n}^{\delta^\infty} = \mathbf{v}_{\lambda_n}^*$ for $n \geq 1$.

This means that $\mathbf{v}_{\lambda_n}^{\delta^\infty}$ satisfies the discounted Bellman equation (5.48):

$$\mathbf{0} = \underset{d' \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_{d'} + (\lambda_n \mathbf{P}_{d'} - \mathbf{I})\mathbf{v}\}$$

for all $n \geq 1$. Thus it follows that

$$\mathbf{0} = \mathbf{r}_\delta + (\lambda_n \mathbf{P}_\delta - \mathbf{I})\mathbf{v}_{\lambda_n}^{\delta^\infty} = \underset{d' \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_{d'} + (\lambda_n \mathbf{P}_{d'} - \mathbf{I})\mathbf{v}_{\lambda_n}^{\delta^\infty}\} \geq \mathbf{r}_d + (\lambda_n \mathbf{P}_d - \mathbf{I})\mathbf{v}_{\lambda_n}^{\delta^\infty} \tag{7.54}$$

[23] The stronger concept of *Blackwell optimality* is relevant here. A policy π is said to be *Blackwell optimal* if there exists a λ^* for which π is discount optimal for all $\lambda \in [\lambda^*, 1)$.

for all $d \in D^{\text{MD}}$.

From Theorem 7.4, there exist g^{δ^∞} and $\mathbf{h}^{\delta^\infty}$ for which

$$\mathbf{v}_{\lambda_n}^{\delta^\infty} = \frac{g^{\delta^\infty}\mathbf{e}}{1-\lambda_n} + \mathbf{h}^{\delta^\infty} + \mathbf{f}(\lambda_n), \tag{7.55}$$

where $\mathbf{f}(\lambda_n) \to \mathbf{0}$ as $n \to \infty$.

Now substitute (7.55) into the right hand side of (7.54) to obtain for *any* $d' \in D^{\text{MD}}$ that

$$\begin{aligned}\mathbf{r}_{d'} + (\lambda_n \mathbf{P}_{d'} - \mathbf{I})\mathbf{v}_{\lambda_n}^{\delta^\infty} &= \mathbf{r}_{d'} + (\lambda_n \mathbf{P}_{d'} - \mathbf{I})\left(\frac{g^{\delta^\infty}\mathbf{e}}{1-\lambda_n} + \mathbf{h}^{\delta^\infty} + \mathbf{f}(\lambda_n)\right) \\ &= \mathbf{r}_{d'} - g^{\delta^\infty}\mathbf{e} + (\lambda_n \mathbf{P}_{d'} - \mathbf{I})\mathbf{h}^{\delta^\infty} + \mathbf{f}'(\lambda_n), \end{aligned} \tag{7.56}$$

where $\mathbf{f}'(\lambda_n) \to \mathbf{0}$ as $n \to \infty$. Choose d' equal to δ and d, substitute this expression into (7.54) and let $n \to \infty$ to yield

$$\mathbf{0} = \mathbf{r}_\delta - g^{\delta^\infty}\mathbf{e} + (\mathbf{P}_\delta - \mathbf{I})\mathbf{h}^{\delta^\infty} \geq \mathbf{r}_d - g^{\delta^\infty}\mathbf{e} + (\mathbf{P}_d - \mathbf{I})\mathbf{h}^{\delta^\infty}$$

for all $d \in D^{\text{MD}}$. From this it follows that:

$$\mathbf{0} = \underset{d' \in D^{\text{MD}}}{\text{c-max}}\{\mathbf{r}_{d'} - g^{\delta^\infty}\mathbf{e} + (\mathbf{P}_{d'} - \mathbf{I})\mathbf{h}^{\delta^\infty}\}. \tag{7.57}$$

Hence part 1 follows. Moreover since the pair $(g^{\delta^\infty}, \mathbf{h}^{\delta^\infty})$ satisfy the optimality equation, part 3 of Theorem 7.7 implies that $g^{\delta^\infty} = g^*$, so δ^∞ is average optimal. □

7.5.4 The Bellman equation and optimal policies

The following result shows how to use solutions of the Bellman equation to identify optimal policies.

Theorem 7.9. Suppose $(g^*, \mathbf{h}^*)$ are solutions of the Bellman equation and for all $s \in S$

$$d^*(s) \in \underset{a \in A_s}{\arg\max}\left\{r(s,a) + \sum_{j \in S} p(j|s,a)h^*(j)\right\} \tag{7.58}$$

or equivalently

$$d^* \in \underset{d \in D^{\text{MD}}}{\arg\,\text{c-max}}\left\{\mathbf{r}_d + \mathbf{P}_d\mathbf{h}^*\right\}. \tag{7.59}$$

Then $(d^*)^\infty$ is average optimal.

Proof. From (7.59)

$$\mathbf{h}^* = \max_{d \in D^{\text{MD}}}\{\mathbf{r}_d - g^*\mathbf{e} + \mathbf{P}_d\mathbf{h}^*\} = \mathbf{r}_{d^*} - g^*\mathbf{e} + \mathbf{P}_{d^*}\mathbf{h}^*.$$

As a consequence of Corollary 7.4, $g^* = g^{(d^*)^\infty}$ establishing the result. □

Some comments about this result follow:

1. In a finite action model, there always exists such a d^*. This result holds in greater generality but requires some technical considerations.

2. Surprisingly, neither (7.58) nor (7.59) includes g^* directly. However, they do so indirectly since the pair $(g^*, \mathbf{h}^*)$ satisfies the Bellman equation. Moreover, h^* is defined using g^*.

3. Some authors refer to any decision rule d that satisfies

$$d \in \arg\max_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \mathbf{P}_d \mathbf{h}\}$$

   as *greedy* with respect to $\mathbf{h}$.

4. As (7.58) and (7.59) suggest, d^* need not be unique.

Does the gain and bias of every average optimal policy satisfy the Bellman equation?

Theorem 7.9 does *not* imply that the gain and bias of *every* average optimal policy satisfies the Bellman equation. In particular, if d^* is average optimal, then it must satisfy (7.58) for all recurrent states but need not satisfy the optimality equation for transient states. This is because the reward accumulated in transient states, which in expectation are only visited a finite number of times, does not affect the overall average reward of a policy. Thus, average optimal decision rules do not discriminate on the basis of behavior in transient states.

The following example sheds some light on the relationship between solutions of the average optimality equation and average optimal policies. It shows that there exist average optimal policies with gain and bias that do not satisfy the Bellman equation. In other words, the Bellman equation is sufficient, but not necessary, for optimality, in contrast to previous chapters.

Moreover the following example delves into why the gain and bias of certain average optimal policies satisfy the Bellman equation while others do not.

Example 7.7. Let $S = \{s_1, s_2\}$; $A_{s_1} = \{a_{1,1}, a_{1,2}\}$, $A_{s_2} = \{a_{2,1}\}$; $p(s_1|s_1, a_{1,1}) = p(s_2|s_1, a_{1,1}) = 0.5$, $p(s_2|s_1, a_{1,2}) = 1$, $p(s_2|s_2, a_{2,1}) = 1$; and $r(s_1, a_{1,1}) = 5$, $r(s_1, a_{1,2}) = 10$ and $r(s_2, a_{2,1}) = 1$. Rewards need not be defined for zero probability transitions. See Figure 7.7[a].

Let δ denote the decision rule that uses action $a_{1,1}$ in s_1 and γ denote the decision rule which uses action $a_{1,2}$ in s_1. There is no action choice in s_2. This model is unichain with s_2 absorbing and s_1 transient under both stationary

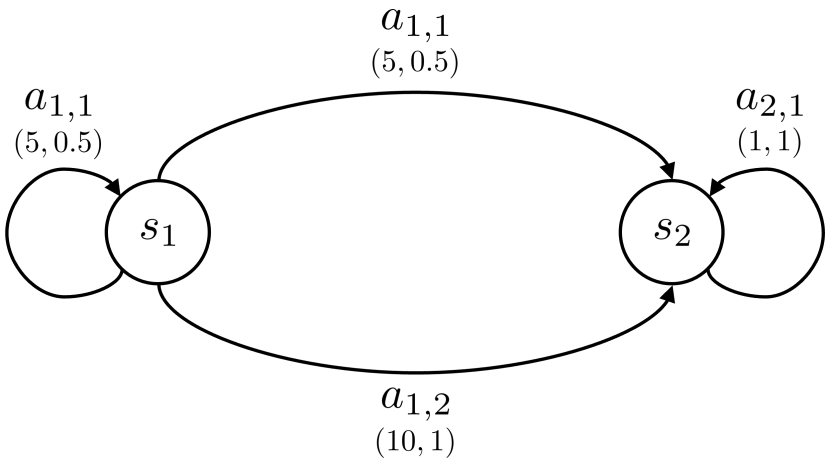


Figure 7.7: Graphical representation of model in Example 7.7. The labels (r, p) refer to the reward and transition probabilities, respectively, associated with using the specified action in the given state.

policies. Clearly

$$g^{\delta^\infty} = g^{\gamma^\infty} = g^* = 1$$

so that *both* deterministic stationary policies are average optimal.

To find the gain and bias of each stationary policy solve

$$\mathbf{h} = \mathbf{r}_d - g\mathbf{e} + \mathbf{P}_d\mathbf{h}$$

subject to $\mathbf{P}_d^*\mathbf{h} = \mathbf{0}$. Since

$$\mathbf{P}_\delta^* = \mathbf{P}_\gamma^* = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$$

$h^{\delta^\infty}(s_2) = h^{\gamma^\infty}(s_2) = 0$ so that it follows that $h^{\delta^\infty}(s_1) = 8$ and $h^{\gamma^\infty}(s_1) = 9$. From the Bellman equation,

$$\begin{aligned} h(s_1) &= \max\{5 - g + 0.5h(s_1) + 0.5h(s_2), 10 - g + h(s_2)\} \\ h(s_2) &= 1 - g + h(s_2) \end{aligned}$$

it follows that $(g^{\gamma^\infty}, \mathbf{h}^{\gamma^\infty})$ satisfies the Bellman equation but $(g^{\delta^\infty}, \mathbf{h}^{\delta^\infty})$ does not.

[a] A variant of this example is analyzed throughout Puterman [1994], most notably in slightly modified form as Example 8.4.3.

In this example, both policies maximize the gain. Moreover, starting in state s_1, each accumulates an expected total reward of 10 prior to reaching s_2. But one would believe that a decision maker would prefer the policy γ^∞ to δ^∞ because the reward of

10 would be received sooner and with certainty.

This anomaly can be understood in two ways:

1. The proof of Theorem 7.8 constructs a solution to the Bellman equation using the partial Laurent series expansion of a policy that is discount optimal for a sequence of λ converging to one. Thus the solution of the Bellman equation corresponds to the gain and bias (up to a constant) of a discount optimal policy for λ near 1. It is easy to prove[24] that γ^∞ is discount optimal for λ near 1, while δ^∞ is not.

2. Computing the bias of each policy in s_1 is constructive. From the definition of bias in (7.17);

$$h^{\gamma^\infty}(s_1) = r(s_1, a_{1,2}) - g^{\gamma^\infty} = 10 - 1 = 9$$
$$h^{\delta^\infty}(s_1) = (5-1) + 0.5(5-1) + (0.5)^2(5-1) + \ldots = 8.$$

   Hence the bias of γ^∞ is greater than that of δ^∞.

As a consequence of the second observation above, one might conjecture that the solution of the Bellman equation also maximizes the bias over all average optimal policies. This is not true in general[25].

Note that the above example might seem a bit contrived, however the same issue arises in many practical problems such as queuing admission control[26] and (s, S)-inventory models. In such models good policies drive the system to a set of recurrent states where it remains in perpetuity so that actions chosen on transient states do not affect the average reward.

7.5.5 State-action value functions

As in the cases of previously considered optimality criteria, in average reward models the policy evaluation equation and Bellman equation can be expressed in terms of state-action value functions. This representation will not be central to this chapter, but will be fundamental in Chapter 10 which studies simulation methods for average reward models. Discussion will be restricted to recurrent/unichain models and be brief.

Let g^* denote the optimal average reward and π^* denote an optimal policy. Define the *optimal state-action value function* as follows.

[24] Apply policy iteration starting from policy γ^∞.

[25] See Chapter 10 of Puterman [1994] for a detailed discussion of this issue.

[26] See Haviv and Puterman [1998] for an analysis of this issue in a continuous time queuing model.

Definition 7.9. The *optimal state-action value function*, $q^*(s,a)$, for all $a \in A_s$ and $s \in S$, is defined as

$$q^*(s,a) := E^{\pi^*}\left[\sum_{n=1}^{\infty}\left(r(X_n,Y_n)-g^*\right)\middle| X_1=s, Y_1=a\right]. \tag{7.60}$$

Separating out the first term it follows that

$$\begin{aligned} q^*(s,a) &= r(s,a) - g^* + E^{\pi}\left[\sum_{n=2}^{\infty}\left(r(X_n,Y_n)-g^*\right)\middle| X_1=s, Y_1=a\right] \\ &= r(s,a) - g^* + \sum_{j\in S} p(j|s,a)h^*(j) \end{aligned} \tag{7.61}$$

where the scalar g^* and $h^*(s)$ for all $s \in S$ solve the recurrent/unichain optimality equation (7.46). Since $h^*(s)$ is unique up to a scalar factor, the state-action value function depends on its specification. The most common choices are the bias or a relative value.

For a stationary policy[27] d^∞, the *policy-specific state-action value function* is defined by

$$q^{d^\infty}(s,a) := r(s,a) - g^{d^\infty} + \sum_{j\in S} p(j|s,a)h^{d^\infty}(j) \tag{7.62}$$

for all $a \in A_s$ and $s \in S$ where g^{d^∞} and $h^{d^\infty}(s)$ solve the policy evaluation equations defined in Theorem 7.6. Again note that $h^{d^\infty}(s)$ is unique up to a constant.

Bellman equation for state-action value functions

As noted above, g^* and $h^*(s)$ are solutions of the Bellman equation in recurrent and unichain models. Hence

$$h^*(s) = \max_{a\in A_s}\left\{r(s,a) - g^* + \sum_{j\in S} p(j|s,a)h^*(j)\right\}.$$

Setting

$$q^*(s,a) = r(s,a) - g^* + \sum_{j\in S} p(j|s,a)h^*(j) \tag{7.63}$$

it follows from above that

$$h^*(s) = \max_{a\in A_s} q^*(s,a).$$

[27] Note the state-action value functions can be defined for arbitrary policies but this level of generality will not be needed.

Hence substituting this representation into (7.63) establishes that the Bellman equation expressed in terms of state-action value functions is equivalent to finding a scalar g^* and a function $q^*(s,a)$ defined for $s \in S$ and $a \in A_s$ that satisfy:

Average reward Bellman equation for state-action value functions:

$$q(s,a) = r(s,a) - g + \sum_{j \in S} p(j|s,a) \max_{a' \in A_s} q(j,a'). \tag{7.64}$$

When g^* and $q^*(s,a)$ are solutions of (7.64), choosing $d^*(s) \in \arg\max_{a' \in A_s} q^*(s,a')$ for all $\in S$ identifies an optimal stationary policy.

Note that:

1. Equation (7.64) involves action choice at *both* the current decision epoch and the subsequent decision epoch.

2. The solution of (7.64) need not be unique. One way of resolving this is to add the extra condition that $q(s^*,a^*) = 0$ for some designated state-action pair (s^*,a^*). This approach is generalized when discussing relative Q-learning in Chapter 10.

3. To evaluate the deterministic stationary policy d^∞, the analogous evaluation equation would be

$$q(s,a) = r(s,a) - g + \sum_{j \in S} p(j|s,a) q(j,d(s)). \tag{7.65}$$

Written as an expected value, (7.64) is equivalent to

$$q(s,a) = E^{\pi^*}\left[r(X,Y) - g + \max_{a \in A_{X'}} q(X',a) \,\middle|\, X = s, Y = a \right], \tag{7.66}$$

where π^* is an optimal policy and X' is a random variable distributed according to $p(\cdot|s,a)$. The previous equation extends to models in which $r(s,a,j)$ is the model primitive as follows:

$$q(s,a) = E^{\pi^*}\left[r(X,Y,X') - g + \max_{a \in A_{X'}} q(X',a) \,\middle|\, X = s, Y = a \right]. \tag{7.67}$$

Similar to state-action value functions in discounted Markov decisions processes, these expressions differ from the usual Bellman equation in that the maximization is *inside* the expectation. This will be important in Chapter 10, where expectations are estimated by simulated or realized rewards and transitions.

7.5.6 The Bellman equation in multi-chain models*

For completeness, this brief section[28] provides the Bellman equation for multi-chain models.

In component notation there are two equations, given by

$$g(s) = \max_{a \in A_s} \left\{ \sum_{j \in S} p(j|s,a) g(j) \right\} \tag{7.68}$$

$$h(s) = \max_{a \in G_s} \left\{ r(s,a) - g(s) + \sum_{j \in S} p(j|s,a) h(j) \right\} \tag{7.69}$$

where

$$G_s := \left\{ a' \in A_s \;\middle|\; g(s) = \sum_{j \in S} p(j|s,a') g(j) \right\}$$

for all $s \in S$.

Note that:

1. This pair of equations is said to be *nested* because the maximization in the second equation is over actions that achieve the maximum in the first equation.

2. The equations uniquely determine $g(s)$ and determine $h(s)$ up to one or more constants. The number of constants depends on the chain structure of optimal policies.

3. In a recurrent/unichain model, the gain of every policy is constant so that the first equation only guarantees that $g(s)$ is constant, in which case these reduce to recurrent/unichain Bellman equation.

4. The multi-chain equations are required in communicating or weakly communicating models because in such models, some policies might have non-constant gain.

5. The most direct way to solve these equations is by policy iteration. The proof of convergence is quite involved.

7.6 Value iteration

Discussion now shifts to algorithms for solving average reward models. Average reward value iteration is based on the following recursion expressed in component notation as

[28] A comprehensive discussion of multi-chain average reward models appears in Chapter 9 of Puterman [1994].

$$v'(s) \leftarrow \max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a) v(j) \right\} \tag{7.70}$$

or equivalently in vector form as:

$$\mathbf{v}' \leftarrow L\mathbf{v} := \underset{d \in D^{\mathrm{MD}}}{\text{c-max}} \{ \mathbf{r}_d + \mathbf{P}_d \mathbf{v} \}. \tag{7.71}$$

The astute reader will note that these recursions have a similar form to value iteration in expected total reward models. In that case, a convergent algorithm was constructed by restricting attention to positive, negative, transient and stochastic shortest path models. A relevant explicit or implicit feature of these cases was that in all them, the system reached zero-reward absorbing states so that the optimal average reward equaled zero. In this chapter, models have non-zero average rewards so different analyses are required.

Consequently, several challenges emerge to create a convergent algorithm from these recursions.

1. The average reward g does not appear in the above expressions; a formal algorithm requires a method to approximate the optimal gain from a sequence of iterates.

2. The iterates of an algorithm based on these recursions may diverge; a modifications is required to obtain a convergent sequence.

3. The iterates of an algorithm based on these recursions may oscillate. Models that have this property must be characterized so that approaches to address this difficulty can be developed.

4. Span-based stopping criteria may not result in algorithm termination in multichain models.

7.6.1 Examples

The following two examples illustrate the challenges of using value iteration in average reward models.

A straightforward example

The first example applies value iteration to the two-state model to illustrate that values may diverge.

Example 7.8. Consider the two-state example from Section 2.5. Begin the iterative scheme implicit in (7.70) or (7.71) at $\mathbf{v}^0 = \mathbf{0}$ to obtain the sequence of iterates $\mathbf{v}^n$ reported in Table 7.5.

Observe that the iterates diverge but that $\mathrm{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n)$ converges to zero exponentially quickly. Observe further that for $s \in S$, $v^{10}(s) - v^9(s) = 2.857 = g^*$ in agreement with Example 7.5. See (7.34) for an indication why this might be the case. This will be made precise below.

n	$v^n(s_1)$	$v^n(s_2)$	$\mathrm{sp}(\mathbf{v}^n - \mathbf{v}^{n-1})$
0	0.00000	0.00000	n/a
1	5.00000	2.00000	3.0000000
2	7.40000	5.20000	0.8000000
3	10.20000	8.08000	0.0800000
4	13.08000	10.92800	0.0320000
5	15.92800	13.78880	0.0128000
6	18.78880	16.64448	0.0051200
7	21.64448	19.50221	0.0020480
8	24.50221	22.35912	0.0008192
9	27.35912	25.21635	0.0003277
10	30.21635	28.07346	0.0001311

Table 7.5: Iterates of value iteration obtained using the methods in Example 7.8.

A periodic example

The following example provides an "edge-case" in which the iterates of value iteration oscillate.

Example 7.9. Consider a two-state periodic model with $S = \{s_1, s_2\}$ and a single decision rule $d \in D^{\mathrm{MD}}$ with reward and transition probabilities given by:

$$\mathbf{r}_d = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{P}_d = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Set $\mathbf{v}^0 = \begin{bmatrix} a \\ b \end{bmatrix}$. Since $\mathbf{v}^n = \mathbf{P}_d^n \mathbf{v}^0$,

$$\mathbf{v}^n = \begin{bmatrix} a \\ b \end{bmatrix} \quad \text{for } n \text{ even and} \quad \mathbf{v}^n = \begin{bmatrix} b \\ a \end{bmatrix} \quad \text{for } n \text{ odd.}$$

Hence, unless $a = b$, the iterates oscillate. Moreover, $\mathrm{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n) = 2|a - b|$.

Since

$$\mathbf{P}_d^* = \begin{bmatrix} 1/2 & 1/2 \\ \\ 1/2 & 1/2 \end{bmatrix},$$

$g^{d^\infty} = 0$.

These observations suggest that in a periodic model value iteration oscillates and need not converge with respect to the span semi-norm.

7.6.2 Removing periodicity

Periodicity is the most significant challenge to developing a convergent value iteration algorithm. In practice most applications will be aperiodic. However, a model with periodic policies can be transformed into an aperiodic model by a simple transformation that preserves values (up to constant) and optimality by a simple perturbation.

To do so, for $0 < \tau < 1$ define a transformed model in which rewards and transition probabilities are denoted by a tilde as follows:

$$\tilde{r}(s,a) = \tau r(s,a) \quad \text{and} \quad \tilde{p}(j|s,a) = (1-\tau)\delta(j|s) + \tau p(j|s,a) \tag{7.72}$$

for $s \in S$, $a \in A_s$ and $j \in S$ where $\delta(j|s) = 1$ if $j = s$ and 0 otherwise.

Alternatively in vector notation this transformation can be expressed for $d \in D^{\text{MD}}$ by:

$$\tilde{\mathbf{r}}_d = \tau \mathbf{r}_d \quad \text{and} \quad \tilde{\mathbf{P}}_d = (1-\tau)\mathbf{I} + \tau \mathbf{P}_d. \tag{7.73}$$

This transformation adds a self-transition to the model so that no Markovian decision rule has a periodic transition matrix. Another way to think about this transformation is to regard it as the result of inspecting the state more frequently in a model in which transitions occur uniformly throughout the period between two decision epochs. For example if $\tau = 0.5$, the system state is viewed twice as often. Consequently at such an inspection time, a transition may have occurred with probability τ and the system remains in the same state with probability $1 - \tau$.

Example 7.10. Apply the transformation to the model in Example 7.9 above to obtain:

$$\tilde{\mathbf{r}}_d = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \text{and} \quad \tilde{\mathbf{P}}_d = \begin{bmatrix} 1-\tau & \tau \\ \tau & 1-\tau \end{bmatrix}.$$

The transformed model has period 1 so that it is aperiodic.

Since the iterates of value iteration satisfy $\mathbf{v}^n = \tilde{\mathbf{P}}_d^n \mathbf{v}^0$, an eigenvalue decomposition of $\tilde{\mathbf{P}}_d$ simplifies calculations. Applying the eigenvalue decomposition Theorem B.2 in Appendix B results in

$$\mathbf{\Lambda} = \begin{bmatrix} 1 & 0 \\ 0 & 1-2\tau \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad \text{and} \quad \mathbf{W}^{-1} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix}.$$

So,

$$\tilde{\mathbf{P}}_d^n = \mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1} = \frac{1}{2}\begin{bmatrix} 1+(1-2\tau)^n & 1-(1-2\tau)^n \\ 1-(1-2\tau)^n & 1+(1-2\tau)^n \end{bmatrix}.$$

Consequently for $\mathbf{v}^0 = \begin{bmatrix} a \\ b \end{bmatrix}$

$$\mathbf{v}^{n+1} - \mathbf{v}^n = \tilde{\mathbf{P}}_d^n(\tilde{\mathbf{P}}_d - \mathbf{I})\mathbf{v}_0 = \tau\tilde{\mathbf{P}}_d^n \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \mathbf{v}^0 = \tau(1-2\tau)^n \begin{bmatrix} b-a \\ a-b \end{bmatrix}.$$

Therefore $\text{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n) = 2\tau(1-2\tau)^n|b-a|$ converges to 0 as $n \to \infty$ so that in this modified model, value iteration is convergent with respect to the span semi-norm.

The following result shows that this transformation preserves solutions of the optimality equation and consequently optimal policies.

Proposition 7.2. Suppose that the pair $(g^*, \mathbf{h}^*)$ satisfies (7.47). Then

1. The pair $(\tau g^*, \mathbf{h}^*)$ satisfies

$$\mathbf{h} = \underset{d \in D^{\text{MD}}}{\text{c-max}} \{\tilde{\mathbf{r}}_d - g\mathbf{e} + \tilde{\mathbf{P}}_d \mathbf{h}\}. \tag{7.74}$$

2. The same policy is optimal for both models.

Proof. Rewriting (7.47),

$$\mathbf{h}^* + g^*\mathbf{e} = \underset{d \in D^{\text{MD}}}{\text{c-max}} \{\mathbf{r}_d + \mathbf{P}_d \mathbf{h}^*\}.$$

Multiply this equation by τ and add $(1-\tau)\mathbf{h}^*$ to both sides to obtain

$$\mathbf{h}^* + \tau g^*\mathbf{e} = \underset{d \in D^{\text{MD}}}{\text{c-max}} \{\tau\mathbf{r}_d + (1-\tau)\mathbf{I}\mathbf{h}^* + \tau\mathbf{P}_d\mathbf{h}^*\} = \underset{d \in D^{\text{MD}}}{\text{c-max}} \{\tilde{\mathbf{r}}_d + \tilde{\mathbf{P}}_d\mathbf{h}^*\},$$

which establishes the result. □

Observe that the above result holds regardless how $\mathbf{h}^*$ is specified. Moreover, this result is consistent with interpreting the aperiodic transformation as increased model inspection. When the state is inspected twice as often, that is $\tau = 0.5$, the per period reward is half as great, that is $0.5g^*$

Replacing D^{MD} by a single policy establishes:

Corollary 7.5. Suppose for some $d \in D^{\text{MD}}$ that the pair $(g^{d^\infty}, \mathbf{h}^{d^\infty})$ satisfy (7.43), then the pair $(\tau g^{d^\infty}, \mathbf{h}^{d^\infty})$ satisfy

$$\mathbf{h} = \tilde{\mathbf{r}}_d - g\mathbf{e} + \tilde{\mathbf{P}}_d\mathbf{h}.$$

Since this transformation applies to any model, the remainder of this section assumes that *all policies are aperiodic*. On occasion, this will be restated for emphasis.

7.6.3 A value iteration algorithm

This section states and investigates the properties of the following value iteration algorithm expressed in component form:

Algorithm 7.1. Value iteration for an average reward model: component form

1. **Initialize:** Specify $v'(s)$ for all $s \in S$, $\epsilon > 0$ and $\sigma > \epsilon$.
2. **Iterate:** While $\sigma \geq \epsilon$:
   (a) $v(s) \leftarrow v'(s)$ for all $s \in S$.
   (b) For all $s \in S$,
   $$v'(s) \leftarrow \max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a)v(j) \right\}. \tag{7.75}$$
   (c) $\sigma \leftarrow \text{sp}(\mathbf{v}' - \mathbf{v})$.
3. **Terminate:** For all $s \in S$, return
   $$d_\epsilon(s) \in \arg\max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a)v(j) \right\}$$
   and
   $$g^\epsilon = \frac{1}{2}\left(\max_{s \in S}\{v'(s) - v(s)\} + \min_{s \in S}\{v'(s) - v(s)\} \right). \tag{7.76}$$

Note that the recursion (7.75) is identical to that in the expected total reward model, however the above algorithm uses a span-based stopping criterion and returns an estimate of the average reward.

Convergence of value iteration in unichain and recurrent models*

In the absence of contraction properties that were applicable in discounted models and in transient expected total reward models, a proof that value iteration converges and eventually achieves the span-based stopping criterion requires subtle analysis. For ease of exposition, this convergence is established under the assumption that the transition probabilities of every stationary policy are aperiodic and *recurrent or unichain*. Almost all of the results hold under the weaker condition of communicating or weakly communicating models, which ensure that the optimal gain is constant[29].

A key technical result

A proof of the convergence of value iteration in an average reward model is based on the following technical result.

Proposition 7.3. Assume an aperiodic recurrent or unichain model and let $v^n(s)$ denote a sequence of iterates generated by the average reward value iteration algorithm. Then

$$\lim_{n\to\infty} \left(v^n(s) - ng^* - h^*(s)\right) \tag{7.77}$$

exists where g^* and $h^*(s)$ are solutions of the Bellman equation (7.46).

A formal proof is omitted but follows from the steps outlined below.

1. Establish upper and lower bounds on $e_n(s) := v^n(s) - ng^* - h^*(s)$ for each $s \in S$.
2. Use the well-known result that every bounded infinite sequence contains a convergent subsequence[30].
3. Show that $\lim_{n\to\infty} e_n(s)$ exists for s in the recurrent states of an optimal policy by proving that the $\liminf_{n\to\infty} e_n(s) = \limsup_{n\to\infty} e_n(s)$.
4. Show that the above limit exists on transient states by combining a modified argument to that on recurrent states and then appealing to the result on recurrent states.

[29] This development is adapted from the analysis of multi-chain value iteration in Section 9.4 of Puterman [1994].

[30] Known as the Bolzano-Weierstrass Theorem.

The hypothesis of this proposition can be weakened to "there exists an optimal policy with an aperiodic transition probability matrix". Alternatively, by applying the aperiodicity transformation (7.72) prior to analysis, this applies to all recurrent and unichain models. Note that this result is valid in multi-chain models as well but requires a subtle intermediate step[31].

Consequences

When the limit in (7.77) exists, the following important results follows easily.

Proposition 7.4. Suppose in an aperiodic recurrent or unichain model that the sequence $v^n(s)$ is generated by (7.75) with $v^0(s)$ arbitrary and that the limit in (7.77) exists.

Then for all $s \in S$,

$$\lim_{n\to\infty} (v^{n+1}(s) - v^n(s)) = g^* \tag{7.78}$$

and

$$\lim_{n\to\infty} \operatorname{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n) = 0. \tag{7.79}$$

Proof. To prove the first result choose $s' \in S$ and denote the limit in (7.77) by K. Then given $\epsilon > 0$ there exists an N such that for $n \geq N$

$$|v^n(s') - ng^* - h^*(s') - K| < \epsilon.$$

Hence

$$\begin{aligned}
&|v^{n+1}(s') - v^n(s') - g^*| \\
&\quad = |(v^{n+1}(s') - (n+1)g^* - h^*(s') - K) - (v^n(s') - ng^* - h^*(s') - K)| \\
&\quad \leq |v^{n+1}(s') - (n+1)g^* - h^*(s') - K| + |v^n(s') - ng^* - h^*(s') - K| \\
&\quad < 2\epsilon.
\end{aligned}$$

Since s' and ϵ were arbitrary, the result follows.

To prove the second result, let

$$s^+ = \arg\max_{s\in S}\{v^{n+1}(s) - v^n(s)\} \text{ and } s^- = \arg\min_{s\in S}\{v^{n+1}(s) - v^n(s)\}.$$

Since g^* is constant, it follows from the first result that

$$\operatorname{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n) = \left(v^{n+1}(s^+) - v^n(s^+)\right) - \left(v^{n+1}(s^-) - v^n(s^-)\right) \tag{7.80}$$

$$= \left(v^{n+1}(s^+) - v^n(s^+) - g^*\right) - \left(v^{n+1}(s^-) - v^n(s^-) - g^*\right) \tag{7.81}$$

[31] Details appear in Section 9.4.1 in Puterman [1994].

$$\begin{aligned} &< |v^{n+1}(s^+) - v^n(s^+) - g^*| + |v^{n+1}(s^-) - v^n(s^-) - g^*| \qquad (7.82)\\ &< 4\epsilon \end{aligned}$$

Hence the second result follows. Note that the assumption of constant g^* was crucial to expression (7.81). □

The following result provides bounds on the optimal gain. It can be used to identify an ϵ-optimal policy when using value iteration and also estimate the optimal gain at termination. Moreover it may be useful for estimating the optimal gain when using simulation methods to solve average reward problems.

Proposition 7.5. In an aperiodic recurrent or unichain model, for any $v(s)$,

$$\min_{s'\in S}\{L\mathbf{v}(s') - v(s')\} \leq g^{d^\infty} \leq g^* \leq \max_{s'\in S}\{L\mathbf{v}(s') - v(s')\}, \qquad (7.83)$$

where

$$d(s) \in \arg\max_{a\in A_s}\left\{r(s,a) + \sum_{j\in S} p(j|s,a)v(j)\right\}. \qquad (7.84)$$

Proof. First establish the lower bound. Suppose d satisfies (7.84) and let R_d denote its set of recurrent states. Then since $\mathbf{P}_d^* = \mathbf{P}_d^*\mathbf{P}_d$

$$\begin{aligned} g^{d^\infty}\mathbf{e} = \mathbf{P}_d^*\mathbf{r}_d = \mathbf{P}_d^*(\mathbf{r}_d + \mathbf{P}_d\mathbf{v} - \mathbf{v}) &= \mathbf{P}_d^*(L\mathbf{v} - \mathbf{v})\\ &\geq \min_{s\in R_d}\{L\mathbf{v}(s) - v(s)\}\mathbf{e} \geq \min_{s\in S}\{L\mathbf{v}(s) - v(s)\}\mathbf{e}. \end{aligned}$$

Note that the minimum in the first inequality above is over R_d because columns corresponding to transient states of $\mathbf{P}_d$ will have zeroes in columns of $\mathbf{P}_d^*$ corresponding to transient states.

To establish the upper bound, from Theorem 7.8, there exists a g^* and a decision rule d^* for which

$$\begin{aligned} g^*\mathbf{e} = \mathbf{P}_{d^*}^*\mathbf{r}_{d^*} = \mathbf{P}_{d^*}^*(\mathbf{r}_{d^*} + \mathbf{P}_{d^*}\mathbf{v} - \mathbf{v}) &= \mathbf{P}_{d^*}^*(L\mathbf{v} - \mathbf{v})\\ &\leq \max_{s\in R_{d^*}}\{L\mathbf{v}(s) - v(s)\}\mathbf{e} \leq \max_{s\in S}\{L\mathbf{v}(s) - v(s)\}\mathbf{e}. \end{aligned}$$

□

Combining the above propositions gives the main result of this section. The second and third parts follow immediately from Proposition 7.5.

Theorem 7.10. Let $v^n(s)$ denote the sequence of iterates of value iteration in an aperiodic recurrent or unichain model. Then:

1. Value iteration converges and the stopping criterion in step 2 of the algorithm is satisfied for n sufficiently large.

2. For every $n > 0$,

$$\min_{s\in S}\left\{v^{n+1}(s) - v^n(s)\right\} \leq g^{d^\infty} \leq g^* \leq \max_{s\in S}\left\{v^{n+1}(s) - v^n(s)\right\} \tag{7.85}$$

   where

$$d(s) \in \arg\max_{a\in A_s}\left\{r(s,a) + \sum_{j\in S} p(j|s,a)v^n(j)\right\}. \tag{7.86}$$

3. If $\text{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n) < \epsilon$ for some $\epsilon > 0$ then

$$\left|\max_{s\in S}\left\{v^{n+1}(s) - v^n(s)\right\} + \min_{s\in S}\left\{v^{n+1}(s) - v^n(s)\right\} - 2g\right| < \epsilon \tag{7.87}$$

   for $g = g^*$ or $g = g^{d^\infty}$ where d satisfies (7.86).

Some comments about the above results follow:

1. Part 1 of Theorem 7.10 gives the most significant result. Namely in a model in which $g^*(s)$ is constant, the value iteration algorithm converges and the span-based stopping criterion is achieved. Aperiodicity is crucial for this result. Without it, the iterates could oscillate and not satisfy the stopping criterion.

2. In a model in which optimal policies are multi-chain, the algorithm will converge, but since the optimal average reward need not be constant, the span need not converge to zero and hence does not provide a valid stopping rule.

3. As noted above, the assumption that the model be aperiodic is not limiting. It can always be achieved by applying the aperiodicity transformation described in the preceding section.

4. The proof of part 2 of Proposition 7.4 holds under the weaker assumption that the optimal gain is constant. This is true when either:

   (a) The model is regular or unichain.

   (b) The model is communicating.

   (c) The model is weakly communicating.

   Thus this result holds in considerable generality.

5. Part 1 of Proposition 7.4 justifies the approximation

$$g^*(s) \approx v^{n+1}(s) - v^n(s).$$

Part 3 of Theorem 7.10 makes it precise and justifies the calculation in the step "Approximate the optimal average reward" in the value iteration algorithm statements.

6. Part 2 of Theorem 7.10 shows how to obtain an ϵ-optimal policy d^∞ when the condition

$$\text{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n) < \epsilon$$

holds. It provides the basis for the estimate in (7.76).

Referring back to Example 7.8, Table 7.5 shows that $\text{sp}(\mathbf{v}^{n+1} - \mathbf{v}^n)$ converges to zero. At termination, (7.76) gives the estimate $g^* = 2.857$.

7.6.4 Relative value iteration

Observe that in Example 7.8 the iterates of value iteration diverge. While this does not impact theory, it might become problematic in practice, especially in large models. To overcome this and as well provide a basis for a simulation approach, *relative value iteration* can be used instead. Relative value iteration distinguishes a state $\bar{s}$ and normalizes by subtracting $v(\bar{s})$ from $v(s)$ after each iteration. A formal statement of an algorithm follows.

Algorithm 7.2. Relative value iteration: component form

1. **Initialize:** Specify a state $\bar{s}$, $\epsilon > 0$, $\sigma > \epsilon$ and $w'(s)$ satisfying $w'(\bar{s}) = 0$.
2. **Iterate:** While $\sigma \geq \epsilon$:
   (a) For all $s \in S$, $w(s) \leftarrow w'(s)$.
   (b) For all $s \in S$

$$v(s) \leftarrow \max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a) w(j) \right\}. \tag{7.88}$$

   (c) **Normalize:** For all $s \in S$,

$$w'(s) \leftarrow v(s) - v(\bar{s}).$$

   (d) $\sigma \leftarrow \text{sp}(\mathbf{w}' - \mathbf{w})$.

3. **Terminate:** For all $s \in S$, return

$$d_\epsilon(s) \in \arg\max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a) w(j) \right\}$$

and

$$g^\epsilon = \frac{1}{2} \left(\max_{s \in S} \{v(s) - w(s)\} + \min_{s \in S} \{v(s) - w(s)\} \right).$$

To illustrate this algorithm, apply relative value iteration to the model in Example 7.8 normalized by setting $w(s_2) = 0$. Comparing Table 7.6 to Table 7.5 shows that the span expressed in terms of $\mathbf{w}^n$ agrees with that expressed in terms of $\mathbf{v}^n$. (Why?) Moreover the estimates of g^* converge quickly. Note also that $w^n(s_1)$ converges to $h^*_{\text{rel}}(s_1) = 15/7$ (see Example 7.5). Thus with obvious modifications, Theorem 7.10 applies to relative value iteration and moreover, $w^n(s)$ converges to $h^*_{\text{rel}}(s)$.

n	$w^n(s_1)$	$w^n(s_2)$	$\text{sp}(\mathbf{w}^n - \mathbf{w}^{n-1})$	Estimate of g^*
0	0.00000	0	n/a	n/a
1	3.00000	0	3.0	3.5
2	2.20000	0	0.8	2.8
3	2.12000	0	0.08	2.84
4	2.15200	0	0.032	2.864
5	2.13900	0	0.0128	2.8544
6	2.14432	0	0.00512	2.85824
7	2.14227	0	0.002048	2.85670
8	2.14309	0	0.000819	2.85732
9	2.14276	0	0.0003278	2.85707
10	2.14290	0	0.0001311	2.85717

Table 7.6: Iterates of relative value iteration for Example 7.8.

Some concluding remarks regarding average reward value iteration

Given the complexity of this analysis of value iteration, a concluding recommendation may be helpful. In practice, one should use relative value iteration. An aperiodicity transformation can be applied if necessary to ensure convergence in most cases, the exception being multi-chain models in which the optimal policy does *not* have constant gain. Convergent rate estimates require additional analyses beyond the scope of this book.

7.7 Policy iteration

This section describes a policy iteration algorithm for recurrent and unichain models, illustrates its application in an example, proves its convergence and comments on some salient features.

7.7.1 An algorithm

The following policy iteration algorithm, expressed in vector notation, provides a high-level perspective on its key steps.

Algorithm 7.3. Policy iteration for recurrent and unichain models: vector form

1. **Initialize:** Specify $d \in D^{\text{MD}}$ and set $\Gamma \leftarrow S$.
2. While $\Gamma \neq \emptyset$:
   (a) **Evaluate:** Obtain g' and $\mathbf{h}'$ by solving
   $$\mathbf{h} = \mathbf{r}_d - g\mathbf{e} + \mathbf{P}_d\mathbf{h}. \tag{7.89}$$
   (b) **Improve:** Choose
   $$d' \in \arg\underset{\delta \in D^{\text{MD}}}{\text{c-max}}\{\mathbf{r}_\delta + \mathbf{P}_\delta \mathbf{h}'\}. \tag{7.90}$$
   setting $d' = d$ if possible.
   (c) $\Gamma \leftarrow \{s \in S \,|\, d'(s) \neq d(s)\}$.
   (d) **Update:** $d \leftarrow d'$.
3. **Terminate:** Return $d^* = d$, $g^* = g'$ and $\mathbf{h}^* = \mathbf{h}'$.

Its restatement in component notation better describes the details of an implementable algorithm. This description emphasizes that the improvement step is implemented state by state.

Algorithm 7.4. Policy iteration for recurrent and unichain models: component form

1. **Initialize:** Specify $a_s \in A_s$ for all $s \in S$ and set $\Gamma \leftarrow S$.
2. While $\Gamma \neq \emptyset$:

(a) **Evaluate:** Obtain g' and $h'(s)$ for all $s \in S$ by solving the system of linear equations

$$h(s) = r(s, a_s) - g + \sum_{j \in S} p(j|s, a_s)h(j). \tag{7.91}$$

(b) **Improve:** For all $s \in S$, choose

$$a'_s \in \arg\max_{a \in A_s} \left\{ r(s, a) + \sum_{j \in S} p(j|s, a)h'(j) \right\}, \tag{7.92}$$

setting $a'_s = a_s$ if possible.

(c) $\Gamma \leftarrow \{s \in S \,|\, a'_s \neq a_s\}$.

(d) **Update:** For all $s \in S$, $a_s \leftarrow a'_s$.

3. **Terminate:** Return $d^*(s) = a_s$ and $h^*(s) = h'(s)$ for all $s \in S$, and $g^* = g'$.

Some comments follow:

1. The evaluation step uniquely determines g and determines $h(s)$ up to a constant. This arbitrary constant does not impact decision rule or action choice in the subsequent improvement step, however its specification is required to implement it numerically. The easiest approach is to set $h(s') = 0$ for some distinguished state s'. In this case the evaluation step generates relative values. Note this does not apply to multi-chain models.

2. The value of g does not enter into the improvement step. Therefore $h(s)$ contains all of the information required to possibly find an improved decision rule.

3. This algorithm is not impacted by periodicity since solution of the evaluation equation is exact.

4. The improvement step requires the same effort as one pass through value iteration. It is implemented component-wise to avoid enumerating all Markovian deterministic decision rules. Modified policy iteration, which is not described here, replaces this step with an iterative method for approximating the value function of $(d')^\infty$.

5. The instruction "set $d'' = d'$ if possible" prevents cycling when the maximizer is non-unique in some states. It is also fundamental to enable the stopping condition to be satisfied. Without this rule the stopping criterion can be replaced by one based on the value functions being equal at two iterations. However, one must account for the numerical precision to which these quantities have been computed. Alternatively a span-based stopping condition could be used.

7.7.2 An example

This section solves the two-state example from Section 2.5 using the above policy iteration algorithm expressed in component form. It is easy to see that this model is unichain because state s_2 is absorbing under action $a_{2,1}$. This implementation arbitrarily sets $h(s_2) = 0$ for convenience.

1. Choose $a'_{s_1} = a_{1,2}$ and $a'_{s_2} = a_{2,1}$.

2. The evaluation equations (7.91) become:

$$\begin{aligned} h(s_1) &= 5 - g + h(s_2) \\ h(s_2) &= -5 - g + h(s_2). \end{aligned}$$

   Setting $h(s_2) = 0$ gives $g = -5$ and $h(s_1) = 10$.

3. In the improvement step (7.92)

$$\begin{aligned} a''_{s_1} &\in \arg\max_{i \in \{1,2\}} \left\{ r(s_1, a_{1,i}) + \sum_{j=1}^{2} p(s_j|s_1, a_{1,i}) h(s_j) \right\} \\ &= \arg\max\{3 + 0.8h(s_1) + 0.2h(s_2), 5 + h(s_2)\} \\ &= \arg\max\{11, 5\} = a_{1,1} \end{aligned}$$

   and

$$\begin{aligned} a''_{s_2} &\in \arg\max_{i \in \{1,2\}} \left\{ r(s_2, a_{2,i}) + \sum_{j=1}^{2} p(s_j|s_2, a_{2,i}) h(s_j) \right\} \\ &= \arg\max\{-5 + h(s_2), 2 + 0.4h(s_1) + 0.6h(s_2))\} \\ &= \arg\max\{-5, 6\} = a_{2,2} \end{aligned}$$

4. Since $a''_s \neq a'_s$ for both s, replace a'_s by a''_s for all $s \in S$ and return to the evaluation step.

5. Evaluating this decision rule ($(a'_{s_1}, a'_{s_2}) = (a_{1,1}, a_{2,2})$) using (7.91) together with $h(s_2) = 0$ yields $g = 8/3 = 2.6667$ and $h(s_1) = 5/3 = 1.6667$. Hence g has increased.

6. Applying the improvement step gives

$$\begin{aligned} a''_{s_1} &\in \arg\max\{13/3, 5\} = a_{1,2} \\ a''_{s_2} &\in \arg\max\{-5, 8/3\} = a_{2,2}. \end{aligned}$$

7. Since $a''_s \neq a'_s$ for s_1, replace a'_s by a''_s for all $s \in S$ and return to the evaluation step.

8. Evaluating this decision rule ($(a'_{s_1}, a'_{s_2}) = (a_{1,2}, a_{2,2})$) using (7.91) together with $h(s_2) = 0$ yields $g = 20/7 = 2.857$ and $h(s_1) = 15/7 = 2.143$. Hence g has increased.

9. Applying the improvement step again gives

$$a''_{s_1} \in \arg\max\{29.735, 30.147\} = a_{1,2}$$
$$a''_{s_2} \in \arg\max\{20.146, 27.941\} = a_{2,2}.$$

10. Since $a''_s = a'_s$ for all $s \in S$, stop.

The above calculations showed that it required two improvement steps to find the optimal policy and an additional improvement step to confirm its optimality. This is because the algorithm was initiated with the worst policy. A better starting value would have led to faster convergence.

Observe that the algorithm terminates with:

1. the optimal policy d^*,
2. the optimal gain,
3. the relative values[32] of d^*.
4. a solution of the Bellman equation.

Moreover g increased from iteration to iteration. This need not be the case in general; at some iterations g may remain the same and h increase. This observation will be the basis for a convergence proof for policy iteration.

7.7.3 Convergence of recurrent and unichain policy iteration

This section establishes that policy iteration finds an optimal policy in a finite number of iterations. The proof is based on the following observations:

1. If there is a strict improvement in a recurrent state of a decision rule obtained in the improvement step, then the gain increases.
2. If there is a strict improvement in a transient state of a decision rule obtained in the improvement step and no change in action in any recurrent state, then the gain remains the same but the bias increases.

[32] Note that if the condition $h(s_2) = 0$ was replaced by the more computationally intensive condition $\mathbf{P}_{d^*}\mathbf{h} = \mathbf{0}$, $h(s)$ would equal the bias of d^*.

The consequence of this is that the iterates of policy iteration are monotone in a *lexicographic* (or dictionary-ordering) sense[33]. What this means that if the solution of the evaluation equations is expressed as a vector $(g, \mathbf{h})$ then either the first component g increases or g remains the same and $\mathbf{h}$ increases. Note that there are several other ways of proving this result[34]; we believe this approach is the most informative.

Improvements on recurrent states

The following proposition establishes the first statement above. It expresses the gain of a stationary policy δ^∞ in terms of a one-step "Bellman update" of the gain and bias of a different stationary policy d^∞. It then explores the implications of δ^∞ representing a strict improvement of d^∞ in the recurrent states of δ^∞. Note the result applies to any scalar g and $\mathbf{h} \in \mathbb{R}^{|S|}$ but it is stated to be compatible with the average reward policy iteration algorithm. It is often referred to as a "comparison lemma".

Proposition 7.6. Suppose in a recurrent or unichain model that for some $d \in D^{\mathrm{MD}}$, g and $\mathbf{h}$ satisfy

$$\mathbf{0} = \mathbf{r}_d - g\mathbf{e} + \mathbf{P}_d\mathbf{h} - \mathbf{h}. \tag{7.93}$$

Then for any $\delta \in D^{\mathrm{MD}}$:

1. $$g^{\delta^\infty}\mathbf{e} = g\mathbf{e} + \mathbf{P}^*_\delta(\mathbf{r}_\delta - g\mathbf{e} + (\mathbf{P}_\delta - \mathbf{I})\mathbf{h}). \tag{7.94}$$

2. Suppose
$$r(s,\delta(s)) - g + \sum_{j\in S} p(j|s,\delta(s))h(j) - h(s) \geq 0 \tag{7.95}$$
for all recurrent states under $\mathbf{P}_\delta$ with strict inequality for some s that is recurrent under $\mathbf{P}_\delta$, then $g^{\delta^\infty} > g$.

3. If
$$r(s,\delta(s)) - g + \sum_{j\in S} p(j|s,\delta(s))h(j) - h(s) = 0 \tag{7.96}$$
for all s that are recurrent under δ, then $g^{\delta^\infty} = g$.

Proof. Since $g^{\delta^\infty}\mathbf{e} = \mathbf{P}^*_\delta\mathbf{r}_\delta$, $\mathbf{P}^*_\delta(\mathbf{P}_\delta - \mathbf{I}) = \mathbf{0}$ and $\mathbf{P}^*_\delta\mathbf{e} = \mathbf{e}$, it follows that

$$\begin{aligned} g^{\delta^\infty}\mathbf{e} =& \mathbf{P}^*_\delta\mathbf{r}_\delta + g\mathbf{e} - g\mathbf{e} + \mathbf{P}^*_\delta(\mathbf{P}_\delta - \mathbf{I})\mathbf{h} \\ =& g\mathbf{e} + \mathbf{P}^*_\delta(\mathbf{r}_\delta - g\mathbf{e} + (\mathbf{P}_\delta - \mathbf{I})\mathbf{h}) \end{aligned}$$

[33] A vector $\mathbf{a}$ is lexicographicaly larger (smaller) than a vector $\mathbf{b}$ of the same length if the first non-zero component of $\mathbf{a} - \mathbf{b}$ is positive (negative).

[34] One can use the partial Laurent expansion or the representation $\mathbf{v}_n = ng\mathbf{e} + \mathbf{h} + \mathbf{o}(1)$.

from which part 1 follows.

Parts 2 and 3 follow immediately from Lemma B.4 in Appendix B, which shows that the components of the limiting matrix $\mathbf{P}^*_\delta$ satisfy $p^*(j|s) > 0$ when both s and j are recurrent states and $p^*(j|s) = 0$ for all s and all transient states j .

□

Some comments follow:

1. The statement of the above proposition combines vector and component notation. This is intended to make the results as transparent as possible.

2. In the context of policy iteration, δ in part 1. will be chosen according to

$$\delta \in \arg\max_{d' \in D^{\text{MD}}} \{\mathbf{r}_{d'} + \mathbf{P}_{d'}\mathbf{h}\}. \tag{7.97}$$

3. Since $p^*_\delta(j|s) = 0$ when j is transient under δ, changes on transient states of δ do not effect the average reward. Note this structure was fundamental to the analysis of transient expected total reward models.

4. In a recurrent model the above result is sufficient to establish the convergence of policy iteration.

5. This result can be generalized to multi-chain models[35].

Improvement on transient states

The following proposition provides the basis for the statement that there is a strict improvement in a transient state of a greedy decision rule and no change in action in any recurrent state, then the gain remains the same but the bias increases. the beginning of this section. Since it is concerned with behavior on transient states, it is vacuous when applied to a recurrent model. The proof relies heavily on the structure of $\mathbf{P}^*_\delta$ for unichain policies.

The proposition statement is simplified by using the following additional notation. For a unichain decision rule δ, let R_δ and T_δ denote the recurrent and transient states of $\mathbf{P}_\delta$, for an arbitrary vector $\mathbf{v}$ let $\mathbf{v}_R$ and $\mathbf{v}_T$ respectively denote the sub-vectors of $\mathbf{v}$ restricted to R_δ and T_δ and let $(\mathbf{P}_\delta)_{TT}$ denote the sub-matrix of $\mathbf{P}_\delta$ corresponding to transitions between the transient states of $\mathbf{P}_\delta$. The proof is based on applying Theorem 7.3, partitioning

$$\mathbf{H}_d = \begin{bmatrix} (\mathbf{H}_d)_{RR} & (\mathbf{H}_d)_{RT} \\ (\mathbf{H}_d)_{TR} & (\mathbf{H}_d)_{TT} \end{bmatrix} \tag{7.98}$$

for a Markovian decision rule d, and using the properties of the sub-matrices of $\mathbf{H}_d$.

[35] See Lemma 9.2.5 in Puterman [1994].

Proposition 7.7. Suppose in a unichain model that for some $d \in D^{\mathrm{MD}}$, g and $\mathbf{h}$ satisfy (7.93) subject to $\mathbf{P}_d^*\mathbf{h} = \mathbf{0}$. Then for any $\delta \in D^{\mathrm{MD}}$:

1. $$\mathbf{h}^{\delta^\infty} = (\mathbf{I} - \mathbf{P}_\delta^*)\mathbf{h} + \mathbf{H}_\delta(\mathbf{r}_\delta - g\mathbf{e} + \mathbf{P}_\delta\mathbf{h} - \mathbf{h}). \tag{7.99}$$

2. Suppose $\delta(s) = d(s)$ for $s \in R_d$. Then $\mathbf{h}_R^{\delta^\infty} = \mathbf{h}_R$ and
$$\mathbf{h}_T^{\delta^\infty} = \mathbf{h}_T + (\mathbf{I} - \mathbf{P}_\delta)_{TT}^{-1}(\mathbf{r}_\delta - g\mathbf{e} + \mathbf{P}_\delta\mathbf{h} - \mathbf{h})_T. \tag{7.100}$$

3. Suppose
$$r(s,\delta(s)) - g + \sum_{j\in S} p(j|s,\delta(s))h(j) - h(s) = 0 \text{ for } s \in R_\delta \tag{7.101}$$
and
$$r(s,\delta(s)) - g + \sum_{j\in S} p(j|s,\delta(s))h(j) - h(s) \geq 0 \text{ for } s \in T_\delta \tag{7.102}$$
with strict inequality for some $s \in T_\delta$.
Then $g^{\delta^\infty} = g$ and $h^{\delta^\infty}(s) \geq h(s)$ for all $s \in S$ with $h^{\delta^\infty}(s') > h(s')$ for some $s' \in S$.

4. Suppose
$$r(s,\delta(s)) - g + \sum_{j\in S} p(j|s,\delta(s))h(j) - h(s) = 0 \tag{7.103}$$
for all $s \in S$. Then $g^{\delta^\infty} = g$ and $\mathbf{h}^{\delta^\infty} = \mathbf{h}$.

Proof. Since $\mathbf{h}^{\delta^\infty} = \mathbf{H}_\delta\mathbf{r}_\delta$, $\mathbf{H}_\delta\mathbf{e} = \mathbf{0}$ and $\mathbf{H}_\delta(\mathbf{P}_\delta - \mathbf{I}) = \mathbf{P}_\delta^* - \mathbf{I}$ it follows after a bit of manipulation that

$$\begin{aligned}\mathbf{h}^{\delta^\infty} &= \mathbf{H}_\delta\mathbf{r}_\delta + \mathbf{H}_\delta\big((\mathbf{P}_\delta - \mathbf{I})\mathbf{h} - (\mathbf{P}_\delta - \mathbf{I})\mathbf{h}\big)\\ &= (\mathbf{I} - \mathbf{P}_\delta^*)\mathbf{h} + \mathbf{H}_\delta\big(\mathbf{r}_\delta - g\mathbf{e} + (\mathbf{P}_\delta - \mathbf{I})\mathbf{h}\big)\end{aligned}$$

establishing part 1.

To prove part 2, suppose $\delta(s) = d(s)$ when s is recurrent under d, then $(\mathbf{r}_\delta - g\mathbf{e} + \mathbf{P}_\delta\mathbf{h} - \mathbf{h})_R = \mathbf{0}$ by the hypothesis that g and $\mathbf{h}$ satisfy (7.93) subject to $\mathbf{P}_d^*\mathbf{h} = \mathbf{0}$. By the properties of limiting matrices, $\mathbf{P}_d^* = \mathbf{P}_\delta^*$ so it follows that $\mathbf{P}_\delta^*\mathbf{h} = \mathbf{P}_d^*\mathbf{h} = \mathbf{0}$. From Lemma B.5, $(\mathbf{H}_\delta)_{RT} = \mathbf{0}$ and $(\mathbf{H}_\delta)_{TT} = (\mathbf{I} - \mathbf{P}_\delta)_{TT}^{-1}$. Hence part 2. follows by substituting these observations into (7.99).

To prove part 3, note that under (7.101), it follows from Proposition 7.6 that $g^{\delta^\infty} = g$. Since $(\mathbf{I} - \mathbf{P}_\delta)_{TT}^{-1} \geq \mathbf{I}$ from part 3 of Lemma B.5, combining (7.102) with representation (7.100) establishes the result.

Part 4 follows immediately from part 2. For both parts 3 and 4, $g^{\delta^\infty} = g$ follows from Proposition 7.6. □

Some comments about this proposition follow:

1. The additional hypothesis that $\mathbf{P}_d^* \mathbf{h} = \mathbf{0}$ implies that $\mathbf{h} = \mathbf{h}^{\delta^\infty}$. It is not required to derive (7.99) but is fundamental for deriving (7.100).

2. Note that the increase in $\mathbf{h}$ in part 3 occurs in transient states of $\mathbf{P}_\delta$.

3. The above propositions do not account for the eventuality that

$$r(s, \delta(s)) - g + \sum_{j \in S} p(j|s, \delta(s))h(j) - h(s)$$

is positive on some states and negative in others. When δ is derived from d in the policy iteration improvement step, this is impossible.

4. It might at first appear that these propositions do not consider the the possibility that δ represents an improvement in **both** recurrent and transient states. However as a consequence of Proposition 7.6, this would result in an increase in the average reward. This is not of concern since it will not impact the proof of convergence of policy iteration below.

Convergence of policy iteration

The following theorem applies Propositions 7.6 and 7.7 to establish convergence of policy iteration.

Theorem 7.11. In a recurrent or unichain model, policy iteration converges in a finite number of iterations to a solution $(g^*, \mathbf{h})$ of the Bellman equation (7.47). Moreover if

$$d^* \in \arg \operatorname*{c\text{-}max}_{d \in D^{\mathrm{MD}}} \{\mathbf{r}_d + \mathbf{P}_d \mathbf{h}\} \tag{7.104}$$

then $(d^*)^\infty$ is average optimal.

Proof. The proof uses the vector version of the recurrent and unichain policy iteration algorithm above. Let $d' \in D^{\mathrm{MD}}$. Clearly

$$\operatorname*{c\text{-}max}_{d \in D^{\mathrm{MD}}} \left\{ \mathbf{r}_d - g^{(d')^\infty} \mathbf{e} + \mathbf{P}_d \mathbf{h}^{(d')^\infty} - \mathbf{h}^{(d')^\infty} \right\} \geq \mathbf{r}_{d'} - g^{(d')^\infty} \mathbf{e} + \mathbf{P}_{d'} \mathbf{h}^{(d')^\infty} - \mathbf{h}^{(d')^\infty} = \mathbf{0}.$$

Thus when d'' is chosen in the improvement step using (7.90),

$$\mathbf{r}_{d''} - g^{(d')^\infty} \mathbf{e} + \mathbf{P}_{d''} \mathbf{h}^{(d')^\infty} - \mathbf{h}^{(d')^\infty} \geq \mathbf{0}. \tag{7.105}$$

If equality holds in (7.105), $(g^{(d')^\infty}, \mathbf{h}^{(d')^\infty})$ is a solution of the unichain Bellman equation, completing the proof. If not, and there is strict inequality in (7.105), for some state in $R_{d''}$ then $g^{(d'')^\infty} > g^{(d')^\infty}$ from Proposition 7.6.

Finally, if equality holds in (7.105) on $R_{d''}$ and there is strict inequality for some state in $T_{d''}$, Proposition 7.7 implies $g^{(d'')^\infty} = g^{(d')^\infty}$ and $h^{(d'')^\infty}(s) \geq h^{(d')^\infty}(s)$ for all s with strict inequality in some state. Since there are only finitely many stationary deterministic policies, only a finite number of such increases are possible. Thus, the result follows. □

Some comments follow:

1. The above result shows the average reward policy iteration algorithm finds a solution of the optimality equation, an optimal policy and its average reward.

2. Depending on the specification used to uniquely determine $\mathbf{h}$, it finds either the bias or relative values of an optimal policy.

3. If policy iteration is implemented with $h(s') = 0$ for some distinguished state s', the above proof still holds because Proposition 7.7 shows the bias increases when the only strict improvement is in a transient state.

4. The above theorem also provides a constructive proof of the existence of a solution for the Bellman equation (7.47).

A curious example

Note that the above analysis does not conclude anything regarding optimality with respect to the bias. The following example shows that policy iteration finds an average optimal policy but not one with optimal bias among average optimal policies.

Example 7.11. Let $S = \{s_1, s_2\}$; $A_{s_1} = \{a_{1,1}, a_{1,2}\}$, $A_{s_2} = \{a_{2,1}\}$; $r(s_1, a_{1,1}) = 2$, $r(s_1, a_{1,2}) = 0$, $r(s_2, a_{2,1}, s_1) = 6$, $r(s_2, a_{2,1}, s_2) = 0$; $p(s_1|s_1, a_{1,1}) = 1$, $p(s_2|s_1, a_{1,2}) = 1, p(s_1|s_2, a_{2,1}) = p(s_1|s_2, a_{2,1}) = 0.5$. Note that $r(s_2, a_{2,1}) = 3$ and the model is unichain. See Figure 7.8.

Since actions are only chosen in state s_1, Markovian deterministic decision rules can be specified by their choice of action in s_1. Let d denote the deterministic decision rule that uses action $a_{1,1}$ and f denote the deterministic decision rule that uses action $a_{1,2}$.

Suppose policy iteration begins with policy f^∞. Evaluating this policy gives $g^{f^\infty} = 2$, $h^{f^\infty}(s_1) = -4/3$ and $h^{f^\infty}(s_2) = 2/3$. The improvement step in s_1 gives

$$\max_{a \in A_{s_1}} \left\{ r(s_1, a) + p(s_1|s_1, a) h^{f^\infty}(s_1) + p(s_2|s_1, a) h^{f^\infty}(s_2) \right\}$$

$$= \max\left\{2 + h^{f^\infty}(s_1), h^{f^\infty}(s_2)\right\} = \max\left\{\frac{2}{3}, \frac{2}{3}\right\}.$$

Thus, policy iteration terminates with policy f^∞.

Evaluating d^∞ gives $g^{d^\infty} = 2$, $h^{d^\infty}(s_1) = 0$ and $h^{d^\infty}(s_2) = 2$. Therefore the bias of d^∞ exceeds that of f^∞. Consequently in this example, policy iteration finds an average optimal policy but not a policy with optimal bias among average optimal policies. This is because d and f have different sets of recurrent states.

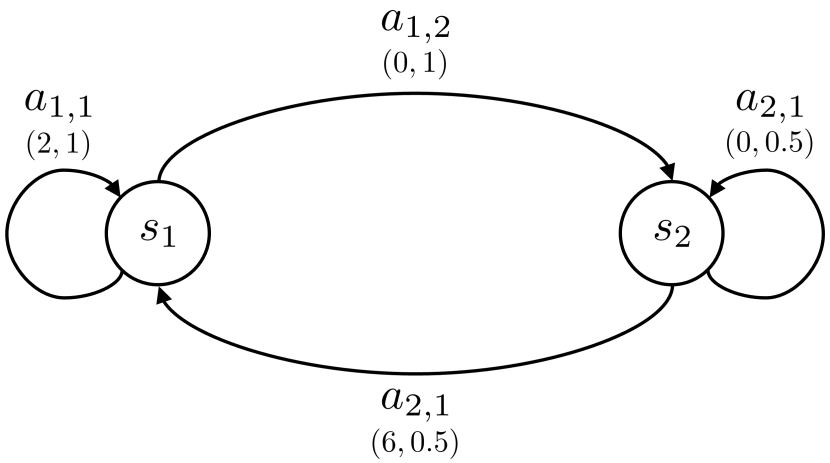


Figure 7.8: Graphical representation of model in Example 7.11 illustrating that policy iteration need not find a policy with optimal bias among average optimal policies. The labels (r, p) refer to the reward and transition probabilities, respectively, associated with using the specified action in the given state.

7.8 Linear programming

The linear programming formulation of the average reward Markov decision process is noteworthy because:

1. Dual variables represent state-action probabilities generated by randomized policies, and

2. The model allows direct inclusion of meaningful and practical constraints.

This section includes a detailed analysis of linear programming in recurrent models followed by generalization to unichain models. Linear programming in multi-chain models is more complicated and its analysis is beyond the scope of this book.

7.8.1 Linear programming formulation

In both recurrent and unichain models, in which the average reward is constant, a solution of the *single* Bellman equation (7.46) can be used to identify optimal policies. As a consequence of part 1 of Theorem 7.7, if there exist a scalar g and vector $\mathbf{h}$ such that

$$h(s) \geq r(s,a) - g + \sum_{j \in S} p(j|s,a)h(j), \tag{7.106}$$

for all $a \in A_s$ and $s \in S$, then $g \geq g^*$. Hence g^* represents the smallest g for which there exists $h(s)$ satisfying (7.106) for all $s \in S$ and $a \in A_s$. This observation gives rise to the following primal linear program for finding such a quantity.

Primal LP formulation of average reward model

$$\underset{g,\mathbf{h}}{\text{minimize}} \quad g \tag{7.107a}$$

$$\text{subject to} \quad g + h(s) - \sum_{j \in S} p(j|s,a)h(j) \geq r(s,a), \quad a \in A_s, s \in S \tag{7.107b}$$

The corresponding dual linear program is given by:

Dual LP formulation of average reward model

$$\text{maximize} \quad \sum_{s \in S} \sum_{a \in A_s} r(s,a)x(s,a) \tag{7.108a}$$

$$\text{subject to} \quad \sum_{a \in A_j} x(j,a) - \sum_{s \in S} \sum_{a \in A_s} p(j|s,a)x(s,a) = 0, \quad j \in S, \tag{7.108b}$$

$$\sum_{s \in S} \sum_{a \in A_s} x(s,a) = 1. \tag{7.108c}$$

$$x(s,a) \geq 0 \text{ for } a \in A_s \text{ and } s \in S. \tag{7.108d}$$

Note that one of the constraints is redundant. Hence the set of constraints of the dual linear program have at most $|S|$ linearly independent rows. To see this, sum (7.108b) over $j \in S$ to obtain

$$\sum_{j \in S} \sum_{a \in A_j} x(j,a) - \sum_{j \in S} \sum_{s \in S} \sum_{a \in A_s} p(j|s,a)x(s,a)$$
$$= \sum_{j \in S} \sum_{a \in A_j} x(j,a) - \sum_{s \in S} \sum_{a \in A_s} x(s,a) = 0.$$

The following example illustrates the linear programming formulations of the two-state model.

Example 7.12.
The primal LP for the two-state model in Section 2.5 is

$$\begin{aligned} \text{minimize} \quad & g \\ \text{subject to} \quad & g + h(s_1) - 0.8h(s_1) - 0.2h(s_2) \geq 3 \\ & g + h(s_1) - h(s_2) \geq -5 \\ & g + h(s_2) - h(s_2) \geq -5 \\ & g + h(s_2) - 0.4h(s_1) - 0.6(s_2) \geq 2 \end{aligned}$$

Observe that the primal LP has three variables, four constraints (one for each state-action pair) and no non-negativity constraints. After simplification the constraints can be written as

$$\begin{aligned} & g + 0.2\big(h(s_1) - h(s_2)\big) \geq 3 \\ & g + \big(h(s_1) - h(s_2)\big) \geq -5 \\ & g \geq -5 \\ & g - 0.4\big(h(s_1) - h(s_2)\big) \geq 2. \end{aligned}$$

These inequalities show that $h(s_1)$ and $h(s_2)$ cannot be independently determined.

The dual LP is given by

$$\begin{aligned} \text{maximize} \quad & 3x(s_1, a_{1,1}) - 5x(s_1, a_{1,2}) - 5x(s_2, a_{2,1}) + 2x(s_2, a_{2,2}) \\ \text{subject to} \quad & x(s_1, a_{1,1}) + x(s_1, a_{1,2}) - 0.8x(s_1, a_{1,1}) - 0.4x(s_2, a_{2,2}) = 0 \\ & x(s_2, a_{2,1}) + x(s_2, a_{2,2}) - 0.2x(s_1, a_{1,1}) \\ & \quad - x(s_1, a_{1,2}) - x(s_2, a_{2,1}) - 0.6x(s_2, a_{2,2}) = 0 \\ & x(s_1, a_{1,1}) + x(s_1, a_{1,2}) + x(s_2, a_{2,1}) + x(s_2, a_{2,2}) = 1 \\ & x(s_1, a_{1,1}), x(s_1, a_{1,2}), x(s_2, a_{2,1}), x(s_2, a_{2,2}) \geq 0. \end{aligned}$$

The dual linear program has four variables (one for each state-action pair), three equality constraints (one for each of the two state-action pairs and one corresponding to the sum of the dual variables equaling one), and non-negativity constraints.

Rearranging terms in the first two constraints of the dual shows that one of them is redundant. Hence the dual has a total of two independent constraints.

As in Chapters 5 and 6, an analysis of the dual LP is the most insightful. Recurrent and unichain models require distinct analyses.

7.8.2 Recurrent models

This section establishes for recurrent models that:

1. There is a one-to-one relationship between randomized stationary policies and feasible solutions of the dual linear program,
2. There is a one-to one relationship between deterministic stationary policies and basic feasible solutions of the dual linear program, and
3. An optimal basic feasible solution to the dual linear program exists. From it an optimal deterministic policy can be derived.

Feasible solutions and randomized policies

The following theorem relates feasible[36] solutions of the dual LP to the stationary probability that a Markovian randomized decision rule occupies state $s \in S$ and chooses action $a \in A_s$. Recall that $w_d(a|s)$ denotes the probability that Markovian randomized decision rule d chooses action a in state s and for a Markovian randomized decision rule d, that $p_d(j|s) = \sum_{a \in A_s} p(j|s,a) w_d(a|s)$.

The proof of the theorem below uses part 3 of Theorem B.1 in Appendix B, which is restated here.

Proposition 7.8. In a recurrent model, the stationary distribution $q_d(s)$ of the Markov chain corresponding to Markovian decision rule d is the unique non-negative solution of

$$\sum_{j \in S} q(s) p_d(j|s) = q(s) \quad \text{for } s \in S \tag{7.109}$$

subject to $\sum_{s \in S} q(s) = 1$. Moreover, $q_d(s) > 0$ for all $s \in S$.

Note that in a unichain model, $q_d(s)$ is positive on the recurrent states of $\mathbf{P}_d$ and zero on its transient states.

[36] Recall that a *feasible solution* of a linear program is one that satisfies all of its constraints.

Theorem 7.12. In a recurrent model:

1. For each $d \in D^{\mathrm{MR}}$,
$$x_d(s,a) := w_d(a|s)q_d(s) \tag{7.110}$$
is a feasible solution to the dual linear program.

2. (a) Let $x(s,a)$ be a feasible solution to the dual linear program. Then for each $s \in S$,
$$\sum_{a\in A_s} x(s,a) > 0. \tag{7.111}$$

   (b) Suppose the Markovian randomized decision rule $d_{\mathbf{x}}$ has action choice probability
$$w_{d_{\mathbf{x}}}(a|s) := \frac{x(s,a)}{\sum_{a\in A_s} x(s,a)}. \tag{7.112}$$
   Then $x_{d_{\mathbf{x}}}(s,a) = w_{d_{\mathbf{x}}}(a|s)q_{d_{\mathbf{x}}}(s)$ is a feasible solution to the dual linear program.

   (c) For each $s \in S$ and $a \in A_s$,
$$x_{d_{\mathbf{x}}}(s,a) = x(s,a). \tag{7.113}$$

Proof. To prove part 1 substitute $x_d(s,a)$ into the left hand side of (7.108b). As a consequence of (7.109), that $p_d(j|s) = \sum_{a\in A_s} p(j|s,a)w_d(a|s)$ and $\sum_{a\in A_s} w_d(a|s) = 1$, it follows for all $s \in S$ that

$$\sum_{a\in A_s} w_d(a|s)q_d(s) - \sum_{s'\in S}\sum_{a\in A_{s'}} p(s|s',a)w_d(a|s')q_d(s') = q_d(s) - \sum_{j\in S} p_d(j|s)q_d(s) = 0.$$

Moreover it is easy to see that $x_d(s,a)$ satisfies (7.108c) and (7.108d) so the first result follows.

To prove 2(a) define $f(s) := \sum_{a\in A_s} x(s,a)$ and let $S' \subseteq S$ denote the set of $s \in S$ for which $f(s) > 0$. To show that $S' = S$, note first that

$$\begin{aligned}
\sum_{s\in S'}\sum_{a\in A_s} p(j|s,a)x(s,a) &= \sum_{s\in S'}\sum_{a\in A_s} p(j|s,a)x(s,a)\frac{f(s)}{f(s)} \\
&= \sum_{s\in S'}\sum_{a\in A_s} p(j|s,a)\frac{x(s,a)}{\sum_{a\in A_s} x(s,a)} f(s) \\
&= \sum_{s\in S'}\sum_{a\in A_s} p(j|s,a)w_{d'}(a|s)f(s) = \sum_{s\in S'} p_{d'}(j|s)f(s),
\end{aligned}$$

where the randomized decision rule $d'(s)$ selects action $a \in A_s$ for $s \in S'$ with proba-

bility[37]

$$w_{d'}(a|s) := \frac{x(s,a)}{\sum_{a\in A_s} x(s,a)}.$$

From (7.108b) and the above equality it follows that for $j \in S'$,

$$\begin{aligned} 0 &= f(j) - \sum_{s\in S}\sum_{a\in A_s} p(j|s,a)x(s,a) \\ &= f(j) - \sum_{s\in S'}\sum_{a\in A_s} p(j|s,a)x(s,a) - \sum_{s\in S\setminus S'}\sum_{a\in A_s} p(j|s,a)x(s,a) \\ &= f(j) - \sum_{s\in S'}\sum_{a\in A_s} p(j|s,a)x(s,a) \\ &= f(j) - \sum_{s\in S'} p_{d'}(j|s)f(s), \end{aligned} \tag{7.114}$$

where the third equality follows from the fact that $f(s) = 0$ for $s \in S\setminus S'$ and $x(s,a) \geq 0$ together imply $x(s,a) = 0$ for all $a \in A_s, s \in S \setminus S'$. Moreover, because $f(s) = 0$ for $s \in S \setminus S'$ and (7.108c),

$$\sum_{s\in S'} f(s) = \sum_{s\in S} f(s) = \sum_{s\in S}\sum_{a\in A_s} x(s,a) = 1. \tag{7.115}$$

Since $f(s) > 0$ solves (7.114) and (7.115), it follows from Proposition 7.8, $f(s)$ is the stationary distribution of d', that is $f(s) = q_{d'}(s)$ for all $s \in S'$ and by (7.115), $\sum_{s\in S'} q_{d'}(s) = 1$. Since in a recurrent model, all states are recurrent under every stationary policy, $q_{d'}(s) > 0$ for all $s \in S$. Hence, $S' = S$, proving 2(a).

As a consequence of 2(a), $w_{d_{\mathbf{x}}}(a|s) \geq 0$ is well defined for all $s \in S$ and $a \in A_s$, non-negative and $\sum_{a\in A_s} w_{d_{\mathbf{x}}}(a|s) = 1$. Therefore $w_{d_{\mathbf{x}}}(a|s)$ is a probability distribution on A_s for each $s \in S$ so that $d_{\mathbf{x}}$ is a Markovian randomized decision rule. Hence from part 1 $x_{d_{\mathbf{x}}}(s,a)$ satisfies the dual linear program.

Finally

$$x(s,a) = x(s,a)\frac{f(s)}{\sum_{a\in A_s} x(s,a)} = w_{d_{\mathbf{x}}}(a|s)q_{d_{\mathbf{x}}}(s) = x_{d_{\mathbf{x}}}(s,a),$$

where the second equality was established in the proof of 2(a). □

The following comments provide insight into the significance of the above result:

1. From an application perspective, (7.112) is fundamental. It shows how to derive a randomized stationary policy from a feasible solution of the dual linear program.

[37] Note that this is the same action choice probability of the decision rule $d_{\mathbf{x}}(s)$ defined in (7.112) but it is not yet determined that $S' = S$ so $d'(s)$ is only defined on S'.

2. As a consequence of part 2(c) of the above theorem, $x(s,a)$ is the joint stationary probability that the system corresponding to $d_{\mathbf{x}}$ occupies state s **and** chooses action a in steady state.

3. The previous comment implies that the dual objective function equals the stationary reward corresponding to the policy $(d_{\mathbf{x}})^{\infty}$. That is,

$$\sum_{s\in S}\sum_{a\in A_s} r(s,a)x(s,a) = g^{(d_{\mathbf{x}})^{\infty}}$$

4. The proof of 2(a) of Theorem 7.12 is quite subtle. It exploits the property that in a recurrent Markov chain, the stationary distribution is strictly positive in every state.

Basic feasible solutions and deterministic policies.

This section explores the relationship between basic feasible solutions[38] and deterministic stationary policies. The next result follows immediately from the previous theorem and the definition of a basic feasible solution.

Theorem 7.13. In a recurrent model:

1. Suppose $d \in D^{\mathrm{MD}}$. Then

$$x_d(s,a) := \begin{cases} q_d(s) & \text{when } d(s) = a \\ 0 & \text{when } d(s) \neq a \end{cases} \tag{7.116}$$

   is a basic feasible solution of the dual linear program.

2. Let $x(s,a)$ denote a basic feasible solution of the dual linear program.

   (a) Then $x(s,a) > 0$ for exactly one $a \in A_s$ for each $s \in S$.

   (b) For each $s \in S$, $d_{\mathbf{x}}(s) = a$ corresponds to $x(s,a) > 0$ so that $d_{\mathbf{x}} \in D^{\mathrm{MD}}$.

Proof. Part 1 follows immediately from part 1 of the preceding theorem.

As shown above, the dual has at most $|S|$ linearly independent rows. Thus, a basic feasible solution has at most $|S|$ positive components. Since part 2(a) of Theorem 7.12 shows that $\sum_{a\in A_s} x(s,a) > 0$ for each $s \in S$, each basic feasible solution has exactly one positive $x(s,a)$ for each $s \in S$. Hence Part 2(a) holds. Hence from (7.112), $w_{d_{\mathbf{x}}}(a|s) = 1$ for exactly one $a \in A_s$ for each $s \in S$ from which part 2(b) follows. □

[38] The proof of the following theorem uses the result that in a linear program with m constraints and n variables with $n > m$, a *basic feasible solution* has at most m non-zero entries. A formal definition of this concept appears in Appendix C.

This theorem represents the next building block in analyzing the dual linear program in recurrent average reward models, namely:

1. The assumption of a recurrent model is crucial to this result. It is used to show that $\sum_{a\in A_s} x(s,a) > 0$ for each $s \in S$. The next section considers transient models in which for some $s' \in S$, $x(s',a) = 0$ for all $a \in A_{s'}$.

2. Part 2(b) shows how to identify deterministic policies from basic feasible solutions. Namely for each state, choose the action a for which $x(s,a) > 0$.

3. Part 1 shows that the solution of the dual gives the stationary probability distribution for the Markov chain corresponding to $d_{\mathbf{x}}$.

Optimal solutions and optimal policies

Finally, this section relates optimal solutions to the dual with optimal deterministic stationary policies.

Theorem 7.14. Suppose in a recurrent model that $r(s,a)$ is bounded for all $s \in S$, and $a \in A_s$. Then there exists an optimal basic feasible solution $x^*(s,a)$ to the dual linear program. Moreover $(d_{\mathbf{x}^*})^\infty$ is an optimal deterministic stationary policy where $d_{\mathbf{x}^*}(s) = a$ when $x^*(s,a) > 0$.

Proof. Since $r(s,a)$ is bounded, Theorems C.1, C.2 and C.4 together imply that the dual LP has at least one optimal basic feasible solution. Let $x^*(s,a)$ denote one.

For any $d \in D^{\mathrm{MR}}$, $x_d(s,a)$ defined in part 1 of Theorem 7.12 is a feasible solution of the dual linear program so that by the optimality of $x^*(s,a)$,

$$\sum_{s\in S}\sum_{a\in A_s} r(s,a)x^*(s,a) \geq \sum_{s\in S}\sum_{a\in A_s} r(s,a)x_d(s,a).$$

Hence from Theorem 7.13 there exists a deterministic decision rule $d_{\mathbf{x}^*}(s)$ corresponding to $x^*(s,a)$ for which

$$\begin{aligned} g^{(d_{\mathbf{x}^*})^\infty} &= \sum_{s\in S}\sum_{a\in A_s} r(s,a)x_{d_{\mathbf{x}^*}}(s,a) = \sum_{s\in S}\sum_{a\in A_s} r(s,a)x^*(s,a) \\ &\geq \sum_{s\in S}\sum_{a\in A_s} r(s,a)x_d(s,a) = g^{d^\infty} \end{aligned}$$

for any $d \in D^{\mathrm{MR}}$. Hence $(d_{\mathbf{x}^*})^\infty$ is an optimal policy. □

Note that this theorem guarantees the existence of an optimal stationary deterministic policy but it does not exclude the possibility that there exist multiple optimal policies. In such a case there also exist optimal randomized policies.

Moreover Theorem C.6 guarantees the existence of an optimal solution $(g^*, \mathbf{h}^*)$ to the primal LP with $g^* = \sum_{s\in S}\sum_{a\in A_s} r(s,a)x^*(s,a)$. Hence the above analysis provides an alternative approach to establishing the existence of a solution to the average reward Bellman equation.

7.8.3 Unichain models

The situation is more complex in unichain models. The following is the most significant result; it is stated without proof[39].

Theorem 7.15. In a unichain model with $r(s,a)$ is bounded for all $a \in A_s$, $s \in S$:

1. There exists a bounded optimal basic feasible solution $x^*(s,a)$ to the dual linear program.

2. Let $S_{\mathbf{x}^*}$ denote the set of $s \in S$ for which $x^*(s,a) > 0$ for some $a \in A_s$. Then the policy $d^\infty_{\mathbf{x}^*}$ defined by

$$d_{\mathbf{x}^*}(s) = \begin{cases} a & \text{if } x^*(s,a) > 0, s \in S_{\mathbf{x}^*} \\ \text{arbitrary} & s \in S \setminus S_{\mathbf{x}^*} \end{cases} \tag{7.117}$$

is optimal and $S_{\mathbf{x}^*}$ equals its set of recurrent states.

Note that in a recurrent model, $S_{\mathbf{x}^*} = S$, so there is no arbitrariness in selecting an optimal policy to correspond to the solution. The following example illustrates the implications of this result by solving the dual linear program formulated in Example 7.12. Recall that the two-state model in Section 2.5 is unichain so that Theorem 7.15 applies.

Example 7.13. This example solves the dual linear program for the model in Example 7.12 and a variant. The analysis is divided into two cases, one in which the optimal policy is recurrent and the other in which it is unichain.
Case 1: Rewrite the dual LP as

$$\begin{aligned} \text{maximize} \quad & 3x(s_1,a_{1,1}) - 5x(s_1,a_{1,2}) - 5x(s_2,a_{2,1}) + 2x(s_2,a_{2,2}) \\ \text{subject to} \quad & 0.2x(s_1,a_{1,1}) + x(s_1,a_{1,2}) - 0.4x(s_2,a_{2,2}) = 0 \\ & -0.2x(s_1,a_{1,1}) - x(s_1,a_{1,2}) + 0.4x(s_2,a_{2,2}) = 0 \end{aligned}$$

[39] See Section 8.8.2 in Puterman [1994].

$$x(s_1, a_{1,1}) + x(s_1, a_{1,2}) + x(s_2, a_{2,1}) + x(s_2, a_{2,2}) = 1$$
$$x(s_1, a_{1,1}), x(s_1, a_{1,2}), x(s_2, a_{2,1}), x(s_2, a_{2,2}) \geq 0.$$

Note that $x(s_2, a_{2,1})$ does not appear in the first or second constraint and that one of those two constraints is redundant (one is a multiple of the other).

Solving this linear program using the simplex algorithm gives the optimal basic feasible solution $x(s_1, a_{1,1}) = 0.6667$, $x(s_1, a_{1,2}) = 0$, $x(s_2, a_{2,1}) = 0$ and $x(s_2, a_{2,2}) = 0.3333$ with objective function value of 2.6667. By Theorem 7.15, $S_{\mathbf{x}^*} = S$ and the stationary policy $d_{\mathbf{x}^*}(s_1) = a_{1,1}$ and $d_{\mathbf{x}^*}(s_1) = a_{2,2}$ is optimal.

Case 2: Suppose instead that $r(s_2, a_{2,1}) = 4$. In this case, the optimal solution is $x(s_1, a_{1,1}) = 0, x(s_1, a_{1,2}) = 0, x(s_2, a_{2,1}) = 1, x(s_2, a_{2,2}) = 0$ with an the objective function value of 4. Thus $S_{\mathbf{x}^*} = \{s_2\}$ and an optimal policy uses action $a_{2,1}$ in state s_2. Since $x(s_1, a_{1,1}) = x(s_1, a_{1,2}) = 0$, s_1 is transient and either action in s_1 gives the same average reward. Which action would you choose in s_1? (Why?)

7.8.4 Constrained Markov decision processes

Because variables in the dual linear program represent the stationary probability that the system is in state $s \in S$ and chooses action $a \in A_s$, the dual linear program provides a natural framework for adding one or more constraints of the form

$$\sum_{s \in S} \sum_{a \in A_s} c(s, a) x(s, a) \leq C. \tag{7.118}$$

The following examples show why such constraints could be useful.

Example 7.14. Consider the queuing service rate control model of Section 3.4.1. To model the constraint that the percentage of time that the fastest service rate a_f is used is at most α, (7.118) becomes

$$\sum_{s \in S} x(s, a_f) \leq \alpha,$$

corresponding to $c(s, a) = 1$ if $a = a_f$ and 0 otherwise.

Example 7.15. Consider the inventory model of Section 3.1. To add a constraint that the percentage of time that an order is backlogged[a] is less than or equal to α, (7.118) can be written as

$$\sum_{s \leq 0} \sum_{a \in A_s} x(s, a) \leq \alpha,$$

corresponding to $c(s,a) = 1$ if $s \leq 0$ and 0 otherwise.

[a]Recall that *backlogging* means an inventory level of 0 or below.

The following example examines the impact of adding a constraint in the two-state example.

Example 7.16. Consider the model analyzed as Case 1 in Example 7.13. Limiting the frequency at which the system occupies state s_1 to be at most 0.5 in steady state is captured through the constraint

$$x(s_1, a_{1,1}) + x(s_1, a_{1,2}) \leq 0.5.$$

Solving the model with this extra constraint yields $x^c(s_1, a_{1,1}) = 0.375$, $x^c(s_1, a_{1,2}) = 0.125$, $x^c(s_2, a_{2,1}) = 0$ and $x^c(s_2, a_{2,2}) = 0.5$. Since there are three non-zero variables, this is not a basic feasible solution for the unconstrained problem, but of course it is a basic feasible solution for the constrained problem.

Through (7.112) it generates the *randomized* policy $(d_{\mathbf{x}^c})^\infty$ with the following action selection probabilities:

$$w_{d_{\mathbf{x}^c}}(a_{1,1}|s_1) = \frac{0.375}{0.375 + 0.125} = 0.75, \quad w_{d_{\mathbf{x}^c}}(a_{1,2}|s_1) = \frac{0.125}{0.375 + 0.125} = 0.25,$$

$$w_{d_{\mathbf{x}^c}}(a_{2,1}|s_2) = 0, \qquad w_{d_{\mathbf{x}^c}}(a_{2,2}|s_2) = 1.$$

Moreover $g^{(d_{\mathbf{x}^c})^\infty} = 1.5$, which is considerably less than its value of 2.667 in the unconstrained model. Also, note that when $\alpha \geq 0.6667$ the model has the same solution as the unconstrained model since this constraint becomes non-binding.

The takeaway from this example is that in a constrained Markov decision process with one constraint, the optimal stationary policy may be randomized. In particular, when the constraint is binding, as when $\alpha = 0.5$ above, the optimal policy randomizes in a single state. In general, when there are K constraints, an optimal policy will randomize in at most K states.

7.9 Optimality of structured policies

The optimality of a structured policy in discounted and expected total reward models in Chapters 5 and 6 was a consequence of showing that:

1. the form of the value function (such as convexity) is preserved under application of the Bellman operator,

2. the limit of functions of the specific form retains the same form, and

3. greedy decision rules with respect to a function of this form has the desired structure.

A proof of this result was based on inductively showing that if value iteration were started with a value function of a particular form, all iterates of value iteration would retain this form as would the limits of the iterates. Since the limit is the optimal value function and greedy decision rules correspond to optimal stationary policies, the optimal stationary policy has the desired structure.

Since in an average reward model the iterates of value iteration diverge, the above analysis requires some modification. Instead, analysis can be based on the iterates of relative value iteration. To do so, distinguish a particular state s' and form iterates $w^n(s)$ based on the recursions:

$$v^{n+1}(s) = \max_{a \in A_s} \left\{ r(s,a) + \sum_{j \in S} p(j|s,a) w^n(j) \right\} \quad \text{and} \quad w^{n+1}(s) = v^{n+1}(s) - v^{n+1}(s').$$

A formal statement of the relevant result follows. Note that it requires assumptions regarding the model class that ensure convergence of relative value iteration.

Let V^F_{rel} denote the set of real-valued functions $\mathbf{w}$ on S for which $w(s') = 0$ for some designated $s' \in S$ and let D^F denote a set of decision rules that are compatible with V^F_{rel}.

Theorem 7.16. In an aperiodic recurrent, unichain, communicating or weakly communicating model, suppose that

1. V^F_{rel} is non-empty,

2. $\mathbf{w} \in V^F_{\text{rel}}$ implies $\mathbf{w}'$ defined by

$$\mathbf{v} = \underset{d \in D^{\text{MD}}}{\text{c-max}} \{\mathbf{r}_d + \mathbf{P}_d \mathbf{w}\} \text{ and } \mathbf{w}' = \mathbf{v} - v(s')\mathbf{e}$$

   is in V^F_{rel}, and

3. $\mathbf{w}^n \in V^F_{\text{rel}}$ for $n = 1, 2, \ldots$ and $\lim_{n \to \infty} \|\mathbf{w}^n - \mathbf{w}^*\| = 0$ implies $\mathbf{w}^* \in V^F_{\text{rel}}$.

Then if D^F is compatible with V^F_{rel}, there is an optimal stationary policy $(d^*)^\infty$ with $d^* \in D^F$.

Applying this result to the equipment replacement model in Section 5.10.2 is left as an exercise.

7.10 Queuing service rate control

This section focuses on numerical solutions in a infinite horizon average cost version of the discrete time service rate control model described in Section 3.4.1 and previously analyzed in the finite horizon (Section 4.3.1) and discounted (Section 5.11) cases. This section illustrates some of the challenges faced when solving average reward models.

7.10.1 The model

Suppose the controller chooses among three service probabilities $a_1 = 0.2$, $a_2 = 0.4$ and $a_3 = 0.6$ and the arrival probability is $b = 0.2$. (Note that if $b > 0.4$, the row sums of transition probabilities when using a_3 will exceed 1.) Assume further that the delay cost is $f(s) = s^2$ and the cost per period of serving at rate a_k is $m(a_k) = 5k^3$. To simplify notation set $c(s, a_k) := f(s) + m(a_k)$.

To obtain a finite state space, truncate the state space at N and assume that the transition probabilities in state N for $k = 1, 2, 3$ are given by:

$$p(j|N, a_k) = \begin{cases} a_k & j = N-1 \\ 1 - a_k & j = N. \end{cases}$$

Since every state is accessible from every other state (see Exercise 19), the model is recurrent and aperiodic (regular) so that the average reward of every policy is constant. Hence the Bellman equation for this model becomes

$$h'(s) = \min_{k=1,2,3} \begin{cases} c(0, a_k) - g + (1-b)h(0) + bh(1) & s = 0 \\ c(s, a_k) - g + a_k h(s-1) + (1-b-a_k)h(s) + bh(s+1)) & 1 \le s \le N-1 \\ c(N, a_k) - g + a_k h(N-1) + (1-a_k)h(N) & s = N. \end{cases}$$

Since $f(0) = 0$ and the choice of service rate has no impact on transition behavior in state 0, the optimal decision in this state is to choose the least expensive service rate with cost $m(a_1)$. Hence, the Bellman equation in state 0 simplifies to

$$h'(0) = m(a_1) - g + (1-b)h(0) + bh(1).$$

7.10.2 Value iteration

The beauty of this model is that as a result of the simple transition structure, when implementing value iteration there is no need to write out the entire transition matrix. Coding is simplified when value iteration is expressed in two steps as follows.

Given $v(s)$, define $q(s, a)$ by

$$q(s, a_k) = \begin{cases} c(0, a_k) + (1-b)v(0) + bv(1) & s = 0 \\ c(s, a_k) + a_k v(s-1) + (1-b-a_k)v(s) + bv(s+1) & 1 \le s \le N-1 \\ c(N, a_k) + a_k v(N-1) + (1-a_k)v(N) & s = N. \end{cases}$$

Then

$$v'(s) = \min_{k=1,2,3} q(s, a_k) \quad \text{for all } s \in S.$$

Value iteration was applied to models with $N = 50, 200, 500$, and 1,000 setting $v^0(s) = 0$ for all $s \in S$ and $\epsilon = 0.0001$. Convergence required $350, 796, 1661$, and 3055 iterations, respectively.

The left-hand figure in Figure 7.9 shows the ϵ-optimal policy when $N = 50$. Observe that it is monotone and uses action a_1 for $s = 0, 1, 2$, action a_2 when $s = 3, 4, \ldots, 8$ and action a_3 for $s \geq 9$. Moreover the same policy is optimal for all choices of N with the estimate $g^* = 19.4247$. That g does not vary with N is a result of the fact that under the optimal policy, the steady state probability that there are 10 or more jobs in the system is less than 0.001. Thus, under the optimal policy the system remains in the low-occupancy states[40] regardless of the value of N.

Because the values diverged, for example when $N = 1{,}000$, $v(1{,}000) = 834, 930, 712$ at termination, relative value iteration was applied setting $w^n(s) = v^n(s) - v^n(0)$ after each iteration. Convergence required the same number steps as above, but as noted in the text $w^n(s)$ converged. The right-hand image in Figure 7.9 depicts $w^n(s)$ at termination. Observe that it is convex and non-decreasing.

Using policy iteration the next section shows the ϵ-optimal policy found by value iteration is indeed optimal and moreover when $N = 50$, $w(s)$ and $h_{\text{rel}}(s)$ differ by at most by 0.002. Thus relative value iteration well approximates the relative value function.

Since under the optimal policy the system spends most of it time in states 0 to 9, for large s one can view the relative value function $h_{\text{rel}}(s)$ as the expected total excess cost associated with starting in state s.

7.10.3 Policy iteration

This section solves several instances of this model using policy iteration. For each problem size, iteration is initialized with the following "silly" stationary policy: use action a_1 in state 0, a_2 in state 1, a_3 in state 2 and repeat this pattern up to state N.

Policy iteration is implemented by adding the constraint $h(0) = 0$ so as to uniquely specify $h(s)$ for all $s \in S$. Consequently the evaluation equation can be expressed in matrix notation as

$$\begin{bmatrix} \mathbf{e} & \mathbf{I} - \mathbf{P}_d \\ 0 & \mathbf{e}_1^{\mathsf{T}} \end{bmatrix} \begin{bmatrix} g \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{r}_d \\ 0 \end{bmatrix}, \tag{7.119}$$

where $\mathbf{e}$ is a vector of length $|S|$ with all entries equal to 1 and $\mathbf{e}_1^{\mathsf{T}}$ is a row vector of length $|S|$ with a 1 in the first component (corresponding to state 0) and the remaining entries equal to zero. Note the last row of this equation corresponds to the constraint

[40] This will provide a challenge when using simulation based methods because there will be few observations in high occupancy states.

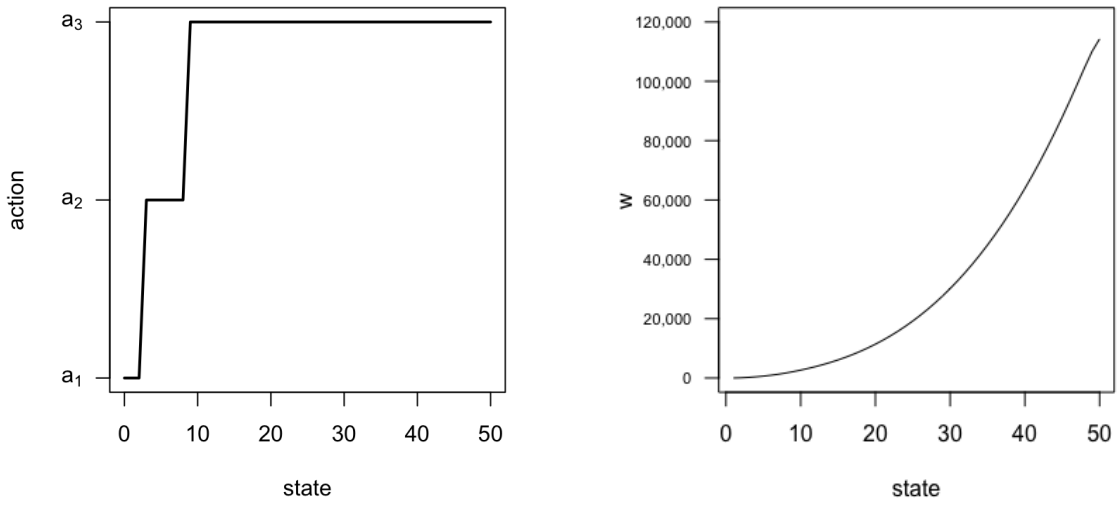


Figure 7.9: The ϵ-optimal stationary policy d^∞ (left) identified by value iteration and relative value iteration and the iterate $w^n(s)$ (right) of relative value iteration at termination for the queuing service rate control model with $N = 50$.

$h(0) = 0$[41]. As a result of this specification, the solution will give the gain and relative value function for each policy.

The model was solved for $N = 50, 200, 500$, and 1,000. It was observed that:

1. For each value of N, policy iteration required 3 iterations to satisfy the termination criterion $d' = d$. Computation time was negligible on a MacBook Pro M3. The resulting optimal policy was identical to that found using value iteration with $g^* = 19.4247$.

2. The optimal policy for this model agrees with the optimal policy for a discounted model with $\lambda = 0.999$. This observation is consistent with the logic used to prove Theorem 7.8[42].

Policy iteration was also used to solve a queuing service rate control model with 6 actions and 5,000 states. It required 4 iterations and 47.2 seconds on a MacBook Pro M3. Again the optimal policy was monotone non-decreasing in the state and the relative value function was convex non-decreasing. It used the fastest rate when $s > 22$.

[41] Expressing the matrix in this format is convenient for using the solver in R or other software.

[42] This observation is a consequence of Blackwell optimality, namely that in a finite state and action model there exists a stationary policy that is optimal for all λ sufficiently close to 1.

7.10.4 Linear programming

Formulation of the linear programming model for solution by a solver[43] requires specification of

1. a matrix representing constraints,
2. the right hand side vector, and
3. objective function coefficients.

Moreover, since this model seeks to minimize costs, the dual becomes a minimization instead of a maximization problem[44].

In a model with $S = \{0, 1, \ldots, N-1\}$, there are $3N$ dual variables so that the constraint matrix is $(N+1) \times 3N$. The rows of the matrix correspond to the N constraints involving the transition probabilities plus an additional constraint that the sum of the dual variables equals 1. Although one of the first N constraints is redundant, it is left to the solver to address.

It is convenient to order the dual variables as $x(0,a_1), \ldots, x(N-1,a_1), x(0,a_2), \ldots, x(N-1,a_2), x(0,a_3), \ldots, x(N-1,a_3)$ so the constraint matrix can be constructed by concatenating matrices corresponding to each action. Let $\mathbf{P}_{a_k}$ denote the $N \times N$ transition matrix that uses action a_k in every state. That is

$$\mathbf{P}_{a_k} = \begin{bmatrix} 1-b & b & 0 & 0 & \ldots & 0 & 0 & 0 \\ a_k & 1-b-a_k & b & 0 & \ldots & 0 & 0 & 0 \\ 0 & a_k & 1-b-a_k & b & \ldots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots \\ 0 & \ldots & \ldots & 0 & \ldots & 0 & a_k & 1-a_k \end{bmatrix}.$$

Thus the portion of the constraint matrix representing transition probabilities can be written as

$$\left[(\mathbf{I} - \mathbf{P}_{a_1})^\mathsf{T} \quad (\mathbf{I} - \mathbf{P}_{a_2})^\mathsf{T} \quad (\mathbf{I} - \mathbf{P}_{a_3})^\mathsf{T} \right].$$

The vector of objective function coefficients is given by

$$(c(0,a_1), \ldots, c(N-1,a_1), c(0,a_2), \ldots, c(N-1,a_2), c(0,a_3), \ldots, c(N-1,a_3))$$

and the right hand side vector equals $\mathbf{e}_{N+1}$ (a vector with a 1 in the $N+1$-th component and zeroes elsewhere).

Solving the model with $N = 20$ and noting (7.112) gives the same solution as that obtained by value iteration and policy iteration. Note that when using this solution, the system use action a_3 with probability less than 0.0002. This is because most of the time the system remains in low occupancy states.

[43] Computations here used the package lpSolver in R.

[44] As always one can formulate it as a maximization problem with negative rewards.

If instead the problem is solved with arrival probability equal to 0.35, the average optimal policy is $(d^*)^\infty$ where

$$d^*(s) = \begin{cases} a_1 & s = 0, 1 \\ a_2 & s = 2, 3, 4 \\ a_3 & s = 5, 6, \ldots . \end{cases}$$

Because arrivals occur more often, the system uses higher service rates at lower occupancy levels than when $b = 0.2$. In fact the system now uses action a_3 with probability 0.195 and optimal average cost equals to 60.27.

Adding constraints

To illustrate the use of constraints, consider imposing the condition that *the system use action a_3 at most* 15% *of the time in steady state*. This corresponds to the constraint:

$$\sum_{s \in S} x(s, a_3) \leq 0.15.$$

Solving the model with this constraint results in $x(s, a_1) > 0$ for $s = 0, 1$, $x(s, a_2) > 0$ for $s = 2, \ldots, 6$ and $x(s, a_3) > 0$ for $s \geq 6$. Thus since $x(6, a_2) = 0.0158$ and $x(6, a_3) = 0.053$ the optimal policy to the constrained model differs from the above policy by randomizing action choice when $s = 6$. Using (7.112) gives

$$w_{d^*}(a_2|6) = \frac{0.016}{0.016 + 0.053} = 0.229 \quad \text{and} \quad w_{d^*}(a_3|6) = 0.771.$$

The optimal average reward to the constrained problem is 60.46, only slightly greater than that of the unconstrained solution. Since this constraint is binding, the system uses action a_3 with probability 0.15.

If instead of 15% the constraint was set at 10%, the optimal policy changes and uses action a_1 with certainty when $s = 0$, a_2 with certainty when $s = 1, \ldots, 6$ and randomizes action choice between a_2 and a_3 when $s = 7$ and chooses a_3 with certainty when $s \geq 8$. Note that $w_{d^*}(a_2|7) = 0.795$ and the optimal average cost equals 62.85.

When trying to solve larger problems (say $N = 51$) using linear programming it was challenging to identify the optimal policy because it was difficult to distinguish true zeroes from those due to underflow, even when the right hand side of the equality constraint was scaled considerably.

Thus, the main advantage of using linear programming is the ability to add constraints. However, this may be outweighed by

1. the challenge of identifying the optimal policy in moderate sized problems,
2. the work required to generate the constraint matrix, and
3. the need to have an in depth understanding of linear programming to apply it successfully.

7.10.5 Concluding remarks on computation

Current trends suggest that Markov decision process users will want to solve much larger problems than those considered in this chapter. Consequently, when a model is available, relative value iteration or policy iteration are the most attractive computational approaches. Although not considered here, modified policy iteration might be an attractive alternative because it requires less computational effort than (relative) value iteration because it avoids many unnecessary maximizations and avoids generating the transition probability matrix that is required to evaluate a policy using policy iteration. Linear programming is not recommended for most computational studies but is discussed here because of its elegant theory and historical significance. Moreover it has proved useful in approximate dynamic programming (Chapter 9).

Bibliographic remarks

Our development closely follows Puterman [1994] but omits many technical issues specifically around multi-chain models.

Howard was the first to study average reward Markov decision problems although there are apparently antecedents in the game theory literature. The monograph Howard [1960], based on his MIT doctoral dissertation, formulates the model, introduces policy iteration and observe that distinct analyses were required for unichain and multi-chain models.

Howard's work motivated the seminal paper Blackwell [1962], which studies the average reward as the limit of the discounted reward as the discount rate approaches 1 and relies heavily on the partial Laurent expansion (see Theorem 7.4). He shows that in finite state and action models multi-chain policy iteration converges and moreover there exists a stationary policy that is optimal for all discount rates sufficiently close to 1, a concept that is now referred to as Blackwell optimality. The classic book Kemeny and Snell [1960] describes the concepts of limiting and fundamental matrices that underlie Blackwell's paper.

As noted above, Howard pointed out the need for distinguishing between unichain and multi-chain models. However Bather [1973] showed that classification excludes many applications. He introduces a class of models referred to as *communicating*, which include a wide-range of applications and in which the optimal gain is constant. Platzman [1977] generalizes this concept to the class of weakly communicating models.

The proof of existence of solutions to the unichain Bellman equation uses concepts from Blackwell [1962]. Derman [1970] and Sennott [1999] provide an alternative approach.

The use of value iteration in average reward models has been extensively studied. Howard [1960] conjectured that the limit in (7.77) exists. Theorem 7.10 shows that when this limit exists, value iteration converges. However, it required many papers to establish the existence of this limit. Schweitzer and Federgruen [1977] provide a

comprehensive analysis of this problem that applies also to countable state models. The discussion of the aperiodicity transformation as proposed by Schweitzer [1971] follows Section 6.6 in Sennott [1999]. Relative value iteration is due to White [1963].

The use of linear programming in average reward models originates with Ghelinick [1960] and Manne [1960] who focus on regular models. This work was extended to more general models by Denardo and Fox [1968]. Derman [1970] studies the use of linear programming in constrained models and Kallenberg [1983] provides a comprehensive overview and development in his thesis.

Exercises

1. Consider a stationary version of the three-state model from exercise 1 of Chapter 2.

   (a) Verify that the model is regular.

   (b) Give the Bellman equation for this model.

   (c) Find an average ϵ-optimal policy using value iteration and relative value iteration with $\epsilon = 0.0001$. Compare the number of iterations each require to achieve the same degree of precision.

   (d) Find an average optimal policy using policy iteration starting with the decision rule that uses a_2 in each state.

   (e) Find an average optimal policy using linear programming.

   (f) For what values of λ does the average optimal policy agree with the discount optimal policy?

2. Consider the decision rule $d'(s_i) = a_2$ for $i = 1, 2, 3$ in the model in Problem 1. Investigate the quality of the approximation $\mathbf{v}_n \approx ng\mathbf{e}+\mathbf{h}$ and $\mathbf{v}_\lambda \approx (1-\lambda)^{-1}g\mathbf{e}+\mathbf{h}$ for the stationary policy $(d')^\infty$. Show your results graphically for each state.

3. Classify the model in Problem 1 with:

   (a) the addition of the action $a_{1,3}$ with transition probability $p(s_1|s_1, a_{1,3}) = 1$ and $r(s_1, a_{1,3}) = 2$.

   (b) the addition of the action $a_{2,3}$ with transition probability $p(s_2|s_2, a_{2,3}) = 1$ and $r(s_2, a_{2,3}) = 4$.

   Use value iteration to find a 0.0001-optimal policy for the model with the two additional actions. What is the chain structure of the transition probability matrix corresponding to an optimal policy? What is the form of the ϵ-optimal average reward?

4. Consider a Markovian decision rule with periodic transition matrix

$$\mathbf{P}_d = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad \text{and} \quad \mathbf{r}_d = \begin{bmatrix} a \\ b \end{bmatrix}.$$

   (a) Show numerically that (7.29) is not valid when $a \neq b$.

   (b) Derive a relationship similar to (7.29) in terms of

$$\frac{1}{N} \sum_{j=1}^{N} \sum_{i=1}^{j} \mathbf{v}_i^{d^\infty}$$

   by modifying the proof of Theorem 7.5.

   (c) Show numerically that this new expression holds for $a \neq b$.

5. Show that the limit in (7.1) exists for all stationary policies in the example in Section 7.2.2.

6. (This example was suggested to us by Nicolas Gast in a personal communication.) Consider a three state deterministic model with $S = \{s_1, s_2, s_3\}$ and $A_{s_1} = \{a_{1,1}\}, A_{s_2} = \{a_{2,1}, a_{2,2}\}, A_{s_3} = \{a_{3,1}\}$ where $p(s_2|s_1, a_{1,1}) = p(s_1|s_2, a_{2,1}) = p(s_3|s_2, a_{2,2}) = p(s_2|s_3, a_{3,1}) = 1$ and $r(s_1, a_{1,1}) = 0$, $r(s_2, a_{2,1}) = 2$, $r(s_2, a_{2,2}) = 2$, $r(s_1, a_{1,1}) = 4$.

   (a) Show this model is communicating.

   (b) Show that there exists no Markovian deterministic policy that generates a recurrent Markov chain.

   (c) Find an optimal Markovian deterministic policy and show that it is unichain.

7. Prove in general that in a communicating model there exists a Markovian randomized decision rule that generates a Markov chain with a single recurrent class. (Note that since optimizing over randomized decision rules is equivalent to optimizing over deterministic decision rules, this implies the optimality of a deterministic Markovian policy.)

8. Find 0.001-optimal and optimal policies for the replacement model from Exercise 4 from Chapter 5. Compare your results to those in the discounted case.

9. Find 0.001-optimal and optimal policies for the inventory model in Exercise 3 from Chapter 5. Compare your results to those in the discounted case.

10. Find an average optimal policy for an infinite horizon average reward variant of the Gridworld navigation problem of Section 3.2 in which after delivering the coffee (cell 1) or falling down the stairs (cell 7), the robot begins the subsequent decision epoch in the coffee room (cell 13). Assume the system starts in the

coffee room. Choose appropriate values for the parameters. How does the optimal policy for this model compare with the optimal solution to the single pass problem?

11. Consider the following **deterministic** stationary model formulated in Table 7.7:

State	Action	Next state	Reward
s_1	$a_{1,1}$	s_3	4
s_1	$a_{1,2}$	s_2	2
s_2	$a_{2,1}$	s_3	3
s_3	$a_{3,1}$	s_3	0

Table 7.7: Description of transitions and rewards for Exercise 11.

    (a) Depict the model graphically and classify its states.

    (b) Find all policies that maximize the long-run average reward. What does this imply about the long-run average reward criterion?

    (c) Find all policies that maximize the expected total reward.

    (d) Find all policies that maximize the discounted reward for λ close to 1.

    (e) Suppose action $a_{3,1}$ is replaced by a zero-reward transition to s_1. Find all policies that maximize the average reward for this modified problem. How is this policy related to those above?

    (f) How does this observation apply to the previous problem?

12. Consider an admission control queuing model (Section 3.4.2) with finite buffer in which at the start of each period a job arrives with certainty and is processed in that period with probability p and not processed with probability $1-p$. Assume further that when the buffer is full, no jobs arrive.

    (a) Show that there exist stationary policies with two or more closed classes.

    (b) Show that the model is communicating.

13. Consider an inventory model with a capacity of 3 units in which the probability that the demand in any period is 1 unit equals p and the probability demand is 0 equals $1-p$. Assume unfilled demand is lost so that $S = \{0, 1, 2, 3\}$.

    (a) Show that there exists a stationary deterministic policy that has a transition probability matrix with two closed classes so that the model is multi-chain.

    (b) Show that the model is communicating.

    (c) What algorithm would you use to find an optimal policy?

14. Provide an example of a weakly communicating Markov decision process and prove that the optimal reward in a weakly communicating model is constant.

15. In Example 7.5, show that g and $\mathbf{h}$ as given in the example satisfy the optimality equation.

16. Verify all calculations in Example 7.7. Show that when $r(s_2, a_{2,1}, s_2) = 0$ that the gain and bias of both stationary policies satisfies the Bellman equation.

17. Consider a deterministic decision rule $d \in D^{\text{MD}}$ with

$$\mathbf{r}_d = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad \text{and} \quad \mathbf{P}_d = \begin{bmatrix} 0 & 1/2 & 1/2 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

    (a) Verify that $\mathbf{P}_d$ is periodic. What is its period?

    (b) Find g^{d^∞} and $\mathbf{h}^{d^\infty}$ by solving (7.43).

    (c) Show through calculations that the result in Corollary 7.5 holds.

    (d) Transform $\mathbf{P}_d$ to $\tilde{\mathbf{P}}_d$ using aperiodicity transformation (7.73).

    (e) Provide a graphical representation of the transformed model.

    (f) Compare the eigenvalues of $\mathbf{P}_d$ to those of $\tilde{\mathbf{P}}_d$ when $\tau = 0.05$. What difference do you observe?

18. Verify all calculations and statements in Example 7.10.

19. Show that the queuing service rate control model from Section 7.10 is communicating.

20. Prove that an optimal policy of the equipment replacement model is a control limit policy.

21. Show by induction on n or direct expansion of the left hand side that for $n \geq 1$, $\mathbf{P}_d^n - \mathbf{P}_d^* = (\mathbf{P}_d - \mathbf{P}_d^*)^n$.

22. Prove that $\mathbf{P}_d^* = \tilde{\mathbf{P}}_d^*$ where $\tilde{\mathbf{P}}_d^*$ is defined by (7.73).

23. Assuming that (7.77) is valid, show that all parts of Theorem 7.10 hold.

24. Write out one step of relative value iteration for a fixed decision rule in the form $\mathbf{w}' = \mathbf{Q}\mathbf{v}'$ where $\mathbf{Q}$ is a rank-one matrix that subtracts the first component from all other elements of $\mathbf{v}'$ where $\mathbf{v}' = \mathbf{r}_d + \mathbf{P}_d\mathbf{w}$.

    (a) Show by example that when $\mathbf{P}_d$ is regular, the largest eigenvalue of $\mathbf{Q}\mathbf{P}_d$ is strictly less than one.

    (b) Prove this result.

25. Develop, apply and demonstrate the convergence of Gauss-Seidel and modified policy iteration algorithms for unichain average reward models.

26. Create an alternative non-stationary policy to the one analyzed in Section 7.2.2 that has a different lim inf and lim sup average rewards.

Chapter 8

Partially Observable Markov Decision Processes

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

Confucious said: "To know that we know what we know, and that we do not know what we do not know, that is true knowledge."[1]

Henry David Thoreau, American naturalist and philosopher, 1817-1862.

This chapter studies a class of widely applicable models referred to as *partially observable Markov decision processes* or *POMDPs*. In a POMDP, the decision maker cannot observe the state of the system but instead receives a *signal* that provides noisy information about the *hidden* system state. Since the state is unobservable, actions must be chosen on the basis of the only available information, namely the sequence of previously observed signals and actions. As in the previously discussed Markov decision process model, the decision maker chooses actions so as to maximize a performance measure such as the expected total reward or discounted reward over a finite or infinite horizon.

The key idea in analyzing a POMDP is summarized in the following sequence of events:

1. Given a prior estimate (probability distribution) of the hidden state, the decision maker chooses an action and observes a signal.

2. Based on this action and signal, Bayes' Theorem[2] is used to obtain a posterior distribution of the hidden state.

[1] Thoreau [1854], Chapter 1.

[2] The process of using Bayes' Theorem in this way is sometimes referred to as *Bayesian filtering*.

3. This posterior distribution becomes the prior distribution at the next decision epoch and provides the basis for choosing the next action.

These probability distributions are referred to as *belief states*. After establishing that the information encoded in a belief state is sufficient for decision making and Markovian, the POMDP can be transformed into a related Markov decision process with state space equal to the set of all probability distributions on S or equivalently the $(|S|-1)$-dimensional unit simplex[3].

The model resulting from this transformation has an uncountable compact state space. On the surface, this makes the model intractable but a remarkable result of Smallwood and Sondik [1973] shows that optimal value functions are piecewise linear and convex. Since such functions can be represented by the maximum of a *finite* number of linear functions, exact and approximate algorithms can be developed to exploit the structure.

Note that modern applications, such as autonomous vehicle navigation or robotic control, base action choice on the observed signal instead of the belief state. Although sub-optimal, such an approach is the only option available when a system model is not available or too complex to represent. This chapter will assume a known model and focus on methods based on belief states.

8.1 Model overview

Consider a Markov decision process in which the decision maker cannot directly observe the system state, but instead observes a *signal* from the system that provides some information about the true system state but does not completely describe it. Since action choice can only depend on available information, it must be based on previously observed signals and actions, and not the hidden system state.

Such a model is widely applicable. For example:

1. In medicine, clinicians use diagnostic tests such as x-rays or blood tests to provide information regarding the unobservable health state of a patient. Test results inform future treatment decisions and the need for further tests.

2. In equipment maintenance, the operator of a machine, in the face of uncertainty about its true working condition, bases preventive maintenance decisions on signals obtained through sensors or on the number of defects in a sample of the machine's output.

3. In robot navigation, a robot uses sensor information such as camera images to estimate its true location and make navigational decisions.

[3]The $(N-1)$-dimensional unit simplex is the set of N-dimensional non-negative vectors $(x_1, \ldots, x_N)$ satisfying $\sum_{i=1}^{N} x_i = 1$

Information acquisition

The observation process may be conceptualized in two different ways:

1. *Passive information acquisition*: From this perspective, action choice does not directly influence the information gathering process.

2. *Active information acquisition*: Actions may be employed to gather information about the system state. These actions can vary in accuracy and cost, and may or may not affect the system state.

While this distinction will not impact analysis, it provides a framework for understanding and modeling the information acquisition process in a POMDP.

The following two examples shed light on this distinction. As a model of passive information acquisition, consider a robot navigating in a Gridworld that acquires information about its location through a noisy sensor readings. The robot chooses its next move based on these noisy observations, together with past actions and observations. This represents passive information acquisition, since the robot's actions do not directly influence the information gathering process.

Active information acquisition is prevalent in medical decision making in which a clinician must decide which of several tests to use to assess a patient's health status. Usually the test does not alter the patent's health state, but instead provides important information about it. For instance, consider the challenge of monitoring and managing the blood-oxygen level in a newborn child. Using a clip attached to the baby's finger provides a noisy but non-invasive measure. Alternatively, the clinician may decide to draw a blood sample so as to obtain a more accurate measure. Because the baby's blood supply is limited, the more invasive test must be used judiciously and only when the degree of uncertainty about the blood oxygen level is high.

These examples as described above can be easily modified to include both passive and active features.

8.1.1 POMDP dynamics

In the models analyzed here, the signal is an element of a finite discrete *signal space* O. Let the random variable, Z_n, denote the signal received at decision epoch n. Its probability distribution is represented by[4]

$$u(o|s,a) := P[Z_{n+1} = o \,|X_{n+1} = s, Y_n = a] \tag{8.1}$$

for $o \in O$, $a \in A_s$, $s \in S$ and $n \geq 1$. Note that the signal distribution is conditional on the action chosen at the *preceding* decision epoch and the state at the *current* decision

[4]If the signal space were continuous, $u(o|s,a)$ could represent a probability density or distribution function.

epoch. Also assume $u(o|s,a)$ is stationary, that is, it does not vary with decision epoch. In passive-information acquisition models, u will also be independent of a and only a function of the state. Note also that a Markov decision process is a special case of a POMDP in which there is a one-to-one mapping between observations and states, namely, $u(o|s,a) = 1$ when $o = s$ and 0 otherwise.

A POMDP evolves as follows. At decision epoch n, the system is in unobservable state s^n and the decision maker observes a signal, o^n. Using this and all past information, the decision maker chooses an action a^n. The consequence of choosing this action is an (unobserved) system transition to state s^{n+1} and a reward $r^n = r_n(s^n, a^n, s^{n+1})$ (or $r_n(s^n, a^n)$) being accrued. A new signal o^{n+1} becomes available at the start of the next decision epoch. Figure 8.1 illustrates the timing of events between two decision epochs.

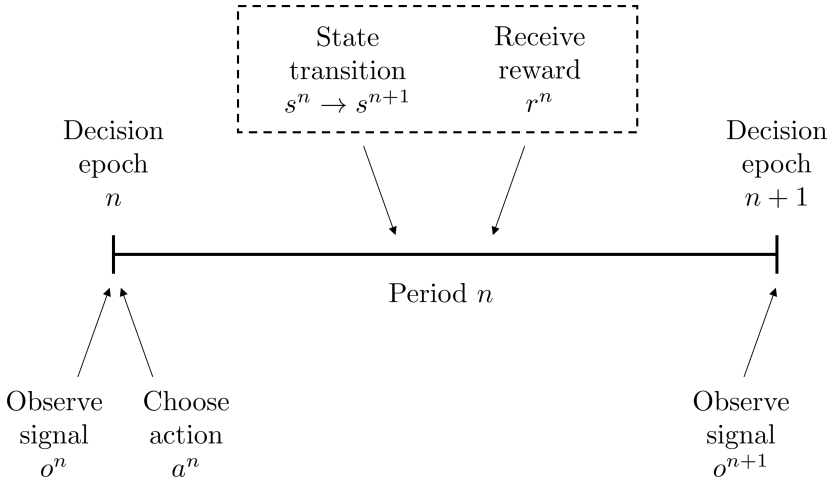


Figure 8.1: Timeline of events in period $n > 1$ of a POMDP. The decision maker observes the signal and the action choice. Events in the dashed box, namely the state transition and reward, are unobservable. Note that the first period ($n = 1$) begins with the decision maker's prior assessment of the state of the unobservable Markov decision process, so a signal is unnecessary.

The POMDP model assumes that the decision maker observes the past sequence of actions and signals, but not the sequence of state transitions or rewards since the system state is unobservable[5]. Note that the first decision epoch is distinguished because this iterative process begins with a prior assessment of the system state $b_1(s) := P[X_1 = s]$ instead of a signal[6].

[5]In some applications, the rewards may be observable, but in such a case must not provide any extra information about the current hidden state. For example, in Gridworld navigation, all transitions might incur a cost of one unit.

[6]This is consistent with the notion of a Markov decision process beginning in a random state with a known probability distribution $\rho(s)$.

A comment on action choice

We emphasize that there is some subtlety that is often overlooked regarding how actions are implemented in a POMDP. In most POMDP applications, $A_s = A$ for all $s \in S$ so that any action can be used in *any* unobservable state. For example a robot may try to move north, south, east or west at any location or a doctor may choose one of several tests in any health state.

However, it is also possible that A_s varies with s, which means that different actions are available in different (hidden) states such as in the model in Section 8.1.2. In this situation the POMDP decision maker must inform the Markov decision process controller[7] how to select an action in each possible state. What this means is that:

1. An action in a POMDP is a decision rule in the hidden Markov decision process.
2. A decision rule in the POMDP is a function mapping the current signal and past signals and actions to decison rules in the hidden Markov decision process.

To emphasize this distinction, consider first a model in which $A_s = A$ for all $s \in S$ and restrict attention to Markovian deterministic decision rules. If this were a Markov decision process, a decision rule might specify a different action in each state. However in a POMDP, this is not possible since the state of the underlying Markov decision process is unknown. Therefore the only decision rules that can be specified for use in the hidden Markov decision process are of the form $d(s) = a$ for all $s \in S$. Therefore a decision rule in the POMDP maps each possible current signal and past signals and actions to a single action in the underlying Markov decision process that can be applied in every state. For example "use test B" in a clinical decision making problem. This argument applies as well to using Markovian randomized decision rules in the underlying Markov decision process.

When A_s varies with s, the distinction and control mechanism is more problematic. For this model to be implementable, there must exist a controller in the hidden Markov decision process that can implement *any* decision rule handed down from the decision maker in the POMDP. In this case an action in the POMDP corresponds to a Markovian decision rule in the hidden Markov decision process. The example in Section 8.1.2 illustrates such a model.

[7] To distinguish the two levels of this model, the entity making decisions in the POMDP is referred to as the *decision maker* and the entity making decisions in the Markov decision process is referred to as the *controller*.

The POMDP model

In summary, a POMDP consists of an underlying[8] Markov decision process with unobservable states and rewards, plus a set of possible observations O and a corresponding probability distribution $u(\cdot|s,a)$. More formally, a POMDP can be represented by the collection of objects

$$\big\{T, S, \{A_s : s \in S\}, O, r(s,a), p(j|s,a)\big\},$$

where T is the horizon length and the rest of these quantities have been defined above.

While the added dynamics associated with partial observability are straightforward to understand, the mathematics of the model are considerably more complex than in a fully observable Markov decision process.

An illustrative example

The following is a high-level description of a prototypical clinical decision problem to illustrate some model features. The decision is whether to treat or test a patient when the true disease state is not known. Assume the disease state can be represented by

$$S = \{\text{none}, \text{mild}, \text{moderate}, \text{severe}\}$$

and

$$A_s = \{\text{do nothing}, \text{test}, \text{treat}\}$$

for each $s \in S$. If not treated the (non-fatal) disease may progress and if treated the disease may improve. Let $p(j|s,a)$ represent the probability of change in disease state when the patient is treated or untreated (corresponding to the actions "do nothing" or "test"). This becomes a meaningful problem when the treatment is effective in controlling the disease, expensive, has serious side effects and can only be administered once. So in essence this is a partially observable optimal stopping problem that terminates upon treatment.

In most settings, test results will be imprecise with possible outcomes

$$O = \{\text{negative}, \text{inconclusive}, \text{positive}\},$$

which are observed with probability $u(o|s, \text{test})$ upon testing.

In contrast to the model in the next section, *each action can be applied in each state.* That is the $A_s = A$ for all $s \in S$. Moreover, the model allows for both active and passive information acquisition.

[8] Also referred to as a hidden, unobservable or core MDP in the literature.

8.1.2 A two-state POMDP model

To provide a model that illustrates the generality of the POMDP model, consider a modified version of the two-state model in Section 2.5 that adds a signal space $O = \{o_1, o_2\}$ and signal probabilities $u(o_1|s_1) = 0.8$, $u(o_2|s_1) = 0.2$, $u(o_1|s_1) = 0.4$ and $u(o_2|s_2) = 0.6$ that are independent of decision epoch and action choice. Note that it is more likely to receive signal o_1 when the hidden state is s_1 and o_2 when the hidden state is s_2. This may be regarded as a passive information acquisition model because the observation probability does not depend on action choice.

Assume an initial state distribution $b_1(s_1) = P[X_1 = s_1]$ and $b_1(s_2) = P[X_1 = s_2]$. Let $\mathbf{b}_1 = (b_1(s_1), b_1(s_2))$, denote a column vector with components $b_1(s_1)$ and $b_1(s_2)$. Since there are only two states, letting $b_1(s_1) = q$ implies $b_1(s_2) = 1 - q$ for $0 \leq q \leq 1$. Note that in this example, the unit simplex is $\{(x_1, x_2)|x_1 + x_2 = 1, x_i \geq 0, x_2 \geq 0\}$.

Figure 8.2 provides a visual representation of the model.

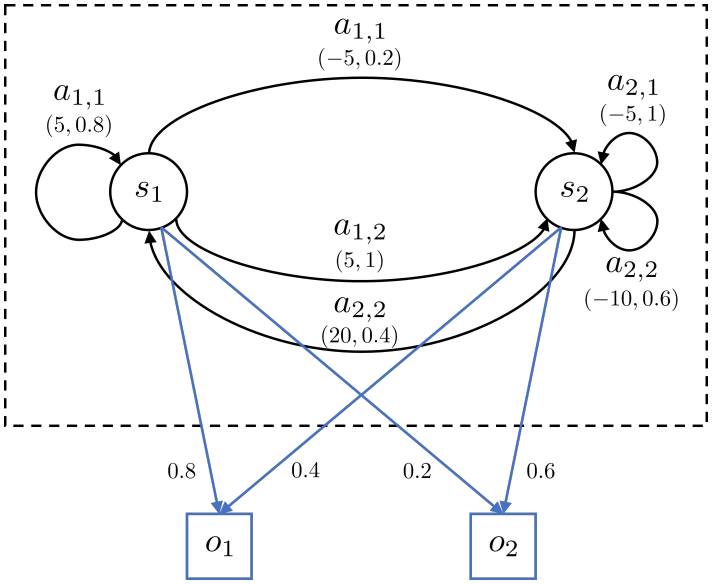


Figure 8.2: Graphical representation of the two-state POMDP model. The entities in the dashed box represents the hidden (unobservable) Markov decision process. The square boxes indicate the observable signals, with probabilities of observation on the arcs originating in the hidden states. Note the model can be generalized to allow the signal to depend on the action as well.

Computing the optimal value function in a one-period model

The following calculations for a one-period model illustrate some of the key steps in the analysis of a POMDP and the subtlety of decision rule specification. Suppose (for

illustrative purposes) that the POMDP decision maker wishes to restrict attention to *only* the two Markovian deterministic decision rules, f and g, in the underlying Markov decision process defined by:

$$f(s) := \begin{cases} a_{1,1}, & \text{if } s = s_1 \\ a_{2,1}, & \text{if } s = s_2 \end{cases} \tag{8.2}$$

and

$$g(s) := \begin{cases} a_{1,2}, & \text{if } s = s_1 \\ a_{2,2}, & \text{if } s = s_2. \end{cases} \tag{8.3}$$

Note that f can be interpreted as choosing the "first action" in each state and g can be interpreted as choosing the "second action" in each state. Moreover, there are other decision rules available for consideration such as choosing the first action in state s_1 and the second action in state s_2, or even randomizing between actions in each hidden state. However, restricting attention to f and g is sufficient to illustrate the key points in the calculations below.

In this one-period model, the only information available to the POMDP decision maker is $\mathbf{b}_1$ the POMDP decision maker's belief that the system state at decision epoch 1 is s_1 or s_2. Hence a decision rule in the POMDP must specify when to choose f and g as a function of $\mathbf{b}_1$. As an example consider a POMDP decision rule of the form:

$$d(\mathbf{b}_1) = \begin{cases} f, & b_1(s_1) \geq \overline{q} \\ g, & b_1(s_1) < \overline{q} \end{cases} \tag{8.4}$$

for some pre-specified $\overline{q} \in [0, 1]$. Alternatively the same decision rule can be expressed in terms of Markov decision process primitives as:

$$d(\mathbf{b}_1) = \begin{cases} a_{1,1}, & s = s_1, b_1(s_1) \geq \overline{q} \\ a_{1,2}, & s = s_1, b_1(s_1) < \overline{q} \\ a_{2,1}, & s = s_2, b_1(s_1) \geq \overline{q} \\ a_{2,2}, & s = s_2, b_1(s_1) < \overline{q}. \end{cases} \tag{8.5}$$

Suppose $\overline{q} = 0.5$. Then d chooses decision rule f if the probability of being in state s_1 is at least 0.5 and chooses decision rule g if the probability of being in state s_1 is less than 0.5. The expression for the decision rule d in (8.5) makes clear that while decision rules in the POMDP are functions of belief state, they specify what action to choose in each hidden state.

POMDP decision rules that always choose f and g represent extreme cases. Using either one of them for every belief state is most likely sub-optimal. The question then becomes whether there is an optimal value for $\overline{q}$, or even if there is a better decision rule with multiple breakpoints $\overline{q}_1, \ldots, \overline{q}_k$.

To address this question, computation of the expected total reward under decision rules f and g is illustrated below. The goal is to understand under what values for

$\mathbf{b}_1$ does f produce higher expected total reward than g. Note that even though these decision rules do not depend on $\mathbf{b}_1$, the expected total reward will depend on $\mathbf{b}_1$ since the total reward will depend on the starting state of the system. Furthermore, since this is a two-state model, a single parameter q fully specifies $\mathbf{b}_1$.

Assume a terminal reward $r_2(s_1) = x$ and $r_2(s_2) = y$. Let $v_f(q)$ and $v_g(q)$ denote the expected total reward obtained using f and g when $b_1(s_1) = q = 1 - b_1(s_2)$. Then,

$$\begin{aligned} v_f(q) &= (r(s_1, a_{1,1}, s_1) + r_2(s_1))p(s_1|s_1, a_{1,1})b_1(s_1) + (r(s_1, a_{1,1}, s_2) + r_2(s_2))p(s_2|s_1, a_{1,1})b_1(s_1) \\ &\quad + (r(s_2, a_{2,1}, s_1) + r_2(s_1))p(s_1|s_2, a_{2,1})b_1(s_2) + (r(s_2, a_{2,1}, s_2) + r_2(s_2))p(s_1|s_2, a_{2,1})b_1(s_2) \\ &= (5 + x)0.8q + (-5 + y)0.2q + (0 + x)0(1 - q) + (-5 + y)1(1 - q) \\ &= (3 + 0.8x + 0.2y)q + (-5 + y)(1 - q) \\ &= -5 + y + (8 + 0.8x - 0.8y)q, \end{aligned} \tag{8.6}$$

and

$$\begin{aligned} v_g(q) &= (r(s_1, a_{1,2}, s_1) + r_2(s_1))p(s_1|s_1, a_{1,2})b_1(s_1) + (r(s_1, a_{1,2}, s_2) + r_2(s_2))p(s_2|s_1, a_{1,2})b_1(s_1) \\ &\quad + (r(s_2, a_{2,2}, s_1) + r_2(s_1))p(s_1|s_2, a_{2,2})b_1(s_2) + (r(s_2, a_{2,2}, s_2) + r_2(s_2))p(s_1|s_2, a_{2,2})b_1(s_2) \\ &= (0 + x)0q + (5 + y)1q + (20 + x)0.4(1 - q) + (-10 + y)0.6(1 - q) \\ &= (5 + y)q + (2 + 0.4x + 0.6y)(1 - q) \\ &= 2 + 0.4x + 0.6y + (3 - 0.4x + 0.4y)q. \end{aligned} \tag{8.7}$$

Figure 8.3 shows how $v_f(q)$ and $v_g(q)$ vary as a function of q when $x = 10$ and $y = 0$. The two functions intersect at $q = 11/17$, $v_f(q) > v_g(q)$ for $q > 11/17$ and $v_g(q) > v_f(q)$ for $q < 11/17$. Thus if the POMDP decision maker believes $q < 11/17$ then the decision maker will inform the Markov decision process controller to use decision rule g. Accordingly, the Markov decision process controller will use action $a_{1,2}$ if the Markov decision process is in state s_1 and $a_{2,2}$ if the Markov decision process is in state s_2.

It is left as an exercise to show that for other choices of x and y, $v_f(q)$ may lie above or below $v_g(q)$ for all values of q.

Some observations that motivate the general analysis of POMDP models follow:

1. The function $\max\{v_f(q), v_g(q)\}$ is piecewise linear and convex in q.

2. The distribution of the state at decision epoch 2, X_2, is required to evaluate the expected total reward. It involves both the prior distribution $\mathbf{b}_1$ and the transition probabilities in the hidden Markov decision process, $p(j|s, a)$.

3. The expected total reward under f and g are linear functions of $\mathbf{b}_1$. For example, $v_f(q) = \boldsymbol{\gamma}^{\mathsf{T}}\mathbf{b}_1$, where $\boldsymbol{\gamma} = (3 + 0.8x + 0.2y, -5 + y)$ and $\mathbf{b}_1 = (q, 1 - q)$.

4. Figure 8.3 shows that the optimal policy partitions the interval $[0, 1]$ into two disjoint intervals.

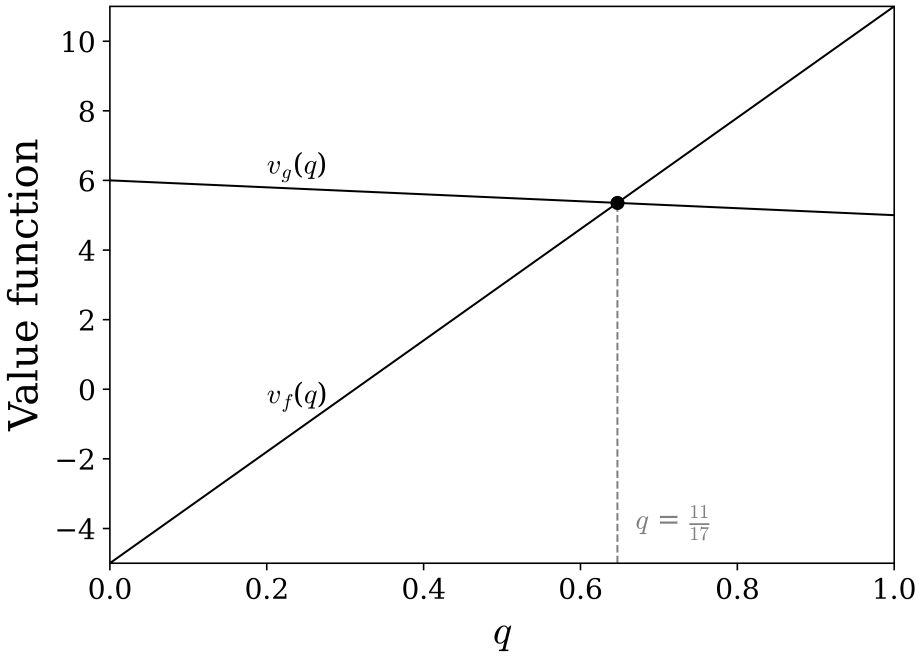


Figure 8.3: Plot of v_f and v_g as functions of q for $q \in [0, 1]$ for terminal reward $x = 10$ and $y = 0$.

5. Since the decision maker cannot observe the hidden state, the choice of decision rule must be based on the probability distribution $\mathbf{b}_1$, or equivalently, q.

6. The above calculations do not use any information regarding the observed signal, which does not become known to the decision maker until the second decision epoch, after the transition from X_1 to X_2. This signal will be used to update the subsequent belief state.

Updating the belief probability at decision epoch 2

The probability distribution $\mathbf{b}_1$ reflects the decision maker's "belief" as to what the current state is. The probability distribution of the hidden state in decision epoch 2, $\mathbf{b}_2$ can be easily calculated assuming that $\mathbf{b}_1$ and the signal at decision epoch 2 are known. In this example, that means that the calculation depends on the observed signal o_1 or o_2 and whether the POMDP decision rule informed the Markov decision process controller to chose the decision rule f or g at decision epoch 1.

Let X_1 be a random variable with distribution $\mathbf{b}_1 = (q, 1 - q)$ and $b_2(s)$ be the probability that the hidden state is s at decision epoch 2. Suppose that for a specific value of $\mathbf{b}_1$ the decision maker uses decision rule f at decision epoch 1 and observes o_1

at decision epoch 2. Then $b_2(s_1)$ and $b_2(s_2)$ can be calculated as follows:

$$\begin{aligned} b_2(s_1) &= P[X_2 = s_1 | Z_2 = o_1, Y_1 = f(X_1), X_1] \\ &= \frac{P[Z_2 = o_1 | X_2 = s_1, Y_1 = f(X_1), X_1] P[X_2 = s_1 | Y_1 = f(X_1), X_1]}{P[Z_2 = o_1 | Y_1 = f(X_1), X_1]} \\ &= \frac{u(o_1|s_1)p(s_1|s_1, a_{1,1})b_1(s_1) + u(o_1|s_1)p(s_1|s_2, a_{2,1})b_1(s_2)}{\sum_{s \in \{s_1, s_2\}} (u(o_1|s)p(s|s_1, a_{1,1})b_1(s_1) + u(o_1|s)p(s|s_2, a_{2,1})b_1(s_2))} \\ &= \frac{0.8 \cdot 0.8q + 0.8 \cdot 0(1-q)}{\big(0.8 \cdot 0.8q + 0.8 \cdot 0(1-q)\big) + \big(0.4 \cdot 0.2q + 0.4 \cdot 1(1-q)\big)} \\ &= \frac{0.64q}{0.4 + 0.32q} \end{aligned} \tag{8.8}$$

and

$$b_2(s_2) = 1 - b_2(s_1) = \frac{0.4 - 0.32q}{0.4 + 0.32q}.$$

To make this concrete, when $b_1(s_1) = q = 0.2$, $b_2(s_1) = 0.128/0.464 = 0.276$. This means that after using f at decision epoch 1 and observing o_1 at decision epoch 2, the belief that the hidden state is s_1 has increased from 0.2 to 0.276. Similarly if $q = 1$, $b_2(s_1) = 0.64/0.72 = 0.889$. Note that when $q = 0$, $b_2(s_1) = 0$ regardless of what signal is observed.

Similar calculations apply when o_2 is observed, or when decision rule g is used instead of f. These calculations are left as exercises for the reader. Thus, for each region in Figure 8.3, corresponding to either decision rule f or g, $\mathbf{b}_2$ assumes one of two potential values. This value is determined by the observed outcome, o_1 or o_2.

The astute reader will note that the calculations shown in (8.8) are precisely those described by *Bayes' Theorem*. Moreover, conditional on the observation in decision epoch 2, the probability distribution of the state in decision epoch 2 is a deterministic function of the probability distribution in decision epoch 1. This process is referred to as *Bayesian updating*. It use is formalized in the next section.

8.2 Information and belief states

If the POMDP decision maker chooses an action based on *all* information that is available at a given epoch, including all prior signals and actions, that would be analogous to using a history-dependent decision rule in a fully observable Markov decision process. As the process evolves, this information vector would grow and quickly make analysis unwieldy.

Instead, a succinct representation of this information is needed, so that decision making in a POMDP can be done in a similar fashion as using a Markovian decision rule in a MDP, in which the current state alone contains the relevant information needed to choose an action. It turns out that the information encoded in the belief state is sufficient for decision making.

Information states

Up to now the discussion of belief states has been quite informal. The following definitions makes this concept precise.

Definition 8.1. Let $a^1, \ldots, a^{n-1}$ and $o^2, \ldots, o^n$ respectively denote the sequence of actions chosen and signals observed by the POMDP decision maker. Define an *information state* to be a vector I_n for $n \geq 1$ represented by

$$I_n := \begin{cases} \mathbf{b}_1 & n = 1 \\ (o^n, a^{n-1}, \ldots, o^2, a^1, \mathbf{b}_1) & n \geq 2. \end{cases} \tag{8.9}$$

Note that:

1. The information state I_n contains all knowledge available to the POMDP decision maker at decision epoch n.

2. The information state I_n assumes values in the set of all realizations of the signal and action choice processes $(Z_2, \ldots, Z_n)$ and $(Y_1, \ldots, Y_{n-1})$. Moreover, since action choice at the first decision is based on $\mathbf{b}_1$, this quantity is appended to the information vector.

3. Note that I_n satisfies the recursion

$$I_{n+1} = (o^{n+1}, a^n, I_n) \tag{8.10}$$

   for $n = 1, 2, \ldots$.

4. The information vector in a POMDP differs from the history vector in a fully observable Markov decision processin that the observed state is replaced by the observed signal. Therefore, I_n replaces the history H_n in this setting[9].

5. Most descriptions of POMDPs in the literature include $\mathbf{b}_1$ in the model formulation.

Belief states

Belief states describe the POMDP decision maker's assessment of the distribution of states in the hidden Markov decision process at each decision epoch.

[9] Although I_n is a vector, it is unbolded so as to avoid confusion with the identity matrix $\mathbf{I}$, as well as to remain consistent with the notation for the history H_n, which plays an analogous role in the fully observable Markov decision process model.

Definition 8.2. For each $s \in S$ and probability distribution on S, $\mathbf{b}_1$, define the *belief state* $\mathbf{b}_n$ to be a vector with components

$$b_n(s) := P[X_n = s | I_n] \tag{8.11}$$

for $s \in S$ and $n \geq 2$.

Some comments about the belief state follow:

1. Suppose $a^1, \ldots, a^{n-1}$ and $o^2, \ldots, o^n$ denote the sequence of actions and signals available to the POMDP decision maker at decision epoch $n \geq 2$. Then

$$b_n(s) = P[X_n = s | Z_n = o^n, Y_{n-1} = a^{n-1}, \ldots, Z_2 = o^2, Y_1 = a^1, \mathbf{b}_1]$$

2. The belief state summarizes the decision maker's beliefs about what the system state is at a given decision epoch. It is a function of past actions and observations, and not the hidden states. In other words, it is a function of only the observable parts of the process to the decision maker.

3. Although $b_n(s)$ explicitly depends on the past sequence of actions and observations, they are suppressed in the notation.

4. The first signal is observed prior to decision epoch 2. There is no signal at decision epoch 1; that decision is based entirely on the initial state distribution $b_1(s) = P[X_1 = s]$.

5. For each n and $s \in S$, $0 \leq b_n(s) \leq 1$ and $\sum_{s \in S} b_n(s) = 1$. When S is a finite set with M elements, $b_n(\cdot)$ is an element of the $(M-1)$-dimensional unit simplex, $\mathcal{S}_{M-1}$, defined by

$$\mathcal{S}_{M-1} := \left\{ (x_1, \ldots, x_M) \;\middle|\; \sum_{i=1}^{M} x_i = 1, 0 \leq x_i \leq 1 \text{ for } i = 1, 2, \ldots, M \right\}. \tag{8.12}$$

6. In the two-state example above, $\mathcal{S}_1 = [0, 1]$, so $b_1(s_1) = q$ for a scalar $q \in [0, 1]$. In a three-state model, $\mathcal{S}_2$ is an equilateral triangle with sides of length 1 and in a four-state model, it is a tetrahedron.

A belief state recursion

The following fundamental result shows that in a POMDP:

1. The current belief state is a function of the previous belief state, the previous action and the current signal.

2. There is an explicit formula for updating the belief state.

What this means is that conditional on the belief state, no extra information about the hidden state of the system can be obtained by including the entire sequence of past actions and observations.

Theorem 8.1. For $n \geq 1$ the components of $\mathbf{b}_{n+1}$ satisfy

$$b_{n+1}(s) = P[X_{n+1} = s|I_{n+1}] = P[X_{n+1} = s|Z_{n+1} = o^{n+1}, Y_n = a^n, I_n] \quad (8.13)$$

for all $s \in S$. Moreover,

$$b_{n+1}(s) = \frac{u(o^{n+1}|s, a^n) \sum_{j \in S} p(s|j, a^n) b_n(j)}{\sum_{s' \in S} u(o^{n+1}|s', a^n) \sum_{j \in S} p(s'|j, a^n) b_n(j)}. \quad (8.14)$$

Proof. Choose n and apply Bayes' Theorem to obtain,

$$\begin{aligned} b_{n+1}(s) &= P[X_{n+1} = s|I_{n+1}] = P[X_{n+1} = s|Z_{n+1} = o^{n+1}, Y_n = a^n, I_n] \\ &= \frac{P[Z_{n+1} = o^{n+1}|X_{n+1} = s, Y_n = a^n, I_n] P[X_{n+1} = s|Y_n = a^n, I_n]}{P[Z_{n+1} = o^{n+1}|Y_n = a^n, I_n]} \end{aligned}$$

By assumption, Z_{n+1} does not depend on I_n, so that

$$P[Z_{n+1} = o^{n+1}|X_{n+1} = s, Y_n = a^n, I_n] = u(o^{n+1}|s, a^n).$$

From the law of total probability $P[X_{n+1} = s|Y_n = a^n, I_n]$ can be written as

$$\begin{aligned} P[X_{n+1} = s|Y_n = a^n, I_n] &= \sum_{j \in S} P[X_{n+1} = s|X_n = j, Y_n = a^n, I_n] P[X_n = j|Y_n = a^n, I_n] \\ &= \sum_{j \in S} p(s|j, a^n) b_n(j), \end{aligned}$$

where

$$P[X_{n+1} = s|X_n = j, Y_n = a^n, I_n] = p(s|j, a^n)$$

since the transition probability in the hidden Markov decision process does not depend on I_n. The second expression in this summation is the probability distribution of the hidden state at decision epoch n which does not depend on the subsequent action Y_n. By definition

$$P[X_n = j|Y_n = a^n, I_n] = P[X_n = j|I_n] = b_n(j)$$

so putting all of this together gives,

$$b_{n+1}(s) = \frac{u(o^{n+1}|s, a^n) \sum_{j \in S} p(s|j, a^n) b_n(j)}{\sum_{s' \in S} u(o^{n+1}|s', a^n) \sum_{j \in S} p(s'|j, a^n) b_n(j)}.$$

which establishes both results in the theorem. □

Thus, computing $\mathbf{b}_{n+1}$ requires only the most recent signal, action, and belief state[10]. Equation (8.14) shows that the updated belief state is computed by the following four-step process.

1. The decision maker chooses action a^n on the basis of the belief state $\mathbf{b}_n$ at decision epoch n.

2. The hidden process moves to state s according to the probability distribution $\sum_{j\in S} p(s|j,a^n)b_n(j)$.

3. The system generates the signal o^{n+1} from the probability distribution $u(\cdot|s,a^n)$.

4. The decision maker updates the belief state to $\mathbf{b}_{n+1}$ using (8.14).

Note that steps 2 and 3 are probabilistic, while step 4 is deterministic. Moreover, the numerator of (8.14) equals the probability that the hidden state is s and that the decision maker observes o^{n+1}, while the denominator equals the probability that the decision maker observes o^{n+1}, independent of s. These computations also show that $b_{n+1}(s)$ takes into account all the information available to the decision maker at decision epoch $n+1$, through $\mathbf{b}_n$, the action at decision epoch n and the observation o^{n+1} at decision epoch $n+1$.

As a consequence of this result, the POMDP dynamics can be expressed in terms of the belief state as follows. After observing signal o^n in decision epoch $n \geq 2$, the decision maker updates the belief state using equation (8.14), and then chooses an action. The rest of the dynamics remain unchanged. Figure 8.4 illustrates how the belief state enters into the sequence of events in period n. At decision epoch n, the decision maker observes signal o^n, updates the belief state $\mathbf{b}_n$ using (8.14) and chooses action a^n. Then prior to decision epoch $n+1$ the hidden system transitions to s^{n+1} and generates a signal o^{n+1}, which the decision maker then uses to update the belief state to $\mathbf{b}_{n+1}$.

8.3 Transforming a POMDP into a Markov decision process

This section shows how to transform a POMDP with a finite state space into an a fully observable MDP with a continuous state space, namely, the $(|S|-1)$-dimensional unit simplex, or equivalently, the set of all probability distributions on S. To evaluate policies and find optimal policies in a POMDP, the POMDP is converted into an MDP in which the belief state of the POMDP becomes the state of the equivalent MDP. Accordingly, the action sets, rewards, and transition probabilities must be redefined in terms of the belief states.

[10]This is related to the concept of a *sufficient statistic*. More formally one needs to establish that optimal value functions in a POMDP only depend on the history through the belief state.

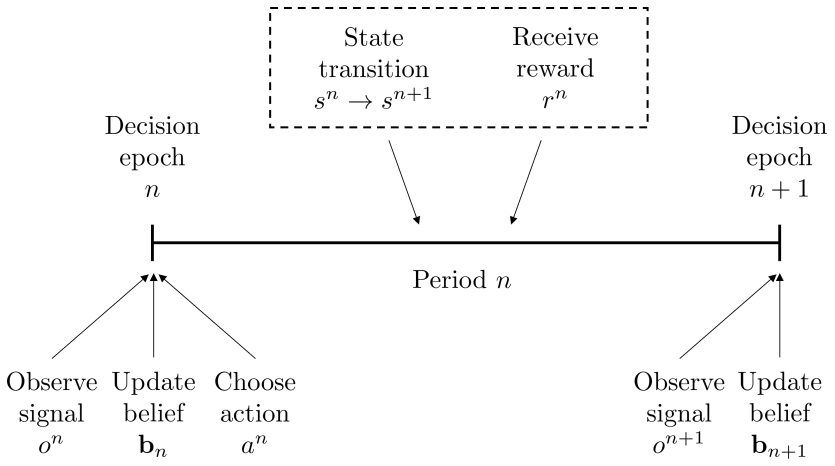


Figure 8.4: Timeline for a POMDP in period $n > 1$. The decision maker observes the signal, the updated belief, and the action choice. Events in the dashed box, namely the state transition and reward, are unobservable. Note that at decision epoch 1, o^1 is superfluous.

Note that there are two interrelated Markov decision processes under consideration, the original hidden model underlying the POMDP model, and the equivalent Markov decision process formulation of the POMDP model. The former is referred to as the *underlying Markov decision process* and the latter the *derived Markov decision process*. A tilde will be used to distinguish entities in the derived Markov decision process from those in the underlying Markov decision process. For example, the derived state space will be written $\tilde{S}$.

8.3.1 The derived Markov decision process

Recall that a Markov decision process is defined by its decision epochs, states, actions, rewards and transitions probabilities. In constructing the derived Markov decision process of a POMDP, assume S and O are discrete and finite, and written as $S = \{s_1, \ldots, s_M\}$ and $O = \{o_1, \ldots, o_K\}$.

Decision epochs: Decision epochs correspond to the point in time immediately after the decision maker updates the belief state. The model may be either finite or infinite horizon.

$$\tilde{T} = \{1, 2, \ldots, N\}, \quad N \leq \infty.$$

States: Assuming S has M elements, then $\tilde{S}$ is the $(M-1)$-dimensional unit simplex, that is

$$\tilde{S} = \mathcal{S}_{M-1}.$$

The states of the derived Markov decision process correspond to the belief states in the POMDP and are denoted by $\mathbf{b}$. Thus, $\tilde{S}$ is an uncountable and compact set even when S is finite.

When the underlying Markov decision process has two states, $\tilde{S}$ is the unit interval, when it has three states, it is equivalent to an equilateral triangle in the plane and when S has four states, $\tilde{S}$ is equivalent to a tetrahedron with equal sides.

Actions: Since the POMDP decision maker is unable to observe the hidden state, an action in the POMDP must correspond to a decision rule that *can be implemented* in the underlying Markov decision process.

In most applications, A_s will not vary with s, that is $A_s = A$ for all $s \in S$, so that selecting an action in the POMDP will result in implementing the same action in all hidden states of the underlying Markov decision process. In this case,

$$\tilde{A} = A.$$

This specification applies to the clinical example described above in which the true health state of a patient is unknown to the POMDP decision maker and the inspection problem in Section 8.6.1 below.

In contrast, when A_s varies with s such as in Example 8.1.2, a POMDP action must specify what action the MDP controller should take in each hidden state. In other words, an action in the derived Markov decision process is a decision rule in the hidden MDP. This means that it is a vector of length M that specifies an action for each hidden state. In this case,

$$\tilde{A} \subseteq \prod_{s \in S} A_s = \{(a_{s_1}, a_{s_2}, \ldots, a_{s_M}) \mid a_{s_m} \in A_{s_m}, m = 1, 2, \ldots, M\}.$$

Note that $\tilde{A}$ is specified to be a subset of $\Pi_{s \in S} A_s$ to emphasize that it may only contain a subset of all possible decision rules in the underlying Markov decision process. Since $\tilde{A}$ consists of vectors of length M, actions in the derived Markov decision process will be written as vectors $\mathbf{a}$, with components a_s for each $s \in S$. The two-state model in Section 8.1.2 provides an example in which an action in the POMDP corresponds to a (Markovian deterministic) decision rule in the underlying MDP and the set of possible actions in the derived MDP is a proper subset of the set of all decision rules in the hidden MDP.

The development in this chapter will continue with the specification that actions in the POMDP are decision rules in the MDP. However, examples where $A_s = A$ for all $s \in S$ will be highlighted to illustrate the simpler formulation.

Rewards: In the derived Markov decision process, rewards and transition probabilities of the hidden MDP must be converted to functions of the belief state $\mathbf{b}$ which is the state of the derived MDP. This is done as follows.

Since the reward in the underlying MDP, $r(s, a, j)$, is a function of hidden states s and j, it can be converted to a function of the belief state $\mathbf{b}$ as follows. Define $\tilde{r}(\mathbf{b}, \mathbf{a})$ for $\mathbf{b} \in \tilde{S}$ and $\mathbf{a} \in \tilde{A}$ by

$$\tilde{r}(\mathbf{b}, \mathbf{a}) = \sum_{s \in S} \sum_{j \in S} r(s, a_s, j) p(j|s, a_s) b(s). \tag{8.15}$$

In the above expression, the inner sum (over j) gives the expected reward conditional on the hidden system occupying state s and choosing action a_s. The outer sum (over s) takes into account that s is unobservable and that the decision maker enters the decision epoch with the belief state $\mathbf{b}$. The example in Section 8.1.2 carries out the calculation implicit in (8.15) in two steps.

When the underlying reward function is independent of j, (8.15) simplifies to

$$\tilde{r}(\mathbf{b}, \mathbf{a}) = \sum_{s \in S} r(s, a_s) b(s). \tag{8.16}$$

Equations (8.15) and (8.16) show that $\tilde{r}(\mathbf{b}, \mathbf{a})$ is **linear** in $\mathbf{b}$. This observation will play a key role in subsequent computations. Note also that even when the underlying reward depends on the subsequent state of the hidden process as in (8.15), $\tilde{r}(\mathbf{b}, \mathbf{a})$ depends only on the belief state at the current decision epoch and not on the belief state at the subsequent decision epoch.

For $\mathbf{b} \in \tilde{S}$, the terminal reward in a finite horizon model becomes

$$\tilde{r}_N(\mathbf{b}) = \sum_{s \in S} r_N(s) b(s). \tag{8.17}$$

Transition probabilities: Introducing some additional notation simplifies and provides insight into some of the expressions below. For each $\mathbf{b} \in \tilde{S}$ and $\mathbf{a} \in \tilde{A}$, define the probability of observing $o \in O$ by

$$\eta(o|\mathbf{b}, \mathbf{a}) := \sum_{s' \in S} \sum_{s \in S} u(o|s', a_s) p(s'|s, a_s) b(s). \tag{8.18}$$

Let the updated belief state be represented by $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$. It is a vector of dimension M and depends on o, $\mathbf{a}$, and $\mathbf{b}$. Therefore, it follows from (8.14) that the j-th component of the updated belief state when observing $o \in O$ is given by

$$\tau_j(o, \mathbf{a}, \mathbf{b}) := \frac{\sum_{s \in S} u(o|j, a_s) p(j|s, a_s) b(s)}{\sum_{s' \in S} \sum_{s \in S} u(o|s', a_s) p(s'|s, a_s) b(s)} = \frac{\sum_{s \in S} u(o|j, a_s) p(j|s, a_s) b(s)}{\eta(o|\mathbf{b}, \mathbf{a})}. \tag{8.19}$$

Consequently

$$\tilde{p}(\mathbf{b}'|\mathbf{b}, \mathbf{a}) = \begin{cases} \eta(o|\mathbf{b}, \mathbf{a}) & \text{when } b'(j) = \tau_j(o, \mathbf{a}, \mathbf{b}) \text{ for all } j \in S \text{ and for each } o \in O \\ 0 & \text{otherwise.} \end{cases} \tag{8.20}$$

Hence, the probability of observing a particular belief state is equal to the probability of observing the specific signal that leads to that belief state.

Observe that when O has K elements, $\tilde{p}(\mathbf{b}'|\mathbf{b}, \mathbf{a})$ assumes at most K non-zero values, at most one for each $o \in O$. Moreover it follows from (8.14) that the next belief state is a deterministic function of the current belief state and signal.

Table 8.1 summarizes the relationship between the elements in the hidden MDP and the derived MDP.

Element	Hidden MDP	Derived MDP
State	s	$\mathbf{b}$
Action (A_s independent of s)	a	a
Action (A_s varies with s)	a	$\mathbf{a} = (a_{s_1}, \ldots, a_{s_M})$
Rewards	$r(s, a)$	$\tilde{r}(\mathbf{b}, \mathbf{a})$
Transition probabilities	$p(j\|s, a)$	$\tilde{p}(\mathbf{b}'\|\mathbf{b}, \mathbf{a})$

Table 8.1: Relationship between entities in the hidden and derived Markov decision processes.

Matrix notation

Much of the literature represents the rewards and transition probabilities using the following matrix notation. It simplifies calculations and makes the dimensions of various objects more apparent.

$\mathbf{b}$: Column vector with components $b_{s_1}, \ldots, b_{s_M}$.

$\mathbf{P_a}$: $M \times M$ matrix with its (s, j)-th component equal to $p(j|s, a_s)$[a].

$\mathbf{R_a}$: $M \times M$ matrix with (s, j)-th entry $r(s, a_s, j)$.

$\mathbf{r_a}$: M-dimensional column vector with components $r(s, a_s)$ for all $s \in S$.

$\mathbf{r}_N$: M-dimensional column vector with components $r_N(s)$ for all $s \in S$ (for the finite horizon setting).

$\mathbf{U}^o_{\mathbf{a}}$: $M \times M$ **diagonal** matrix with entries $u(o|s_1, a_{s_1}), \ldots, u(o|s_M, a_{s_M})$.

$\mathbf{e}$: M-component column vector with all entries equal to 1.

[a] This definition assumes the use of deterministic decision rules in the derived MDP. It can be extended to allow randomized decision rules. Randomized rules arise when applying policy gradient algorithms (Chapter 11) to a POMDP.

When the reward function of the underlying MDP is $r(s, a, j)$, then from (8.15)

$$\tilde{r}(\mathbf{b}, \mathbf{a}) = \text{diag}(\mathbf{R_a P_a^\mathsf{T}})\mathbf{b}, \tag{8.21}$$

where $\text{diag}(\mathbf{R_a P_a^\mathsf{T}})$ denotes an $M \times M$ diagonal matrix containing the diagonal elements of $\mathbf{R_a P_a^\mathsf{T}}$, that is, the elements $(\mathbf{R_a P_a^\mathsf{T}})_{j,j}$ for $j = 1, \ldots, M$. When the underlying reward function $r(s, a)$ does not depend on j, then from (8.16)

$$\tilde{r}(\mathbf{b}, \mathbf{a}) = \mathbf{r_a^\mathsf{T}}\mathbf{b}. \tag{8.22}$$

In this notation,

$$\eta(o|\mathbf{b}, \mathbf{a}) = \mathbf{e}^\mathsf{T}\mathbf{U_a^o P_a^\mathsf{T}}\mathbf{b} \quad \text{and} \quad \boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b}) = \frac{\mathbf{U_a^o P_a^\mathsf{T}}\mathbf{b}}{\eta(o|\mathbf{b}, \mathbf{a})}. \tag{8.23}$$

It is important to emphasize that $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$ is a vector with $|S|$-components and $\eta(o|\mathbf{b}, \mathbf{a})$ is a scalar in $[0, 1]$.

The following expression summarizes this calculation:

$$\tilde{p}(\mathbf{b}'|\mathbf{b}, \mathbf{a}) = \begin{cases} \eta(o|\mathbf{b}, \mathbf{a}) & \text{when } \mathbf{b}' = \boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b}) \text{ for each } o \in O \\ 0 & \text{otherwise.} \end{cases} \tag{8.24}$$

Equation (8.24) shows that the only values for $\mathbf{b}'$ with non-zero probability are those represented by $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$ for $o \in O$. When O has K elements, there are at most K distinct vectors $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$.

8.3.2 The two-state model revisited

This section illustrates the derived MDP model for the two-state POMDP.

Decision epochs:

$$\tilde{T} = \{1, 2, \ldots, N\}, \quad N \leq \infty$$

States:

$$\tilde{S} = \{(q_1, q_2) \mid q_1 + q_2 = 1, q_1 \geq 0, q_2 \geq 0\} = \mathcal{S}_1$$

For each i, $i = 1, 2$, q_i represents the probability that the hidden state is s_i. Note that the state space can be equivalently represented as

$$\{q \mid 0 \leq q \leq 1\} = [0, 1],$$

where q is the probability that the hidden state is s_1, as shown in Section 8.1.2. Below, $\mathcal{S}_1$ is used to emphasize the fact that the belief state is a probability distribution with two components.

Actions: Consider an action set corresponding to the two decision rules defined in equations (8.2) and (8.3).

$$\tilde{A} = \{(a_{1,1}, a_{2,1}), (a_{1,2}, a_{2,2})\} = \{f, g\}.$$

The first component of each action in $\tilde{A}$ denotes the action used when the hidden process is in state s_1 and the second component denotes the action used if the hidden process is in state s_2. Note that this $\tilde{A}$ is a proper subset of the set of Markovian deterministic decision rules in the underlying two-state Markov decision process. That is, the decision rules $(a_{1,1}, a_{2,2})$ and $(a_{1,2}, a_{2,1})$ are excluded.

Rewards: The reward when the decision maker chooses action $(a_{1,1}, a_{2,1}) \in \tilde{A}$ after computing $\mathbf{b} = (q_1, q_2) \in \tilde{S}$ is

$$\begin{aligned}\tilde{r}((q_1, q_2), (a_{1,1}, a_{2,1})) &= (r(s_1, a_{1,1}, s_1)p(s_1|s_1, a_{1,1}) + r(s_1, a_{1,1}, s_2)p(s_2|s_1, a_{1,1}))q_1 \\ &\quad + (r(s_2, a_{2,1}, s_1)p(s_1|s_2, a_{2,1}) + r(s_2, a_{2,1}, s_2)p(s_2|s_2, a_{2,1}))q_2 \\ &= (5 \cdot 0.8 + (-5)0.2)q_1 + (0 \cdot 0 + (-5)1)q_2 \\ &= 3q_1 - 5q_2\end{aligned} \tag{8.25}$$

Observe that the reward is a linear function of the vector (q_1, q_2). Since $q_2 = 1 - q_1$ the reward reduces to $-5 + 8q_1$. It is left to the reader to show that $\tilde{r}\big((q_1, q_2), (a_{1,2}, a_{2,2})\big) = 5q_1 + 2q_2$.

Transition probabilities: The probability that the hidden state occupies s_1 when the decision maker chooses action $(a_{1,1}, a_{2,1}) \in \tilde{A}$ given belief state $\mathbf{b} = (q_1, q_2) \in \tilde{S}$ is calculated as follows. From (8.18), the probability of observing o_1 is given by

$$\begin{aligned}\eta(o_1|(q_1, q_2), (a_{1,1}, a_{2,1})) &= u(o_1|s_1)(p(s_1|s_1, a_{1,1})q_1 + p(s_1|s_2, a_{2,1})q_2) \\ &\quad + u(o_1|s_2)(p(s_2|s_1, a_{1,1})q_1 + p(s_2|s_2, a_{2,1})q_2) \\ &= 0.8(0.8q_1 + 0q_2) + 0.4(0.2q_1 + 1q_2) = 0.72q_1 + 0.4q_2\end{aligned}$$

and the probability of observing o_2 equals $0.28q_1 + 0.6q_2$. Then, from (8.19), the probability the hidden state equals s_1 is given by

$$\begin{aligned}\tau_{s_1}(o_1, (a_{1,1}, a_{2,1}), (q_1, q_2)) &= \frac{u(o_1|s_1)(p(s_1|s_1, a_{1,1})q_1 + p(s_1|s_2, a_{2,1})q_2)}{\eta(o_1(q_1, q_2), (a_{1,1}, a_{2,2}))} \\ &= \frac{0.8(0.8q_1 + 0q_2)}{0.72q_1 + 0.4q_2} = \frac{0.64q_1}{0.72q_1 + 0.4q_2}.\end{aligned} \tag{8.26}$$

The probability the hidden state equals s_2 is given by $1 - \tau_{s_1}(o_1, (a_{1,1}, a_{2,1}), (q_1, q_2))$, which becomes

$$\tau_{s_2}(o_1, (a_{1,1}, a_{2,1}), (q_1, q_2)) = \frac{0.08q_1 + 0.4q_2}{0.72q_1 + 0.4q_2}. \tag{8.27}$$

Hence the updated belief state when observing o_1 is

$$\boldsymbol{\tau}(o_1, (a_{1,1}, a_{2,1}), (q_1, q_2)) = \left(\frac{0.64q_1}{0.72q_1 + 0.4q_2}, \frac{0.08q_1 + 0.4q_2}{0.72q_1 + 0.4q_2} \right) \tag{8.28}$$

and when observing o_2 is

$$\boldsymbol{\tau}(o_2, (a_{1,1}, a_{2,1}), (q_1, q_2)) = \left(\frac{0.16q_1}{0.28q_1 + 0.6q_2}, \frac{0.12q_1 + 0.6q_2}{0.28q_1 + 0.6q_2} \right). \tag{8.29}$$

Putting it all together, the transition probabilities in the derived model satisfy

$$\tilde{p}((q_1', q_2')|(q_1, q_2), (a_{1,1}, a_{2,1})) = \begin{cases} 0.72q_1 + 0.4q_2 & \text{for } (q_1', q_2') = \boldsymbol{\tau}(o_1, (a_{1,1}, a_{2,1}), (q_1, q_2)) \\ 0.28q_1 + 0.6q_2 & \text{for } (q_1', q_2') = \boldsymbol{\tau}(o_2, (a_{1,1}, a_{2,1}), (q_1, q_2)) \\ 0 & \text{otherwise,} \end{cases} \tag{8.30}$$

so that $\tilde{p}(\cdot|(q_1, q_2), (a_{1,1}, a_{2,1}))$ is indeed a transition probability. It only takes two positive values, one for each observation.

To provide more insight into these calculations, suppose the belief state is $(0.2, 0.8)$, the decision maker chooses action $(a_{1,1}, a_{2,1})$, and observes o_1. Figure 8.5 shows that the subsequent belief state, given by (8.28), equals $(0.276, 0.724)$. This belief state is observed by the decision maker with probability 0.464. If instead the decision maker observed o_2, the subsequent belief state becomes $(0.06, 0.94)$ and is observed with probability 0.536. It is left to the reader to compute the values of the belief state, corresponding to using action $(a_{1,2}, a_{2,2})$.

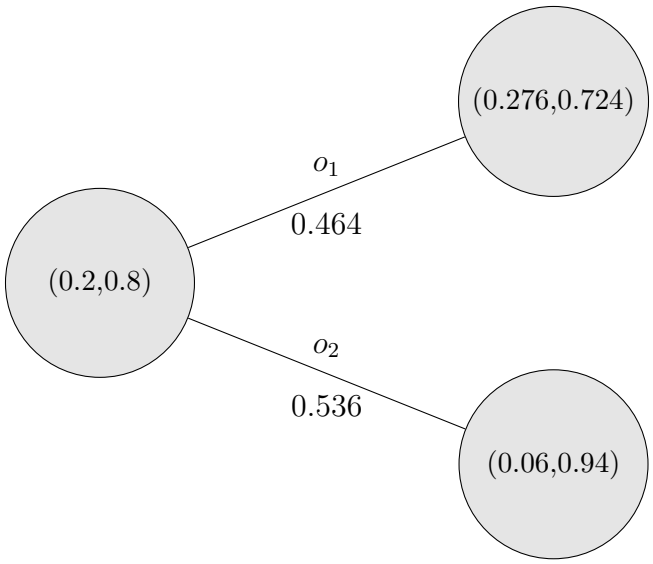


Figure 8.5: Change in belief states when using action $(a_{1,1}, a_{2,1})$.

Matrix representation

The matrix representation for the entities in the above model follow. For action $\mathbf{a} = \mathbf{a}_1 := (a_{1,1}, a_{2,1})$:

$$\mathbf{P}_{\mathbf{a}_1} = \begin{bmatrix} 0.8 & 0.2 \\ 0 & 1 \end{bmatrix}, \ \mathbf{U}^{o_1}_{\mathbf{a}_1} = \begin{bmatrix} 0.8 & 0 \\ 0 & 0.4 \end{bmatrix}, \ \mathbf{U}^{o_2}_{\mathbf{a}_1} = \begin{bmatrix} 0.2 & 0 \\ 0 & 0.6 \end{bmatrix}, \ \mathbf{R}_{\mathbf{a}_1} = \begin{bmatrix} 5 & -5 \\ 0 & -5 \end{bmatrix}. \quad (8.31)$$

When $\mathbf{a} = \mathbf{a}_2 := (a_{1,2}, a_{2,2})$,

$$\mathbf{P}_{\mathbf{a}_2} = \begin{bmatrix} 0 & 1 \\ 0.4 & 0.6 \end{bmatrix}, \ \mathbf{U}^{o_1}_{\mathbf{a}_2} = \begin{bmatrix} 0.8 & 0 \\ 0 & 0.4 \end{bmatrix}, \ \mathbf{U}^{o_2}_{\mathbf{a}_2} = \begin{bmatrix} 0.2 & 0 \\ 0 & 0.6 \end{bmatrix}, \ \mathbf{R}_{\mathbf{a}_2} = \begin{bmatrix} 0 & 5 \\ 20 & -10 \end{bmatrix}. \quad (8.32)$$

Note that $\mathbf{U}^o_{\mathbf{a}}$ is the same for both actions since the observation depends on the hidden state and not the action. Using these quantities, it is left to the reader to evaluate $\eta(o|\mathbf{b}, \mathbf{a})$ and $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$ from (8.23).

As noted above, the observation process in the two-state POMDP model is passive. Suppose there is a costly action $\bar{\mathbf{a}}$ defined by

$$\mathbf{P}_{\bar{\mathbf{a}}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \ \mathbf{U}^{o_1}_{\bar{\mathbf{a}}} = \begin{bmatrix} 0.9 & 0 \\ 0 & 0.05 \end{bmatrix}, \ \mathbf{U}^{o_2}_{\bar{\mathbf{a}}} = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.95 \end{bmatrix}, \ \mathbf{r}_{\bar{\mathbf{a}}} = \begin{bmatrix} -15 \\ -20 \end{bmatrix}$$

The presence of this action adds an active information acquisition possibility.

8.4 A finite horizon model

This section provides a framework for optimization in a finite horizon POMDP model. It presents the Bellman equation and a backwards induction algorithm to determine optimal values and policies. Using the assumption that the set of observations is finite, it establishes the piecewise linearity and convexity of the optimal value function and describes some implementable solution methods. This development uses results from Chapter 4. It assumes that S and A_s for each $s \in S$ are finite (and discrete) and stationary to simplify exposition.

8.4.1 Optimality criterion

As in the fully observable model, the decision maker seeks to maximize

$$v^\pi(\mathbf{b}) := E^\pi \left[\sum_{n=1}^{N-1} r_n(X_n, Y_n) + r_N(X_N) \,\middle|\, \mathbf{b}_1 = \mathbf{b} \right] \quad (8.33)$$

over history-dependent randomized policies $\pi = (d_1, \ldots, d_{N-1})$. We emphasize that in this definition, d_n is a *history-dependent randomized decision rule in the POMDP*, which means it is a function of the information at decision epoch n, namely $I_n = (o^n, a^{n-1}, \ldots, o^2, a^1, \mathbf{b}_1)$ as defined in (8.9).

Note that in this definition, the policy is based on the entities that are observable in the POMDP, while the quantity in the expectation is based on unobservable hidden states. The following theorem addresses this apparent mismatch.

Fundamentals

The following important result shows that the above expression is equivalent to the related expression in the derived Markov decision process.

Theorem 8.2. Suppose for $\pi \in \Pi^{\text{HR}}$, $v^\pi(\mathbf{b})$ is defined by (8.33). Then

$$v^\pi(\mathbf{b}) = E^\pi \left[\sum_{n=1}^{N-1} \tilde{r}(\mathbf{b}_n, Y_n) + \tilde{r}_N(\mathbf{b}_N) \,\middle|\, \mathbf{b}_1 = \mathbf{b} \right]. \tag{8.34}$$

The proof follows by noting that

$$\begin{aligned} E^\pi \left[r_n(X_n, Y_n) \,|\, \mathbf{b}_1 = \mathbf{b} \right] =& E^\pi \left[E^\pi \left[r_n(X_n, Y_n) \,|\, I_n \right] \,\middle|\, \mathbf{b}_1 = \mathbf{b} \right] \\ =& E^\pi \left[\sum_{s \in S} r(s, Y_n)\ b_n(s) \,\middle|\, \mathbf{b}_1 = \mathbf{b} \right] \\ =& E^\pi [\, \tilde{r}(\mathbf{b}_n, Y_n) \,|\, \mathbf{b}_1 = \mathbf{b} \,], \end{aligned} \tag{8.35}$$

where the equality in (8.35) is a result of Theorem 8.1. Equivalently,

$$E^\pi \left[r_N(X_N) \,\middle|\, \mathbf{b}_1 = \mathbf{b} \right] = E^\pi \left[\tilde{r}_N(\mathbf{b}_N) \,\middle|\, \mathbf{b}_1 = \mathbf{b} \right].$$

From a different perspective, this theorem can be interpreted as saying that $\mathbf{b}_n$ is a *sufficient statistic* for I_n with respect to the finite horizon policy value function.

Optimal policies and values

An optimal policy and value are defined similarly to fully observable models. The key distinction is in the definition of Π^{HR}, the class of history-dependent randomized policies. In a POMDP, such policies are composed of decision rules that are functions from the set of information states (previous observations and actions) to the set of probability distributions over the action set.

Definition 8.3. An *optimal policy* $\pi^* \in \Pi^{\text{HR}}$ satisfies

$$v^{\pi^*}(\mathbf{b}) \geq v^\pi(\mathbf{b}) \tag{8.36}$$

for all $\pi \in \Pi^{\text{HR}}$ and $\mathbf{b} \in \tilde{S}$.

Definition 8.4. The *value* of the POMDP is defined by

$$v^*(\mathbf{b}) := \sup_{\pi \in \Pi^{\mathrm{HR}}} v^\pi(\mathbf{b}) \tag{8.37}$$

for all $\mathbf{b} \in \tilde{S}$.

The optimal value from decision epoch $n = 1, \ldots, N-1$ to the end of the planning horizon is defined by

$$v_n^*(\mathbf{b}) := \sup_{\pi \in \Pi^{\mathrm{HR}}} v_n^\pi(\mathbf{b}), \tag{8.38}$$

where

$$v_n^\pi(\mathbf{b}) := E^\pi \left[\sum_{i=n}^{N-1} \tilde{r}(\mathbf{b}_i, Y_i) + \tilde{r}_N(\mathbf{b}_N) \,\middle|\, \mathbf{b}_n = \mathbf{b} \right]. \tag{8.39}$$

Moreover, $v_1^*(\mathbf{b}) = v^*(\mathbf{b})$ by definition.

Note that in the derived MDP, history-dependent randomized decision rules are functions of the history $(\mathbf{b}_1, \ldots, \mathbf{b}_n)$ while Markovian decision rules are functions of $\mathbf{b}_n$ only.

As a result of Theorem 4.2[11], it suffices to consider Markovian deterministic policies to obtain optimal policies.

Theorem 8.3. There exists $\pi^* \in \Pi^{\mathrm{MD}}$ that achieves the optimal value $v^*(\mathbf{b})$ for all $\mathbf{b} \in \tilde{S}$. That is,

$$v^*(\mathbf{b}) = \max_{\pi \in \Pi^{\mathrm{MD}}} v^\pi(\mathbf{b}). \tag{8.40}$$

8.4.2 The Bellman equation and its solution

In the derived MDP, given a belief state $\mathbf{b}$, choosing an action $\mathbf{a}$ and observing o, the process transitions to a new belief state $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$ with probability $\eta(o|\mathbf{b}, \mathbf{a})$ and accrues reward $\tilde{r}(\mathbf{b}, \mathbf{a})$. Define the finite horizon Bellman equation for the derived MDP as follows.

[11] Note that the proof of this result in Chapter 4 applies as well to models with compact state spaces and finite action spaces.

Definition 8.5. In a POMDP, the *Bellman equation*[a] for the derived MDP refers to the system of equations:

$$u_n(\mathbf{b}) = \max_{\mathbf{a}\in\tilde{A}} \left\{ \tilde{r}(\mathbf{b},\mathbf{a}) + \sum_{o\in O} \eta(o|\mathbf{b},\mathbf{a}) u_{n+1}(\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})) \right\} \tag{8.41}$$

for $n = 1,\ldots,N-1$ together with the boundary condition $u_N(\mathbf{b}) = \tilde{r}_N(\mathbf{b})$.

[a]Following the convention used throughout this book, the singular expression "Bellman equation" is used to refer to the system of equations (8.41) and its boundary condition.

A conceptual algorithm

Since $\tilde{A}$ is finite, the maximum in (8.41) is attained. Hence values that satisfy the Bellman equation can be computed by the following backwards induction algorithm. Of course, since the unit simplex $\tilde{S}$ is a non-finite compact set, the algorithm cannot be implemented directly.

One possible implementation is to evaluate (8.41) on a grid of values and interpolate as necessary. Unfortunately, such an approach can be prohibitive in high dimensions. More practical implementable computational algorithms are described later.

Algorithm 8.1. Finite horizon policy optimization for a POMDP

1. **Initialize:** Set $n \leftarrow N-1$ and $u_N(\mathbf{b}) \leftarrow \tilde{r}_N(\mathbf{b})$ for all $\mathbf{b}\in\tilde{S}$.
2. **Iterate:** While $n \geq 1$:
   (a) For all $\mathbf{b}\in\tilde{S}$, evaluate $u_n(\mathbf{b})$ according to

$$u_n(\mathbf{b}) \leftarrow \max_{\mathbf{a}\in\tilde{A}} \left\{ \tilde{r}(\mathbf{b},\mathbf{a}) + \sum_{o\in O} \eta(o|\mathbf{b},\mathbf{a}) u_{n+1}(\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})) \right\}, \tag{8.42}$$

   set

$$\tilde{A}^*_{n,\mathbf{b}} \leftarrow \arg\max_{\mathbf{a}\in\tilde{A}} \left\{ \tilde{r}(\mathbf{b},\mathbf{a}) + \sum_{o\in O} \eta(o|\mathbf{b},\mathbf{a}) u_{n+1}(\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})) \right\} \tag{8.43}$$

   and select $d_n(\mathbf{b}) \in \tilde{A}^*_{n,\mathbf{b}}$.
   (b) $n \leftarrow n-1$.
3. **Terminate:** Return $u_1(\mathbf{b})$ for all $\mathbf{b}\in\tilde{S}$ and $\pi = (d_1,\ldots,d_{N-1})$.

Some comments about this algorithm follow:

1. The main challenge in implementing this algorithm is that the right hand side of (8.42) must be evaluated for a non-finite number of belief states $\mathbf{b}$. This is addressed in several ways below.

2. Observe that the summation in (8.42) is indexed by the observation set O. This is because transition probabilities in this model, as given by (8.20), only assume a *finite* number of positive values indexed by elements of O.

3. Referring to the fully observable Markov decision process model in Chapter 4, where the backwards induction recursion is of the form

$$u_n(s) = \max_{a \in A_s} \left\{ r_n(s,a) + \sum_{j \in S} p_n(j|s,a) u_{n+1}(j) \right\},$$

one observes that $\eta(o|\mathbf{b},\mathbf{a})$ replaces the transition probability and $\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})$ replaces the subsequent state j.

The following result is the analog of Theorem 4.3 applied to the derived MDP. It is a fundamental result in the theory of POMDPs.

Theorem 8.4. Suppose $\tilde{A}$ is finite and $u_n(\mathbf{b})$ for $n = 1, \ldots, N$ is obtained by applying Algorithm 8.1. Then

1. For $n = 1, \ldots, N$ and $\mathbf{b} \in \tilde{S}$, $u_n(\mathbf{b}) = v_n^*(\mathbf{b})$ and $u_1(\mathbf{b}) = v^*(\mathbf{b})$.
2. Suppose $d_n^*(\mathbf{b}) \in \tilde{A}_{n,\mathbf{b}}^*$ for $n = 1, \ldots, N-1$ and $\mathbf{b} \in \tilde{S}$. Then $\pi^* = (d_1^*, \ldots, d_{N-1}^*)$ is an optimal policy.
3. Moreover, $v^{\pi^*}(\mathbf{b}) = v^*(\mathbf{b})$, for all $\mathbf{b} \in \tilde{S}$ and $v_n^{\pi^*}(\mathbf{b}) = v_n^*(\mathbf{b})$, for all $\mathbf{b} \in \tilde{S}$ and $n = 1, \ldots, N$.

Policy evaluation

Note by replacing the maximum in (8.42) by actions corresponding $\pi = (d_1, d_2, \ldots, d_{N-1}) \in \Pi^{\text{MD}}$, it follows that $v_n^\pi(\mathbf{b})$ can be determined by the recursion

$$u_n(\mathbf{b}) = \tilde{r}(\mathbf{b}, d_n(\mathbf{b})) + \sum_{o \in O} \eta(o|\mathbf{b}, d_n(\mathbf{b})) u_{n+1}(\boldsymbol{\tau}(o, d_n(\mathbf{b}), \mathbf{b})). \tag{8.44}$$

subject to $u_N(\mathbf{b}) = \tilde{r}_N(\mathbf{b})$ for $n = 1, \ldots, N-1$ and $\mathbf{b} \in \tilde{S}$.

When π is stationary, this recursion (with the inclusion of a discount factor befoe the summation) may be used to evaluate a fixed policy in an infinite horizon discounted model.

8.4.3 Structure of optimal value functions in a POMDP

This section establishes the elegant[12] result that $v_n^*(\mathbf{b})$ is piecewise linear and convex.

Piecewise linear convex functions

This brief section defines and describes the properties of piecewise linear convex functions that are fundamental in the following sections.

Definition 8.6. Let X be a subset of $\mathbb{R}^n$. A real function $f(\mathbf{x})$ on X is said to be *piecewise linear convex (PLC)* if for some finite set, Γ, of n-dimensional vectors:

$$f(\mathbf{x}) = \max_{\boldsymbol{\gamma}\in\Gamma}\{\boldsymbol{\gamma}^{\mathsf{T}}\mathbf{x}\}. \tag{8.45}$$

The above definition means that piecewise linear convex functions may be written as the maximum of a finite number of linear functions[13]. Useful properties of piecewise linear convex functions follow, proofs are left as an exercise:

1. If $f(\mathbf{x})$ and $g(\mathbf{x})$ are PLC, then $f(\mathbf{x}) + g(\mathbf{x})$ is PLC.
2. If $f(\mathbf{x})$ and $g(\mathbf{x})$ are PLC, then $\max\{f(\mathbf{x}), g(\mathbf{x})\}$ is PLC.
3. Let $\boldsymbol{\gamma}_{\mathbf{x}} \in \arg\max_{\boldsymbol{\gamma}\in\Gamma}\{\boldsymbol{\gamma}^{\mathsf{T}}\mathbf{x}\}$. If $f(\mathbf{x})$ is PLC, then $f(\mathbf{x}) = (\boldsymbol{\gamma}_{\mathbf{x}})^{\mathsf{T}}\mathbf{x}$ for each $\mathbf{x} \in X$.
4. For each $\boldsymbol{\gamma} \in \Gamma$, $G_{\boldsymbol{\gamma}} := \{\mathbf{x} \in X | f(\mathbf{x}) = \boldsymbol{\gamma}^{\mathsf{T}}\mathbf{x}\}$ is a closed convex subset of X or the empty set.
5. If Γ is finite, there are finitely many $G_{\boldsymbol{\gamma}}$.

[12] This result appears in Smallwood and Sondik [1973] and was established in Sondik's PhD dissertation.

[13] In $\mathbb{R}^1$, this is equivalent to

$$f(x) = \begin{cases} c_1 x + d_1, & x \in [a_1, b_1) \\ c_2 x + d_2, & x \in [a_2, b_2) \\ \dots & \\ c_m x + d_m, & x \in [a_m, b_m). \end{cases}$$

The fourth property does not exclude the possibility that there may be some $\boldsymbol{\gamma}' \in \Gamma$ that do not attain the maximum in (8.45) for any $\mathbf{x} \in X$. In other words $(\boldsymbol{\gamma}')^\mathsf{T}\mathbf{x} < f(\mathbf{x})$ for all $\mathbf{x} \in X$. Figure 8.7 illustrates this observation.

The following additional terminology will facilitate exposition.

Definition 8.7. Refer to elements in Γ as $\boldsymbol{\gamma}$-*vectors* and a vector $\boldsymbol{\gamma}^*$ chosen according to

$$\boldsymbol{\gamma}^* \in \arg\max_{\boldsymbol{\gamma}\in\Gamma}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{x}^*\}$$

as a *support*[a] of the convex function $f(\mathbf{x}) = \max_{\boldsymbol{\gamma}\in\Gamma}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{x}\}$ at $\mathbf{x}^*$.

[a]Some authors refer to such $\boldsymbol{\gamma}$ as a *subgradient* or *gradient* and $\boldsymbol{\gamma}^\mathsf{T}\mathbf{x}$ as a *supporting hyperplane*.

The key structural result

The following theorem establishes the fundamental result regarding the structure of the optimal value function in a POMDP. It provides the basis for all POMDP algorithms based on belief states[14]. Its straightforward proof is informative.

[14]Some simulation-based algorithms seek policies that are functions of the observations only.

Theorem 8.5. Suppose $v_n^*(\mathbf{b})$ satisfies the Bellman equation (8.41) for $n = 1, \ldots, N$ and all $\mathbf{b} \in \tilde{S}$ and $\mathbf{r}_N$, $\mathbf{r}_\mathbf{a}$, $\mathbf{P}_\mathbf{a}$, $\mathbf{U}_\mathbf{a}^o$, and $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$ are as defined in Section 8.3.1.

1. Then $v_n^*(\mathbf{b})$ is a piecewise linear convex function of $\mathbf{b}$ for $n = 1, \ldots, N$.
2. For $n = 1, \ldots, N$
$$v_n^*(\mathbf{b}) = \max_{\boldsymbol{\gamma} \in \Gamma_n} \{\boldsymbol{\gamma}^\mathsf{T} \mathbf{b}\}, \tag{8.46}$$
where $\Gamma_N = \{\mathbf{r}_N\}$,
$$\Gamma_n := \left\{ \mathbf{r}_\mathbf{a} + \sum_{o \in O} \mathbf{P}_\mathbf{a} \mathbf{U}_\mathbf{a}^o \boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})} \;\middle|\; \mathbf{a} \in \tilde{A} \right\} \tag{8.47}$$
is finite for $n = 1, \ldots, N-1$, and
$$\boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})} \in \arg\max_{\boldsymbol{\gamma} \in \Gamma_{n+1}} \{\boldsymbol{\gamma}^\mathsf{T} \boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})\} \tag{8.48}$$
3. For each $n = 1, \ldots, N$, the state space $\tilde{S}$ can be partitioned into at most $|\Gamma_n|$ polyhedra[a]. Furthermore, all states in each polyhedron have the same optimal action.

[a] Recall that a polyhedron is the set of solutions of a finite number of linear inequalities or equivalently the intersection of a finite number of half-spaces. The expression *polyhedra* refers to more than one polyhedron.

Proof. The proof of parts 1 and 2 is by (backwards) induction on n. For $n = N$, $v_N^*(\mathbf{b}) = \mathbf{r}_N^\mathsf{T} \mathbf{b}$, so it is PLC[15].

Assume that $v_m^*(\mathbf{b})$ is PLC and (8.46)-(8.48) hold for $m = n+1, \ldots, N-1$. This means in particular that for each $\mathbf{b}' \in \tilde{S}$
$$v_{n+1}^*(\mathbf{b}') = \max_{\boldsymbol{\gamma} \in \Gamma_{n+1}} \{\boldsymbol{\gamma}^\mathsf{T} \mathbf{b}'\}$$
for a *finite* set Γ_{n+1}. From (8.20), for each $\mathbf{b} \in \tilde{S}$ and $\mathbf{a} \in \tilde{A}$, there are only finitely many values for the subsequent belief state $\mathbf{b}' = \boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$ at decision epoch n, each corresponding to a different $o \in O$. Thus
$$v_n^*(\mathbf{b}) = \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) v_{n+1}^*(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\}$$

[15] Actually it is linear.

$$= \max_{\mathbf{a}\in\tilde{A}} \left\{ \tilde{r}(\mathbf{b},\mathbf{a}) + \sum_{o\in O} \eta(o|\mathbf{b},\mathbf{a}) \left(\max_{\boldsymbol{\gamma}\in\Gamma_{n+1}} \boldsymbol{\gamma}^{\mathsf{T}} \boldsymbol{\tau}(o,\mathbf{a},\mathbf{b}) \right) \right\} \tag{8.49}$$

$$= \max_{\mathbf{a}\in\tilde{A}} \left\{ \mathbf{r}_{\mathbf{a}}^{\mathsf{T}}\mathbf{b} + \sum_{o\in O} \eta(o|\mathbf{b},\mathbf{a}) \left(\max_{\boldsymbol{\gamma}\in\Gamma_{n+1}} \boldsymbol{\gamma}^{\mathsf{T}} \left(\frac{\mathbf{U}_{\mathbf{a}}^{o}\mathbf{P}_{\mathbf{a}}^{\mathsf{T}}\mathbf{b}}{\eta(o|\mathbf{b},\mathbf{a})} \right) \right) \right\} \tag{8.50}$$

$$= \max_{\mathbf{a}\in\tilde{A}} \left\{ \mathbf{r}_{\mathbf{a}}^{\mathsf{T}}\mathbf{b} + \sum_{o\in O} \left(\max_{\boldsymbol{\gamma}\in\Gamma_{n+1}} \boldsymbol{\gamma}^{\mathsf{T}}\mathbf{U}_{\mathbf{a}}^{o}\mathbf{P}_{\mathbf{a}}^{\mathsf{T}}\mathbf{b} \right) \right\} \tag{8.51}$$

$$= \max_{\mathbf{a}\in\tilde{A}} \left\{ \sum_{o\in O} \max_{\boldsymbol{\gamma}\in\Gamma_{n+1}} \left(\frac{\mathbf{r}_{\mathbf{a}}^{\mathsf{T}}}{|O|} + \boldsymbol{\gamma}^{\mathsf{T}}\mathbf{U}_{\mathbf{a}}^{o}\mathbf{P}_{\mathbf{a}}^{\mathsf{T}} \right) \mathbf{b} \right\}$$

$$= \max_{\mathbf{a}\in\tilde{A}} \left\{ \sum_{o\in O} \max_{\boldsymbol{\gamma}\in\Gamma_{n+1}} \left(\frac{\mathbf{r}_{\mathbf{a}}}{|O|} + \mathbf{P}_{\mathbf{a}}\mathbf{U}_{\mathbf{a}}^{o}\boldsymbol{\gamma} \right)^{\mathsf{T}} \mathbf{b} \right\}, \tag{8.52}$$

where (8.49) follows from the induction hypothesis and (8.50) follows from (8.23) and (8.24). The next expression (8.51) follows by canceling out the probability $\eta(o|\mathbf{b},\mathbf{a})$ and the last two expressions follow by rearranging terms. Note that moving $\mathbf{r}_{\mathbf{a}}$ inside the summation and "max" requires dividing it by the number of terms in the summation $|O|$. Since

$$\frac{\mathbf{r}_{\mathbf{a}}}{|O|} + \mathbf{P}_{\mathbf{a}}\mathbf{U}_{\mathbf{a}}^{o}\boldsymbol{\gamma} \tag{8.53}$$

is a vector, each term in (8.52) is PLC. Thus, the expression inside the braces is PLC because the sum of PLC functions is PLC. Finally since the max of PLC functions is PLC, $v_n^*(\mathbf{b})$ is PLC for $n = 1,\ldots,N$.

Defining $\boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})}$ by (8.48), substituting it into (8.53) and rearranging terms establishes part 2.

To prove the result in part 3, represent the finite set of vectors in Γ_n by $\{\boldsymbol{\gamma}_1,\ldots,\boldsymbol{\gamma}_K\}$. For an arbitrary $\boldsymbol{\gamma}_i$ in this set, consider the set B_i, which contains all $\mathbf{b}$ for which $\mathbf{v}_n^*(\mathbf{b}) = \boldsymbol{\gamma}_i^{\mathsf{T}}\mathbf{b}$. Thus

$$B_i := \{\mathbf{b}\in\tilde{S} \mid \boldsymbol{\gamma}_i^{\mathsf{T}}\mathbf{b} \geq \boldsymbol{\gamma}^{\mathsf{T}}\mathbf{b},\ \forall\boldsymbol{\gamma}\in\Gamma_n\} = \bigcap_{j\neq i} \{\mathbf{b}\in\tilde{S} \mid (\boldsymbol{\gamma}_i - \boldsymbol{\gamma}_j)^{\mathsf{T}}\mathbf{b} \geq \mathbf{0}\}. \tag{8.54}$$

Consequently, B_i is a polyhedron since it is the intersection of a finite number of half-spaces. Since $\boldsymbol{\gamma}_i$ corresponds to a particular action, that action is optimal for all belief states in B_i.

□

The following interpretation of the quantities in (8.47) and (8.48) provides motivation for algorithms below. The set Γ_n contains at most $|\tilde{A}| \times |\Gamma_{n+1}|$ elements, one for each action and element in Γ_{n+1}. The quantity $\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})$ is the belief state at decision epoch $n+1$ if the belief state at decision epoch n is $\mathbf{b}$ and action $\mathbf{a}$ is chosen at decision epoch n after observing o. The quantity $\boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})}$ is the support of $v_{n+1}^*(\cdot)$ at $\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})$. In light of these observations, the following definition is provided for convenience.

Definition 8.8. Define $\boldsymbol{\gamma}_{\mathbf{a}}(\mathbf{b})$ by

$$\boldsymbol{\gamma}_{\mathbf{a}}(\mathbf{b}) := \mathbf{r}_{\mathbf{a}} + \sum_{o \in O} \mathbf{P}_{\mathbf{a}} \mathbf{U}_{\mathbf{a}}^{o} \boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})} \tag{8.55}$$

for all $\mathbf{a} \in \tilde{A}$.

Note that $\boldsymbol{\gamma}_{\mathbf{a}}(\mathbf{b})$ can take on at most $|\tilde{A}| \times |\Gamma_{n+1}|$ values because $\boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})}$ is chosen from the finite set Γ_{n+1}. Hence $v_n^*(\mathbf{b})$ can be evaluated inductively using any of the following expressions.

Equivalent representations for $v_n^*(\mathbf{b})$

$$\begin{aligned} v_n^*(\mathbf{b}) &= \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) v_{n+1}^*(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\} \\ &= \max_{\mathbf{a} \in \tilde{A}} \left\{ \left(\mathbf{r}_{\mathbf{a}} + \sum_{o \in O} \mathbf{P}_{\mathbf{a}} \mathbf{U}_{\mathbf{a}}^{o} \boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})} \right)^{\mathsf{T}} \mathbf{b} \right\} \qquad (8.56) \\ &= \max_{\mathbf{a} \in \tilde{A}} \{ \boldsymbol{\gamma}_{\mathbf{a}}(\mathbf{b})^{\mathsf{T}} \mathbf{b} \} \qquad (8.57) \\ &= \max_{\boldsymbol{\gamma} \in \Gamma_n} \{ \boldsymbol{\gamma}^{\mathsf{T}} \mathbf{b} \}. \end{aligned}$$

The challenge in applying these results on all of $\tilde{S}$ is to efficiently determine Γ_n for $n = 1, \ldots, N-1$. Referring to these equivalent representations will enhance your understanding of the algorithms below.

Some further remarks about Theorem 8.5 follow:

1. Although the state space is infinite dimensional, the optimal value function has a finite dimensional characterization, where it is defined as the maximum of a finite set of linear functions on the unit simplex in $\mathbb{R}^{|S|}$. Note that this result holds when there is a finite observation set and a finite action set.

2. While each $\boldsymbol{\gamma} \in \Gamma_n$ is associated with a single action, the same action may be associated with different values of $\boldsymbol{\gamma}$ in different regions of $\tilde{S}$, as shown in a later example.

3. This theorem is in the spirit of results in Section 4.4 on the optimality of structured policies. Here, the special structure of $v_n^*(\mathbf{b})$ being piecewise linear convex is used to show that an optimal policy is constant over the polyhedrons in (8.54) that define $v_n^*(\mathbf{b})$.

4. In an expressions of the form $f(\mathbf{b}) = \boldsymbol{\gamma}^\mathsf{T}\mathbf{b}$, the ith component of $\boldsymbol{\gamma}$ is the marginal rate of change of $f(\mathbf{b})$ with respect to b_i. This description does not account for the condition that the components of $\mathbf{b}$ sum to 1.

Figure 8.3 shows the consequences of this theorem in a one-period model ($N = 2$) with two states. The optimal value function is piecewise linear and the one-dimensional belief space (the probability of being in the first state) is separated into two intervals, one in which the first action is optimal and one in which the second action is optimal. Note that where these intervals meet, at $q = 11/17$, both actions are optimal. A multi-period example below provides additional insights.

8.4.4 An exact algorithm for finite horizon POMDPs

As noted above, the non-finiteness of $\tilde{S}$ makes direct implementation of Algorithm 8.1 impossible. The PLC representation for $v_n(\mathbf{b})$ in Theorem 8.5 introduces a sense of finiteness that underlies several implementable algorithms. We believe the algorithm proposed by Monahan [1982] is the most transparent and provides good insight into the issues that arise when solving a POMDP exactly.

Motivation

Theorem 8.5 established that

$$v_n^*(\mathbf{b}) = \max_{\boldsymbol{\gamma}\in\Gamma_n}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\}$$

for a particular finite set Γ_n where each element of Γ_n corresponds to an action in $\tilde{A}$. Thus if Γ_n is known, for any belief state $\mathbf{b}$, the optimal action at decision epoch n can be recovered by choosing

$$\gamma_\mathbf{b} \in \arg\max_{\boldsymbol{\gamma}\in\Gamma_n}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\}$$

and finding an action $\mathbf{a}$ that corresponds to $\gamma_\mathbf{b}$ or equivalently $\boldsymbol{\gamma}_\mathbf{a}(\mathbf{b}) = \gamma_\mathbf{b}$. Moreover, as a result of part 3 of Theorem 8.5 the belief space can be partitioned into polyhedra in which a single action is optimal on the interior of each polyhedra so that an optimal decision rule can be specified explicitly.

The following algorithm generates Γ_n as follows. Assuming that Γ_{n+1} is available, Γ_n for each n can be constructed using (8.47). However, the computations quickly become complex given that $\boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})}$, which is required to compute Γ_n, depends on $o, \mathbf{a}$, and $\mathbf{b}$. In other words, computing Γ_n exactly would require determining which elements of Γ_{n+1} attain the "argmax" in equation (8.48) for *some* value of $o, \mathbf{a}$, and $\mathbf{b}$.

Rather than generating Γ_n exactly using $\boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})}$, the algorithm below generates a larger set $G_n \supseteq \Gamma_n$, and then *prunes* the unnecessary elements. Generating Γ_n exactly may require further pruning anyway, since some of the elements may not be supports of $v_n^*(\mathbf{b})$ for any $\mathbf{b}$. Thus, if G_n can be generated and pruned efficiently, it is possible to avoid computing $\boldsymbol{\gamma}_{\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})}$ for each value of $o, \mathbf{a}$, and $\mathbf{b}$.

Define G_n inductively by

$$G_n := \left\{ \mathbf{r_a} + \sum_{o \in O} \mathbf{P_a U_a^o} \bar{\boldsymbol{\gamma}}_o \;\middle|\; \mathbf{a} \in \tilde{A}, \bar{\boldsymbol{\gamma}}_o \in \Gamma_{n+1} \right\}, \tag{8.58}$$

where the recursion is initialized by $\Gamma_N = \{\mathbf{r}_N\}$. The subtle difference is to simply use *all* elements of Γ_{n+1}, to define G_n, even if they would not have attained the maximum in (8.48) for a particular $o, \mathbf{a}$, and $\mathbf{b}$.

Pruning G_n

Pruning is a key step in a POMDP algorithm. That is, elements of G_n that do not achieve the maximum in $\max_{\bar{\boldsymbol{\gamma}} \in G_n} \bar{\boldsymbol{\gamma}}^\mathsf{T} \mathbf{b}$ for any $\mathbf{b}$ need to be pruned (removed from consideration). One way to do this is to solve the following linear program for *each* $\bar{\boldsymbol{\gamma}} \in G_n$:

$$\begin{aligned} \underset{z, \mathbf{b}}{\text{minimize}} \quad & z - \bar{\boldsymbol{\gamma}}^\mathsf{T} \mathbf{b} \\ \text{subject to} \quad & z \geq \boldsymbol{\gamma}^\mathsf{T} \mathbf{b}, \quad \forall\, \boldsymbol{\gamma} \in G_n \\ & \mathbf{b} \in \tilde{S}, z \in \mathbb{R}^1. \end{aligned} \tag{8.59}$$

Note that in the objective function $\bar{\gamma}$ is *fixed* and $\mathbf{b}$ and z are variable. As a result of the first set of constraints, the objective function is bounded below by zero. The second set of constraints ensure that $\mathbf{b}$ is a belief state, that is, it lies in the unit simplex, and that z is a scalar.

The following lemma shows that solutions of this linear program identifies dominated elements of G_n.

Lemma 8.1. Suppose for some $\bar{\boldsymbol{\gamma}} \in G_n$ that the optimal objective function value of the above linear program is positive. Then $\bar{\boldsymbol{\gamma}} \notin \Gamma_n$.

Proof. As a result of constraint (8.59), the objective function value is non-negative. Furthermore if

$$v_n(\bar{\mathbf{b}}) = \max_{\boldsymbol{\gamma} \in G_n} \boldsymbol{\gamma}^\mathsf{T} \bar{\mathbf{b}} = \bar{\boldsymbol{\gamma}}^\mathsf{T} \bar{\mathbf{b}}$$

for some $\mathbf{b} \in \tilde{S}$ then the objective function of the above linear program will be zero. Hence if it is positive, $\bar{\boldsymbol{\gamma}} \notin \Gamma_n$. □

An algorithm to solve a finite horizon POMDP

The following algorithm due to Monahan [1982] implements the process described above. For each n it starts with a set of potential supports for $v_n(\mathbf{b})$ denote by G_n and

eliminates $\boldsymbol{\gamma}$-vectors one at a time using the result in Lemma 8.1. It returns the set Γ_n of $\boldsymbol{\gamma}$-vectors that are supports for $v_n(\mathbf{b})$ for at least one $\mathbf{b} \in \tilde{S}$ for $n = 1, \ldots, N$.

Algorithm 8.2. A support-based pruning algorithm

1. **Initialize:** Set $n \leftarrow N-1$ and $\Gamma_N \leftarrow \{\mathbf{r}_N\}$.
2. **Iterate:** While $n \geq 1$:
   (a) **Generate candidate vectors:**

$$G_n \leftarrow \left\{ \mathbf{r}_{\mathbf{a}} + \sum_{o \in O} \mathbf{P}_{\mathbf{a}} \mathbf{U}_{\mathbf{a}}^{o} \bar{\boldsymbol{\gamma}}_o \;\middle|\; \mathbf{a} \in \tilde{A}, \bar{\boldsymbol{\gamma}}_o \in \Gamma_{n+1} \right\}. \tag{8.60}$$

   (b) **Prune:** For each $\bar{\boldsymbol{\gamma}} \in \Gamma_n$, solve

$$\begin{aligned} \underset{z,\mathbf{b}}{\text{minimize}} \quad & z - \bar{\boldsymbol{\gamma}}^{\mathsf{T}} \mathbf{b} \\ \text{subject to} \quad & z \geq \boldsymbol{\gamma}^{\mathsf{T}} \mathbf{b}, \quad \forall \boldsymbol{\gamma} \in G_n \\ & \mathbf{b} \in \tilde{S}, z \in \mathbb{R} \end{aligned} \tag{8.61}$$

   and let $(z^*, \mathbf{b}^*)$ be an optimal solution. If $z^* - \bar{\boldsymbol{\gamma}}^{\mathsf{T}} \mathbf{b}^* > 0$,

$$G_n \leftarrow G_n \setminus \{\bar{\boldsymbol{\gamma}}\}.$$

   (c) Set $\Gamma_n \leftarrow G_n$ and $n \leftarrow n-1$.
3. **Terminate:** Return Γ_n for $n = 1, \ldots, N$.

Using the Γ_n found by the above algorithm, the optimal value in state $\mathbf{b}$ is given by

$$v_n^*(\mathbf{b}) = \max_{\boldsymbol{\gamma} \in \Gamma_n} \{\boldsymbol{\gamma}^{\mathsf{T}} \mathbf{b}\}. \tag{8.62}$$

To determine an optimal action in $\mathbf{b}$, first choose $\boldsymbol{\gamma}_{\mathbf{b}}$ to satisfy

$$\boldsymbol{\gamma}_{\mathbf{b}} \in \arg\max_{\boldsymbol{\gamma} \in \Gamma_n} \{\boldsymbol{\gamma}^{\mathsf{T}} \mathbf{b}\}, \tag{8.63}$$

and then determine action $\mathbf{a}^*$ for which:

$$\boldsymbol{\gamma}_{\mathbf{b}} = \mathbf{r}_{\mathbf{a}^*} + \sum_{o \in O} \mathbf{P}_{\mathbf{a}^*} \mathbf{U}_{\mathbf{a}}^{o} \boldsymbol{\gamma}_{\boldsymbol{\tau}(o, \mathbf{a}^*, \mathbf{b})}. \tag{8.64}$$

Some comments follow:

1. Prior to implementing the pruning step, dominated constraints can be removed to reduce the size of G_n. This is a standard procedure in modern LP solvers and illustrated in the example below.

2. Note that in implementing step 2(b), the number of constraints in (8.61) is decreasing as vectors are eliminated from G_n.

3. If one is interested in the structure of the optimal policy, one can use the result in part 3 of Theorem 8.5 to partition $\tilde{S}$ into polyhedra in which a single action is optimal. Of course more than one action may be optimal on the boundaries.

4. The calculations described in (8.62)-(8.64) can be avoided by *maintaining a list of which actions correspond to which elements of* Γ_n.

Other approaches

POMDP algorithms can be classified[16] as support-based, belief-based or approximate.

As noted above, the above algorithm is *support-based*; at each iteration it starts with a set of possible supports and prunes it to obtain a reduced set of supports that define the optimal value function at that iteration. From these supports the polyhedra defining an optimal policy can be determined.

An alternative class of algorithms is *belief-based*. They start with a belief state $\mathbf{b}'$ and seek to define the region in which the support of $v_n(\mathbf{b})$ at $\mathbf{b}'$, denote by $\boldsymbol{\gamma}_{\mathbf{b}'}$ is optimal. The intent is to perform this calculation at sufficiently many such $\mathbf{b}'$ so as to characterize $v_n(\mathbf{b})$ for all $\mathbf{b} \in \tilde{S}$. These points are sometimes referred to as *witness points* because they "testify" to the existence of a polyhedron in which $\boldsymbol{\gamma}_{\mathbf{b}'}$ is a support of $v_n(\mathbf{b})$.

Approximate methods focus on extending exact calculations on finite subsets of $\tilde{S}$ to approximate $v_n(\mathbf{b})$ for all belief states. These subsets can be fixed as described in the next section or generated throughout the computational process.

An application of Algorithm 8.2

The calculations in this section demonstrate the application of Algorithm 8.2 to the two-state example from Section 8.3.2 with $N = 3$. In this example, express $\mathbf{b}$ by (q_1, q_2) where $q_1 \geq 0$, $q_2 \geq 0$ and $q_1 + q_2 = 1$ and let $\mathbf{a}_1 = (a_{1,1}, a_{2,1})$ and $\mathbf{a}_2 = (a_{1,2}, a_{2,2})$.

To make the example more insightful, set $r(s_2, a_{2,1}, s_2) = 4$ instead of -5. This means that the reward matrix in (8.31) changes to

$$\mathbf{R}_{\mathbf{a}_1} = \begin{bmatrix} 5 & -5 \\ 0 & 4 \end{bmatrix}.$$

With this change, $r(s_2, a_{2,1}) = 4$ so that $\mathbf{r}_{\mathbf{a}_1} = (3, 4)$. Moreover set $r_3(s) = 0$ for all $s \in S$ so that $\tilde{\mathbf{r}}_3(\mathbf{b}) = \mathbf{0}$ for all $\mathbf{b} \in \tilde{S}$.

[16] Spaan and Vlassis [2005] provides a nice overview of POMDP algorithms.

The iterates of the algorithm follow:

1. For $n = 3$, $\Gamma_3 = \{\mathbf{r}_3\} = \{(0, 0)\}$ so that $v_3^*(\mathbf{b}) = \max_{\boldsymbol{\gamma}\in\Gamma_3}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\} = 0$.

2. For $n = 2$, since $\Gamma_3 = \{\mathbf{0}\}$,

$$G_2 = \{\mathbf{r}_{(a_{1,1},a_{2,1})}, \mathbf{r}_{(a_{1,2},a_{2,2})}\} = \{(3, 4), (5, 2)\}.$$

It is clear from Figure 8.6 that both vectors in G_2 are required to define $v_2(\mathbf{b})$ so that $\Gamma_2 = G_2$.

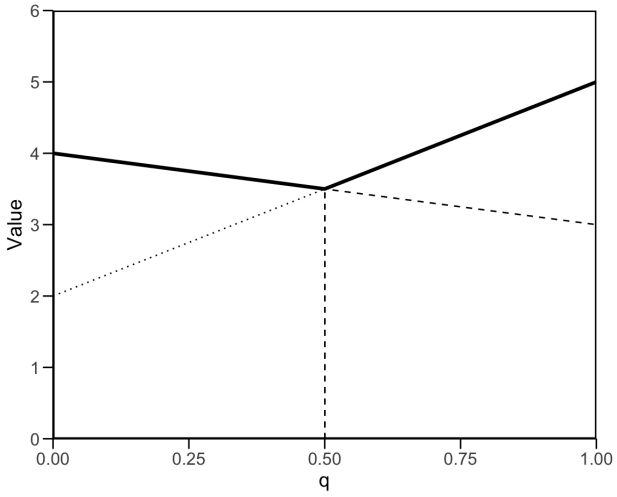


Figure 8.6: Graphical representation of $v_2^*(\mathbf{b})$. Note that $q_1 = q$ and $q_2 = 1 - q$.

Hence

$$v_2^*(\mathbf{b}) = \max_{\boldsymbol{\gamma}\in\Gamma_2}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\} = \max\{3q_1 + 4q_2, 5q_1 + 2q_2\},$$

where the first term in the max corresponds to action $(a_{1,1}, a_{2,1})$ and the second term corresponds to action $(a_{1,2}, a_{2,2})$. Using the equivalent representation $q_1 = q$ and $q_2 = 1 - q$, the optimal value function can be written as

$$v_2^*(\mathbf{b}) = \max\{4 - q, 2 + 3q\},$$

and represented in Figure 8.6. Thus,

$$d_2^*(\mathbf{b}) = \arg\max_{\boldsymbol{\gamma}\in\Gamma_2}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\} = \begin{cases}(a_{1,1}, a_{2,1}), & 0 \le q \le 0.5\\ (a_{2,1}, a_{2,2}), & 0.5 < q \le 1.\end{cases} \tag{8.65}$$

That is, for belief states where the probability of being in state s_1 is less than 0.5, action $(a_{1,1}, a_{2,1})$ should be chosen. Otherwise, choose action $(a_{2,1}, a_{2,2})$. Hence $\tilde{S}$ is partitioned into two polyhedra, which in this case are intervals.

3. The case $n = 1$ provides more insight into the workings of the algorithm. In it

$$G_1 = \Big\{ \mathbf{r}_{(a_{1,1},a_{2,1})} + \mathbf{P}_{(a_{1,1},a_{2,1})}\mathbf{U}^{o1}_{(a_{1,1},a_{2,1})}\boldsymbol{\gamma}_1 + \mathbf{P}_{(a_{1,1},a_{2,1})}\mathbf{U}^{o2}_{(a_{1,1},a_{2,1})}\boldsymbol{\gamma}_2,$$
$$\mathbf{r}_{(a_{1,2},a_{2,2})} + \mathbf{P}_{(a_{1,2},a_{2,2})}\mathbf{U}^{o1}_{(a_{1,2},a_{2,2})}\boldsymbol{\gamma}_1 + \mathbf{P}_{(a_{1,2},a_{2,2})}\mathbf{U}^{o2}_{(a_{1,2},a_{2,2})}\boldsymbol{\gamma}_2 \;\Big|\; (\boldsymbol{\gamma}_1, \boldsymbol{\gamma}_2) \in \Gamma_2 \times \Gamma_2 \Big\}.$$

A comment about the above expression may be helpful. Note that since Γ_2 has two elements, $(3, 4)$ and $(5, 2)$, the pair (γ_1, γ_2) can take on **four** values, that is

$$\Gamma_2 \times \Gamma_2 = \big\{\big((3,4),(3,4)\big), \big((3,4),(5,2)\big), \big((5,2),(3,4)\big), \big((5,2),(5,2)\big)\big\}.$$

Since there are two actions, G_1 has eight elements. For example, when $(\boldsymbol{\gamma}_1, \boldsymbol{\gamma}_2) = \big((3,4),(3,4)\big)$, then the element of G_1 corresponding to action $\mathbf{a}_1 = (a_{1,1}, a_{2,1})$ is

$$\mathbf{r}_{(a_{1,1},a_{2,1})} + \mathbf{P}_{(a_{1,1},a_{2,1})}\mathbf{U}^{o1}_{(a_{1,1},a_{2,1})}\gamma_1 + \mathbf{P}_{(a_{1,1},a_{2,1})}\mathbf{U}^{o2}_{(a_{1,1},a_{2,1})}\boldsymbol{\gamma}_2 =$$

$$\begin{bmatrix}3\\4\end{bmatrix} + \begin{bmatrix}0.8 & 0.2\\ 0 & 1\end{bmatrix}\begin{bmatrix}0.8 & 0\\ 0 & 0.4\end{bmatrix}\begin{bmatrix}3\\4\end{bmatrix} + \begin{bmatrix}0.8 & 0.2\\ 0 & 1\end{bmatrix}\begin{bmatrix}0.2 & 0\\ 0 & 0.6\end{bmatrix}\begin{bmatrix}3\\4\end{bmatrix} = \begin{bmatrix}6.2\\8\end{bmatrix}.$$

Similar calculations show that

$$G_1 = \left\{\begin{bmatrix}6.2\\8\end{bmatrix}, \begin{bmatrix}6.28\\6.8\end{bmatrix}, \begin{bmatrix}7.32\\7.2\end{bmatrix}, \begin{bmatrix}7.4\\6\end{bmatrix}, \begin{bmatrix}9\\5.6\end{bmatrix}, \begin{bmatrix}7.8\\5.04\end{bmatrix}, \begin{bmatrix}8.2\\5.76\end{bmatrix}, \begin{bmatrix}7\\5.2\end{bmatrix}\right\}, \tag{8.66}$$

where the first four vectors correspond to action $\mathbf{a}_1 = (a_{1,1}, a_{2,1})$ and the last four correspond to action $\mathbf{a}_2 = (a_{1,2}, a_{2,2})$. Observe that there are three dominated elements (the second, sixth, and eighth vectors) so that G_1 can be reduced to

$$G_1 = \left\{\begin{bmatrix}6.2\\8\end{bmatrix}, \begin{bmatrix}7.32\\7.2\end{bmatrix}, \begin{bmatrix}7.4\\6\end{bmatrix}, \begin{bmatrix}9\\5.6\end{bmatrix}, \begin{bmatrix}8.2\\5.76\end{bmatrix}\right\}.$$

To obtain Γ_1 requires solving five linear programs. Setting $\bar{\gamma} = (6.2, 8)$, the first linear program becomes

$$\begin{aligned} \underset{z,q_1\geq 0,q_2\geq 0}{\text{minimize}} \quad & z - (6.2q_1 + 8q_2) \\ \text{subject to} \quad & z \geq 6.2q_1 + 8q_2, \\ & z \geq 7.32q_1 + 7.2q_2, \\ & z \geq 7.4q_1 + 6q_2, \\ & z \geq 9q_1 + 5.6q_2, \\ & z \geq 8.2q_1 + 5.76q_2, \\ & 1 = q_1 + q_2. \end{aligned}$$

Since the objective function is positive, $(6.2, 8)$ can be eliminated from G_1 so the next linear program uses $\bar{\gamma} = (7.32, 7.20)$ in the objective function and has one fewer constraint. Continuing in this way establishes that

$$\Gamma_1 = \{(6.2, 8), (7.32, 7.2), (9, 5.6)\},$$

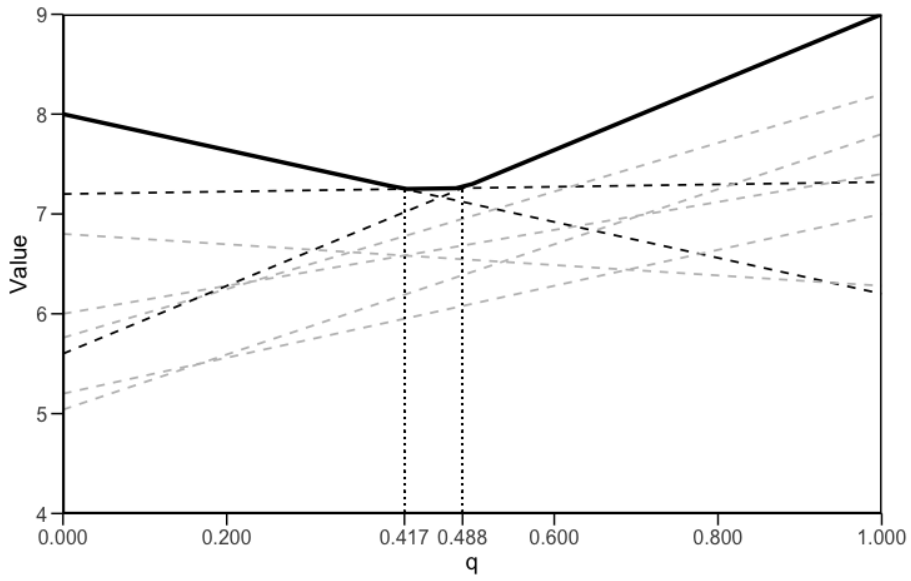


Figure 8.7: Graphical representation of $v_1^*(\mathbf{b})$. It shows $\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}$ for all $\boldsymbol{\gamma} \in G_1$ and that $q_1 = q$ and $q_2 = 1 - q$. Note that lines corresponding to $\boldsymbol{\gamma} \notin \Gamma_1$ are represented by grey dashed lines and those corresponding to $\boldsymbol{\gamma} \in \Gamma_1$ by black dashed lines. The bold line represents the piecewise linear convex function $v_1^*(\mathbf{b})$.

where the first two vectors correspond to $\mathbf{a}_1$ and the third vector corresponds to $\mathbf{a}_2$.

Thus Figure 8.7 shows that

$$\begin{aligned} v_1^*(\mathbf{b}) &= \max_{\boldsymbol{\gamma}\in\Gamma_1}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\} \\ &= \max\{6.2q_1 + 8q_2, 7.32q_1 + 7.2q_2, 9q_1 + 5.6q_2\} \\ &= \max\{8 - 1.8q, 7.2 + 0.12q, 5.6 + 3.4q\} \\ &= \begin{cases} 8 - 1.8q & \text{for } 0 \le q \le 0.417 \\ 7.2 + 0.12q & \text{for } 0.417 \le q \le 0.488 \\ 5.6 + 3.4q & \text{for } 0.488 \le q \le 1. \end{cases} \end{aligned}$$

Hence $v_1^*(\mathbf{b})$ is PLC and it can be shown that for

$$d_1^*(\mathbf{b}) = \begin{cases} \mathbf{a}_1, & 0 \le q \le 0.488 \\ \mathbf{a}_2, & 0.488 < q \le 1. \end{cases} \tag{8.67}$$

Figure 8.7 shows that $v_1^*(\mathbf{b})$ is piecewise linear convex with three distinct supports $(6.2, 8)$, $(7.32, 7.2)$, and $(9, 5.6)$. The first breakpoint occurs at $q = 5/12 = 0.417$ and the second breakpoint occurs at $q = 20/41 = 0.488$. On both sides of the first breakpoint, the optimal action is $(a_{1,1}, a_{2,1})$, so the fact that a breakpoint exists there

is due to the fact that $\boldsymbol{\gamma}_{\boldsymbol{\tau}(o_1,(a_{1,1},a_{2,1}),(q_1,q_2))}$ changes from $(3,4)$ to $(5,2)$. In other words, it is due to the breakpoint in $v_2^*(\mathbf{b})$. The second breakpoint is due to a change in the optimal action, where for $q < 0.488$, $\mathbf{a}_1 = (a_{1,1}, a_{2,1})$ is optimal, and for $q > 0.488$, $\mathbf{a}_2 = (a_{1,2}, a_{2,2})$ is optimal.

8.4.5 A finite-grid approximation for finite horizon POMDPs

This section outlines a straightforward and implementable approach for approximating solutions to a POMDP. The core principle involves evaluating the optimal value function at a *finite grid* of points within the belief space $\tilde{S}$ rather than across the entire continuous space. This yields a set of $\boldsymbol{\gamma}$-vectors, which can be used to:

1. Generate upper and lower bounds on the unknown true optimal value function.
2. Derive a policy that is guaranteed to be within a known distance of the optimal policy (based on the bounds).

This provides a practical method for tackling the computational challenges associated with solving POMDPs and has provided the basis for several approximate algorithms for solving[17] larger models.

A lower bound

Generating a lower bound is straightforward. Since the optimal value function at decision epoch n can be written as

$$v_n^*(\mathbf{b}) = \max_{\boldsymbol{\gamma}\in\Gamma_n}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\}, \tag{8.68}$$

a lower bound on it can be obtained by instead choosing the maximum over a **subset** of Γ_n. To see why this is true, let H be a finite subset set of $\tilde{S}$ and let Γ_n^H be the set of $\boldsymbol{\gamma} \in \Gamma_n$ that define the optimal value function on H. That is,

$$\Gamma_n^H := \left\{ \boldsymbol{\gamma}_\mathbf{b} \;\middle|\; \boldsymbol{\gamma}_\mathbf{b} \in \arg\max_{\boldsymbol{\gamma}\in\Gamma_n}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\}, \mathbf{b} \in H \right\}. \tag{8.69}$$

Then

$$\underline{v}_n(\mathbf{b}) := \max_{\boldsymbol{\gamma}\in\Gamma_n^H}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\} \tag{8.70}$$

is a lower bound on $v_n^*(\mathbf{b})$ for all $\mathbf{b}$. In particular

$$\underline{v}_n(\mathbf{b}) = \begin{cases} v_n^*(\mathbf{b}) & \text{for } \mathbf{b} \in H \\ \leq v_n^*(\mathbf{b}) & \text{for } \mathbf{b} \in \tilde{S} \setminus H. \end{cases} \tag{8.71}$$

Note that if Γ_n is not available, then (8.69) can be applied with any subset of Γ_n. In this case there may be an inequality for $\mathbf{b} \in H$ in (8.71) if a $\boldsymbol{\gamma}$-vector that supports $v_n^*(\mathbf{b})$ has been omitted. In this case, $\underline{v}_n(\mathbf{b}) \leq v_n^*(\mathbf{b})$.

[17] See the Bibliographic Remarks section for references.

An upper bound

To generate an upper bound, find $v_n^*(\mathbf{b})$ for $\mathbf{b} \in H$ and interpolate the optimal value function between points in H.

Let $H = \{\mathbf{b}_1, \ldots, \mathbf{b}_m\}$. Suppose $\mathbf{b} \in \tilde{S} \setminus H$ can be written as a convex combination of elements in H[18]. Then, $v_n^*(\mathbf{b})$ for $\mathbf{b} \in \tilde{S} \setminus H$ can be bounded above by

$$\overline{v}_n(\mathbf{b}) := \sum_{i=1}^{m} \alpha_i v_n^*(\mathbf{b}_i), \tag{8.72}$$

where $\mathbf{b} = \sum_{i=1}^m \alpha_i \mathbf{b}_i$, $\alpha_i \geq 0, i = 1, \ldots, m$, and $\sum_{i=1}^m \alpha_i = 1$. The convexity of $v_n^*(\mathbf{b})$ and Jensen's inequality implies that

$$\sum_{i=1}^{m} \alpha_i v_n^*(\mathbf{b}_i) \geq v_n^* \left(\sum_{i=1}^{m} \alpha_i \mathbf{b}_i \right) = v_n^*(\mathbf{b}),$$

so that

$$\overline{v}_n(\mathbf{b}) = \begin{cases} v_n^*(\mathbf{b}), & \mathbf{b} \in H \\ \sum_{i=1}^m \alpha_i v_n^*(\mathbf{b}_i), & \mathbf{b} \in \tilde{S} \setminus H \end{cases} \tag{8.73}$$

is an upper bound on $v_n^*(\mathbf{b})$ for all $\mathbf{b}$.

Obtaining a decision rule

One approach to obtain a policy for an arbitrary $\mathbf{b} \in \tilde{S}$ is to use the lower bound as follows.

$$d_n(\mathbf{b}) \in \arg\max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) \underline{v}_{n+1}\big(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})\big) \right\}. \tag{8.74}$$

The reason for using the lower bound is that the difference between the value of this policy and the optimal value can be bounded using (8.77) below.

To evaluate the above requires computing $\eta(o|\mathbf{a}, \mathbf{b})$ and $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$ using (8.24) and then evaluating

$$\underline{v}_{n+1}\big(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})\big) = \max_{\boldsymbol{\gamma} \in \Gamma_{n+1}^H} \{\boldsymbol{\gamma}^\mathsf{T} \boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})\}. \tag{8.75}$$

To do this requires only storing Γ_{n+1}^H. Note that since $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$ most likely will not be an element of H, $\underline{v}_{n+1}\big(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})\big)$ must be evaluated as in (8.75). With this representation, an action that achieves the arg max in (8.74) can be tracked via the corresponding $\boldsymbol{\gamma}$-vector.

[18] To ensure all points in $\tilde{S}$ can be represented in this way requires that H contain the extreme points of the unit simplex.

The policy $\pi = (d_1, \ldots, d_{N-1})$ where each $d_n(\mathbf{b})$ is defined by (8.74) is referred to as the *lower bound approximate policy.*

Properties of the bounds

Using the upper and lower bounding functions, the value of the policy defined by (8.74) and the optimal policy can be bounded as follows.

Theorem 8.6. For any $\mathbf{b} \in \tilde{S}$, suppose $\underline{v}_n(\mathbf{b})$ and $\overline{v}_n(\mathbf{b})$ are computed using (8.70) and (8.72) for n=$1, \ldots, N$ and let $\pi = (d_1, d_2, \ldots, d_{N-1})$ be a policy that uses decision rules d_n obtained from (8.74). Then

$$\underline{v}_n(\mathbf{b}) \leq v_n^{\pi}(\mathbf{b}) \leq v_n^*(\mathbf{b}) \leq \overline{v}_n(\mathbf{b}). \tag{8.76}$$

for any $n = 1, \ldots, N$.

Proof. The proof is by induction. For $n = N$, the inequalities hold by definition. Assume

$$\underline{v}_j(\mathbf{b}) \leq v_j^{\pi}(\mathbf{b}) \leq v_j^*(\mathbf{b}) \leq \overline{v}_j(\mathbf{b})$$

for $j = n+1, \ldots, N$. Then for all $\mathbf{b} \in \tilde{S}$

$$\begin{aligned}
\underline{v}_n(\mathbf{b}) &= \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) \underline{v}_{n+1}(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\} \\
&\leq \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) v_{n+1}^{\pi}(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\} \\
&= v_n^{\pi}(\mathbf{b}) \leq v_n^*(\mathbf{b}) \\
&= \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) v_{n+1}^*(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\} \\
&\leq \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) \overline{v}_{n+1}(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\} \\
&= \overline{v}_n(\mathbf{b}),
\end{aligned}$$

where the first inequality and last inequalities are a result of the induction hypothesis. □

As a consequence of this theorem, one obtains the following bound.

Corollary 8.1. Under the hypothesis of Theorem 8.6,

$$0 \leq v_n^*(\mathbf{b}) - v_n^\pi(\mathbf{b}) \leq \overline{v}_n(\mathbf{b}) - \underline{v}_n(\mathbf{b}) \tag{8.77}$$

for $n = 1, \ldots, N$ and all $\mathbf{b} \in \tilde{S}$.

Another upper bound

The following provides a different and potentially useful upper bound on the optimal value function. For all $\mathbf{b} \in \tilde{S}$ and $n = 1, \ldots, N-1$, inductively define

$$\overline{v}'_n(\mathbf{b}) := \sum_{i=1}^m \alpha_i \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}_i, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}_i, \mathbf{a}) \overline{v}'_{n+1}(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b}_i)) \right\}, \tag{8.78}$$

where $\mathbf{b} = \sum_{i=1}^m \alpha_i \mathbf{b}_i$, $\alpha_i \geq 0, i = 1, \ldots, m$, and $\sum_{i=1}^m \alpha_i = 1$, with $\overline{v}'_N(\mathbf{b}) = \tilde{r}_N(\mathbf{b})$.

Proposition 8.1. For all $\mathbf{b} \in \tilde{S}$ and $n = 1, \ldots, N$,

$$v_n^*(\mathbf{b}) \leq \overline{v}'_n(\mathbf{b}). \tag{8.79}$$

Proof. For $n = N$, the inequality holds by definition. For a given $n < N$, assume the result holds for $n+1, \ldots, N$. Then,

$$\begin{aligned}
\overline{v}'_n(\mathbf{b}) &= \sum_{i=1}^m \alpha_i \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}_i, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}_i, \mathbf{a}) \overline{v}'_{n+1}(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b}_i)) \right\} \\
&\geq \sum_{i=1}^m \alpha_i \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}_i, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}_i, \mathbf{a}) v^*_{n+1}(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b}_i)) \right\} \\
&= \sum_{i=1}^m \alpha_i v_n^*(\mathbf{b}_i) \geq v_n^* \left(\sum_{i=1}^m \alpha_i \mathbf{b}_i \right), = v_n^*(\mathbf{b})
\end{aligned}$$

where the last inequality is due to Jensen's inequality. □

Since both (8.73) and (8.78) generate upper bounds on the optimal value function, the minimum of the two is also an upper bound.

Choosing a finite grid

To generate the lower bound, any set $H \subseteq \tilde{S}$ suffices. However, to generate the upper bound, all points in $\tilde{S}$ need to be expressable as convex combinations of elements of H. Thus, H must include the vertices of $\tilde{S}$. Furthermore, the set H should balance being

fine enough so the approximation is of sufficient quality while being coarse enough to remain computationally tractable.

One particular set H that satisfies these requirements is based on the *Freundenthal triangulation* of $\mathbb{R}^m$, which induces a grid over the unit simplex (i.e., the belief space) and allows the modeler to adjust its granularity. Let K be a positive integer and consider the set[19]

$$H = \left\{ \frac{\mathbf{k}}{K} \;\middle|\; \mathbf{k} \in \mathbb{Z}_+^m, \sum_{i=1}^{m} k_i = K \right\}. \tag{8.80}$$

This set H generates a grid (and a corresponding partition) over the unit simplex that becomes increasingly fine as K increases.

Another choice is to choose H randomly by simulating belief states. Ideally these points should in some way represent beliefs that will be observed in practice and as well cover a large portion of $\tilde{S}$. These belief states can be based on random sampling from the prior or on updates resulting from implementing a policy. Several approximate algorithms are based on different approaches to choosing and modifying the grid points.

Algorithms to compute bounds and approximate policies

Computations based on Lovejoy [1991a] and Lovejoy [1991b] are summarized in the following two algorithms. The first iteratively constructs the elements required to derive lower and upper bounds on the optimal value function and the lower bound approximate policy by restricting attention to belief states in the finite grid H. The second algorithm uses the output of the first algorithm to compute bounds and the lower bound approximate policy at an arbitrary belief state $\mathbf{b}'$.

Note that two algorithms are described when a single statement would suffice. This is so as to distinguish objectives and calculations. In practice one would combine them at each decision epoch.

Algorithm 8.3. Bounding the value of a finite horizon POMDP on a grid

1. **Initialize:**
   (a) Specify a finite set $H \leftarrow \{\mathbf{b}_1, \ldots, \mathbf{b}_m\} \subseteq \tilde{S}$.
   (b) $\Gamma_N^H \leftarrow \{\mathbf{r}_N\}$ and $\Gamma_N \leftarrow \{\mathbf{r}_N\}$.
   (c) For all $\mathbf{b} \in H$, $\underline{v}_N(\mathbf{b}) \leftarrow \tilde{r}_N(\mathbf{b})$, $v_N^*(\mathbf{b}) \leftarrow \tilde{r}_N(\mathbf{b})$, and $\overline{v}_N(\mathbf{b}) \leftarrow \tilde{r}_N(\mathbf{b})$.
   (d) $n \leftarrow N - 1$.
2. **Iterate:** While $n \geq 1$:

[19] Recall that $\mathbb{Z}_+^m$ denotes the m-dimensional space of positive integers.

(a) **Generate lower bound on H:**

i.
$$\Gamma_n \leftarrow \left\{ \mathbf{r}_\mathbf{a} + \sum_{o \in O} \mathbf{P}_\mathbf{a} \mathbf{U}_\mathbf{a}^o \bar{\boldsymbol{\gamma}}_o \;\middle|\; \mathbf{a} \in \tilde{A}, \bar{\boldsymbol{\gamma}}_o \in \Gamma_{n+1}^H \right\}. \tag{8.81}$$

ii.
$$\Gamma_n^H \leftarrow \left\{ \boldsymbol{\gamma}_\mathbf{b} \;\middle|\; \boldsymbol{\gamma}_\mathbf{b} \in \arg\max_{\boldsymbol{\gamma} \in \Gamma_n} \{\boldsymbol{\gamma}^\mathsf{T} \mathbf{b}\}, \mathbf{b} \in H \right\}. \tag{8.82}$$

iii. For $\mathbf{b} \in H$[a]
$$\underline{v}_n(\mathbf{b}) \leftarrow \max_{\boldsymbol{\gamma} \in \Gamma_n^H} \{\boldsymbol{\gamma}^\mathsf{T} \mathbf{b}\}. \tag{8.83}$$

(b) **Generate upper bound on H:** For all $\mathbf{b} \in H$,
$$\overline{v}_n(\mathbf{b}) \leftarrow \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) \overline{v}_{n+1}(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\}. \tag{8.84}$$

(c) $n \leftarrow n - 1$.

3. **Terminate:** Return Γ_n^H and $\overline{v}_n(\mathbf{b})$ for $\mathbf{b} \in H$ and $n = 1, \ldots, N$.

[a] This calculation applies to $\mathbf{b} \in \tilde{S} \setminus H$ as well. It is not necessary to include this in the algorithm; computing Γ_n^H suffices.

The above algorithm summarizes calculations on the grid. The following algorithm applies them to obtain bounds and approximate policies for any belief state.

Algorithm 8.4. Extending bounds to arbitrary belief states.

1. **Initialize:**
   (a) Specify $\mathbf{b}' \in \tilde{S} \setminus H$.
   (b) Apply Algorithm 8.3.
   (c) Set $n \leftarrow N - 1$.
2. **Iterate:** While $n \geq 1$:
   (a) **Generate lower bound at $\mathbf{b}'$:**
$$\underline{v}_n(\mathbf{b}') \leftarrow \max_{\boldsymbol{\gamma} \in \Gamma_n^H} \{\boldsymbol{\gamma}^\mathsf{T} \mathbf{b}'\}. \tag{8.85}$$

(b) **Generate upper bound at $\mathbf{b}'$:**

$$\overline{v}_n(\mathbf{b}') \leftarrow \sum_{i=1}^{m} \alpha_i \overline{v}_n(\mathbf{b}_i), \tag{8.86}$$

where $\mathbf{b}' = \sum_{i=1}^{m} \alpha_i \mathbf{b}_i, \sum_{i=1}^{m} \alpha_i = 1, \alpha_i \geq 0$ for $i = 1, \ldots, m$ and $\mathbf{b}_i \in H$ are the closest points to $\mathbf{b}'$ that allow $\mathbf{b}'$ to be written as their convex combination.

(c) **Generate lower bound approximate policy at $\mathbf{b}'$**: Set

$$d_n(\mathbf{b}') \in \arg\max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}', \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}', \mathbf{a}) \underline{v}_{n+1}\big(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b}')\big) \right\}, \tag{8.87}$$

where

$$\underline{v}_{n+1}\big(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b}')\big) = \max_{\boldsymbol{\gamma} \in \Gamma^H_{n+1}} \{\boldsymbol{\gamma}^\mathsf{T} \boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b}')\}.$$

(d) $n \leftarrow n - 1$.

3. **Terminate:** Return $\underline{v}_n(\mathbf{b}')$, $\overline{v}_n(\mathbf{b}')$ and $d_n(\mathbf{b}')$ for $n = 1, \ldots, N$.

Some comments on these algorithms follow:

1. When applying these algorithms in practice, the calculations can be combined for each n when structural results or a policy are required. Alternatively, they can be applied online by first applying Algorithm 8.3 and then applying Algorithm 8.4 when $\mathbf{b}'$ is observed at decision epoch n.

2. Using $\Gamma^H_n = \{\boldsymbol{\gamma}_1, \ldots, \boldsymbol{\gamma}_{|H|}\}$, the structure of the lower bound approximate policy can be determined by partitioning $\tilde{S}$ into $|H|$ polyhedra defined by (breaking ties arbitrarily)

$$\left\{ \mathbf{b} \in \tilde{S} \;\middle|\; \boldsymbol{\gamma}_i = \arg\max_{\boldsymbol{\gamma} \in \Gamma^H_n} \{\boldsymbol{\gamma}^\mathsf{T} \mathbf{b}\} \right\}.$$

3. For each n, $|\Gamma^H_n| \leq |H|$. This avoids the exponential growth of $|\Gamma_n|$ in Algorithm 8.2.

4. Note that (8.81) differs from (8.60) in Algorithm 8.2 by replacing Γ_n by the smaller set Γ^H_n. Consequently Γ^H_n may not contain $\boldsymbol{\gamma}$-vectors corresponding to the optimal value function.

5. The quality of the approximation depends on H. Note that H can vary with n and be defined adaptively. Moreover, different specifications of H can be used to obtain the upper and lower bound.

6. The upper bound for arbitrary $\mathbf{b} \in \tilde{S} \setminus H$ is the interpolation of the values $\overline{v}_n(\mathbf{b}_i)$, where $\mathbf{b}_i \in H$ are the nearest points to $\mathbf{b}$ that allow $\mathbf{b}$ to be written as a convex combination of those points. If H corresponds to the Freundenthal triangulation, then for any point $\mathbf{b} \in \tilde{S}$, it is straightforward to find the closest integer lattice points in $\mathbb{Z}_+^m$ around $K\mathbf{b}$.

An example

The following example applies Algorithms 8.3 and 8.4 to an $N = 4$ version of the two-state model analyzed above. However, to avoid using different notation to refer to the same constructs in the exact calculations above, $N = 3$ represents the final decision epoch and the decision epoch $n = 0$ is added prior to decision epoch 1 to represent the extra period.

Choose the finite grid $H = \{\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3\} = \{(0,1), (0.4, 0.6), (1,0)\}$.

1. For $n = 3$, $\Gamma_3^H = \Gamma_3 = \{\mathbf{r}_3\} = \{(0,0)\}$. Then, $\underline{v}_3(\mathbf{b}) = \overline{v}_3(\mathbf{b}) = 0$ for $\mathbf{b} \in H$.

2. For $n = 2$, obtain Γ_2 using (8.81). Since $\Gamma_3^H = \Gamma_3$, $\Gamma_2 = \{(3,4), (5,2)\}$ as above. Following (8.82), $\Gamma_2^H = \{(3,4), (5,2)\}$. Then

$$\underline{v}_2(\mathbf{b}) = \max\{3q_1 + 4q_2, 5q_1 + 2q_2\},$$

where $\mathbf{b} = (q_1, q_2)$. Thus, $\underline{v}_2((0,1)) = 4$, $\underline{v}_2((0.4, 0.6)) = 3.6$ and $\underline{v}_2((1,0)) = 5$. Moreover,

$$\underline{v}_2(\mathbf{b}) = \max_{\boldsymbol{\gamma} \in \Gamma_2^H} \{\boldsymbol{\gamma}^\mathsf{T} \mathbf{b}\}$$

for $\mathbf{b} \in \tilde{S}$.

To obtain the upper bound using (8.84) for $\mathbf{b} \in H$,

$$\overline{v}_2(\mathbf{b}) = \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) \overline{v}_3(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\}, \tag{8.88}$$

but since $\overline{v}_3(\mathbf{b}) = 0$ for all $\mathbf{b}$, $\overline{v}_2(\mathbf{b}) = \max_{\mathbf{a} \in \tilde{A}} \{\tilde{r}(\mathbf{b}, \mathbf{a})\} = \max\{3q_1 + 4q_2, 5q_1 + 2q_2\}$. Thus, $\overline{v}_2((0,1)) = 4$, $\overline{v}_2((0.4, 0.6)) = 3.6$ and $\overline{v}_2((1,0)) = 5$, identical to the corresponding lower bounds.

For $\mathbf{b} \in \tilde{S} \setminus H$ and $\alpha_i \geq 0$, $i = 1, 2, 3$,

$$\overline{v}_2(\mathbf{b}) = \begin{cases} \alpha_1 \overline{v}_2(\mathbf{b}_1) + \alpha_2 \overline{v}_2(\mathbf{b}_2), & \alpha_1 + \alpha_2 = 1, 0.4\alpha_2 = q_1, 0 \leq q_1 \leq 0.4, \\ \alpha_2 \overline{v}_2(\mathbf{b}_2) + \alpha_3 \overline{v}_2(\mathbf{b}_3) & \alpha_2 + \alpha_3 = 1, 0.4\alpha_2 + \alpha_3 = q_1, 0.4 < q_1 \leq 1 \end{cases}$$

$$= \begin{cases} 4\alpha_1 + 3.6\alpha_2, & \alpha_1 + \alpha_2 = 1, 0.4\alpha_2 = q_1, 0 \leq q_1 \leq 0.4, \\ 3.6\alpha_2 + 5\alpha_3, & \alpha_2 + \alpha_3 = 1, 0.4\alpha_2 + \alpha_3 = q_1, 0.4 < q_1 \leq 1. \end{cases} \tag{8.89}$$

These bounds are shown for all $\mathbf{b}$ in Figure 8.8a. Note that since the belief state is 1-dimensional, at most two of the α_i need to be positive for any given $\mathbf{b}'$. In particular, if $\mathbf{b}'$ is to the left of the breakpoint at 0.4, only α_1 and α_2 are positive, otherwise α_2 and α_3 are positive.

Note also that the lower bound equals the optimal value found earlier. So, the lower bound approximate policy matches the optimal policy given in (8.65).

3. For $n = 1$, obtain Γ_1 using (8.81) and (8.82). Since $\Gamma_2^H = \Gamma_2$,

$$\Gamma_1 = \{(6.2, 8), (7.32, 7.2), (9, 5.6)\},$$

from calculations using the exact algorithm. It is easy to see from (8.82) that

$$\Gamma_1^H = \{\boldsymbol{\gamma}' \mid \boldsymbol{\gamma}' \in \arg\max_{\boldsymbol{\gamma} \in \Gamma_1} \{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}\}, \mathbf{b} \in \{(0, 1), (0.4, 0.6), (1, 0)\}\} = \{(6.2, 8), (9, 5.6)\}.$$

As noted in the example of applying Algorithm 8.2, the first $\boldsymbol{\gamma}$-vector corresponds to $\mathbf{a}_1$ and the second to $\mathbf{a}_2$. Moreover observe that Γ_1^H is a **proper subset** of Γ_1. Hence from (8.85),

$$\underline{v}_1(\mathbf{b}) = \max\{6.2q_1 + 8q_2, 9q_1 + 5.6q_2\}$$

for all $\mathbf{b} = (q_1, q_2) \in \tilde{S}$ and $\underline{v}_1((0, 1)) = 8$, $\underline{v}_1((0.4, 0.6)) = 7.28$ and $\underline{v}_1((1, 0)) = 9$.

To obtain the upper bound for $\mathbf{b} \in H$, from (8.84)

$$\overline{v}_1(\mathbf{b}) = \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) \overline{v}_2(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\}. \tag{8.90}$$

This requires computing $\eta(o|\mathbf{b}, \mathbf{a})$ and $\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})$ for all $\mathbf{b} \in H$ and $o \in O$. For $\mathbf{b} = (0, 1)$, $\eta(o_1|\mathbf{b}, \mathbf{a}_1) = 0.4$, $\eta(o_2|\mathbf{b}, \mathbf{a}_1) = 0.6$, $\eta(o_1|\mathbf{b}, \mathbf{a}_2) = 0.56$, $\eta(o_2|\mathbf{b}, \mathbf{a}_2) = 0.44$, $\boldsymbol{\tau}(o_1, \mathbf{a}_1, \mathbf{b}) = (0, 1)$, $\boldsymbol{\tau}(o_2, \mathbf{a}_1, \mathbf{b}) = (0, 1)$, $\boldsymbol{\tau}(o_1, \mathbf{a}_2, \mathbf{b}) = (4/7, 3/7)$, and $\boldsymbol{\tau}(o_2, \mathbf{a}_2, \mathbf{b}) = (2/11, 9/11)$. Thus,

$$\begin{aligned} \overline{v}_1((0, 1)) &= \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) \overline{v}_2(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\} \\ &= \max\{(0, 1)^\mathsf{T}(3, 4) + 0.4\overline{v}_2((0, 1)) + 0.6\overline{v}_2((0, 1)), \\ &\qquad (0, 1)^\mathsf{T}(5, 2) + 0.56\overline{v}_2((4/7, 3/7)) + 0.44\overline{v}_2((2/11, 9/11))\}. \end{aligned}$$

Note that the second term in the braces requires $\overline{v}_2((4/7, 3/7))$ and $\overline{v}_2((2/11, 9/11))$, that is, at values of $\mathbf{b}' \notin H$. From (8.89) and noting that:

$$\left(\frac{4}{7}, \frac{3}{7}\right) = \frac{5}{7}(0.4, 0.6) + \frac{2}{7}(1, 0) \quad \text{and} \quad \left(\frac{2}{11}, \frac{9}{11}\right) = \frac{6}{11}(0, 1) + \frac{5}{11}(0.4, 0.6),$$

it follows that

$$\begin{aligned}\overline{v}_1((0,1)) = \max\{&4 + 0.4\overline{v}_2((0,1)) + 0.6\overline{v}_2((0,1)),\\ &2 + 0.56\left(\frac{5}{7}\overline{v}_2((0.4,0.6)) + \frac{2}{7}\overline{v}_2((1,0))\right)\\ &+ 0.44\left(\frac{6}{11}\overline{v}_2((0,1)) + \frac{5}{11}\overline{v}_2((0.4,0.6))\right)\Big\}.\end{aligned}$$

Substituting in the values above results in $\overline{v}_1((0,1)) = 8$. Carrying out similar computations for the other values of $\mathbf{b} \in H$, $\overline{v}_1((0.4,0.6)) = 7.429$ and $\overline{v}_1((1,0)) = 9$.

For $\mathbf{b} \in \tilde{S} \setminus H$ and $\alpha_i \geq 0$, $i = 1,2,3$,

$$\begin{aligned}\overline{v}_1(\mathbf{b}) &= \begin{cases}\alpha_1\overline{v}_1(\mathbf{b}_1) + \alpha_2\overline{v}_1(\mathbf{b}_2), & \alpha_1 + \alpha_2 = 1, 0.4\alpha_2 = q_1, 0 \leq q_1 \leq 0.4,\\ \alpha_2\overline{v}_1(\mathbf{b}_2) + \alpha_3\overline{v}_1(\mathbf{b}_3) & \alpha_2 + \alpha_3 = 1, 0.4\alpha_2 + \alpha_3 = q_1, 0.4 < q_1 \leq 1\end{cases}\\ &= \begin{cases}8\alpha_1 + 7.429\alpha_2, & \alpha_1 + \alpha_2 = 1, 0.4\alpha_2 = q_1, 0 \leq q_1 \leq 0.4,\\ 7.429\alpha_2 + 9\alpha_3, & \alpha_2 + \alpha_3 = 1, 0.4\alpha_2 + \alpha_3 = q_1, 0.4 < q_1 \leq 1.\end{cases}\end{aligned} \tag{8.91}$$

Figure 8.8b displays the bounds and the optimal value function. Since Γ_1^H is a proper subset of Γ_1, the lower bound lies below the optimal value function on the region $q_1 \in [0.417, 0.488]$ as result of the calculations displayed in Figure 8.7.

Noting the relationship between supports and actions referred to above, the lower bound approximate policy can be shown to be (the breakpoint in the lower bound occurs at 0.462):

$$d_1(\mathbf{b}) = \begin{cases}\mathbf{a}_1, & 0 \leq q \leq 0.462\\ \mathbf{a}_2, & 0.462 < q \leq 1.\end{cases} \tag{8.92}$$

Note that this policy differs from the optimal policy for $0.462 \leq q < 0.488$. The bound in (8.77) can be used to assess the quality of this approximate policy.

4. For $n = 0$,

$$\Gamma_0^H = \{(9.56, 12), (11.16, 11.04), (13, 9.28)\}.$$

Thus, $\underline{v}_0((0,1)) = 12$, $\underline{v}_0((0.4,0.6)) = 11.088$ and $\underline{v}_0((1,0)) = 13$. Thus for $\mathbf{b}' = (q_1, q_2) \notin H$

$$\underline{v}_0(\mathbf{b}') = \max_{\boldsymbol{\gamma} \in \Gamma_0^H}\{\boldsymbol{\gamma}^\mathsf{T}\mathbf{b}'\} = \max\{9.56q_1 + 12q_2, 11.16q_1 + 11.04q_2, 13q_1 + 9.28q_2\}.$$

Following the same approach as above, the upper bound is

$$\begin{aligned}\overline{v}_0((0,1)) = \max\{&1 + 0.4\overline{v}_1((0,1)) + 0.6\overline{v}_1((0,1)),\\ &2 + 0.56\overline{v}_1((4/7,3/7)) + 0.44\overline{v}_1((2/11,9/11))\} = 12\end{aligned}$$

$\overline{v}_0((0.4,0.6)) = 11.325$ and $\overline{v}_1((1,0)) = 13$.

For $\mathbf{b} \in \tilde{S} \setminus H$ and $\alpha_i \geq 0$, $i = 1, 2, 3$,

$$\begin{aligned}\overline{v}_0(\mathbf{b}) &= \begin{cases} \alpha_1 \overline{v}_0(\mathbf{b}_1) + \alpha_2 \overline{v}_0(\mathbf{b}_2), & \alpha_1 + \alpha_2 = 1, 0.4\alpha_2 = q_1, 0 \leq q_1 \leq 0.4, \\ \alpha_2 \overline{v}_0(\mathbf{b}_2) + \alpha_3 \overline{v}_0(\mathbf{b}_3) & \alpha_2 + \alpha_3 = 1, 0.4\alpha_2 + \alpha_3 = q_1, 0.4 < q_1 \leq 1 \end{cases} \\ &= \begin{cases} 12\alpha_1 + 11.325\alpha_2, & \alpha_1 + \alpha_2 = 1, 0.4\alpha_2 = q_1, 0 \leq q_1 \leq 0.4, \\ 11.325\alpha_2 + 13\alpha_3, & \alpha_2 + \alpha_3 = 1, 0.4\alpha_2 + \alpha_3 = q_1, 0.4 < q_1 \leq 1. \end{cases} \end{aligned} \tag{8.93}$$

The lower bound approximate policy can be shown to be

$$d_1(\mathbf{b}) = \begin{cases} \mathbf{a}_1, & 0 \leq q \leq 0.375 \\ \mathbf{a}_1, & 0.375 < q \leq 0.489 \\ \mathbf{a}_2, & 0.489 < q \leq 1. \end{cases} \tag{8.94}$$

Let Γ^{opt} denote the set of $\boldsymbol{\gamma}$-vectors generated by Algorithm 8.2. Applying Algorithm 8.2 for an additional decision epoch it can be shown that optimal value function for $n = 0$ is defined by

$$\Gamma_0^{\text{opt}} = \{(9.56, 12), (10.213, 11.68), (11.16, 11.04), (13, 9.28)\}.$$

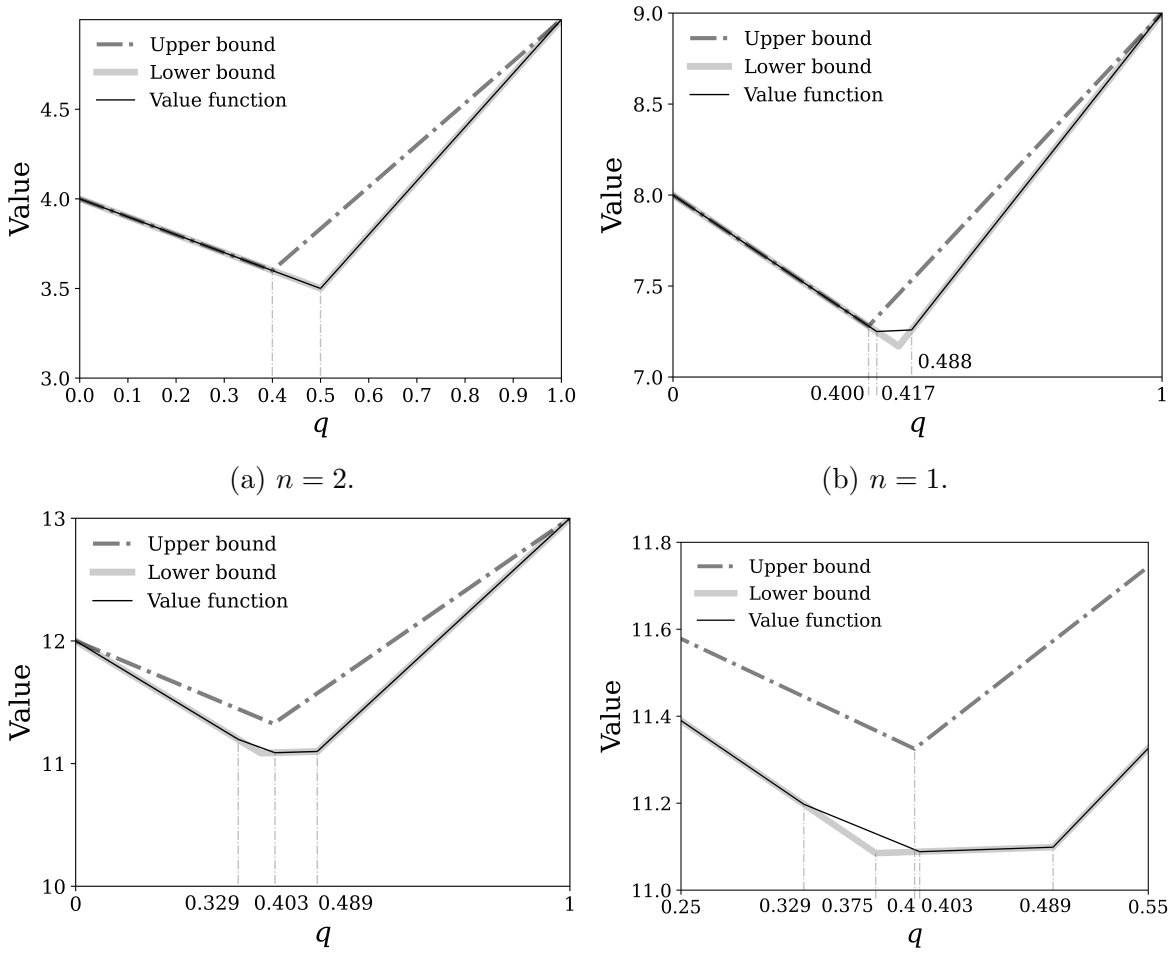
and the corresponding optimal policy is

$$d_1(\mathbf{b}) = \begin{cases} \mathbf{a}_1, & 0 \leq q \leq 0.329 \\ \mathbf{a}_1, & 0.329 < q \leq 0.403 \\ \mathbf{a}_1, & 0.403 < q \leq 0.489 \\ \mathbf{a}_2, & 0.489 < q \leq 1. \end{cases} \tag{8.95}$$

Thus, the lower bound approximate policy matches the optimal policy.

To clearly illustrate the workings of this algorithm, the optimal value function and the bounds at each iteration are plotted below. Again, let $q = q_1 = 1 - q_2$ denote the probability of being in the first state. Figure 8.8 illustrates the bounds for $n = 0, 1, 2$. Figure 8.8a shows that optimal value function for $n = 2$ is defined by $\Gamma_2^{\text{opt}} = \{(3, 4), (5, 2)\}$. The lower bound matches the optimal value function, since $\Gamma_2^{\text{opt}} = \Gamma_2^H$. The upper bound matches the optimal value function for $q \in [0, 0.4]$ and lies above the optimal value function for $q \in (0.4, 1)$. Figure 8.8b shows that the optimal value function for $n = 1$ is defined by $\Gamma_1^{\text{opt}} = \{(6.2, 8), (7.32, 7.2), (9, 5.6)\}$. Since $\Gamma_1^H = \{(6.2, 8), (9, 5.6)\}$, the lower bound lies below the optimal value function for q between 0.417 and 0.488. The upper and lower bounds continue to match the optimal value function on H.

Finally, Figures 8.8c and 8.8d, show that the optimal value function for $n = 0$ lies strictly between the upper and lower bounds: in contrast to the previous iterations, there is a gap between the upper and lower bound at $q = 0.4$. However, as shown above, the lower bound approximate policy matches the optimal policy.



(a) $n = 2$.

(b) $n = 1$.

(c) $n = 0$.

(d) $n = 0$ (zoomed in).

Figure 8.8: Optimal value function and its upper and lower bounds for the two-state POMDP model.

8.5 Infinite horizon models

This section presents a brief discussion of infinite horizon POMDPs focused on the discounted model.

8.5.1 Optimality criterion

For $0 \leq \lambda < 1$, let $v_\lambda^\pi(\mathbf{b})$ be the expected discounted reward of a policy $\pi = (d_1, d_2, \ldots)$, given an initial belief state $\mathbf{b} \in \tilde{S}$:

$$v_\lambda^\pi(\mathbf{b}) := E^\pi \left[\sum_{n=1}^{\infty} \lambda^{n-1} \tilde{r}(\mathbf{b}_n, d_n(\mathbf{b}_n)) \;\middle|\; \mathbf{b}_1 = \mathbf{b} \right]. \tag{8.96}$$

Definition 8.9. An *optimal policy* $\pi^* \in \Pi^{\text{HR}}$ satisfies

$$v_\lambda^{\pi^*}(\mathbf{b}) \geq v_\lambda^{\pi}(\mathbf{b}) \tag{8.97}$$

for all $\pi \in \Pi^{\text{HR}}$ and $\mathbf{b} \in \tilde{S}$.

Definition 8.10. The *value* of the POMDP is defined by

$$v_\lambda^*(\mathbf{b}) := \sup_{\pi \in \Pi^{\text{HR}}} v_\lambda^{\pi}(\mathbf{b}). \tag{8.98}$$

for all $\mathbf{b} \in \tilde{S}$.

Similar to the fully observable case, stationary deterministic policies are optimal within the class of all polices for a POMDP.

Theorem 8.7. There exists $\pi^* \in \Pi^{\text{SD}}$ that achieves the optimal value $v_\lambda^*(\mathbf{b})$ for all $\mathbf{b} \in \tilde{S}$. That is,

$$v_\lambda^{\pi^*}(\mathbf{b}) = v_\lambda^*(\mathbf{b}). \tag{8.99}$$

Going forward, focus is restricted to stationary deterministic policies.

8.5.2 Computing optimal values and finding optimal policies

Extending equation (8.44) to the discounted infinite horizon setting, the value of a stationary deterministic policy $\pi = (d, d, \ldots)$ can be written as

$$v_\lambda^{\pi}(\mathbf{b}) = \tilde{r}(\mathbf{b}, d(\mathbf{b})) + \lambda \sum_{o \in O} \eta(o|\mathbf{b}, d(\mathbf{b})) v_\lambda^{\pi}(\boldsymbol{\tau}(o, d(\mathbf{b}), \mathbf{b})). \tag{8.100}$$

Thus, the optimal value function can be written as

$$v_\lambda^*(\mathbf{b}) = \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \lambda \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) v_\lambda^*(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\}. \tag{8.101}$$

If $d^*(\mathbf{b})$ achieves the maximum in equation (8.101), then $\pi^* = (d^*, d^*, \ldots)$ is a stationary optimal policy with value $v_\lambda^*(\mathbf{b})$ for each $\mathbf{b} \in \tilde{S}$.

Similar to the fully observable case, the Bellman operator is a contraction mapping, which means that the Banach fixed-point theorem (Theorem 5.5) applies and there exists a unique solution to (8.101) that can be obtained by value iteration. Because the limit of PLC functions need not be piecewise linear, all that can be guaranteed is that $v_\lambda^*(\mathbf{b})$ is convex.

As frequently pointed out above there are challenges to applying value iteration. They will be discussed after describing a value iteration algorithm conceptually.

Algorithm 8.5. Value iteration for an infinite horizon discounted POMDP

1. **Initialize:**
   (a) Specify $v'(\mathbf{b})$ for all $\mathbf{b} \in \tilde{S}$.
   (b) Specify $\epsilon > 0$ and $\sigma > \epsilon$.
2. **Iterate:** While $\sigma \geq \epsilon$:
   (a) $v(\mathbf{b}) \leftarrow v'(\mathbf{b})$ for all $\mathbf{b} \in \tilde{S}$.
   (b) For all $\mathbf{b} \in \tilde{S}$, compute
   $$v'(\mathbf{b}) \leftarrow \max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \lambda \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) v(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\}. \quad (8.102)$$
   (c) $\sigma \leftarrow \|\mathbf{v}' - \mathbf{v}\|$.
3. **Terminate:** For all $\mathbf{b} \in \tilde{S}$, return
   $$d_\epsilon(\mathbf{b}) \in \arg\max_{\mathbf{a} \in \tilde{A}} \left\{ \tilde{r}(\mathbf{b}, \mathbf{a}) + \lambda \sum_{o \in O} \eta(o|\mathbf{b}, \mathbf{a}) v(\boldsymbol{\tau}(o, \mathbf{a}, \mathbf{b})) \right\}.$$

Stopping criteria

Most approaches use a stopping criterion of the form

$$\|\mathbf{v}' - \mathbf{v}\| := \max_{\mathbf{b} \in \tilde{S}} |v'(\mathbf{b}) - v(\mathbf{b})| < \epsilon.$$

Note that the "max" rather than the "sup" is appropriate since for a finite number of iterations, $v(\mathbf{b})$ is continuous and $\tilde{S}$ is compact so the "max" is attained. The challenge in applying this criterion is to compute the "max", which itself involves solving an optimization problem. In small instances this may not be a problem because the iterates can be represented in closed form. In larger instances an option is to evaluate this criterion at a large number of states. Alternatives include stopping when there are small changes in the $\boldsymbol{\gamma}$-vectors or in the policy itself.

Optimal value function updates

For problems with a few states and actions, the update can be implemented exactly as in Algorithm 8.2. This approach may not be suitable for large problems since each step is computationally intensive especially when G_n is large.

Most workable alternatives are based on the ideas underlying Algorithms 8.3 and 8.4, namely, to restrict calculation of supports to a limited number of belief states. Such algorithms are referred to as *point-based value iteration*.

Algorithms differ on how the points at which to compute updates are chosen. Algorithms 8.3 and 8.4 use a fixed grid of points, some are based on sampling points, and some are based on using a tree to represent the possible realizations of the belief states. The details are beyond the scope of this book and the reader is referred to references in the Bibliographic Remarks section.

Large problems

The most promising approaches for very large problems appear to be those that are based on reinforcement learning methods (Chapter 11). Policy gradient type methods do not use the belief state approach and instead derive a randomized policy based on observations. In such methods, policies are determined by neural networks.

Bounds

The bound from Theorem 5.12 in Chapter 5 may be useful in assessing the quality of the greedy policy obtained by the above algorithm. In the notation of this chapter, for a vector $\mathbf{v}$, $\overline{\mathbf{v}} := \max_{\mathbf{b}\in\tilde{S}} v(\mathbf{b})$ and $\underline{\mathbf{v}} := \min_{\mathbf{b}\in\tilde{S}} v(\mathbf{b})$.

Theorem 8.8. For all $\mathbf{b} \in \tilde{S}$,

$$\begin{aligned} v(\mathbf{b}) + & (1-\lambda)^{-1}(\underline{\mathbf{v}' - \mathbf{v}}) \\ & \leq v'(\mathbf{b}) + \lambda(1-\lambda)^{-1}(\underline{\mathbf{v}' - \mathbf{v}}) \\ & \leq v_\lambda^{d^\infty}(\mathbf{b}) \leq v_\lambda^*(\mathbf{b}) \\ & \leq v'(\mathbf{b}) + \lambda(1-\lambda)^{-1}(\overline{\mathbf{v}' - \mathbf{v}}) \\ & \leq v(\mathbf{b}) + (1-\lambda)^{-1}(\overline{\mathbf{v}' - \mathbf{v}}), \end{aligned} \tag{8.103}$$

where

$$d(\mathbf{b}) \in \arg\max_{\mathbf{a}\in\tilde{A}} \left\{ \tilde{r}(\mathbf{b},\mathbf{a}) + \lambda \sum_{o\in O} \eta(o|\mathbf{b},\mathbf{a}) v\big(\boldsymbol{\tau}(o,\mathbf{a},\mathbf{b})\big) \right\}.$$

The challenge in applying these bounds is computing the quantities $(\overline{\mathbf{v}' - \mathbf{v}})$ and $(\underline{\mathbf{v}' - \mathbf{v}})$, which involve computing the maximum and minimum over the uncountable set of belief states.

Note that if $v(\mathbf{b}) = 0$, then $v'(\mathbf{b}) = \max_{a\in\tilde{A}} \tilde{r}(\mathbf{b},\mathbf{a})$ so that

$$\underline{r} := \min_{s\in s, a\in A_s} r(s,a) \leq v'(\mathbf{b}) \leq \max_{s\in s, a\in A_s} r(s,a) := \overline{r}$$

and the outer bounds become the obvious inequalities

$$\frac{\underline{r}}{1-\lambda} \leq v_\lambda^{d^\infty}(\mathbf{b}) \leq v_\lambda^*(\mathbf{b}) \leq \frac{\overline{r}}{1-\lambda}$$

for all $\mathbf{b} \in \tilde{S}$.

Note that Lovejoy [1991a] provided the following bound on the difference between the optimal n-period value function $v_n^*(\mathbf{b})$ and the optimal infinite horizon value function:

$$\frac{\underline{r}\,\lambda^n}{1-\lambda} \leq v_n^*(\mathbf{b}) \leq v_\lambda^*(\mathbf{b}) \leq \frac{\overline{r}\,\lambda^n}{1-\lambda}.$$

Hence the inequality

$$0 \leq v_\lambda^*(\mathbf{b}) - v_n^*(\mathbf{b}) \leq \frac{(\overline{r}-\underline{r})\,\lambda^n}{1-\lambda}$$

can determine how many value iteration steps to apply to obtain a good approximation of the infinite horizon optimal value function.

8.6 Examples

This section formulates POMDP models in several applications. In contrast to an MDP formulation, a POMDP formulation requires the additional specification of the observations. Derived MDP formulations for several examples are also provided.

8.6.1 Preventive maintenance and inspection

A machine that manufactures a product can be in one of two conditions, "good" or "bad". At any decision epoch, the decision maker can do one of three things: 1) minimal observation of the output, 2) actively inspect the output or 3) repair the machine. As a consequence of choosing an action, the decision maker receives a signal regarding the status of the equipment. Assume all actions each require one period and generate a signal at the end of the period. The first two actions provide a signal of the machine's status with active inspection providing a more accurate signal; neither affects machine state. The repair action results in the machine being in the "good" state by the start of the next period.

Different costs are associated with observing and repairing the machine. Revenue depends on the quality of the product, and thus depends on the machine state. This model has both passive and active information acquisition actions. The first action is passive and amounts to a visual inspection of the product. The second action is active and could come in the form of a diagnostic of the product's functionality. A POMDP formulation follows.

Decision epochs: The horizon may be either finite or infinite.

$$T = \{1, 2, \ldots, N\}, \quad N \leq \infty.$$

Hidden States:

$$S = \{s_1, s_2\},$$

where s_1 = "good" and s_2 = "bad".

Actions:

$$A = \{a_1, a_2, a_3\},$$

where a_1 = "do nothing", a_2 = "inspect" and a_3 = "repair". Note that all actions are applicable in each state.

Observations:

$$O = \{o_1, o_2\},$$

where o_1 = "normal" and o_2 = "abnormal". The probability of these observations depends on the hidden state and the action at the previous decision epoch. Since there are only two signals, it suffices to specify the probability of observing o_1:

$$u(o_1|s,a) = \begin{cases} \alpha_1 & s = s_1, a = a_1 \\ \alpha_2 & s = s_2, a = a_1 \\ \beta_1 & s = s_1, a = a_2 \\ \beta_2 & s = s_2, a = a_2 \\ 1 & s \in \{s_1, s_2\}, a = a_3. \end{cases}$$

The expression for $u(o_1|s,a)$ assumes that it is more likely to observe a "normal" signal in the "good" state than the "bad" state ($\alpha_1 > \alpha_2$, $\beta_1 > \beta_2$) and that inspection provides more accurate information than observation ($\beta_1 > \alpha_1$, $\beta_2 > \alpha_2$). If the decision maker decides to repair the equipment, it takes one period, the hidden state becomes "good" at the next decision epoch and the observed signal will be "normal" with probability 1.

Rewards: Rewards depend on assumptions regarding when the equipment changes state between decision epochs. For ease of exposition assume that a transition between states occurs at the end of a period, immediately preceding the subsequent decision epoch. Under this assumption, rewards are accrued prior to, and thus are independent of, the subsequent state. The reward function is given by

$$r(s,a) = \begin{cases} r_1 & s = s_1, a = a_1 \\ r_2 & s = s_2, a = a_1 \\ r_1 - k & s = s_1, a = a_2 \\ r_2 - k & s = s_2, a = a_2 \\ -K & s \in \{s_1, s_2\}, a = a_3. \end{cases}$$

Assume that the equipment generates greater revenue in the "good" state ($r_1 > r_2$) and that it costs more to repair than to inspect ($K > k$). When the machine is being repaired it does not produce any product, hence there is no revenue generated during that period.

Transition probabilities:

$$p(s'|s,a) = \begin{cases} 1-\gamma & s' = s_1, s = s_1, a \in \{a_1, a_2\} \\ \gamma & s' = s_2, s = s_1, a \in \{a_1, a_2\} \\ 0 & s' = s_1, s = s_2, a \in \{a_1, a_2\} \\ 1 & s' = s_2, s = s_2, a \in \{a_1, a_2\} \\ 1 & s' = s_1, s \in \{s_1, s_2\}, a = a_3 \\ 0 & s' = s_2, s \in \{s_1, s_2\}, a = a_3 \end{cases} \tag{8.104}$$

When the decision maker decides to observe or inspect, (8.104) shows that the probability that the equipment changes status from "good" to "bad" in one period equals γ and that it remains in the "bad" state with certainty once it reaches there. When the decision maker decides to repair, the equipment moves to the "good" state with certainty.

The dynamics are depicted graphically in Figure 8.9. Note that the observation probabilities depend on action choice.

The derived MDP

Next, the derived MDP for this example is formulated. Decision epochs remain unchanged.

States: Since there are two hidden states,

$$\tilde{S} = \{(q_1, q_2) \mid q_1 + q_2 = 1, q_1 \geq 0, q_2 \geq 0\} = \mathcal{S}_1,$$

where q_1 is the probability the system is in the "good" state and q_2 is the probability the system is in the "bad" state.

Actions: In this example, the action set is the same for each hidden state and can be implemented in each state. Thus, in the derived MDP, the action set simplifies to

$$\tilde{A} = A = \{a_1, a_2, a_3\},$$

where a_1, a_2, a_3 are as defined in the POMDP.

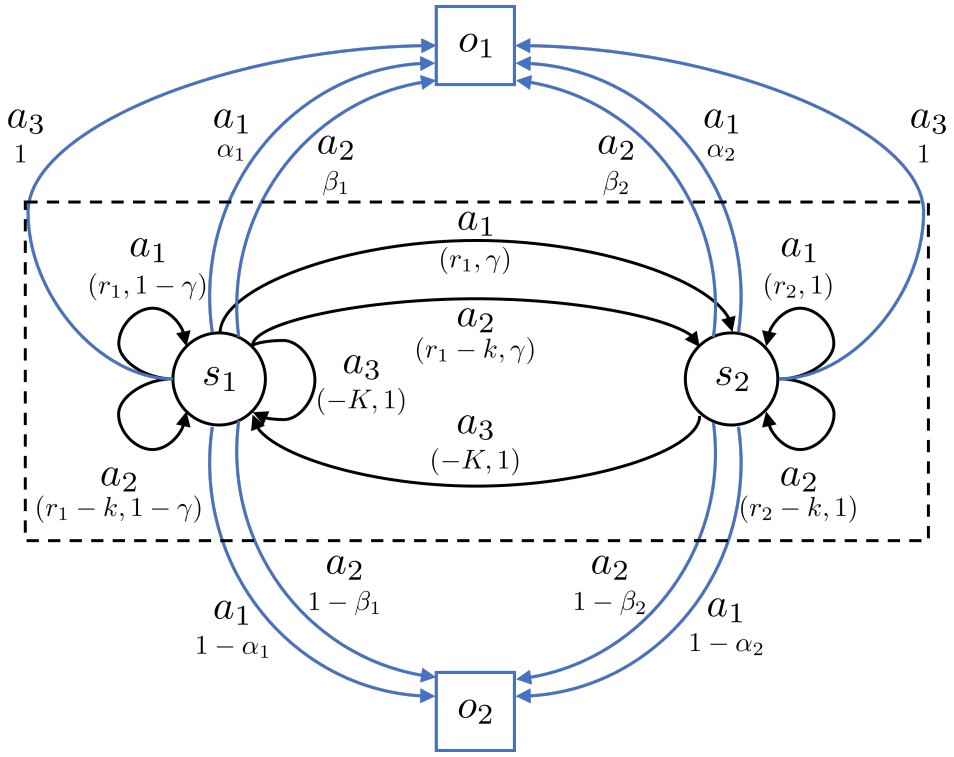


Figure 8.9: Graphical representation of the preventive maintenance POMDP model. The area in the dashed box represents the hidden (unobservable) Markov decision process. Labels (r, p) on the unobservable transitions refer to rewards and probabilities, respectively. The squared boxes indicate the observable signals, with probabilities of observation on the arcs from the hidden states.

Rewards: Let $\mathbf{b} = (q_1, q_2)$. From equation (8.16), with a scalar action,

$$\tilde{r}(\mathbf{b}, a) = \sum_{s \in S} r(s, a) b(s) = \begin{cases} r_1 q_1 + r_2 q_2 & a = a_1 \\ r_1 q_1 + r_2 q_2 - k & a = a_2 \\ -K & a = a_3. \end{cases}$$

Transition Probabilities: Let $\mathbf{b} = (q_1, q_2)$ and $\mathbf{b}' = (q_1', q_2')$. From equation (8.18),

$$\begin{aligned} \eta(o_1 | \mathbf{b}, a) &= \sum_{s' \in S} u(o_1 | s', a) \sum_{s \in S} p(s' | s, a) b(s) \\ &= \begin{cases} \alpha_1(1-\gamma)q_1 + \alpha_2(\gamma q_1 + q_2) & a = a_1 \\ \beta_1(1-\gamma)q_1 + \beta_2(\gamma q_1 + q_2) & a = a_2 \\ 1 & a = a_3 \end{cases} \end{aligned}$$

and

$$\begin{aligned}\eta(o_2|\mathbf{b},a) &= \sum_{s'\in S} u(o_2|s',a)\sum_{s\in S} p(s'|s,a)b(s)\\ &= \begin{cases}(1-\alpha_1)(1-\gamma)q_1+(1-\alpha_2)(\gamma q_1+q_2) & a=a_1\\ (1-\beta_1)(1-\gamma)q_1+(1-\beta_2)(\gamma q_1+q_2) & a=a_2\\ 0 & a=a_3.\end{cases}\end{aligned}$$

From equation (8.19),

$$\boldsymbol{\tau}(o_1,a_1,\mathbf{b}) = \left(\frac{\alpha_1(1-\gamma)q_1}{\alpha_1(1-\gamma)q_1+\alpha_2(\gamma q_1+q_2)}, \frac{\alpha_2(\gamma q_1+q_2)}{\alpha_1(1-\gamma)q_1+\alpha_2(\gamma q_1+q_2)}\right) \tag{8.105}$$

$$\boldsymbol{\tau}(o_1,a_2,\mathbf{b}) = \left(\frac{\beta_1(1-\gamma)q_1}{\beta_1(1-\gamma)q_1+\beta_2(\gamma q_1+q_2)}, \frac{\beta_2(\gamma q_1+q_2)}{\beta_1(1-\gamma)q_1+\beta_2(\gamma q_1+q_2)}\right) \tag{8.106}$$

$$\boldsymbol{\tau}(o_1,a_3,\mathbf{b}) = (1,0) \tag{8.107}$$

$$\boldsymbol{\tau}(o_2,a_1,\mathbf{b}) = \left(\frac{(1-\alpha_1)(1-\gamma)q_1}{(1-\alpha_1)(1-\gamma)q_1+(1-\alpha_2)(\gamma q_1+q_2)}, \frac{(1-\alpha_2)(\gamma q_1+q_2)}{(1-\alpha_1)(1-\gamma)q_1+(1-\alpha_2)(\gamma q_1+q_2)}\right) \tag{8.108}$$

$$\boldsymbol{\tau}(o_2,a_2,\mathbf{b}) = \left(\frac{(1-\beta_1)(1-\gamma)q_1}{(1-\beta_1)(1-\gamma)q_1+(1-\beta_2)(\gamma q_1+q_2)}, \frac{(1-\beta_2)(\gamma q_1+q_2)}{(1-\beta_1)(1-\gamma)q_1+(1-\beta_2)(\gamma q_1+q_2)}\right) \tag{8.109}$$

Note that $\boldsymbol{\tau}(o_2,a_3,\mathbf{b})$ is not well-defined, since $\eta(o_2|\mathbf{b},a_3)=0$. Under a_3, with probability 1 the system transitions to s_1 and observation o_1 is generated. This is an example where there are fewer updated belief states than there are observations, for a given current belief state and action.

Finally,

$$\tilde{p}(\mathbf{b}'|\mathbf{b},a) = \begin{cases}\eta(o_1|\mathbf{b},a_1) & \mathbf{b}'=\boldsymbol{\tau}(o_1,a_1,\mathbf{b})\\ \eta(o_1|\mathbf{b},a_2) & \mathbf{b}'=\boldsymbol{\tau}(o_1,a_2,\mathbf{b})\\ \eta(o_1|\mathbf{b},a_3) & \mathbf{b}'=\boldsymbol{\tau}(o_1,a_3,\mathbf{b})\\ \eta(o_2|\mathbf{b},a_1) & \mathbf{b}'=\boldsymbol{\tau}(o_2,a_1,\mathbf{b})\\ \eta(o_2|\mathbf{b},a_2) & \mathbf{b}'=\boldsymbol{\tau}(o_2,a_2,\mathbf{b})\\ 0 & \text{otherwise.}\end{cases}$$

8.6.2 Models with unknown parameters: Bayesian decision problems

Models with unknown parameters represent a particularly useful class of POMDPs. In such models the decision maker knows the form of the distribution of a random variable that affects rewards and transition probabilities up to the unknown parameter values. For example, in a production process the defect rate may be binomial with unknown defect probability. The decision maker acquires information about the parameter through a sequence of actions that affect both the information received and the reward earned. Formulations of such models are illustrated through examples below. The key feature is that the state represents the unknown parameters and remains unchanged over time.

Newsvendor with unknown demand

As an example, consider a variant of the newsvendor problem introduced in Section 3.1.2, in which the demand has a discrete distribution with unknown parameter μ. Denote its probability mass function by $f(d|\mu)$. Assume that μ can assume a finite number of values $\{\mu_1, \mu_2 \ldots, \mu_M\}$ and the newsvendor has a prior distribution on μ, denoted $w_1(\mu)$.

In such a model, inventory is perishable and lasts only one period. Each period, the decision maker purchases units of a product from a supplier at a cost c and sells them at price $g > c$ throughout the period. If any items remain unsold at the end of the period, they are scrapped at a reduced price $h < c$. Although the model with a known demand distribution is on a one-period model, this model has multiple decision epochs that account for updates on the knowledge regarding the value of the unknown parameter.

Decision epochs: The model may be either finite or infinite horizon.

$$T = \{1, 2, \ldots, N\}, \quad N \leq \infty.$$

Hidden states: States represent possible values for the unknown parameter. Under the above assumption,

$$S = \{\mu_1, \mu_2 \ldots, \mu_M\}.$$

In greater generality, S could be an interval (or the positive real line).

Actions: Actions represent the number of units to order at each decision epoch,

$$A = \{1, \ldots, a_{\max}\},$$

where $a_{\max}$ is some large finite value representing the maximum amount that can be ordered in a period.

Observations: The newsvendor observes only the number of units sold. If external demand is less than the quantity ordered, the newsvendor observes demand, but when demand exceeds inventory, demand is *censored* by the order quantity. That is, the decision maker is only guaranteed to observe a lower bound on the true demand. Thus,

$$O = \{0, 1, \ldots, a_{\max}\}.$$

It is important to note that action choice determines the values of o that have positive probability. The observation distribution can be written as

$$u(o|\mu, a) = \begin{cases} f(o|\mu) & o < a, \mu \in S \\ \sum_{i=a}^{\infty} f(i|\mu) & o = a, \mu \in S \\ 0 & o > a, \end{cases}$$

where the state is written as μ to emphasize its meaning.

Rewards: The newsvendor receives rewards that depend on the values of a and μ. The following expression for the *expected* reward accounts for the effect of the unknown parameter:

$$r(\mu, a) = \sum_{d=0}^{a-1} (Gd - L(a-d)) f(d|\mu) + Ga \sum_{d=a}^{\infty} f(d|\mu). \tag{8.110}$$

When the order quantity a exceeds demand d, the first expression on the right hand side of (8.110) combines the gain from sales at a profit $G = g - c$ with the loss of $L = h - c$ for unsold items when the order quantity a exceeds demand d. The second expression corresponds to the demand being greater than or equal to the order quantity. In this case, only the number of ordered units can be sold.

Transition probabilities: Since the parameter value remains constant over the planning horizon, for all $a \in A$

$$p(j|s, a) = \begin{cases} 1 & j = s \\ 0 & j \neq s. \end{cases}$$

Searching for an object

A searcher seeks to find an object in a region divided into an M-cell grid. A prior distribution on the object's location initiates the search. At each decision epoch the searcher decides which cell to search. Search is not perfect: if the searched cell contains the object, the searcher finds the item with probability $p_f > 0$. Upon finding the object, the searcher receives a reward of R and terminates the search. Model variants include multiple within-cell search routines with different costs, search costs that depend on the cell, moving objects, and a maximum search budget.

The POMDP formulation below describes particular model features. It includes some of the same features as the optimal stopping problem in Section 3.8 and may be classified as an *episodic* model.

Decision epochs: Since the object is not found with certainty, the number of decision epochs to find it is unbounded. Therefore

$$T = \{1, 2, \ldots\}.$$

If search were perfect, at most M epochs would be required.

Hidden states: States represent the cell number that contains the object and a stopped state Δ corresponding to the search being completed after finding the object.

$$S = \{1, 2, \ldots, M, \Delta\}.$$

Note that this model combines unobservable states corresponding to the location of the hidden object and an observable state resulting from the object being found.

Actions: Actions represent the cell to search at each decision epoch. Once the object is found the decision maker may only choose the zero-cost action ϕ leaving it in the stopped state. So

$$A_s = \begin{cases} \{1, \ldots, M\} & s \neq \Delta \\ \{\phi\} & s = \Delta \end{cases}$$

Observations: Set $O = \{F, NF, \Delta_o\}$ where F indicates "object found", NF indicates "object not found" and Δ_o corresponds to being in the stopped state. The searcher observes F only when successfully searching the cell that contains the object. Observation probabilities follow. When the search is ongoing, that is when $s \neq \Delta$,

$$u(o|s, a) = \begin{cases} p_f & o = F,\ a = s \\ 1 - p_f & o = NF,\ a = s \\ 1 & o = NF,\ a \neq s \end{cases}$$

and when the search has stopped, $u(\Delta_o|\Delta, \phi) = 1$.

Rewards: It costs one time unit to search a cell. Rewards in the stopped state are distinguished from those in the non-stopped state. When $s \neq \Delta$

$$r(s, a, j) = \begin{cases} R & a = s,\ j = \Delta \\ -1 & a = s,\ j = s \\ -1 & a \neq s. \end{cases}$$

The first case corresponds to finding the object, which results in a transition to the stopped state Δ. The second case corresponds to searching the correct cell, but not finding it. The third case corresponds to searching an incorrect cell. When the search has stopped, $r(\Delta, \phi, \Delta) = 0$.

Transition probabilities: When $s \neq \Delta$

$$p(j|s,a) = \begin{cases} 1-p_f & a = s,\ j = s \\ p_f & a = s,\ j = \Delta \\ 0 & a = s,\ j \neq s \\ 1 & a \neq s,\ j = a \\ 0 & a \neq s,\ j \neq a \end{cases}$$

and when $s = \Delta$

$$p(j|s,a) = \begin{cases} 1 & a = \phi,\ j = \Delta \\ 0 & a = \phi,\ j \neq \Delta. \end{cases}$$

The reasoning behind these transition probabilities is as follows. The hidden state s represents the true but unknown location of the object. The system state only changes state when the searcher finds the object, in which case the system moves to the stopped state Δ. This occurs with probability p_f when the state containing the object is searched. No other transitions are possible corresponding to the 0 probabilities above. If the searcher searches a state that does not contain the object, that is $a \neq s$, the state does not change. Once in the stopped state the system remains there forever.

Minimally invasive surgery

An interesting example of an optimal search problem arises in minimally invasive (robotic) surgery. A surgeon seeks to maneuver instruments from an initial incision on the surface of the body to a target organ while avoiding damage to critical structures such as arteries along the way. Pre-operative images provide some indication of the location of these structures but their precise location might change during the surgery. The surgeon has two search options: quick cuts through overlying tissue or slowly peeling away layers of tissue to safely learn what lies behind the tissue.

In practice there is a bound on the time to reach the organ and search is limited to contiguous regions. That means that the choice of the location for the next action must be contiguous to the site of the previous action.

8.6.3 Multi-armed bandits

Bandit models have received a great deal of attention in the literature; they are simple to describe, widely applicable and have produced some elegant mathematical results. The expression "bandit" refers to a slot machine, a common feature in a gambling casino. In this context, a player selects a machine, inserts a coin or token, pulls its "arm", and receives an uncertain payoff. The reward received from playing a particular machine does not depend on or provide information about any of the other machines available to the player. Questions of interest are which machine to play, when to switch to other machines, and when to stop playing.

Bandit models abstract the key features of this process. At each decision epoch, the decision maker selects one of K bandits to play or equivalently "arms to pull". Bandit models may be classified on the basis of the reward generating mechanism as follows:

1. In a *classical statistical bandit* "pulling an arm" results in receiving a reward from a probability distribution with unknown parameters corresponding to that arm.

2. In a *restless Markov bandit*, "pulling an arm" results in receiving a reward that depends on the state of an unobservable Markov chain corresponding to that arm.

Models may include a cost for switching between arms.

Applications

Selecting an arm to pull involves a trade-off between *exploitation* and *exploration*. The decision maker may either exploit the arm that is thought to have the highest expected payoff or explore other arms (pull other arms) to find one with a higher expected payoff. Specific examples include:

1. **Optimal Foraging:** To maximize survival, an animal may either continue to forage in the current area or move to another area (patch) with the hope of acquiring more nutrition while expending less energy. The bandit model applies where an "arm" corresponds to which patch to explore. This application requires including energy expenditure associated with changing patches.

2. **Project Management:** Faced with K projects in progress, at each decision epoch, the decision maker decides which project to work on.

3. **Sequential Clinical Trials:** Randomized controlled clinical trials have become the gold standard for establishing the effectiveness of new drugs, therapies and medical procedures. In a multi-armed clinical trial, patients are randomly assigned to a control or to one of several treatments under study. The goal is to determine which treatment is most effective and which have a larger effect than placebo. Random assignment of patients to a treatment or control arm accounts for the effect of potentially confounding factors.

   When patients are assigned to a treatment or control sequentially[20], the challenge is to develop a mechanism to assign of a patient to a treatment "arm" based on previously acquired data. The use of a multi-armed bandit framework has been shown to efficiently determine the best treatment and as well assign the most patients to the best treatment during the trial.

[20] This contrasts with the standard approach of randomizing all participants at the start of the trial.

4. **A/B Testing:** Similarly to a controlled clinical trial, A/B testing refers to a controlled statistical experiment with two "treatments" referred to as Treatment A and Treatment B. While deeply rooted in the statistical literature, it has become a widely used approach for choosing between alternative web site designs and online marketing campaigns such as the form and content of banner ads on web pages. When used sequentially, this may be viewed as a two-armed bandit. In the case of online advertising, when a user clicks a website link, the decision maker (i.e., website) chooses whether to display ad design A or ad design B, seeking to determine which design generates the most clicks or the most revenue. Subsequently, new designs are introduced and compared to the previous best design. Multi-armed bandits apply when there are more than two designs to compare.

The next two sections formulate the above bandit models as POMDPs.

POMDP formulation of classical statistical bandit

In the classical statistical bandit model, pulling arm k, $k = 1, \ldots, K$, results in an observation drawn from a discrete probability distribution $w(\cdot|\mu_k)$ with unknown mean μ_k on a discrete set $O = \{o_1, o_2, \ldots, o_L\}$, where L may be finite or infinite. When $O = \{0, 1\}$ the model is referred to as a Bernoulli bandit and applies to settings when the outcome may be viewed as "success" or "failure" such as in A/B testing or clinical trials. Assume that for $k = 1, \ldots, K$, μ_k assumes values in the finite set $M = \{m_1, \ldots, m_J\}$. This means that the unknown mean for each arm may take on any value in M.

Continuous variants of this model have been widely studied but are outside of the scope of this book. In such models, O is a continuum with probability density functions replacing probability mass functions.

Decision epochs: The model may be either finite or infinite horizon

$$T = \{1, 2, \ldots, N\}, \quad N \leq \infty.$$

Hidden states: Hidden states represent the possible values for the *unknown* mean for each of the K arms. Thus

$$S = M \times M \times \cdots \times M = M^K = \{(s_1, \ldots, s_K) \mid s_k \in M, k = 1, \ldots, K\}.$$

Actions: Actions represent which arm to play

$$A = \{1, \ldots, K\}.$$

Observations: When choosing the k-th arm, the decision maker observes the value $o \in O$ with probability $w(o|s_k)$ so that

$$u(o|(s_1, \ldots, s_K), k) = w(o|s_k).$$

This specification uses the fact that since $s_k \in M$, it equals one of $m_1, m_2, ..., m_J$.

Reward: Assuming no cost for switching arms,

$$r((s_1, \ldots, s_K), k) = \sum_{o \in O} f(o) w(o|s_k),$$

where $f(\cdot)$ is a known real-valued function on O. Usually the reward and observation are the same so $f(o) = o$ in which case $r((s_1, \ldots, s_K), k) = s_k$.

Transition probabilities: Regardless of the action chosen, the state of the arm does not change, so that for each $a \in A$

$$p((s'_1, s'_2, \ldots, s'_K)|(s_1, s_2, \ldots, s_K), a) = \begin{cases} 1 & \text{for } s'_k = s_k,\ k = 1, 2, \ldots, K \\ 0 & \text{otherwise.} \end{cases}$$

Implementation of this model requires an initial belief state or prior distribution on M for each arm. When the decision maker plays arm k, only that distribution is updated. When the state space for each Markov chain is M-dimensional, the POMDP formulation of the model has M^K states, so the model might appear intractable. However, it has been shown that the model decomposes into K independent problems that can be analyzed independently.

A Bernoulli bandit example

This example shows how this formulation applies to a three-armed Bernoulli bandit and provides the equivalent Markov decision process formulation. In this example $K = 3$, $O = \{0, 1\}$ and $M = \{m_1, m_2\}$ with $m_2 > m_1$. The state of the system, (s_1, s_2, s_3) indicates possible values for the mean of each arm, that is $s_k = m_1$ or $s_k = m_2$ for each $k = 1, 2, 3$. The Bernoulli assumption means that if arm k is in state s_k,

$$1 - w(0|s_k) = w(1|s_k) = s_k.$$

This means that s_k is the *probability* of observing a success, equivalently a 1, if arm k is selected. Hence, the observation probabilities are given by

$$u(0|(s_1, s_2, s_3), k) = 1 - s_k \quad \text{and} \quad u(1|(s_1, s_2, s_3), k) = s_k$$

Assume further that $f(o) = o$ so that the decision maker receives reward 1 if the arm pull generates a success and 0 if it generates a failure. Thus in this example rewards and observations are not distinguished.

The equivalent MDP provides additional insight into this example and the interpretation of quantities in this transformation. Let q_k denote the decision maker's assessment that $\mu_k = m_1$ so that $1 - q_k$ denotes the assessment that $\mu_k = m_2$. There are eight possible values for (s_1, s_2, s_3) so that a belief state provides the joint probability of each. For example, belief states are of the form, assuming independent arms:

$$b_1(m_2, m_2, m_1) = P\big[X_1 = (m_2, m_2, m_1)\big] = (1 - q_1)(1 - q_2)q_3.$$

Choosing action k results[21] in an (expected) reward

$$\tilde{r}(\mathbf{b}, k) = m_1 q_k + m_2(1 - q_k)$$

and observation probabilities

$$\eta(0|\mathbf{b}, k) = (1 - m_1)q_k + (1 - m_2)(1 - q_k) \quad \text{and} \quad \eta(1|\mathbf{b}, k) = m_1 q_k + m_2(1 - q_k).$$

Since the underlying state does not change, choosing action k only impacts q_k. Let $q'_k(o)$ denote the posterior probability of the state of arm k after observing signal $o = 0$ or $o = 1$. When $o = 0$,

$$\begin{aligned} q'_k(0) &= P[\mu_k = m_1 | o = 0] \\ &= \frac{P[o = 0|\mu_k = m_1]P[\mu_k = m_1]}{P[o = 0|\mu_k = m_1]P[\mu_k = m_1] + P[o = 0|\mu_k = m_2]P[\mu_k = m_2]} \\ &= \frac{(1 - m_1)q_k}{(1 - m_1)q_k + (1 - m_2)(1 - q_k)} \end{aligned}$$

and when $o = 1$,

$$q'_k(1) = \frac{m_1 q_k}{m_1 q_k + m_2(1 - q_k)}.$$

Then $\boldsymbol{\tau}(0, k, \mathbf{b})$ and $\boldsymbol{\tau}(1, k, \mathbf{b})$ are each 8-dimensional vectors of joint probabilities with $q'_k(0)$ and $q'_k(1)$ replacing q_k in the joint probability distributions.

To gain more insight into this formulation, assume $m_1 = 0.5$ and $m_2 = 0.6$ and suppose the prior probability that $s_1 = m_1$ is $q_1 = 0.7$, the prior probability that $s_2 = m_1$ is $q_2 = 0.5$ and the prior probability that $s_3 = m_1$ is $q_3 = 0.2$. Then the components of $\mathbf{b}_1$ are given by

$$b_1(s_1, s_2, s_3) = \begin{cases} 0.7 \cdot 0.5 \cdot 0.2 & \text{if } (s_1, s_2, s_3) = (m_1, m_1, m_1) \\ 0.7 \cdot 0.5 \cdot 0.8 & \text{if } (s_1, s_2, s_3) = (m_1, m_1, m_2) \\ \quad \cdots & \\ 0.3 \cdot 0.5 \cdot 0.8 & \text{if } (s_1, s_2, s_3) = (m_2, m_2, m_2). \end{cases}$$

[21] This calculation is actually more complex and must account for all eight possible values of (s_1, s_2, s_3).

Suppose arm 3 is selected and a 1 is observed. then

$$q_3'(1) = \frac{0.5 \cdot 0.2}{0.5 \cdot 0.2 + 0.6 \cdot 0.8} = 0.172.$$

In this case, this means that the probability that $s_3 = m_2$ has increased from 0.8 to 0.828 and in $\mathbf{b}_2$, 0.172 replaces 0.2 and 0.828 replaces 0.8 in the above expressions for the components of $\mathbf{b}_1$.

POMDP formulation of the restless Markov bandit

The unobservable state of arm k, $k = 1, \ldots, K$ changes according to a Markov chain with state space S_k and transition probability $p_k(j|s)$. Assume the following timing of events. After selecting arm k the decision maker receives a reward $r_k(s)$ and an observation o with probability $u_k(o|s)$ if the hidden state of arm k equals s. Subsequently the arm changes state to state j with probability $p_k(j|s)$.

Note that to ensure the decision maker receives no information from the reward, it can be assumed that each $r_k(\cdot)$ has the same range for each k, that is $r_k(\cdot) = r(\cdot)$ for $k = 1, \ldots, K$. Note that this model may be viewed as choosing between K independent unobservable Markov reward processes.

Decision epochs: The model may be either finite or infinite horizon

$$T = \{1, 2, \ldots, N\}, \quad N \leq \infty.$$

Hidden states:

$$S = S_1 \times S_2 \ldots \times S_K = \{(s_1, \ldots, s_K) \mid s_k \in S_k, k = 1, \ldots, K\}.$$

Actions: Actions represent which arm to play

$$A = \{1, \ldots, K\}.$$

Observations: When the decision maker chooses action $k \in A$ and Markov chain k is in state s_k the decision maker observes $o \in O$ with probability $u_k(o|s)$. Therefore

$$u(o|(s_1, \ldots, s_K), k) = u_k(o|s_k).$$

This means that the observation depends on the state of arm k.

Rewards: When the decision maker chooses action $k \in A$ and Markov chain k is in state s, the decision maker receives reward

$$r((s_1, \ldots, s_K), k) = r_k(s_k).$$

Transition probabilities: For $k \in A$,

$$p((s'_1, \ldots, s'_K)|(s_1, \ldots, s_K), k) = \begin{cases} p_k(s'_k|s_k) & \text{for } s'_i = s_i, i \neq k \\ 0 & \text{otherwise} \end{cases}$$

8.6.4 Breast cancer screening

The American Cancer Society [2024] estimates that approximately one in eight women will be diagnosed with breast cancer in their lifetime. Using mammography to screen for breast cancer has been shown to reduce breast cancer mortality through early detection. Fundamental issues are at what age to start screening and how frequently to screen. Since women have different risk factors for breast cancer, screening policies may differ between individuals of the same age. The particular model developed below is an example of one in which some of the transition probabilities depend on the observation.

Consider the situation where a woman's true health state, that is, whether or not she has breast cancer and possibly what type, is unknown. Mammography provides a noisy preliminary indication of breast cancer. The model recommends periodically whether a woman should or should not have a mammogram. If the "no mammogram" option is chosen, the patient is encouraged to use self-examination as a rough indicator of the presence of cancer.

Action choice involves trade-offs. The downsides of too frequent mammograms are that they are painful, expose a patient to radiation, and false positives may heighten the patient's anxiety and tax health system resources. On the other hand, not having them frequently enough may miss the presence of disease. The goal is to develop a personalized screening policy so that diagnosis and subsequent timely treatment occurs as early as possible. The model is limited to screening. It assumes that after a positive mammogram the patient undergoes a biopsy, which is a more invasive and a highly reliable test. A positive biopsy results in the start of treatment; a negative biopsy returns the patient to their screening regimen. The model can be generalized to include follow-up decisions and treatment policies.

Consider the following sequence of events.

1. Immediately prior to the current decision epoch, the decision maker updates the belief state.

2. The decision maker chooses either mammography or self-examination in the current period.

3. The result of the mammogram, if performed, is found to be positive or negative.

4. A positive mammogram results in a biopsy. If the biopsy is also positive, treatment starts prior to the next period. A negative biopsy indicates the patient does not have breast cancer.

5. If the decision maker chose self-examination, the result is revealed prior to the next decision epoch.

6. The health state (cancer free, cancer present, or death) is updated.

Decision epochs: Decisions are made relative to a woman's age. Assume screening starts at age T_0 and decisions are made periodically up to age T_F[22]. Hence

$$T = \{T_0, T_1, \ldots, T_F\}.$$

Hidden states: States represent a woman's health condition at each decision epoch. The set of states is given by

$$S = \{CF, BC, TR, DE\}$$

where CF represents cancer free, BC represents breast cancer present, TR represents a patient under treatment and DE represents death. Assuming treatment begins once cancer is diagnosed, states TR and DE are *observable* and absorbing, while CF and BC are *hidden*. Thus this model combines both observable and unobservable states. Cancer states can be further distinguished by the stage of breast cancer, or whether the cancer is localized versus invasive, if needed.

Actions: In hidden states CF and BC, the decision maker chooses from MA (mammography) and SE (self-examination). In observable states TR and DE, the decision maker may only choose an artificial action ϕ, which leaves the process in the same state. Therefore:

$$A_s = \begin{cases} \{SE, MA\} & \text{for } s \in \{CF, BC\} \\ \{\phi\} & \text{for } s \in \{TR, DE\} \end{cases}$$

Observations: Set $O = \{+, -, \Delta\}$ where $+$ denotes a positive mammogram or self-exam, $-$ denotes a negative test and Δ denotes the screening process has terminated.

[22] Epochs could be every six months, or more frequently for self-examination even if the mammogram option is not available. The final epoch T_F should be chosen to be sufficiently high, e.g., 100. The reality that a woman may die before age 100 is accounted for by adding an absorbing "death" state to the hidden state space.

The probabilities of these observations are given for $t = T_0, T_1, \ldots, T_F$ by

$$u_t(o|s,a) = \begin{cases} \alpha_{t,SE} & \text{for } o = -, s = CF, a = SE \\ 1 - \alpha_{t,SE} & \text{for } o = +, s = CF, a = SE \\ 1 - \beta_{t,SE} & \text{for } o = -, s = BC, a = SE \\ \beta_{t,SE} & \text{for } o = +, s = BC, a = SE \\ \alpha_{t,MA} & \text{for } o = -, s = CF, a = MA \\ 1 - \alpha_{t,MA} & \text{for } o = +, s = CF, a = MA \\ \beta_{t,MA} & \text{for } o = +, s = BC, a = MA \\ 1 - \beta_{t,MA} & \text{for } o = -, s = BC, a = MA \\ 1 & \text{for } o = \Delta, s \in \{TR, DE\}, a = \phi \\ 0 & \text{otherwise.} \end{cases}$$

Observation probabilities are based on the medical testing concepts of *specificity* and *sensitivity*. In this case, the specificity of a test is the probability of a negative test $(\alpha_{t,SE}, \alpha_{t,MA})$ in a cancer-free woman while the sensitivity is the probability of a positive test $(\beta_{t,SE}, \beta_{t,MA})$ in a woman with cancer. These probabilities are obtainable from the medical literature. Moreover, sensitivity and specificity have been shown to be age related so that the observation probabilities vary with decision epoch. In general, for mammography to be a useful screening test, its sensitivity and specificity must exceed that of self-examination.

Rewards: The units for rewards are *Quality Adjusted Life Years* (QALYs). Note that the reward depends on the observation. Then

$$r_t(s,a,o) = \begin{cases} L_t & \text{for } s \in \{CF, BC\}, a = SE, o \in \{+, -\} \\ L_t - k & \text{for } s \in \{CF, BC\}, a = MA, o = - \\ L_t - K & \text{for } s = CF, a = MA, o = + \\ q_t & \text{for } s = BC, a = MA, o = + \\ 0 & \text{for } s = \{TR, DE\}, a = \phi, 0 = \Delta \end{cases}$$

with a terminal reward $r_{T_F}(s) = q_{T_F}(s)$ for $s \in S$. In the above, L_t denotes the expected QALYs in the period following decision epoch t, k the disutility of having a mammogram, K the disutility of a false positive mammogram, and q_t the estimated QALYs for an age t woman beginning cancer treatment (which implicitly includes the disutility of a positive mammogram).

Transition probabilities: For $t = T_0, T_1, \ldots, T_F$,

$$p_t(s'|s,a,o) = \begin{cases} (1-\lambda_t)\delta_t & \text{for } s' = BC, s = CF, a = SE, o \in \{+,-\} \\ (1-\lambda_t)(1-\delta_t) & \text{for } s' = CF, s = CF, a = SE, o \in \{+,-\} \\ 1 & \text{for } s' = TR, s = BC, a = MA, o = + \\ (1-\lambda_t) & \text{for } s' = BC, s = BC, a = MA, o = - \\ (1-\lambda_t)\delta_t & \text{for } s' = BC, s = CF, a = MA, o = + \\ (1-\lambda_t)\delta_t & \text{for } s' = CF, s = CF, a = SE, o = + \\ \lambda_t & \text{for } s' = DE, s \in \{CF, BC\}, a = SE, o \in \{+,-\} \\ 1 & \text{for } s' = s, s = \{TR, DE\}, a = \phi, o = \Delta \\ 0 & \text{otherwise} \end{cases}$$

where λ_t denotes the probability a woman dies in the period following decision epoch t and δ_t denotes the probability an age t cancer-free woman develops cancer in the period following decision epoch t. A transition from BC to TR can only occur if a woman with cancer has a positive mammogram.

To apply this model requires obtaining patient-specific data from the medical literature, modifying belief state updating and optimality equations to account for rewards and transition probabilities that depend on the observation, and developing a computational algorithm. A policy will partition the first two components of the belief state (probability of being cancer free and probability of breast cancer) into regions where the action choice is mammography versus self examination. Presumably, the model will recommend a mammogram when the risk of cancer is high and self-examination when the probability is low.

8.6.5 Controlling autonomous robots with noisy sensors

An autonomous controlled robot (ACR) possesses the following components:

1. **Sensors** to detect features of the robot's surroundings. Sensors may use light (cameras or lasers) or sound (sonar) to acquire information about the local environment.

2. **Controllers** to process the sensor information and determine appropriate actions.

3. **Actuators** to execute the action specified by the controller.

In addition, the robot may incorporate a low-level object avoidance system that works independently of the controller.

Robotic navigation competitions have demonstrated that ACRs are prone to *localization errors*. Robots often lose track of their position, hindering their ability to reach

their targets. This observation underscores the need for intelligent control algorithms that avoid localization errors and enable the robot to reach its target.

POMDP models provide a framework for robotic control. The following stylized example is intended to illustrate some of the modeling issues. In this model, the ACR strives to complete a task of moving through a series of hallways or a maze to reach a target location. Assume the robot has a stored map of its environment and knows the location of the target. Randomness enters through imprecise movements and noisy sensor readings.

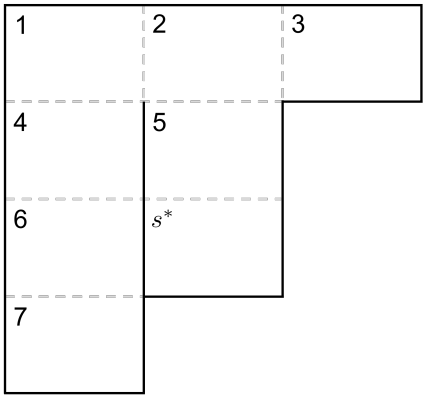


Figure 8.10: Sample grid for robot navigation POMDP model. Thick lines indicate walls.

The following combines features of many models in the literature. The environment (see Figure 8.10 for an example) is divided into discrete cells, which may be enclosed by up to three walls. This might be regarded as movement through the corridors of an office environment or through a maze. Given a map of the area to be navigated, the ACR can either move or activate its sensors. An onboard compass orients the robot. Since the robot does not know its precise location it uses sensor readings to update its belief distribution.

Decision epochs: Decision epochs correspond to the instance right before action choice. The number of periods to achieve the target is not known a priori. Good policies should achieve the target in finitely many periods, but cycling is possible so an infinite horizon is required to formulate the model:

$$T = \{1, 2, \ldots\}.$$

Hidden states: In an environment with K states plus a target state s^*,

$$S = \{1, \ldots, K, s^*\},$$

where the state denotes the robot's actual location.

Actions: Actions model the robot's movement and sensing capabilities. The action set is given by:

$$A = \{N, S, E, W, SNS, SEW\}$$

where N, S, E, W represent an attempt to move one cell in the north, south, east or west directions. It is important to emphasize that these actions represent the robot's *intended* movement. Transition probabilities account for the impact of the environment on the robots intended move.

Actions SNS and SEW refer to the robot performing a sensor sweep to detect walls immediately adjacent to its position in the north-south or east-west directions, respectively. When executing these actions, the robot does not move.

These actions represent both active and passive information acquisition.

Observations: When actions N, S, E and W are chosen, the robot gains no additional information about its state. Let ϕ denote this "none-observation" corresponding to choosing these actions. Also assume the robot knows when it reaches the target state, denoting this information by γ.

When either sensing action is chosen, the robot's sensors provide noisy information about the number of walls (0, 1 or 2) in the directions searched. Thus

$$O = \{0, 1, 2, \phi, \gamma\}.$$

Denote by $\omega(o|i)$ the probability the robot interprets the sensor information as there being o walls when in fact there are i walls. For example, when there are two walls in the directions searched, $\omega(o|2)$ may be given by

$$\omega(o|2) = \begin{cases} 0.05 & o = 0, \\ 0.15 & o = 1, \\ 0.8 & o = 2. \end{cases}$$

For sensors to be effective, one would expect that $\omega(i|i)$ for $i = 0, 1, 2$ are the largest probabilities. Thus,

$$u(o|s, a) = \begin{cases} 1 & s \neq s^*, a \in \{N, S, E, W\} \text{ and } o = \phi \\ 1 & s = s^* \text{ and } o = \gamma \\ \omega(o|W_s) & s \neq s^*, a \in \{SNS, SEW\}, W_s \in \{0, 1, 2\}, o \in \{0, 1, 2\}, \end{cases}$$

where W_s denotes the (unknown) number of walls in the specified directions in cell s.

Rewards: The following reward functions captures most of the intended features of the model:

$$r(s, a, j) = \begin{cases} R & s \neq s^*, j = s^* \\ -c & s \neq s^*, j \neq s^* \\ 0 & s = s^*, j = s^*, \end{cases}$$

where R denotes the reward for reaching the target state and c denotes the cost per action taken. Alternatively there could be different costs associated with sensing and movement actions. Assuming the process terminates when the target state is reached, the robot remains there and receives no subsequent reward.

Transition probabilities: Transition probabilities depend on the precise orientation and configuration of cells in the grid. When sensing, the robot does not move so that

$$p(j|s,a) = \begin{cases} 1 & j = s, a \in \{SNS, SEW\} \\ 0 & j \neq s, a \in \{SNS, SEW\} \end{cases}$$

Assume that the robot's movement is subject to randomness. When it tries to move in a direction it succeeds with probability 0.7 and moves in each other direction with probability 0.1. If it bumps into a wall it does not move. For the environment in Figure 8.10, some selected probabilities are given below:

$$p(j|6,N) = \begin{cases} 0.7 & j = 4 \\ 0.1 & j = 7 \\ 0.2 & j = 6 \end{cases}, \quad p(j|2,S) = \begin{cases} 0.7 & j = 5 \\ 0.1 & j \in \{1,2,3\} \end{cases}, \quad p(j|4,E) = \begin{cases} 0.8 & j = 4 \\ 0.1 & j \in \{1,6\} \end{cases}$$

For all $a \in A$

$$p(j|s^*,a) = \begin{cases} 1 & j = s^* \\ 0 & j \neq s^* \end{cases}$$

The derived MDP of this POMDP can be formulated with actions that are actions in the underlying MDP. That is, assume that every action is possible in every state, $\tilde{A} = A$ for all $s \in S$, since infeasible actions (asking the robot to walk through a wall) lead the robot to stay in the same cell. This logic is encoded in the transition probabilities as described above.

As an alternative, consider the situation where a robot is not able to detect an infeasible action before implementing it, and thus would attempt to walk through a wall if asked to do so, which is undesirable. In this case, the derived MDP needs to be formulated with actions that are decision rules in the underlying MDP, $\tilde{A} \subset \Pi_{s \in S} A_s$, with A_s including only feasible actions in hidden state s.

Belief updating

The following belief states updates using (8.14) distinguishes the effect of the two types of actions. Note that the results simplify because under a move action, the only signal is ϕ and under a sensor action, the system state is not altered.

Suppose a movement action is selected. Then applying (8.19) gives the updated belief state

$$b'(s') = P[X' = s'|Y = a, Z = \phi, \mathbf{b}] = \sum_{s \in S} p(s'|s,a) b(s),$$

where the expression simplifies because $u(\phi|s', a) = 1$ for $s' \neq s^*$ and $a \in \{N, S, E, W\}$.

For example, under the action N, cell 1 can only be reached from cells 1, 2 and 4 so that if $b(s) = 1/7$ for $s \neq s^*$,

$$b'(1) = p(1|1, N)b(1) + p(1|2, N)b(2) + p(1|4, N)b(4) = \frac{0.8 + 0.1 + 0.7}{7} = 0.229$$

Suppose a sensor action a is chosen and *oo* walls are observed, then

$$b'(s') = \frac{u(o|s', a)b(s')}{\sum_{s \in S} u(o|s, a)b(s)}$$

where the expression simplifies because choosing a sensor action does not result in robot movement, that is $p(j|s, a) = 1$ for $j = s$ and 0 otherwise.

For example suppose action SNS is chosen and 1 wall is observed and $b(s)$ is as above. Then assuming $\omega(1|0) = 0.1, \omega(1|1) = 0.8$ and $\omega(1|2) = 0.15$, and noting that $b(s)$ is constant,

$$\begin{aligned} b'(1) =& \frac{u(1|1, SNS)b(1)}{\sum_{s=1}^{7} u(1|s, SNS)b(s)} = \frac{u(1|1, SNS)}{\sum_{s=1}^{7} u(1|s, SNS)} \\ =& \frac{0.8}{0.8 + 0.8 + 0.15 + 0.1 + 0.1 + 0.1 + 0.8} = 0.280. \end{aligned}$$

Note that the probability of receiving this signal is 0.407. Thus in this sample calculation, sensing in the north-south direction increases the probability of being in cell 1 more than trying to move north.

Note that more complex (and perhaps realistic) models combine sensor readings and movement in single actions.

Bibliographic remarks

Our development in this chapter is in the spirit of Krishnamurthy [2016] and draws heavily on the classical operations research literature. The finite horizon and infinite horizon models were presented and analyzed in Smallwood and Sondik [1973] and Sondik [1978]. The survey by Monahan [1982] includes the algorithm that we present in this chapter for solving the finite horizon model. The linear support model by Cheng [1988] builds up the set Γ instead of pruning in Monahan's algorithm and is the basis for several subsequent algorithms. The approximation method presented in this chapter is due to Lovejoy [1991a]. The subsequent survey by Lovejoy [1991b] provides a clear exposition of this and several algorithms for solving POMDPs. Subsequent advances in approximate algorithms include the witness algorithm of Cassandra et al. [1997] and the point-based value iteration method described in Pineau et al. [2003] and Spaan and Vlassis [2005].

The preventive maintenance and inspection example is adapted from Monahan [1982]. The searching for an object example comes from Shechter et al. [2015] and was

motivated by a challenge in minimally invasive surgery. Kohavi and Thomke [2017] describes the impact that A/B testing has had on major technology companies. An optimal policy for the multi-armed bandit problem is characterized by the well-known Gittins Index [Gittins and Jones, 1974], which allows the problem to be solved by analyzing each arm separately. The Markov chain bandit model follows Krishnamurthy [2016]. The restless bandit model was proposed by Whittle [1988]. The model for breast cancer screening was developed by Ayer et al. [2012]. The robotic control example is based on the discussion in Simmons and Koenig [1995].

The website Cassandra [2025] contains an insightful verbal tutorial, links to some key references and code for some popular algorithms. We strongly encourage you to consult it.

Exercises

1. Consider the two-state model and calculations in Section 8.1.2.
   (a) Calculate the expected total reward when $q > 0.5$ with terminal reward 0 and when $x = 3$ and $y = 4$.
   (b) Let q^* denote the point at which the value of the decision rules (8.2) and (8.3) with terminal reward 0 are equal. Plot $v_d(q)$ as a function of q for $q \in [0, 1]$ using the decision rule in (8.2) for $q \leq q^*$ and the decision rule in (8.3) for $q > q^*$. What do you observe about the shape of $v_d(q)$?
   (c) Compute $P[X_2 = s_1|Z_2 = o_1, Y_1 = g(X_1), X_1]$ and $P[X_2 = s_1|Z_2 = o_2, Y_1 = f(X_1), X_1]$ using the approach in Section 8.1.2.
2. Suppose in the two-state model that the policy can choose between **any** of the four deterministic Markovian decision rules in the underlying Markov decision process.
   (a) Generate the analog of Figure 8.3 for several choices of x and y.
   (b) Specify the optimal policy as a function of q for each choice of x and y.
   (c) Compute the reward and transition probabilities for the two remaining actions in $\tilde{A}$ in a similar manner to (8.25) and (8.30).
   (d) Compute and interpret transition probabilities and rewards when $q_1 = 0.4$.
3. **The tiger problem** [Cassandra et al., 1997]. You are faced with the opportunity of opening one of two doors, one to the left and one to the right. Behind one door is a tiger and behind the other door is a valuable prize. Unfortunately, you do not know which object is behind which door. At each instance, you can open the left door, open the right door or listen. If you decide to listen, you will hear the tiger behind the correct door with probability 0.85 and behind the incorrect door with probability 0.15.

Assume listening costs 1 unit, opening the door with the tiger results in a penalty of 100 units and opening the door with the prize results in a reward of 10 units. The objective is to determine how long to listen before deciding to open a door. Clearly this depends on your belief regarding which door the tiger is behind.

(a) Formulate this as a POMDP and provide the equivalent MDP.

(b) Solve it for $N = 5, 10$ with Algorithm 8.2. Plot the optimal value function and represent the optimal policy.

(c) Use Algorithms 8.3 and 8.4 to find bounds and the lower bound approximate policy for a range of belief states, some that are in H and some that are not, for $N = 5, 10, 20$. Compare the quality of the approximations and the lower bound approximate policy to the optimal policy.

(d) Reformulate the model with three doors, two prizes and one tiger. How does this change analysis?

4. **Machine maintenance** Smallwood and Sondik, 1973. A machine has two identical components, both of which must operate on a product before it is completed. Refer to the time required to produce an item as a *production cycle*. The components fail, that is become inoperative, independently. The probability each component fails during a production cycle is 0.1. Assume for simplicity that if components fail, they do so at the start of the production cycle.

   The number of operative components affects the quality of the product as follows. If both components are operative, the product will be non-defective. If a single component is inoperative, the product will be defective with probability 0.5 and if two components are inoperative, the probability the product is defective is 0.75. Assume that a non-defective product generates 1 unit of revenue and a defective product generates no revenue.

   Assume that four control options are available at the start of each production cycle:

   - Continue as is: Do not inspect the product.
   - Inspect output: Inspect the finished product to determine if it is defective at a cost of 0.25 units. Assume this action can be completed in the current production cycle.
   - Condition-based maintenance: Inspect and repair components if necessary. The cost of replacing a failed component is 1 unit in addition to a cost of 0.5 units for inspection. This requires an entire production cycle so a product is not produced.
   - Preventive maintenance: Replace both internal components without inspection. This costs 2 units but foregoes the inspection cost. This also requires an entire production cycle so a product is not produced.

The objective is to maximize expected total revenue over a finite horizon.

(a) Draw a timeline of events within a period.

(b) Formulate this model as a POMDP.

(c) Express the model entities in matrix form.

(d) Solve the problem using Algorithm 8.2 for $N = 3, 4, 7, 11$. (These are the problem length in the paper.) Assume a terminal cost of 0 if both components are operative, 1 if a single component is operative, and 2 if both components are inoperative at termination.

(e) Represent the optimal policy by partitioning $\tilde{S}$ into regions where each action is optimal.

(f) Use Algorithms 8.3 and 8.4 to find bounds and the lower bound approximate policy for a range of belief states some that are in H and some that are not, for $N = 3, 4, 7, 11$.

5. Consider the calculations in the examples in Sections 8.4.4 and 8.4.5.

(a) Apply Algorithm 8.2 to find optimal policies and optimal value functions in models with $N = 4$ and $N = 5$.

(b) Apply Algorithms 8.3 and 8.4 to find the lower bound approximate policy and bounds for models with $N = 4$ and $N = 5$.

(c) Compare the policies and assess the quality of the bounds.

(d) Suppose the exact solution for $N = 5$ is regarded as an approximation to the infinite horizon version of the problem. Apply the bounds in (8.103) to assess the quality of the approximation.

6. Verify that the quantities in the matrix formulation of the derived MDP (8.21) -(8.24) are equivalent to the corresponding quantities in the component formulation of the derived MDP.

7. **Some formulation exercises:**

(a) Formulate a POMDP model of the classical statistical bandit model in which there is a fixed cost C associated with changing arms.

(b) Formulate the derived MDP for the newsvendor problem from Section 8.6.2 when $M = 3$.

(c) Formulate the derived MDP for the search problem from Section 8.6.2. This is challenging because of the presence of the stopped state.

(d) Reformulate the search problem when the searcher finds the object with certainty.

(e) Reformulate the search model when the object can change cells between decision epochs. Assume the object follows a random walk where p_L denotes the probability the object moves from cell i to cell $i-1$ and p_H denotes the probability the object moves from cell i to cell $i+1$.

(f) Formulate the two derived MDPs of the robot control problem described at the end of Section 8.6.5.

8. Prove that a function written in the form (8.45) is convex and establish the five properties of piecewise linear convex functions listed immediately below (8.45).

9. Derive all probabilities for the robot control model in Section 8.6.5.

10. Formulate the following model as a POMDP. Consider the case of a male patient who may have prostate cancer. In this setting, observation becomes "active surveillance", inspection corresponds to having a PSA test, and advance testing may correspond to a biopsy or imaging. This setting becomes more complex in that as a result of a positive biopsy, a patient may decide to receive hormone therapy, have surgery or continue active surveillance. Moreover more than two states may be necessary to represent different disease stages.

Part III - Reinforcement Learning

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

With four parameters I can fit an elephant, and with five I can make him wiggle his trunk[1].

John von Neumann, Hungarian-American mathematician, physicist, computer scientist, game theorist and more, 1903-1957.

This part of the book contains three chapters:

- Chapter 9: Value function approximation
- Chapter 10: Simulation in tabular models
- Chapter 11: Simulation with function approximation

The material in Chapter 9 is often referred to as *approximate dynamic programming (ADP)* and that in Chapters 10 and 11 as *reinforcement learning*.

While von Neumann's quote was relevant for statistical models in the 1950s, things have changed. Many modern reinforcement learning methods, particularly those based on neural networks, use models with millions of parameters and still generalize well to new settings. This progress comes from advances in neural network techniques, access to massive datasets, and powerful optimization algorithms. As a result, the link between model complexity and performance is far more subtle than in von Neumann's time.

Reinforcement learning can be viewed from two perspectives:

- The *artificial intelligence (AI)* perspective is that reinforcement learning methods seek to develop an "agent which can interpret any environment, and learn a task to superhuman ability, all with minimal user interaction[2]."

[1] Dyson [2004].
[2] McKenzie and McDonnell [2022]

- The *operations research (OR)* perspective is that in a model-based environment in which rewards and transition probabilities are known, reinforcement learning provides a toolbox of methods that can be used to find effective policies.

Although these appear to be distinct, the boundaries are far from clear cut in practice. Table 8.2 sheds light on how these perspectives have evolved in practice.

Aspect	OR perspective	AI perspective
Horizon	Infinite or finite and fixed	Finite and random
Model	Specified	Not specified
Rewards	At each decision epoch	At termination
Possible actions	Pre-specified	Pre-specified
Transition probabilities	Known	Not known
Algorithms	Offline	Online

Table 8.2: Distinguishing features of the OR and AI perspectives on reinforcement learning. These are not intended to be exhaustive or mutually exclusive, the boundary between them is often blurred.

Overview

Chapters 4 - 8 provided computational algorithms (e.g., value iteration, policy iteration, linear programming) for finding optimal policies in a fully or partially observable Markov decision process that can be stored on a computer. Since these methods are computationally prohibitive in models with extremely large (or continuous) state spaces, this section of the book and most current research develops and investigates methods to do so.

Motivated by the need to reduce the dimension of the state space, Chapter 9 provides generalizations of value iteration, policy iteration and linear programming for analyzing Markov decision processes in which the underlying value function is approximated by a low-dimensional linear or nonlinear function of features of the state space or state-action space. Such algorithms are applied directly to the Markov decision process model. No simulation is involved.

The penultimate chapter, Chapter 10, represents a major departure from previous chapters. In it, simulation (or real-time experimentation) replaces exact computation. It focuses on models that can be represented in *tabular* form. That is, models that are sufficiently small so that value functions and state-action value functions can be easily stored in memory. In such models, approximating value and state-action value functions is unnecessary. The reason for focusing on tabular models is to compare the performance of simulation to classical methods. Approaches include Monte Carlo methods, temporal differencing and Q-learning. This chapter also distinguishes *model-based* analyses, in which components of the underlying Markov decision process are used in algorithms, and *model-free* analyses, which are based only on the generated or

observed sequence of states, actions and rewards. The underlying mathematical tools are stochastic approximation and stochastic gradient descent.

The concluding chapter, Chapter 11, combines the concepts from the previous two chapters, namely value function or state-action value function approximation and simulation and introduces policy-based approaches as well. It is motivated by applications in which the state space (and/or action space) is so large (or continuous) that tabular representations and exact computation are impractical. The two distinct approaches are:

1. **Value-based methods** use the reward to update the stored state-action value function or an approximation to it. When rewards corresponding to an action are greater than expected, the state-action value function for the specified state-action pair is increased, making the chosen action in that state more attractive. Conversely, when the reward is less than expected, the state-action value function is decreased making the action less attractive. Thus, the rewards lead these algorithms to learn a good approximations to the "true" underlying state-action value function from which actions can be selected greedily in subsequent implementations.

2. **Policy-based methods** use the reward to directly update a randomized policy by gradient ascent. When an action chosen in a state produces a larger than expected reward, the probability of choosing that action in the state is increased. Conversely if the reward is less than expected, the probability of choosing that action is decreased. Repeating this process produces effective policies.

Objectives

This part of the book offers an accessible introduction to reinforcement learning concepts and algorithms, rather than a comprehensive (and soon outdated) overview. By mastering these fundamental methods, readers will establish a solid base from which to delve more deeply into rapidly evolving reinforcement learning research and applications.

So, how should one learn this material? Following the reinforcement learning philosophy, you must learn by doing. This means:

1. Carefully formulating your applications as Markov decision processes.

2. Developing your own code for the algorithms[3] in these chapters.

Often your codes will not replicate published results. Unlike the methods in Chapters 4 – 8, reinforcement learning methods offer few practical guarantees on convergence and have many parameters to tune. Trial-and-error is a natural part of the process, and a necessity for developing the insights needed to create robust and effective algorithms.

[3]This is not recommended for linear programming and regression, where excellent commercial codes are available.

Figure III.1: Image a Brio Labyrinth owned by one of the authors of this book.

An example: The Brio labyrinth

To provide a flavor of the type of problem that the reinforcement learning community seeks to solve, consider the challenge of optimally guiding a ball through the Brio Labyrinth shown in Figure III.1. Bi and D'Andrea [2023] use reinforcement learning to find and implement a good strategy for playing this game. Details and methods are beyond the scope of this book but hopefully will be understandable after reading these chapters.

In this game, a player places a metal ball at a location designated as the start at the top center of the maze. Using the two knobs on the sides of the labyrinth, the player dynamically tilts the plane of the surface so as to guide the ball to a destination at the right center without falling through any of the 39 numbered holes[4].

To "solve" this problem using reinforcement learning requires complex engineering, considerable ingenuity and the use of state-of-the-art algorithms. The researchers developed a robotic system that used computer-controlled motors attached to the rotors to adjust the tilt of the surface. A camera mounted above the surface provided real-time information about the ball position, surface angles and the geometry around the ball.

An MDP model

The system was modeled by a discrete-time undiscounted infinite horizon (episodic) MDP. States represent information about the ball, the surface, and the local geometry (position of walls and holes) around the ball. The ball position is given by its (x, y)-

[4]Holes may be numbered twice when they can interact with the ball at two different segments of its path.

coordinates. The surface is summarized in terms of the angle of tilt of the surface in the x and y directions. Rates of change of these quantities are estimated using sequences of images. The local geometry around the ball is captured by 6 cm $\times$ 6 cm images centered at the ball, taken at discrete time steps and encoded in a 64×64 RGB image. Actions represent the angular velocity of the two motors attached to the knobs controlling the surface tilt.

Camera images also provided a basis for a reward structure that measures progress (in cms) along the directional path represented by the black line from the origin to the destination in Figure III.1. Note in episodic models such as this, it is customary to only receive a reward when achieving a goal or incur a penalty when terminating in an undesirable state. Such a reward structure presents challenges for reinforcement learning methods so the introduction of these intermediate rewards facilitates learning.

Algorithms and results

Good policies are found using a model-based actor-critic algorithm (see Section 11.7.1) called DreamerV3 [Hafner et al., 2025]. Transition probabilities (obtained from "a world model"), policies, and values are approximated by neural networks. Underlying symmetries in the board structure improve algorithmic efficiency.

The system learned to "solve" the game in 5 hours (1 million time steps) using real-world experiences. Running the resulting policy over 50 replicates resulted in a success rate (reaching the target location) of 76% and a completion time of 15.73 ± 0.36 seconds. To put this in perspective, the human record is 15.95 seconds.

Chapter 9

Value Function Approximation

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

Far better an approximate answer to the right question, which is often vague, than an exact answer to the wrong question . . .[1]

John Tukey, American statistician, 1915-2000.

9.1 Introduction

In models with large state spaces or action sets or both, direct evaluation and storage of policy and optimal value functions or state-action value functions may be prohibitive. As Tukey suggests, approximations may be preferable in such instances. To address this issue, this chapter describes methods in which value functions are often approximated by lower-dimensional functions such as polynomials, splines or neural networks. Doing so presents several challenges that are discussed below. Approaches based on function approximations are often referred to as *approximate dynamic programming* (ADP).

As an example, consider the medical appointment scheduling model of Section 3.7, analyzed in Section 9.6 below. In it, the vector-valued state represents the number of booked appointments each day over a multi-day booking horizon and the number of requests for appointments of each type[2] arriving on a particular day. Actions represent the number of incoming appointment requests of each type to book in available appointment slots on each day in the booking horizon. The objective in this application

[1] Tukey (1962).

[2] In the application that motivated development of these models, type is an urgency level determined by a medical professional. Each urgency type has a different wait time target.

is to find a policy for assigning appointment slots to incoming appointment requests that uses resources efficiently but meets wait time targets.

In a realistic application of this model, the booking horizon $N = 30$, the number of appointment types $K = 3$, the daily capacity $C = 20$ and the maximum number of daily arrivals of each type $M = 10$. Subject to this specification the model will have $(21)^{30} \times (11)^3$ states and as well, a very large number of actions. Clearly in such a model, calculation and storage of exact value functions or state-action value functions in tabular form would be impossible. This chapter offers alternatives.

To motivate the approach, let $\hat{v}^*_\lambda(s)$ denote an approximation to the optimal value function $v^*_\lambda(s)$ in an infinite horizon discounted model. This chapter considers approaches in which the approximation is a lower-dimensional *parametric* function of model characteristics referred to as *basis functions* or *features*. Instead of storing $v^*_\lambda(s)$ in tabular form, the methods herein store only the parameter estimates. Using this information, a "good" action in a specific state s' is chosen as follows. Evaluate $\hat{v}^*_\lambda(j)$ in states j that can be reached from s' in one transition and choose an action $a^*_{s'}$ greedily based on the one-step Bellman update

$$a^*_{s'} \in \arg\max_{a \in A_{s'}} \left\{ r(s', a) + \lambda \sum_{j \in S} p(j|s', a) \hat{v}^*_\lambda(j) \right\}. \tag{9.1}$$

Since in most applications, there are few successor states to s', this calculation can be very efficient. Recall that this is equivalent[3] to solving a one-period problem (see Chapter 2) with terminal reward $\hat{v}^*_\lambda(s)$. Note that such an approach was used in Section 8.4.5 in which the POMDP optimal value function was approximated by supports obtained on a small set of belief points.

As an alternative to approximating value functions, approximations can be applied to state-action value functions to obtain $\hat{q}^*(s, a)$. Then $a^*_{s'}$ can be determined from

$$a^*_{s'} \in \arg\max_{a \in A_{s'}} \{\hat{q}^*(s', a)\}.$$

Moreover, approximations can be also used to directly represent policies in terms of model states such as in the policy gradient methods of Section 11.5.

Note that a policy based on greedy action choice using an approximation to the value function need not be optimal. The quality of the action depends on the accuracy of the approximation.

Returning to the appointment scheduling problem, suppose one has an approximation $\hat{v}^*_\lambda(j)$ to the optimal value function. Instead of a finding and storing the decision rule for all states, an action is chosen as needed as follows:

1. Observe the system state s', namely the number of appointments previously scheduled for each day in the booking horizon and the number of arrivals of each type.

[3] Some authors refer to this procedure as *rollout*.

2. Obtain an action to implement by evaluating the expression in braces in (9.1) for each $a \in A_{s'}$ and choosing the maximizing action,, $a^*_{s'}$.

3. Assign patients to future appointment dates according to $a^*_{s'}$.

In this case, as in most implementations of Markov decision processes, the optimal action in all states is not needed a priori; it can be determined on an as-needed basis using the above approach or one based on an approximation $\hat{q}^*(s,a)$. The advantage of using $\hat{q}^*(s,a)$ will become apparent in model-free implementations described in Chapters 10 and 11.

Questions arising are:

1. How to obtain good value function approximations?

2. How close to optimal is the stationary policy based on decision rule $\hat{d}^*(s) = a^*_s$?

Note that in most of the literature, approximation and simulation are discussed together. To identify issues particular to each, these discussions are separated. This chapter focuses on approximation. Chapter 10 discusses simulation methods. Then, Chapter 11 combines simulation and approximation, which is the cornerstone of *reinforcement learning*. Moreover, the focus in this chapter is on discounted models. Extensions to average and total reward models are left to the reader.

9.2 Approximating policy value functions

This section describes approaches for approximating value functions and subsequently generalizes them to state-action value functions. Moreover, the focus will be on approximating the expected discounted reward of a *fixed* stationary deterministic policy. The following example provides motivation.

Example 9.1. Recall that in the queuing service rate control model (Section 3.4.1) the state represents the number of jobs in the system and the action represents the chosen service probability. In each period a job arrives with probability $b = 0.2$ and the controller can choose among three service probabilities $a_1 = 0.2$, $a_2 = 0.4$ and $a_3 = 0.6$. In any period, there is either an arrival, a service or neither. There is a delay cost of $f(s) = s^2$ and a cost per period of serving at rate a_k is $m(a_k) = 5k^3$.

To illustrate approximations, the unbounded state space is truncated to $S = \{0, 1, \ldots, 50\}$ and the discount rate is set to $\lambda = 0.98$. Approximations to the value function of stationary policy $\pi = d^\infty$, which uses the deterministic decision

rule

$$d(s) = \begin{cases} a_1 & \text{for } s < 20 \\ a_3 & \text{for } s \geq 20, \end{cases}$$

are shown.

Since the state space is ordered and interpretable, simple parametric functions of the state can be used as value function approximators. To investigate the quality of approximations, the exact policy value function[a] is evaluated using methods of Chapter 5. Approximations to the exact policy value function based on linear, quadratic and exponential functions are considered. Parameter estimates for the polynomials are obtained using least squares linear regression, while those for the exponential function are obtained using nonlinear least squares. Regression is based on the data $\{(s, v(s))|s \in S\}$ or a subset thereof. In this example, this data was obtained from exact methods. In practice the exact $v(s)$ will not be available.

Figure 9.1a shows the true value function and its linear and quadratic approximations. Clearly the linear fit is poor while the quadratic fit appears to be better. It is left as an exercise to evaluate the quality of cubic polynomial and spline approximations. The equations of the two fitted models are

$$\hat{v}^{\pi}_{\lambda}(s) = -7603.3 + 1320.9s$$

and

$$\hat{v}^{\pi}_{\lambda}(s) = 2096.3 + 133.2s + 23.78s^2.$$

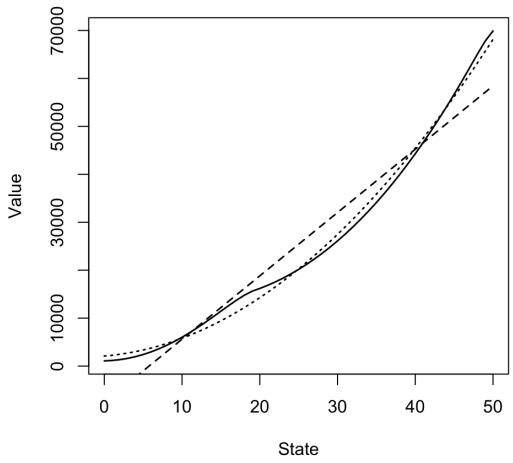
In addition the value function is approximated by the nonlinear function $\beta_0 + \beta_1 e^{\beta_2 s}$, which requires software that implements nonlinear least squares. The analysis here uses the *nls* package in R [R Core Team, 2024], which implements Gauss-Newton iteration to obtain parameter estimates. Given the sensitivity of nonlinear least squares to starting values, experimentation was required to obtain the fitted equation:

$$\hat{v}^{\pi}_{\lambda}(s) = -11403.2 + 12222.5e^{0.038s}.$$

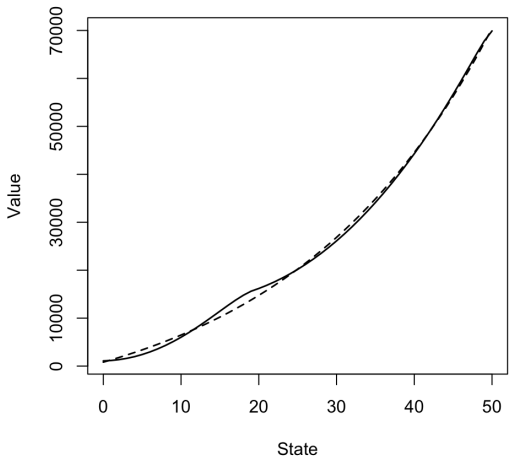
Figure 9.1b shows the actual value (solid line) and fitted value (dashed line). It appears that the exponential fit is superior to the quadratic fit.

Of course, value function approximation is not necessary in such a small instance; the above calculations are provided to motivate for the material in this chapter. In practice $v^{\pi}_{\lambda}(s)$ would not be known for all $s \in S$, so approximation would be necessary. As an aside, if the focus was the countable state version of this model, the approximation on $s \leq 50$ could be extrapolated to $s > 50$.

[a] Of course, if the exact policy value function is available, such as in this small example, there is no need to approximate it.



(a) Value function for d^∞ (solid line) with linear (dashed line) and quadratic (dotted line) approximations.



(b) Value function for d^∞ (solid line) with exponential approximation (dashed line).

Figure 9.1: Example 9.1 value functions and approximations.

9.2.1 Features

It is customary to use features of the state space as variables when approximating value functions. Feature generation is similar to the choice of independent variables in regression models. Features are usually low dimension summaries of key aspects of the state space that can be used to well approximate a value function. More formally:

Definition 9.1. A *feature* or *basis function* is a real-valued function defined on S.

Denote the set of features evaluated at s by the vector[4] $\mathbf{b}(s) = \big(b_0(s), b_1(s), \ldots, b_I(s)\big)$ Often $b_0(s) := 1$ to correspond to a constant in a regression model.

For $S = \{s_1, s_2, \ldots, s_M\}$ define the $M \times (I+1)$ matrix $\mathbf{B}$ of features by

$$\mathbf{B} := \begin{bmatrix} b_0(s_1) & \ldots & b_I(s_1) \\ \cdot & \ldots & \cdot \\ \cdot & \ldots & \cdot \\ \cdot & \ldots & \cdot \\ b_0(s_M) & \ldots & b_I(s_M) \end{bmatrix}. \tag{9.2}$$

[4]Recall that Chapter 8 used the vector $\mathbf{b}$ to denote a belief state. Hopefully this will not cause confusion.

Assume that features are constructed so that the columns of $\mathbf{B}$ are linearly independent (in the linear algebra sense[5]) so that the matrix $\mathbf{B}^\mathsf{T}\mathbf{B}$ is invertible.

Specifying features

Feature choice depends on the setting:

1. If the state space is a set of **real numbers or ordered integers** possible feature choices include powers of the state, B-spline components, trigonometric functions or exponentials. In Example 9.1 when using a quadratic value function approximation, $I = 2$ and $b_0(s) = 1$, $b_1(s) = s$ and $b_2(s) = s^2$ for all $s \in S$.

2. If the state space is **vector-valued**, possible feature choices include component values, powers of component values, interactions between components or nonlinear functions of the component values. For example in the appointment scheduling model described in the introduction to this chapter when there is a five-day booking window and two appointment types the state space is

$$S = \{(x_1, x_2, \ldots, x_5, y_1, y_2) \mid 0 \leq x_i \leq C, i = 1, \ldots, 5; 0 \leq y_k \leq L, k = 1, 2\},$$

   where C denotes the maximum daily capacity and L denotes the maximum number of appointments of each type that can arrive on a given day.

   A possible set of features is:

$$\begin{aligned}
&b_0\big((x_1, x_2, \ldots, x_5, y_1, y_2\big) = 1; \\
&b_i\big((x_1, x_2, \ldots, x_5, y_1, y_2)\big) = x_i \qquad \text{for } i = 1, \ldots, 5; \\
&b_{i,j}\big((x_1, x_2, \ldots, x_5, y_1, y_2)\big) = x_i x_j \quad \text{for } i = 1, \ldots, 5,\ j = 1, \ldots, 5 \quad \text{and} \\
&b'_k\big((x_1, x_2, \ldots, x_5, y_1, y_2)\big) = y_k \qquad \text{for } k = 1, 2.
\end{aligned}$$

   These features correspond to a quadratic model in the number of booked appointments and a linear function of the number of appointment requests.

3. In **more general settings**, application and subject matter knowledge dictates feature choice. For example, in checkers[6] (Figure 9.2) there are 32 squares on the board that can be occupied (numbered in some way) and the two players, referred to as "Red" and "Black", each start with 12 pieces. Thus

$$S = \left\{(x_1, \ldots, x_{32}) \;\middle|\; x_i \in \{E, B, R, BK, RK\} \text{ for } i = 1, \ldots, 32 \text{ and } \sum_{i=1}^{32} I_{\{B,BK\}}(x_i) \leq 12,\ \sum_{i=1}^{32} I_{\{R,RK\}}(x_i) \leq 12\right\},$$

[5] This means that the only solution of $\mathbf{Bx} = \mathbf{0}$ is $\mathbf{x} = \mathbf{0}$. Equivalently the rank of $\mathbf{B}$ is $I + 1$ (since it is assume $I + 1 < M$).

[6] Checkers is a two-person game but it can be analyzed by *self play*, that is, playing against yourself.

Figure 9.2: Checkerboard showing positions of pieces at the start of a game. Credit: gmatsuno/E+ via Getty Images.

where $I_A(x)$ is the indicator function for $x \in A$, E indicates that the square is empty, B denotes it is occupied by a single black checker, R denotes it is occupied by a single red checker, BK denotes it is occupied by a black king, and RK denotes it is occupied by a red king. The summations indicate that there are at most 12 pieces of each color on the board. Since $|S| = 5^{32}$, approximations are necessary to analyze this game.

Possible features include

$$b_0((x_1, \ldots, x_{32})) = 1;$$

$$b_1((x_1, \ldots, x_{32})) = \sum_{i=1}^{32} I_{\{B\}}(x_i); \quad b_2((x_1, \ldots, x_{32})) = \sum_{i=1}^{32} I_{\{BK\}}(x_i);$$

$$b_3((x_1, \ldots, x_{32}) = \sum_{i=1}^{32} I_{\{R\}}(x_i); \quad b_4((x_1, \ldots, x_{32})) = \sum_{i=1}^{32} I_{\{RK\}}(x_i).$$

The above features can be interpreted as follows: $b_1(\cdot)$ and $b_3(\cdot)$ denote the number of single checkers of each color and $b_2(\cdot)$ and $b_4(\cdot)$ denote the number of kings of each color. Other possible features include the number of pieces of each color that are within κ rows of becoming kings for specific choices of κ.

In summary, possible features include:

1. the function 1,

2. powers and cross products of state variables,
3. expressions that interpolate between designated states, and
4. indicator functions of subsets of states.

Choosing the set of features equal to the indicator of each state, that is,

$$b_i(s) = \begin{cases} 1 & \text{if } s = s_i, \\ 0 & \text{otherwise} \end{cases}$$

for $i = 1, \ldots, M$, is equivalent to a *tabular* representation of the model. This means that expressing the value function as a linear combination of such indicator functions is equivalent to using the standard Markov decision process formulation in terms of $v(s_i)$ for $i = 1, \ldots, M$. In this case the methods of earlier chapters apply directly.

9.2.2 Feature-based value function approximators

This section proposes some value function approximations based on features. These approximations may be linear or nonlinear with neural networks representing an important and widely used class of nonlinear approximations. Some authors refer to the specified functional form as an *architecture*.

Linear architectures

The expression *linear approximation*[7], refers to a value function approximation of the form:

$$v(s) \approx \beta_0 b_0(s) + \beta_1 b_1(s) + \ldots + \beta_I b_I(s), \tag{9.3}$$

where $b_0(s) = 1$ for all $s \in S$ so that β_0 denotes the model constant. With judicious choice of features, such models can represent a wide range of functional forms. Advantages of linear approximations include the ability to estimate parameters using least squares regression (i.e., where estimates are available in closed form; see the chapter appendix) and having a functional form that can be directly used in a linear programming model. Moreover, when meaningful, the parameters can be interpreted as marginal values or costs.

Linear approximations can be nicely represented using matrix notation as follows:

$$\mathbf{v} \approx \mathbf{B}\boldsymbol{\beta}, \tag{9.4}$$

where

$$\mathbf{B} = \begin{bmatrix} b_0(s_1) & \ldots & b_I(s_1) \\ \cdot & \ldots & \cdot \\ \cdot & \ldots & \cdot \\ \cdot & \ldots & \cdot \\ b_0(s_M) & \ldots & b_I(s_M) \end{bmatrix} \quad \text{and} \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \cdot \\ \cdot \\ \cdot \\ \beta_I \end{bmatrix} \tag{9.5}$$

[7] This may also be referred to as an *affine* approximation. Precisely speaking, the expression affine is more accurate when the approximation includes a constant term.

and $S = \{s_1, \ldots, s_M\}$. In statistical literature, the matrix $\mathbf{B}$ is often referred to as the *design matrix*[8].

Note that the above expression does not explicitly include an error term ϵ because the models in this chapter do not involve statistical sampling. The only source of imprecision is the inaccuracy of the approximation.

Nonlinear architectures

Although in many applications, a linear approximation will suffice, a nonlinear function f of features can also be used to represent value functions. In general, nonlinear architectures can be written as

$$v(s) \approx f(b_0(s), b_1(s), \ldots, b_I(s)|\boldsymbol{\beta}), \tag{9.6}$$

where the unknown vector of parameters in state s is represented by $\boldsymbol{\beta}$. In nonlinear models, the number of parameters need not equal the number of features used. That is, the dimension of $\boldsymbol{\beta}$ need not be $I + 1$. For example, with two features $b_0(s)$ and $b_1(s)$ a possible choice for $f(\cdot)$ is

$$f(b_0(s), b_1(s)|\boldsymbol{\beta}) = \beta_0 b_0(s) + \beta_1 e^{\beta_2 b_1(s)}$$

where $\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2)$. In this case there are two features and three parameters. In the special case when $b_0(s) = 1$ and $b_1(s) = s$ for all $s \in S$, the above approximation becomes

$$f(s|\boldsymbol{\beta}) = \beta_0 + \beta_1 e^{\beta_2 s},$$

which is the same as the exponential function from Example 9.1.

Neural networks

Artificial neural networks or neural networks (NNs)[9] have been widely used in large modern applications of Markov decision processes. In the Markov decision process context they transform states or features into approximate value functions, state-action value functions or even the probability of selecting an action. However they also have been used in numerous significant machine learning implementations including image recognition, recommendation systems, forecasting, and natural language processing.

The most basic type of neural network is a *feedforward neural network* or FNN. A FNN consists of *nodes* organized in *layers*, uni-directional *arcs* connecting nodes from "left" to "right" and *activation functions* corresponding to nodes in interior *hidden* layers. Layers are classified as input layers, hidden layers and output layers. Figure 9.3 depicts a feedforward neural network with an input layer, two hidden layers and an output layer.

[8]The expression *design matrix* originates in the experimental design literature where the analyst chooses the allocation of treatments to experimental units. This allocation can be summarized in terms of a matrix.

[9]Some authors refer to neural networks as *multi-layer perceptrons*.

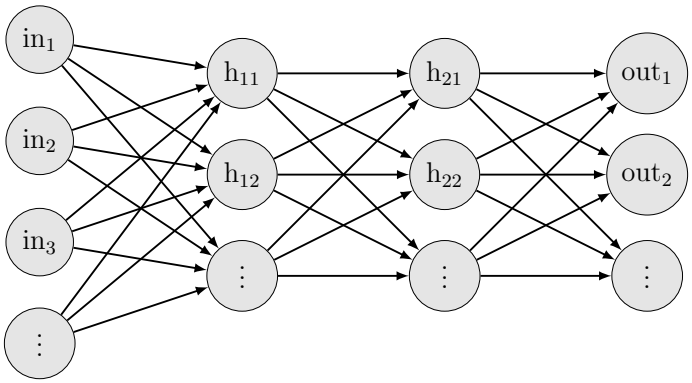


Figure 9.3: A generic feedforward neural network. The nodes labeled in_j denote nodes in the input layer, those labeled $\text{h}_{i,j}$ denote nodes in hidden layer j, and those labeled out_j denote nodes in the output layer.

The FNN processes information as follows: Values in the input layer are weighted by parameters corresponding to each arc and summed to generate inputs to the nodes at next layer where they are transformed nonlinearly by the activation function to produce outputs. These outputs are then weighted and summed and fed to the next layer, and so on.

Activation functions mimic the behavior of neurons that "fire" when the incoming signal reaches a threshold. They can be modeled by a step function that equals 0 for inputs less than a threshold and 1 for inputs greater than the threshold. Alternatively they can be represented by non-decreasing continuous functions $f : \mathbb{R} \to [0, 1]$ such as logistic or sigmoidal functions.

Other types of neural networks include:

1. **recurrent neural networks**, which generate outputs by combining inputs with previous hidden states, and

2. **convolutional neural networks**, which include filters that extract features from grid-like data such as camera images.

Details of these models are beyond the scope of this book but are important for applications in robotics and autonomous vehicle guidance. From the perspective of this chapter, neural networks are viewed as "black box" nonlinear function approximators.

An example

This example applies a single-layer FNN to the queuing service rate control example analyzed in Example 9.1 and discusses some practical issues arising when applying neural networks.

Example 9.2. Consider the queuing service rate control model analyzed in Example 9.1. This example approximates the value function of the specified stationary policy with $\lambda = 0.98$ using the FNN represented in Figure 9.4. This network consists of an input layer, a hidden layer with two nodes plus a node denoted by 1, which is often referred to as an *offset* or *bias*, and a single node in the output layer. The objective is to estimate parameters of this neural network to accurately approximate the value function. This may be regarded as using a neural network to perform nonlinear regression with features $b_0(s) = 1$ and $b_1(s) = s$.

In the hidden layer, the FNN model transforms a linear combination of inputs using the activation function

$$f(x) = \frac{1}{1 + e^{-x}} \tag{9.7}$$

and outputs the value function approximation using a linear combination of outputs from the hidden layer plus a constant. The model has seven parameters, one corresponding to each arc denoted by $w^k_{i,j}$, where i denotes the node at the start of the arc, j denotes the destination of the arc and $k = 2$ or 3 denotes the layer of the destination.

The FNN may be expressed as a single function as follows. The input to h_1 in the hidden layer is a linear function of the inputs 1 and s

$$w^1_{11} + w^1_{21} s$$

and the input to node 2 in the hidden layer is

$$w^1_{12} + w^1_{22} s.$$

The activation function transforms these inputs to

$$\frac{1}{1 + e^{-(w^1_{11} + w^1_{21} s)}} \quad \text{and} \quad \frac{1}{1 + e^{-(w^1_{12} + w^1_{22} s)}}.$$

Finally the input to the output layer can be represented by the linear function of outputs from the hidden layer as follows:

$$w^2_{01} 1 + w^2_{11} \frac{1}{1 + e^{-(w^1_{11} + w^1_{21} s)}} + w^2_{21} \frac{1}{1 + e^{-(w^1_{12} + w^1_{22} s)}}. \tag{9.8}$$

Note that this function contains the constant w^2_{01}. In light of the von Neumann quote in the Part III introduction, one would expect an accurate approximation of the value function using this architecture.

The *nnet* package in R produced the parameter estimates shown on the arcs in Figure 9.4. Achieving reliable estimates required scaling both the state and the

value function to the interval [0, 1] prior to training, followed by a reverse transformation to the original scale. Furthermore, the inclusion of a small amount of random noise during training proved beneficial. Without scaling, the estimation process produced unreliable results.

Figure 9.5 compares the true value function (solid line) and its FNN approximation (dashed line). The RMSE was 395 and while the fit looks good, the residuals had a sinusoidal pattern suggesting a systematic lack of fit. Ultimately, the quality of the approximation is determined based on the policy derived from the approximate value function.

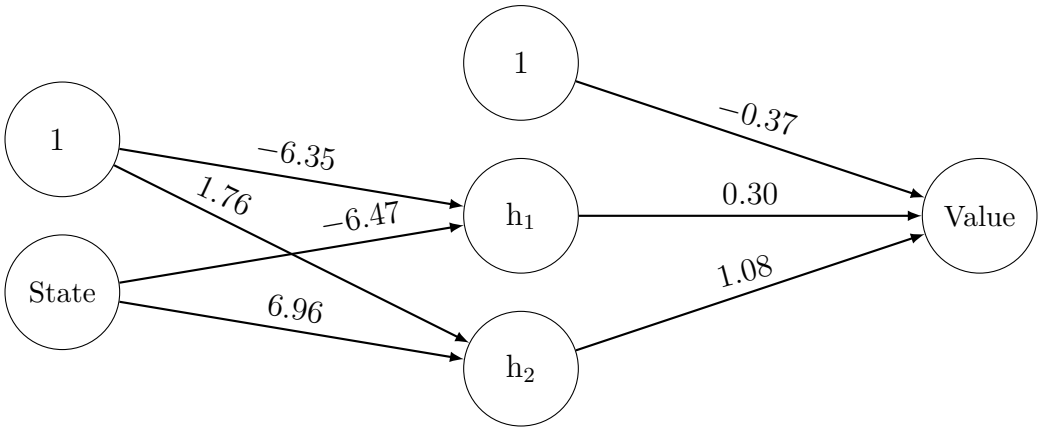


Figure 9.4: Feedforward neural network used in queuing service rate control approximation. Values on the arcs are estimated weights (parameter estimates).

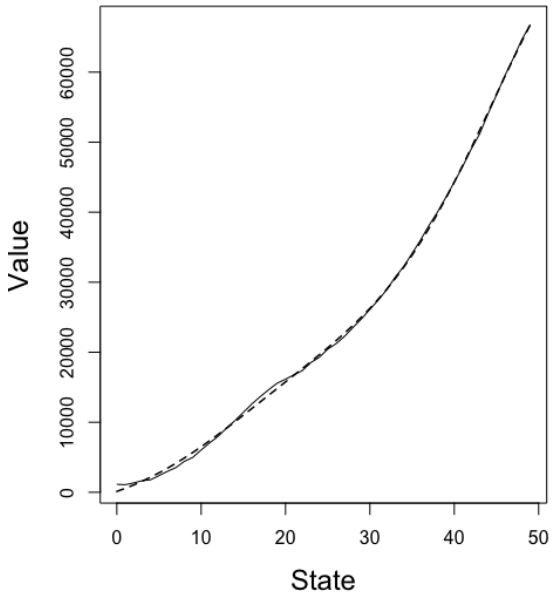


Figure 9.5: FNN approximation to the queuing system service rate control value function for the specified stationary policy. Solid line is the true value function while the dashed line is the approximation based on the FNN.

9.2.3 State-action value functions

The above sections describe how to approximate value functions using basis functions and linear or nonlinear architectures. Alternatively these constructs can be used to approximate state-action value functions $q(s,a)$. While such an approach may be of use in reinforcement learning environments when the model components are unknown, in this chapter, which assumes knowledge of $r(s,a)$ and $p(j|s,a)$, it is preferable to approximate $q(s,a)$ for all $a \in A_s$ by

$$\hat{q}(s,a) := r(s,a) + \lambda \sum_{j \in S} p(j|s,a)\hat{v}(j),$$

where $\hat{v}(s)$ is a value function approximation based on basis functions. Thus, the focus of this chapter will be on value function approximations.

9.3 Parameter estimation for policy value functions: Linear architectures

This section provides methods for finding good approximations to the value function of a *fixed* stationary policy in the form of linear combinations of basis functions. The

proposed methods are based on least squares estimation and linear programming. The next section considers nonlinear approximations. We strongly recommend referring to the Appendix of this chapter, which reviews least squares regression emphasizing matrix methods.

Motivation

Let d^∞ denote a stationary randomized policy. To determine its value, $\mathbf{v}_\lambda^{d^\infty}$, involves solving the following linear system of $|S|$ equations, which appeared previously in vector form in (5.31) as

$$\mathbf{v} = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v} = L_d \mathbf{v}. \tag{9.9}$$

Recall that when $\lambda < 1$, this system has a unique solution that can be represented by

$$\mathbf{v}_\lambda^{d^\infty} = (\mathbf{I} - \lambda \mathbf{P}_d)^{-1} \mathbf{r}_d. \tag{9.10}$$

Using matrix notation, the general form of a policy value function estimator[10] based on a linear combination of basis functions can be written as $\mathbf{v} = \mathbf{B}\boldsymbol{\beta}$, where the columns of $\mathbf{B}$ are vectors of basis functions evaluated for each $s \in S$. That is, a row of $\mathbf{B}$ is of the form $\big(b_0(s), \ldots, b_I(s)\big)$ for some $s \in S$. Let $\text{col}(\mathbf{B})$ denote the subspace of $\mathbb{R}^M$ spanned by the columns of $\mathbf{B}$, that is, it contains all real-valued functions that can be written as linear combinations of basis functions.

The benefit of seeking approximations in $\text{col}(\mathbf{B})$ is that there are $I+1$ parameters to determine, as opposed to $|S|$. In applications where approximation is necessary, I will be orders of magnitude smaller than $|S|$. But replacing exact calculation by an approximation introduces *approximation error*. Moreover the methods below seek to do this without first computing $\mathbf{v}_\lambda^{d^\infty}$.

Least squares estimation: $\mathbf{v}_\lambda^{d^\infty}$ known

To motivate approximation methods and introduce some notation, assume first that $\mathbf{v}_\lambda^{d^\infty}$ has been evaluated[11] and one seeks a good approximation $\text{col}(\mathbf{B})$. Thus one wants to find

$$\hat{\boldsymbol{\beta}} \in \underset{\boldsymbol{\beta} \in \mathbb{R}^{I+1}}{\arg\min} \, \|\mathbf{B}\boldsymbol{\beta} - \mathbf{v}_\lambda^{d^\infty}\|, \tag{9.11}$$

where $\|\cdot\|$ is either the Euclidean norm

$$\|\mathbf{v}\|_2 := \left(\sum_{s \in S} v(s)^2 \right)^{\frac{1}{2}} = (\mathbf{v}^\mathsf{T}\mathbf{v})^{\frac{1}{2}}$$

[10] The statistical expression *estimator* is used, even though there is no randomness in the model.

[11] Of course if this quantity were known, there would no need to approximate it unless one intended to interpolate or extrapolate, as is the case when truncating a countable state model.

or the weighted Euclidean norm

$$\|\mathbf{v}\|_2^w := \left(\sum_{s\in S} w(s)v(s)^2\right)^{\frac{1}{2}} = (\mathbf{v}^\mathsf{T}\mathbf{W}\mathbf{v})^{\frac{1}{2}}, \tag{9.12}$$

where $w(s) > 0$ for all $s \in S$ and $\mathbf{W}$ is a diagonal matrix with entries $w(s)$ on the diagonal.

The ordinary least squares (OLS) parameter and policy value function estimates (equations (9.68) and (9.70) in the chapter Appendix) become

$$\hat{\boldsymbol{\beta}}_{\text{OLS}} = (\mathbf{B}^\mathsf{T}\mathbf{B})^{-1}\mathbf{B}^\mathsf{T}\mathbf{v}_\lambda^{d^\infty} = \boldsymbol{\Gamma}\mathbf{v}_\lambda^{d^\infty} = \boldsymbol{\Gamma}(\mathbf{I} - \lambda\mathbf{P}_d)^{-1}\mathbf{r}_d \tag{9.13}$$

and

$$\hat{\mathbf{v}}_{\text{OLS}} = \mathbf{B}(\mathbf{B}^\mathsf{T}\mathbf{B})^{-1}\mathbf{B}^\mathsf{T}\mathbf{v}_\lambda^{d^\infty} = \boldsymbol{\Pi}\mathbf{v}_\lambda^{d^\infty} = \boldsymbol{\Pi}(\mathbf{I} - \lambda\mathbf{P}_d)^{-1}\mathbf{r}_d, \tag{9.14}$$

respectively, where

$$\boldsymbol{\Gamma} := (\mathbf{B}^\mathsf{T}\mathbf{B})^{-1}\mathbf{B}^\mathsf{T} \quad \text{and} \quad \boldsymbol{\Pi} := \mathbf{B}\boldsymbol{\Gamma} = \mathbf{B}(\mathbf{B}^\mathsf{T}\mathbf{B})^{-1}\mathbf{B}^\mathsf{T}. \tag{9.15}$$

The matrix $\boldsymbol{\Pi}$ is the projection of $\mathbb{R}^M$ onto the $(I+1)$-dimensional subspace $\text{col}(\mathbf{B})$. Note that $\boldsymbol{\Gamma}\mathbf{v}$ is a vector of parameters and $\boldsymbol{\Pi}\mathbf{v}$ is a vector of values.

Note that this approach was used to illustrate concepts earlier in Example 9.1. It is often referred to as a *direct method*.

Direct methods will not be used in practice since the goal of approximate dynamic programming is to approximate the value function without first computing it.

Least squares estimation: $\mathbf{v}_\lambda^{d^\infty}$ unknown

In contrast to the direct method described above, *indirect methods* approximate a value function by appealing to the Bellman equation and a variant. Suppose $\mathbf{v}'$ is an estimate of $\mathbf{v}_\lambda^{d^\infty}$. Then its accuracy can be measured with respect to the Bellman equation according to

$$\|\mathbf{v}' - L_d\mathbf{v}'\| \tag{9.16}$$

or with respect to the *projected Bellman evaluation equation*

$$\mathbf{v} = \boldsymbol{\Pi}L_d\mathbf{v} \tag{9.17}$$

and the corresponding error

$$\|\mathbf{v}' - \boldsymbol{\Pi}L_d\mathbf{v}'\|, \tag{9.18}$$

where as above, the norm can be either the Euclidean or weighted Euclidean norm.

Thus, setting $\mathbf{v}' = \mathbf{B}\boldsymbol{\beta}'$, the two above approaches for choosing a value of $\boldsymbol{\beta}'$ lead to:

1. **Bellman[12] residual minimization:** Choose $\hat{\boldsymbol{\beta}}_{\text{BR}}$ according to

$$\hat{\boldsymbol{\beta}}_{\text{BR}} \in \underset{\boldsymbol{\beta} \in \mathbb{R}^{I+1}}{\arg\min} \|\mathbf{B}\boldsymbol{\beta} - L_d \mathbf{B}\boldsymbol{\beta}\| \tag{9.19}$$

2. **Projection error minimization:** Choose $\hat{\boldsymbol{\beta}}_{\text{PE}}$ according to

$$\hat{\boldsymbol{\beta}}_{\text{PE}} \in \underset{\boldsymbol{\beta} \in \mathbb{R}^{I+1}}{\arg\min} \|\mathbf{B}\boldsymbol{\beta} - \boldsymbol{\Pi} L_d \mathbf{B}\boldsymbol{\beta})\|, \tag{9.20}$$

where as above, the norms may be either the Euclidean or weighted Euclidean norm.

Geometric interpretation

Geometrically these two approaches minimize different quantities as shown in Figure 9.6. The Bellman residual minimization approach finds a value for $\boldsymbol{\beta}$ that minimizes the distance between $L_d\mathbf{B}\boldsymbol{\beta}$ and $\mathbf{B}\boldsymbol{\beta}$. In this case $L_d\mathbf{B}\boldsymbol{\beta}$ need **not** be in col($\mathbf{B}$). On the other hand the projection error minimization approach minimizes the difference between $\mathbf{B}\boldsymbol{\beta}$ and $\boldsymbol{\Pi}L_d\mathbf{B}\boldsymbol{\beta}$, where both expressions lie in col($\mathbf{B}$). Note that least squares regression theory (see chapter appendix) establishes that $\boldsymbol{\Pi}L_d\mathbf{B}\boldsymbol{\beta}$ represents the element of col($\mathbf{B}$) closet to $L_d\mathbf{B}\boldsymbol{\beta}$ in the least squares sense.

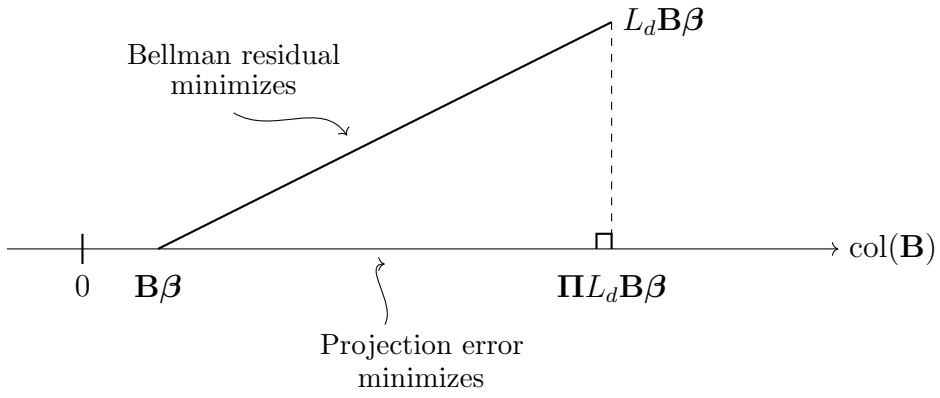


Figure 9.6: Graphical comparison of two approaches for estimating $\boldsymbol{\beta}$. This figure assumes that $\mathbf{B}$ contains a single column (corresponding to a single basis function) so that col($\mathbf{B}$) is a one-dimensional subspace of $\mathbb{R}^2$.

In principle, these two estimates can be found by varying $\boldsymbol{\beta}$ so as to make each of the distances as small as possible. In the one-dimensional example shown in Figure 9.6, this would correspond to altering the value of the scalar $\boldsymbol{\beta}$, or equivalently sliding $\mathbf{B}\beta$ along the horizontal axis and noting how $\|\mathbf{B}\boldsymbol{\beta} - L_d\mathbf{B}\boldsymbol{\beta}\|$ and $\|\mathbf{B}\boldsymbol{\beta} - \boldsymbol{\Pi}L_d\mathbf{B}\boldsymbol{\beta}\|$ change.

To distinguish these methods and gain familiarity with this notation, consider the following example.

[12]Note that as stated here, this approach corresponds to minimizing the error using the policy evaluation equation, which is a special case of the Bellman equation.

Example 9.3. Let $S = \{1, 2\}$,

$$\mathbf{r}_d = \begin{bmatrix} 2 \\ 8 \end{bmatrix} \quad \text{and} \quad \mathbf{P}_d = \begin{bmatrix} 0.25 & 0.75 \\ 0.10 & 0.90 \end{bmatrix}.$$

Choose a single basis function $b(s) = s$ so that $\mathbf{B} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. Consequently $\boldsymbol{\beta}$ is a scalar denoted by β so that approximation of $v_\lambda^{d^\infty}(s)$ is of the form

$$\mathbf{B}\beta = \begin{bmatrix} \beta \\ 2\beta \end{bmatrix}.$$

The Bellman residual error is given by

$$\|\mathbf{B}\beta - L_d\mathbf{B}\beta\| = \left\| \begin{bmatrix} \beta \\ 2\beta \end{bmatrix} - \left(\begin{bmatrix} 2 \\ 8 \end{bmatrix} + \lambda \begin{bmatrix} 1.75\beta \\ 1.9\beta \end{bmatrix} \right) \right\| = \left\| \begin{bmatrix} (1 - 1.75\lambda)\beta - 2 \\ (2 - 1.90\lambda)\beta - 8 \end{bmatrix} \right\|.$$

On the other hand noting that

$$\mathbf{\Pi} = \mathbf{B}(\mathbf{B}^\mathsf{T}\mathbf{B})^{-1}\mathbf{B}^\mathsf{T} = \frac{1}{5}\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix},$$

the least squares projection error is given by

$$\left\| \begin{bmatrix} \beta \\ 2\beta \end{bmatrix} - \frac{1}{5}\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \left(\begin{bmatrix} 2 \\ 8 \end{bmatrix} + \lambda \begin{bmatrix} 1.75\beta \\ 1.90\beta \end{bmatrix} \right) \right\| = \left\| \begin{bmatrix} (1 - 1.11\lambda)\beta - 2.4 \\ (2 - 2.22\lambda)\beta - 4.8 \end{bmatrix} \right\|.$$

Setting $\lambda = 0.6$ and using the Euclidean norm gives

$$\mathbf{v}_\lambda^{d^\infty} = \begin{bmatrix} 18.40 \\ 25.10 \end{bmatrix}, \quad \mathbf{\Pi}\mathbf{v}_\lambda^{d^\infty} = \begin{bmatrix} 10.09 \\ 20.18 \end{bmatrix}, \quad \mathbf{B}\hat{\beta}_{\text{BR}} = \begin{bmatrix} 9.14 \\ 18.27 \end{bmatrix} \quad \text{and} \quad \mathbf{B}\hat{\beta}_{\text{PE}} = \begin{bmatrix} 10.78 \\ 21.55 \end{bmatrix}.$$

Because the value function is **not** in col($\mathbf{B}$) neither value function estimate accurately approximates $\mathbf{v}_\lambda^{d^\infty}$. However, the estimate based on projection error minimization provides a better estimate of the projection of the value function on col($\mathbf{B}$) as represented by $\mathbf{\Pi}\mathbf{v}_\lambda^{d^\infty}$.

9.3.1 Least squares policy evaluation

This section describes a flexible iterative algorithm for projection error minimization referred to as *least squares policy evaluation (LSPE)*. It is based on iteratively solving the *projected* policy evaluation equation $\mathbf{v} = \mathbf{\Pi}L_d\mathbf{v}$. Its convergence depends on contraction properties of $\mathbf{\Pi}L_d$. Bellman residual minimization will be briefly discussed in a subsequent starred section and not be explored beyond that.

The following iterative algorithm (stated in matrix form) corresponds to applying value iteration to a fixed decision rule. Its beauty is that it:

1. is intuitive,
2. avoids computation of $\mathbf{v}_\lambda^{d^\infty}$,
3. can be applied on a fixed or randomly generated subset of S as described below, and
4. easily generalizes to nonlinear architectures and optimization problems.

Note that the algorithm is structured so that value function approximations remain in col($\mathbf{B}$).

Algorithm 9.1. Least squares policy evaluation: linear architectures

1. **Initialize:** Specify $\boldsymbol{\beta}$ arbitrary, $\epsilon > 0$, and $\Delta > \epsilon$.
2. **Iterate:** While $\Delta \geq \epsilon$:
   (a) $\mathbf{v}' \leftarrow \mathbf{B}\boldsymbol{\beta}$.
   (b) $\mathbf{v} \leftarrow \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}'$.
   (c) $\boldsymbol{\beta}' \leftarrow \boldsymbol{\Gamma}\mathbf{v}$[a].
   (d) $\Delta \leftarrow \|\boldsymbol{\beta}' - \boldsymbol{\beta}\|_2$.
   (e) $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}'$.
3. **Terminate:** Return $\boldsymbol{\beta}$.

[a] In practice this step would use a regression routine such as *lm* in R to implement.

Closed form representation of the LSPE estimate

The parameter estimate generated by LSPE can be represented in closed form as follows. Starting with $\boldsymbol{\beta}$, 2(a) generates the approximate value function $\mathbf{v}' = \mathbf{B}\boldsymbol{\beta}$. Then step 2(b) performs a one-step update on the approximation to obtain

$$\mathbf{v} = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}' = \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{B}\boldsymbol{\beta}.$$

Step 2(c) represents projection (least squares regression of $\mathbf{v}$ on the basis vectors) on col($\mathbf{B}$) to obtain

$$\boldsymbol{\beta}' = (\mathbf{B}^\mathsf{T}\mathbf{B})^{-1}\mathbf{B}^\mathsf{T}\mathbf{v} = \boldsymbol{\Gamma}\mathbf{v}$$

Combining the above two expressions it follows that LSPE generates $\boldsymbol{\beta}'$ from $\boldsymbol{\beta}$ according to

$$\boldsymbol{\beta}' = \boldsymbol{\Gamma}\Big(\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{B}\boldsymbol{\beta}\Big) = \boldsymbol{\Gamma}\mathbf{r}_d + \lambda \boldsymbol{\Gamma}\mathbf{P}_d\mathbf{B}\boldsymbol{\beta}. \tag{9.21}$$

As a result of (9.21), if $\lambda\boldsymbol{\Gamma}\mathbf{P}_d\mathbf{B}$ is a contraction, LSPE converges to the solution of

$$(\mathbf{I} - \lambda\boldsymbol{\Gamma}\mathbf{P}_d\mathbf{B})\boldsymbol{\beta} = \boldsymbol{\Gamma}\mathbf{r}_d, \tag{9.22}$$

where $\mathbf{I}$ is the $(I+1)\times(I+1)$ identity matrix. In this case, $(\mathbf{I} - \lambda\boldsymbol{\Gamma}\mathbf{P}_d\mathbf{B})$ is invertible so that the fixed point of the projection equation can be represented by

$$\hat{\boldsymbol{\beta}} = (\mathbf{I} - \lambda\boldsymbol{\Gamma}\mathbf{P}_d\mathbf{B})^{-1}\boldsymbol{\Gamma}\mathbf{r}_d. \tag{9.23}$$

Thus, in general, the updated approximation at the subsequent application of step 2(a) becomes

$$\mathbf{v}' = \mathbf{B}\boldsymbol{\Gamma}(\mathbf{r}_d + \lambda\mathbf{P}_d\mathbf{v}) = \boldsymbol{\Pi}(\mathbf{r}_d + \lambda\mathbf{P}_d\mathbf{v}) = \boldsymbol{\Pi}L_d\mathbf{v} \tag{9.24}$$

in agreement with Figure 9.6 where $\mathbf{v} = \mathbf{B}\boldsymbol{\beta}$ and $\mathbf{v}' = \boldsymbol{\Pi}L_d\mathbf{B}\boldsymbol{\beta}$. As a result of this observation, the above algorithm may also be referred to as "projected value iteration".

Examples

The following example[13] shows that the inverse in (9.23) need not exist.

Example 9.4. Let $S = \{s_1, s_2\}$, $\mathbf{r}_d$ be arbitrary, $\lambda = 5/5.4 = 0.926$,

$$\mathbf{P}_d = \begin{bmatrix} 0.2 & 0.8 \\ 0.2 & 0.8 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Hence

$$\boldsymbol{\Gamma} = \left(\begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right)^{-1} \begin{bmatrix} 1 & 2 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} 1 & 2 \end{bmatrix}$$

so that

$$\lambda\boldsymbol{\Gamma}\mathbf{P}_d\mathbf{B} = \frac{5}{5.4}\left(\frac{1}{5} \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 0.2 & 0.8 \\ 0.2 & 0.8 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right) = 1.$$

This implies that $\lambda\boldsymbol{\Gamma}\mathbf{P}_d\mathbf{B}$ is not a contraction so that $\mathbf{I} - \lambda\boldsymbol{\Gamma}\mathbf{P}_d\mathbf{B}$ is not invertible.

In this example, the iterative scheme implicit in (9.21) reduces to

$$\boldsymbol{\beta}' \leftarrow \boldsymbol{\Gamma}\mathbf{r}_d + \boldsymbol{\beta}.$$

Hence, if $\boldsymbol{\Gamma}\mathbf{r}_d \neq \mathbf{0}$, LSPE diverges and if $\boldsymbol{\Gamma}\mathbf{r}_d = \mathbf{0}$, it does not uniquely determine $\boldsymbol{\beta}$.

[13] This example appears in de Farias and Roy [2000].

This example is an *edge case* that is unlikely to occur in practice. In most settings, LSPE will converge as shown in the example below. Note that under the conditions that $\mathbf{P}_d$ is recurrent and that the basis vectors are linearly independent, $\mathbf{\Pi}L_d$ is a contraction mapping[14] with respect to the weighted Euclidean norm (9.12) with weight function equal to the (positive) steady state distribution of $\mathbf{P}_d$.

Example 9.5. This example illustrates the calculations described above in the context of the queuing service rate control model analyzed in Example 9.1. It approximates the value function of the previously specified stationary policy d^∞ by linear and quadratic functions of the state and compares the estimates based on LSPE (or its fixed point representation based on (9.23)) to the exact value function and its direct OLS estimate.

Set $c(s,a) := f(s) + m(a)$. Step 2(b) of LSPE is implemented component-wise using the expressions:

$$v(0) = c(0, d(0)) + \lambda((1-b)v'(0) + bv'(1)),$$

$$v(50) = c(50, d(50)) + \lambda(d(50)v'(49) + (1 - d(50))v'(50)) \tag{9.25}$$

and

$$v(s) = c(s, d(s)) + \lambda((1-b)v'(s-1) + (1-b-d(s))v'(s) + d(s)v'(s)),$$

for $s = 1, \ldots, 49$ where, $d(s)$ denotes the service rate in state s.

Note that there is no need to distinguish the truncation state 50 as in (9.25) since the approximation $v'(51)$ is available. Choosing such an approach may be regarding as evaluating the infinite state model with $S = \{0, 1, \ldots\}$ by an approximation of $\boldsymbol{\beta}$ based on the reduced set of states $S' = \{0, \ldots, 50\}$.

The analysis below chooses $v'(s)$ to be either the linear approximation

$$v'(s) = \beta_0 + \beta_1 s$$

or the quadratic approximation

$$v'(s) = \beta_0 + \beta_1 s + \beta_2 s^2.$$

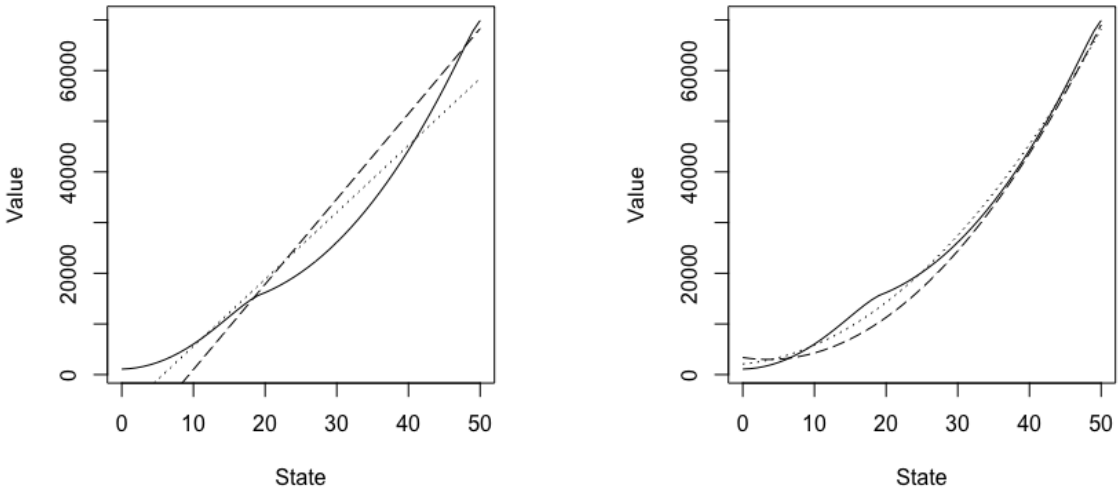
Applying LSPE for the linear approximation yields $\hat{\beta}_0 = -15825.3$ and $\hat{\beta}_1 = 1682.6$ compared to OLS values $\hat{\beta}_0 = -7603.3$ and $\hat{\beta}_1 = 1320.9$. Graphical comparisons are provided in Figure 9.7a. The main takeaways are that these two estimates are quite different and neither approximates $v_\lambda^{d^\infty}(s)$ well.

Using the quadratic approximation, LSPE gives $\hat{\beta}_0 = 3371.2$, $\hat{\beta}_1 = -216.1$ and $\hat{\beta}_2 = -30.6$ in contrast to the OLS values $\hat{\beta}_0 = 2096.3$, $\hat{\beta}_1 = 133.2$ and $\hat{\beta}_2 = 23.8$.

[14] See Bertsekas [2012], pp. 423-427.

Graphical comparisons of fitted values appear in Figure 9.7b. Observe that the quadratic fit is superior to the linear fit. Moreover, both approximations are close to the true value function for $s \geq 30$ but differ at lower values. Note also that the LSPE approximation is not monotone in s for low occupancy levels.

Note that improved fits may be possible by taking into account that $d(s)$ has a step change at $s = 20$. However letting the form of a particular policy determine the approximation will have downsides when turning to optimization. It is left to the reader to compare approximations that use cubic polynomials or a splines with knots at 20.



(a) Linear approximation.

(b) Quadratic approximation.

Figure 9.7: Linear and quadratic approximations to the exact value function for the specified policy in Example 9.5. The solid line indicates the exact value function, the dotted line indicates the OLS estimate of the exact value function and the dashed line indicates the LSPE approximation.

Using subsets of S in LSPE and other iterative algorithms

The numerical examples above applied LSPE using basis functions and updates defined for **all** $s \in S$. But in practice, especially when $S = \{s_1, \ldots, s_M\}$ is large, one would seek to obtain estimates based on basis functions and updates using a small subset $S' = \{s'_1, \ldots, s'_n\} \subset S$. This affects Algorithm 9.1 and all other iterative methods in this chapter as follows.

To begin with, $\mathbf{B}$ has n rows with entries in each column corresponding to the values of basis functions on S'. Then instead of applying steps 2(a) and 2(b) for all $s \in S$, the algorithm generates $v(s')$ for $s' \in S'$ according to

$$v(s') \leftarrow r\big(s', d(s')\big) + \lambda \sum_{j \in S} \left(p\big(j|s', d(s')\big) \sum_{i=0}^{I} \beta_i b_i(j) \right). \tag{9.26}$$

Note that summation over j involves states that are not in S'. Therefore in such states the update uses the current value of $\boldsymbol{\beta}$ to approximate the value in these states by $\sum_{i=0}^{I} \beta_i b_i(j)$. Fortunately in most practical examples, such as queuing control or Gridworld navigation, $p(j|s', d(s')) > 0$ is positive for only a small number of successor states so computing this sum is not burdensome.

Applying (9.26) for all $s' \in S'$ results in a "data set" suitable for regression. This data, in matrix form, consists of

$$\mathbf{v} = \begin{bmatrix} v(s'_1) \\ \vdots \\ v(s'_n) \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_0(s'_1) & \dots & b_I(s'_1) \\ \vdots & \vdots & \vdots \\ b_0(s'_n) & \dots & b_I(s'_n) \end{bmatrix}.$$

Thus estimates of $\boldsymbol{\beta}'$ can be obtained in closed form as

$$\boldsymbol{\beta}' = \boldsymbol{\Gamma}\mathbf{v} = (\mathbf{B}^\mathsf{T}\mathbf{B})^{-1}\mathbf{B}^\mathsf{T}\mathbf{v}$$

or by using a regression package. In the nonlinear setting, $f(b_0(j), b_1(j), \dots, b_I(j); \boldsymbol{\beta})$ replaces $\sum_{i=0}^{I} \beta_i b_i(j)$ in the update.

Bellman residual minimization*

As noted above, Bellman residual minimization obtains a linear approximation to the value function by substituting $\mathbf{v} = \mathbf{B}\boldsymbol{\beta}$ into $(\mathbf{I} - \lambda\mathbf{P}_d)\mathbf{v} = \mathbf{r}_d$ to obtain

$$(\mathbf{I} - \lambda\mathbf{P}_d)\mathbf{B}\boldsymbol{\beta} = (\mathbf{B} - \lambda\mathbf{P}_d\mathbf{B})\boldsymbol{\beta} \approx \mathbf{r}_d. \tag{9.27}$$

This approach regresses $\mathbf{r}_d$ on the columns of $(\mathbf{I} - \lambda\mathbf{P}_d)\mathbf{B}$ directly. Defining $\mathbf{G}_d := \mathbf{B} - \lambda\mathbf{P}_d\mathbf{B}$, the least squares approximations based on (9.27) become

$$\hat{\mathbf{v}} = \mathbf{G}_d(\mathbf{G}_d^\mathsf{T}\mathbf{G}_d)^{-1}\mathbf{G}_d^\mathsf{T}\mathbf{r}_d \tag{9.28}$$

and

$$\hat{\boldsymbol{\beta}}_{\text{BR}} = (\mathbf{G}_d^\mathsf{T}\mathbf{G}_d)^{-1}\mathbf{G}_d^\mathsf{T}\mathbf{r}_d. \tag{9.29}$$

Note that in the above, the matrix $\mathbf{G}_d = \mathbf{B} - \lambda\mathbf{P}_d\mathbf{B}$ is $M \times (I+1)$ so that $\mathbf{G}_d^\mathsf{T}\mathbf{G}_d$ is $(I+1) \times (I+1)$.

To avoid matrix inversion, one can instead solve the normal equations

$$\mathbf{G}_d^\mathsf{T}\mathbf{G}_d\hat{\boldsymbol{\beta}} = \mathbf{G}_d^\mathsf{T}\mathbf{r}_d. \tag{9.30}$$

directly.

Analysis of the quality of this approximation is left to the reader.

9.3.2 Linear programming approximations for policy value functions

Linear programming offers another way of finding approximations for linear architectures.

Approximate linear programming – primal model

Chapter 5 formulated the following (primal) LP to determine optimal value functions:

$$\begin{aligned} \text{minimize} \quad & \sum_{s\in S} \alpha(s)v(s) \\ \text{subject to} \quad & v(s) - \lambda \sum_{j\in S} p(j|s,a)v(j) \geq r(s,a), \quad a \in A_s, s \in S. \end{aligned}$$

When there is **only one** deterministic stationary policy $d^\infty(s)$ to evaluate, this primal LP reduces to

$$\text{minimize} \quad \sum_{s\in S} \alpha(s)v(s) \tag{9.31a}$$

$$\text{subject to} \quad v(s) - \lambda \sum_{j\in S} p(j|s,d(s))v(j) \geq r(s,d(s)), \quad s \in S. \tag{9.31b}$$

Observe that when restricted to a single stationary policy, this LP has $|S|$ variables and $|S|$ constraints. Now suppose $v(s)$ is approximated by a linear function of prespecified basis functions $(b_0(s), b_1(s), \ldots, b_I(s))$ of the form

$$v(s) = \beta_0 b_0(s) + \beta_1 b_1(s) + \ldots + \beta_I b_I(s).$$

Substituting this expression into (9.31a) and (9.31b) and rearranging terms gives the approximate primal linear program:

Approximate primal linear program (APLP) – fixed decision rule

$$\text{minimize} \quad \sum_{i=0}^{I} \beta_i \sum_{s\in S} \alpha(s) b_i(s) \tag{9.32a}$$

$$\text{subject to} \quad \sum_{i=0}^{I} \beta_i b_i(s) - \lambda \left(\sum_{i=0}^{I} \beta_i \sum_{j\in S} p(j|s,d(s)) b_i(j) \right) \geq r(s,d(s)), \quad s \in S. \tag{9.32b}$$

Observe that this LP has $I+1$ variables $\beta_0, \beta_1, \ldots, \beta_I$ and $|S|$ constraints. Since in most applications $|S| \gg I$, this system has many more constraints than variables. The following example illustrates simple cases of this approximation and provides insight into the geometry underlying the LP approximation.

Example 9.6. Suppose there is a single feature $b_0(s) = 1$ for all $s \in S$ so that

$$\mathbf{B} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Consequently the value function will be estimated[a] by a constant β_0. Assuming $\sum_{s\in S} \alpha(s) = 1$, the APLP becomes

$$\begin{aligned} \text{minimize} \quad & \beta_0 \\ \text{subject to} \quad & (1-\lambda)\beta_0 \geq r(s, d(s)), \quad s \in S. \end{aligned} \tag{9.33}$$

Moreover this easy to solve LP has one variable and $|S|$ constraints. Its optimal solution is

$$\hat{\beta}_0^{\text{LP}} = \max_{s\in S} \left\{ \frac{r(s, d(s))}{1-\lambda} \right\}.$$

Now consider as a special case the two-state ($S = \{0, 1\}$) version of the queuing service rate control model previously analyzed in this chapter. Assume an arrival rate of 0.2 and service rate of $d(0) = a_1 = 0.2$ and $d(1) = a_2 = 0.4$. To be consistent with the LP formulation, represent it as a reward maximization problem with $r(0, d(0)) = -5$ and $r(1, d(1)) = -41$.

Therefore the approximate linear program (9.33) when $\lambda = 0.5$ becomes

$$\begin{aligned} \text{minimize} \quad & \beta_0 \\ \text{subject to} \quad & 0.5\beta_0 \geq -5 \\ & 0.5\beta_0 \geq -41 \end{aligned}$$

and its optimal solution is $\hat{\beta}_0^{\text{LP}} = -10$. Consequently the LP approximation to the value function is $\hat{v}_{\text{LP}}(0) = \hat{v}_{\text{LP}}(1) = -10$. Note that if S were instead truncated at $N = 50$, there would be 51 inequalities of a similar form to those above.

The exact LP for this model is

$$\begin{aligned} \text{minimize} \quad & \alpha_0 v(0) + \alpha_1 v(1) \\ \text{subject to} \quad & 0.6v(0) - 0.1v(1) \geq -5 \\ & -0.2v(0) + 0.7v(1) \geq -41 \end{aligned}$$

The solution of the exact LP is $v_\lambda^{d^\infty}(0) = -19$ and $v_\lambda^{d^\infty}(1) = -64$. Note that the least squares approximation to this solution would be the average of these

two values, $\hat{\beta}_0^{\text{OLS}} = -41.5$, so that the least squares approximation is $\hat{v}_{\text{OLS}}(0) = \hat{v}_{\text{OLS}}(1) = -41.5$.

Figure 9.8 illustrates these estimates. Its caption explains what is shown in considerable detail. The takeaway is that when the subspace spanned by the basis functions intersects the feasible region near the optimal vertex, then the LP approach will provide good estimates of the optimal value function. Unfortunately, this cannot be known priori, so care must be taken when choosing basis functions.

[a] In a regression model with only a constant, the parameter estimate is the mean of the observations.

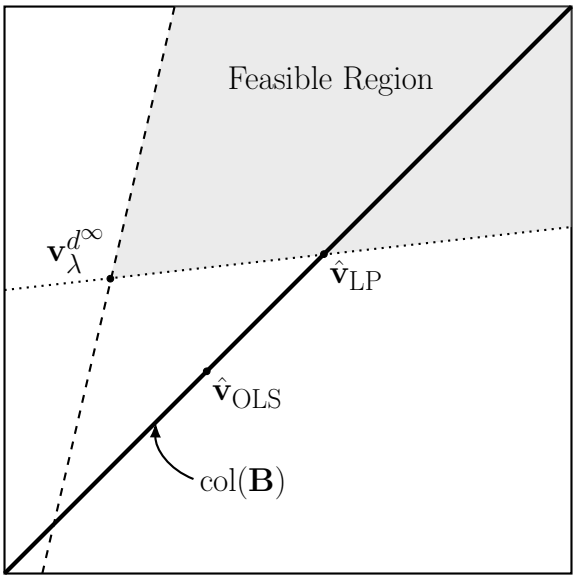


Figure 9.8: Graphical representation of the value function and two approximations for Example 9.6. It shows the feasible region for the exact LP, its solution $\mathbf{v}_\lambda^{d^\infty}$, and the subspace col($\mathbf{B}$). The quantities $\hat{\mathbf{v}}_{\text{LP}}$ and $\hat{\mathbf{v}}_{\text{OLS}}$ denote the LP and OLS estimates in that subspace. Since the LP minimizes β_0, its value is the minimum value of the intersection of col($\mathbf{B}$) and the feasible region of the LP. The OLS estimate is the orthogonal projection of the exact value function on to that subspace.

The approximate LP model will be discussed further below when it is used to find approximate optimal values.

9.4 Parameter estimation for a fixed policy: Nonlinear architectures

This section modifies Algorithm 9.1 to use nonlinear architectures. This requires replacing computation of $\mathbf{v}'$ in step 2(a) by

$$v'(s) = f(s, \boldsymbol{\beta}),$$

written as $\mathbf{v}' \leftarrow \mathbf{f}(\boldsymbol{\beta})$. Step 2(c), replaces the computation of $\mathbf{v}'$ by its least squares estimate obtained using Gauss-Newton iteration or some other method expressed as $\boldsymbol{\beta}' \in \arg\min_{\boldsymbol{\beta}} \|\mathbf{f}(\boldsymbol{\beta}) - \mathbf{v}\|_2$ since a closed form representation for $\hat{\boldsymbol{\beta}}$ is unavailable for nonlinear architectures.

Algorithm 9.2. Least squares policy evaluation: nonlinear architectures

1. **Initialize:** Specify $\boldsymbol{\beta}$ arbitrary, $\epsilon > 0$, and $\Delta > \epsilon$.
2. **Iterate:** While $\Delta \geq \epsilon$:
   (a) $\mathbf{v}' \leftarrow \mathbf{f}(\boldsymbol{\beta})$.
   (b) $\mathbf{v} \leftarrow \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}'$.
   (c) $\boldsymbol{\beta}' \in \arg\min_{\boldsymbol{\beta}} \|\mathbf{f}(\boldsymbol{\beta}) - \mathbf{v}\|_2$.
   (d) $\Delta \leftarrow \|\boldsymbol{\beta}' - \boldsymbol{\beta}\|_2$.
   (e) $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}'$.
3. **Terminate:** Return $\boldsymbol{\beta}$.

The following example applies Algorithm 9.2 to fit a nonlinear exponential approximation to the value function in the queuing service rate control model.

Example 9.7. This example illustrates the nonlinear LSPE algorithm above in the queuing service rate control model in Example 9.1. In it, the value function for the specified stationary policy d^∞ is approximated by an exponential function. It compares the estimate based on LSPE to the exponential fit when the value function is known. To implement the algorithm, chose $\boldsymbol{\beta} = \mathbf{0}$ so that the corresponding initial value function $\tilde{\mathbf{v}} = \mathbf{0}$ and $\mathbf{v}$ in step 2(b) becomes $\mathbf{v} = \mathbf{r}_d$. Step 2(c) used the *nls* function in R. Estimation required some adjustment of starting values to ensure convergence at every iteration of step 2(c).

With $\epsilon = 0.001$, the algorithm terminated in 386 iterations with the approxi-

mation

$$\hat{v}_{\lambda}^{d^{\infty}}(s) = -13427.3 + 13004.1e^{0.037s}$$

which is shown in Figure 9.9. Observe that $\hat{\beta}_1$ and $\hat{\beta}_2$ differ from that in Example 9.1 but $\hat{\beta}_3$ is almost identical to that obtained when the value function was known. Moreover the approximation was considerably more accurate than those based on linear and quadratic approximations.

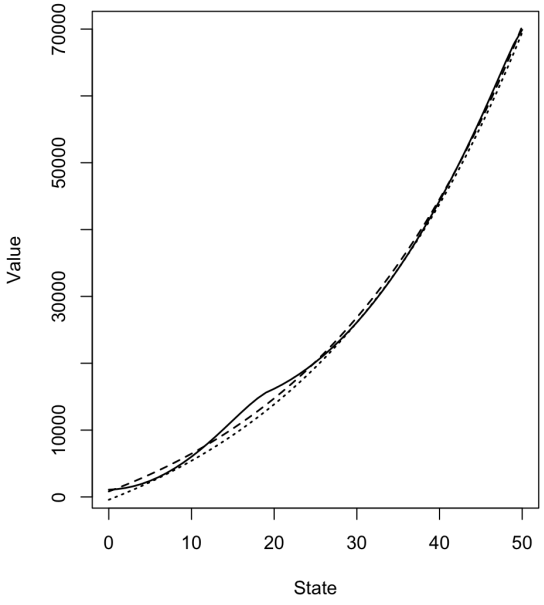


Figure 9.9: Value function (solid line) for queuing service rate control model with nonlinear least squares approximation (dashed line) and nonlinear LSPE approximation (dotted line).

This example suggest that applying nonlinear architectures requires considerably more care than in the linear case because of the sensitivity of nonlinear algorithms to starting values and other algorithmic parameters. As in the case of a linear architecture, this method can be applied to a subset of states that are pre-determined or sampled.

Note that Algorithm 9.2 can also be used to fit a neural network iteratively. In this case step 2(b) would require repeated training of the neural network.

9.5 Combining approximation and optimization

This section generalizes the previous section by providing methods that seek good policies in a Markov decision process based on approximations to the optimal value

function. Such methods are usually referred to as approximate dynamic programming (ADP). It presents value iteration, policy iteration and modified policy iteration algorithms based on linear, nonlinear and neural network architectures and a linear programming approach, which by necessity applies only to linear architectures.

In practice these iterative methods can be applied to pre-specified or randomly selected subsets of the state space, but most of our examples implement them on the entire state space to illustrate the most optimistic outcomes.

9.5.1 Iterative methods for linear architectures

The following algorithm is an optimization version of LSPE. It is then generalized to obtain policy iteration and modified policy iteration algorithms. The approaches all assumes a pre-specified set of basis functions and a set of states at which to evaluate them so that the matrix $\mathbf{B}$ can be specified a priori.

Least squares value iteration (LSVI) – linear architectures

The following algorithm generalizes Algorithm 9.1 to incorporate optimization. The only change is in step 2(b) where the Bellman operator replaces the one-step policy update $\mathbf{v} \leftarrow \mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}'$.

Algorithm 9.3. Least squares value iteration: linear architectures

1. **Initialize:** Specify $\boldsymbol{\beta}$ arbitrary, $\epsilon > 0$, and $\Delta > \epsilon$.
2. **Iterate:** While $\Delta \geq \epsilon$:
   (a) $\mathbf{v}' \leftarrow \mathbf{B}\boldsymbol{\beta}$.
   (b) $\mathbf{v} \leftarrow \text{c-max}_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}'\}$.
   (c) $\boldsymbol{\beta}' \leftarrow \boldsymbol{\Gamma}\mathbf{v}$.
   (d) $\Delta \leftarrow \|\boldsymbol{\beta}' - \boldsymbol{\beta}\|_2$.
   (e) $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}'$.
3. **Terminate:** Return $\boldsymbol{\beta}$.

As noted in Example 9.4 above, the algorithm cannot be guaranteed to converge because the operator implicitly defined by the algorithm need not be a contraction mapping. However, empirically, this algorithm performs well as illustrated by the examples below.

Least squares policy iteration (LSPI) – linear architectures

This section describes a policy iteration method for finding "good" policies. Instead of choosing the value in step 2(b) of Algorithm 9.3, it obtains the arg c-max, evaluates the resulting policy and then finds the best approximation. The algorithm can be initiated with either:

1. a policy,
2. a value, or
3. a parameter vector.

The order of steps in the first pass through the algorithm depends on how it is initialized. With linear architectures it appeared best to begin with a parameter or value; with nonlinear architectures, methods based on initialization with a value seemed to work best. In the latter case, the first pass through step 2 begins with an improvement.

Algorithm 9.4. Least squares policy iteration: linear architectures

1. **Initialize:** Specify $d' \in D^{\text{MD}}$, $\boldsymbol{\beta}$ arbitrary, $\epsilon > 0$, and $\Delta > \epsilon$.
2. **Iterate:** While $\Delta \geq \epsilon$:
   (a) **Parameter evaluation:** $\boldsymbol{\beta}' \leftarrow (\mathbf{I} - \lambda \boldsymbol{\Gamma} \mathbf{P}_{d'} \mathbf{B})^{-1} \boldsymbol{\Gamma} \mathbf{r}_{d'}$.
   (b) **Value function approximation:** $\mathbf{v}' \leftarrow \mathbf{B}\boldsymbol{\beta}'$.
   (c) **Improvement:**
   $$d' \leftarrow \underset{d \in D^{\text{MD}}}{\arg\,\text{c-max}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}'\}.$$
   (d) $\Delta \leftarrow \|\boldsymbol{\beta}' - \boldsymbol{\beta}\|_2$.
   (e) $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}'$.
3. **Terminate:** Return d' and $\boldsymbol{\beta}$.

Some comments about this algorithm follow.

1. This algorithm can be applied to a subset of S using methods described in the section on LSPE.
2. The stopping criterion in step 2 is stated in terms of parameter estimates. It can be replaced by either: "Stop when $d' = d$" or stop when the change in $\mathbf{v}$ is

sufficiently small. Our calculations found no difference in algorithmic behavior between these criteria in examples. Note if a stopping criterion is based on decision rules, the specification "set $d' = d$ if possible" to avoid cycling needs to be added.

3. Note that the only reason to carry $\boldsymbol{\beta}'$ forward in step 2(e) is to evaluate the stopping criterion at the next iteration, it is not used in steps 2(a)-2(c).

4. Step 2(a) provides the closed-form representation for $\boldsymbol{\beta}$ corresponding to decision rule d. This representation is valid for linear architectures. In practice, it might be easier to implement it using LSPE. When using nonlinear architectures, this step would be replaced by LSPE.

5. Recall that the matrix $\boldsymbol{\Gamma} = (\mathbf{B}^\mathsf{T}\mathbf{B})^{-1}\mathbf{B}^\mathsf{T}$.

6. When the algorithm is initialized with a starting value for $\boldsymbol{\beta}$, step 2(a) can be skipped at the first pass through the algorithm.

7. This algorithm can be modified easily to use approximations of state-action value functions instead of value functions.

Example 9.8. This example applies LSPI to the queuing service rate control model with $S = \{0, \ldots, 50\}$. It compares linear, quadratic and cubic polynomial approximations. The algorithm was initiated with the previously considered decision rule and used $\epsilon = 0.0001$ for the stopping criterion. All instances required 3 iterations for convergence.

Figure 9.10 shows the optimal policy and the policies determined by the algorithm. In all cases the approximations resulted in monotone non-decreasing decision rules. Clearly the linear approximation was inadequate. The quadratic and cubic approximations produced similar policies differing only in states 11, 12, 13 and 30. The policy identified by the cubic approximation was optimal.

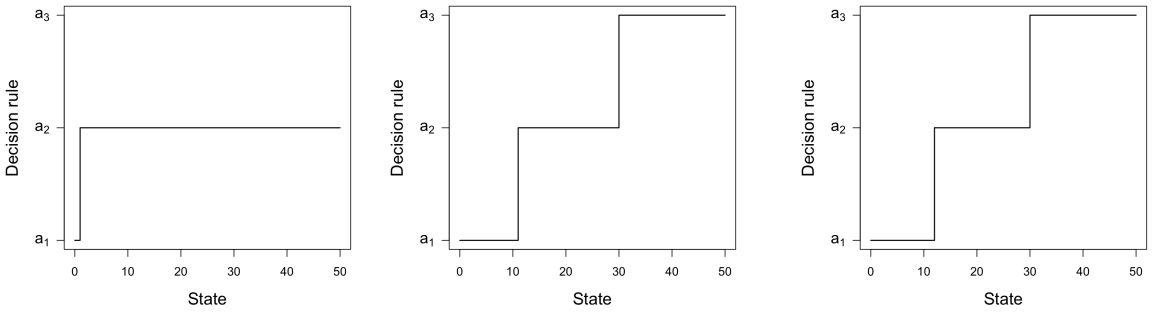
A larger model: Additional calculations revealed that the policies identified by the cubic approximation agreed with the optimal policy for $S = \{0, \ldots, 500\}$ and $\lambda = 0.9$[a]. When $\lambda = 0.98$, the policy obtained from the cubic approximation d^∞_{cub} and the optimal policy d^∞_{opt} differ as follows:

$$d_{\text{cub}}(s) = \begin{cases} a_1 & s \in \{0, \ldots, 20\} \\ a_2 & s \in \{21, 22, 23\} \\ a_3 & s \in \{24, \ldots, 500\} \end{cases} \quad \text{and} \quad d_{\text{opt}}(s) = \begin{cases} a_1 & s \in \{0, \ldots, 8\} \\ a_2 & s \in \{10, \ldots, 15\} \\ a_3 & s \in \{16, \ldots, 500\}. \end{cases}$$

To gain further insight into the effect of approximation, the values of these two policies were compared. Note that LSPI does not provide the value of d^∞_{cub}; this requires computing $(\mathbf{I} - \lambda \mathbf{P}_{d_{\text{cub}}})^{-1}\mathbf{r}_{d_{\text{cub}}}$. These calculations show that values

differ most at low occupancy levels. Bounds in the next subsection will provide further insight. However, it appears that more accurate approximations may be needed when λ is close to 1.

[a]When solving the larger instance, the approximating polynomials were centered at 250 to avoid numerical instability in step 2(a) when computing $(\mathbf{B}^\mathsf{T}\mathbf{B})^{-1}$. That is, the components in the $\mathbf{B}$ matrix were of the form $s - 250$, $(s - 250)^2$ and $(s - 250)^3$. Alternatively, this step could be replaced by iterative policy evaluation such as in Algorithm 9.5 below.



(a) Linear (b) Quadratic (c) Cubic

Figure 9.10: Stationary policies obtained using LSPI with linear, quadratic and cubic approximations. The policy obtained using a cubic approximation was optimal. In these figures, $S = \{0, \ldots, 50\}$ and $\lambda = 0.9$.

The following example shows that LSPI may not converge when using linear approximations.

Example 9.9. This example applies LSPI to the two-state example from Section 2.5 using the single basis function $b(s_1) = 0.05$ and $b(s_2) = 1$, and setting $\lambda = 0.9$. In this case,

$$\mathbf{B} = \begin{bmatrix} 0.05 \\ 1 \end{bmatrix} \quad \text{and} \quad \mathbf{B}\beta = \begin{bmatrix} 0.05\beta \\ \beta \end{bmatrix}.$$

The Bellman update (in component notation) applied to the approximation becomes:

$$v(s_1) \leftarrow \max(3 + \lambda(0.8 \cdot 0.05\beta + 0.2\beta),\ 5 + \lambda\beta)$$
$$v(s_2) \leftarrow \max(-5 + \lambda\beta,\ 3 + \lambda(0.4 \cdot 0.05\beta) + 0.6\beta)$$

It is left as an exercise to show that for initial values $\beta = 0$ and $\beta = -50$, LSVI converges quickly and identifies an optimal policy. However, for both

initial values, LSPI does not converge. Initially it chooses the decision rule $(a_{1,2}, a_{2,2})$ but then it **cycles** between the parameter estimates -51.87 and 26.92, and decision rules $(a_{1,2}, a_{2,1})$ and $(a_{1,1}, a_{2,2})$ indefinitely.

On the other hand, for other basis functions and starting values of β both LSPI and LSVI converge to either an optimal policy or a sub-optimal policy[a].

This simple example shows that it is difficult to predict the behavior of LSPI and LSVI. This cycling phenomenon is explored further in Bertsekas [2011] and Bertsekas [2012].

[a] In LSVI, the resulting policy is defined as the greedy decision rule based on the value approximation, once a convergence criterion is met.

LSPI bounds

This section applies the bounds from Section 5.5.3 to LSPI. Step 2(c) of the algorithm implicitly computes the quantity

$$L\mathbf{v}' = \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}'\}.$$

Therefore with minimal additional computation the bounds from Theorem 5.12 can be applied to obtain

$$\begin{aligned} \mathbf{v}' + \frac{1}{(1-\lambda)}\underline{(L\mathbf{v}' - \mathbf{v}')}\mathbf{e} \leq L\mathbf{v}' + \frac{\lambda}{(1-\lambda)}\underline{(L\mathbf{v}' - \mathbf{v}')}\mathbf{e} \leq \mathbf{v}_\lambda^{(d')^\infty} \leq \mathbf{v}_\lambda^* \\ \leq L\mathbf{v}' + \frac{\lambda}{(1-\lambda)}\overline{(L\mathbf{v}' - \mathbf{v}')}\mathbf{e} \leq \mathbf{v}' + \frac{1}{(1-\lambda)}\overline{(L\mathbf{v}' - \mathbf{v}')}\mathbf{e}, \end{aligned} \tag{9.34}$$

where as before $\overline{\mathbf{u}} = \max_{s \in S} u(s)$, $\underline{\mathbf{u}} = \min_{s \in S} u(s)$ and

$$d' \in \arg \underset{d \in D^{\mathrm{MD}}}{\text{c-max}}\{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}'\}.$$

Most importantly, since $\mathbf{v}_\lambda^{(d')^\infty}$ is never computed using LSPI[15], the following bound provides a measure of how close it is to the optimal value:

$$\|\mathbf{v}_\lambda^{(d')^\infty} - \mathbf{v}_\lambda^*\| \leq \frac{\lambda}{(1-\lambda)}\mathrm{sp}(L\mathbf{v}' - \mathbf{v}'), \tag{9.35}$$

where as before $\mathrm{sp}(\mathbf{v}) = \overline{\mathbf{v}} - \underline{\mathbf{v}}$. Moreover, averaging the inner upper and lower bounds gives the approximation

$$L\mathbf{v}' + \frac{\lambda}{2(1-\lambda)}\mathrm{sp}(L\mathbf{v}' - \mathbf{v}')\mathbf{e} \approx \mathbf{v}_\lambda^*. \tag{9.36}$$

[15] It only computes the approximation $\mathbf{v}' = \mathbf{B}\boldsymbol{\beta}'$.

The beauty of these bounds is that they can be applied to any $\mathbf{v}'$ and $L\mathbf{v}'$ regardless of how they have been obtained. Moreover, (9.35) provides an alternative stopping criterion for LSPI (or LSVI). However, if the stopping criterion is too strict or the basis functions are inappropriate, it need not be attained. Also, the factor of $(1-\lambda)^{-1}$ may result in the bounds not being very tight.

The following example applies these bounds to the queuing control example.

Example 9.10. This example computes the bounds in (9.34) for the queuing service rate control model with capacity 50.

Since $\mathbf{v}'$ is computed in step 2(b) and $L\mathbf{v}'$ can be easily obtained from step 2(c), the quantity $L\mathbf{v}'-\mathbf{v}'$ and all of the bounds can be easily computed. Using a cubic approximation, at termination of LSPI, $\underline{L\mathbf{v}'-\mathbf{v}'} = -113.9$ and $\overline{L\mathbf{v}'-\mathbf{v}'} = 38.2$. Using these values gives the inner bounds in (9.34), which are shown in Figure 9.11b.

Since $\text{sp}(L\mathbf{v}'-\mathbf{v}') = 152.1$, (9.35) yields the estimate

$$\|\mathbf{v}_\lambda^{(d')^\infty} - \mathbf{v}_\lambda^*\| \leq 1369.5.$$

Since in fact $(d')^\infty$ is optimal, this bound is not very accurate. However, the average absolute error of the approximation in (9.36) is 38.8 indicating that the approximation based on (9.36) is very accurate.

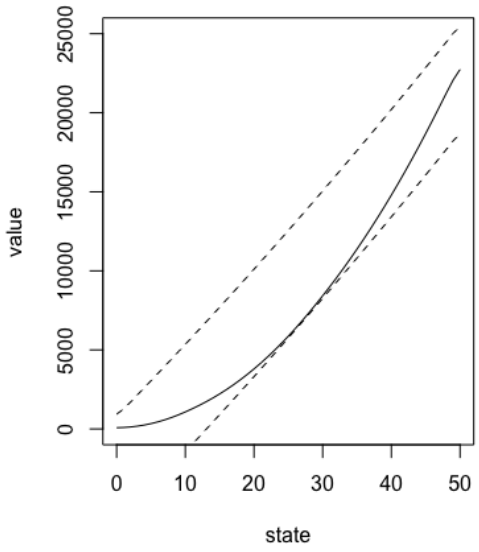
Further calculations show that bounds based on a quadratic approximation are only slightly less tight than those based on the cubic approximation but, those based on a linear approximation are considerably less accurate (Figure 9.11a).

Least squares modified policy iteration – linear architectures

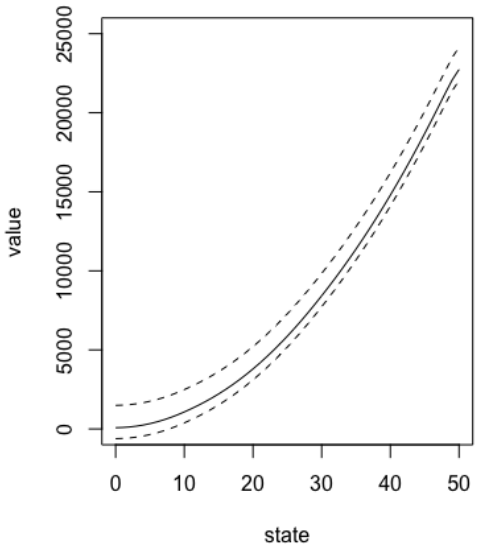
In large applications, it might be challenging to construct and solve the linear system implicit in step 2(a) of the LSPI iteration algorithm. The following generalization of MPI, referred to as *least squares modified policy iteration* and denoted by LSMPI addresses this issue.

Algorithm 9.5. Least squares modified policy iteration: linear architectures

1. **Initialize:** Specify $M \in \mathbb{Z}_+$, $\boldsymbol{\beta}$ arbitrary, $\epsilon > 0$, and $\Delta > \epsilon$.
2. **Iterate:** While $\Delta \geq \epsilon$:
   (a) $\mathbf{v} \leftarrow \mathbf{B}\boldsymbol{\beta}$.



(a) Bounds based on a linear approximation to the value function.



(b) Bounds based on a cubic approximation to the value function.

Figure 9.11: Bounds (dashed lines) for value function (solid line) derived from the LSPI solution using inner inequalities in (9.34).

(b) $\mathbf{v}' \leftarrow \text{c-max}_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}$.

(c) $d' \leftarrow \arg \text{c-max}_{d \in D^{\text{MD}}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}\}$.

(d) $m \leftarrow 1$.

(e) While $m \leq M$:

i. $\boldsymbol{\beta}' \leftarrow \boldsymbol{\Gamma} \mathbf{v}'$.

ii. $\mathbf{v}' \leftarrow \mathbf{r}_{d'} + \lambda \mathbf{P}_{d'} \mathbf{B} \boldsymbol{\beta}'$.

iii. $m \leftarrow m + 1$.

(f) $\boldsymbol{\beta}' \leftarrow \boldsymbol{\Gamma} \mathbf{v}'$.

(g) $\Delta \leftarrow \|\boldsymbol{\beta}' - \boldsymbol{\beta}\|_2$.

(h) $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}'$.

3. **Terminate:** Return d' and $\boldsymbol{\beta}$.

Some comments about using this algorithm follow:

1. The algorithm is presented in a way that enables its implementation on a *subset* of S.

2. Step 2(e) corresponds to terminating LSPE after $M+1$ iterations. If $M=0$, step 2(e) is skipped and the algorithm is identical LSVI and if $M=\infty$, it is equivalent to LSPI.

3. The integer M denotes the order of the algorithm. It can vary from iteration to iteration. See the discussion in Chapter 5 for guidance on selection of M. Choosing M adaptively to ensure that the norm of successive values of $\boldsymbol{\beta}'$ computed in 2(e)i is less than some pre-specified small tolerance is equivalent to LSPI.

4. It may be easier (especially when applying to LSMPI to nonlinear architectures[16]) to initialize the algorithm with some $\tilde{\mathbf{v}}$ instead of $\boldsymbol{\beta}$. In this case step 2(a) is not necessary at the first pass through the step 2.

5. Steps 2(b) and 2(c) are implemented together avoiding double computation.

6. As an alternative to a parameter-based stopping criterion, one could use the value based stopping criterion $\|\mathbf{v}'-\tilde{\mathbf{v}}\|<\epsilon$. Such a modification was necessary when using neural network approximations in which parameter estimates were highly unstable.

7. The concluding observations in Example 9.9 apply as well to LSMPI.

9.5.2 Iterative methods for nonlinear architectures

This brief section generalizes LSVI and LSPI by developing similar methods for finding good policies using nonlinear approximations based on features.

Least squares value iteration – nonlinear architectures

This section modifies the LSVI algorithm (Algorithm 9.3) to allow for nonlinear approximation architectures $\mathbf{v}\approx\mathbf{f}(\boldsymbol{\beta})$. The main difference with the linear architecture variant is that the estimate of $\boldsymbol{\beta}$ cannot be obtained in closed form. That is, the closed form representation in step 2(c) of Algorithm 9.3 is replaced by an estimate derived by nonlinear least squares. This step is implemented using appropriate code.

Algorithm 9.6. Least squares value iteration: nonlinear architectures

1. **Initialize:** Specify $\boldsymbol{\beta}$ arbitrary, $\epsilon>0$, and $\Delta>\epsilon$.

2. **Iterate:** While $\Delta\geq\epsilon$:

   (a) $\mathbf{v}'\leftarrow\mathbf{f}(\boldsymbol{\beta})$.

[16] In complicated nonlinear models such as neural networks, it may be challenging to accurately estimate parameter values.

(b) $\mathbf{v} \leftarrow \text{c-max}_{d \in D^{\text{MD}}}\{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}'\}$.

(c) $\boldsymbol{\beta}' \in \arg\min_{\boldsymbol{\beta}} \|\mathbf{f}(\boldsymbol{\beta}) - \mathbf{v}\|_2$.

(d) $\Delta \leftarrow \|\boldsymbol{\beta}' - \boldsymbol{\beta}\|_2$.

(e) $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}'$.

3. **Terminate:** Return $\boldsymbol{\beta}$.

Least squares policy iteration – nonlinear architectures

The following generalizes LSPI (Algorithm 9.4) to nonlinear architectures.

Algorithm 9.7. Least squares policy iteration: nonlinear architectures

1. **Initialize:** Specify $d \in D^{\text{MD}}$, $\boldsymbol{\beta}$ arbitrary, $\epsilon > 0$, and $\Delta > \epsilon$.
2. **Iterate:** While $\Delta \geq \epsilon$:

(a) $\mathbf{v}' \leftarrow \mathbf{f}(\boldsymbol{\beta})$.

(b)
$$d' \leftarrow \arg \underset{d \in D^{\text{MD}}}{\text{c-max}}\{\mathbf{r}_d + \lambda \mathbf{P}_d \mathbf{v}'\}.$$

(c) Apply LSPE using d' to return $\boldsymbol{\beta}'$.

(d) $\Delta \leftarrow \|\boldsymbol{\beta}' - \boldsymbol{\beta}\|_2$.

(e) $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}'$.

3. **Terminate:** Return d' and $\boldsymbol{\beta}$.

Note that step 2(c) produces a parameter estimate corresponding to policy $(d')^\infty$. Its value is computed at the next iteration by $\mathbf{f}(\boldsymbol{\beta})$. Note also that both of these algorithms can be applied on subsets of S using the modification described in the LSPE section.

The following example illustrates the use of nonlinear LSPI in the queuing service rate control model.

Example 9.11. As in Example 9.7 the value function is approximated by

$$v(s) = \beta_0 + \beta_1 e^{\beta_2 s}$$

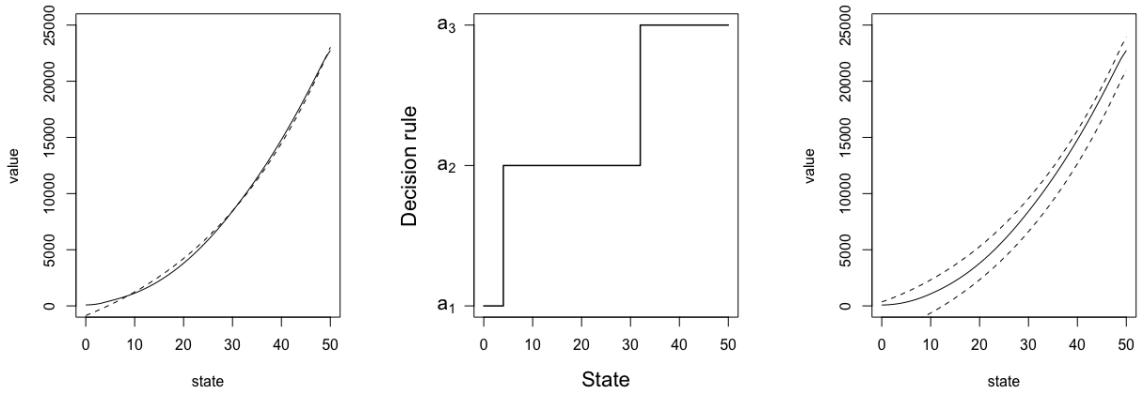
and $\lambda = 0.9$.

Applying the stopping criterion "stop if $d' = d$", nonlinear LSPI terminates

in three iterations. Note that since the algorithm identified the same decision rule at two successive iterations, it also generated the same values of $\boldsymbol{\beta}$ at these iterations so a stopping rule based on $\|\boldsymbol{\beta} - \boldsymbol{\beta}'\|_2$ would also terminate after three iterations.

Figure 9.12a shows the optimal value function and the exponential approximation found with Algorithm 9.7. Figure 9.12b shows the stationary policy based on the optimal exponential approximation. Figure 9.12c shows bounds on the optimal value function based on the exponential approximation.

Note that the approximation does not fit well for low state values and the policy obtained using the approximation deviates from the optimal policy in states 4 to 13 and in state 31. (see Figure 9.10). Therefore one can conclude that the cubic polynomial approximation is preferable for this instance.



(a) Comparison of exponential approximation (dashed line) and the true value function (solid line).

(b) Stationary policy obtained using exponential approximation.

(c) Bounds on optimal value function based on exponential approximation.

Figure 9.12: Value function, policy and bounds for Example 9.11.

9.5.3 Optimization using linear programming: Linear architectures

The linear programming approach in Section 9.3.2 was used to approximate the policy value function of a fixed policy. This section extends this approach to obtain approximations to the optimal value function. As noted above, this estimate differs from the least squares estimates obtained using LSVI, LSPI or LSMPI. And of course, this approach is only appropriate for approximations that are linear functions of the parameters.

Primal LP approximation

In a reward maximization problem, the primal LP approximation in component form can be written as follows:

Approximate primal linear program (APLP): component form

$$\text{minimize} \quad \sum_{i=0}^{I} \beta_i \sum_{s \in S} \alpha(s) b_i(s) \tag{9.37a}$$

$$\text{subject to} \quad \sum_{i=0}^{I} \beta_i b_i(s) - \lambda \left(\sum_{i=0}^{I} \beta_i \sum_{j \in S} p(j|s,a) b_i(j) \right) \geq r(s,a), \quad a \in A_s, s \in S. \tag{9.37b}$$

In a cost minimization problem this APLP becomes:

$$\text{maximize} \quad \sum_{i=0}^{I} \beta_i \sum_{s \in S} \alpha(s) b_i(s) \tag{9.38a}$$

$$\text{subject to} \quad \sum_{i=0}^{I} \beta_i b_i(s) - \lambda \left(\sum_{i=0}^{I} \beta_i \sum_{j \in S} p(j|s,a) b_i(j) \right) \leq r(s,a), \quad a \in A_s, s \in S. \tag{9.38b}$$

Observe that both models have $I+1$ variables and $\sum_{s \in S} |A_s|$ constraints. Note that the difference between formulation (9.37) and formulation (9.32) is that the constraints are for all $a \in A_s$ rather than for a fixed policy d. Hence, there are $\sum_{s \in S} |A_s|$ constraints instead of $|S|$ constraints.

Matrix version of the approximate linear program

Instead of implementing the model in component notation it is considerably easier to approach it from a matrix perspective. Recall that the (exact) primal model in Section 5.9 can be written as:

$$\begin{aligned} \text{minimize} \quad & \boldsymbol{\alpha}^{\mathsf{T}} \mathbf{v} \\ \text{subject to} \quad & \mathbf{A}\mathbf{v} \geq \mathbf{r}, \end{aligned} \tag{9.39}$$

where $\mathbf{r}$ is a column vector with entries $r(s,a)$ listed in the same order as the constraints represented in $\mathbf{A}$ and $\boldsymbol{\alpha}$ is an arbitrary positive $|S|$-dimensional vector. Substituting the linear approximation $\mathbf{v} \approx \mathbf{B}\boldsymbol{\beta}$ into (9.39) yields:

Approximate primal linear program: vector form

$$\begin{aligned} \text{minimize} \quad & (\boldsymbol{\alpha}^{\mathsf{T}}\mathbf{B})\boldsymbol{\beta} \\ \text{subject to} \quad & (\mathbf{AB})\boldsymbol{\beta} \geq \mathbf{r}. \end{aligned} \tag{9.40}$$

This suggests the following approach for finding policies based on the APLP.

Algorithm 9.8. Using APLP to find an approximate optimal policy

1. Specify an approximation $\mathbf{v} \approx \mathbf{B}\boldsymbol{\beta}$.
2. Formulate the exact primal LP (9.39).
3. Transform the model to (9.40).
4. Solve (9.40) to obtain $\hat{\boldsymbol{\beta}}$.
5. Set $\hat{\mathbf{v}} \leftarrow \mathbf{B}\hat{\boldsymbol{\beta}}$.
6. Choose
$$\hat{d} \in \underset{d \in D^{\text{MD}}}{\arg\,\text{c-max}} \{\mathbf{r}_d + \lambda \mathbf{P}_d \hat{\mathbf{v}}\}.$$
7. Obtain bounds on $\mathbf{v}_\lambda^*$ using (9.34).

The beauty of this approach is that $\mathbf{B}$ can easily be modified to try different approximations. The bounds in the last step can be used to assess the quality of the approximation[17]. Note that $\hat{\mathbf{v}}$ only equals $\mathbf{v}_\lambda^{d^\infty}$ when $\hat{\mathbf{v}}$ is the optimal value (which it most likely will not be the case for an arbitrary $\mathbf{B}$ and $\hat{\boldsymbol{\beta}}$).

Dual LP approximation

Note that the approximate **dual** LP has the following form:

$$\begin{aligned} \text{maximize} \quad & \mathbf{r}^{\mathsf{T}}\mathbf{x} \\ \text{subject to} \quad & (\mathbf{AB})^{\mathsf{T}}\mathbf{x} = \mathbf{B}^{\mathsf{T}}\boldsymbol{\alpha} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned} \tag{9.41}$$

This dual linear program has $I + 1$ equality constraints and $\sum_{s \in S} |A_s|$ variables. Whether the primal or dual is more efficiently solved is largely a function of the problem instance. Modern solvers are capable of deciding the best approach to solving a given

[17] We believe the idea of using these bounds in the linear programming context is new. Moreover they can be obtained when finding an approximately optimal policy in the previous step.

LP formulation. Note that if the dual is solved, the primal variables are also available because they are the shadow prices of the dual constraints (and vice versa). This is important because the primal variables are the quantities needed to construct the approximation to the optimal value function. Assuming the dual has many more columns than variables, it may also be solved using column generation[18] methods.

An example

The following example uses linear programming to find linear approximations to the optimal value function in the queuing service rate control model.

Example 9.12. Recall that in this model $A_s = \{a_1, a_2, a_3\}$ with $a_1 = 0.2$, $a_2 = 0.4$ and $a_3 = 0.6$, the probability of an arrival $b = 0.2$ and the cost $c(s, a_k) = s^2 + 5k^3$, which is the sum of the delay cost $f(s) = s^2$ and the serving cost per period $m(a_k) = 5k^3$. The state space is truncated at $N = 50$, $\lambda = 0.9$ and approximations of the form

$$v(s) = \beta_0 + \beta_1 s + \beta_2 s^2$$

are considered.

Since the objective is to minimize costs, formulation (9.38) is used. Because the above quadratic approximation has three parameters, the primal LP has 3 variables and 153 constraints. Note that the corresponding dual linear program, has 153 variables and 3 constraints (plus non-negativity constraints).

Choosing $\alpha(s) > 0$ so that $\sum_{s\in S} \alpha(s) = 1$ and letting $M_1 = \sum_{s\in S} \alpha(s)s$ and $M_2 = \sum_{s\in S} \alpha(s)s^2$ the APLP becomes (after considerable algebra):

$$\begin{aligned}
\text{maximize} \quad & \beta_0 + M_1\beta_1 + M_2\beta_2 \\
\text{subject to} \quad & (1-\lambda)\beta_0 - \lambda b\beta_1 - \lambda b\beta_2 \le c(s, a_k), \quad s = 0, k = 1, 2, 3, \\
& (1-\lambda)\beta_0 + (s + \lambda(a_k - b))\beta_1 + (s^2 - \lambda(b + a_k) - 2\lambda(b - a_k)s)\beta_2 \\
& \quad \le c(s, a_k), \quad s \in \{1, \ldots, 49\}, k = 1, 2, 3, \\
& (1-\lambda)\beta_0 + (s + \lambda a_k)\beta_1 + (s^2 - \lambda a_k + 2\lambda a_k s)\beta_2 \\
& \quad \le c(s, a_k), \quad s = 50, k = 1, 2, 3.
\end{aligned}$$

Using the matrix formulation of the APLP (9.40) avoids the error-prone and inflexible manipulations used to derive the above constraints in component form. As noted previously the matrices of basis function values for linear and quadratic

[18] Desrosiers and Lübbecke [2005] provides a primer on column generation in LPs.

approximations are given by

$$\mathbf{B} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ \cdot & \cdot \\ \cdot & \cdot \\ \cdot & \cdot \\ 1 & 50 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 2^2 \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ 1 & 50 & 50^2 \end{bmatrix},$$

respectively. Hence the primal LP model becomes:

$$\begin{aligned} \text{maximize} \quad & (\boldsymbol{\alpha}^\mathsf{T}\mathbf{B})\boldsymbol{\beta} \\ \text{subject to} \quad & (\mathbf{AB})\boldsymbol{\beta} \leq \mathbf{r}. \end{aligned}$$

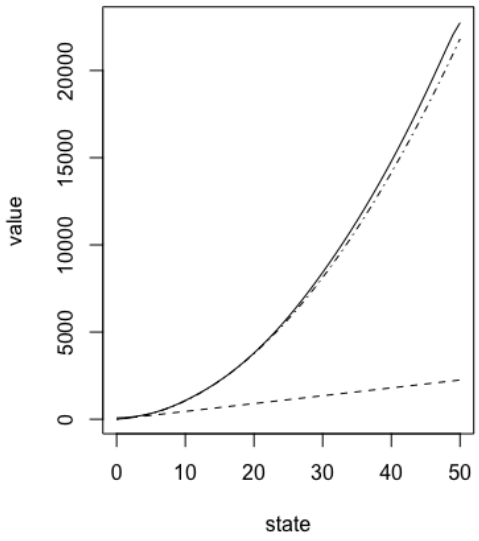
Solving it using the command *simplex* in R provides the estimates $\hat{\boldsymbol{\beta}} = (0, 45)$ for the linear approximation and $\hat{\boldsymbol{\beta}} = (0, 23.2, 8.26)$ for the quadratic approximation.

Figure 9.13a shows the resulting linear and quadratic approximations to the optimal value function. Observe that the linear approximation is inadequate and the quadratic approximation fits well. In both cases, the approximations lie below the true optimal value function. This is because the LP maximizes the greatest lower bound to the optimal value function that satisfies all of the constraints. Moreover the APLP objective function based on the quadratic approximation equals 7532 compared with the optimal objective function of 7538 obtained from the exact LP.

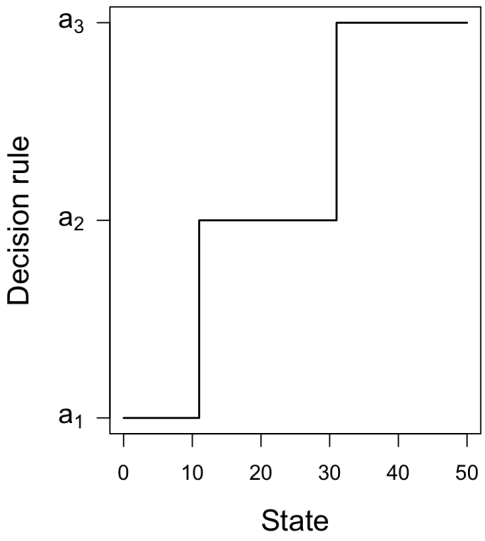
Figure 9.13b shows the stationary policy obtained using the above algorithm based on the quadratic approximation. It is monotone and only differs from the optimal policy in states 30 and 31 where it chooses actions a_2 instead of a_3.

Note also that for this instance, a cubic approximation resulted in identical values to the quadratic approximation because the coefficient of the cubic term was zero. To further explore this issue, $\lambda = 0.999$ was used. In this case the quadratic and cubic approximations differed, but the greedy policy based on each was the same.

Based on the observations, the linear programming approach described in Algorithm 9.8 offers an attractive and flexible approach for obtaining approximations to the optimal value function.



(a) Linear and quadratic approximations to exact value function (solid line) obtained using APLP.



(b) Stationary policy obtained using quadratic LP approximation.

Figure 9.13: Results of using the approximate LP in the queuing service rate control model with $N = 50$.

9.6 Application: strategic scheduling

This section applies the above methods to the following simplified version of the advanced scheduling problem in Section 3.7. The lengthy discussion in the later part of this section illustrates the type of insights that can be gained by solving a Markov decision process and issues arising when using approximations.

In the simplified model, requests for appointments arrive randomly throughout the current day (today). At the end of the day, appointment requests are scheduled for service on some day in the future (tomorrow and subsequent days). Once scheduled, an appointment cannot be rescheduled or delayed. On account of these assumptions, appointment requests arriving today cannot be scheduled until tomorrow.

There are two appointment request types (Class 1 and Class 2) and two types of service (regular time and overtime[19]). Class 1 appointments are urgent and Class 2 appointments are less urgent. Class 1 appointments **must** be served the next day (tomorrow) through either regular time or overtime and Class 2 appointments can be scheduled for regular time tomorrow (if space is available after assigning the Class

[19]In the medical setting, overtime is often referred to as *surge capacity*.

1 requests), between 2 and N days into the future or through overtime tomorrow[20]. Refer to the quantity N as the booking horizon; no appointments can be assigned to an appointment beyond that day. If a Class 1 appointment is scheduled for regular service tomorrow, **no** cost is incurred. If it is scheduled for overtime, the cost is C. If a Class 2 appointment request is scheduled for regular service on day n in the future, the cost is c_n, $n = 1, \ldots, N$[21] and the cost is C if it is served through overtime. It is reasonable to assume c_n is non-decreasing in n and that $C > c_N$[22]. Figure 9.14 illustrates the cost structure and decision problem symbolically.

The number of Class 1 and Class 2 appointment requests arriving each day are random and independent. For each day, the probability of j Class 1 requests is p_j and the probability of k Class 2 requests in q_k. The vectors $\mathbf{p} = (p_0, \ldots, p_J)$ and $\mathbf{q} = (q_0, \ldots, q_K)$ represent the probability distributions. Assume a capacity of M regular time appointments each day, no limit on overtime and at most J Class 1 arrivals and K Class 2 arrivals on any day. Moreover assume demand on successive days is independent.

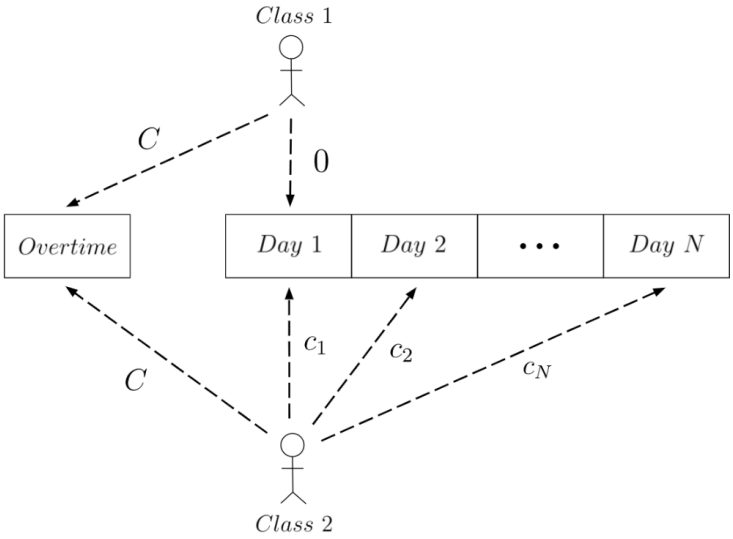


Figure 9.14: Symbolic representation of simplified advance appointment scheduling problem to be analyzed in this section. Note that Day 1 corresponds to "tomorrow".

Model formulation

A cost minimization Markov decision process formulation follows.

[20] If an appointment is to be scheduled for overtime, it is better to do so tomorrow instead of incurring delay costs.

[21] Note $n = 1$ corresponds to tomorrow. It seems reasonable to set $c_1 = 0$ but it is left arbitrary to simplify notation.

[22] Otherwise it would be preferable to use overtime rather than schedule an appointment on or before day N.

Decision epochs: Decision epochs correspond to the specific point of time at the end of a day when the scheduler assigns requests to appointments. Although there is a finite booking horizon, the problem is ongoing so that an infinite horizon formulation is appropriate. Therefore

$$T = \{1, 2, \ldots\}.$$

States: A state is a vector $(s_1, \ldots, s_N, d_1, d_2)$ where s_n for $n = 1, \ldots, N$ denotes the number of previously booked appointments on days n and d_i for $i = 1, 2$ denotes the number of Class i appointment requests to schedule. Therefore $|S| = (M+1)^N \times (J+1) \times (K+1)$. Note that an alternative state description would replace the number of booked appointments on day n by the number of available appointment slots on day n.

Actions: Actions are the number of Class 2 requests to assign to day n or to overtime. More formally let a_n denote the number of Class 2 requests assigned to day n, $n = 1, \ldots, N$ and a_0 denote the number of Class 2 appointments assigned to overtime. Then the following rather complicated expression describes the feasible assignments given demands d_1 and d_2:

$$A_{(s_1,\ldots,s_n,d_1,d_2)} = \{(a_0, \ldots, a_N) \,|\, a_0 \geq 0;\ a_1 = (M - s_1 - d_1)^+;$$
$$0 \leq a_n \leq M - s_n, n = 2, \ldots, N;\ \sum_{n=0}^{N} a_n = d_2\}.$$

An interpretation of these constraints follows:

1. The condition $a_0 \geq 0$ means that there is no bound on the number of units served through overtime.

2. The condition $a_1 = (M - s_1 - d_1)^+$ means that if after scheduling Class 1 demand for day 1 (tomorrow) there is any capacity available, as much Class 2 demand as possible will be assigned to day 1. Since it is least expensive to schedule demand on day 1, any optimal policy will use this assignment. Moreover it is the only way in which Class 1 demand explicitly interacts with Class 2 demand.

3. Since $M - s_n$ represents the number of available appointment slots on day n, the condition $0 \leq a_n \leq M - s_n$ constrains the number of Class 2 appointments that can be assigned on day n.

4. The condition $\sum_{n=0}^{N} a_n = d_2$ means that all Class 2 demand has to be assigned to appointment slots (including overtime).

This is one of the few examples in the book where the action varies with the state. This complicates coding because it requires deriving and storing or generating action sets as needed. Note that there is no decision to make for Class 1 appointments because they are served in regular time if capacity is available and otherwise through overtime.

Costs: Costs are incurred on the basis of the number of appointments booked in overtime time, C, and the number of Class 2 appointments booked on day n, c_n. That is

$$c\big((s_1,\ldots,s_N,d_1,d_2),(a_0,\ldots,a_N)\big) = C\big(d_1-(M-s_1)\big)^+ + Ca_0 + \sum_{n=1}^{N} c_n a_n.$$

In this expression, the first term corresponds to overtime charges for Class 1 appointments where $M - s_1$ denotes the number of free spots on day 1. If d_1 exceeds this number, overtime costs are incurred. Recall that it is assumed Class 1 appointment requests incur no cost if assigned to regular time on day 1. Note that the model is formulated in terms of a cost $c(s,a)$ instead of a negative reward $-r(s,a)$.

Transition probabilities: Transition probabilities can be expressed as:

$$p((s_1',\ldots,s_N',d_1',d_2') \mid (s_1,\ldots,s_N,d_1,d_2),(a_0,\ldots,a_N)) = \begin{cases} p_{d_1'}q_{d_2'}, & s_n' = s_{n+1}+a_{n+1}, n=1,\ldots,N-1, s_N'=0 \\ 0, & \text{otherwise.} \end{cases}$$

Transition probabilities are simpler than they appear. When there is a transition from today to tomorrow, after scheduling appointment requests, tomorrow's state becomes today's state and the state on day N becomes 0 since no appointments were previously scheduled on that day. The only probabilistic term corresponds to the arrival of Class 1 and Class 2 appointment requests today.

The Bellman equation

The Bellman equation for the infinite horizon discounted cost minimization version is given by:

$$v\big((s_1,\ldots,s_N,d_1,d_2)\big) = \min_{(a_0,\ldots,a_N)\in A_{(s_1,\ldots,s_n,d_1,d_2)}} \Big\{ C\Big(\big(d_1-(M-s_1)\big)^+ + a_0\Big) + \sum_{n=1}^{N} c_n a_n + \lambda \sum_{j=0}^{J}\sum_{k=0}^{K} p_j q_k v\big((s_2+a_2,\ldots,s_N+a_N,0,j,k)\big) \Big\} \tag{9.42}$$

for all $(s_1,\ldots,s_N,d_1,d_2) \in S$. Note that the argument of $v(\cdot)$ inside the "min" represents tomorrow's state after allocating appointment requests that arrived today. The probabilities p_j and q_k only impact the last two components of the state vector.

> The challenge faced by the scheduler is to determine on which day to assign Class 2 demand so as to retain sufficient capacity for future random Class 1 demand and, at the same time, not waste capacity.

Scheduling in practice presents many challenges, most notably anticipating future demand when scheduling current demand. Schedulers often act myopically and schedule appointments at the earliest possible date instead of trying to reserve capacity to meet future demand. The greedy policies based on approximations to the optimal value function do the latter.

9.6.1 A numerical example

This section formulates a small numerical example chosen so that an ϵ-optimal solution can be obtained without resorting to approximations. Let $N = 3, M = 4, J = K = 4$ so that the model has $5^3 \times 5 \times 5 = 3125$ states. The number of actions in each state varies from 1 to 35[23] so that the number of state-action pairs is large[24]. Clearly increasing any of these quantities to practical levels will result in a model that requires solution by approximation.

A comparison of ϵ-optimal policies and values found with value iteration to those found using LSVI with linear and quadratic approximations follows. Results are for an instance with $\lambda = 0.95, c_n = 2n, C = 20, \mathbf{p} = (0, 0.2, 0.2, 0.3, 0.3)$ and $\mathbf{q} = (0.3, 0, 0, 0.3, 0.4)$. The arrival probabilities were chosen so that the total expected daily demand of $5.2 = 2.7 + 2.5$ exceeds the capacity of 4, making strategic scheduling a necessity. Results and interpretations appear in the following sections. Because of the complexity of the action set, policy iteration and linear programming are left to the reader.

9.6.2 Value iteration without approximation

Value iteration, using the recursion implicit in iterating the Bellman equation (9.42), was the easiest to code[25] and as well converged quickly. For this instance, the iterates of value iteration achieved the stopping criterion $\text{sp}(\mathbf{v}' - \mathbf{v}) < 0.0001$ in 11 iterations. Columns 2 and 3 of Table 9.1 gives actions and values obtained using value iteration for selected states.

To interpret these results, consider the case $s = (1, 1, 0, 2, 4)$. This means that there are 3 units of regular-time capacity available on days 1 and 2 and 4 units available on day 3. Since the Class 1 demand is 2, it can be scheduled on day 1 leaving 1 unit of capacity available for Class 2 demand. Thus 3 units of Class 2 demand remain to be scheduled. The greedy action based on the ϵ-optimal value function, assigns one unit to day 2 and two units to day 3. Thus at the next decision epoch, $(s'_1, s'_2, s'_3) = (2, 2, 0)$.

[23] In state $(0, 0, 0, 0, 4)$ there are 35 actions and in any state with $d_2 = 0$ or no available capacity, there is only one action.

[24] It is left as an exercise to determine the exact number of actions.

[25] Generating the set of actions for each state and manipulating arrays was challenging to code. Complexities included a "feature" of R that converted one-dimensional matrices to vectors.

State $(s_1, s_2, s_3, d_1, d_2)$	ϵ-optimal action	ϵ-optimal value	Linear approx. action	Linear approx. value
(2,1,2,4,2)	(0,0,1,1)	63.39	(0,0,0,2)	66.06
(2,3,0,2,4)	(0,0,0,4)	46.64	(0,0,0,4)	48.11
(1,1,0,2,4)	(0,1,1,2)	28.98	(0,1,0,3)	31.45
(1,2,2,2,3)	(0,1,0,2)	30.58	(0,1,0,2)	33.09
(0,0,0,4,4)	(0,0,2,2)	30.98	(0,0,0,4)	33.04
(0,0,0,2,4)	(0,2,1,1)	19.22	(0,2,0,2)	19.82

Table 9.1: This table gives results for the value iteration solution (columns 2 and 3) and that found using LSVI with a linear value function approximation (columns 4 and 5) for the instance described in the text. The first component of the action, a_0, corresponds to Class 2 demand assigned to overtime and the next three components of the action (a_1, a_2, a_3) represent the number of units of Class 2 demand scheduled on day $i = 1, 2, 3$.

In contrast, the greedy policy based on the linear approximation schedules the 3 units of Class 2 demand to day 3, perhaps causing challenges in the future.

9.6.3 LSVI with linear value function approximation

LSVI was used to obtain parameter estimates based on a linear architecture and basis functions that are linear in the state. That is,

$$\hat{v}((s_1, s_2, s_3, d_1, d_2)) = \beta_0 + \beta_1 s_1 + \beta_2 s_2 + \beta_3 s_3 + \beta_4 d_1 + \beta_5 d_2. \tag{9.43}$$

The advantages of using this approximation are:

1. It is easy to obtain parameter estimates using readily available linear regression packages.
2. The number of quantities to approximate and store is reduced from the 3125 components of $\mathbf{v}$ (in its tabular representation) to the six components of $\boldsymbol{\beta}$.
3. The parameter estimates provide insight into the effect demand and capacity have on cost. In (9.43), $\beta_i, i = 1, 2, 3$ represents the *marginal cost* of one less unit of capacity on day i, and β_4 and β_5 represent the marginal costs of an extra unit of Class 1 and Class 2 demand.
4. Greedy actions corresponding to the approximate value function can be easily determined.
5. Parameter estimates for this approximation can also be found using linear programming.

LSVI[26] as stated in Algorithm 9.3 was implemented as follows. It was initialized

[26]LSVI was used instead of LSPI because it was simpler to code and converged quickly.

with $\boldsymbol{\beta} = \mathbf{0}$ and $\epsilon = 0.0001$. Steps 2(a) and 2(b) were implemented component-wise instead of in vector form. Step 2(a) used representation (9.43) instead of the matrix $\mathbf{B}$. The right hand side of step 2(b) becomes

$$\min_{(a_0,a_1,a_2,a_3)\in A_{(s_1,s_2,s_3,d_1,d_2)}} \Big\{ c\big((s_1,s_2,s_3,d_1,d_2),(a_0,a_1,a_2,a_3)\big) + \lambda \sum_{j=0}^{4}\sum_{k=0}^{4} p_j q_k \hat{v}\big((s_2+a_2,s_3+a_3,0,j,k)\big) \Big\}. \tag{9.44}$$

After substituting the linear approximation, the summation above becomes

$$\begin{aligned} &\sum_{j=0}^{4}\sum_{k=0}^{4} p_j q_k \hat{v}((s_2+a_2,s_3+a_3,0,j,k)) \\ &\quad = \sum_{j=0}^{4}\sum_{k=0}^{4} p_j q_k \big(\beta_0 + \beta_1(s_2+a_2) + \beta_2(s_3+a_3) + \beta_3 0 + \beta_4 j + \beta_5 k\big) \\ &\quad = \beta_0 + \beta_1(s_2+a_2) + \beta_2(s_3+a_3) + \beta_4 E(D_1) + \beta_5 E(D_2), \end{aligned} \tag{9.45}$$

where the random variables D_1 and D_2 represent the amount of Class 1 and Class 2 demand, respectively. Since after substitution, several of the expressions in (9.44) do not involve the action explicitly, the greedy action based on the linear approximation chooses

$$(\hat{a}_0,\hat{a}_1,\hat{a}_2,\hat{a}_3) \in \operatorname*{arg\,min}_{(a_0,a_1,a_2,a_3)\in A_{(s_1,s_2,s_3,d_1,d_2)}} \Big\{ Ca_0 + c_1 a_1 + (c_2+\lambda\beta_1)a_2 + (c_3+\lambda\beta_2)a_3 \Big\} \tag{9.46}$$

The example was solved using the representation (9.44) to allow easy generalization to other approximations for $\hat{v}(\cdot)$. LSVI converged in 13 iterations. Curiously, convergence of the quantity $\|\boldsymbol{\beta}' - \boldsymbol{\beta}\|_2$ was not monotone; at one iteration the norm of the difference $\boldsymbol{\beta}' - \boldsymbol{\beta}$ exceeded that at a previous iteration. This is consistent with the observation above that the LSVI operator need not be a contraction mapping.

The linear approximation to the optimal value function obtained by substituting the value of $\hat{\boldsymbol{\beta}}$ is given by

$$\hat{v}\big((s_1,s_2,s_3,d_1,d_2)\big) = -43.98 + 12.59 s_1 + 6.54 s_2 + 4.28 s_3 + 12.60 d_1 + 8.98 d_2. \tag{9.47}$$

Observe that all coefficients (except the constant) are positive meaning that the greater the occupancy or demand, the greater the expected discounted cost. More importantly, the expected discounted cost of an extra unit of occupancy decreases from day 1 to day 3 and an extra unit of Class 1 demand contributes more to the expected discounted cost than a unit of Class 2 demand because of the greater flexibility in scheduling Class 2 demand.

The structure of a greedy policy based on the linear approximation can be obtained as follows. Recall that the daily capacity is $M = 4$ units, overtime cost is $C = 20$ and the cost of scheduling a unit of Class 2 demand on day $n = 1, 2, 3$ is $2n$. Substituting the parameter estimates in (9.47) into (9.46) implies that

$$(\hat{a}_0, \hat{a}_1, \hat{a}_2, \hat{a}_3) \in \underset{(a_0,a_1,a_2,a_3)\in A_{(s_1,s_2,s_3,d_1,d_2)}}{\arg\min} \{20a_0 + 2a_1 + 10.21a_2 + 10.06a_3\}. \tag{9.48}$$

where $10.21 = 4+0.95\cdot 6.54$ and $10.06 = 6+0.95\cdot 4.28$. The action set can be expressed as

$$A_{(s_1,s_2,s_3,d_1,d_2)} = \left\{ (a_0, a_1, a_2, a_3) \;\middle|\; a_0 \geq 0; a_1 = (4 - s_1 - d_1)^+; \right. \\ \left. 0 \leq a_n \leq 4 - s_n, n = 2, 3; \sum_{n=0}^{3} a_n = d_2 \right\}. \tag{9.49}$$

Observe further that the arg min only involves the current state through its constraints.

As a result of (9.48) and (9.49) it follows that the greedy action based on the linear approximation has the following structure:

> In each state in which $d_2 > 0$, assign as much Class 2 demand as possible to day 1, then assign as much as possible to day 3, then assign as much as possible to day 2. Only assign Class 2 demand to overtime when there is no other capacity available.

This policy can be summarized succinctly as: after using up capacity on day 1, fill capacity starting from the end of the booking horizon and working backwards to the current day. Note that this characterization avoids the need to compute the optimal action in each state although it is done so in Table 9.1 for comparison.

9.6.4 LSVI with quadratic value function approximation

LSVI was also applied using a value function approximation of the form:

$$\hat{v}((s_1, s_2, s_3, d_1, d_2)) = \beta_0 + \beta_1 s_1 + \beta_2 s_2 + \beta_3 s_3 + \beta_4 d_1 + \beta_5 d_2 \\ + \beta_6 s_1^2 + \beta_7 s_2^2 + \beta_8 s_3^2 + \beta_9 d_1^2 + \beta_{10} d_2^2.$$

A more general quadratic model would include 10 additional cross product terms of the form $s_i s_j$, $s_i d_j$ and $d_1 d_2$. Since the model is linear in the parameters, they can be easily estimated using linear regression. In contrast to the approximation in the previous section, the marginal costs vary with the state vector. For example the marginal cost

of an additional unit of Class 2 demand would be $\beta_5 + 2\beta_{10}d_2$. Again LSVI converged in 13 iterations and the norms of the differences of successive parameter estimates were not monotone.

The estimated value function is given by

$$\hat{v}((s_1, s_2, s_3, d_1, d_2)) = -34.93 + 5.93s_1 + 3.55s_2 + 1.92s_3 + 5.93d_1 + 5.07d_2 + 1.68s_1^2 + 0.85s_2^2 + 0.70s_3^2 + 1.68d_1^2 + 1.04d_2^2. \quad (9.50)$$

Using the same approach as in the case of the linear value function approximation, substituting $\hat{v}(s)$ into the right hand side of the Bellman equation, the analogous expression to (9.48), after some algebra, becomes

$$20a_0 + 2a_1 + \left(7.37a_2 + 0.81(s_2 + a_2)^2\right) + \left(7.82a_3 + 0.66(s_3 + a_3)^2\right). \quad (9.51)$$

Given a state, choosing the action to minimize this expression gives the greedy action. Since the action set is finite, this minimization can be done by enumeration. For example, in state $(2, 1, 1, 2, 1)$ there are three feasible actions as shown in Table 9.2. Evaluating the above expression for each action shows that the greedy action is $(0, 0, 1, 0)$; that is, assign the single unit of Class 2 demand to day 2. This contrasts with the action based on the linear approximation which would assign this unit of Class 2 demand to day 3.

Action	Value of (9.51)
$(1, 0, 0, 0)$	20.00
$(0, 0, 1, 0)^*$	11.21
$(0, 0, 0, 1)$	11.27

Table 9.2: Greedy action choice in state $(2, 1, 1, 2, 1)$ based on quadratic value function approximation. The greedy action is indicated with an asterisk.

9.6.5 Comparison of policies and values

The three left most columns of Table 9.1 above give the action chosen by the ϵ-optimal policy and its value in some selected high demand states. Observe that in these states, the policy does not assign Class 2 demand to the earliest day possible, instead it **reserves** capacity for future Class 1 demand. For example, in state $(2, 1, 2, 4, 2)$, all day 1 capacity is used by Class 1 demand and the two units of Class 2 demand are split between days 2 and 3. In state $(0, 0, 0, 4, 4)$, all Class 1 demand is served on day 1 and the 4 units of Class 2 demand are split between days 2 and 3. Moreover in these states Class 2 demand is never served with overtime.

Columns 4 and 5 of Table 9.1 show the greedy action obtained using the linear value function approximation and the value $\hat{v}$. Observe that in several of these states, the action based on the linear approximation differs from that using the ϵ-optimal policy.

In cases where it differed, it assigned the Class 2 demand to later appointments[27]. For example in state $(0, 0, 0, 4, 4)$ the policy chosen using the linear approximation assigned **all** Class 2 demand to day 3 while the ϵ-optimal policy split it between days 2 and 3. Overall, the action based on the linear approximation differed from that chosen by the ϵ-optimal policy in 632 out of 3125 states.

Remarkably, the greedy actions chosen using the quadratic approximation were **identical** to those corresponding to the ϵ-optimal policy. Moreover the RMSE of the difference between the ϵ-optimal value and that found using a linear approximation was 1.99 and the RMSE of the difference between the ϵ-optimal value and that found using a quadratic approximation was 1.11. Hence the quadratic approximation fit was considerably better and might be appropriate in larger instances of the model.

9.6.6 Further insights

This section provides some additional insights that can be gained by investigating other aspects of the model and solution methods.

Using overtime strategically

To test whether overtime was ever used strategically, overtime costs were chosen to be class specific. Setting the overtime cost for Class 1 to 40 and for Class 2 to 10, made it more essential to reserve capacity for Class 1 demand. For example the ϵ-optimal policy in state $(2, 3, 0, 2, 4)$ chose action $(2, 0, 0, 2)$. This means that it assigns 2 units of Class 2 demand to overtime and 2 units as late as possible, that is to day 3. Thus the ϵ-optimal policy uses overtime strategically by reserving capacity for Class 1 demand.

Note that the greedy action corresponding to the linear value function approximation remained $(0, 0, 0, 0, 4)$ but the quadratic approximation again identified the optimal action. This provides further support for using a quadratic approximation.

State sampling

In the above analysis, LSVI approximations were based on estimating parameters using value function updates for all states. However, good results may be possible if LSVI is applied to only a subset of the states. More importantly, in larger models, obtaining approximations by updating the above recursion in all states would be computationally prohibitive.

The advantage of state sampling is that it reduces the number of times that the Bellman operator

$$L\hat{\mathbf{v}}(s) = \min_{a \in A_s} \left\{ c(s, a) + \lambda \sum_{j \in S} p(j|s, a)\hat{v}(j) \right\}$$

[27] This observation is similar to the policy found by Patrick et al. [2008] using linear programming with a linear value function approximation.

needs to be evaluated at each iteration of LSVI.

The following offline approach was used to apply LSVI using a subset of S. A subset of states was selected by:

1. specifying the fraction of states to sample,
2. randomly sampling the specified number of states,
3. discarding duplicates[28], and
4. applying LSVI as described in Section 9.3.1 using the designated sample.

Thus after selecting a random subset of states, LSVI uses it for all iterations. The next two chapters consider online versions of these algorithms, which sample states sequentially and avoid computing the expectation $\sum_{j\in S} p(j|s,a)\hat{v}(j)$.

The following analysis is based on using the above quadratic approximation. It applies LSVI to a large number of samples, in which the fraction of states sampled was varied between 0.1 to 0.9. The quality of fit was compared on the basis of

$$\text{RMSE} = \left(\frac{1}{11}\sum_{i=0}^{10}(\hat{\beta}_i - \hat{\beta}_i^{\text{all}})^2\right)^{0.5},$$

where β_i^{all} denotes the parameter estimates based on using all states.

Results of this experiment appear in Figure 9.15, which shows two things:

1. the average quality of the fits improve with sample size, and
2. there is considerable variability between samples.

We speculate that by instead choosing the state samples to be representative of the entire state space, much of this between sample variability can be reduced. Moreover, it appears that the accuracy based on sampled fractions greater than 0.6 are similar.

The quadratic value function approximation based on a random sample of half of the states is given by

$$\begin{aligned}\hat{v}((s_1,s_2,s_3,d_1,d_2)) = -36.86 + 5.61s_1 + 3.75s_2 + 2.21s_3 + 7.21d_1 + 5.12d_2 \\ + 1.73s_1^2 + 0.80s_2^2 + 0.70s_3^2 + 1.37d_1^2 + 1.03d_2^2. \quad (9.52)\end{aligned}$$

Comparing it to (9.50) obtained using **all** states shows that the estimates are reasonably close and the order of magnitude of the corresponding parameters is similar. Derivation of the corresponding greedy policy and comparison to that obtained using all states is left to the reader.

[28] Since there is no variability in this approach other than state sampling, each replication of state s would yield the same value for $L\hat{\mathbf{v}}(s)$. Thus, it was unnecessary to evaluate the same state more than once. In a more general simulation approach, this would not be the case.

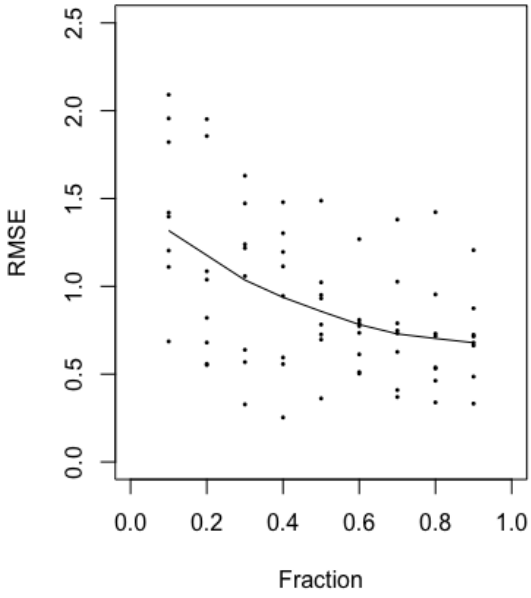


Figure 9.15: RMSE as a function of the fraction of states sampled over multiple state samples. The dots correspond to a specific sample and the line is a Lowess smoothing of the individual RMSEs.

Numerical example: Neural network approximation

Obtaining good results using a neural network approximation proved challenging when using the package "neuralnet" in R. When incorporated in LSVI, the algorithm either oscillated and did not satisfy a stopping criterion or the parameter estimation algorithm did not converge. Several alternatives were explored. Most notably, when the ϵ-optimal value function on the entire state space was approximated by several different neural network structures, the greedy policy was similar to that found with a linear value function approximation. In this application, we believe that neural networks **do not** provide an attractive approximation architecture but leave it to the reader to prove the supposition false.

Approximate linear programming

An analysis based on linear programming is not presented here. The main challenge was to construct constraint matrices. This model has been analyzed in considerable detail in Patrick et al. [2008]. Therein, the linear program was solved using column generation on the dual, which contained few rows and many columns. The coefficients for the linear approximation were obtained from the corresponding primal solution.

This work identified the same greedy policy as described above for the LSVI solution of the linear approximation.

9.7 Application: Golf strategy

Often, in real-life situations, transition probabilities, reward functions and the policy used are not known. This section provides a case study of the estimation of a value function based on multiple realizations of finite sequences of states and rewards that terminate when a *goal state* is reached. This case study, drawn from the game of golf, illustrates this approach and provides a transition to the next two chapters.

9.7.1 Value functions and strokes gained in golf

This section describes dynamic programming based concepts developed by Broadie [2014] and now widely used in all levels of competitive golf.

Golf as a Markov decision process

Golf is a game where players strive to propel a 1.68-inch diameter golf ball into a 4.25-inch diameter cup (often referred to as a "hole") in the fewest number of strokes. A typical golf course comprises 18 holes, each featuring a tee box, a putting green (where the cup is located), a fairway, and possibly some rough, sand traps, obstructed areas, out-of-bounds areas, and natural obstacles like woods, deserts, or water. The distance from the tee to the cup typically ranges from 100 to 600 yards.

Play begins from the tee, and subsequent shots are played from the point where the previous shot came to rest. A penalty stroke is added if a ball is lost or deemed unplayable. Playing **each hole** in a round of golf may be viewed as a Markov decision process as follows.

Decision epochs: Decision epochs correspond to the instance before a shot is hit:

$$T = \{1, 2, \ldots\}.$$

Note that an infinite horizon formulation is appropriate because the number of shots required to complete a hole of golf is not fixed; it is determined by the quality of play. In theory it could take an arbitrarily large number of strokes to hit the ball into the hole from any location of the golf course. In reality this is not supported by data but often it might seem like the case to a frustrated golfer[29].

[29] In men's professional (PGA) golf, the highest recorded hole score to date is 16 in 2011 by Kevin Na at the Valero Open.

States: The state represents the distance from the hole $\delta \in \mathcal{D}$ and the terrain type $\tau \in \mathcal{T}$ (fairway, rough, etc.) where the ball lies immediately before a shot is taken. Note that the state $(0, \text{green})$ is the zero-reward absorbing state corresponding to the ball being "in the hole". Therefore,

$$S = \{\mathcal{D} \times \mathcal{T}\}.$$

On most golf courses $\mathcal{D} = \{100, 101, \ldots, 600\}$ and

$$\mathcal{T} = \{\text{tee, fairway, rough, sand trap, hazard, obstructed, green}\}.$$

Actions: Each action represents the type of shot a player tries to hit and its intended target. These vary by location, wind and course configuration.

$$A_{(\delta,\tau)} = \{\text{Possible targets and shot types from } (\delta, \tau)\}$$
$$A_{(0,\text{green})} = \{a_0\},$$

where a_0 indicates that the hole is completed.

Elements of $A_{(\delta,\tau)}$ can be thought of as capturing the choice of a golfer to aim the ball in a certain direction, to hit the ball at a certain speed and angle, and to attempt to give the shot a particular curvature. Note that at this level, shot types are not described explicitly because they vary greatly with the skill of the player and the terrain. For example, if the ball is behind a tree then the player may hit a recovery shot. If the ball is on the green, the player will putt. In the fairway, the player may pick a target 10 feet short of the hole and attempt to hit a low draw[30].

Transition probabilities: The transition probabilities $p((\delta', \tau')|(\delta, \tau), a)$ are determined from a probability distribution of shot outcomes given the target and shot type. One would expect that shots from closer to the hole will have less variable outcomes than shots from greater distances.

Since the hole terminates when $(0, \text{green})$ is reached, $p((0, \text{green})|(0, \text{green}), a_0) = 1$.

The assumption that the transition probability depends only on the current state and action makes sense since given the location of the ball and its terrain, the outcome of the current shot should not depend on the outcome of previous shots[31].

Rewards: It requires one stroke to hit a shot so that:

$$r((\delta, \tau), a, (\delta', \tau')) = -1 \quad \text{for all} \quad (\delta, \tau) \in S$$

and

$$r((0, \text{green}), a_0, (0, \text{green})) = 0.$$

[30] A shot that curves from the right to the left for a right-handed golfer.

[31] Unless of course the golfer is really frustrated after hitting a poor shot, since mood may affect execution of future shots.

Because the objective in golf is to minimize the number of shots on each hole, the reward represents the "cost" of a shot. Because this is formulated as an infinite horizon model, the state $(0, \text{green})$ is the zero-cost (reward-free) absorbing state.

Note that it is preferable to model this as a minimum cost problem, in which case $r((\delta, \tau), a, (\delta', \tau')) = -1$ is replaced by $c((\delta, \tau), a, (\delta', \tau')) = 1$.

Value functions in golf

Define the value of each location as the expected number of shots to "hole out", that is get the ball into the cup, from that location. Both the distance and terrain affect this value, that is

$$v(\delta, \tau) = \text{the expected number of strokes to hole out from distance } \delta \text{ and terrain } \tau.$$

Of course $v(\delta, \tau) \geq 0$ and $v(0, \text{green}) = 0$. Moreover for a fixed τ, $v(\delta, \tau)$ should be non-decreasing in δ.

As an example, for an average male professional golfer[32],

$$v(100, \text{fairway}) = 2.80 \quad \text{and} \quad v(100, \text{rough}) = 3.02.$$

This means that the expected number of shots for an average male professional golfer to hole out from 100 yards in the fairway is 2.80 and from 100 yards in the rough is 3.02. Thus, on average a player adds 0.22 shots if a shot ends up in the rough as opposed to the fairway. This may not sound like much but over a tournament made up of four 18 hole rounds, missing the fairway several times can have a significant impact on the player's final score.

The value function can be overlaid on a picture of a hole to show the expected number of shots from each location (see Figure 9.16). Such a figure can also guide player strategy as discussed below. Knowing this value from every distance and terrain is fundamental to developing a performance metric referred to as *strokes gained*, described below.

A case study

The following case study is based on Marty's collaboration with the University of British Columbia (UBC) women's golf team. The objective of this study was to establish value functions for female golf team members. Unlike in the case of male professional golfers, such specialized information was not available at the time of the study.

The following approach was used to estimate values of shots from the fairway for distances between 10 and 350 yards from the hole. Note that the data contained shots from all terrains, however for illustrative purposes only shots from the fairway are analyzed here. Therefore the component τ will be dropped from the state description

[32] Broadie [2014] p. 85.

Figure 9.16: Graphical representation of golf value functions for male professional golfers based on Broadie [2014]. For example, on average it requires 2.8 shots to hole out from anywhere on the black arc at 100 yards from the green.

for the remainder of this section. Hence, the objective of this analysis is to estimate $v(\delta)$; the expected number of shots to hole out from distance δ on the fairway.

The complete data set, which contained the starting location (state) and ending location (state) for 10,100 shots executed during the 2016/2017 golf season, was entered by team members in a custom application. It was manipulated to create a data set consisting of a large number of pairs of distances and number shots to "hole out" from those distances. For example, the data point $(\delta, v) = (200, 4)$ indicates that for a particular player, on a particular hole in a particular round, it required 4 shots to "hole out" from 200 yards on the fairway.

A value function approximation

The value function was approximated by a linear combination of the basis functions $b_0(\delta) = 1, b_1(\delta) = \delta, b_2(\delta) = \delta^2$ and $b_3(\delta) = \delta^3$. That is, it was approximated by a cubic polynomial. In contrast to other examples in this chapter based on an MDP model,

an observed value was represented by

$$v(\delta) = \beta_0 + \beta_1\delta + \beta_2\delta^2 + \beta_3\delta^3 + \epsilon, \tag{9.53}$$

where ϵ represented an unobservable random disturbance.

The reason for developing an approximation using basis functions as opposed to separate estimates of $v(\delta)$ for each δ based on the average of shots from that distance was to account for data sparsity and to obtain a value function approximation that was non-decreasing in distance. For some distances, there were many records while for others there were none. Moreover, it made sense that the further from the hole, the greater the number of strokes it should take to hole out so that $v(\delta)$ should be non-decreasing in δ.

Using least squares regression resulted in the following fitted model:[33]

$$\hat{v}(\delta) = 1.95 + 0.015\,\delta - 0.000040\,\delta^2 - 0.000000049\,\delta^3. \tag{9.54}$$

The data together with the fitted model are shown in Figure 9.17. Note that more complicated models or including random effects had little effect on estimated values.

Based on the fitted model, $\hat{v}(100) = 3.11$, which is 0.31 higher than for a male professional golfer from that location or equivalent to the value for a male professional from 185 yards.

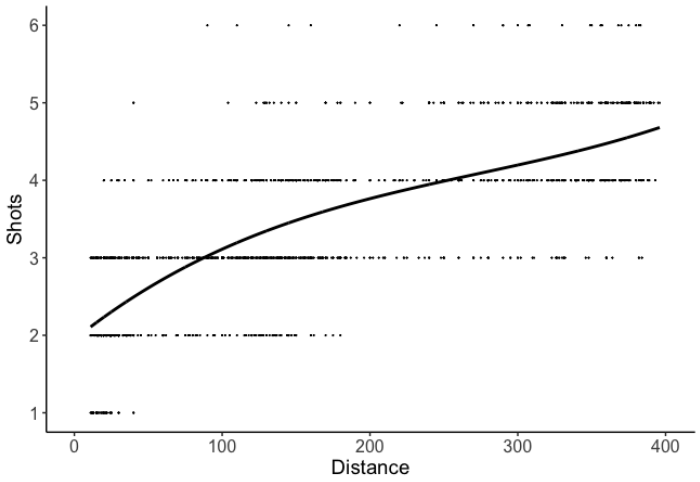


Figure 9.17: Data and fitted model for (9.54). The data values are discrete since they represented the number of shots required to "hole out" from each distance on the fairway. Note that the fitted function is monotone increasing.

[33] An alternative is to fit a Poisson regression since the dependent variable represents a count. Thanks to Antoine Sauré for this suggestion.

Golf strategy based on value functions

In the context of the Markov decision process framework, the value functions can be used to determine strategy. Strategic choices in golf include anticipated shot distance and shot type. Outcomes are impacted by shot dispersion patterns represented by probability distributions. Shorter shots have less dispersion than longer shots.

As an example, consider the option of a shot from the fairway when the hole is 240 yards away and there is a water hazard in front of the green. Suppose the player considers two choices. Choice 1 is to "go for the green", which means to be aggressive and try to reach green in one shot. Choice 2 is to "lay up", which means to be conservative and hit a shorter shot that most likely avoids the water, but will require a subsequent shot to get on the green.

To simplify exposition, suppose that under Choice 1 if the player aims at the green and lands on it, the ball ends up 50 feet from the hole which occurs with probability p. The value from that point would be $v(50, \text{green}) = 2.14$[34]. However, if the ball ends up in the water, which occurs with probability $1 - p$, the player incurs a one stroke penalty and must hit his next shot from 60 yards in the fairway where the value is $v(60, \text{fairway}) = 2.70$. Thus the expected value of the action "go for the green" is

$$v_1 = 2.14p + (2.70 + 1)(1 - p) = 3.70 - 1.56p.$$

Suppose the alternative action is to aim for a target 80 yards from the green where there is a 0.9 chance of being close to that target where $v(80, \text{fairway}) = 2.75$ and a 0.1 chance of being in the rough at the same distance where $v(80, \text{rough}) = 2.96$. The expected value of the action "lay up" is $v_2 = 2.77$. Since the goal is to minimize the number of strokes taken to hole out, the decision to go for the green is optimal if $v_1 \leq v_2$, otherwise the golfer should lay up. Therefore noting that $3.70 - 1.56p = 2.77$ implies $p = 0.596$, the optimal decision rule is

$$d(240, \text{fairway}) = \begin{cases} \text{go for green}, & p \geq 0.596 \\ \text{lay up}, & p < 0.596. \end{cases}$$

Hence if the player thinks there is greater than a 59.6% chance of hitting the green, then he[35] should go for the green; otherwise he should lay up. In reaching this decision, a player must take into account factors including the weather, the lie in the fairway and his stress level.

Using value functions: Strokes gained

Suppose a golfer scores a 3 on a par 4 hole. To which shot (or shots) can this outcome be attributed? Was it the result of an outstanding drive, a precise approach shot or

[34] Distance on the green are measured in feet and distances from other terrain are measured in yards.
[35] The calculations above use men's professional values from p. 85 of Broadie [2014].

an excellent putt or was it some combination of these shots? This is the question that the concept of *strokes gained* seeks to answer.

Determining which action contributes to success in an episodic model is referred to in the machine learning literature to as the *the credit assignment problem*[36]. This challenge arises in domains such as the games chess and Go, robotic navigation and natural language processing. Value functions and state-action value functions provide a way to approach this problem in general.

Solving the credit assignment problem is crucial for effective learning. If a learning system cannot assign credit (or blame) to actions, then it will not be able to learn which actions are beneficial and which are detrimental. Without being able to make this distinction, the learner will not be able to improve performance over time.

Returning to golf, *strokes gained* measures the difference between the expected value of a shot and its result. Let X_D be a random variable denoting the distance of the ball to the hole after the current shot and X_T be a random variable denoting the terrain of the ball after the current shot. Also, let

$$(\delta', \tau') = \text{Actual location of the ball after the current shot}$$
$$(\delta, \tau) = \text{Actual location of the ball immediately prior to the current shot.}$$

Then strokes gained, SG[37], is defined as

$$\text{SG}(\delta, \tau) = E_{(\delta,\tau)}[v(X_D, X_T)] - v(\delta', \tau'). \tag{9.55}$$

Recalling the MDP formulation of golf above, the policy evaluation equation

$$v(s) = r(s) + \sum_{j \in S} p(j|s)v(j)$$

expressed in terms of random variables and costs becomes

$$v(\delta, \tau) = 1 + E_{(\delta,\tau)}[v(X_D, X_T)]. \tag{9.56}$$

So rearranging terms and substituting (9.56) into (9.55) yields

$$\text{SG}(\delta, \tau) = v(\delta, \tau) - (v(\delta', \tau') + 1) \tag{9.57}$$

The first quantity on the right hand side of (9.57) denotes the expected number of shots to hole out prior the current shot. The second quantity is the expected number of shots to hole out from where the current shot ended up plus the "cost" of 1 stroke to reach this point.

[36] A nice discussion of this issue appears in Bennett [2023].

[37] see Broadie [2014]

To interpret this quantity, recall that the goal in golf is to *minimize* the expected number of strokes to hole out. If the player hits a better than average shot, $v(\delta', \tau') + 1$ will be less than $v(\delta, \tau)$ and strokes gained will be positive. If the shot outcome is worse than average, strokes gained will be negative. When a player hits an "average" shot, $v(\delta, \tau) = v(\delta', \tau') + 1$ so that strokes gained will be zero.

Example 9.13. To illustrate this concept consider the following scenario where values are based on the fitted curve (9.54). Suppose a women's golf team member is at 275 yards in the fairway, then $v(275, \text{fairway}) = 4.09$. After hitting an average shot from this location, her strokes gained would be $4.09 - 1 = 3.09$. This is equivalent to a distance of 98 yards in the fairway, so an average shot to the fairway would be $275 - 98 = 177$ yards. Hitting the ball to 75 yards in the fairway results in a value of 2.88 and the corresponding strokes gained is $4.09 - 2.88 - 1 = 0.21$. If instead she hits the ball to 125 yards in the fairway, the value is 3.31 corresponding to a strokes gained of -0.22. Note that hitting the ball into the rough or a sand trap at 125 yards would result in more negative stokes gained.

This calculation ignores the possibility that player may have made a strategic choice to lay up to specific distance so as to avoid a hazard.

From the credit assignment problem perspective, strokes gained averaged over several rounds identifies a player's strengths and weaknesses and which areas need improvement. Hence, this information can be used to develop a practice plan. For example, if average strokes gained for putts in the range 8 to 12 feet is negative, then this is an area where the player needs to improve. This was the objective of the UBC golf team project.

9.8 Technical appendix: Regression and nonlinear optimization

To effectively apply and interpret value function approximation, familiarity with the basics of linear regression as described in this appendix is crucial.

9.8.1 Linear regression

Given observations of a dependent variable y_j and independent variables $x_{j,1}, \ldots, x_{j,I}$ for $j = 1, \ldots, J$, the objective of linear regression is to find a set of *regression coefficients*, *weights* or *parameters* β_i, $i = 0, \ldots, I$ so that

$$f(x_{j,1}, \ldots, x_{j,I}; \beta_0, \ldots, \beta_I) := \beta_0 + \beta_1 x_{j,1} + \cdots + \beta_I x_{j,I} \tag{9.58}$$

well approximates y_j for $j = 1, \ldots, J$. This is referred to as a *linear* regression problem because $f(x_{j,1}, \ldots, x_{j,I}; \beta_0, \ldots, \beta_I)$ is a linear function of its parameters.

The standard approach[38] to estimation is to obtain parameters that minimize the sum of squared errors

$$g(\beta_0, \ldots, \beta_I) := \sum_{j=1}^{J} \left(y_j - (\beta_0 + \beta_1 x_{j,1} + \cdots + \beta_I x_{j,I})\right)^2 \tag{9.59}$$

$$= \sum_{j=1}^{J} g_j(\beta_0, \ldots, \beta_I)^2 \tag{9.60}$$

where

$$g_j(\beta_0, \ldots, \beta_I) := y_j - (\beta_0 + \beta_1 x_{j,1} + \cdots + \beta_I x_{j,I})$$

for $j = 1, \ldots, J$.

The function $g_j(\cdot)$ is introduced to provide more elegant expressions and simplify the derivation of Gauss-Newton iteration below. Note that this notation expresses dependence on observation $(y_j, x_{j,1}, \ldots, x_{j,I})$ through the subscript j.

Parameters that minimize $g(\beta_0, \ldots, \beta_I)$ are referred to as *least squares* or *ordinary least squares (OLS)* estimates. Using a matrix formulation leads to a closed form representation. Parameter estimates are often denoted by $\hat{\beta}_i, i = 0, \ldots, I$.

Note that the statistical literature formulates the regression problem in the context of the statistical model

$$y_j = \beta_0 + \beta_1 x_{j,1} + \cdots + \beta_I x_{j,I} + \epsilon_j, \tag{9.61}$$

where ϵ_j represents an *unobservable* random disturbance (or error) drawn from a distribution with mean 0 and constant variance σ^2. Furthermore, it is usually assumed that $E[\epsilon_j \epsilon_k] = 0$ for $j \neq k$, that is, that the random disturbances are uncorrelated between observations. When in addition, the random disturbances are assumed to be normally distributed, the least squares estimates of β_i, $i = 0, \ldots, I$ are also *maximum likelihood estimates* and consequently have many desirable theoretical properties that allow for statistical inference, such as the computation of confidence intervals and hypothesis testing. Usually, these statistical concepts are not taken into account in approximate dynamic programming.

When the errors are assumed to be normal, the above model is often expressed as

$$y_j \sim NID(\beta_0 + \beta_1 x_{j,1} + \cdots + \beta_I x_{j,I}, \sigma^2), \tag{9.62}$$

where the expression $NID(\mu, \sigma^2)$ corresponds to a set of independent normally distributed random variables with mean μ and variance σ^2. From this perspective, the mean of y_j varies from observation to observation and the variance is constant. The beauty of writing the model this way is that it allows generalization to other distributions such as Poisson or binomial, or different non-constant variance, or auto-correlated errors.

[38] Other choices for $g(\cdot)$ can be used including sums of absolute values or other weighting functions that may downweight outlying observations.

9.8.2 Matrix formulation

The most elegant approach for analyzing regression models is through its matrix formulation. It is widely used in approximate dynamic programming and reinforcement learning, and leads to elegant formulae for predicted values and parameter estimates. To do so define the following vectors and matrices:

$$\mathbf{y} := \begin{bmatrix} y_1 \\ \cdot \\ \cdot \\ \cdot \\ y_J \end{bmatrix}, \ \mathbf{X} := \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,I} \\ \cdot & \cdot & \dots & \cdot \\ \cdot & \cdot & \dots & \cdot \\ \cdot & \cdot & \dots & \cdot \\ 1 & x_{J,1} & \dots & x_{J,I} \end{bmatrix}, \ \boldsymbol{\beta} := \begin{bmatrix} \beta_0 \\ \cdot \\ \cdot \\ \cdot \\ \beta_I \end{bmatrix} \text{ and } \boldsymbol{\epsilon} := \begin{bmatrix} \epsilon_1 \\ \cdot \\ \cdot \\ \cdot \\ \epsilon_J \end{bmatrix}. \tag{9.63}$$

In (9.63), $\mathbf{y}$ denotes the $J \times 1$ column vector of observations of the dependent variable, $\mathbf{X}$ is the $J \times (I+1)$ matrix of values of the independent variables, $\boldsymbol{\epsilon}$ denotes the $J \times 1$ column vector of random disturbances and $\boldsymbol{\beta}$ is the $(I+1) \times 1$ column vector of parameters. It is also convenient to denote the j-th row of $\mathbf{X}$ by the vector $\mathbf{x}_j$, that is

$$\mathbf{x}_j := (1, x_{j,1}, \dots, x_{j,I})$$

for $j = 1, \dots, J$. When written as a row vector, it will be expressed as $\mathbf{x}_j^\mathsf{T}$.

Using this matrix notation, the data generating model, which provides an elegant way of representing (9.61) for all j in a single equation, is given by

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}, \tag{9.64}$$

where $E[\boldsymbol{\epsilon}] = \mathbf{0}$ and $\boldsymbol{\Sigma} := \text{cov}[\boldsymbol{\epsilon}] = \sigma^2\mathbf{I}$.

The squared error criterion can be expressed in matrix form as

$$g(\boldsymbol{\beta}) = \sum_{j=1}^{J} g_j(\boldsymbol{\beta})^2 = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\mathsf{T}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = \mathbf{y}^\mathsf{T}\mathbf{y} - 2\boldsymbol{\beta}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{y} + \boldsymbol{\beta}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{\beta}. \tag{9.65}$$

Note that $g(\boldsymbol{\beta})$ is often written as

$$g(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2. \tag{9.66}$$

Computing the gradient of $g(\boldsymbol{\beta})$ and setting it equal to zero shows that $\hat{\boldsymbol{\beta}}$, the least squares estimator of $\boldsymbol{\beta}$, satisfies the "normal equation"

$$\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{\beta} = \mathbf{X}^\mathsf{T}\mathbf{y}. \tag{9.67}$$

When $\mathbf{X}^\mathsf{T}\mathbf{X}$ is invertible[39] the least square parameter estimates can be written in closed form as:

[39] This is true when the columns of $\mathbf{X}$ are linearly independent. Otherwise they are said to be *collinear*. In regression theory this phenomenon is referred to as *multi-collinearity*; several approaches, most notably ridge regression, have been developed to address this issue.

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{y} := \boldsymbol{\Gamma}\mathbf{y}. \tag{9.68}$$

Note that $\hat{\boldsymbol{\beta}}$ determined by either (9.67) or (9.68) can also be written as

$$\hat{\boldsymbol{\beta}} \in \underset{\boldsymbol{\beta}\in\mathbb{R}^{I+1}}{\arg\min} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2. \tag{9.69}$$

The vector of predicted (fitted) values $\hat{\mathbf{y}}$ can be expressed as:

$$\hat{\mathbf{y}} = \mathbf{X}(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{y} := \boldsymbol{\Pi}\mathbf{y}. \tag{9.70}$$

The matrices $\boldsymbol{\Gamma}$ and $\boldsymbol{\Pi}$ defined in (9.68) and (9.70) are fundamental to the discussion of iterative algorithms in this chapter. The matrix $\boldsymbol{\Pi}$ is sometimes referred to as the *hat matrix* because it maps the vector of observed values $\mathbf{y}$ into the vector $\hat{\mathbf{y}}$ of fitted values; that is, it puts a "hat" on $\mathbf{y}$. It is also referred to as a *projection matrix* for reasons described below.

The following example illustrates this notation.

Example 9.14. Quadratic regression: Suppose the dependent variable y_i is represented by a quadratic function of the independent variable x_i written as

$$y_j = \beta_0 + \beta_1 x_j + \beta_2 x_j^2 + \epsilon_j$$

for $j = 1, \ldots, J$. To form the matrix representation for this model, $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$, set

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ 1 & x_J & x_J^2 \end{bmatrix} \quad \text{and} \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}.$$

There is no necessity that the x_j be distinct. Also, they can be pre-specified or randomly sampled. The subject *design of experiments* addresses how to specify these values so as to obtain the most precise estimates.

Least squares geometry

A beautiful geometric representation underlies least squares estimation and is fundamental to the LSVI and LSPI algorithms.

Since $\mathbf{\Pi}^2 = \mathbf{\Pi}$, $\mathbf{\Pi}$ is a projection matrix and it maps $\mathbf{y}$ onto the space of vectors spanned by the columns of $\mathbf{X}$ denoted col($\mathbf{X}$) (see Figure 9.18). In other words $\mathbf{\Pi y}$ is the linear combination of the columns of $\mathbf{X}$ that is closest in the Euclidean norm sense to $\mathbf{y}$.

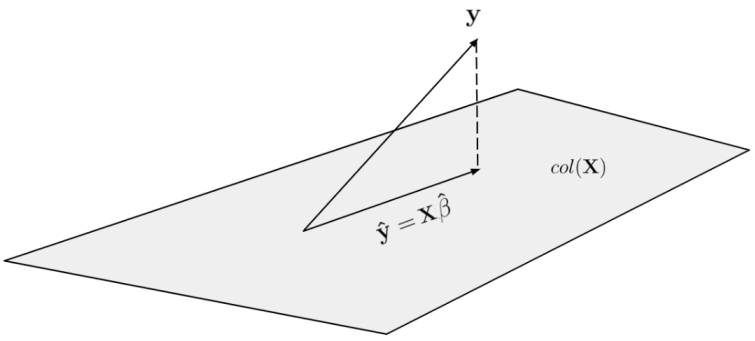


Figure 9.18: Illustration of the geometry underlying least squares regression. It shows the plane col($\mathbf{X}$), the vector $\mathbf{y}$ which lies outside the plane and its projection $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}}$ on col($\mathbf{X}$). The dashed line corresponds to the vector of residuals $\mathbf{y} - \hat{\mathbf{y}}$.

Furthermore, rewriting the normal equations (9.67) as

$$(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\intercal \mathbf{X} = \mathbf{0}$$

shows that at the least squares estimator $\hat{\boldsymbol{\beta}}$, the vector of residuals, $\mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}$, is orthogonal to the space spanned by the columns of $\mathbf{X}$ as shown in Figure 9.18. What this means is that $\mathbf{X}\hat{\boldsymbol{\beta}}$ is the closest point to $\mathbf{y}$ in the subspace col($\mathbf{X}$). Moreover the vector of residuals $\mathbf{y} - \mathbf{X}\boldsymbol{\beta}$ and the predicted values $\mathbf{y} = \mathbf{X}\boldsymbol{\beta}$ are uncorrelated. Hence the independent variables (represented by columns of $\mathbf{X}$) contain no additional explanatory power about the dependent variable.

Weighted least squares

In some situations, for example when the variance of y_i is a function of the independent variables, it might be preferable to minimize the weighted sum of squared errors

$$g_{\mathbf{w}}(\boldsymbol{\beta}) := \sum_{j=1}^{J} w_j g_j(\boldsymbol{\beta})^2, \tag{9.71}$$

where $\mathbf{w} = (w_1, \ldots, w_J)$ is a vector of positive scalar weights. Defining the *squared weighted Euclidean norm* of $\mathbf{u} \in \mathbb{R}^J$ as

$$\|\mathbf{u}\|_{\mathbf{w}}^2 := \sum_{j=1}^{J} w_j u_j^2, \tag{9.72}$$

then

$$g_{\mathbf{w}}(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_{\mathbf{w}}^2.$$

The matrix form of this expression is

$$g_{\mathbf{w}}(\boldsymbol{\beta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\mathsf{T}\mathbf{W}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}),$$

where $\mathbf{W} = \mathbf{I}\mathbf{w}$ is a diagonal matrix with entries $w_1, \ldots, w_J$.

Corresponding to this optimality criterion, the *weighted least squares (WLS)* parameter estimates are given by

$$\boldsymbol{\beta}_{\text{WLS}} = (\mathbf{X}^\mathsf{T}\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{W}\mathbf{y}. \tag{9.73}$$

Note that when $\mathbf{w} = \mathbf{I}$, these reduce to the ordinary least squares parameter estimates.

There are several reasons one might want to use the weighted sum of squares as an optimality criterion:

1. Some regions of the state space might be more important than others, so it is desirable that estimates be more precise in such regions.

2. In statistical models it might be the case that $\text{cov}[\boldsymbol{\epsilon}] = \sigma^2\mathbf{W}$. For example if for $j = 1, \ldots, J$,

   $$y_j = \beta_0 + \beta_1 x_j + \epsilon_j$$

   and $\text{var}[\epsilon_j] = \sigma^2 x_j$, then it would be appropriate to use WLS with $w_j = 1/x_j$. This simple model applies widely, especially in economic data. It means observations become more variable the greater the value of the independent variable.

3. To adjust for unequal state sampling proportions when the data is either an average or a total of the observations at each state.

4. In the body of this chapter it is noted that the operator corresponding to LSPE is a contraction mapping with respect to the weighted *supremum* norm with weight function equal to the stationary distribution of the underlying Markov chain.

9.8.3 Nonlinear regression

In some cases it may be preferable to represent value functions by nonlinear models, which can be denoted in scalar notation as[40]

$$y_j = f(x_{j,1}, \ldots, x_{j,I}; \beta_0, \ldots, \beta_I) + \epsilon_j \tag{9.74}$$

for $j = 1, \ldots, J$ or in vector notation as

$$\mathbf{y} = \mathbf{f}(\mathbf{X}; \boldsymbol{\beta}) + \boldsymbol{\epsilon}, \tag{9.75}$$

where

$$\mathbf{f}(\mathbf{X}; \boldsymbol{\beta}) := \begin{bmatrix} f(x_{1,1}, \ldots, x_{1,I}; \beta_0, \ldots, \beta_I) \\ \cdot \\ \cdot \\ \cdot \\ f(x_{J,1}, \ldots, x_{J,I}; \beta_0, \ldots, \beta_I) \end{bmatrix} = \begin{bmatrix} f(\mathbf{x}_1; \boldsymbol{\beta}) \\ \cdot \\ \cdot \\ \cdot \\ f(\mathbf{x}_J; \boldsymbol{\beta}) \end{bmatrix}.$$

As in the case of linear regression, the least squares estimates of the parameters $\beta_0, \ldots, \beta_I$ minimize the squared error

$$g(\beta_0, \ldots, \beta_I) = \sum_{j=1}^{J} (y_j - f(x_{j,1}, \ldots, x_{j,I}; \beta_0, \ldots, \beta_I))^2, \tag{9.76}$$

which can be expressed in matrix form as

$$g(\boldsymbol{\beta}) = \sum_{j=1}^{J} g_j(\boldsymbol{\beta})^2 = \mathbf{g}(\boldsymbol{\beta})^\mathsf{T} \mathbf{g}(\boldsymbol{\beta}), \tag{9.77}$$

where $g_j(\boldsymbol{\beta}) = y_j - f(\mathbf{x}_j; \boldsymbol{\beta})$ and $\mathbf{g}(\boldsymbol{\beta})$ denotes an $J \times 1$ vector with components $g_j(\boldsymbol{\beta})$. Finding $\boldsymbol{\beta}$ such that $g(\boldsymbol{\beta})$ is minimized can be done by applying nonlinear optimization methods to (9.77) such as those described in the next subsection.

9.8.4 Nonlinear optimization

Many approaches can be used to find least squares estimates of nonlinear models. The most widely used are variants of gradient descent and Gauss-Newton iteration, which are described here.

Gradient descent

Let $g(\boldsymbol{\beta})$ denote a real-valued function of the $(I+1)$-dimensional parameter vector $\boldsymbol{\beta} = (\beta_0, \ldots, \beta_I)$. *Gradient descent* refers to algorithms based on recursions of the form

$$\boldsymbol{\beta}' \leftarrow \boldsymbol{\beta} - \tau \nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}). \tag{9.78}$$

[40] Recall that in general the number of independent variables does not need to equal I.

Written in terms of its iterates this recursion is expressed as:

$$\boldsymbol{\beta}^{n+1} = \boldsymbol{\beta}^n - \tau_n \nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}^n), \tag{9.79}$$

where the gradient of g evaluated at $\boldsymbol{\beta}$ is defined by

$$\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}) := \begin{bmatrix} \frac{\partial g(\boldsymbol{\beta})}{\partial \beta_0} \\ \cdot \\ \cdot \\ \cdot \\ \frac{\partial g(\boldsymbol{\beta})}{\partial \beta_I} \end{bmatrix} \tag{9.80}$$

and τ_n is the *step size* or *learning rate* at iteration n.

The motivation for this method is that $-\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}^n)$ represents the direction of steepest descent of g at $\boldsymbol{\beta}^n$ so that decreasing $\boldsymbol{\beta}$ in this direction will decrease g the fastest. Unfortunately, this method often converges very slowly and is very sensitive to starting values.

In the context of nonlinear regression, the i-th component of the gradient of g evaluated at $\boldsymbol{\beta}$ is given by

$$\frac{\partial g(\boldsymbol{\beta})}{\partial \beta_i} = -2 \sum_{j=1}^{J} \left(\big(y_j - f(x_{j,1}, \ldots, x_{j,I}; \beta_0, \ldots, \beta_I)\big) \frac{\partial f(x_{j,1}, \ldots, x_{j,I}; \beta_0, \ldots, \beta_I)}{\partial \beta_i} \right)$$

for $i = 0, \ldots, I$. In matrix form this can be written as

$$\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}) = -2\mathbf{J}(\boldsymbol{\beta})^{\mathsf{T}} \mathbf{g}(\boldsymbol{\beta}), \tag{9.81}$$

where $\mathbf{J}(\boldsymbol{\beta})$ denotes the $J \times (I+1)$ *Jacobian matrix* of $\mathbf{f}(\mathbf{X}; \boldsymbol{\beta})$ with components

$$\mathbf{J}(\boldsymbol{\beta})_{j,i} := \frac{\partial f(\mathbf{x}_j; \boldsymbol{\beta})}{\partial \beta_i}$$

for $j = 1, \ldots, J$ and $i = 0, \ldots, I$. Alternatively,

$$\mathbf{J}(\boldsymbol{\beta}) = \begin{bmatrix} \nabla_{\boldsymbol{\beta}} f(\mathbf{x}_1; \boldsymbol{\beta})^{\mathsf{T}} \\ \cdot \\ \cdot \\ \cdot \\ \nabla_{\boldsymbol{\beta}} f(\mathbf{x}_J; \boldsymbol{\beta})^{\mathsf{T}} \end{bmatrix}.$$

The following example illustrates the above elements used to specify gradient descent.

Example 9.15. The expressions below are used to fit the nonlinear[a] model

$$y_j = f(x_j; \beta_0, \beta_1, \beta_2) + \epsilon_j = \beta_0 + \beta_1 e^{\beta_2 x_j} + \epsilon_j$$

$j = 1, \ldots, J$ to data using gradient descent. In this model, $\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2)$. Let

$$g(\boldsymbol{\beta}) = \sum_{j=1}^{J} (y_j - f(x_j; \beta_0, \beta_1, \beta_2))^2 = \sum_{j=1}^{J} g_j(\boldsymbol{\beta})^2,$$

where $g_j(\boldsymbol{\beta}) = y_j - f(x_j; \beta_0, \beta_1, \beta_2)$.

For this model

$$-\frac{1}{2}\nabla_{\boldsymbol{\beta}}\, g(\boldsymbol{\beta}) = \begin{bmatrix} \sum_{j=1}^{J} (y_j - (\beta_0 + \beta_1 e^{\beta_2 x_j})) \\ \sum_{j=1}^{J} (y_j - (\beta_0 + \beta_1 e^{\beta_2 x_j})) e^{\beta_2 x_j} \\ \sum_{j=1}^{J} (y_j - (\beta_0 + \beta_1 e^{\beta_2 x_j})) \beta_1 x_j e^{\beta_2 x_j} \end{bmatrix} = \mathbf{J}(\boldsymbol{\beta})^{\mathsf{T}} \mathbf{g}(\boldsymbol{\beta}),$$

where

$$\mathbf{J}(\boldsymbol{\beta}) = \begin{bmatrix} 1 & e^{\beta_2 x_1} & \beta_1 x_1 e^{\beta_2 x_1} \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ 1 & e^{\beta_2 x_J} & \beta_1 x_J e^{\beta_2 x_J} \end{bmatrix} \quad \text{and} \quad \mathbf{g}(\boldsymbol{\beta}) = \begin{bmatrix} y_1 - (\beta_0 + \beta_1 e^{\beta_2 x_1}) \\ \cdot \\ \cdot \\ \cdot \\ y_J - (\beta_0 + \beta_1 e^{\beta_2 x_J}) \end{bmatrix}.$$

[a] If the errors were multiplicative and $\beta_0 = 0$, this model would be linearizable by taking logarithms.

Newton's method

Because gradient descent may converge slowly, Newton's method and its variants provide attractive alternative approaches to solve nonlinear least squares problems. Recall that to find a zero of the scalar function $f(u) = 0$, Newton's method uses the iteration scheme

$$u^{n+1} = u^n - \frac{f(u^n)}{f'(u^n)}$$

For a J-dimensional vector-valued function $(\mathbf{u})$ with components $f_j(\mathbf{u})$ for $j = 1, \ldots, J$ and $\mathbf{u} = (u_1, \ldots, u_J)$ the iterates of Newton's method become

$$\mathbf{u}^{n+1} = \mathbf{u}^n - (\mathbf{J}(\mathbf{u}^n))^{-1} \mathbf{f}(\mathbf{u}^n),$$

where as before $\mathbf{J}(\mathbf{u}^n)$, which is assumed to be invertible, denotes the Jacobian matrix of $\mathbf{f}(\mathbf{u})$.

The nonlinear least squares regression problem can be solved by applying Newton's method to the first order optimality condition $\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}) = \mathbf{0}$. This results in the recursion

$$\boldsymbol{\beta}^{n+1} = \boldsymbol{\beta}^n - (\nabla^2_{\boldsymbol{\beta}} g(\boldsymbol{\beta}^n))^{-1} \nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}^n), \tag{9.82}$$

where the *Hessian matrix* $\nabla^2_{\boldsymbol{\beta}} g(\boldsymbol{\beta}^n)$ has components

$$[\nabla^2_{\boldsymbol{\beta}} g(\boldsymbol{\beta}^n)]_{i,k} = \frac{\partial^2 g(\boldsymbol{\beta})}{\partial \beta_i \partial \beta_k}$$

for $i = 0, \ldots, I$ and $k = 0, \ldots, I$.

Gauss-Newton iteration provides an alternative to Newton's method that avoids computing the second partial derivatives.

Gauss-Newton iteration

Gauss-Newton is based on approximating the real-valued function $g_j(\boldsymbol{\beta})$ as defined in (9.77) by the first-order Taylor series approximation

$$g_j(\boldsymbol{\beta}) \approx g_j(\boldsymbol{\beta}^n) + (\nabla_{\boldsymbol{\beta}} g_j(\boldsymbol{\beta}^n))^\mathsf{T} (\boldsymbol{\beta} - \boldsymbol{\beta}^n). \tag{9.83}$$

Since $\mathbf{g}(\boldsymbol{\beta}) = (g_1(\boldsymbol{\beta}), \ldots, g_J(\boldsymbol{\beta}))$ and $g(\boldsymbol{\beta}) = \mathbf{g}(\boldsymbol{\beta})^\mathsf{T} \mathbf{g}(\boldsymbol{\beta})$

$$g(\boldsymbol{\beta}) \approx (\mathbf{g}(\boldsymbol{\beta}^n) + \mathbf{J}(\boldsymbol{\beta}^n)(\boldsymbol{\beta} - \boldsymbol{\beta}^n))^\mathsf{T} (\mathbf{g}(\boldsymbol{\beta}^n) + \mathbf{J}(\boldsymbol{\beta}^n)(\boldsymbol{\beta} - \boldsymbol{\beta}^n)), \tag{9.84}$$

where $\mathbf{J}(\boldsymbol{\beta}^n)$ denotes the $J \times (I+1)$ Jacobian matrix of $\mathbf{g}(\boldsymbol{\beta}^n)$. Note that in (9.84) $\boldsymbol{\beta}^n$ is fixed and $\boldsymbol{\beta}$ is variable so that $\mathbf{g}(\boldsymbol{\beta}^n)$ is a fixed $J \times 1$ vector and $\mathbf{J}(\boldsymbol{\beta}^n)$ is a fixed $J \times (I+1)$ matrix. Hence, analogous to linear regression, this expression is minimized by choosing

$$\boldsymbol{\beta} - \boldsymbol{\beta}^n = -(\mathbf{J}(\boldsymbol{\beta}^n)^\mathsf{T} \mathbf{J}(\boldsymbol{\beta}^n))^{-1} \mathbf{J}(\boldsymbol{\beta}^n)^\mathsf{T} \mathbf{g}(\boldsymbol{\beta}^n).$$

Rearranging terms yields the Gauss-Newton recursion:

$$\boldsymbol{\beta}^{n+1} = \boldsymbol{\beta}^n - (\mathbf{J}(\boldsymbol{\beta}^n)^\mathsf{T} \mathbf{J}(\boldsymbol{\beta}^n))^{-1} \mathbf{J}(\boldsymbol{\beta}^n)^\mathsf{T} \mathbf{g}(\boldsymbol{\beta}^n). \tag{9.85}$$

The following comments are worth noting:

1. The recursion in (9.85) involves matrix inversion. Instead of inverting this matrix, one can solve the system of equations

   $$(\mathbf{J}(\boldsymbol{\beta}^n)^\mathsf{T} \mathbf{J}(\boldsymbol{\beta}^n)) \boldsymbol{\Delta}^{n+1} = -\mathbf{J}(\boldsymbol{\beta}^n) \mathbf{g}(\boldsymbol{\beta}^n).$$

   and set $\boldsymbol{\beta}^{n+1} = \boldsymbol{\beta}^n + \boldsymbol{\Delta}^{n+1}$.

2. Often it might be inconvenient to compute $\mathbf{J}(\boldsymbol{\beta}^n)$ analytically. Instead it can be approximated numerically.

3. Convergence of Gauss-Newton is sensitive to the starting value $\boldsymbol{\beta}^0$. A good choice can enhance convergence.

4. Modifications are available when $\mathbf{J}(\boldsymbol{\beta}^n)^\mathsf{T}\mathbf{J}(\boldsymbol{\beta}^n)$ is close to singular.

5. Gauss-Newton can also be derived as a special case of Newton's method in which the Hessian is approximated by the Jacobian.

Bibliographic remarks

The concept of using approximations in Markov decision processes appears to originate in the clear and concise paper Schweitzer and Seidmann [1985]. This paper, motivated by work on operations research problems, introduced the idea of approximating value functions by low order multi-variable polynomials and showed how to incorporate these approximations in linear programming and policy iteration. Moreover it considers both discounted and average reward formulations and, like this chapter, did not consider the use of simulation in estimation. We do not believe the authors anticipated how significant and widely used this approach would become.

The texts by Bertsekas [2012], Bertsekas and Tsitsiklis [1996] and Powell [2007] provide comprehensive discussions of value function approximation.

The use of features to reduce state spaces appears to originate with Samuel [1959], who is also credited with coining the expression "Machine Learning" to represent the study of using computers to learn tasks without explicitly programming them to do so.

We base our discussion of least squares parameter estimates in policy iteration on Lagoudakis and Parr [2003] and the surveys by Bertsekas [2011] and Buşoniu et al. [2012]. The bounds for LSVI and LSPI appear to be new and useful, although other bounds exist in the literature.

de Farias and Roy [2003] provides an in depth study of the approximate LP model. Moreover they analyze an instance of the queuing admission control model used for examples in this chapter and elsewhere.

Nielsen [2015] provides a very readable step-by-step introduction to neural networks. Private communications with several researchers corroborate the difficulties we observed when incorporating neural networks in LSVI and LSPI. In fact, most analyses in the literature are based on linear approximations.

Our discussion and analysis of the advanced scheduling model draws on work of Patrick et al. [2008] and Sauré et al. [2015]. Sauré et al. [2015] also analyzes the queuing service rate control model. The shots gained metric was developed by Broadie [2014]. A similar metric, points gained, was developed for American football by Chan et al. [2021].

Draper and Smith [1998] provide a good introduction to and overview of regression.

Exercises

1. Consider the queuing service rate control model analyzed throughout this chapter. Describe the basis functions you would use if you intended to approximate the value function by:

   (a) Aggregating states into subgroups.

   (b) A piecewise linear approximation that interpolates between a subgroup of states.

   (c) A piecewise quadratic approximation that interpolates between a subgroup of states.

2. Consider a two-state model with a single policy $d(s)$, with $\lambda = 1/2$,

$$\mathbf{r}_d = \begin{bmatrix} 4 \\ 2 \end{bmatrix} \quad \text{and} \quad \mathbf{P}_d = \begin{bmatrix} 0.5 & 0.5 \\ 1 & 0 \end{bmatrix}. \tag{9.86}$$

   (a) Find $\hat{\boldsymbol{\beta}}_{\text{OLS}}$, $\hat{\boldsymbol{\beta}}_{\text{LSVI}}$ and $\hat{\boldsymbol{\beta}}_{\text{BR}}$ when the basis function is $b_0(s) = 1$.

   (b) Represent these estimates graphically as in Figure 9.6. Include the coordinates for all quantities.

3. Consider the model in Example 9.3 using an approximation based on $b(s) = s$.

   (a) Obtain parameter estimates based on LSPE and compare them to those in the text.

   (b) Repeat your analysis using APLP.

   (c) Repeat your analysis with $b(s) = s^{0.5}$.

4. Compute linear and quadratic approximations to the value function in Example 9.5 using Bellman residual minimization and compare them to those in the text.

5. Repeat the analysis in Example 9.5 for $S = 0, \ldots, 100$ and a decision rule that always uses action a_2.

   (a) Compare the quality of quadratic and cubic approximations obtained using OLS, LSPE and Bellman residual minimization to the exact value function and its projection on col($\mathbf{B}$).

6. Consider the queuing service rate control model with $S = \{0, \ldots, 60\}$.

   (a) Solve the model exactly using a method of your choice.

   (b) **Extrapolation.** Find a quadratic value function approximation using LSVI or LSPI on $S' = \{0, \ldots, N\}$ for $N = 20, 30, 40$ and use them to obtain greedy policies for all of S. How do these policies compare to the optimal policy?

(c) Use the value function approximations obtained using the above subsets to obtain bounds on the value functions on all of S using (9.35).

(d) **State sampling.** Find a quadratic value function approximation by sampling a fraction of states and then applying LSVI or LSPI on this subset. Compare the resulting greedy policies and values functions to the optimal values and policies as in Figure 9.15. What can you conclude?

(e) **State aggregation.** Consider the effect of state aggregation on the accuracy of approximate value functions and corresponding greedy policies where basis functions are indicator functions of disjoint sequences of m consecutive states. Investigate the impact of m on the quality of the approximations.

7. **Oscillation of LSPI.**[41] Let $S = \{s_1, s_2\}$; $A_{s_1} = \{a_{1,1}, a_{1,2}\}$, $A_{s_2} = \{a_{2,1}\}$; $r(s_1, a_{1,1}) = 0.1$, $r(s_1, a_{1,2}) = 0$, $r(s_2, a_{2,1}) = 0$; $p(s_1|s_1, a_{1,1}) = 0.8$, $p(s_2|s_1, a_{1,1}) = 0.2$, $p(s_2|s_1, a_{1,2}) = 1$, $p(s_1|s_2, a_{2,1}) = 1$ and $\lambda = 0.9$.

   (a) Find the optimal policy by enumeration.

   (b) Consider the basis function represented by $b(s_1) = 1, b(s_2) = 2$ so that the linear approximation to the value function is of the form $\tilde{\mathbf{v}} = \begin{bmatrix} \beta \\ 2\beta \end{bmatrix}$. For what values of β is it optimal to use action $a_{1,1}$, respectively $a_{1,2}$, in s_1.

   (c) Solve the problem using LSPI starting with $\beta = 1$. What behavior of the iterates do you observe?

8. Consider the replacement model in Section 5.10.2 with $M = 500, p = 0.1, \lambda = 0.95$, $K = 1200$ and a linear operating cost $2s$ in state s.

   (a) Find an optimal policy by a method of your choosing and describe its structure.

   (b) Find linear and quadratic value function approximations using LSVI, LSMPI, LSPI and linear programming. Compare the quality of the approximations and the greedy policies identified by each.

9. Solve the instance of the advanced scheduling problem in Section 9.6 using policy iteration, linear programming and evaluate quadratic approximations obtained using LSPI and approximate linear programming.

10. For the advanced appointment scheduling model in Section 9.6, compare the greedy policy based on (9.52) obtained from a random sample of half the states to that in (9.50) based on applying LSVI to the entire state space.

11. (Mini project.) This open-ended problem asks you to carry out your own analysis of the advanced appointment scheduling model analyzed in Section 9.6. To do

[41] This example is adopted from Example 1 in Bertsekas [2011].

so entails coding the model, solving it optimally, analyzing it with LSVI using polynomial and neural network approximations, and comparing policies and the quality of approximations. You must specify λ, N, M and K, costs and demand distributions. Some questions to consider:

(a) How large a problem can you solve exactly?

(b) What is the structure of the greedy policies?

(c) What is the impact of state sampling?

(d) How does the quality of the approximations vary with cost structure?

(e) How can you extend your analyses to three or more classes?

12. (Mini project.) Consider an inventory system that manages L products it obtains from a single source. Every time an order is placed, there is a fixed charge of K dollars and a per unit charge of c_l for product l. Assume orders placed arrive prior to the next business day. The cost for storing one unit of product l for one day is h_l.

    Customer orders for products arrive daily and are filled immediately from stock on hand. Assume demand for product l is Poisson distributed with mean μ_l. The selling price for product l is r_l and orders that can not be filled from stock on hand are "lost", that is the customer seeks the product elsewhere.

    (a) Formulate this as a Markov decision problem where the objective is to maximize discounted expected revenue over an infinite horizon.

    (b) What quantities would need to be truncated in order to solve this problem. How would you truncate them?

    (c) Consider a model with $L = 5$, $K = 200$, $c_l = 50$, $r_l = 100$, $h_l = 2$ and $\mu_l = 5$. Find approximately optimal policies using LSVI, LSPI, LSMPI and linear programming. Choose suitable basis functions and architectures. Derive bounds to assess the quality of your approximations.

    (d) Is there any obvious structure to the approximate policies? Investigate how they vary when the model parameters change and the products are not identical.

13. Show that the LSPE operator is a contraction in the weighted sup-norm with weighting function given by the stationary distribution of the underlying regular Markov chain.

14. Show that all equivalences in (9.65) hold.

15. Consider the model in Example 9.15.

    (a) Write out the Newton's method and Gauss-Newton recursions for this model.

(b) Fit this model to the data in Table 9.3 using gradient descent, Newton's Method and Gauss-Newton. Compare the effort to set the recursion up, the rate of convergence and the sensitivity to starting values.

Variable / Observation	1	2	3	4	5	6	7
y_j	8.37	15.39	4.93	8.92	33.74	7.14	62.95
x_j	2	5	1	3	8	2	10

Table 9.3: Data for Exercise 15. The data generating model was $y_j = 2 + 3e^{0.3x_j} + \epsilon_j$ where $\epsilon_j \sim NID(0, 1)$.

16. Formulate the game of Scrabble as an MDP. What features would you use when approximating a value function?

17. Formulate the game of Tetris as an MDP. What features would you use when approximating a value function?

Chapter 10

Simulation in Tabular Models

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

You've got to win a little, lose a little, yes, and always have the blues a little[1].

From the song *The Glory of Love*, by Billy Hill, 1899-1940.

This is the first of two chapters on reinforcement learning, focusing on how trial-and-error learning is implemented in simulated and real-world environments. As the song lyric suggests, learning is an ongoing interplay of winning and losing. Successes reinforce behavior while failures diminish it. This dynamic lies at the heart of reinforcement learning and underpins the temporal difference and Q-learning methods introduced in this chapter.

This chapter describes simulation-based methods for evaluating value and state-action value functions, a crucial step in identifying *effective* policies. The focus is on models that can be represented in *tabular* form, that is, when the sets of states and actions are sufficiently small so that value functions, state-action value functions and stationary policies can be represented in look-up tables indexed by either states or state-action pairs. Chapter 11 builds on this foundation by extending methods to complex environments that instead rely on approximations to represent value functions, state-action value functions, and policies.

A brief comment on terminology:

1. The term *methods* replaces algorithms[2] because the proposed approaches do not

[1] Hill (1936).

[2] Note that in the text, they are still indexed as algorithms.

have stopping criteria that guarantee precise error bounds. The number of iterations or replicates is often chosen arbitrarily.

2. The expressions *good* or *effective* distinguish policies identified by the methods herein from optimal or ϵ-optimal policies generated by the algorithms in Chapters 5, 6 and 7.

3. The term *simulation* refers broadly to any mechanism, be it computer-based or a physical system operating in the real world, that as shown in Figure 10.1, generates successor states and rewards from current states and actions based on an underlying known or unknown MDP.

Figure 10.1: Symbolic representation of the data generation process underlying the methods in this chapter. The simulator, be it computer-based or a real-time process, generates the subsequent state-reward pair (s', r) from a current state-action pair (s, a).

This chapter will consider three broad classes of methods:

1. Monte Carlo

2. Temporal differencing

3. Q-learning

Each has numerous variants that will be explored below. Policy gradient methods, which are compatible with simulation, will be discussed in the next chapter.

Model classification

The methods herein are frequently classified as either *model-based* or *model-free*. The expression "model-free" is somewhat a misnomer; it refers to settings where system dynamics are governed by a Markov decision process, but where the Markov decision process is not used to estimate value functions, state-action value functions or policies. Instead the system learns by interacting directly with the environment; this feedback guides policy choice. Model-free approaches may be used when the underlying model is very complex and challenging to formulate, when the state space and/or reward function is unknown, or when complex dynamics makes it easier to use a simulator.

More explicitly:

Model-based: Analysis is based on a Markov decision process model of the system. This means that S, $\{A_s|s \in S\}$, $r(s,a,j)$ and $p(j|s,a)$ are known and can be used to generate data by sampling s' from $p(\cdot|s,a)$ and setting $r' = r(s,a,s')$. Moreover, they can be used directly in computation. Examples include inventory management, queuing control, and revenue management models.

Model-free: There is an underlying model but it is not used for analysis because it may be complex or not known. *At a minimum the analyst knows the actions available at each decision epoch.* The set of states may or may not be known. State transitions and rewards are generated by a simulator or a real-world process as depicted in Figure 10.1. Methods must be based on value functions, state-action value functions or policies only. Applications include gaming, autonomous vehicle control, and robotics. This approach is often applied to episodic models in which there is a task to complete. Moreover, there may not be an explicit reward structure; the analyst must design one to achieve the objectives as efficiently as possible.

The distinction between model-based and model-free reinforcement learning is conceptually useful but often blurred in practice. Model-based methods rely on explicit knowledge or estimation of the underlying MDP — namely, the transition probabilities and reward function — and use this model to evaluate or optimize policies. Model-free methods, in contrast, learn value functions or policies directly from experience, without requiring knowledge of the MDP's structure.

For example, in applications such as advance appointment scheduling (Section 3.7) a Markov decision process model can be explicitly formulated, but it may be preferable to adopt a model-free approach[3].

This chapter describes and illustrates policy evaluation and optimization methods, focusing on episodic and infinite horizon discounted models, with a brief discussion of average reward models. Its intent is not to be comprehensive and theoretically rigorous, but to provide a high-level framework for delving into Chapter 11 and the voluminous and rapidly advancing reinforcement learning literature.

The key concept underlying the methods in this chapter and the next is that:

> Simulation replaces computation of expectations in Bellman equations.

10.1 Preliminaries

This section discusses data generation and underlying trade-offs when choosing methods.

[3]The idea here is that instead of deriving a (complicated) analytical model for the MDP, it might be easier to simulate events, apply a policy, and observe the outcome.

10.1.1 Data generation

The two primary sources of data are computer simulation and real-world interaction. They can be used for policy evaluation or optimization. Each creates data sequences by transforming states and actions to successor states and rewards as discussed above:

> **Simulation:** A computer-based simulator generates successor states and rewards using either underlying probabilistic mechanisms or MDP model components.
>
> **Real-world interaction:** A system, process or experiment generates successor states and rewards.

Repeated application of this mechanism generates *trajectories*. A few examples may help clarify this distinction:

1. In a queuing service rate control model, sampling the number of arrivals and service completions in a period transforms a state and action to a successor state and reward. This is a classic example of an application of simulation. It is preferable to simulating directly from the MDP model, which would require first computing a convolution of the service and arrival distributions. Clearly, this latter approach would not be scalable to larger models.

2. A physical navigational robot interacts with its environment by choosing an action in a given state and (possibly) moving to a new location and generating a reward. This could be costly if some actions could damage the robot. Alternatively a simulation of robot motion could avoid such costs. However if the robot is complex, policies generated through simulation might not perform identically in real life.

3. In Chess, White's choice of a move results in a new board configuration prior to White's next move. Randomness arises through the implementation of Black's strategy. This can be implemented in actual play or more likely through a simulator. Self-play is often used to develop computer-based Chess players.

4. In the pre-board screening application described in the Section 8.6.5, one approach is to develop an MDP model and derive and then simulate system behavior using the derived transition probabilities. Clearly, this is an onerous task. Instead, one could develop a simulation of the system where arrival and service probability distributions combine with operational policies to update the system state and determine one-period costs. Real-world interaction would be impractical since it would impact passenger wait times.

5. In managing patient flow in a health care system, historical data on lengths of stay and transfer decisions could provide appropriate data for analyses.

Our computational experiments suggest that distinguishing computer-based simulations from real-world interaction was important to bring to the forefront for the following reason. In a simulation environment, the analyst may restart the system at any state at any time, while in real-world interaction, states usually follow system dynamics. As a case in point, consider the pre-board screening application referred to above. Clearly the system state cannot be altered in real time. It is determined by passenger arrivals and inspection completions. Computer simulation allows testing policies in a wide range of configurations that might not have been observed or that occur infrequently.

Data generation mechanisms

Starting at $s^1 \in S$, a simulator or process will generate a sequence

$$(s^1, a^1, r^1, s^2, a^2, r^2, \ldots, s^N, r^N),$$

where N may be random or fixed. In the model-based environment, $r^n = r(s^n, a^n, s^{n+1})$, so it can be computed from a sequence of state-action pairs. In the model-free environment, the reward will be observed directly.

Data in reinforcement learning applications may be generated in a variety of ways, depending on the setting, objectives, and available resources. The following four categories capture common approaches, each with different implications for learning, evaluation, and exploration. Their significance will become clearer in the examples that follow.

Trajectory-based: Specify a stationary randomized or deterministic policy π. Starting from an initial state $s^1 \in S$, generate a trajectory $(s^1, a^1, r^1, s^2, a^2, r^2, \ldots, s^N, r^N)$ from a simulator or a real-world process. The initial state may be fixed or chosen randomly from a pre-specified distribution.

State-based: Specify a stationary randomized or deterministic decision rule $d(s)$. Sample $s \in S$ from a specified distribution. Apply $d(s)$ to generate[4] a successor state s' and a reward r. In state-based data generation, s is not the result of a transition from the previous state; it is sampled from a pre-specified probability distribution. This is sometimes referred to as *random restart* data generation.

Action-based: Given $s \in S$, generate $a \in A_s$ by some mechanism that does not correspond[5] to a pre-specified policy or decision rule. Then generate s' and r.

[4]When d is randomized, this action is generated from $w_d(a|s)$. When d is deterministic, $d(s)$ denotes an action.

[5]Logically, any way of generating actions from states may be viewed as a policy but it need not be explicitly stated as such or used in the system.

State-action pair-based: Sample $s \in S$ and $a \in A_s$ from specified distributions, and then generate r and s'. State-action based generation combines exploration in *both* the state and action spaces.

Comments regarding the use, benefits, and limitations of these data generation approaches follow:

1. Data may be generated either *sequentially* or by *sampling states or state-action pairs after each iteration*. Sequential data may be realized in real-time as the output of a real or simulated process. In contrast, in most applications, simulation must be used to generate random restart data because random restarts would not be able to be physically realized.

2. Trajectory-based and state-based approaches provide the basis for policy evaluation.

3. Trajectory-based methods generate data that behaves like the stationary distribution of a Markov chain generated by a policy.

4. State-based methods, in which states are generated either randomly or deterministically, enable investigation of system behavior in states that are infrequently (or never) visited under trajectory-based data generation.

5. Action-based methods are particularly useful in optimization because they facilitate exploration in the action space (and state space) enabling identification of potentially beneficial actions.

6. State-action pair-based methods combine the features of state-based and action-based methods. They combine exploration in the action space with the ability to generate states that are infrequently observed under policies generated by optimization methods.

Common random numbers

Simulation-based comparisons of policies or methods can be enhanced using *common random numbers*. The idea is to use the same random number stream to evaluate multiple policies or algorithmic parameter choices. The benefit is that by eliminating one source of variability, namely the random number sequence seed, the variability of comparisons is reduced[6].

[6]Suppose X and Y are two random variables. Then

$$\mathrm{Var}(X - Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) - 2\mathrm{Cov}(X, Y).$$

When X and Y are positively correlated, that is when $\mathrm{Cov}(X, Y) > 0$, the variance of the difference of X and Y is reduced. Common random numbers seek to achieve this by removing one source of variability, the random number sequence seed.

In the context of this chapter and the next, common random numbers potentially reduce the variance of the *differences* of estimates of value function and state-action value functions across policies or methods. In this chapter, they will be primarily used for comparing the performance of algorithms under different parameter choices or implementation methods.

Examples of applications of common random numbers include using the same stream of arrivals and service times in queuing control models or the same demand stream in inventory models such as the newsvendor model.

10.1.2 Trade-offs

This section describes some terminology and trade-offs that may be encountered when using the methods described in subsequent sections.

1. **Offline vs. online:** *Online* methods update value functions or state-action value functions after each transition. *Offline* (*batch*) methods use a complete realization of a trajectory for computation. In practice one may combine these two approaches by first using an offline implementation to obtain good starting values for subsequent application of an online method in real-world interaction.

2. **Exploration vs. exploitation:** In light of one of the most famous quotes commonly mis-attributed[7] to Albert Einstein (1879-1955):

   > *Insanity is doing the same thing over and over again and expecting different results,*

   it is advantageous to try new things occasionally. Thus, to find good policies, a method needs to deviate from greedy action selection so as to investigate the impact of other actions on system performance. Greedy action selection is referred to as *exploitation* and non-greedy action selection as *exploration*. Exploration methods include ϵ-greedy and softmax sampling, which are described below. Exploitation may be attractive when the decision maker wishes to accrue rewards during the learning phase or improve estimation of value functions and state-action value functions.

3. **On-policy vs. off-policy:** A method is said to be *on-policy* if it evaluates the data-generating policy, often referred to as the *learning* or *behavioral* policy. A method is said to be *off-policy* if it evaluates a policy that differs from the behavioral policy. This distinction is relevant when designing algorithms and when the reward received during learning is of concern to the investigator. Off-policy methods should be considered when reusing previously collected data or when safety or cost constraints limit exploration under the behavioral policy.

[7] The source of the quote appears to be the book *Sudden Death* by Rita Mae Brown, Bantam, 1983, p. 68, as noted in *The Ultimate Quotable Einstein*, by Alice Calaprice, Princeton University Press, 2011, p. 474.

4. **Variance vs. bias:** In statistical estimation one often seeks a minimum-variance unbiased estimator. When evaluating a method, the analyst may be willing to accept some bias in exchange for reduced variance. The *root mean squared error* for distinguished states or averaged over states captures this trade-off. (See Appendix 10.11.1 for a detailed discussion of this issue.)

5. **Synchronous vs. asynchronous:** A *synchronous* method simultaneously updates value functions in **all** states or state-action value functions in **all** state-action pairs in a similar way to value and policy iteration. An *asynchronous* method updates these quantities one state or state-action pair at a time in a similar way to Gauss-Seidel iteration. With the exception of TD(γ), all the methods in this chapter are asynchronous. The reason for this distinction is that in asynchronous methods, the precision of estimates varies over states or state-action pairs.

Table 10.1 distinguishes the methods discussed in this chapter with respect to their input data, use, and updating approach.

Method	Data	Use	Updating
Monte Carlo	Trajectory-based	Policy Evaluation	Offline
Temporal differencing	Trajectory-based or state-based	Policy Evaluation	Online
Q-learning	Trajectory-based or state-action pair-based	Optimization	Online

Table 10.1: Methods, compatible data generation mechanisms, and purpose.

10.2 Methods overview: The learning newsvendor

This section provides an overview of the methods introduced in this chapter by demonstrating how they can be used to determine the optimal order quantity in the classic newsvendor model, which was introduced in Section 3.1.2. The newsvendor model occupies a central role in the operations research literature due to its analytical tractability and real-world relevance. It applies to a wide-range of products and services with uncertain demand and a limited selling season including bread and dairy products, daily (printed) newspapers, fashion items and time-sensitive services such as hotel rooms and airplane seats.

The reason for focusing on the newsvendor problem is that it provides a transparent yet rich framework, namely a single-state, one-period model, in which to highlight core concepts without the complicating features of multi-period dynamics and multiple states. Thus, it provides an ideal setting to introduce Monte Carlo methods, temporal differencing, and Q-learning.

As noted in Section 3.1.2, the newsvendor's objective is to choose an order quantity $a^* \in A_0$ that maximizes the expected revenue

$$E[r(a, Z)] = \sum_{z=0}^{M} r(a, z) f(z), \tag{10.1}$$

where A_0 denotes the set of possible order quantities and M denotes a finite upper bound on the demand[8]. Recall that the random variable Z denotes the demand, $f(z) := P[Z = z]$, and $r(a, z)$, the revenue if a units of the item are ordered and the demand is z, was shown to be equal to

$$r(a, z) := G \min(z, a) - L(a - z)^+. \tag{10.2}$$

In this expression G denotes the profit (selling price minus cost) obtained by selling a unit and L the loss (cost minus salvage value) associated with disposing of an unsold item.

Moreover, it was shown that if the demand distribution is known and continuous, the optimal order quantity is available in closed form as

$$a^* = F^{-1}\left(\frac{G}{G + L}\right), \tag{10.3}$$

where $F(z) := P[Z \leq z]$.

10.2.1 Analysis by simulation

This may be regarded as either an episodic model with all episodes of length one, or equivalently, as a one-period model as formulated in Chapter 3. The following rather formal notation will help relate it to later material in this chapter and as well to general Markov decision process notation. Let[9] $q(a), a \in A_0$, denote the expected reward when choosing order quantity (action) a in state 0, that is,

$$q(a) := E[r(a, Z)].$$

Since in this simple model decision rules correspond to actions, the expected total reward of using decision rule $d'(0) = a'$ is given by $v^{d'}(0) = q(a')$. Note the equivalence between the value function and state-action value function here because in general:

> A value function equals a state-action value function evaluated at a specific decision rule.

Offline (batch) and online approaches for obtaining estimates $\hat{q}(a)$ of $q(a)$ are described below.

[8] In contrast to Chapter 3, the demand is truncated to facilitate demand simulation and analyses.

[9] In the standard state-action value function notation, this quantity would be written as $q(0, a)$. The expression $q(a)$ is used for simplicity, since there is only one state.

Monte Carlo estimation

Monte Carlo estimation is an offline (batch) approach. A policy corresponds to a single fixed order quantity $\bar{a}$. To evaluate it using a Monte Carlo approach, take N samples $\{z^1, \ldots, z^N\}$ from $f(\cdot)$, set

$$r^n := r(\bar{a}, z^n) \tag{10.4}$$

and

$$\hat{q}(\bar{a}) = \frac{1}{N} \sum_{n=1}^{N} r^n. \tag{10.5}$$

To optimize using Monte Carlo, repeat the above simulation for each $a \in A_0$ and set

$$a^* \in \arg\max_{a \in A_0} \hat{q}(a). \tag{10.6}$$

Note that

1. the same demand sequence could be used for evaluating all actions,
2. the ordering of r^n does not affect the estimate, and
3. estimates of the standard deviation and other distributional properties of $q(a)$ can be easily obtained.

Temporal differencing

Temporal differencing provides an online approach for policy value function estimation[10]. Instead of waiting until all N values of r^n have been realized, estimates of $q(\bar{a})$ are updated as data arrives. It works as follows. To evaluate the policy $\bar{a}$, set $q^1(\bar{a})$ to an arbitrary value[11] and after realizing z^n, compute r^n using (10.4) and update the estimate of $q(a)$ according to

$$q^{n+1}(\bar{a}) = q^n(\bar{a}) + \tau_n\big(r^n - q^n(\bar{a})\big), \tag{10.7}$$

where $0 < \tau_n \leq 1$ for $n = 1, 2, \ldots$ is a sequence of positive constants converging to 0. The expression (10.7) can be thought of as follows. The quantity $q^n(\bar{a})$ is the "prediction" of $q(\bar{a})$ before observing r^n. After observing r^n a "new prediction" is obtained by adding a *fraction* of the difference between the observation and the previous prediction to the previous prediction. Referring to τ_n as the *learning rate*, the above recursion can be expressed as

> New prediction = old prediction + learning rate × (observation − old prediction).

[10] This is referred to as a *prediction problem* in the reinforcement learning literature.
[11] 0 is a good choice.

Choosing a learning rate less than 1 avoids over-correction and smooths out noise. Requiring that it approaches 0 as n increases stabilizes the estimate. Moreover, choosing the learning rate to satisfy some technical conditions described in this chapter and Appendix 10.11.2 provides theoretical guarantees of convergence.

Some comments about temporal differencing follow:

1. This approach is referred to as TD(0) for reasons to be discussed below.
2. Choosing $\tau_n = 1/n$ is equivalent to online estimation of the sample mean[12].
3. Choosing the learning rate in practice requires experimentation.
4. The order of the r^n does impact the value of the estimate for non-constant learning rate sequences.
5. Distributional properties of $q^n(\bar{a})$ can be obtained by choosing multiple permutations or bootstrap[13] samples from $(r^1, \ldots, r^n)$.

Q-learning

Instead of computing an estimate of $q(a)$ for each action and then maximizing, one could instead incorporate the maximization directly in the recursion. This is the idea underlying the *Q-learning* recursion:

$$q^{n+1}(a) = q^n(a) + \tau_n \left(\max_{a \in A_0} r(a, z^n) - q^n(a) \right). \tag{10.8}$$

When A_0 consists of a single action, Q-learning is equivalent to temporal differencing.

Because of noise (sampling variability) alternatives to greedy action choice may be preferable. Recursions based on this approach can be expressed as

$$q^{n+1}(a) = q^n(a) + \tau_n \big(r(a', z^n) - q^n(a) \big), \tag{10.9}$$

where a' is chosen randomly in some way[14].

One approach is to use a version of ϵ-*greedy* action selection that samples a' according to:

$$a' = \begin{cases} a^* \in \arg\max_{a \in A_0} r(a, z^n) & \text{with probability } 1 - \epsilon_n \\ a \in A_0 \setminus \{a^*\} & \text{with probability } \epsilon_n / (|A_0| - 1) \end{cases} \tag{10.10}$$

[12] See Example 10.14 in the chapter appendix

[13] Bootstrapping here refers to the statistical procedure that resamples from a data set with replacement to obtain properties of estimators.

[14] In a one-period model such as this, this algorithm is identical to an algorithm referred to as SARSA below.

for some sequence of $\epsilon_n, n = 1, 2 \ldots$. Note that decreasing ϵ_n leads to less exploration later on in the iterative process. Moreover, setting $\epsilon_n = 1$ for all n corresponds to randomly sampling actions and $\epsilon_n = 0$ for all n to using greedy action selection.

An alternative is to use *softmax* sampling (action selection) in which action a' is chosen with probability

$$p(a') := \frac{e^{\eta_n q(a')}}{\sum_{a \in A_0} e^{\eta_n q(a)}}, \tag{10.11}$$

where the sequence $\eta_n, n = 1, 2, \ldots$, affects the relative weighting of actions. Large values of η_n put more weight on actions with the largest values of $q(a')$, while small values of η_n make the weight on actions more uniform. Note that when all values of $q(a)$ are equal, softmax sampling corresponds to random sampling.

The ϵ-greedy method selects actions other than the greedy action with equal probability, while the softmax approach samples among actions with greater rewards. When there a few actions in each state, either method would be appropriate. However, when there are many actions, the softmax sampling may be more appropriate because it concentrates exploration on fewer actions, namely those with greater q-values. This distinction is significant in the calculations below.

Q-learning: An "Earning while learning" perspective

Instead of simply describing a way to find good actions, recursions (10.8) and (10.9) may be viewed from the perspective of an agent that learns by trial-and-error and accumulates reward beginning as follows.

1. Choose $q^1(a)$ and set $R^0 = 0$.
2. Choose a^1 greedily, ϵ-greedily or by softmax sampling.
3. Implement a^1 and observe r^1.
4. Set $R^1 = R^0 + r^1$.
5. Update $q^2(a^1)$ using (10.8) or (10.9).

The fourth step highlights the subtle distinction with the Q-learning approach described previously. Here, the perspective is that rewards earned are tracked while learning takes place.

By repeating this procedure[15] for N steps, the newsvendor obtains total reward R^N, and uses $q^N(a)$ to choose order quantities. Variants of this approach[16] can be compared on the basis of a running average of the cumulative reward and on how accurately they determine the known optimal order quantity.

[15] We choose not to state this formally here.

[16] Variants may correspond to different choices of tuning parameters and update methods.

10.2.2 A numerical study

This section illustrates and discusses computation using the above approaches.

Model

Demand is generated from either a discrete (rounded) and truncated normal distribution with mean 50 and standard deviation 15 or a gamma distribution[17] with shape parameter 20 and scale parameter 4. The item cost is 20, the salvage value is 5, and price varies among 21, 30, and 100. The corresponding G values are 1, 10, and 80, and $L = 15$. The corresponding ratios of $G/(G+L)$ are $0.0625, 0.400$, and 0.842, illustrating several distinct quantiles of the demand distribution. Set $A_0 = \{0, 1, \ldots, 120\}$.

Monte Carlo estimation

Monte Carlo methods are compared using sample sizes of $N = 100$ and $N = 5{,}000$. A replicate consists of estimating $q(a)$ for each $a \in A_0$ for each price. The expected reward is estimated using (10.5) and the optimal order quantity is chosen according to (10.6). Using a common random number seed, the same realized demand was used in each replicate. There was no need to resample; estimates could be based on stored demand samples.

Table 10.2 gives the optimal order quantity (for a typical replicate) using the closed form representation (10.3) and that based on the arg max of the Monte Carlo estimates of $q(a)$. Of course, choosing $N = 5{,}000$ produces more accurate estimates of the optimal order quantity than using $N = 100$ but the difference was surprisingly small. An exception was when demand was normal and price was 21, when the optimal order quantity corresponded to $G/(G + L) = 0.0625$. This is because larger sample sizes are needed to estimate values in the tail of a distribution.

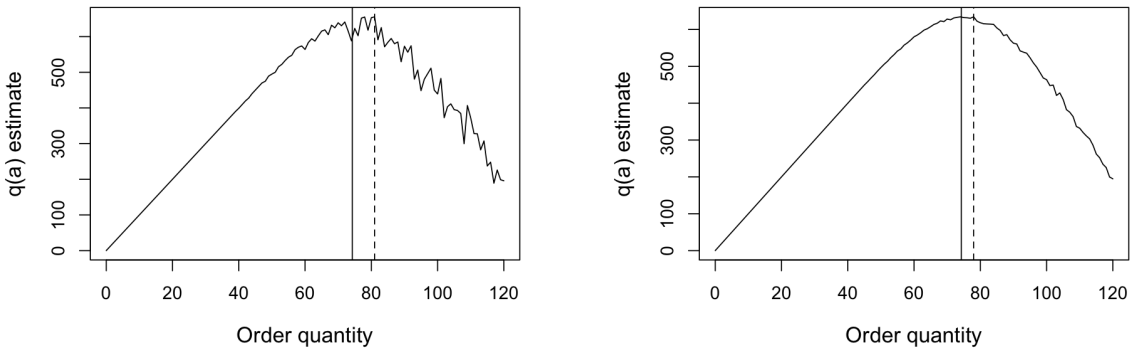
Distribution	Price	Optimal	Monte Carlo ($N = 100$)	Monte Carlo ($N = 5{,}000$)
Normal	21	27	36	26
	30	46	42	44
	100	65	69	64
Gamma	21	55	57	55
	30	74	81	78
	100	98	109	104

Table 10.2: Optimal order quantity and that based on Monte Carlo estimates of $q(a)$ in a typical replicate of the newsvendor model using $N = 100$ and $N = 5{,}000$.

Figure 10.2 shows $\hat{q}(a)$ based on a single replicate with gamma distributed demand, price equal to 30 and two values of N. Observe that the estimates with $N = 100$ are

[17] For this choice of parameters, the mean of the gamma distribution is 80 and its standard deviation is 17.89.

considerably noisier than those with $N = 5{,}000$. However, surprisingly, the estimated optimal order quantities vary little. Note that for both values of N the curves are quite similar for small values of a. Moreover, Figure 10.2b shows that $\hat{q}(a)$ is quite flat near the true optimum, suggesting that there may be considerable variability in choosing the action using Monte Carlo estimates.



(a) $N = 100$ (b) $N = 5{,}000$

Figure 10.2: Estimates of $q(a)$ based on a single replicate with gamma distributed demand and price equal to 30 for two sample sizes. The solid vertical line represents the theoretical optimal order quantity and the dashed line that obtained using (10.6).

Temporal differencing

This section illustrates the use of temporal differencing by applying it to estimate $q(80)$ for $G = 80$, $L = 15$ and gamma distributed demand. It compares learning rates $\tau_n = 0.05, n^{-1}, 0.5n^{-0.75}$, and $10/(40 + n)$. Figure 10.3 shows the evolution of the iterates of (10.7) over a typical replicate with $N = 1{,}000$ iterations starting from 0 and the same demand sequence for each learning rate. Observe that the sample mean, which corresponds to $\tau_n = n^{-1}$, best approximates the true value, with the estimate using $\tau_n = 0.5n^{-0.75}$ quite close. Note also that the estimate corresponding to $\tau_n = 0.05$ oscillates. More formally, estimates can be compared using the RMSE as described in Appendix 10.11.1. The respective RMSE values corresponding to the above learning rates are 467.21, 163.87, 214.29 and 419.08, consistent with Figure 10.3. Such calculations over multiple configurations and with many replicates provide the basis for parameter studies in this chapter and the next.

Q-learning

Applying Q-learning using (10.9) was more challenging than the methods above. The ϵ-greedy action selection method was unreliable. Even when ϵ_n was slightly less than one,

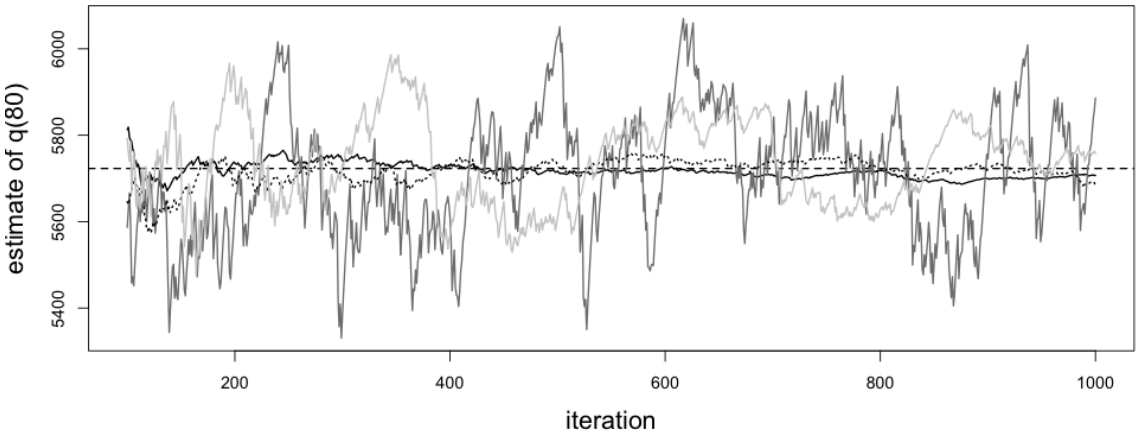


Figure 10.3: Comparison, based on a single replicate, for estimates of $q(80)$ for the four choices of τ_n described in the text. The horizontal dashed line represents the exact value $q(80) = 5{,}723$, and the other four lines represent different choices of τ_n. The dark grey line corresponds to $\tau_n = 0.05$, the solid line to $\tau_n = n^{-1}$, the dotted line to $\tau_n = 0.5/(n^{0.75})$ and the light grey line to $\tau_n = 10/(n + 40)$.

a single high-value action was sampled too frequently; all other actions were sampled equally and infrequently. Thus the recursion "got stuck" at a potentially non-optimal action.

Softmax action selection was better suited to this problem since there are 121 actions and $q(a)$ was quite flat near the optimum and dropped off quickly as shown in Figure 10.2b. Consequently, softmax sampled high-value actions more frequently, thus providing more accurate estimates of $q(a)$ close to the optimum.

After preliminary experimentation, three learning rates, $\tau_n = 5/(20 + n)$, $\tau_n = 0.5n^{-0.5}$ and $\tau_n = 0.05/(1 + (10^{-5})n^2)$, were chosen for further analyses. The third choice for τ_n is justified later in this chapter. Note that estimates based on $\tau_n = n^{-1}$ and $\tau_n = 0.5n^{-0.75}$ converged too quickly, producing very inaccurate estimates of $q(a)$ and the optimal order quantity.

This study used the scenarios in the model description above and generated 40 replicates for each using common random number seeds for each replicate. For each scenario, the recursions began with $q(a) = 0$, set $N = 50{,}000$ and the softmax parameter $\eta_n = 1/6{,}000$ for all n. The number of times an action was previously chosen (as opposed to the iteration number) was used as the index of the learning rate.

Table 10.3 summarizes results in terms of the mean and standard deviation of the optimal order quantify over 40 replicates for each configuration of distribution, price and learning rate. The learning rate $\tau_n = 0.5n^{-0.5}$ best estimated the optimal order quantity on average. However, the estimates were more variable when the price equaled 100. Note that when $N = 10{,}000$, $\tau_n = 0.5n^{-0.5}$ produced the most accurate estimates

but their variability was greater.

Distribution	Price	Optimal	$\tau_n = 5/(20+n)$	$\tau_n = 0.05/\left(1+(10^{-5})n^2\right)$	$\tau_n = 0.5n^{-0.5}$
Normal	21	27	29.79 (2.89)	30.72 (3.97)	28.59 (2.73)
	30	46	48.64 (3.09)	48.41 (4.00)	46.62 (2.92)
	100	65	67.69 (5.91)	68.74 (6.70)	67.18 (8.01)
Gamma	21	55	57.62 (3.05)	57.15 (3.66)	55.18 (2.60)
	30	74	76.41 (3.91)	76.64 (4.22)	74.64 (4.54)
	120	98	99.87 (6.32)	104.13 (6.71)	99.15 (8.65)

Table 10.3: Optimal order quantity in the newsvendor model and mean (standard deviation) of estimates based on 40 replicates of Q-learning with three different learning rates.

Q-learning: An "Earning while learning" perspective

The above analysis is from the perspective of solving an optimization problem; that is, Q-learning is an online method to optimize the order quantity. Alternatively, from a simultaneous earning and learning perspective, the newsvendor might seek to maximize revenue while learning what quantity to order.

To illustrate this approach, consider an instance with normal demand, price equal 30, $N = 100{,}000$ iterates, the three learning rates used above, and softmax action selection with $\eta_n = 600n^{0.2}$. The index for the learning rate was the number of times an action was previously used and the index for softmax sampling was the iteration number. The effect of this specification for η_n is to allow random action choice at the beginning and then make it more likely that sampling is done from actions with high values of the estimate of $q(a)$, which is shown in Figure 10.4. Observe that in this replicate, the estimate of $q(a)$ is smoothest around the optimal order quantity where most of the actions are chosen.

Figure 10.5 shows the running average revenue in a typical replicate[18]. Observe that all three learning rates result in learning as indicated by increasing average revenue per iterate. Using the learning rate $\tau_n = 5/(20+n)$ resulted in the greatest revenue early on. Eventually, revenue streams from all three learning rates become approximately the same but that using $\tau_n = 5/(20+n)$ continued to have the greatest average revenue. Note that over 100,000 iterates even a small increase in average revenue contributes to a large increase in total revenue.

10.2.3 Conclusions

The purpose of this section was to provide an introduction to the simulation methods in this chapter. By choosing a practical and easy to understand **one-state** example, several implementation issues that arise in multi-state models were swept under the rug.

[18] In 40 random sequences, the average revenues always exhibited this pattern.

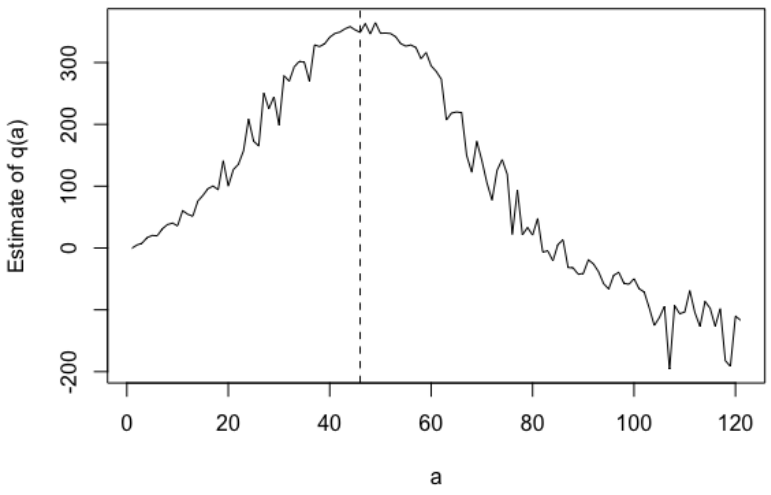


Figure 10.4: Estimate of $q(a)$ based on 100,000 iterates of Q-learning. The vertical dashed line corresponds to a^* found using (10.3).

However, this example was sufficiently rich to provide insight into issues arising when using these techniques. The remainder of this chapter generalizes these approaches to multi-state models.

Some noteworthy observations include:

1. Monte Carlo methods are easy to understand, however learning does not occur until all data has been collected. As the above calculations show, Monte Carlo methods involve costly experimentation for actions that may be far from optimal. However, because $q(a)$ is relatively flat near the maximum, large sample sizes near this value are useful. Note that Monte Carlo could be integrated with a search procedure to reduce experimentation.

2. Temporal differencing provides an online approach for policy value function estimation. This means that estimates are available after each observation. However, the quality of the approximation depends significantly on the learning rate. Note that $\tau_n = n^{-1}$ corresponds to an online version of Monte Carlo.

3. Q-learning worked best with softmax sampling and on-policy updates using (10.9). Again, outcomes were sensitive to the learning rates. After preliminary experimentation three promising learning rates were used in a more detailed study. Optimal order quantity estimates were quite similar with the exponential learning rate working slightly better than the others in determining an order quantity. From the perspective of learning, the ratio learning rate generated the greatest revenue. Differences in revenue over learning rates was greatest initially.

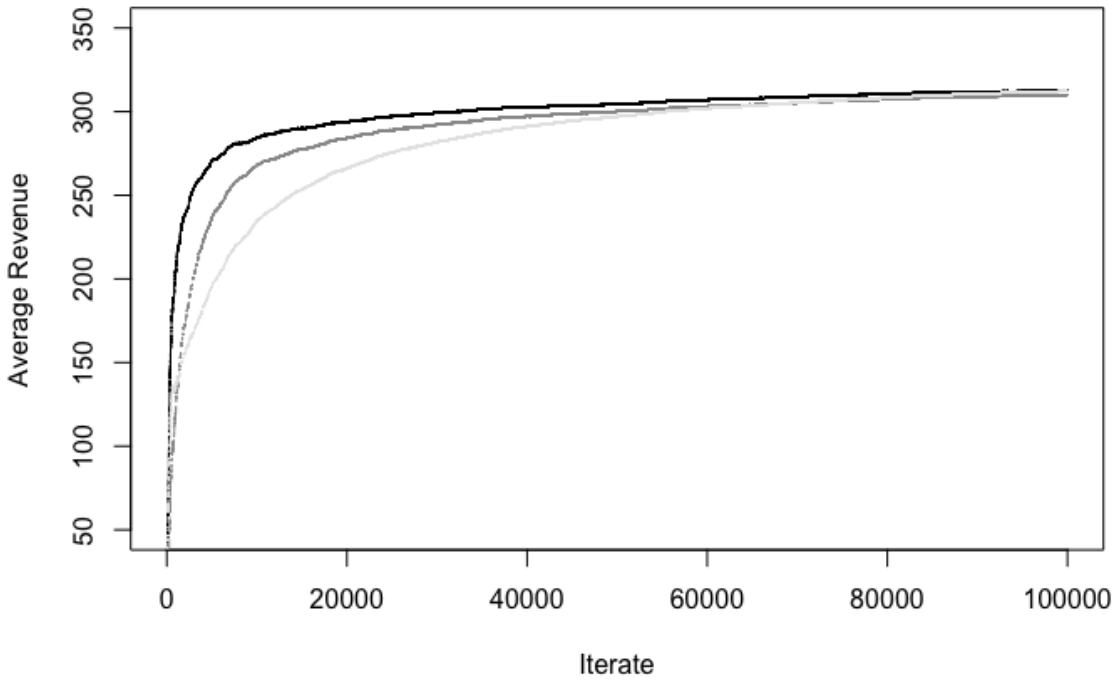


Figure 10.5: Cumulative average revenue vs. iterate using Q-learning. Data is from a single replicate of newsvendor model using softmax sampling and three learning rates. The black line corresponds to $\tau_n = 5/(20 + n)$, the dark grey line (between the other two) to $\tau_n = 0.05/(1 + (10^{-5})n^2)$ and the light grey to $\tau_n = 0.5n^{-0.5}$.

10.3 Policy evaluation: Episodic models

In an episodic model, the system accrues rewards until it reaches a reward-free absorbing state at some (possibly) random stopping time in the future. Usually this will be when a task is completed or a game ends. This section focuses on models in which stopping times are determined by the evolution of the system. For example, in a robot navigation model, an episode terminates when the robot achieves its goal[19].

As noted in Section 2.3.2, a discounted model may be viewed as an episodic model in which the episode terminates at a geometrically distributed stopping time independent of the evolution of the system. Policy evaluation for discounted models will be discussed in Section 10.4 below.

Finite horizon models may also be regarded to be episodic models in which the stopping time is fixed and after which the system transitions to a zero-reward absorbing

[19]The episode could end when the robot fails, but computation in various applications suggests that it is more expedient to terminate an episode only when the goal is achieved.

state. This chapter will not discuss finite horizon models[20] but many of the techniques here can be generalized to apply to them.

In episodic models, the expected total reward of stationary policy $\pi = d^\infty$ is given by

$$v^\pi(s) = E^\pi \left[\sum_{n=1}^{N-1} r(X_n, Y_n, X_{n+1}) + g(X_N) \middle| X_1 = s \right], \tag{10.12}$$

where the random variable N represents the random time at which the process terminates, X_n denotes the state at decision epoch n, Y_n denotes the action chosen at decision epoch n, X_{n+1} denotes the state at decision epoch $n+1$ that results from choosing action Y_n in state X_n, and $g(X_N)$ denotes the reward when terminating in state X_N[21]. The random variable $N = \min\{n \geq 1 \mid X_n \in \Delta\}$, sometimes referred to as the *effective horizon*, denotes the time the system enters a set of zero-reward absorbing states, Δ.

Chapter 6 established the optimality of deterministic policies[22] for such models. However, allowing for randomized policies broadens the applicability of the methods herein. Chapter 6 shows that in several classes of models[23], $v^\pi(s)$ is well-defined for most policies, including any policy that could be optimal.

10.3.1 Monte Carlo methods

The most obvious approach for estimating $v^\pi(\bar{s})$, for a fixed state $\bar{s}$, is to sum up the rewards for each episode and average over episodes. This leads to the following *Monte Carlo* algorithms, which differ according to which quantities are retained from each episode. Each assumes that an episode terminates when the system reaches a state in the set Δ.

Starting-state Monte Carlo

This simple Monte Carlo algorithm is stated for illustrative purposes. It produces an estimate for a single state $\bar{s}$ only. First-visit and every-visit variants are preferable and slightly more involved to state but easy to implement.

Algorithm 10.1. Starting-state Monte Carlo policy evaluation for an episodic model

[20] Chapter 4 showed that in finite horizon models, optimal policies are usually non-stationary.

[21] One could alternatively set $g(\cdot) = 0$ and include the termination reward in the definition of $r(X_{N-1}, Y_{N-1}, X_N)$. This is how the model was written in Chapter 6 but the representation with a specified terminal reward is more natural here.

[22] When T is stochastic, optimal policies will also be stationary; when T is fixed and constant, non-stationary policies are often optimal.

[23] Transient models, stochastic shortest path models, positive models and negative models.

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MR}}$ and the number of episodes K.
   (b) Specify $\bar{s} \in S \setminus \Delta$ and create an empty list VALUES.
   (c) $k \leftarrow 1$.
2. **Iterate:** While $k \leq K$:
   (a) $s \leftarrow \bar{s}$ and $v \leftarrow 0$.
   (b) **Generate episode:** While $s \notin \Delta$:
      i. Sample a from $w_d(\cdot|s)$.
      ii. Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
      iii. Update the value:
      $$v \leftarrow r + v. \tag{10.13}$$
      iv. $s \leftarrow s'$.
   (c) **Terminate episode ($s \in \Delta$):**
      i. Append $v + g(s)$ to VALUES.
      ii. $k \leftarrow k + 1$.
3. **Terminate:** Return
$$\hat{v}^{d^\infty}(\bar{s}) = \mathrm{mean}(\textsc{Values}).$$

Some comments about the algorithm follow:

1. The above algorithm estimates $v^{d^\infty}(\bar{s})$ by setting it equal to the average of observed total rewards for all episodes that start in state $\bar{s}$. In most cases, one would seek estimates for all $s \in S \setminus \Delta$.
2. If $d \in D^{\mathrm{MD}}$, step 2(b)i is replaced by "Set $a = d(s)$".
3. The algorithm statement includes a model-free variant in which state transitions and rewards are simulated or observed, and a model-based variant in which states are sampled and rewards computed using the triple (s, a, s').
4. Since the total rewards from different episodes are independent and identically distributed, $\hat{v}^{d^\infty}(\bar{s})$ converges almost surely to its mean, $v^{d^\infty}(\bar{s})$, by the *strong law of large numbers*, and is asymptotically normal by the *central limit theorem*.
5. By storing estimates of $v(\bar{s})$ in the list VALUES instead of updating the mean sequentially, one can investigate distributional properties of the estimates such as the standard deviation and percentiles. Moreover, one can compute standard

errors (i.e., standard deviation divided by the square root of the number of observations) and confidence intervals for the estimates, and use the width to guide the choice of K.

6. The algorithm as stated above *updates values only for the starting state* $\bar{s}$ of each episode. Obviously data would be used more efficiently if it also estimated $v(s)$ for *all* states visited during an episode. If a state repeats, the estimate for that state may be based on the total return accumulated from the first visit to that state (or from every visit to that state). If such an approach is used, $v(s)$ should be updated for *all* states previously visited in step 2(b). The first-visit algorithm is stated below.

7. Algorithm 10.1 is an offline algorithm since it requires all data to estimate $v^{d^\infty}(s)$ in step 3.

First-visit Monte Carlo

The following algorithm uses data more efficiently by storing cumulative rewards for each distinct state visited during an episode.

Algorithm 10.2. First-visit Monte Carlo policy evaluation for an episodic model

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MR}}$ and the number of episodes K.
   (b) For all $s \in S \setminus \Delta$ create an empty list VALUES(s).
   (c) $k \leftarrow 1$.
2. **Iterate:** While $k \leq K$:
   (a) Create empty list VISITED.
   (b) $v(s) \leftarrow 0$ for all $s \in S \setminus \Delta$.
   (c) Specify $s \in S \setminus \Delta$.
   (d) **Generate episode:** While $s \notin \Delta$:
      i. If $s \notin$ VISITED, append s to VISITED.
      ii. Sample a from $w_d(\cdot|s)$.
      iii. Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
      iv. For all $s'' \in$ VISITED

$$v(s'') \leftarrow r + v(s''). \tag{10.14}$$

      v. $s \leftarrow s'$.

(e) **Terminate episode ($s \in \Delta$):**

i. For all $s'' \in$ VISITED, append $v(s'') + g(s)$ to VALUES(s'').

ii. $k \leftarrow k + 1$.

3. **Terminate:** For all $s \in S \setminus \Delta$, return

$$\hat{v}^{d^\infty}(s) = \text{mean}(\text{VALUES}(s)).$$

Some comments about this algorithm follow:

1. The list VISITED keeps track of the *distinct* states visited during an episode and 2(d)iv updates the value for each.

2. An alternative to this algorithm would be an every-visit version, which in each replicate starts a new total every time a state is visited. That is, if the sequence of states was $(s_1, s_2, s_5, s_2, \ldots)$, such an algorithm would generate two estimates for s_2, one beginning from the first visit and one beginning form the second visit. It is left to the reader to formally state and apply such an algorithm.

An example

This section illustrates the starting-state and first-visit Monte Carlo algorithms by evaluating a policy for the Gridworld model of Section 3.2. The calculations assume an episode ends when *either* the robot reaches the office (cell 1) or falls down the stairs (cell 7). Thus $S = \{1, \ldots, 15\}$ and $\Delta = \{1, 7\}$. The cell configuration diagram is repeated in Figure 10.6 below.

Consider the Gridworld model represented in Figure 10.6 with $p = 0.8$, a cost of 200 for falling down the stairs, a reward of 50 for successfully delivering the coffee and a per unit move cost of 1. This example analyzes the stationary policy d^∞ in which the robot seeks to move to the lowest numbered neighboring cell except when in state 10 when it instead tries to move toward cell 11. It moves to the intended cell with probability p and to each other adjacent cell with equal probability. The formulation assumes that the robot cannot move diagonally. For example, from cell 14, the robot moves to cell 11 with probability p and to cells 13 or 15 with probability $(1 - p)/2$.

The example compares Algorithm 10.1 and Algorithm 10.2, using $K = 2{,}000$ replicates for each starting state. Figures 10.7a and 10.7b show the estimates of the values starting in each state using the two algorithms.

Observe that estimates based on first-visit Monte Carlo had narrower simultaneous 95% confidence intervals[24] than those based on starting-state Monte Carlo. For example, the standard errors when starting in cell 13 were 2.34 for starting-state and 2.07

[24]Such confidence intervals are constructed so that in 95% of samples all intervals contain the true mean. A Bonferroni correction (0.025 divided by the number of estimates) is used to obtain the appropriate t-value.

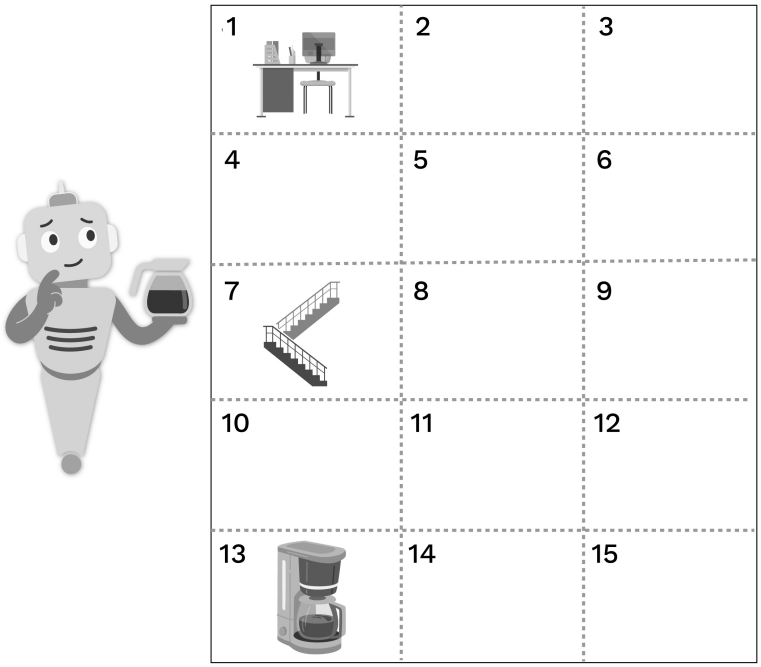


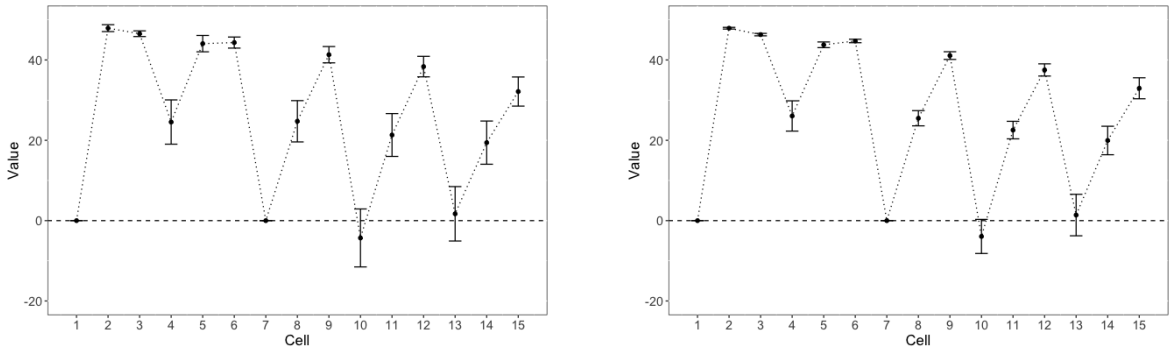
Figure 10.6: Schematic layout for Gridworld navigation example.

for first-visit. When starting in cell 2, the respective standard errors were 0.30 and 0.08. Thus, as expected, the first-visit Monte Carlo algorithm provides more accurate estimates with the same number of replicates especially for the most frequently visited states.

Figure 10.8 provides a histogram of values for cell 13 when using first-visit Monte Carlo. Observe that the distribution is extremely bi-modal corresponding to the possibilities of falling down the stairs or successfully delivering the coffee. Consequently, the mean does not represent this distribution well and is perhaps a poor choice of a value of a policy. Note that the 5th percentile is -204, the 25th percentile is 38, the median is 42 and the 75th percentile and 95th percentiles equal 44. Thus, one might wish to consider the use of different summaries of the distribution (such as the median or 75th percentile) when analyzing such models.

10.3.2 Temporal differencing

Temporal differencing represents one of the most significant innovations in reinforcement learning. It refers to methods that update policy value function estimates based



(a) Starting-state Monte Carlo

(b) First-visit Monte Carlo

Figure 10.7: Value function estimates for the specified policy based on 2,000 replicates for each starting state and $p = 0.8$. Error bars in Figures 10.7a and 10.7b represent simultaneous (Bonferroni) 95% confidence intervals.

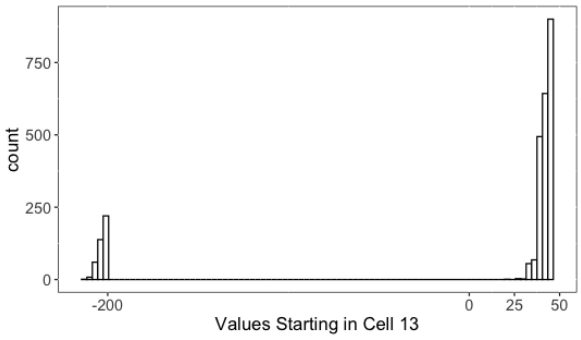


Figure 10.8: Histogram of estimates of $v(13)$ using Algorithm 10.2. Note the mean of these estimates is 0.64.

on a fraction of the difference between a current estimate and a one-step prediction equal to an observed reward and the value in a successor state. The difference is called the *temporal difference*. The term, temporal differencing, has its roots in behavioral neuroscience referring to situations in which actions are reinforced when predictions exceed expectations and degraded when they do not.

To motivate this approach, consider the operator

$$L_d\mathbf{v} := \mathbf{r}_d + \lambda \mathbf{P}_d\mathbf{v}$$

on the set of real-valued vectors on S when using a fixed decision rule $d \in D^{\text{MR}}$. When $0 \leq \lambda < 1$, it plays a fundamental role in discounted models (Chapter 5), and when $\lambda = 1$, it is fundamental in transient models and applies to proper policies in stochastic shortest path models (Chapter 6). In these models, value iteration, expressed as

$$\mathbf{v}' = L_d\mathbf{v} \tag{10.15}$$

converges to the unique fixed point of L_d, which was shown to equal the value function of d^∞. Rewriting (10.15) as

$$\mathbf{v}' = \mathbf{v} + (L_d\mathbf{v} - \mathbf{v}) \tag{10.16}$$

provides a conceptually different approach for thinking of value iteration, namely:

> new estimate = old estimate + correction.

A related numerical iterative scheme, referred to as *successive over-relaxation*, modifies value iteration by introducing a scaling factor τ satisfying $0 \leq \tau \leq 1$ that down weights the correction to obtain:

$$\mathbf{v}' = \mathbf{v} + \tau(L_d\mathbf{v} - \mathbf{v}). \tag{10.17}$$

It is not hard to show that the expression on the right hand side of (10.17) is a contraction mapping and that it also converges to the unique fixed point of L_d.

Suppose instead of computing $L_d\mathbf{v}$ directly, one instead observes a random sample $\mathbf{u}$ from a distribution with expectation[25] $L_d\mathbf{v}$, then the obvious generalization of (10.17) becomes

$$\mathbf{v}' = \mathbf{v} + \tau\boldsymbol{\delta} \tag{10.18}$$

where

$$\boldsymbol{\delta} = \mathbf{u} - \mathbf{v}.$$

Temporal differencing applies such an approach on a state-by-state basis by choosing $s \in S$, using the decision rule $d(s)$ to obtain action $a \in A_s$, observing r' and s', and setting:

$$\delta = r' + \lambda v(s') - v(s)$$

and

$$v'(s) = v(s) + \tau\delta.$$

In this expression, δ is the temporal difference. This approach uses a concept that is often referred to as *bootstrapping*, namely it uses current value estimates to update other value estimates.

Stochastic approximation using whole trajectories

As a different motivation for temporal differencing, consider updates obtained using stochastic approximation based on *complete* episodes. Appendix 10.11.2 of this chapter introduces the following recursion for estimating the mean μ of a random variable:

[25] This means that $E[\mathbf{u}] = L_d\mathbf{v}$, for example when $\mathbf{u} = L_d\mathbf{v} + \boldsymbol{\epsilon}$ with $E[\boldsymbol{\epsilon}] = \mathbf{0}$.

$$\mu^{n+1} = \mu^n + \tau_n\big(y^n - \mu^n\big). \tag{10.19}$$

The appendix shows that this recursion is the result of applying stochastic gradient descent to obtain least squares estimates of a mean. The parameters $\tau_n, n = 1, 2, \ldots$ are referred to as *learning rates* or *step-sizes*.

Convergence of μ^n to μ occurs with probability 1 if the following conditions[26] are satisfied.

Definition 10.1. The sequence $\tau_n, n = 1, 2, \ldots$, satisfies the *Robbins-Monro step-size conditions* if

$$\tau_n \geq 0, \quad \sum_{n=1}^{\infty} \tau_n = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \tau_n^2 < \infty. \tag{10.20}$$

Let $(h^1, h^2, \ldots)$ denote independent samples from a distribution $H(\cdot)$ with expectation equal to the right hand side of (10.12). Then, applying (10.19) provides recursive estimates of $v^{d^\infty}(s)$ given by

$$v^{n+1}(s) = v^n(s) + \tau_n\big(h^n - v^n(s)\big). \tag{10.21}$$

One way to produce these samples is by generating complete episodes and setting

$$h^n = \sum_{l=1}^{N^n} r_l^n$$

where r_l^n is the l-th reward in the n-th episode and N^n is the length of the n-th episode. The astute reader will recognize that this is equivalent to generating a sequence of starting-state Monte Carlo estimates and updating the value function estimate using (10.19) after each episode completes[27].

The downsides of this approach are that:

1. It delays updating the estimate until the completion of an episode.
2. It wastes considerable within-episode information.
3. A separate estimate must be obtained for each $s \in S$.

[26] These are frequently referred to simply as the *Robbins-Monro conditions* or the *usual conditions*.

[27] Sutton and Barto (2018) refers to such an approach as Monte Carlo-α and h^n as the *target* of the update.

An online alternative

To address these concerns, instead of basing updates on an entire reward sequence, temporal differencing bases updates on the (one-step) evaluation equation

$$v(s) = E^d\left[r(X, Y, X') + v(X') \,\middle|\, X = s\right] = E^d\left[r(s, Y, X') + v(X')\right] \tag{10.22}$$

derived in (6.23). In this expression, the expectation is with respect to the distribution of the action a chosen by decision rule d in state s (when it is randomized) and the resulting transition probabilities $p(\cdot|s, a)$.

An implementable recursion is based on replacing the sampled trajectories h^n in (10.21) by the results of a single simulation step. In this step, a is deterministically chosen as $a = d(s)$ or sampled from the randomized distribution $w_d(\cdot|s)$. The successor state s' and reward r are jointly simulated or s' is sampled from $p(\cdot|s, a)$ and the reward is computed by $r = r(s, a, s')$. This enables direct calculation of $r + v^n(s')$. Thus, the following recursion

$$v^{n+1}(s) = v^n(s) + \tau_n\big(\, r + v^n(s') - v^n(s)\,\big) \tag{10.23}$$

provides an online update of an estimate of $v(s)$ without waiting for an episode to complete.

Often this recursion appears as

$$v^{n+1}(s) = v^n(s) + \tau_n \delta^n \tag{10.24}$$

where

$$\delta^n := r + v^n(s') - v^n(s).$$

As noted above, the quantity δ^n is referred to as a temporal difference.

When applying this recursion, the usual approach is to start in some state, generate a transition using the policy, update value estimates using (10.23) and terminate when reaching a reward-free absorbing state. However, to obtain accurate estimates in each state, one would need to repeat this over multiple episodes by restarting this process in a state chosen either deterministically or randomly.

The following subtleties arise when using this recursion:

1. **Approximation:** This recursion uses *bootstrapping*[28]. That is, it uses an approximation of the value function $v^n(\cdot)$ at each step instead of a known quantity.

[28] This use of the expression bootstrapping here is consistent with the reinforcement learning literature. In the statistical literature, bootstrapping refers to methods based on resampling from a given set of data.

2. **Multiplicity of successor states:** The recursion requires estimates of $v^n(s')$ for all states reachable from s in one step under d.

3. **Asynchronicity:** The quality (variance) of the approximations will vary across states. Values of states frequently visited under d will be estimated accurately while those infrequently visited will be less accurate.

As a consequence of these remarks, a rigorous theoretical analysis of this algorithm requires methods above and beyond those used to establish convergence of stochastic approximation. In spite of this, many proofs of the following result are available[29]. This issue is explored in Chapter 11 in a more transparent setting where value functions are parameterized by a vector $\boldsymbol{\beta}$ and a distinction is made between gradients and semi-gradients.

Theorem 10.1. Let $\mathbf{v}^{d^\infty}$ denote the unique solution to (10.22). Then for every $\mathbf{v}^0$, if every state is visited infinitely often and τ_n satisfies (10.20), $\mathbf{v}^n$ converges to $\mathbf{v}^{d^\infty}$ with probability 1.

Note that the expression "with probability 1" can be interpreted as "on almost every sample path", where sample paths consist of sequences of states, actions and rewards determined by the decision rule, the transition probabilities, and the distribution used to start a new episode. To ensure that every state is visited infinitely often, after reaching a terminal state, the system must be restarted using a distribution that ensures all states are reached with positive probability.

Example 10.1. The example depicted in Figure 10.9 illustrates the above challenges of applying (10.23). In this example of an episodic model, there is a single stationary policy that uses action a_1 in s_1, a_2 in s_2 and a_3 in s_3. Actions choose arcs with the indicated probabilities and generate the indicated rewards. Note s_3 is a zero-reward absorbing state. When s_3 is reached, an episode ends.

Suppose the system starts in s_1, then in 999 out of 1,000 trajectories will immediately jump to s_2, remain there for on average 500 transitions and then jump to either s_1 or s_3 with equal probability. Therefore the value of s_2 will be accurately estimated but the value of s_1 need not be.

State-based sampling provides an attractive alternative to requiring a large number of episodes to accurately estimate $v(s_1)$. Of course, in a "real-world" system such restarts may not be possible.

[29]The proofs of this result are beyond the level of this book; references appear in the Bibliographic Remarks section.

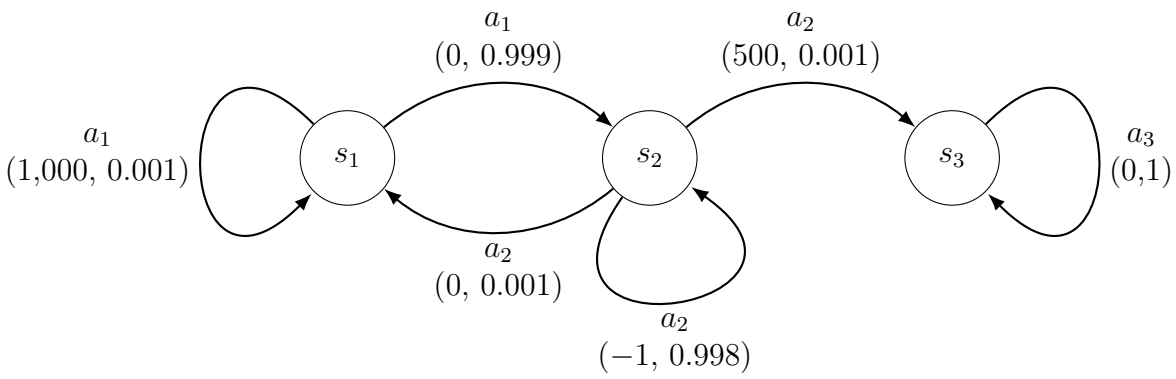


Figure 10.9: Graphical representation of model in Example 10.1. Numbers in parentheses represent $(r(s,a,s'), p(j|s,a))$.

Temporal differencing in algorithmic form

This section describes an algorithm to implement (10.23) in an episodic model. The algorithm evaluates a randomized stationary policy d^∞, and combines both model-free and model-based implementations.

Algorithm 10.3. Temporal differencing for policy evaluation for an episodic model

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MR}}$.
   (b) Specify the number of episodes K.
   (c) Specify the learning rate sequence $\tau_n, n = 1, 2, \ldots$.
   (d) $v(s) \leftarrow 0$ for all $s \in S$.
   (e) $\mathrm{count}(s) \leftarrow 0$ for all $s \in S$.
   (f) $k \leftarrow 1$.
2. **Iterate:** While $k \leq K$:
   (a) Specify $s \in S \setminus \Delta$.
   (b) **Generate episode:** While $s \notin \Delta$:
      i. $\mathrm{count}(s) \leftarrow \mathrm{count}(s) + 1$.
      ii. Sample a from $w_d(\cdot|s)$.
      iii. Simulate (s', r) or sample s' from $p(\cdot|s,a)$ and set $r \leftarrow r(s,a,s')$.

iv. Evaluate temporal difference

$$\delta \leftarrow r + v(s') - v(s). \tag{10.25}$$

v. Set

$$v(s) \leftarrow v(s) + \tau_{\text{count}(s)}\delta. \tag{10.26}$$

vi. $s \leftarrow s'$.

(c) $k \leftarrow k + 1$.

3. **Terminate:** Return $v(s)$ for all $s \in S$.

Some comments about Algorithm 10.3 follow:

1. Temporal differencing is an online algorithm with asynchronous updates.
2. It provides estimates of the policy value function for every state visited during at least one episode when episodes are replicated.
3. In contrast to Monte Carlo estimation in which value function updates occur at the end of an episode, this algorithm updates the value function estimate within each episode after each transition. At the end of an episode, a new initial state is generated in step 2(a).
4. The initial state may be generated by any, possibly random, mechanism that is appropriate for a problem. For example, it is often advantageous to start far from absorbing states so that many states may be visited (and the values corresponding to those states updated) during each episode. However, a random start over all states may provide accurate estimates for states visited infrequently under decision rule d. This will be particularly relevant when seeking an optimal policy.
5. Monte Carlo methods might be used to obtain good starting values for $v(s)$ before applying temporal differencing.
6. The algorithm uses the number of visits to a state as the index for the learning rate parameter τ_n to account for asynchronicity in updates.
7. Choosing the learning rate τ_n is somewhat of an art. A systematic approach (sometimes referred to as a *parameter study*) that combines RMSE with graphical summaries for guidance is described in the chapter appendix 10.11.1 and illustrated in examples below. The next section expands on this issue.

8. After applying (10.26) in state s, it is possible to update values in states previously visited during an episode. This is the idea underlying TD(γ) algorithms, which are described below.

9. In step 2(b)iii, the next state s' (and the reward r) may be directly obtained using a simulator in a model-free environment or generated from transition probabilities and rewards in a model-based environment. Note that when d is deterministic, step 2(b)ii becomes "Set $a = d(s)$".

Learning rate choice

The following observation[30] is consistent with our experience choosing the learning rates (step-sizes): *"Step-size rules are important. As you experiment with ADP[31], it is possible to find problems where provably convergent algorithms simply do not work, and the only reason is a poor choice of step-sizes."*

Ideally, learning rates should produce fast learning early and slow learning later. Theorem 10.1 provides conditions (10.20) that do this and as well guarantee convergence of temporal differencing. However, ensuring that learning rates satisfy these conditions is not sufficient to obtain accurate estimates in practice.

Learning rate sequences that satisfy the Robbins-Monro conditions include:

Polynomial:

$$\tau_n = \alpha n^{-\beta}$$

where $0.5 < \beta \leq 1$. Note that $1/n$ is a special case corresponding to $\alpha = \beta = 1$.

Ratio:

$$\tau_n = \frac{\alpha}{\beta + n}$$

for $\alpha > 0$ and $\beta \geq 0$. Note that this sequence starts at $\alpha/(\beta + 1)$. Moreover when $\alpha = 1$ and $\beta = 0$ this reduces to $1/n$.

Search-then-converge (STC)[32]:

$$\tau_n = \alpha \left(\frac{1 + \frac{\beta}{\alpha}\frac{n}{\gamma}}{1 + \frac{\beta}{\alpha}\frac{n}{\gamma} + \frac{n^2}{\gamma}} \right)$$

When n is much smaller than γ, τ_n is nearly constant while for large n it behaves as β/n. Notice that choosing $\beta = 0$ makes τ_n decay more quickly.

Log:

$$\tau_n = \log(n + 1)/n$$

A multiplicative constant can be added to increase flexibility.

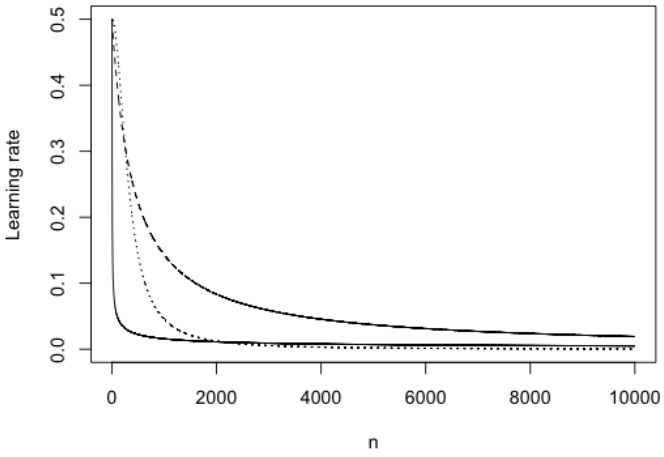


Figure 10.10: Graphical representation of three learning rate sequences. The solid line represents the polynomial sequence with $\alpha = \beta = 0.5$, the dashed line indicates the ratio sequence with $\alpha = 200$ and $\beta = 400$ and the dotted line corresponds to the STC sequence with $\alpha = 0.5$, $\beta = 0.0001$ and $\gamma = 100{,}000$.

Figure 10.10 shows how the learning rate decays with n for three specific τ_n. Observe that for the indicated parameter values, the polynomial sequence decays very quickly, the ratio sequence decays more slowly and the STC sequence lies somewhere in between.

Note that the quantity n in the above specifications may represent either:

1. the iteration number, or
2. the number of times a state[33] has been visited.

The iteration number benefits from simplicity. However, in algorithms using trajectory-based sampling, in which some states are visited infrequently, using the number of times a state has been visited seems more reasonable. It ensures that infrequently visited states are updated at a larger learning rate than if the iteration number had been used. Of course this involves extra storage but when a tabular model specification is appropriate, this should not be problematic. When states, or state-action pairs, are sampled randomly, either specification can be used.

[30] Powell, p. 184.
[31] In this context, ADP refers to simulation-based methods.
[32] This approach was described in Darken et al. [1992].
[33] Or state-action pair when estimating a state-action value function.

Temporal differencing in the Gridworld model

This section applies Algorithm 10.3 to the Gridworld model analyzed above using Monte Carlo methods in Section 10.3.1. It estimates the value function for the policy specified in that example. It compares four learning rates over 40 replicates of 10,000 episodes each with common random number seeds for each replicate. Each episode starts in cell 13 (the coffee room) and terminates when the robot reaches either cell 1 or 7.

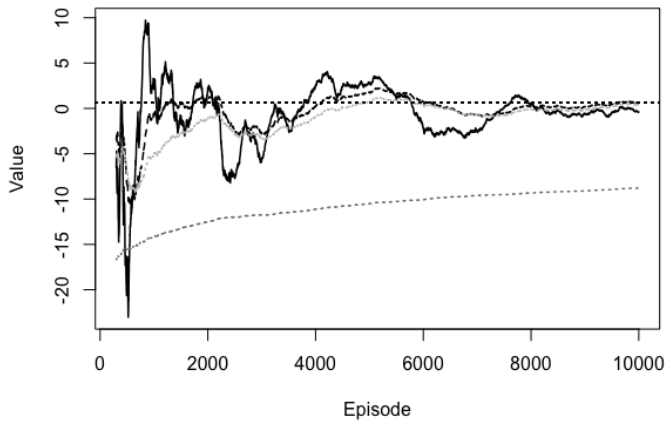


Figure 10.11: Estimates of $v^{d^\infty}(13)$ in the Gridworld model using Algorithm 10.3 based on a single replicate of 10,000 episodes. The figure shows the sequence of iterates for $\tau_n = 20/(50+n)$ (solid black), $\tau_n = \log(n+1)/n$ (dashed black), $\tau_n = 0.8n^{-0.75}$ (light grey) and $\tau_n = 0.8n^{-0.95}$ (dashed grey). The horizontal dotted line corresponds to the theoretical (true) value of this quantity 0.638.

Figure 10.11 displays a sequence of temporal difference estimates of $v^{d^\infty}(13)$, the expected total reward achieved by the robot starting in the coffee room. It shows iterates for the four choices of τ_n described in the figure caption. These were chosen based on some preliminary parameter studies.

Observe that $\tau_n = \log(n+1)/n$ and $\tau_n = 0.8n^{-0.75}$ produced the best estimates in this replicate. They converged quickly to the true value and oscillations damped out. Estimates using $\tau_n = 20/(50+n)$ oscillated around the true value but were more variable than the other two choices. That for $\tau_n = 0.8n^{-0.95}$ stabilized quickly and had low variance but was extremely inaccurate.

Figure 10.12 illustrates the variability in RMSE across 40 replicates where the

RMSE is computed over non-terminal states using

$$\text{RMSE} = \left(\frac{1}{13} \sum_{s \in S \setminus \Delta} \big(\hat{v}(s) - v(s) \big)^2 \right)^{\frac{1}{2}}. \tag{10.27}$$

In this expression, $\hat{v}(s)$ is based on the final episode for that state, while the true value is determined exactly using the methods from Chapter 6. This measure accounts for estimation error in all non-terminal states.

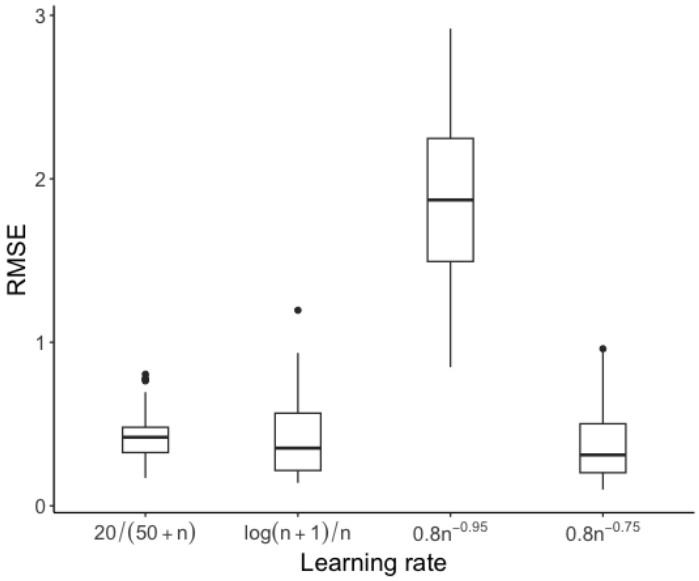


Figure 10.12: Boxplots of RMSE showing effect of learning rate when applying temporal differencing in Gridworld model.

It shows that the ratio estimate was least variable but slightly less accurate than those based on the logarithmic learning rate and the polynomial learning rate $0.8n^{-0.75}$. Clearly $\tau_n = 0.8n^{-0.95}$ was the worst choice. Note that RMSE weights all states equally. Alternative weightings are possible.

10.3.3 TD(γ)

The temporal differencing method above updates values one state at a time. Consequently, it ignores the effect of the current temporal difference on previously visited state values. This section describes an approach that overcomes this. It may be viewed as either an *offline or online method*.

As motivation, suppose in the Gridworld model (Figure 10.6) that the robot starts in cell 4 and visits cells 5, 6, 3, and 2 before terminating in cell 1. Recall that each step costs 1 and termination in cell 1 generates a reward of 50.

Consider updates starting in cell 6. After a transition to cell 3, the temporal difference update would be

$$v(6) \leftarrow v(6) + \tau(-1 + v(3) - v(6)).$$

But the value function could just as well be updated using a two-step update based on transitions from 6 to 3 and then 3 to 2 according to

$$v(6) \leftarrow v(6) + \tau(-1 - 1 + v(2) - v(6)),$$

or even a three-step update

$$v(6) \leftarrow v(6) + \tau(-1 - 1 - 1 + 50 + v(1) - v(6)).$$

Instead of selecting just one, it might be expedient to combine these three updates in some way. TD(γ)[34] provides a structured way to do so.

Two-step evaluation equations

The following formalizes the above argument in a model-based environment. The temporal difference algorithm above updates an estimate of the value of stationary policy d^∞ in state s according to

$$v(s) \leftarrow v(s) + \tau\big(r(s, a, s') + v(s') - v(s)\big),$$

where a is sampled from $w_d(\cdot|s)$ and the subsequent state s' is sampled from $p(\cdot|s, a)$. As noted above, this may be viewed as applying gradient descent to minimize the squared difference between the two sides of

$$v(s) = E^{d^\infty}\big[r(X_n, Y_n, X_{n+1}) + v(X_{n+1}) \,\big|\, X_n = s\big]. \tag{10.28}$$

Applying a similar recursion for $v(X_{n+1})$ and substituting it into (10.28) leads to the two-step evaluation equation beginning at decision epoch n:

$$v(s) = E^{d^\infty}\big[r(X_n, Y_n, X_{n+1}) + r(X_{n+1}, Y_{n+1}, X_{n+2}) + v(X_{n+2})\big|X_n = s\big]. \tag{10.29}$$

This suggests an alternative two-step update for $v(s)$ given by

$$\begin{aligned} v(s) &\leftarrow v(s) + \tau\big(r(s, a, s') + r(s', a', s'') + v(s'') - v(s)\big) \\ &= v(s) + \tau\big(r(s, a, s') + v(s') - v(s) + r(s', a', s'') + v(s'') - v(s')\big) \\ &= v(s) + \tau\big(\delta + \delta'\big), \end{aligned} \tag{10.30}$$

[34] Note that most of the literature refers to this approach as TD(λ) but since this book uses λ to denote the discount factor, it instead uses γ to represent the algorithmic parameter.

where a' denotes the realization of the action at decision epoch $n+1$ and s'' is a realization of the state at decision epoch $n+2$,

$$\delta = r(s,a,s') + v(s') - v(s) \quad \text{and} \quad \delta' = r(s',a',s'') + v(s'') - v(s').$$

This implies that the update of $v(s)$ is delayed until two decision epochs later.

However, this update can be viewed as two one-step updates as follows. After the first transition, update

$$v(s) \leftarrow v(s) + \tau_1 \delta$$

and leave the values the same for all other states. Then, in the next iteration, update

$$v(s) \leftarrow v(s) + \tau_2 \delta' \text{ and } v(s') \leftarrow v(s') + \tau_2 \delta'$$

and leave values for other states unchanged. Note that in contrast to the combined two-step update in (10.30), different learning rates can be used in each iteration.

Online TD(γ) modifies the second update for state s by reducing its impact by a factor of γ, $0 \leq \gamma \leq 1$, so that the second update becomes

$$v(s) \leftarrow v(s) + \tau_2 \gamma \delta' \quad \text{and} \quad v(s') \leftarrow v(s') + \tau_2 \delta'.$$

Multi-step and weighted evaluation equations

Repeating the above argument leads to the m-step evaluation equation (beginning at decision epoch n):

$$v(s) = E^{d^\infty}\left[\sum_{k=0}^{m-1} r(X_{n+k}, Y_{n+k}, X_{n+k+1}) + v(X_{n+m}) \middle| X_n = s\right]. \tag{10.31}$$

Define

$$R_n^m := \sum_{k=0}^{m-1} r(X_{n+k}, Y_{n+k}, X_{n+k+1}) + v(X_{n+m}) \tag{10.32}$$

for $m \geq 1$. The random variable R_n^m represents the total reward over decision epochs $n, n+1, \ldots, n+m-1$ plus a terminal reward $v(X_{n+m})$. These cumulative rewards may be viewed as the result of *rolling out* a policy over multiple periods starting at decision epoch n. Observe that (10.31) can be rewritten compactly as

$$v(s) = E\left[R_n^m | X_n = s\right].$$

Instead of arbitrarily combining these rollouts or using a pre-specified number of them, an option is to use the following exponential weighted average:

$$v(s) = E\left[\sum_{m=1}^{\infty} (1-\gamma)\gamma^{m-1} R_n^m \,\middle|\, X_n = s\right]. \tag{10.33}$$

From (10.32), R_n^m is itself a summation, so interchanging the order of summation as carried out in Appendix 10.11.4 leads to the following equivalent representation for $v(s)$:

$$v(s) = v(s) + \sum_{m=0}^{\infty} \gamma^m D_{n+m}, \tag{10.34}$$

where

$$D_{n+m} := E\big[r(X_{n+m}, Y_{n+m}, X_{n+m+1}) + v(X_{n+m+1}) - v(X_{n+m})|X_n = s\big].$$

Offline TD(γ)

Equation (10.34) provides the basis for deriving an offline representation for TD(γ). As shown in Appendix 10.11.4, the offline TD(γ) representation becomes

$$v(s^n) \leftarrow v(s^n) + \tau \sum_{m=n}^{\infty} \gamma^{m-n} \delta_m, \tag{10.35}$$

where

$$\delta_k := r(s^k, a^k, s^{k+1}) + v(s^{k+1}) - v(s^k).$$

Under the assumption of a transient model or a proper policy in a stochastic shortest path model, the rewards based on a trajectory $(s^1, a^1, s^2, \ldots)$ equal zero after some finite number of terms. Therefore, even though (10.35) has an infinite number of terms, they will equal zero after some trajectory-specific index N so that the above sum can be evaluated.

This can be applied offline in a similar way to a sequential implementation of first-visit or every-visit Monte Carlo over many trajectories. To do so, start with initial values of $v(s)$ for all $s \in S \setminus \Delta$. After each trajectory is completed, update the values for all states visited during that trajectory using (10.35) with τ decreasing over trajectories.

Observe that when $\gamma = 0$, (10.35) becomes

$$v(s^n) \leftarrow v(s^n) + \tau \delta_n,$$

which is exactly the temporal differencing update encountered earlier. Thus:

Temporal differencing is equivalent to TD(0).

When $\gamma = 1$, (10.35) becomes

$$v(s^n) \leftarrow v(s^n) + \tau \sum_{m=n}^{\infty} \delta_m = (1-\tau)v(s^n) + \tau \sum_{m=n}^{\infty} r(s^m, a^m, s^{m+1}).$$

Since the quantity in the sum is a Monte Carlo estimate of the total reward starting in state s^n, TD(1) is equivalent to a stochastic approximation method that updates the value function estimate for each state using the Monte Carlo update as in (10.21). Thus:

A variant of Monte Carlo estimation is equivalent to TD(1).

From a theoretical perspective[35], offline (batch) TD(γ) converges with probability one to $v^{d^\infty}(s)$ for all $s \in S \setminus \Delta$ provided there are no inaccessible states under the initial distribution, $v^{d^\infty}(s)$ is finite for all $s \in S \setminus \Delta$, and the Robbins-Monro step-size conditions hold.

Online TD(γ)

To obtain an online version of TD(γ) requires a change in perspective. In each episode, it uses the most recently observed temporal difference to update values in *all previously* visited states in that episode. This is in contrast to the temporal differencing method described above which updates values for the most recently visited state. The weightings are chosen to be consistent with the geometric weightings in offline TD(γ).

Suppose that, under a specified stationary policy, the states visited during an episode of length N are denoted by $(s^1, s^2, \ldots, s^N)$. Then after observing s^n and computing δ_n, online TD(γ) updates the policy value function in states $(s^1, s^2, \ldots, s^n)$ according to:

$$\begin{aligned}
v(s^n) &\leftarrow v(s^n) + \tau_n \delta_n \\
v(s^{n-1}) &\leftarrow v(s^{n-1}) + \tau_n \gamma \delta_n \\
&\vdots \\
v(s^{n-k}) &\leftarrow v(s^{n-k}) + \tau_n \gamma^k \delta_n \\
&\vdots \\
v(s^1) &\leftarrow v(s^1) + \tau_n \gamma^{n-1} \delta_n.
\end{aligned} \tag{10.36}$$

[35] See Dayan and Sejnowski [1994] which establishes this result for models with linear value function approximations, tabular models being a special case.

Observe that the update in state s^n is the same as in the temporal differencing algorithm and that the temporal difference δ_n is used to update all previously visited states.

It is important to note that when the current state s^n has been visited more than once in a trajectory, the above scheme updates its value multiple times. An alternative approach would be to only update the value *once* using the temporal differencing update $v(s^n) \leftarrow v(s^n) + \tau_n \delta_n$.

Eligibility traces

The *eligibility trace* is an innovation that assigns the appropriate weighting of the current temporal difference when updating values for previously visited states. Two variants have been proposed, the *accumulating eligibility trace* and the *replacement eligibility trace*, differing with respect to how values in repeat observations are updated. Often they will be referred to as simply an accumulating trace or a replacement trace.

More formally, an eligibility trace is a sequence of non-negative real-valued functions defined on a sequence of states $(s^1, s^2, \ldots)$ in $S \setminus \Delta$. The accumulating trace, which corresponds to updating values corresponding to all previous visits to the current state, is defined recursively by $e_1(s) = I_{\{s^1\}}(s)$ and

$$e_n(s) := \gamma e_{n-1}(s) + I_{\{s^n\}}(s). \tag{10.37}$$

Observe that for $n = 1$,

$$e_1(s) = \begin{cases} 1 & \text{for} \quad s = s^1 \\ 0 & \text{for} \quad s \neq s^1 \end{cases}$$

and $n > 1$,

$$e_n(s) = \begin{cases} \gamma e_{n-1}(s) + 1 & \text{for} \quad s = s^n \\ \gamma e_{n-1}(s) & \text{for} \quad s \neq s^n. \end{cases} \tag{10.38}$$

Using the eligibility trace as defined above, the sequence of updates (10.36) for $s \in S \setminus \Delta$ can be written in condensed form as:

$$v(s) \leftarrow v(s) + \tau_n \delta_n e_n(s). \tag{10.39}$$

The replacement trace modifies (10.38) for $n > 1$ to be

$$e_n(s) = \begin{cases} 1 & \text{for} \quad s = s^n \\ \gamma e_{n-1}(s) & \text{for} \quad s \neq s^n. \end{cases} \tag{10.40}$$

The effect of this modification becomes clear by using it in (10.39). It is easy to see that the consequence of using the replacement trace is that the update of $v(s^n)$ at iteration n is $v(s^n) \leftarrow v(s^n) + \tau_n \delta_n$, whereas when using the accumulating trace the expression $\tau_n \delta_n$ would be multiplied by a polynomial in γ.

An example

The following example[36] compares online and offline TD(γ) and also illustrates the effect of trace types on online updates. Consider a two-state model with a single policy. The state space consists of a state s_0 and a zero-reward absorbing state Δ. There is a single policy that generates a reward of r in s_0 at each decision epoch and the system remains in s_0 with probability p or jumps to Δ with probability $1-p$.

Since there is a single non-absorbing state, all temporal differences prior to absorption in Δ simplify to

$$\delta_k = r + v(s_0) - v(s_0) = r.$$

It is easy to see that $v(s_0) = r/(1-p)$, but that is not the point of this example. Suppose the system starts in s_0 and is absorbed in Δ after two transitions so that the trajectory is (s_0, s_0, Δ). The following calculations compare online and offline TD(γ).

Initialize calculations with $v(s_0) = c$. Then the offline update corresponding to this trajectory is

$$v^{\text{offline}}(s_0) \leftarrow c + \tau(1+\gamma)r.$$

The online updates using the accumulating trace and assuming $\tau_1 = \tau_2 = \tau$ are $e_1(s_0) = 1$ and

$$v_1^{\text{online}}(s_0) \leftarrow c + \tau r$$

corresponding to the first transition and since $e_2(s_0) = 1 + \gamma$,

$$v_2^{\text{online}}(s_0) \leftarrow c + \tau r + \tau(1+\gamma)r = c + 2\tau r + \tau\gamma r$$

corresponding to the second transition.

Using the replacement trace, $e_1(s_0) = e_2(s_0) = 1$ so that the first update is the same as using the accumulation trace, but the second update becomes

$$v_2^{\text{online'}}(s_0) \leftarrow c + 2\tau r.$$

Observe that:

1. The initial condition $v(s_0) = c$ persists in all updates. Therefore, it must be set to $c = 0$ to ensure convergence to the appropriate quantity.

2. The online and offline updates differ by a factor that is linear in τ.

3. The two online updates differ by a factor of $\tau\gamma$.

4. In trajectories of length n, the online update with replacement trace will be $c + n\tau r$. Since the expected total reward is $E[N]r$, the online estimate has exactly the same form as the quantity being estimated.

Consequently, if τ is small, estimates based on longer trajectories will be close.

[36] This is motivated by Example 5.7 in Bertsekas and Tsitsiklis [1996]. Their example has two non-absorbing states so it also shows the effect of the learning rate on transitions.

An online TD(γ) algorithm

The following algorithm implements online TD(γ). It incorporates the eligibility trace in the algorithmic flow. The precise update depends on whether an accumulating or replacement trace is used.

Algorithm 10.4. TD(γ) for policy evaluation for an episodic model

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MR}}$.
   (b) Set $v(s) \leftarrow 0$, $e(s) \leftarrow 0$ and $\mathrm{count}(s) \leftarrow 0$ for all $s \in S$.
   (c) Specify $0 \leq \gamma \leq 1$ and learning rate sequence $\tau_n, n = 1, 2, \ldots$.
   (d) Specify the number of episodes K and set $k \leftarrow 1$.
2. **Iterate:** While $k \leq K$:
   (a) Specify $s \in S \setminus \Delta$.
   (b) **Generate episode:** While $s \notin \Delta$:
      i. $e(s) \leftarrow e(s) + 1$ (accumulating trace) or $e(s) \leftarrow 1$ (replacement trace).
      ii. $\mathrm{count}(s) \leftarrow \mathrm{count}(s) + 1$.
      iii. Sample a from $w_d(\cdot|s)$.
      iv. Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
      v. Evaluate temporal difference
      $$\delta \leftarrow r + v(s') - v(s). \tag{10.41}$$
      vi. For all $s \in S$,
      $$v(s) \leftarrow v(s) + \tau_{\mathrm{count}(s)} \delta e(s) \tag{10.42}$$
      and
      $$e(s) \leftarrow \gamma e(s). \tag{10.43}$$
      vii. $s \leftarrow s'$.
   (c) $k \leftarrow k + 1$.
3. **Terminate:** Return $v(s)$ for all $s \in S$.

Some observations regarding this algorithm follow:

1. It is an online algorithm. It differs from Algorithm 10.3 in that at each state it updates values for **all** previously visited states. The magnitude of the update

decreases exponentially at rate γ.

2. A variant of the algorithm would reset eligibility traces to 0 at the end of each episode.

3. Small values of γ result in updating values for only a few previously visited states while values near 1 update this value far into the past.

4. The algorithm updates the eligibility trace online in steps 2(b)i and 2(b)vi. These ensure that the eligibility trace values agree with (10.38) and (10.40).

5. Convergence results apply to the algorithm implemented over infinitely many episodes. If every state is visited infinitely often over repeated episodes and the Robbins-Monro conditions are met, the online TD(γ) estimates converge[37] with probability one to the true policy value function.

An example

Before applying this algorithm in an example, examine how the eligibility trace updating works. Suppose in the Gridworld example the robot follows the trajectory $4 \to 5 \to 2 \to 5 \to 4 \to 1$. The above algorithm updates the eligibility trace at step 2(b)i prior to applying (10.42) and again in step 2(b)vi through (10.43).

Non-zero values of $e(s)$ (when using the accumulating trace) prior to applying (10.43) are given below:

1. $e(4) = 1$.

2. $e(5) = 1$, $e(4) = \gamma$.

3. $e(2) = 1$, $e(4) = \gamma^2$, $e(5) = \gamma$.

4. $e(5) = 1 + \gamma$, $e(2) = \gamma$, $e(4) = \gamma^3$.

5. $e(4) = 1 + \gamma^3$, $e(5) = \gamma(1 + \gamma)$, $e(2) = \gamma^2$.

Note that in step 5, $e(5)$ combines two terms to account for the two visits to cell 5.

If instead, the replacement trace had been used, the last two steps would be different. In step 4, $e(5) = 1$, and in step 5, $e(4) = 1$ and $e(5) = \gamma$. Note that if γ were near 1, the accumulating trace can exceed 1 in value.

The following example illustrates the application of TD(γ) to evaluate a policy in the Gridworld model.

[37] See Bertsekas and Tsitsiklis (1996) and Jaakkola et al. (1994) for details.

Example 10.2. Policy evaluation using TD(γ) in an episodic model

This example applies TD(γ) to the Gridworld model. Each episode starts in cell 13 and terminates when the robot reaches either cell 1 or cell 7. TD(γ) is used to again evaluate the policy that aims to move the robot to the lowest number adjacent cell, except in cell 10 when it attempts to move it to cell 11. It explores the effect of γ, trace type and learning rate on the accuracy of estimates.

Forty replicates of $K = 5{,}000$ episodes were compared for each combination of $\gamma \in \{0, 0.2, 0.4, 0.6, 0.8, 0.95\}$, accumulating and replacement eligibility trace and $\tau_n = 0.8n^{-0.75}$, $\tau_n = 20/(50+n)$, $\tau_n = \log(n+1)/n$ with the same seed in each replicate for all combinations. Results were assessed on the basis of the RMSE of the final estimates as defined in (10.27).

When $p = 0.8$, the most accurate estimates were obtained using $\tau_n = 0.8n^{-0.75}$, $0 \leq \gamma \leq 0.6$ with the trace type having little effect when γ was small (Figure 10.13). Observe also that for large values of γ, using the replacement trace resulted in more accurate estimates.

Moreover with this learning rate, the TD(γ) estimates were similar for $\gamma = 0, 0.2, 0.4$ with minimal differences between their accuracy. Consequently, there was minimal benefit using TD(γ) instead of temporal differencing (TD(0)).

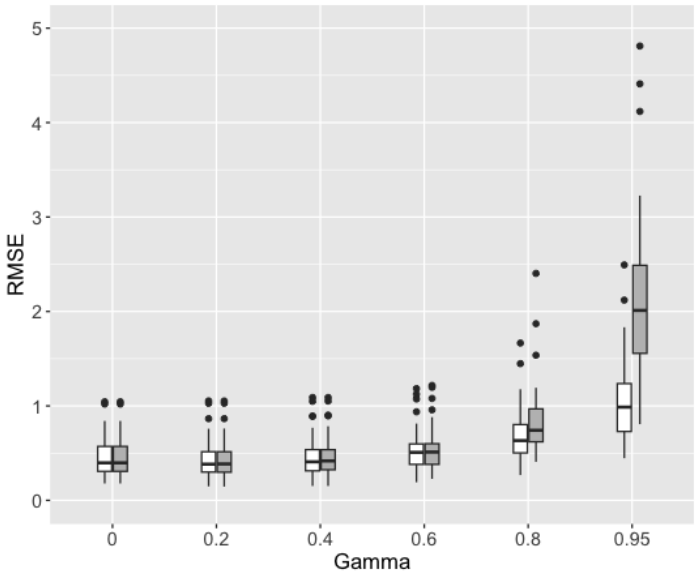


Figure 10.13: Boxplot of RMSE for Gridworld model showing the effect of γ and trace type (white represents replacement trace and grey represents accumulating trace) for learning rate $\tau_n = 0.8n^{-0.75}$.

10.4 Policy evaluation: Infinite horizon discounted models

This section discusses Monte Carlo and TD(γ) approaches for estimating the value of a policy in a discounted model.

10.4.1 Monte Carlo methods

Monte Carlo methods in an infinite horizon discounted model are based on either truncating the reward sequence or viewing the discount rate as the parameter of a geometrically distributed stopping time. Only starting-state Monte Carlo is applicable because the truncation index or geometric stopping time are based on the first decision epoch. For example, suppose an episode is truncated (or stopped) after 50 decisions, the system starts in state s^1 and visits states $s^2, s^3, \ldots, s^{50}$. Then an episode starting in state s^{25} would be based on accumulating only 25 rewards, while that starting in state s^1 would be based on accumulating 50 rewards. Therefore, an estimate based on s^{25} would not estimate the value function to the same precision as one based on s^1. This differs from the discussion above of episodic models where stopping occurs when a state in Δ is reached.

More formally, a policy value function in a discounted model may be written as

$$v_\lambda^{d^\infty}(s) = E^{d^\infty}\left[\sum_{n=1}^{\infty} \lambda^{n-1} r(X_n, Y_n, X_{n+1}) \,\middle|\, X_1 = s\right].$$

Because of the infinite sum, Monte Carlo methods cannot be applied directly to evaluate $v_\lambda^{d^\infty}(s)$. Instead, they can be based on approximating $v_\lambda^{d^\infty}(s)$ by either a *truncated sum of discounted rewards*

$$v_\lambda^{d^\infty}(s) \approx E^{d^\infty}\left[\sum_{n=1}^{M} \lambda^{n-1} r(X_n, Y_n, X_{n+1}) \,\middle|\, X_1 = s\right] \tag{10.44}$$

for some finite and sufficiently large M or the *undiscounted random sum* representation

$$v_\lambda^{d^\infty}(s) = E_T^{d^\infty}\left[\sum_{n=1}^{T} r(X_n, Y_n, X_{n+1}) \,\middle|\, X_1 = s\right], \tag{10.45}$$

where T is an independent geometric random variable with parameter 1–λ. In the latter case, the expectation is with respect to the probability distribution of $(X_1, Y_1, X_2, \ldots)$ generated by the specified policy and T. In (10.45), the discount factor appears implicitly in the expectation of T. Note that equation 10.45 is exact while 10.44 is an approximation.

Truncated reward sequences

Given a realization of states and actions, $(s^1, a^1, s^2, \ldots)$, at issue is when to truncate $\sum_{n=1}^{\infty} \lambda^{n-1} r(s^n, a^n, s^{n+1})$. This can be done *a priori* by choosing M so that

$$\lambda^{M-1} \max_{s \in S, a \in A_s, s' \in S} |r(s, a, s')| \leq \epsilon \tag{10.46}$$

for some pre-specified tolerance ϵ. Alternatively, M can be chosen arbitrarily or by stopping when the change in values of successive estimates is small.

Algorithm 10.5. Monte Carlo policy evaluation for a discounted model using truncation

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MR}}$.
   (b) Specify the number of replicates K and set $k \leftarrow 1$.
   (c) Specify the truncation level M.
   (d) Specify $\bar{s} \in S$.
   (e) Create an empty list VALUES.
2. **Replicate:** While $k \leq K$:
   (a) $s \leftarrow \bar{s}$, $m \leftarrow 1$ and $u \leftarrow 0$.
   (b) **Generate a replicate:** While $m \leq M$:
      i. Sample a from $w_d(\cdot|s)$.
      ii. Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
      iii. $u \leftarrow u + \lambda^{m-1} r$.
      iv. $s \leftarrow s'$.
      v. $m \leftarrow m + 1$.
   (c) **Update:**
      i. Append u to VALUES.
      ii. $k \leftarrow k + 1$.
3. **Terminate:** Return

$$\hat{v}_{\lambda}^{d^\infty}(\bar{s}) = \mathrm{mean}(\textsc{Values}).$$

Some comments about the algorithm follow:

1. This algorithm uses the expression *replicate* to refer to a single realization of a

truncated reward series in contrast to the use of the expression *episode* in episodic Monte Carlo.

2. As stated, the algorithm updates *online* within each replicate to obtain an estimate u to append to the list of values generated so far. Alternatively, this algorithm can be implemented *offline* by generating a sequence $(r^1, \ldots, r^M)$ and setting $u = \sum_{m=1}^{M} \lambda^{m-1} r^m$.

3. As stated, the algorithm generates a list VALUES. The advantage of doing so is that it can be used to investigate distributional properties of estimators.

4. After evaluating u based on a complete replicate, the estimate of $v_\lambda^{d^\infty}(s)$ can be updated using the stochastic approximation recursion:

$$v(\bar{s}) \leftarrow v(\bar{s}) + \tau_n\big(u - v(\bar{s})\big). \tag{10.47}$$

Of course, choosing $\tau_n = n^{-1}$ gives an estimate of the mean.

Geometric stopping times

Using representation (10.45) for $v_\lambda^{d^\infty}(s)$ is equivalent to terminating each episode at the first "success" in a sequence of Bernoulli trials with success probability $1-\lambda$.

To implement this approach modify Algorithm 10.5 as follows:

1. At the start of each replicate, generate a random stopping time from a geometric distribution[38].

2. Set $\lambda = 1$ in 2(b)iii so that the update is $u \leftarrow u + r$.

An example

The following example compares variants of Monte Carlo estimation using the two-state model of Section 2.5. Of course it would not be prudent to use simulation in such a simple model but it provides simple illustrations of the issues arising when estimating policy value functions for a discounted model using simulation.

Example 10.3. Comparison of truncation and random geometric stopping time estimates

This example analyzes the two-state model using Monte Carlo estimation with truncation and geometric stopping times. It evaluates the deterministic stationary policy d^∞ with $d(s_1) = a_{1,1}$ and $d(s_2) = a_{2,2}$. Setting $\lambda = 0.9$ and solving

[38] Note that some programming languages (such as R) represent the geometric distribution as the number of failures up to the first success. If this is the case, it is necessary to add 1 to the simulated value to be consistent with (10.45).

$(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d$ establishes that $v_\lambda^{d^\infty}(s_1) = 30.147$ and $v_\lambda^{d^\infty}(s_2) = 27.941$.

The example compares the estimate of $v_\lambda^{d^\infty}(s)$ based on fixed truncation at $M = 30, 50, 70$ and random geometric truncation where the estimates are updated between replicates using (10.47) with learning rates $\tau_n = \log(1 + n)/n$, $20/(50 + n)$, n^{-1}, $0.8n^{-0.75}$. As noted earlier, $\tau_n = n^{-1}$ provides a running estimate of the mean. For each combination of method and learning rate, 100 instances of 1,000 replicates each were generated with common random number seeds for each instance. The quality of the estimates was assessed on the basis of the RMSE over the two states for the 100 instances.

Figure 10.14 provides boxplots of the RMSEs by method and learning rate. Observe that:

1. For each method, the estimate based on the mean ($\tau_n = n^{-1}$) had the smallest median RMSE and was the least variable. This is to be expected since the mean over instances is the minimum variance unbiased estimator of the mean.

2. Estimates using the ratio learning rate were the least accurate and most variable.

3. The quality of truncation estimates improved with increasing M with the average accuracy of those for $M = 50$ and $M = 70$ similar. The mean RMSE for $M = 50$ and $M = 70$ differed by 0.005 and the standard deviation when $M = 50$ was 0.44 and that for $M = 70$ was 0.48.

4. For all learning rates, the estimates based on geometric stopping times were most variable. This could be partially explained by noting that the mean number of episodes evaluated using geometric stopping times is $(1-\lambda)^{-1} = 10$.

The above observations indicate that using the Monte Carlo estimate with $M = 50$ or $M = 70$ gave the most precise estimates. Nothing was to be gained by using the hybrid TD-Monte Carlo approach based on (10.47). The reason for the imprecision of the geometric estimates was the presence of an additional source of variability resulting from simulating the reward sequence length.

10.4.2 Online TD(γ) in an infinite horizon discounted model

Because reward sequences for discounted models are infinitely long, offline TD(γ) faces the same challenges that impacted Monte Carlo estimation. Consequently, TD(γ) is typically implemented *online* using either trajectory-based or state-based data. While both data types are appropriate in simulations, only trajectory-based data is viable for real-time applications.

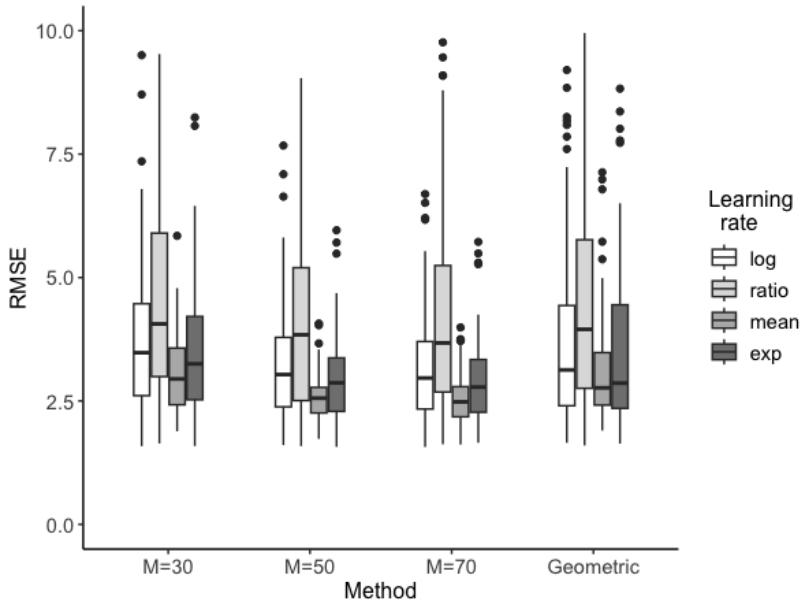


Figure 10.14: Boxplots of RMSEs for Example 10.3 by method and learning rate.

The TD(γ) algorithm below assumes that data is trajectory-based and the accumulating trace is used. A modification for a state-based approach is described below.

Algorithm 10.6. TD(γ) for policy evaluation for a discounted model

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MR}}$.
   (b) Set $v(s) \leftarrow 0$, $e(s) \leftarrow 0$ and $\mathrm{count}(s) \leftarrow 0$ for all $s \in S$.
   (c) Specify $0 \leq \gamma \leq 1$ and learning rate sequence $\tau_n, n = 1, 2, \ldots$.
   (d) Specify the number of iterations N, set $n \leftarrow 1$ and specify $s \in S$.
2. **Iterate:** While $n \leq N$:
   (a) $e(s) \leftarrow e(s) + 1$.
   (b) $\mathrm{count}(s) \leftarrow \mathrm{count}(s) + 1$.
   (c) Sample a from $w_d(\cdot|s)$.
   (d) Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
   (e) Evaluate temporal difference

$$\delta \leftarrow r(s, d(s), s') + \lambda v(s') - v(s). \tag{10.48}$$

(f) For all $s \in S$,

$$v(s) \leftarrow v(s) + \tau_{\text{count}(s)} \delta e(s) \tag{10.49}$$

and

$$e(s) \leftarrow \lambda\gamma e(s). \tag{10.50}$$

(g) $s \leftarrow s'$ and $n \leftarrow n + 1$.

3. **Terminate:** Return $v(s)$ for all $s \in S$.

Comments on online TD(γ) for discounted models follow:

1. In contrast to the undiscounted episodic model, the discount factor λ appears explicitly in (10.48) and (10.50).

2. The above algorithm is stated for an accumulating eligibility trace. Because in (10.50) the eligibility trace is dampened by the factor $\lambda\gamma$, the impact on previous terms is less than in the episodic model. Thus, distinguishing between the accumulating trace and the replacement trace should have less effect on estimates than in an episodic model. To use a replacement trace, replace step 2(a) by $e(s) \leftarrow 1$. The example below investigates the impact of trace type on estimate quality.

3. The Markov chain generated in step 2(d) may visit some states infrequently leading to high variability in policy value function estimates. Instead, when data is simulated, a state-based approach may be used. If that is the case, the first part of step 2(g) would be replaced by "Sample s from a discrete distribution on S." The distribution could be uniform or put more weight on critical states. This approach is sometimes referred to as *random restart*.

4. The algorithm may be sensitive to initial values. Choosing a good initial value such as

$$v_0(s) = (1 - \lambda)^{-1} r_d(s) \tag{10.51}$$

where $r_d(s) = E[r(X_0, d(X_0), X_1)|X_0 = s]$ for each $s \in S$ may enhance convergence. This value of $v_0(s)$ corresponds to starting the system in state s and receiving an identical reward of $r_d(s)$ at each decision epoch over the infinite horizon.

5. Online TD(γ) converges if all states are visited infinitely often and the usual conditions on τ_n apply.

6. Temporal differencing or TD(0) is a special case corresponding to $\gamma = 0$. In this case, the discount rate enters only through the temporal difference (10.48) since (10.50) sets $e(s) = 0$ for all $s \in S$.

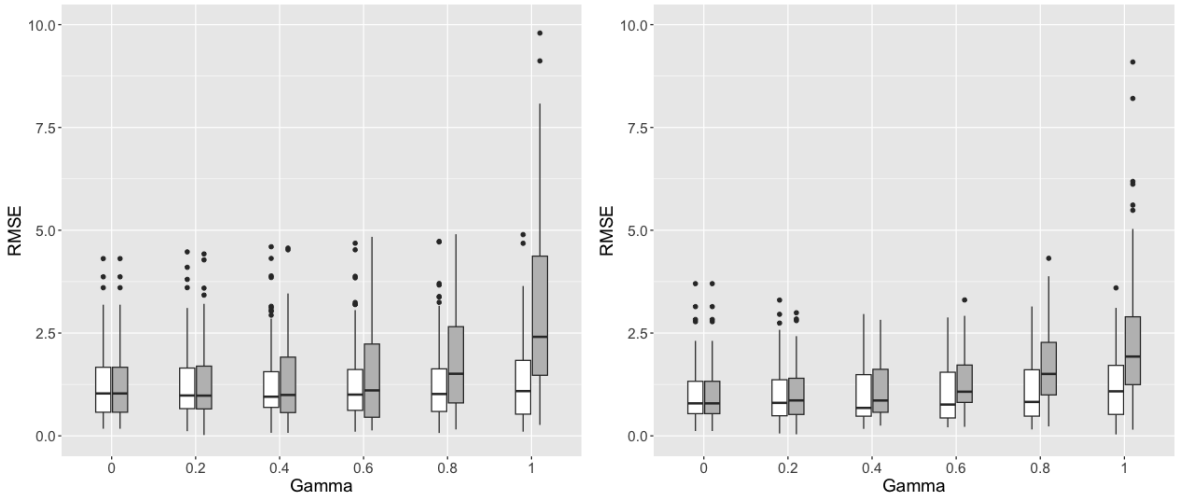
An example

The following example applies online TD(γ) to the two-state model in Section 2.5. It estimates the expected infinite horizon discounted reward, $v_\lambda^{d^\infty}(s)$, for the stationary deterministic policy d^∞ with $d(s_1) = a_{1,2}$ and $d(s_2) = a_{2,2}$. Setting $\lambda = 0.9$ and solving $(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d$ establishes that $v_\lambda^{d^\infty}(s_1) = 30.147$ and $v_\lambda^{d^\infty}(s_2) = 27.941$. Estimation methods were compared using the RMSE of the final observation averaged over both states.

Experiments consisting of 40 replicates of length $N = 5{,}000$ with common random number seeds. They compared the following four factors:

1. Trace: accumulating vs. replacement;
2. Learning rate[39]: $\tau_n = 20/(50 + n)$ vs. $\tau_n = \log(n + 1)/n$;
3. TD(γ): $\gamma = 0, 0.2, 0.4, 0.6, 0.8$, and 1;
4. Simulation method: Trajectory-based vs. state-based (random restart using a uniform distribution).

Output was analyzed graphically and using analysis of variance[40] (ANOVA). Results include:



(a) Trajectory-based (b) State-based

Figure 10.15: Graphical summary of an experiment applying online TD(γ) to a discounted version of the two-state example. It shows variability in RMSE by γ and trace type (accumulating - grey; replacement - white) over 40 replicates of length 5,000.

[39] These values were chosen based on preliminary experimentation.

[40] *Analysis of variance* is a statistical methodology for investigating the individual and simultaneous effect of discrete factors on the mean value of a response variable.

1. The most accurate estimate of $v_\lambda^{d^\infty}(s)$ over all combinations of factors was the combination of logarithmic learning rate, replacement trace, random restart and $\gamma = 0.4$. For this configuration, the average RMSE was 0.96.

2. ANOVA established that the effect of learning rate on RMSE was statistically insignificant but that simulation method, trace type and γ were highly significant.

3. ANOVA also showed that the interaction between γ and trace type was significant, meaning that the effect of trace type differed depending on γ as shown in Figure 10.15. As expected, γ affected results for the accumulating trace and not the replacement trace. This observation was to be expected because in this two-state example, using the replacement trace, especially in the case of trajectory-based data, frequently reset $e(s)$ to 1 so that the effect of γ was minimal. In contrast, when using the accumulating trace, γ impacted the values of $e(s)$ and hence the estimates of $v_\lambda^{d^\infty}(s)$.

4. For trajectory-based data, Figure 10.15a shows that $\gamma = 0$ resulted in the minimal median and least variable RMSE (1.25) over all configurations. Of course, trace type does not impact TD(0) estimates.

5. For state-based data, the results were more complicated. Figure 10.15b shows that the (median) RMSE was increasing in γ for the accumulating trace. For the replacement trace, the median RMSE was minimal at $\gamma = 0.4$ but the differences with respect to the values associated with other γ values were small.

6. Estimates for trajectory-based data were more variable than those for the random restart data. This is to be expected since, when following the Markov chain corresponding to long trajectory state generation, state s_1 is visited twice as frequently as state s_2.

The above results suggest that in this small example, the replacement trace is preferable to the accumulating trace, and that TD(0) provides the best estimates in trajectory-based data. If state-based simulation is used, there is a small benefit to choosing $\gamma > 0$. It is left to the reader to compare this approach to results using Monte Carlo estimation.

Of course, the same conclusions need not hold in general. However, the data generation and analysis of this example suggest an approach to follow in other applications.

10.5 Policy evaluation: Infinite horizon average reward models

This section provides a brief overview of issues arising when applying simulation in the context of infinite horizon average reward models. As a prelude to policy optimization, computing only the average reward (or gain) of a policy does not suffice; an estimate of the bias is also required to apply the Bellman equation.

Most research in simulation-based dynamic programming has focused on infinite horizon episodic and discounted models. This is because the average reward criterion is less sensitive to short-term actions; it depends more on what happens in the distant future. *Therefore, while online learning is fundamental in episodic and discounted models, it is less relevant when using the average reward criterion.* This means that to evaluate policies and find optimal policies for average reward models:

1. Large data sets are required for accurate estimation of the average reward and bias.

2. Offline analyses are practical in average reward settings but online methods exist and still can be used.

This section will describe Monte Carlo and TD(0) approaches for estimating the gain and bias of a stationary policy. Generalization to TD(γ) is left to the reader.

Technical preliminaries

The focus of this section will be on policies with constant gain, therefore, it implicitly assumes a recurrent or unichain model[41]. Recall from Chapter 7 that for a stationary policy d^∞, its average reward (or gain) is defined by

$$g^{d^\infty}(s) = \lim_{N\to\infty} \frac{1}{N} E^{d^\infty}\left[\sum_{n=1}^{N} r(X_n, Y_n, X_{n+1}) \middle| X_1 = s\right] \tag{10.52}$$

and its bias by

$$h^{d^\infty}(s) = E^{d^\infty}\left[\sum_{n=1}^{\infty} \left(r(X_n, Y_n, X_{n+1}) - g^{d^\infty}(X_n)\right) \middle| X_1 = s\right]. \tag{10.53}$$

Moreover, Chapter 7 establishes that in unichain models, the gain and bias of a stationary policy d^∞ satisfy the system of equations

$$h(s) = \sum_{j\in S} p(j|s, d(s)) \left(r(s, d(s), j) + h(j)\right) - g, \quad \forall s \in S. \tag{10.54}$$

[41] Or a regular or unichain policy in a multi-chain model.

Since there are $|S|$ equations in $|S|+1$ unknowns, an extra condition is required to guarantee a unique solution. Solving (10.54) subject to $h(s_0)=0$ for some s_0 provides the *relative values*, denoted $h^{d^\infty}_{\text{rel}}(s)$, which suffice for optimization using the Bellman equation.

When setting $h(s_0)=0$, the equation corresponding to s_0 becomes

$$g=\sum_{j\in S}p(j|s_0,d(s_0))\big(r(s_0,d(s_0),j)+h(j)\big). \tag{10.55}$$

This equation will prove useful when motivating TD(0) below.

10.5.1 Monte Carlo methods

Monte Carlo methods are based on the representations for the gain and bias in (10.52) and (10.53). For the same reasons as in the discounted case, only starting-state Monte Carlo provides estimates with the same degree of accuracy for all states.

Algorithm 10.7. Monte Carlo policy evaluation for an average reward model

1. **Initialize:**
   (a) Specify $d\in D^{\text{MR}}$.
   (b) Specify the number of replicates K_g and K_h, and set $k_g\leftarrow 1$ and $k_h\leftarrow 1$.
   (c) Specify the truncation levels M_g and M_h, and set $m_g\leftarrow 1$ and $m_h\leftarrow 1$.
   (d) Specify $\bar{s}\in S$.
   (e) Create empty lists $\textsc{Values}_g$ and $\textsc{Values}_h$.
2. **Estimate the gain:** While $k_g\leq K_g$:
   (a) $m_g\leftarrow 1$ and $u\leftarrow 0$.
   (b) Sample $s\in S$ from a random distribution.
   (c) **Generate a replicate:** While $m_g\leq M_g$:
      i. Sample a from $w_d(\cdot|s)$.
      ii. Simulate (s',r) or sample s' from $p(\cdot|s,a)$ and set $r\leftarrow r(s,a,s')$.
      iii. $u\leftarrow u+r$.
      iv. $s\leftarrow s'$.
      v. $m_g\leftarrow m_g+1$.
   (d) **Update:**

i. Append u/M_g to $\textsc{Values}_g$.
ii. $k_g \leftarrow k_g + 1$.

3. Set $g = \text{mean}(\textsc{Values}_g)$.
4. **Estimate the bias**: While $k_h \leq K_h$:
   (a) $m_h \leftarrow 1$ and $u \leftarrow 0$.
   (b) $s \leftarrow \bar{s}$.
   (c) **Generate a replicate:** While $m_h \leq M_h$:
      i. Sample a from $w_d(\cdot|s)$.
      ii. Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
      iii. $u \leftarrow u + (r - g)$.
      iv. $s \leftarrow s'$.
      v. $m_h \leftarrow m_h + 1$.
   (d) **Update:**
      i. Append u to $\textsc{Values}_h$.
      ii. $k_h \leftarrow k_h + 1$.
5. Set $h(\bar{s}) = \text{mean}(\textsc{Values}_h)$.
6. **Terminate:** Return g and $h(\bar{s})$.

Some comments follow:

1. This rather lengthy algorithmic statement provides estimates of g and $h(\bar{s})$ using truncation with possibly different truncation levels and number of replicates for each quantity estimated.
2. Replicates to estimate g can start from the same state or be randomly chosen.
3. It returns estimates of the bias for a single state. To obtain estimates of the bias for all states requires repeating step 4 for all $\bar{s} \in S$.
4. Note that an estimate of the gain is required before estimating the bias.
5. Of course, Monte Carlo is an offline algorithm. It applies to both model-free and model-based settings.

Example 10.4. Average reward Monte Carlo policy evaluation
This example evaluates the gain and bias in the two-state model for the station-

ary policy d^∞ with $d(s_1) = a_{1,2}$ and $d(s_2) = a_{2,2}$. Solving the evaluation equations (10.54) with $h(s_1) = 0$ gives $g^{d^\infty} = 2.857$ and $h^{d^\infty}_{\text{rel}}(s_2) = -2.143$. Alternatively, solving (10.54) subject to $\mathbf{P}^*_d\mathbf{h} = \mathbf{0}$ or equivalently using the matrix representation in Theorem 7.3, establishes that $h^{d^\infty}(s_1) = 1.531$ and $h^{d^\infty}(s_2) = -0.612$.

The gain, bias and relative values were estimated using Algorithm 10.7. Preliminary calculations showed that estimates, especially of the relative value, were extremely variable. Consequently a large number of replicates was required. The following values were used to obtain the estimates in Figure 10.4: $K_g = 850$, $M_g = 850$, $K_h = 10{,}000$ and $M_h = 500$. Results described below were based on 50 instances using the above configuration with different random number seeds for each instance.

Estimates of the gain had mean 2.850 and standard deviation of 0.038 over all instances. Estimates of the bias $h^{d^\infty}(s)$ were extremely variable and unreliable, however estimates of relative values based on $h^{d^\infty}(s_2) - h^{d^\infty}(s_1)$ were more accurate with mean -2.130 and standard deviation 0.204 over all instances. Figure 10.4 displays the variability of the estimates of the gain and relative value across instances.

Because the estimate of g^{d^∞} in step 3 of the algorithm impacts the estimate of the bias in step 4(c)iii, one might hypothesize that the estimates of the gain and relative values were correlated. Surprisingly, this was not the case.

One can conclude that Monte Carlo methods require very large samples to estimate relative values. So, they may not be appropriate for algorithmic optimization.

10.5.2 Temporal differencing (TD(0))

This section provides a temporal differencing method to evaluate the average reward or gain, g^{d^∞}, and relative values $h^{d^\infty}_{\text{rel}}(s)$ of a stationary policy d^∞. For simplicity, they are expressed as g and $h(s)$.

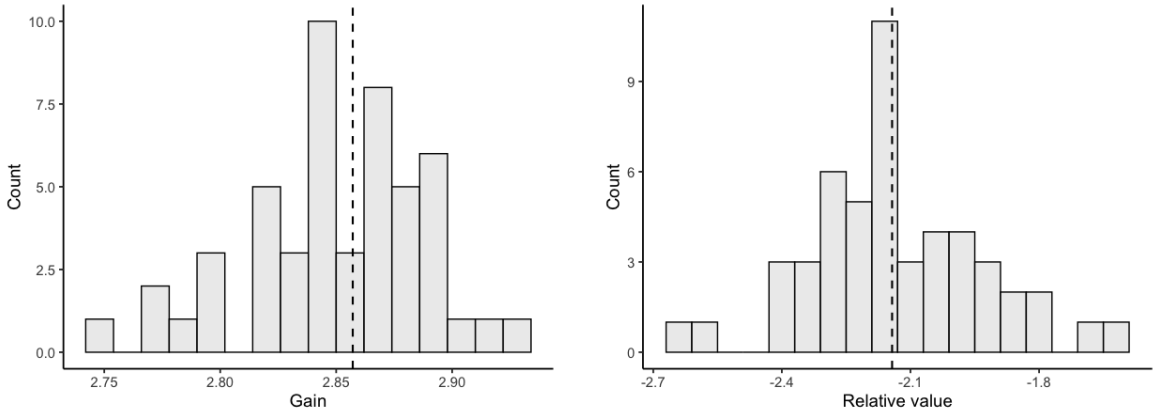
To motivate temporal differencing, express (10.54) in expectation form as

$$h(s) = E^{d^\infty}\left[r(X, Y, X') + h(X') - g \,\middle|\, X = s\right] \tag{10.56}$$

and (10.55) as

$$g = E^{d^\infty}\left[r(X, Y, X') + h(X') \,\middle|\, X = s_0\right]. \tag{10.57}$$

Applying stochastic approximation to these two expressions and replacing g and $h(s)$ by the most recent estimates leads to the following algorithm. Note that it applies to trajectory-based and state-based data.



(a) Estimates of g^{d^∞}.

(b) Estimates of $h_{\text{rel}}^{d^\infty}(s_2)$.

Figure 10.16: Histograms showing variability of Monte Carlo estimates of gain and relative value over 50 instances in Example 10.4. Vertical dashed lines give the true value of the quantities being estimated. Note that the axis scale differs for the two estimates.

Algorithm 10.8. Temporal differencing for policy evaluation for an average reward model

1. **Initialize:**
   (a) Specify $d \in D^{\text{MR}}$ and a distinguished state s_0.
   (b) Specify sequences τ_n and β_n, $n = 1, 2, \ldots$.
   (c) Set $g \leftarrow 0$, $h(s) \leftarrow 0$ and $\text{count}(s) \leftarrow 0$ for all $s \in S$.
   (d) Specify $s \in S$.
   (e) Specify the number of iterations N and set $n \leftarrow 1$.
2. **Iterate:** While $n \leq N$:
   (a) Sample a from $w_d(\cdot|s)$.
   (b) Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
   (c) $\text{count}(s) \leftarrow \text{count}(s) + 1$.
   (d) If $s \neq s_0$,

$$h(s) \leftarrow h(s) + \tau_{\text{count}(s)} \left(r(s, d(s), s') - g + h(s') - h(s) \right) \tag{10.58}$$

   and if $s = s_0$,

$$g \leftarrow g + \beta_{\text{count}(s_0)} \left(r(s_0, d(s_0), s') + h(s') - g \right). \tag{10.59}$$

(e) $n \leftarrow n + 1$.

(f) $s \leftarrow s'$ (trajectory-based) or sample s from a specified distribution on S (state-based).

3. **Terminate:** Return g, and $h(s)$ for all $s \in S$.

Some comments follow:

1. The above algorithm uses the relative evaluation equation with $h(s_0) = 0$ in step 2(d). It is based on (10.57). Consequently, the algorithm generates relative values.

2. A variant of the algorithm[42] replaces equation (10.59) with

$$g \leftarrow g + \beta_n(r(s, d(s), s') - g) \tag{10.60}$$

at **every** iteration. Thus, the index for β_n is the iteration number. Note that $\beta_n = n^{-1}$ is equivalent to recursive estimation of the mean of g based on the observed rewards. This is equivalent to online Monte Carlo estimation of the average reward.

3. The algorithm allows for different learning rates in (10.58) and (10.59).

4. Note that in both variants of the algorithm, updates are asynchronous; when using the algorithm with (10.59), g is only updated whenever s_0 is visited and $h(s)$ is only updated when visiting state $s \neq s_0$. When using the algorithm with (10.60), g is updated at every iteration.

5. A TD(γ) variant of this algorithm using update equation (10.60) converges[43] with probability one under the assumption that the Markov chain corresponding to d^∞ is irreducible and aperiodic, and the learning rates satisfy the Robbins-Monro conditions and are linearly related.

The following example investigates the two variants of the algorithm with different learning rates using the frequently analyzed two-state model.

Example 10.5. Temporal differencing in an average reward model. This example applies the above algorithm and its variant to analyze the policy d^∞ where $d(s_1) = a_{1,2}$ and $d(s_2) = a_{2,2}$. The gain and relative values are given in Example 10.4.

It compares the quality of estimates based on Algorithm 10.8 and its variant

[42] This variant has been proposed by Tsitsiklis and Roy (1999).

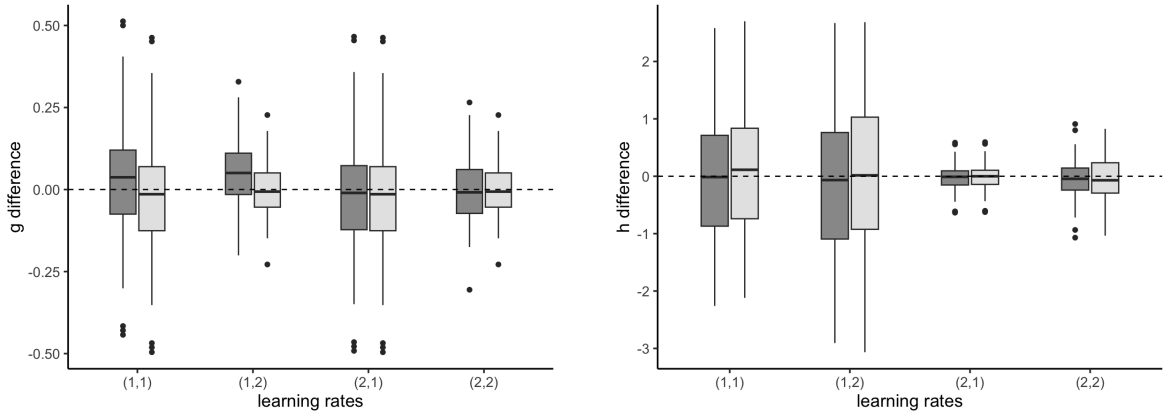
[43] Tsitsiklis and Roy (1999) established this result as well for models with function approximation.

that replaces (10.59) with (10.60) using 50 replicates of $N = 25{,}000$ iterates. After some preliminary calculations, two values for each learning rate were chosen for further investigation: $\tau_n^1 = n^{-1}\log(n+1)$, $\tau_n^2 = 150/(300+n)$ and $\beta_n^1 = n^{-1}\log(n+1)$, $\beta_n^2 = n^{-1}$. Thus, for each random number seed, eight instances were compared: each instance corresponded to a different pair of learning rate and update equation for g. Only the trajectory-based version of the algorithm was investigated.

Estimates are compared on the basis of the difference between the final estimates g and $h(s_2)$ and the true values g^{d^∞} and $h_{\text{rel}}^{d^\infty}(s_2)$. Results are displayed as boxplots in Figure 10.17. The figure shows that when estimating the gain, estimates based on (10.60) were more accurate for all learning rate choices. Clearly, using $\beta_n^2 = n^{-1}$ provided the least variable estimates of the gain.

However, the results for estimating the relative value differed. Using the learning rate τ_n^2 provided less variable estimates than using τ_n^1. Moreover estimates based on the two specifications for the gain differed slightly when using τ_n^2. Note also that the combination of learning rates (τ_n^2, β_n^1) produced the most accurate estimates of the bias.

Overall the estimates of the gain were more accurate than those of the relative value. This is important because the relative value update (10.58) uses the estimate of the gain explicitly. However, accurate estimates of the relative value are required when seeking optimal policies so that precision of the relative values should guide algorithmic and learning rate choice.



(a) $g - g^{d^\infty}$ (b) $h(s_2) - h_{\text{rel}}^{d^\infty}(s_2)$

Figure 10.17: Boxplots comparing accuracy of estimates of the gain and relative values for the eight instances in Example 10.5. Learning rates τ_n^i and β_n^j are represented as (i, j). Dark fill corresponds to g update equation (10.59) and light fill to (10.60). Note that the vertical scale differs in the two plots.

10.6 Policy optimization

This section describes simulation-based methods for finding effective policies. The methods described here either optimize step-by-step, in a similar fashion to value iteration, or combine one of Monte Carlo or temporal differencing with some form of policy improvement. Most are based on state-action value functions, as opposed to value functions. One such approach, Q-learning, represents one of the most significant advances in the development of reinforcement learning methods.

10.6.1 Two perspectives

As noted earlier, simulation-based optimization can be viewed from two perspectives:

1. That of a decision maker or agent interacting with its environment in order to *learn* a good policy through experimentation. Often, the agent may also seek "earning while learning", that is to maximize accumulated rewards during the learning phase.

2. That of an external controller (or analyst) using simulation or real-world experimentation as a tool to learn good policies to be used in subsequent applications.

Of course these are not mutually exclusive; policies identified during learning may be subsequently implemented in the future[44].

10.6.2 Model selection

To assess the quality of estimates of the value of a fixed policy, standard *goodness-of fit* metrics (Section 10.11.1) can be used. When a model is available, these estimates can be compared to the exact policy value function obtained by solving appropriate evaluation equations. Alternatively, comparisons can be based on results of real-time implementations or simulation output.

Since the optimization methods in this chapter are based on state-action value functions, similar RMSE and graphical approaches can be used to assess how well a method estimates the optimal state-action value function. However these should *not* be the primary measure of the quality of a method. Such approaches are limited by two considerations:

[44] In the reinforcement learning literature this is referred to as *planning*.

1. The objective of optimization is to identify high-quality policies. State-action value functions are a construct to do so.

2. The better the method, the more quickly it will identify effective greedy policies. As a result, many state-action pairs will be visited infrequently resulting in inaccurate estimates of state-action value functions for some state-action pairs.

The following *policy-based* considerations should guide method selection:

1. **Policy agreement:** In experiments with known optimal policies, the agreement between these and ones identified by a method can be assessed graphically or through measures of policy closeness.

2. **Long-run policy quality:** The long-run performance of the policies can be compared on the basis of the specified optimality criterion. In a model-based environment this can be based on comparing the exact value of the identified policy computed by solving policy evaluation equations to the optimal value function obtained through algorithms such as policy iteration. In a model-free environment, these can be assessed against simulations using good heuristics.

3. **Policy variability:** Methods that produce similar policies across multiple replications are preferable.

4. **Accumulated reward:** In learning environments, the agent may seek to maximize the accumulated reward during the learning phase of an experiment. Methods that achieve higher cumulative rewards during training are preferable.

5. **Data efficiency:** Methods that require less data to obtain high quality policies are preferable.

6. **Robustness:** Methods that identify high quality policies across different applications and parameter settings are preferable.

Definitive statements require appropriate metrics and carefully designed experiments spanning diverse environments[45].

10.6.3 Preliminaries

Given an estimate of the value function $v(\cdot)$, iterative optimization algorithms in Chapters 5-7 are based on evaluating expressions of the form

$$\max_{a \in A_s} \left(\sum_{j \in S} p(j|s,a)(r(s,a,j) + \lambda v(j)) \right),$$

[45] Patterson et al. 2024 provides an in-depth discussion of issues to consider when evaluating a reinforcement learning method.

where $0 < \lambda \leq 1$ and choosing actions *greedily*, that is, those in

$$\arg\max_{a \in A_s} \left(\sum_{j \in S} p(j|s,a)(r(s,a,j) + \lambda v(j)) \right).$$

Actions chosen in this way are referred to as *greedy*[46] actions. Instead of focusing on value functions, the methods in this section focus on state-action value functions

$$q(s,a) := \sum_{j \in S} p(j|s,a)(r(s,a,j) + \lambda v(j))$$

with $\lambda = 1$ in episodic and average reward[47] models and $0 \leq \lambda < 1$ in discounted models. In particular, they seek to find the optimal state-action value function

$$q^*(s,a) := \sum_{j \in S} p(j|s,a)(r(s,a,j) + \lambda v^*(j)), \tag{10.61}$$

where $v^*(\cdot)$ is the optimal value function for the corresponding MDP. If this quantity were available, optimal policies can be found by selecting actions greedily using $q^*(s,a)$.

The elegance of these methods is that they regard $q(s,a)$ as the primitive. Implementation does not require knowledge of the underlying model. All that is needed are:

1. methods to generate actions in states,
2. methods to generate rewards and transitions to new states, and
3. methods to recursively update state-action value functions.

Recall that $q^*(s,a)$ solves the Bellman equation

$$q(s,a) = \sum_{j \in S} p(j|s,a)\big(r(s,a,j) + \lambda \max_{a' \in A_j} q(j,a')\big) \tag{10.62}$$

with $\lambda = 1$ in episodic models and $0 \leq \lambda < 1$ in discounted models. The solution is unique in transient and stochastic shortest path episodic models and discounted models (Theorems 6.6 and 5.7).

[46] They are also referred to as **v**-greedy actions below.

[47] In average reward models, the bias or relative value functions replace $v(s)$.

10.6.4 Selecting actions: Exploration vs. exploitation

To find optimal policies using simulation or real-time experimentation, **all** state-action pairs must be evaluated sufficiently often to obtain accurate estimates of state-action value functions $q(s,a)$. Doing this is referred to as *exploration* to distinguish it from *exploitation*, which only implements greedy actions.

When seeking a good policy, it is desirable to:

1. begin by exploring broadly and frequently,
2. then narrow exploration by searching among actions with large $q(s,a)$ values, and
3. finally, implement greedy actions frequently.

Two approaches to achieve these objectives are ϵ-greedy sampling and softmax sampling. Both use randomized decision rules to sample actions.

ϵ-greedy sampling

This approach selects[48] an action a' in state $s \in S$ according to

$$a' = \begin{cases} a^* \in \arg\max_{a \in A_s} q(s,a) & \text{with probability } 1-\epsilon \\ a \in A_s \setminus \{a^*\} & \text{with probability } \epsilon/(|A_s|-1). \end{cases} \tag{10.63}$$

Alternatively,

$$a' = \begin{cases} a^* \in \arg\max_{a \in A_s} q(s,a) & \text{with probability } 1-\epsilon+\epsilon/|A_s| \\ a \in A_s \setminus \{a^*\} & \text{with probability } \epsilon/|A_s|. \end{cases}$$

When A_s contains only a few elements, such as in the examples below, the former specification enables more exploration. To see this, suppose A_s contains three elements and $\epsilon = 0.3$. The first specification chooses the greedy action with probability 0.7 and each other action with probability 0.15, while the second specification chooses the greedy action with probability 0.8 and each other action with probability 0.1. In models with large action sets, such a distinction is irrelevant.

From another perspective, ϵ-greedy action selection corresponds to implementing the *randomized* decision rule $d(s)$ defined by

$$w_d(a|s) := \begin{cases} 1-\epsilon & a \in \arg\max_{a' \in A_s} q(s,a') \\ \epsilon/(|A_s|-1) & a \neq \arg\max_{a' \in A_s} q(s,a'). \end{cases} \tag{10.64}$$

[48] This specification needs some modification if the arg max is not unique. For example if there are two elements attaining the arg max select each with probability $(1-\epsilon)/2$ when using (10.63).

Softmax (Boltzmann) sampling

Another approach to exploration is to choose action a in state s with probability determined by the *softmax* function:

$$\frac{e^{\eta q(s,a)}}{\sum_{a' \in A_s} e^{\eta q(s,a')}}. \tag{10.65}$$

The quantity η is a parameter that affects action choice probabilities and also helps avoid numerical instability[49]. Sometimes $1/\eta$ is referred to as the *temperature parameter*. When η is small, the differences in $q(s,a)$ have less effect on action choice probabilities than when η is large. For small η, softmax sampling selects actions with nearly equal probabilities. Conversely, when η is large, the softmax function makes it more likely to select actions with large $q(s,a)$ values.

As above, softmax sampling may be viewed as implementing a *randomized* decision rule $d(s)$ defined by

$$w_d(a|s) := \frac{e^{\eta q(s,a)}}{\sum_{a' \in A_s} e^{\eta q(s,a')}}. \tag{10.66}$$

Note that viewing softmax sampling from the perspective of a randomized decision rule is fundamental to the policy gradient and actor-critic methods in Chapter 11.

Discussion

Both methods depend on parameters that trade-off exploration and exploitation. Clearly these parameters should be varied through the learning (or computational) process to impose early exploration and late exploitation. This means that ϵ should decrease and η increase with respect to either the iteration number or the count of the number of visits to a state. Therefore algorithms are usually specified in terms of sequences of ϵ_n or η_n, $n = 1, 2, \ldots$.

The main difference between these methods is that for each $s \in S$, softmax sampling assigns probabilities to actions based on the relative values of $q(s,a)$, ensuring that all actions have a non-zero probability of being selected but favoring those with the greatest values of $q(s,a)$. In contrast, ϵ-greedy sampling selects greedy actions with probability $1-\epsilon$ and does not distinguish between non-greedy actions; all have a small and equal probability of being selected regardless of their $q(s,a)$ values.

The use of either of these methods with appropriate parameter choices ensures that all state-action pairs are visited infinitely often, which is usually sufficient for theoretical convergence.

[49] We have frequently encountered divergence of parameter estimates in our calculations.

10.7 Q-learning and SARSA

How would one go about using state-action value functions to learn good policies? An obvious iterative approach would be to start with a guess for $q(s,a)$, select a state s, use softmax or ϵ-greedy sampling to select an action a, implement it and observe a reward r and a transition to a subsequent state s'.

Q-learning and SARSA follow this general approach, but differ in how they use this information to update $q(s,a)$. Each may use exploration to generate an action a' in state s', but Q-learning updates $q(s,a)$ according to

$$q(s,a) \leftarrow q(s,a) + \tau \left(r + \lambda \max_{a' \in A_{s'}} q(s',a') - q(s,a) \right) \tag{10.67}$$

while SARSA updates $q(s,a)$ according to

$$q(s,a) \leftarrow q(s,a) + \tau \big(r + \lambda q(s',a') - q(s,a) \big), \tag{10.68}$$

where a' denotes an action selected by exploration in the subsequent state. The acronym SARSA stands for "state-action-reward-state-action", corresponding to the sequence of objects (s,a,r,s',a') involved in the update in (10.68) [50].

Another way of thinking of this is through the concept of a *learning* or *behavioral* policy.

Definition 10.2. A *learning policy* or *behavioral policy* is the policy used to generate rewards in a simulation or real-world implementation.

Suppose both Q-learning and SARSA use the same behavioral policy to generate action a'. SARSA corresponds to an on-policy update of $q(s,a)$ in agreement with the behavioral policy, while the off-policy update using Q-learning may differ[51] as shown in Figure 10.18.

More formally, Q-learning updates estimates of state-action value functions using a temporal difference relationship of the form

$$q(s,a) \leftarrow q(s,a) + \tau \left(r(s,a,s') + \lambda \max_{a' \in A_s} q(s',a') - q(s,a) \right) \tag{10.69}$$

where $0 \leq \tau \leq 1$. This recursion is based on using a variant[52] of gradient descent to

[50] Note that when the reward depends on the subsequent state, the unappealing acronym SASRA would be more appropriate.

[51] They will agree when $a' \in \arg\max_{a \in A_{s'}} q(s',a)$.

[52] This variant, based on a construct referred to as a *semi-gradient*, is discussed in Section 11.2.2 in the next chapter.

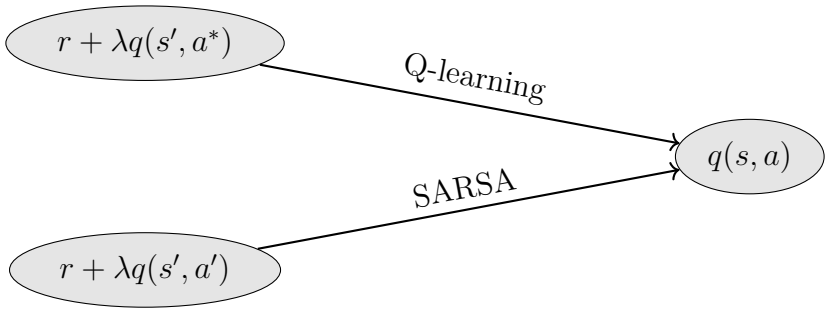


Figure 10.18: Comparison of quantity used to update $q(s, a)$ using SARSA and Q-learning. These updates differ when the action chosen by the greedy policy a^* differs from the action chosen by the learning policy a'.

minimize the expected squared error between the two sides of the expression

$$q(s,a) = E\left[r(X,Y,X') + \lambda \max_{a' \in A_{X'}} q(X',a') \,\middle|\, X = s, Y = a\right], \tag{10.70}$$

which is the Bellman equation (10.62) written in expectation form. Note that the expectation is with respect to the transition probability $p(j|s,a)$ that specifies the distribution of X'. Note that in episodic and average reward models, $\lambda = 1$. In the latter case, (10.69) would also contain a $-g$ term to correspond to the appropriate recursion for the gain and bias.

SARSA

SARSA updates estimates of state-action value functions according to:

$$q(s,a) \leftarrow q(s,a) + \tau\big(r(s,a,s') + \lambda q(s',a') - q(s,a)\big), \tag{10.71}$$

where a' in state s' is chosen based on the randomized policy that was used to choose a in state s. The justification for SARSA is more subtle than Q-learning because after generating s' from $p(\cdot|s,a)$, it requires a distribution for choosing a' at the subsequent decision epoch.

Given a randomized decision rule d, SARSA is based on replacing (10.70) by

$$q(s,a) = E^d\left[r(X,Y,X') + \lambda q(X',Y') \,\middle|\, X = s, Y = a\right], \tag{10.72}$$

where X' has distribution $p(j|s,a)$ and Y' has conditional distribution $w_d(\cdot|X')$ corresponding to the exploration method.

Thus, in essence, SARSA is applying a variant of gradient descent to estimate the state-action value function corresponding to the behavioral policy d^∞. This would

be appropriate if the objective was to estimate $q^{d^\infty}(s,a)$, but the objective is to find the **optimal** state-action value function. Therefore, the decision d must evolve over time so that it approaches the decision rule corresponding to an optimal policy. This is typically achieved using ϵ-greedy or softmax action selection with parameters that ensure that eventually only greedy actions are selected. This requirement is described in more detail following the description of SARSA in algorithmic form.

10.7.1 Episodic models

Q-learning and SARSA algorithms for episodic models are presented, discussed and compared below. Recall that in an episodic model, an episode terminates when a state in Δ is reached.

Q-learning

The following online algorithm can be implemented in a model-free or model-based environment.

Algorithm 10.9. Q-learning for an episodic model

1. **Initialize:**
   (a) Set $q(s,a) \leftarrow 0$ and $\text{count}(s,a) \leftarrow 0$ for each $a \in A_s$ and $s \in S$.
   (b) Specify the learning rate $\tau_n, n = 1, 2, \ldots$.
   (c) Specify $\bar{s} \in S$ and $\bar{a} \in A_{\bar{s}}$.
   (d) Specify the number of episodes K.
   (e) Specify a sequence $\epsilon_n, n = 1, 2, \ldots$ (or η_n for softmax sampling).
   (f) $k \leftarrow 1$.
2. **Iterate:** While $k \leq K$:
   (a) $s \leftarrow \bar{s}$ and $a \leftarrow \bar{a}$.
   (b) **Generate episode:** While $s \notin \Delta$:
      i. $\text{count}(s,a) \leftarrow \text{count}(s,a) + 1$.
      ii. Simulate (s', r) or sample s' from $p(\cdot|s,a)$ and set $r \leftarrow r(s,a,s')$.
      iii. **Update** $q(s,a)$**:**

$$q(s,a) \leftarrow q(s,a) + \tau_{\text{count}(s,a)} \left(r + \max_{a' \in A_{s'}} q(s',a') - q(s,a) \right). \tag{10.73}$$

iv. Choose $a \in A_{s'}$ using ϵ-greedy (or softmax) sampling.

v. $s \leftarrow s'$.

(c) $k \leftarrow k + 1$.

3. **Terminate:** Return $q(s, a)$ for all $a \in A_s$ and $s \in S$.

Some comments follow:

1. Note that in (10.73), r depends only on the *current* action a while $\max_{a' \in A_{s'}} q(s', a')$ accounts for action choice at the *next* decision epoch. The consequence of this is that updates of $q(s, a)$ use the value associated with a greedy action although the action generated by the learning policy in step 2(b)iv to generate rewards and transition probabilities at the next decision epoch need not be the greedy action. Because the learning policy may differ from the policy used to compute the temporal difference in (10.73), Q-learning is said to be an *off-policy algorithm*.
2. The index for ϵ or η is best set to $\sum_{a \in A_s} \text{count}(s, a)$, corresponding to the number of times the state was previously visited.
3. The nature of the application determines whether it is most prudent to start each episode in the same state (as the algorithm is currently written) or in a random state.
4. Using a similar approach to TD(γ), past state-action values can be updated at each iteration.
5. Under the assumption that each state-action pair is visited infinitely often and the learning rate satisfies the Robbins-Monro conditions, the state-action value function estimates converge with probability one to the optimal state-action value function and hence can be used to identify an optimal policy in the limit.

SARSA

SARSA provides the following *on-policy* variant to Q-learning.

Algorithm 10.10. SARSA for an episodic model

1. **Initialize:** Same as in Algorithm 10.9.
2. **Iterate:** While $k \leq K$:
   (a) $s \leftarrow \bar{s}$ and $a \leftarrow \bar{a}$.
   (b) **Generate episode:** While $s \notin \Delta$:
      i. $\text{count}(s, a) \leftarrow \text{count}(s, a) + 1$.

ii. Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.

iii. Choose $a' \in A_{s'}$ using ϵ-greedy (or softmax) sampling.

iv. **Update** $q(s, a)$:

$$q(s, a) \leftarrow q(s, a) + \tau_{\text{count}(s,a)} \left(r + q(s', a') - q(s, a)\right). \quad (10.74)$$

v. $s \leftarrow s'$, $a \leftarrow a'$.

(c) $k \leftarrow k + 1$.

3. **Terminate:** Return $q(s, a)$ for all $a \in A_s$ and $s \in S$.

Note that both SARSA and Q-learning use ϵ-greedy or softmax action selection but SARSA uses the action to *both* update the state-action value function and implement at the subsequent iteration. On the other hand, Q-learning uses ϵ-greedy or softmax action selection *only* to select the next action. Thus, the updates of $q(s, a)$ using SARSA correspond to the "implemented" action or the behavioral policy, while the updates using Q-learning correspond to the greedy action, which may differ from the implemented action (see Figure 10.18).

Without further conditions, *SARSA may converge to a sub-optimal policy.* For example, when the exploration rate does not decrease to zero, the state-action value functions may correspond to a non-optimal randomized policy. In order for SARSA to converge to the optimal state-action value function and identify an optimal policy requires, the learning policy must

1. sample each state-action pair infinitely often, and
2. select greedy actions in the limit with probability one.

Such learning policies are referred to as *greedy in the limit with infinite exploration*[53] . Thus, if the learning policy is greedy in the limit with infinite exploration, SARSA will converge to the optimal state-action value function under the Robbins-Monro step-size conditions.

The on-policy versus off-policy dichotomy

Why should one care about this distinction? Analysis of a **deterministic** version of the Gridworld example in Figure 10.6 provides some insight.

Example 10.6. Distinguishing on-policy and off-policy updates
Suppose in a deterministic model the coffee-delivering robot occupies cell 11

[53] Singh et al. [2000] introduces this concept and provides technical conditions that ensure convergence of SARSA.

(see Figure 10.6), plans to move upward (denoted action 1) to cell 8 and the learning policy based on ϵ-greedy or softmax sampling in cell 8 selects the action that moves it to the left (denoted action 2). As a result, the robot falls down the stairs and incurs a penalty of 200. If this happens, the SARSA and Q-learning updates of $q(11, 1)$ differ.

The SARSA update is

$$q(11, 1) \leftarrow q(11, 1) + \tau\big(-1 + q(8, 2) - q(11, 1)\big)$$

while the Q-learning update is

$$q(11, 1) \leftarrow q(11, 1) + \tau\left(-1 + \max_{a' \in A_8} q(8, a') - q(11, 1)\right).$$

Choosing $q(s, a)$ equal to the optimal value (in the deterministic problem) sets $q(11, 1) = 46$, $q(8, 2) = -201$ and $q(8, 1) = 47$. Thus, the SARSA update becomes

$$q(11, 1) \leftarrow 46 + \tau(-1 - 201 - 46) = 46 - 248\tau$$

while the Q-learning update becomes

$$q(11, 1) \leftarrow 46 + \tau(-1 + 47 - 46) = 46.$$

Thus in future episodes, under SARSA, it would be less likely for the robot to choose the action "move up' in cell 11 and instead move to the right to cell 12, while under Q-learning the robot would move to cell 8 risking the likelihood of falling down the stairs.

This example shows that because of the presence of action sampling, the SARSA guided robot will take more conservative actions than the Q-learning guided robot. However, Q-learning preserves the optimal state-action value function while SARSA does not.

What this example shows is that when using exploration with either ϵ-greedy or softmax sampling:

Use Q-learning if your objective is to find an optimal policy for future use, while SARSA might be preferable when reward acquisition during the learning phase is of concern.

Computational examples

This section compares Q-learning and SARSA using two versions of the Gridworld model.

Example 10.7. Model-free deterministic Gridworld

In this version of the Gridworld model, the robot can select any of the actions "up", "right", and "left" in any cell. Action choice results in a deterministic transition in the intended direction when possible. Otherwise the robot remains in the current cell. For example, if the robot is in cell 13 and tries to move left, it will remain in cell 13. Each time the robot chooses an action it incurs a cost of 1, even if it cannot move.

This example applies Q-learning and SARSA with $q(s, a)$ initialized at 0 and with initial state designated as cell 13. In contrast to earlier Gridworld examples, an episode ends when the robot reaches cell 1. Thus, an episode may consist of instances when the robot falls down the stairs (cell 7) and incurs a penalty. In such instances, the robot returns to cell 13, refills the coffee cup and then attempts to deliver the coffee.

Exploration introduces randomness. The results below apply softmax sampling with $\eta_n = 0.04$ and updates $q(s, a)$ using a learning rate of $\tau_n = 0.8n^{-0.75}$ where n denotes the number of visits to pair (s, a). These parameter choices were based on considerable experimentation. Both Q-learning and SARSA used $K = 500$ episodes and used the same random number seed for each episode.

Using Q-learning, the estimated state-action value function obtained from a typical replicate of 500 episodes appears in Table 10.4. The entries labeled "n/a" correspond to the termination cells 1 and 7, where no action choice is available. Starting from cell 13, the greedy policy's path, represented by the bold entries in the table, is $13 \rightarrow 10 \rightarrow 11 \rightarrow 8 \rightarrow 5 \rightarrow 2 \rightarrow 1$. Note that $q(13, \text{"up"}) = 42.89$, while the optimal deterministic policy total reward starting in 13 and following the shortest path generates a reward of 44. The discrepancy is a result of exploration and the small number of episodes. When K is increased to 5,000, $q(13, \text{"up"}) = 44$ and there also were several optimal policies.

SARSA with softmax sampling and $K = 500$ identified the same optimal policy but the $q(s, a)$ values were considerably smaller than those obtained using Q-learning. Moreover "optimal" action choice differed in cell 15. In cell 15 SARSA chose the more conservative action "up" and Q-learning chose "left".

Figure 10.19 shows the sequence of running-average rewards acquired through the 500 episodes using Q-learning and SARSA. Observe that in this instance the running-average reward during learning using SARSA exceeded that of Q-learning. The mean (standard deviation) of the total reward per episode for Q-learning was 31.36 (14.94) and for SARSA was 34.58 (16.92). This pattern persisted across all experiments.

As a result of the above computational results, the robot can abandon exploration, encode the optimal policy and obtain the maximum reward.

The results in Example 10.7 are quite remarkable. *With no intervention, and no knowledge of the world, the robot identified the optimal policy and accurately estimated*

state	up	right	left
1	n/a	n/a	n/a
2	48.00	46.99	**49.00**
3	47.00	47.00	48.00
4	49.00	46.87	47.96
5	**47.99**	45.81	47.97
6	46.98	45.86	46.89
7	n/a	n/a	n/a
8	**46.94**	44.41	-160.8
9	45.81	42.25	42.86
10	-160.8	**44.52**	43.14
11	**45.78**	42.25	42.86
12	44.22	41.49	44.20
13	**42.89**	42.33	40.52
14	44.17	37.08	39.95
15	39.40	27.90	41.41

Table 10.4: Table of estimates of $q^*(s,a)$ using Algorithm 10.9 in a single experiment consisting of 500 episodes using Q-learning.

the optimal state-action value function.

The following example considers the Gridworld model with noisy transitions.

Example 10.8. Comparison of Q-learning and SARSA in Gridworld

Using the Gridworld model with $p = 0.8$, this example investigates the effect of learning rate and algorithm (Q-learning versus SARSA) on:

1. The extent of agreement between the greedy policy based on the estimated state-action value function and the optimal policy,
2. the difference between the value function of the greedy policy and that of the optimal policy, and
3. the RMSE of the estimated state-action value function.

Since the model is available it can be solved to find the optimal policy and the optimal value function. If it were not, Monte Carlo policy evaluation could be used.

Preliminary calculations investigating exploration method, learning rate and number of episodes suggested the following choices:

1. **Exploration:** ϵ-greedy sampling with

$$\epsilon_n = \frac{200}{500+n}, \tag{10.75}$$

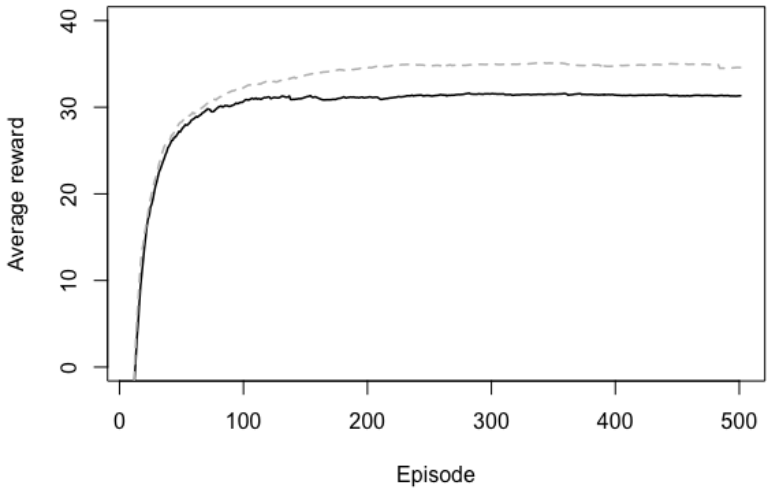


Figure 10.19: Running-average reward using SARSA (dashed line) and Q-learning (solid line) based on a single experiment with 500 episodes.

where n denotes the episode number, produced the most accurate estimates of $q(s, a)$.

2. **Learning rates:**

   Exp1: $\tau_n = 0.4n^{-0.5}$

   Exp2: $\tau_n = 0.8^{-0.75}$

   STC: $\tau_n = 0.1/(1 + 10^{-5}n^2)$

   Log: $\tau_n = n^{-1}\log(n + 1)$

   where the learning rate index n represented the number of times a state-action pair was previously visited.

3. **Replicates:** 40 starting in cell 13 using common random number seeds for each combination of factors in each replicate.

4. **Number of episodes:** 5,000

Table 10.5 and Figure 10.20 summarize the results. Observe that:

1. **Policy optimality:** The greedy policy derived from the final estimate of $q(s, a)$ agreed with the optimal policy in at most 21 of the 40 replications (SARSA with learning rate Exp2). Averaged over the four learning rates,

the policy identified by SARSA was more likely to be optimal than that identified by Q-learning.

2. **Features of non-optimal policies:** When greedy policies were non-optimal, they differed from the optimal policy in at most 2 states (cells 4 and cell 8) for all but one instance[a]. These states were visited infrequently under the optimal policy, which steered the robot towards the right wall before reaching the stairs. This could be remedied by additional sampling of infrequently sampled state-action pairs.

3. **Impact of non-optimal actions:** Table 10.6 shows the effect of incorrect action choice on policy value functions found using the exact methods in Chapter 6. Observe that having one or two non-optimal actions had little effect on the estimate for replications starting in cell 13.

4. **State-action value function estimation:** For all learning rates, Q-learning estimates of $q(s, a)$ had smaller median and mean RMSEs[b] than those obtained with SARSA. The effect of learning rate on the accuracy of the estimate of the optimal state-action value function was not consistent for the two methods.

5. **Value vs. policy** The learning rates that most accurately estimated the optimal state-action value function differed from those that were more likely to identify the optimal policy.

The take away from this example is that, in most cases, greedy policies obtained from estimated state-action value functions identified policies with values that differed little from those of the optimal policy. On the other hand, accuracy of the estimates of state-action value functions varied considerably. Increased sampling of infrequently visited state-action pairs may improve the quality of estimates and perhaps policies.

[a] In this instance, the greedy policy differed from the optimal policy in cells 4, 8 and 14. Cells 4 and 8 were sampled infrequently and in cell 14 there was only a small difference in estimated state-action values.

[b] The RMSE compared the estimated state-action value function to the optimal state-action value function obtained using exact methods.

10.7.2 Infinite horizon discounted models

This section describes and examines the use of Q-learning and SARSA in infinite horizon discounted models assuming trajectory-based data.

Non-optimal actions	Q-learning				SARSA			
	Exp1	Exp2	STC	Log	Exp1	Exp2	STC	Log
0	14	7	14	20	20	21	17	16
1	20	26	20	17	16	15	18	21
2	6	6	6	3	4	4	5	3
3	0	1	0	0	0	0	0	0

Table 10.5: Comparison of update method and learning rate on the basis of the number of non-optimal actions determined by the greedy policy.

Non-optimal action(s)	$v^*(13) - v(13)$	Median difference
4	0.12	0.12
8	0.35	0.30
4,8	0.44	0.41
4,8,14	2.22	0.54

Table 10.6: Effect of non-optimal actions on the deviation of the optimal value function from that of policies with indicated non-optimal actions. The last column indicates the median difference over all states. As a point of reference $v^*(13) = 29.22$ and the median of $v^*(s)$ over all s was 35.29.

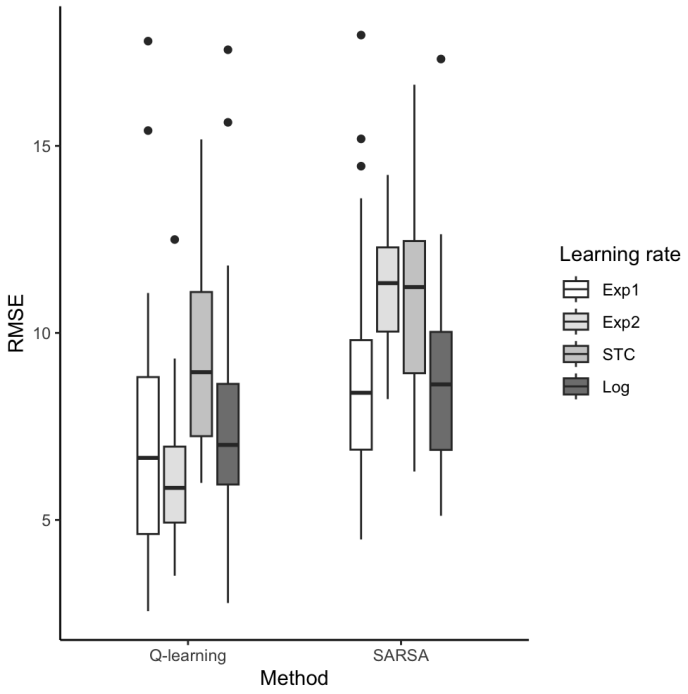


Figure 10.20: Boxplots of RMSE by learning rate and method for Example 10.8.

Algorithm 10.11. Q-learning for a discounted model

1. **Initialize:**
   (a) Set $q(s,a) \leftarrow 0$ and $\text{count}(s,a) \leftarrow 0$ for each $a \in A_s$ and $s \in S$.
   (b) Specify the learning rate $\tau_n, n = 1, 2, \ldots$.
   (c) Specify the number of iterations N.
   (d) Specify a sequence $\epsilon_n, n = 1, 2, \ldots$ (or η_n for softmax sampling).
   (e) Specify $s \in S$ and $a \in A_s$.
   (f) $n \leftarrow 1$.
2. **Iterate:** While $n \leq N$:
   (a) $\text{count}(s,a) \leftarrow \text{count}(s,a) + 1$.
   (b) Simulate (s', r) or sample s' from $p(\cdot|s,a)$ and set $r \leftarrow r(s,a,s')$.
   (c) **Update** $q(s,a)$**:**

$$q(s,a) \leftarrow q(s,a) + \tau_{\text{count}(s,a)} \left(r + \lambda \max_{a' \in A_s} q(s',a') - q(s,a) \right). \quad (10.76)$$

   (d) Choose $a \in A_{s'}$ using ϵ-greedy (or softmax) sampling.
   (e) $s \leftarrow s'$.
   (f) $n \leftarrow n + 1$.
3. **Terminate:** Return $q(s,a)$ for all $a \in A_s$ and $s \in S$.

Some comments follow:

1. The above algorithm is trajectory-based. It can be modified to be state-action pair-based by randomly sampling states and actions prior to generating s' and r in step 2(b) and removing step 2(e). Such a modification can be implemented in a simulator and may apply in some real systems.
2. As noted in the episodic case, SARSA replaces steps 2(c)-2(e) with:
   (c) Sample $a' \in A_{s'}$
   (d) Update $q(s,a)$ by

$$q(s,a) \leftarrow q(s,a) + \tau_{\text{count}(s,a)} \left(r + \lambda q(s',a') - q(s,a) \right). \quad (10.77)$$

   (e) $s \leftarrow s'$, $a \leftarrow a'$.

The difference is that it uses action a' to *both* update $q(s,a)$ and to generate a state and reward at the next iteration.

3. The algorithm returns a state-action value function that can be used to identify a greedy decision rule $d^*(s)$ and policy $(d^*)^\infty$. The value of this policy can be approximated by $q(s,d^*(s))$, evaluated exactly by solving the policy evaluation equations if a model is available, or by simulation or real-time implementation otherwise.

4. If every state-action pair is visited infinitely often and the learning rates satisfy the Robbins-Monro conditions, then the iterates of Q-learning converge with probability one to the optimal state-action value function.

Example of Q-learning and SARSA in a discounted model

The following example illustrates the challenges of selecting parameters for Q-learning and SARSA by investigating the impact of exploration and learning rate on policy choice and state-action value function estimation.

It applies Algorithm 10.11 and its SARSA variant to find estimates of $q^*(s,a)$ in a discounted ($\lambda = 0.9$) version of the two-state model from Section 2.5. Although this example has only four stationary policies, it provides insights that can be applied in more complex settings such as the example in Section 10.9.

Results below are based on setting $N = 30{,}000$ and using trajectory-based data. They compare learning rates $\tau_n = 0.5n^{-0.6}$ (Exp), $\tau_n = 150/(300+n)$ (Ratio), $\tau_n = 0.1(1+n^2/10^5)^{-1}$ (STC) and $\tau_n = \log(n+1)/n$ (Log), and exploration approaches ϵ-greedy with step function decay $\epsilon_n = 0.01 + 0.34I_{[1,\,5{,}000]}(n) + 0.09I_{[0,\,10{,}000]}(n)$ (Step), ϵ-greedy with ratio decay $\epsilon_n = 150/(500+n)$ (Ratio) and with $\eta_n = 0.01n^{0.5}$ (Softmax).

For the learning rate, n equals the number of times a state-action pair was selected. For action choice, n was set to number of times a state was visited. This was done to account for the asynchronous evaluation of state-action pairs. Comparison to setting n equal to the iteration number is left as an exercise.

The simulation study compared all 24 combinations of learning rate, exploration method and recursion (SARSA vs. Q-learning) using 40 replicates with common random number seed for each. Results are summarized in terms of:

1. the fraction of instances in which the greedy policy and the optimal policy were identical, and

2. the RMSE of the estimated matrix of state-action values compared to the exact optimal state-action value function represented by the matrix $\mathbf{Q}^*$.

Solving for $v_\lambda^{d^\infty}(s)$ and applying (10.61) yields

$$\mathbf{Q}^* = \begin{bmatrix} 29.74 & \mathbf{30.15} \\ 20.15 & \mathbf{27.94} \end{bmatrix},$$

where the entries in bold represent the maximum of the row. Thus the optimal policy $(d^*)^\infty$, corresponds to $d^*(s_1) = a_{1,2}$ and $d^*(s_2) = a_{2,2}$, with value $\mathbf{v}^*_\lambda = (30.15, 27.94)$. Recall that the next best policy with $d(s_1) = a_{1,1}$ and $d(s_2) = a_{2,2}$ had value $(27.18, 25.62)$.

		Q-learning			SARSA		
Exploration		Step	Ratio	Softmax	Step	Ratio	Softmax
Learning Rate	Exp	67	74	56	79	18	85
	Ratio	62	74	67	28	51	**97**
	STC	**92**	90	62	0	46	46
	Log	67	90	72	79	38	92

Table 10.7: Percentage of replicates in which the greedy policy agreed with the optimal policy.

Table 10.7 shows the percentage of replicates in which the optimal policy was identified by greedy action choice using the estimated state-action value function. Observe that:

1. There was considerable variability in accuracy.
2. **Main effects:** Averaged over all other factors, Q-learning chose the optimal policy more frequently than SARSA (73% vs. 55%), softmax exploration chose the optimal policy most frequently (72%) among exploration methods, and the Log learning rate chose the optimal policy most frequently (73%) among learning rates.
3. **Q-learning:** STC with step function and ratio exploration chose the optimal policy most frequently. Softmax exploration was the least accurate.
4. **SARSA:** Softmax exploration was most accurate, especially in the case of Ratio and Log learning rates.

Figure 10.21 provides boxplots of the RMSE as it varied across 40 replicates for each combination of learning rate, exploration method and recursion.

They show that:

1. Q-learning estimates were considerably more accurate than SARSA estimates.
2. Q-learning estimates were less sensitive to exploration method than SARSA estimates. This is to be expected since, as noted above, SARSA evaluates the learning policy implied by the exploration method.
3. For Q-learning, the Log learning rate produced the most accurate estimates. In this case, estimates based on exploration with the step function or ratio estimate produced the most accurate and least variable estimates, respectively.

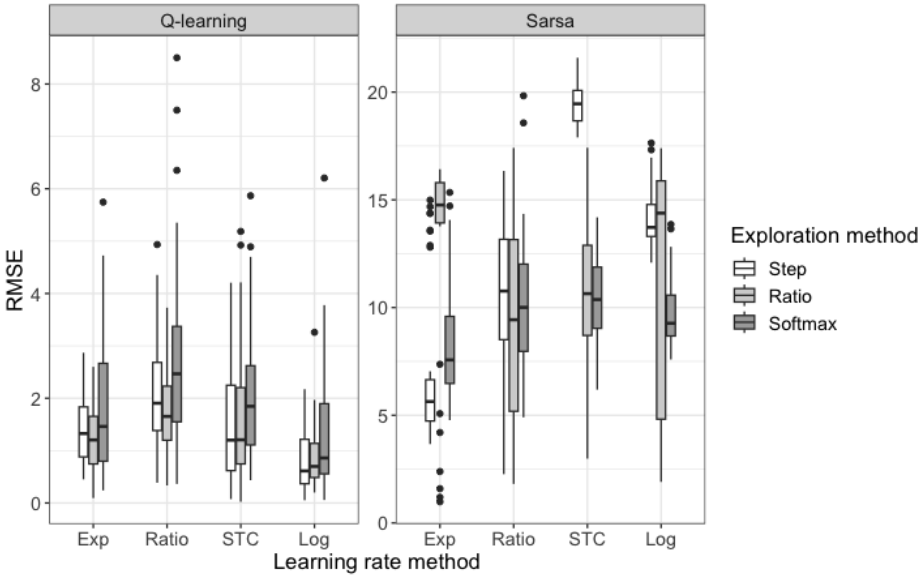


Figure 10.21: Boxplot of RMSE based on 40 simulated replicates. Note that the vertical scales on the left and right plots differ.

4. For SARSA, the exponential decay learning rate with step function exploration produced the most accurate estimates. The estimates using a ratio learning rate were highly variable.

These results show that the most accurate estimates in terms of RMSE were not always in agreement with the methods that chose the best policy. Since SARSA methods were less accurate in estimating the q-function, softmax may be able to account for this by using the relative values of the q-function as its basis for action choice.

The following state-action value matrices, generated using a specific random number seed, offer deeper insight into the effects of parameters and method on estimates. The matrix labels use subscripts to denote the method (Q-learning or SARSA), learning rate, and exploration technique. The bold entry within each row denotes the greedy action in that state.

$$\mathbf{Q}_{Q,STC,Step} = \begin{bmatrix} 30.36 & \mathbf{30.51} \\ 21.00 & \mathbf{28.51} \end{bmatrix} \quad \mathbf{Q}_{Q,Log,Rat} = \begin{bmatrix} 29.92 & \mathbf{30.64} \\ 21.00 & \mathbf{28.42} \end{bmatrix}$$

$$\mathbf{Q}_{Q,Exp,Smax} = \begin{bmatrix} \mathbf{30.93} & 30.79 \\ 20.75 & \mathbf{29.86} \end{bmatrix} \quad \mathbf{Q}_{S,Rat,Smax} = \begin{bmatrix} 15.00 & \mathbf{30.56} \\ 13.58 & \mathbf{29.17} \end{bmatrix}$$

$$\mathbf{Q}_{S,Exp,Step} = \begin{bmatrix} 20.25 & \mathbf{26.90} \\ 16.68 & \mathbf{26.20} \end{bmatrix} \quad \mathbf{Q}_{S,Log,Rat} = \begin{bmatrix} \mathbf{16.15} & 11.90 \\ 7.45 & \mathbf{13.95} \end{bmatrix}.$$

Observe that for Q-learning, the instances $Q, STC, Step$ and Q, Log, Rat accurately estimate the optimal state-action value functions and identify the optimal policy, while the instance $Q, Exp, Smax$ quite accurately estimated $\mathbf{Q}^*$ but the greedy action was

not optimal. In the case of SARSA, no instance provided an accurate estimate of $\mathbf{Q}^*$. The instances for $S, Rat, Smax$ and $S, Exp, Step$ identified the optimal policy while S, Log, Rat did not. Note also that when the greedy policy was not optimal, the discrepancy occurred in state s_1 where the two values in $\mathbf{Q}^*$ are quite close.

The above analysis provides a framework for use in other applications but cautions against generalizing specific recommendations regarding selection of learning rates and exploration methods. As in the episodic example, results are sensitive to learning rates and exploration method.

10.7.3 Infinite horizon average reward models

This section describes and illustrates two Q-learning approaches for finding effective policies for infinite horizon models with the average reward criterion. Reviewing Chapter 7 provides relevant background.

Following (7.63), the optimal state-action value function $q^*(s,a)$ for a recurrent or unichain average reward model is given by:

$$q^*(s,a) = \sum_{j \in S} p(j|s,a)\big(r(s,a,j) - g^* + h^*(j)\big) \tag{10.78}$$

and the optimal state-action value function and the optimal gain is a solution of the Bellman equation:

$$q(s,a) = \sum_{j \in S} p(j|s,a)\left(r(s,a,j) - g + \max_{a \in A_j} q(j,a)\right). \tag{10.79}$$

Written as an expected value, (10.79) is equivalent to:

$$q(s,a) = E^{d^*}\left[r(X,Y,X') - g + \max_{a' \in A_{X'}} q(X',a') \,\middle|\, X = s, Y = a\right].$$

Applying gradient descent[54] to minimize the squared difference between $q(s,a)$ and its realized values suggests the following model-based recursion for estimating $q(s,a)$ using simulation or process data:

$$q(s,a) \leftarrow q(s,a) + \tau\left(r(s,a,s') - g + \max_{a' \in A_{s'}} q(s',a') - q(s,a)\right) \tag{10.80}$$

where s' is sampled from $p(\cdot|s,a)$. The Q-learning algorithm below implements this recursion by combining it with a recursion that updates the estimate of g at *every* iteration. Note that the model-free version replaces $r(s,a,s')$ by an observed reward r.

[54] With the caveat that the semi-gradient replaces the gradient.

Q-learning

The following Q-learning algorithm generalizes the variant of Algorithm 10.8 that uses (10.60) to update the gain.

Algorithm 10.12. Q-learning for an average reward model

1. **Initialize:**
   (a) Set $q(s,a) \leftarrow 0$, $g \leftarrow 0$ and $\text{count}(s,a) \leftarrow 0$ each $a \in A_s$ and $s \in S$.
   (b) Specify learning rates $\tau_n, n = 1, 2, \ldots$ and $\beta_n, n = 1, 2, \ldots$.
   (c) Specify the number of iterations N.
   (d) Specify a sequence $\epsilon_n, n = 1, 2, \ldots$ (or η_n for softmax sampling).
   (e) Specify $s \in S$ and $a \in A_s$.
   (f) $n \leftarrow 1$.
2. **Iterate:** While $n \leq N$:
   (a) $\text{count}(s,a) \leftarrow \text{count}(s,a) + 1$.
   (b) Simulate (s', r) or sample s' from $p(\cdot|s,a)$ and set $r \leftarrow r(s,a,s')$.
   (c) **Update** $q(s,a)$**:**

$$q(s,a) \leftarrow q(s,a) + \tau_{\text{count}(s,a)} \left(r - g + \max_{a' \in A_{s'}} q(s',a') - q(s,a) \right). \tag{10.81}$$

   (d) **Update** g**:**

$$g \leftarrow g + \beta_n (r - g). \tag{10.82}$$

   (e) Generate $a \in A_{s'}$ using ϵ-greedy (or softmax) sampling.
   (f) $s \leftarrow s'$.
   (g) $n \leftarrow n + 1$.
3. **Terminate:** Return $q(s,a)$ for all $a \in A_s$ and $s \in S$.

Some comments follow:

1. Note that since g is updated at every iteration, its learning rate β_n in (10.82) is indexed by the iteration index n. On the other hand, the learning rate for $q(s,a)$ in (10.81) may be indexed by n or the number of times that state-action pair (s,a) has been previously evaluated.

2. Perhaps better estimates could be obtained by updating g using (10.82) before updating $q(s,a)$ using (10.81).

3. Observe that there are four hyperparameters to specify: two learning rates, the exploration rate and the number of iterations. Experimentation with these parameters in the context of policy evaluation through temporal differencing (Algorithm 10.8) might guide choice.

4. As noted earlier in this chapter, the above implementation is off-policy, that is the action chosen for execution at the next iteration need not correspond to the value $\max_{a' \in A_s} q(s',a')$ used in the update (10.81). A SARSA (on-policy update) replaces this expression by

$$q(s,a) \leftarrow q(s,a) + \tau_{\text{count}(s,a)} \left(r - g + q(s',a') - q(s,a) \right).$$

where a' denotes an ϵ-greedy action choice in state s'. Note that in numerical experiments, SARSA did not perform well.

5. Algorithm 10.12 converges[55] under the conditions that every state-action pair is visited infinitely often, the Markov chain of every stationary policy is regular and aperiodic, and the Robbins-Monro conditions on the learning rates hold.

6. If the algorithm is implemented using simulation, state-action based sampling at the start of step 2 might enhance empirical performance through more accurate estimates of $q(s,a)$ for infrequently visited states.

Relative Q-learning

Numerical studies suggest that the iterates in the above algorithm frequently diverge. The following alternative approach[56] generalizes relative value iteration (Algorithm 7.2). It considers updates of the form

$$q(s,a) \leftarrow q(s,a) + \tau \left(r - f(q) + \max_{a' \in A_{s'}} q(s',a') - q(s,a) \right), \tag{10.83}$$

where possible choices for $f(q)$ include:

1. $f(q) = q(\bar{s},\bar{a})$ for some specified pair $(\bar{s},\bar{a})$,

2. $f(q) = \max_{a \in A_{\bar{s}}} q(\bar{s},a)$ for some specified state $\bar{s}$, and

3. $f(q)$ equal to the average of $q(s,a)$ over all state-action pairs.

[55] See Tsitsiklis and Roy (1999).

[56] This algorithm and variants was proposed and analyzed by Abounadi et al. (2001).

Many implementations of relative value iteration use the specification $f(q) = q(\bar{s}, \bar{a})$, which was used to analyze (numerically) relative value iteration in Chapter 7. However, its performance is sensitive to the designation of $(\bar{s}, \bar{a})$. If visited infrequently, estimates will be inaccurate resulting in inaccurate estimates of $q(s, a)$. If possible, frequent restarts in $(\bar{s}, \bar{a})$ may result in improved performance.

Algorithm 10.13. Relative Q-learning for an average reward model

1. **Initialize:**
   (a) Specify a real-valued function $f(q)$.
   (b) Set $q(s, a) \leftarrow 0$ and $\text{count}(s, a) \leftarrow 0$ each $a \in A_s$ and $s \in S$.
   (c) Specify the learning rate $\tau_n, n = 1, 2, \ldots$
   (d) Specify the number of iterations N.
   (e) Specify a sequence $\epsilon_n, n = 1, 2, \ldots$ (or η_n for softmax sampling).
   (f) Specify $s \in S$ and $a \in A_s$.
   (g) $n \leftarrow 1$.
2. **Iterate:** While $n \leq N$:
   (a) $\text{count}(s, a) \leftarrow \text{count}(s, a) + 1$.
   (b) Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
   (c) **Update** $q(s, a)$**:**

$$q(s, a) \leftarrow q(s, a) + \tau_{\text{count}(s,a)} \left(r - f(q) + \max_{a' \in A_{s'}} q(s', a') - q(s, a) \right). \quad (10.84)$$

   (d) Generate $a \in A_{s'}$ using ϵ-greedy (or softmax) sampling.
   (e) $n \leftarrow n + 1$ and $s \leftarrow s'$.
3. **Terminate:** Return $q(s, a)$ for all $a \in A_s$ and $s \in S$.

Observe that Relative Q-learning and Q-learning differ in how they estimate g. Relative Q-learning computes the estimate of g, $f(q)$, in step 2(c) using the previously evaluated $q(s, a)$, while Q-learning directly updates g at every iteration using (10.82).

From a theoretical perspective, in a unchain model:

1. The iterates of $f(q)$ converge to g^*,
2. The iterates of $q(s, a)$ converge to $q^*(s, a)$ normalized so that $f(q^*) = g$,
3. The greedy policy based on $q^*(s, a)$ is optimal.

These results hold under the Robbins-Monro step-size conditions and any of the above specifications for $f(q)$.

The two-state example

Applying Q-learning and Relative Q-learning to the two-state model of Section 2.5 highlights some inherent challenges when using these methods. Calculations in Section 7.7.2 establish that the stationary policy $(d^*)^\infty$ that chooses $d^*(s_1) = a_{1,2}$ and $d^*(s_2) = a_{2,2}$ is optimal, that $g^* = 2.86$, and the relative value obtained setting $h^*(s_1) = 0$ is $h^*_{\text{rel}}(s_2) = h^*(s_2) - h^*(s_1) = -2.14$.

An immediate issue is how to compare methods and model specifications. Comparing resulting policies on the basis of their gain and agreement with the optimal policy is straightforward. Comparisons based on the state-action value functions is problematic because $q^*(s,a)$ is unique up to a constant that is independent of (s,a). Instead results can be compared on the basis of relative values[57], under a specification such as $h^*(s_1) = 0$.

To determine an easy to use representation for relative values, recall that $q^*(s,a)$ is defined in Chapter 7 by

$$q^*(s,a) = E^{(d^*)^\infty}\left[\sum_{n=1}^{\infty}\left(r(X_n,Y_n) - g^*\right)\middle| X_1 = s, Y_1 = a\right].$$

Therefore $h^*(s) = q^*(s, d^*(s))$ for all $s \in S$ where $(d^*)^\infty$ is an optimal stationary policy. Since

$$d^*(s) \in \arg\max_{a \in A_s} q^*(s,a),$$

$h^*(s) = \max_{a\in A_s} q^*(s,a)$. To ensure $h^*(s_1) = 0$, subtract $h^*(s_1)$ from $h^*(s)$ for all $s \in S$ so that

$$h^*_{\text{rel}}(s) := h^*(s) - h^*(s_1)$$

for all $s \in S$.

Putting this altogether, if in this example $q(s,a)$ is obtained by applying Q-learning or Relative Q-learning, an estimate of $h^*_{\text{rel}}(s_2)$ is given by

$$h_{\text{rel}}(s_2) = q(s_2, d^*(s_2)) - q(s_1, d^*(s_1)) = \max_{a\in A_{s_2}} q(s,a) - \max_{a\in A_{s_1}} q(s_1,a) \tag{10.85}$$

Therefore, $h_{\text{rel}}(s_2)$ can be compared to $h^*(s_2) = -2.14$. The beauty of using this representation for $h_{\text{rel}}(s)$ is that it extends easily to larger applications and avoids evaluating the greedy policy explicitly. It also could be the basis for a formal termination criterion for Q-learning or Relative Q-learning.

Estimates using Q-learning and Relative Q-learning set $\tau_n = n^{-1}\log(n+1)$, $\beta_n = n^{-1}$ (for Q-learning), and used ϵ_n-greedy exploration with $\epsilon_n = 150/(300+n)$. The

[57] This quantity is more stable than $q^*(s,a)$ since some state-action pairs may be visited infrequently and not accurately estimated.

index for τ_n was the number of visits to each state-action pair and the index for ϵ_n was the number of previous visits to that state[58]. Relative Q-learning used seven choices for $f(q)$: $f(q) = q(\bar{s}, \bar{a})$ where $(\bar{s}, \bar{a})$ varied over all (four) state-action pairs (denoted RQL1-RQL4), $f(q) = \max_{a \in A_s} q(\bar{s}, a)$ for some $\bar{s}$ (denoted RQL5-RQL6) and $f(q) = \text{mean}\, q(\cdot, \cdot)$ (denoted RQL7). Comparisons were based on 40 independent replicates. Each replicate evaluated Q-learning and Relative Q-learning across all 7 choices of $f(q)$, with $N = 20{,}000$, 35,000, and 50,000. Common random number seeds were used for each of 24 instances evaluated within a replicate.

Table 10.8 gives the number of replications in which there were the designated number of non-optimal actions, classified by algorithm version and number of iterations. Observe that:

1. Q-learning most accurately identified the optimal policy. The impact of run length was minimal.

2. Relative value results varied with the form of $f(q)$. Accuracy for all improved as the number of iterations increased.

3. The differences between RQL1-RQL4 show that the method is sensitive to choice of $(\bar{s}, \bar{a})$.

4. RQL5, which used $f(q) = \max_{a \in A_{s_1}} q(s_1, a)$, more accurately chose the optimal policy than RQL6, which used $\bar{s} = s_2$.

Iterations	Non-optimal actions	QL	RQL1	RQL2	RQL3	RQL4	RQL5	RQL6	RQL7
20,000	0	38	33	23	30	27	34	28	32
	1	0	5	10	7	4	3	7	6
	2	2	2	7	3	9	3	5	2
35,000	0	39	34	28	34	26	36	29	34
	1	0	4	10	5	6	2	8	5
	2	1	2	2	1	8	2	3	1
50,000	0	39	34	28	37	26	37	31	36
	1	0	6	10	2	8	1	6	3
	2	1	0	2	1	6	2	3	1

Table 10.8: Distribution of the number of non-optimal actions in 40 replicates broken down by method and run-length for average reward version of the two-state model.

Figure 10.22 shows the accuracy and variability of estimates of g^* and $h^*(s_2)$, classified by run length and optimization method. Observe that:

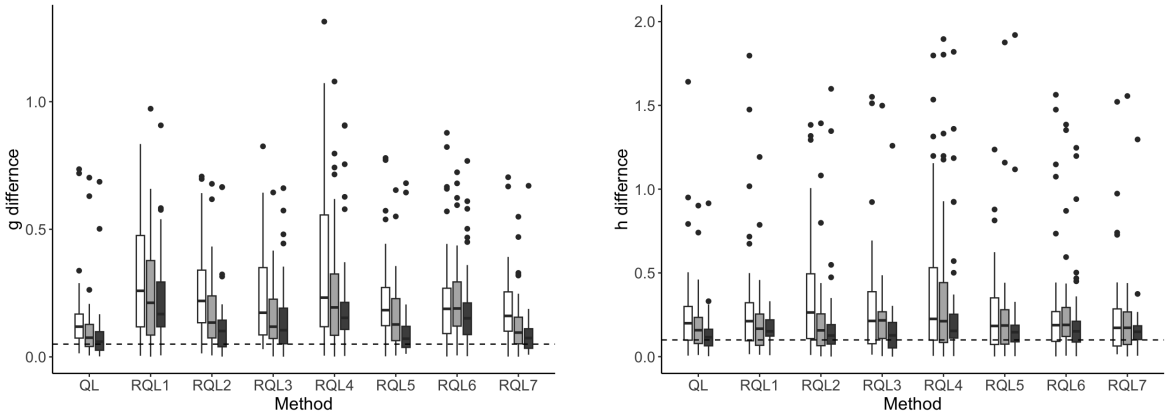
1. Q-learning produced the most accurate and least variable estimates of g^*. Its variability decreased as N increased.

[58] For state s, $n = \sum_{a \in A_s} \text{count}(s, a)$.

2. RQL7 and RQL5 provided the best estimates of g^* among the Relative Q-learning methods. These methods used the most iterations to estimate $f(q)$. Overall, Relative Q-learning methods were more sensitive to the number of iterations than Q-learning.

3. Q-learning estimates of $h^*(s_2)$ were as at least as accurate as those of any other methods. Precision increased and variability decreased with the number of iterations.

4. The accuracy and variability of Relative Q-learning estimators were similar across all choices of $f(q)$ and most were comparable to that of Q-learning. Accuracy increased and variability decreased with increasing number of iterations.



(a) $|g - g^*|$ (b) $|h - \text{rel}(s_2) - h^*_{\text{rel}}(s_2)|$

Figure 10.22: Difference between estimates and true values classified by estimation and number of iterations (white fill - 25,000; gray fill - 35,000; dark gray fill - 50,000). Dashed lines are included to facilitate comparisons and not meant as targets for the estimators. Note that the scale of the two figures differ.

As a consequence of these observations, Q-learning, as implemented in Algorithm 10.12, most frequently and efficiently[59] identified the optimal policy. Moreover it provided the most accurate and least variable estimates of g^* and $h^*(s_2)$.

One reason for the accuracy of estimating the gain was that estimates were updated at *every* iteration (using a running mean). Accuracy and variability of Relative Q-learning methods were sensitive to choice of $f(q)$ and were surpassed by Q-learning.

Of course, these conclusions apply only to this example and the specified learning rate and action sampling parameters. To establish broader validity requires similar comparisons in other environments and a review of relevant recommendations in the literature.

[59]The accuracy of the policies identified with $N = 35{,}000$ and $N = 50{,}000$ were the same.

10.8 Policy iteration type algorithms*

There is little general agreement on the structure of policy iteration algorithms in simulation-based environments. The methods proposed below provide plausible implementations.

This section focuses on discounted models; development of simulation-based policy iteration algorithms for episodic and average reward models is left to the reader. Q-learning and SARSA may be viewed as adaptations of value iteration algorithms from Section 5.6 to a simulation environment. Each iterate involves some form of maximization with Q-learning using the "max" to update $q(s,a)$ and SARSA using ϵ-greedy or softmax sampling to select actions at each update.

As was seen in Chapter 5, policy iteration and modified policy iteration involved less frequent maximizations and faster convergence, so it may be fruitful to investigate simulation-based versions of policy iteration (Section 5.7) and modified policy iteration (Section 5.8). In general, the structure of such algorithms can be represented by Figure 10.23. In earlier chapters, such an algorithm was implemented using the known Markov decision process model to compute $q^{d^\infty}(s,a)$ exactly using (5.64). This section explores the use of simulation methods to estimate this intermediate quantity. As a consequence of the variability introduced through simulation, the stopping rule "stop when $d' = d$" may require modification.

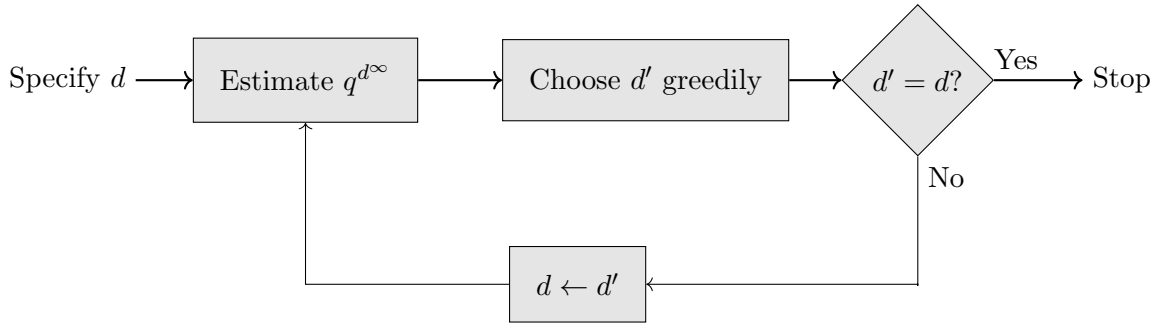


Figure 10.23: Structure of a policy iteration algorithm.

Simulation-based policy iteration methods can implement the estimation step in different ways:

1. Estimate $v_\lambda^{d^\infty}(s)$ and compute $q_\lambda^{d^\infty}(s,a)$ using the model.
2. Sample to estimate $r(s,a,j)$ and $p(j|s,a)$, then use these quantities to estimate $v_\lambda^{d^\infty}(s)$ and $q_\lambda^{d^\infty}(s,a)$.
3. Estimate $v_\lambda^{d^\infty}(s)$ and then simulate to estimate $q_\lambda^{d^\infty}(s,a)$.
4. Estimate $q_\lambda^{d^\infty}(s,a)$ directly.

A discussion of the first and last methods follow. Analysis of the other two approaches is left to the reader.

10.8.1 Hybrid policy improvement

First, consider a value function-based algorithm in which the evaluation step uses simulation to estimate the policy value function and then implements "exact" improvement in the same way as deterministic policy iteration in Chapter 5. While this may be impractical to implement when either S or A_s are large, and contrasts with the philosophy of this chapter, it will provide a useful baseline for algorithms in which improvement also involves simulation.

Algorithm 10.14. Hybrid policy improvement for a discounted model

1. **Initialize:** Specify $d \in D^{\mathrm{MD}}$, stopping tolerance $\nu \in \mathbb{Z}_+$, and set $\Gamma = S$.
2. **Iterate:** While $|\Gamma| > \nu$:
   (a) **Evaluation:** Evaluate d using TD(0), TD(γ) or Monte Carlo to obtain $\hat{v}(s)$ for $s \in S$.
   (b) **Construct $\hat{q}(s,a)$:** For all $s \in S$ and $a \in A_s$, set
   $$\hat{q}(s,a) \leftarrow p(j|s,a)(r(s,a,j) + \lambda \hat{v}(j)). \tag{10.86}$$
   (c) **Exact improvement:** For all $s \in S$,
   $$d'(s) \in \arg\max_{a \in A_s} \hat{q}(s,a), \tag{10.87}$$
   setting $d'(s) = d(s)$ if possible.
   (d) $\Gamma \leftarrow \{s \in S \,|\, d'(s) \neq d(s)\}$.
   (e) $d \leftarrow d'$.
3. **Terminate:** Return d.

Note that the termination criterion is based on the number of states in which actions change in successive iterations of step 2. While the condition $\nu = 0$ works for exact methods, empirical results suggest that it is too stringent for practical applications.

This algorithm does not apply to model-free settings in which $p(j|s,a)$ and $r(s,a,j)$ are not known. However, it can be adapted to a model-free setting using a two-stage approach in which $\hat{p}(j|s,a)$ and $\hat{r}(s,a,j)$ are learned from data. To facilitate this, the algorithm should start with a decision rule that randomizes action choice in all states. After obtaining these estimates, Algorithm 10.14 can be applied using

$$\hat{q}(s,a) := \hat{p}(j|s,a)(\hat{r}(s,a,j) + \lambda \hat{v}(j)). \tag{10.88}$$

Furthermore these estimates can be improved while evaluating subsequent decision rules.

Example 10.9. Hybrid policy improvement in the two-state model.

This example applies Algorithm 10.14 to the two-state model from Section 2.5 with $\lambda = 0.9$ and stopping tolerance $\nu = 0$. Results from the example in Section 10.7.2 guide parameter choice. It uses TD(0) for evaluation, sets $N = 5{,}000$ and compares two learning rates: $\tau_n = \log(n+1)/n$ and $\tau_n = 20/(50+n)$, where n represents the number of visits to the state. It also compares implementing TD(0) using state-based uniform random sampling with trajectory-based sampling. The algorithm is initialized with the sub-optimal policy $d = (a_{1,1}, a_{2,1})$.

The state-action value functions corresponding to this policy are

$$\begin{aligned}
\hat{q}(s_1, a_{1,1}) &= 3 + 0.9\big(0.8\hat{v}(s_1) + 0.2\hat{v}(s_2)\big) \\
\hat{q}(s_1, a_{1,2}) &= 5 + 0.9\hat{v}(s_2) \\
\hat{q}(s_2, a_{2,1}) &= -5 + 0.9\hat{v}(s_2) \\
\hat{q}(s_1, a_{2,2}) &= 2 + 0.9\big(0.4\hat{v}(s_1) + 0.6\hat{v}(s_2)\big)
\end{aligned}$$

where $\hat{v}(s)$ is an estimate of $v_\lambda^{d^\infty}(s)$ for the policy being evaluated.

Experiments consisted of 50 replicates using common random number seeds to compare learning rates and methods for generating the next state. Comparisons were based on the number of iterations to converge, the policy identified and the RMSE between the estimated $\hat{q}(s,a)$ and the known value of $q_\lambda^*(s,a)$.

In *all* cases the algorithm terminated with the optimal policy in a few iterations. For $N = 5{,}000$, the termination condition was achieved in most cases in five or fewer iterations. When N was increased to 20,000, trajectory-based sampling ensured convergence within three iterations for both learning rate choices. However, with state-based sampling, a few replications required up to five iterations.

Table 10.9 shows that TD(0) implemented with the log learning rate and trajectory-based state sampling most accurately estimated $q_\lambda^*(s,a)$ for both values of N. Furthermore, increasing N from 5,000 to 20,000 significantly improved the quality of estimates and resulted in faster algorithmic convergence.

It is left to the reader to apply variants of policy iteration that:

1. Estimate $r(s,a,j)$ and $p(j|s,a)$ assuming a model-free environment and then using them to estimate $q_\lambda^{d^\infty}(s,a)$, and
2. Estimate $\sum_{j\in S} p(j|s,a)\big(r(s,a,j) + \lambda\hat{v}(j)\big)$ using sampling after estimating $\hat{v}(\cdot)$.

Learning rate	State sampling	RMSE	
		$N = 5{,}000$	$N = 20{,}000$
$\log(n+1)/n$	State-based	1.88 (1.37)	0.92 (0.73)
	Trajectory-based	1.59 (1.43)	0.66 (0.55)
$20/(50+n)$	State-based	2.36 (1.54)	1.00 (0.75)
	Trajectory-based	2.22 (1.31)	0.77 (0.79)

Table 10.9: Mean (standard deviation) of RMSE errors of q-function estimates using hybrid policy iteration classified by learning rates and state generation method based on 50 replicates.

10.8.2 Hybrid policy iteration bounds

While theoretically justified in exact computation, the stopping rule in Algorithm 10.14 lacks a theoretical basis. However, one would expect that if values were accurately estimated, the condition to stop when $d' = d$ would eventually hold. Value function bounds, previously discussed in Chapter 5, provide a more rigorous approach for stopping, as well as bounding the difference between the value of a policy and the optimal value function.

Theorem 5.12 shows that for any function $v(s)$ on S,

$$v(s) + (1-\lambda)^{-1} \min_{s' \in S}\{L\mathbf{v}(s') - v(s')\} \leq v_\lambda^{d^\infty}(s) \leq v_\lambda^*(s) \leq v(s) + (1-\lambda)^{-1} \max_{s' \in S}\{L\mathbf{v}(s') - v(s')\}$$

where

$$L\mathbf{v}(s) := \max_{a \in A_s} \left\{ \sum_{j \in S} p(j|s,a) \left(r(s,a,j) + \lambda v(j) \right) \right\} = \max_{a \in A_s} q(s,a)$$

and

$$d(s) \in \arg\max_{a \in A_s} q(s,a).$$

Applying the above expressions to the iterates of hybrid policy iteration results in the following readily applicable bounds[60].

[60] We do not believe these have previously been used with simulated data.

Theorem 10.2. Suppose $\hat{v}(s)$, $\hat{q}(s,a)$ and d' are determined in step 2 of Algorithm 10.14. Then

$$\hat{v}(s) + (1-\lambda)^{-1} \min_{s' \in S} \left\{ \max_{a' \in A_{s'}} \hat{q}(s',a') - \hat{v}(s') \right\} \leq v_\lambda^{(d')^\infty}(s) \tag{10.89}$$
$$\leq v_\lambda^*(s) \leq \hat{v}(s) + (1-\lambda)^{-1} \max_{s' \in S} \left\{ \max_{a' \in A_{s'}} \hat{q}(s',a') - \hat{v}(s') \right\}$$

It is important to emphasize that for (10.89) to be valid $\hat{q}(s,a)$ must be defined by (10.86). While one might hypothesize that this inequality remains valid when $\hat{q}(s,a)$ is estimated directly (as in Q-learning), our numerical experiments indicated that this supposition is false. Obtaining efficient stopping rules for algorithms based on state-action value functions remains an open question. One approach in a model-based environment would be to intermittently set $\hat{v}(s) = \max_{a \in A_s} \hat{q}(s,a)$ and subsequently use (10.86) to obtain $\hat{q}(s,a)$ that is compatible with the bounds.

Since all of the quantities in (10.89) are available after step 2(b) of hybrid policy iteration, the above bounds provide the basis for the following stopping criterion for Algorithm 10.14.

Corollary 10.1. Let

$$\delta := (1-\lambda)^{-1} \left(\max_{s' \in S} \left\{ \max_{a' \in A_{s'}} \hat{q}(s',a') - \hat{v}(s') \right\} - \min_{s' \in S} \left\{ \max_{a' \in A_{s'}} \hat{q}(s',a') - \hat{v}(s') \right\} \right). \tag{10.90}$$

Then if $\delta < \epsilon$,

$$v_\lambda^*(s) - \hat{v}(s) < \epsilon \quad \text{for all } s \in S.$$

Moreover (10.89) provides upper and lower bounds on $v_\lambda^*(s)$.

Example 10.10. This example illustrates the application of the stopping criterion in Corollary 10.1 and the bounds on the optimal value function in Theorem 10.2 when hybrid policy iteration is applied to the two-state model. Guided by results in Example 10.9, it generates 50 replicates of hybrid policy iteration with the policy value function estimated by TD(0) with trajectory-based state sampling, a log learning rate and $N = 20{,}000$.

The stopping criterion was implemented with $\epsilon = 2$ meaning that the algorithm terminates when $\delta < 2$. This tolerance was chosen since it was known for this model with $\lambda = 0.9$ that the difference[a] between the value of the optimal policy and that of its closest stationary policy exceeded 2. Smaller values of δ were problematic because of variability in estimates of $\hat{v}(s)$.

Table 10.10 shows the distribution of iterations required to satisfy $\delta < 2$ across replicates. Observe that the algorithm terminated in four or fewer iterations in 86% of the replicates.

Upper and lower bounds along with the value of δ for a typical replicate appear in Table 10.11. In this replicate, hybrid policy iteration would terminate after four iterations. Note that δ is not monotone as a result of variability in TD(0) estimates of $\hat{v}(s)$.

The empirical validity of the bound is supported by the observation that $\mathbf{v}^*_\lambda = (30.15, 27.94)$ lies within *all* computed upper and lower bounds. Moreover, the value of the second-best stationary policy, $(27.19, 25.63)$, lies outside of the bounds for all iterations except the first.

[a] Of course this tolerance would not be known a priori in practice.

Iterations	2	3	4	5	6	7
Count	10	15	18	2	4	1

Table 10.10: Distribution of the number of iterations for hybrid policy iteration to satisfy $\delta < 2$ in 50 replicates.

Iteration	Lower bound	Upper bound	δ
1	$\begin{bmatrix} -59.42.62 \\ -109.09 \end{bmatrix}$	$\begin{bmatrix} 248.51 \\ 198.83 \end{bmatrix}$	307.92
2	$\begin{bmatrix} 27.71 \\ 26.03 \end{bmatrix}$	$\begin{bmatrix} 34.91 \\ 33.23 \end{bmatrix}$	7.20
3	$\begin{bmatrix} 28.54 \\ 26.69 \end{bmatrix}$	$\begin{bmatrix} 33.29 \\ 31.43 \end{bmatrix}$	4.74
4	$\begin{bmatrix} 30.10 \\ 27.90 \end{bmatrix}$	$\begin{bmatrix} 30.25 \\ 28.05 \end{bmatrix}$	0.15
5	$\begin{bmatrix} 29.11 \\ 26.79 \end{bmatrix}$	$\begin{bmatrix} 30.67 \\ 28.36 \end{bmatrix}$	1.57

Table 10.11: Upper and lower bounds for $v^*_\lambda(s)$, and the bound, δ, obtained from a typical replicate in Example 10.10.

10.8.3 Online policy improvement

The following policy iteration algorithm is suitable for both model-free and model-based implementations and is designed for online implementation. Its beauty is that it estimates the optimal state-action function directly without first estimating a policy value function, making a model unnecessary. Simulation-based evaluation of $q(s,a)$ is analogous to modified policy iteration.

The improvement step of a policy improvement algorithm requires an estimate of

$$q_\lambda^{d^\infty}(s,a) := \sum_{j\in S} p(j|s,a)\big(r(s,a,j) + \lambda v_\lambda^{d^\infty}(j)\big). \tag{10.91}$$

Therefore, it is not sufficient to just evaluate d through simulation since that will only provide an estimate of $q_\lambda^{d^\infty}(s,d(s)) = v_\lambda^{d^\infty}(s)$.

To overcome the absence of estimates of $q(s,a)$ for $a \neq d(s)$, the proposed algorithm estimates $q(s,a)$ for all states and actions by replacing a deterministic policy d^∞ by a randomized policy that is "close" to d^∞. To do this, the algorithm evaluates the *ϵ-tweaked policy* d_ϵ that chooses actions in state s according to

$$w_{d_\epsilon}(a|s) = \begin{cases} 1-\epsilon & a = d(s) \\ \epsilon/(|S|-1) & a \neq d(s). \end{cases} \tag{10.92}$$

It is important to emphasize that this is different than an ϵ-greedy policy because the policy it is based on need not be optimal. This policy can be implemented in a simulation by sampling action $d(s)$ with probability $1-\epsilon$ and choosing a specific other action with probability $\epsilon/(|S|-1)$. It may be prudent to decrease ϵ as calculations ensue.

Algorithm 10.15. Online simulated policy improvement for a discounted model

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MD}}$, stopping tolerance $\nu \in \mathbb{Z}_+$, and set $\Gamma = S$.
   (b) Specify the number of evaluation iterations N.
   (c) Specify a sequence $\epsilon_n, n = 1,2,\ldots$ for ϵ-tweaked action sampling and learning rate sequence $\tau_n, n = 1,2,\ldots$.
2. **Iterate:** While $|\Gamma| > \nu$:
   (a) Specify $s \in S$ and set $n \leftarrow 1$.
   (b) $q(s,a) \leftarrow 0$ and $\mathrm{count}(s,a) \leftarrow 0$ for all $a \in A_s$ and $s \in S$.
   (c) **Evaluate** $q^{d^\infty}(s,a)$**:** While $n \leq N$:
      i. $i \leftarrow \sum_{a\in A_s} \mathrm{count}(s,a)$ and $\epsilon \leftarrow \epsilon_i$.

ii. Sample a from $w_{d_\epsilon}(\cdot|s)$.

iii. Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.

iv. **Update** $q(s, a)$**:**

$$q(s, a) \leftarrow q(s, a) + \tau_{\text{count}(s,a)}(r + \lambda q(s', d(s')) - q(s, a)). \quad (10.93)$$

v. $\text{count}(s, a) \leftarrow \text{count}(s, a) + 1$ and $n \leftarrow n + 1$.

vi. $s \leftarrow s'$.

(d) **Greedy improvement:** For all $s \in S$,

$$d'(s) \in \arg\max_{a \in A_s} q(s, a), \quad (10.94)$$

setting $d'(s) = d(s)$ if possible.

(e) $\Gamma = \{s \in S \,|\, d'(s) \neq d(s)\}$.

(f) $d \leftarrow d'$.

3. **Terminate:** Return d.

Some comments about applying this algorithm follow:

1. The algorithm might be initiated with a randomized policy that chooses actions in each state with equal probability to obtain a good preliminary estimate of $q(s, a)$, and ϵ could be set to 1 for the first pass through the evaluation step.

2. Choosing $\epsilon_1 = 1 - (1/|S|)$ starts the evaluation with a decision rule that randomizes over all actions. In general, ϵ_n should decrease with n but not too quickly. The algorithm uses the number of times a state has been visited (as opposed to the iteration number) to ensure that there is sufficient action sampling in each state.

3. Step 2(a) can be implemented with a fixed or random s.

4. The algorithm stops when d' and d differ in at most a small number, ν, of states. Setting $\nu = 0$ recovers the stopping rule "stop when a decision rule repeats". However, this rule may be too stringent when q-functions are estimated directly. A simpler option is to specify the number of times to execute step 2.

5. There are two options for the update of $q(s, a)$ in (10.93). As stated, it is an off-policy algorithm that corresponds to Q-learning where the maximum is over the single policy d. Alternatively an on-policy update, corresponding to SARSA, requires sampling the action a' in state s' according to the ϵ-tweaked decision rule d_ϵ and replacing $d(s')$ in (10.93) by a'. This would require adding $a \leftarrow a'$ at the end of the Evaluate loop.

One would suspect that the off-policy variant would be preferable since the objective is to obtain an estimate of $q_\lambda^{d^\infty}(s,a)$ and not $q_\lambda^{d_\epsilon^\infty}(s,a)$.

6. Step 2(c)vi is specified for trajectory-based sampling. State-based sampling replaces $s \leftarrow s'$ by "Sample $s \in S$". Alternatively, state-action pair sampling would also replace Step 2(c)ii by "sample $a \in A_s$".

7. The algorithm is stated with TD(0) updates. Alternatively, TD(γ) updates could be used.

Example 10.11. Online simulated policy iteration in the two-state model

This example applies Algorithm 10.15 to the two-state model with $\tau_n = \log(n+1)/n$, $\epsilon_n = 1{,}500/(3{,}000+n)$, $N = 50{,}000$ and $\nu = 0$ corresponding to stopping when two successive policies are identical. Forty replicates with common random number seeds were used to investigate the combined effect of using:

1. on-policy of off-policy updates in (10.93), and

2. trajectory-based or state-based sampling in step 2(c)vi.

In all cases, the algorithm terminated with an optimal policy. The on-policy variant terminated in three iterations regardless of how the next state was generated while the off-policy variant converged in three iterations in 73 out of 80 cases and in five iterations in 7 out of 80 cases.

Table 10.12 shows the accuracy of the q-function estimates based on the RMSE between the estimated and true values. As expected, the off-policy methods, which are related to Q-learning, were considerably more accurate than the on-policy methods, which are related to SARSA.

The off-policy trajectory-based version produced the most accurate estimates suggesting that an online implementation produces good results. But, the algorithm cycled with less judicious choices of τ_n and ϵ_n so care is needed when specifying parameters.

Update	State sampling	RMSE (mean ± SD)
On-Policy	State-based	17.48 ± 7.70
	Trajectory-based	20.69 ± 5.35
Off-Policy	State-based	0.88 ± 1.21
	Trajectory-based	0.31 ± 0.35

Table 10.12: Summary of the accuracy of q-function estimates using variants of online simulated policy iteration for the two-state model based on 40 replicates with common random number seeds.

10.9 Queuing service rate control revisited

The infinite horizon version of the queuing service rate control model introduced in Section 3.4.1 provides a good platform to investigate some challenges that arise when implementing the above algorithms in a simple but more realistic model. Particular features that make this problem attractive for analysis include:

1. the ordered and interpretable state space,
2. the known structure of optimal value functions, and
3. the proven monotonicity of the optimal policy.

This section focuses on the discounted version but a short discussion of the average reward version is also included. The objective of this analysis is to find an optimal policy for future use so it is not necessary to use an online implementation or evaluate performance while learning.

Even though implementation involves only 51 states with three actions in each, the problem is sufficiently complex to raise a range of implementation issues that must be considered when dealing with modern large-scale applications. Experiments with a wide range of options are reported and provide a framework that may be applied broadly.

The analysis truncates the length of the queue at 50 and uses the parameters: arrival rate $b = 0.2$, service rates $a_1 = 0.2$, $a_2 = 0.4$ and $a_3 = 0.6$, holding cost $f(s) = s^2$, service rate cost $m(a_k) = 5k^3$ for $k = 1, 2, 3$, and discount rate $\lambda = 0.9$. Because the objective is to minimize the expected discounted cost, it is formulated as a *minimization* problem with positive costs.

Using value iteration and policy iteration (Chapter 5) when $\lambda = 0.9$, the optimal policy $(d^*)^\infty$ was found to be

$$d^*(s) = \begin{cases} a_1 = 0.2 & s \in \{0, \ldots, 10\} \\ a_2 = 0.4 & s \in \{11, \ldots, 29\} \\ a_3 = 0.6 & s \in \{30, \ldots, 50\} \end{cases} \tag{10.95}$$

and the optimal value function was monotone increasing and had monotone increasing differences (was convex) on $\{0, 1, \ldots, 48\}$[61]. Under the average reward criterion, the optimal policy uses actions $a_1 = 0.2$ in states $\{0, 1, 2\}$, $a_2 = 0.4$ in states $\{3, \ldots, 8\}$ and $a_3 = 0.6$ in states $\{9, \ldots, 50\}$.

10.9.1 Monte Carlo policy evaluation

This section explores the use of truncation and geometric stopping time Monte Carlo estimates of the discounted value function for the policy d^∞ with $d(s) = a_2$ for all

[61] As a result of truncation, the values for $s \in \{49, 50\}$ deviated from this pattern.

$s \in S$. Recall that estimates of $v_\lambda^{d^\infty}(s)$ in each state are obtained by averaging over replicates in that state.

Some observations and comments follow:

1. Only starting-state Monte Carlo applies. This is because the truncation horizon or geometric stopping time are defined in terms of the length of the horizon starting from the first transition. Hence, first-visit estimates, which would start later, would correspond to shorter horizons.

2. In this model, the estimated standard deviation $\hat{\sigma}(s)$ of $\hat{v}(s)$ (Figure 10.24a) varied over the states. Thus, sample sizes (number of Monte Carlo replicates) needed to obtain the same precision for all states must vary with the state. Note that the "artificial" boundary at $s = 50$ resulted in the non-monotone behavior near the upper boundary.

3. By the Central Limit Theorem, the Monte Carlo estimate for state s is asymptotically normal with standard deviation approximately equal to $\hat{\sigma}(s)/n^{-0.5}$. This relationship can be used to choose the sample size to obtain a pre-specified level of confidence.

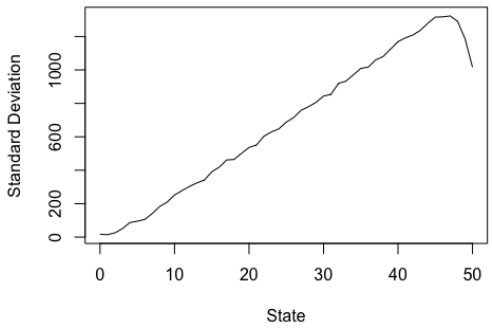
4. In light of the previous comment, the truncation estimator used $K = \max\{100, 4s^2\}$ replicates in state s resulting in a total of 172,080 replicates.

5. Differences between the estimated and true value function are shown in Figure 10.24b for the truncation estimate. Note the RMSE of these estimates (over states) was 12.3. These estimates were more precise than TD(0) estimates described below (Figure 10.26a). However, the total number of replicates was considerably larger.

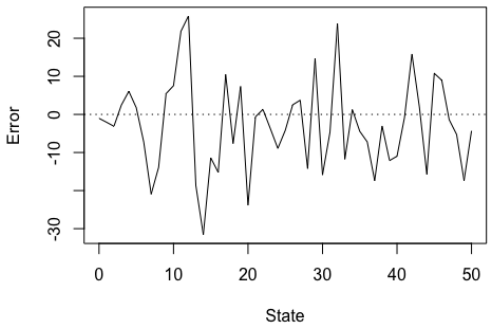
6. Geometric stopping time estimates were considerably more variable than the truncation estimates (standard deviation in state 50 was 200 times greater than that of the truncation estimator) and much less accurate overall (RMSE = 103.7). Note that the expected number of terms in the geometric sums was 11 as opposed to 125 used in truncation.

10.9.2 TD(γ) policy evaluation

Evaluation using TD(γ) presents some challenges. Under the policy that uses action a_1 in every state, the system is a symmetric reflecting random walk on $\{0, \ldots, 50\}$ so that in steady state all states are visited with equal probability. Therefore applying TD(γ) for evaluation of this policy with trajectory-based sampling is straightforward since all states will be visited approximately the same number of times in a simulation. However, evaluating other reasonable policies becomes problematic because the system



(a) Estimated standard deviation of Monte Carlo policy value function estimates.



(b) Difference between estimated and true policy value function. The number of replicates in state s was set at $\max\{100, 4s^2\}$.

Figure 10.24: Some results for Monte Carlo estimation based on reward sequence truncation at $M = 125$.

primarily visits the low-numbered states[62]. Hence, estimates of value functions or state-action value functions using trajectory-based sampling will be highly variable for the infrequently visited, high-occupancy states. Therefore, it seems plausible that state-based sampling after each transition would improve the accuracy of TD(γ) methods.

The following small study investigated the effect of learning rate, parameter of TD(γ), trace type and policy on the accuracy of policy value function estimates. The study consisted of 40 replicates of 100,000 iterations for each of five learning rates (exponential, ratio $(150/(300+n)$ and $15/(30+n))$, STC, and Log), four values of γ, two trace types (replacement and accumulation) and two stationary policies (one that used action a_2 in every state and another that repeated the sequence (a_1, a_2, a_3) beginning in state 0 and continuing through state 50). The discount rate was set equal to $\lambda = 0.9$, states were generated randomly at each iteration and common random number seeds were used in each replicate. The index of the learning rate was the number of previous visits to a state.

Since the true policy value function could be found using exact methods for policy evaluation, replicates were summarized in terms of the RMSE of the policy value function estimates over all states. Figure 10.25 displays the main results graphically. Observe that:

1. Trace type and policy had insignificant effect on results and were not included in the figure.

[62] It is left as an exercise to compute the stationary distribution under policies that use action a_2 or a_3 in all states.

2. The Log and ratio $(150/(300+n))$ learning rate generated very high RMSEs and were not included in the figure.

3. The ratio learning rate $\tau_n = 15/(30+n)$ generated estimates with the smallest RMSEs. For this learning rate, $\gamma = 0$, corresponding to TD(0), gave the smallest RMSE (41.2). For the STC learning rate, $\gamma = 0.2$ gave the smallest RMSE.

4. Combined TD(0) and ratio estimates were most accurate in low-numbered states on an absolute level (Figure 10.26).

Figure 10.26 shows that the TD(0) policy value function estimates deviated from the true policy value function by at most 112. On a relative basis this error is at most 2.3%.

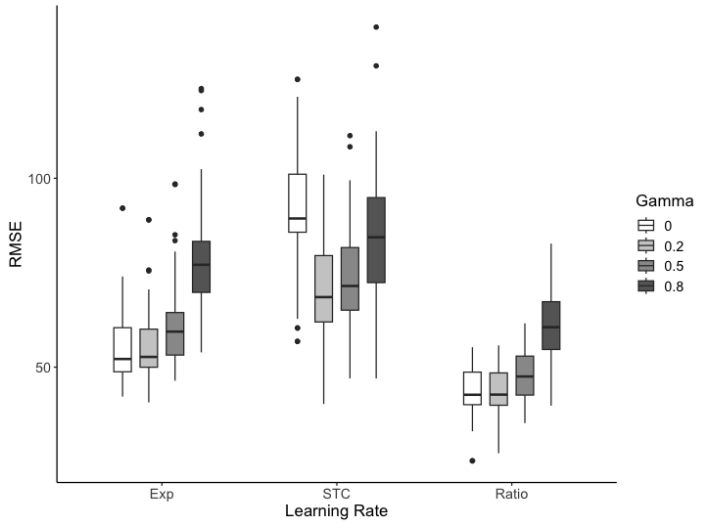
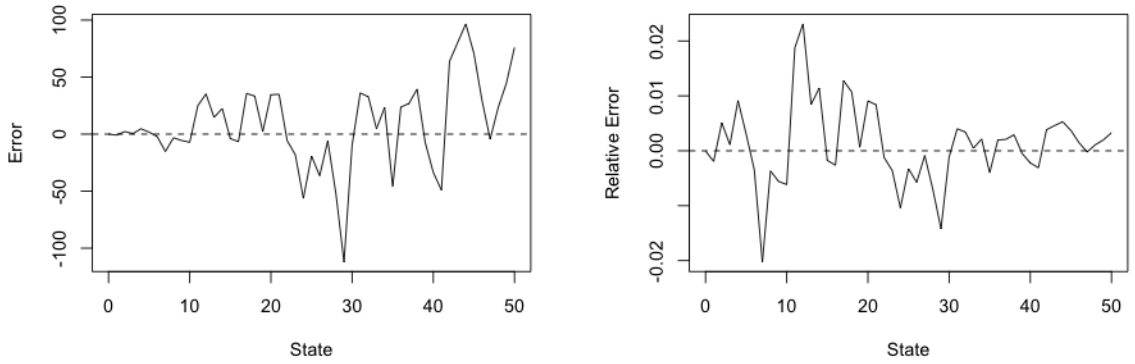


Figure 10.25: Graphical summary of the dependence of RMSE on learning rate and γ when using TD(γ) for policy value function approximation for d^∞ with $d(s) = a_2$ for all $s \in S$. Each boxplot represents a summary over 40 replicates. Note the effect of trace and policy were insignificant and are not shown.

Comparison of TD(γ) and Monte Carlo estimates

Table 10.13 compares two Monte Carlo estimates with the best TD(γ) estimate on the basis of RMSE and the number of transitions evaluated to obtain a policy value function estimate for all states. Note that the RMSE for TD(0) was the average over 40 replicates of length 100,000 but in practice one would only generate one such estimate. On the other hand the large number of replicates required to obtain Monte Carlo estimates was due to the need to obtain multiple replicates for each starting state. Thus it appears that the TD(0) method is the computationally most efficient (at the expense of parameter tuning).



(a) Difference between estimated and true policy value function.

(b) Relative difference between estimated and true policy value function.

Figure 10.26: Error and relative error using TD(0) with learning rate $\tau_n = 15/(30+n)$ for estimating the policy value function of d^∞ where $d(s) = a_2$ for all $s \in S$ based on single randomly selected replicate.

Method	RMSE	Total number of transitions to estimate the policy value function
Monte Carlo - Truncation	12.3	182,384 × 125
Monte Carlo - Geometric	103.7	182,384 × 11
TD(0) with ratio learning	41.2	100,000

Table 10.13: Summary of value function calculations. Note 182,384 represents the total number of sequences generated when obtaining estimates.

10.9.3 Q-learning

This section explores the use of the Q-learning algorithm (Algorithm 10.11) and its SARSA variant. Since the optimal value function and policy can be computed using methods in Chapter 5, the quality of the estimates can be assessed by comparing the policy value function of the greedy policy[63] to the optimal value function on the basis of RMSE over all states. In addition, the greedy policy can be compared to the optimal policy on the basis of the percentage of states on which they agree.

State-based versus trajectory-based sampling

The first step in the analysis was a comparison of trajectory-based sampling and uniform state-based sampling in individual replicates. Results for a typical replicate are displayed in Figure 10.27. Figures 10.27a and 10.27c show the greedy policy (points)

[63] Found by solving $(\mathbf{I} - \lambda \mathbf{P}_d)\mathbf{v} = \mathbf{r}_d$ where d^∞ denotes the greedy policy.

and the optimal policy (dashed line). Figures 10.27b and 10.27d show the difference in values (on different scales) of the greedy policy and the optimal policy.

Observe that the greedy policy derived using state-based sampling closely matched the optimal policy and achieved a value very close to it. On the other hand, the greedy policy derived using trajectory-based sampling had a similar structure and value to the optimal policy only in states $0 - 17$.

The reason for such poor performance in high-occupancy states using trajectory-based sampling is that in 450,000 replicates, states $18 - 50$ were visited infrequently: for example, states $30 - 50$ were visited at most three times. Hence, in these states, the $q(s, a)$ estimates were extremely inaccurate. Theoretically, although the underlying Markov chain is regular, the limiting probabilities for high-occupancy states are extremely small making them rarely observed under trajectory-based sampling.

In practice, this limitation may not impact overall system performance since high-occupancy state are seldom encountered. However, this issue can be addressed by:

1. Comparing values using RMSE weighted by the limiting probabilities of the optimal policy[64].

2. Using state-based (or state-action based sampling) to ensure all states or state-action pairs are sampled sufficiently.

This issue is not limited to this queuing model. Similar limitations arise when using Q-learning with trajectory-based sampling in other process-type applications. Thus, where the objective is to find an optimal policy applicable in **all** states, state-based or state-action based sampling is preferred.

State-based sampling

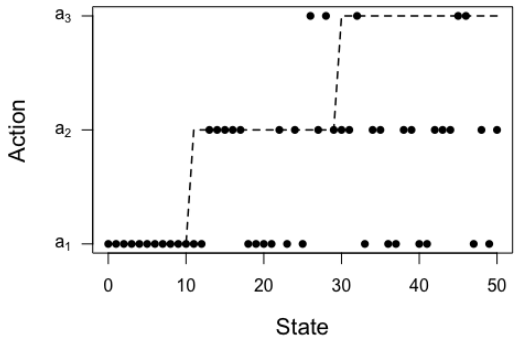
The following results were based on an experiment that used 40 replicates of length 450,000 with random state-based sampling at each iteration. It compared five learning rate specifications (used for policy evaluation above), four choices (0.1, $150/(300+n)$, $15/(30+n)$, $0.7n^{-0.5}$) for ϵ_n and two estimation methods (Q-learning and SARSA).

Q-learning and SARSA produced equivalent results. The most accurate learning rate τ_n and ϵ_n-greedy specifications among those considered were

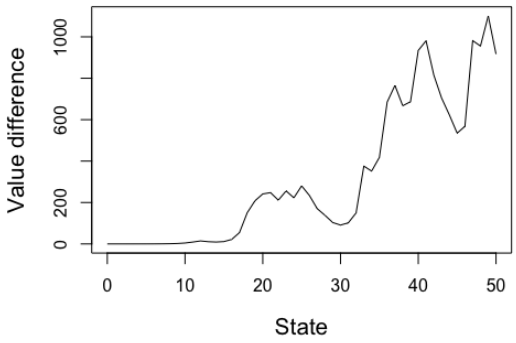
$$\tau_n = \frac{0.1}{1 + n^2/10^5} \quad \text{and} \quad \epsilon_n = \frac{150}{300 + n}. \tag{10.96}$$

For the learning rate, n represented the number of visits to a state-action pair and for ϵ_n, n represented the number of visits to a state. This combination generated the

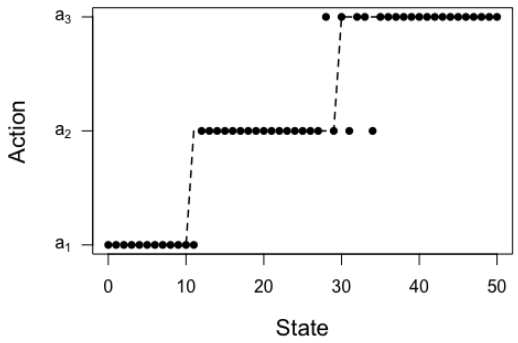
[64] Of course these probabilities would not be known in practice. Instead, one could assign weights based on observed empirical frequencies or the importance of states. To use a sports analogy, training should focus on skills for situations most likely to be encountered in competition and limited for unlikely scenarios.



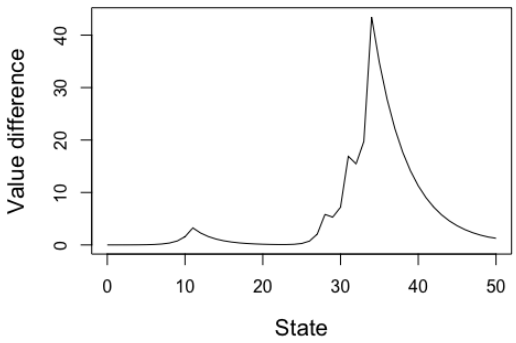
(a) Policy - Trajectory-based



(b) Value - Trajectory-based



(c) Policy - State-based



(d) Value - State-based

Figure 10.27: Comparison of policies and values using Q-learning with trajectory-based and state-based sampling. Note the difference in the scales in Figures 10.27b and 10.27d.

most accurate estimates of the state-action value function and the highest percentage of states in which the greedy policy was optimal over 40 replicates.

Results in Figures 10.27c and 10.27d were typical for this specification. Using Q-learning, the state-action value function estimates were extremely accurate. The mean RMSE of the state-action value function (across replicates) was 19.8 with standard deviation of 11.1, which is small relative to the range of the $q(s, a)$ values spanning 70 to 22,600. Greedy policies deviated from the optimal policy in only a few states. On average, 90.6% of actions generated by the greedy policy agreed with the optimal policy, with a standard deviation of 3.7%.

10.9.4 Policy iteration type algorithms

This section describes the use of hybrid policy iteration (Algorithm 10.14) and online policy iteration (Algorithm 10.15). Recall hybrid policy iteration estimates a policy value function using simulation and then uses the model to compute the q-function, while online policy iteration estimates q-functions directly. Implementation choices[65] include:

Policy evaluation method: TD(0), TD(γ), or Monte Carlo with truncated discounted rewards or geometric stopping times;

Algorithmic parameters: learning rates, ϵ-greedy exploration settings, and number of iterations;

Sampling method: state-based or trajectory-based updates;

Improvement step: exact vs. estimated q-functions;

Policy value function estimates: smoothed vs. unsmoothed value functions;

q-function estimates: smoothed vs. unsmoothed state-action value functions;

Smoothing technique: Parametric model used for smoothing;

Stopping criterion: Extent of agreement of policies at successive iterations.

Configurations were compared on the basis of whether the algorithms terminated or cycled, how similar the identified policy was to the optimal policy, and how well the estimated value function approximated the optimal value function.

To obtain convergence in online policy iteration, it was necessary to smooth $q(s, a)$ as a function of s prior to greedy improvement. Moreover, smoothing could potentially enhance convergence of hybrid policy iteration. A convenient choice for this purpose was a *quadratic B-spline*[66]. Chapter 11 provides a systematic analysis for combining simulation and function approximation.

Hybrid policy iteration: Implementation

Algorithm 10.14 combines policy value function estimation with greedy improvement. Given a policy value function estimate $\hat{v}(s)$, it uses the model to evaluate (10.86) according to

$$q(s, a) = \sum_{j \in S} p(j|s, a)(r(s, a, j) + \hat{v}(j)) \tag{10.97}$$

[65] Not all apply to each method.

[66] B-splines are linear combinations of polynomials that flexibly approximate nonlinear functions. In R, they are specified with the function `bs()` and fit with function `lm()`. To avoid pre-specifying knot locations, our implementations here used the specification `degree=2` and `df=2`.

$$= \begin{cases} 5a^3 + (1-q)\hat{v}(0) + q\hat{v}(1) & s = 0 \\ s^2 + 5a^3 + a\hat{v}(s-1) + (1-a-q)\hat{v}(s) + q\hat{v}(s+1) & s \in \{1, 2, \ldots, 49\} \\ 50^2 + 5a^3 + a\hat{v}(49) + (1-a)\hat{v}(50) & s = 50. \end{cases}$$

Note that $r(s, a, j)$ is independent of j.

Policy evaluation used TD(0) and starting-state Monte Carlo with truncation and geometric stopping times, and both smoothed and unsmoothed q-functions. For each of these six combinations, 40 replicates with common seeds across each configuration within a replicate were evaluated.

Details of the implementation follow:

1. TD(0) parameters were specified as $\tau_n = 15/(30 + n)$ and $N = 100{,}000$. States were resampled after each transition.

2. Each replicate of Monte Carlo evaluated all starting states. Both truncated and geometric stopping time versions used $\max(100, 3s^2)$ iterations in state s. Truncation was set at $M = 100$[67].

3. The stopping criterion was "stop when $|\Gamma| \leq 2$" where Γ denotes[68] the set of states on which the policy at two successive iterates differ. More restrictive conditions led to cycling.

Hybrid policy iteration: Results

Results of the comparisons are summarized in Table 10.14. Without smoothing, hybrid policy iteration with TD(0) and Monte Carlo with truncation achieved the stopping criteria in approximately six improvement steps, with identical ranges of 2 to 18 steps. The algorithm did not converge when Monte Carlo estimates were based on geometric stopping times.

Average RMSE[69] for both approaches were comparable but values of greedy policies based on Monte Carlo replicates were less variable. Hybrid policy iteration with TD(0) identified the optimal policy in 4 of 40 cases while the Monte Carlo variant with truncation identified the optimal policy in 5 of 40 cases. However, when q-function smoothing was applied, both methods reduced the number of improvement steps to two and almost always identified the optimal policy.

Since both TD(0) and Monte Carlo with truncation accurately estimated the policy value function, hybrid policy iteration using these estimates achieved the stopping criterion and produced accurate estimates without smoothing. Given the greater computational effort to implement Monte Carlo with truncation, TD(0) is the preferred evaluation choice if hybrid policy iteration is to be used without smoothing. When

[67] These values were modified from Section 10.9.1 to speed up execution. Nonetheless, Monte Carlo with truncation was extremely time consuming (several hours on a Macbook Pro M3).

[68] See step 2(d) in the algorithm.

[69] Between the greedy policy and the optimal policy.

Evaluation Method	Smoothing q-function	Average Iterations	Range Iterations	Average RMSE	Range RMSE
TD(0)	No	5.58	(2, 18)	15.48	(0, 71.07)
	Yes	2	2	0	0
Monte Carlo (truncation)	No	5.85	(2, 18)	17.0	(0, 39.8)
	Yes	2	2	0	0
Monte Carlo (geometric stopping)	No	Did not converge	-	-	-
	Yes	2	2	0.31	(0, 1.52)

Table 10.14: Comparison of results of hybrid policy improvement methods based on 40 replications with common random number seeds.

smoothing is applied, all evaluation methods, including Monte Carlo with geometric stopping, perform similarly.

Online policy iteration: Implementation

Algorithm 10.15 was applied using $N = 450{,}000$, $\nu = 2$, learning rates $\tau_n = 0.1/(1 + n^2/10^5)$ and exploration parameters $\epsilon_n = 1{,}500/(3{,}000 + n)$. State transitions were modeled using random state-based sampling and actions were sampled using a ϵ_n-tweaked decision rule d_{ϵ_n}.

Both off-policy and on-policy updates for the state-action value function were examined. The on-policy update used

$$q(s,a) = q(s,a) + \tau_n\big(r(s,a,a') + \lambda q(s,a') - q(s,a)\big), \tag{10.98}$$

where a' is chosen using the ϵ_n-tweaked decision rule d_{ϵ_n}. The off-policy update uses (10.93).

Additionally, the effect of q-function smoothing was investigated. A total of 40 replicates were evaluated using common random numbers in each.

Online policy iteration: Results

Table 10.15 summarizes the results. Results for unsmoothed q-functions are omitted because decision rules generated using both on-policy and off-policy updates failed to achieve the proposed stopping criterion, even after 25 evaluation-improvement cycles. In fact, between 20 and 30 actions changed at each cycle.

Results in the above table suggests that using d for updating the q-function provides more reliable performance. This makes sense because $q(s,a)$ reflects the value of using action a in state s for one period and then following the policy d^∞, not the exploratory policy based on d_{ϵ_n}.

Thus, when applying online policy iteration, smoothing appears necessary to obtain convergence. Even in small problems, this suggests the potential value of function approximation.

Decision rule used in (10.93)	Smoothing q-function	Average Iterations	Range Iterations	Average RMSE	Range RMSE
d	Yes	3.95	(2, 10)	15.02	(0, 68.3)
d_{ϵ_n}	Yes	4.33	(2, 10)	12.81	(0, 58.4)

Table 10.15: Summary of results based on 40 replicates of Algorithm 10.15. In two replicates, the variant using d_{ϵ_n} in the update terminated after two iterations with sub-optimal policies (RMSE = 1523). These two outliers are excluded from the reported RMSE values for d_{ϵ_n}.

10.9.5 Computational summary: Discounted model

Analysis of the discounted queuing control model suggests:

1. Convergence and accuracy of algorithms was highly sensitive to learning rate and, when appropriate, to the exploration parameter ϵ_n. The specification of n (state-action pair visits for learning rate or state visits for exploration rate) impacted convergence.

2. TD(0) provided reliable estimates of the policy value function. TD(γ) estimates were less accurate.

3. Starting-state Monte Carlo provided accurate policy value function estimates but required more replicates than TD(0) to obtain comparable precision. Methods based on truncated discounted rewards were more accurate than those using geometric stopping times.

4. Optimization methods required state-based sampling to obtain near optimal policies. Trajectory-based methods sampled some parts of the state space too infrequently.

5. Using state-based sampling, Q-learning produced policies that differed from the optimal policies in a few states. Although smoothing the q-function was not necessary to obtain convergence, it likely would have generated monotone policies.

6. Policy iteration algorithms behaved best with a stopping criterion of the form "Stop when the decision rule at successive iterates differs in at most two states." Tighter stopping rules often led to cycling.

7. Hybrid policy improvement with state-based sampling terminated without q-function smoothing. However, q-function smoothing resulted in faster convergence to the optimal policy.

8. Online policy iteration required q-function smoothing for convergence. The off-policy variant as stated in Algorithm 10.15 converged more reliably.

These findings suggest that when a model is available, hybrid policy iteration with TD(0) or truncated Monte Carlo policy value function estimation were most reliable and precise. However, the Monte Carlo estimates required considerably more computation. In the absence of a model, Q-learning was easiest to apply and provided good results without smoothing the q-function. Simulated policy iteration required q-function smoothing to ensure convergence. In contrast to the two-state example above, methods were most reliable when states were randomly sampled at each transition as opposed to trajectory-based sampling.

Smoothing the q-function enhanced convergence and produced monotone policies. Smoothing involved two-steps at each iteration, estimating the q-function for all state-action pairs and then approximating it by a smooth function prior to greedy action selection. This was possible in a small model, the next chapter investigates incorporating function approximation directly in the optimization algorithm enabling generalization to larger models.

10.9.6 Average reward queuing control

This section investigates the application of Algorithms 10.12 and 10.13 to an average reward version of the queuing service rate control model. Preliminary analyses suggest that the only effective approach was Relative Q-learning with state-based sampling.

Q-learning (Algorithm 10.12) consistently identified suboptimal policies that deviated significantly from the optimal policy under any state sampling regime. The key issue was the inaccuracy of estimates of g, which resulted in inaccurate estimates of state-action value functions. Relative Q-learning with trajectory-based sampling also failed to identify good policies, primarily because of infrequent visits to high-occupancy states.

Implementation and results

A discussion of the application of Relative Q-learning with state-based sampling follows. The implementation specified $f(q) = q(0, a_1)$ meaning this quantity was used to estimate the gain in (10.83). Computations compared configurations with learning rates $100/(200+n)$ and $0.2/(1+10^{-6}n^2)$ paired with either ϵ-greedy exploration with $\epsilon_n = 100/(200+n)$ or softmax exploration[70] with $\eta = 1/2{,}000$.

Each configuration was evaluated over 40 replicates with common random number seeds, simulating $N = 300{,}000$ total iterates (roughly 6,000 per state) per replicate. Methods were assessed on the accuracy of estimates of the optimal average reward g^* and the number of states in which the greedy policy differed from the optimal policy.

Table 10.16 summarizes the results. Clearly ϵ-greedy exploration was preferable to softmax exploration, the latter being highly sensitive to parameter choice. There was

[70] Results were sensitive to specification of η; choosing it too small resulted in limited search across actions.

a small benefit to using the STC learning rate as opposed to the ratio learning rate. As the table shows, its estimate of the optimal gain was more accurate.

Learning Rate	Exploration	Non-optimal Actions	Average Reward Difference
Ratio	ϵ-greedy	2.18 (1.31)	2.41 (1.70)
	Softmax	11.40 (3.33)	4.71 (2.73)
STC	ϵ-greedy	2.18 (1.20)	0.95 (0.84)
	Softmax	4.85 (2.19)	2.96 (1.49)

Table 10.16: Accuracy of Relative Q-learning estimates as a function of learning rate and exploration method. Entries are means (standard deviations) over 40 replicates.

Figure 10.28 compares the greedy policy to the optimal policy for a typical selected replicate. Observe that the resulting policy closely approximates the optimal policy, differing from it in only three states. In this replicate, the estimate of $g^* = 19.42$ was 16.58.

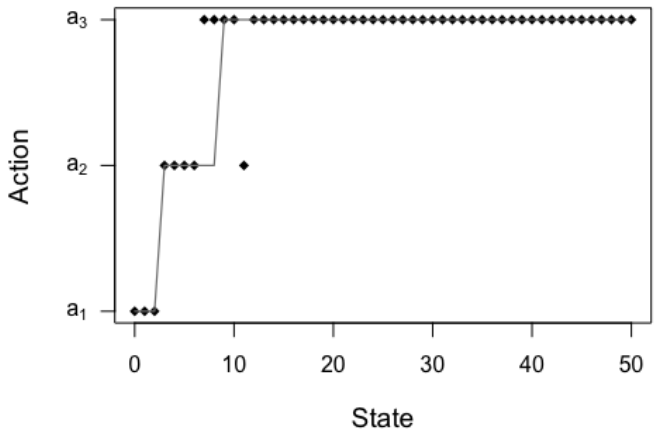


Figure 10.28: Plot of greedy decision rule (diamonds) and average optimal decision rule (line) for a randomly selected replicate using STC learning rate and ϵ-greedy exploration.

It was encouraging to find that Relative Q-learning, without any additional smoothing, identified policies that are close to optimal. The implementation used state-based sampling; actions were chosen by exploration. It is possible that state-action based sampling would result in even more accurate results.

10.10 Conclusion

The foundational methods in this chapter provide a "toolbox" of potential approaches (Monte Carlo, temporal differencing, TD(γ) for policy evaluation; Q-learning, SARSA and policy iteration type algorithms for optimization) to apply in simulated or process-based environments suitable for tabular representations. However, applying them requires considerable trial-and error tuning of learning rates, exploration parameters, sampling methods and run lengths to produce useful results. Moreover, appropriate specifications for a given application are not obvious *a priori*.

10.11 Technical appendix

10.11.1 Choosing good estimators

Policy evaluation methods in this chapter are concerned with estimates of an unknown quantity based on a sequence of observations or replicates of an experiment. Estimates may be computed offline using the whole data set, for example a sample mean, or online, updated sequentially using stochastic approximation algorithms described in Appendix 10.11.2. Standard statistical concepts such as bias, variance and mean squared error are useful for assessing the quality of an estimator.

Many examples in this chapter require choosing among estimates such as those represented by the different lines in Figure 10.29. They may be generated by different algorithms or from a single algorithm under different parameter specifications. In the figure, the light gray estimate is the least variable but fails to converge to the true value given by the horizontal black line, in other words, it is biased. The dark gray and black estimates both oscillate around the true value but the black is less variable and therefore preferred. This illustrates that choosing an estimator involves trading-off bias and variability as described below.

First, some caveats are in order.

1. The estimates in Figure 10.29 are based on a single replicate of a simulation using a common random number seed. Results may differ with different random sequences.

2. In practice, a single estimation method and a pre-specified number of iterates will be specified.

3. Results are sensitive to run lengths. If sampling stopped after approximately 4,200 steps, one might conclude that the estimate corresponding to the dark grey line gave the best approximation of the unknown quantity.

Visualizing results across different run lengths, as in this figure, facilitates selection of estimators that produce good estimates over a range of run lengths.

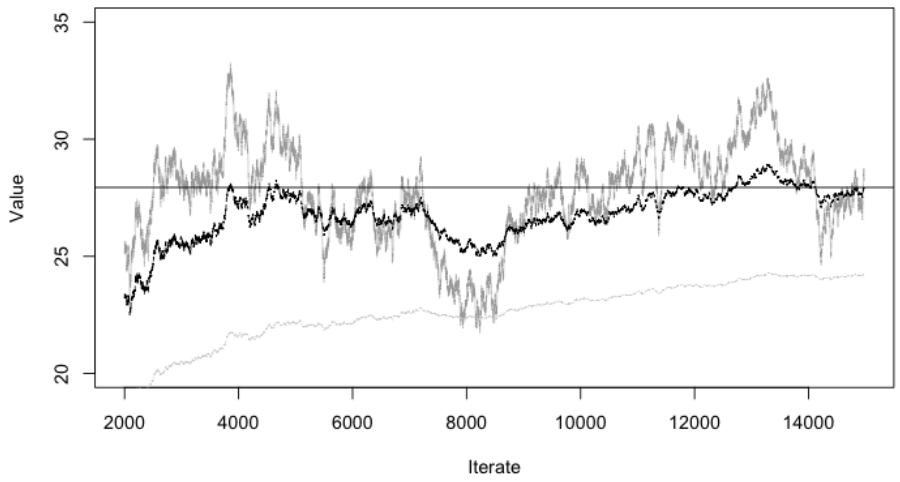


Figure 10.29: Three sequences of estimates of a quantity based on a single replicate of an online algorithm with run length of 15,000. The horizontal line represents the true value which is unknown in practice but known in the illustrative experiments used in many of the examples in Chapters 10 and 11.

RMSE for point estimation

Consider an experiment with K replicates that generates estimates $y^k, k = 1, 2, \ldots, K$ of a fixed scalar C. For example, y^k may be the estimate of a policy value function in a specified state obtained from a single Monte Carlo replicate.

The quality of the estimate is often assessed using the *mean squared error (MSE)* defined by

$$\text{MSE} := \frac{1}{K}\sum_{k=1}^{K}(y^k - C)^2.$$

Since the MSE is expressed in squared units, the *root mean squared error (RMSE)*, defined by

$$\text{RMSE} = \sqrt{\text{MSE}},$$

is preferred because it is on the same scale as the estimates and hence provides a more interpretable measure of accuracy.

The MSE is related to two other statistical measures, the *sample bias* given by

$$\text{Bias} := |\bar{y} - C|$$

and the *sample variance* given by

$$\text{Variance} := \frac{1}{K-1}\sum_{k=1}^{K}(y^k - \bar{y})^2,$$

where $\bar{y}$ denotes the sample mean. Note that the variance and MSE measure deviations from different quantities, the sample mean and true value, respectively. When K replaces $K-1$ in the variance, this quantity is referred to as the *population variance*.

These quantities are related through the expression:

$$\text{MSE} = \left(\frac{K-1}{K}\right)\text{Variance} + \text{Bias}^2. \tag{10.99}$$

To see this, note that

$$\begin{aligned}\sum_{k=1}^{K}(y^k - C)^2 &= \sum_{k=1}^{K}(y^k - \bar{y} + \bar{y} - C)^2 \\ &= \sum_{k=1}^{K}(y^k - \bar{y})^2 + 2\sum_{k=1}^{K}(y^k - \bar{y})(\bar{y} - C) + \sum_{k=1}^{K}(\bar{Y} - C)^2 \\ &= \sum_{k=1}^{K}(y^k - \bar{Y})^2 + K(\bar{Y} - C)^2,\end{aligned}$$

since the cross product term sums to zero, the result follows by dividing both sides by K.

Hence the mean squared error contains both a variance and a bias component. If the population variance replaces the sample variance in (10.99) the fraction disappears so that this relationship becomes:

$$\text{MSE} = \text{Population Variance} + \text{Bias}^2. \tag{10.100}$$

Many conclusions in examples will be based on the RMSE, so the breakdown in either (10.99) or (10.100) is intended primarily for interpretability of this concept. It is referred to as the *bias-variance tradeoff*.

Table 10.17 provides summary measures[71] for the estimators represented in Figure 10.29. Observe that the estimator represented by the light grey line has small

[71] This example does not fall exactly within the above framework, since in it, y^k represents the k-th value in a single replicate of length 15,000 for a single estimator. To interpret results, assume that statistical measures are computed after obtaining all 15,000 values so $\bar{y}$ is the overall mean value for that estimator.

variance and high bias, the estimator represented in dark grey has high variance and low bias, and that in black has both small variance and small bias. Moreover, the RMSE of the estimator in black is the smallest. On this basis one would prefer the estimator corresponding to the black line, consistent with the conclusions based on inspecting the figure directly.

Estimator	RMSE	MSE	Population Variance	Bias^2
Black	1.59	2.54	1.17	1.37
Dark-grey	2.09	4.37	4.33	0.04
Light-grey	5.47	29.93	1.58	28.35

Table 10.17: Statistical summaries of estimators in Figure 10.29.

RMSE for value function estimation

The RMSE is frequently used in a different context in this chapter, namely for assessing the accuracy of different approaches for estimating policy or optimal value functions. Suppose $\hat{v}(s)$ is an estimator of a known function $v(s)$ defined over a finite state space S. The RMSE of the estimate is defined by:

$$\text{RMSE}(\hat{\mathbf{v}}) := \sqrt{\frac{1}{|S|}\sum_{s\in S}(\hat{v}(s) - v(s))^2}. \tag{10.101}$$

Studies of simulation-based algorithms, such as in Section 10.9, often generate K replicates $\hat{v}^1(s), \ldots, \hat{v}^K(s)$ by varying the random number seed. In such settings, algorithmic performance is often assessed using summary statistics such as the mean, standard deviation and range of the sequence[72] of RMSE values $\text{RMSE}(\hat{\mathbf{v}}^1), \ldots, \text{RMSE}(\hat{\mathbf{v}}^K)$ displayed in tables or graphically using boxplots.

Of course when the primary objective is to find effective policies, as discussed in Section 10.6.2, precise estimation of value or state-action value functions may be of secondary importance. Algorithmic comparisons should be based on how close policies and their theoretical or simulated policy value functions are to optimal.

Weighted root mean squared error

Often it makes sense to replace RMSE by a *weighted RMSE*, especially when:

1. evaluating value functions which differ by orders of magnitude over S, or
2. in trajectory-based simulations where subsets of states are visited infrequently so that values in these states are not accurately estimated.

[72] Recall $\mathbf{v}$ represents a vector of values over states.

Moreover, a related concept, the weighted supremum norm is sometimes used to derive theoretical properties of estimators, especially when S is not finite.

The weighted RMSE of an estimator $\hat{\mathbf{v}}$ of a vector $\mathbf{v}$ is defined by

$$\text{wRMSE}(\hat{\mathbf{v}}) := \sqrt{\sum_{s \in S} w(s)\big(\hat{v}(s) - v(s)\big)^2}, \tag{10.102}$$

where $w(s) \geq 0$ for $s \in S$. Note setting $w(s) = 1/|S|$ yields the unweighted RMSE.

Suppose $w(s)$ is chosen to be the *stationary distribution* of the Markov chain corresponding to a policy being evaluated or that identified by an algorithm when optimizing. Such a measure better represents the accuracy (or potential loss of optimality) that will be observed in practice when implementing the policy. Of course, this distribution will not be known in optimization problems. However, it can be calculated in experimental settings in which the optimal policy is known.

Alternative choices for weights include the reciprocal of the variance of $\hat{v}(s)$ over multiple replicates of an experiment or an exponential or polynomial growth function.

10.11.2 Stochastic approximation

> *A new type of approximation process called "stochastic approximation" will play an important role in multistage decision processes*[73].
>
> Richard P. Bellman, Mathematician and father of dynamic programming, 1920-1984.

This prescient remark by Bellman underscores our decision to include this lengthy appendix on stochastic approximation. As anticipated by Bellman, stochastic approximation underlies the temporal differencing, Q-learning and policy gradient methods described in this chapter and in Chapter 11.

Stochastic approximation originates with the seminal paper Robbins and Monro [1951][74], which developed an online algorithm for finding the root of a function based on noisy observations of the function value at selected points. It has become especially important for analyzing simulation data because it avoids storing large amounts of data.

The Robbins-Monro procedure

Stochastic approximation is an *online* method for finding an x^* that solves

$$E[Y|X = x] = 0.$$

[73] Bellman [1963].

[74] As an aside, Herbert Robbins was the PhD advisor of Cy Derman, who was the PhD advisor of Arthur F. Veinott Jr., who was the PhD advisor of author MLP.

It uses samples $y^n, n = 1, 2, \ldots$ from a distribution with conditional mean $f(x) := E[Y|X = x]$ as the basis of an estimator. For example, the sequence y^n may be generated by $y^n = f(x^n) + \epsilon_n$ where ϵ_n denotes a random error term with mean 0 and the sequence $x^n, n = 1, 2, \ldots$ is chosen in some way.

A Monte Carlo approach would generate multiple samples at different values of x and use them to approximate $f(x)$ and estimate where it equals 0. Robbins and Monro [1951] proposed something different. Their procedure generates a sequence $x^n, n = 1, 2, \ldots$ that represents both estimates of x^* and points at which to apply a recursion of the form:

The Robbins-Monro recursion:

$$x^{n+1} = x^n - \tau_n y^n, \tag{10.103}$$

where y^n denotes an observation from a distribution with conditional mean $E[Y|X = x^n]$ and τ_n, referred to as the *step-size* or *learning rate*, represents a sequence of non-negative constants converging to 0.

The intuition underlying (10.103) (see Figure 10.30) is that when y^n is greater than 0, x^{n+1} will be *corrected* downward from x^n and conversely when y^n is less than 0, x^{n+1} will be corrected upward from x^n. Moreover, when y^n is close to zero the correction will be smaller as will be the case for large n when τ_n is smaller.

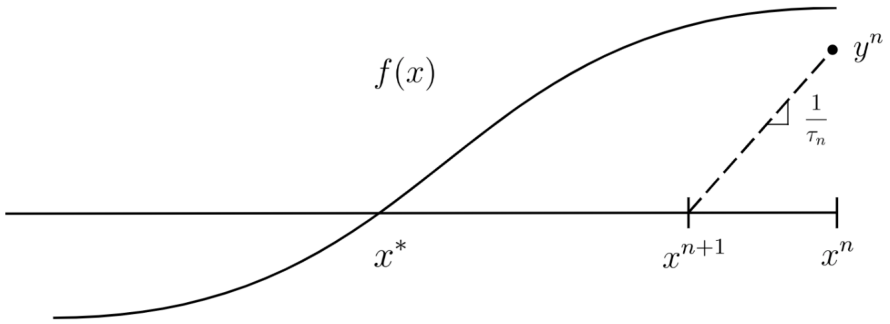


Figure 10.30: Geometric representation of Robbins-Monro recursion for solving $f(x) = 0$. Since $y^n/(x^n - x^{n+1}) = 1/\tau^n$, $x^{n+1} = x^n - \tau_n y^n$.

This recursion may be viewed as a stochastic generalization of Newton's method. When $f(x^n)$ is observable (and differentiable), Newton's method finds its zero by iter-

ating

$$x^{n+1} = x^n - (f'(x^n))^{-1} f(x^n). \tag{10.104}$$

Equation (10.103) replaces the reciprocal of the derivative by the constant τ_n and $f(x^n)$ by an observation sampled from a distribution with mean $f(x^n)$.

The following theorem (stated without proof) describes the limiting behavior of the sequence of iterates generated by (10.103).

Theorem 10.3. Suppose $f(x) = E[Y|X = x]$,

1. $f(x)$ is monotone,
2. $E[(Y - f(X))^2|X = x] < M < \infty$ for some constant M, and
3. $|f(x)| \leq d|x| + c$ for finite positive constants c and d,
4. the sequence $\tau_n, n = 1, 2, \ldots$ satisfies

$$\tau_n \geq 0, \quad \sum_{n=1}^{\infty} \tau_n = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \tau_n^2 < \infty. \tag{10.105}$$

Then for any x^1, the sequence

$$x^{n+1} = x^n - \tau_n y^n$$

converges almost surely to x^* where $f(x^*) = 0$.

Some comments about this theorem follow:

1. In most of the applications of this result in this book, x^n will represent an estimate of a parameter (weight) or vector of parameters and be denoted by $\boldsymbol{\beta}^n$.
2. The theorem provides a stronger version of the result in Robbins and Monro [1951] under slightly different conditions on $f(x)$. It is due to Blum [1954]. It establishes almost sure convergence[75] as opposed to L^2 convergence. The key conditions on the step-size in (10.105) remain the same.
3. Proofs of this theorem and its variants are beyond the scope of this book. They use subtle arguments and advanced probabilistic concepts. See the Bibliographic Remarks section for references.
4. Much of the literature refers to (10.105) as "the usual conditions on τ_n".

[75] Also called convergence with probability one.

5. The conditions on τ_n in Theorem 10.3 are satisfied when $\tau_n = k/n^b$ for $b \in (.5, 1]$, $\tau_n = c/(d+n)$ where c and d are positive constants and $\tau_n = \log(n+1)/n$. Note the choice $\tau_n = 1/n$ satisfies these conditions.

6. Note that in addition to specifying the correction term to the current estimate of the solution of $f(x) = 0$, the Robbins-Monro recursion specifies at which value of x^n to generate a new observation.

7. This theorem applies to a univariate method; a vector version is also valid. It will be discussed in the context of gradient methods below.

8. Equation (10.103) can be rewritten as

$$x^{n+1} = (1 - \tau_n)x^n + \tau_n(x^n - y^n)$$

This representation writes x^{n+1} as a weighted average of the previous estimate x^n and its difference with the new observation y^n. This is referred to as *exponential smoothing* in the forecasting literature.

Example 10.12 below shows that in agreement with theory, when $\sum_{n=1}^{\infty} \tau_n < \infty$, the step-sizes converge to 0 too quickly leading to convergence to an incorrect value. Moreover, when $\sum_{n=1}^{\infty} \tau_n^2 = \infty$, variability is not damped out so that iterates oscillate rapidly. When the conditions on the sums of τ_n and τ_n^2 are satisfied, the iterates converge.

Example 10.12. Impact of τ_n on stochastic approximation estimates.
This example shows how the conditions in (10.105) impact convergence of the stochastic approximation iterates. Consider the problem of finding the fifth root of 2. Put into the stochastic approximation framework, it can be expressed as solving

$$f(x) := x^{1/5} - 2 = 0$$

based on realizations of $f(x) + \epsilon$ where ϵ is normally distributed with mean 0 and standard deviation 1.

The example compares estimates based on four choices for τ_n corresponding to cases when (10.105) hold and are violated in two different ways. Each estimate is based on 50,000 iterates, common random number sequences for the four choices of τ_n and $x_0 = 20$. Of course $f(32) = 0$ so that $x^* = 32$.

Clearly conditions 1 and 3 of Theorem 10.3 hold and since $\text{var}(\epsilon) = 1$, condition 2 holds. Choosing $\tau_n = 0.9n^{-0.5}$ and $\tau_n = 100/(400+n)$ correspond to the cases in which both summation conditions in (10.105) are satisfied, choosing $\tau_n = n^{-1.0000001}$ corresponds to $\sum_{n=1}^{\infty} \tau_n < \infty$ and $\sum_{n=1}^{\infty} \tau_n^2 < \infty$ and choosing $\tau_n = n^{-0.25}$ corresponds to $\sum_{n=1}^{\infty} \tau_n = \infty$ and $\sum_{n=1}^{\infty} \tau_n^2 = \infty$. Note that $\sum_{n=1}^{\infty} \tau_n < \infty$ and $\sum_{n=1}^{\infty} \tau_n^2 = \infty$ is impossible since for $\tau_n > 0$ small, $\tau_n^2 < \tau_n$. (As an exercise,

verify that these sequences satisfy the indicated conditions.)

Figure 10.31 displays the sequence x^n corresponding to each of the four step-size sequences. The figure legend identifies the sequences. The figure shows that:

1. sequences corresponding to $\tau_n = 0.9n^{-0.5}$ and $\tau_n = 100/(400+n)$ converge to x^* with that based on the ratio more stable,
2. the sequence corresponding to $\tau_n = n^{-1.0000001}$, converges but to the wrong value, and
3. the sequence corresponding to $\tau_n = n^{-0.25}$ oscillates around the target value $x^* = 32$.

Consequently, different violations of the assumptions on τ_n have different impacts on the sequence of iterates x^n. Moreover, the ratio step-size parameter choice provides the most stable estimates.

The following example illustrates the use of stochastic approximation in a non-standard application.

Example 10.13. Using stochastic approximation to find a percentile

Since the p-th, $0 \leq p \leq 100$, percentile of the distribution of a random variable Y can be represented as the solution x^* of $E[I_{\{Y \leq x\}}] = p/100$, (10.103) can be used to find this percentile. To do so, specify x, sample y from its distribution and compute $I_{\{y \leq x\}}$.

For example, consider the problem of finding the 75th percentile of a standard Cauchy distribution[a]. In this case, (10.103) becomes

$$x^{n+1} = x^n - \tau_n \left(I_{\{y^n \leq x^n\}} - 0.75 \right).$$

Note that the quantity $I_{\{y^n \leq x^n\}} - 0.75$ only takes on the values 0.25 and -0.75.

Stochastic approximation is applied to estimate the 10th, 20th, 50th, 75th and 85th percentile of the Cauchy distribution with $\tau_n = 20/(200+n)$, $\tau_n = n^{-0.75}$ and $\tau_n = n^{-1}$. A total of 40 replicates of 10,000 iterates were generated using a common random number seed for each replicate and an initial value $x^1 = 0$.

Estimates were compared on the basis of within-replicate RMSE values (after deleting first 200 estimates) and visual inspection of plots. Table 10.18 summarizes the results. Observe that estimates using $\tau_n = n^{-1}$ (the sample average) produced the least accurate estimates for all percentiles so clearly it is advantageous to explore using stochastic approximation with learning rates other than $\tau_n = n^{-1}$. The reason that the running mean performed so poorly is that τ_n

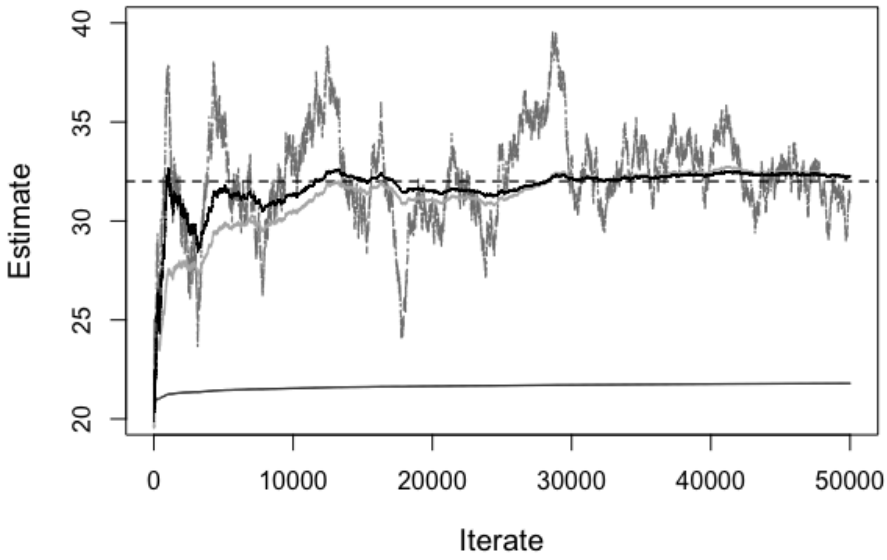


Figure 10.31: Graphical representation of sequence of estimates of x^* in Example 10.12. The horizontal dashed line gives the solution $x^* = 32$, the dark grey line, the iterates when $\tau_n = n^{-0.25}$, the line near the bottom of the figure corresponds to $\tau_n = n^{-1.0000001}$ and the grey and black lines that stabilize around 32 correspond to $\tau_n = 0.9n^{-0.5}$ and $\tau_n = 100/(400 + n)$, respectively.

decreased too quickly to allow learning.

The quality of estimates declined the further the percentile was from the mean 0. When accounting for both the mean and standard deviation, the ratio learning rate produced the most accurate estimates, especially in the tails of the distribution. Figure 10.32 shows a typical replicate comparing the three learning rates, substantiating the conclusions above.

[a]Recall that the Cauchy distribution has infinite variance so large values occur frequently. A standard Cauchy distribution has location parameter 0 and scale parameter 1.

10.11.3 Stochastic approximation and optimization

Stochastic approximation plays a key role in optimization where the objective is to minimize an expected loss function. It serves as a foundational concept for the online learning algorithms in this chapter and the next as follows.

Consider the problem of finding the value that minimizes a real-valued function of

Percentile	$\tau_n = \frac{20}{200+n}$	$\tau_n = n^{-0.75}$	$\tau_n = \frac{1}{n}$
10	3.43 (5.06)	5.02 (6.60)	17.40 (28.91)
20	0.64 (0.36)	0.57 (0.68)	5.58 (7.81)
50	0.29 (0.08)	0.13 (0.07)	0.72 (0.77)
75	0.45 (0.23)	0.38 (0.39)	5.43 (9.30)
85	1.08 (0.97)	3.82 (8.34)	12.42 (22.16)

Table 10.18: Mean (standard deviation) of within-replicate RMSE over 40 replicates of stochastic approximation estimates of percentiles of a Cauchy distribution using three learning rates.

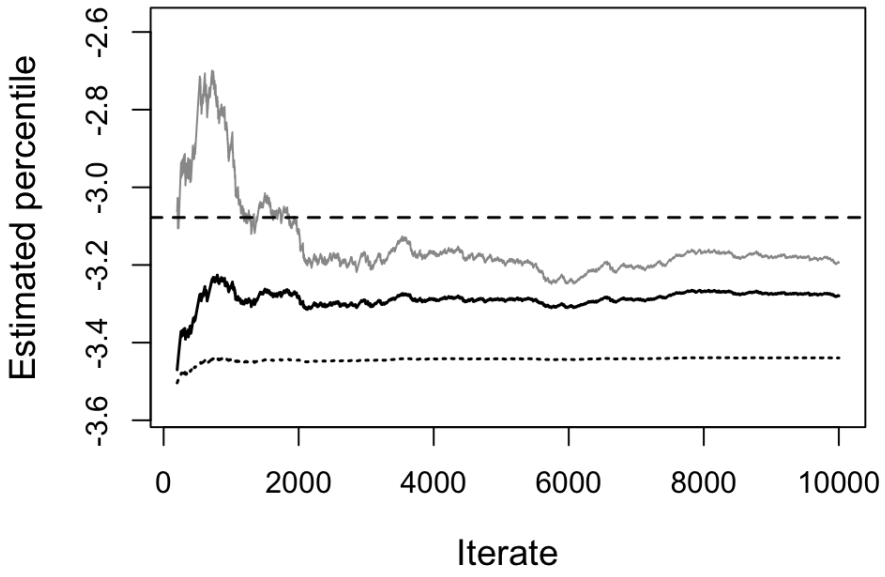


Figure 10.32: Plot of a single replicate of estimates for three learning rates of the 10th percentile of a Cauchy distribution in Example 10.13. The grey line corresponds to $\tau_n = 20/(200+n)$, the black line to $\tau_n = n^{-0.75}$ and the dotted line to $\tau_n = n^{-1}$. The horizontal line represents the true value of the 10th percentile.

a vector of variables $\boldsymbol{\beta}$, $G(\boldsymbol{\beta})$. That is

$$\boldsymbol{\beta}^* \in \arg\min_{\boldsymbol{\beta}} G(\boldsymbol{\beta}). \tag{10.106}$$

Gradient descent methods achieve this by solving the equation

$$\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta}) = 0, \tag{10.107}$$

where $\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta})$ denotes the gradient of $G(\boldsymbol{\beta})$. Implicit is the assumption that $G(\boldsymbol{\beta})$ is differentiable with respect to each component of $\boldsymbol{\beta}$. Under mild conditions, this minimum is unique but its possible that there may be multiple minima.

Gradient descent is an iterative procedure that chooses $\boldsymbol{\beta}$, observes $G(\boldsymbol{\beta})$, evaluates its gradient at $\boldsymbol{\beta}$ and updates $\boldsymbol{\beta}$ according to the recursion

$$\boldsymbol{\beta}' \leftarrow \boldsymbol{\beta} - \tau \nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta}), \tag{10.108}$$

where τ is the learning rate or step-size. This method should be well known to readers of this book.

Instead of observing $G(\boldsymbol{\beta})$ and computing its gradient, many of the methods in Chapters 10 and 11 are based on estimating the gradient using **sampled** data. That is, instead of computing the gradient directly, observations are generated from a distribution with **expectation** equal to $\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta})$. Then stochastic approximation is applied using the sample gradient $\widehat{\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta})}$ to update $\boldsymbol{\beta}$ according to

$$\boldsymbol{\beta}' \leftarrow \boldsymbol{\beta} - \tau \widehat{\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta})} \tag{10.109}$$

This recursion is often referred to as *stochastic gradient descent (SGD)* but other definitions of SGD appear in the literature. This book will refer to recursion (10.109) as SGD. Note that when tracking iteration number (as in Appendix of Chapter 9) this expression can be written as

$$\boldsymbol{\beta}^{n+1} = \boldsymbol{\beta}^n - \tau_n \widehat{\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta}^n)}.$$

Doing so allows specifying the explicit dependence of τ on n.

The following section illustrates this approach in a very important application.

Least squares regression

Least squares regression assumes a model for data of the form

$$y_j = f(\mathbf{x}_j; \boldsymbol{\beta}) + \epsilon_j \tag{10.110}$$

for $j = 1, \ldots, J$, where $\mathbf{x}_j$ denotes the j-th value of a $(I+1)$-dimensional vector of features[76], $\boldsymbol{\beta}$ denotes a $(I+1)$-dimensional vector of parameters, y_j denotes the j-th observation of the dependent variable and ϵ_j is an unobserved random disturbance

[76] Features are referred to as covariates or independent variables in statistical models.

with mean 0 and constant variance. The function $f(\mathbf{x}_j;\boldsymbol{\beta})$ denotes a **known** real-valued function that can be either linear or nonlinear in the parameters.

Least squares regression seeks parameter values $\boldsymbol{\beta}^*$ that minimize the *expected squared error loss function*

$$G(\boldsymbol{\beta}) := E[g(\boldsymbol{\beta})] = E[(Y - f(\mathbf{x};\boldsymbol{\beta}))^2], \tag{10.111}$$

where the expectation is with respect to the joint distribution of column vector $\mathbf{x}$ and scalar Y and the random variable

$$g(\boldsymbol{\beta}) := (Y - f(\mathbf{x};\boldsymbol{\beta}))^2.$$

This representation assumes that the random vector $\mathbf{x}$ is sampled from some distribution and Y is sampled from the conditional distribution of Y given $\mathbf{x}$ which has mean $f(\mathbf{x};\boldsymbol{\beta})$.

A standard approach to obtaining parameter estimates is to solve the equation

$$\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta}) = \mathbf{0}.$$

However, doing this might be complicated because of the expectation inside the gradient. Instead, under mild regularity conditions,

$$\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta}) = E[\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta})] \tag{10.112}$$

so that

$$\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta}) = E[-2(Y - f(\mathbf{x};\boldsymbol{\beta}))\nabla_{\boldsymbol{\beta}} f(\mathbf{x};\boldsymbol{\beta})], \tag{10.113}$$

which in the linear case, when $f(\mathbf{x};\boldsymbol{\beta}) = \mathbf{x}^\mathsf{T}\boldsymbol{\beta}$, simplifies to

$$E[\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta})] = E[-2(Y - \mathbf{x}^\mathsf{T}\boldsymbol{\beta})\mathbf{x}]. \tag{10.114}$$

Note that in this expression the expected gradient is a column vector with the number of components equal to the dimension of $\boldsymbol{\beta}$.

The reason for this formality is to set things up to apply stochastic approximation. Instead of solving $\nabla_{\boldsymbol{\beta}} G(\boldsymbol{\beta}) = \mathbf{0}$ directly to minimize the expected loss in (10.111), samples are obtained from a *distribution* with mean $E[\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta})]$ and the Robbins-Monro recursion is applied as follows.

Let $(\mathbf{x}^1, y^1), \ldots, (\mathbf{x}^N, y^N)$ denote a sequence of samples from the joint distribution of $\mathbf{x}$ and Y[77]. Then this sequence is used to estimate the gradient according to:

$$[\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta})]^n := -2(y^n - f(\mathbf{x}^n;\boldsymbol{\beta}))\nabla_{\boldsymbol{\beta}} f(\mathbf{x}^n;\boldsymbol{\beta}) \tag{10.115}$$

where the superscript refers to the n-th sample of the gradient. When $f(\mathbf{x};\boldsymbol{\beta})$ is a linear function of the parameters this expression becomes

$$[\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta})]^n = -2\big(y^n - (\mathbf{x}^n)^\mathsf{T}\boldsymbol{\beta}\big)\mathbf{x}^n. \tag{10.116}$$

[77] In the context of regression, subscripts indexed by j indicate data points, whereas superscripts indexed by n indicate a sequence of iterates.

In the linear case, the Robbins-Monro recursion becomes

$$\boldsymbol{\beta}' \leftarrow \boldsymbol{\beta} + \tau_n\big(y^n - (\mathbf{x}^n)^\mathsf{T}\boldsymbol{\beta}\big)\mathbf{x}^n, \tag{10.117}$$

where the multiplier 2 in the gradient expression has been absorbed into the learning rate and the negative sign in the gradient leads to a plus sign before τ_n.

Figure 10.33 summarizes the approach used to apply stochastic approximation when optimizing an expected value.

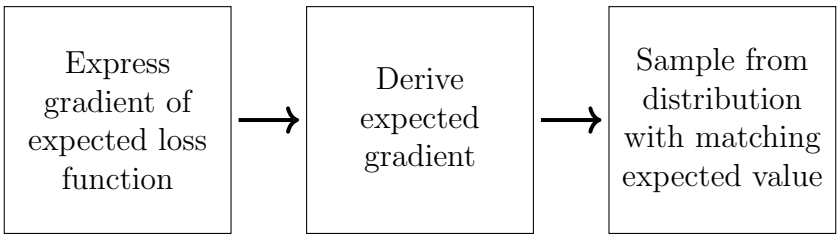


Figure 10.33: General approach for applying Robbins-Monro stochastic approximation to minimize expected loss.

A key theoretical result is that if the hypotheses of Theorem 10.3 hold for the gradient of the loss function, the sequence of iterates generated by stochastic gradient descent converges (in expectation) to a zero of the expected gradient.

Examples

Two applications of this approach follow.

Example 10.14. Using stochastic approximation to find a mean.
This example applies (10.117) to obtain an online recursion for estimating the mean μ of a random variable Y. In the additive error case, this can be viewed as a least squares regression problem in which $\boldsymbol{\beta}$ is a scalar equal to μ and $\mathbf{x}$ is a scalar equal to 1. This means that observations are generated according to

$$y^n = \mu + \epsilon^n$$

and the Robbins-Munro recursion becomes

$$\mu^{n+1} = \mu^n + \tau_n(y^n - \mu^n). \tag{10.118}$$

This recursion is the basis for the standard online formula for the sample mean

$\bar{y}^n$

$$\bar{y}^n = \frac{1}{n}\big((n-1)\bar{y}^{n-1} + y^n\big). \tag{10.119}$$

To see this, set $\tau_n = 1/n$ for $n = 1, 2, \ldots$, choose $\mu^1 = 0$ and apply (10.118) to obtain $\mu^2 = y^1$,

$$\mu^3 = \mu^2 + \frac{1}{2}(y^2 - \mu^2) = \frac{1}{2}(y^1 + y^2) = \bar{y}^2$$

and in general $\mu^{n+1} = \sum_{i=1}^n y^i/n = \bar{y}^n$. Then using (10.118)

$$\bar{y}^n = \mu^{n+1} = \mu^n + \frac{1}{n}(y^n - \mu^n) = \frac{n-1}{n}\mu^n - \frac{1}{n}y^n = \frac{1}{n}\big((n-1)\bar{y}^{n-1} + y^n\big),$$

which is the desired result. Observe that it requires storing only the current estimate of the mean and the number of observations. This approach has been referred to often in this chapter.

To illustrate this approach numerically, (10.118) is applied to estimate the mean of an exponential distribution. For illustrative purposes, learning rates $\tau_n = 20/(200+n)$, $n^{-0.75}$, and n^{-1} are compared. Simulations use a common random number seed and 5,000 iterates starting with $y^1 = 1$.

Figure (10.34) shows that the estimate using $\tau_n = n^{-1}$ (corresponding to the sample mean) is least variable and best estimates the true mean. Moreover its RMSE was 1.62 compared to 34.14 for the ratio learning rate and 10.10 when the learning rate was $n^{-0.75}$. Note that this conclusion appears to be valid even when observations are auto-correlated.

The following example illustrates the vector version of (10.117).

Example 10.15. Online regression

This example shows how to obtain estimates of regression parameters using (10.117). Consider a quadratic regression model of the form

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon.$$

To write it in vector form, define

$$\boldsymbol{\beta} = \begin{bmatrix}\beta_0\\ \beta_1\\ \beta_2\end{bmatrix} \text{ and } \mathbf{x} = \begin{bmatrix}1\\ x\\ x^2\end{bmatrix}$$

so that (10.117) becomes

$$\begin{bmatrix}\beta_0'\\ \beta_1'\\ \beta_2'\end{bmatrix} \leftarrow \begin{bmatrix}\beta_0\\ \beta_1\\ \beta_2\end{bmatrix} + \tau_n(y^n - \boldsymbol{\beta}^{\mathsf{T}}\mathbf{x}^n)\begin{bmatrix}1\\ x^n\\ (x^n)^2\end{bmatrix}. \tag{10.120}$$

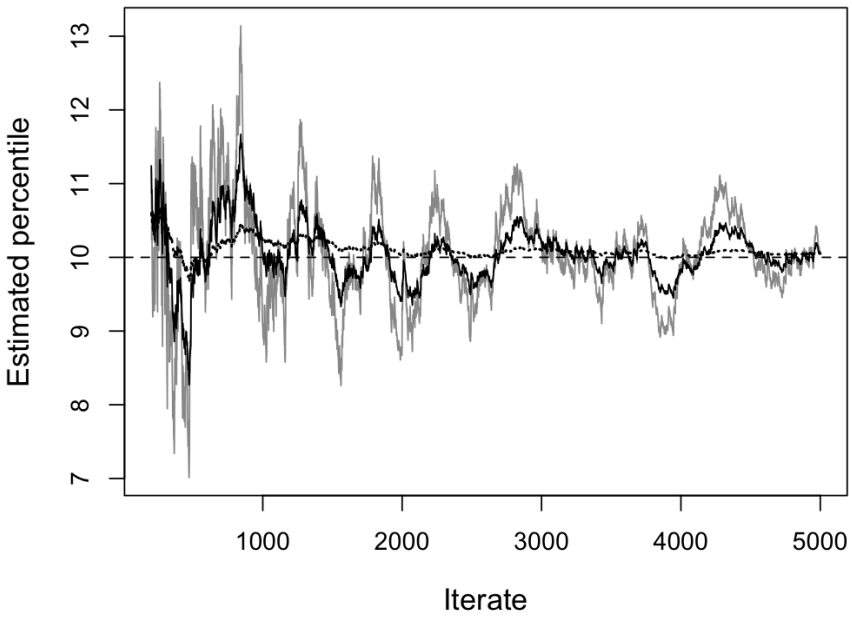


Figure 10.34: Results from a single replicate showing stochastic approximation estimates using (10.118) based on observations generated by an exponential distribution with mean 10. The gray line corresponds to $\tau_n = 20/(200+n)$, the solid black line to $\tau_n = n^{-0.75}$ and the dashed black line to $\tau_n = n^{-1}$.

Observe that each component of the vector $\mathbf{x}^n$ is multiplied by the same scalar quantity $\tau_n(y^n - \boldsymbol{\beta}^\mathsf{T}\mathbf{x}^n)$. Note also that is common practice to scale $\mathbf{x}^n$ (and sometimes y^n) to have mean zero and standard deviation one. This can be done by keeping online estimates of the mean and standard deviations of $\mathbf{x}^n$ and y^n.

As a numerical example suppose

$$y = -2 + 4x + 2x^2 + \epsilon$$

with ϵ sampled from a uniform distribution on $[-2, 2]$. Applying (10.120) with $\tau_n = 10/(80+n)$, each component of $\boldsymbol{\beta}$ initialized to zero and x randomly sampled from a normal distribution with mean 0 and standard deviation 1, the estimates from a typical replicate appear in Figure 10.35. They show convergence to true values. Note that when x was more variable, the estimates often diverged.

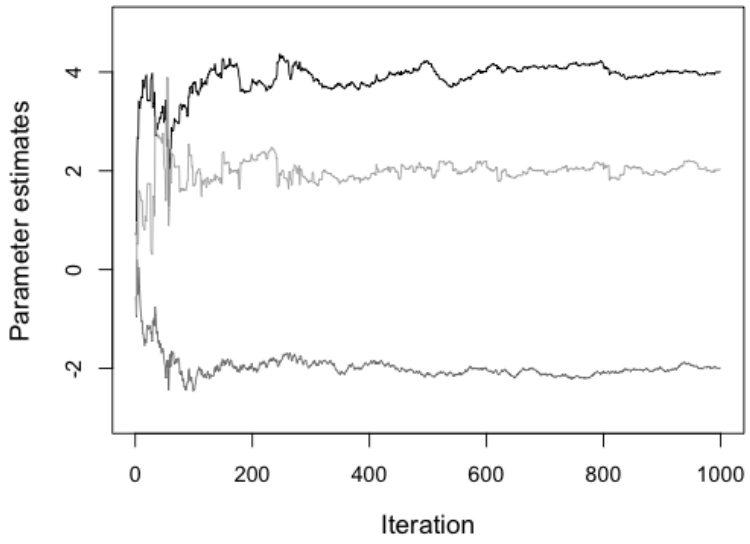


Figure 10.35: Parameter estimates from a single replicate in Example 10.15. The dark grey line corresponds to $\beta_0 = -2$, the black line to $\beta_1 = 4$ and the light grey line to $\beta_2 = 2$.

10.11.4 Offline TD(γ): Technical details

This technical section derives a representation for TD(γ) that is fundamental to the use of eligibility traces. The motivation is that value function estimation may be enhanced by using a weighted average of rollouts of different lengths.

The following lemma is fundamental to deriving the forward looking expression (10.34). Is proof is straightforward and involves interchanging the order of summation.

Lemma 10.1. Let $0 \leq \gamma \leq 1$ and r^k and v^k for $k = 0, 1, \ldots$ denote scalars for which the summations in the following expressions are convergent. Then

$$(1-\gamma)\sum_{m=1}^{\infty}\gamma^{m-1}\left(\sum_{k=0}^{m-1}r^k + v^m\right) = v^0 + \sum_{m=0}^{\infty}\gamma^m\left(r^m + v^{m+1} - v^m\right). \quad (10.121)$$

Proof. Reverse the order of summation in the first term on the left-hand side of (10.121) to obtain

$$(1-\gamma)\sum_{m=1}^{\infty}\left(\gamma^{m-1}\sum_{k=0}^{m-1}r^k\right) = (1-\gamma)\sum_{m=0}^{\infty}r^m\sum_{k=m}^{\infty}\gamma^k = \sum_{m=0}^{\infty}\gamma^m r^m,$$

where the last equality follows by using the standard formula for the sum of a geometric series.

Next consider the second term. Simple algebra shows

$$(1-\gamma)\sum_{m=1}^{\infty}\gamma^{m-1}v^m = \sum_{m=1}^{\infty}(\gamma^{m-1}-\gamma^m)v^m = v^1 + \sum_{m=1}^{\infty}\gamma^m(v^{m+1}-v^m)$$
$$= v^0 + \sum_{m=0}^{\infty}\gamma^m(v^{m+1}-v^m),$$

where the last equality comes from adding and subtracting v^0. Combining these expressions and rearranging terms yields (10.121). □

To apply this result to expression (10.33) expand it as

$$v(s^n) = (1-\gamma)E^{d^\infty}\left[\sum_{m=1}^{\infty}\gamma^{m-1}\left(\sum_{k=0}^{m-1}r(X_{n+k},Y_{n+k},X_{n+k+1}) + v(X_{n+m})\right)\middle| X_n = s^n\right],$$

set

$$r^k = E^{d^\infty}\left[r(X_{n+k},Y_{n+k},X_{n+k+1})\middle| X_n = s^n\right]$$

and

$$v^m = E^{d^\infty}\left[v(X_{n+m})\middle| X_n = s^n\right].$$

Applying the above lemma shows that:

$$v(s^n) = v(s^n) + E^{d^\infty}\left[\sum_{m=0}^{\infty}\gamma^m\big(r(X_{n+m},Y_{n+m},X_{n+m+1}) \right. \tag{10.122}$$
$$\left. + v(X_{n+m+1}) - v(X_{n+m})\big)\middle| X_n = s^n\right]. \tag{10.123}$$

Offline TD(γ) applies stochastic approximation to (10.122) as follows. Given an observed trajectory $(s^n, a^n, s^{n+1}, \ldots)$, stochastic approximation results in the recursion

$$v(s^n) \leftarrow v(s^n) + \tau\sum_{m=0}^{\infty}\gamma^m\Big(r(s^{n+m},a^{n+m},s^{n+m+1}) + v(s^{n+m+1}) - v(s^{n+m})\Big)$$
$$= v(s^n) + \tau\sum_{m=0}^{\infty}\gamma^m\delta_{n+m}$$

where

$$\delta_k := r(s^k,a^k,s^{k+1}) + v(s^{k+1}) - v(s^k).$$

Then changing variables for the summation index yields the closed form representation for an offline implementation:

$$v(s^n) \leftarrow v(s^n) + \tau \sum_{m=n}^{\infty} \gamma^{m-n} \delta_m \tag{10.124}$$

in which τ denotes a learning rate.

This representation provides the basis for the theoretical analysis of TD(γ) in Bertsekas and Tsitsiklis [1996], generalization to function approximation and provides motivation for an online implementation. To make the argument above rigorous in an episodic model requires that the decision rule be transient[78], that is, it generates a process that terminates in a finite time with probability one.

A similar representation is also valid in discounted models with discount factor λ in which case (10.124) becomes

$$v(s^n) \leftarrow v(s^n) + \tau \sum_{m=n}^{\infty} (\lambda\gamma)^{m-n} \delta_m. \tag{10.125}$$

Derivation of this expression is left as an exercise.

[78] Or proper in a stochastic shortest path problem.

Bibliographic Remarks

This chapter aimed to provide a user-friendly and practical introduction to simulation-based methods applicable to both model-free and model-based environments, with a focus on tabular models. It is not intended to be comprehensive or fully up to date, as this area of research and application is rapidly advancing.

Gosavi [2015] provides an accessible and informative overview of simulation methods for Markov decision processes, well-suited to readers of this book. His insights and comments significantly improved both the results and exposition in this chapter, particularly regarding the use of importance ratios, stochastic trace correction (STC), log-based learning rates, and choices for ϵ_n in ϵ-greedy exploration. In addition, our discussions with him regarding Relative Q-learning in average reward models enabled us to solve that version of the queuing control problem.

Noteworthy is the book Sutton and Barto [2018], which presents a reinforcement learning perspective on this material in a clear and engaging style. It served as an important source for this chapter and the next. For more rigorous and in-depth approaches grounded in Markov decision process and control theory, see Bertsekas and Tsitsiklis [1996], Bertsekas [2012], and Meyn [2022], which were our primary references for the underlying theory. In addition, there are a wealth of teaching notes and lectures available online to support further learning.

The computational results in this chapter highlight the importance of distinguishing among data generation approaches. Xiao et al. [2022] addresses this issue from a complexity-theoretic perspective, which motivates the discussion in Section 10.1.1.

Temporal-difference (TD) learning, which originated in the seminal work of Sutton (see references in Sutton and Barto [2018]), is based on applying stochastic approximation to policy evaluation, which is referred to as *prediction* in that context. Our discussion of learning rates draws on Powell [2007] and other sources.

The analysis of TD(γ) follows Sutton and Barto [2018], Tsitsiklis and Roy [1997], and especially Bertsekas and Tsitsiklis [1996], pp. 195–197. TD(γ) originated in foundational papers by Sutton cited in these works.

Solving Markov decision processes in a model-free online environment traces back to Watkins' Ph.D. thesis and the seminal paper Watkins and Dayan [1992] on Q-learning. That work includes a convergence proof for Q-learning under a discounted model when values are stored in a lookup table and all state-action pairs are sampled infinitely often. Singh et al. [2000] provides an in-depth analysis of single-step SARSA.

Average-reward temporal-difference learning and Q-learning are based on Tsitsiklis and Roy [1999], Mahadevan [1996], and pp. 548–551 in Bertsekas [2012]. Our treatment of relative value iteration follows Abounadi et al. [2001].

We believe the material on simulated policy iteration is original. It was partially inspired by Buşoniu et al. [2010], which describes a more general approach that incorporates function approximation. To our knowledge, the introduction of bounds for hybrid policy iteration is new.

The description of stochastic approximation in the appendix draws on the seminal paper by Robbins and Monro [1951] and the follow-up by Blum [1954]. More modern treatments appear in Bertsekas [2012], Powell [2007], and Meyn [2022]. The example using stochastic approximation to estimate percentiles is due to Ribes et al. [2019]. Sugiyama [1994] provides an excellent overview of stochastic approximation with several detailed examples. That work also drew our attention to Bellman's prescient remark quoted at the start of Section 10.11.2.

The next chapter extends these methods to models with function approximation and contains many additional references.

Exercises

1. The exercise compares data generation mechanisms in Section 10.1.1 in the context of the queuing service rate control model analyzed in Section 10.9.

   (a) Compare and contrast trajectory-based sampling and state-based sampling by developing code snippets for evaluating a fixed randomized stationary policy.

   (b) Compare and contrast all four sampling methods by developing code snippets for finding an optimal policy using Q-learning.

2. Repeat the Q-learning analysis of the newsvendor model for other prices, demand distributions, learning rates and choices of η_n.

3. Extend Algorithm 10.2 to the every-visit variant and apply it to evaluate the policy in the Gridworld model in Section 10.3.1. Compare its computational effort to starting-state and first-visit Monte Carlo.

4. Evaluate the policy in Example 10.1 using Monte Carlo methods and TD(0). Note the challenges encountered in obtaining accurate estimates.

5. Show that the operator defined by the right hand side of (10.17) is a contraction mapping and that the recursion converges to the same fixed point for every choice of τ.

6. Extend the study of Q-learning and SARSA in Gridworld (Example 10.8) by considering other exploration methods, varying the number of episodes in each replicate and model parameters, especially p.

7. Repeat the analysis in the example in Section 10.7.2 in which the index of the learning rate and exploration method is the number of iterations as opposed to the count of state-action pair or state, visits respectively. How do these changes impact results?

8. This example illustrates the difference between SARSA and Q-learning. Consider the snippet of a deterministic Markov decision process depicted in Figure 10.36. In state s there are two actions, a and b and in state u there are two actions $\bar{a}$

and $\bar{b}$. Choosing action a in state s yields a reward of 1 and a transition to state u. All other rewards are immaterial.

Suppose that $q(s,a) = 0,\ q(u,\bar{a}) = 5,\ q(u,\bar{b}) = 3$ and exploration resulted in choosing action $\bar{b}$ in state u. Show that the Q-learning and SARSA updates of $q(s,a)$ differ.

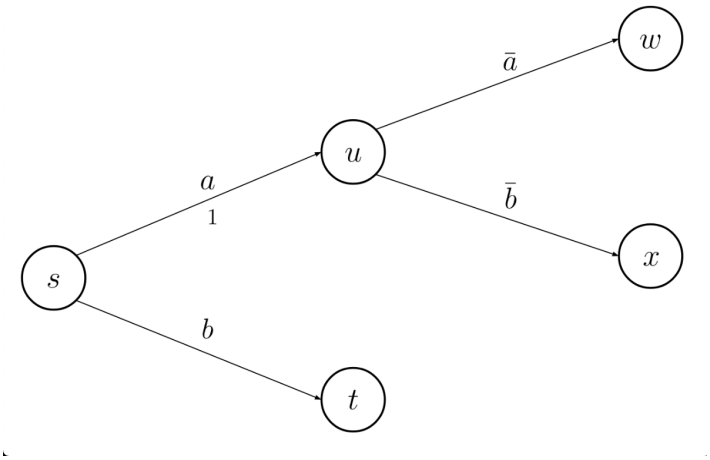


Figure 10.36: Graphical representation of model in Exercise 8.

9. Consider the deterministic version of Gridworld as in Example 10.7 except assume that the robot can move "up", "down", "right" and "left". Apply Q-learning and SARSA to find good policies using softmax and ϵ-greedy exploration. Compare these methods on the basis of the mean and standard deviation of the total reward per replicate.

10. Develop simulation-based policy iteration algorithms for an episodic model and apply them to the Gridworld model. Summarize your conclusions verbally and graphically.

11. Repeat the analysis in Example 10.8 with a small fraction of episodes starting in cell 4 in which the state-action frequencies were inaccurately estimated when beginning all episodes in cell 13. How do your results differ from those in the example? Also explore the use of softmax exploration instead of ϵ-greedy exploration.

12. **Finite Horizon Q-learning:** Develop a Q-learning algorithm for a **finite horizon** model. Apply it to solve the revenue management problem analyzed in Section 4.3.2.

13. **Cliff walking**[79]**.** In this problem on a 4×12 grid, graphically represented in Figure 10.37, a robot must traverse the grid by moving from the origin "O" in

[79] This colorful example was proposed by Sutton and Barto [2018]. It has a similar structure to our Gridworld example of a coffee delivering robot.

cell (1,1) in the lower left hand corner of the grid to the destination "D" in cell (1,12) in the lower right hand corner of the grid without falling off the cliff in cells (1,2) to (1,11). The robot incurs a cost of 1 unit for each step it takes and a penalty of 100 if it falls off the cliff. If it falls off the cliff it returns to the origin and starts again.

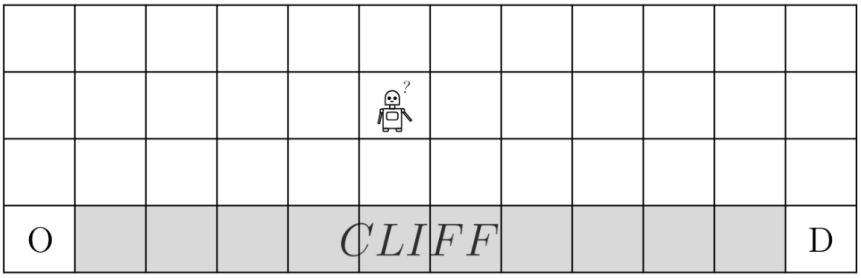


Figure 10.37: The cliff walking problem

The robot moves deterministically and can choose to move "up", "down", "left" or "right" in every cell. If it tries to move in a direction which is not possible, it incurs a cost of one unit and remains in that cell.

The goal is to learn an optimal path through this grid. Assume the robot does not know the layout and must learn it by ϵ-greedy exploration.

(a) Formulate this problem as an episodic model in which an episode starts with the robot at the origin and concludes when it reaches its destination.

(b) Find (and state) an optimal policy using Q-learning and SARSA with a learning rate $\tau = 0.1$, ϵ-greedy exploration parameter $\epsilon = 0.1$ and a discount rate of 1.

(c) Accumulate the total reward per episode for Q-learning and SARSA for the first 500 episodes of each. Compare them graphically.

(d) Investigate the impact of τ and ϵ on algorithmic performance.

14. Show that using TD(γ) for policy evaluation is equivalent to temporal differencing when $\gamma = 0$ and Monte Carlo estimation when $\gamma = 1$. Note in the latter case some care has to go into choosing the trace, specifying τ_n, and distinguishing the various forms of Monte Carlo estimation.

15. Analyze the model in Example 10.9 using the following two variants of hybrid policy iteration:

(a) Estimate $r(s,a,j)$ and $p(j|s,a)$ assuming a model-free environment and then use them to estimate $q^{d^\infty}(s,a)$ as in the example.

(b) Estimate $\sum_{j\in S} p(j|s,a)\big(r(s,a,j) + \lambda\hat{v}(j)\big)$ using Monte Carlo simulation after estimating $\hat{v}(\cdot)$.

Apply the bounds in (10.89) and stopping criterion implicit in (10.90) and summarize your observations.

16. Show either theoretically or using simulation that in the queuing control model, the steady state probabilities when the service rate and arrival rates are identical (for example, each equal to 0.2) is a symmetric random walk on the state space and that the limiting probabilities are equal for each state. Find the limiting probabilities when the service rate exceeds the arrival rate for example when action $a_2 = 0.4$ or 0.6.

17. In the context of the queuing control model in Section 10.9, investigate the convergence of a variant of Algorithm 10.14 that smooths the policy value function prior to exact derivation using (10.86). Compare the effects of smoothing both the policy value function and the q-function to smoothing only the policy value function.

18. **A large queuing service rate control model.** Use Q-learning and the policy iteration algorithms of Section 10.8 to analyze the larger version of the discounted queuing service rate control model previously solved with (deterministic) policy iteration in Section 5.11.4. In it the queue capacity is 5,000, $A_s = \{0.2, 0.3, \ldots, 0.7\}$, $b = 0.2$, $m(a_k) = 2k^3$, $f(s) = s^2$ and the discount rate equals 0.4. A low discount rate was chosen to obtain an interesting optimal policy.

    Use Relative Q-learning to solve an average reward version of this problem instance.

19. **Inventory control with hidden Markov demand.** Consider the following inventory model. At the end of each day, the inventory manager can order j units from a supplier that arrive the next morning at a cost of c per item plus a fixed ordering cost of k per order. When the demand d on any day exceeds the number of units on hand, the excess demand is unfilled and lost. If demand is less than or equal the number of units received, the system receives a revenue r for each unit sold. Unsold inventory is carried over to the next day at a cost of h per unit.

    Suppose that daily demand follows a binomial distribution with parameters D and p and moreover p takes on the values p_L and p_H, which vary from day to day according to a Markov chain with transition probability matrix

$$P = \begin{bmatrix} 0.2 & 0.8 \\ 0.6 & 0.4 \end{bmatrix},$$

where row 1 corresponds to p_L. Suppose that $c = 10, K = 5, r = 20, h = 3, p_H = 0.5, p_L = 0.2$ and $D = 20$.

(a) Develop a simulation model of this system and use it to evaluate the policy that orders 5 units every day.

(b) Find a good policy for the infinite horizon discounted model with $\lambda = 0.95$.

20. Consider the model in Exercise 1 of Chapter 2. In performing the following calculations investigate the effect of learning rate sequence $\tau_n, n = 1, 2, , \ldots$ and sample size on the accuracy of policy evaluation. Compare results on the basis of RMSE and visual inspection of the policy value functions.

    In addition, when optimizing, investigate the impact of exploration method. Summarize results in terms of the similarity of the policies to the optimum, the true values of the policies, and appropriate RMSE measures.

    (a) Use temporal differencing, TD(γ), and Monte Carlo with truncated discounted rewards and geometric sampling to estimate the infinite horizon discounted reward with $\lambda = 0.9$ for the following policies:

        i. The deterministic stationary policy that uses $d(s_i) = a_{i,1}$ in state s_i for $i = 1, 2, 3$.

        ii. The randomized stationary policy that uses action $a_{i,1}$ with probability $e^{-0.5i}$ and action $a_{i,2}$ with probability $1 - e^{-0.5i}$ in state s_i for $i = 1, 2, 3$.

    (b) Evaluate the average reward and bias of these policies using Monte Carlo methods and temporal differencing.

    (c) Develop and apply a TD(γ) algorithm for an average reward model.

    (d) Use Q-learning, SARSA, hybrid policy iteration and online policy iteration to find an optimal policy for a discounted version of the model. Consider both $\lambda = 0.9$ and $\lambda = 0.999$. Comment on the effect of λ on the execution of the algorithm.

    (e) Use Q-learning and Relative Q-learning to find an optimal policy for the average reward version of this model.

21. Consider the admission control queuing model model of Section 3.4.2 with reward $R = 20$, holding cost $f(j) = 1.05j$, service probability $w = 0.45$, and arrival probability $b = 0.5$. Truncate the state space at $M = 100$.

    (a) Write down recursions for the state-action value function assuming:

        i. The holding cost is incurred before an arrival or service completion.

        ii. The holding cost is incurred after an arrival or service completion.

    (b) Find an optimal policy for a discounted version of this model with $\lambda = 0.95$ and investigate its sensitivity to the problem data.

(c) Use Q-learning, hybrid policy iteration and online policy iteration to find a good policy for this model using simulation.

(d) Investigate the impact of truncation on the optimal policy and its gain by varying the truncation level M.

(e) Use Q-learning and Relative Q-learning to find good policies for an average reward variant of this model.

22. Develop simulation-based policy iteration algorithms for an average reward model and use it to solve the queuing service rate control model. Compare results to those obtained using Relative Q-learning in this chapter.

23. Develop a Q-learning algorithm for a finite horizon model and apply it to the revenue management problem in Section 3.3. Note that this requires defining state-action value functions that vary with decision epoch.

    Based on these observations, what can you conclude about the challenges of using trajectory-based sampling for value function estimation and optimization?

24. Use the recursion (10.103) to find a root of the function $f(x) = 1/(1+e^{-0.5x})-0.5$ based on observing $f(x) + \epsilon$ where ϵ is normally distributed with mean 0 and standard deviation 0.1. Investigate the variability and accuracy of estimates as you vary the learning rate τ_n and the number of observations.

25. Derive (10.125) by modifying the argument at the start of Section 10.11.4 to include a discount factor.

26. **Queuing with a budget constraint.** Formulate and analyze the queuing service rate control model in which an episode ends when a finite budget has been expended and the objective is to maximize *throughput*, that is the number of customers served. Assuming the same parameter values as in Section 10.9, investigate the impact of the budget constraint level on your results.

Chapter 11

Simulation with Function Approximation

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

> *In fact, everything we know is only some kind of approximation, because we know that we do not know all the laws as yet. Therefore, things must be learned only to be unlearned again or, more likely, to be corrected*[1].
>
> Richard P. Feynman, American theoretical physicist, 1918-1988.

As Feynman so eloquently puts it, every model is an approximation that must be revised as new information becomes available. This is exactly the principle underlying this chapter that combines function approximation with simulation.

Modern applications of Markov decision processes require both function approximation (Chapter 9) and simulation[2] (Chapter 10). Such methods are usually referred to as *reinforcement learning* (RL), but as noted at the beginning of Part III, RL refers to any setting in which the decision maker (or agent) learns by trial and error.

This chapter motivates, describes, and applies methods for:

- value function approximation,
- state-action value function approximation, and
- policy approximation.

[1] Feynman et al. (1963).

[2] As noted earlier the expression *simulation* refers to either computer-based simulation or real-time experimentation.

Whereas value function and state-action value function approximation generalize the methods introduced in Chapters 9 and 10, this chapter describes approaches that approximate the action-choice probability distribution of a randomized policy directly, that is, without using value functions as intermediaries.

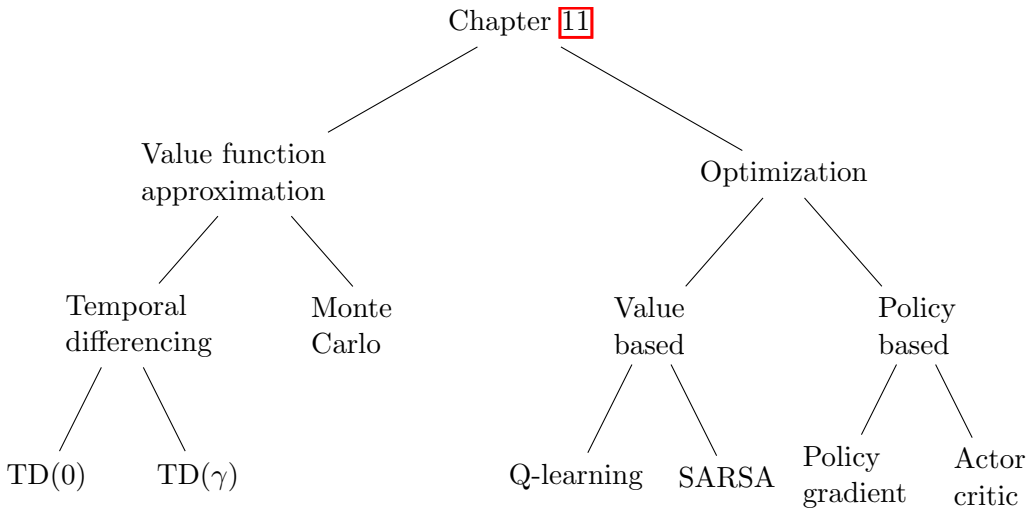


Figure 11.1: Schematic representation of methods described in Chapter 11

The methods in this chapter represent value functions, state-action value functions, and randomized policies by parametric functions of features and then estimate the parameters online or offline using simulated data. Parameters are often referred to as *weights*.

Figure 11.1 summarizes the methods described in this chapter. Note that this chapter considers only discounted and episodic models. Generalization to average reward models is left to the reader. Technical prerequisites include familiarity with gradient descent and the matrix formulation of least squares regression (see Chapter 9 Appendix).

11.1 The challenge of large models

In contrast to the tabular models analyzed in Chapter 10, models with large state (and action) spaces require methods that represent value functions and decision rules in forms suitable for computation and application. The following sections discuss methods for doing so.

11.1.1 Features

The underlying approach is to summarize the set of states or state-action pairs by lower-dimensional sets of real-valued functions defined on the set of states or state-action pairs. Such functions are referred to as *features* or *basis functions*.

Specifying features

Choosing suitable features presents challenges. In physical models, such as navigating a drone through three-dimensional space, features may represent the position, velocity and angular orientation. In discrete models, with numerical-valued states and actions such as controlling a multi-dimensional queuing system, features may be (scaled) powers and products of powers of the number of entities in each queue.

In Gridworld models, specifying features may be more challenging. For example, in a robot guidance problem on an $M \times N$ grid, a possible choice of features is the row index, the column index and some higher order and cross-product terms of these quantities. Another possibility is to aggregate cells by choosing features to be indicator functions of overlapping or disjoint subsets of cells.

Note that:

> Specifying features to be indicator functions of individual states or state-action pairs is equivalent to using a *tabular model*.

Consequently, the results and methods in Chapter 10 are special cases of those in this chapter.

Combining features

Once features are specified, the issue of how to combine them to obtain estimates of quantities of interest arises. In general, value functions, state-action value functions and action-choice probabilities can be approximated by:

1. linear functions of features
2. nonlinear functions of features
3. neural networks[3].

Regarding notation, vectors of features evaluated at s or (s, a) are denoted by $\mathbf{b}(s)$ or $\mathbf{b}(s, a)$, respectively. Components of these vectors are written as $b_i(s)$ or $b_{i,j}(s, a)$. In both cases, vectors of weights are denoted by $\boldsymbol{\beta}$, with components β_i and $\beta_{i,j}$, respectively.

11.1.2 Policy value function approximation

This section briefly reviews value function approximation[4] for a fixed policy, which was introduced in Chapter 9. For ease of exposition and to encompass many practical

[3] Of course, neural networks are nonlinear functions but they are distinguished here because they have a vast and specialized array of methods for parameter estimation.

[4] The problem of estimating a value function for a specified policy is referred to as *prediction* in the computer science literature.

applications, assume a linear value function approximation of the form

$$v(s;\beta_0,\ldots,\beta_I) := \beta_0 b_0(s) + \beta_1 b_1(s) + \ldots + \beta_I b_I(s), \tag{11.1}$$

where $\beta_0,\ldots,\beta_I$ denote weights (parameters) and $b_0(s),\ldots,b_I(s)$ denote the value of the features evaluated at $s \in S$. Note that indexing begins at 0 to conform with the convention in linear regression models in which $b_0(s) = 1$ for all $s \in S$ so that the first term in the sum corresponds to a constant β_0. The above approximation is linear in the weights, however, features may be nonlinear functions (for example powers) of the state.

In vector notation (which is preferable) (11.1) can be written succinctly as:

$$v(s;\boldsymbol{\beta}) = \boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s), \tag{11.2}$$

where $\mathbf{b}(s)$ denotes a (column) vector of pre-specified features evaluated at $s \in S$ and $\boldsymbol{\beta}$ denotes a weight vector.

The beauty of such a representation is that in (11.1) the coefficients can be interpreted as *marginal* rewards or costs. For example, if s denotes the number of entities in a single-server queuing system and $v(s;\boldsymbol{\beta}) = \beta_0 + \beta_1 s$ represents an approximation to the expected infinite horizon discounted cost, then β_1 represents the increase in expected discounted cost that results from adding one customer to the system.

In greater generality, $v(s;\boldsymbol{\beta})$ may be a nonlinear function such as a neural network. As a convention, estimates of $\boldsymbol{\beta}$ will be denoted by $\hat{\boldsymbol{\beta}}$ and estimates of value functions by either $\hat{v}(s)$ or $v(s;\hat{\boldsymbol{\beta}})$.

11.1.3 State-action value function approximation

Approximations of state-action value functions require features expressed in terms of states and actions. Determination of suitable functional forms for the components of the vector $\mathbf{b}(s,a)$ can present some challenges.

One can set

$$b_{i,j}(s,a) = f_i(s)h_j(a), \tag{11.3}$$

where $f_i(s)$ for $i = 1,\ldots,I$ is a function defined on states and $h_j(a)$ for $j = 1,\ldots,J$ is a function defined on actions. In the linear case, the approximation can be represented by

$$q(s,a;\boldsymbol{\beta}) = \sum_{i=1}^{I}\sum_{j=1}^{J}\beta_{i,j}b_{i,j}(s,a) = \sum_{i=1}^{I}\sum_{j=1}^{J}\beta_{i,j}f_i(s)h_j(a), \tag{11.4}$$

where $\boldsymbol{\beta}$ is the "long" vector $(\beta_{1,1}, \ldots, \beta_{I,J})$. In greater generality, $q(s, a; \boldsymbol{\beta})$ may be a pre-specified nonlinear function of features.

When A_s contains a small number of elements and does not vary with the state, such as in the queuing service rate control model, one may choose $h_j(a)$ to be the indicator function of the action a_j, that is

$$h_j(a) = I_{\{a_j\}}(a) = \begin{cases} 1 & a = a_j \\ 0 & a \neq a_j. \end{cases} \tag{11.5}$$

In this case, $J = |A_s|$. Consequently,

$$b_{i,j}(s, a) = \begin{cases} f_i(s) & a = a_j \\ 0 & a \neq a_j \end{cases} \tag{11.6}$$

for $a \in A_s$ and $s \in S$. This is equivalent to representing $q(s, a)$ as a different linear combination of the functions $f_i(s)$ for each $a \in A_s$.

To make this concrete, consider a queuing control model (such as in Section 10.9) with service rates a_1 and a_2, and suppose $f_0(s) = 1$[5], $f_1(s) = s$ and $h_j(a) = I_{\{a_j\}}(a)$ as in (11.5). Then using the product-form representation in (11.4), the approximation of $q(s, a)$ can be written as

$$\begin{aligned} q(s, a; \boldsymbol{\beta}) &= \beta_{0,1} b_0(s, a_1) + \beta_{0,2} b_0(s, a_2) + \beta_{1,1} b_1(s, a_1) + \beta_{1,2} b_1(s, a_2) \\ &= \beta_{0,1} f_0(s) h_1(a) + \beta_{0,2} f_0(s) h_2(a) + \beta_{1,1} f_1(s) h_1(a) + \beta_{1,2} f_1(s) h_2(a) \\ &= \beta_{0,1} I_{\{a_1\}}(a) + \beta_{0,2} I_{\{a_2\}}(a) + \beta_{1,1} s I_{\{a_1\}}(a) + \beta_{1,2} s I_{\{a_2\}}(a). \end{aligned}$$

This is equivalent to using the following, possibly different, linear functions for each action:

$$\begin{aligned} q(s, a_1; \boldsymbol{\beta}) &= \beta_{0,1} + \beta_{1,1} s \\ q(s, a_2; \boldsymbol{\beta}) &= \beta_{0,2} + \beta_{1,2} s. \end{aligned}$$

Hence, this representation corresponds to approximating $q(s, a)$ by lines with different slopes and intercepts for each action.

Vector representation

It is convenient to write (11.4) in vector form as

$$q(s, a; \boldsymbol{\beta}) = \boldsymbol{\beta}^{\mathsf{T}} \mathbf{b}(s, a), \tag{11.7}$$

[5] Recall in Chapter 9 that the index started at 0 in the linear case so as to include an explicit constant term in the function approximation. This example and several others in this chapter adopt this convention.

where $\boldsymbol{\beta}$ is a column vector of weights of length IJ and $\mathbf{b}(s,a)$ is a column vector of features evaluated at (s,a) of length IJ. Of course, the weights and features must be ordered consistently. For example, if $|A_s| = J$ for each s, then $\boldsymbol{\beta}$ can be written as

$$\boldsymbol{\beta} = \begin{bmatrix} \boldsymbol{\beta}_{a_1} \\ \vdots \\ \boldsymbol{\beta}_{a_J} \end{bmatrix}, \tag{11.8}$$

where the column vector $\boldsymbol{\beta}_{a_j}$ contains the coefficients corresponding to action a_j for each state s. Following this approach, the approximation in the above queuing example can be represented using

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_{0,1} \\ \beta_{1,1} \\ \beta_{0,2} \\ \beta_{1,2} \end{bmatrix} = \begin{bmatrix} \boldsymbol{\beta}_{a_1} \\ \boldsymbol{\beta}_{a_2} \end{bmatrix}, \quad \mathbf{b}(s,a_1) = \begin{bmatrix} 1 \\ s \\ 0 \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{b}(s,a_2) = \begin{bmatrix} 0 \\ 0 \\ 1 \\ s \end{bmatrix}. \tag{11.9}$$

Another possible approximation for $q(s,a)$ is to write it as a linear combination of the product of state space features and action space features of the form $f(s)h(a)$. For example, suppose $f(s) = 1 + s$ and $g(a) = 1 + a$. Then

$$f(s)h(a) = 1 + s + a + as.$$

In this case

$$q(s,a;\boldsymbol{\beta}) = \beta_{0,0} + \beta_{1,0}s + \beta_{0,1}a + \beta_{1,1}as.$$

where $\boldsymbol{\beta} = (\beta_{0,0}, \beta_{1,0}, \beta_{0,1}, \beta_{1,1})$.

Without the interaction term, $\beta_{1,1}as$, greedy action choice based on this approximation to $q(s,a)$ would depend only on the value of $\beta_{0,1}$, independent of s. When it is present, it would depend on the state through $\beta_{0,1} + \beta_{1,1}s$.

When the states and actions are vectors $\mathbf{s}$ and $\mathbf{a}$ respectively, it may be preferable to approximate $q(\mathbf{s},\mathbf{a};\boldsymbol{\beta})$ by a neural network with inputs $\mathbf{s}$ and $\mathbf{a}$. As pointed out above, if all basis functions are indicator functions of state-action pairs, the approximation is equivalent to the tabular representation studied in Chapter 10.

Recovering value functions

Recall that given a state-action value function, the value function of the stationary policy d^∞ can be represented by:

$$v_d(s) = \begin{cases} q(s,d(s)) & \text{if } d \text{ is deterministic} \\ \sum_{a \in A_s} w_d(a|s) q(s,a) & \text{if } d \text{ is randomized.} \end{cases}$$

Combining this observation with (11.7) results in using approximations

$$\hat{v}_d(s) = \begin{cases} \boldsymbol{\beta}^\mathsf{T} \mathbf{b}(s,d(s)) & \text{if } d \text{ is deterministic} \\ \sum_{a \in A_s} w_d(a|s) \boldsymbol{\beta}^\mathsf{T} \mathbf{b}(s,a) & \text{if } d \text{ is randomized.} \end{cases}$$

Decision rules

When analyzing tabular models, it was possible to state decision rules **explicitly**. Deterministic decision rules could be specified in a look-up table and randomized decision rules could be specified as a distribution over the set of actions. However in models with large state spaces this will not be possible, or even necessary.

When using function approximation, the quantity $\boldsymbol{\beta}$ corresponds to a specific decision rule. That is, instead of explicitly encoding the policy, one can use $\boldsymbol{\beta}$ to generate actions as follows.

Note that algorithms in this chapter assume that basis functions or features are pre-specified and fixed.

Algorithm 11.1. Implicit action choice

1. Specify $\boldsymbol{\beta}_0$.
2. Specify a state $s \in S$.
3. Compute $q(s, a; \boldsymbol{\beta}_0) \leftarrow \boldsymbol{\beta}_0^{\intercal} \mathbf{b}(s, a)$ for all $a \in A_s$.
4. (a) **Deterministic action selection:** Set $a_s \in \arg\max_{a \in A_s} q(s, a; \boldsymbol{\beta}_0)$.
   (b) **Randomized action selection** Sample a_s using ϵ-greedy or softmax sampling based on $q(s, a; \boldsymbol{\beta}_0)$.
5. Return a_s.

To think of this another way, instead of carrying around a look-up table, our agent will store a (much lighter) vector $\boldsymbol{\beta}_0$. When in state s, the agent can then apply Algorithm 11.1 to choose an action a_s to apply. In applications, one would prefer a greedy action; randomized action selection is appropriate for exploration or using randomized policies generated by policy approximation methods.

From this perspective, it would be difficult to determine which $\boldsymbol{\beta}$ corresponds to a specific policy. In the special case when the model is known and rewards are independent of the subsequent state, choosing $\boldsymbol{\beta}_0 = \mathbf{0}$ would result in setting $q(s, a; \boldsymbol{\beta}_0) = r(s, a)$ so that greedy action selection would generate a myopic policy.

11.1.4 Policy approximation

A third approach focuses on parameterizing the policy directly. It represents $w(a|s) = P(Y_n = a|X_n = s)$ as a parametric function of features defined in terms of states and actions. Of course the functional form must ensure that $w(a|s) \geq 0$ for all $a \in A_s$ and $\sum_{a \in A_s} w(a|s) = 1$. Features are represented by the vector $\mathbf{b}(s, a)$ and weights

by the vector $\boldsymbol{\beta}$. A convenient representation, referred to as a softmax[6] or logistic transformation, is given by

$$w(a|s;\boldsymbol{\beta}) = \frac{e^{\boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s,a)}}{\sum_{a' \in A_s} e^{\boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s,a')}}. \tag{11.10}$$

An alternative to using (11.10) is to represent $w(a|s;\boldsymbol{\beta})$ by a *neural network* where $\boldsymbol{\beta}$ is a vector of weights.

11.2 Policy value function approximation

This section focuses on simulation methods for approximating a policy value function. It describes Monte Carlo and temporal differencing methods.

11.2.1 Monte Carlo policy value function approximation

The idea is quite simple. Simulate (or observe) the process under a fixed policy for a set of starting states, store the observed values and then estimate the policy value function using least squares. For ease of exposition, this section only provides a starting-state version of the algorithm; the first-visit variant is a simple modification while the every-visit version takes some care to describe.

The following discussion takes the perspective that policy value functions are to be approximated without direct analyst input — such is the case when they are intermediaries in a method to find optimal policies. Consequently, the set of starting states and features are not selected to align with any particular policy.

Monte Carlo estimation of policy value function approximations in an episodic model

Recall that Δ denotes the set of stopped states and $g(s)$ denotes the value on termination in state s. The algorithm assumes a randomized stationary policy and pre-specified features.

Algorithm 11.2. Starting-state Monte Carlo policy value function approximation for an episodic model

1. **Initialize:**

   (a) Specify $d \in D^{\mathrm{MR}}$ and the number of episodes K.

[6] Previously, the softmax function was used for exploration. Here it is used to represent a policy directly. Note the constant η previously used to parameterize the function is accounted for by the scale of $\boldsymbol{\beta}$.

(b) Specify a subset of starting states $\bar{S} \subseteq S \setminus \Delta$.

(c) For all $s \in \bar{S}$, create an empty list VALUES(s).

2. **Generate values:** For all $\bar{s} \in \bar{S}$:

   (a) $k \leftarrow 1$.

   (b) While $k \leq K$:

      i. $s \leftarrow \bar{s}$ and $v \leftarrow 0$.

      ii. **Generate episode:** While $s \notin \Delta$:

         A. Sample a from $w_d(\cdot|s)$.

         B. Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.

         C. Update the value:

$$v \leftarrow r + v.$$

         D. $s \leftarrow s'$.

      iii. **Terminate episode ($s \in \Delta$):**

         A. Append $v + g(s)$ to VALUES($\bar{s}$).

         B. $k \leftarrow k + 1$.

3. **Estimate parameters:** Use data $\{(s, v) \,|\, s \in \bar{S}, v \in \text{VALUES}(s)\}$ to obtain estimates of $\hat{\boldsymbol{\beta}} = (\hat{\beta}_0, \ldots, \hat{\beta}_I)$ using least squares or weighted least squares.

4. **Terminate:** Return $\hat{\boldsymbol{\beta}}$.

Some comments follow:

1. Instead of specifying a set of states at which to evaluate $v(s)$, states can be chosen randomly. Specifying states judiciously may result in more accurate parameter estimates.

2. The above algorithm generates $|\bar{S}|K$ data points of the form $(s, v(s))$. When $v(s)$ is linear in the weights or parameters, least squares estimates are available in closed form. When a nonlinear approximation is used, iterative estimation methods are required. Since these methods will be coded, both approaches can easily be included in optimization algorithms.

3. The above algorithm records values for the starting state only. As noted in Chapter 10 it is easy and more efficient to modify the algorithm to store estimates for all visited states.

4. When the variability of the Monte Carlo estimates depends on the state, weighted least squares provides a more appropriate approach for parameter estimation.

5. The key difference between this algorithm and Algorithm 10.1 (starting-state Monte Carlo without value function approximation) is the addition of the concluding step, which estimates the parameters $\hat{\boldsymbol{\beta}}$ from pairs of states-value function estimates. Note also that estimation requires values associated with multiple states (those in $\bar{S}$), whereas Algorithm 10.1 provides an estimate of the value for a single state.

6. Note that an alternative is to take the mean of all values for each s, and then regress on the $(s, \text{mean}(\textsc{Values}(s)))$ pairs, where there is now a single value for each s. However, this approach would lose information on the variability of different v estimates for the same s.

An example

The following model of optimal stopping in a random walk illustrates the application of the above algorithm.

Example 11.1. Monte Carlo policy value function estimation for a random walk with stopping.

Consider the random walk model described in Example 6.19 but with $S = \{1, \ldots, 500\}$, continuation cost $c = 8$, stopping reward $g(s) = 0.3s + 0.7s^2$ and $p = 0.51$, where p denotes the probability of a transition from s to $s + 1$ except in state 500 where it denotes the probability of remaining in state 500.

The decision maker trades off the cost of continuing versus the benefit of reaching the high reward states. There are two regions where the decision maker might consider stopping: in very low states when the cost of reaching the high reward states might exceed the benefit of waiting, or in very high states.

This example investigates the use of a linear function of polynomials of the form

$$v(s; \beta_0, \beta_1, \ldots, \beta_I) = \beta_0 + \beta_1 s + \beta_2 s^2 + \ldots + \beta_I s^I$$

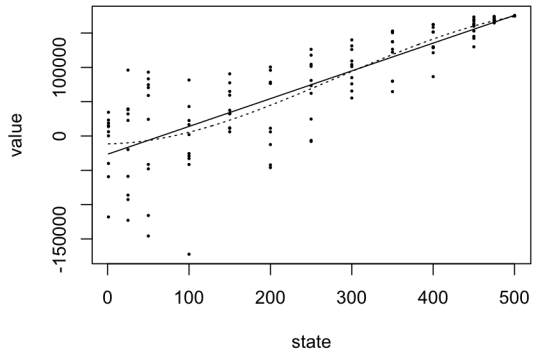
for small values of I and considers three policies:

π_1: Stop in all states, that is $\Delta = S$.

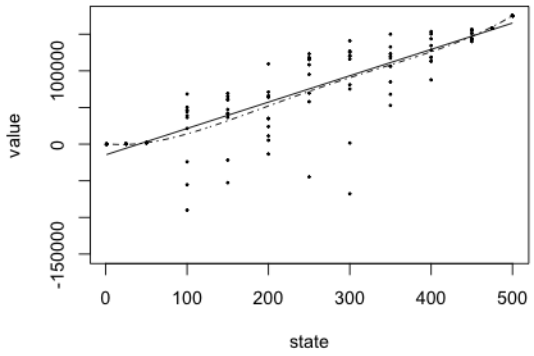
π_2: Stop only when in state 500, that is $\Delta = \{500\}$.

π_3: Stop in states $\{1, \ldots, 50\}$ and $\{475, \ldots, 500\}$, that is $\Delta = \{1, \ldots, 50\} \cup \{475, \ldots, 500\}$.

Under π_1, no approximation is necessary and $v^{\pi_1}(s) = g(s)$. To apply Algorithm 11.2 to π_2 and π_3, choose $\bar{S} = \{1, 25, 50, 100, 150, \ldots, 400, 450, 475, 500\}$ and $K = 10$ replications for each $s \in \bar{S}$[a].



(a) Linear (solid line) and cubic (dashed line) value function approximations for policy π_2.



(b) Linear (solid line) and fourth-order (dashed line) value function approximations for policy π_3.

Figure 11.2: Monte Carlo value function estimation for two policies in Example 11.1.

Figure 11.2a) shows linear ($I = 1$, solid line) and cubic ($I = 3$, dashed line) approximations to the Monte Carlo values for policy π_2. The linear approximation was

$$v^{\pi_2}(s; \hat{\beta}_0, \hat{\beta}_1) = -26{,}549.1 + 404.7s.$$

and the cubic approximation was

$$v^{\pi_2}(s; \hat{\beta}_0, \hat{\beta}_1, \hat{\beta}_2, \hat{\beta}_3) = -11{,}459.94 + 17.90s + 1.72s^2 - 0.0021s^3.$$

The cubic fit was slightly more accurate with a smaller standard error. The fourth order fit was almost identical to that of the cubic model.

Figure 11.2b shows linear and fourth-order approximations for policy π_3. Note that a fourth-order polynomial was necessary to represent well the values in the stopped region. The estimated approximations were:

$$v^{\pi_3}(s; \hat{\beta}_0, \hat{\beta}_1) = -14594.7 + 359.6s$$

and

$$v^{\pi_3}(s, \hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_4) = 647.3 - 123.43s + 3.41s^2 - 0.009s^3 + 0.000009s^4.$$

It is left as an exercise to investigate other choices for $\bar{S}$ and parameter values.

[a] The example chooses extra values close to the endpoints to achieve greater accuracy. Note also that replications in the stopping regions were unnecessary since $v(s) = g(s)$ therein.

Discounted models using truncation and Monte Carlo estimation

The approach to approximating the policy value function for a discounted model is similar to that for an episodic model, however, it is necessary to either truncate trajectories after a fixed number of iterations or use geometric stopping times. The truncation version accumulates the discounted reward for some pre-specified number, M, of decision epochs. The geometric stopping time version is identical to that for an episodic model in which at each decision epoch the system can enter an absorbing state with probability $1 - \lambda$. A formal statement of the truncation version of the algorithm is provided here; the geometric stopping time version is left to the reader. The algorithm assumes a randomized stationary policy.

Algorithm 11.3. Starting-state Monte Carlo policy value function approximation for a discounted model with truncation

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MR}}$.
   (b) Specify the number of replicates K and the truncation level M.
   (c) Specify a subset of starting states $\bar{S} \subseteq S$.
   (d) For all $s \in \bar{S}$, create an empty list VALUES(s).
2. **Generate values:** For all $\bar{s} \in \bar{S}$:
   (a) $k \leftarrow 1$.
   (b) While $k \leq K$:
      i. $s \leftarrow \bar{s}$, $v \leftarrow 0$ and $m \leftarrow 1$.
      ii. **Generate replicate:** While $m \leq M$:
         A. Sample a from $w_d(\cdot|s)$.
         B. Simulate (s', r) or sample s' from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
         C. $v \leftarrow \lambda^{m-1} r + v$.
         D. $s \leftarrow s'$.
         E. $m \leftarrow m + 1$.
      iii. Append v to VALUES$(\bar{s})$.
      iv. $k \leftarrow k + 1$.
3. **Estimate parameters:** Use data $\{(s, v) \,|\, s \in \bar{S},\, v \in \text{VALUES}(s)\}$ to estimate parameters $\hat{\boldsymbol{\beta}} = (\hat{\beta}_0, \ldots, \hat{\beta}_I)$ using least squares or weighted least squares.
4. **Terminate:** Return $\hat{\boldsymbol{\beta}}$.

As previously discussed, a first-visit and an every-state version of this algorithm would not be applicable because truncation lengths would vary from state to state.

Example 11.2. Monte Carlo policy value function estimation for the queuing service rate control model.

This example applies Algorithm 11.3 to the queuing control model with $S = \{0, \ldots, 50\}$. It explores the quality of cubic polynomial approximations to the policy value function for the deterministic stationary policies derived from decision rules

$$d_1(s) = \begin{cases} a_1 & \text{for } s \leq 25 \\ a_3 & \text{for } s > 25 \end{cases}$$

and

$$d_2(s) = a_2 \quad \text{for } 0 \leq s \leq 50,$$

using 40 replicates of the algorithm for $\lambda = 0.9, 0.95, 0.98$ and common random numbers in each replicate. Each replicate consists of $K = 10$ episodes for each $s \in \bar{S} = \{0, 5, 10, \ldots, 45, 50\}$. Cumulative discounted rewards are truncated at $M = 600$.

Outcomes were compared on the basis of the RMSE of the fit:

$$\text{RMSE} = \left(\frac{1}{|S|} \sum_{s \in S} (v(s, \hat{\boldsymbol{\beta}}) - v_\lambda^{d^\infty}(s))^2 \right)^{\frac{1}{2}}. \tag{11.11}$$

Note that $v_\lambda^{d^\infty}(s)$ can easily be determined exactly using the policy evaluation methods in Chapter 5.

Table 11.1 summarizes results. The column "True" establishes a baseline by providing the RMSE of a cubic polynomial fit to the exact policy value function. Observe that:

1. The accuracy of a cubic fit to the true policy value function decreases with λ as does the accuracy of each of the Monte Carlo estimates.

2. The cubic model fits the policy value function of $(d_2)^\infty$ better than that of $(d_1)^\infty$. This is expected because of the step change in d_1 at $s = 26$ (see Figure 11.3a).

3. Least squares estimates of policy value functions are more accurate than weighted least squares estimates based on the estimated variance of values at each $s \in \bar{S}$.

4. Results (not shown) indicated that the accuracy of the estimates varied considerably with the truncation level N. When $N = 300$ the RMSE was more than three times greater than that shown when $\lambda = 0.98$.

Figure 11.3 compares, for a single replicate, the true policy value function, its cubic polynomial fit and a cubic polynomial approximation based on Monte Carlo estimates. Figure 11.3a shows that for decision rule d_1 and $\lambda = 0.95$, the cubic polynomial fit deviates from the true policy value function, and the cubic approximation based on Monte Carlo estimation lies below the true policy value function. Figure 11.3b shows that for decision rule d_2 and $\lambda = 0.98$, the cubic polynomial fit accurately approximates the true policy value function, but the cubic approximation based on Monte Carlo estimation deviates from the true policy value function at high occupancy states.

The impact of these discrepancies on optimization is investigated in subsequent sections of this chapter.

	d_1			d_2		
λ	True	MC-LS	MC-WLS	True	MC-LS	MC-WLS
0.90	67.99	551.14	569.17	59.15	567.90	579.10
0.95	344.73	1,029.94	1,135.56	153.05	1,098.57	1,151.50
0.98	1,261.33	2,161.07	2,801.03	432.18	2,480.18	2,770.44

Table 11.1: Mean of RMSE over 40 replicates for least squares (MC-LS) and weighted least squares (MC-WLS) fit to Monte Carlo estimates and the RMSE for a cubic regression fit to the exact policy value (True) as a function of discount rate and decision rule.

11.2.2 Temporal differencing with policy value function approximation

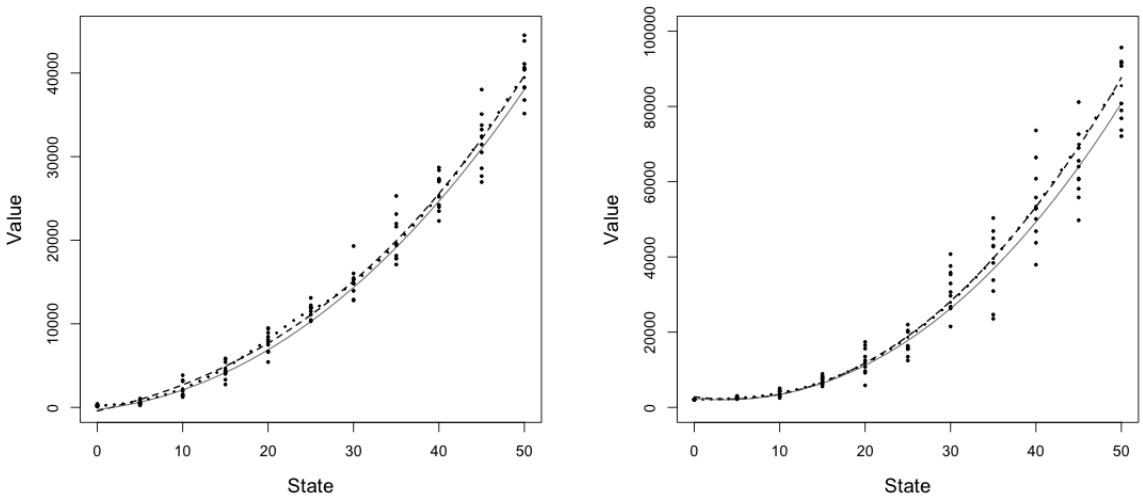
As an alternative to Monte Carlo estimation in which estimates are derived after simulating (or observing) many trajectories, this section describes an online approach that generalizes TD(0) by updating parameter estimates after each state transition[7].

Motivation

This section applies stochastic gradient descent type methods from the Appendix of Chapter 10 to obtain a recursive method for estimating a vector of weights in an approximate policy value function. As detailed below, this approach modifies gradient descent to simplify computation and improve numerical stability.

Let d denote a Markovian randomized decision rule, d^∞ the corresponding stationary policy, and s an arbitrary state in S. The objective is to find a vector of weights

[7] As noted above, when the features are indicators of states or states and actions, this is equivalent to the online methods in Chapter 10.



(a) Decision rule d_1 with $\lambda = 0.95$.

(b) Decision rule d_2 with $\lambda = 0.98$.

Figure 11.3: Plots of estimated and true policy value functions based on a single replicate of starting-state Monte Carlo estimation consisting of $K = 10$ episodes for each $s \in \bar{S}$. Points indicate episode value estimates, the dotted line equals the true policy value function, the dashed line indicates the fitted cubic polynomial approximation to the true policy value function and the gray solid line denotes the least squares estimate based on Monte Carlo estimation. Note that the y-axis scales for the two figures differ.

$\boldsymbol{\beta} \in \mathbb{R}^I$ so as to "solve" the policy evaluation (Bellman) equation

$$v(s;\boldsymbol{\beta}) = E^{d^\infty}\left[r(X,Y,X') + \lambda v(X';\boldsymbol{\beta})\middle| X = s\right]$$

defined for all $s \in S$. In this equation the expectation is with respect to the conditional distribution of the action Y and successor state X' under decision rule d given state $s \in S$. To make this more concrete, the goal is to find a $\boldsymbol{\beta}$ that minimizes the conditional expected squared error loss

$$E^{d^\infty}\left[\left(r(X,Y,X') + \lambda v(X';\boldsymbol{\beta}) - v(X;\boldsymbol{\beta})\right)^2\middle| X = s\right] \tag{11.12}$$

for all $s \in S$.

Since a different $\boldsymbol{\beta}$ may achieve the minimum above for each $s \in S$, an alternative is to choose $\boldsymbol{\beta}$ to minimize the *unconditional* squared error loss:

$$G_d(\boldsymbol{\beta}) := E^{d^\infty}\left[\left(r(X,Y,X') + \lambda v(X';\boldsymbol{\beta}) - v(X;\boldsymbol{\beta})\right)^2\right], \tag{11.13}$$

where the expectation is now with respect to state X as well. Possible choices for a distribution for X include a uniform distribution over states or the stationary distribution of the Markov chain corresponding to d^∞.

If one could evaluate the above expectation directly, the gradient descent recursion

$$\boldsymbol{\beta}' = \boldsymbol{\beta} - \tau \nabla_{\boldsymbol{\beta}} G_d(\boldsymbol{\beta})$$

would provide an estimate of $\boldsymbol{\beta}$.

An alternative approach suitable for use on trajectory-based data can be obtained as follows. Given a value of $\boldsymbol{\beta}$, suppose in state s one observes a pair (r, s'), evaluates $u := r + \lambda v(s'; \boldsymbol{\beta})$ and seeks a vector $\boldsymbol{\beta}$ to minimize

$$g(\boldsymbol{\beta}) := (u - v(s; \boldsymbol{\beta}))^2,$$

regarding the scalar u as a fixed value independent of $\boldsymbol{\beta}$ (which of course it is not).

In this case, temporal differencing applies gradient descent to $g(\boldsymbol{\beta})$ by taking the gradient of $g(\boldsymbol{\beta})$,

$$\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}) = -2(u - v(s; \boldsymbol{\beta})) \nabla_{\boldsymbol{\beta}} v(s; \boldsymbol{\beta})$$

and using the recursion

$$\boldsymbol{\beta}' = \boldsymbol{\beta} + \tau (u - v(s; \boldsymbol{\beta})) \nabla_{\boldsymbol{\beta}} v(s; \boldsymbol{\beta}), \tag{11.14}$$

where the factor 2 is absorbed into τ.

When $v(s; \boldsymbol{\beta})$ is a linear function of $\boldsymbol{\beta}$, $v(s; \boldsymbol{\beta}) = \boldsymbol{\beta}^{\mathsf{T}} \mathbf{b}(s)$ and $\nabla_{\boldsymbol{\beta}} v(s; \boldsymbol{\beta}) = \mathbf{b}(s)$, in which case the gradient descent recursion becomes

$$\boldsymbol{\beta}' = \boldsymbol{\beta} + \tau \big(u - \boldsymbol{\beta}^{\mathsf{T}} \mathbf{b}(s)\big) \mathbf{b}(s). \tag{11.15}$$

Another way to think of (11.15) is to treat u as a "target" and at each iteration $\boldsymbol{\beta}$ is modified so as to move $v(s; \boldsymbol{\beta})$ closer to its target. The reinforcement learning literature refers to this approach as *bootstrapping* because it uses an approximation to the policy value function in the target.

The scalar quantity $\delta := r + \lambda v(s'; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta})$ is often referred to as a *temporal difference* or *Bellman error* and the following recursion is referred to as temporal differencing or TD(0) with function approximation. Using this additional notation, the recursion can be represented by:

$$\boldsymbol{\beta}' = \boldsymbol{\beta} + \tau \, \delta \, \mathbf{b}(s). \tag{11.16}$$

Note that in this expression τ and δ are scalars, while $\boldsymbol{\beta}$ and $\mathbf{b}(s)$ are column vectors of the same dimension.

Gradients and semi-gradients

The following discussion describes how gradient descent is usually employed in reinforcement learning recursions.

Given a simulated update, $r + v(s'; \boldsymbol{\beta})$, to minimize the squared-error loss

$$g(\boldsymbol{\beta}) = \big(r + v(s'; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta})\big)^2$$

using gradient descent, the gradient of $g(\boldsymbol{\beta})$ is given by

$$\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}) = 2\big(r + v(s'; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta})\big)\big(\nabla_{\boldsymbol{\beta}} v(s'; \boldsymbol{\beta}) - \nabla_{\boldsymbol{\beta}} v(s; \boldsymbol{\beta})\big). \tag{11.17}$$

In this expression the gradient includes **two** terms that correspond to the gradient of $v(\cdot; \boldsymbol{\beta})$ evaluated at two states, s' and at s. Temporal differencing is based on *ignoring the first expression* $\nabla_{\boldsymbol{\beta}} v(s'; \boldsymbol{\beta})$, that is, it approximates the gradient of $g(\boldsymbol{\beta})$ by

$$\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta}) \approx -2\big(r + v(s'; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta})\big)\nabla_{\boldsymbol{\beta}} v(s; \boldsymbol{\beta}). \tag{11.18}$$

The expression in (11.18) is referred to in the reinforcement learning literature as a *semi-gradient*. Using the semi-gradient in TD(0) is equivalent to treating the sampled update (target) $r + v(s'; \boldsymbol{\beta})$ as fixed and independent of $\boldsymbol{\beta}$.

When features are chosen to be indicator functions of states and $v(s; \boldsymbol{\beta})$ is a linear function of the features, the components of $\boldsymbol{\beta}$ correspond to the value function. Hence the semi-gradient also underlies tabular TD(0), Q-learning, and SARSA first described in Chapter 10.

What is the effect of using the semi-gradient instead of the gradient?

1. Using the semi-gradient is simpler because it avoids evaluating the gradient at *both* s and s'.

2. Despite being an approximation, the semi-gradient is often used in practice due to its *computational efficiency and empirical success*, especially in large-scale or deep reinforcement learning problems.

3. In trajectory-based on-policy applications, that is when the trajectory is generated by the policy being evaluated, both methods typically give similar results.

4. In off-policy applications using function approximation, that is when the trajectory is generated by a different policy than the one being evaluated, semi-gradient methods may diverge[8].

5. Semi-gradient methods do not correspond to the gradient of any objective function in general. This limits their theoretical justification and makes convergence analysis more difficult.

6. Semi-gradients underlie the recursions in both SARSA and Q-learning.

Exercise 17 asks you to investigate some of these issues in an example.

[8] See Baird (1995).

An algorithm

The following algorithm implements TD(0) in a discounted model. It evaluates a Markovian random decision rule d and allows for flexibility in selecting the sequence of states. It assumes features are fixed and pre-specified.

Algorithm 11.4. TD(0) with linear policy value function approximation for a discounted model

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MR}}$, $\boldsymbol{\beta}$ arbitrary, and $s \in S$.
   (b) Specify the number of iterations N and set $n \leftarrow 1$.
   (c) Specify the learning rate sequence $\tau_n, n = 1, 2, \ldots$.
2. **Iterate:** While $n \leq N$:
   (a) Sample a from $w_d(\cdot|s)$.
   (b) Simulate (s', r) or sample $s' \in S$ from $p(s'|s, a)$ and set $r \leftarrow r(s, a', s')$.
   (c) **Compute the temporal difference:** Set $v(s; \boldsymbol{\beta}) \leftarrow \boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s)$, $v(s'; \boldsymbol{\beta}) \leftarrow \boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s')$ and
   $$\delta \leftarrow r + \lambda v(s'; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta}). \tag{11.19}$$
   (d) **Update $\boldsymbol{\beta}$:**
   $$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_n \delta \mathbf{b}(s). \tag{11.20}$$
   (e) **Next state generation:**
   (Trajectory-based) $s \leftarrow s'$, or
   (State-based) Generate s from a uniform distribution on S, or
   (Hybrid) Specify ϵ small and a state s_0. With probability ϵ, $s \leftarrow s_0$ and with probability $1 - \epsilon$, $s \leftarrow s'$.
   (f) $n \leftarrow n + 1$.
3. **Terminate:** Return $\boldsymbol{\beta}$.

Some comments on implementing this algorithm follow:

1. **Specifying a decision rule to evaluate:** The algorithm is expressed in a form that assumes $d(s)$ can be explicitly specified for all $s \in S$. Unfortunately when S is large, this may not be possible. Optimization algorithms, described below, get around this requirement in different ways.

Q-learning chooses a' using ϵ-greedy or softmax sampling based on an estimated state-action value function $q(s, a; \boldsymbol{\beta})$.

Actor-critic[9] samples a' from $w(\cdot|s; \boldsymbol{\beta}^C)$ where the estimate of the weight $\boldsymbol{\beta}$, denoted by $\boldsymbol{\beta}^C$, is updated using gradient ascent.

2. **Convergence:** When $v(s; \boldsymbol{\beta})$ is a linear function of $\boldsymbol{\beta}$ and the features are linearly independent, the iterates of $\boldsymbol{\beta}$ converge[10] with probability 1 provided the samples follow the trajectory of the Markov chain corresponding to d and $\tau_n, n = 1, 2, \ldots$, satisfies the Robbins-Monro conditions. Of course, this also means that the corresponding policy value functions converge.

3. **Initialization:** As in most nonlinear optimization problems, convergence is sensitive to the initial specification of $\boldsymbol{\beta}$. A limited Monte Carlo analysis based on one or more replicates for a selected subset of states followed by least squares parameter estimation may provide reliable starting values.

4. **Stopping Rules:** They can be based on either changes in parameter values

$$\left(\frac{1}{I}\sum_{i=1}^{I}(\beta_i' - \beta_i)^2\right)^{\frac{1}{2}} \tag{11.21}$$

or changes in fitted values:

$$\left(\frac{1}{|S|}\sum_{s\in S}(v(s;\boldsymbol{\beta}') - v(s;\boldsymbol{\beta}))^2\right)^{\frac{1}{2}}. \tag{11.22}$$

or a weighted variant thereof. Use of these criteria have limitations: (11.21) may be dominated by the impact of a few large coefficients, while (11.22) may be computationally prohibitive due to the magnitude of S. Note that (11.22) may be more appropriate because it is consistent with the least squares objective function used to derive TD(0).

5. **State updating:** Step 2(e) provides three approaches for generating "subsequent" states for evaluation: following the trajectory, randomly restarting, or a combined approach. When this algorithm is used in a simulation environment such as in the queuing service rate control example immediately below, the random restart and hybrid methods will provide better coverage of S, especially if s_0 is chosen judiciously. This is because under a specified decision rule, the process may occupy only a small portion of the state space. In real-world implementations, random restarts may be difficult to implement so that the long-trajectory approach may be necessary. The hybrid approach provides a restart method that might be easier to implement.

[9] See Section 11.6 below.
[10] See Tsitsiklis and Roy [1997].

6. **Episodic models:** Since episodic models terminate at a state in Δ after a finite (but random) number of iterations, it is necessary to provide a mechanism to generate enough data to accurately estimate parameters. One possibility is to jump to a random state after termination. Also, in an episodic model, λ may be set equal to 1.

Policy value function approximation in the queuing service rate control model

This example applies Algorithm 11.4 to the model in Example 11.2 with $\lambda = 0.95$. It uses a cubic polynomial approximation to the policy value function of three deterministic stationary policies derived from the decision rules:

$$d_1(s) = \begin{cases} a_1 & 0 \le s \le 25 \\ a_3 & 26 \le s \le 50 \end{cases}$$
$$d_2(s) = \quad a_2 \quad 0 \le s \le 50$$
$$d_3(s) = \begin{cases} a_2 & 0 \le s \le 9 \\ a_3 & 10 \le s \le 29 \\ a_1 & 30 \le s \le 50 \end{cases}$$

Step 2(e) used trajectory-based, state-based and a hybrid variant that with probability 0.008 causes the process to jump to $s_0 = 50$ and with probability 0.992 followed the existing trajectory. Note that under d_3 choosing $s_0 = 0$ would be more appropriate.

Preliminary analysis suggested the following four learning rates for further evaluation: $\tau_n = 0.1n^{-0.5}$, $0.1\log(n+1)/n$, $0.1/(1+10^{-5}n^2)$, $150/(750+n)$ referred to respectively as polynomial, logarithmic, STC and ratio. Experiments compared all 36 combinations of decision rule, state updating method and learning rate over 40 replicates of length $N = 20{,}000$ using common random number seeds for all instances in a replicate.

The "convergence" of estimates was highly sensitive to starting values and feature scaling so that the following steps were taken to obtain initial weights:

1. Weights were initialized with one replicate of starting-state Monte Carlo implemented on a subset $\bar{S} = \{5, 15, 25, 35, 45\}$ of S;

2. States were scaled so as to have mean zero and standard deviation one, and

3. Least squares estimation was used to obtain preliminary estimates of the parameters of a cubic polynomial approximation.

Figure 11.4 compares the effect of the factors on the quality of the fit for each configuration. Observations include:

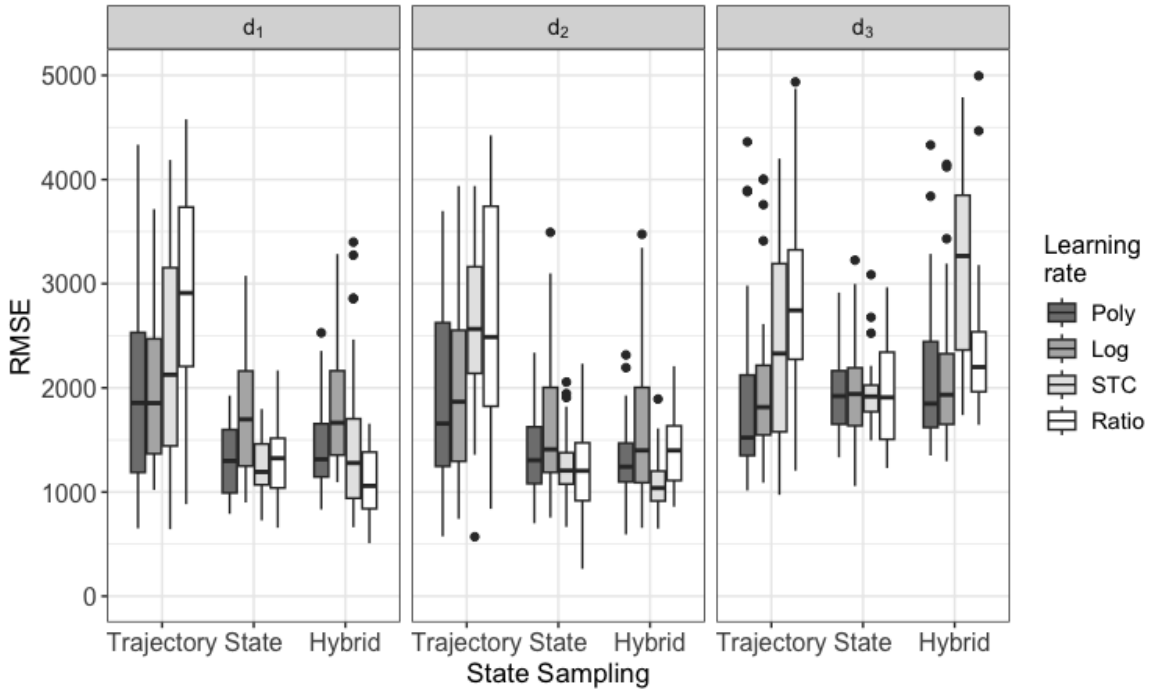


Figure 11.4: Comparison of the RMSEs of policy value function estimates for the queuing control model. To put results in perspective, Table 11.1 showed that a cubic polynomial approximation derived from Monte Carlo estimates had an average RMSE of 1,029.04 for d_1 and 1,098.57 for d_2.

1. With the exception of trajectory-based sampling, value function estimates for d_1 and d_2 had smaller RMSEs than for d_3. As d_3 is unlikely to arise in optimization, the following observations focus on d_1 and d_2.

2. Trajectory-based sampling resulted in the least accurate and most variable policy value function estimates for d_1 and d_2.

3. For d_1 and d_2, state-based sampling using a uniform distribution and hybrid state sampling produced similar results. For d_3, state-based sampling was less sensitive to learning rate than trajectory-based or hybrid sampling.

4. With state-based sampling, the effect of learning rate was similar for d_1 and d_2. Estimates based on STC were least variable and most accurate.

5. For hybrid sampling, the quality of estimates varied with decision rule. For d_1, the ratio learning rate resulted in the most accurate estimates and for d_2, the STC estimate was the most accurate and least variable.

Table 11.2 provides the RMSE of the policy value function estimate obtained from five randomly selected replicates. Each replicate used decision rule d_1 and the STC

	Replicate				
Method	1	2	3	4	5
MC - initialization only	1,209	1,259	794	973	2,111
MC + TD(0) - hybrid	967	1,209	873	882	1620
MC + TD(0) - state-based	1,270	1,070	1,441	926	1,131

Table 11.2: RMSE of fitted value functions for 5 replicates for decision rule d_1. It compares the RMSE of estimates based on Monte Carlo initialization only and Monte Carlo initialization combined with TD(0) using state-based and hybrid state updating.

learning rate, and compared value function estimates obtained by the Monte Carlo initialization only to those based on Monte Carlo initialization followed by TD(0) estimates with state-based and hybrid state generation. As a baseline, the RMSE from fitting a cubic regression function to the true value function is 344.73 so it would be unlikely that any approximation would generate a better fit. Moreover, Monte Carlo estimates obtained previously (Table 11.1) had RMSE 1,029.04 for d_1 and 1,098.57 for d_2 averaged over 40 replicates.

From Table 11.2 observe that:

1. For replicate 3, both TD(0) methods gave a worse fit than the Monte Carlo initialization that was based on one realization at five states. For all other replicates at least one of the TD(0) methods improved the fit from the initial value.

2. There was no clear pattern as to whether hybrid or random state-based updating gave more accurate estimates.

3. Neither method gave estimates with RMSE close to that of the regression fit to the true policy value function.

In conclusion, for this example, estimates based on Algorithm 11.4 were highly variable and not significantly better than Monte Carlo estimates, particularly when using trajectory-based state updating. There was some evidence suggesting that under hybrid state-sampling, a well-chosen learning rate may yield more accurate estimates than when using random state-based sampling. The next section considers a TD(γ) variant.

11.2.3 TD(γ) with linear policy value function approximation

This section extends TD(γ) to models with policy value function approximation. It presents an algorithm, provides some comments and includes an illustrative example. The focus is restricted to discounted models with linear policy value function approximations. This chapter uses the vector $\mathbf{z}$ to represent the eligibility trace to avoid confusion with the vector $\mathbf{e}$ of all ones.

Algorithm 11.5. TD(γ) for linear policy value function approximation for a discounted model

1. **Initialize:**
   (a) Specify $d \in D^{\mathrm{MR}}$, $\boldsymbol{\beta}$ arbitrary, and $s \in S$.
   (b) Specify the number of iterations N and set $n \leftarrow 1$.
   (c) Set the eligibility trace vector $\mathbf{z} \leftarrow \mathbf{0}$ and specify $\gamma \in [0, 1]$.
   (d) Specify the learning rate sequence $\tau_n, n = 1, 2, \ldots$.
2. **Iterate:** While $n \leq N$:
   (a) Sample a from $w_d(\cdot|s)$.
   (b) Simulate (s', r) or sample $s' \in S$ from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
   (c) **Compute the temporal difference:** Set $v(s; \boldsymbol{\beta}) \leftarrow \boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s)$, $v(s'; \boldsymbol{\beta}) \leftarrow \boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s')$ and
   $$\delta \leftarrow r + \lambda v(s'; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta}). \tag{11.23}$$
   (d) **Update eligibility trace:**
   $$\mathbf{z} \leftarrow \gamma\lambda\mathbf{z} + \mathbf{b}(s). \tag{11.24}$$
   (e) **Update $\boldsymbol{\beta}$:**
   $$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_n\delta\mathbf{z}. \tag{11.25}$$
   (f) **Next state generation:**
   (Trajectory-based) $s \leftarrow s'$, or
   (State-based) Generate s from a uniform distribution on S, or
   (Hybrid) Specify ϵ small and a state s_0. With probability ϵ, $s \leftarrow s_0$ and with probability $1 - \epsilon$, $s \leftarrow s'$.
   (g) $n \leftarrow n + 1$.
3. **Terminate:** Return $\boldsymbol{\beta}$.

Some comments follow:

1. The eligibility trace is a weighted linear combination of past feature vectors. The smaller the value of γ, the more quickly the effect of past features dies out. Note that when using nonlinear policy value function approximations, the gradient of the value function (with respect to its parameters) replaces the feature vector in (11.24).

2. When $\gamma = 0$, the algorithm reduces to TD(0).

3. It is left as an exercise to show that this algorithm reduces to Algorithm 10.6 when features are indicator functions of individual states.

Example 11.3. This example investigates the application of TD(γ) to the queuing model. Its primary purpose is to examine if and when there is an increase in accuracy or a reduction in variability that results from using TD(γ) with $\gamma > 0$ instead of TD(0).

Previous analysis suggested the best estimates occurred with STC and ratio learning rates. Accordingly, this example compares the impact on estimation of using these two learning rates, $\gamma \in \{0, 0.3, 0.6\}$ and three state-updating regimes. Each of 40 replicates sets $N = 20{,}000$ and uses decision rule d_1 and common random number seeds. Estimates are compared on the basis of the RMSE of the fitted values. Figure 11.5 summarizes results graphically.

Observe that the effect of γ varied with learning rate and state updating method:

1. The effect of γ was inconsistent.

   (a) For trajectory-based state updating with a ratio learning rate, the estimates when $\gamma > 0$ were more accurate than when $\gamma = 0$. However, this effect was not observed for the STC learning rate.

   (b) For state-based updating, the effect of γ was similar for both learning rates. Estimates based on $\gamma = 0.3$ were more accurate and less variable than those using TD(0) but the effect was small.

   (c) For hybrid updating, the effect of γ varied with the learning rate. For the STC learning rate, $\gamma = 0.6$ produced the most accurate and least variable estimates. However, when using the ratio learning rate, the TD(0) estimates were most accurate and least variable.

2. As before, trajectory-based sampling produced the least accurate and most variable estimates and state-based uniform random sampling produced the most accurate and least variable estimates. Estimates using hybrid state-updating behaved similarly to state-based updating although they were more variable.

3. Overall, the STC learning rate appeared to be preferable to the ratio learning rate. With this learning rate, there was a small benefit to using $\gamma > 0$ for state-based and hybrid state updating methods. For trajectory-based sampling, the effect of γ was minimal.

Overall, it appears there are small benefits of using TD(γ) with $\gamma > 0$, but the effect varies with the configuration. It is unlikely that the analyst would be able to anticipate a priori when that might be. The effect of the state updating method is far greater.

Analysis of variance methods would allow a formal analysis of the above observations. It seems likely that only some of the results observed above would be statistically significant.

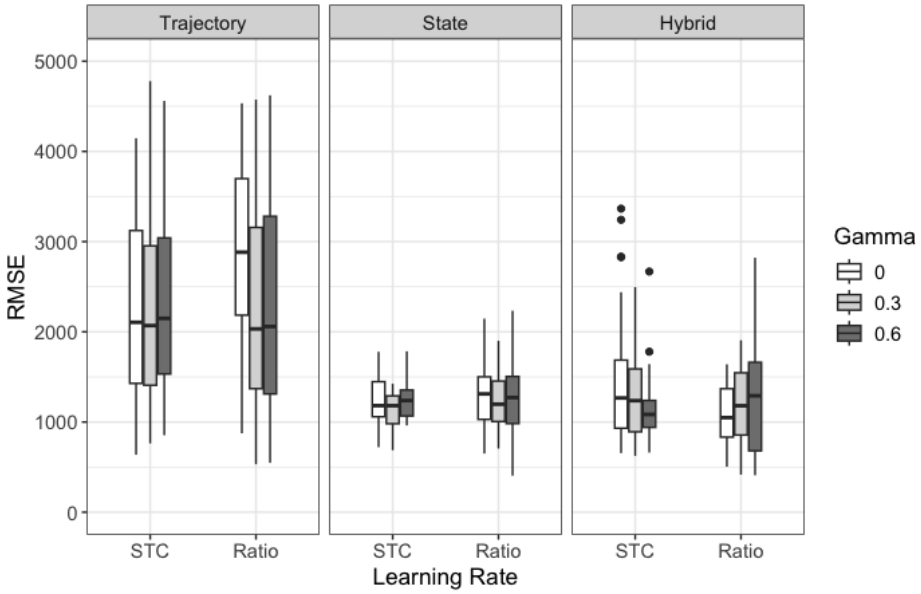


Figure 11.5: Boxplots of the RMSEs of the TD(γ) estimates based on 40 replicates with two learning rates and three state generation methods for the model in Example 11.3.

From this example, it appears that the effect of using TD(γ) with $\gamma > 0$ is small. Moreover, applying it could be problematic when the dimension of the eligibility trace, $\mathbf{z}$, is high, which is the case when using large neural network value function representations.

11.3 Optimization based on value function approximation: Q-learning

This section extends tabular Q-learning to settings in which state-action value functions are approximated by functions of features. An algorithm is provided for discounted models. It can be easily generalized to episodic models by adding a mechanism to

choose the starting state for subsequent episodes. Examples illustrate both the discounted and episodic cases.

11.3.1 Motivation

Q-learning in the discounted case is based on the Bellman equation for optimal state-action value functions. It was shown in Chapter 5 that the Bellman equation can be expressed as

$$q(s,a) = \sum_{j \in S} p(j|s,a)\big(r(s,a,j) + \lambda \max_{a \in A_s} q(j,a)\big)$$

and in expectation form as

$$q(s,a) = E\left[r(X,Y,X') + \lambda \max_{a' \in A_{X'}} q(X',a') \,\middle|\, X = s, Y = a \right]. \tag{11.26}$$

Recall that its unique solution satisfies

$$q^*(s,a) = \sum_{j \in S} p(j|s,a)\big(r(s,a,j) + \lambda v^*(j)\big)$$

or in expectation form as

$$q^*(s,a) = E\Big[r(X,Y,X') + \lambda v^*(X') \Big| X = s, Y = a \Big], \tag{11.27}$$

where $v^*(\cdot)$ denotes the optimal value function. Note that the expectation representation is interpretable directly in a model-free environment.

State-action value function approximation replaces $q(s,a)$ by $q(s,a;\boldsymbol{\beta})$ in (11.26) to obtain

$$q(s,a;\boldsymbol{\beta}) = E\left[r(X,Y,X') + \lambda \max_{a' \in A_{X'}} q(X',a';\boldsymbol{\beta}) \,\middle|\, X = s, Y = a \right]. \tag{11.28}$$

As in the case of a tabular model, Q-learning seeks to find a $\boldsymbol{\beta}$ that minimizes the discrepancy between the two-sides of (11.28) in the squared error sense as

$$E\left[\left(r(X,Y,X') + \lambda \max_{a' \in A_X} q(X',a';\boldsymbol{\beta}) - q(X,Y;\boldsymbol{\beta}) \right)^2 \,\middle|\, X = s, Y = a \right], \tag{11.29}$$

where the expectation is with respect to the distribution $p(\cdot|s,a)$.

The semi-gradient[11] of this expression with respect to $\boldsymbol{\beta}$ (see (11.18)) is written as

$$-2E\left[\left(r(s,a,X') + \lambda \max_{a' \in A_{X'}} q(X',a';\boldsymbol{\beta}) - q(s,a;\boldsymbol{\beta}) \right) \nabla_{\boldsymbol{\beta}} q(s,a;\boldsymbol{\beta}) \right].$$

[11] The full gradient would include an additional term corresponding to the gradient of $\lambda \max_{a' \in A_{X'}} q(X',a';\boldsymbol{\beta})$.

When $q(s,a)$ is approximated by a linear function of features $\mathbf{b}(s,a)$, the above expression becomes

$$-2E\left[\left(r(s,a,X')+\lambda\max_{a'\in A_{X'}}\boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(X',a')-\boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s,a)\right)\mathbf{b}(s,a)\right].$$

This expression leads to the following recursion for estimating $\boldsymbol{\beta}$:

$$\boldsymbol{\beta}\leftarrow\boldsymbol{\beta}+\tau\left(r(s,a,s')+\lambda\max_{a\in A_{s'}}\boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s',a')-\boldsymbol{\beta}^{\mathsf{T}}\mathbf{b}(s,a)\right)\mathbf{b}(s,a), \tag{11.30}$$

where s' is the result of state sampling. Note that in the above expression $\boldsymbol{\beta}$ and $\mathbf{b}(s,a)$ are column vectors and the expression in parentheses is a scalar.

11.3.2 Discounted Q-learning with function approximation

The following Q-learning algorithm seeks to find the coefficients of a linear approximation to an optimal state-action value function $q^*(s,a)$ for a fixed set of features. The output of the algorithm is a vector $\hat{\boldsymbol{\beta}}$ of weights that can be used in subsequent applications to determine action choice in state s according to

$$\hat{a}_s\in\arg\max_{a\in A_s}q(s,a;\hat{\boldsymbol{\beta}}) \tag{11.31}$$

or randomly using softmax sampling based on $q(s,a;\hat{\boldsymbol{\beta}})$. Moreover, the corresponding optimal value function approximation can be obtained from

$$\hat{v}(s)=\max_{a\in A_s}q(s,a;\hat{\boldsymbol{\beta}}). \tag{11.32}$$

Algorithm 11.6. Q-learning with linear state-action value function approximation for a discounted model

1. **Initialize:**
   (a) Specify $\boldsymbol{\beta}$.
   (b) Specify the learning rate $\tau_n, n=1,2,\ldots$.
   (c) Specify the number of iterations N.
   (d) Specify a sequence $\epsilon_n, n=1,2,\ldots$ (or η_n for softmax sampling).
   (e) Specify $s\in S$.
   (f) $n\leftarrow 1$.
2. **Iterate:** While $n\leq N$:

(a) Generate $a \in A_s$ randomly using Algorithm 11.1 applied to $\boldsymbol{\beta}$ in state s.

(b) Simulate (s', r) or sample $s' \in S$ from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.

(c) For all $a \in A_{s'}$

$$q(s', a; \boldsymbol{\beta}) \leftarrow \boldsymbol{\beta}^{\mathsf{T}} \mathbf{b}(s', a). \tag{11.33}$$

(d) Set

$$\delta \leftarrow r + \lambda \max_{a' \in A_{s'}} q(s', a'; \boldsymbol{\beta}) - q(s, a; \boldsymbol{\beta}). \tag{11.34}$$

(e) **Update $\boldsymbol{\beta}$:**

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_n \mathbf{b}(s, a)\delta. \tag{11.35}$$

(f) $s \leftarrow s'$.

(g) $n \leftarrow n + 1$.

3. **Terminate:** Return $\boldsymbol{\beta}$.

Some comments about this algorithm follow:

1. The algorithm requires ϵ-greedy or softmax action selection to ensure exploration. If only greedy actions were chosen in step 2(a), the algorithm would most likely end up evaluating a sub-optimal policy.

2. When computing the learning rate, the index is chosen to be the iteration number to avoid storing the number of visits to each state-action pair. In tabular models, using the frequency of visits to each state-action pair may be preferable since, unlike the more general parametric function approximation cases considered in this chapter, the values for state-action pairs are only updated when that state-action pair is observed. When using function approximation, weights are updated at every iteration so they should stabilize faster. Consequently, smaller learning rates and ϵ-greedy parameters can be used.

3. As stated, the algorithm is trajectory-based, that is, it selects sequences of states according to the underlying transition probabilities. Note that in step 2(f), $s \leftarrow s'$ can be replaced by either state-based or hybrid updating.

4. The above algorithm never explicitly computes a decision rule. Action selection in step 2(a) implicitly corresponds to a policy but as $\boldsymbol{\beta}$ changes, the elusive policy may change. The policy iteration algorithms below avoid this by fixing $\boldsymbol{\beta}$ for several iterations.

5. Note this is an off-policy algorithm since the update in (11.35) may be based on a different action than that followed by the trajectory. A SARSA variant, as in Chapter 10, would provide an on-policy alternative.

6. Unlike its tabular counterpart, which corresponds to using state-action indicators as basis functions in the above algorithm, there are no general convergence guarantees for Q-learning with function approximation.

7. The algorithm can be easily modified to allow for nonlinear state-action value function approximations. This would require replacing 2(c) to allow for a nonlinear functional form and replacing $\mathbf{b}(s,a)$ in 2(e) by the gradient of the nonlinear function.

Assessing policies in computational experiments

This brief digression offers insights into assessing the quality of policies generated by an optimization algorithm. Suppose, as in the example below, the optimal value $v^*(s)$ is known. Let d^∞ be a policy generated by some approximation method and let its true value function[12] be given by $v'(s) := v^{d^\infty}(s)$. Methods for comparing these values were discussed in Section 10.11.1 and are expanded on here.

A key issue is that under d^∞, the system most likely visits only a small region of the state space, say S'. For example, in the queuing service rate control model, the system will spend most of its time in low-occupancy states. Thus, the degree of sub-optimality of the approximate policy should be assessed based on a weighted combination of differences of values in states in S'. This is exactly what the weighted RMSE metric, denoted by wRMSE, defined in (10.102) achieves when weights $w(s)$ are chosen to be the stationary distribution[13] of the implemented policy.

In contrast, using an unweighted RMSE can lead to over-weighting discrepancies in infrequently visited states thereby providing a misleading belief on the loss of optimality faced by the system under d^∞.

An example

The example in this section applies Q-learning with value function approximation to the infinite horizon discounted queuing service rate control model.

Example 11.4. Consider the queuing service rate control model on $S = \{0,\ldots,50\}$. Basis functions are of the form (11.3) where the terms in $f_i(s)$ equal the scaled powers of a cubic polynomial and the terms in $h_j(a)$ represent indicator functions of each possible action. This is equivalent to representing $q(s,a)$ by a cubic polynomial in which the weights vary with the action.

As seen in the example in Section 11.2.2, good starting values are required to ensure convergence. By using the above specification for the approximation,

[12] This could be computed by solving policy evaluation equations or approximated by simulation.

[13] Recall that these can be found by solving $\mathbf{w}^\mathsf{T} = \mathbf{w}^\mathsf{T}\mathbf{P}$ subject to $\mathbf{w}^\mathsf{T}\mathbf{e} = 1$ or approximated by raising $\mathbf{P}$ to a large power.

preliminary weight estimates for the cubic polynomial approximation can be obtained using Monte Carlo estimation for each action separately or alternative randomizing over all actions with equal probability. Note that these estimates are distinct from the optimal state-action value functions sought by the algorithm because after choosing a state and action, they follow the policy value function of the policy used to obtain starting values, not the optimal value function.

The algorithm required considerable tuning to obtain convergence to reasonable policies. Results are described for $N = 150{,}000$, learning rates

$$\tau_n = \frac{5{,}000}{50{,}000 + n},$$

ϵ-greedy parameter

$$\epsilon_n = \frac{100}{400 + n}$$

for $n = 1, 2, \ldots$, and trajectory-based and random state-based state updating. The indices of the learning rate and ϵ-greedy parameter were the iteration number.

For reasons discussed above, results are assessed on the basis of the wRMSE of the greedy policy and the discrepancy between the greedy policy and the known optimal policy. The RMSE is reported for illustrative purposes.

Table 11.3 compares summary measures for the two state updating methods. Policies based on random state-based updating were closer to optimal on all three dimensions and also less variable. To put these quantities in perspective note that the exact optimal value function $v^*(s)$ satisfies $215 \leq v^*(s) \leq 39{,}365$ and $\sum_{s \in S} w(s)v^*(s) = 502$ for the stationary distribution $w(s)$ based on the optimal policy. For almost any reasonable policy, the system occupies states 0-10 most of the time so inaccuracy in high-occupancy and more-costly states is less important.

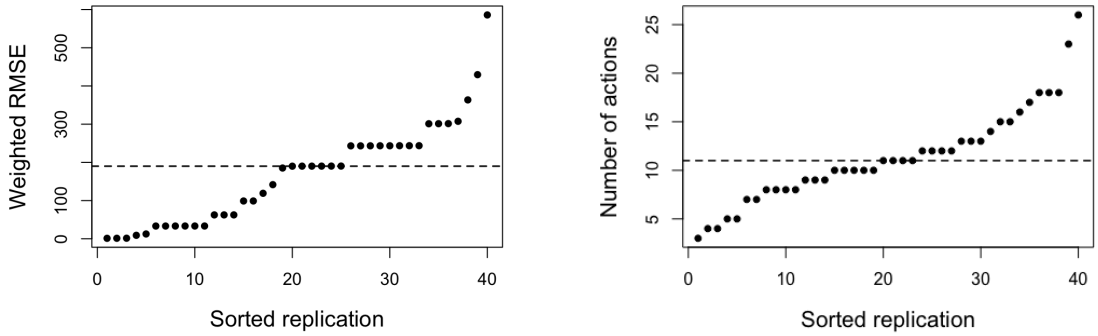
Figure 11.6a shows the sorted wRMSE values in ascending order. It shows the wRMSE was less than 300 in all but three replicates.

Figure 11.6b displays the number of states at which the greedy policy differed from the optimal policy under state-based sampling. For both state sampling methods greedy policies differed from the optimal policy in *all* replicates. Under state-based sampling, in 35 out of 40 replicates the greedy-policy had the same structure as the optimal policy; however, the states at which actions changed differed.

Consequently, from the perspective of near optimality, state-based sampling produced satisfactory results. However, this approach failed to identify the optimal policy in all cases. In contrast, Q-learning using a tabular representation was shown in Section 10.9 to be considerably more accurate. It is left to the reader to explore other approximations.

State updating	RMSE	wRMSE	Non-optimal actions
State-based	28.85 (37.47)	168.41 (133.03)	11.38 (4.93)
Trajectory-based	138.84 (80.36)	332.98 (567.03)	25.70 (10.33)

Table 11.3: Comparison of the mean (standard deviation) of the summary measures obtained from Example 11.4 broken down by state-updating method.



(a) Sorted wRMSEs.

(b) Sorted non-optimal actions.

Figure 11.6: Plots of the sorted wRMSE values and the number of non-optimal actions for 40 replicates in Example 11.4 using state-based updating. In each plot the horizontal dashed line denotes the corresponding median.

11.3.3 Q-learning in an episodic model

This example applies Q-learning to an episodic model. It considers a shortest path problem on a rectangular grid with M rows and N columns. In each cell, the robot can *try* to move up, down, right or left. If a move is impeded by a boundary, the robot remains in its current location. The goal is to reach a pre-specified cell (m^*, n^*). When that cell is reached the robot receives a reward of R. The cost of each attempted step is c (corresponding to a reward of $-c$). The objective is to maximize the expected total reward.

Markov decision process formulation

An MDP formulation follows:

States: $S = \{(m, n) \mid m = 1, \ldots, M, n = 1, \ldots, N\}$

Actions: For all $s = (m, n) \in S$, $A_s = A$ where

$$A = \begin{cases} \{\text{up, down, left, right}\} & (m,n) \neq (m^*, n^*) \\ \{\delta\} & (m,n) = (m^*, n^*) \end{cases}$$

Rewards:

$$r\big((m,n), a, (m',n')\big) = \begin{cases} -c & (m',n') \neq (m^*, n^*),\ a \in A \\ R - c & (m,n) \neq (m^*, n^*),\ (m',n') = (m^*, n^*),\ a \in A \\ 0 & (m,n) = (m^*, n^*), a = \delta \end{cases}$$

Transition probabilities:

$$p((m',n')|(m,n),a) = \begin{cases} 1 & \text{if move from } (m,n) \text{ to } (m',n') \text{ is possible under action } a \\ 0 & \text{otherwise.} \end{cases}$$

Note that the transition probabilities are awkward to write down explicitly because there are many cases to enumerate[14], but easy to code in a simulation.

This model can be analyzed as an undiscounted total return model in which the action choice probabilities introduce randomness in transitions. There are many improper policies, each having reward $-\infty$, however, there exist (many) deterministic optimal policies that can be easily found by inspection.

Feature choice

This example compares three implementations of Q-learning differing by the choice of the form of the approximate state-action value function. All are based on representation (11.4) in which $h_j(a)$ is an indicator function of the action. That is

$$h_1(a) := I_{\{\text{up}\}}(a), \quad h_2(a) := I_{\{\text{down}\}}(a), \quad h_3(a) := I_{\{\text{left}\}}(a), \quad h_4(a) := I_{\{\text{right}\}}(a)$$

for $a \in A$. They differ in the form of the features $f_i(m,n)$ as follows:

1. **Indicator functions of each cell (tabular model):**

   $$f_i(m',n') = I_{\{(m,n)\}}(m',n')$$

   for each $(m,n) \in S$. For each action there are $M \times N$ features (i.e., $i = 1, \ldots, MN$). In code, it might be more convenient to index $f_i(m',n')$ by the row and column index.

[14] For example, if the agent is at the left boundary, that is in state $(m,1)$, and tries to move left, it remains in state $(m,1)$ but if it tries, for example, to move right, a transition to $(m,2)$ occurs.

2. **Indicator functions of each row and column:**

$$f_i(m', n') = \begin{cases} I_{\{m\}}(m', n') & \text{for } m = 1, \ldots, M \text{ and } i = 1, \ldots, M, \\ I_{\{n\}}(m', n') & \text{for } n = 1, \ldots, N \text{ and } i = M+1, \ldots, M+N. \end{cases}$$

   Note that in this case there are $M + N$ features for each action.

3. **A parametric function of the row and column number:**

   Motivated by the calculations in the tabular case that are described below, it was hypothesized that linear features with an interaction term would well approximate the true state-action value function when combined with indicators functions of actions.

$$f_0(m, n) := 1, \quad f_1(m, n) := m, \quad f_2(m, n) := n, \quad f_3(m, n) := mn.$$

   This means that for each action, $q(s, a; \boldsymbol{\beta})$ is approximated by a linear function of 1, m, n, and mn with coefficients varying across actions.

   It is possible to also add higher order terms or even some judiciously chosen indicator functions. Note that when M and/or N are large, it may be necessary to scale the row and column index.

Results: Tabular model

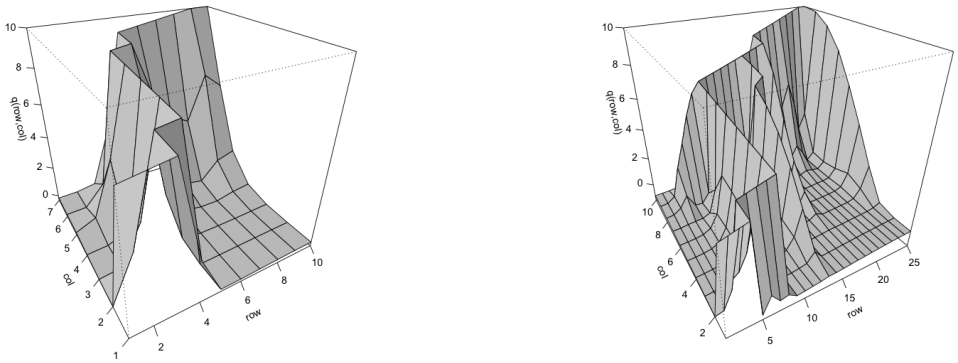
Calculations for the tabular model apply Algorithm 11.6 to two instances with $(m^*, n^*) = (M, N)$, $R = 10$ and $c = 0.1$. In this application the robot seeks to learn a path to cell (M, N) from the starting cell $(1, 1)$.

It uses ϵ-greedy action selection with $\epsilon_n = 10/(100 + n)$ and a learning rate of $\tau_n = 50/(1{,}000 + n)$ where n denotes the episode number. After completion of an episode, the next episode starts from cell $(1, 1)$[15]. The experiments used a discount rate of $\lambda = 1$, 10,000 episodes and set all $q((m, n), a) = 0$ for initialization.

For small values of M and N, it was convenient to use the tabular representation of Q-learning to gain some insight into the form of the q-functions. For all replications of the experiment, this approach found an optimal path through the grid.

Figure 11.7 shows the optimal q-functions from a single replicate for $(M, N) = (10, 7)$ and $(M, N) = (25, 10)$. It shows that in both cases, the algorithm identified a policy such that the robot's path was close to the main diagonal of the grid. This suggested that the parametric model above with a cross-product term might identify optimal policies.

[15] If one sought a good policy for every starting state, episodes could be restarted at a random non-target cell.



(a) 10×7 grid (b) 25×10 grid

Figure 11.7: Optimal q-values for the Gridworld shortest path model obtained using Q-learning with a tabular representation. Horizontal axes are row and column and vertical axis represents the optimal state-action value function.

Results: Indicator functions of rows and columns

The above calculations were repeated using indicator functions of grid row and grid column using the same choice for ϵ_n, τ_n and the two grid sizes. To reliably find an optimal policy required discounting. Setting $\lambda = 0.95$ gave reliable results.

Unlike the policies generated using the tabular representation, which tended to move up along the main diagonal, the greedy policies chose actions that moved the robot along the boundaries (row 1 and column N or column 1 and row M) of the grid. Moreover, the algorithm found an optimal policy from all starting states even when the target was an interior point of the grid.

Results: Parametric representation

Developing a convergent implementation required considerable experimentation. Issues that arose were divergence of q-values or termination with non-optimal greedy policies. The challenge was a consequence of credit assignment. Because rewards are only received when reaching the goal state, it required many iterations for the impact of the reward in the goal state to impact q-values in distant states. In light of these observations, successful implementations[16] used:

1. $\boldsymbol{\beta}$ initialized to be $\mathbf{0}$,
2. high initial exploration rates ($\epsilon_n = 20/(20 + n)$),
3. an upper bound (1,000) on the number of steps in an episode,
4. small learning rates ($\tau_n = 50/(1{,}000 + n)$),

[16]The literature also suggests using experience replay to enhance convergence.

5. a discount factor of 0.9, and

6. random starting states.

With these specifications, greedy policies based on Q-learning always achieved the objective and were optimal when starting in state $(1, 1)$. Moreover, they were similar to those found using the tabular representation, that is, they tended to step up through the interior of the grid. They exhibited fast learning as shown in Figure 11.8.

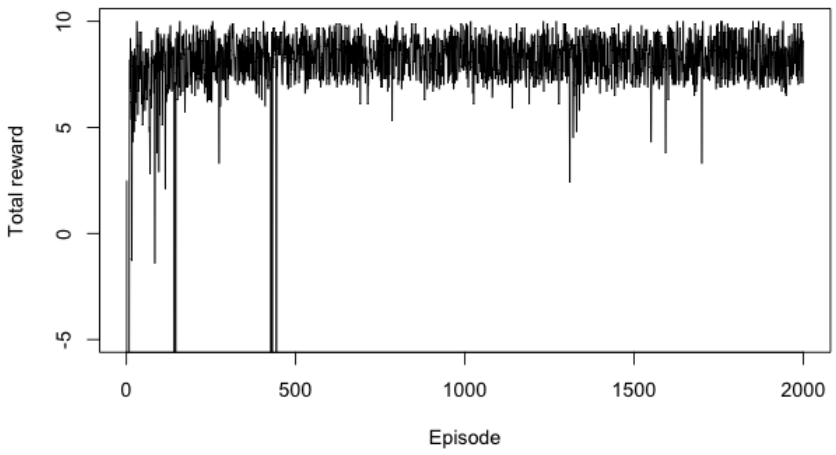


Figure 11.8: Reward per episode in a typical realization of Q-learning in a 10×25 grid. The ongoing variability of the total reward per episode is a result of the random starting state for each epsiode.

The beauty of using this parameterization was that it was not required to specify or even know the dimension of the grid before implementing Q-learning.

Summary

The above approximations used MN, $M + N$, and four features, respectively, for each action. While the first two approximations, which were based on indicator functions, reliably found optimal policies, those based on a low-dimensional parametric function required considerable experimentation to identify optimal policies. It is left to the reader to explore parametric approximations based on other functional forms or neural networks.

11.4 Q-policy iteration

Policy iteration algorithms require methods for both evaluating and improving stationary policies. This presents challenges for large models in which policies are represented

implicitly through vectors of weights. This section proposes, discusses and applies a weight-based policy iteration algorithm.

11.4.1 Motivation

Ideally, a weight-based policy iteration algorithm should include the following steps:

1. **Initialize:** Select a weight $\boldsymbol{\beta}$.
2. **Improve:** Find an "improved" policy using $q(s, a; \boldsymbol{\beta})$.
3. **Evaluate:** Find $\boldsymbol{\beta}'$ corresponding to the improved policy.
4. **Iterate:** Repeat.

Implementing such an approach presents challenges, especially when the state space is large and simulated data is used.

If the state space was small, the improve step could be applied in all states using Algorithm 11.1, the weights corresponding to the improved policy could be evaluated by simulation, and the process repeated. Of course, the evaluation step will only generate state-action pairs corresponding to the improved policy so there may not be sufficient coverage of the state-action space to accurately update the weights. An exploration-based alternative would evaluate an ϵ-greedy or softmax-based policy instead.

However, it is not necessary to explicitly generate the policy and then evaluate it. These steps can be combined by fixing the weight at $\boldsymbol{\beta}_0$, using $q(s, a; \boldsymbol{\beta}_0)$ to generate actions, and computing a revised estimate of the weight of this policy using temporal differencing. This is the logic underlying the following *Q-policy iteration* algorithm.

11.4.2 An algorithm

A Q-policy iteration algorithm follows. It is stated for discounted models; generalization to episodic models is direct and left to the reader. The reason why it may perform better than Q-learning is that actions are chosen on the basis of a q-function that approximates the value of the policy implicitly defined by $q(s, a; \boldsymbol{\beta}_0)$ rather than a more unstable estimate that changes every iteration.

Algorithm 11.7. Q-policy iteration with linear state-action value function approximation for a discounted model

1. **Initialize:**
   (a) Specify $\boldsymbol{\beta}_0$.
   (b) Specify the learning rate sequence $\tau_n, n = 1, 2, \ldots$, and a sequence $\epsilon_n, n = 1, 2, \ldots$ (or $\eta_n, n = 1, 2, \ldots$, for softmax sampling).

(c) Specify the number of evaluation iterations M and evaluation loops N.

(d) Specify $s \in S$, $k \leftarrow 1$, and $n \leftarrow 1$.

2. **Iterate:** While $n \leq N$:

(a) **Policy evaluation for fixed $\boldsymbol{\beta}_0$:** $m \leftarrow 1$ and $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}_0$.

(b) Generate $a \in A_s$ randomly using Algorithm 11.1 applied to $\boldsymbol{\beta}_0$ in state s.

(c) While $m \leq M$:

i. Simulate (s', r) or sample $s' \in S$ from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.

ii. Set
$$q(s, a; \boldsymbol{\beta}_0) \leftarrow \boldsymbol{\beta}_0^{\mathsf{T}} \mathbf{b}(s, a). \tag{11.36}$$

iii. Generate $a' \in A_{s'}$ randomly using Algorithm 11.1 applied to $\boldsymbol{\beta}_0$ in state s'.

iv. Set
$$q(s', a'; \boldsymbol{\beta}_0) \leftarrow \boldsymbol{\beta}_0^{\mathsf{T}} \mathbf{b}(s', a'). \tag{11.37}$$

v. Set
$$\delta \leftarrow r + \lambda q(s', a'; \boldsymbol{\beta}_0) - q(s, a; \boldsymbol{\beta}_0) \tag{11.38}$$

vi. **Update $\boldsymbol{\beta}$:**
$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_k \mathbf{b}(s, a)\delta. \tag{11.39}$$

vii. $s \leftarrow s'$, $a \leftarrow a'$.

viii. $k \leftarrow k + 1$ and $m \leftarrow m + 1$.

(d) **Weight updating:**

i. $\boldsymbol{\beta}_0 \leftarrow \boldsymbol{\beta}$.

ii. $n \leftarrow n + 1$.

3. **Terminate:** Return $\boldsymbol{\beta}_0$.

On the surface this does not look like a policy iteration algorithm since there is no clear improvement step. The algorithm combines improvement in steps 2(b) and 2(c)iii using fixed weights $\boldsymbol{\beta}_0$ with evaluation in step 2(b)vi. While the action-generating policy being evaluated remains the same, the estimate of its corresponding $\boldsymbol{\beta}$ is revised.

On the other hand, the algorithm may be viewed as a simulation-based version of modified policy iteration[17] (Algorithm 5.8) in which improvement and evaluation are

[17] Note that some authors, especially Bertsekas (2012), refer to modified policy iteration as *optimistic* policy iteration.

intertwined.

Some further comments follow:

1. In contrast to Q-learning which uses $q(s, a; \boldsymbol{\beta})$ as the basis for action selection, steps 2(b) and 2(c)iii choose actions using ϵ-greedy or softmax sampling based on $q(s, a; \boldsymbol{\beta}_0)$ for M iterations. This is done to enhance exploration.

2. When $M = 1$, this algorithm is equivalent to a SARSA variant of Q-learning. When $M > 1$, the evaluation step may be viewed as a TD(0) approximation to the state-action value function of the ϵ-greedy or softmax-based policy corresponding to $\boldsymbol{\beta}_0$.

3. Steps 2(c)iii-2(c)v can be replaced by the two steps:

   (a) Set
   $$q(s', a; \boldsymbol{\beta}_0) \leftarrow \boldsymbol{\beta}_0^{\mathsf{T}} \mathbf{b}(s', a) \tag{11.40}$$
   for all $a \in A_{s'}$.

   (b) Set
   $$\delta \leftarrow r + \lambda \max_{a' \in A_{s'}} q(s', a'; \boldsymbol{\beta}_0) - q(s, a; \boldsymbol{\beta}_0). \tag{11.41}$$

   This alternative replaces updating the value of a specific action by the maximum value based on the estimate of the state-action value function using $\boldsymbol{\beta}_0$ for M iterations. When $M = 1$, this updating approach is equivalent to Q-learning and for $M > 1$, it provides an alternative to the on-policy SARSA-like variant in the algorithmic statement.

4. No explicit stopping criterion is specified. One might terminate the algorithm when successive weight vectors differ by a small amount.

5. The algorithm is stated assuming trajectory-based state updating. Of course step 2(c)vii can be modified to update s using state-based or state-action pair-based sampling.

6. The index k is included to decrease the learning rate and exploration rates at *every* iteration. Each may be varied in different ways. For example in step 2(c)vii, k may be replaced by m and in implicit action choice in steps 2(b) and 2(c)iii the ϵ-greedy or softmax parameters can vary with n.

7. Similarly to Q-learning, there is no guarantee of convergence of this algorithm.

8. If one wished to start the algorithm with a decision rule d rather than a vector of weights $\boldsymbol{\beta}_0$, then step 2(b) would need to evaluate an ϵ-tweaked variant of this decision rule.

11.4.3 Newsvendor model

This section applies the following methods to the newsvendor inventory model formulated in Section 3.1.2:

1. Monte Carlo estimation
2. Q-learning
3. Q-policy iteration

The reason for considering this simple model is that by removing the effect of state transitions, subtle differences between Q-learning and Q-policy iteration become obvious.

Since this one-period model always starts in state $s = 0$, the state-action value function $q(a, 0)$ simplifies to the expected revenue from choosing action a, $q(a)$, which was defined in Section 3.1.2. Approximations of $q(a)$ use features corresponding to a cubic polynomial expressed in terms of scaled actions:

$$b_0(a) = 1, b_1(a) = \tilde{a},\ b_2(a) = \tilde{a}^2 \text{ and } b_3(a) = \tilde{a}^3,$$

where $\tilde{a}$ represents a scaled by subtracting the mean and dividing by the standard deviation[18], so that

$$q(a; \boldsymbol{\beta}) = \boldsymbol{\beta}^\mathsf{T} \mathbf{b}(a).$$

Monte Carlo estimation

To implement Monte Carlo estimation, for each action a in a subset A_0 of all actions, sample the demand N times to obtain $z^0, \ldots, z^N$ and set $\hat{q}(a)$ equal to the mean of $r(a, z^1), \ldots, r(a, z^N)$. Then use least squares to approximate $\hat{q}(a)$ by a linear function of its features to obtain $\hat{\boldsymbol{\beta}}_{\text{MC}}$. Note that the same demand sequence can be used for each chosen a.

Q-learning

The following algorithm applies Q-learning to the newsvendor model.

Algorithm 11.8. Q-learning for the newsvendor model

1. **Initialize:**
   (a) Specify $\boldsymbol{\beta}$, $a \in A$, the number of iterations N, and set $n \leftarrow 1$.
   (b) Specify sequences $\epsilon_n, n = 1, 2, \ldots$ and $\tau_n, n = 1, 2, \ldots$.

[18] This means $\tilde{a} = (a - \text{mean}(a))/\text{SD}(a)$ where the mean and standard deviation are over $\{0, 1, \ldots, a_{\max}\}$.

2. **Iterate:** While $n \leq N$:

   (a) Sample a' using ϵ-greedy (or softmax) sampling with respect to

   $$q(a; \boldsymbol{\beta}) = \boldsymbol{\beta}^\mathsf{T} \mathbf{b}(a).$$

   (b) Generate demand z^n.

   (c) Set

   $$\delta \leftarrow r(a', z^n) - \boldsymbol{\beta}^\mathsf{T} \mathbf{b}(a'). \tag{11.42}$$

   (d) **Update $\boldsymbol{\beta}$:**

   $$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_n \delta \mathbf{b}(a'). \tag{11.43}$$

   (e) $n \leftarrow n + 1$ and $a \leftarrow a'$.

3. **Terminate:** Return $\hat{\boldsymbol{\beta}}_{\text{QL}} = \boldsymbol{\beta}$.

Note that the calculation of δ in (11.42) sets $\max_{a' \in A_{s'}} q(a', s') = 0$ because the episode ends after realizing the reward corresponding to demand z^n.

Q-policy iteration

The following algorithm applies Q-policy iteration to the newsvendor model.

Algorithm 11.9. Q-policy iteration for the newsvendor model.

1. **Initialize:**

   (a) Specify $\boldsymbol{\beta}_0$, the number of evaluation iterations M, the number of evaluation loops N, $k \leftarrow 1$ and $n \leftarrow 1$.

   (b) Specify sequences $\epsilon_n, n = 1, 2, \ldots$ and $\tau_n, n = 1, 2, \ldots$.

2. **Iterate:** While $n \leq N$:

   (a) $m \leftarrow 1$ and $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}_0$.

   (b) While $m \leq M$:

      i. Sample a' using ϵ-greedy (or softmax) sampling with respect to

      $$q(a; \boldsymbol{\beta}_0) = \boldsymbol{\beta}_0^\mathsf{T} \mathbf{b}(a).$$

      ii. Generate demand z^m.

      iii. Set

      $$\delta \leftarrow r(a', z^m) - \boldsymbol{\beta}^\mathsf{T} \mathbf{b}(a') \tag{11.44}$$

iv. **Update $\boldsymbol{\beta}$:**

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_k \delta \mathbf{b}(a').$$

v. $k \leftarrow k+1$, $m \leftarrow m+1$, and $a \leftarrow a'$.

(c) $\boldsymbol{\beta}_0 \leftarrow \boldsymbol{\beta}$ and $n \leftarrow n+1$.

3. **Terminate:** Return $\hat{\boldsymbol{\beta}}_{\text{QPI}} = \boldsymbol{\beta}_0$.

While these algorithms appear similar, there are subtle but important differences.

1. In each of these algorithms, the "max" in the Bellman update disappears because the Bellman equation reduces to $\max_{a \in A_0} E[r(a, Z)]$ so that the continuation cost is zero.

2. In Q-learning, action selection in step 2(a) is with respect to a different $\boldsymbol{\beta}$ at each iteration, which is updated in (11.43). In Q-policy iteration, a fixed $\boldsymbol{\beta}_0$ is used for action generation in step 2(b)i for M evaluation steps and a different $\boldsymbol{\beta}$ (corresponding to the policy that is being evaluated) is updated in step 2(b)iv. This is because the inner loop seeks to evaluate a fixed policy based on the ϵ-greedy or softmax based decision rule corresponding to $\boldsymbol{\beta}_0$.

3. When $M = 1$, Q-learning and Q-policy iteration are equivalent.

A numerical example

This example considers the newsvendor problem with two discrete demand distributions:

- a rounded normal distribution, truncated at 0, with mean 50 and standard deviation 15, and
- a rounded gamma distribution with shape parameter 20 and scale parameter 4.

It sets the item cost to 20, the salvage value to 5 and the price to be each of 21, 30, and 120. The corresponding ratios of $G/(G+L)$ that were used to obtain the optimal order quantity, equaled 0.0625, 0.400, and 0.870, representing a range of quantiles of the demand distributions. The possible order quantities were $A_0 = \{0, 1, \ldots, 120\}$.

To obtain a baseline, 10,000 Monte Carlo replicates were generated for each $a \in A_0$ and the expected reward corresponding to each action was set equal to the average reward over the samples for that action. Table 11.4 gives the optimal order quantity using the closed form representation (3.3) denoted "Optimal", the maximum of the Monte Carlo estimates denoted "Monte Carlo", and the maximum obtained when fitting the Monte Carlo estimates with a cubic polynomial, denoted "MC-Cubic", and a cubic spline with knots at 40 and 80, denoted "MC-Spline".

Distribution	Price	Optimal	Monte Carlo	MC-Cubic	MC-Spline
Normal	21	27	26	23	29
	30	46	45	41	44
	120	67	65	76	69
Gamma	21	55	53	47	54
	30	74	76	76	75
	120	100	99	101	99

Table 11.4: Order quantities for the newsvendor model based on Monte Carlo estimates.

Observe that the order quantity that maximizes the Monte Carlo estimates accurately approximates the optimal order quantity. Observe also that the order quantity obtained by maximizing the spline approximation better approximates the Monte Carlo maximum than that derived from the cubic approximation. This is to be expected because the spline is a more flexible function with more parameters than a cubic polynomial.

Q-learning and Q-policy iteration were applied to these instances using a cubic approximation based on scaled values of a. Both algorithms were initiated with $\boldsymbol{\beta} = \mathbf{0}$, used the STC learning rate $\tau_n = 0.05(1 + 10^{-5}n^2)^{-1}$ and ϵ-greedy exploration with parameter $\epsilon_n = 4{,}000/(20{,}000 + n)$. Calculations used $N = 20{,}000$ for Q-learning and $N = 10$ and $M = 2{,}000$ for Q-policy iteration, so that the total number of iterates in each case was the same. For each combination of demand distribution and price, 40 replicates of each algorithm were applied with common random number seeds, or equivalently, the same sequence of demands.

The boxplots in Figure 11.9 shows the variability of the estimates of the optimal order quantity chosen according to

$$a^* \in \arg\max_{a \in A_0} \hat{\boldsymbol{\beta}}^{\mathsf{T}} \mathbf{b}(a)$$

over the 40 replicates.

Comparison with Table 11.4 shows that Q-policy iteration better approximated the order quantity obtained using MC-Cubic than the Q-learning estimates in **all** cases. Moreover, with normal demand:

1. When price equals 21, the Q-learning estimate often resulted in a non-increasing approximating polynomial with $a^* = 0$.

2. The Q-policy iteration estimates were considerably less variable than those using Q-learning.

These observations suggest that Q-policy iteration offers a promising alternative to Q-learning.

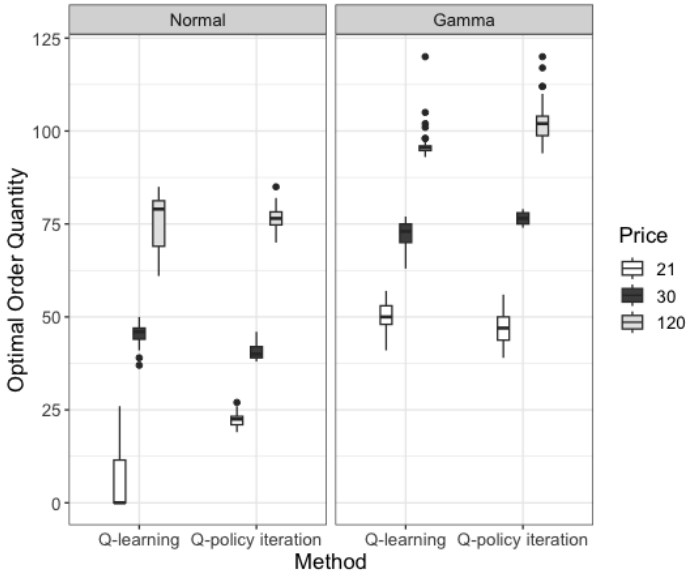


Figure 11.9: Boxplots comparing the actions chosen by Q-learning and Q-policy iteration for the newsvendor model over 40 replicates broken down by demand distribution and price.

11.4.4 Queuing service rate control model

The example in this section applies Algorithm 11.7 to the queuing control model and compares results to those obtained applying Q-learning directly. Again it assumes $q(s, a)$ is approximated by distinct cubic functions of the state for each action.

To obtain reasonable policy estimates required considerable experimentation with starting values, the total number of iterations, the learning rate and the ϵ-greedy parameters. Table 11.5 provides summaries of three performance metrics over 40 replicates of several variants of Q-policy iteration with

$$\tau_n = 5{,}000/(50{,}000 + n)$$

and

$$\epsilon_n = 100/(400 + n),$$

where the index for the parameters was the cumulative iteration number. State updating was based on random state-based sampling and performance measures were derived from a total of 250,000 iterations per replicate apportioned between M and N.

The resulting policies were assessed on the basis of the number of non-optimal actions, the RMSE between the true value of the greedy policy based on the final q-function estimate and the optimal value, and the wRMSE weighted by the stationary distribution of the greedy policy.

The results in Table 11.5 suggest:

Method	Quantity	min	25th %ile	median	75th %ile	max
Q-policy iteration	Non-optimal actions	5	9	10	11	20
	RMSE	3.5	12.9	19.9	25.3	139.0
$(M = 100, N = 2{,}500)$	wRMSE	33.5	105.3	157.9	208.2	585.9
Q-policy iteration	Non-optimal actions	3	6	7.5	10	21
	RMSE	1.7	12.2	16.8	22.5	169.9
$(M = 50, N = 5{,}000)$	wRMSE	12.6	141.7	150.6	212.7	587.0
SARSA	Non-optimal actions	5	9.8	11.5	13	40
	RMSE	3.0	19.2	26.4	30.8	334.4
	wRMSE	1.4	141.8	194.3	585.9	651.9
Q-policy iteration variant	Non-optimal actions	3	6	7	9	18
	RMSE	1.7	8.3	12.6	22.1	126.7
$(M = 50,\ N = 5{,}000)$	wRMSE	12.6	98.9	141.7	229.9	585.9
Q-learning	Non-optimal actions	6	8.8	12	13	17
	RMSE	3.9	13.3	21.0	23.8	95.7
	wRMSE	9.9	62.65	141.8	190.3	440.3

Table 11.5: Summary statistics based on 40 replicates of Q-policy iteration, Q-learning and SARSA applied to the queuing service rate control model. The Q-policy iteration variant uses (11.40) and (11.41) for parameter updating. Fractional values for percentiles of non-optimal actions result from interpolation.

1. No instance identified the optimal policy. Q-policy iteration and its variant with $M = 50$ and $N = 5{,}000$ generated policies that were closest to optimum. However, in a few replicates[19] policies were non-monotone or only chose two actions.

2. Both instances of Q-policy iteration as stated in Algorithm 11.7 were preferable to SARSA with respect to all performance metrics. The combination $M = 50, N = 5{,}000$ gave slightly better results than using $M = 100, N = 2{,}000$.

3. Policies selected by the Q-policy iteration variant were closer than Q-learning to the optimum policy on the basis of number of non-optimal actions chosen. However, this did not translate to consistently improved performance with respect to RMSE and wRMSE.

4. The Q-policy iteration variant was preferable to choosing the implementation in Algorithm 11.7.

The main takeaways from this example are that Q-policy iteration based on the variant using (11.40) and (11.41) was preferable to directly applying Algorithm 11.7 and that there is weak evidence that the Q-policy iteration variant was preferable to Q-learning but the difference in performance was small.

[19] Results not shown.

11.4.5 Concluding remarks on examples

These numerical studies show that for the queuing control model, both Q-learning and Q-policy iteration produced similar results. However, Q-policy iteration was slightly better at identifying a policy in agreement with the optimal policy. In the newsvendor model, Q-policy iteration was better than Q-learning in identifying a policy that agreed with that found using a cubic approximation to Monte Carlo estimates.

Overall, both Q-learning and Q-policy iteration provide the analyst with tools for analyzing models that use function approximation. Their relative performance will depend on many factors including the choice of parameter settings and the specific problem being solved.

11.5 Policy space methods

This lengthy section describes a conceptually different approach to function approximation based on parameterizing the probability that a randomized stationary policy selects an action in a given state. Instead of directly improving a value function or state-action value function, it seeks to find a choice of parameter values that improves the value of a policy directly.

Its advantage is that unlike state-action value function focused methods (such as Q-learning), which may jump between deterministic policies, it allows for gradual changes in action-selection probabilities leading to greater stability in values. This is especially attractive when controlling physical systems in real-time.

Two concepts underlie this approach:

1. *Expressing the probability that a stationary randomized policy d^∞ chooses action a in state s as a parametric function of both the state and action.* This is somewhat similar to the approach used to parameterize a state-action value function in which both s and a are inputs and $q(s, a)$ is the output. In contrast, policy space methods are based on functions of s and a, however, the state s is an input and the probability of choosing action a, $w_d(a|s)$, is the output.

2. *Updating parameter values using (stochastic) gradient ascent.* This approach seeks to iteratively find a zero of the gradient of a policy value function[20] by stochastic approximation in which gradient estimates are obtained by sampling from a distribution with a specified expectation.

[20] Of course, the value function varies over states. To obtain a single value, a weighted average over states is used. Surprisingly, this choice of weights will not affect the policy updating.

11.5.1 Motivation: A policy gradient algorithm for a one-state one-period model

Consider first a one-period, single-state multi-action model such as the newsvendor problem. Since this model does not include state transitions, it reduces to a classical univariate stochastic optimization model. The same notation for approximation is used as when approximating state-action value functions although as noted above, these are two conceptually different approaches. That is, features are represented by vectors $\mathbf{b}(s,a)$ and parameters by compatible vectors[21] $\boldsymbol{\beta}$. In the single-state model, the state, say 0, is fixed so that the vector of features is represented by $\mathbf{b}(a) := \mathbf{b}(0,a)$.

Frequently, policies are expressed in terms of the softmax or logistic function. For single-state models:

$$w(a|\boldsymbol{\beta}) = \frac{e^{\boldsymbol{\beta}^\intercal \mathbf{b}(a)}}{\sum_{a' \in A_s} e^{\boldsymbol{\beta}^\intercal \mathbf{b}(a')}} \tag{11.45}$$

where $\boldsymbol{\beta}$ is a column vector of parameters and $\mathbf{b}(a)$ is a column vector of features associated with action a. In multi-state models (Markov decision processes),

$$w(a|s;\boldsymbol{\beta}) = \frac{e^{\boldsymbol{\beta}^\intercal \mathbf{b}(s,a)}}{\sum_{a' \in A_s} e^{\boldsymbol{\beta}^\intercal \mathbf{b}(s,a')}}. \tag{11.46}$$

Note that the temperature parameter of the softmax function is absorbed into the weight vector $\boldsymbol{\beta}$. However in some settings, such as the optimal stopping model below, it might be convenient to retain it to stabilize estimates.

The goal when analyzing this single-state model is to find a $\mathbf{b}(a)$ that attains the maximum in

$$v^* := \max_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = \max_{\boldsymbol{\beta}} \sum_{a \in A_0} w(a|\boldsymbol{\beta}) r(a) = \max_{\boldsymbol{\beta}} \sum_{a \in A_0} \frac{e^{\boldsymbol{\beta}^\intercal \mathbf{b}(a)}}{\sum_{a' \in A_s} e^{\boldsymbol{\beta}^\intercal \mathbf{b}(a')}} r(a). \tag{11.47}$$

In this expression, $r(a)$ denotes the (expected) reward for choosing action a, $w(a|\boldsymbol{\beta}) := w(a|0;\boldsymbol{\beta})$ and

$$v(\boldsymbol{\beta}) := \sum_{a \in A_0} w(a|\boldsymbol{\beta}) r(a) = E_{\boldsymbol{\beta}}[r(Y)],$$

where Y is the random action selected by $w(\cdot|\boldsymbol{\beta})$.

Policy gradient methods seek to maximize this expression by solving:

$$\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = \mathbf{0}. \tag{11.48}$$

In the newsvendor model, $r(a) = E[r(a,\delta)] = \sum_{z=0}^{M} r(a,z) f(z)$, where $f(z) = P[Z = z]$. (Recall that Z is the random demand with support $\{0,1,\ldots,M\}$.) In this simple model, $v(\boldsymbol{\beta})$ can be computed analytically, numerically or through simulation. To simulate this model requires sampling from both $w(\cdot|\boldsymbol{\beta})$ and the demand distribution $f(\cdot)$.

[21] Many authors represent randomized decision rules by $\pi_{\boldsymbol{\phi}}(a|s)$ where $\boldsymbol{\phi}$ denotes a vector of parameters.

The gradient of $v(\boldsymbol{\beta})$ in the newsvendor model

The machinery to apply stochastic gradient descent is developed next. To do so requires deriving a representation for the gradient of $v(\boldsymbol{\beta})$. This derivation is based on a simple calculus identity that is the key step in generalizing this result to multi-state models.

The gradient of $v(\boldsymbol{\beta})$ can be written as:

$$\begin{aligned}\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) &= \sum_{a\in A_0} \nabla_{\boldsymbol{\beta}} w(a|\boldsymbol{\beta}) r(a) \\ &= \sum_{a\in A_0} w(a|\boldsymbol{\beta}) \nabla_{\boldsymbol{\beta}} \ln(w(a|\boldsymbol{\beta})) r(a) \qquad (11.49)\\ &= E_{\boldsymbol{\beta}}\left[\nabla_{\boldsymbol{\beta}} \ln(w(Y|\boldsymbol{\beta})) r(Y)\right]. \qquad (11.50)\end{aligned}$$

The expression in (11.49) follows by applying the calculus identity

$$\frac{dg(x)}{dx} = g(x)\frac{d\ln(g(x))}{dx} \qquad (11.51)$$

to each component of $\boldsymbol{\beta}$.

The benefits of establishing this equivalence are that:

1. When using the softmax to represent $w(a|\boldsymbol{\beta})$, the gradient of $\ln(w(a|\boldsymbol{\beta}))$ has a nice closed-form representation.

2. As a result of the representation (11.50), the gradient of $v(\boldsymbol{\beta})$ can be estimated by sampling
$$\nabla_{\boldsymbol{\beta}} \ln(w(Y|\boldsymbol{\beta})) r(Y)$$
from $w(\cdot|\boldsymbol{\beta})$.

3. In multi-period models, expectations are based on products of probabilities, in which case logarithms transform the product to a sum making it easier to analyze[22].

It is left as an exercise to show that when $w(a|\boldsymbol{\beta})$ is defined by (11.45),

$$\nabla_{\boldsymbol{\beta}} \ln(w(a|\boldsymbol{\beta})) = \mathbf{b}(a) - \sum_{a'\in A_0} w(a'|\boldsymbol{\beta})\mathbf{b}(a'). \qquad (11.52)$$

Combining (11.49) with (11.52) leads to the following closed-form expression for the gradient of $v(\boldsymbol{\beta})$ in the newsvendor model:

[22]Note that in statistical maximum likelihood estimation one bases results on maximizing log-likelihoods instead of likelihoods.

$$\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = \sum_{a \in A_0} w(a|\boldsymbol{\beta}) \Big(\mathbf{b}(a) - \sum_{a' \in A_0} w(a'|\boldsymbol{\beta}) \mathbf{b}(a') \Big) r(a). \tag{11.53}$$

Online estimation of the gradient in the newsvendor model

This section applies stochastic gradient ascent[23] to solve $\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = \mathbf{0}$ for the newsvendor model. This method involves three steps:

1. Sample action a from $w(\cdot|\boldsymbol{\beta})$,
2. sample the demand from $f(\cdot)$ to estimate $r(a) = E[r(a, Z)]$, and
3. estimate the gradient by $\nabla_{\boldsymbol{\beta}} \ln(w(a|\boldsymbol{\beta}))r(a)$.

Thus, the gradient ascent recursion becomes

$$\boldsymbol{\beta}' \leftarrow \boldsymbol{\beta} + \tau \nabla_{\boldsymbol{\beta}} \ln(w(a|\boldsymbol{\beta}))r(a), \tag{11.54}$$

where the gradient is computed using (11.53) (or by numerical differentiation). This observation leads to the following policy gradient algorithm for the newsvendor model (assuming the simulation of both actions and rewards).

Algorithm 11.10. Policy gradient for the newsvendor model

1. **Initialize:**
   (a) Specify $\boldsymbol{\beta}$ and a learning rate sequence $\tau_n, n = 1, 2, \ldots$.
   (b) Specify the number of iterations N and set $n \leftarrow 1$.
2. **Iterate:** While $n \leq N$:
   (a) Sample a from $w(\cdot|\boldsymbol{\beta})$.
   (b) Generate demand z^n and set $r \leftarrow r(a, z^n)$.
   (c) **Estimate the gradient:**

$$\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = \Big(\mathbf{b}(a) - \sum_{a' \in A_0} w(a'|\boldsymbol{\beta}) \mathbf{b}(a') \Big) r. \tag{11.55}$$

   (d) **Update $\boldsymbol{\beta}$:**

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_n \nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}). \tag{11.56}$$

[23] "Ascent" is used since this is a maximization problem.

(e) $n \leftarrow n + 1$.

3. **Terminate:** Return $\boldsymbol{\beta}$.

Some comments follow:

1. As stated, the algorithm samples both the action and the reward given the action. In some examples the reward may be a deterministic function of the action so it can be computed directly once the action is known.

2. In a model-free environment, instead of sampling the reward, it can be observed. However, one would still sample the action from $w(\cdot|\boldsymbol{\beta})$.

3. The policy gradient algorithm converges to a global maximum under mild conditions on $r(a)$.

4. The algorithm terminates with an estimate of $\boldsymbol{\beta}$ and the corresponding stationary randomized policy $w(a|\boldsymbol{\beta})$.

5. Alternatively to stopping after a specified number of iterations, the algorithm can terminate when the change in $\boldsymbol{\beta}$ achieves a pre-specified tolerance.

How the algorithm works

To illustrate the workings of this algorithm take as features indicator variables of actions as follows:

$$b_i(a) = \begin{cases} 1 & a = a_i \\ 0 & a \neq a_i \end{cases}$$

for $i = \{0, 1, \ldots, a_{\max}\}$. In this case, there are $a_{\max} + 1$ features. Note this choice of features is equivalent to using a tabular representation. The advantage of this choice of features is that it provides very clear insight in to the workings of gradient ascent as the following calculations show.

For concreteness, assume that $a \in \{0, 1, 2, 3\}$ and $\boldsymbol{\beta} = (0, 0, 0, 0)$, so that defining $w(a|\boldsymbol{\beta})$ by (11.45) gives $w(a|\boldsymbol{\beta}) = 0.25$ for $a \in \{0, 1, 2, 3\}$. Suppose sampling generates action $a = 1$ and $r(a) = 17$. Then applying (11.55) shows that

$$\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = 17 \left(\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0.25 \\ 0.25 \\ 0.25 \\ 0.25 \end{bmatrix} \right) = 17 \begin{bmatrix} -0.25 \\ 0.75 \\ -0.25 \\ -0.25 \end{bmatrix}.$$

Observe that the component of the gradient corresponding to $a = 1$ is positive and all other components are negative. Thus, as a result of applying gradient ascent in step 2(d) of Algorithm 11.10, the component of $\boldsymbol{\beta}$ corresponding to $a = 1$ will increase and

all other components will decrease. Hence the policy $w(a|\boldsymbol{\beta}')$, where $\boldsymbol{\beta}'$ corresponds to the updated parameter vector, will select $a = 1$ with greater probability and all other actions with smaller probability than $w(a|\boldsymbol{\beta})$. If on the other hand $r(a) < 0$, the reverse would occur, namely the probability of selecting $a = 1$ would decrease and all other probabilities would increase.

Example 11.5. Policy gradient applied to the newsvendor model.

This example sets $a_{\max} = 25$, the unit cost to 20 and the salvage value to 5. The possible prices are 21, 30, and 120. Three demand distributions are considered:

1. A truncated and rounded normal with mean 10 and standard deviation 3,
2. a binomial distribution with $n = 10$ and $p = 0.6$, and
3. a discrete uniform distribution on $0, \ldots, a_{\max}$.

Algorithm 11.10 is applied with $N = 10{,}000$ and $\tau_n = 0.0001$ for each of 40 replicates across all nine combinations of price and demand distribution.

In all replicates, the distribution $w(a|\boldsymbol{\beta})$ evaluated at the estimated $\boldsymbol{\beta}$ was unimodal in a, with a single action having probability extremely close to 1 and all others near 0, corresponding to a deterministic policy.

Table 11.6 summarizes results. The column "PG: Mean order quantity" reports the average of $\arg\max_{a \in A_0} w(a|\boldsymbol{\beta})$, while the "PG: Standard deviation" column gives its standard deviation across the 40 replicates. In all instances, the mean order quantity closely approximates the "Optimal order quantity" calculated using (3.3). The greatest variability was observed when the price equaled 120, corresponding to the 87th percentile of the demand distribution.

11.5.2 A policy gradient algorithm for a Markov decision process

Intuition and results in the previous section provide the basis of a policy gradient algorithm for an episodic Markov decision process. Recall that the goal in an episodic model is to maximize the expected total reward prior to reaching a set of zero-reward absorbing states Δ. In such models one seeks to find a policy π that maximizes

$$v^{\pi}(s) = E^{\pi}\left[\sum_{n=1}^{N_\Delta} r(X_n, Y_n, X_{n+1}) \middle| X_1 = s\right]$$

over the set of all history-dependent randomized policies, where N_Δ denotes the random time the system enters Δ. Recall (see Section 2.2.3) that the expectation is with respect

Demand distribution	Price	Optimal order quantity	PG: Mean order quantity	PG: Standard deviation
Normal	21	5.40	5.55	0.64
	30	9.23	9.50	0.72
	120	13.37	14.43	2.05
Binomial	21	4	3.73	0.45
	30	6	5.58	0.50
	120	8	7.93	1.16
Uniform	21	1.63	1.15	0.36
	30	10.40	10.75	1.69
	120	22.61	21.93	2.10

Table 11.6: Mean and standard deviation over 40 replicates of the order quantity obtained by maximizing $w(a|\hat{\boldsymbol{\beta}})$ over a where $\hat{\boldsymbol{\beta}}$ is obtained using Algorithm 11.10. The optimal order quantity is based on the closed form expression (3.3) using continuous representations of the normal and uniform distributions and "PG" stands for policy gradient.

to the probability distribution of the sequence of states and actions generated by the stochastic process corresponding to π. The absorption time (effective horizon) N_Δ is implicitly determined by the trajectories. This probability distribution combines both Markov decision process transition probabilities and action choice probabilities corresponding to randomized decision rules.

To compute a gradient requires a single objective function as opposed to one for each $s \in S$. This is easily accomplished by adding an initial state distribution $\rho(s)$ so that the value of policy π becomes scalar-valued and represented by

$$v^\pi = \sum_{s \in S} \rho(s) v^\pi(s). \tag{11.57}$$

As a result of Theorem 6.3, under mild conditions there is a stationary deterministic policy that maximizes (11.57). But instead of focusing on deterministic policies, the methods herein operate within the larger family of randomized stationary policies $d^\infty \in D^{\mathrm{MR}}$ which choose actions a in state $s \in S \setminus \Delta$ according to distribution $w_d(a|s)$.

As above, let $\mathbf{b}(s,a)$ denote a state and action-dependent feature vector with corresponding $\boldsymbol{\beta}$ and let $w(a|s;\boldsymbol{\beta})$ denote the corresponding parameterized action-choice probability distribution. Let $d_{\boldsymbol{\beta}}$ represent the decision rule that chooses actions according $w(a|s;\boldsymbol{\beta})$. That is $w_{d_{\boldsymbol{\beta}}}(a|s) := w(a|s;\boldsymbol{\beta})$.

Hence, the (scalar) objective becomes that of finding a weight vector $\boldsymbol{\beta}$ that maximizes

$$v(\boldsymbol{\beta}) := \sum_{s \in S} \rho(s) v^{(d_{\boldsymbol{\beta}})^\infty}(s) \tag{11.58}$$

or equivalently, a

$$\hat{\boldsymbol{\beta}} \in \arg\max_{\boldsymbol{\beta}} v(\boldsymbol{\beta}). \tag{11.59}$$

11.5.3 The gradient of the policy value function for an undiscounted infinite horizon Markov decision process

Chapter 4 provides details regarding the evaluation of an expression similar to $v(\boldsymbol{\beta})$ in terms of transition probabilities, action choice probabilities and rewards. It shows that

$$v(\boldsymbol{\beta}) = \sum_{s^1 \in S} \sum_{a^1 \in A_{s^1}} \sum_{s^2 \in S} \rho(s^1) w(a^1|s^1;\boldsymbol{\beta}) p(s^2|s^1,a^1) \Big(r(s^1,a^1,s^2) + \sum_{a^2 \in A_{s^2}} \sum_{s^3 \in S} w(a^2|s^2;\boldsymbol{\beta}) p(s^3|s^2,a^2) \big(r(s^2,a^2,s^3) + \dots \big) \Big) \tag{11.60}$$

where eventually the rewards are zero after reaching an absorbing state in Δ.

Now consider the expected reward in the first period only

$$v_1(\boldsymbol{\beta}) := \sum_{s^1 \in S} \sum_{a^1 \in A_{s^1}} \sum_{s^2 \in S} \rho(s^1) w(a^1|s^1;\boldsymbol{\beta}) p(s^2|s^1,a^1) r(s^1,a^1,s^2).$$

In this case,

$$\nabla_{\boldsymbol{\beta}} v_1(\boldsymbol{\beta}) = \sum_{s^1 \in S} \sum_{a^1 \in A_{s^1}} \sum_{s^2 \in S} \nabla_{\boldsymbol{\beta}} \big(\rho(s^1) w(a^1|s^1;\boldsymbol{\beta}) p(s^2|s^1,a^1) r(s^1,a^1,s^2) \big). \tag{11.61}$$

Observe that this gradient involves both the initial state distribution and the transition probabilities and is not amenable (especially in later periods) to direct computation or simulation. Instead using (11.51) yields

$$\nabla_{\boldsymbol{\beta}} v_1(\boldsymbol{\beta}) = \sum_{s^1 \in S} \sum_{a^1 \in A_{s^1}} \sum_{s^2 \in S} \rho(s^1) w(a^1|s^1;\boldsymbol{\beta}) p(s^2|s^1,a^1) r(s^1,a^1,s^2) \times \nabla_{\boldsymbol{\beta}} \ln \big(\rho(s^1) w(a^1|s^1;\boldsymbol{\beta}) p(s^2|s^1,a^1) r(s^1,a^1,s^2) \big). \tag{11.62}$$

Since $\rho(s)$, $p(s^2|s^1,a^1)$ and $r(s^1,a^1,s^2)$ do not involve $\boldsymbol{\beta}$,

$$\begin{aligned} &\nabla_{\boldsymbol{\beta}} \ln \big(\rho(s^1) w(a^1|s^1;\boldsymbol{\beta}) p(s^2|s^1,a^1) r(s^1,a^1,s^2) \big) \\ &\quad = \nabla_{\boldsymbol{\beta}} \ln \big(\rho(s) \big) + \nabla_{\boldsymbol{\beta}} \ln \big(w(a^1|s^1;\boldsymbol{\beta}) \big) + \nabla_{\boldsymbol{\beta}} \ln \big(p(s^2|s^1,a^1) \big) + \nabla_{\boldsymbol{\beta}} \ln \big(r(s^1,a^1,s^2) \big) \\ &\quad = \nabla_{\boldsymbol{\beta}} \ln \big(w(a^1|s^1;\boldsymbol{\beta}) \big). \end{aligned}$$

Therefore, it follows from (11.62) that

$$\nabla_{\boldsymbol{\beta}} v_1(\boldsymbol{\beta}) = \sum_{s^1 \in S} \sum_{a^1 \in A_{s^1}} \sum_{s^2 \in S} \rho(s^1) w(a^1|s^1;\boldsymbol{\beta}) p(s^2|s^1,a^1) r(s^1,a^1,s^2) \nabla_{\boldsymbol{\beta}} \ln \big(w(a^1|s^1;\boldsymbol{\beta}) \big). \tag{11.63}$$

The significance of this expression is that it shows that $\nabla_{\boldsymbol{\beta}} v_1(\boldsymbol{\beta})$ can be evaluated by averaging $\nabla_{\boldsymbol{\beta}} \ln\big(w(a^1|s^1;\boldsymbol{\beta})\big) r(s^1, a^1, s^2)$ over replicates of (s^1, a^1, s^2) generated by the distribution of (X_1, Y_1, X_2). Note that a model is not necessary to evaluate this expression; realizations of the reward can replace $r(s^1, a^1, s^2)$.

The following result, which is proved in the appendix to this chapter, generalizes the above argument to complete trajectories. It provides the basis for estimating the gradient of the policy value function using a sampled trajectory.

Theorem 11.1. Suppose N_Δ is finite with probability 1 and that $w(a|s;\boldsymbol{\beta})$ is differentiable with respect to each component of $\boldsymbol{\beta}$. Then

$$\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = E^{(d_{\boldsymbol{\beta}})^\infty}\left[\sum_{j=1}^{N_\Delta} \nabla_{\boldsymbol{\beta}} \ln\big(w(Y_j|X_j;\boldsymbol{\beta})\big)\left(\sum_{i=j}^{N_\Delta} r(X_i, Y_i, X_{i+1})\right)\right]. \quad (11.64)$$

Observe that the multiplier of the gradient $\nabla_{\boldsymbol{\beta}} \ln\big(w(Y_j|X_j;\boldsymbol{\beta})\big)$ only involves rewards after decision epoch j. Moreover, subtracting a function $B(\cdot)$ that is independent of the action from the cumulative reward, referred to as a *baseline*, does not affect the gradient but often stabilizes calculations.

The quantity $\nabla_{\boldsymbol{\beta}} \ln(w(Y_j|X_j;\boldsymbol{\beta}))$ is often referred to as the *score function* and is a fundamental quantity in maximum likelihood estimation. In fact, if the summation of the rewards was replaced by a constant equal to one, the above expression would correspond to the gradient used when applying gradient ascent to obtain the maximum likelihood estimator of $\boldsymbol{\beta}$ for decision rule $d_{\boldsymbol{\beta}}$.

11.5.4 A simple policy gradient algorithm

The following algorithm, frequently referred to as *the policy gradient algorithm*, describes an early[24] and straightforward implementation of gradient ascent over the space of parameterized randomized policies.

Algorithm 11.11. Policy gradient for an episodic model

1. **Initialize:**
   (a) Specify $\boldsymbol{\beta}$, a learning rate sequence $\tau_n, n = 1, 2, \ldots$, and a fixed baseline $B(s)$ for all $s \in S$.
   (b) Specify the number of episodes K and set $k \leftarrow 1$.
2. **Iterate:** While $k \leq K$:

[24] This is sometimes referred to as *REINFORCE* in the reinforcement learning literature. It represents one of the first implementations of this approach.

(a) **Generate an episode:** Generate an episode using $w(a|s;\boldsymbol{\beta}^A)$ to generate actions. Save the sequence of states, actions and rewards

$$(s^1, a^1, s^2, r^1, \ldots, s^N, a^N, s^{N+1}, r^N),$$

where N denotes the termination epoch of the episode.

(b) $n \leftarrow 1$.

(c) While $n \leq N$:

i. **Update weights sequentially:**

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_k \nabla_{\boldsymbol{\beta}} \ln\left(w(a^n|s^n;\boldsymbol{\beta})\right)\left(\sum_{i=n}^{N} r^i - B(s^n)\right). \tag{11.65}$$

ii. $n \leftarrow n+1$.

(d) $k \leftarrow k+1$.

3. **Terminate:** Return $\boldsymbol{\beta}$.

This algorithm may be viewed as a simulated policy improvement algorithm. Starting with a policy defined by $\boldsymbol{\beta}$, it evaluates it by generating a single episode using this policy. Then, it improves the policy by adjusting the weights in the direction of the gradient of this policy in step 2(c).

The update can also be implemented in a single step by accumulating the terms of the gradient as in (11.64) prior to an update or even averaging the gradient over several trajectories. This variant is often referred to as a *batch policy gradient* algorithm and is the foundation for many effective algorithms. The highly successful *proximal policy optimization algorithm (PPO)* is a batch implementation of this algorithm that restricts how much policies can change at each policy update[25].

To implement this batch policy gradient step, replace the sequential update 2(c) by

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_k \sum_{n=1}^{N} \nabla_{\boldsymbol{\beta}} \ln\left(w(a^n|s^n;\boldsymbol{\beta})\right)\left(\sum_{i=n}^{N} r^i - B(s^n)\right). \tag{11.66}$$

Some further comments follow:

1. After generating an entire episode using $w(a|s;\boldsymbol{\beta})$, step 2(c) updates $\boldsymbol{\beta}$ by stepping backwards through the tail of cumulative rewards (adjusted by the baseline).

2. The above algorithm is on-policy. That is, it improves the same policy that generates the sequence of states, actions and rewards. An off-policy variant

[25] See Schulman et al. 2017.

would improve a policy using episodes generated by a different policy. *Importance sampling* methods adjust iterates appropriately.

3. Many implementations of this algorithm appear without a baseline, which is equivalent to setting $B(s) = 0$. Including $B(s)$ does not invalidate the gradient derivation in Theorem 11.1 since the baseline does not change the expectation in (11.64). Moreover choosing the baseline appropriately enhances convergence by reducing the variance of the estimates. Actor-critic methods below are based on choosing an appropriate baseline.

4. Expression (11.65) provides insight into the qualitative behavior of the policy gradient algorithm. When $\sum_{i=n}^{N} r^i - B(s^n)$ is positive, components of $\boldsymbol{\beta}$ that make action a^n more likely in state s^n will be increased.

   To illustrate this phenomenon, consider an implementation in which features are indicator functions of state-action pairs and $w(a|s;\boldsymbol{\beta})$ is the softmax function. Then

$$\frac{\partial}{\partial \beta_{s,a}} \ln\big(w(a^n|s^n;\boldsymbol{\beta})\big) = \begin{cases} 1 - w(a|s;\boldsymbol{\beta}) & \text{for } s = s^n,\ a = a^n \\ -w(a|s;\boldsymbol{\beta}) & \text{for } s = s^n,\ a \neq a^n \\ 0 & \text{for } s \neq s^n. \end{cases}$$

   Consequently, the only positive component of $\nabla_{\boldsymbol{\beta}} \ln\big(w(a|s^n;\boldsymbol{\beta})\big)$ corresponds to $a = a^n$. Hence, when $\sum_{i=n}^{N} r^i - B(s^n)$ is positive, applying (11.65) increases β_{s^n,a^n} and decreases $\beta_{s^n,a}$ for $a \neq a^n$. Moreover, for $s \neq s^n$, $\beta_{s,a}$ remains unchanged. Thus, the probability of choosing action a^n in state s^n increases for future episodes.

5. The expression $\nabla_{\boldsymbol{\beta}} \ln\big(w(a^n|s^n;\boldsymbol{\beta})\big)$ can be evaluated numerically, in closed form when $w(a|s;\boldsymbol{\beta})$ is a softmax function of features, or by back-propagation when $w(a|s;\boldsymbol{\beta})$ is a neural net.

6. The index for the learning rate can vary according to the replicate (as stated), or the total number of passes through (11.65).

7. Step 2(c) describes a direct application of gradient ascent. Enhanced versions are available[26].

8. The main cost in implementing this algorithm is generating trajectories in step 2(a). In complicated episodic models, N can be very large. *Experience replay* described below strategically stores and replaces past trajectories to reduce this cost. It is especially relevant in batch implementations.

[26] For example, the widely cited smoothed gradient descent algorithm Adam [Kingma and Ba, 2017].

11.5.5 An example

Consider the problem of optimal stopping in a reflecting random walk (Example 6.19). Let $S = \{-M, -(M-1), \ldots, M-1, M\}$ and $A_s = \{C, Q\}$ with C corresponding to continuing and Q corresponding to stopping (Q for quit). Stopping in state s yields a reward of $g(s)$, while continuing costs c and results in a transition to state s' according to

$$p(s'|s, a_1) = \begin{cases} p & \text{if } s' = s+1, s < N \text{ or } s' = s = M \\ 1-p & \text{if } s' = s-1, s > 1 \text{ or } s' = s = -M \\ 0 & \text{otherwise} \end{cases}$$

for $0 < p < 1$. Let Δ denote the stopped state.

This is a stochastic shortest[27] path model with the reward of the improper policy that continues in every state equal to $-\infty$. Under all other policies the system terminates in finite expected time so that it can be analyzed as an episodic model.

Illustrative calculations

Before providing computational results, the update in step 2(c) of Algorithm 11.11 is illustrated using a single hypothetical episode. Suppose a policy based on $\boldsymbol{\beta}$ generates an episode with two continuation actions followed by stopping at the third step. For concreteness, set the observed sequence to $5 \to 6 \to 7 \to \Delta$ and suppose $c = -2$ and $g(s) = s^2$. Then the trajectory of the episode would be

$$(s^1, a^1, r^1, s^2, a^2, r^2, s^3, a^3, r^3, s^4) = (5, C, -2, 6, C, -2, 7, Q, 49, \Delta).$$

The following steps illustrate the calculation in (11.65) with $B = 0$. Ignoring "artificial" transitions once Δ is reached, $\sigma^n := \sum_{i=n}^{N} r^i - B$ takes on the values

$$\sigma^3 = 49, \quad \sigma^2 = 47, \quad \sigma^1 = 45.$$

Now assume the features are linear in the state for each action so that

$$b(s) = \beta_{0,0} I_{\{C\}}(a) + \beta_{0,1} I_{\{Q\}}(a) + \beta_{1,0} s I_{\{C\}}(a) + \beta_{1,1} s I_{\{Q\}}(a)$$

and the softmax function $w(a|s;\boldsymbol{\beta})$ with $\eta = 1$ becomes

$$w(C|s;\boldsymbol{\beta}) = \frac{e^{\beta_{0,0}+\beta_{1,0}s}}{e^{\beta_{0,0}+\beta_{1,0}s} + e^{\beta_{0,1}+\beta_{1,1}s}}$$

and $w(Q|s;\boldsymbol{\beta}) = 1 - w(C|s;\boldsymbol{\beta})$. Some simple calculus establishes that the gradients are given by

$$\nabla_{\boldsymbol{\beta}} \ln\big(w(C|s;\boldsymbol{\beta})\big) = \begin{bmatrix} w(Q|s;\boldsymbol{\beta}) \\ -w(Q|s;\boldsymbol{\beta}) \\ s\,w(Q|s;\boldsymbol{\beta}) \\ -s\,w(Q|s;\boldsymbol{\beta}) \end{bmatrix} = w(Q|s;\boldsymbol{\beta}) \begin{bmatrix} 1 \\ -1 \\ s \\ -s \end{bmatrix}$$

[27] Since rewards are maximized, this is actually a *longest* path model.

and

$$\nabla_{\boldsymbol{\beta}} \ln\big(w(Q|s;\boldsymbol{\beta})\big) = w(C|s;\boldsymbol{\beta}) \begin{bmatrix} -1 \\ 1 \\ -s \\ s \end{bmatrix}$$

where $\boldsymbol{\beta} = (\beta_{0,0}, \beta_{0,1}, \beta_{1,0}, \beta_{1,1})$. Note that in examples such as this it is more convenient to represent the components of $\boldsymbol{\beta}$ in matrix form with columns corresponding to actions so that gradient can be represented in terms of a Jacobian matrix.

Thus, when $a = C$, (11.65) becomes

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_n \left(\sum_{i=n}^{N} r^i \right) w(Q|s;\boldsymbol{\beta}) \begin{bmatrix} 1 \\ -1 \\ s \\ -s \end{bmatrix}. \tag{11.67}$$

Observe that the only stochastic element in this expression is the term $\sum_{i=n}^{N} r^i$ so that the variability of this quantity affects the stability of the estimates of $\boldsymbol{\beta}$. Standardizing rewards can reduce this variability.

Suppose that $\boldsymbol{\beta} = (0, 0, 0, 0)$ and $\tau_n = 0.1$, then the iterates (numbered in the order they are computed) become

$$\boldsymbol{\beta}^1 = 0.1 \cdot 49 \cdot 0.5 \begin{bmatrix} 1 \\ -1 \\ 7 \\ -7 \end{bmatrix} = \begin{bmatrix} 2.45 \\ -2.45 \\ 17.15 \\ -17.15 \end{bmatrix}, \ \boldsymbol{\beta}^2 = \begin{bmatrix} 2.25 \\ -2.25 \\ 11.05 \\ -11.05 \end{bmatrix} \text{ and } \boldsymbol{\beta}^3 = \begin{bmatrix} 2.25 \\ -2.25 \\ 11.05 \\ -11.05 \end{bmatrix}.$$

Note that corresponding to the final iterate $\boldsymbol{\beta}^3$, the action choice probabilities in state 5 are $w(Q|5;\boldsymbol{\beta}^3) = 1$ and $w(C|5;\boldsymbol{\beta}^3) = 0$ and in state 0, $w(Q|0;\boldsymbol{\beta}^3) = 0.99$ and $w(C|0;\boldsymbol{\beta}^3) = 0.01$.

Alternatively, setting $B = 47$, which equals the mean of the σ^n, results in the sequence

$$\boldsymbol{\beta}^1 = \begin{bmatrix} -0.1 \\ 0.1 \\ -0.7 \\ 0.7 \end{bmatrix}, \ \boldsymbol{\beta}^2 = \begin{bmatrix} -0.1 \\ 0.1 \\ -0.7 \\ 0.7 \end{bmatrix} \text{ and } \boldsymbol{\beta}^3 = \begin{bmatrix} -0.3 \\ -0.3 \\ -1.7 \\ 1.7 \end{bmatrix}.$$

Although the $\boldsymbol{\beta}$ estimates have changed and have become smaller in magnitude, the corresponding-action selection probabilities in state 5 remain $w(Q|5;\boldsymbol{\beta}^3) = 1$ and $w(C|5;\boldsymbol{\beta}^3) = 0$, and in state 0, $w(Q|0;\boldsymbol{\beta}^3) = 0.65$ and $w(C|0;\boldsymbol{\beta}^3) = 0.35$. Thus, estimates of $\boldsymbol{\beta}$ and the corresponding probabilities are sensitive to the choice of baseline.

As an alternative, consider implementing the policy gradient update in a single (batch) step. To do this compute the gradient of $v(\boldsymbol{\beta})$ using (11.66) so that with the

baseline $B = 47$ and $\tau_k = 0.1$

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_k \nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + 0.1 \begin{bmatrix} -1.1 \\ 1.1 \\ -6.1 \\ 6.1 \end{bmatrix} = \begin{bmatrix} -0.11 \\ 0.11 \\ -0.61 \\ 0.61 \end{bmatrix}.$$

With the baseline $B = 0$, the updated $\boldsymbol{\beta}$ would be

$$\begin{bmatrix} 0.83 \\ -0.83 \\ 4.56 \\ -4.56 \end{bmatrix}.$$

Thus in both cases, the batch update generates smaller increments in $\boldsymbol{\beta}$ and might be more stable.

A numerical experiment

This section describes a more detailed numerical study that implements Algorithm 11.11 in the context of an optimal stopping problem on $S = \{-50, \ldots, 50\}$ and $g(s) = -s^2$. The objective is to maximize the expected total (undiscounted) reward averaged over the initial state distribution. With this choice for $g(s)$ the goal is stop as close to zero as possible, taking into account the expected cost to reach it.

Figure 11.10 displays the *optimal* value functions and stopping region for three choices of the continuation cost. The optimal value function was found to a high degree of accuracy using value iteration with $v^0(s) = -50^2$. This starting value is a lower bound on the optimal value function, guaranteeing monotone convergence of value iteration (see Chapter 6).

To apply the policy gradient algorithm above, first assume a tabular representation so that features have the form

$$b(s, a) = I_{\{s', a'\}}(s, a), \text{ for } s' \in S \text{ and } a' \in \{C, Q\}$$

so that

$$w(a'|s'; \boldsymbol{\beta}) = \frac{e^{\eta \beta_{s',a'}}}{\sum_{s=-50}^{50} (e^{\eta \beta_{s,C}} + e^{\eta \beta_{s,Q}})}, \tag{11.68}$$

where $\beta_{s',a'}$ is the coefficient corresponding to $b(s', a')$.

For this model, the gradient is available in closed form as:

$$\frac{\partial}{\partial \beta_{s,a}} \ln w(a'|s'; \boldsymbol{\beta}) = \begin{cases} \eta(1 - w(a|s'; \boldsymbol{\beta})) & (s, a) = (s', a') \\ -\eta w(a|s'; \boldsymbol{\beta}) & (s, a) = (s', a) \text{ and } a \neq a' \\ 0 & \text{otherwise.} \end{cases} \tag{11.69}$$

Thus, the only positive term in the gradient corresponds to the state-action pair that is being evaluated. Moreover, the factor η can be absorbed into the learning rate.

Algorithm 11.11 was implemented with the following specifications:

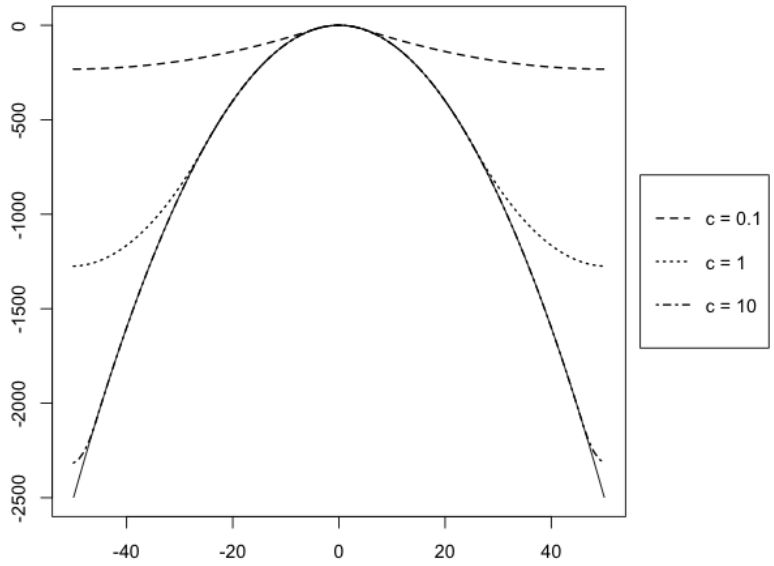


Figure 11.10: Optimal value functions (dashed lines) for three choices of c for the problem of optimally stopping in a random walk with $p = 0.5$ The solid line shows the reward of stopping in each state. The optimal policy is to stop when the optimal value function and the reward from stopping agree. Observe that the stopping region increases with respect to the continuation cost.

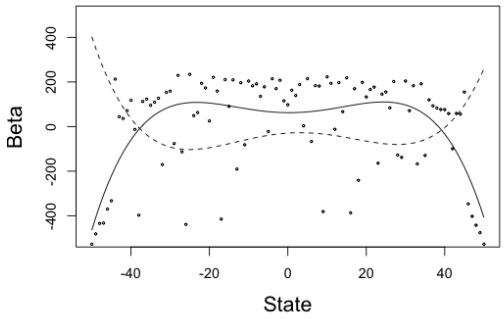
- Uniform initial distribution on S;
- $c \in \{0.1, 10, 50\}$;
- $p = 0.5$;
- $\boldsymbol{\beta}$ initialized to $\mathbf{0}$;
- $K = 500{,}000$;
- episodes were truncated after 100 steps;
- softmax exponent $\eta = 1/100$;
- learning rate, $\tau_k = 1{,}000/(10{,}000 + k)$, where k represented the episode number, and
- baseline $B(s) = -50^2$ for all s.

The baseline was chosen so as to avoid overflow in probability estimates.

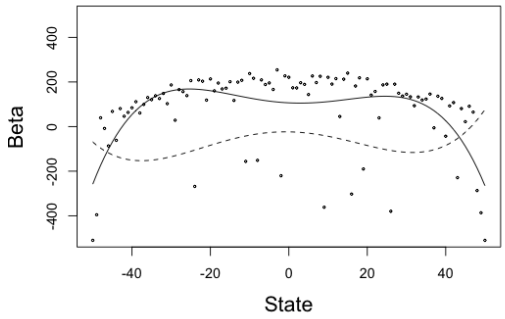
Figure 11.11 below shows estimates of $\boldsymbol{\beta}$ for the three choices of c for a single (typical) replicate and as well for a discounted model with $c = 0$ and the discount rate

$\lambda = 0.95$. The results obtained using a batch implementation of Algorithm 11.11 were similar.

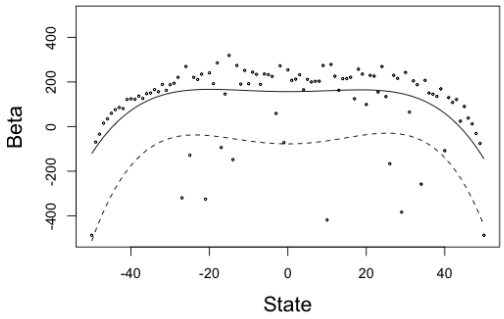
To better understand the trends within the components of $\boldsymbol{\beta}$ for each action, fourth-order polynomials were subsequently fit to the individual, often noisy, component values (represented by dots). The resulting lines, solid for Q and dashed for C, smooth out this variability, making the underlying patterns more readily interpretable.



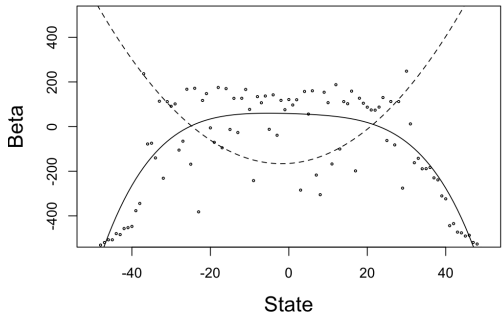
(a) $c = 0.1$ and $\lambda = 1$



(b) $c = 10$ and $\lambda = 1$



(c) $c = 50$ and $\lambda = 1$



(d) $c = 0$ and $\lambda = 0.95$

Figure 11.11: Parameter estimates obtained applying Algorithm 11.11 to optimal stopping on a random walk. The dots correspond to the estimates of $\beta_{s,Q}$, the solid line to the fit to these points using a fourth order polynomial and the dashed line indicates the fitted values to $\beta_{s,C}$ using a fourth order polynomial.

Note that when the solid line lies above the dashed line, it is more likely for the policy to stop[28]. Thus, these figures show that the stopping region is centered on $s = 0$ and increases with respect to c. When c is large, it is optimal to stop in all states, and when $c = 0.1$, and in the discounted model, the stopping region is narrower. Note that when $c = 0.1$, the policy found using the Algorithm 11.11 differs considerably from the optimal policy represented in Figure 11.10.

[28] Alternatively, the estimates can be plotted on the probability scale.

To conclude the analysis, consider an implementation that fits a fourth-order polynomial using centered and standardized states $\tilde{s}$ as features for each action. That is

$$w(a|s;\boldsymbol{\beta}) = \frac{e^{\eta\left(\sum_{i=0}^{4}\beta_{i,a}\tilde{s}^i\right)}}{\sum_{k=\{C,Q\}} e^{\eta\left(\sum_{i=0}^{4}\beta_{i,k}\tilde{s}^i\right)}}$$

for $a \in \{C, Q\}$. This model has 10 parameters as opposed to the tabular model above which has 202 parameters. It is convenient to represent $\boldsymbol{\beta}$ in matrix form as

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_{0,C} & \beta_{0,Q} \\ \beta_{1,C} & \beta_{1,Q} \\ \beta_{2,C} & \beta_{2,Q} \\ \beta_{3,C} & \beta_{3,Q} \\ \beta_{4,C} & \beta_{4,Q} \end{bmatrix} := \begin{bmatrix} \boldsymbol{\beta}_C & \boldsymbol{\beta}_Q \end{bmatrix}$$

Again, the gradient of $\ln w(a|s;\boldsymbol{\beta})$ is available in closed form with components

$$\frac{\partial}{\partial\beta_{i,j}} \ln w(C|s;\boldsymbol{\beta}) = \begin{cases} \eta\big(1 - w(C|s;\boldsymbol{\beta})\big)\tilde{s}^i & j = C \\ -\eta w(Q|s;\boldsymbol{\beta})\tilde{s}^i & j = Q \end{cases} \tag{11.70}$$

and

$$\frac{\partial}{\partial\beta_{i,j}} \ln w(Q|s;\boldsymbol{\beta}) = \begin{cases} -\eta w(C|s;\boldsymbol{\beta})\tilde{s}^i & j = C \\ \eta\big(1 - w(Q|s;\boldsymbol{\beta})\big)\tilde{s}^i & j = Q. \end{cases}$$

Note that $w(Q|s;\boldsymbol{\beta}) = 1 - w(C, s|\boldsymbol{\beta})$, so that the above expressions can be simplified further. They are left in the above form in order to easily generalize to cases with more than two actions.

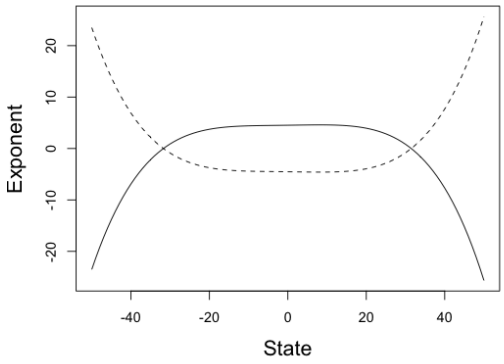
Figure 11.12 shows the fitted exponents as a function of the state using a fourth degree polynomial with $\eta = 1/100$ and $B(s) = -0.3(50)^2$. Observe that the stopping regions differ from the optimal in the case $c = 0.1$ and $\lambda = 1.0$, and also from those obtained using a polynomial fit applied to results from the tabular model. Note also that with these baseline and parameter choices, the estimates when $c = 10$ required a larger value of η in the softmax function to converge.

11.5.6 Policy gradient in a discounted model

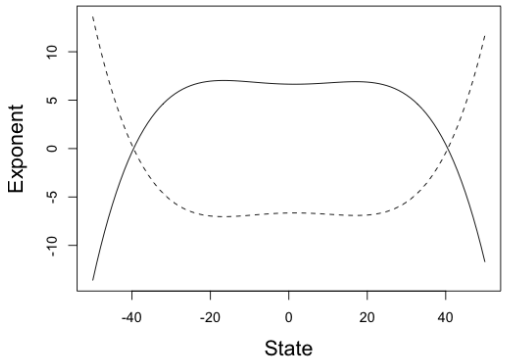
As noted frequently, an infinite horizon discounted model may be analyzed through simulation as either:

1. a random horizon expected total reward model where the horizon length is sampled from a geometric distribution with parameter λ, or

2. a finite horizon model in which the total discounted reward is truncated at a fixed decision epoch.



(a) $c = 0.1$ and $\lambda = 1$



(b) $c = 0$ and $\lambda = 0.95$

Figure 11.12: Exponent values of the softmax function as a function of the state, obtained by applying Algorithm 11.11 to the problem of optimal stopping on a random walk. The values are derived from estimates of the coefficients of a fourth-order polynomial for each action in a single replicate. The solid line corresponds to stopping and the dashed line to continuing.

Algorithm 11.11 applies directly to the first representation where the stopping time is either sampled before each episode or realized by sampling from a Bernoulli distribution at each decision epoch within an episode. To apply truncation, (11.65) must be modified to include the discount factor as follows:

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \tau_k \nabla_{\boldsymbol{\beta}} \ln \left(w(a^n | s^n; \boldsymbol{\beta}) \right) \left(\sum_{i=n}^{N} \lambda^{i-n} r^i - B(s^n) \right) \tag{11.71}$$

Otherwise the algorithm is applied directly.

A numerical example

This example illustrates the truncation approach applied to the frequently analyzed two-state model. Experiments used softmax action selection probabilities and tabular representations for the policy so that $\boldsymbol{\beta} = (\beta_{1,1}, \beta_{1,2}, \beta_{2,1}, \beta_{2,2})$, where the first subscript corresponds to the state and the second subscript corresponds to the action. The implementation used a discount rate $\lambda = 0.9$, $K = 1{,}000$ episodes, truncation at $N = 100$, learning rate $\tau_k = 50/(1{,}000 + n)$ where k denotes the episode number, and weights initialized at $\boldsymbol{\beta} = \mathbf{0}$. Experiments explored the impact of the softmax exponent and the baseline over varying random number seeds.

By choosing the parameter of the softmax function $\eta = 1/10$, the impact of baseline was negligible and the algorithm terminated with action selection probabilities close to 1 for the optimal policy across a wide range of random number seeds. For example,

for a specific random number seed

$$w(a_{1,2}|s_1;\boldsymbol{\beta}) = 0.988 \quad \text{and} \quad w(a_{2,2}|s_2;\boldsymbol{\beta}) = 0.999.$$

Using the softmax parameter $\eta = 1$ resulted in frequent convergence to a non-optimal policy.

11.5.7 Policy gradient: Concluding remarks

In the discrete domain numerical examples in this section, the policy gradient algorithm was extremely sensitive to algorithmic specification including: initialization, baseline specification, state and reward scaling, softmax scaling, and learning rate.

Configurations that identified an optimal policy for one random number seed often converged to a non-optimal policy for another. These results are consistent with observations in the literature[29], which noted that:

- Experiments are difficult to reproduce because numerical results are sensitive to hyper-parameters and can vary over random number seeds.
- A major share of claimed performance increments is achieved less by innovative algorithmic properties but more through clever implementation.
- Results obtained using algorithms based on neural networks can be achieved by simpler models based on linear function approximations.

Therefore, we encourage the reader to experiment with algorithmic features when attempting to use the above policy gradient methods.

11.6 Actor-critic algorithms: Combining policy and policy value function approximation

The previous section argued that adding a baseline to the policy gradient algorithm reduces variance and improves convergence. Therefore, the expression

$$\delta(s^n) := \sum_{i=n}^{N} r^i - B(s^n) \tag{11.72}$$

in (11.65) in the episodic policy gradient algorithm needs to be looked at more closely. In this expression, N represents a realization of the episode length and $\sum_{i=n}^{N} r^i$ a realization of the total reward from decision epoch n onward starting in state[30] s^n and choosing action a^n.

[29] For instance Gronauer et al. [2021]

[30] Recall superscripts correspond to decision epochs.

In other words, the sum represents a realization of the state-action value function $q^{(d_{\boldsymbol{\beta}})^\infty}(s^n, a^n)$ under the randomized stationary policy $(d_{\boldsymbol{\beta}})^\infty$ resulting from using the weights $\boldsymbol{\beta}$. Realizing this, it has been shown[31] that $v^{(d_{\boldsymbol{\beta}})^\infty}(s^n)$, the expected value of the policy $(d_{\boldsymbol{\beta}})^\infty$ starting in state s^n, minimizes the variance of the gradient estimation for the specified policy. In other words, the optimal choice of the baseline is an appropriately specified policy value function.

To simplify notation, write $q(s, a; \boldsymbol{\beta}) := q^{(d_{\boldsymbol{\beta}})^\infty}(s, a)$ and $v(s; \boldsymbol{\beta}) := v^{(d_{\boldsymbol{\beta}})^\infty}(s)$. Then

- If $q(s, a; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta}) > 0$, it is desirable to increase the probability of selecting action a in state s.
- If $q(s, a; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta}) < 0$, it is desirable to decrease the probability of choosing action a in state s.

Thus, with this choice of baseline, *on average* the policy gradient algorithm will generate a new weight vector that corresponds to a policy with an increased value. The expression "on average" accounts for stochastic variation in the underlying simulation or process.

Advantage functions

Definition 11.1. The *advantage function*[a] is defined as

$$A(s, a; \boldsymbol{\beta}) := q(s, a; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta}) \tag{11.73}$$

for $s \in S$, $a \in A_s$ and real-valued vectors $\boldsymbol{\beta}$ of appropriate dimension.

[a] Hopefully, the use of A to represent the advantage function will not cause confusion. Recall that A_s denotes the set of actions in state s.

As argued above, the sign and magnitude of the advantage function impact the change in parameter values and consequently the probability of selecting the designated action. Since $q(s, a; \boldsymbol{\beta})$ and $v(s; \boldsymbol{\beta})$ are not known, the challenge is to incorporate their difference, namely the advantage function, into algorithms. Expressing algorithms in terms of the advantage function can be beneficial, as it may eliminate the need to compute its components separately.

Note that the gradient of the policy value function can be represented by

$$\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = E^{(d_{\boldsymbol{\beta}})^\infty}\left[\sum_{j=1}^{N_\Delta} \nabla_{\boldsymbol{\beta}} \ln\big(w(Y_j|X_j; \boldsymbol{\beta})\big) A(X_i, Y_i; \boldsymbol{\beta})\right]. \tag{11.74}$$

[31] See Sutton et al. [1999] or Grondman et al. [2012].

This representation takes into account that

$$E^{(d_{\boldsymbol{\beta}})^\infty}\left[\sum_{j=1}^{N_\Delta}\nabla_{\boldsymbol{\beta}}\ln\big(w(Y_j|X_j;\boldsymbol{\beta})\big)v(s;\boldsymbol{\beta})\right]=0.$$

Actor-critic algorithm overview

Algorithms that use policy value function estimates as baselines in a policy gradient algorithm are known as *actor-critic algorithms*. The *actor* corresponds to the policy and the *critic* to the value of that policy. Feedback from the critic enables the actor to improve the policy. Policy gradient algorithms may be referred to as actor-only algorithms and Q-learning algorithms as critic-only algorithms. The benefit of the combined actor-critic structure is that while Q-learning with function approximation may diverge, policy gradient algorithms are typically more stable and can converge to a local optimum under reasonable assumptions.

Actor-critic algorithms are based on approximating both the policy and advantage function (or its components) by weighted combinations of features. Let $\mathbf{b}_A(s,a)$ denote the vector of actor (randomized policy) features and $\boldsymbol{\beta}^A$ denote the corresponding vector of weights. Similarly, let $\mathbf{b}_C(s)$ denote the vector of critic (policy value function) features and $\boldsymbol{\beta}^C$ denote the corresponding vector of critic weights. Note that the form of features depend on whether value functions or state-action value functions are being used.

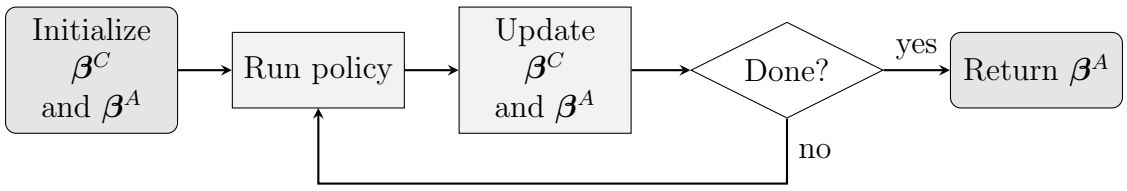


Figure 11.13: Steps in an actor-critic algorithm.

Figure 11.13 provides a high level schematic of an actor-critic algorithm. The algorithm can be implemented online or in batch mode. The former is suitable for continuing tasks modelled by an infinite horizon discounted (or average reward) model, while the latter is designed for episodic tasks (but can be used for discounted models by truncating rewards or including a geometric stopping time).

In an online implementation, "Run policy" generates a single transition after which both weight vectors are updated. In batch mode, "Run policy" generates a trajectory of states, actions and rewards using the current policy weights $\boldsymbol{\beta}^A$ and then both $\boldsymbol{\beta}^A$ and $\boldsymbol{\beta}^C$ are updated. Variants of the algorithm interchange the order of updating the weights as well as the method. The critic weights $\boldsymbol{\beta}^C$ can be updated using TD(0), TD(γ) or iterative least squares and the actor weights $\boldsymbol{\beta}^A$ can be updated using policy gradient with the baseline set equal to an estimate of the critic.

Note that an actor-critic algorithm avoids the need to explicitly evaluate an ϵ-tweaked policy in Q-policy iteration.

11.6.1 An online actor-critic algorithm

The online actor-critic algorithm is the easiest to describe. It applies to discounted infinite horizon applications. It is stated assuming a linear policy value function approximation

$$v(s;\boldsymbol{\beta}^C) = (\boldsymbol{\beta}^C)^{\mathsf{T}}\mathbf{b}_C(s).$$

Algorithm 11.12. Online actor-critic for a discounted model

1. **Initialize:**
   (a) Specify $\boldsymbol{\beta}^C$ and $\boldsymbol{\beta}^A$.
   (b) Specify learning rate sequences $\tau_n^C, n = 1, 2, \ldots$, and $\tau_n^A, n = 1, 2, \ldots$.
   (c) Specify the number of iterations N and $n \leftarrow 1$.
   (d) Specify $s \in S$.
2. **Iterate:** While $n \leq N$:
   (a) Sample a from $w(\cdot|s;\boldsymbol{\beta}^A)$.
   (b) Simulate (s', r) or sample $s' \in S$ from $p(\cdot|s, a)$ and set $r \leftarrow r(s, a, s')$.
   (c) **Evaluate advantage:**

$$A(s, a;\boldsymbol{\beta}^C) \leftarrow r + \lambda v(s';\boldsymbol{\beta}^C) - v(s;\boldsymbol{\beta}^C). \tag{11.75}$$

   (d) **Update critic:**

$$\boldsymbol{\beta}^C \leftarrow \boldsymbol{\beta}^C + \tau_n^C A(s, a;\boldsymbol{\beta}^C)\mathbf{b}_C(s). \tag{11.76}$$

   (e) **Update actor:**

$$\boldsymbol{\beta}^A \leftarrow \boldsymbol{\beta}^A + \tau_n^A \nabla_{\boldsymbol{\beta}^A} \ln\big(w(a|s;\boldsymbol{\beta}^A)\big) A(s, a;\boldsymbol{\beta}^C). \tag{11.77}$$

   (f) $n \leftarrow n + 1$ and $s \leftarrow s'$.
3. **Terminate:** Return $\boldsymbol{\beta}^A$.

Some comments about this algorithm follow:

1. As stated, the critic update uses TD(0). To see this, express (11.76) as:

$$\boldsymbol{\beta}^C \leftarrow \boldsymbol{\beta}^C + \tau_n^C \big(r + \lambda v(s'; \boldsymbol{\beta}^C) - v(s; \boldsymbol{\beta}^C)\big)\mathbf{b}_C(s).$$

   Alternatively, a TD(γ) update may be used in which case the critic update step will be more complex.

2. The advantage function (11.75) depends on the action a chosen in step 2(a).

3. Note that the same temporal difference (advantage) is used to update the actor and critic through (11.76) and (11.77). This means that the updated critic parameter is not used until the next iterate to update the actor. Using the updated critic in the same iteration has been explored in the literature and found to be less stable.

4. If a nonlinear policy value function approximation is used, $\nabla_{\boldsymbol{\beta}^C} v(s; \boldsymbol{\beta}^C)$ replaces the expression $\mathbf{b}_C(s)$ in (11.76).

5. The advantage estimates $q(s, a)$ with its sampled version $r + \lambda v(s'; \boldsymbol{\beta}^C)$. Alternatively, one can approximate $q(s, a; \boldsymbol{\beta}^C)$ and estimate its weights iteratively.

6. Many variants in the literature consider updates of the critic based on sampling states and rewards from parallel implementations.

Convergence of actor-critic

The following conditions[32] ensure convergence with probability one under linear function approximation:

1. **Slower evaluation of the actor:** To ensure that the critic reliably evaluates the actor, its learning rate should be faster than that of the actor. This occurs, for example, if

$$\lim_{n\to\infty} \frac{\tau_n^A}{\tau_n^C} = 0. \tag{11.78}$$

2. **Compatible features:** The critic's approximation of the advantage function must lie in the space spanned by the policy's score function. That is, the critic must approximate the advantage by:

$$\hat{A}(s, a) = \mathbf{u}^\top \nabla_{\boldsymbol{\beta}^A} \ln w(a|s; \boldsymbol{\beta}^A)$$

   for some $\mathbf{u}$. This ensures unbiased estimation of the policy gradient.

3. **The usual conditions:** Both learning rates must satisfy the Robbins-Monro conditions, features and rewards must be bounded, and all states visited infinitely often under the Markov chain of all policies.

[32] See Konda and Tsitsiklis [2000] and Sutton et al. [1999].

In practical applications, such as when the advantage is approximated by the temporal difference (11.75) or when using nonlinear function approximations such as neural networks, the compatibility assumption and others may be violated. Despite the lack of formal convergence guarantees in these settings, actor–critic algorithms often perform well empirically.

An example

This example describes in some detail the application of the online actor-critic algorithm to the frequently analyzed two-state example.

Example 11.6. This example applies Algorithm 11.12 to a discounted ($\lambda = 0.9$) version of the frequently analyzed two-state example. It uses a tabular representation with state-action indicators as features for the actor and state indicators as features for the critic. That is, the features for the critic are represented by the two-dimensional vector $\mathbf{b}_C(s)$ with components $b_C(s_i) = I_{s_i}(s)$ for $i = 1, 2$ and the features for the actor are represented by the four-dimensional vector $\mathbf{b}_A(s, j)$ with components $b_A(s_i, a_{i,j}) = I_{(s_i,a_{i,j})}(a, s)$ for $i = 1, 2$ and $j = 1, 2$.

This means that for $s \in S = \{s_1, s_2\}$

$$v(s_i; \boldsymbol{\beta}^C) = \beta^C_{s_i} \quad \text{for} \quad i = 1, 2,$$

where $\boldsymbol{\beta}^C = (\beta^C_{s_1}, \beta^C_{s_2})$.

To implement the above algorithm, sample a from $w(a|s; \boldsymbol{\beta}^A)$, observe (s', r) and compute the advantage function:

$$A(s, a; \boldsymbol{\beta}^C) = r + \lambda v(s'; \boldsymbol{\beta}^C) - v(s; \boldsymbol{\beta}^C) = r + \lambda \beta^C_{s'} - \beta^C_s.$$

Using this notation, the critic update equation (11.76) becomes

$$\beta^C_s = v(s; \boldsymbol{\beta}^C) \leftarrow v(s; \boldsymbol{\beta}^C) + \tau^C_n \big(r + \lambda v(s'; \boldsymbol{\beta}^C) - v(s; \boldsymbol{\beta}^C)\big) = \beta^C_s + \tau^C_n (r + \lambda \beta^C_{s'} - \beta^C_s).$$

The actor update equation (11.77) is given by

$$\boldsymbol{\beta}^A \leftarrow \boldsymbol{\beta}^A + \tau^A_n \nabla_{\boldsymbol{\beta}^A} \ln \big(w(a|s; \boldsymbol{\beta}^A)\big) A(s, a; \boldsymbol{\beta}^C).$$

This means that updates of the actor weights are given by

$$\beta^A_{s',a'} \leftarrow \begin{cases} \beta^A_{s,a} + \tau^A_n \big(1 - w(a|s; \boldsymbol{\beta}^A)\big) A(s, a; \boldsymbol{\beta}^C) & \text{if } s' = s, a' = a \\ \beta^A_{s,a} + \tau^A_n \big(- w(a|s; \boldsymbol{\beta}^A)\big) A(s, a; \boldsymbol{\beta}^C) & \text{if } s' = s, a' \neq a \\ \beta^A_{s,a} & \text{if } s' \neq s. \end{cases}$$

Note that at each iteration, the critic is updated only in the observed state s, while the actor weights are updated for all actions in state s.

The algorithm converged to the optimal value function and policy for a wide range of learning rates and numbers of iterations. Figure 11.14 shows the sequences of values for a typical replicate with $N = 30{,}000$ iterations and $\tau_n^A = 100/(1{,}000 + n^{0.6})$ and $\tau_n^C = 100/(1{,}000 + n)$ satisfying the time scale condition (11.78). In this replicate the estimated policy selected actions with probabilities:

$$w(a_{1,2}|s_1;\boldsymbol{\beta}^A) = 1.00 \quad \text{and} \quad w(a_{2,2}|s_2;\boldsymbol{\beta}^A) = 1.00,$$

in agreement with the deterministic optimal policy. The critic estimate of the optimal value function was $(29.97, 27.26)$ in close agreement with optimal values (see Figure 11.14).

It was also observed that the algorithm converged to the optimal policy for random number seeds in which the policy gradient algorithm (Algorithm 11.11) converged to a sub-optimal policy. Thus, by replacing an arbitrary baseline with a critic that tracked the "current policy", the algorithm avoided convergence to a sub-optimal policy.

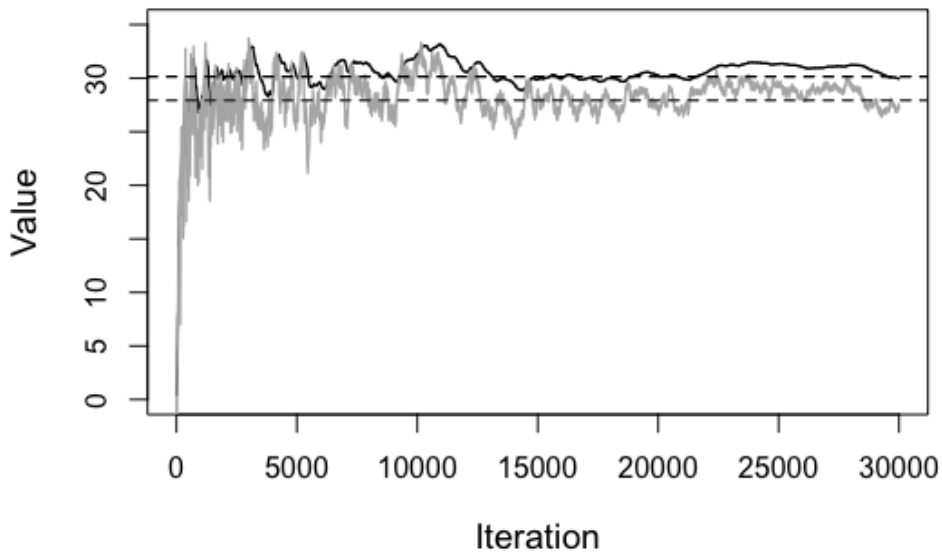


Figure 11.14: Optimal value function estimates in two-state example using Algorithm 11.12 obtained for a typical replicate. Black lines correspond to $v^*(s_1)$ and gray lines to $v^*(s_2)$; solid lines correspond to estimates and dashed lines to the exact value.

11.6.2 A batch actor-critic algorithm

The following algorithm describes an actor-critic implementation suitable for undiscounted episodic models. It generalizes the policy gradient algorithm above (Algorithm 11.11) by using the critic to update the baseline.

The algorithm as stated updates the actor and critic after every episode; the critic update (policy value function estimate) uses the whole episode, while the actor is updated sequentially after updating the critic. An alternative would be to also use a batch update of the actor represented by (11.66).

Algorithm 11.13. Batch actor-critic for an episodic model

1. **Initialize:**
   (a) Specify $\boldsymbol{\beta}^A$.
   (b) Specify learning rate sequence $\tau_n^A, n = 1, 2, \ldots$.
   (c) Specify the number of episodes K and set $k \leftarrow 1$.
2. **Iterate:** While $k \leq K$:
   (a) **Generate an episode:** Generate an episode using $w(a|s; \boldsymbol{\beta}^A)$ to generate actions. Save the sequence of states, actions and rewards
   $$(s^1, a^1, s^2, r^1, \ldots, s^N, a^N, s^{N+1}, r^N),$$
   where N denotes the termination epoch of the episode.
   (b) **Update critic:** Choose
   $$\boldsymbol{\beta}^C \in \arg\min_{\boldsymbol{\beta}} \sum_{n=1}^{N} \left(\sum_{i=n}^{N} r^i - v(s^n; \boldsymbol{\beta}) \right)^2. \tag{11.79}$$
   (c) $n \leftarrow 1$.
   (d) **Update actor:** While $n \leq N$:
      i. **Evaluate advantage:**
      $$A(s^n, a^n; \boldsymbol{\beta}^C) \leftarrow \sum_{i=n}^{N} r^i - v(s^n; \boldsymbol{\beta}^C). \tag{11.80}$$
      ii. **Update actor weights:**
      $$\boldsymbol{\beta}^A \leftarrow \boldsymbol{\beta}^A + \tau_n^A \nabla_{\boldsymbol{\beta}^A} \ln \left(w(a^n|s^n; \boldsymbol{\beta}^A) \right) A(s^n, a^n; \boldsymbol{\beta}^C) \tag{11.81}$$
      iii. $n \leftarrow n + 1$.
   (e) $k \leftarrow k + 1$.
3. **Terminate:** Return $\boldsymbol{\beta}^A$.

Some comments on the algorithm follow.

1. Critic weights $\boldsymbol{\beta}^C$ can also be updated after updating actor weights $\boldsymbol{\beta}^A$. Our experience, as illustrated by the example below, is that the algorithm converged more reliably when the critic was updated before updating the actor.

2. The above implementation does not explicitly describe how to estimate the critic weights. Alternatives include applying linear or nonlinear regression or alternatively to minimize
$$h(\boldsymbol{\beta}) := \sum_{n=1}^{N} \left(\sum_{i=n}^{N} r^i - v(s^n; \boldsymbol{\beta}) \right)^2$$
by gradient descent. To do so, an algorithm would compute
$$\nabla_{\boldsymbol{\beta}} h(\boldsymbol{\beta}) = \nabla_{\boldsymbol{\beta}} \left(\sum_{n=1}^{N} \left(\sum_{i=n}^{N} r^i - v(s^n; \boldsymbol{\beta}) \right)^2 \right),$$
which in the linear case equals
$$\nabla_{\boldsymbol{\beta}} h(\boldsymbol{\beta}) = -2 \sum_{n=1}^{N} \left(\sum_{k=n}^{N} r^k - v(s^n; \boldsymbol{\beta}) \right) \mathbf{b}^C(s^n),$$
where $\mathbf{b}^C(s)$ denotes the critic features evaluated at s. Absorbing the constant into the learning rate gives the recursion for $\boldsymbol{\beta}^C$:
$$\boldsymbol{\beta}^C \leftarrow \boldsymbol{\beta}^C - \tau_n^C \nabla_{\boldsymbol{\beta}} h(\boldsymbol{\beta}). \tag{11.82}$$
Note that this approach requires specifying an additional learning rate and monitoring the stability of gradient estimates.

3. The above algorithm uses the same advantage function as that used by the policy gradient algorithm. An alternative is to use the single-step (bootstrapped) advantage function:
$$A(s^n, a^n; \boldsymbol{\beta}^C) := r^n + v(s^{n+1}; \boldsymbol{\beta}^C) - v(s^n; \boldsymbol{\beta}^C) \tag{11.83}$$

4. Instead of using policy value functions, one could evaluate the advantage in terms of the state-action value function $q(s, a; \boldsymbol{\beta}^C)$ as follows
$$A(s^n, a^n; \boldsymbol{\beta}^C) := q(s^{n+1}, a^{n+1}; \boldsymbol{\beta}^C) - q(s^n, a^n; \boldsymbol{\beta}^C), \tag{11.84}$$
or even update it directly.

An example

This example applies Algorithm 11.13 to an instance of the shortest path model in Section 11.3.3 with $M = 10$, $N = 7$, $(m^*, n^*) = (10, 7)$, $R = 10$ and $c = 0.1$. In this application the robot seeks to learn a path to cell $(10, 7)$ from each starting cell. The implementation represents the critic by the model

$$v\big((m,n);\boldsymbol{\beta}^C\big) = \beta^C_0 + \beta^C_{1,0}m + \beta^C_{0,1}n + \beta^C_{1,1}mn + \beta^C_{2,0}m^2 + \beta^C_{0,2}n^2$$

so that $\boldsymbol{\beta}^C = (\beta^C_0, \beta^C_{1,0}, \beta^C_{0,1}, \beta^C_{1,1}, \beta^C_{2,0}, \beta^C_{0,2})$. Critic weights, $\boldsymbol{\beta}^C$, are computed in step 2(b) using ordinary least squares. The actor is represented in tabular form with $\boldsymbol{\beta}^A = \{\beta^A_{(m,n,a)} \,|\, (m,n) \in S \text{ and } a \in A_{(m,n)}\}$.

The algorithm was initiated with components of $\boldsymbol{\beta}^A$ sampled from a standard normal distribution so as to avoid too many long episodes. Episodes were truncated at 200 iterations and the algorithm was run for 50,000 episodes, each starting from a random cell that differed from the target. The learning rate was $\tau^A_n = 40/(400 + n)$.

Figure 11.15 shows how the total reward per episode varies within a single replicate starting in cell $(1, 1)$ and at random. Observe that the reward increases and stabilizes with more variability in total rewards when starting in a random cell than when starting in cell $(1, 1)$. The advantage of the random start is that it identifies good policies for infrequently visited states.

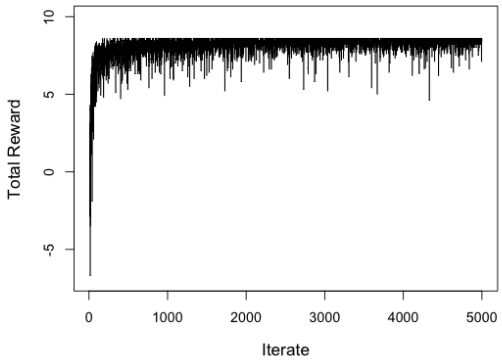
Note that when starting in cell $(1, 1)$, the actor-critic algorithm often finds a sequence of actions[33] with probabilities close to 1 that correspond to a shortest path through the grid. However, since most episodes will follow this path, the optimal probabilities off the path are not well estimated. Moreover, when the critic was evaluated *after* the actor, the algorithm converged to sub-optimal policies that cycled between states and never reached the target cell.

The policy gradient algorithm (Algorithm 11.11) was also applied to this example. The implementation was the same as above but instead of using the critic, it set the baseline (B in Table 11.7) equal to $0, -8$ or the (running) mean of the previous total rewards accumulated from the starting state. Observe from Table 11.7 that the quality of estimates varied significantly with the baseline, with the running mean giving the best results. In fact, the policy gradient algorithm with baseline equal to the running mean of total rewards gave a higher total reward than the actor-critic algorithm. However, there were many instances where the policy gradient diverged and the actor-critic algorithm converged to a good policy.

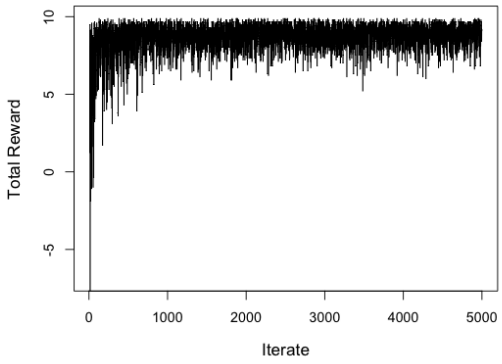
Delivering coffee

This section applies the actor-critic algorithm to find an optimal policy for the coffee delivering robot problem from Section 3.2. Rewards and costs are divided by 10 to

[33] For example, when starting in the upper left corner of the grid, the sequence: "down", "down", "down", "down", "down", "right", "down", "down", "down", "right", "right", "down", "right", "right", "right".



(a) All episodes start in cell $(1, 1)$.



(b) Episodes start in a random non-terminal cell.

Figure 11.15: A typical replicate of the total reward per episode in a model with a 10×7 grid with the target cell in lower right corner.

Method	Total Reward mean	Total Reward SD
PG ($B = 0$)	6.31	6.14
PG ($B = -8$)	5.80	8.06
PG ($B =$ mean)	8.52	0.47
AC	8.24	0.78

Table 11.7: Mean and standard deviation of total reward over all episodes for a single replicate in which all four methods converged. The results (using common random number seeds) were obtained using the policy gradient algorithm with baseline B and the actor-critic algorithm as described above.

improve numerical stability so that the per step cost is 0.1, the reward for delivering the coffee is 5 and the cost for falling down the stairs is 10. An episode ends when the robot successfully delivers the coffee; if it falls down the stairs, it returns to cell 13 and starts over. This example assumes that the robot's moves are deterministic in the sense that if it plans to go right and such a move is possible, it moves right. However, if there is a wall in the way, the robot stays in its current cell and incurs a cost associated with a single step. Randomness is introduced through $w(a|s; \boldsymbol{\beta}^A)$.

Algorithmic details follow. The policy value function is approximated by

$$v\big((m, n); \boldsymbol{\beta}^C\big) = \boldsymbol{\beta}^C_{0,0} + \beta^C_{1,0} m + \beta^C_{0,1} n,$$

where m denotes the row (indexed from the bottom) and n denotes the column (indexed from the left)[34]. Its parameters are estimated in step 2(b) of Algorithm 11.13 using

[34]For example, cell 13 corresponds to row 1 and column 1.

linear regression. The actor is again represented by indicators of state-action pairs and estimated (by gradient ascent) using a learning rate of $\tau_n^A = 40/(400+n)$ where n is the episode number. The model is run for 20,000 episodes with initial values for $\boldsymbol{\beta}^A$ generated randomly from a standard normal distribution.

This implementation identified an optimal policy for roughly half of the random number seeds. When it did not, it identified a policy that cycled without delivering the coffee. Note that for a few earlier episodes, the robot incurred a cost of 10 when it fell down the stairs but eventually it learned to avoid such incidents. For this model, the optimal value function estimate was

$$v\big((m,n);\hat{\boldsymbol{\beta}}^C\big) = 4.164 + 0.127m + 0.055n.$$

With this value function, the effect of moving one row closer to the destination increases the reward by more than twice as much as moving one column to the right.

A larger grid

The batch actor-critic algorithm was also applied to a variant of the above problem on a larger 10×5 grid in which cell $(1,1)$ corresponds to the upper left corner and $(10,5)$ to the lower right corner. An obstacle (stairs) occupies cells $(4,1)$ to $(8,1)$ and the target cell is $(10,1)$.

For an experiment with 30,000 episodes the critic was estimated to be

$$v\big((m,n);\hat{\boldsymbol{\beta}}^C\big) = 3.30 + 0.16m + 0.003n.$$

In comparison to the critic estimate above, moving down one row[35] was considerably more valuable than moving right. In most cells, especially along the policy identified by Algorithm 11.13 in Figure 11.16, one direction choice had probability close to one. This path was slightly longer than optimal but kept the robot at a safe distance from the obstacle and completed the task. Note that for other random number seeds, the policy sometimes moved the robot closer to the obstacle and consequently fell down the stairs more often.

The sequence of total rewards per episode is shown in Figure 11.17. Observe that in several episodes the robot fell down the stairs more than once, with this event occurring less frequently in later episodes. Moreover, estimating the critic weights by gradient descent failed to identify a policy that reliably reached the destination.

11.6.3 Actor-critic methods: Concluding remarks and enhancements

Actor-critic algorithms combine the key features of the methods in Chapters 10 and 11. Namely, they combine the policy evaluation features of temporal differencing with the

[35] So that coefficients are comparable, note that moving down in this formulation is equivalent to moving up in the previous formulation.

→	↓			
	↓			
	→	→	↓	
×			↓	
×			↓	
×			→	↓
×				↓
×			↓	←
			↓	
*	←	←	←	

Figure 11.16: Grid including policy identified by actor-critic algorithm for variant of the coffee delivering problem.

policy optimizing features of policy gradient methods. They provide a suite of elegant methods for solving problems using simulation.

Using a well-tuned critic reduces the variability observed in policy gradient methods and, by modifying the objective function, large swings in estimates of actor weights can be avoided. On the other hand, these algorithms contain many hyperparameters to tune and may limit exploration by using the current estimate of the actor for action selection.

Policy gradient and actor-critic algorithms have been researched extensively in the literature. The following variations have been suggested to enhance performance:

Normalization: Centering and scaling states and rewards, especially when they are highly variable) can enhance convergence.

Parallelization: Running many episodes in parallel (or on several identical robots) can provide better critic estimates.

Natural gradients: The gradients described above are sensitive to both the parameterization and the geometry (curvature) of the underlying surface. The natural gradient accounts for this in the actor by multiplying the gradient by the inverse of the Fisher information matrix given by

$$E\left[\Big(\nabla_{\boldsymbol{\beta}} \ln w(a|s;\boldsymbol{\beta})\Big)\Big(\nabla_{\boldsymbol{\beta}} \ln w(a|s;\boldsymbol{\beta})\Big)^{\mathsf{T}}\right].$$

Generalized advantage estimation: Generalized advantage estimation provides a TD(γ)-like approach for improved estimation of the advantage in actor-critic algorithms. It is based on taking weighted sums of expressions of the form:

$$r^n + \ldots + r^{n+m} + v(s^{n+m+1};\boldsymbol{\beta}^C) - v(s^n;\boldsymbol{\beta}^C).$$

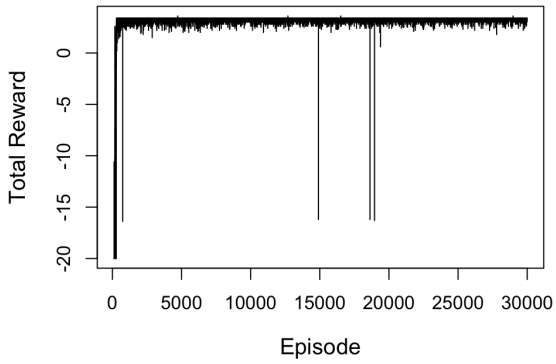


Figure 11.17: Total reward by iterate for a single replicate of the coffee delivering robot problem on a 10×5 grid obtained using Algorithm 11.13.

Improved gradient descent: Adaptive moment estimation[36] provides enhanced estimates of gradients based on moving averages of the mean and variance of the gradient. It is a widely used tool in gradient descent methods.

Policy and value function networks: Instead of using linear parameterizations for $w(a|s;\boldsymbol{\beta}^A)$ and $v(s;\boldsymbol{\beta}^C)$, one can replace them with neural networks using common or different features.

Thus, the analyst is faced with numerous options to improve the performance of policy gradient and actor-critic methods. However, achieving reliable results requires substantial experimentation and tuning. This part of the book has attempted to provide a foundational understanding of these methods and through several examples some guidance on how to apply them effectively.

11.7 Further topics

As the intent of this part of the book is to provide an introduction to reinforcement learning methods, it has been necessary to omit many topics. The following provides a brief overview of additional considerations.

11.7.1 Simultaneous learning and control

This book has frequently emphasized the distinction between model-based and model-free methods. Methods referred to as *model-based reinforcement learning* learn transition probabilities and rewards at the same time as they update a policy or value

[36] See Kingma and Ba (2017) for details regarding the algorithm Adam.

function or both. Sometimes the learned models are referred to as *world models*. In them, the transition probabilities may be estimated by parametric functions including neural networks.

These approaches have been applied extensively in robotics (such as the Brio labyrinth described in the introduction to Part III of this book), game playing (Minecraft, chess and Go) and autonomous driving, where accurate modeling of the environment and the ability to plan based on simulated experiences is critical for success.

Benefits of this approach are:

Data augmentation: Training data can be augmented by data simulated from the derived model. The benefit of using simulation in this setting is that regions of the state-action space that may not be observed in real-life can be evaluated through simulation. For example, in the Brio labyrinth, one can simulate episodes starting at later points in the maze that would be reached infrequently during the learning phase.

Deriving state-action value functions from value functions: Given the challenge of specifying a functional form for state-action value functions, having a model enables the analyst to derive q-functions from value functions according to

$$q(s,a) = \hat{r}(s,a) + \sum_{j \in S} \hat{p}(j|s,a)v(j),$$

where $\hat{r}(s,a)$ and $\hat{p}(j|s,a)$ represent the estimated rewards and transition functions. Of course, computing this summation may be problematic in large models. However, in most real examples (with Gridworld as a prototype), $\hat{p}(\cdot|s,a)$ will be positive for only a few states. Alternatively, this expectation can be estimated by simulation.

Many options are available for how to combine data from the estimated model with data from real experiences. Algorithms that implement this approach include Dyna (Sutton and Barto [2018]) and DreamerV3 (**?**). The latter algorithm was used to find solutions to the Brio Labyrinth.

11.7.2 Experience replay

Experience replay is an offline method developed to improve the efficiency and stability of learning algorithms. It applies to Q-learning, policy gradient and actor-critic algorithms. The key concepts of experience replay include:

Replay buffer: A replay buffer (or memory) stores observed data encountered by the agent during training in the form (state, action, reward, next state, done?). This buffer can hold a fixed number of experiences; less relevant older experiences are discarded as new ones are added.

Random sampling: Instead of using the most recent experience to update weights, experiences are randomly sampled from the replay buffer. This breaks the temporal correlation between consecutive experiences that arise when using long trajectories, leading to more stable and efficient learning.

Batch updating: The agent updates its policy or state-action value function using a batch of experiences sampled from the replay buffer, rather than a single experience. In the actor-critic environment this allows for more robust gradient estimates and reduces the variance of updates.

The main benefits of using experience replay are:

Efficient use of data: By reusing past experiences, the agent can learn more from the same amount of data. This is important when data is expensive to collect and/or episodes are long.

Breaking correlations: Experiences encountered in a sequential manner are often highly correlated, which can lead to poor training performance. Experience replay mitigates this issue by shuffling the data.

Implementation issues include the size of the buffer, the refresh rate and the sampling strategy.

11.7.3 Deep learning

Deep learning refers to the use of high-dimensional neural networks to represent value functions and policies. We have chosen *not* to explicitly apply them in the book because using them presents a unique set of challenges. Neural networks provide alternatives to the function approximators described in this chapter. However, the simple examples analyzed herein do not warrant such powerful techniques.

Most modern applications of reinforcement learning use large-scale neural networks to approximate value functions, state-action value functions and policies.

11.7.4 Importance sampling

Importance sampling is a statistical technique used to estimate properties (expectations) of a specific (target) distribution while sampling from a different distribution. It is particularly useful when direct sampling from the target distribution is difficult, computationally expensive or does not provide wide coverage of the sample space. Importance sampling is used extensively in Bayesian statistics, risk analysis where costly rare events occur infrequently, and Monte Carlo methods, as it uses data more efficiently and can reduce variance of the estimates when properly applied.

In reinforcement learning, importance sampling is commonly used for off-policy learning, that is when the agent learns about a *target policy* from data generated by a *behavioral policy*. This is especially relevant for off-policy updates in policy gradient

methods but can also be applied to policy improvement like variants of Q-learning using experience replay or function approximation.

General use

Importance sampling works as follows. Suppose one seeks to estimate the expected value of a real-valued function $f(\cdot)$ with respect to a target distribution $q(\cdot)$ but wants to do so using samples obtained from a different distribution $p(\cdot)$. Assuming both distributions are discrete with the same domain X,

$$E_q[f(X)] := \sum_{x \in X} f(x)q(x) = \sum_{x \in X} f(x)\frac{q(x)}{p(x)}p(x). \tag{11.85}$$

For this to make sense, it requires $p(x) > 0$ for all $x \in X$. As a result of (11.85),

$$E_q[f(X)] = E_p\left[f(X)\frac{q(X)}{p(X)}\right],$$

where the subscript on the expectation denotes the distribution used to compute the expectation. The quantities

$$\omega(x) := \frac{q(x)}{p(x)}$$

are referred to as *importance weights* or *importance ratios*.

Empirically, if $x^1, \ldots, x^N$ represent samples from $p(\cdot)$, an estimate of $E_q[f(X)]$ is given by

$$\widehat{E_q[f(X)]} = \frac{1}{N}\sum_{n=1}^{N} f(x^n)\frac{q(x^n)}{p(x^n)} = \frac{1}{N}\sum_{n=1}^{N} f(x^n)\omega(x^n).$$

Importance sampling in reinforcement learning

In reinforcement learning, importance sampling is used to evaluate or improve a target policy when using data obtained from implementing or simulating a behavioral policy. In particular, suppose actions are sampled from policy δ and one wishes to compute the expected reward in period 1 under policy γ starting in state s. Then

$$E^{\gamma}[r(X_1, Y_1, X_2)|X_1 = s] = E^{\delta}\left[r(X_1, Y_1, X_2)\frac{p(X_2|X_1, Y_1)w_{\gamma}(Y_1|X_1)}{p(X_2|X_1, Y_1)w_{\delta}(Y_1|X_1)}\middle| X_1 = s\right].$$

In this expression, the quantity

$$\omega(Y_1|s) := \frac{p(X_2|s, Y_1)w_{\gamma}(Y_1|s)}{p(X_2|s, Y_1)w_{\delta}(Y_1|s)} = \frac{w_{\gamma}(Y_1|s)}{w_{\delta}(Y_1|s)}$$

is the importance weight. It is noteworthy that the importance weight does not depend on the underlying transition probabilities that govern both policies.

Example 11.7. Consider a model with a single state and two actions $\{a_1, a_2\}$. Suppose the cost of choosing action a_1 is 10 and the cost of choosing action a_2 is 2. The objective is to estimate the expected cost under decision rule γ, which chooses action a_1 with probability 0.1. In a short realization, a_1 will be infrequently observed using γ.

Importance sampling can be used to estimate the cost under γ by sampling from a different policy, δ, which more frequently samples the high cost, low probability (important) action a_1. In this setting, γ is the target policy and δ is the behavioral policy.

For concreteness, assume δ selects actions with equal probability. Suppose in a sample of size 5 using δ one observes the sequence $(a_2, a_1, a_2, a_2, a_1)$ and corresponding rewards $(2, 10, 2, 2, 10)$. Define the importance weights $\omega(a_1) = 0.1/0.5 = 0.2$ and $\omega(a_2) = 0.9/0.5 = 1.8$. Then the estimated cost under decision rule γ is

$$\frac{1}{5}\big(2 \cdot 1.8 + 10 \cdot 0.2 + 2 \cdot 1.8 + 2 \cdot 1.8 + 10 \cdot 0.2\big) = 2.9$$

Note that this well approximates the true expected cost 2.8. One would expect that estimates based on samples from γ would have high variance, hence requiring larger samples to estimate its mean accurately.

Suppose now that one wished to evaluate $E^{\gamma^\infty}[r(X_n, Y_n, X_{n+1})|X_1 = s]$ for arbitrary n. Then by a similar argument to that above

$$E^{\gamma^\infty}[r(X_n, Y_n, X_{n+1})|X_1 = s] = E^{\delta^\infty}\left[r(X_n, Y_n, X_{n+1})\frac{w_\gamma(Y_1|X_1)}{w_\delta(Y_1|X_1)} \cdots \frac{w_\gamma(Y_n|X_n)}{w_\delta(Y_n|X_n)}\middle| X_1 = s\right].$$

Hence, by summing the expressions in the above equation, the value of policy γ^∞ can be calculated using trajectories generated by δ^∞.

Such a calculation is fundamental in policy evaluation, especially when generating trajectories is costly. Using trajectories from one policy enables estimating expected costs for other policies. This is especially applicable when using policy gradient methods but also can be used in the Q-learning context.

See Sutton and Barto [2018] for examples of calculations based on this approach. Recent research, for example, Thomas and Brunskill [2016], has extended importance sampling methods in many ways.

11.7.5 Monte Carlo tree search

Monte Carlo tree search (MCTS) is a simulation-based method for evaluating the likelihood of an outcome (usually *win* or *lose*) when a specific action is chosen in a given state. It does so by rolling out and evaluating trajectories through simulation. MCTS applies to episodic models such as games against an opponent (where the outcome may

be *win* or *lose*) and bandit models. MCTS underlies Google's AlphaGo and AlphaZero that obtained outstanding successes in Chess and Go.

The MCTS algorithm repeatedly executes the following steps:

1. **Select state:** Specify state to be evaluated.
2. **Select action:** Select an action on the basis of a current estimate of the probability of outcomes (payoffs or winning) for each action (or an upper confidence bound on this probability in multi-armed bandit models). Exploration can be introduced at this stage by sampling infrequently visited actions.
3. **Generate next state:** The action is implemented (in real life or a simulation) to obtain the next state. This may correspond to the opponent's move after choosing your move.
4. **Generate a sample path:** Play or simulate a game until an outcome occurs or a state in which the value is accurately estimated is encountered. This is sometimes referred to as a *rollout*.
5. **Update path values:** Step back through the sample path and update the number of times a state has been visited and the number of times the game was won or task successfully completed when that state was reached. This is sometimes referred to as *back-propagation*, but its meaning differs from that used in neural networks.

The result of applying the above algorithm is a state-action value function that gives the probability of winning or the expected reward associated with choosing an action in a state. Use of this function can guide future strategies.

When applied to models with large state and action spaces such as Chess and Go, approximations are required. Often, neural networks are used to represent the values and opponent's action choice probabilities.

Wordle

As an example of MCTS, consider the popular online game Wordle, which is available on the New York Times website.

The goal in Wordle is to guess an unknown target five-letter word in a minimum number of attempts by using feedback on the quality of the guess. Feedback is provided to the player by highlighting letters in the current guess as shown in Figure 11.18:

- Triangle, if the letter is in the same position as in the target word,
- Circle, if the letter is in the word but in the wrong position, and
- Nothing, if the letter is not in the word.

Figure 11.18: Typical Wordle output.

In Figure 11.18, the first guess "SALET" had no correct letters, the second guess "PRION" contained one correct letter "I" but in a different position than in the target word, and the fourth guess "WINDY" had two letters "I" and "Y" that were in the same position as in the target word. The keyboard below the guesses indicates the previously guessed letters with applicable highlighting (circle, triangle or no highlight) in accordance with their positions in the target word.

Note that the target word in this instance was "JIFFY". Data provided by Wordle after completing the game showed that there were only three remaining words after guessing "WINDY", namely "DIZZY", "DITZY" and "JIFFY". Moreover, the average number of steps to guess the target word over all players was 4.9. (Note this is a hard word to guess as the usual average is below 4.)

In the online version of the game, the hidden target word is chosen from a dictionary of 2,316 words and only 6 guesses are allowed. However, in modeling it, the game can proceed until a success is achieved.

This game can be analyzed using MCTS as follows. The state is rather complex; it encodes:

- Which letters have been guessed,
- For the guessed letters that are in the correct position in the target word, the exact positions they occupy, and
- For the guessed letters that are in the target word but not in the correct position, the positions previously guessed for those letters.

An action represents the word to guess, chosen from the set of not previously guessed words. Selection may be based on the distribution of the number of steps to find the

target word from each guess or the probability of eventually guessing the target word if a word is chosen.

After a guess, MCTS will generate sample paths of states and guesses starting from the current guess and ending when the target word is guessed. The value assigned to this state will be the number of guesses starting from the current state.

Clearly, this is non-trivial to implement and requires a more precise description of states and actions. Alternatively, Wordle can be solved using Q-learning.

11.7.6 Partially observable models

Reinforcement learning methods, especially policy gradient and actor-critic, apply directly to POMDPs. To use them, the observation, rather than the state, provides the input to the policy and/or a value function. That is, instead of using $w(a|s)$ to represent the policy and $v(s)$ to represent a policy value, one uses $w(a|o)$ and $v(o)$, where o denotes the observation. The consequence of doing so is that the action chosen by a policy is a function of an observation, rather than of the belief distribution over the unobservable states as analyzed in Chapter 8.

The benefit of this approach is that using observations simplifies the policy representation and learning process, as there is no need to update the belief state after each observation. However, policies based solely on observations might be less effective in environments where the history of observations is crucial for decision-making, as they lack explicit memory of past states and actions.

It would be informative to carry out a comprehensive study comparing these two approaches.

11.8 Technical appendix: Derivation of policy gradient representation

This appendix provides a proof of what is often referred to as the "policy gradient theorem". This fundamental equivalence underlies policy gradient and actor-critic methods. The proof below is given for finite horizon models. Discussion of extensions to infinite horizon models follows.

Theorem 11.2. Let N be finite and suppose $w(a|s;\boldsymbol{\beta})$ is differentiable with respect to each component of $\boldsymbol{\beta}$. Then

$$\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = E^{(d_\beta)^\infty}\left[\sum_{j=1}^{N} \nabla_{\boldsymbol{\beta}} \ln(w(Y_j|X_j;\boldsymbol{\beta}))\left(\sum_{i=j}^{N} r(X_i, Y_i, X_{i+1})\right)\right]. \quad (11.86)$$

The proof is an easy consequence of the following two results.

Lemma 11.1. Let a_i for $i = 1, \ldots, N$ and b_j for $j = 1, \ldots, N$ be real numbers. Then for N finite

$$\sum_{i=1}^{N} \left(\sum_{j=1}^{i} b_j \right) a_i = \sum_{j=1}^{N} b_j \left(\sum_{i=j}^{N} a_i \right). \tag{11.87}$$

This lemma and its proof for $N = \infty$ is Theorem 8.3 in Rudin [1964]. The following provides a representation for the expected value of a Markov decision process that receives a reward $r(s, a, j)$ *only* at the end of period n.

Lemma 11.2. Consider an n-period Markov decision process in which the reward function satisfies[a]

$$r_i(s, a, j) = \begin{cases} 0 & i < n \\ r(s, a, j) & i = n. \end{cases}$$

Let $d_{\boldsymbol{\beta}}$ denote a randomized decision rule that chooses actions in each decision epoch according to $w(a|s; \boldsymbol{\beta})$ and let $v_n(\boldsymbol{\beta})$ denote the expected total reward of $(d_{\boldsymbol{\beta}})^\infty$ averaged over initial state distribution $\rho(\cdot)$.

Then if $w(a|s; \boldsymbol{\beta})$ is differentiable with respect to each component of $\boldsymbol{\beta}$:

$$\nabla_{\boldsymbol{\beta}} v_n(\boldsymbol{\beta}) = E^{(d_{\boldsymbol{\beta}})^\infty} \left[\left(\sum_{j=1}^{n} \nabla_{\boldsymbol{\beta}} \ln(w(Y_j|X_j; \boldsymbol{\beta})) \right) r(X_n, Y_n, X_{n+1}) \right]. \tag{11.88}$$

[a] Recall that $r_i(s, a, j)$ denotes the reward in period i in a non-stationary Markov decision process.

Proof. From the definition of $v_n(\boldsymbol{\beta})$,

$$v_n(\boldsymbol{\beta}) := \sum_{h \in H_n} P_{\boldsymbol{\beta}}(h) r(s_n, a_n, s_{n+1}) = E^{(d_{\boldsymbol{\beta}})^\infty} \left[r(X_n, Y_n, X_{n+1}) \right], \tag{11.89}$$

where H_n denotes the set of all histories of the form $h = (s_1, a_1, \ldots, s_n, a_n, s_{n+1})$ and $P_{\boldsymbol{\beta}}(h)$ denotes the probability of history h where actions are chosen in each decision epoch according to $w(a|s; \boldsymbol{\beta})$. By the Markov property,

$$P_{\boldsymbol{\beta}}(h) = \rho(s_1) w(a_1|s_1; \boldsymbol{\beta}) p(s_2|s_1, a_1) \cdots w(a_n|s_n; \boldsymbol{\beta}) p(s_{n+1}|s_n, a_n). \tag{11.90}$$

Since $w(a|s; \boldsymbol{\beta})$ is differentiable with respect to each component of $\boldsymbol{\beta}$, using (11.51) establishes that

$$\nabla_{\boldsymbol{\beta}} P_{\boldsymbol{\beta}}(h) = P_{\boldsymbol{\beta}}(h) \nabla_{\boldsymbol{\beta}} \ln(P_{\boldsymbol{\beta}}(h)). \tag{11.91}$$

Therefore,

$$\nabla_{\boldsymbol{\beta}} v_n(\boldsymbol{\beta}) = \sum_{h \in H_n} P_{\boldsymbol{\beta}}(h) \nabla_{\boldsymbol{\beta}} \ln(P_{\boldsymbol{\beta}}(h)) r(s_n, a_n, s_{n+1}). \tag{11.92}$$

From (11.90)

$$\ln(P_{\boldsymbol{\beta}}(h)) = \ln(\rho(s_1)) + \ln(w(a_1|s_1;\boldsymbol{\beta})) + \ln(p(s_2|s_1,a_1)) \\ + \cdots + \ln(w(a_n|s_n;\boldsymbol{\beta})) + \ln(p(s_{n+1}|s_n,a_n)).$$

Since $\rho(s)$ and $p(j|s,a)$ do not depend on $\boldsymbol{\beta}$,

$$\nabla_{\boldsymbol{\beta}} \ln(P_{\boldsymbol{\beta}}(h)) = \sum_{j=1}^{n} \nabla_{\boldsymbol{\beta}} \ln(w(a_j|s_j;\boldsymbol{\beta})). \tag{11.93}$$

Hence, combining (11.92) and (11.93) gives

$$\nabla_{\boldsymbol{\beta}} v_n(\boldsymbol{\beta}) = \sum_{h \in H_n} P_{\boldsymbol{\beta}}(h) \left(\sum_{j=1}^{n} \nabla_{\boldsymbol{\beta}} \ln(w(a_j|s_j;\boldsymbol{\beta})) \right) r(s_n, a_n, s_{n+1}). \tag{11.94}$$

Rewriting this in expectation form gives (11.88). □

Proof of Theorem 11.2. It is easy to see that

$$\nabla_{\boldsymbol{\beta}} v(\boldsymbol{\beta}) = \sum_{n=1}^{N} \nabla_{\boldsymbol{\beta}} v_n(\boldsymbol{\beta}),$$

so that substituting the representation in (11.88) into the above result and applying Lemma 11.1, with $b_j = \ln w(Y_j|X_j;\boldsymbol{\beta})$ and $a_i = r(X_i, Y_i, X_{i+1})$, gives (11.86). □

Extensions*

The proof above establishes the policy gradient theorem for finite N. The same proof applies for a random stopping time N that satisfies $P(N < \infty) = 1$; the result follows by applying the same proof to each sample path. Regarding an infinite horizon model as a model with random geometric stopping times allows extension to the infinite horizon case.

To directly analyze infinite horizon discounted models requires additional assumptions. Specifically Lemma 11.1 must apply with $N = \infty$. For this to hold, the series in Lemma 11.1 must converge absolutely. Moreover such an analysis requires the validity of interchanging the order of the gradient, expectation and infinite summation. The conditions $\lambda < 1$ and bounded rewards are sufficient for both requirements.

Bibliographic remarks

This chapter has merely touched the surface of the vast literature on reinforcement learning with function approximation, including value-based, policy-based, and actor-critic methods.

Function approximation has played a central role in scaling reinforcement learning to large or continuous state-action spaces. The seminal work of Sutton [1988] introduced temporal-difference learning with linear approximators. Tsitsiklis and Roy [1997] provided a rigorous theoretical foundation for TD methods, establishing convergence results that inspired much subsequent research. The instability of off-policy learning, notably highlighted in Baird [1995], remains a core challenge. Stable alternatives such as least squares TD and policy iteration methods were proposed by Lagoudakis and Parr [2003] for linear approximations.

Policy-based methods trace back to the REINFORCE[37] algorithm of Williams [1992], which appears here as Algorithm 11.11. Sutton et al. [1999] later proved its convergence. These methods enable direct optimization in continuous action spaces and have evolved into robust frameworks. Notable extensions include Trust Region Policy Optimization (TRPO) Schulman et al. [2015] and Proximal Policy Optimization (PPO) Schulman et al. [2017], which improved sample efficiency and training stability.

The actor-critic framework was foreshadowed in the adaptive elements of Widrow et al. [1973], and formally developed in Barto et al. [1983], which shows how a system with *"two neuron-like elements can solve a difficult learning control problem"*. A survey of policy gradient methods is provided by Lehmann [2024].

The use of nonlinear function approximators, especially neural networks, gained momentum with the success of deep reinforcement learning. The Deep Q-Network (DQN) introduced by Mnih et al. [2015] represented a landmark, combining Q-learning with convolutional networks and stabilizing techniques such as experience replay and target networks.

Monte Carlo Tree Search (MCTS), used in AlphaGo and related systems, is closely related to the adaptive multistage sampling algorithm of Chang et al. [2005]. An accessible overview is provided in Fu [2018]. The game Wordle, used in this text to illustrate MCTS, was created by Wardle [2021].

Several books offer accessible and insightful introductions to the field, including Sutton and Barto [2018] and Kochenderfer et al. [2022] from a computer science perspective. Comprehensive theoretical treatments are found in Bertsekas and Tsitsiklis [1996], Szepesvari [2010], and Bertsekas [2019]. Historical perspectives on the evolution of reinforcement learning appear in Sutton and Barto [2018], Bertsekas and Tsitsiklis [1996], and Gosavi [2015]. Moreover, excellent lecture notes are available online; we especially recommend those by Levine [2024] on policy gradients and actor-critic methods, and by Katselis [2019] on Q-learning.

Exercises

You are encouraged to replicate the computational results in this chapter and as well to try out all methods herein on problems of personal or research interest. Chapter 3

[37] REINFORCE is an acronym for the expression "REward Increment = Non-negative Factor × Offset Reinforcement × Characteristic Eligibility".

provides a wide range of problems to analyze. In particular, the advanced appointment scheduling problem in Section 9.6 offers an excellent vehicle for testing these methods. Analyzing it with neural networks offers considerable research potential.

1. Specify first-visit and every-visit Monte Carlo approximation algorithms for estimating the value of a policy in an episodic model. Apply these algorithms to the shortest path Gridworld model in Section 11.3.3.

2. **Optimal stopping in a random walk (Example 11.1).**

   (a) Repeat the analysis in the example for other choices of $\bar{S}$ and K.

   (b) Obtain a linear value approximation (i.e., polynomial or piecewise-polynomial) for π_2 and π_3 when $p = 0.45, 0.48, 0.5, 0.52, 0.55$. Compare your analysis to that in the text.

   (c) Find suitable policy value function approximations when a linear fit is inadequate.

3. (a) Describe a Monte Carlo policy value function approximation algorithm for a discounted MDP based on geometric stopping. Recall that in this case, you accumulate the *undiscounted* total reward up to a geometric stopping time. This is in contrast to the version based on truncation, which accumulated the *discounted* reward.

   (b) Use this algorithm to approximate the policy value function in the two instances described in Example 11.2.

4. Fit a cubic spline with two knots to the queuing control model in and use Algorithm 11.4 to estimate its parameters. Why may it be problematic to use splines in Q-learning?

5. Provide explicit expressions for features when $f_i(s)$ and $h_j(a)$ are polynomials in s and a, respectively.

6. **Tabular models and function approximation.** Show that when features are indicators of state:

   (a) Algorithm 11.5 reduces to the tabular TD(γ) Algorithm 10.6.

   (b) Algorithm 11.6 reduces to the tabular Q-learning Algorithm 10.11.

   (c) Describe a tabular policy gradient algorithm.

7. **Queuing service rate control experiments.** Consider the discounted queuing service rate control model in Example 11.4. Use wRMSE and policy structure to guide interpretation of results.

(a) Conduct an in-depth study of the model using Q-learning that varies model parameters, discount rate, learning and exploration parameters and data generation methods.

(b) Repeat your analysis using policy gradient and actor-critic algorithms.

8. **Gridworld experiments.** Consider the episodic shortest path model in Section 11.3.3.

   (a) Verify the conclusions in Section 11.3.3 by developing and implementing your own codes.

   (b) Repeat the calculations using a neural network and other parametric state-action value function approximations.

   (c) Implement a SARSA variant of Q-learning Algorithm 11.6 .

   (d) Solve the model using the Q-policy iteration Algorithm 11.7.

9. **State aggregation in optimal stopping.** Apply policy gradient and actor-critic to the optimal stopping model as follows. Let $N = 60$ and compare approximations that aggregate consecutive states into groups of $M = 1, 2$ or 4. For example when $M = 4$ the state space is partitioned into K groups of states of the form $G_1 := \{1, 2, 3, 4\}, G_2 := \{5, 6, 7, 8\}, \ldots, G_{15} = \{57, 58, 59, 60\}$. In this case features can be written as

$$b_{k,j}(s, a) = \begin{cases} 1 & s \in G_k,\ a = a_j \\ 0 & \text{otherwise} \end{cases}$$

   for $k = 1, \ldots, 15$ and $j = 1, 2$. These features can be represented as 30-component vectors with the first 15 components corresponding to a_1 and the next 15 components corresponding to a_2.

10. Derive the expression in (11.52).

11. Consider the shortest path through a grid analyzed in Section 11.6.2.

    (a) Replicate the analysis in the text using function approximations for both the actor and critic. See Section 11.3.3 for guidance on specification of basis functions.

    (b) Repeat the analysis assuming the cost per step $c(m, n)$ in cell (m, n) depends on the row m and column n. For example, suppose $c(m, n) = -0.1 - 0.1n$. How do you think modifying the costs in this way affects policy choice. (Note we found actor-critic and policy gradient frequently diverged or converged to sub-optimal policies for this modification.)

12. **Comparing methods for evaluating the policy gradient.** Consider a two-state model with $S = \{1, \Delta\}$ where Δ is a zero-reward absorbing state and the system starts in state 1 so that $\rho(1) = 1$. There are two actions to choose from in state 1, a_1 and a_2. For $i = 1, 2$, under action a_i, $p(1|1, a_i) = p_i$, $r(1, a_i, 1) = r_i$, and $r(1, a_i, \Delta) = 0$. Suppose the parameter vector is a scalar β and

$$w(a_1|\beta) = \frac{e^\beta}{e^\beta + 1}, \qquad w(a_2|\beta) = \frac{1}{e^\beta + 1}.$$

Compute $v_1(\beta)$ as a function of β. Then evaluate the gradient of $v_1(\beta)$ in the following two ways and compare results:

(a) Using the representation in (11.61).

(b) Using the policy gradient theorem result expressed as (11.63).

Each case requires enumerating trajectories and computing the probability of each as a function of β.

13. **Expectation of a baseline in the Policy Gradient Theorem.** Prove that if the baseline $B(s)$ is independent of the action a, the policy gradient satisfies

$$E^{(d_{\boldsymbol{\beta}})^\infty}\left[\sum_{j=1}^{N} \nabla_{\boldsymbol{\beta}} \ln\big(w(Y_j|X_j;\boldsymbol{\beta})\big) B(X_j)\right] = 0. \tag{11.95}$$

Consequently, subtracting a baseline in the policy gradient representation does not change its expectation but can reduce its variance.

14. **Partially observable models and reinforcement learning:** Consider the partially observable model in Example 3.

(a) Apply a policy gradient algorithm to a version of this problem in which policies are formulated as functions of *the observation* rather than belief states. Note that this is an episodic model that terminates when choosing a door.

(b) Develop and apply a policy gradient algorithm in which the policies are functions of belief states as in Chapter 8. Use it to find a good policy. This requires developing an approach for updating belief states and representing a continuous state variable.

(c) Compare the form, the information used, and the quality of policies obtained using both approaches. Which would you prefer?

15. **Importance sampling.** Perform the following experiment for the setting in Example 11.7. Simulate 100 replicates of samples of length five using decision rule γ and decision rule δ. Compute the mean and standard deviation of the sample means from γ and from the estimates of γ obtained by using importance sampling based on realizations of δ. What conclusions can be drawn?

16. **Monte Carlo tree search.** Apply MCTS to the Gridworld model in Figure 11.16.

17. **Comparing gradient and semi-gradient TD updates.** Consider a policy in an episodic Markov decision process with two non-terminal states, s_1 and s_2, and one terminal state, s_3. Under this policy, transitions are deterministic and the only reward is obtained upon entering the terminal state. That is:

    - $s_1 \to s_2$ with reward $r = 0$
    - $s_2 \to s_3$ with reward $r = 1$
    - s_3 is terminal

    The value function is approximated using a linear function approximator:

    $$v(s; \boldsymbol{\beta}) = \beta_1 x_1(s) + \beta_2 x_2(s)$$

    with features defined as:

    $$b(s_1) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad b(s_2) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad b(s_3) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

    For this example, do the following:

    (a) For a single transition $s \to s'$ with reward r, define the squared temporal difference error:
    $$g(\boldsymbol{\beta}) = \left(r + \lambda v(s'; \boldsymbol{\beta}) - v(s; \boldsymbol{\beta})\right)^2.$$
    Derive the full gradient $\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta})$ and the semi-gradient approximation used in TD learning.

    (b) For $\boldsymbol{\beta} = (0, 0)$, compute the following for the transition $s_1 \to s_2$:
    - $v(s_1; \boldsymbol{\beta})$, $v(s_2; \boldsymbol{\beta})$, the TD target, and TD error
    - The full gradient and semi-gradient update directions
    - The updated $\boldsymbol{\beta}$ after one gradient step with learning rate $\alpha = 0.1$

    (c) Repeat part (b) for the transition $s_2 \to s_3$ with reward $r = 1$.

    (d) Compare the results of full gradient and semi-gradient updates. Which update more effectively reduces the TD error? How does the difference between $b(s)$ and $b(s')$ affect this?

    (e) Modify the example so that transitions are random rather than deterministic and repeat your analysis.

    (e) Implement both the full gradient and semi-gradient TD updates in a programming language of your choice. Simulate several transitions and observe the learning behavior. Plot the squared TD error over time for both methods.

Appendix A

Notation and Conventions

This material will be published by Cambridge University Press as "Markov Decision Processes and Reinforcement Learning" by Martin L. Puterman and Timothy C. Y. Chan. This pre-publication version is free to view and download for personal use only. Not for re-distribution, re-sale, or use in derivative works. ©Martin L. Puterman and Timothy C. Y. Chan, 2025.

A.1 General math notation

- := for definition.
- $\mathbb{R}^N$ for N-dimensional Euclidean space. $\mathbb{R}^N_+$ restricts it to non-negative vectors.
- $\mathbb{Z}^N$ for N-dimensional integer lattice. $\mathbb{Z}^N_+$ restricts the lattice to non-negative integers.
- "sup" for supremum and "max" for maximum.
  - The supremum of a real-valued function $f(w)$ over a set W refers to the smallest upper bound of the elements in the set $\{f(w) \mid w \in W\}$. Importantly, the supremum need not be an element of that set.
  - The maximum denotes the largest element of a set and must be contained in that set.
  - As a convention, sup is used when a set is non-finite and max is reserved for finite sets. When it is shown that suprema are attained, sup is replaced by max.
  - inf and min are the analogues to sup and max, respectively, when refering to lower bounds.
- $\arg\max\{\}$ and $\arg\min\{\}$ refer to the set of arguments that maximize or minimize the expression in braces.

  - For a real-valued function $f(w)$ and a set W,

$$\arg\max_{w \in W} f(w) := \{w^* \in W \mid f(w^*) \geq f(w) \text{ for all } w \in W\}.$$

  - arg min is defined similarly as arg max, except with respect to minimization.
  - Note that arg max and arg min may return sets consisting of multiple elements. Only if there is a unique maximizer or minimizer do they return a single element.

- $u^+ = \max\{u, 0\}$, given a scalar u.
- $\mathbf{e}$ is a vector with all components equal to one.
- $\mathbf{e}_s$ is a vector with a 1 in the s-th component and zeroes elsewhere.
- $\mathbf{I}$ is the identity matrix.
- $\mathbf{0}$ is a vector or matrix with all components equal to zero.
- $\mathbf{x}^\mathsf{T}$ denotes the transpose of the vector $\mathbf{x}$. The same operator is applied to matrices.
- $\emptyset$ is the empty set.
- $|A|$ is the number of elements in a set A.
- $I_A(x)$ is the indicator function of the set A. It equals 1 if $x \in A$ and 0 otherwise.
- $\mathcal{S}_N$ is the unit simplex for N-dimensional vectors. That is,

$$\mathcal{S}_N := \left\{ (x_1, \ldots, x_N) \;\middle|\; \sum_{i=1}^{N} x_i = 1, x_i \geq 0, i = 1, \ldots, N \right\}.$$

- $o(1)$ is a generic expression for a function $f(n)$ on the non-negative integers, that converges to zero as $n \to \infty$. It is used when one is concerned about the limiting property of a function and not its particular form. More generally a function is $o(g)$ if $f(n)/g(n) \to 0$.
- A^B denotes the set of all functions mapping elements of B to elements of A. Equivalently, it can be viewed as the Cartesian product $A \times A \times \ldots \times A$ taken $|B|$ times.
- $A \setminus B$ denotes the set of elements in set A that are not in set B.
- $\sigma(\mathbf{B})$ denotes the spectral radius (largest eigenvalue in absolute value) of the square matrix $\mathbf{B}$.

- V denotes a vector space of real-valued vectors, usually of dimension $|S|$.
- $\nabla_{\boldsymbol{\beta}} g(\boldsymbol{\beta})$ denotes the gradient of a real-valued function $g(\boldsymbol{\beta})$ respect to the components of the vector $\boldsymbol{\beta}$.
- $\leftarrow$ denotes the assignment of value to a variable in an algorithm. $x \leftarrow 0$ means "x gets the value of 0" or equivalently that "x equals 0".

A.2 Notation specific to MDPs

- $p^{\pi}(\cdot)$ is the probability of a sequence of states and actions under π
- $P^{\pi}[\cdot|\cdot]$ is the conditional probability of an event under policy π.
- $p_d(j|s)$ is the one-step conditional probability of a transition to state j beginning in state s and using Markovian decision rule d.
- $\mathbf{P}_d$ is the transition probability matrix corresponding to Markovian decision rule d.
- $p_d^n(j|s)$ is the n-step conditional probability of being in state j in n steps, beginning in state s and using Markovian decision rule d. It is the (s, j)-th component of the matrix $\mathbf{P}_d^n$.
- $\rho(s)$ is an initial state distribution for a Markov chain.
- $w_d(a|s)$ is the probability that randomized decision rule d chooses action a in state s. This notation may include a superscript n if the probability depends on the epoch n (e.g., in a finite horizon model).
- $\mathbf{r}_d$ is the vector of (expected) rewards corresponding to Markovian decision rule d.
- $E^{\pi}[\cdot]$ is the unconditional expectation of a random variable under policy π.
- $E^{\pi}[\cdot|\cdot]$ is the conditional expectation of a random variable under policy π.
- $\mathbf{v}$ is a $|S|$-dimensional real-valued vector representing or approximating a value function. Value functions are usually distinguished as policy value functions or optimal value functions.
- d^{∞} denotes the stationary policy that uses decision rule d at every decision epoch.
- $d_{\mathbf{v}}$ denotes a $\mathbf{v}$-greedy decision rule. A *greedy* decision rule maximizes $L_d\mathbf{v}$ (or similar expression) over some set of decision rules. A *greedy* policy is any stationary policy composed of greedy decision rules.

- "c-max" is the component-wise maximum. Applied to a set of vectors, each component is maximized independently of the others, meaning that the vector that achieves the maximum may be different for each component.
- arg c-max{} returns the set of arguments that achieve the c-max. In the MDP case, c-max is applied to the set of deterministic decision rules, corresponding to the Cartesian product of all actions in all states. Hence, arg c-max will return one of the decision rules in that set.
- L is the Bellman operator. Its form varies by optimality criterion.
- L_d is the operator that applies decision rule d for one period. This varies by optimality criterion.
- B is the operator on a value function $\mathbf{v}$ defined as $L\mathbf{v} - \mathbf{v}$.
- $x(s, a)$ are dual variables in the linear programming formulation of an MDP.
- $\underline{\mathbf{v}}$ is the minimum of all components of $\mathbf{v}$.
- $\overline{\mathbf{v}}$ is the maximum of all components of $\mathbf{v}$.
- $v(s; \boldsymbol{\beta})$ denotes an approximate value function in state s parameterized by weight vector $\boldsymbol{\beta}$.
- $q(s, a; \boldsymbol{\beta})$ denotes an approximate state-action value function when the state-action pair equals (s, a), parameterized by the weight vector $\boldsymbol{\beta}$.
- Δ is an absorbing state (or set of absorbing states) in a Markov chain.
- N_Δ is a random variable representing the time to reach absorbing state(s) Δ in an expected total reward model.

A.3 Conventions

- Random variables are denoted by capital letters and their values by lower case letters.
- Vectors and matrices are bolded, while their components are not bolded. A vector written as $\mathbf{x} = (x_1, \ldots, x_n)$ should be considered a column vector, unless it is transposed explicitly. Thus, $\mathbf{x}^\mathsf{T}$ is a row vector.
- Subscripts and superscripts are defined based on the context.
  - Subscripts denote epochs for value functions, state-action value functions, rewards and transition probabilities.

  - Superscripts denote epochs for realizations of states, actions and histories.
  - Subscripts denote different states, actions or histories, within the sets of states, actions or histories, respectively.
  - Superscripts on a value function denote a specific policy (π) or an optimal policy ($*$)

- Hat denotes approximation, typically applied to a value function or parameter vector.
- Square brackets are reserved for probability and expectation.

A.4 Abbreviations

- MDP: Markov decision process
- POMDP: Partially observable Markov decision process
- HR: History-dependent and randomized
- HD: History-dependent and deterministic
- MR: Markovian and randomized
- MD: Markovian and deterministic
- MSE: Mean squared error
- RMSE: Root mean squared error (the square root of the MSE)
- TD: Temporal differencing
- PI: Policy iteration
- MPI: Modified policy iteration
- VI: Value Iteration
- LP: Linear Program

Appendix B

Markov Chains

Markov chain theory underlies Markov decision process models, especially under the average reward and expected total reward criteria. Markov chains have a long history. We especially like the seminal book Kemeny and Snell [1960] for its transparent approach and innovative applications. It provides the basis for the vanishing discount approach developed by Blackwell [1962] to analyze average reward models. Chapter 4 of Gallagher [1996] also provides an insightful discussion of the use of eigenvalues and eigenvectors to explore limiting properties of powers of transition probability matrices and of the challenges faced when analyzing countable state Markov chains.

Markov chains are now widely applied and provide the basis for Google's PageRank algorithm, speech recognition software and many reinforcement learning applications.

B.1 What is a finite Markov chain?

Let S denote a finite set of states and let $\mathcal{X} = (X_n : n = 0, 1, 2, \ldots)$ denote a sequence of random variables with values in $S = \{s_1, \ldots, s_m\}$[1]. Then $\mathcal{X}$ is a *Markov chain* if

$$P(X_n = s^n | X_{n-1} = s^{n-1}, X_{n-2} = s^{n-2}, \ldots, X_0 = s^0) = P(X_n = s^n | X_{n-1} = s^{n-1}) \tag{B.1}$$

for all $n = 0, 1, \ldots$ and $s^k \in S$ for $k = 0, 1, \ldots$.

Equation (B.1) is known as the *Markov property*, which can be stated succinctly as *"the future conditional on the present is independent of the past"*.

A Markov chain $\mathcal{X}$ is *stationary* or *homogeneous* whenever $P(X_n = s^n | X_{n-1} = s^{n-1})$ is independent of n. In this case, define for $k = 0, 1, \ldots$, the *(one-step) transition probability*

$$p(j|s) := P(X_k = j | X_{k-1} = s) \tag{B.2}$$

and the *n-step transition probability* by

$$p^n(j|s) := P(X_{n+k} = j | X_k = s). \tag{B.3}$$

[1] Recall that subscripts are used to denote different states and superscripts to denote the state visited at a decision epoch.

We emphasize that this quantity does not equal $(p(j|s))^n$.

Let $\mathbf{P}$ denote the $|S| \times |S|$ matrix with (s, j)-th component $p(j|s)$. Using the law of total probability n-times shows that this matrix provides provides a convenient way of computing the n-step transition probabilities. It follows that (s, j)-th component of the matrix product $\mathbf{P}^n$ equals $p^n(j|s)$. Also, $\mathbf{P}^0 = \mathbf{I}$.

For $s \in S$ define the initial distribution $\rho^0(s) := \rho(s) := P(X_0 = s)$ and the unconditional distribution $\rho^n(s) := P(X_n = s)$. Then again by the law of total probability, the unconditional distribution

$$\rho^n(s) = \sum_{j \in S} \rho(j) p^n(s|j),$$

which in matrix-vector notation can be written as $\boldsymbol{\rho}^n = \boldsymbol{\rho}\mathbf{P}^n$.

Finally, for a real valued function[2] $r(\cdot)$ on S define the conditional expectation

$$E_s[r(X_n)] := E[r(X_n)|X_0 = s] = \sum_{j \in S} r(j) p^n(j|s),$$

which in matrix-vector notation can be written $E[r(X_n)|X_0 = s] = \mathbf{P}^n\mathbf{r}(s)$. Note that the expressions $\mathbf{P}^n$ and $\mathbf{P}^n\mathbf{r}$ will appear throughout this book when using matrices and vectors to analyze stationary policies because of the following important fact.

> A stationary policy d^∞ generates a Markov chain with transition probability matrix $\mathbf{P}_d$ and reward vector $\mathbf{r}_d$.

Consequently, given a stationary policy, each of the models in Chapter 3 provides an application of a Markov chain.

B.2 Classifying states

The limiting behavior of the Markov chain depends on relationships between its states. State j is said to be *accessible* from state s, written $s \rightsquigarrow j$, if $p^n(j|s) > 0$ for some $n \geq 0$. Otherwise j is *inaccessible* from S. If $s \rightsquigarrow j$ and $j \rightsquigarrow s$ then states j and s are said to *communicate*, which is written as $s \leftrightsquigarrow j$.

State $s \in S$ is said to be:

- *recurrent* if the time to return to state s is finite with probability one. This occurs if state s is accessible from all states that are accessible from s. That is, if $s \rightsquigarrow j$, then $j \rightsquigarrow s$.
- *absorbing* whenever $p(s|s) = 1$. This means that once the chain visits state s, it remains there forever.

[2] A Markov chain together with a reward function is often referred to as a *Markov reward process*.

- *transient* if the time to return to state s is finite with probability *less than* one, or equivalently. if there exists a state j for which $s \rightsquigarrow j$ but s is not accessible from j. That is after a transition to j, $p^n(s|j) = 0$ for $n = 0, 1, \ldots$.
- *periodic with period* m if the greatest common divisor of $\{n = 0, 1, \ldots \,|\, p^n(s|s) > 0\}$ is m.
- *aperiodic* if 1 is the only common divisor of $\{n = 0, 1, \ldots \,|\, p^n(s|s) > 0\}$ is 1.

Note that if s and j communicate, they will be classified in the same way. That is if $s \leftrightsquigarrow j$ and s is recurrent or periodic with period m, than j is recurrent or periodic with period m. Moreover starting in a recurrent state s, the expected number of returns to state s is infinite, while if s is transient, the expected number of returns to s is finite.

Example B.1. As a simple illustration of a periodic Markov chain, consider one with transition probability matrix

$$\mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ p & 0 & 1-p \\ 0 & 1 & 0 \end{bmatrix}. \tag{B.4}$$

where $S = \{s_1, s_2, s_3\}$ and $0 < p < 1$. Since

$$\mathbf{P}^2 = \begin{bmatrix} p & 0 & 1-p \\ 0 & 1 & 0 \\ p & 0 & 1-p \end{bmatrix} \tag{B.5}$$

it follows that $p^2(s_i|s_i) > 0$ for $i = 1, 2, 3$. For $n \geq 1$, $\mathbf{P}^{2n+1} = \mathbf{P}$ and $\mathbf{P}^{2(n+1)} = \mathbf{P}^2$ so that it follows that each state is periodic with period 2.

Observe that when $p = 0$ or $p = 1$ the chain behaves differently.

Periodicity represents an "edge case" that complicates many analyses and necessitates averaging to ensure convergence (see Section B.4).

B.3 Classes and class structure

Call a subset C of S *closed* if no state outside of C is accessible from any state in C. Moreover C is *irreducible* if no proper subset of C is closed. An irreducible closed set C that consists of a single element is said to be *absorbing*.

A Markov chain can be partitioned into closed irreducible subsets of states $C_1, \ldots, C_M$, in which all states in each C_i are recurrent, and a set of transient states T. If the Markov chain starts at a state in some C_k, it remains in C_k forever, however if it starts in T it

eventually leaves it and ends up in some C_k. Obviously when S is finite, the Markov chain contains at least one closed class.

Classification depends on the arrangement of 0 entries in $\mathbf{P}$. For example, consider the transition probability matrix with $S = \{s_1, s_2, s_3, s_4, s_5\}$,

$$\mathbf{P} = \begin{bmatrix} a & b & 0 & 0 & 0 \\ c & d & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & e & 0 & 0 & f \\ g & 0 & h & u & v \end{bmatrix},$$

where lower case letters denote non-zero probabilities with row sums equal to one. Consequently $C_1 = \{s_1, s_2\}$, $C_2 = \{s_3\}$ and $T = \{s_4, s_5\}$. Moreover s_3 is absorbing. Note that starting in state s_4, the system can either jump to C_1 in one step or remain in T for a few steps and then jump to either C_1 or C_2. Observe also the zeroes in rows 1-3 of columns 4 and 5.

A matrix partitioned as above is said to be in *canonical form*. Any transition matrix can be converted to the canonical form[3]

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_1 & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{P}_2 & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} \\ \cdot & \cdot & \cdot & & \cdot & \cdot \\ \cdot & \cdot & & \cdot & \cdot & \cdot \\ \mathbf{0} & \mathbf{0} & \dots & & \mathbf{P}_M & \mathbf{0} \\ \mathbf{Q}_1 & \mathbf{Q}_2 & \dots & & \mathbf{Q}_M & \mathbf{Q}_{M+1} \end{bmatrix},$$

where $\mathbf{P}_k$ corresponds transitions between states in C_k and Q_{M+1} to transitions between states in T.

B.3.1 Chain structure

Many results in this book, especially in Chapters 6 and 7, depend on the structure of Markov chains corresponding to stationary policies. A Markov chain on S is said to be:

- *regular*[4] if S is a single closed aperiodic class.
- *recurrent* or *ergodic* if it consists of a single closed class. A recurrent chain may be either periodic or aperiodic. When it is aperiodic, it is regular.
- *unichain* if it consists of a single closed class and a non-empty set T of transient states, and

[3] The Fox-Landi algorithm does this, see Section A.3 in Puterman [1994].

[4] Kemeny and Snell [1960] refer to a Markov chain as *regular* if $\mathbf{P}^N$ has all positive entries for some N. Clearly that is equivalent to our notion.

- *multi-chain* if it consists of two or more closed classes and possibly some transient states.

The analysis in the text will focus primarily on models in which all stationary policies correspond to Markov chains that are either recurrent or unichain. In the Markov decision process context, they will be referred to as recurrent or unichain models.

B.4 Limiting behavior

From a Markov decision process perspective, classification of the limiting behavior of $\mathbf{P}$ is extremely important. A sequence of matrices $\mathbf{A}_n$ converges to a matrix $\mathbf{A}$ if for each s and j in S, its components $a_n(s,j)$ converges to the components of $\mathbf{A}$, $a(s,j)$[5]. This is sometimes referred to as component-wise convergence[6].

The following important result regarding limiting behavior is stated without proof.

[5]When the matrices represent transition probability matrices, these terms will be written equivalently as $p_n(j|s)$ and $p(j|s)$, respectively.

[6]This is equivalent to convergence in norm when S is finite.

Theorem B.1. Let $\mathbf{P}$ be a transition probability matrix of a Markov chain on a finite state space S.

1. Then the limit
$$\mathbf{P}^* := \lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} \mathbf{P}^{n-1} \tag{B.6}$$
exists.

2. $\mathbf{P}^*$ is a transition probability matrix and satisfies
$$\mathbf{P}^*\mathbf{P} = \mathbf{P}\mathbf{P}^* = \mathbf{P}^*\mathbf{P}^* = \mathbf{P}^*. \tag{B.7}$$

3. When $\mathbf{P}$ is recurrent,
$$\mathbf{P}^* = \mathbf{e}\mathbf{q}^\mathsf{T} \tag{B.8}$$
where $\mathbf{q}$ satisfies $\mathbf{q}^\mathsf{T} = \mathbf{q}^\mathsf{T}\mathbf{P}$ subject to $\mathbf{q}^\mathsf{T}\mathbf{e} = 1$[a]. Moreover each entry of $\mathbf{q}$ is strictly positive.

4. If $s \in T$, then $p^*(s|j) = 0$ for all $j \in S$.

5. When $\mathbf{P}$ is regular,
$$\mathbf{P}^* = \lim_{N\to\infty} \mathbf{P}^n. \tag{B.9}$$

6. When $\mathbf{P}$ is regular, the convergence in (B.9) is exponentially fast. That is for each j and s in S, there exists constants $k > 0$ and $0 \le a < 1$ for which
$$|p^n(j|s) - p^*(j|s)| < ka^n. \tag{B.10}$$

[a] Recall that $\mathbf{e}$ denotes a column vector of onnes so that $\mathbf{q}^\mathsf{T}\mathbf{e}$ is a scalar and $\mathbf{e}\mathbf{q}^\mathsf{T}$ is a $|S| \times |S|$ matrix with all rows equal to $\mathbf{q}$.

Some comments on the significance of the above result follow:

1. The representation of $\mathbf{P}^*$ in (B.6) is particularly relevant in the Markov decision process setting. In this case $\sum_{n=0}^{N-1} \mathbf{P}^n\mathbf{r}$ denotes the expected total reward over N decision epochs so that $\mathbf{P}^*\mathbf{r}$ equals the *limiting average expected reward*.

2. As a consequence of (B.9), in a regular chain, $\mathbf{P}^*\mathbf{r}$ can be interpreted as the *steady state* reward. Note that in a periodic chain such as in (B.5), this limit does not exist. However the limit in (B.6) always exists (see Example B.2).

3. Part 3 provides an approach for computing $\mathbf{P}^*$ for a recurrent chain, that is by solving the system of equations
$$q(s) = \sum_{j\in S} q(j)p(s|j)$$

subject to $\sum_{j \in S} q(j) = 1$. In this case

$$\mathbf{P}^* = \begin{bmatrix} q(s_1) & q(s_2) & \dots & q(s_M) \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ q(s_1) & q(s_2) & \dots & q(s_M) \end{bmatrix}$$

The vector $\mathbf{q}$ is called the *stationary distribution* of the Markov chain.

4. The approach described in the previous comment applies to computing the stationary distribution and the components of $\mathbf{P}^*$ for any closed class.

5. Part 4 says that the long run average time spent in a transient state is zero. When the Markov chain is unichain, this means that for all $s \in S$

$$p^*(j|s) = \begin{cases} q(j) & j \in C \\ 0 & j \in T, \end{cases}$$

where $p^*(j|s)$, denotes the entries of the matrix $\mathbf{P}^*$.

6. Puterman [1994] p.593-594 describes an approach for computing $p^*(j|s)$ for $s \in T$ and j in any closed class.

7. Part 6 follows from Gallagher [1996], which shows that for a regular chain the second largest eigenvalue of $\mathbf{P}$ is strictly less than 1 and equals a.

Example B.2. This continues Example B.1. Since $\mathbf{P}$ is not regular, (B.9) does not hold. Using the result in part 3 of Theorem B.1 shows that:

$$\mathbf{P}^* = \begin{bmatrix} p/2 & 1/2 & (1-p)/2 \\ p/2 & 1/2 & (1-p)/2 \\ p/2 & 1/2 & (1-p)/2 \end{bmatrix}. \tag{B.11}$$

Observe that when $0 < p < 1$, all rows of $\mathbf{P}^*$ are equal since $\mathbf{P}$ consists of a single closed class. Moreover $\mathbf{P}^* = (\mathbf{P} + \mathbf{P}^2)/2$ consistent with its representation in part 1 and the observation that for $n \geq 1$, $\mathbf{P}^{2n+1} = \mathbf{P}$ and $\mathbf{P}^{2(n+1)} = \mathbf{P}^2$.

Note that when $p = 1$, states s_1 and s_2 form a recurrent class of period 2 and s_3 is transient. Similarly when $p = 0$, states s_2 and s_3 form a recurrent class of period 2 and s_1 is transient.

B.5 An important lemma

This section provides an important result regarding convergence of geometric series of matrices that is fundamental for Chapters 5-7. It holds in considerable generality and can be proved under a range of hypotheses, but the following finite dimensional version suffices for this book. Note the proof generalizes that used to derive the scalar identity $\sum_{n=0}^{\infty} a^n = (1-a)^{-1}$ when $|a| < 1$. The proof follows Kemeny and Snell [1960].

Lemma B.1. Let $\mathbf{Q}$ denote an $|S| \times |S|$ real-valued matrix for which $\mathbf{Q}^N \to \mathbf{0}$ as $N \to \infty$. Then the inverse of $\mathbf{I} - \mathbf{Q}$ exists and satisfies

$$\sum_{n=0}^{\infty} \mathbf{Q}^n = (\mathbf{I} - \mathbf{Q})^{-1}. \tag{B.12}$$

Proof. Let

$$\mathbf{S}_N := \sum_{n=0}^{N} \mathbf{Q}^n$$

Then it is easy to see that

$$(\mathbf{I} - \mathbf{Q})\mathbf{S}_{N-1} = \mathbf{I} - \mathbf{Q}^N.$$

Letting $N \to \infty$ and applying the hypothesis $\mathbf{Q}^N \to \mathbf{0}$ yields

$$(\mathbf{I} - \mathbf{Q}) \sum_{n=0}^{\infty} \mathbf{Q}^n = \mathbf{I},$$

from which the result follows. □

Since $\mathbf{Q}^n \to \mathbf{0}$ is equivalent[7] to the condition that the spectral radius (largest eigenvalue in absolute value) of $\mathbf{Q}$, $\sigma(\mathbf{Q})$, is strictly less than 1, it follows that:

Lemma B.2. The inverse of $\mathbf{I} - \mathbf{Q}$ exists and satisfies

$$\sum_{n=0}^{\infty} \mathbf{Q}^n = (\mathbf{I} - \mathbf{Q})^{-1} \tag{B.13}$$

if and only if $\sigma(\mathbf{Q}) < 1$.

Note that $\sigma(\mathbf{Q}) < 1$ even in some cases where some row sums of $\mathbf{Q}$ are greater than or equal to 1. Since $\sigma(\mathbf{Q}) \leq ||\mathbf{Q}||$ a sufficient condition for (B.12) to hold is that $\|\mathbf{Q}\| < 1$.

[7] This is proved in Faddeeva [1959] and other numerical linear algebra texts.

Lemma B.1 is applied in the book as follows:

1. In discounted models, $\mathbf{Q} = \lambda \mathbf{P}$ with $0 \leq \lambda < 1$.
2. In transient and stochastic shortest path expected total reward models, $\mathbf{Q}$ equals the sub-matrix of $\mathbf{P}$ restricted to its transient states.
3. In average reward models, $\mathbf{Q} = \mathbf{P} - \mathbf{P}^*$.

The following generalization, stated without proof, will be useful when $\mathbf{P}$ is periodic. The summation below and in (B.6) are referred to as *Cesaro summation*[8].

Lemma B.3. Let $\mathbf{Q}$ denote an $M \times M$ matrix for which $\lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} \mathbf{Q}^{N-1} \to \mathbf{0}$. Then

$$\lim_{N\to\infty} \frac{1}{N} \sum_{k=1}^{N} \left(\sum_{n=1}^{k} \mathbf{Q}^n \right) = (\mathbf{I} - \mathbf{Q})^{-1}. \tag{B.14}$$

B.6 The deviation matrix

The *deviation matrix*

$$\mathbf{H} := \sum_{n=0}^{\infty} (\mathbf{P}^n - \mathbf{P}^*)$$

is fundamental in the analysis of average reward models. It has the following closed form representation:

Proposition B.1. Let $\mathbf{H}$ be defined as above. Then if $\mathbf{P}^n \to \mathbf{P}^*$,

$$\mathbf{H} = (\mathbf{I} - (\mathbf{P} - \mathbf{P}^*))^{-1} - \mathbf{P}^* = (\mathbf{I} - (\mathbf{P} - \mathbf{P}^*))^{-1}(\mathbf{I} - \mathbf{P}^*). \tag{B.15}$$

Proof. Using equalities in (B.7), it is easy to establish directly or by induction that for $n \geq 1$, $\mathbf{P}^n - \mathbf{P}^* = (\mathbf{P} - \mathbf{P}^*)^n$.

[8] The *Cesaro summation* of the series $x_n, n = 0, 1, 2, \ldots$ equals

$$\lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N-1} x_n.$$

It often exists when $\lim_{n\to\infty} x_n$ does not. For example the non-convergent series $1, 0, 1, 0, \ldots$ has a *Cesaro limit* equal to $1/2$. In the Markov chain context, (component-wise) Cesaro summation is used to obtain limits in periodic chains.

Hence

$$\mathbf{H} = \sum_{n=0}^{\infty}(\mathbf{P}^n - \mathbf{P}^*) = (\mathbf{I} - \mathbf{P}^*) + \sum_{n=1}^{\infty}(\mathbf{P}^n - \mathbf{P}^*)$$
$$= \mathbf{I} + \sum_{n=1}^{\infty}(\mathbf{P} - \mathbf{P}^*)^n - \mathbf{P}^* = \sum_{n=0}^{\infty}(\mathbf{P} - \mathbf{P}^*)^n - \mathbf{P}^*.$$

Since $\mathbf{P}^n \to \mathbf{P}^*$, $(\mathbf{P}-\mathbf{P}^*)^n$ converges to zero so that the result follows from Lemma B.1. The equivalence in (B.15) follows from applying (B.7)[9]. □

Note that when a Markov chain is periodic or some recurrent class is periodic then the representation for $\mathbf{H}$ in (B.15) is still valid but the summation is in the Cesaro-sense. This means that

$$\lim_{N\to\infty} \frac{1}{N} \sum_{k=0}^{N-1} \sum_{i=0}^{k} \left(\mathbf{P}^k - \mathbf{P}^*\right) = (\mathbf{I} - (\mathbf{P} - \mathbf{P}^*))^{-1} - \mathbf{P}^*.$$

Note that in both cases, $\mathbf{H}$ has the following useful and easy to prove properties:

$$(\mathbf{I} - \mathbf{P})\mathbf{H} = \mathbf{H}(\mathbf{I} - \mathbf{P}) = \mathbf{I} - \mathbf{P}^* \tag{B.16}$$
$$\mathbf{H}\mathbf{P}^* = \mathbf{P}^*\mathbf{H} = \mathbf{0}. \tag{B.17}$$

B.7 Structure of $\mathbf{P}^*$ and $\mathbf{H}$

Proofs of convergence of average reward value iteration and policy iteration in Chapter 7 exploit the following properties of $\mathbf{P}^*$ and $\mathbf{H}$ in unichain Markov chains.

Let R denote the set of recurrent states and T denote the set of transient states of $\mathbf{P}$. Then $\mathbf{P}$ can be written as the partitioned matrix

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_{RR} & \mathbf{P}_{RT} \\ \mathbf{P}_{TR} & \mathbf{P}_{TT} \end{bmatrix}, \tag{B.18}$$

where the sub-matrix $\mathbf{P}_{RR}$ corresponds to transitions between recurrent states, $\mathbf{P}_{TR}$ corresponds to transitions from transient to recurrent states, $\mathbf{P}_{RT}$ corresponds to transitions from recurrent states to transient states and $\mathbf{P}_{TT}$ corresponds to transitions between transient states. Note that all entries of $\mathbf{P}_{RT}$ equal zero since by definition such transitions cannot occur. The dimensions of these sub-matrices are *conformable*, that is they depend on the number of recurrent and transient states. The matrices $\mathbf{I}$ and $\mathbf{0}$ will be conformable with the partition.

For ease of reference parts 3 and 4 of Theorem B.1 can be restated as follows.

[9] See Appendix A in Puterman [1994] or Blackwell [1962] for two very different proofs of this result.

Lemma B.4. Let $\mathbf{P}$ be unichain. Then

$$\mathbf{P}^* = \begin{bmatrix} \mathbf{Q} & \mathbf{0} \end{bmatrix}, \tag{B.19}$$

where $\mathbf{Q}$ denotes an $|S| \times |R|$ matrix with each row equal to $\mathbf{q}$ as defined in part 3 of Theorem B.1 and $\mathbf{0}$ denotes an $|S| \times |T|$ matrix of zeroes.

The following proposition describes an important property of $\mathbf{P}_{TT}$ that provides the basis for analysis of transient models in Chapter 6.

Proposition B.2. The matrix $\mathbf{I} - \mathbf{P}_{TT}$ is invertible and satisfies

$$(\mathbf{I} - \mathbf{P}_{TT})^{-1} = \sum_{n=0}^{\infty} \mathbf{P}_{TT}^n. \tag{B.20}$$

Proof. By the definition of transience, for each $s \in T$, a state in R is accessible from s in a finite number of transitions. That is there exists a $j \in R$ for which $p^n(j|s) > 0$ for some n. Since there are finitely many states (in T), there exists a $k \geq 1$ for which each row sum of $\mathbf{P}_{TT}^k$ is strictly less than one. Hence $\mathbf{P}_{TT}^n \to \mathbf{0}$ and the result follows from Lemma B.1. □

An immediate consequence of (B.20) is that the (s, j)-th component of the matrix $(\mathbf{I} - \mathbf{P}_{TT})^{-1}$ equals the expected total number of times the chain is in state $j \in T$ starting from $s \in T$[10].

The matrix $\mathbf{H}$ can be represented in partitioned form

$$\mathbf{H} = \begin{bmatrix} \mathbf{H}_{RR} & \mathbf{H}_{RT} \\ \mathbf{H}_{TR} & \mathbf{H}_{TT} \end{bmatrix}. \tag{B.21}$$

The following lemma describes useful properties of the sub-matrices of $\mathbf{H}$ in unichain models.

Lemma B.5. Let $\mathbf{P}$ be unichain. Then

- All entries of $\mathbf{H}_{RT}$ equal zero,
- $\mathbf{H}_{TT} = (\mathbf{I} - \mathbf{P}_{TT})^{-1}$, and
- $\mathbf{H}_{TT} \geq \mathbf{I}$.

[10] See Chapter III in Kemeny and Snell [1960].

Proof. Writing $\mathbf{Z} := \mathbf{I} - (\mathbf{P} - \mathbf{P}^*)$ in partitioned form gives

$$\mathbf{Z} = \begin{bmatrix} \mathbf{U} & \mathbf{0} \\ \mathbf{W} & \mathbf{I} - \mathbf{P}_{TT} \end{bmatrix},$$

where $\mathbf{I}$ denotes a $|T| \times |T|$ identity matrix and $\mathbf{0}$ denotes an $|R| \times |T|$ matrix of zeroes and $\mathbf{U}$ and $\mathbf{W}$ are specific matrices that will not be further examined. Standard formulae for inverting a partitioned matrix establish that

$$\mathbf{Z}^{-1} = \begin{bmatrix} \mathbf{X} & \mathbf{0} \\ \mathbf{Y} & (\mathbf{I} - \mathbf{P}_{TT})^{-1} \end{bmatrix},$$

where $\mathbf{X}$ and $\mathbf{Y}$ are specific matrices that are not of interest. From Lemma B.4

$$\mathbf{I} - \mathbf{P}^* = \begin{bmatrix} \mathbf{I} - \mathbf{P}^*_{RR} & \mathbf{0}_{RT} \\ -\mathbf{P}^*_{TR} & \mathbf{I}_{TT} \end{bmatrix},$$

where subscripts suggest the sub-matrix dimensions. Since $\mathbf{H} = \mathbf{Z}^{-1}(\mathbf{I} - \mathbf{P}^*)$, parts 1 and 2 follow. Part c follows from Proposition B.2 and the fact that the entries of $\mathbf{P}_{TT}$ are non-negative. □

B.8 Some examples

Example B.3. A two-state Markov chain.
Suppose $S = \{s_1, s_2\}$ and

$$\mathbf{P} = \begin{bmatrix} a & 1-a \\ 1-b & b \end{bmatrix}.$$

Limiting properties of P depend on the values of a and b as follows:

1. If $0 < a < 1$ and $0 < b < 1$, the Markov chain is regular and $\mathbf{P}^n \to \mathbf{P}^*$ where
$$\mathbf{P}^* = \begin{bmatrix} \frac{1-b}{2-a-b} & \frac{1-a}{2-a-b} \\ \\ \frac{1-b}{2-a-b} & \frac{1-a}{2-a-b} \end{bmatrix}.$$
2. If $a = 1$ and $0 < b < 1$, then the Markov chain is unichain, s_1 is absorbing, $\mathbf{P}^n \to \mathbf{P}^*$ and
$$\mathbf{P}^* = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix},$$
as noted in Lemma B.4.
3. When $a = b = 1$ the Markov chain is multi-chain and $\mathbf{P}^n = \mathbf{P}^*$ for all $n = 0, 1, \ldots$.

4. Suppose $a = b = 0$, the chain consists of single closed periodic class (with period 2), $\mathbf{P}^n$ does not converge to a limiting matrix, but from part 3 of Theorem B.1,

$$\mathbf{P}^* = \lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} \mathbf{P}^n = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & 1/2 \end{bmatrix}.$$

This means that the average time in each state is 1/2, which is obvious from the structure of $\mathbf{P}$.

It is left as an exercise to compute $\mathbf{H}$ for each of these cases.

Example B.4. A multi-state Markov chain.
This example analyzes a slightly more complicated Markov chain in which $S = \{s_1, \ldots, s_6\}$ and

$$\mathbf{P} = \begin{bmatrix} a & 1-a & 0 & 0 & 0 & 0 \\ 1-b & b & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ c & d & e & f & g & h \end{bmatrix}.$$

Assume $0 < a < 1$ and $0 < b < 1$. This multi-chain matrix is in canonical form and corresponds to closed classes $C_1 = \{s_1, s_2\}$, $C_2 = \{s_3, s_4, s_5\}$ and transient class $T = \{s_6\}$. Because C_2 is periodic, $\mathbf{P}^n$ does not converge, but $\mathbf{P}^*$ can be defined using (B.6).

Letting $w = (1-b)/(2-a-b)$, gives:

$$\mathbf{P}^* = \begin{bmatrix} w & 1-w & 0 & 0 & 0 & 0 \\ w & 1-w & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 1/3 & 1/3 & 0 \\ 0 & 0 & 1/3 & 1/3 & 1/3 & 0 \\ 0 & 0 & 1/3 & 1/3 & 1/3 & 0 \\ (\frac{c+d}{1-h})w & (\frac{c+d}{1-h})(1-w) & (\frac{e+f+g}{1-h})\frac{1}{3} & (\frac{e+f+g}{1-h})\frac{1}{3} & (\frac{e+f+g}{1-h})\frac{1}{3} & 0 \end{bmatrix}.$$

A general formula for deriving the last row appears on p. 594 of Puterman [1994] however in this example, it can be argued that starting in state s_6, the chain ends in C_1 with probability $(c+d)/(1-h)$ and ends in C_2 with probability $(e+f+g)/(1-h)$. Then the probability of being in a particular state in steady-state is the probability of ending in that state's class times the probability of being in that state given it is in that class in steady-state.

B.9 Eigenvalues and eigenvectors of a transition matrix*

Eigenvectors and eigenvalues provide insight into the limiting behavior of Markov chains. This section uses some basic linear algebra results[11].

The following key result will shed much insight into the rate at which a Markov chain converges to its limit.

Theorem B.2. Suppose the $m \times m$ matrix $\mathbf{P}$ has m independent eigenvectors. Then

$$\mathbf{P} = \mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1}, \tag{B.22}$$

where $\mathbf{W}$ is a matrix with columns equal to the right eigenvectors of $\mathbf{P}$ and $\mathbf{\Lambda}$ is a diagonal matrix with entries equal to eigenvalues of $\mathbf{P}$.
Moreover

$$\mathbf{P}^n = \mathbf{W}\mathbf{\Lambda}^n\mathbf{W}^{-1}. \tag{B.23}$$

The following example illustrates this result and its consequences using the two-state example.

Example B.5. Let

$$\mathbf{P} = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}.$$

Note that this matrix corresponds to the decision rule $d(s_1) = a_{1,1}$ and $d(s_2) = a_{2,2}$ in Example 2.2. Note that the Markov chain corresponding to $\mathbf{P}$ is regular.

It is easy to see that[a]

$$\mathbf{\Lambda} = \begin{bmatrix} 1.0 & 0 \\ 0 & 0.4 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 1.0 & 0.2 \\ 1.0 & -0.4 \end{bmatrix} \quad \text{and} \quad \mathbf{W}^{-1} = \begin{bmatrix} 2/3 & 1/3 \\ 5/3 & -5/3 \end{bmatrix}.$$

Expanding the right hand side of (B.22) gives

$$\mathbf{P} = 1 \begin{bmatrix} 2/3 & 1/3 \\ 2/3 & 1/3 \end{bmatrix} + 0.4 \begin{bmatrix} 1/3 & -1/3 \\ 2/3 & -2/3 \end{bmatrix}. \tag{B.24}$$

It follows from (B.23) that for $n \geq 1$

$$\mathbf{P}^n = 1^n \begin{bmatrix} 2/3 & 1/3 \\ 2/3 & 1/3 \end{bmatrix} + 0.4^n \begin{bmatrix} 1/3 & -1/3 \\ 2/3 & -2/3 \end{bmatrix}. \tag{B.25}$$

[11] For example see Strang [2023] and his brilliant online lectures.

Therefore

$$\mathbf{P}^n \to \mathbf{P}^* = \begin{bmatrix} 2/3 & 1/3 \\ \\ 2/3 & 1/3 \end{bmatrix} \tag{B.26}$$

and the convergence is at rate 0.4^n as noted in Theorem B.1.

[a]Recall that the sum of the eigenvalues equals the trace of a matrix and the product of eigenvalues equals its determinant.

Observe that in this example the eigenvalues of $\mathbf{P}$ are 1 and 0.4. This is an example of the following important result often referred to as the *Perron-Frobenius theorem*.

Theorem B.3. Let $\mathbf{P}$ be a transition probability matrix. Then

1. 1 is an eigenvalue of $\mathbf{P}$,
2. all eigenvalues of $\mathbf{P}$ are less than or equal to 1 in absolute value, and
3. the eigenvector corresponding to eigenvalue 1 has non-negative components.

Observe also that the stationary distribution $(2/3, 1/3)$ arises naturally in this example as the left eigenvector of $\mathbf{P}$ corresponding to the eigenvalue 1. Note that the right eigenvector of 1 equals $(1, 1)$. Note also that when $\mathbf{P}$ is regular, all eigenvalues other than 1 are strictly less than 1.

The following example analyzes the periodic model in Example B.3.

Example B.6. Let

$$\mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Then

$$\mathbf{\Lambda} = \begin{bmatrix} 1 & 0 \\ \\ 0 & -1 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 1 & 1 \\ \\ 1 & -1 \end{bmatrix} \quad \text{and} \quad \mathbf{W}^{-1} = \begin{bmatrix} 1/2 & 1/2 \\ \\ 1/2 & -1/2 \end{bmatrix}.$$

Since $\mathbf{\Lambda}^n$ does not converge, $\lim_{n\to\infty} \mathbf{P}^n$ does not exist. But

$$\lim_{n\to\infty} \frac{1}{n} \sum_{k=0}^{n-1} \mathbf{\Lambda}^n = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

so that from (B.6)

$$\mathbf{P}^* = \lim_{n\to\infty} \frac{1}{n} \sum_{k=0}^{n-1} \mathbf{P}^n = \mathbf{W} \left[\lim_{n\to\infty} \frac{1}{n} \sum_{k=0}^{n-1} \mathbf{\Lambda}^n \right] \mathbf{W}^{-1} = \begin{bmatrix} 1/2 & 1/2 \\ \\ 1/2 & 1/2 \end{bmatrix}.$$

The following result (stated without proof) is important for the analysis of models in Chapter 7.

Theorem B.4. If the Markov chain corresponding to the transition matrix $\mathbf{P}$ has k closed classes, then the eigenvalue 1 has multiplicity k and k independent eigenvectors.

A consequence of this theorem is that when a Markov chain is unichain or recurrent, the eigenvalue 1 has multiplicity 1 and (right) eigenvector $\mathbf{e} = (1, \ldots, 1)$.

B.10 Absorbing chains

Chapter 6 analyzes Markov decision process models in which Markov chains corresponding to stationary policies can be transformed into a model with $|S-1|$ transient states and 1 absorbing state. That is $\mathbf{P}$ has the form

$$\mathbf{P} = \begin{bmatrix} \mathbf{Q} & \mathbf{R} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix}, \tag{B.27}$$

where at least one component of the $(|S|-1) \times 1$ matrix $\mathbf{R}$ is positive. Thus by the above the theorem, 1 is an eigenvalue of $\mathbf{P}$ of multiplicity 1. Moreover the following important result follows directly from Lemma B.2.

Theorem B.5. Suppose $\mathbf{P}$ can be partitioned as in (B.27). Then the spectral radius of $\mathbf{Q}$ is strictly less than 1, $\mathbf{Q}^n \to \mathbf{0}$ and

$$(\mathbf{I} - \mathbf{Q})^{-1} = \sum_{n=1}^{\infty} \mathbf{Q}^{n-1}. \tag{B.28}$$

The following example illustrates this result.

Example B.7. Let $S = \{s_1, s_2, s_3\}$ and

$$\mathbf{P} = \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ 0.4 & 0.6 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

In this case $\mathbf{Q} = \begin{bmatrix} 0.5 & 0.3 \\ 0.4 & 0.6 \end{bmatrix}$ and $\mathbf{R} = \begin{bmatrix} 0.2 \\ 0 \end{bmatrix}$. The eigenvalues of $\mathbf{P}$ are $1, 0.9$ and 0.2 so that as a consequence of (B.23),

$$\mathbf{P}^* = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Moreover as stated in Theorem B.5 the eigenvalues of $\mathbf{Q}$ are 0.9 and 0.2 implying that $\mathbf{Q}^n \to \mathbf{0}$, so the $(\mathbf{I} - \mathbf{P})^{-1}$ exists. Direct computation shows that

$$(\mathbf{I} - \mathbf{Q})^{-1} = \begin{bmatrix} 5.00 & 3.75 \\ 5.00 & 6.25 \end{bmatrix}.$$

Note that the (s, j)-th component of $(\mathbf{I} - \mathbf{Q})^{-1}$ can be interpreted as the (finite) expected number of visits to state j. Hence, starting in s_1, the expected number of visits to s_1 equals 5 and the expected number of visits to s_2 equal 3.75 so on average starting in state s_1 the absorbing state will be reached after 8.75 transitions.

The following example provides another illustration of the consequences of Theorem B.4.

Example B.8. Consider the Markov chain with states $\{s_1, s_2, s_3, s_4\}$ and transition probability matrix

$$\mathbf{P} = \begin{bmatrix} 0.8 & 0.2 & 0 & 0 \\ 0.4 & 0.6 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0.2 & 0.2 & 0.4 & 0.2 \end{bmatrix}.$$

Then the Markov chain corresponding to this matrix has 2 closed classes $C_1 = \{s_1, s_2\}$ and $C_2 = \{s_3\}$ and a transient state s_4. Hence Theorem B.4 implies that the eigenvalue 1 has multiplicity 2.
Direct computation shows that $\mathbf{P}$ has 4 linearly independent eigenvectors so as

a result of Theorem B.2 it can be expressed as $\mathbf{P} = \mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1}$ where[a]

$$\mathbf{\Lambda} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0.4 & 0 \\ 0 & 0 & 0 & 0.2 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 1 & 0 & 0.2 & 0 \\ 1 & 0 & -0.4 & 0 \\ 0 & 1 & 0 & 0 \\ 0.5 & 0.5 & 0.2 & 1 \end{bmatrix},$$

$$\text{and } \mathbf{W}^{-1} = \begin{bmatrix} 0.667 & 0.333 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -1.667 & 1.667 & 0 & 0 \\ 0 & -0.5 & -0.375 & 1 \end{bmatrix}.$$

Letting $\mathbf{L} = \lim_{n\to\infty} \mathbf{\Lambda}^n$ yields:

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Hence $\mathbf{P}^* = \mathbf{W}\mathbf{L}\mathbf{W}^{-1}$ is given by:

$$\mathbf{P}^* = \begin{bmatrix} 0.667 & 0.333 & 0 & 0 \\ 0.667 & 0.333 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0.333 & 0.1667 & 0.5 & 0 \end{bmatrix}.$$

Note that starting in state s_4, the chain ends in C_1 or C_2 with probability 0.5 and then follows the limiting distribution for that class.

[a] To find the eigenvectors of $\mathbf{P}$ first find the eigenvectors for each closed class and adjust values in transient states to ensure that $\mathbf{P}\mathbf{v} = \lambda\mathbf{v}$ for all states. See Example B.5.

B.11 Countable state chains*

Although not used in this book, a brief discussion of some distinctions that arise when analyzing countable state Markov chains is included. Countable state chains arise naturally in queuing models in which the state represents the number of jobs in the queue. In most places in the book this issue has been avoided by truncating the state space.

This brief section points out some of the challenges that arise when analyzing countable state Markov chains as the following simple example shows.

Example B.9. This example shows that when S is countable, the limit

$$\lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} \mathbf{P}^n \tag{B.29}$$

exists but that its limit $\mathbf{P}^*$ need not be a transition probability matrix. Let $S = \{1, 2, \dots\}$ and consider a Markov chain with transition probabilities $p(s+1|s) = 1$, $p(j|s) = 0$ for $j \neq s+1$. That is at each transition the Markov chain moves to the right with probability one.

Clearly all states are transient, aperiodic and $\lim_{n\to\infty} \mathbf{P}^n$ exists so that the limit in (B.29) also exists. But the limiting matrix $\mathbf{P}^*$ with elements $p^*(j|s) = 0$ is not a transition probability matrix since all the mass "went off to infinity". Hence if there exists a non-zero reward $\mathbf{r}$, the long run average reward *cannot* be represented by $\mathbf{P}^*\mathbf{r}$.

Such a model is not typical of Markov decision process applications in queuing in which good policies (such as control limit policies in admission control models) result in a Markov chain that can be partitioned into a finite set of recurrent states and infinite set of transient states.

Note further that there are some distinctions with respect to the concepts of recurrence and transience, which are defined in terms of random variables describing transition behavior. Define ν_s to be the number of times a Markov chain visits state s. If s is transient $E_s[\nu_s] < \infty$ and if s is recurrent $E_s[\nu_s] = \infty$. However, recurrence can be further refined.

Let τ_s denote the time of the *first visit time to state* s[12]. For a recurrent state s $P(\tau_s < \infty | X_0 = s) = 1$ and if s is transient, $P(\tau_s < \infty | X_0 = s) < 1$. That is, starting from a recurrent state, the Markov chain returns to it in a finite time with probability one. Starting from a transient state, however, it does not necessarily return. This leads to a refinement on the concept of recurrent: a state s is *positive recurrent* if $E[\tau_s | X_0 = s] < \infty$ and *null recurrent* if $E[\tau_s | X_0 = s] = \infty$. Null recurrence is a curious phenomenon. It refers to a situation in which the chain returns to its starting state with certainty but the expected time to do so is infinite.

The important consequence of this is that in a transient or null recurrent class, the limiting probabilities are 0, while in a positive recurrent class, the limiting probabilities are non-zero. The following example adopted from Sennott [1999] (p.293-295) illustrates this distinction.

Example B.10. Consider a single server queue with batch arrivals and deterministic service rate of one job per period. Assume further that jobs are admitted

[12] If the chain starts in s, τ_s denotes the first return time.

only when the server is idle, which occurs when the queue is empty.
To model this let $S = \{0, 1, 2, ..\}$, $p(j|0) = q_j$ for $j \geq 1$, $p(s-1|s) = 1$ for $s \geq 1$ and $p(j|s) = 0$ otherwise. Thus each time the queue is empty, the system jumps to state j with probability q_j. It is left as an exercise to show that:

- All states are recurrent.
- If $\sum_{j=1}^{\infty} jq_j < \infty$, all states are positive recurrent.
- If $\sum_{j=1}^{\infty} jq_j = \infty$, all states are null recurrent.

Thus if the batch sizes are too large, the Markov chain is null recurrent.

Appendix C

Linear Programming

A linear program is an optimization problem that comprises an *objective function* and *constraints*, which are restricted to linear functions of the *decision variables*. Let $\mathbf{c} \in \mathbb{R}^n$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$. Consider the following linear program.

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} a_{ij} x_j \geq b_i, \quad \forall i = 1, \ldots, m. \end{aligned} \tag{C.1}$$

It has a linear objective function and constraints in the form of linear inequalities with respect to the decision variables $x_1, \ldots, x_n$. The *parameters* $c_j, j = 1, \ldots, n, a_{ij}, i = 1, \ldots, m, j = 1, \ldots, n$, and $b_i, i = 1, \ldots, m$ are fixed and typically derived from data.

The above formulation can be written compactly in vector form:

$$\begin{aligned} \text{minimize} \quad & \mathbf{c}^\mathsf{T}\mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \geq \mathbf{b}. \end{aligned} \tag{C.2}$$

The i-th row of the $\mathbf{A}$ matrix will be written as a row vector $\mathbf{a}_i^\mathsf{T}$. The j-th column of $\mathbf{A}$ will be written $\mathbf{A}_j$.

Note that many equivalent forms of an LP can be written in the above manner. For example, if the objective is to maximize instead of minimize, simply replace $\mathbf{c}$ with $-\mathbf{c}$. Similarly, for "less than or equal to" inequalities, multiply $\mathbf{A}$ and $\mathbf{b}$ by -1. Equality constraints can also be represented in the above form by noting that for a given i, $\mathbf{a}_i^\mathsf{T}\mathbf{x} = b_i$ can be enforced with two constraints: $\mathbf{a}_i^\mathsf{T}\mathbf{x} \geq b_i$ and $-\mathbf{a}_i^\mathsf{T}\mathbf{x} \geq -b_i$. Sign constraints on the variables (e.g., $x_1 \geq 0$ or $x_2 \leq 0$) can be enforced with appropriate choices of $\mathbf{A}$ and $\mathbf{b}$.

A *standard form* linear program is written

$$\begin{aligned} \text{minimize} \quad & \mathbf{c}^\mathsf{T}\mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned} \tag{C.3}$$

where the constraints comprise equality constraints involving $\mathbf{A}$ and $\mathbf{b}$, and non-negativity constraints on the decision variables. Any linear program can be transformed to one in standard form as follows. An unconstrained variable x_j can be replaced by $x_j^+ - x_j^-$, with constraints $x_j^+ \geq 0$ and $x_j^- \geq 0$. An inequality $\mathbf{a}_i^\mathsf{T}\mathbf{x} \geq b_i$ can be written as $\mathbf{a}_i^\mathsf{T}\mathbf{x} - s_i = b_i$, where $s_i \geq 0$. If the inequality is a "less than or equal to", add s_i instead of subtracting it.

A vector $\mathbf{x}$ that satisfies all constraints is a *feasible solution*. The set of vectors that satisfy the constraints is referred to as the *feasible region*. The feasible region of a linear program is a *polyhedron* by definition, since a polyhedron is defined as the intersection of a finite set of linear inequalities. Among the feasible solutions, the one with the lowest value of $\mathbf{c}^\mathsf{T}\mathbf{x}$, if it exists, is an *optimal solution*, denoted $\mathbf{x}^*$. If a linear program is feasible, but for every real number r there exists a feasible solution $\mathbf{x}$ such that $\mathbf{c}^\mathsf{T}\mathbf{x} < r$, then the problem is *unbounded* or has *unbounded objective function value*. Such an LP may be referred to as having an optimal value of $-\infty$. The following theorem establishes when an LP will have an optimal solution.

Theorem C.1. Every feasible LP with bounded objective function has an optimal solution.

If the feasible region of an LP is non-empty and bounded, then the objective function value cannot go to $-\infty$. Hence, the LP will have an optimal solution.

Corollary C.1. An LP with a non-empty bounded feasible region has an optimal solution.

Consider an LP with the set of constraints $\mathbf{a}_i^\mathsf{T}\mathbf{x} \geq b_i, i = 1, \ldots, m$. If a vector $\mathbf{x}$ satisfies a subset of the constraints at equality $\mathbf{a}_i^\mathsf{T}\mathbf{x} = b_i, i \in I \subseteq \{1, \ldots, m\}$, and if there are n coefficient vectors in this subset $\{\mathbf{a}_i | i \in I\}$ that are linearly independent, then $\mathbf{x}$ is a *basic feasible solution*. Figure 5.14 from Chapter 5 illustrates a polyhedron. Basic feasible solutions are the "corner points" of the polyhedron. Basic feasible solutions are important because under mild conditions, optimal solutions to an LP, if they exist, can be found there.

Theorem C.2. If a linear program has an optimal solution and its feasible region contains at least one basic feasible solution, then at least one basic feasible solution is an optimal solution.

The following characterizes when the feasible region of a linear program will have an extreme point.

Theorem C.3. A polyhedron has at least one extreme point if and only if it does not contain a line.

The well-known Simplex Algorithm progressively searches basic feasible solutions with improving objective function value until it finds an optimal basic feasible solution or determines that the optimal value is $-\infty$.

Note that there are LPs that have optimal solutions, but no corner points. Consider the feasible region $\{(x_1, x_2) \mid x_2 \geq 0\}$, which is a half space of $\mathbb{R}^2$. If the objective is to minimize x_2, then all points on the line $(x_1, 0), x_1 \in (-\infty, \infty)$ are optimal but none are corner points.

To guarantee that a polyhedron has a corner point, it is necessary and sufficient that it does not contain a line. That is, there must not be a vector $\mathbf{x}$ and nonzero vector $\mathbf{d}$ such that $\mathbf{x} + \alpha\mathbf{d}$ remains in the polyhedron for all scalars (negative and positive) α. Note that a linear program in standard form does not contain a line, since the feasible region is a subset of $\{\mathbf{x} \mid \mathbf{x} \geq \mathbf{0}\}$. Hence, if a linear program in standard form has an optimal solution, there must be one at a basic feasible solution.

Theorem C.4. A polyhedron has at least one extreme point if and only if it does not contain a line.

C.1 Duality

Given any linear program, there exists a *dual* linear program. The former or original linear program is referred to as the *primal*. The following pair of linear programs are dual to each other.

$$
\begin{array}{rlrl}
\text{minimize} & \mathbf{c}^\mathsf{T}\mathbf{x} & \text{maximize} & \mathbf{b}^\mathsf{T}\mathbf{y} \\
\text{subject to} & \mathbf{a}_i^\mathsf{T}\mathbf{x} \geq b_i, \quad i \in I_1, & \text{subject to} & y_i \geq 0, \quad i \in I_1, \\
& \mathbf{a}_i^\mathsf{T}\mathbf{x} \leq b_i, \quad i \in I_2, & & y_i \leq 0, \quad i \in I_2, \\
& \mathbf{a}_i^\mathsf{T}\mathbf{x} = b_i, \quad i \in I_3, & & y_i \text{ free}, \quad i \in I_3, \\
& x_j \geq 0, \quad j \in J_1, & & \mathbf{A}_j^\mathsf{T}\mathbf{y} \leq c_j, \quad i \in J_1, \\
& x_j \leq 0, \quad j \in J_2, & & \mathbf{A}_j^\mathsf{T}\mathbf{y} \geq c_j, \quad i \in J_2, \\
& x_j \text{ free}, \quad j \in J_3, & & \mathbf{A}_j^\mathsf{T}\mathbf{y} = c_j, \quad i \in J_3.
\end{array}
$$

Let the minimization problem be the primal. Then the dual problem is a maximization problem. A dual variable is associated with every non-sign constraint in the primal. Whether the dual variable is subject to a sign constraint or is free depends on whether the corresponding primal constraint is an inequality or equality, respectively. Similarly, every primal variable is associated with a non-sign constraint in the dual, and whether the primal variable is sign-constrained or is free determines whether the dual constraint is an inequality or equality, respectively. Notice that the constraints involving $\mathbf{A}$ in the dual use the transpose of the $\mathbf{A}$ matrix. That is, the constraints in the primal are written based on the rows, $\mathbf{a}_i^\mathsf{T}$, of $\mathbf{A}$. However, the constraints in the dual are written based on the columns, $\mathbf{A}_j$, of $\mathbf{A}$. Correspondingly, the parameters $\mathbf{c}$

and $\mathbf{b}$ have switched places. The objective coefficients $\mathbf{c}$ in the primal have become the right hand side parameters in the constraints in the dual, and the right hand side parameters $\mathbf{b}$ in the primal have become the objective coefficients in the dual.

First, it is straightforward to show that the value of the dual (maximization) objective is a lower bound on the value of the primal (minimization) objective. This result is known as *Weak Duality*.

Theorem C.5. Let $\mathbf{x}$ be a feasible solution to the primal and $\mathbf{y}$ be a feasible solution to the dual. Then $\mathbf{b}^\mathsf{T}\mathbf{y} \leq \mathbf{c}^\mathsf{T}\mathbf{x}$.

An immediate consequence of this result is that if there is a primal solution and dual solution that have the same objective function value, they are optimal solutions for their respective problems.

Corollary C.2. Let $\mathbf{x}$ be a feasible solution to the primal and $\mathbf{y}$ be a feasible solution to the dual. If $\mathbf{b}^\mathsf{T}\mathbf{y} = \mathbf{c}^\mathsf{T}\mathbf{x}$, then $\mathbf{x}$ and $\mathbf{y}$ are optimal solutions to the primal and dual, respectively.

A fundamental theorem of linear programming is the theorem of *Strong Duality*.

Theorem C.6. Let $\mathbf{x}^*$ be an optimal solution for an LP. Then its dual also has an optimal solution, $\mathbf{y}^*$, and their optimal values are equal.

Finally, the *Complementary Slackness* conditions provide another set of necessary and sufficient conditions for feasible solutions to the primal and dual to be optimal.

Theorem C.7. Let $\mathbf{x}$ be a feasible solution to the primal and $\mathbf{y}$ be a feasible solution to the dual. They are optimal if and only if

$$\begin{aligned} y_i(\mathbf{a}_i^\mathsf{T}\mathbf{x} - b_i) &= 0, \quad \forall i \\ (c_j - \mathbf{A}_j^\mathsf{T}\mathbf{y})x_j &= 0, \quad \forall j. \end{aligned}$$

Bibliography

Optimal decision procedures for finite Markov chains, I], author= J. A. Bather, journal=Adv. Appl. Prob., volume=5, pages=328-339, year=1973.

J. Abounadi, D. Bertsekas, and V.S. Borkar. Learning algorithms for Markov decision processes with average cost. *SIAM J. Control Optim.*, 40(3):681–698, 2001.

O. Alagoz, L. M. Maillart, A. J. Schaefer, and M. S. Roberts. Determining the acceptance of cadaveric livers using an implicit model of the waiting list. *Oper. Res.*, 55: 24–36, 2007.

American Cancer Society. *Cancer Facts and Figures 2024*. American Cancer Society, Atlanta, GA, 2024.

Aristotle. *Nicomachean Ethics*. Oxford University Press, Oxford, 1925. Book II, chapter 1 (1103a31–32). Translated by W. D. Ross.

K. J. Arrow, D. Blackwell, and M. A. Girshick. Bayes and minimax solutions of sequential decision problems. *Econometrica*, 17:213–244, 1949.

K. J. Arrow, T. Harris, and J. Marschak. Optimal inventory policy. *Econometrica*, 19: 250–272, 1951.

K. J. Arrow, S. Karlin, and H. E. Scarf. *Studies in the Mathematical Theory of Inventory and Production*. Stanford University Press, Stanford, CA, 1958.

T. Ayer, O. Alagoz, and N. K. Stout. Or forum–A POMDP approach to personalize mammography screening decisions. *Oper. Res.*, 60:1019–1034, 2012.

L. C. Baird. Residual algorithms: Reinforcement learning with function approximation. In Armand Prieditis and Stuart Russell, editors, *Machine Learning Proceedings 1995*, pages 30–37, San Francisco, CA, USA, 1995. Morgan Kaufmann.

A. G. Barto, R. S. Sutton, and C. W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Trans. Syst. Man. Cybern.*, SMC-13 (5):834–846, 1983.

A. G. Barto, R. S. Sutton, and C. J. C. H. Watkins. Sequential decision problems and neural networks. In D.S. Touretzky, editor, *Advances in Neural Information Processing Systems*, volume 2, pages 686–693. 1989.

J. A. Bather. Optimal decision procedures for finite Markov chains, II. *Adv. Appl. Prob.*, 5:521–540, 1973.

J. A. Bather. Dynamic programming. In W. Lederman and S. Vajda, editors, *Analysis, Vol. 4., Handbook of Applicable Analysis*. John Wiley and Sons, NY, 1982.

R. Bellman. Some directions of research in dynamic programming. *Unternehmensforschung*, 7:97–102, 1963.

R. E. Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957.

M. Bennett. *A brief history of intelligence*. William Collins, 2023.

A. Berman and R. Plemmons. *Nonnegative Matrices in the Mathematical Sciences*. Academic Press, 1979.

D. P. Bertsekas. *Dynamic Programming: Deterministic and Stochastic Models*. Prentice Hall, 1987.

D. P. Bertsekas. Approximate policy iteration: a survey and some new methods. *J. Control Theory Appl.*, 9:310–335, 2011.

D. P. Bertsekas. *Dynamic Programming and Optimal Control, Volume 2, 4th Edition*. Athena Scientific, 2012.

D. P. Bertsekas. *Reinforcement Learning and Optimal Control*. Athena Scientific, 2019.

D. P. Bertsekas. *Lessons from AlphaZero for Optimal, Model Predictive and Adaptive Control*. Athena Scientific, 2022.

D. P. Bertsekas and J. N. Tsitsiklis. An analysis of stochastic shortest path problems. *Math. Oper. Res.*, 16:580–595, 1991.

D. P. Bertsekas and J. N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.

D. Bertsimas and R. Shioda. Restaurant revenue management. *Oper. Res.*, 51:472–486, 2003.

D. Bertsimas and J. N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, Belmont, Massachusetts, 1997.

T. Bi and R. D'Andrea. Sample-efficient learning to solve a real-world labyrinth game using data-augmented model-based reinforcement learning, 2023. URL https://arxiv.org/abs/2312.09906.

D. Blackwell. Discrete dynamic programming. *Ann. Math. Stat.*, 33:719–726, 1962.

D. Blackwell. Discounted dynamic programming. *Ann. Math. Stat.*, 36:226–235, 1965.

D. Blackwell. Positive dynamic programming. In *Proceedings of the 5th Berkeley Symposium on Probability and Statistics, Volume 1*, pages 415–418, 1967.

J. E. Blum. Approximation methods which converge with probability one. *Ann. Math. Stat.*, 25:382–386, 1954.

M. Broadie. *Every Shot Counts*. Avery, 2014.

W. J. Bryan. America's Mission. Speech, Virginia Democratic Association Banquet, Washington, D.C., 1899. Delivered on February 22, 1899.

L. Buşoniu, D. Ernst, B. De Schutter, and R. Babuška. Online least-squares policy iteration for reinforcement learning control. In *Proceedings of the 2010 American Control Conference*, pages 486–491, 2010.

L. Buşoniu, A. Lazaric, M. Ghavamzadeh, M. Rémi, and R. Babuška. Least-squares methods for policy iteration. In M. Wiering and M. van Otterio, editors, *Reinforcement Learning: State of the Art*, pages 75–109. Springer Berlin, 2012.

P. G. Canbolat and U. G. Rothblum. (Approximate) iterated successive approximations algorithm for sequential decision processes. *Ann. Oper. Res.*, 208:309–320, 2013.

L. Carroll. *Alice's Adventures in Wonderland*. Macmillan and Co., London, 1865.

V. Carter and R. E. Machol. Technical note–Operations research on football. *Oper. Res.*, 19:541–544, 1971.

A. R. Cassandra. The POMDP Page, 2025. URL https://www.pomdp.org/. Accessed: 2025-01-30.

A. R. Cassandra, M. L. Littman, and N. L. Zhang. Incremental pruning: A simple, fast, exact method for partially observable Markov decision processes. In *Proceedings of the Thirteenth Conference on Uncertainty in Artificial Intelligence (UAI1997)*, pages 54–61, 1997.

A. Cayley. Mathematical questions with their solutions, no. 4528. *Educational Times*, 23:18, 1875.

E. Çinlar. *Introduction to Stochastic Processes*. Prentice Hall, Englewood Cliffs, N.J., 1975.

T. C. Y. Chan and R. Singal. A Markov Decision Process-based handicap system for tennis. *Journal Quant. Anal. Sports*, 12:179–189, 2016.

T. C. Y. Chan, C. Fernandes, and M. L. Puterman. Points gained in football: Using Markov process-based value functions to assess team performance. *Oper. Res.*, 69: 877–894, 2021.

T. C. Y. Chan, C. Fernandes, and R. Walker. No more throwing darts at the wall: Developing fair handicaps for darts using a Markov Decision Process. In *Proceedings of the 18th Annual MIT Sloan Sports Analytics Conference*, 2024.

H. S. Chang, M. C. Fu, J. Hu, and S. I. Marcus. An adaptive sampling algorithm for solving Markov decision processes. *Oper. Res.*, 53(1):126–139, 2005.

R. R. Chen, T. C. E. Cheng, T. M. Choi, and Y. Wang. Novel advances in applications of the newsvendor model. *Decis. Sci.*, 47(1):8–10, 2016.

H. Cheng. *Algorithms for partially observed Markov decision processes*. PhD thesis, University of British Columbia, 1988.

Y. S. Chow, H. Robbins, and D. Siegmund. *Great Expectations: The theory of optimal stopping*. Houghton-Mifflin, New York, 1971.

C. W. Clark. The lazy adaptable lions: A Markovian model of group foraging. *Anim. Behav.*, 35:361–368, 1987.

Confucius. *Analects 15.10*, page 178. Hackett Publishing, Indianapolis, 2003. Translated by E. Slingerland.

C. Darken, J. Chang, and J. Moody. Learning rate schedules for faster stochastic gradient search. In *Neural Networks for Signal Processing II Proceedings of the 1992 IEEE Workshop*, pages 3–12, 1992.

P. Dayan and T.J. Sejnowski. TD(λ) converges with probability 1. *Machine Laearning*, 14:295–301, 1994.

D. P. de Farias and B. Van Roy. On the existence of fixed points for approximate value iteration and temporal-difference learning. *J. Opt. Theor. Appl.*, 105:589–608, 2000.

D. P. de Farias and B. Van Roy. The linear programming approach to approximate dynamic programming. *Oper. Res.*, 51:850–865, 2003.

V. de Paul. Letter 1796 to Charles Ozenne, 13 Nov. 1654. In Sr. Marie Poole et al., editors, *Correspondence, Conferences, Documents*, volume 5, page 219. New City Press, Brooklyn, NY, 1995. Translated by Sr. Marie Poole and Rev. Francis Germovnik.

E. Denardo and B. Fox. Multichain markov renewal programs. *SIAM J. Appl. Math.*, 16:468–487, 1968.

F. D'Epenoux. Sur un problème de production et de stockage dans l'aléatoire. *RAIRO*, 14:3–16, 1960.

C. Derman. *Finite State Markovian Decision Processes*. Academic Press, New York, NY, 1970.

J. Desrosiers and M. E. Lübbecke. A primer in column generation. In G. Desaulniers, J. Desrosiers, and M. M. Solomon, editors, *Column Generation*, pages 1–32. Springer, 2005.

N. Draper and H. Smith. *Applied Regression Analysis, 3rd Edition*. Wiley-Interscience, 1998.

A. Dvoretsky, J. Kiefer, and J. Wolfowitz. The inventory problem: I. case of known distributions of demand. *Econometrica*, 20:187, 1952.

F. Dyson. A Meeting with Enrico Fermi. *Nature*, 427(6972):297, 2004.

J. H. Eaton and L. A. Zadeh. Optimal pursuit strategies in discrete state probabilistic systems. *Trans. ASME, Series D*, 84:23–29, 1962.

F. Edgeworth. The mathematical theory of banking. *J. R. Stat. Soc.*, 51(1):113–127, 1888.

G. Eliot. *Middlemarch: A Study of Provincial Life*. William Blackwood and Sons, Edinburgh and London, 1874.

V. N. Faddeeva. *Computational Methods of Linear Algebra*. Dover, New York, 1959.

W. Feller. *An Introduction to Probability Theory and Its Applications, Volume 1*. John Wiley & Sons., 1982.

T. S. Ferguson. Who solved the secretary problem? *Stat. Sci.*, 4(3):282–296, 1989.

R. P. Feynman, R. B. Leighton, and M. Sands. Lecture 1: Atoms in motion. In *The Feynman Lectures on Physics*, volume I, pages 1–2. Addison–Wesley, 1963.

J. Forrest and the COIN-OR contributors. *COIN-OR CLP: Simplex Solver*. COIN-OR Foundation, 2024. URL `https://www.coin-or.org/Clp`.

M. C. Fu. Monte Carlo tree search: a tutorial. In *Proceedings of the 2018 Winter Simulation Conference*, pages 222–236, 2018.

R. G. Gallagher. *Discrete Stochastic Processes*. Springer, 1996.

G. Gallego and G. van Ryzin. Optimal dynamic pricing of inventories with stochastic demand over finite horizons. *Manag. Sci.*, 40:999–1020, 1994.

J. Gessford and S. Karlin. Optimal policies for hydroelectric operations. In K. Arrow, S. Karlin, and H. Scarf, editors, *Studies in the Mathematical Theory of Inventory and Production*, pages 179–200. Stanford University Press, Stanford, CA, 1958.

G. T. De Ghelinick. Les problèmes de décisions séquentielles. *Cahiers du Centre d'Études Rec. Opér.*, 2:161–179, 1960.

J. C. Gittins and D. M. Jones. A dynamic allocation index for the sequential design of experiments. In *Progress in Statistics*. North-Holland, Amsterdam, 1974.

Y. Goçgun and M. L. Puterman. Dynamic scheduling with due dates and time windows: An application to chemotherapy patient appointment booking. *Health Care Manag. Sci.*, 17:60–76, 2014.

A. Gosavi. *Simulation-Based Optimization - Parametric Optimization Techniques and Reinforcement Learning*. Springer, 2015.

S. Gronauer, M. Gottwald, and K. Diepold. The sucessful ingredients of policy gradient algorithms. In *Proceedings of the Thirtieth Joint Conference on Artificial Intelligence (IIJCAI-21)*, pages 2455–2461, 2021.

I. Grondman, L. Busoniu, A. Lopes, and R. Babuska. A survey of actor-critic reinforcement learning: Standard and natural policy gradient. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 42(6):1291–1307, 2012.

LLC Gurobi Optimization. *Gurobi Optimizer Reference Manual*, 2024. URL `https://www.gurobi.com`.

D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025.

M. Hall. The state of goalie pulling in the NHL. `https://hockey-graphs.com/2020/05/18/the-state-of-goalie-pulling-in-the-nhl/`, May 2020. [accessed 14-September-2021].

N. A. J. Hastings. Some notes on dynamic programming and replacement. *Op. Res. Quart.*, 19:453–464, 1968.

M. Haviv and M. L. Puterman. Bias optimality in controlled queueing systems. *Jour. Appl. Prob.*, 35:136–150, 1998.

J. Herrman. The Great Amazon Flip-a-Thon. *The New York Times*, 2021. URL `https://www.nytimes.com/2021/03/17/style/amazon-brand-flippers.html`. Published online 17 Mar. 2021.

D. P. Heyman. Optimal operating policies for M/G/1 queueing systems. *Oper. Res.*, 16:362–382, 1968.

B. Hill. The glory of love. Sheet music, 1936.

F. S. Hillier and G. J. Lieberman. *Introduction to Operations Research*. McGraw-Hill, 2021.

R. Howard. *Dynamic Programming and Markov Processes*. MIT Press, Cambridge, MA, 1960.

T. Hubel, J. Myatt, N. Jordan, O. Dewhirst, J. McNutt, and A. Wilson. Energy cost and return in African wild dogs and cheetahs. *Nat. Commun.*, 7:11034, 2016.

G. Hübner. Improved procedures for eliminating suboptimal actions in Markov programming by the use of contraction properties. In J. Kožešnik, editor, *Transactions of the Seventh Prague conference on Information Theory, Statistical Decision Functions, Random Processes and the 1974 European Meeting of Statisticians*, pages 257–263, 1977.

IBM. *IBM ILOG CPLEX Optimization Studio*, 2024. URL https://www.ibm.com/products/ilog-cplex-optimization-studio.

T. Jaakkola, M. I. Jordan, and S. P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. *Neural Comput.*, 6(6):1185–1201, 1994.

L. C. M. Kallenberg. *Linear Programming and Finite Markovian Control Problems*. Mathematisch Centrum, Amsterdam, 1983.

S. Karlin. Stochastic models and optimal policies for selling an asset. In K. Arrow, S. Karlin, and H. Scarf, editors, *Studies in Applied Probability and Management Science*, pages 148–158. Stanford University Press, Stanford, CA, 1962.

D. Katselis. ECE586: MDPs and reinforcement learning: Lecture 10, 2019. URL http://katselis.web.engr.illinois.edu/ECE586/Lecture10.pdf.

E. J. Kelly and P. L. Kennedy. A dynamic stochastic model of mate desertion. *Ecology*, 74:351–366, 1993.

J. G. Kemeny and J. L. Snell. *Finite Markov Chains*. Van Nostrand-Reinhold, New York, 1960.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization, 2017. URL https://arxiv.org/abs/1412.6980.

L. Kleinrock. *Queuing Systems, Volume 1*. John Wiley & Sons, 1975.

M. J. Kochenderfer, T. A. Wheeler, and K. H. Wray. *Algorithms for Decision Making*. MIT Press, 2022.

R. Kohavi and S. Thomke. The surprising power of online experiments. *Harvard Business Review*, pages 74–82, September 2017.

V. Konda and J. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 12. MIT Press, 2000. URL https://proceedings.neurips.cc/paper_files/paper/2000/file/0199d4b5c234ed1324952503c3e02160-Paper.pdf.

V. Krishnamurthy. *Partially Observed Markov Decision Processes: From Filtering to Controlled Sensing*. Cambridge University Press, Cambridge, UK, 2016.

M. Kurt, B. T. Denton, A. J. Schaefer, N. D. Shah, and S. A. Smith. The structure of optimal statin initiation policies for patients with type 2 diabetes. *IIE Trans. Health. Syst. Eng.*, 1:49–65, 2011.

H. Kushner and A.J. Kleimnan. Accelerated procedures for the solution of discrete Markov control problems. *IEEE Trans. Autom. Control*, 16:147–152, 1971.

M. G. Lagoudakis and R. Parr. Least-squares policy evaluation. *J. Mach. Learn. Res.*, 4:1107–1149, 2003.

M. Lehmann. The definitive guide to policy gradients in deep reinforcement learning: Theory, algorithms and implementations, 2024. URL https://arxiv.org/abs/2401.13662.

S. Levine. CS285: Deep reinforcement learning; Lectures 4-9, 2024. URL https://rail.eecs.berkeley.edu/deeprlcourse/.

S. A. Lippman. Applying a new device in the optimization of exponential queuing systems. *Oper. Res.*, 23:687–710, 1975.

J. London. What Life Means to Me. *Cosmopolitan Magazine*, 40(5):526–530, March 1906.

W. S. Lovejoy. Computationally feasible bounds for partially observed Markov decision processes. *Oper. Res.*, 39:162–175, 1991a.

W. S. Lovejoy. A survey of algorithmic methods for partially observed Markov decision processes. *Ann. Oper. Res.*, 28:47–66, 1991b.

J. MacQueen. A modified dynamic programming method for Markov decision problems. *J. Math. Anal. Appl.*, 14:38–43, 1966.

J. MacQueen. Letter to the editor – A test for suboptimal actions in Markov decision problems. *Oper. Res.*, 15:559–561, 1967.

S. Mahadevan. Average reward reinforcement learning: Foundations, algorithms and empirical results. *Mach. Learn.*, 22:159–195, 1996.

A. Makhorin. *GNU Linear Programming Kit, Version 5.0*. GNU Project, 2024. URL `https://www.gnu.org/software/glpk/`.

M. Mangel and C. W. Clark. Towards a unified foraging theory. *Ecology*, 67:1127–1138, 1986.

A. Manne. Linear programming and sequential decisions. *Manag. Sci.*, 6:259–267, 1960.

P. Massé. *Les reserves et la regulation se l'avenir dans la vie economique, Vol I and II*. Hermann, 1946.

M. C. McKenzie and M. D. McDonnell. Modern value based reinforcement learning: A chronological review. *IEEE Access*, 10:134704–134725, 2022.

S. Meyn. *Control Systems and Reinforcement Learning*. Cambridge University Press, 2022.

V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing Atari with deep reinforcement learning. In *Proceedings of the 27th International Conference on Machine Learning*, pages 330–337, 2013.

V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

G. E. Monahan. State of the art–A survey of partially observable Markov decision processes: Theory, models, and algorithms. *Manag. Sci.*, 28:1–16, 1982.

D. G. Morrison. On the optimal time to pull the goalie: A Poisson model applied to a common strategy used in ice hockey. *TIMS Studies in Management Science*, 4: 67–78, 1976.

P. Naor. On the regulation of queue size by levying tolls. *Econometrica*, 37:15–24, 1969.

I. Newton. Letter to Robert Hooke, 15 Feb. 1676. In H. W. Turnbull, editor, *The Correspondence of Isaac Newton: 1661–1675*, volume 1, page 416. Cambridge University Press, Cambridge, 1959.

M. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015.

J. M. Norman. Dynamic programming in tennis – When to use a fast serve. *J. Oper. Res. Soc.*, 36:75–77, 1985.

NVIDIA Isaac. AI robot development platform. https://developer.nvidia.com/isaac/, 2025. Retrieved Feb. 20, 2025.

J. Patrick, M. L. Puterman, and M. Queyranne. Dynamic multi-priority patient scheduling. *Oper. Res.*, 56:1507–152, 2008.

A. Patterson, S. Neumann, M. White, and A. White. Empirical design in reinforcement learning. *J. Mach. Learn. Res.*, 25:1–63, 2024.

J. Pineau, G.. Gordon, and S. Thrun. Point-based value iteration: An anytime algorithm for POMDPs. In *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1025–1032, 2003.

L. K. Platzman. Negative dynamic programming. *Oper. Res.*, 25:529–533, 1977.

E. Porteus. *Foundations of Stochastic Inventory Theory*. Stanford Business Books, 2002.

W. Powell. *Approximate Dynamic Programming*. J. Wiley and Sons, 2007.

M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons., 1994.

M. L. Puterman and S. Brumelle. On the convergence of policy iteration in stationary dynamic programming. *Math. Oper. Res.*, 4:60–69, 1979.

M. L. Puterman and M. C. Shin. Modified policy iteration algorithms for discounted Markov decision problems. *Manag. Sci.*, 24:1127–1137, 1978.

R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024. URL https://www.R-project.org/.

A. Ribes, T. Terraz, B. Iooss, Y. Fournier, and B. Raffin. Large scale in transit computation of quantiles for ensemble runs. 2019. URL https://arxiv.org/abs/1905.04180.

H. Robbins and S. Monro. A stochastic approximation method. *Ann. Math. Stat.*, 22: 400–407, 1951.

O. Roeder. *Seven games: a human history*. W. W. Norton, New York, 2022.

S. Ross. *Introduction to Stochastic Dynamic Programming*. Academic Press, New York, 1983.

C. Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nat. Mach. Intell.*, 1:206–215, 2019.

W. Rudin. *Principles of Mathematical Analysis, 2nd Edition*. McGraw-Hill, Inc., 1964.

A. Samuel. Some studies in machine learning using the game of checkers. *IBM J. Res. and Dev.*, 3:210–229, 1959.

A. Sauré, J. Patrick, and M. L. Puterman. Optimal multi-appointment scheduling. *Eur. J. Oper. Res.*, 223:573–584, 2012.

A. Sauré, J. Patrick, and M.L. Puterman. Simulation-based approximate policy iteration with generalized logistic functions. *INFORMS J. Comput.*, 27:579–595, 2015.

H. Scarf. The optimality of (s, S)-policies in the dynamic inventory problem. In S. Karlin K. Arrow and P. Suppes, editors, *Studies in the Mathematical Theory of Inventory and Production*, pages 196–202. Stanford University Press, Stanford, CA, 1960.

J. Schulman, S. Levine, P. Abbeel, M. I. Jordan, and P. Moritz. Trust region policy optimization. In *International Conference on Machine Learning*, 2015.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. 2017. URL `https://arxiv.org/abs/1707.06347`.

W. Schultz, P. Dayan, and P. R. Montague. A neural substrate of prediction and reward. *Science*, 275:1593–1599, 1997.

P. J. Schweitzer. Iterative solution of the functional equations of undiscounted Markov renewal programming. *J. Math. Anal. Appl.*, 34:495–501, 1971.

P. J. Schweitzer and A. Federgruen. Asymptotic behavior of value iteration in Markov decision problems. *Math. Oper. Res.*, 2:360–381, 1977.

P. J. Schweitzer and A. Seidmann. Generalized polynomial approximations in Markov decision processes. *J. Math. Anal. Appl.*, 110:568–582, 1985.

L. Sennott. *Stochastic Dynamic Programming and the Control of Queueing Systems*. John Wiley and Sons, 1999.

R. Serfozo. Monotone optimal policies for Markov decision processes. *Math. Program. Stud.*, 6:202–215, 1976.

L. S. Shapley. Stochastic games. *Proc. Natl. Acad. Sci. USA*, 39:1095–1100, 1953.

S. M. Shechter, M. D. Bailey, A. J. Schaefer, and M. S. Roberts. The optimal time to initiate HIV therapy under ordered health states. *Oper. Res.*, 56:20–33, 2008.

S. M. Shechter, F. Ghassemi, Y. Gocgun, and M. L. Puterman. Technical note–Trading off quick versus slow actions in optimal search. *Oper. Res.*, 63:353–362, 2015.

D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.

R. Simmons and S. Koenig. Probabilistic robot navigation in partially observable environments. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1080–1087, 1995.

S. Singh, T. Jaakkola, M. Littman, and C. Szepesvári. Convergence results for some reinforcement learning algorithms. In *Proceedings of the seventeenth International Conference on Machine Learning*, pages 791–798, 2000.

E. Sirot and C. Bernstein. Time sharing between host searching and food searching in parasitoids: state-dependent optimal strategies. *Behav. Ecol.*, 7:189–194, 1996.

R. D. Smallwood and E. J. Sondik. The optimal control of partially observable Markov processes over a finite horizon. *Oper. Res.*, 21:1071–1088, 1973.

B. C. Smith, J. F. Leimkuhler, and R. M. Darrow. Yield management at American Airlines. *Interfaces*, 22:8–31, 1992.

G. L. Smuts. Diet of lions and spotted hyenas assessed from stomach contents. *S. Afr. J. Wildl. Res.*, 9:19–25, 1979.

M. J. Sobel. Optimal average-cost policy for a queue with start-up and shut-down costs. *Oper. Res.*, 17:145–162, 1969.

E. J. Sondik. The optimal control of partially observable Markov processes over the infinite horizon: Discounted costs. *Oper. Res.*, 26:282–304, 1978.

M. Spaan and N. Vlassis. Perseus: Randomized point-based value iteration for POMDPs. *J. Artif. Intell. Res.*, 24:195–220, 2005.

D. Stengos and L. C. Thomas. The blast furnaces problem. *Eur. J. Oper. Res.*, 4: 330–336, 1980.

G. Strang. *Introduction to Linear Algebra, 6th edition.* Wellesley-Cambridge Press, 2023.

R. Strauch. Negative dynamic programming. *Ann. Math. Stat.*, 37:871–890, 1966.

H. Sugiyama. Solving deterministic problems via stochastic approximation – A simple yet powerful numerical method. *Computers Math. Applic.*, 27(9/10):129–144, 1994.

R. S. Sutton. Learning to predict by the methods of temporal differences. *Mach. Learn.*, 3:9–44, 1988.

R. S. Sutton and A. G. Barto. *Reinforcement Learning: An introduction, second edition.* MIT Press, Cambridge, MA, 2018.

R. S. Sutton, D. MacAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, pages 1057–1063, 1999.

C. Szepesvari. *Algorithms for Reinforcement Learning.* Morgan and Claypool, 2010.

K. Talluri and G. van Ryzin. *The Theory and Practice of Revenue Management.* Springer US, 2004.

J. C. Y. Tang, V. Paixao, F. Carvalho, A. Silva, A. Klaus, J. Alves da Silva, and R. M. Costa. Dynamic behaviour restructuring mediates dopamine-dependent credit assignment. *Nature*, 626:583–592, 2024.

G. Tesauro. Practical issues in temporal difference learning. *Mach. Learn.*, 8:257–277, 1992.

P. S. Thomas and E. Brunskill. Data-efficient off-policy policy evaluation for reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning, PMLR*, volume 48, pages 2139–2148, 2016.

H. D. Thoreau. *Walden; or, Life in the Woods.* Ticknor and Fields, Boston, 1st edition, 1854.

J. N. Tsitsiklis. Asynchronous stochastic approximation and Q-learning. *Mach. Learn.*, 16:185–202, 1994.

J. N. Tsitsiklis and B. Van Roy. An analysis of temporal-difference learning with function approximations. *IEEE Trans. Autom. Control*, 42:674–690, 1997.

J. N. Tsitsiklis and B. Van Roy. Average cost temporal-difference learning. *Automatica*, 35:1799–1808, 1999.

J. W. Tukey. The future of data analysis. *Ann. Math. Stat.*, 33(1):1–67, 1962.

J. A. E. E. van Nunen. A set of successive approximation methods for discounted Markovian decision problems. *Z. Oper. Res.*, 20:203–208, 1976.

A. F. Veinott, Jr. Discrete dynamic programming programming with sensitive discount optimality criteria. *Ann. Math. Stat.*, 40:1635–1660, 1969a.

A. F. Veinott, Jr. Dynamic programming and stochastic control course notes and homework. OR351, Stanford University, 1969b. Unpublished.

A. Wald. *Sequential Analysis*. John Wiley & Sons, New York, 1947.

A. Wald and J. Wolfowitz. Optimal character of the sequential probability ratio test. *Ann. Math. Stat.*, 19:326–339, 1948.

J. Wardle. Wordle, 2021. URL `https://www.nytimes.com/games/wordle/index.html`. Accessed: 2024-07-12.

A. Washburn. Still more on pulling the goalie. *Interfaces*, 21:59–64, 1991.

C. Watkins and P. Dayan. Q-learning. *Mach. Learn.*, 8:279–292, 1992.

J. Wessels. Markov programming by successive approximations with respect to weighted supremum norms. *J. Math. Anal. Appl.*, 58:326–335, 1977.

D. J. White. Dynamic programming, markov chains and the method of sucessive approximations. *J. Math. Anal. Appl.*, 6:373–376, 1963.

P. Whittle. *Optimization over time, dynamic programming and stochastic control, Volume II*. Wiley, 1983.

P. Whittle. Restless bandits: Activity allocation in a changing world. *J. Appl. Probab.*, 25:287–298, 1988.

B. Widrow, N. K. Gupta, and S. Maitra. Punish/reward: learning with a critic in adaptive threshold systems. *IEEE Trans. Syst. Man. Cybern.*, 3(5):455–465, 1973.

A. A. Wieschenberg. Making mathematics: The coffee connection. *College Teaching*, 47:102–105, 1999.

R. Williams. Some statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.*, 8:229–256, 1992.

C. Xiao, I. Lee, B. Dai, D. Schuurmans, and C. Szepesvari. The curse of passive data collection in batch reinforcement learning. In *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 151, pages 8413–8438, 2022.

M. Yadin and P. Naor. On queueing systems with variable service capacities. *Naval Res. Logist. Quart.*, 14:43–53, 1967.

Index

N-stage contraction, 348
v-greedy, 228
ϵ-greedy action selection, 695
ϵ-greedy sampling, 644
ϵ-optimal policy, 205
ϵ-tweaked policy, 725

Absorption time, 328
Action, 38
Action elimination, 231
Action set, 38
Actor-critic, 786, 830
    Online, 833
Advantage function, 831
Aperiodicity transformation, 433
Approximate dynamic programming, 559
    Linear programming, 581
Approximate linear program
    Dual, 597
    Primal, 596
Asynchronous value iteration, 244
Average reward
    Aperiodic case, 403
    Bellman equation, 418
    Bias, 403
    Bounds, 420
    Chain structure, 397
    Communicating models, 399
    Constrained models, 460
    Constraints, 467
    Evaluation equations, 410
    Gain, 392
    Linear programming, 451, 466
    Multi-chain, 430
    Optimal policy, 396
    Periodic case, 403
    Policy iteration, 442, 464
    Relative value iteration, 440, 463
    Relative values, 417
    State-action value function, 427
    Structured policies, 461
    TD(0), 689
    Temporal differencing, 689
    Value function, 397
    Value iteration, 430, 435, 463
Average reward optimality, 63

Backwards induction, 137, 233
Banach fixed point theorem, 209
Bang-bang policy, 154
Baseline, 820, 822
Basic feasible solution
    Linear programming, 884
Basis functions, 563, 769
Batch policy gradient, 821
Behavioral policy, 640, 697
Behavioral science, 26
Belief state, 475, 484, 486
Bellman equation
    Average reward, 418
    Derived MDP, 498
    Discounted model, 205
    Existence of solution, 210
    Finite horizon, 147
    Negative model, 360
    POMDP, 498
    Positive model, 357
    Solution, 211
    State-action value function, 175, 216, 793
    Transient model, 335, 346
    Value function approximation, 793

Bellman error, 783
Bellman operator
  Discounted model, 207
  Expected total reward, 335
Bellman residual minimization, 580
Bias, 403, 742
Bias-variance trade-off, 743
Bootstrapping, 658, 661, 783

c-max, 206
Cesaro sum, 871
Classical statistical bandit, 537, 538
Common random numbers, 639
Complementary slackness
  Linear programming, 886
Component-wise maximum, 194
Contraction mapping, 208, 240
Control limit policy, 165, 286
Convergence at rate β^n, 237
Convergence with order α, 237
Convexity, 169
Credit assignment, 27, 29, 618, 801

Data generation
  Action-based, 638
  State-action pair-based, 639
  State-based, 638
  Trajectory-based, 638
Decision epoch, 37
Decision rule, 42
  Deterministic, 42
  History-dependent, 42
  Markovian, 42
  Randomized, 42
Derived stochastic process, 45
Deviation matrix, 871
Discount factor, 49
Discount optimal, 61
Discounted model
  Bellman operator, 207
  Modified policy iteration, 256
  Policy iteration, 246
  Stationary policies, 201
  Value iteration, 233
Dopamine, 27
Duality
  Linear programming, 885
Dynamic programming, 147

Effective horizon, 54, 328, 652
Eigenvalues, 876
Eigenvectors, 876
Eligibility trace, 672
  Accumulating, 672
  Replacement, 672
Episodic model, 651, 817
  Q-learning, 699
Examples
  Advance appointment scheduling, 108, 559, 600
  Brio Labyrinth, 557
  Checkers, 27
  Delivering coffee, 839
  Golf, 612
  Gridworld, 88, 323, 332, 344, 364, 367, 374, 401, 655, 666, 676, 701, 702, 704, 798, 839
  Inventory management, 24, 80
    Backlogged demand, 82
  Liver transplantation, 105, 297
  Newsvendor, 85, 641, 806, 814, 817
  Online dating, 116, 157
  Optimal parking, 115
  Optimal stopping, 113, 376, 777, 823, 825
  Optimal stopping on a random walk, 378
  Periodic, 432, 433
  Pre-boarding security, 20
  Preventive maintenance, 164, 285
  Pulling the goalie, 118
  Queuing admission control, 97, 178
  Queuing service rate control, 95, 151, 166, 286, 463, 561, 568, 578, 582, 588, 591, 598, 728, 780, 791, 796, 810

Revenue management, 92, 154
Robotic navigation, 21
Secretary problem, 22, 116, 157
Selling an asset, 115, 379
Slow policy iteration, 250
Strategic scheduling, 600
Tennis handicapping, 120
Two-state, 33, 70, 139, 150, 173, 177, 217, 225, 229, 237, 243, 249, 261, 267, 272, 400, 405, 432, 444, 453, 459, 461, 589, 679, 683, 687, 690, 709, 716, 721, 723, 829, 835
Wordle, 848
Expected total reward
Bellman equation, 334
Bellman operator, 335
Modified policy iteration, 365
Policy iteration, 362
State-action value function, 342
Structured policies, 373
Value iteration, 342
Experience replay, 822, 844
Exploitation, 695
Exploration, 695

Features, 563, 769
Feedforward neural network, 567
Finite horizon
ϵ-optimal policy, 145
Backwards induction, 138, 147, 180
Bellman equation, 147
Optimal policy, 144
Optimality equation, 147
Policy evaluation, 138, 143
Policy optimization, 147
Value function, 136
Fixed point, 207
Freudenthal triangulation, 517

Gain, 62, 392
Optimality, 396
Gain optimality, 63
Gauss-Newton iteration, 628
Gauss-Seidel
Bounds, 306
Value iteration, 238
Geometric convergence, 238, 348
Geometric distribution, 59
Geometric stopping times, 679
Gradient descent, 625, 750
Greedy action, 67, 693
Greedy action choice, 213
Greedy decision rule, 425
Greedy in the limit with infinite exploration, 701
Greedy policy, 213, 860

Hidden state, 474

Importance sampling, 822, 845
Information states, 485

Laurent series, 406
Learning policy, 640, 697
Learning rates, 664
Least squares
Modified policy iteration, 591
Policy evaluation, 575
Policy iteration, 587
Regression, 620
Value iteration, 586
Lexicographic ordering, 445
Linear approximation, 566
Linear programming, 883
Average reward, 451, 466
Basic feasible solution, 884
Complementary slackness, 886
Discounted model, 265
Dual, 271
Duality, 885
Interior point algorithms, 283
Polyhedron, 884
Primal, 266
Simplex method, 282
Strong duality, 886
Transient model, 368
Weak duality, 886

Linear regression, 619
  Matrix formulation, 621
Logistic function, 775
Lookup table, 45
Lovejoy's algorithm, 517, 518
LSPI, 587
  Bounds, 590
LSVI, 586

Markov chains, 863
  Countable state, 880
  Limits, 867
  Regular, 866
  State classification, 864
  Structure, 866
Markov reward process, 48, 864
Matrix series, 870
Mean squared error, 742
Model classification
  Communicating model, 398
  Recurrent model, 398
  Regular model, 398
  Unichain model, 398
  Weakly communicating, 398
Model-based, 635
Model-based reinforcement learning, 843
Model-free, 635
Modified policy iteration
  Discounted model, 256
  SSP model, 368
  Transient model, 365
Monte Carlo methods
  Discounted model, 779
  Episodic, 652
  Estimation, 643
  First visit, 654
  Starting state, 652
  Value function approximation, 775
Monte Carlo tree search, 847
Multi-armed bandits, 536
  Restless Markov bandit, 541
Multi-chain Bellman equations, 430

Negative model, 330
  Bellman equation, 360
  Value iteration, 360
Neural networks, 567, 845
Newton's method, 253, 310, 627
Nonlinear regression, 625
Norm, 195
  Matrix, 196
Normed linear space
  Complete, 208

Off-policy, 701
Off-policy learning, 845
On-policy, 701
One-period problem, 63
Operator, 193
  B, 225
  L, 207
  Maximum return, 207
Optimal policy, 56
  Average reward, 396
  Discounted model, 204
  Expected total reward, 334
  Finite horizon, 144
  POMDP, 497
Optimal stationary policy, 212
  Average reward, 423
  Discounted model, 212
  Expected total reward, 338
Optimal value function
  Average reward, 397
  Discounted model, 204
  Expected total reward, 333
  Finite horizon, 144
Optimality criteria, 50
  Average reward, 61, 396
  Expected discounted reward, 57
  Expected total reward, 51
Optimality equation
  Finite horizon, 147

Partial Laurent series, 406
Partial order, 194
Perron-Frobenius Theorem, 877

Piecewise linear convex function, 501
Planning horizon, 36
Policy, 44
  History-dependent deterministic, 44
  History-dependent randomized, 44
  Markovian deterministic, 44
  Markovian randomized, 44
  Stationary, 44
Policy approximation, 774
Policy evaluation
  Discounted model, 197
  Finite horizon, 138, 143
Policy gradient
  Algorithm, 813, 815
  Discounted model, 828
  Episodic model, 817
  Theorem, 850
Policy iteration
  Average reward, 442, 464
  Convergence, 251
  Discounted model, 246
  Expected total reward, 362
  Hybrid, 720
  Online, 725
  Quadratic convergence, 312
  Simplex method, 282
  Simulated, 719
  SSP model, 365
  Transient model, 362
Policy optimization
  Finite horizon, 147
Policy value function approximation, 775
Polyhedron, 277, 503
  Linear programming, 884
POMDP
  Approximate algorithm, 513
  Backwards induction, 499
  Belief state, 475, 483, 484
  Classical statistical bandit, 537
  Derived Markov decision process, 489
  Finite horizon model, 496
  Hidden state, 474
  Infinite horizon discounted model, 524
  Lovejoy's algorithm, 513, 517, 518
  Monahan's algorithm, 508
  Point-based value iteration, 527
  Pruning, 507
  Restless bandit, 537
  Restless Markov bandit, 541
  Underlying Markov decision process, 489
  Value function structure, 501
POMDP examples
  Breast cancer screening, 542
  Minimally invasive surgery, 536
  Multi-armed bandits, 536
  Newsvendor, 533
  Newsvendor with unknown demand, 533
  Preventive maintenance, 528, 551
  Robotic control, 545
  Search, 534
  Tiger problem, 550
  Two-state, 480, 509, 520
Positive model, 330
  Bellman equation, 357
  Value iteration, 356
Positively similar models, 349
Post-decision state, 100, 179
Principle of optimality, 145, 149
Projected Bellman equation, 573
Proper policy, 329
Proximal policy optimization, 821

Q-learning, 644, 697, 786
  Average reward, 712
  Discounted model, 706
  Episodic model, 699
  Value function approximation, 792
Q-policy iteration, 803
Quadratic convergnce, 253

Randomized policy, 774, 775
Regression, 752
Reinforcement learning, 768
  Behavioral, 25
  Computational, 25

Relative Q-learning, 714
Relative value iteration
    Average reward, 440
Replay buffer, 844
Restless bandit, 537
Reward, 39, 191
    Stationary, 40
Reward process, 47
Robbins-Monro recursion, 745
Rolling horizon, 112
Root mean squared error, 742
    Weighted, 744

SARSA, 697, 795, 805
    Discounted model, 706
Score function, 820
Semi-gradient, 784, 793
Signal space, 476
Simulation, 52
Softmax, 775, 813
Softmax sampling, 645, 696
Span, 220
Span contractions, 223
Spectral radius, 327
SSP model, 329
    Modified policy iteration, 368
    Policy iteration, 365
    Value iteration, 356
State, 38
State space, 38
State-action value function, 68
    Approximation, 571
    Average reward, 427
    Bellman equation, 175, 216
    Discounted model, 215
    Expected total reward, 342
    Finite horizon, 171
    Policy evaluation, 172
    Policy optimization, 176
Step-size, 664
Stochastic approximation, 745
Stochastic gradient descent, 752
Stochastic shortest path model, 329
Strokes gained, 612, 617
Strong duality
    Linear programming, 886
Structured policies, 23, 24, 162
    Average reward, 461
    Discounted model, 284
    Expected total reward, 373
Sufficient statistic, 497

Tabular model, 45, 634, 770
TD(γ), 667, 757, 789
    Discounted models, 681
    Offline, 670
    Online, 671, 674
TD(0), 785
Temporal difference, 660, 783
Temporal differencing, 643, 783
    Average reward, 689
    Episodic model, 656
    Policy value function approximation, 781
Threshold policy, 165
Trajectory, 46
Transient Bellman equation, 335
Transient model, 326, 878
    Bellman equation, 335, 346
    Error bounds, 355
    Linear programming, 368
    Modified policy iteration, 365
    Policy iteration, 362
    Value iteration, 346
Transient reward, 405
Transition probability, 39
    Stationary, 39
Trial-and-error learning, 25
Truncation, 678
Turnpike theorem, 153

Unichain model, 326
Utility, 49

Value function
    Average reward, 397
    Discounted model, 197

Expected total reward, 324
Finite horizon, 136
Optimal, 64, 144, 333
Policy, 51, 63, 136, 197
POMDP, 498
State-action, 68, 171
Value function approximation, 561, 566, 770
Direct methods, 573
Indirect methods, 573
Using subsets of S, 579
Value iteration
Average reward, 430, 435, 463
Bounds, 439
Discounted model, 233
Expected total reward, 342
Negative model, 360
Positive model, 356
SSP model, 356
Transient model, 346
Vanishing discount rate, 406
Variance, 742
Vector space, 193

Weak duality
Linear programming, 886
Weighted sup-norm, 353
Weights, 769
World models, 844